PROGRAMACIÓN

https://martinezpenya.es/1DAMProgramacion/

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

1º Programacion (CFGS Desarrollo de Aplicaciones Multiplataforma)

IES Eduardo Primo Marques (Carlet)

David Martinez Peña

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Indice de contenidos

1. UD00 10
   1.1 🚀 Información importante 10
      1.1.1 📛 Denominación del curso 10
      1.1.2 📋 Contenidos 10
      1.1.3 🎯 Resultados de Aprendizaje (RA) 11
      1.1.4 📜 Legislación vigente 18
      1.1.5 📝 Evaluación 18
2. UD01 19
   2.1 Elementos de un programa informático 19
      2.1.1 Piensa como un programador 19
      2.1.2 Problemas, algoritmos y programas 23
      2.1.3 Java 29
      2.1.4 Componentes del lenguaje Java 33
      2.1.5 Herramientas útiles para empezar 41
      2.1.6 Ejemplos UD01 41
      2.1.7 Resumen — Conceptos clave 45
      2.1.8 Autoevaluación 46
      2.1.9 Vídeos recomendados 46
   2.2 Ejercicios de la UD01 47
      2.2.1 Retos 47
      2.2.2 Ejercicios 47
      2.2.3 Expresiones Lógicas 50
      2.2.4 Actividades 51
   2.3 Talleres 53
      2.3.1 Taller UD01_01: Instalar Veyon para el control del aula 53
      2.3.2 Taller UD01_01: Instalación de Veyon en Linux 55
      2.3.3 Taller UD01_01: Instalación de Veyon en Windows 57
      2.3.4 Taller UD01_01: Instalación de Veyon en macOS 59
      2.3.5 Taller UD01_01: Resolución de problemas con Veyon 60
      2.3.6 Taller UD01_02: Instalación y uso de entornos de desarrollo 62
      2.3.7 Taller UD01_03: Crear cuenta en GitHub 87
      2.3.8 Taller UD01_T04: Markdown 94
3. UD02 104
   3.1 Utilización de Objetos y Clases 104
      3.1.1 Introducción a la POO 104

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

3.1.2 Características de la POO 105
3.1.3 Objetos y Clases 105
3.1.4 Utilización de Objetos 108
3.1.5 Utilización de Métodos 112
3.1.6 **Librerías de Objetos (Paquetes)** 115
3.1.7 Cadenas de caracteres. La clase `String` 116
3.1.8 Ejemplos UD02 118
3.1.9 Resumen — Conceptos clave 121
3.1.10 Autoevaluación 122
3.1.11 Vídeos recomendados 122
3.2 Ejercicios de la UD02 123
3.2.1 Actividades 123
3.3 Talleres 131
3.3.1 Taller UD02_01: Classroom 50 131
3.3.2 Taller UD02_02: Configurar OBS para grabar los exámenes 145
4. UD03 153
4.1 Estructuras de control y Excepciones 153
4.1.1 Introducción 153
4.1.2 Sentencias y bloques 154
4.1.3 Estructuras de selección 155
4.1.4 Estructuras de repetición 159
4.1.5 Estructuras de salto 165
4.1.6 Excepciones 166
4.1.7 Aserciones (`Assertions`) 171
4.1.8 Ejemplos UD03 172
4.1.9 Resumen — Conceptos clave 183
4.1.10 Autoevaluación 183
4.1.11 Vídeos recomendados 184
4.2 Ejercicios de la UD03 185
4.2.1 Retos 185
4.2.2 Ejercicios 187
4.2.3 Actividades 208
4.3 Talleres 213
4.3.1 Taller UD03_01: Classroom 50 213
4.3.2 Taller UD03_02: Acepta el reto 214
5. UD04 215
5.1 Estructuras de datos: Arrays y matrices. Recursividad. 215
5.1.1 Introducción 215

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

5.1.2 Arrays 216
5.1.3 Problemas de recorrido, búsqueda y ordenación 219
5.1.4 Arrays bidimensionales: matrices 224
5.1.5 Arrays multidimensionales 227
5.1.6 Recursividad 227
5.1.7 Ejemplos UD04 231
5.1.8 Resumen — Conceptos clave 236
5.1.9 Autoevaluación 237
5.1.10 Vídeos recomendados 237
5.2 Anexo Cheatsheet Strings en Java 238
5.2.1 Introducción 238
5.2.2 Construyendo `String` 238
5.2.3 Operando con Métodos de la clase `String` 238
5.2.4 Ejemplo de todos los métodos de `String` 241
5.2.5 Arrays de `String` 242
5.2.6 Los `String` son inmutables 242
5.2.7 `String` en Argumentos de Línea de Comandos 243
5.2.8 Concatenar cadenas en `Java` 244
5.2.9 Formato de salida: `printf` y `String.format` 245
5.3 Ejercicios de la UD04 247
5.3.1 Arrays. Ejercicios de recorrido 247
5.3.2 Arrays. Ejercicios de búsqueda 249
5.3.3 Matrices 252
5.3.4 Recursividad 256
5.4 Talleres 258
5.4.1 Taller UD04_01: Classroom 50 258
6. UD05 259
6.1 Desarrollo de clases 259
6.1.1 Introducción 260
6.1.2 Estructura y miembros de una clase 260
6.1.3 Atributos 262
6.1.4 Métodos 264
6.1.5 Encapsulación, control de acceso y visibilidad. 269
6.1.6 Utilización de los métodos y atributos de una clase 271
6.1.7 Constructores 271
6.1.8 Clases Anidadas, Clases Internas (*Inner Class*) [NUEVO] 273
6.1.9 Introducción a la herencia. [NUEVO] 276
6.1.10 Conversión entre objetos (Casting) [NUEVO] 277

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

6.1.11 Acceso a métodos de la superclase [NUEVO] 278
6.1.12 Empaquetado de clases [NUEVO] 279
6.1.13 Ejercicios resueltos 280
6.1.14 Ejemplos UD05 285
6.1.15 Resumen — Conceptos clave 292
6.1.16 Autoevaluación 292
6.1.17 Vídeos recomendados 293
6.2 Anexo Wrappers 294
6.2.1 Wrappers (Envoltorios) 294
6.2.2 Ejemplo Anexo UD05 297
6.3 Anexo Fechas 298
6.3.1 Clase `Date` 298
6.3.2 Ejemplo Anexo UD05 305
6.4 Ejercicios de la UD05 308
6.4.1 Ejercicios 308
6.4.2 Actividades 314
6.5 Talleres 317
6.5.1 Taller UD05_01: Classroom 50 317
7. UD06 318
7.1 Lectura y escritura de información 318
7.1.1 Streams (Flujos) 318
7.1.2 Ficheros 321
7.1.3 Serialización 325
7.1.4 Sockets 326
7.1.5 Manejo de ficheros y carpetas (`File`) 326
7.1.6 Ejemplos UD06 328
7.1.7 Resumen — Conceptos clave 337
7.1.8 Autoevaluación 337
7.1.9 Vídeos recomendados 338
7.2 Comparativa CRUD 339
7.3 Ejercicios de la UD06 340
7.3.1 Paquetes Completos 340
7.3.2 **Los flujos estándar** 344
7.3.3 **`InputStreamReader`** 344
7.3.4 **Entrada "orientada a líneas".** 344
7.3.5 **Lectura/escritura en ficheros** 344
7.3.6 **Uso de buffers** 345
7.3.7 **Streams para información binaria** 345

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

7.3.8 **Streams de objetos. Serialización.** 346
7.3.9 Sockets 346
7.3.10 Más ejercicios (Lionel) 346
7.3.11 Aún más ejercicios 348
7.4 Talleres 350
7.4.1 Taller UD06_01: Classroom 50 350
7.4.2 Taller UD06_T02: Sockets en la nube (AWS) 351
8. UD07 361
8.1 Colecciones 361
8.1.1 Introducción 361
8.1.2 Estructuras de almacenamiento 362
8.1.3 Clases y métodos genéricos 362
8.1.4 Colecciones 365
8.1.5 Iteradores 376
8.1.6 Comparadores 377
8.1.7 Extras 378
8.1.8 Programación funcional 379
8.1.9 Ejemplos UD07 383
8.1.10 Resumen — Conceptos clave 395
8.1.11 Autoevaluación 396
8.1.12 Vídeos recomendados 396
8.2 Ejercicios de la UD07 397
8.2.1 Ejercicios 397
8.2.2 Actividades 399
8.2.3 Ejercicios Genericidad 401
8.2.4 Programación funcional. Funciones Lambda. 402
8.3 Talleres 405
8.3.1 Taller UD07_01: Classroom 50 405
9. UD08 406
9.1 Composición, Herencia y Polimorfismo 406
9.1.1 Relaciones entre clases. 406
9.1.2 Composición 409
9.1.3 Herencia 412
9.1.4 Clases Abstractas 417
9.1.5 Interfaces 420
9.1.6 Polimorfismo 427
9.1.7 Ejemplos UD08 431
9.1.8 Resumen — Conceptos clave 456

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

9.1.9 Autoevaluación 457
9.1.10 Vídeos recomendados 457
9.2 Ejercicios de la UD08 458
9.2.1 Ejercicios Herencia 458
9.2.2 Ejercicios Polimorfismo 461
9.2.3 Actividades 463
9.2.4 Ejercicios Lionel 464
9.3 Talleres 471
9.3.1 Taller UD08_01: Classroom 50 471
9.3.2 Taller UD08_02: Librerias Maven vs Jar 472
9.3.3 Taller UD08_T03: Introducción a JSON y YAML 475
10. UD09 479
10.1 Interfaz gráfica 479
10.1.1 Introducción 479
10.1.2 Gráfico de escena 480
10.1.3 Controles de la interfaz de usuario 484
10.1.4 Diseño (Layouts) 490
10.1.5 Gestión de excepciones y ventanas modales`Alert` 505
10.1.6 Estructura de la aplicación 505
10.1.7 Ejemplos UD09 506
10.1.8 Resumen — Conceptos clave 520
10.1.9 Autoevaluación 520
10.1.10 Vídeos recomendados 520
10.2 Ejercicios de la UD09 521
10.2.1 Cuestiones generales 521
10.2.2 Ejercicios (con `SceneBuilder`) 521
10.2.3 Actividades 529
10.2.4 Más ejercicios (Sin `SceneBuilder`) 530
10.3 Talleres 533
10.3.1 Taller UD09_02: Scene Builder, ScenicView y FXMLManager 533
10.3.2 Taller UD09_03: Proyecto JavaFX (con Maven) 538
10.3.3 Taller UD09_04: Calculadora en JavaFX (IntelliJ) 542
11. UD10 556
11.1 Acceso a Bases de Datos Relacionales 556
11.1.1 Introducción 556
11.1.2 JDBC 559
11.1.3 Patrones de diseño aplicables 563
11.1.4 Acceso a BBDD 566

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

11.1.5 Navegabilidad y concurrencia 571
11.1.6 Consultas (`Query`) 572
11.1.7 Modificación (`update`) 574
11.1.8 Inserción (`insert`) 575
11.1.9 Borrado (`delete`) 575
11.1.10 Sentencias predefinidas 576
11.1.11 Ejemplos UD10 577
11.1.12 Resumen — Conceptos clave 588
11.1.13 Autoevaluación 588
11.1.14 Vídeos recomendados 589
11.2 Ejercicios de la UD10 590
11.2.1 Actividades 590
11.3 Talleres 595
11.3.1 Taller UD10_02: Conectores 595
11.3.2 Taller UD10_03: BBDD en la nube (AWS) con IntelliJ 600
11.3.3 Taller UD10_04: Patron DAO (CRUD completo) 621
12. UD11 645
12.1 Bases de Datos Orientadas a Objetos 645
12.1.1 Introducción 645
12.1.2 Los lenguajes `ODL` y `OQL` 646
12.1.3 La librería `ObjectDB` 648
12.1.4 Anotaciones para `ObjectDB` 649
12.1.5 Comparativa BDR vs BDOO vs ORM 659
12.1.6 Ejemplos 660
12.1.7 Resumen — Conceptos clave 664
12.1.8 Autoevaluación 664
12.1.9 Vídeos recomendados 664
12.2 Ejercicios de la UD11 665
12.2.1 Actividades 665
12.2.2 Ejercicios 674
12.2.3 Ejercicios propuestos 676
12.2.4 Supuestos prácticos 677
12.2.5 Auto evaluación 677
12.3 Talleres 680
12.3.1 Taller UD11_02: Añadir ObjectDB a un proyecto IntelliJ (Maven) 680
12.3.2 Taller UD11_03: CRUD con ObjectDB 686
13. 👋 Sobre mí... 687
13.1 👨‍💻 David Martinez Peña 687

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

13.2 📬 Contacto: 687
14. Fuentes de información 688
14.1 Canales de vídeo 688

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

1. UD00

1.1 🚀 Información importante

PROGRAMACIÓN

https://martinezpenya.es/DAM/Programacion
2025 David Martínez licensed under CC BY-NC-SA 4.0

1.1.1 📛 Denominación del curso

📚 **Ciclo formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma**

☕ Programación (PRG)

1.1.2 📋 Contenidos

		Horas
Bloque	**P R I M E R** TRIMESTRE	**89**
B1	UD01: Elementos de un programa informático	19
B2	UD02: Utilización de Objetos	20
	PRUEBA UNIDADES 1 Y 2	**6**
B3	UD03: Estructuras de control y Excepciones	20
B4	UD04: Estructuras de datos Arrays y matrices. Recursividad	18
	1ª EVALUACIÓN	**6**
	S E G U N D O TRIMESTRE	**77**
B5	UD05: Desarrollo de clases	25
B6	UD06: Lectura y escritura de información	25
B4	UD07: Colecciones y Funciones Lambda	21
	2ª EVALUACIÓN	**6**
	T E R C E R TRIMESTRE	**90**
B8	UD08: Composición, Herencia y Polimorfismo	20
B9	UD09: Creación de interfaces gráficas	20
B10	UD10: Acceso a Bases de datos	24
B11	UD11: BBDD OO	16
	3ª EVALUACIÓN	**6**
	CONVOCATÒRIA ORDINÀRIA	**4**

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

	Horas
T O T A L	**256**

1.1.3 🎯 Resultados de Aprendizaje (RA)

	Descripció	Pes	UNITAT	Avaluació
RA1	**Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.**	**10%**		
A	Se han identificado los bloques que componen la estructura de un programa informático.	11%	1	=[[1AVA]]
B	Se han creado proyectos de desarrollo de aplicaciones	11%	2	=[[1AVA]]
C	Se han utilizado entornos integrados de desarrollo.	11%	2	=([[1AVA]]*0,5)+ ([[FEE]]*0,5)
D	Se han identificado los distintos tipos de variables y la utilidad específica de cada uno.	11%	1	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
E	Se ha modificado el código de un programa para crear y utilizar variables.	11%	1	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
F	Se han creado y utilizado constantes y literales.	11%	1	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
G	Se han clasificado, reconocido y utilizado en expresiones los operadores del lenguaje.	11%	1	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
H	Se ha comprobado el funcionamiento de las conversiones de tipo explícitas e implícitas.	11%	1	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
I	Se han introducido comentarios en el código.	11%	1	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
	Descripció	**Pes**	**UNITAT**	**Avaluació**
RA2	**Escribe y prueba programas sencillos, reconociendo y aplicando los fundamentos de la programación orientada a objetos.**	**10%**		

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

	Descripció	Pes	UNITAT	Avaluació
A	Se han identificado los fundamentos de la programación orientada a objetos.	11%	2	=[[1AVA]]
B	Se han escrito programas simples.	11%	2	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
C	Se han instanciado objetos a partir de clases predefinidas.	11%	2	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
D	Se han utilizado métodos y propiedades de los objetos.	11%	2	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
E	Se han escrito llamadas a métodos estáticos.	11%	2	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
F	Se han utilizado parámetros en la llamada a métodos.	11%	2	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
G	Se han incorporado y utilizado librerías de objetos.	11%	2	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
H	Se han utilizado constructores.	11%	2	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
I	Se ha utilizado el entorno integrado de desarrollo en la creación y compilación de programas simples.	11%	2	=([[1AVA]]*0,05)+ ([[2AVA]]*0,15)+ ([[3AVA]]*0,30)+ ([[FEE]]*0,50)
	Descripció	**Pes**	**UNITAT**	**Avaluació**
RA3	**Escribe y depura código, analizando y utilizando las estructuras de control del lenguaje.**	**10%**		
A	Se ha escrito y probado código que haga uso de estructuras de selección.	11%	3	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
B	Se han utilizado estructuras de repetición.	11%	3	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
C	Se han reconocido las posibilidades de las sentencias de salto.	11%	3	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
D	Se ha escrito código utilizando control de excepciones.	11%	3	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
E	Se han creado programas ejecutables utilizando diferentes estructuras de control.	11%	3	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

	Descripció	Pes	UNITAT	Avaluació
F	Se han probado y depurado los programas.	11%	3	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
G	Se ha comentado y documentado el código.	11%	3	=([[1AVA]]*0,05)+ ([[2AVA]]*0,15)+ ([[3AVA]]*0,30)+ ([[FEE]]*0,50)
H	**Se han creado excepciones.**	11%	3	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
I	**Se han utilizado aserciones para la detección y corrección de errores durante la fase de desarrollo.**	11%	3	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
	Descripció	**Pes**	**UNITAT**	**Avaluació**
RA4	**Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos.**	**10%**		
A	Se ha reconocido la sintaxis, estructura y componentes típicos de una clase.	11%	5	=([[2AVA]]*0,5)+ ([[3AVA]]*0,5)
B	Se han definido clases.	11%	5	=([[2AVA]]*0,5)+ ([[3AVA]]*0,5)
C	Se han definido propiedades y métodos.	11%	5	=([[2AVA]]*0,5)+ ([[3AVA]]*0,5)
D	Se han creado constructores.	11%	5	=([[2AVA]]*0,5)+ ([[3AVA]]*0,5)
E	Se han desarrollado programas que instancien y utilicen objetos de las clases creadas anteriormente.	11%	5	=([[2AVA]]*0,5)+ ([[3AVA]]*0,5)
F	Se han utilizado mecanismos para controlar la visibilidad de las clases y de sus miembros.	11%	5	=([[2AVA]]*0,5)+ ([[3AVA]]*0,5)
G	Se han definido y utilizado clases heredadas.	11%	5	=([[2AVA]]*0,5)+ ([[3AVA]]*0,5)
H	Se han creado y utilizado métodos estáticos.	11%	5	=([[2AVA]]*0,5)+ ([[3AVA]]*0,5)
I	Se han creado y utilizado conjuntos y librerías de clases.	11%	5	=([[2AVA]]*0,5)+ ([[3AVA]]*0,5)
	Descripció	**Pes**	**UNITAT**	**Avaluació**

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

	Descripció	Pes	UNITAT	Avaluació
RA5	**Realiza operaciones de entrada y salida de información, utilizando procedimientos específicos del lenguaje y librerías de clases.**	**15%**		
A	Se ha utilizado la consola para realizar operaciones de entrada y salida de información.	10%	6	=([[1AVA]]*0,05)+ ([[2AVA]]*0,15)+ ([[3AVA]]*0,30)+ ([[FEE]]*0,50)
B	Se han aplicado formatos en la visualización de la información.	10%	6	=([[1AVA]]*0,2)+ ([[2AVA]]*0,3)+ ([[3AVA]]*0,5)
C	Se han reconocido las posibilidades de entrada / salida del lenguaje y las librerías asociadas.	10%	6	=([[2AVA]]*0,5)+ ([[3AVA]]*0,5)
D	Se han utilizado ficheros para almacenar y recuperar información.	10%	6	=([[2AVA]]*0,5)+ ([[3AVA]]*0,5)
E	Se han creado programas que utilicen diversos métodos de acceso al contenido de los ficheros.	10%	6	=([[2AVA]]*0,5)+ ([[3AVA]]*0,5)
F	Se han utilizado las herramientas del entorno de desarrollo para crear interfaces gráficos de usuario simples.	20%	9	'=([[3AVA]]*0,25)+ ([[FEE]]*0,50)+ (([[UD09_T01]]*0,25)+ ([[UD09_T02]]*0,25)+ ([[UD09_T03]]*0,5)*0,25)
G	Se han programado controladores de eventos.	15%	9	'=([[3AVA]]*0,4)+ (([[UD09_T01]]*0,25)+ ([[UD09_T02]]*0,25)+ ([[UD09_T03]]*0,5)*0,6)
H	Se han escrito programas que utilicen interfaces gráficos para la entrada y salida de información.	15%	9	'=([[3AVA]]*0,4)+ (([[UD09_T01]]*0,25)+ ([[UD09_T02]]*0,25)+ ([[UD09_T03]]*0,5)*0,6)
	Descripció	**Pes**	**UNITAT**	**Avaluació**
RA6	**Escribe programas que manipulen información seleccionando y utilizando tipos avanzados de datos.**	**20%**		
A	Se han escrito programas que utilicen **matrices (arrays).**	50%	4	=([[2AVA]]*0,5)+ ([[3AVA]]*0,50)
B	Se han reconocido las librerías de clases	5%	7	=[[3AVA]]

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

	Descripció	Pes	UNITAT	Avaluació
	relacionadas con tipos de datos avanzados.			
C	Se han utilizado listas para almacenar y procesar información.	5%	7	=[[3AVA]]
D	Se han utilizado iteradores para recorrer los elementos de las listas.	5%	7	=[[3AVA]]
E	Se han reconocido las características y ventajas de cada una de la colecciones de datos disponibles.	10%	7	=[[3AVA]]
F	Se han creado clases y métodos genéricos.	5%	7	=[[3AVA]]
G	Se han utilizado expresiones regulares en la búsqueda de patrones en cadenas de texto.	5%	7	=[[3AVA]]
H	**Se han identificado las clases relacionadas con el tratamiento de documentos escritos en diferentes lenguajes de intercambio de datos.**	5%	7	=([[UD08_T01]]*0,5)+ ([[UD08_T02]]*0,5)
I	**Se han realizado programas que realicen manipulaciones sobre documentos escritos en diferentes lenguajes de intercambio de datos.**	5%	7	=([[UD08_T01]]*0,5)+ ([[UD08_T02]]*0,5)
J	**Se han utilizado operaciones agregadas para el manejo de información almacenada en colecciones.**	5%	7	=[[3AVA]]
	Descripció	**Pes**	**UNITAT**	**Avaluació**
RA7	**Desarrolla programas aplicando características avanzadas de los lenguajes orientados a objetos y del entorno de programación.**	**10%**		
A	Se han identificado los conceptos de herencia, superclase y subclase.	10%	8	=[[3AVA]]
B		10%	8	=[[3AVA]]

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

	Descripció	Pes	UNITAT	Avaluació
	Se han utilizado modificadores para bloquear y forzar la herencia de clases y métodos.			
C	Se ha reconocido la incidencia de los constructores en la herencia.	10%	8	=[[3AVA]]
D	Se han creado clases heredadas que sobrescriban la implementación de métodos de la superclase.	10%	8	=[[3AVA]]
E	Se han diseñado y aplicado jerarquías de clases.	10%	8	=[[3AVA]]
F	Se han probado y depurado las jerarquías de clases.	10%	8	=[[3AVA]]
G	Se han realizado programas que implementen y utilicen jerarquías de clases.	10%	8	=[[3AVA]]
H	Se ha comentado y documentado el código.	10%	8	=([[3AVA]]*0,50)+([[FEE]]*0,50)
I	**Se han identificado y evaluado los escenarios de uso de interfaces.**	10%	8	=[[3AVA]]
J	**Se han identificado y evaluado los escenarios de utilización de la herencia y la composición.**	10%	8	=[[3AVA]]
	Descripció	**Pes**	**UNITAT**	**Avaluació**
RA8	**Utiliza bases de datos orientadas a objetos, analizando sus características y aplicando técnicas para mantener la persistencia de la información.**	**5%**		
A	Se han identificado las características de las bases de datos orientadas a objetos.	13%	11	=([[UD11_T1]]*0,3)+([[UD11_T2]]*0,7)
B	Se ha analizado su aplicación en el desarrollo de aplicaciones mediante lenguajes orientados a objetos.	13%	11	=([[UD11_T1]]*0,3)+([[UD11_T2]]*0,7)

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

	Descripció	Pes	UNITAT	Avaluació
C	Se han instalado sistemas gestores de bases de datos orientados a objetos.	13%	11	=([[UD11_T1]]*0,3)+([[UD11_T2]]*0,7)
D	Se han clasificado y analizado los distintos métodos soportados por los sistemas gestores para la gestión de la información almacenada.	13%	11	=([[UD11_T1]]*0,3)+([[UD11_T2]]*0,7)
E	Se han creado bases de datos y las estructuras necesarias para el almacenamiento de objetos.	13%	11	=([[UD11_T1]]*0,3)+([[UD11_T2]]*0,7)
F	Se han programado aplicaciones que almacenen objetos en las bases de datos creadas.	13%	11	=([[UD11_T1]]*0,3)+([[UD11_T2]]*0,7)
G	Se han realizado programas para recuperar, actualizar y eliminar objetos de las bases de datos.	13%	11	=([[UD11_T1]]*0,3)+([[UD11_T2]]*0,7)
H	Se han realizado programas para almacenar y gestionar tipos de datos estructurados, compuestos y relacionados.	13%	11	=([[UD11_T1]]*0,3)+([[UD11_T2]]*0,7)
	Descripció	**Pes**	**UNITAT**	**Avaluació**
RA9	**Gestiona información almacenada en bases de datos relacionales manteniendo la integridad y consistencia de los datos.**	**10%**		
A	Se han identificado las características y métodos de acceso a sistemas gestores de bases de datos relacionales.	14%	10	=([[UD10_T1]]*0,2)+([[UD10_T2]]*0,3)+([[UD10_T3]]*0,5)
B	Se han programado conexiones con bases de datos.	14%	10	=([[UD10_T1]]*0,2)+([[UD10_T2]]*0,3)+([[UD10_T3]]*0,5)
C	Se ha escrito código para almacenar información en bases de datos.	14%	10	=([[UD10_T1]]*0,2)+([[UD10_T2]]*0,3)+([[UD10_T3]]*0,5)
D	Se han creado programas para recuperar y mostrar	14%	10	=([[UD10_T1]]*0,2)+([[UD10_T2]]*0,3)+([[UD10_T3]]*0,5)

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

	Descripció	Pes	UNITAT	Avaluació
	información almacenada en bases de datos.			
E	Se han efectuado borrados y modificaciones sobre la información almacenada.	14%	10	=([[UD10_T1]]*0,2)+ ([[UD10_T2]]*0,3)+ ([[UD10_T3]]*0,5)
F	Se han creado aplicaciones **que muestren la información almacenada en** bases de datos.	14%	10	=([[UD10_T1]]*0,05)+ ([[UD10_T2]]*0,15)+ ([[UD10_T3]]*0,3)+ ([[FEE]]*0,5)
G	Se han creado aplicaciones para gestionar la información presente en bases de datos **relacionales**.	14%	10	=([[UD10_T1]]*0,05)+ ([[UD10_T2]]*0,15)+ ([[UD10_T3]]*0,3)+ ([[FEE]]*0,5)

1.1.4 📜 Legislación vigente

- 📄 RD 405/2023, de 29 de mayo (BOE núm. 132, 03-06-2023) — título y enseñanzas mínimas vigentes
- 📄 DECRETO 114/2025, de 29 de julio, del Consell — currículo CV (DOGV núm. 10165, 04-08-2025) — deroga expresamente la ORDE 58/2012 e incluye el currículo de DAM (junto con SMR, ASIR y DAW)
- 🕒 Secuenciación y horario: **incluidos como Anexo II** del mismo DECRETO 114/2025

1.1.5 📝 Evaluación

- 🔍 La evaluación del módulo se realizará con base en los **Resultados de Aprendizaje (RA)** definidos en el currículo del ciclo formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma. Cada RA estará asociado a **criterios de evaluación** (**CE**) que serán los que determinen el grado de adquisición de las competencias previstas para el módulo.
- 📊 La nota final del módulo sale de la ponderación de los RA. Cada RA se evalúa de forma independiente, con **calificación de 1 a 10, sin decimales** (Orden 8/2025, art. 5.1).
- ✅ El alumno debe obtener al menos una nota de **5** en cada **RA** para aprobar el módulo.
- 🔄 Si un alumno obtiene menos de un **5** en algún RA, tendrá que recuperarlo mediante las actividades/exámenes de recuperación diseñadas específicamente para esos resultados de aprendizaje.
- 📊En programación los primeros RA's se distribuyen entre las 3 evaluaciones, así que tener una buena nota en la primera evaluación no quiere decir que has aprobado los RA de esa evaluación.
- ⏰ **Las entregas fuera de plazo tienen un máximo de 5.** El plazo de cada entrega se cierra en la fecha que indica Moodle. Si necesitas entregar después, **avísame** y te reabro esa tarea concreta un tiempo limitado; en ese caso la nota máxima del trabajo es **5 sobre 10**, y la rúbrica lo recoge de forma explícita. Vale para **todo el curso**, así que no se repite en cada unidad.
- ❗**NUEVO SISTEMA DUAL!!** → Busca tu empresa! 120H (aproximadamente en el mes de mayo, también a partir del 2º trimestre por las mañanas)

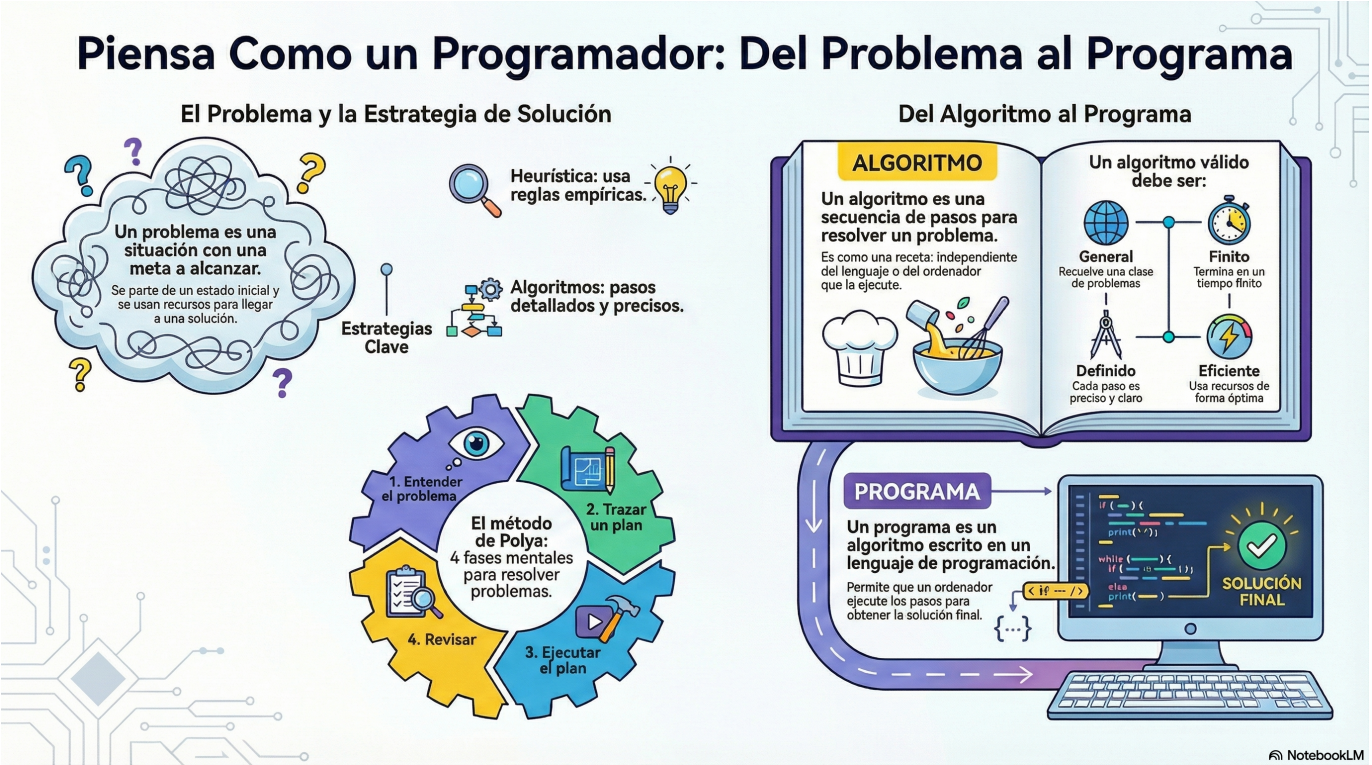
> ⚠️ **Importante**
>
> - ❗ Aprobar las distintas evaluaciones no garantiza aprobar el curso.
> - 📌 Puedes aprobar (y con muy buena nota) las dos evaluaciones, tener un `RA` suspendido y por tanto suspender el módulo.

14 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

2. UD01

2.1 Elementos de un programa informático



Al finalizar esta unidad serás capaz de...

- Identificar los elementos fundamentales de un programa informático
- Definir qué es un problema, un algoritmo y un programa
- Conocer la historia y características del lenguaje Java
- Declarar variables y constantes utilizando los tipos de datos adecuados
- Utilizar operadores aritméticos, relacionales y lógicos
- Realizar conversiones de tipo (implícitas y explícitas)
- Compilar y ejecutar un programa Java desde la consola

2.1.1 Piensa como un programador

Una de las acepciones que trae el Diccionario de Real Academia de la Lengua Española (RAE) respecto a la palabra Problema es **"Planteamiento de una situación cuya respuesta desconocida debe obtenerse a través de métodos científicos"**. Con miras a lograr esa respuesta.

Definición

Un problema es una situación en la cual se trata de alcanzar una meta y para lograrlo se deben hallar y utilizar unos medios y unas estrategias.

La mayoría de problemas tienen algunos elementos en común: un estado inicial; una meta, lo que se pretende lograr; un conjunto de recursos, lo que está permitido hacer y/o utilizar; y un dominio, el estado actual de conocimientos, habilidades y energía de quien va a resolverlo (Moursund, 1999).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Casi todos los problemas requieren, que quien los resuelve, los divida en submetas que, cuando son dominadas (por lo regular en orden), llevan a alcanzar el objetivo. La solución de problemas también requiere que se realicen operaciones durante el estado inicial y las submetas, actividades (conductuales, cognoscitivas) que alteran la naturaleza de tales estados (Schunk, 1997).

Cada disciplina dispone de estrategias específicas para resolver problemas de su ámbito; por ejemplo, resolver problemas matemáticos implica utilizar estrategias propias de las matemáticas. Sin embargo, algunos psicólogos opinan que es posible utilizar con éxito estrategias generales, útiles para resolver problemas en muchas áreas. A través del tiempo, la humanidad ha utilizado diversas estrategias generales para resolver problemas. Schunk (1997), Woolfolk (1999) y otros, destacan los siguientes métodos o estrategias de tipo general:

- **Ensayo y error** : Consiste en actuar hasta que algo funcione. Puede tomar mucho tiempo y no es seguro que se llegue a una solución. Es una estrategia apropiada cuando las soluciones posibles son pocas y se pueden probar todas, empezando por la que ofrece mayor probabilidad de resolver el problema.

Por ejemplo, una bombilla que no prende: revisar la bombilla, verificar la corriente eléctrica, verificar el interruptor.

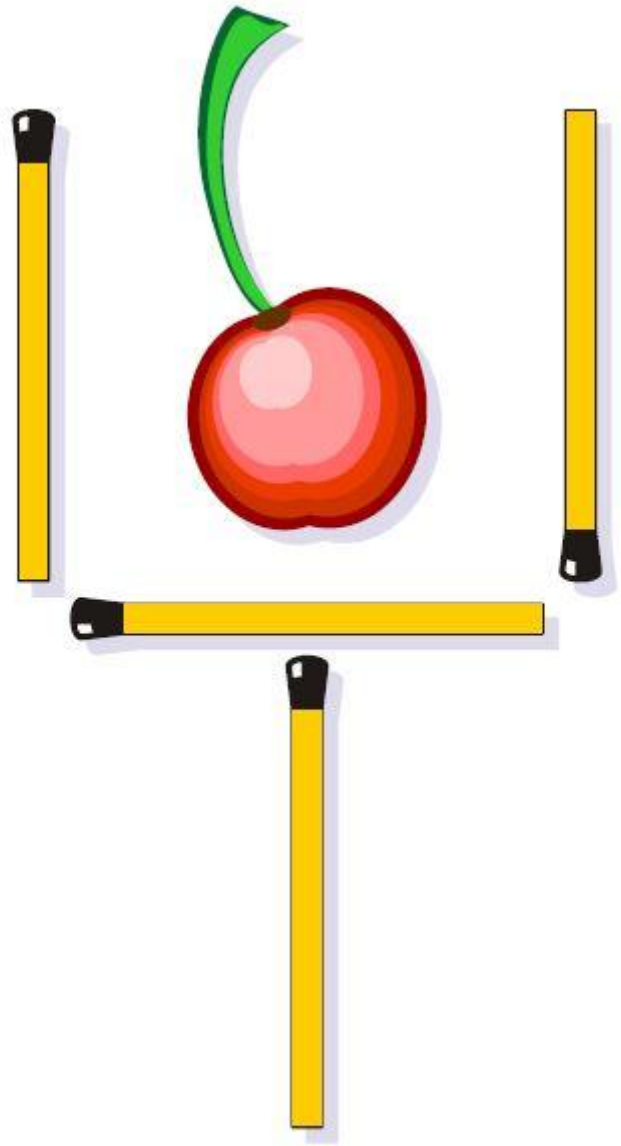
- **Iluminación** : Implica la súbita conciencia de una solución que sea viable. Es muy utilizado el modelo de cuatro pasos formulado por Wallas (1921): preparación, incubación, iluminación y verificación.

Estos cuatro momentos también se conocen como proceso creativo. **Algunas investigaciones han determinado que cuando en el periodo de incubación se incluye una interrupción en el trabajo sobre un problema se logran mejores resultados desde el punto de vista de la creatividad**. La incubación ayuda a "olvidar" falsas pistas, mientras que no hacer interrupciones o descansos puede hacer que la persona que trata de encontrar una solución creativa se estanque en estrategias inapropiadas.

Ejemplos:

- Dispones de 6 lapices/palillos/cerillas igual de largos, ¿como puedes formar 4 triángulos iguales y equiláteros?
- Mueve 2 cerillas para seguir teniendo una copa pero con la cereza fuera:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



- **Heurística** : Se basa en la utilización de reglas empíricas para llegar a una solución. El método heurístico conocido como “IDEAL”, formulado por Bransford y Stein (1984), incluye cinco pasos: Identificar el problema; definir y presentar el problema; explorar las estrategias viables; avanzar en las estrategias; y lograr la solución y volver para evaluar los efectos de las actividades (Bransford & Stein, 1984).

El matemático Polya (1957) también formuló un método heurístico para resolver problemas que se aproxima mucho al ciclo utilizado para programar computadores. A lo largo de esta Guía se utilizará este método propuesto por Polya.

- **Algoritmos** : Consiste en aplicar adecuadamente una serie de pasos detallados que aseguran una solución correcta. Por lo general, cada algoritmo es específico de un dominio del conocimiento. La programación de computadores se apoya en este método.
- **Modelo de procesamiento de información** : El modelo propuesto por Newell y Simon (1972) se basa en plantear varios momentos para un problema (estado inicial, estado final y vías de solución). Las posibles soluciones avanzan por subtemas y requieren que se realicen operaciones en cada uno de ellos.
- **Análisis de medios y fines** : Se funda en la comparación del estado inicial con la meta que se pretende alcanzar para identificar las diferencias.

Luego se establecen submetas y se aplican las operaciones necesarias para alcanzar cada submeta hasta que se alcance la meta global. Con este método se puede proceder en retrospectiva (desde la meta hacia el estado inicial) o en prospectiva (desde el estado inicial hacia la meta).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- **Razonamiento analógico** : Se apoya en el establecimiento de una analogía entre una situación que resulte familiar y la situación problema. Requiere conocimientos suficientes de ambas situaciones.
- **Lluvia de ideas** : Consiste en formular soluciones viables a un problema. El modelo propuesto por Mayer (1992) plantea: definir el problema; generar muchas soluciones (sin evaluarlas); decidir los criterios para estimar las soluciones generadas; y emplear esos criterios para seleccionar la mejor solución. Requiere que los estudiantes no emitan juicios con respecto a las posibles soluciones hasta que terminen de formularlas.
- **Sistemas de producción** : Se basa en la aplicación de una red de secuencias de condición y acción (Anderson, 1990).
- **Pensamiento lateral** : Se apoya en el pensamiento creativo, formulado por Edwar de Bono (1970), el cual difiere completamente del pensamiento lineal (lógico). El pensamiento lateral requiere que se exploren y consideren la mayor cantidad posible de alternativas para solucionar un problema. Su importancia para la educación radica en permitir que el estudiante: explore (escuche y acepte puntos de vista diferentes, busque alternativas); avive (promueva el uso de la fantasía y del humor); libere (use la discontinuidad y escape de ideas preestablecidas); y contrarreste la rigidez (vea las cosas desde diferentes ángulos y evite dogmatismos). Este es un método adecuado cuando el problema que se desea resolver no requiere información adicional, sino un reordenamiento de la información disponible; cuando hay ausencia del problema y es necesario apercibirse de que hay un problema; o cuando se debe reconocer la posibilidad de perfeccionamiento y redefinir esa posibilidad como un problema (De Bono, 1970).

Ejemplos:

- **El dilema del náufrago.** Un náufrago necesita trasladar a su isla de residencia algunos restos del naufragio de su barco, que afloraron en la orilla de la isla de enfrente. Allí tiene un zorro, un conejo y un racimo de zanahorias, que en su bote puede llevar a razón de uno por viaje. ¿Cómo puede llevarlo todo a su isla, sin que el zorro se coma al conejo, ni éste a las zanahorias?

  Respuesta: Deberá llevar primero al conejo y dejar al zorro con las zanahorias. Luego volver y llevarse al zorro, que dejará a solas en su isla, tomar al conejo y llevarlo de vuelta a la de enfrente. Después llevará las zanahorias, dejando al conejo solo y depositándolas junto al zorro. Finalmente regresará para hacer un último viaje con el conejo.
- **El dilema del ascensor.** Un hombre que vive en el décimo piso de un edificio, toma todos los días el ascensor hasta la planta baja, para ir a trabajar. En la tarde, sin embargo, toma de nuevo el mismo ascensor, pero si no hay nadie con él, baja en el séptimo piso y sube el resto de los pisos por la escalera. ¿Por qué?

  Respuesta: El hombre es bajito y no logra presionar el botón del décimo piso.
- **La paradoja del globo.** ¿De qué manera podemos pinchar un globo con una aguja, sin que se fugue el aire y sin que el globo estalle?

  Respuesta: Debemos pinchar el globo estando desinflado.
- **El dilema del bar.** Un hombre entra a un bar y le pide al barman un vaso de agua. El barman busca debajo de la barra y de golpe apunta al hombre con un arma. Este último da las gracias y se marcha. ¿Qué acaba de ocurrir?

  Respuesta: El barman se percató de que el hombre tenía hipo, y decide curárselo dándole un buen susto.

Resumen

Existen muchas estrategias de resolución de problemas, pero esta Guía se enfoca principalmente en dos: **Heurística** (basada en reglas empíricas) y **Algorítmica** (pasos detallados que aseguran una solución correcta).

Según Polya (1957), cuando se resuelven problemas, intervienen cuatro operaciones mentales:

1. Entender el problema;
2. Trazar un plan;
3. Ejecutar el plan (resolver);
4. Revisar;

Es importante notar que estas son flexibles y no una simple lista de pasos como a menudo se plantea en muchos de esos textos (Wilson, Fernández & Hadaway, 1993). Cuando estas etapas se siguen como un modelo lineal, resulta contraproducente para cualquier actividad encaminada a resolver problemas.

Es necesario hacer énfasis en la naturaleza dinámica y cíclica de la solución de problemas. En el intento de trazar un plan, los estudiantes pueden concluir que necesitan entender mejor el problema y deben regresar a la etapa anterior; O cuando han trazado un plan y tratan de ejecutarlo, no encuentran cómo hacerlo; entonces, la actividad siguiente puede ser intentar con un nuevo plan o regresar y desarrollar una nueva comprensión del problema (Wilson, Fernández & Hadaway, 1993; Guzdial, 2000).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Recuerda

Las cuatro fases de Polya (Entender, Planificar, Ejecutar, Revisar) son dinámicas y cíclicas, no una simple lista lineal de pasos. Su aplicación a la programación es directa: Analizar, Diseñar el algoritmo, Traducir a código, Depurar.

Referencia

La mayoría de los textos escolares de matemáticas abordan la Solución de Problemas bajo el enfoque planteado por Polya.

Importante

Las fases de la programación son una traducción directa del método de Polya:

Polya	Programación
Entender el problema	Analizar el problema
Trazar un plan	Diseñar un algoritmo
Ejecutar el plan	Traducir a lenguaje de programación
Revisar	Depurar el programa

Como se puede apreciar, hay una similitud entre las metodologías propuestas para solucionar problemas matemáticos (Clements & Meredith, 1992; Díaz, 1993; Melo, 2001; NAP, 2004) y las cuatro fases para solucionar problemas específicos de áreas diversas, mediante la programación de computadores.

Actividad

Problema de la Jirafa

Primera pregunta: ¿Cómo podríamos meter una jirafa dentro de una nevera? Piensa que es un problema para niños y a ellos no se les pasaría por la cabeza trocear al bello animal para resolver un problema.

Segunda pregunta: Repetimos la jugada con distinto protagonista. ¿Cómo metemos un elefante dentro de la nevera?

Tercera pregunta: Imaginemos que el Rey León está celebrando su cumpleaños y ha invitado a todos los animales del reino. Acuden todos excepto uno. ¿Quién falta?

Cuarta pregunta: Estamos frente a un río que debemos cruzar como sea para continuar nuestro camino. El único problema es que esa zona es el hogar de unos cocodrilos muy agresivos y no disponemos de ningún tipo de embarcación para ir al otro lado. ¿Cómo harías para cruzar el río sin morir en el intento?

2.1.2 Problemas, algoritmos y programas

Problemas

Definición

La **programación** es una forma de resolución de problemas mediante una secuencia de instrucciones que pueden ejecutarse de forma automática en un ordenador.

Para que un problema pueda resolverse utilizando un programa informático, éste tiene que poder resolverse de forma mecánica, es decir, mediante una secuencia de instrucciones u operaciones que se puedan llevar a cabo de manera **automática** por un ordenador.

Ejemplos de problemas resolubles mediante un ordenador:

- Determinar el producto de dos números a y b.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- Determinar la raíz cuadrada positiva del número 2.
- Determinar la raíz cuadrada positiva de un número n cualquiera.
- Determinar si el número n, entero mayor que uno, es primo.
- Dada la lista de palabras, determinar las palabras repetidas.
- Determinar si la palabra p es del idioma castellano.
- Ordenar y listar alfabéticamente todas las palabras del castellano.
- Dibujar en pantalla un círculo de radio r.
- Separar las silabas de una palabra p.
- A partir de la fotografía de un vehículo, reconocer y leer su matrícula.
- Traducir un texto de castellano a inglés.
- Detectar posibles tumores a partir de imágenes radiográficas.

Por otra parte, el científico Alan Turing, demostró que existen problemas irresolubles, de los que ningún ordenador será capaz de obtener nunca su solución.

Atención

Los problemas deben definirse de forma **general** y **precisa**, evitando ambigüedades. Una definición ambigua conduce a interpretaciones distintas y, por tanto, a soluciones incorrectas.

Ejemplo: Raíz cuadrada.

- Determinar la raíz cuadrada de un número n.
- Determinar la raíz cuadrada de un número n, entero no negativo, cualquiera.

Ejemplo: Dividir.

- Calcular la división de dos números de dos números a y b.
- Calcular el cociente entero de la división a/b, donde a y b son números enteros y b es distinto de cero. (5/2 = 2).
- Calcular el cociente real de la división a/b, donde a y b son números reales y b es distinto de cero (5/2 = 2.5).

Algoritmos

Definición

Dado un problema P, un **algoritmo** es un conjunto de reglas o pasos que indican cómo resolver P en un tiempo finito. Es independiente del lenguaje de programación y del dispositivo donde se ejecute.

Ejemplo

Secuencias de reglas básicas que utilizamos para realizar operaciones aritméticas: sumas, restas, productos y divisiones.

Algoritmo para desayunar

```
Begin
    Sentarse
    Servirse café con leche
    Servirse azucar
    If tengo tiempo
        While tenga apetito
            Untar mantequilla en una tostada
            Añadir mermelada
            Comer la tostada
        End While
    End If
    Beberse el café con leche
    Levantarse
End
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Un algoritmo, por tanto, no es más que la secuencia de pasos que se deben seguir para solucionar un problema específico. La descripción o nivel de detalle de la solución de un problema en términos algorítmicos depende de qué o quién debe entenderlo, interpretarlo y resolverlo.

Los algoritmos son independientes de los lenguajes de programación y de las computadoras donde se ejecutan. Un mismo algoritmo puede ser expresado en diferentes lenguajes de programación y podría ser ejecutado en diferentes dispositivos. Piensa en una receta de cocina, ésta puede ser expresada en castellano, inglés o francés, podría ser cocinada en fogón o vitrocerámica, por un cocinero o más, etc. Pero independientemente de todas estas circunstancias, el plato se preparará siguiendo los mismos pasos.

Importante

La diferencia fundamental entre **algoritmo** y **programa**: el algoritmo describe los pasos para resolver un problema (independiente del lenguaje), mientras que el programa es la implementación concreta de ese algoritmo en un lenguaje de programación específico para ser ejecutado en un ordenador.

CARACTERÍSTICAS DE LOS ALGORITMOS

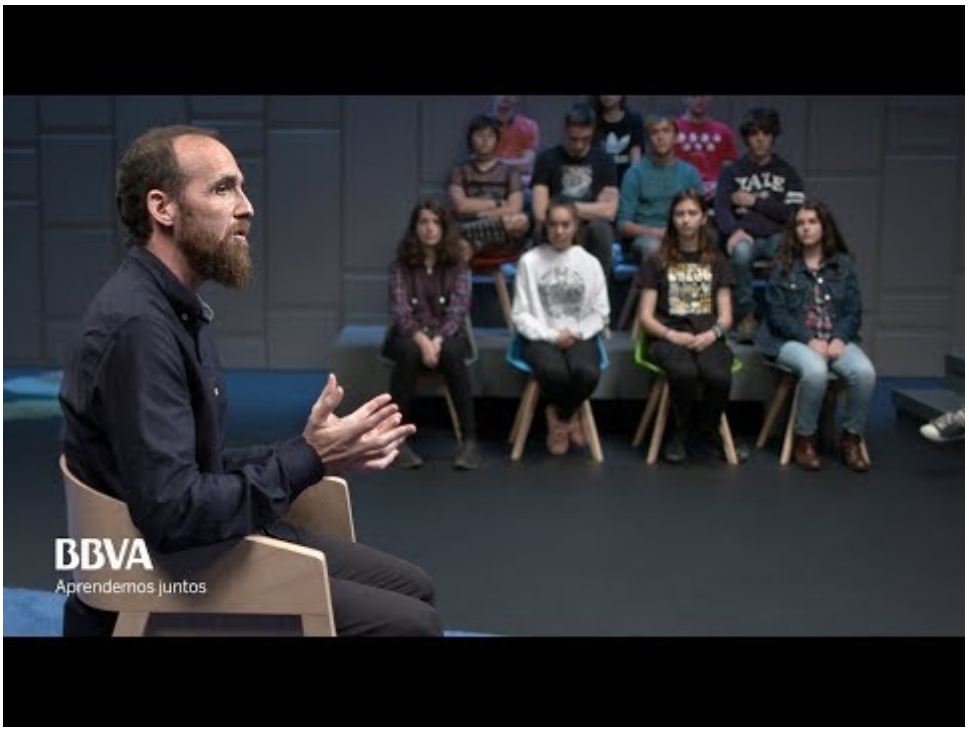
Atención

Un algoritmo debe cumplir cuatro características para ser válido:

- **Generalidad**: debe resolver toda una clase de problemas, no uno aislado.
- **Finitud**: debe acabar necesariamente tras un número finito de pasos.
- **Definibilidad**: debe estar definido de forma exacta y precisa, sin ambigüedades.
- **Eficiencia**: debe resolver el problema de forma rápida y eficiente.

Vídeo

Juego de las monedas (Eduardo Sáenz Cabezón)



Desde el comienzo del enlace hasta 7 minutos después.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

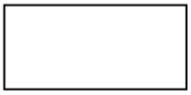
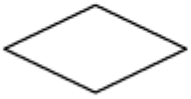
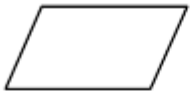
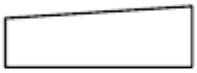
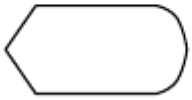
REPRESENTACIÓN DE ALGORITMOS

Los métodos más usuales para representar algoritmos son los diagramas de flujo y el pseudocódigo. Ambos son sistemas de representación independientes de cualquier lenguaje de programación. Hay que tener en cuenta que el diseño de un algoritmo constituye un paso previo a la codificación de un programa en un lenguaje de programación determinado (C, C++, Java, Pascal). La independencia del algoritmo del lenguaje de programación facilita, precisamente, la posterior codificación en el lenguaje elegido.

Definición

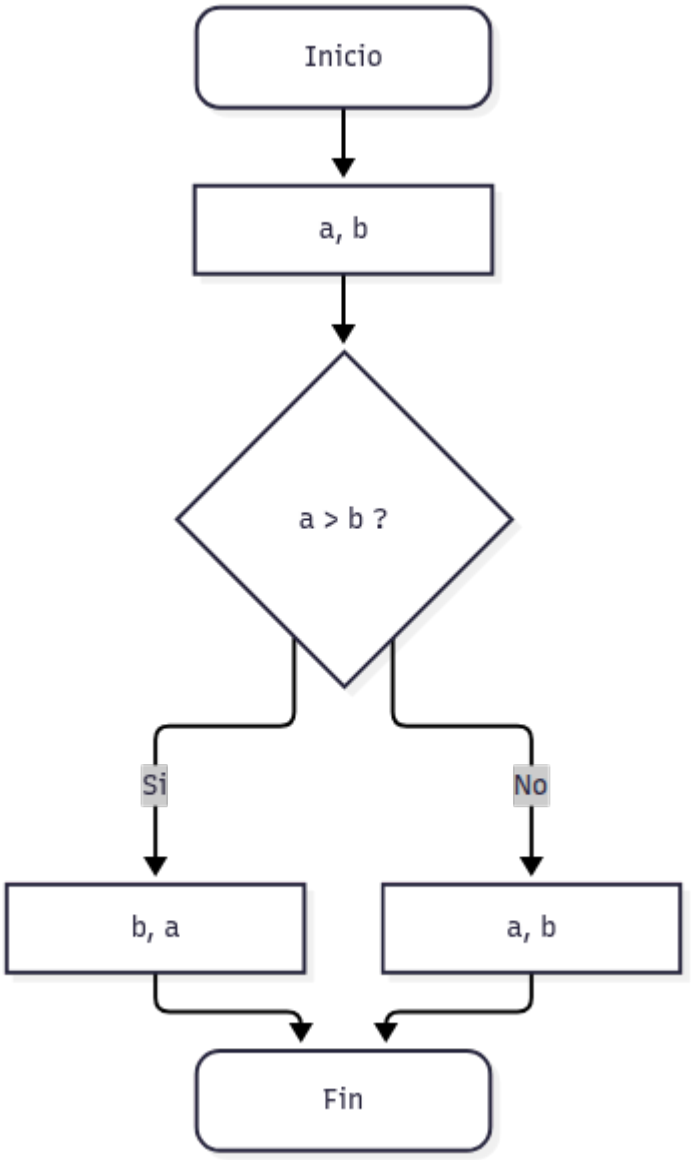
Un **diagrama de flujo** es una representación gráfica de un algoritmo mediante símbolos estándar (inicio/fin, proceso, decisión, entrada/salida) unidos por flechas que indican el orden de ejecución.

Los símbolos más utilizados son:

Proceso		Cualquier tipo de operación que pueda originar cambio de valor, formato, operaciones aritméticas etc
Decisión		Iindica operaciones lógicas o de comparación entre datos y en función del resultado determina el camino a seguir.
Terminal		Representa el comienza y el final de un programa o un módulo
Entrada / Salida de información		Este símbolo se puede subdividir en otros de teclado, pantalla, impresora disco etc.
Teclado		Representa las entradas de datos desde teclado
Pantalla		Representa las salidas de datos en pantalla
Dirección del flujo		Indica el sentido de ejecución de las operaciones

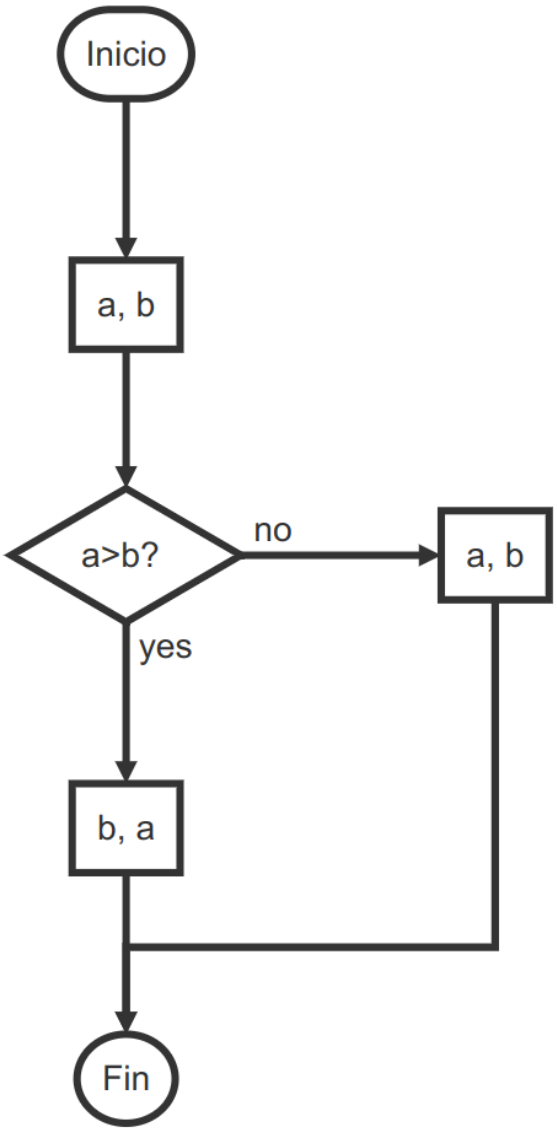
Ejemplo: Mostrar dos números ordenados de menor a mayor.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



O también en otra representación:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Definición

El **pseudocódigo** es un lenguaje informal de descripción de algoritmos, cercano a la sintaxis de los lenguajes de programación, que NO puede ejecutarse en un ordenador. Debe traducirse (codificarse) a un lenguaje real.

El pseudocódigo no se puede ejecutar nunca en el ordenador, sino que tiene que traducirse a un lenguaje de programación (codificación). La ventaja del pseudocódigo, frente a los diagramas de flujo, es que se puede modificar más fácilmente si detecta un error en la lógica del algoritmo, y puede ser traducido fácilmente a los lenguajes estructurados como Pascal, C, fortran, Java, etc.

El Pseudocódigo utiliza palabras reservadas (en sus orígenes se escribían en inglés) para representar las sucesivas acciones. Para mayor legibilidad utiliza la **identación** (sangría en el margen izquierdo) de sus líneas.

Ejemplo: Mostrar dos números ordenados de menor a mayor.

```
Begin
    Leer (A, B)
    If (A>B) then
        Escribir (B, A)
    Else
        Escribir (A, B)
    End If
End
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Programas

Los lenguajes de programación son sólo un medio para expresar el algoritmo y el ordenador un procesador para ejecutarlo. El diseño de los algoritmos será una tarea que necesitará de la creatividad y conocimientos de las técnicas de programación. Estilos distintos, de distintos programadores a la hora de obtener la solución del problema, darán lugar a programas diferentes, igualmente válidos.

Pero cuando los problemas son complejos, es necesario descomponer éstos en subproblemas más simples y, a su vez, en otros más pequeños. Estas estrategias reciben el nombre de diseño descendente (Metodología de diseño de programas, consistente en la descomposición del problema en problemas más sencillos de resolver) o diseño modular (top-down design) (Metodología de diseño de programas, que consiste en dividir la solución a un problema en módulos más pequeños o subprogramas. Las soluciones de los módulos se unirán para obtener la solución general del problema). Este sistema se basa en el lema **divide y vencerás**.

> **Recuerda**
>
> El **diseño descendente** (divide y vencerás) es una de las estrategias más importantes en programación: descomponer un problema complejo en subproblemas más simples, y estos a su vez en otros más pequeños, hasta que cada pieza sea fácil de resolver.

2.1.3 Java

¿Qué y cómo es Java?

Java es un lenguaje sencillo de aprender, con una sintaxis parecida a la de C++, pero en la que se han eliminado elementos complicados y que pueden originar errores. Java es orientado a objetos, con lo que elimina muchas preocupaciones al programador y permite la utilización de gran cantidad de bibliotecas ya definidas, evitando reescribir código que ya existe. Es un lenguaje de programación creado para satisfacer nuevas necesidades que los lenguajes existentes hasta el momento no eran capaces de solventar.

> **Información**
>
> La principal virtud de Java es su **independencia de plataforma**: el código se compila a *bytecode*, que se ejecuta sobre la Máquina Virtual Java (JVM), permitiendo que un mismo programa funcione en cualquier sistema operativo. Lema: "*Write once, run everywhere*".

Antes de que apareciera Java, el lenguaje C era uno de los más extendidos por su versatilidad. Pero cuando los programas escritos en C aumentaban de volumen, su manejo comenzaba a complicarse. Mediante las técnicas de programación estructurada y programación modular se conseguían reducir estas complicaciones, pero no era suficiente.

Fue entonces cuando la Programación Orientada a Objetos (POO) entra en escena, aproximando notablemente la construcción de programas al pensamiento humano y haciendo más sencillo todo el proceso. Los problemas se dividen en objetos que tienen propiedades e interactúan con otros objetos, de este modo, el programador puede centrarse en cada objeto para programar internamente los elementos y funciones que lo componen.

> **Resumen**
>
> Características clave de Java: independiente de plataforma, orientado a objetos, sintaxis similar a C/C++, distribuido (TCP/IP), amplias bibliotecas, robusto (comprobaciones en compilación y ejecución), y seguro.

Breve historia.

Java surgió en 1991 cuando un grupo de ingenieros de Sun Microsystems trataron de diseñar un nuevo lenguaje de programación destinado a programar pequeños dispositivos electrónicos. La dificultad de estos dispositivos es que cambian continuamente y para que un programa funcione en el siguiente dispositivo aparecido, hay que reescribir el código. Por eso la empresa Sun quería crear un lenguaje independiente del dispositivo.

Pero no fue hasta 1995 cuando pasó a llamarse Java, dándose a conocer al público como lenguaje de programación para computadores. Java pasa a ser un lenguaje totalmente independiente de la plataforma y a la vez potente y orientado a objetos. Esa filosofía y su facilidad para crear aplicaciones para redes TCP/IP ha hecho que sea uno de los lenguajes más utilizados en la actualidad.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

El factor determinante para su expansión fue la incorporación de un intérprete Java en la versión 2.0 del navegador Web Netscape Navigator, lo que supuso una gran revuelo en Internet. A principios de 1997 apareció Java 1.1 que proporcionó sustanciales mejoras al lenguaje. Java 1.2, más tarde rebautizado como Java 2, nació a finales de 1998.

El principal objetivo del lenguaje Java es llegar a ser el nexo universal que conecte a los usuarios con la información, esté ésta situada en el ordenador local, en un servidor Web, en una base de datos o en cualquier otro lugar.

Para el desarrollo de programas en lenguaje Java es necesario utilizar un entorno de desarrollo denominado JDK (Java Development Kit), que provee de un compilador y un entorno de ejecución (JRE – Java RunEnvironment) para los bytecodes generados a partir del código fuente. Al igual que las diferentes versiones del lenguaje han incorporado mejoras, el entorno de desarrollo y ejecución también ha sido mejorado sucesivamente.

Java 2 es la tercera versión del lenguaje, pero es algo más que un lenguaje de programación, incluye los siguientes elementos:

- Un lenguaje de programación: Java.
- Un conjunto de bibliotecas estándar que vienen incluidas en la plataforma y que son necesarias en todo entorno Java. Es el Java Core.
- Un conjunto de herramientas para el desarrollo de programas, como es el compilador de bytecodes, el generador de documentación, un depurador, etc.
- Un entorno de ejecución que en definitiva es una máquina virtual que ejecuta los programas traducidos a bytecodes.

Compilar y ejecutar un programa `Java`. Uso de la consola.

Veamos los pasos para compilar e interpretar nuestro primer programa escrito en lenguaje Java.

ESTRUCTURA Y BLOQUES FUNDAMENTALES DE UN PROGRAMA.

Ejemplo Holamundo.java

Consulta el Ejemplo01 — Hola Mundo para ver el código completo.

Atención

En Java, el nombre del archivo `.java` debe coincidir **exactamente** con el nombre de la clase pública que contiene (respetando mayúsculas y minúsculas). Por ejemplo: la clase `public class Ejemplo` debe guardarse en `Ejemplo.java`.

```
public class Holamundo {
    [...]
}
```

El código java en las clases se agrupa en funciones o métodos. Cuando java ejecuta el código de una clase busca la función o método `main()` para ejecutarla. Es público (`public`) estático (`static`) para llamarlo sin instanciar la clase. No devuelve ningún valor (`void`) y admite parámetros (`Strings[] args`) que en este caso no se han utilizado.

```
[...]
    public static void main (String[] args)
    {
        [...]
    }
[...]
```

El código de la función `main` se escribe entre las llaves. Por ejemplo:

```
[...]
        System.out.println("Hola Mundo");
[...]
```

Muestra por pantalla el mensaje `Hola Mundo`, ya que la clase `System` tiene un atributo `out` con dos métodos: `print()` y `println()`. La diferencia es que `println` muestra mensaje e introduce un retorno de carro.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Importante

En Java, toda instrucción debe terminar con punto y coma (`;`). Las únicas excepciones son las llaves de apertura `{` y cierre `}`.

SANGRADO O TABULADO

Recomendación

Aplica sangrado (indentación) consistente de 2-4 espacios en cada nivel lógico. Para principiantes se recomiendan 4 espacios. No mezcles tabuladores con espacios; elige uno y sé consistente.

Recomendación

Limita las líneas de código a 70-90 caracteres. Si una línea es más larga, divídela: - Tras una coma - Antes de un operador (que pasará a la línea siguiente) - En una construcción de alto nivel (paréntesis) - La línea continuada debe alinearse con un sangrado lógico adicional

Unos pocos ejemplos, para comprender mejor:

Unos pocos ejemplos, para comprender mejor:

Dividir tras una coma:

```
funcion(expresionMuuuuyLarga1,
          expresionMuuuyyyyLarga2,
          expresionMuuuyyyLarga3);
```

Mantener la expresión entre paréntesis en la misma línea:

```
nombreLargo = nombreLargo2*
              (nombreLargo3 + nombreLArgo4)+
              4*nombreLargo5;
```

Siempre hay excepciones. Puede resultar que al aplicar estas reglas, en operaciones muy largas, o expresiones lógicas enormes, el sangrado sea ilegible. En estos casos, el convenio se puede relajar.

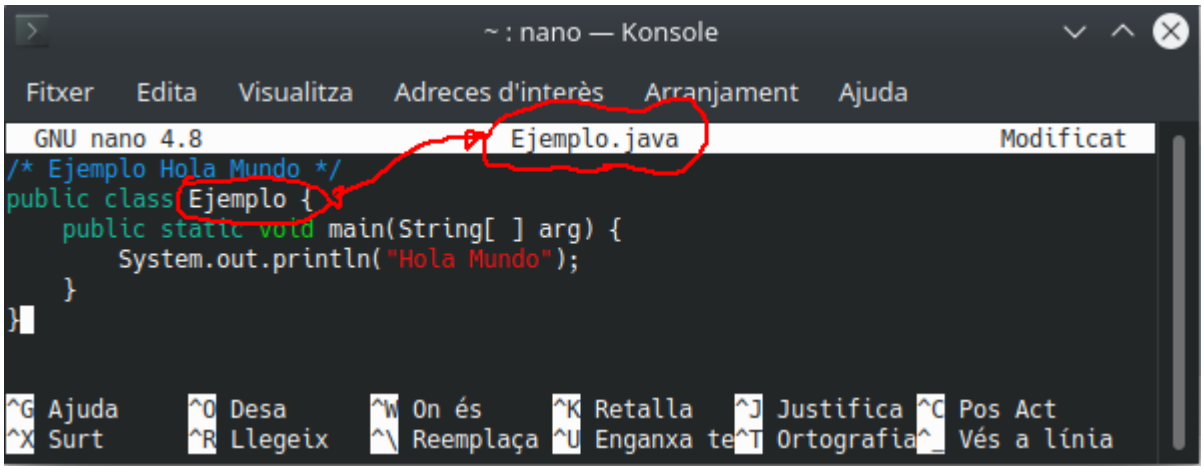
PASO 1: CREACIÓN DEL CÓDIGO FUENTE

Abrimos un editor de texto (da igual cual sea, siempre que sea capaz de almacenar "texto sin formato" en código ASCII). Una vez abierto escribiremos nuestro primer programa, que mostrará un texto "Hola Mundo" en la consola. De momento no te preocupes si no entiendes lo que escribes, más adelante le daremos sentido. Ahora solo queremos ver si podemos ejecutar java en nuestro equipo.

El código de nuestro programa en Java será el siguiente (consulta el Ejemplo02 — Compilación y ejecución para ver el ciclo completo):

A continuación guardamos nuestro archivo y le ponemos como nombre `Ejemplo.java`. Debemos seguir una norma dictada por Java, hemos de hacer coincidir nombre del archivo y nombre del programa, tanto en mayúsculas como en minúsculas, y la extensión del archivo habrá de ser siempre `.java`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Debemos recordar exactamente la ruta donde guardamos el archivo de ejemplo `Ejemplo.java`.

PASO 2: COMPILACIÓN DEL PROGRAMA

Vamos a proceder a compilar e interpretar este pequeño programa Java (no te preocupes si todavía no entiendes el significado de las palabras compilar e interpretar, lo veras en la asignatura de `Entornos de Desarrollo`). Para ello usaremos la consola. Una vez en la consola debemos colocarnos en la ruta donde previamente guardamos el archivo `Ejemplo.java`.

A continuación daremos la instrucción para que se realice **el proceso de compilación del programa**, para lo que escribiremos `javac Ejemplo.java`, donde `javac` es el nombre del compilador (`java c` ompiler) que transformará el programa que hemos escrito nosotros en lenguaje Java al lenguaje de la máquina virtual Java (`bytecode`), dando como resultado un nuevo archivo `Ejemplo.class` que se creará en este mismo directorio. Comprueba que no aparezca ningún error y que `javac` esté instalado en tu sistema (desde la consola lo puedes comprobar con el comando `javac --version` y debería aparece el número de versión que tienes instalada). Si aparecen los dos archivos tanto `Ejemplo.java` (código fuente) como `Ejemplo.class` (bytecode creado por el compilador) puedes continuar.

> **Recuerda**
>
> El proceso de compilación (`javac`) transforma código fuente `.java` en bytecode `.class`. El proceso de ejecución (`java`) interpreta el bytecode en la JVM. Dos pasos separados: primero compilar, luego ejecutar.

PASO 3: EJECUCIÓN DEL PROGRAMA

Finalmente, vamos a pedirle al intérprete (JVM) que ejecute el programa, es decir, que transforme el código de la máquina virtual Java en código máquina interpretable por nuestro ordenador y lo ejecute. Para ello escribiremos en la ventana consola: `java Ejemplo`.

El resultado será que se nos muestra la cadena `Hola Mundo`. Si logramos visualizar este texto en pantalla, ya hemos desarrollado nuestro primer programa en Java.

> **Pregunta frecuente**
>
> ¿Por qué no necesito compilar mi archivo `.java` antes de ejecutarlo y funciona directamente si me salto ese paso?
>
> https://stackoverflow.com/questions/54493058/running-a-java-program-without-compiling

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

2.1.4 Componentes del lenguaje Java

Variables, identificadores, convenciones.

VARIABLES

Definición

Una **variable** es una posición de memoria identificada por un nombre, con un tipo de dato que determina qué clase de información puede almacenar y qué rango de valores admite.

Las variables vienen determinadas por: - un **nombre**, que permite al programa acceder al valor que contiene en memoria. Debe ser un identificador válido. - un **tipo de dato**, que especifica qué clase de información guarda la variable en esa zona de memoria - un **rango de valores** que puede admitir dicha variable.

Las variables declaradas dentro de un bloque `{ }` son accesibles solo dentro de ese bloque. Una variable local no puede ser declarada como `static`. Una variable no puede declararse fuera de la clase.

Importante

El ámbito (scope) de una variable es la zona del código donde es accesible. Las variables declaradas dentro de un bloque `{ }` solo existen y son visibles dentro de ese bloque. Fuera de él, no están disponibles.

Al nombre que le damos a la variable se le llama identificador. Los identificadores permiten nombrar los elementos que se están manejando en un programa. Vamos a ver con más detalle ciertos aspectos sobre los identificadores que debemos tener en cuenta.

IDENTIFICADORES

Definición

Un **identificador** es un nombre que damos a variables, clases, métodos, etc. Debe empezar por letra, `_` o `$`, y puede contener letras, dígitos, `_` y `$` (sin espacios). Java distingue mayúsculas de minúsculas. Por ejemplo, son válidos los siguientes identificadores:

- x5
- αtη
- NUM_MAX
- numCuenta

Unicode es un código de caracteres o sistema de codificación, un alfabeto que recoge los caracteres de prácticamente todos los idiomas importantes del mundo. Además, el código Unicode es “compatible” con el código ASCII, ya que para los caracteres del código ASCII, Unicode asigna como código los mismos 8 bits, a los que les añade a la izquierda otros 8 bits todos a cero. La conversión de un carácter ASCII a Unicode es inmediata.

CONVENCIONES

Normas de estilo para nombrar variables

Recomendación

Sigue siempre las convenciones de nombres en Java:

- **Variables y métodos:** `lowerCamelCase` (ej. `numAlumnos`, `obtieneValor`)
- **Constantes:** `MAYÚSCULAS_CON_GUIONES` (ej. `TAM_MAX`, `PI`)
- **Clases:** `UpperCamelCase` (ej. `MiClase`, `String`)

Esto hace tu código más legible y profesional.

Identificador	Convención	Ejemplo
		numAlumnos, suma

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Identificador	Convención	Ejemplo
nombre de variable	Comienza por letra minúscula, y si tienen más de una palabra se colocan juntas y el resto comenzando por mayúsculas. A esto se le llama *lowerCamelCase*.	
nombre de constante	En letras mayúsculas, separando las palabras con el guión bajo, por convenio el guión bajo no se utiliza en ningún otro sitio	TAM_MAX, PI
nombre de una clase	Comienza por letra mayúscula, y si tienen más de una palabra se colocan juntas y el resto comenzando por mayúsculas. A esto se le llama *UpperCamelCase*.	String, MiTipo
nombre de función	Comienza por letra minúscula, y si tienen más de una palabra se colocan juntas y el resto comenzando por mayúsculas. A esto se le llama *lowerCamelCase*.	modificaValor, obtieneValor

Puedes consultar estas y otras convenciones sobre código Java en este enlace.

Palabras reservadas

> **Atención**
>
> Las **palabras reservadas** (`class`, `public`, `static`, `void`, `int`, etc.) NO pueden usarse como identificadores (nombres de variables, clases o métodos). Si intentas usarlas, el compilador producirá un error.

Las palabras reservadas en Java son:

```
abstract, continue, for, new, switch, assert, default, goto, package, synchronized, boolean, do, if, private, this, break, double, implements, protected, throw, byte, else, import, public, throws, case, enum, instanceof, return, transient, catch, extends, int, short, try, char, final, interface, static, void, class, finally, long, strictfp, volatile, const, float, native, super, while.
```

Tipos de datos.

Los tipos de datos se utilizan para declarar variables y el compilador sepa de antemano que tipo de información contendrá la variable.

Java dispone de los siguientes tipos de datos simples:

Tipo de dato	Representación	Tamaño (Bytes)	Rango de Valores	Valor por defecto	Clase Asociada
byte	Numérico Entero con signo	1	-128 a 127	0	Byte
short	Numérico Entero con signo	2	-32768 a 32767	0	Short
int	Numérico Entero con signo	4	-2147483648 a 2147483647	0	Integer
long	Numérico Entero con signo	8	-9223372036854775808 a 9223372036854775807	0	Long
float	Numérico en Coma flotante de precisión simple Norma IEEE 754	4	$-3.4x10^{-38}$ a $3.4x10^{38}$	0.0	Float
double	Numérico en Coma flotante de precisión doble Norma IEEE 754	8	$-1.8x10^{-308}$ a $1.8x10^{308}$	0.0	Double
char	Carácter Unicode	2	\u0000 a \uFFFF	\u0000	Character

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Tipo de dato	Representación	Tamaño (Bytes)	Rango de Valores	Valor por defecto	Clase Asociada
boolean	Dato lógico	-	true ó false	false	Boolean
void	-	-	-	-	Void

Referencia

Sobre valores por defecto y inicialización de variables: https://stackoverflow.com/questions/19131336/default-values-and-initialization-in-java

Ejemplo de declaración y asignación de valores a variables:

Tipo de datos	código
byte	`byte a;`
short	`short b, c=3;`
int	`int d=-30;` `int e=0xC125; //la 0x significa Hexadecimal`
long	`long b=46240;` `long b=5L; // La L en este caso indica Long`

char	`char car1='c';`
	`char car2=99; //car1 y car2 son iguales, la c equivale al ascii 99`
	`char letra = '\u0061'; //código unicode del carácter "a"`

float	`float pi=3.1416F; //La F significa float. Sin ella no compila: un literal con decimales es double por defecto`
	`float medio=1/2F; //0.5. Sin la F, 1/2 es una división entera y el resultado sería 0.0`

double	`double millon=1e6; // 1x10^6`
	`double medio=1/2D; //0.5, la D significa double`
	`double z=.123; //si la parte entera es 0 se puede omitir`

boolean	`boolean esPrimero;` `boolean esPar=false;`

Precaución

Ojo con los tipo float: https://jvns.ca/blog/2023/01/13/examples-of-floating-point-problems/

Tipos referenciados

A partir de los ocho tipos datos primitivos, se pueden construir otros tipos de datos. Estos tipos de datos se llaman tipos referenciados o referencias, porque se utilizan para almacenar la dirección de los datos en la memoria del ordenador.

```
1  int[] arrayDeEnteros;
2  Cuenta cuentaCliente;
```

En la primera instrucción declaramos una lista de números del mismo tipo, en este caso, enteros. En la segunda instrucción estamos declarando la variable u objeto `cuentaCliente` como una referencia de tipo `Cuenta`.

Cualquier aplicación de hoy en día necesita no perder de vista una cierta cantidad de datos. Cuando el conjunto de datos utilizado tiene características similares se suelen agrupar en estructuras para facilitar el acceso a los mismos, son los llamados datos estructurados.

Son datos estructurados los `arrays`, `listas`, `árboles`, etc. Pueden estar en la memoria del programa en ejecución, guardados en el disco como ficheros, o almacenados en una base de datos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Importante

`String` en Java es un tipo referenciado (objeto), no un tipo primitivo. Sin embargo, Java le da un tratamiento especial que permite usarlo con una sintaxis simplificada, como si fuera un tipo básico. Lo estudiaremos en profundidad en la siguiente unidad.

```
String mensaje;
mensaje= "El primer programa";
```

Hemos visto qué son las variables, cómo se declaran y los tipos de datos que pueden adoptar. Anteriormente hemos visto un ejemplo de creación de variables, en esta ocasión vamos a crear más variables, pero de distintos tipos primitivos y los vamos a mostrar por pantalla. Los tipos referenciados los veremos en la siguiente unidad.

Para mostrar por pantalla un mensaje utilizamos `System.out`, conocido como la salida estándar del programa. Este método lo que hace es escribir un conjunto de caracteres a través de la línea de comandos. Podemos utilizar `System.out.print` o `System.out.println`. En el segundo caso lo que hace el método es que justo después de escribir el mensaje, sitúa el cursor al principio de la línea siguiente.

El texto en color gris que aparece entre caracteres // son comentarios que permiten documentar el código, pero no son tenidos en cuenta por el compilador y, por tanto, no afectan a la ejecución del programa.

Tipos enumerados

Definición

Un **tipo enumerado** (`enum`) es un tipo de dato definido por el programador que solo puede tomar un conjunto restringido de valores predefinidos. Por ejemplo: `enum Dias {Lunes, Martes, Miercoles, Jueves, Viernes, Sabado, Domingo}`.

Los tipos de datos enumerados son una forma de declarar una variable con un conjunto restringido de valores. Es como si definiéramos nuestro propio tipo de datos.

Al considerar Java este tipo de datos como si de una clase se tratara, no sólo podemos definir los valores de un tipo enumerado, sino que también podemos definir operaciones a realizar con él y otro tipo de elementos, lo que hace que este tipo de dato sea más versátil y potente que en otros lenguajes de programación.

Tipos enumerados

Consulta el Ejemplo03 — Tipos enumerados para ver el código completo y su salida.

Tenemos una variable `Dias` que almacena los días de la semana. Para acceder a cada elemento del tipo enumerado se utiliza el nombre de la variable seguido de un punto y el valor en la lista. Más tarde veremos que podemos añadir métodos y campos o variables en la declaración del tipo enumerado, ya que como hemos comentado un tipo enumerado en Java tiene el mismo tratamiento que las clases.

En este ejemplo hemos utilizado el método `System.out.print`. Como podrás comprobar si lo ejecutas, la instrucción `print` escribe el texto que tiene entre comillas pero no salta a la siguiente línea, por lo que la instrucción `println` escribe justo a continuación.

Sin embargo, también podemos escribir varias líneas usando una única sentencia. Así lo hacemos en la instrucción `println`, la cual imprime como resultado tres líneas de texto. Para ello hemos utilizado un carácter especial, llamado carácter escape (`\`). Este carácter sirve para darle ciertas órdenes al compilador, en lugar de que salga impreso en pantalla. Después del carácter de escape viene otro carácter que indica la orden a realizar, juntos reciben el nombre de secuencia de escape. La secuencia de escape `\n` recibe el nombre de carácter de nueva línea. Cada vez que el compilador se encuentra en un texto ese carácter, el resultado es que mueve el cursor al principio de la línea siguiente. En el próximo apartado vamos a ver algunas de las secuencias de escape más utilizadas.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Constantes y literales.

Definición

Una **constante** es una variable cuyo valor no puede cambiar una vez asignado. En Java se declara con la palabra clave `final`:

```
final double IVA = 0.21;
```

Los **literales** pueden ser de tipo simple, null o string, como por ejemplo 230, null o "Java".

Respecto a los literales existen unos caracteres especiales que se representan utilizando secuencias de escape:

Secuencia de escape	Significado	Secuencia de escape	Significado
\b	Retroceso	\r	Retorno de carro
\t	Tabulador	\''	Carácter comillas dobles
\n	Salto de línea	\'	Carácter comillas simples
\f	Salto de página	\\	Barra diagonal

Operadores y expresiones.

OPERADORES ARITMÉTICOS

Los **Operadores Aritméticos** permiten realizar operaciones matemáticas:

Operador	Uso	Operación
+	A + B	Suma
-	A - B	Resta
*	A * B	Multiplicación
/	A / B	División
%	A % B	Módulo o resto de una división entera

Operadores aritméticos

Consulta el Ejemplo04 — Operadores aritméticos para ver ejemplos de cada operación.

OPERADORES RELACIONALES

Los **Operadores Relacionales** permiten evaluar (la respuesta es un booleano: si o no) la igualdad de los operandos:

Operador	Uso	Operación
`<`	`a < b`	a menor que b
`>`	`a > b`	a mayor que b
`<=`	`a <= b`	a menor o igual que b
`>=`	`a >= b`	a mayor o igual que b
`!=`	`a != b`	a distinto de b
`==`	`a == b`	a igual a b

Operadores relacionales

Consulta el Ejemplo05 — Operadores relacionales para ver ejemplos de cada operador.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

OPERADORES LÓGICOS

Los **Operadores Lógicos** permiten realizar operaciones lógicas:

Operador	Uso	Operación
`&&` o `&`	`a&&b` o `a&b`	a **AND** b. El resultado será *true* si ambos operadores son *true* y *false* en caso contrario.
`||` o `|`	`a||b` o `a|b`	a **OR** b. El resultado será *false* si ambos operandos son *false* y *true* en caso contrario
`!`	`!a`	**NOT** a. Si el operando es *true* el resultado es *false* y si el operando es *false* el resultado es *true*.
`^`	`a^b`	a **XOR** b. El resultado será *true* si un operando es *true* y el otro *false*, y *false* en caso contrario.

Operadores lógicos

Consulta el Ejemplo06 — Operadores lógicos para ver ejemplos con AND, OR, NOT y XOR.

Para representar resultados de operadores Lógicos también se pueden usar tablas de verdad a las que conviene acostumbrarse:

a	b	a && b	a || b	!a	a^b
false	false	false	false	true	false
true	false	false	true	false	true
false	true	false	true	true	true
true	true	true	true	false	false

OPERADORES UNARIOS O UNITARIOS

Los **Operadores Unarios** o **Unitarios** permiten realizar incrementos y decrementos:

Operador	Uso	Operación
`++`	`a++` o `++a`	Incremento de a
`--`	`a--` o `--a`	Decremento de a

Operadores unarios

Consulta el Ejemplo07 — Operadores unarios para ver la diferencia entre prefijo y sufijo.

OPERADORES DE ASIGNACIÓN

Los **Operadores de Asignación** permiten asignar valores:

Operador	Uso	Operación
`=`	`a = b`	Asignación (como ya hemos visto)
`*=`	`a *= b`	Multiplicación y asignación. La operación `a*=b` equivale a `a=a*b`
`/=`	`a /= b`	División y asignación. La operación `a/=b` equivale a `a=a/b`
`%=`	`a %= b`	Módulo y asignación. La operación `a%=b` equivale a `a=a%b`
`+=`	`a += b`	Suma y asignación. La operación `a+=b` equivale a `a=a+b`
`-=`	`a -= b`	Resta y asignación. La operación `a-=b` equivale a `a=a-b`

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Operadores de asignación

Consulta el Ejemplo08 — Operadores de asignación para ver ejemplos.

OPERADORES DE DESPLAZAMIENTO

Los **Operadores de desplazamiento** permiten desplazar los bits de los valores:

Operador	Utilización	Resultado
`<<`	`a << b`	Desplazamiento de `a` a la izquierda en `b` posiciones. Multiplica por 2 el número b de veces.
`>>`	`a >> b`	Desplazamiento de `a` a la derecha en `b` posiciones, tiene en cuenta el signo. Divide por 2 el número `b` de veces.
`>>>`	`a >>> b`	Desplazamiento de a a la derecha en b posiciones, no tiene en cuenta el signo. (simplemente agrega ceros por la izquierda)
`&`	`a & b`	Operación AND a nivel de bits
`|`	`a | b`	Operación OR a nivel de bits
`^`	`a^b`	Operación XOR a nivel de bits
`~`	`~a`	Complemento de A a nivel de bits

Operadores de desplazamiento

Consulta el Ejemplo09 — Operadores de desplazamiento para ver ejemplos detallados con bits.

OPERADOR CONDICIONAL O TERNARIO `?:`

El **operador condicional** `?:` sirve para evaluar una condición y devolver un resultado en función de si es verdadera o falsa dicha condición. Es el único operador ternario de Java, y como tal, necesita tres operandos para formar una expresión.

El primer operando se sitúa a la izquierda del símbolo de interrogación, y siempre será una expresión booleana, también llamada **condición**. El siguiente operando se sitúa a la derecha del símbolo de interrogación y antes de los dos puntos, y es el **valor** que devolverá el operador condicional **si la condición es verdadera**. El último operando, que aparece después de los dos puntos, es la expresión cuyo **resultado se devolverá si la condición evaluada es falsa**.

Operador ternario

Consulta el Ejemplo10 — Operador ternario para ver cómo funciona el operador `?:`.

El operador condicional se puede sustituir por la sentencia `if...then...else` que veremos más adelante.

PREVALENCIA DE OPERADORES

Los operadores tienen diferente **Prioridad** por lo que es interesante utilizar paréntesis para controlar las operaciones sin necesidad de depender de la prioridad de los operadores.

Prevalencia de operadores, ordenados de arriba a abajo de más a menos prioridad:

Descripción	Operadores
operadores posfijos	`op++ op--`
operadores unarios	`++op --op +op -op ~ !`
multiplicación y división	`* / %`
suma y resta	`+ -`
desplazamiento	`<< >> >>>`

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Descripción	Operadores
operadores relacionales	`< > <= =>`
equivalencia	`== !=`
operador AND	`&`
operador XOR	`^`
operador OR	`|`
AND booleano	`&&`
OR booleano	`||`
condicional	`?:`
operadores de asignación	`= += -= *= /= %= &= ^= |= <<= >>= >>>=`

Prevalencia de operadores

Consulta el Ejemplo11 — Prevalencia de operadores para ver la diferencia con y sin paréntesis.

Recuerda

"Los paréntesis son como las patatas fritas, cuantas más, mejor!" — Usa paréntesis para controlar explícitamente el orden de las operaciones y evitar depender de la tabla de precedencia, que es fácil de olvidar.

Conversiones de tipo.

Atención

Las conversiones de tipo pueden provocar **pérdida de información**. Una conversión implícita (automática) solo es segura cuando el tipo destino tiene mayor precisión. Las conversiones explícitas (cast) son forzadas por el programador y deben usarse con cuidado.

CONVERSIONES IMPLÍCITAS

Las **Conversiones Implícitas** se realizan de forma automática y requiere que la variable destino tenga más precisión que la variable origen para poder almacenar el valor.

Conversiones de tipo

Consulta el Ejemplo12 — Conversiones de tipo para ver ejemplos de conversión implícita y explícita.

Comentarios.

Los comentarios son muy importantes a la hora de describir qué hace un determinado programa. A lo largo de la unidad los hemos utilizado para documentar los ejemplos y mejorar la comprensión del código. Para lograr ese objetivo, es normal que cada programa comience con unas líneas de comentario que indiquen, al menos, una breve descripción del programa, el autor del mismo y la última fecha en que se ha modificado.

Recomendación

Documenta siempre tu código con comentarios. Cada programa debe incluir al menos: breve descripción de lo que hace, autor y fecha de última modificación. Java soporta tres tipos: `//` (una línea), `/* ... */` (multilínea), y `/** ... */` (Javadoc para documentación automática).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Todos los lenguajes de programación disponen de alguna forma de introducir comentarios en el código. En el caso de Java, nos podemos encontrar los siguientes tipos de comentarios:

- Comentarios de **una sola línea**. Utilizaremos el delimitador // para introducir comentarios de sólo una línea.

  Consulta el Ejemplo13 — Comentarios para ver ejemplos de cada tipo.

2.1.5 Herramientas útiles para empezar

Generar números aleatorios.

Podemos generar números aleatorios entre 0 y 1 utilizando el método random de la clase `Math`.

> **Generación de números aleatorios**
>
> Consulta el Ejemplo14 — Números aleatorios para ver el código completo.

Introducir un texto desde el teclado.

> **Importante**
>
> `System.console().readLine()` **no funciona en la mayoría de IDEs** (Eclipse, IntelliJ, NetBeans). Para entrada de datos en entornos de desarrollo, usa la clase `Scanner` (`java.util.Scanner`), que veremos en la siguiente unidad.

Podemos introducir texto desde el teclado utilizando `System.console().readLine();`

> **Introducción de datos por teclado**
>
> Consulta el Ejemplo15 — Entrada por teclado para ver el código completo.

2.1.6 Ejemplos UD01

Descarga el código fuente completo de los ejemplos

Ejemplo01

Programa mínimo en Java que muestra un mensaje por consola (Hola Mundo).

```java
public class Holamundo {
    // programa Hola Mundo
    public static void main(String[] args) {
        /* lo único que hace este programa es mostrar
            la cadena "Hola Mundo!" por pantalla */
        System.out.println("Hola Mundo!");
    }
}
```

Compilación y ejecución:

```
$ javac Holamundo.java
$ java Holamundo
Hola Mundo!
```

⬆ Volver a teoría

Ejemplo02

Ciclo completo de creación, compilación y ejecución de un programa Java.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
/* Ejemplo Hola Mundo */
public class Ejemplo {
    public static void main(String[] args) {
        System.out.println("Hola Mundo");
    }
}
```

Compilación:

```
$ javac Ejemplo.java
```

Ejecución:

```
$ java Ejemplo
Hola Mundo
```

⬆ Volver a teoría

Ejemplo03

Declaración y uso de un tipo `enum`.

```java
public class tiposEnumerados {
    public enum dias {Lunes, Martes, Miercoles, Jueves, Viernes, Sábado, Domingo};

    public static void main(String[] args) {
        dias diaActual = dias.Martes;
        dias diaSiguiente = dias.Miercoles;

        System.out.print("Hoy es: ");
        System.out.println(diaActual);
        System.out.println("Mañana\nes\n"+diaSiguiente);
    }
}
```

Salida:

```
Hoy es: Martes
Mañana
es
Miercoles
```

⬆ Volver a teoría

Ejemplo04

Operaciones aritméticas básicas con `double`.

```java
double num1, num2, suma, resta, producto, division, resto;
num1 = 8;
num2 = 5;
suma = num1 + num2;      // 13
resta = num1 - num2;     // 3
producto = num1 * num2;  // 40
division = num1 / num2;  // 1.6
resto = num1 % num2;     // 3
```

⬆ Volver a teoría

Ejemplo05

Comparaciones entre valores enteros.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
int valor1 = 10;
int valor2 = 3;
boolean compara;
compara = valor1 > valor2;  // true
compara = valor1 < valor2;  // false
compara = valor1 >= valor2; // true
compara = valor1 <= valor2; // false
compara = valor1 == valor2; // false
compara = valor1 != valor2; // true
```

⬆ Volver a teoría

Ejemplo06

Operaciones lógicas con `&&`, `||`, `!` y `^`.

```java
double sueldo = 1400;
int edad = 34;
boolean logica;
logica = (sueldo>1000 & edad<40);   //true
logica = (sueldo>1000 && edad >40); //false
logica = (sueldo>1000 | edad>40);   //true
logica = (sueldo<1000 || edad >40); //false
logica = !(edad <40);               //false
logica = (sueldo>1000 ^ edad>40);   //true
logica = (sueldo<1000 ^ edad>40);   //false
```

⬆ Volver a teoría

Ejemplo07

Incremento y decremento con prefijo y sufijo.

```java
int m = 5, n = 3;
m++; // 6
n--; // 2

int a = 1, b;
b = ++a; // a vale 2 y b vale 2
b = a++; // a vale 3 y b vale 2
```

⬆ Volver a teoría

Ejemplo08

Operadores compuestos de asignación.

```java
int dato1 = 10, dato2 = 2, dato;
dato = dato1;   // dato vale 10
dato2 *= dato1; // dato2 vale 20
dato2 /= dato1; // dato2 vale 2
dato2 += dato1; // dato2 vale 12
dato2 -= dato1; // dato2 vale 2
dato1 %= dato2; // dato1 vale 0
```

⬆ Volver a teoría

Ejemplo09

Desplazamiento de bits a nivel binario.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
int j = 33;
int k = j << 2;
// 00000000000000000000000000100001 : j = 33
// 00000000000000000000000010000100 : k = 33 << 2 ; k = 132

int o = 132;
int p = o >> 2;
// 00000000000000000000000010000100 : o = 132
// 00000000000000000000000000100001 : p = 132 >> 2 ; p = 33

int x = -1;
int y = x >>> 2;
// 11111111111111111111111111111111 : x = -1
// 00111111111111111111111111111111 : y = x >>> 2; y = 1073741823

int q = 132;   // q: 00000000000000000000000010000100
int r = 144;   // r: 00000000000000000000000010010000

int s = q & r; // s: 00000000000000000000000010000000 = 128
int t = q | r; // t: 00000000000000000000000010010100 = 148
int u = q ^ r; // u: 00000000000000000000000000010100 = 20
int v = ~q;    // v: 11111111111111111111111101111011 = -133
```

⬆ Volver a teoría

Ejemplo10

Uso del operador condicional `?:` para elegir entre dos valores.

```java
int mayor, exp1 = 15, exp2 = 25;
mayor = (exp1 > exp2) ? exp1 : exp2;
// mayor valdrá 25
```

⬆ Volver a teoría

Ejemplo11

Diferencia en el resultado según el uso de paréntesis.

```java
int x, y1 = 6, y2 = 2, y3 = 8;
x = y1 + y2 * y3;   // 22 (por prevalencia de *)
x = (y1 + y2) * y3; // 64
```

⬆ Volver a teoría

Ejemplo12

Conversión implícita (automática) y explícita (cast).

```java
// Conversión Implícita
byte origen = 5;
short destino;
destino = origen;  // 5

// Conversión Explícita
short origen2 = 3;
byte destino2;
destino2 = (byte) origen2; // 3
```

⬆ Volver a teoría

Ejemplo13

Los tres tipos de comentarios en Java.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
// comentario de una sola línea
byte estoEsUnByte = 1;

/* Esto es un
comentario
de varias líneas */

/** Comentario de documentación.
 * Javadoc extrae los comentarios del código y
 * genera un archivo html a partir de este tipo de comentarios
 */
```

⬆ Volver a teoría

Ejemplo14

Generación de números aleatorios con `Math.random()`.

```java
double numero;
int entero;
numero = Math.random();
System.out.println("El número es: " + numero); //entre 0 y 0.999...

numero = Math.random() * 100;
System.out.println("El número es: " + numero); //entre 0 y 99.999...

entero = (int)(Math.random() * 100);
System.out.println("El número sin decimales es: " + entero); //entre 0 y 99

int lado = ((int)(Math.random() * 6)) + 1;
char letra = (char)((Math.random() * 26) + 65); //65..90
System.out.println(letra); //A..Z
```

⬆ Volver a teoría

Ejemplo15

Lectura de datos desde la consola con `System.console().readLine()`.

```java
// Ejemplo 1: Introducción de texto
String texto;
System.out.print("Introduce un texto: ");
texto = System.console().readLine();
System.out.println("El texto introducido es: " + texto);

// Ejemplo 2: Introducción de un número entero
String texto2;
int entero2;
System.out.print("Introduce un número: ");
texto2 = System.console().readLine();
entero2 = Integer.parseInt(texto2);
System.out.println("El número introducido es: " + entero2);

// Ejemplo 3: Introducción de un número decimal
String texto3;
double doble3;
System.out.print("Introduce un número decimal: ");
texto3 = System.console().readLine();
doble3 = Double.parseDouble(texto3);
System.out.println("Número decimal introducido es: " + doble3);
```

⬆ Volver a teoría

2.1.7 Resumen — Conceptos clave

Concepto	Definición
Problema	Situación que requiere una solución mediante métodos y estrategias
Algoritmo	Conjunto de pasos finitos y ordenados para resolver un problema
Programa	Algoritmo codificado en un lenguaje de programación
JVM	Máquina Virtual Java que ejecuta el bytecode

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Concepto	Definición
Variable	Zona de memoria con nombre que almacena un valor
Tipo de dato	Categoría de valor que puede tomar una variable
Operador	Símbolo que realiza una operación sobre uno o más operandos

2.1.8 Autoevaluación

- [ ] Comprendo qué es un problema, un algoritmo y un programa
- [ ] Sé compilar y ejecutar un programa Java
- [ ] Conozco los tipos de datos primitivos de Java
- [ ] Puedo declarar variables y constantes
- [ ] Utilizo correctamente los operadores aritméticos, relacionales y lógicos
- [ ] Entiendo la diferencia entre conversión implícita y explícita

2.1.9 Vídeos recomendados

Canal	Vídeo	Contenido
Píldoras Informáticas	Curso Java 2026 — Presentación (vídeo 1)	Presentación del curso completo
Píldoras Informáticas	Curso Java — Estructuras principales VII — Clase Math II (vídeo 10)	Clase Math y operaciones matemáticas
Programación ATS	Playlist completa del curso Java	Vídeos 1–9: introducción, Hola Mundo, tipos de datos, entrada/salida
MoureDev	Curso Completo de Java desde Cero	8 horas — cubre todos los temas del curso
DiscoDurodeRoer	Curso Java SE — playlist	Instalación, Hola Mundo, variables, tipos

27 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

2.2 Ejercicios de la UD01

2.2.1 Retos

1. (Reto1) Haga un programa que evalúe una expresión que contenga literales de los cuatro tipos de datos (booleano, entero, real y carácter) y la muestre por pantalla.
2. (Reto2) En su entorno de trabajo, cree el programa siguiente. Obsérvese que pasa exactamente. Entonces, intente arreglar el problema.

```
// Un programa que usa un entero muuuuy grande
public class TresMilMilions {
    public static void main (String [] args) {
        System.out.println (3000000000);
    }
}
```

3. (Reto3) Haga un programa con dos variables que, sin usar ningún literal ninguna parte excepto para inicializar estas variables, vaya estimando e imprimiendo sucesivamente los 5 primeros valores de la tabla de multiplicar del 4. Puede usar operadores aritméticos y de asignación, si desea.
4. (Reto4) Haga dos programas, uno que muestre por pantalla la tabla de multiplicar del 3, y otro, la del 5. Los dos deben ser exactamente iguales, letra por letra, excepto en un único literal dentro de todo el código.
5. (Reto5) Experimente qué pasa si en el siguiente programa inicializa la variable realLargo con un valor con varios decimales. El programa continúa compilando? ¿Qué resultado da? Después inténtelo asignando un valor superior al rango de los enteros (por ejemplo, 3000000000.0).

```
public class ConversionExplicita {
    public static void main (String[] args) {
        double realLlarg = 3000000000.0;
        // Asignación incorrecta. ¿Un real tiene decimales, no?
        long enterLlarg = (long) realLlarg;
        // Asignación incorrecta. ¿Un entero largo tiene un rango mayor que un entero, no?
        int enter = (int) enterLlarg;
        System.out.println (enter);
    }
}
```

6. (Reto6) Haga un programa que muestre en pantalla de forma tabulada la tabla de verdad de una expresión de disyunción entre dos variables booleanas.
7. (Reto7) Haga un programa que muestre por pantalla la multiplicación de tres números reales entrados por teclado.

2.2.2 Ejercicios

> **Importante**
>
> `Scanner` solo se puede usar en esta actividad porque no se ha explicado en profundidad en este tema. No lo confundas con `System.console().readLine()`, que no funciona en la mayoría de IDEs.

1. (Ejs1) Probar la E/S elemental: Escribe el pequeño programa que aparece a continuación.

```
import java.util.*;
public class EntradaSalida {
    public static void main (String arg[]){
        Scanner tec = new Scanner(System.in);
        int a, b;
        System.out.println("Introduce un número entero");
        a = tec.nextInt();
        System.out.println("Introduce otro número entero");
        b = tec.nextInt();
        System.out.println("Los números introducidos son " + a + " y " + b);
    }
}
```

   Ejecútalo para ver como se comporta el programa.

   ¿Qué ocurre si cuando nos pide un número entero le damos un número real? ¿Y si le damos un carácter no numérico?

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

¿Qué ocurre si eliminamos la instrucción `import java.util.*`;

2. (Ejs2) Averigua mediante pruebas:
   a. ¿Es posible escribir dos instrucciones en la misma línea de un programa?
   b. ¿Se puede "romper" una instrucción entre varias líneas?
   c. Algunos lenguajes de programación dan un valor por defecto a las variables cuando las declaramos sin inicializarlas. Otros no permiten usar el contenido de una variable que no haya sido previamente inicializada. ¿Cuál es comportamiento de Java?
3. (Esj3) ¿Cuáles de los siguientes identificadores son válidos y cuales no? Pruébalos cuando tengas duda
   a. `n`
   b. `MiProblema`
   c. `MiJuego`
   d. `Mi Juego`
   e. `Int`
   f. `Jose&Co`
   g. `A b`
   h. `1rApellido`
   i. `aaaaaaaaaaaa`
   j. `NombreApellidos`
   k. `Saldo-actual`
   l. `Universidad Alicante`
   m. `Juan=Rubio`
   n. `Edad5`
   o. `_5Java`
   p. `true`
   q. `_false`
   r. `f_alse`
4. (Por2) Escribir un programa que lea un entero desde teclado, lo multiplique por 2, y a continuación escriba el resultado en la pantalla:

   Ejemplo de ejecución:

```
Escribe un número:
3
El doble de 3 es 6
```

5. (Intercambio) Escribir un programa que ...
   a. Lea desde teclado dos valores de texto. Llama a las variables s1 y s2.
   b. Muestre los valores introducidos por el usuario
   c. Intercambie el valor de s1 y s2 (s1 pasa a valer lo que valía s2 y viceversa)
   d. Muestre de nuevo los valores, ahora con su valor intercambiado

   Ejemplo de ejecución:

```
Escribe un texto para s1: David
Escribe un texto para s2: Maria
Antes de intercambiar    s1: David  y   s2: Maria
Después de intercambiar  s1: Maria  y   s2: David
```

6. (ExpresionesMatematicas) Escribir las siguientes expresiones siguiendo la sintaxis de Java.
   a. \(\frac{x}{y}+1\)
   b. \(\frac{x+y}{x-y}\)
   c. \([\frac{b}{c+d}]\)
   d. \((a+b)²\)
   e. \(\frac{x+\frac{y}{2}}{x-\frac{y}{z}}\)
   f. \(\frac{xy}{1-4Zx}\)
   g. \((a+b)\frac{c}{d}\)

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

   h. \(\frac{xy}{mn}\)

7. (Superficie) Escribir un programa que solicite al usuario la longitud y la anchura de una habitación y a continuación muestre su superficie (longitud por anchura).
8. (Medidas) Escribir un programa que convierta una medida dada en pies a sus equivalentes en yardas, pulgadas, centímetros y metros, sabiendo que 1 pie = 12 pulgadas, 1 yarda = 3 pies, 1 pulgada = 2.54 cm, 1 m = 100 cm.
9. (Segundos) Escribir un programa que, dada una cantidad de segundos, introducida por teclado, la desglose en días, horas, minutos y segundos.

   Ejemplo de ejecución:

```
Introduce cantidad de segundos: 3661
3661 segundos son:
0 dias
1 horas
1 minutos
1 segundos
```

10. (Fuerza) La fuerza de atracción entre dos masas m1 y m2 separadas por una distancia d, está dada por la fórmula: $$ F= \frac{(G · m1 · m2)}{d^2} $$

    donde G es la constante de gravitación universal G= 6.67430 · 10 $^{-11}$.

    Escribir un programa que lea la masa de dos cuerpos y la distancia entre ellos y a continuación obtenga su fuerza de atracción.
11. (Círculo) Escribir un programa que calcule la longitud de la circunferencia y el área del círculo para un valor del radio introducido por teclado.
12. (Dados) Escribir un programa que simula el lanzamiento de dos dados.

```
Dado 1 : 5
Dado 2: 4
Puntuación total: 9
```

13. (UltimaCifra) Escribir un programa que muestre la última cifra de un número entero que introduce el usuario por teclado.

    Pista: ¿Qué devuelve `a%10` ?

```
Introduce un número entero: 3761
La última cifra de 3761 es 1
```

14. (PenultimaCifra) Escribir un programa que muestre la penúltima cifra de un número entero que introduce el usuario por teclado.

```
Introduce un número entero: 3761
La penúltima cifra de 3761 es 6
```

    Una vez hayas comprobado que el programa funciona correctamente, prueba qué ocurre si el usuario introduce un valor de una sola cifra (por ejemlo 4). Explica el resultado mostrado por el programa.
15. (Redondear1) `Math.round(x)` redondea x de manera que este queda sin decimales. (*`Math.round(35.5289)` da como resultado `36`*)

    Trata de escribir un programa en el que el usuario introduzca un número real y a continuación se muestre redondeado a un decimal. *Pista : combinar productos, divisiones y Math.round()*

    Ejemplo de ejecución:

```
Introduce un número real: 35.5289
El número 35.5289, redondeado a un decimal es 35.5
```

16. (ExpresionesAritmeticas) 16. Cuál es el valor resultante de dada una de las siguientes expresiones
    a. `5 * 4 - 3 * 6`
    b. `4 * 5 * 2`
    c. `(24 + 2 * 6) / 4`
    d. `8 / 2 / 2 * 5`
    e. `3 + 4 * (8 * (4 - (9 + 3) / 6 ))`
    f. `4 * 3 * 5 + 8 * 4 * 2`
    g. `4 - 40 % 5`

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

h. `4 * 3 / 2`
i. `4 / 2 * 3`
j. `213 /100`

17. (Einstein) La famosa ecuación de Einstein para la conversión de una masa m en energía viene dada por la fórmula $E=mc^2$, donde c es la velocidad de la luz que vale $2.997925 \cdot 10^8$ m/s. Escribir un programa que lea el valor de la masa y obtenga la energía correspondiente según la anterior fórmula.
18. (FragmentosCodigo) Indica cuales serán los valores de las variables después de ejecutar cada uno de los siguientes fragmentos de código. Resuelve el ejercicio sin escribir los programas correspondientes y probarlos.

a. `java int a=3, b = 2; a = b + b; b = a + a;`
b. `java int a=3,b=0; b = b - 1; a = a + b;`
c. `java int a, b=5; b++; ++b; a= b+1;`
d. `java int a = 5,b; b = a++;`
e. `java int a = 5,b; b = ++a;`
f. `java int a=2, b=3; b+=a;`
g. `java int a=2, b=3; b-=a; a=-b;`
h. `java int a=2, b=3; b%=a;`
i. `java int a=2,b=3,c=4; a = --b + c++; b+=a;`

2.2.3 Expresiones Lógicas

1. Sean 4 variables enteras:

```
1  int m, j, p, v ;
```

que contienen respectivamente la edad de Miguel, Julio, Pablo y Vicente.

Expresar las siguientes afirmaciones utilizando operadores lógicos y relacionales

Ejemplo: `Miguel es mayor de edad.`

Solución: `m >= 18`

a. (Logica1) Miguel es menor de edad.
b. (Logica2) Miguel es mayor que Julio
c. (Logica3) Miguel es el más viejo.
d. (Logica4) Miguel es el más joven.
e. (Logica5) Miguel no es el más joven.
f. (Logica6) Miguel no es el más viejo.
g. (Logica7) Alguno de ellos es mayor de edad.
h. (Logica8) Miguel y Julio son los más jóvenes.
i. (Logica9) Entre todos tienen más de 100 años.
j. (Logica10) Entre Miguel y Julio suman más edad que Pablo.
k. (Logica11) Entre Miguel y Julio suman más edad que Pablo y Vicente juntos.
l. (Logica12) Si los ordenamos por edades de menor a mayor, Julio es el segundo.
m. (Logica13) Si los ordenamos por edades de menor a mayor, Julio es el segundo y Pablo el tercero.
n. (Logica14) Al menos uno de ellos es menor de edad.
o. (Logica15) Al menos dos de ellos son menores de edad.
p. (Logica16) Todos son menores de edad.
q. (Logica17) Solo dos de ellos son menores de edad.
r. (Logica18) Al menos dos de ellos nacieron el mismo año.
s. (Logica19) Solo dos de ellos nacieron el mismo año.
t. (Logica20) Al menos uno de ellos es menor que Julio
u. (Logica21) Solo uno de ellos es menor que Julio
v. (Logica22) Miguel es mayor de edad y alguno de los otros es menor de edad.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

2.2.4 Actividades

1. (Actividad1) Realiza un conversor de euros a pesetas. La cantidad de euros que se quiere convertir debe ser introducida por teclado.
2. (Actividad2) Realiza un conversor de pesetas a euros. La cantidad de pesetas que se quiere convertir debe ser introducida por teclado.
3. (Actividad3) Escribe un programa que calcule el área de un rectángulo. (`area = base * altura`)
4. (Actividad4) Escribe un programa que calcule el área de un triángulo. (`area = (base * altura) / 2`)
5. (Actividad5) Escribe un programa que calcule el salario semanal de un empleado en base a las horas trabajadas, a razón de 12 euros la hora.
6. (Actividad6) Realiza un conversor de MiB a KiB. Ayuda
7. (Actividad7) Realiza un conversor de Kib a Mib. Ayuda
8. (Actividad8) Realiza un programa en Java que genere letras de forma aleatoria.
9. (Actividad9) Realiza un programa en Java que genere el número premiado del Cupón de la ONCE.
10. (Actividad10) Modificar el siguiente programa para que compile y funcione:

```java
public class Activ10 {
    public static void main(String[] args) {
        int n1 = 50, int n2 = 30,
        boolean suma = 0;
        suma = n1 + n2;
        System.out.println("LA SUMA ES: " + suma);
    }
}
```

11. (Actividad11) Modificar el siguiente programa para que compile y funcione:

```java
public class Activ11 {
    public static void main(String[] args) {
        int numero = 2;
        cuad = numero * número;
        System.out.println("EL CUADRADO DE "+NUMERO+" ES: "+cuad);
    }
}
```

12. (Actividad12) Indicar que valor devolverá la ejecución del siguiente programa:

```java
public class Activ12 {
    public static void main(String[] args) {
        int num = 5;
        num += num - 1 * 4 + 1;
        System.out.println(num);
    }
}
```

13. (Actividad13) Indicar que valor devolverá la ejecución del siguiente programa:

```java
public class Activ13 {
    public static void main(String[] args) {
        int num = 4;
        num %= 7 * num % 3 * 3;
        System.out.println(num);
    }
}
```

14. (Actividad14) Realizar un programa que muestre por pantalla respetando los saltos de carro el siguiente texto (con un solo `println`):

```
Me gusta la programación
cada día más
```

15. (Actividad15) Realiza un programa en Java que tenga las variables edad, nivel de estudios e ingresos y almacene en una variable llamada jasp el valor verdadero si la edad es menor o igual a 28 y el nivel de estudios es mayor a 3, o bien la edad es menor de 30 y los ingresos superiores a 28000. En caso contrario almacenar el valor falso.
16. (Actividad16) Realizar un programa que realice el cálculo del precio de un producto teniendo en cuenta que el producto vale 120 €, tiene un descuento del 15% y el IVA que se le aplica es del 21%.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

17. (Actividad17) Realiza un programa que calcule la nota que hace falta sacar en el segundo examen de la asignatura Programación para obtener la media deseada. Hay que tener en cuenta que la nota del primer examen cuenta el 40% y la del segundo examen un 60%. Ejemplo 1:

```
Introduce la nota del primer examen: 7
¿Qué nota quieres sacar en el trimestre? 8.5
Para tener un 8.5 en el trimestre necesitas sacar un 9.5 en el segundo examen.
```

Ejemplo 2:

```
Introduce la nota del primer examen: 8
¿Qué nota quieres sacar en el trimestre? 7
Para tener un 7 en el trimestre necesitas sacar un 6.333333333 en el segundo examen.
```

18. (Actividad18) Realizar un programa que dado un importe en euros nos indique el mínimo número de billetes y la cantidad sobrante de euros. Debes usar el operador condicional `?:`

```
¿Cuántos euros tienes?: 232
1 billete de 200 €
1 billete de 20 €
1 billete de 10 €
Sobran 2 €
```

19 de julio de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

2.3 Talleres

2.3.1 Taller UD01_01: Instalar Veyon para el control del aula

¿Qué es `Veyon`?

Veyon (*Virtual Eye On Networks*) es un software **libre, gratuito y de código abierto** para la gestión de aulas informáticas. Permite al profesor ver en tiempo real las pantallas de los equipos del alumnado, tomar el control de un equipo para ayudar, bloquear las pantallas durante una explicación o mostrar su propia pantalla a toda la clase.

Veyon se compone de varias piezas que se instalan por separado:

Componente	Dónde se instala	Qué hace
Veyon Master	Solo en el equipo del profesor	Consola gráfica desde la que se ve y controla el aula. **Tú no la necesitas.**
Veyon Service	En tu portátil	Agente en segundo plano que permite que el profesor vea y controle tu equipo.
Interception Driver	En tu portátil	Controlador de bajo nivel necesario para que el profesor pueda bloquear el teclado y ratón durante una explicación.
Veyon Configurator	En tu portátil (se instala junto con Veyon Service)	Herramienta gráfica de configuración (claves, red, permisos).

¿Por qué usamos `Veyon`?

Hasta ahora utilizábamos `NoMachine` para el control remoto de los equipos, pero ha pasado a ser **de pago**. `Veyon` es una alternativa libre y gratuita que, además, está pensada específicamente para aulas y funciona en **Linux, Windows y macOS**, por lo que cubre todos los portátiles que traéis a clase.

En clase lo utilizaremos para:

- **Pedir ayuda y resolver dudas**: el profesor puede ver tu pantalla (o tomar el control) sin tener que desplazarse hasta tu sitio.
- **Corregir las tareas diarias** directamente en tu equipo.
- **Bloquear las pantallas** durante las explicaciones para mantener la atención.
- **Mostrar la pantalla del profesor** (o la de un compañero) a toda la clase.
- **Supervisar los exámenes** que se realicen con el ordenador.

¿Cómo funciona? Seguridad y privacidad

Veyon utiliza un **par de claves asimétrico** (pública/privada), identificado con el nombre `Docent`:

- La **clave privada** se queda **únicamente en el equipo del profesor** y nunca se distribuye.
- La **clave pública** (el fichero `veyon-aula-profesor.pem`) es la que te proporcionará el profesor a través de AULES y tendrás que importar en tu portátil. Con ella, tu equipo puede comprobar que las peticiones de control vienen realmente del profesor y de nadie más. Repartirla no supone ningún riesgo: es pública por diseño.

Además, la comunicación se realiza por el puerto **TCP 11100** y la detección de equipos solo funciona **dentro de la red del aula**. Es decir, el profesor solo podrá conectarse a tu portátil mientras estés conectado a la red del instituto; **no podrá acceder cuando estés en casa**.

Instalación según tu sistema operativo

Sigue el manual correspondiente a tu sistema operativo:

- 🐧 Instalación de Veyon en Linux
- 🪟 Instalación de Veyon en Windows
- 🍎 Instalación de Veyon en macOS

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Consejo

Ten en cuenta que los navegadores pueden cambiar las extensiones de los archivos. Es conveniente activar las opciones de "ver extensiones de archivos" y "ver archivos ocultos" en tu gestor de archivos habitual y comprobar que la clave conserva la extensión `.pem`.

Resolución de problemas

Si tu equipo no aparece en la consola del profesor o falla la autenticación, consulta la guía de Resolución de problemas con Veyon.

Tarea

Debes enviar un archivo `*.pdf` a la plataforma de AULES que contenga:

1. Una captura del **Veyon Configurator** en la página **"Autenticación"**, donde se vea seleccionado el método **"Autenticación mediante fichero de clave"**.
2. Una captura de la página **"Claves de autenticación"**, donde se vea importada la clave pública `Docent`.
3. Una captura que demuestre que el profesor se ha podido conectar a tu equipo (por ejemplo, tu equipo visible en la consola del profesor o el aviso de conexión en tu portátil).

Obligatorio

Debes mantener `Veyon Service` instalado y activo, y permitir las conexiones por parte del profesor, para pedir ayuda y consultar dudas en clase, para corregir las tareas diarias y para realizar los exámenes.

25 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

2.3.2 Taller UD01_01: Instalación de Veyon en Linux

Este manual instala en tu portátil el componente **Veyon Service**, que permite que el profesor vea y controle tu pantalla durante la clase. No necesitas instalar el "Master" (esa parte solo la usa el profesor).

Qué necesitas

- Tu portátil con Linux conectado a la red del aula.
- El fichero de **clave pública** que te ha entregado el profesor (por ejemplo `veyon-aula-profesor.pem`), descargado desde AULES o recibido por USB.

1. Instalar Veyon

UBUNTU / DEBIAN / LLIUREX

```
wget -qO- https://download.veyon.io/apt/veyon.gpg | sudo gpg --dearmor -o /usr/share/keyrings/veyon-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/veyon-archive-keyring.gpg] https://download.veyon.io/apt/ ./" | \
  sudo tee /etc/apt/sources.list.d/veyon.list
sudo apt update
sudo apt install veyon
```

FEDORA / OPENSUSE / OTRAS DISTROS

Consulta las instrucciones específicas para tu distribución en https://veyon.io/download. Si no hay repositorio para tu distro, descarga el paquete `.rpm` / `.deb` correspondiente desde https://github.com/veyon/veyon/releases.

2. Configurar la autenticación por clave

1. Abre el Configurador de Veyon:

   ```
   veyon-configurator
   ```

2. Ve a la página **"Autenticación" (Authentication)** y en **Método** selecciona **"Autenticación mediante fichero de clave" (Key file authentication)**.
3. Ve a la página **"Claves de autenticación" (Authentication keys)**, pulsa **"Importar" (Import)**, ponle el mismo nombre que usó el profesor (`Docent`), y elige el fichero de clave pública que te ha dado el profesor (`veyon-aula-profesor.pem`).

3. Comprobar que el servicio está activo

El servicio arranca solo al iniciar sesión. Compruébalo con:

```
pgrep -fa veyon-service
```

Si no aparece, ejecútalo manualmente:

```
veyon-service start &
```

4. Firewall

Si tienes `ufw` activo, permite el puerto que usa Veyon:

```
sudo ufw allow 11100/tcp
```

5. Comprobación final

Avisa al profesor: tu equipo debería aparecer automáticamente en su consola en unos segundos, dentro del aula. Si no aparece o pide autenticación repetidamente, revisa la guía de Resolución de problemas o avisa al profesor.

⬅ Volver al taller T01

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

23 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

2.3.3 Taller UD01_01: Instalación de Veyon en Windows

Este manual instala en tu portátil el componente **Veyon Service**, que permite que el profesor vea y controle tu pantalla durante la clase.

Qué necesitas

- Tu portátil con Windows conectado a la red del aula.
- El fichero de **clave pública** que te ha entregado el profesor (por ejemplo `veyon-aula-profesor.pem`), descargado desde AULES o recibido por USB.

1. Descargar el instalador

1. Ve a https://veyon.io/download y descarga el instalador de Windows (`veyon-<versión>.exe`), o descárgalo directamente desde https://github.com/veyon/veyon/releases. No lo descargues de ningún otro sitio.
2. Ejecuta el instalador.

AVISO NORMAL DE WINDOWS AL DESCARGAR/EJECUTAR EL INSTALADOR

Es muy probable que veas alguno de estos avisos, tanto al descargar como al ejecutar el `.exe`:

- En el navegador: "*Este archivo no se descarga con frecuencia*" o "*Archivo potencialmente peligroso*".
- Al ejecutarlo, **SmartScreen de Windows**: pantalla azul "*Windows ha protegido su PC*", con el mensaje de que la app es de un editor no reconocido.

> ⚠ **Esto no significa que el programa sea malware**
>
> Veyon es software libre y de código abierto; el aviso aparece porque su instalador no acumula suficientes descargas/reputación en los sistemas de SmartScreen de Microsoft (algo habitual en programas menos populares que no pagan por un certificado de firma de código EV), no porque haya sido detectado como malicioso. Puedes comprobarlo tú mismo: el propio Windows Defender (antivirus), al analizar el fichero, no lo marca como virus; solo SmartScreen avisa por baja reputación de descarga.

Para continuar de forma segura:

1. **Verifica primero que lo has descargado del sitio oficial**: la URL debe empezar por `https://veyon.io/` o `https://github.com/veyon/`. Si el aviso apareciera al descargarlo de cualquier otra web, ahí sí desconfía y no lo instales.
2. Si el navegador bloquea la descarga, pulsa **"Conservar"** / **"Mostrar más"** → **"Conservar de todos modos"**.
3. Si al ejecutarlo aparece la pantalla azul de SmartScreen, pulsa **"Más información"** y luego **"Ejecutar de todas formas"**.

2. Elegir los componentes a instalar (importante)

En la pantalla de selección de componentes verás tres casillas:

Componente	¿Marcar?	Motivo
Veyon Service	✅ Sí	Es el agente que permite que el profesor vea/controle tu equipo. Imprescindible.
Veyon Master	❌ No	Es la consola de control del aula: solo la necesita el profesor, no los alumnos.
Interception Driver	✅ Sí	Controlador de bajo nivel necesario para que el profesor pueda bloquear el teclado/ratón (por ejemplo, durante una explicación en la que bloquea las pantallas). Sin él, esa función puede no funcionar bien.

No aparece ninguna casilla para "Veyon Configurator": se instala automáticamente junto con Veyon Service, no hay que seleccionarlo aparte.

3. Configurar la autenticación por clave

1. Abre **"Veyon Configurator"** desde el menú Inicio (se instaló automáticamente junto con Veyon Service).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

2. Ve a la página **"Autenticación" (Authentication)** y en **Método** selecciona **"Autenticación mediante fichero de clave" (Key file authentication)**.
3. Ve a la página **"Claves de autenticación" (Authentication keys)**, pulsa **"Importar" (Import)**, ponle el mismo nombre que usó el profesor (`Docent`), y selecciona el fichero de clave pública que te ha dado el profesor.

4. Firewall de Windows

El instalador suele crear automáticamente la regla de Firewall de Windows necesaria para `veyon-service.exe`. Verifícalo:

1. Abre **"Firewall de Windows Defender" → "Permitir una aplicación a través del Firewall"**.
2. Comprueba que **Veyon Service** aparece marcado, al menos para redes **privadas**.
3. Si no aparece o la conexión no funciona desde la consola del profesor, añade manualmente una regla de entrada para el puerto **TCP 11100**.
4. En caso que el puerto 11100 este abierto pero continúes sin tener la opción de conectarte puede que debas acceder al apartado de servicios, activar las opciones avanzadas, Ver(pate superior)- Avanzados y vigilar que en el apartado de Miscellanious settings no este seleccionada la opción de "Solo permitir conexiones desde equipo local"

5. Comprobación final

El servicio se inicia automáticamente al arrancar Windows. Avisa al profesor: tu equipo debería aparecer en su consola en unos segundos. Si no aparece o pide autenticación repetidamente, revisa la guía de Resolución de problemas o avisa al profesor.

⬅ Volver al taller T01

27 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

2.3.4 Taller UD01_01: Instalación de Veyon en macOS

Este manual instala en tu portátil el componente **Veyon Service**, que permite que el profesor vea y controle tu pantalla durante la clase.

Qué necesitas

- Tu portátil con macOS conectado a la red del aula.
- El fichero de **clave pública** que te ha entregado el profesor (por ejemplo `veyon-aula-profesor.pem`), descargado desde AULES o recibido por USB.

1. Descargar e instalar

1. Ve a https://veyon.io/download y descarga el instalador de macOS (`veyon-<versión>.pkg`), o descárgalo directamente desde https://github.com/veyon/veyon/releases.
2. Abre el `.pkg` y sigue el asistente de instalación.

> ⚠ **Aviso de Gatekeeper**
>
> Al abrir el instalador o la app por primera vez, macOS puede avisar de que es de un "desarrollador no identificado". Ve a **Ajustes del Sistema → Privacidad y seguridad**, baja hasta el aviso sobre Veyon y pulsa **"Abrir de todos modos"**.

2. Conceder permisos del sistema (imprescindible)

macOS exige permiso explícito para que una app pueda ver o controlar la pantalla. Sin esto, el profesor no podrá verte aunque el servicio esté instalado:

1. Ve a **Ajustes del Sistema → Privacidad y seguridad → Grabación de pantalla**.
2. Activa el permiso para **Veyon Service**.
3. Ve también a **Ajustes del Sistema → Privacidad y seguridad → Accesibilidad**.
4. Activa el permiso para **Veyon Service** (necesario para que el profesor pueda controlar el ratón/teclado, no solo ver la pantalla).
5. Es posible que macOS te pida **cerrar sesión o reiniciar** para que los permisos surtan efecto.

3. Configurar la autenticación por clave

1. Abre **"Veyon Configurator"** (Launchpad o Aplicaciones).
2. Ve a la página **"Autenticación" (Authentication)** y en **Método** selecciona **"Autenticación mediante fichero de clave" (Key file authentication)**.
3. Ve a la página **"Claves de autenticación" (Authentication keys)**, pulsa **"Importar" (Import)**, ponle el mismo nombre que usó el profesor (`Docent`), y selecciona el fichero de clave pública que te ha dado el profesor.

4. Firewall de macOS

Si tienes activado el Firewall (**Ajustes del Sistema → Red → Firewall**), asegúrate de que no está bloqueando **Veyon Service**; si lo bloquea, permite las conexiones entrantes para esa aplicación.

5. Comprobación final

Avisa al profesor: tu equipo debería aparecer en su consola en unos segundos. Si no aparece, revisa que has concedido **ambos** permisos (Grabación de pantalla y Accesibilidad), ya que es la causa más común de fallo en macOS. Si el problema persiste, consulta la guía de Resolución de problemas o avisa al profesor.

⬅ Volver al taller T01

23 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

2.3.5 Taller UD01_01: Resolución de problemas con Veyon

El equipo del alumno no aparece en la consola del profesor

1. **¿Está `veyon-service` en marcha?**
   - Linux/macOS: `pgrep -fa veyon-service`
   - Windows: comprobar en el Administrador de tareas que `veyon-service.exe` está en ejecución.
2. **¿Están en la misma red/VLAN?** La detección automática por red (broadcast) solo funciona dentro de la misma subred. Si el portátil está en una red de invitados/VLAN distinta a la del aula, no se detectará: conéctalo a la red correcta.
3. **¿Bloquea el firewall el puerto 11100/TCP?** Revisa el firewall del propio portátil (ufw / Firewall de Windows / Firewall de macOS) y, si el centro tiene firewall de red entre VLANs, confirma que permite ese puerto entre alumnos y profesor.
4. **macOS:** la causa más habitual es no haber concedido los permisos de **Grabación de pantalla** y **Accesibilidad** a Veyon Service (ver Instalación de Veyon en macOS).
5. Comprueba que el equipo está dado de alta en la ubicación del aula con el hostname o la IP fija correctos. Esto lo gestiona el profesor desde su equipo: si sospechas que es la causa, avísale. Si se usó el hostname, comprueba primero que resuelve (siguiente punto).

El hostname del alumno no resuelve (`ping <hostname>` falla, pero `ping <IP>` funciona)

En la red del aula puede ocurrir que no se resuelvan los nombres de equipo (ni por NetBIOS ni por mDNS/DNS), aunque los portátiles sean accesibles por IP. Veyon **no admite dar de alta un equipo solo por MAC** — la MAC es un campo opcional exclusivo para Wake-on-LAN; para conectar necesita hostname o IP.

Solución: en vez de fiar la identificación a un hostname que no resuelve, se fija la IP de cada portátil con una **reserva DHCP por dirección MAC** en el DHCP del centro, y se da de alta esa IP fija en el Configurador del profesor. Así el portátil sigue recibiendo la IP "por DHCP" pero siempre la misma. Esto lo gestiona el profesor: facilítale la dirección MAC de tu portátil si te la pide.

El equipo aparece pero falla la autenticación / no se puede conectar

1. Comprueba que en **ambos** equipos (profesor y alumno) el método de autenticación configurado en `veyon-configurator` es **el mismo**: "Autenticación mediante fichero de clave" (Key file authentication). Si uno tiene "logon" y otro "clave", fallará.
2. Comprueba que has importado la clave **pública** correcta (la llamada `Docent`, generada en el equipo del profesor), y no una clave antigua o de otra aula.
3. Si el profesor ha regenerado el par de claves (por ejemplo tras reinstalar su equipo), hay que **volver a repartir e importar** la nueva clave pública en todos los portátiles: la clave antigua deja de ser válida.

Error "Autenticación imposible / No se encontraron archivos de clave..." al abrir Veyon Master (Linux)

Este mensaje aparece aunque la clave exista, si el usuario normal no tiene permiso de lectura sobre el fichero de la clave **privada** (en Linux suele quedar accesible solo para `root` al crearla). La solución es asignarle un grupo de acceso del que sea miembro el usuario, desde **"Claves de autenticación"** en el Configurador. Solo afecta al equipo del profesor (Veyon Master), no a los portátiles del alumnado.

Al importar la clave, el diálogo busca ficheros `*.perm` en vez de `*.pem` (Linux)

En Linux, el diálogo **"Importar" (Import)** de la página **"Claves de autenticación"** del Configurador puede mostrar el filtro de ficheros `*.perm` en lugar de `*.pem` (parece un bug de Veyon). Como consecuencia, el fichero de la clave (por ejemplo `veyon-aula-profesor.pem`) **no aparece en la lista** aunque esté en la carpeta correcta, y parece que no exista.

El diálogo no ofrece la opción **"Todos los ficheros" (All files)**, así que no se puede cambiar el filtro.

Solución (comprobada): **renombra el fichero de la clave cambiando la extensión `.pem` por `.perm`** y vuelve a importarlo. Desde una terminal, en la carpeta donde está la clave:

```
mv veyon-aula-profesor.pem veyon-aula-profesor.perm
```

Ahora el fichero sí aparece en el diálogo; selecciónalo e impórtalo con el nombre `Docent` como de costumbre.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Puertos y protocolo usados por Veyon

- **TCP 11100**: comunicación entre Veyon Master y Veyon Service (control remoto, autenticación).
- La detección automática de equipos usa tráfico de difusión (broadcast/anuncio) dentro de la misma subred; no atraviesa routers ni VLANs distintas.

Cambios de versión

Los nombres exactos de menús, botones o rutas de los ficheros de configuración pueden variar ligeramente entre versiones de Veyon. Ante cualquier duda, consulta la documentación oficial en https://docs.veyon.io.

⬅ Volver al taller T01

23 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

2.3.6 Taller UD01_02: Instalación y uso de entornos de desarrollo

Java

Cada software y cada entorno de desarrollo tiene unas características y funcionalidades específicas. Esto también se verá reflejado en la instalación y configuración del software. Dependiendo de la plataforma, entorno o sistema operativo en el que se vaya a instalar el software, se utilizará un paquete de instalación u otro, y habrá que tener en cuenta unas opciones u otras en su configuración. A continuación se muestra cómo instalar una herramienta de desarrollo de software integrada, como Eclipse. Pero también podrás observar los procedimientos para instalar otras herramientas necesarias o recomendadas para trabajar con el lenguaje de programación JAVA, como Tomcat o la Máquina Virtual de Java. Debes tener en cuenta los siguientes conceptos:

Definición

- **JVM**: Máquina Virtual Java — ejecuta el bytecode.
- **JRE**: Java Runtime Environment — incluye la JVM + bibliotecas para ejecutar aplicaciones.
- **JDK**: Java Development Kit — incluye el JRE + herramientas de desarrollo (compilador `javac`, etc.).

El proceso de instalación consta de los siguientes pasos:

1. Descargue, instale y configure el JDK.
2. Descargue e instale un servidor web o de aplicaciones.
3. Descargue, instale y configure el IDE (Netbeans o Eclipse).
4. Configurar JDK con IDE.
5. Configure el servidor web o de aplicaciones con el IDE instalado.
6. Si es necesario, instalación de conectores.
7. Si es necesario, instale un nuevo software.

DESCARGUE E INSTALE EL JDK

Podemos diferenciar entre:

- Java SE (Java Standard Edition): es la versión estándar de la plataforma, siendo esta plataforma la base para todos los entornos de desarrollo Java ya sea de aplicaciones cliente, de escritorio o web.
- Java EE (Java Enterprise Edition): esta es la versión más grande de Java y generalmente se utiliza para crear grandes aplicaciones cliente/servidor y para el desarrollo de servicios web.

En este curso se utilizarán las funcionalidades de Java SE. El archivo es diferente según el sistema operativo donde se tenga que instalar. Así:

- Para los sistemas operativos Windows y Mac OS hay un archivo instalable.
- Para los sistemas operativos GNU/Linux que admiten paquetes .rpm o .deb, también están disponibles paquetes de este tipo.
- Para el resto de sistemas operativos GNU/Linux existe un archivo comprimido (terminado en .tar.gz).

En los dos primeros casos, simplemente hay que seguir el procedimiento de instalación habitual del sistema operativo con el que estamos trabajando. En este último caso, sin embargo, hay que descomprimir el archivo y copiarlo en la carpeta donde se desea instalar. Normalmente, todos los usuarios tendrán permisos de lectura y ejecución en esta carpeta.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Atención

Desde JDK 11, Oracle cambió la licencia. Para evitar restricciones, usa **OpenJDK** (descargable desde https://adoptium.net/), que es gratuito y de código abierto.

Una alternativa es utilizar https://adoptium.net/ antes conocido como adoptOpenJDK, que ahora se ha integrado en la fundación Eclipse. Desde allí podemos descargar los binarios de la versión openJDK para nuestra plataforma sin restricciones. [Noticia completa] (https://es.wikipedia.org/wiki/OpenJDK).

GNU/Linux

En GNU/Linux podemos utilizar los comandos:

- `sudo apt install default-jdk` para instalar el jdk predeterminado.
- `java --version` para ver las versiones disponibles en nuestro sistema.
- `sudo update-alternatives --config java` para elegir cuál de las versiones instaladas queremos usar por defecto o incluso ver la ruta de las diferentes versiones que tenemos instaladas.

CONFIGURAR LAS VARIABLES DE ENTORNO "JAVA_HOME" Y "PATH"

Una vez descargado e instalado el JDK, debes configurar algunas variables de entorno:

- La variable `JAVA_HOME`: indica la carpeta donde se ha instalado el JDK. No es obligatorio definirla, pero es muy cómodo hacerlo, ya que muchos programas buscan en ella la ubicación del JDK. Además, resulta muy fácil definir las dos variables siguientes.
- La variable `PATH`. Debe apuntar al directorio que contiene el ejecutable de la máquina virtual. Suele ser la subcarpeta `bin` del directorio donde hemos instalado el JDK.

Variable CLASSPATH Otra variable que tiene en cuenta el JDK es la variable `CLASSPATH`, que apunta a las carpetas donde se encuentran las librerías de la aplicación que se quiere ejecutar con el comando java. Es preferible, no obstante, indicar la ubicación de estas carpetas con la opción `-cp` del mismo comando java, ya que cada aplicación puede tener diferentes librerías y las variables de entorno afectan a todo el sistema. Establecer la variable `PATH` es esencial para que el sistema operativo encuentre los comandos JDK y pueda ejecutarlos.

Eclipse

Eclipse es una aplicación de código abierto desarrollada actualmente por Eclipse Foundation, una organización independiente, sin fines de lucro, que fomenta una comunidad de código abierto y el uso de un conjunto de productos, servicios, capacidades y complementos para la divulgación del uso de código abierto en el desarrollo de aplicaciones informáticas. Eclipse fue desarrollado originalmente por IBM como sucesor de VisualAge. Como Eclipse está desarrollado en Java, es necesario, para su ejecución, tener un JRE (Java Runtime Environment) previamente instalado en el sistema. Para saber si tienes este JRE instalado, puedes hacer el test en la web oficial de Java, en la sección ¿Tengo Java? Si vamos a desarrollar con Java, como es nuestro caso, deberemos tener instalado el JDK (recordemos que es un superconjunto del JRE).

INSTALACIÓN

Las versiones actuales del entorno Eclipse se instalan con un instalador. Este, básicamente, se encarga de descomprimir, solucionar algunas dependencias y crear los accesos directos. Este instalador se puede obtener descargándolo directamente desde la página oficial del Proyecto Eclipse www.eclipse.org. Podrás encontrar las versiones para los diferentes sistemas operativos e instrucciones para su uso. No son nada complejas. En el caso de GNU/Linux y MAC OS, el archivo es un archivo comprimido, por lo que hay que descomprimirlo y luego ejecutar el instalador. Se trata del archivo *eclipse-inst*, dentro de la carpeta eclipse, que es una subcarpeta del resultado de descomprimir el archivo anterior. Si sólo el usuario actual va a utilizar el IDE, la instalación se puede realizar sin utilizar privilegios de administrador o root y seleccionando para la instalación una carpeta perteneciente a este usuario. Si se desea compartir la instalación entre distintos usuarios, se debe indicar al instalador una carpeta sobre la que todos estos usuarios tengan permisos de lectura y ejecución.

Al iniciar el instalador veremos una pantalla similar a esta:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

El instalador nos preguntará qué versión queremos instalar. La versión que utilizaremos es “Eclipse IDE for Java EE Developers”.

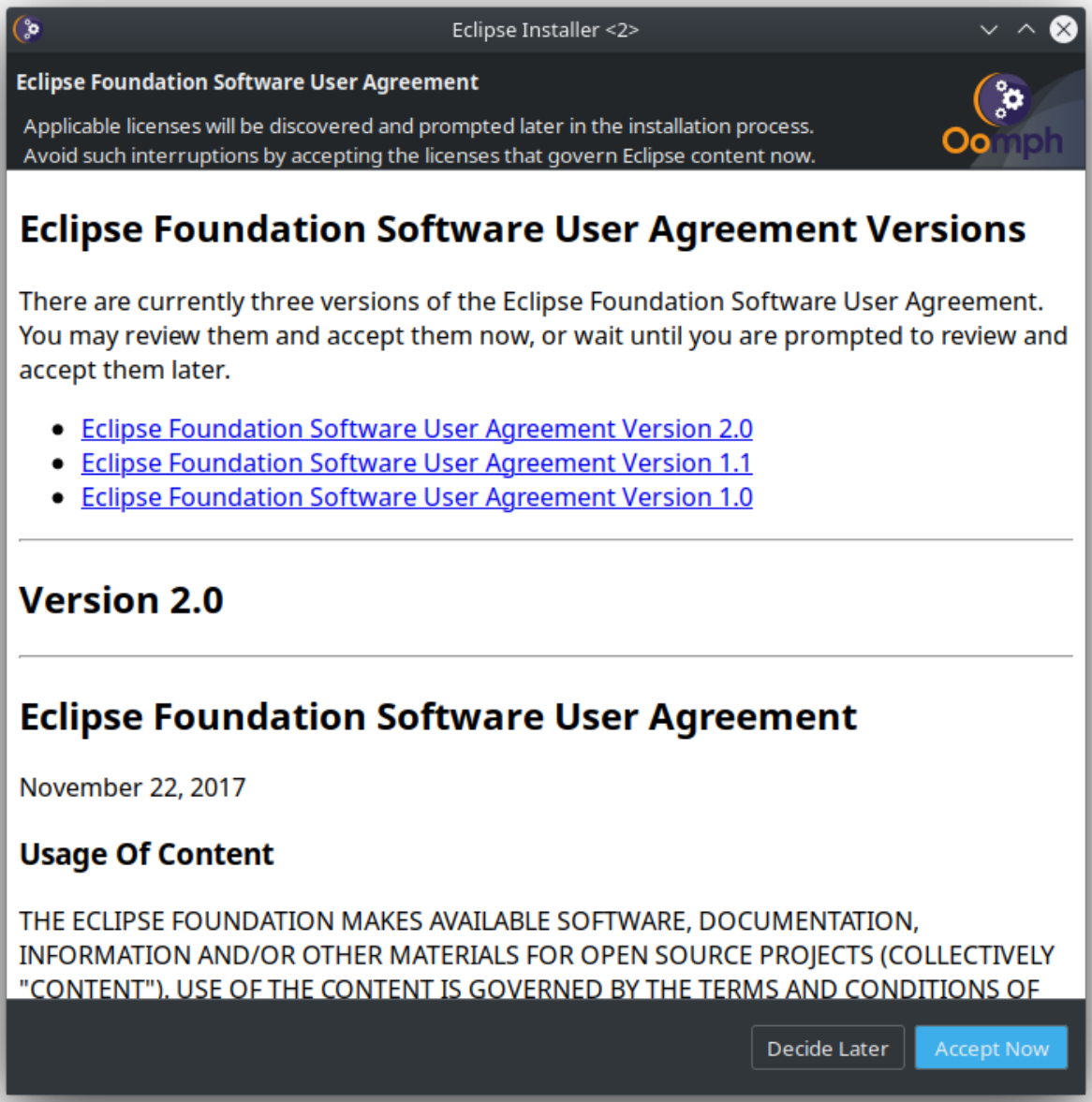
Luego nos pedirá la versión de JDK/JRE que vamos a utilizar (en la captura aparece con letras blancas). También nos pide la carpeta donde la instalaremos. Y dos check boxes para indicar si queremos que nos cree el acceso directo al menú de aplicaciones ya en el escritorio.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Para seleccionar la carpeta correcta hay que tener en cuenta qué usuarios van a utilizar el entorno. Todos ellos deben tener permisos de lectura y ejecución sobre la carpeta en cuestión. Una vez introducida la carpeta podemos pulsar el botón INSTALAR para iniciar la instalación.

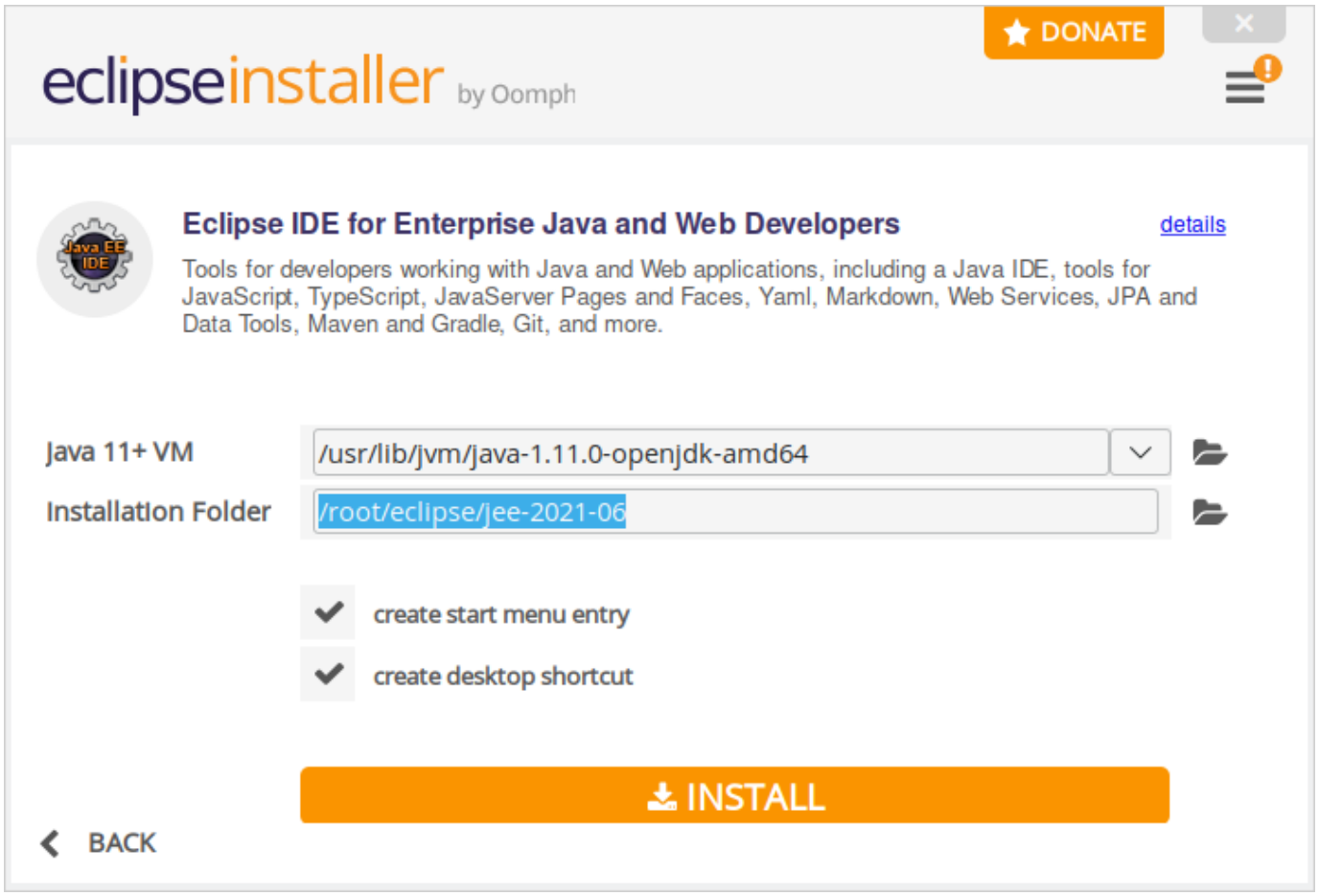
También se nos pedirá que aceptemos las licencias del software a instalar, como muestra la captura de pantalla:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



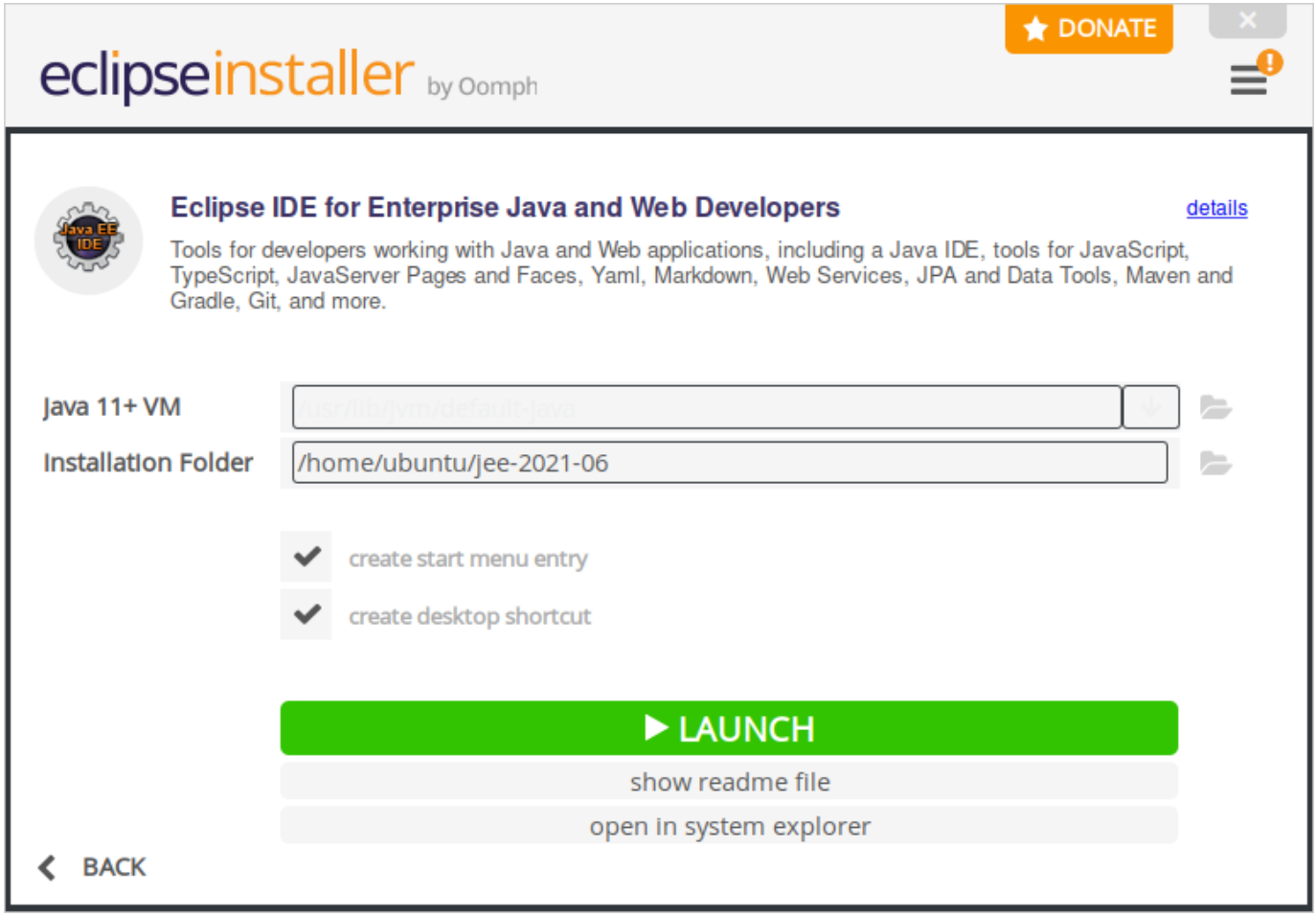
Durante la instalación veremos una pantalla de progreso como la que se muestra a continuación:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Una vez finalizada la instalación, se nos muestra una pantalla que nos invita a ejecutar directamente el entorno.

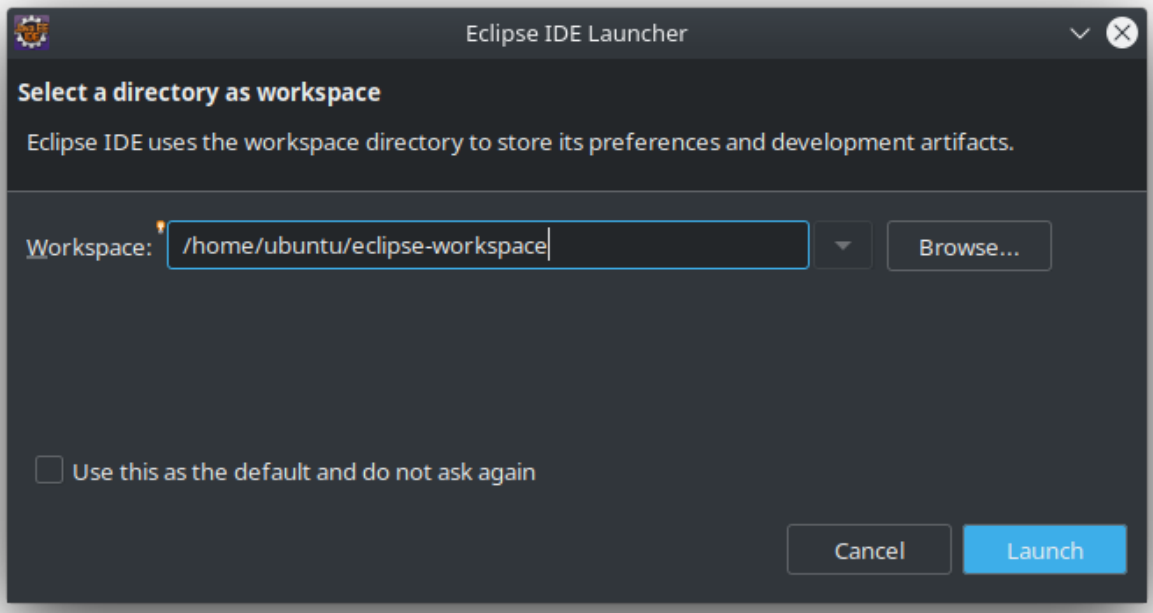
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



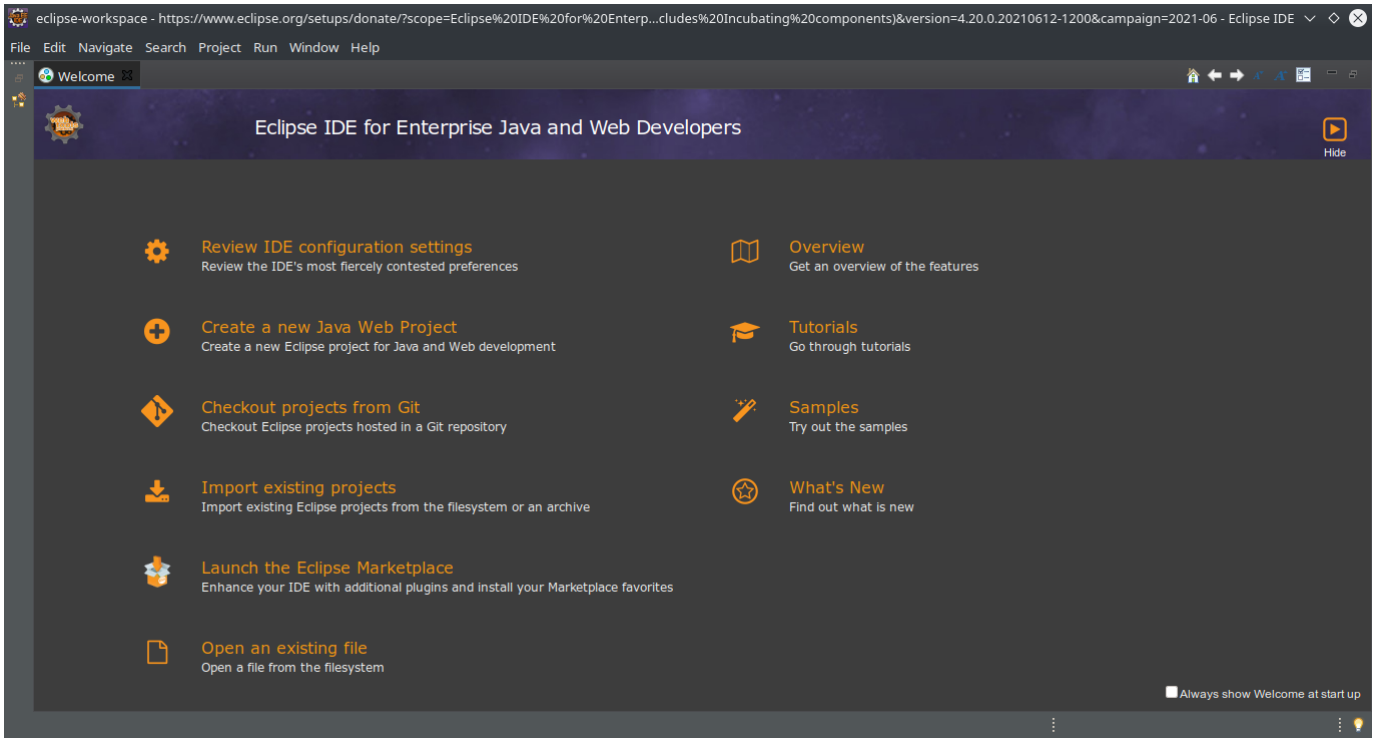
Esta primera vez podremos ejecutar el entorno Eclipse pulsando el botón LAUNCH. El resto de las veces será necesario invocarlo desde los accesos directos o lanzadores, si se han creado o, en caso contrario, invocando directamente el ejecutable. Este se llama eclipse y lo encontrarás en una subcarpeta de la carpeta de instalación también llamada eclipse. La ruta exacta puede variar de una versión a otra. Si en el futuro es necesario desinstalarlo, sólo se debe borrar la carpeta donde ha sido instalado ya que la instalación de Eclipse no aparece en el repositorio de GNU/Linux ni en el panel de control en Windows. Cuando ejecutamos el entorno nos aparecerá una pantalla como la siguiente:

Inmediatamente se nos preguntará en qué carpeta se ubicará el workspace. Podemos pedirle que lo recuerde para el resto de ejecuciones activando la opción "*Usar esto como predeterminado y no volver a preguntar*".

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

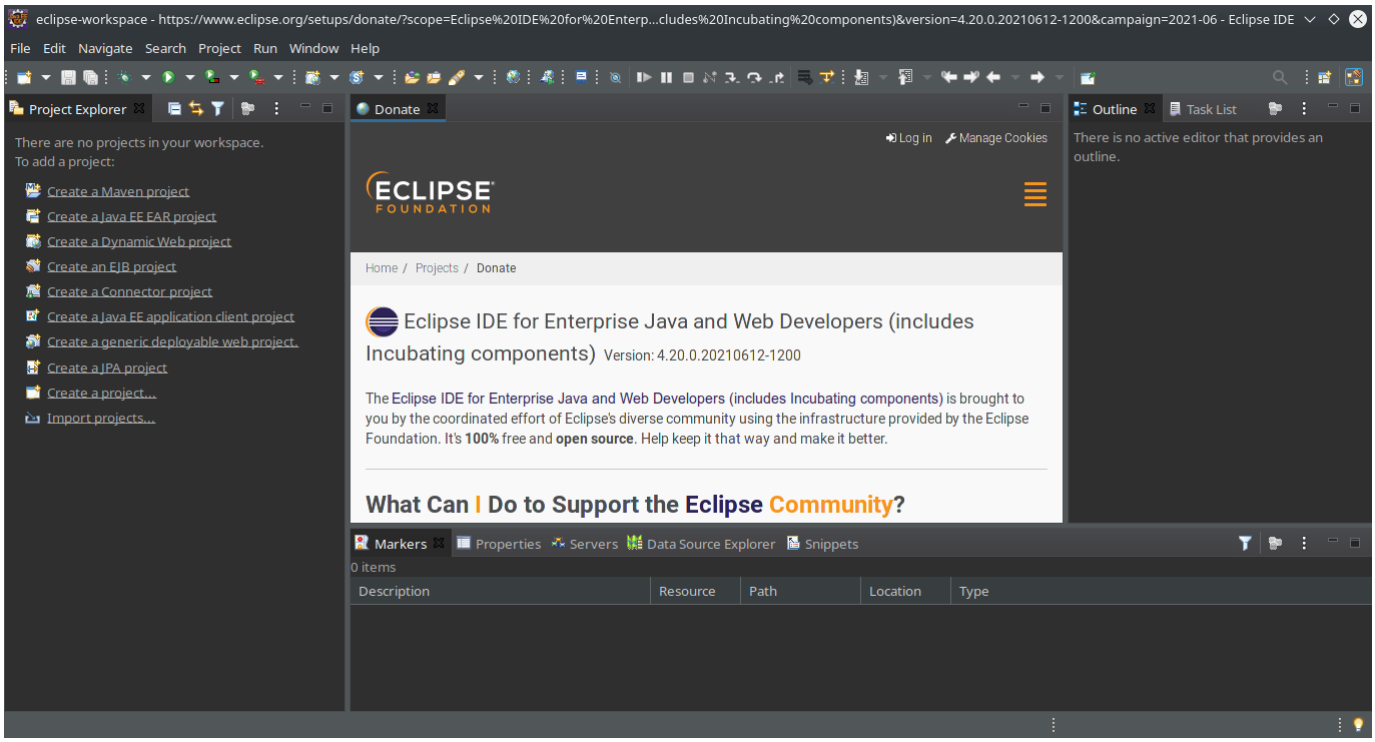


La primera vez que lo ejecutemos se mostrará la pestaña de bienvenida. Podemos pedirle que no nos la muestre más desactivando la opción “Mostrar siempre la bienvenida al iniciar”.



Una vez cerrada esta pestaña, el entorno de trabajo será similar a esto:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Nota

Por defecto Eclipse nos ofrece la descarga del instalador más ligero que descargará de Internet los paquetes necesarios para completar la instalación según nuestras elecciones. Si esta instalación nos da problemas, podemos descargar la versión "package" en la que previamente deberemos elegir el paquete de instalación que queramos, ocupará bastante más, pero descargará todos los paquetes necesarios. Después solo tendremos que descomprimir el archivo descargado en una carpeta de nuestra elección y ya tendremos eclipse instalado. Tendremos que crear nuestro propio menú de inicio e iconos del escritorio (podéis seguir esta [guía] (https://www.donovanbrown.com/post/Añadir-Eclipse-al-Launcher-en-Ubuntu-1604) cambiando la ruta donde habéis descomprimido vuestra versión de eclipse).

CONFIGURACIÓN

Versión Java

Por defecto Eclipse intenta utilizar las nuevas características del JDK 16, pero en nuestro caso por ejemplo tenemos la versión 11. Podemos personalizar estas opciones en el apartado `Ventana/Preferencias/Java/Compilador` y elegir en el campo `Nivel de conformidad del compilador` la versión correcta, en nuestro caso la 11.

Además, si lo necesitamos, podemos configurar los JDKs que están disponibles, añadirlos o eliminarlos desde la opción `Ventana/Preferencias/Java/JRE instalados`.

Perspectiva

Eclipse llama a la distribución de los paneles en la ventana Perspectiva, hay unos cuantos predefinidos y podemos configurar los nuestros, a nuestro gusto en la sección `Ventana/Perspectiva`.

Apariencia

Eclipse nos permite personalizar cualquier aspecto de la apariencia de nuestro entorno, cambiar tanto el tema del IDE como el tamaño de fuente y los colores para el coloreado del código fuente. Todas estas opciones están disponibles en `Ventana/Apariencia`.

MÓDULOS

Las opciones y funcionalidades de Eclipse se pueden ampliar añadiendo módulos desde su "store" de plugins. En `Help/Eclipse Marketplace...` podemos por ejemplo buscar por texto, o buscar en la pestaña de populares. Eso nos mostrará todos los complementos que contengan la palabra buscada, o los complementos más descargados del marketplace. Podemos instalar, por ejemplo, `SonarLint 6.0` que nos ayuda a mantener nuestro código limpio de errores comunes, para ello simplemente tenemos que

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

pulsar el botón `INSTALAR` que aparece a su lado en el listado, aceptar la licencia de uso y automáticamente nos pedirá que reiniciemos el `IDE`.

USO BÁSICO ("¡HOLA MUNDO!")

Eclipse proporciona información sobre su uso en la sección de `Ayuda`, y podemos aprender a crear nuestro primer proyecto en Java (el típico "¡Hola Mundo!"). Para ello debemos abrir la ventana de `Bienvenido`, que es la que nos aparece cuando abrimos eclipse por primera vez, o bien podemos abrirla desde `Ayuda/Bienvenido`, desde esta ventana podemos elegir la sección de `Tutoriales`, y dentro de la sección de Desarrollo Java, elegir el primer ítem "Crear una aplicación Hola Mundo", y el propio Eclipse nos irá guiando paso a paso para crear y ejecutar nuestro primer proyecto Java en Eclipse.

ACTUALIZACIÓN Y MANTENIMIENTO

En la misma sección `Ayuda` Eclipse nos proporciona las opciones para actualizar el propio Eclipse o los complementos que tengamos instalados `Ayuda/Buscar actualizaciones`.

Podemos personalizar el comportamiento respecto a las actualizaciones en la sección `Ventana/Preferencias/Instalar/Actualizar/Actualizaciones automáticas`.

Netbeans

NetBeans es una herramienta de entorno de desarrollo integrado (IDE) muy potente que se utiliza principalmente para el desarrollo en Java y C/C++. Permite desarrollar fácilmente aplicaciones web, de escritorio y móviles desde su marco modular. Puede agregar soporte para otros lenguajes de programación como PHP, HTML, JavaScript, C, C++, Ajax, JSP, Ruby on Rails, etc. mediante extensiones.

Se ha lanzado NetBeans IDE 12 con soporte para Java JDK 11. También incluye las siguientes características:

- Soporte para PHP 7.0 a 7.3, PHPStan y Twig.
- Incluir módulos en el clúster "webcommon". Es decir, todas las funciones de JavaScript en Apache NetBeans GitHub son parte de Apache NetBeans 10.
- Los módulos de clúster "groovy" están incluidos en Apache NetBeans 10.
- OpenJDK puede detectar automáticamente JTReg desde la configuración de OpenJDK y registrar el JDK expandido como una plataforma Java.
- Soporte para JUnit 5.3.1

INSTALACIÓN

Podemos instalar NetBeans de tres maneras:

Instalar desde binarios

Paso 1: Descargue el archivo NetBeans

Descargue el archivo binario de NetBeans 12 `netbeans-12.4-bin.zip`.

Paso 2: Extraer el archivo

Espere a que finalice la descarga y luego extráigala.

```
$ unzip netbeans-12.4-bin.zip
```

Confirme el contenido del archivo de directorio creado:

```
$ ls netbeans
apisupport    enterprise  groovy   javafx    netbeans.css  profiler
bin           ergonomics  harness  LICENSE   NOTICE        README.html
cpplite       etc         ide      licenses  php           webcommon
DEPENDENCIES  extide      java     nb        platform      websvccommon
```

Step 3: Move the `netbeans` folder to `/opt`

Ahora movamos la carpeta `netbeans/` a `/opt`

```
$ sudo mv netbeans/ /opt/
```

Paso 4: Ruta de configuración

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

El binario ejecutable de Netbeans se encuentra en `/opt/netbeans/bin/netbeans`. Necesitamos agregar su directorio principal a nuestro `$PATH` para poder iniciar el programa sin especificar la ruta absoluta al archivo binario. Abra su archivo `~/.bashrc` o `~/.zshrc`.

```
$ nano ~/.bashrc
```

Añade la siguiente línea al final

```
export PATH = "$PATH:/opt/netbeans/bin/"
```

Obtenga el archivo para iniciar Netbeans sin reiniciar el shell.

```
$ source ~/.bashrc
```

Paso 5: Crear el iniciador de escritorio NetBeans IDE (opcional)

Cree un nuevo archivo en `/usr/share/applications/netbeans.desktop`.

```
$ sudo nano /usr/share/applications/netbeans.desktop
```

Añade los siguientes datos.

```
[Desktop Entry]
Name=Netbeans IDE
Comment=Netbeans IDE
Type=Application
Encoding=UTF-8
Exec=/opt/netbeans/bin/netbeans
Icon=/opt/netbeans/nb/netbeans.png
Categories=GNOME;Application;Development;
Terminal=false
StartupNotify=true
```

Para desinstalar NetBeans debemos eliminar la carpeta `netbeans/` que está dentro de la carpeta /opt/, podemos utilizar el comando:

```
$ sudo rm /opt/netbeans -rf
```

Paso 6: Configurar correctamente el JDK (opcional)

En el fichero `/opt/netbeans/etc/netbeans.conf` debemos especificar correctamente la ruta de nuestro JDK en la variable `netbeans_jdkhome`. En GNU/Linux podemos saber los JDK disponibles con el comando `sudo update-alternatives --config java` que nos mostrará un resultado similar a este:

```
Hi ha 3 possibilitats per a l'alternativa java (que proveeix /usr/bin/java).

  Selecció    Camí                                        Prioritat  Estat
------------------------------------------------------------
* 0            /usr/lib/jvm/java-14-openjdk-amd64/bin/java      1411      mode automàtic
  1            /usr/lib/jvm/java-11-openjdk-amd64/bin/java      1111      mode manual
  2            /usr/lib/jvm/java-14-openjdk-amd64/bin/java      1411      mode manual
  3            /usr/lib/jvm/java-8-openjdk-amd64/jre/bin/java   1081      mode manual

Premeu retorn per a mantenir l'opció per defecte[*], o introduïu un número de selecció:
```

En la configuración de netbeans no es necesario especificar el final de la ruta `bin/java`

```
netbeans_jdkhome="/usr/lib/jvm/java-11-openjdk-amd64/"
```

Instalar desde script

Paso 1: Descargue el archivo NetBeans

También puede instalar Netbeans 12.4 en GNU/Linux desde un script proporcionado para descargar `Apache-NetBeans-12.4-bin-linux-x64.sh`.

Paso 2: Ejecutar el script

Debes ejecutar el script de instalación

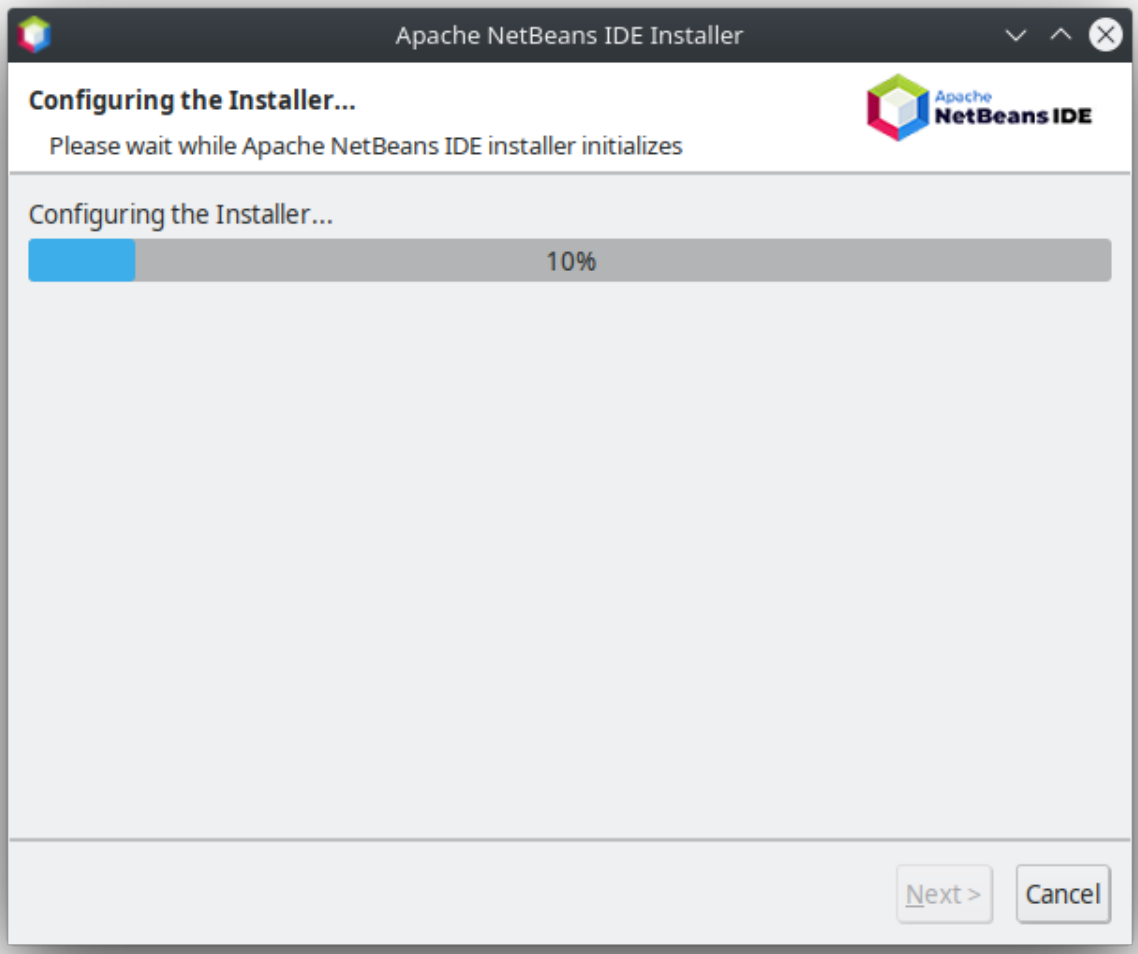
© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
$ sudo sh ./Apache-NetBeans-12.4-bin-linux-x64.sh
```

Instalación

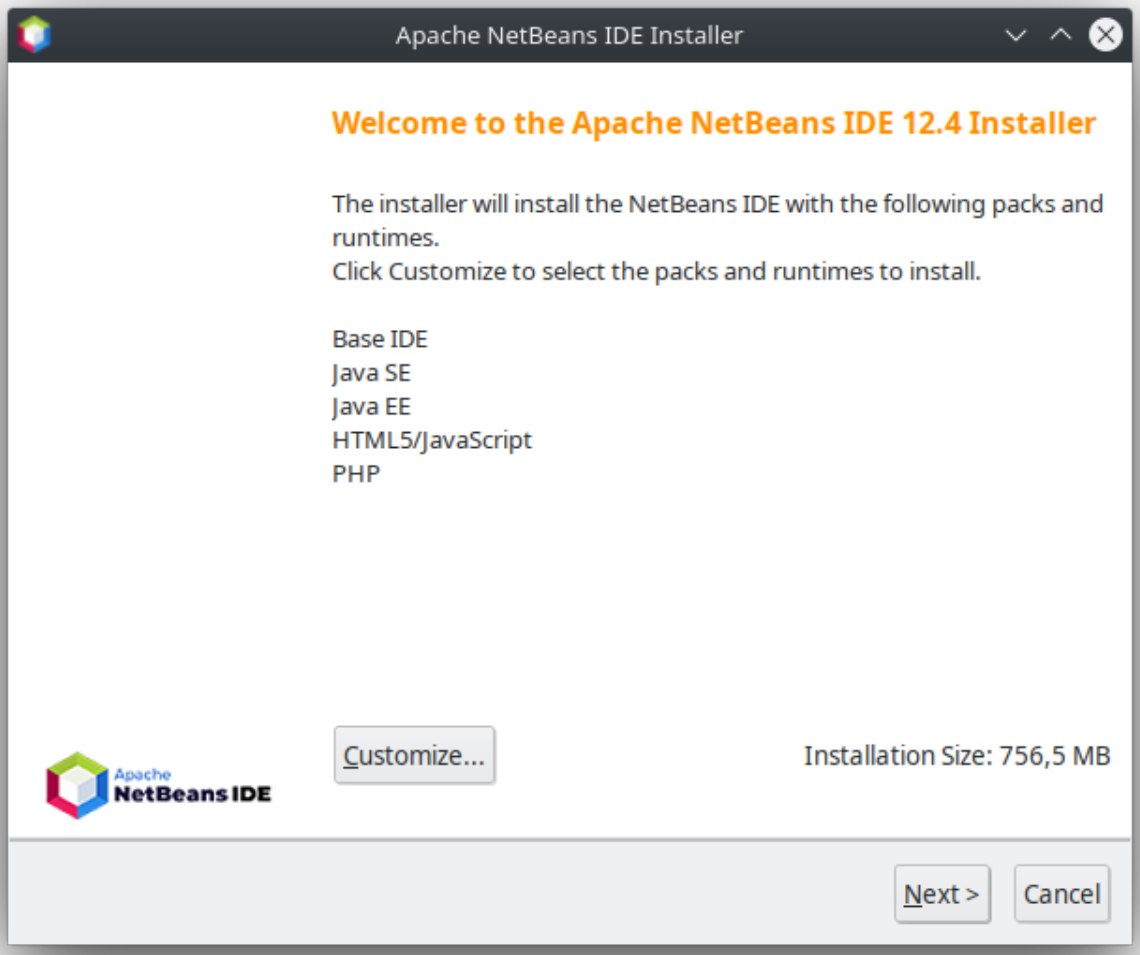
Si ejecuta el script como `root` (`sudo`) Netbeans estará disponible para todos los usuarios. Por el contrario, si ejecuta el usuario sin `sudo`, solo estará disponible para su usuario.

Aparecerá una barra de progreso como esta:



Ahora podemos elegir los componentes que queremos instalar con el IDE de Netbeans, lo dejaremos por defecto y pulsaremos el botón siguiente.

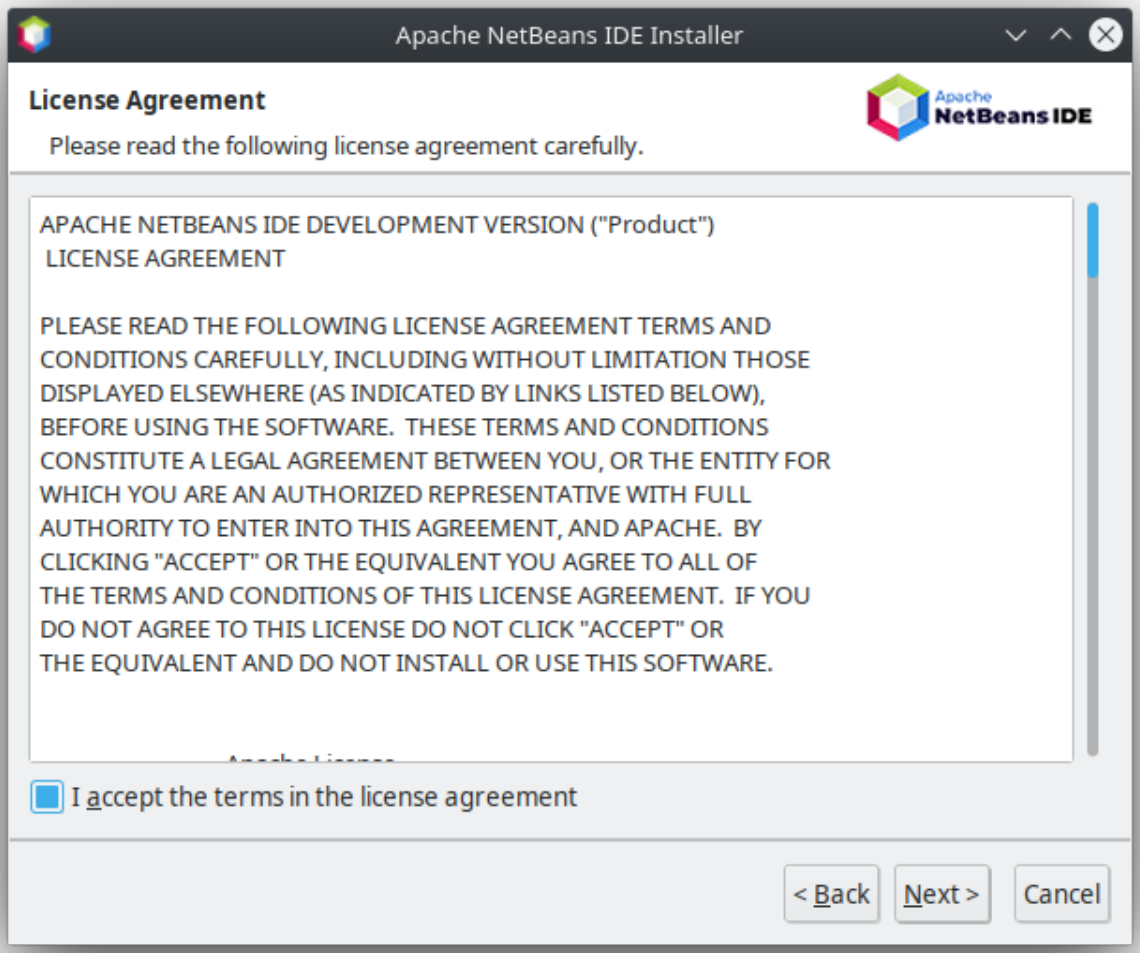
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Paso 3: Aceptar la licencia

Luego debemos aceptar el acuerdo de licencia de uso marcando la casilla y presionando el botón siguiente.

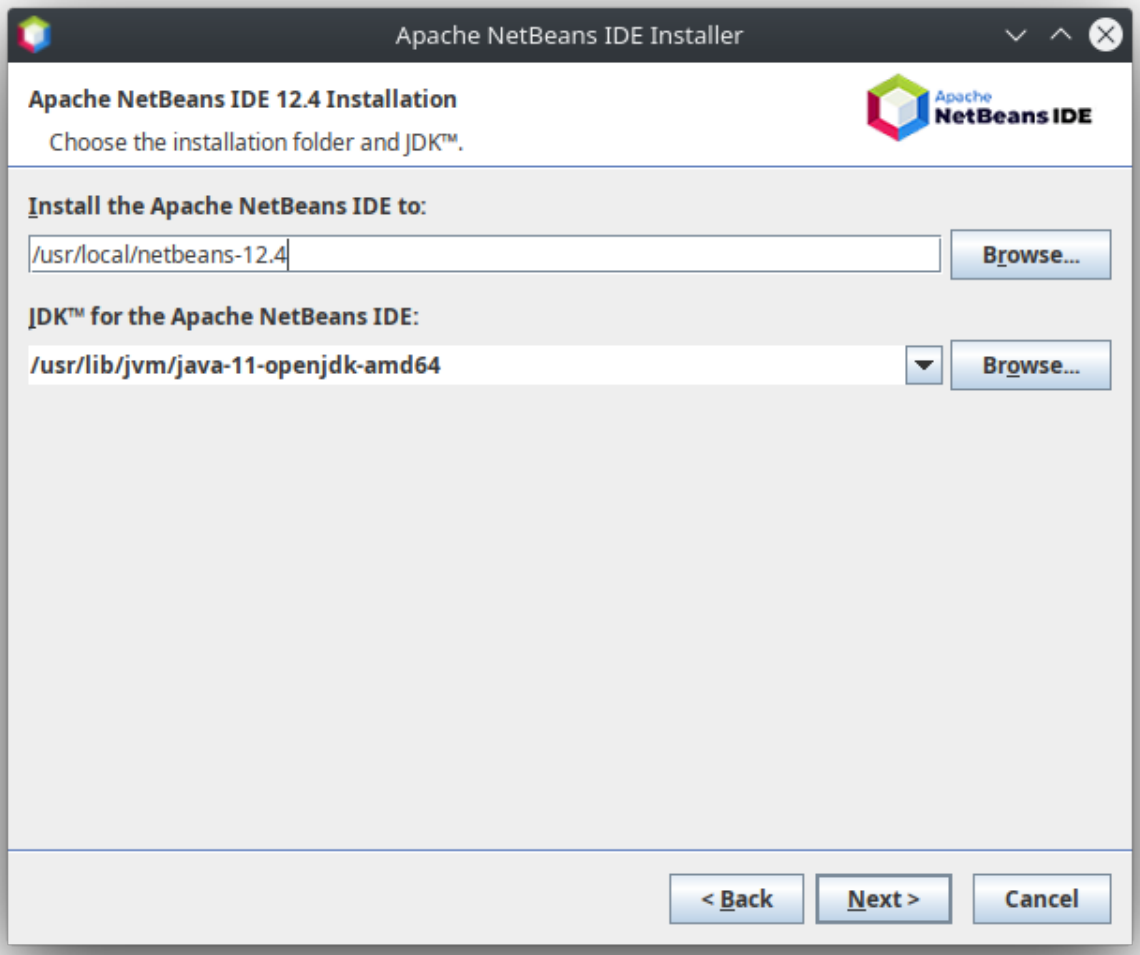
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Paso 4: Elija la ruta de instalación y el JDK

Ahora debemos elegir la ruta donde se instalará Netbeans 12.4. Y debemos elegir la ruta donde se encuentra el JDK (por defecto indica `/usr`, pero debemos especificar la ubicación como por ejemplo `/usr/lib/jvm/java-11-openjdk-amd64`).

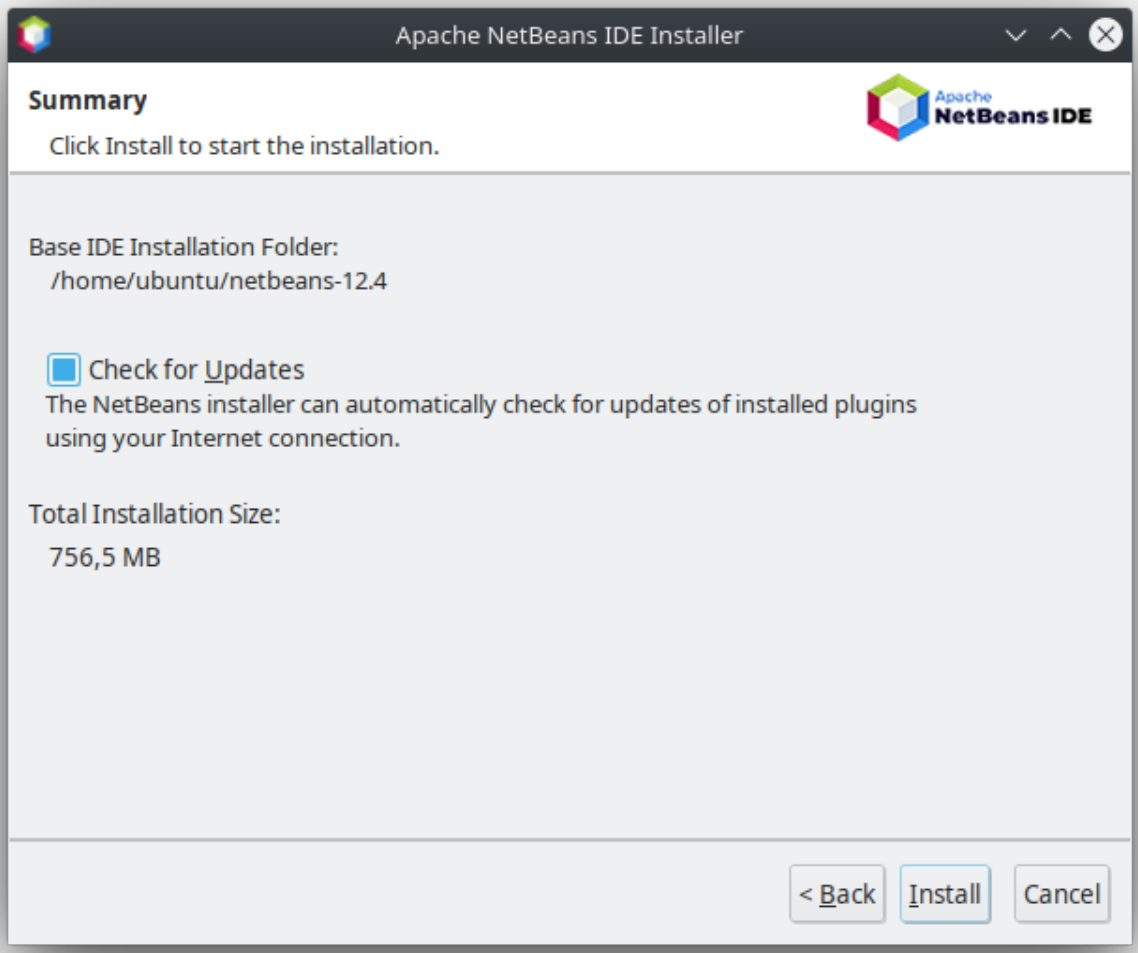
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Paso 5: Actualizaciones automáticas

En este punto se muestra un resumen de la instalación, y podemos elegir si queremos que NetBeans busque e instale actualizaciones desde Internet, y pulsar el botón instalar.

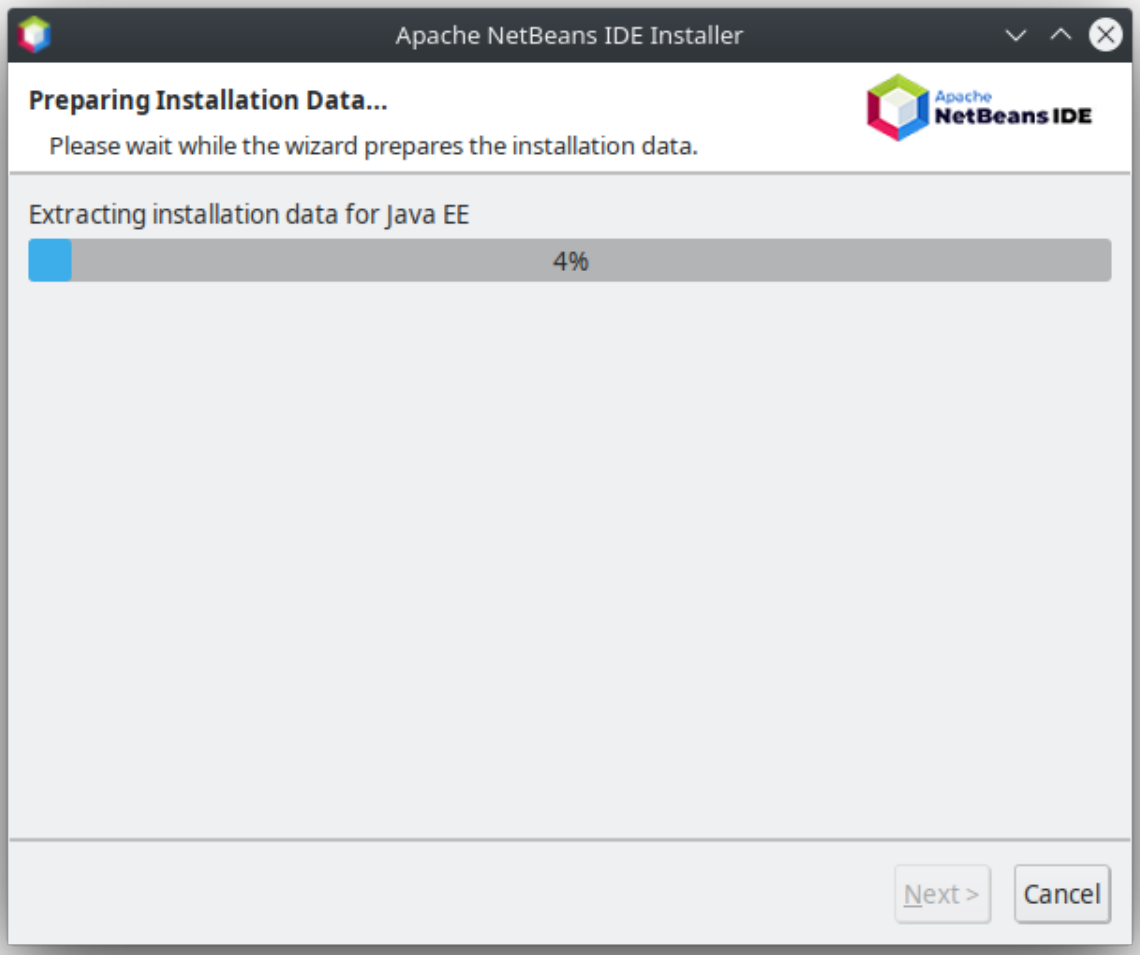
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Paso 6: Instalación

Aparecerá una barra de progreso.

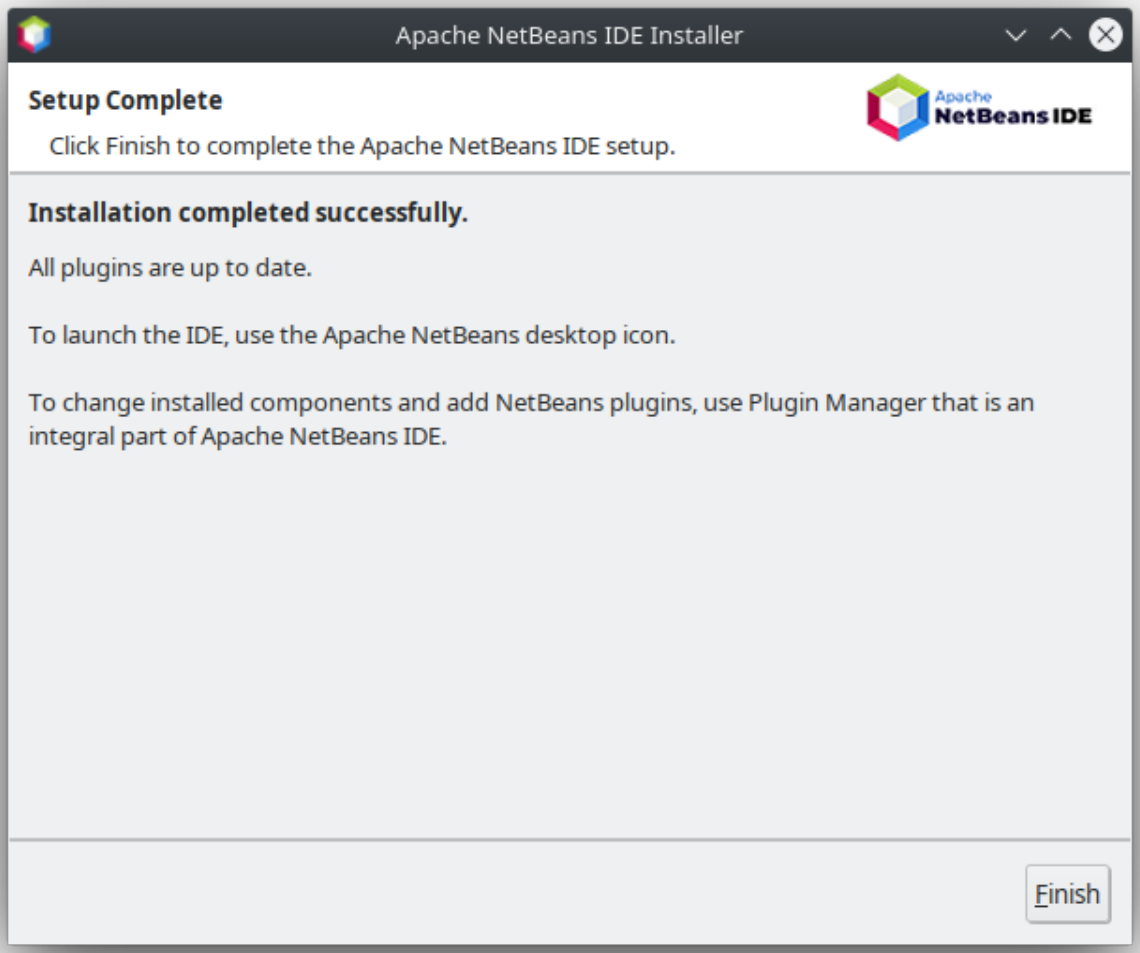
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Paso 7: Paso final

Al terminar, aparecerá una pantalla con las acciones realizadas por el instalador y ya tendremos los launchers creados en el menú de aplicaciones.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Instalar mediante snap

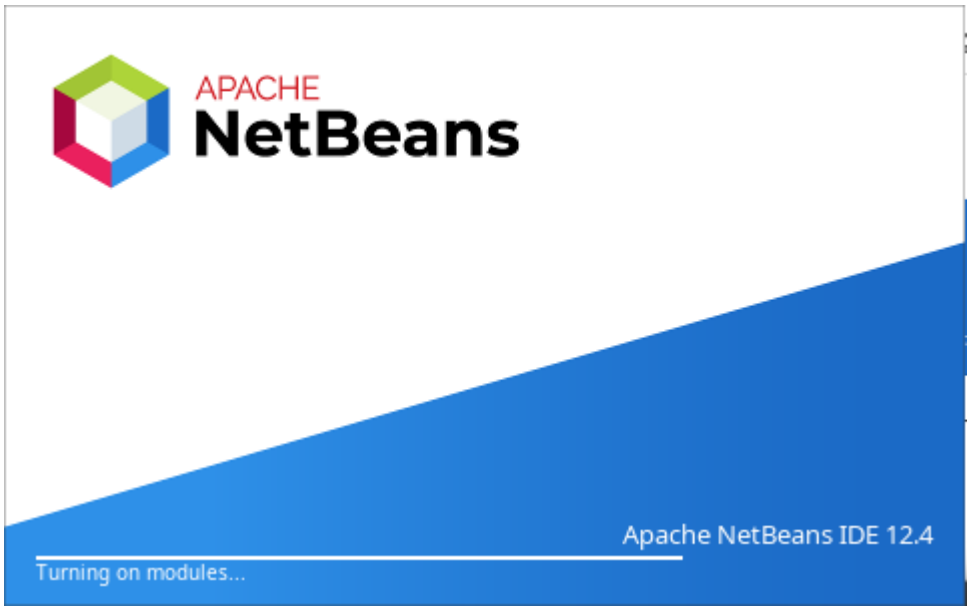
Quizás una forma más sencilla de instalar la última versión de Netbeans en nuestro sistema GNU/Linux es a través de `snap`:

```
$ sudo snap install netbeans --classic
```

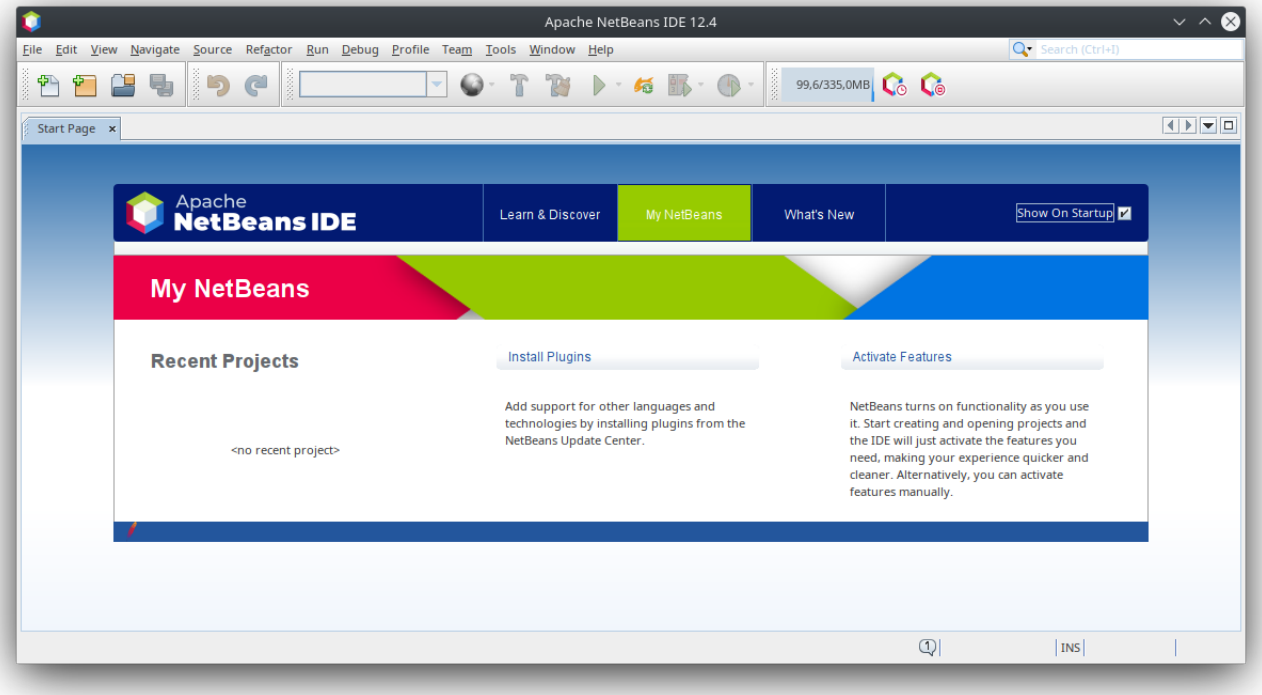
Primera ejecución

Cuando ejecutamos el entorno nos aparecerá una pantalla como la siguiente:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

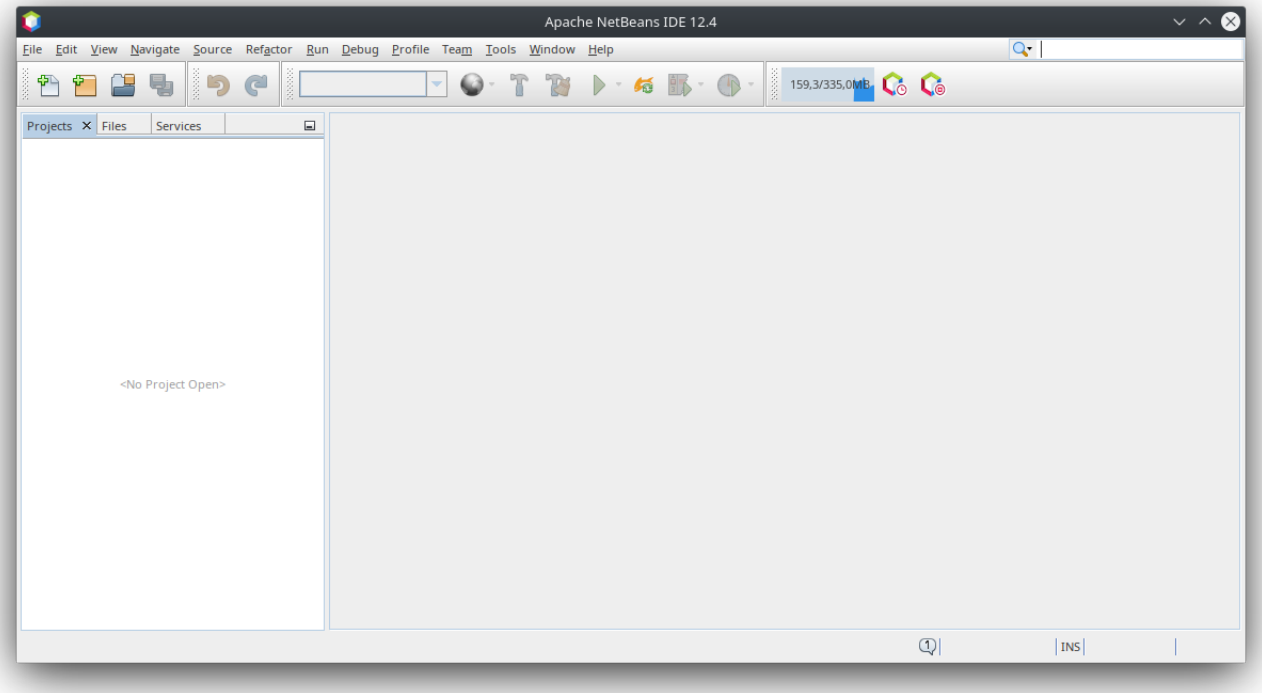


La primera vez que lo ejecutemos se mostrará la pestaña de bienvenida. Podemos pedir que no se nos muestre más desactivando la opción “Mostrar al iniciar”.

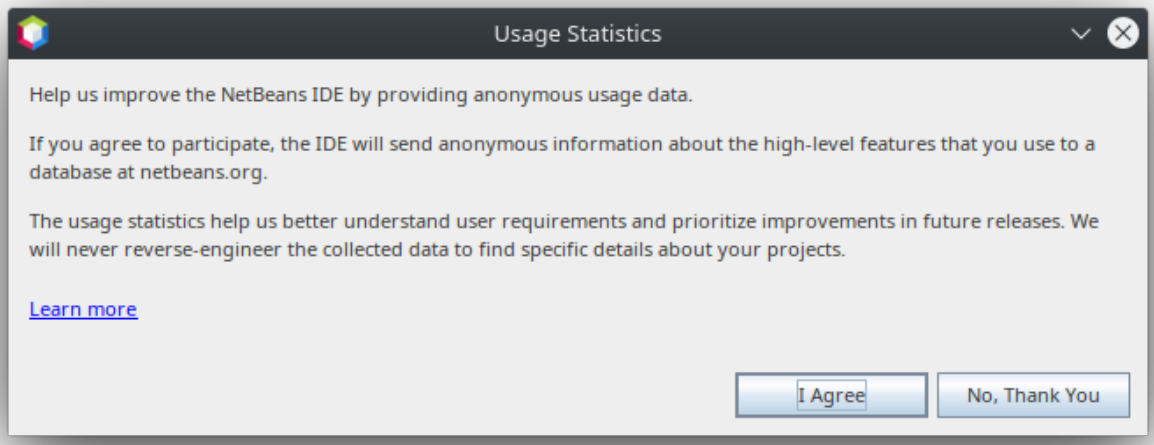


Una vez cerrada esta pestaña, el entorno de trabajo será similar a esto:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



NetBeans puede solicitarnos permiso para utilizar nuestra información a nivel estadístico, elegimos el comportamiento deseado y aceptamos.



Para desinstalar NetBeans en este caso debemos ejecutar el archivo `uninstall.sh` que se encuentra en la carpeta de instalación.

CONFIGURACIÓN

Activar módulos

Por defecto Netbeans tiene los módulos desactivados y será la primera vez que los necesitemos cuando pasen a estar activos y disponibles. Por ejemplo, si creamos un nuevo proyecto y elegimos `Java Application` dentro de la categoría `Java with Ant`, veremos en la parte inferior que Netbeans nos avisa de que el módulo necesario no está activo y que debemos pulsar `Next` para

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

que esté disponible. Lo hacemos, y a continuación nos pedirá que activemos el módulo `nb-javac Impl`, dejamos el check marcado y pulsamos el botón `Activate`, y nos aparecerá el asistente para crear nuestro primer proyecto Java.

Versión Java

Dentro del menú `Herramientas/Plataformas Java` podemos cambiar o ver la ubicación de nuestra instalación JDK.

Perspectiva

En Netbeans las perspectivas no son necesarias, el entorno de Netbeans, aunque es personalizable, se adapta automáticamente a las tareas que estés realizando en cada momento.

Apariencia

Netbeans nos permite personalizar cualquier aspecto de la apariencia de nuestro entorno, cambiar el tema del IDE así como el tamaño de fuente y los colores para el coloreado del código fuente. Todas estas opciones están disponibles en `Herramientas/Opciones`, y dentro de esta ventana elegimos la tercera pestaña `Fuente y Colores` y la penúltima pestaña `Apariencia`.

Configuración de exportación/importación

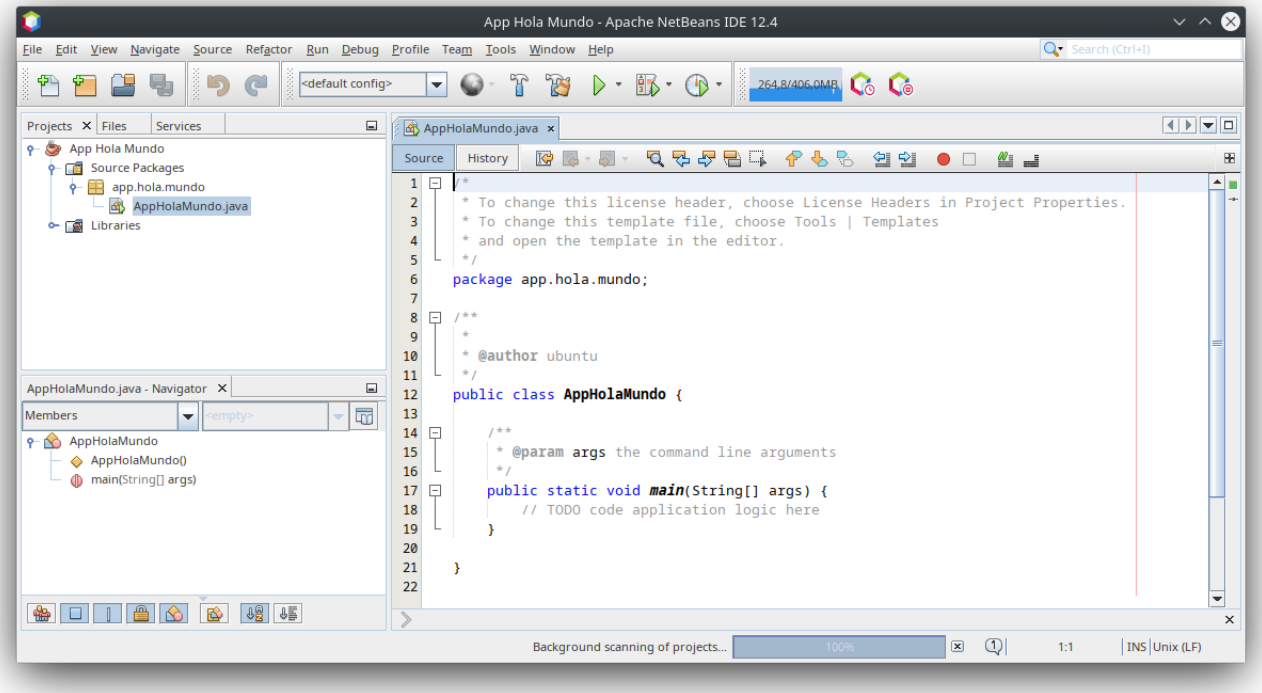
Una opción muy interesante de Netbeans es que nos permite exportar o importar configuraciones y compartirlas con otros compañeros o incluso entre nuestros equipos o diferentes instalaciones. La opción está disponible en `Herramientas/Opciones`, abajo a la izquierda encontramos los botones `Exportar...` e `Importar...`.

MÓDULOS

Las opciones y funcionalidades de Netbeans se pueden ampliar añadiendo módulos desde su sección de plugins. En `Tools/Plugins` podemos por ejemplo buscar por texto, o buscar en la pestaña de plugins disponibles. Eso nos mostrará todos los plugins que contienen la palabra buscada, o los plugins disponibles. Podemos instalar por ejemplo `sonarlint4netbeans` que nos ayuda a mantener nuestro código limpio de errores comunes, para ello simplemente tenemos que marcar la casilla delante del nombre del plugin, y pulsar el botón `INSTALAR` que aparece más abajo, pulsar siguiente, aceptar la licencia de uso e instalar. Cuando termine la instalación nos pedirá que reiniciemos el `IDE`.

USO BÁSICO ("¡HOLA MUNDO!")

Para crear nuestra primera aplicación en Netbeans, debemos crear una aplicación Java, desde el menú `Archivo/Nuevo Proyecto...` debemos elegir `Aplicación Java` dentro de la categoría `Java con Ant`. A continuación debemos especificar el nombre del proyecto, por ejemplo "App Hello World", y nos aseguramos de dejar marcada la opción `Crear clase principal app.hello.world.AppHolaWorld` y nos debería aparecer algo como esto:



© 2025 David Martínez licensed under CC BY-NC-SA 4.0

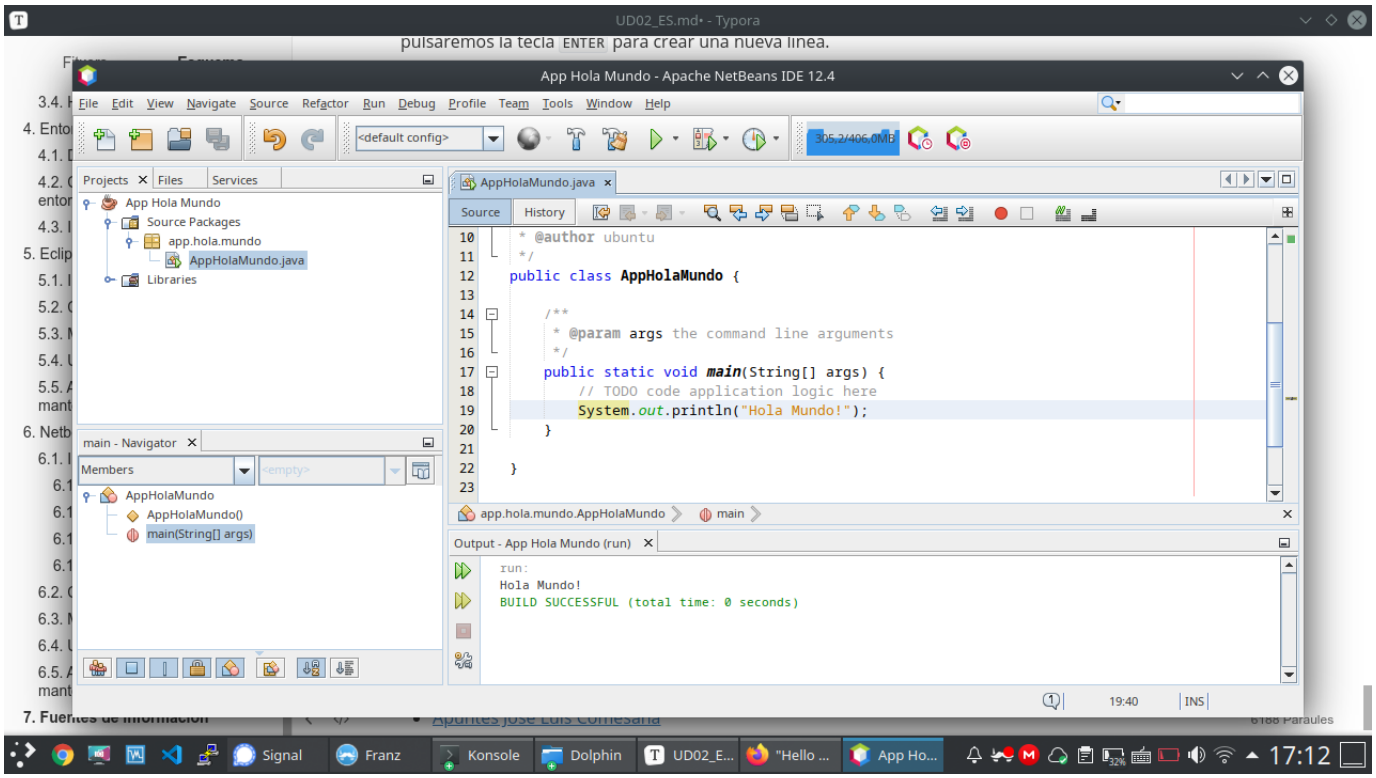
En este punto, sólo nos queda incluir la línea de código necesaria para imprimir el mensaje de texto en pantalla. Para ello, nos dirigiremos al final de la línea `// TODO code application logic here` y pulsaremos la tecla `ENTER` para crear una nueva línea.

Una vez situados en el lugar adecuado utilizaremos una de las funcionalidades más interesantes de Netbeans, que son las plantillas de código. Tecleamos la palabra "sout" y luego pulsamos la tecla `TAB` y Netbeans la sustituirá por el código correcto: `System.out.println ("");`.

Ahora debemos escribir entre las dos comillas dobles el mensaje de texto que debe aparecer en pantalla, y debe quedar así:

```
1  System.out.println("Hola Mundo!");
```

Luego podemos presionar el botón superior con un triángulo verde (`Ejecutar proyecto`) o presionar la tecla `F6` del teclado:



Aparecerá una nueva sección en la ventana (en la parte inferior) llamada `Salida` en la que podremos visualizar el resultado de la ejecución de nuestro primer programa.

ACTUALIZACIÓN Y MANTENIMIENTO

En la sección `Ayuda`, Netbeans nos proporciona las opciones para actualizar el propio Netbeans con la opción `Ayuda/Buscar actualizaciones`.

IntelliJ (recomendado)

IntelliJ IDEA es un entorno de desarrollo integrado (IDE) escrito en Java para desarrollar software informático escrito en Java, Kotlin, Groovy y otros lenguajes basados en JVM. Está desarrollado por JetBrains (antes conocido como IntelliJ) y está disponible como una edición comunitaria con licencia Apache 2 y en una edición comercial propietaria. Ambas se pueden utilizar para el desarrollo comercial.

Nuestra institución dispone de licencias para nuestros alumnos mientras tengáis correo electrónico @ieseduardoprimo.es.

INSTALACIÓN

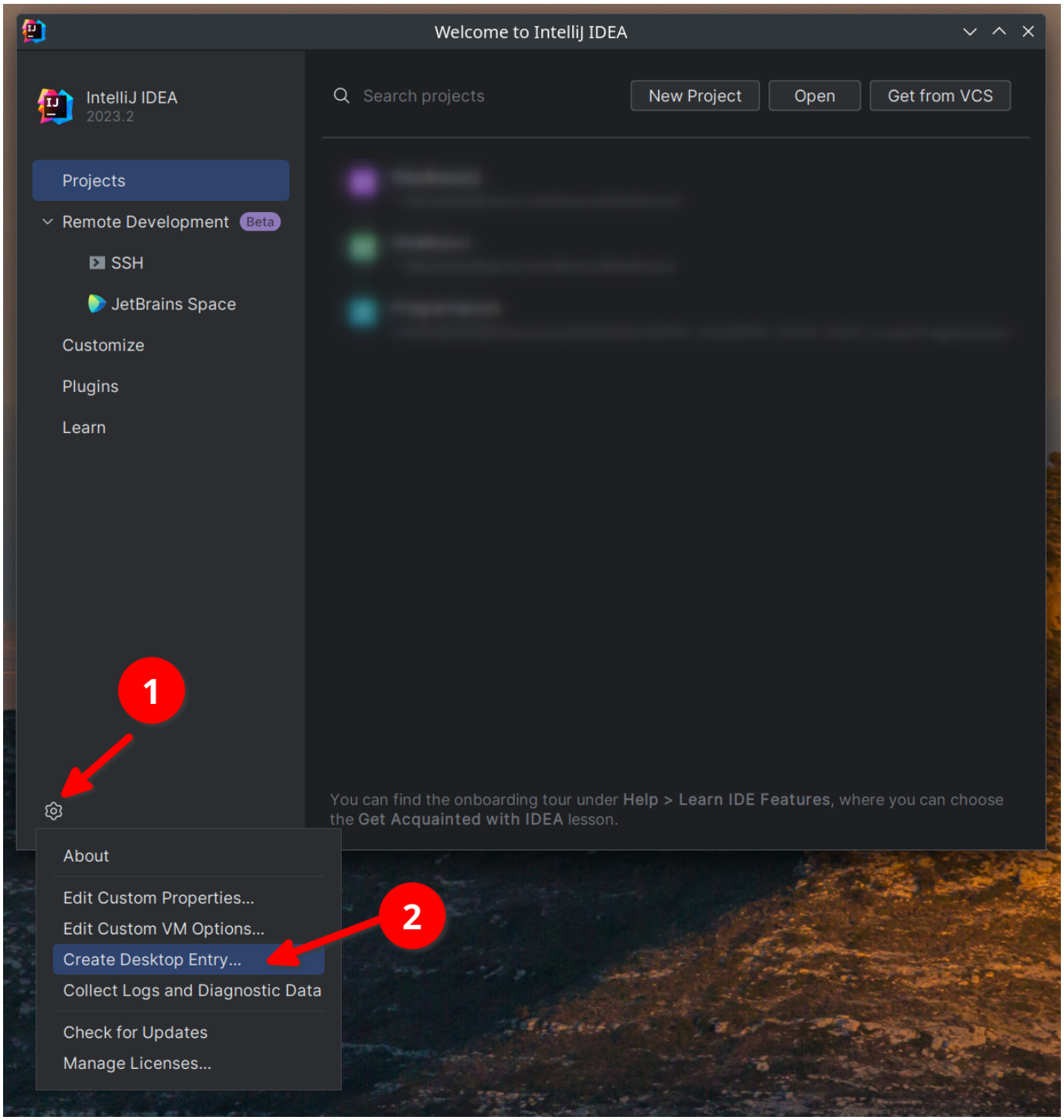
Descargue desde https://www.jetbrains.com/idea/ la versión de la herramienta toolbox correspondiente a su sistema operativo.

Siga las instrucciones para su sistema operativo desde https://www.jetbrains.com/help/idea/installation-guide.html#toolbox

Una vez instalada la caja de herramientas, puede elegir instalar todos los productos de JetBrains.

Una vez instalada la Idea (IDE) puedes crear una entrada de escritorio desde la pantalla inicial:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Y en la opción Administrar licencias debes seguir estas instrucciones: https://www.jetbrains.com/help/license_server/Activating_license.html

La dirección del servidor es: https://iesepm.fls.jetbrains.com/

AJUSTES

Documentos para configurar su IDE: https://www.jetbrains.com/help/idea/configuring-project-and-ide-settings.html

MÓDULOS

Puedes agregar complementos siguiendo estas instrucciones:

https://www.jetbrains.com/help/idea/managing-plugins.html

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

USO BÁSICO ("¡HOLA MUNDO!")

Los documentos te ayudan con tu primer programa en Java: https://www.jetbrains.com/help/idea/creating-and-running-your-first-java-application.html

Mucha más información:

- Si vienes de Eclipse: https://www.jetbrains.com/help/idea/migrating-from-eclipse-to-intellij-idea.html
- Si estuvieras en NetBeans: https://www.jetbrains.com/help/idea/netbeans.html
- Si quieres aprender por tu cuenta: https://www.jetbrains.com/help/idea/product-educational-tools.html

Por qué debería elegir IntelliJ en lugar de VsCode para la codificación en Java

IDEA INTELLIJ:

Ventajas:

1. **Entorno integrado completo:** IntelliJ IDEA está diseñado específicamente para el desarrollo de Java y ofrece un conjunto completo de herramientas y características optimizadas para esta tarea.
2. **Análisis estático avanzado:** Proporciona un análisis de código en profundidad que detecta errores y problemas potenciales antes de la compilación.
3. **Depuración avanzada:** ofrece un potente conjunto de herramientas de depuración que ayudan a identificar y resolver problemas en el código.
4. **Refactorización guiada:** Proporciona herramientas para reorganizar y optimizar el código de forma segura, promoviendo buenas prácticas de programación.
5. **Compatibilidad con marcos y tecnologías Java:** Integración nativa con muchos marcos y tecnologías utilizados en el desarrollo Java, lo que facilita la creación de aplicaciones completas.
6. **Generación automática de código:** ayuda a los programadores a generar automáticamente fragmentos de código repetitivos, como captadores y definidores.
7. **Integración con herramientas de compilación:** facilita la integración con herramientas de compilación como Maven y Gradle.
8. **Soporte para pruebas unitarias:** Ofrece integración con marcos de prueba como JUnit para el desarrollo basado en pruebas.
9. **Facilidad de configuración:** Proporciona asistentes guiados para configurar de manera eficiente proyectos Java.

Contras:

1. **Mayor consumo de recursos:** Debido a su naturaleza integral y rica en funciones, IntelliJ IDEA puede consumir más recursos del sistema en comparación con IDE más livianos.
2. **Curva de aprendizaje:** Dado que ofrece una amplia gama de funciones, los principiantes pueden tardar un tiempo en familiarizarse con todas las herramientas disponibles.

VISUAL STUDIO CODE (VSCODE):

Ventajas:

1. **Ligero y rápido:** VSCode es un editor de código liviano y rápido, lo que lo hace ideal para proyectos más pequeños o para aquellos que prefieren una experiencia más ágil.
2. **Amplia gama de extensiones:** Tiene una amplia comunidad que desarrolla extensiones para diversas tecnologías y lenguajes, incluido Java.
3. **Versatilidad:** Si bien no está diseñado específicamente para Java, se puede personalizar para que funcione con Java a través de extensiones.
4. **Integración de control de versiones:** ofrece integración nativa con sistemas de control de versiones como Git.
5. **Curva de aprendizaje rápida**: Debido a su enfoque más ligero, puede resultar más sencillo para los principiantes comenzar a trabajar con él.

Contras:

1. **Funcionalidad limitada de Java:** Aunque existen extensiones de Java, VSCode no ofrece el mismo conjunto completo de herramientas optimizadas para Java que IntelliJ IDEA.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

2. **Análisis menos profundo:** Las capacidades de análisis estático y corrección de código podrían no ser tan avanzadas como las de IntelliJ IDEA.
3. **Depuración limitada:** si bien ofrece depuración, es posible que no sea tan avanzada o completa como la de IntelliJ IDEA.
4. **Configuración manual del proyecto:** La configuración de proyectos Java puede requerir más pasos y configuración manual en comparación con IntelliJ IDEA.

Tarea

Debes entregar un documento `*.pdf` explicando que IDE has elegido para empezar a programar (más adelante lo puedes cambiar si quieres), justificando porqué lo has elegido.

Además envia una captura de pantalla en la que se vea el resultado del comando:

```
1  java --version
```

Y por último capturas de pantalla donde se pueda ver que editas el fichero fuente (HolaMundo.java), lo compilas y lo ejecutas dentro del IDE que has elegido (explica los pasos que has seguido)

2 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

2.3.7 Taller UD01_03: Crear cuenta en GitHub

¿Qué es GitHub?

Github es una plataforma en la nube basada en Git que permite a los desarrolladores almacenar, gestionar y colaborar en proyectos de código. Es el portafolio universal de los programadores.

Crear una cuenta es esencial para quien aprende o busca trabajar en programación porque: sirve como tu currículum técnico, donde muestras tus proyectos y evolución; te permite colaborar en proyectos open source para ganar experiencia real; y es una herramienta fundamental para el control de versiones y trabajo en equipo, usada por prácticamente todas las empresas tech.

Crea tu cuenta

Accede a la plataforma GitHub: https://github.com/

Pulsa sobre el botón [Sign Up] y sigue las instrucciones para crear tu cuenta.

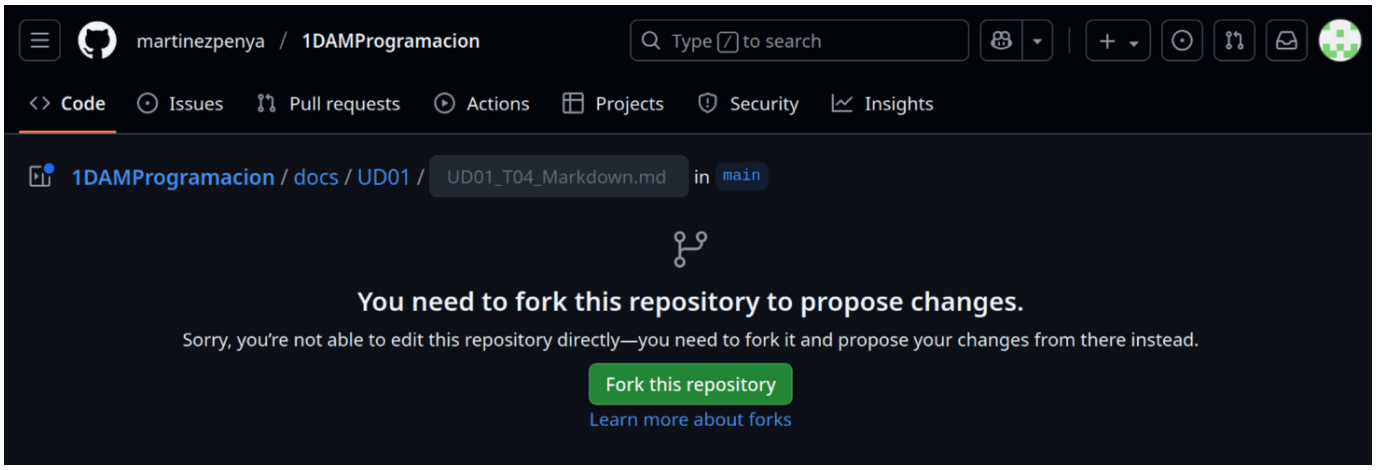
Una vez creada tu cuenta, entra en tu página principal, por ejemplo la mia es esta: https://github.com/martinezpenya (`martinezpenya` es mi usuario de github) y realiza una captura de pantalla.

Solicitar corrección de los apuntes

Ahora, para probar nuestra nueva cuenta y colaborar con algún proyecto, no hay nada mejor que ayudar a mejorar los apuntes del profesor de Programación 😃.

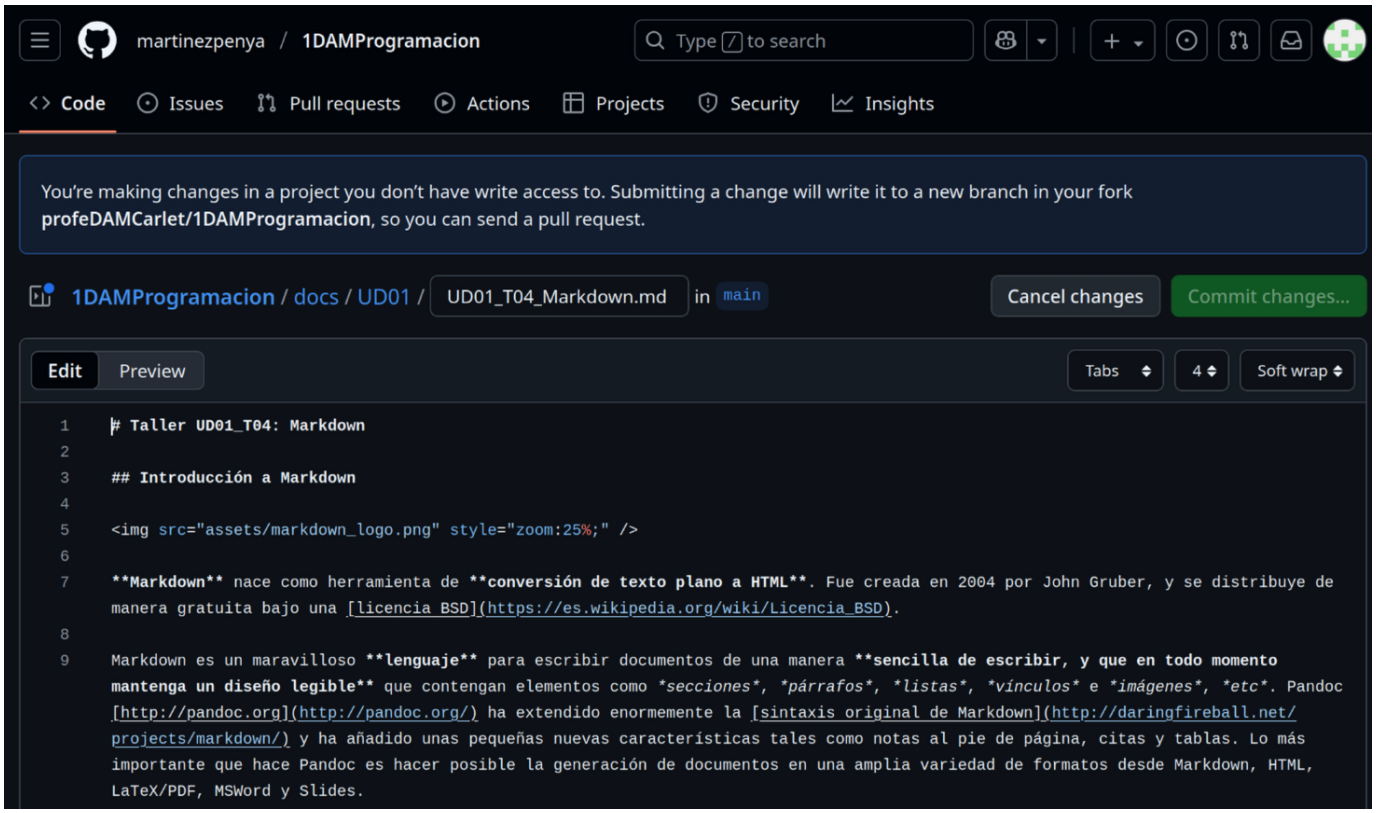
Accedemos a la página de los apuntes en la que hemos detectado el error o queremos sugerir un cambio y en la parte superior derecha debe aparecer el icono:

Esto nos llevará a crear un Fork del repositorio (este concepto lo aprenderás más adelante en el módulo de Entornos de Desarrollo):

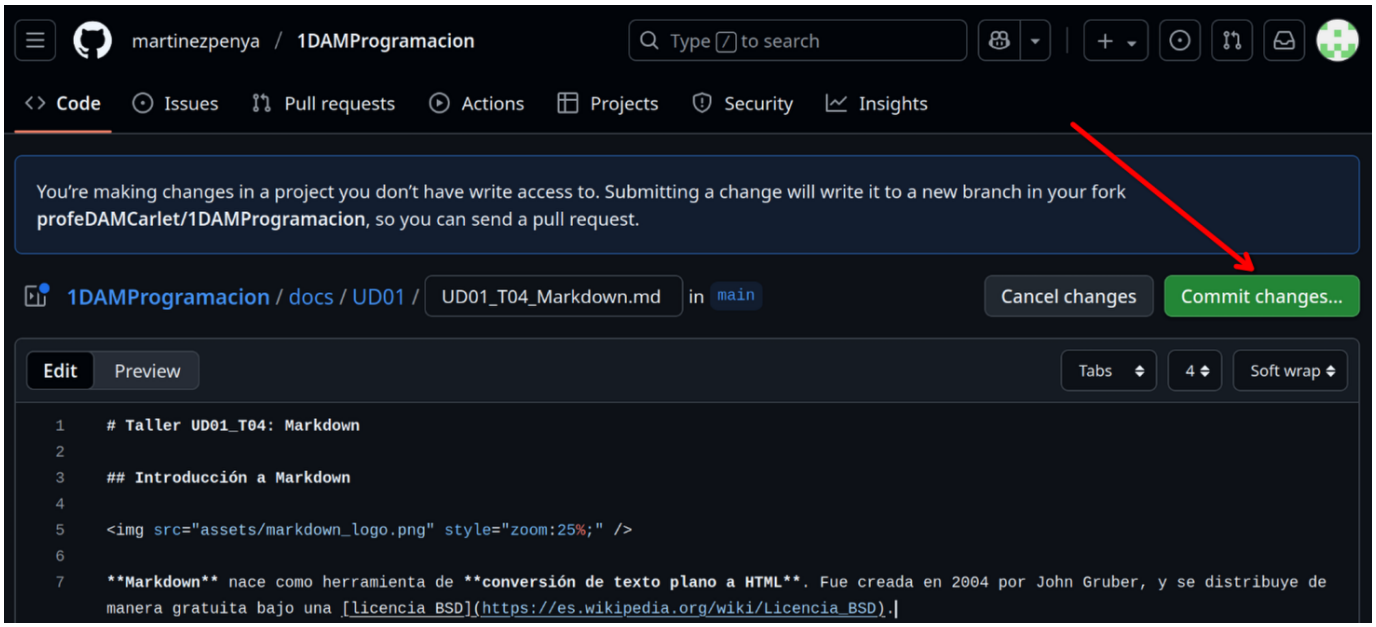


Ahora debemos pulsar el botón **[Fork this repository]**, y a continuación veremos el código de la página en nuestro fork que es `MarkDown` (Puedes aprender más sobre `MarkDown` en el Taller 4):

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

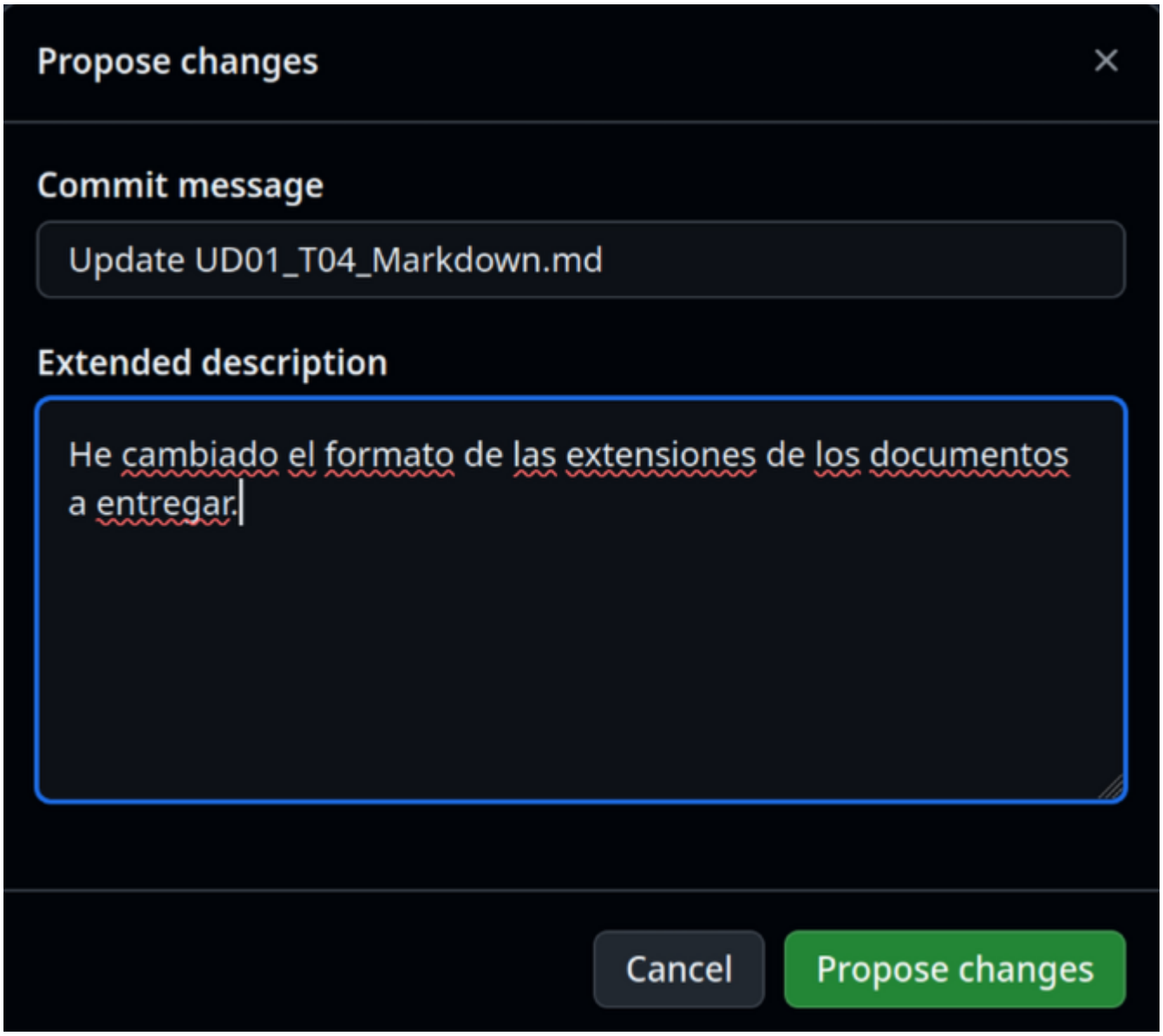


Ahora debemos buscar el texto a modificar y una vez hayamos cambiado algo del documento se activará el botón [Commit changes...]:



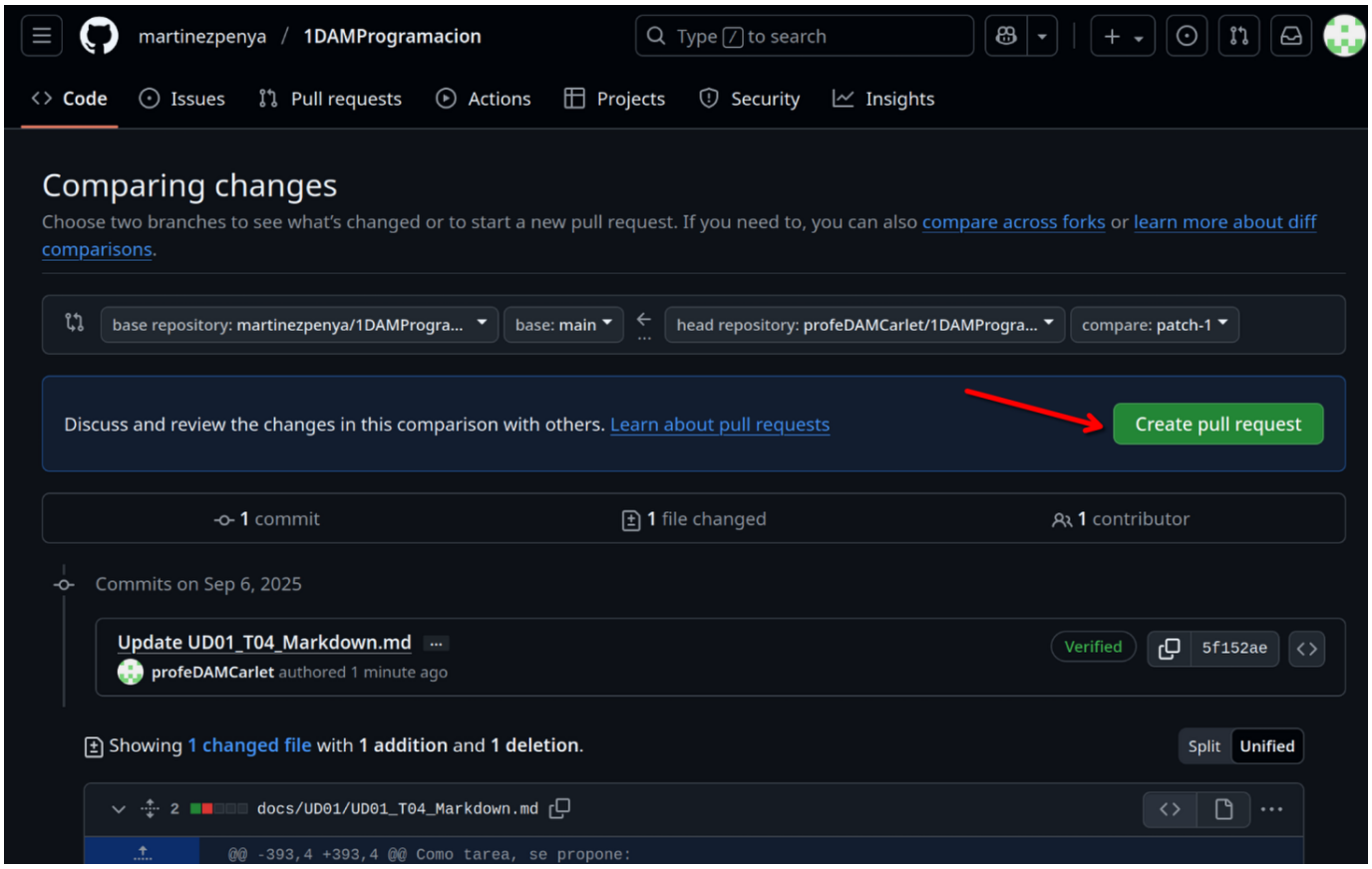
Ahora debes explicar cual ha sido la modificación que hemos realizado y pulsar el botón [Propose changes]:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



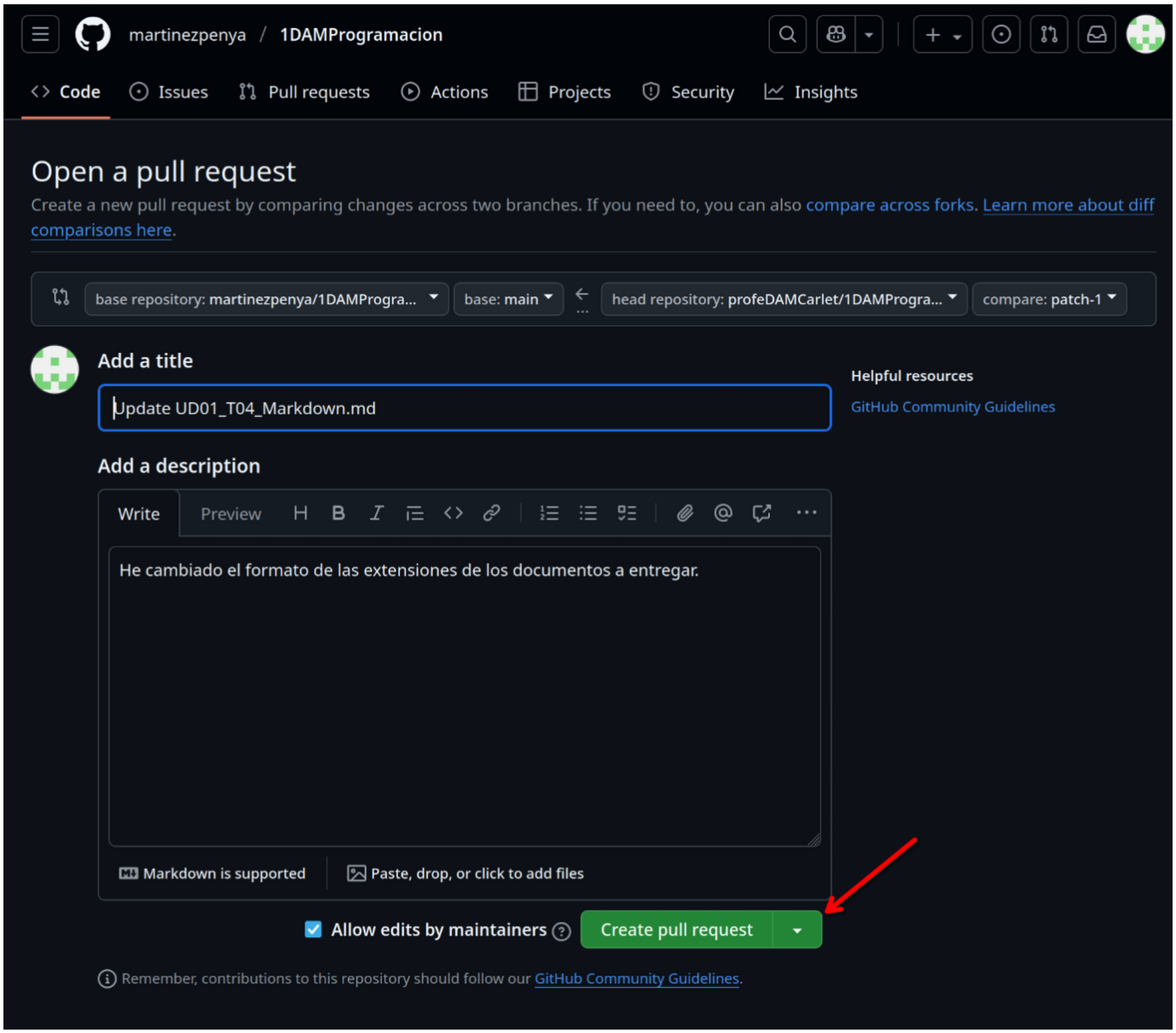
¡Todavía no hemos terminado! ahora hay que comunicar los cambios propuestos en nuestro Fork al propietario del repositorio, para que los visualice y valore si los quiere incluir en la página de documentación. Para ello debemos pulsar el botón [Create pull request]:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



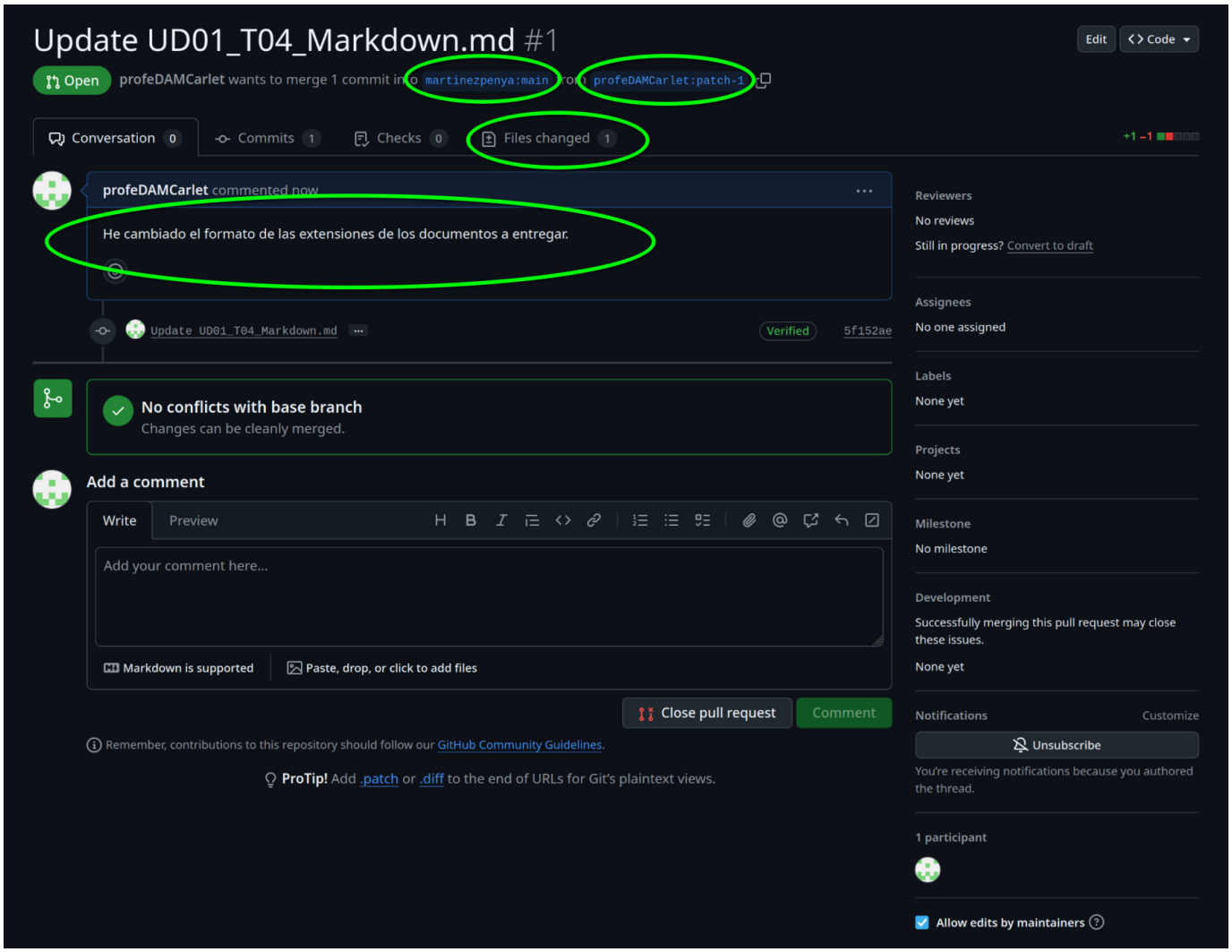
Ahora podemos modificar el mensaje (pero no hace falta), directamente pulsamos sobre el botón [Create pull request]:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Ahora si, deberías ver una página similar a la siguiente, de la que también deberás obtener una captura y adjuntarla al `.pdf`, y además explicar los 4 campos que hay redondeados:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Como resumen:

1. Hemos creado un fork de un repositorio.
2. Hemos modificado un archivo en nuestro fork.
3. Hemos comparado nuestro fork con el original y hemos creado un pull request con las diferencias.

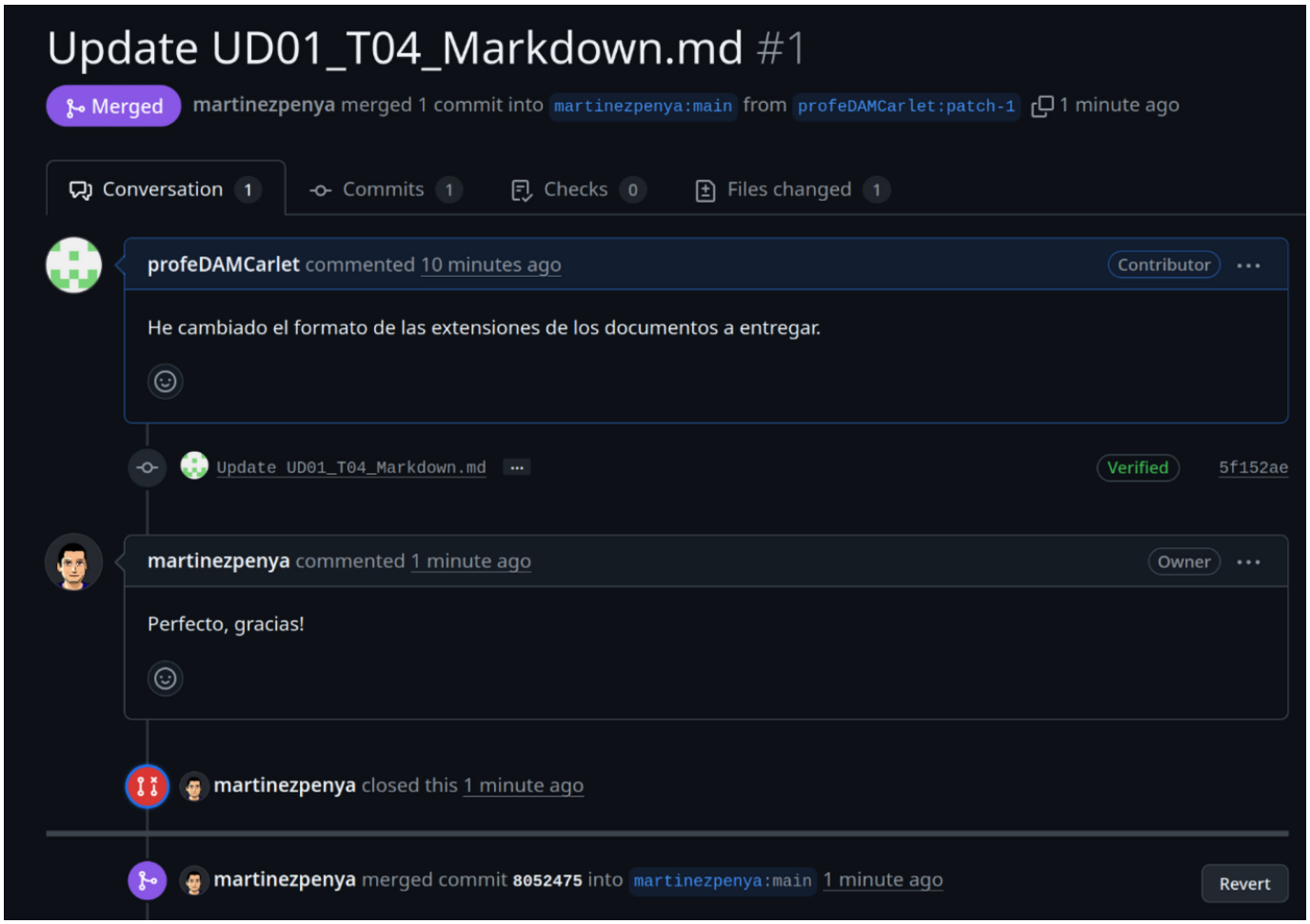
Ahora pueden pasar dos cosas, que el propietario del repositorio original acepte nuestros cambios, y por tanto pasaremos a ser colaboradores del repositorio original.

O bien, que el cambio no sea aceptado.

En cualquiera de los dos casos, si adjuntas las capturas y explicas los campos la actividad estará correcta.

En este caso concreto se ha aceptado la modificación:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Tarea

- Crea un documento `.pdf` donde debes adjuntar la captura de tu perfil de github.
- Añade una **captura** de pantalla donde se vea que has solicitado el **pull request** y que estás esperando a que se integre en el repositorio original.
- Además, **explica** que significan cada uno de los **4 apartados** señalados en la captura.

Adjunta el documento `.pdf` con las capturas y las explicaciones a la tarea de AULES.

2 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

2.3.8 Taller UD01_T04: Markdown

Introducción a Markdown

> **Definición**
>
> **Markdown** es un lenguaje de marcado ligero que permite escribir texto con formato (negritas, cursivas, enlaces, imágenes, código, tablas, etc.) usando texto plano, que luego puede convertirse a HTML, PDF y otros formatos.

Markdown es un maravilloso **lenguaje** para escribir documentos de una manera **sencilla de escribir, y que en todo momento mantenga un diseño legible** que contengan elementos como *secciones*, *párrafos*, *listas*, *vínculos* e *imágenes*, *etc*. Pandoc http://pandoc.org ha extendido enormemente la sintaxis original de Markdown y ha añadido unas pequeñas nuevas características tales como notas al pie de página, citas y tablas. Lo más importante que hace Pandoc es hacer posible la generación de documentos en una amplia variedad de formatos desde Markdown, HTML, LaTeX/PDF, MSWord y Slides.

Este método te permitirá añadir formatos tales como **negritas**, *cursivas* o enlaces, utilizando texto plano, lo que permitirá hacer de tu escritura algo más simple y eficiente al evitar distracciones.

Con Markdown **no vas a reemplazar *todo***, sino cubrir las funcionalidades más comunes que se requieren para escribir un documento relativamente complicado.

Para qué sirve Markdown

Markdown será perfecto para ti sobre todo si publicas de manera constante en Internet, donde el lenguaje HTML está más que presente: WordPress, Squarespace, Jekyll...

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Pero no estoy hablando solo de blogs o páginas web. **Servicios** como Trello o **foros** como Stackoverflow también soportan este lenguaje, y con el paso del tiempo encontrarás aún más lugares que lo utilicen.

Además, Markdown está cada vez más extendido en el **mundo "offline"**. Nada te impedirá utilizar este lenguaje para **tomar notas y apuntes** de tus clases o reuniones en una determinada aplicación (incluso podrías **escribir un libro con él**, ya que puedes exportar fácilmente el resultado final a un formato ePub).

Gracias a la simplicidad de su sintaxis podrás utilizarlo siempre que necesites escribir y dar formato rápidamente, sobre todo si quieres hacerlo desde dispositivos móviles.

Por qué utilizar Markdown

VENTAJAS

- **Markdown para todo**. Para crear apuntes, documentos, notas, sitios web, libros, documentación técnica, etc. de forma off-line.
- **Markdown transportable**. Este tipo de formato siempre será **compatible con todas las plataformas** que utilices, así que utilizar Markdown es una manera de mantener todo tu contenido siempre accesible desde cualquier dispositivo (smartphones, ordenadores de escritorio, tablets...), ya que en cualquiera de ellas siempre encontrarás las aplicaciones adecuadas para leer y editar este tipo de contenido.
- Ideal para escribir un libro, pues permite la exportación fácil en ePub, PDF...

Si en el futuro Microsoft Word desapareciese perderías acceso a todo el contenido que has creado durante años utilizando dicho procesador. Así que lo más inteligente para evitar eso es **generar tu contenido de la manera más sencilla posible**: utilizando texto plano.

DESVENTAJAS

- No tiene muchas funcionalidades (esto es lo que lo hace muy compatible).
- Al no tener todas las opciones de un procesador de textos a veces tendrás que combinar Markdown con HTML para lograr ciertos formatos.

Editores para Markdown

OFF-LINE

- **Typora**
- MarkdownPad
- HarooPad
- Markdown Monster
- ...

ONLINE

- Dillinger
- GitHub
- StackEdit
- ...

Párrafos y saltos de línea

Si queremos generar un nuevo párrafo en Markdown simplemente separa el texto mediante una línea en blanco (**pulsando dos veces intro**).

> **Importante**
>
> Markdown ignora las líneas en blanco múltiples: dos o más líneas en blanco consecutivas se reducen a una sola al procesarse.

Para realizar un salto de línea y empezar **una frase en una línea siguiente dentro del mismo párrafo**, tendrás que pulsar **dos veces la barra espaciadora antes de pulsar una vez intro**.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Por ejemplo si quisieses escribir un poema quedaría tal que así:

«*La tierra estaba seca, No había ríos ni fuentes. Y brotó de tus ojos.*

Donde cada verso tiene **dos espacios en blanco al final**.

Encabezados

Las `#` **almohadillas** son uno de los métodos utilizados en Markdown para crear encabezados. Debes usarlos añadiendo **uno por cada nivel**.

Es decir,

```
# Encabezado 1
## Encabezado 2
### Encabezado 3
#### Encabezado 4
##### Encabezado 5
###### Encabezado 6
```

Se corresponde con:

1. Encapçalament 1

1.1. Encapçalament 2

1.1.1. Encapçalament 3

1.1.1.1. Encapçalament 4

1.1.1.1.0.1. Encapçalament 5

1.1.1.1.0.1. Encapçalament 6

También puedes cerrar los encabezados con el mismo número de almohadillas, por ejemplo escribiendo `### Encabezado 3 ###`. Pero la única finalidad de esto es un **motivo estético**.

Texto básico

Un párrafo no requiere sintaxis especial.

Para aplicar **negrita** al texto, se escribe entre dos asteriscos.

Para aplicar *cursiva* al texto, se escribe entre un solo asterisco.

Para tachar el texto, se escribirá dos virgulillas antes y dos después de éste.

```
Este texto es en **negrita**.
Este texto es en *itálica*.
Este texto está ~~tachado~~.
Este texto es en ambos ***negrita e itàlica***.
```

Se corresponde a:

Este texto es en **negrita**.

Este texto es en *itálica*.

Este texto está ~~tachado~~.

Este texto es en ambos ***negrita e itàlica***.

> **Atención**
>
> Markdown no soporta subrayado de forma nativa. Para subrayar texto debes usar HTML: `<u>texto subrayado</u>`.

Para **ignorar los caracteres** de formato de Markdown, ponga `\` antes del carácter:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Citas

Las citas se generar utilizando el carácter *mayor que* `>` al comienzo del bloque de texto.

```
> No hay que ir para atrás ni para darse impulso.  — Lao Tsé.
```

> No hay que ir para atrás ni para darse impulso. — Lao Tsé.

Si la cita en cuestión se compone de **varios párrafos**, deberás añadir el mismo símbolo `>` al comienzo de cada uno de ellos.

Listas

LISTAS ORDENADAS

Para crear **listas numeradas**, empieza una línea con `1.` or `1)`.

> **Truco**
>
> En listas ordenadas, puedes poner `1.` en todos los elementos y Markdown los numerará automáticamente de forma correcta. Esto facilita reordenar elementos sin tener que renumerar.

```
1. Ítem 1 de la lista.
1. Siguiente ítem de la lista.
1. Siguiente ítem, el tercero, de la lista.
```

Se corresponde con:

1. Ítem 1 de la lista.
2. Siguiente ítem de la lista.
3. Siguiente ítem, el tercero, de la lista.

LISTAS NO ORDENADAS

Para crear listas no numeradas, o de viñetas, empieza una línea con `*`, `-` o `+`, pero no mezcles los formatos dentro de la misma lista. (No mezclar formatos de viñetas, como * y + por ejemplo, dentro del mismo documento).

```
* Ítem 1 de la lista.
* Siguiente ítem de la lista.
* Siguiente ítem, el tercero, de la lista.
```

Se corresponde con:

- Ítem 1 de la lista.
- Siguiente ítem de la lista.
- Siguiente ítem, el tercero, de la lista.

También podremos combinar ambos tipos de listas. Como por ejemplo:

1. element de llista 2 - element de llista 2.2
    - element de llista 2.2.1
    - element de llista 2.2.2

LISTAS DE TAREAS

Para crear listas de tareas basta con que empiece la línea con `- [ ]`, si queremos que no esté el check marcado, y `- [x]`, si queremos que esté el check marcado.

```
- [x] regar plantas.
- [ ] realizar ejercicios de programación.
```

Se corresponde con:

- [x] regar plantas.
- [ ] realizar ejercicios de programación.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Tablas

Las tablas no forman parte de la especificación principal de Markdown, pero Adobe, en cierta forma, las admite.

Para generar una tabla utiliza la barra vertical `|` para generar filas y columnas.

Si insertamos guiones `---` dentro de una celda crearemos el encabezado de la tabla.

```
| encabezado1 | encabezado2 | encabezado3 |
|--- |--- |--- |
| celda 1.1 | celda 1.2 | celda 1.3 |
| celda 2.1 | celda 2.2 | celda 2.3 |
```

Quedaría:

encabezado1	encabezado2	encabezado3
celda 1.1	celda 1.2	celda 1.3
celda 2.1	celda 2.2	celda 2.3

Si queremos una **celda con más de una línea** de texto podremos insertar `\`
(o **Shift+Intro**) al final de ésta.

Enlaces

Para generar en enlace en Markdown se debe poner un código con dos partes:

- `[texto del enlace]`, que es el texto que se va a a mostrar,
- Y después `(nombrefichero.md)`, que es la URL o el nombre de archivo al que se va a vincular.

```
[link text](file-name.md)
```

Un ejemplo:

[enlace a web del centro](https://iesmre.com)

La visualización del ejemplo anterior:

enlace a web del centro

Imágenes

Para insertar una imagen se debe poner un código con dos partes:

- `![texto alternativo]`, que es el texto que se va a a mostrar si la imagen no pudiera visualizarse,
- Seguido de `(nombrefichero.extension)`, que es el archivo imagen (con su dirección).

```
[texto alternativo](file-name.md)
```

Un ejemplo:

[logo markdown](assets/mardown_logo.png)

La visualización de la imagen anterior:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Código de bloque

Uno de los puntos más útiles de Markdown a la hora de crear un documento con texto específico de informática es que admite la colocación de bloques de código tanto en línea como en un bloque "delimitado" independiente entre frases.

Para ello utilizaremos:

- Dos comillas invertidas `` si queremos escribir código dentro de la misma línea de texto del párrafo.
- Si queremos crear un bloque de código multilínea, con un lenguaje específico, pondremos ``` seguido del `nombre del lenguaje del bloque`.

Unos ejemplos:

- En la misma línea:

...estamos escribiendo un párrafo ``insertar el bloque`` y seguimos escribiendo...

- Un bloque de código:

```javascript y escribimos el código.

```
function holamundo(){
    console.log ("hola mundo web");
}
```

#### Línia horizontal

Para crear una línea horizontal, de separación de contenido por ejemplo, se añaden tres guiones: `---`


© 2025 David Martínez licensed under CC BY-NC-SA 4.0
```

Visualización:

Insertar emojis

Para insertar emojis basta con utilizar `:` seguido del nombre del emojii y cerrar con otro `:`.

Podemos observar, que en algunos editores markdown, al escribir, por ejemplo, **:a** nos muestra todos los emojiis con la inicial **a**.

Por ejemplo: `: star :`

Visualización: ⭐

Crear diagrama de flujo

Cuando queremos crear documentos con elementos gráficos como diagramas de flujo, debemos generar una especie de *código* para construirlos.

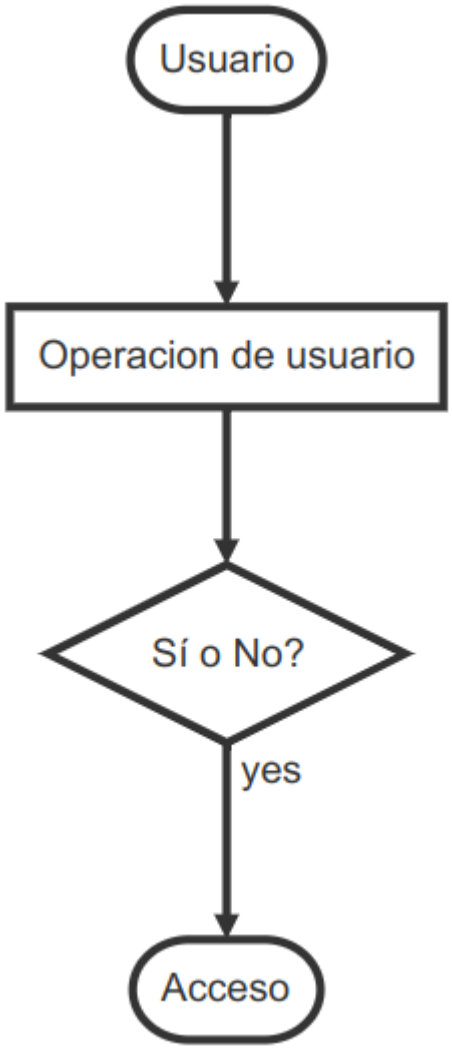
- Por eso, comenzaremos introduciendo la línea de inicio: ```` ```**flow** ````
- Es conveniente asignar un nombre (por ejemplo: st, op, cond, e...) a cada elemento que conforma el diagrama; así, después podremos unir todos estos.
- Forma de inicio: `st=>**start**: Nombre`
- Forma de fin: `e=>**end**: Nombre`
- Rectángulo: `op=>**operation**: texto de nombre`
- Condición: `cond=>**condition**: texto de la condición (Si o No?)`
- Subrutina: `sub1=>**subroutine**: nombre subtarea`
- EntradaSalida: `io1=>**inputoutput**: nombre elemento entrada/salida`
- Líneas: `st->op->cond`
- Caminos de condiciones: `cond(yes)**->**e` y `cond(no)->op`
- Línea de cierre: ```` ``` ````

Ejemplo:

````
```flow
st=>start: Usuario
e=>end: Acceso
op=>operation: Operacion de usuario
cond=>condition: Sí o No?
st->op->cond
cond(yes)->e
cond(no)->op
```
````

Visualización:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Actividad

Intenta realizar un diagrama para "programar" un almuerzo. En él, deberás dar los ***buenos días***, indicar que ***es hora del descanso***, y preguntar si ***alguién quiere almorzar***. Si no hay nadie que quiera almorzar contigo, debes **ir a otro grupo de amigos** y volver a indicar que *es hora del descanso.* Si alguien sí quiere almorzar **escribe en la pizarra que os vais a almorzar** y **sal al patio**.

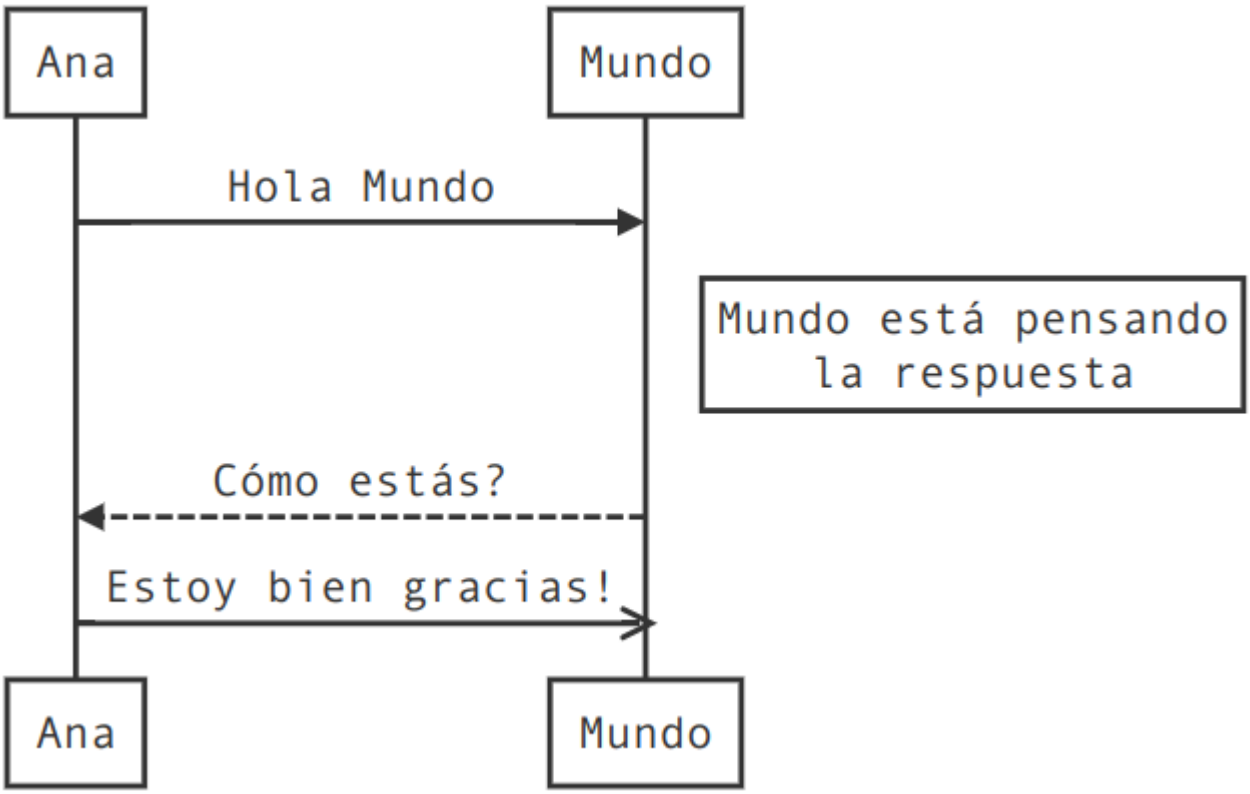
Crear secuencias

En la secuenciación podemos observar que es bastante parecido a la creación de diagramas; pero la primera línea (crear un bloque de código) no serà ~~**flow**~~ sino **sequence**.

```
```sequence
Ana->Mundo: Hola Mundo
Note right of Mundo: Mundo está pensando\nla respuesta
Mundo-->Ana: Cómo estás?
Ana->>Mundo: Estoy bien gracias!
```
```

Visualización:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Crear índice

Para crear el índice a partir de los encabezados creados debemos insertar `[TOC]` .

Resumen

> **Resumen**
>
> Markdown te permite escribir documentos con formato usando texto plano. Sus elementos principales son: encabezados (`#`), énfasis (`**negrita**` , `*cursiva*`), listas, enlaces, imágenes, bloques de código, tablas, citas (`>`) y diagramas. Es ideal para documentación técnica, apuntes y páginas web estáticas.

Tarea

Como tarea, se propone:

- Crear un documento markdown en tu editor markdown favorito (por ejemplo Typora o VSCode) que documente información acerca de tí mismo.
- En dicho documento crear título, índice.
- Añadir 4 encabezados principales (y otros encabezados secundarios dentro de éstos) en el que hables por ejemplo de: *Tus datos*, *Currículum*, *Aficiones* y *Otros datos de interés*. No hace falta que indiques información personal relevante. (O te la puedes inventar)
- Se valorará la inclusión de distintos elementos como: negrita-cursiva-subrayado, listas ordenadas-desordenadas-tareas, enlaces, imágenes, citas, código, etc.
- Si te atreves con ello, crea un diagrama de flujo en el que indiques los pasos que realizas un sábado por la mañana.
- Exporta el documento a pdf.

Subir a la plataforma *AULES* un documento Markdown (*.md) y otro documento PDF (*.pdf) que sea la exportación del primero.

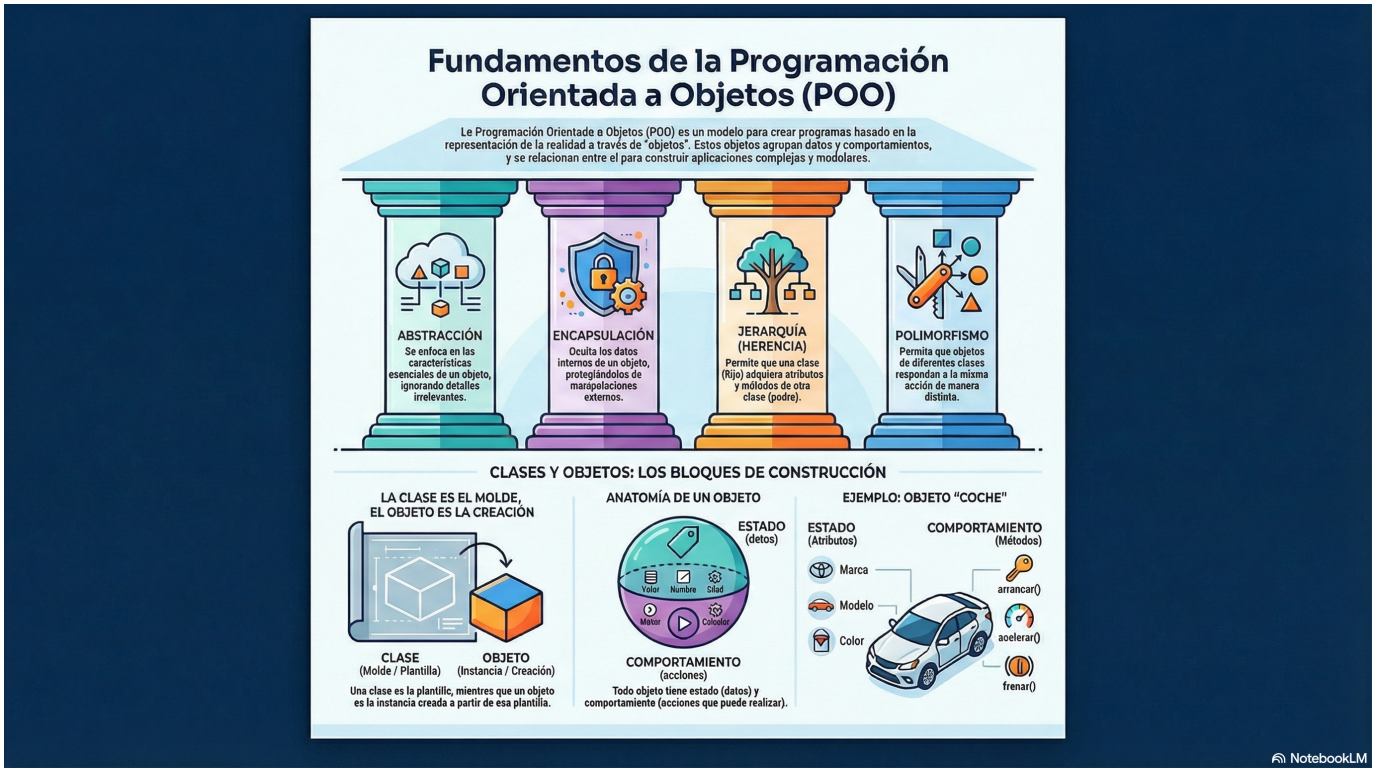
© 2025 David Martínez licensed under CC BY-NC-SA 4.0

8 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

3. UD02

3.1 Utilización de Objetos y Clases



Al finalizar esta unidad serás capaz de...

- Comprender los fundamentos de la Programación Orientada a Objetos
- Identificar las características de los objetos: identidad, estado y comportamiento
- Definir clases con atributos y métodos
- Instanciar objetos a partir de clases predefinidas
- Utilizar métodos y propiedades de los objetos
- Diferenciar entre parámetros y argumentos
- Conocer el uso de constructores

3.1.1 Introducción a la POO

Orientado a objetos hace referencia a una forma diferente de acometer la tarea del desarrollo de software, frente a otros modelos como el de la programación imperativa, la programación funcional o la programación lógica. Supone una reconsideración de los métodos de programación, de la forma de estructurar la información y, ante todo, de la forma de pensar en la resolución de problemas.

La **programación orientada a objetos (POO)** es un modelo para la elaboración de programas que ha impuesto en los últimos años. Este auge se debe, en parte, a que esta forma de programar está fuertemente basada en la representación de la realidad; pero también a que refuerza el uso de buenos criterios aplicables al desarrollo de programas.

Atención

La orientación a objetos no es un tipo de lenguaje de programación. Es una metodología de trabajo para crear programas.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

En POO, un programa es una colección de objetos que se relacionan entre sí de distintas formas.

3.1.2 Características de la POO

Cuando hablamos de Programación Orientada a Objetos, existen una serie de características que se deben cumplir. Cualquier lenguaje de programación orientado a objetos las debe contemplar. Las características más importantes del paradigma de la programación orientada a objetos son:

- **Abstracción**. Es el proceso por el cual definimos las características más importantes de un objeto, sin preocuparnos de cómo se escribirán en el código del programa, simplemente lo definimos de forma general. En la Programación Orientada a Objetos la herramienta más importante para soportar la abstracción es la clase. Básicamente, una clase es un tipo de dato que agrupa las características comunes de un conjunto de objetos. Poder ver los objetos del mundo real que deseamos trasladar a nuestros programas, en términos abstractos, resulta de gran utilidad para un buen diseño del software, ya que nos ayuda a comprender mejor el problema y a tener una visión global de todo el conjunto. Por ejemplo, si pensamos en una clase Vehículo que agrupa las características comunes de todos ellos, a partir de dicha clase podríamos crear objetos como Coche y Camión. Entonces se dice que Vehículo es una abstracción de Coche y de Camión.
- **Modularidad**. Una vez que hemos representado el escenario del problema en nuestra aplicación, tenemos como resultado un conjunto de objetos software a utilizar. Este conjunto de objetos se crean a partir de una o varias clases. Cada clase se encuentra en un archivo diferente, por lo que la modularidad nos permite modificar las características de la clase que define un objeto, sin que esto afecte al resto de clases de la aplicación.
- **Encapsulación**. También llamada "ocultamiento de la información". La encapsulación o encapsulamiento es el mecanismo básico para ocultar la información de las partes internas de un objeto a los demás objetos de la aplicación. Con la encapsulación un objeto puede ocultar la información que contiene al mundo exterior, o bien restringir el acceso a la misma para evitar ser manipulado de forma inadecuada. Por ejemplo, pensemos en un programa con dos objetos, un objeto Persona y otro Coche. Persona se comunica con el objeto Coche para llegar a su destino, utilizando para ello las acciones que Coche tenga definidas como por ejemplo conducir. Es decir, Persona utiliza Coche pero no sabe cómo funciona internamente, sólo sabe utilizar sus métodos o acciones.
- **Jerarquía**. Mediante esta propiedad podemos definir relaciones de jerarquías entre clases y objetos. Las dos jerarquías más importantes son la jerarquía "es un" llamada generalización o especialización y la jerarquía "es parte de", llamada agregación. Conviene detallar algunos aspectos:
- La generalización o especialización, también conocida como herencia, permite crear una clase nueva en términos de una clase ya existente (herencia simple) o de varias clases ya existentes (herencia múltiple). Por ejemplo, podemos crear la clase CochedeCarreras a partir de la clase Coche, y así sólo tendremos que definir las nuevas características que tenga.
- La agregación, también conocida como inclusión, permite agrupar objetos relacionados entre sí dentro de una clase. Así, un Coche está formado por Motor, Ruedas, Frenos y Ventanas. Se dice que Coche es una agregación y Motor, Ruedas, Frenos y Ventanas son agregados de Coche.
- **Polimorfismo**. Esta propiedad indica la capacidad de que varias clases creadas a partir de una antecesora realicen una misma acción de forma diferente. Por ejemplo, pensemos en la clase `Animal` y la acción de expresarse. Nos encontramos que cada tipo de `Animal` puede hacerlo de manera distinta, los `Perros` ladran, los `Gatos` maullan, las `Personas` hablamos, etc. Dicho de otra manera, el polimorfismo indica la posibilidad de tomar un objeto (de tipo `Animal`, por ejemplo), e indicarle que realice la acción de expresarse, esta acción será diferente según el tipo de mamífero del que se trate.

3.1.3 Objetos y Clases

Características de los objetos

En este contexto, un objeto de software es una representación de un objeto del mundo real, compuesto de una serie de características y un comportamiento específico. Pero ¿qué es más concretamente un objeto en Programación Orientada a Objetos? Veámoslo.

> **Definición**
>
> Un objeto es un conjunto de datos con las operaciones definidas para ellos. Los objetos tienen un estado y un comportamiento.

Por tanto, estudiando los objetos que están presentes en un problema podemos dar con la solución a dicho problema. Los objetos tienen unas características fundamentales que los distinguen:

- **Identidad**. Es la característica que permite diferenciar un objeto de otro. De esta manera, aunque dos objetos sean exactamente iguales en sus atributos, son distintos entre sí. Puede ser una dirección de memoria, el nombre del objeto o cualquier otro elemento que utilice el lenguaje para distinguirlos. Por ejemplo, dos vehículos que hayan salido de la misma cadena de fabricación y sean iguales aparentemente, son distintos porque tienen un código que los identifica.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- **Estado**. El estado de un objeto viene determinado por una serie de parámetros o atributos que lo describen, y los valores de éstos. Por ejemplo, si tenemos un objeto `Coche`, el estado estaría definido por atributos como `Marca`, `Modelo`, `Color`, `Cilindrada`, etc.
- **Comportamiento**. Son las acciones que se pueden realizar sobre el objeto. En otras palabras, son los métodos o procedimientos que realiza el objeto. Siguiendo con el ejemplo del objeto Coche, el el comportamiento serían acciones como: `arrancar()`, `parar()`, `acelerar()`, `frenar()`, etc. Definición de clases.

Una clase java se escribe en un fichero con extensión `.java` que tiene el mismo nombre que la clase. Por ejemplo la clase `Vehículo` se escribiría en el fichero `Vehiculo.java`.

Cuando la clase se compila se obtiene un fichero con el mismo nombre que la clase y extensión `.class`. Ej.: `Vehiculo.class`.

Atención

Los identificadores de clase siguen las mismas reglas que otros identificadores de Java (contienen carácter alfanuméricos y especiales, no pueden comenzar por un dígito, no pueden coincidir con una palabra reservada, etc.). Por convenio los identificadores de las clases comienzan por mayúsculas.

Propiedades y métodos de los objetos

Como acabamos de ver todo objeto tiene un estado y un comportamiento. Concretando un poco más, las partes de un objeto son:

- **Campos, Atributos o Propiedades**: Parte del objeto que almacena los datos. También se les denomina Variables Miembro. Estos datos pueden ser de cualquier tipo primitivo (`boolean`, `char`, `int`, `double`, etc) o ser a su vez otro objeto. Por ejemplo, un objeto de la clase `Coche` puede tener un objeto de la clase `Ruedas` (o más concretamente cuatro).
- **Métodos o Funciones Miembro**: Parte del objeto que lleva a cabo las operaciones sobre los atributos definidos para ese objeto.

La idea principal es que el objeto reúne en una sola entidad los datos y las operaciones, y para acceder a los datos privados del objeto debemos utilizar los métodos que hay definidos para ese objeto.

La única forma de manipular la información del objeto es a través de sus métodos. Es decir, si queremos saber el valor de algún atributo, tenemos que utilizar el método que nos muestre el valor de ese atributo. De esta forma, evitamos que métodos externos puedan alterar los datos del objeto de manera inadecuada. Se dice que los datos y los métodos están encapsulados dentro del objeto.

ATRIBUTOS

Los atributos representan la información que almacenan los objetos de la clase. Los atributos son declaraciones de variables dentro de la clase.

Se sigue la siguiente sintaxis (los corchetes indican opcionalidad):

```
[ámbito] tipo nombreDelAtributo;
[ámbito] tipo nombreDelAtributo1, nombreDelAtributo2, ...;
```

donde ...

- ***ámbito*** permite indicar desde qué clases es accesible el atributo.
- ***tipo*** indica el tipo de dato del atributo.
- ***nombreDelAtributo*** es el identificador del atributo.

MÉTODOS

Los métodos determinan qué puede hacer un objeto de la clase, es decir, su comportamiento.

Los métodos realizan algún tipo de acción o tarea y, en ocasiones, devuelven un resultado.

Para realizar su trabajo puede ser necesario que pasemos al método cierta información. Por ejemplo, cuando llamamos al método `round` de la clase `Math`, para redondear un número real, debemos indicar al método cual es el número que queremos redondear. A esa información que pasamos a los métodos se le llama **parámetros** o **argumentos**.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Ejemplo

Consulta el Ejemplo01 para ver la sintaxis de declaración de métodos.

En la definición de un método se distinguen dos partes

- **La cabecera**, en la cual se indica información relevante sobre el método.
- **El cuerpo**, que contiene las instrucciones mediante las cuales el método realiza su tarea.

Para definirlos, se sigue la siguiente sintaxis (los corchetes indican opcionalidad):

Ejemplo

Consulta el Ejemplo01 para ver la sintaxis de declaración de métodos. donde...

- ***ámbito*** permite indicar desde qué clases es accesible el método.
- ***static***, cuando aparece, indica que el método es estático.
- ***tipoDevuelto*** indica el tipo de dato que devuelve el método. La palabra reservada *void* (que no es ningún tipo de dato), indicaría que el método no devuelve nada.
- ***nombreDelMetodo*** es el identificador del método
- ***parámetros*** es una lista, separada por comas, de los parámetros que recibe el método. De cada parámetro se indica el **tipo** y un **identificador**.

Interacción entre objetos

Dentro de un programa los objetos se comunican llamando unos a otros a sus métodos. Los métodos están dentro de los objetos y describen el comportamiento de un objeto cuando recibe una llamada a uno de sus métodos. En otras palabras, cuando un objeto, `objeto1`, quiere actuar sobre otro, `objeto2`, tiene que ejecutar uno de sus métodos. Entonces se dice que el `objeto2` recibe un mensaje del `objeto1`.

Un mensaje es la acción que realiza un objeto. Un método es la función o procedimiento al que se llama para actuar sobre un objeto.

Los distintos mensajes que puede recibir un objeto o a los que puede responder reciben el nombre de **protocolo** de ese objeto.

El proceso de interacción entre objetos se suele resumir diciendo que se ha "enviado un mensaje" (hecho una petición) a un objeto, y el objeto determina "qué hacer con el mensaje" (ejecuta el código del método). Cuando se ejecuta un programa se producen las siguientes acciones:

- **Creación** de los objetos a medida que se necesitan.
- **Comunicación** entre los objetos mediante el envío de mensajes unos a otros, o el usuario a los objetos.
- **Eliminación** de los objetos cuando no son necesarios para dejar espacio libre en la memoria del computador.

Recuerda

Los objetos se pueden comunicar entre ellos invocando a los métodos de los otros objetos.

Clases

Hasta ahora hemos visto lo que son los objetos. Un programa informático se compone de muchos objetos, algunos de los cuales comparten la misma estructura y comportamiento. Si tuviéramos que definir su estructura y comportamiento del objeto cada vez que queremos crear un objeto, estaríamos utilizando mucho código redundante. Por ello lo que se hace es crear una clase, que es una descripción de un conjunto de objetos que comparten una estructura y un comportamiento común. Y a partir de la clase, se crean tantas "copias" o "instancias" como necesitemos. Esas copias son los objetos de la clase.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Recuerda

Las clases constan de datos y métodos que resumen las características comunes de un conjunto de objetos. Un programa informático está compuesto por un conjunto de clases, a partir de las cuales se crean objetos que interactúan entre sí.

En otras palabras, una clase es una plantilla o prototipo donde se especifican:

- Los **atributos** comunes a todos los objetos de la clase.
- Los **métodos** que pueden utilizarse para manejar esos objetos.

Para declarar una clase en Java se utiliza la palabra reservada `class`. La declaración de una clase está compuesta por:

- **Cabecera de la clase**. La cabecera es un poco más compleja que como aquí definimos, pero por ahora sólo nos interesa saber que está compuesta por una serie de modificadores, en este caso hemos puesto public que indica que es una clase pública a la que pueden acceder otras clases del programa, la palabra reservada class y el nombre de la clase.
- **Cuerpo de la clase**. En él se especifican encerrados entre llaves los atributos y los métodos que va a tener la clase.

Ejemplo

Consulta el Ejemplo02 para ver la estructura básica de una clase Java.

Ejemplo

En la unidad anterior ya hemos utilizado clases, aunque aún no sabíamos su significado exacto. Por ejemplo, en los ejemplos de la unidad o en la tarea, estábamos utilizando clases, todas ellas eran clases principales, no tenían ningún atributo y el único método del que disponían era el método `main()`.

Ejemplo

También es una clase `Math` y su método era `random()`, el que nos permitía usar números aleatorios.

Ejemplo

El método `main()` se utiliza para indicar que se trata de una clase principal, a partir de la cual va a empezar la ejecución del programa. Este método no aparece si la clase que estamos creando no va a ser la clase principal del programa.

¿QUÉ SIGNIFICA `public class`?

Significa que la clase que se define es pública. Una clase pública es una clase accesible desde otras clases o, dicho de otra forma, que puede ser utilizada por otras clases. Ya hemos dicho que un programa, de alguna manera, consiste en la creación de objetos de distintas clases, que se relacionan entre sí. Lo más común es que las clases que definimos sean públicas y que en cada fichero de extensión `.java` se defina una única clase.

Sin embargo, en ocasiones se definen clases (`A`) que solo van a ser utilizadas por una clase determinada (`B`). En ese caso, decimos que la clase `A` es una clase privada de la clase `B`. Las clases `A` y `B` se definen en el mismo fichero `.java`.

Atención

En un fichero pueden definirse varias clases pero solo una de ellas puede ser pública. De esta forma, si en un fichero se definen varias clases, una de ellas sería pública y el resto serían clases privadas de la primera, a las que solo ésta tendría acceso.

3.1.4 Utilización de Objetos

Una vez que hemos creado una clase, podemos crear objetos en nuestro programa a partir de esas clases.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Cuando creamos un objeto a partir de una clase se dice que hemos creado una "instancia de la clase". A efectos prácticos, "objeto" e "instancia de clase" son términos similares. Es decir, nos referimos a objetos como instancias cuando queremos hacer hincapié que son de una clase particular.

Los objetos se crean a partir de las clases, y representan casos individuales de éstas.

Ejemplo

Para entender mejor el concepto entre un objeto y su clase, piensa en un molde de galletas y las galletas. El molde sería la clase, que define las características del objeto, por ejemplo su forma y tamaño. Las galletas creadas a partir de ese molde son los objetos o instancias.

Otro ejemplo, imagina una clase Persona que reúna las características comunes de las personas (`color de pelo`, `ojos`, `peso`, `altura`, etc.) y las acciones que pueden realizar (`crecer`, `dormir`, `comer`, etc.). Posteriormente dentro del programa podremos crear un objeto `Trabajador` que esté basado en esa clase Persona. Entonces se dice que el objeto `Trabajador` es una instancia de la clase `Persona`, o que la clase `Persona` es una abstracción del objeto `Trabajador`.

Cualquier objeto instanciado de una clase contiene una copia de todos los atributos definidos en la clase. En otras palabras, lo que estamos haciendo es reservar un espacio en la memoria del ordenador para guardar sus atributos y métodos. Por tanto, cada objeto tiene una zona de almacenamiento propia donde se guarda toda su información, que será distinta a la de cualquier otro objeto. A las variables miembro instanciadas también se les llama variables instancia. De igual forma, a los métodos que manipulan esas variables se les llama métodos instancia.

En el ejemplo del objeto Trabajador, las variables instancia serían `color_de_pelo`, `peso`, `altura`, etc. Y los métodos instancia serían `crecer()`, `dormir()`, `comer()`, etc.

Ciclo de vida de los objetos

Todo programa en Java parte de una única clase, que como hemos comentado se trata de la clase principal.

Esta clase ejecutará el contenido de su método `main()`, el cual será el que utilice las demás clases del programa, cree objetos y lance mensajes a otros objetos.

Las instancias u objetos tienen un tiempo de vida determinado. Cuando un objeto no se va a utilizar más en el programa, es destruido por el recolector de basura para liberar recursos que pueden ser reutilizados por otros objetos.

A la vista de lo anterior, podemos concluir que los objetos tienen un ciclo de vida, en el cual podemos distinguir las siguientes fases:

- **Creación**, donde se hace la reserva de memoria e inicialización de atributos.
- **Manipulación**, que se lleva a cabo cuando se hace uso de los atributos y métodos del objeto.
- **Destrucción**, eliminación del objeto y liberación de recursos.

Declaración

Para la creación de un objeto hay que seguir los siguientes pasos:

- **Declaración**: Definir el tipo de objeto.
- **Instanciación**: Creación del objeto utilizando el operador `new`. Pero ¿en qué consisten estos pasos a nivel de programación en Java? Veamos primero cómo declarar un objeto. Para la definición del tipo de objeto debemos emplear la siguiente instrucción:

```
1  <tipo> nombre_objeto;
```

Donde:

- **tipo** es la case a partir de la cual se va a crear el objeto, y
- **nombre_objeto** es el nombre de la variable referencia con la cual nos referiremos al objeto.

Los tipos referenciados o referencias se utilizan para guardar la dirección de los datos en la memoria del ordenador.

Nada más crear una referencia, ésta se encuentra vacía. Cuando una referencia a un objeto no contiene ninguna instancia se dice que es una referencia nula, es decir, que contiene el valor `null`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Esto quiere decir que la referencia está creada pero que el objeto no está instanciado todavía, por eso la referencia apunta a un objeto inexistente llamado "nulo".

Para entender mejor la declaración de objetos veamos un ejemplo. Cuando veíamos los tipos de datos, decíamos que Java proporciona un tipo de dato especial para los textos o cadenas de caracteres que era el tipo de dato `String`. Veíamos que realmente este tipo de dato es un tipo referenciado y creábamos una variable mensaje de ese tipo de dato de la siguiente forma:

```
String mensaje;
```

Los nombres de la clase empiezan con mayúscula, como `String`, y los nombres de los objetos con minúscula, como `mensaje`, así sabemos qué tipo de elemento utilizando.

Pues bien, `String` es realmente la clase a partir de la cual creamos nuestro objeto llamado mensaje(🤯).

Si observas, poco se diferencia esta declaración de las declaraciones de variables que hacíamos para los tipos primitivos. Antes decíamos que `mensaje` era una variable del tipo de dato `String`. Ahora realmente vemos que `mensaje` es un objeto de la clase `String`. Pero mensaje aún no contiene el objeto porque no ha sido instanciado, veamos cómo hacerlo.

Por tanto, cuando creamos un objeto estamos haciendo uso de una variable que almacena la dirección de ese objeto en memoria. Esa variable es una referencia o un tipo de datos referenciado, porque no contiene el dato si no la posición del dato en la memoria del ordenador.

```
String saludo = new String("Bienvenido a Java");
String s; //s vale null
s = saludo; //asignación de referencias
```

En las instrucciones anteriores, las variables `s` y `saludo` apuntan al mismo objeto de la clase `String`. Esto implica que cualquier modificación en el objeto saludo modifica también el objeto al que hace referencia la variable `s`, ya que realmente son el mismo.

Instanciación

Una vez creada la referencia al objeto, debemos crear la instancia u objeto que se va a guardar en esa referencia. Para ello utilizamos la orden `new` con la siguiente sintaxis:

```
[<tipo>] nombre_objeto = new <Constructor_de_la_Clase>([<par1>, <par2>, ..., <parN>]);
```

Donde:

- **nombre_objeto** es el nombre de la variable referencia con la cual nos referiremos al objeto.
- **new** es el operador para crear el objeto.
- **Constructor_de_la_Clase** es un método especial de la clase, que se llama igual que ella, y se encarga de inicializar el objeto, es decir, de dar unos valores iniciales a sus atributos.
- **par1-parN**, son parámetros que puede o no necesitar el constructor para dar los valores iniciales a los atributos del objeto.

Durante la instanciación del objeto, se reserva memoria suficiente para el objeto. De esta tarea se encarga Java y juega un papel muy importante el `recolector de basura`, que se encarga de eliminar de la memoria los objetos no utilizados para que ésta pueda volver a ser utilizada.

De este modo, para instanciar un objeto `String`, haríamos lo siguiente:

```
mensaje = new String;
```

Así estaríamos instanciando el objeto mensaje. Para ello utilizaríamos el operador `new` y el constructor de la clase `String` a la que pertenece el objeto según la declaración que hemos hecho en el apartado anterior. A continuación utilizamos el constructor, que se llama igual que la clase, `String`.

En el ejemplo anterior el objeto se crearía con la cadena vacía (`""`), si queremos que tenga un contenido debemos utilizar parámetros en el constructor, así:

```
mensaje = new String ("El primer programa");
```

Java permite utilizar la clase `String` como si de un tipo de dato primitivo se tratara, por eso no hace falta utilizar el operador `new` para instanciar un objeto de la clase `String` (pero no es lo habitual en el resto de clases).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
mensaje = "El primer programa";
```

La declaración e instanciación de un objeto puede realizarse en la misma instrucción, así:

```
String mensaje = new String ("El primer programa");
```

o para la clase `String`:

```
String mensaje = "El primer programa";
```

Manipulación

Una vez creado e instanciado el objeto ¿cómo accedemos a su contenido? Para acceder a los atributos y métodos del objeto utilizaremos el nombre del objeto seguido del operador punto (`.`) y el nombre del **atributo** o **método** que queremos utilizar. Cuando utilizamos el operador `punto` se dice que estamos enviando un mensaje al objeto. La forma general de enviar un mensaje a un objeto es:

```
nombre_objeto.mensaje
```

Por ejemplo, para acceder a las variables instancia o atributos se utiliza la siguiente sintaxis:

```
nombre_objeto.atributo
```

Y para acceder a los métodos o funciones miembro del objeto se utiliza la sintaxis es:

```
nombre_objeto.método( [par1, par2, ..., parN] )
```

En la sentencia anterior `par1`, `par2`, etc. son los parámetros que utiliza el método. (*Aparece entre corchetes para indicar son opcionales*).

Para entender mejor cómo se manipulan objetos vamos a utilizar un ejemplo. Para ello necesitamos la Biblioteca de Clases Java o API (Application Programming Interface - Interfaz de programación de aplicaciones). Uno de los paquetes de librerías o bibliotecas es `java.awt`. Este paquete contiene clases destinadas a la creación de objetos gráficos e imágenes. Vemos por ejemplo cómo crear un rectángulo.

En primer lugar, instanciamos el objeto utilizando el método constructor, que se llama igual que el objeto, e indicando los parámetros correspondientes a la posición y a las dimensiones del rectángulo:

```
Rectangle rect = new Rectangle(50, 50, 150, 150);
```

Una vez instanciado el objeto rectángulo si queremos cambiar el valor de los atributos utilizamos el operador punto. Por ejemplo, para cambiar la dimensión del rectángulo:

```
rect.height=100;
rect.width=100;
```

O bien podemos utilizar un método para hacer lo anterior:

```
rect.setSize(200, 200);
```

Ejemplo

Consulta el Ejemplo03 para ver el código completo de manipulación de objetos `Rectangle`.

Destrucción de objetos y liberación de memoria

Cuando un objeto deja de ser utilizado, es necesario liberar el espacio de memoria y otros recursos que poseía para poder ser reutilizados por el programa. A esta acción se le denomina destrucción del objeto.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

En Java la destrucción de objetos corre a cargo del **recolector de basura** (**garbage collector**). Es un sistema de destrucción automática de objetos que ya no son utilizados. Lo que se hace es liberar una zona de memoria que había sido reservada previamente mediante el operador `new`. Esto evita que los programadores tengan que preocuparse de realizar la liberación de memoria.

El recolector de basura se ejecuta en modo segundo plano y de manera muy eficiente para no afectar a la velocidad del programa que se está ejecutando. Lo que hace es que periódicamente va buscando objetos que ya no son referenciados, y cuando encuentra alguno lo marca para ser eliminado.

Después los elimina en el momento que considera oportuno.

Justo antes de que un objeto sea eliminado por el recolector de basura, se ejecuta su método `finalize()`. Si queremos forzar que se ejecute el proceso de finalización de todos los objetos del programa podemos utilizar el método `runFinalization()` de la clase `System`. La clase `System` forma parte de la Biblioteca de Clases de Java y contiene diversas clases para la entrada y salida de información, acceso a variables de entorno del programa y otros métodos de diversa utilidad. Para forzar el proceso de finalización ejecutaríamos:

```
1  System.runFinalization();
```

3.1.5 Utilización de Métodos

Los métodos, junto con los atributos, forman parte de la estructura interna de un objeto. Los métodos contienen la declaración de variables locales y las operaciones que se pueden realizar para el objeto, y que son ejecutadas cuando el método es invocado. Se definen en el cuerpo de la clase y posteriormente son instanciados para convertirse en métodos instancia de un objeto.

Para utilizar los métodos adecuadamente es conveniente conocer la estructura básica de que disponen.

Al igual que las clases, los métodos están compuestos por una cabecera y un cuerpo. La cabecera también tiene modificadores, en este caso hemos utilizado `public` para indicar que el método es público, lo cual quiere decir que le pueden enviar mensajes no sólo los métodos del objeto sino los métodos de cualquier otro objeto externo.

Dentro de un método nos encontramos el cuerpo del método que contiene el código de la acción a realizar. Las acciones que un método puede realizar son:

- **Inicializar** los atributos del objeto
- **Consultar** los valores de los atributos
- **Modificar** los valores de los atributos
- **Llamar a otros métodos**, del mismo del objeto o de objetos externos

Parámetros y valores devueltos

Los métodos se pueden utilizar tanto para consultar información sobre el objeto como para modificar su estado. La información consultada del objeto se devuelve a través de lo que se conoce como valor de retorno, y la modificación del estado del objeto, o sea, de sus atributos, se hace mediante la lista de parámetros. En general, la lista de parámetros de un método se puede declarar de dos formas diferentes:

- **Por valor**. El valor de los parámetros no se devuelve al finalizar el método, es decir, cualquier modificación que se haga en los parámetros no tendrá efecto una vez se salga del método. Esto es así porque cuando se llama al método desde cualquier parte del programa, dicho método recibe una copia de los argumentos, por tanto cualquier modificación que haga será sobre la copia, no sobre las variables originales.
- **Por referencia**. La modificación en los valores de los parámetros sí tienen efecto tras la finalización del método. Cuando pasamos una variable a un método por referencia lo que estamos haciendo es pasar la dirección del dato en memoria, por tanto cualquier cambio en el dato seguirá modificado una vez que salgamos del método.

> **Atención**
>
> En el lenguaje Java, todas las variables se pasan por valor, excepto los objetos que se pasan por referencia.

En Java, la declaración de un método tiene dos restricciones:

- **Un método siempre tiene que devolver un valor (no hay valor por defecto)**. Este valor de retorno es el valor que devuelve el método cuando termina de ejecutarse, al método o programa que lo llamó. Puede ser un tipo primitivo, un tipo referenciado o bien el tipo `void`, que indica que el método no devuelve ningún valor.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- **Un método tiene un número fijo de argumentos**. Los argumentos son variables a través de las cuales se pasa información al método desde el lugar del que se llame, para que éste pueda utilizar dichos valores durante su ejecución. Los argumentos reciben el nombre de parámetros cuando aparecen en la declaración del método.

> **Recuerda**
>
> El valor de retorno es la información que devuelve un método tras su ejecución.

Según hemos visto en el apartado anterior, la cabecera de un método se declara como sigue:

```
public tipo_de_dato_devuelto nombreMetodo (lista_de_parametros);
```

Como vemos, el tipo de dato devuelto aparece después del modificador `public` y se corresponde con el valor de retorno.

La lista de parámetros aparece al final de la cabecera del método, justo después del nombre, encerrados entre signos de paréntesis y separados por comas. Se debe indicar el tipo de dato de cada parámetro así:

```
(tipo_parámetro1 nombre_parámetro1, ..., tipo_parámetroN nombre_parámetroN)
```

> **Atención**
>
> Cuando se llame al método, se deberá utilizar el nombre del método, seguido de los argumentos que deben coincidir con la lista de parámetros.

La lista de argumentos en la llamada a un método debe coincidir en número, tipo y orden con los parámetros del método, ya que de lo contrario se produciría un error de sintaxis.

Constructores

¿Recuerdas cuando hablábamos de la creación e instanciación de un objeto? Decíamos que utilizábamos el operador `new` seguido del nombre de la clase y una pareja de abrir-cerrar paréntesis.

Además, el nombre de la clase era realmente el constructor de la misma, y lo definíamos como un método especial que sirve para inicializar valores. En este apartado vamos a ver un poco más sobre los constructores.

Un constructor es un método especial con el mismo nombre de la clase y que no devuelve ningún valor tras su ejecución.

Cuando creamos un objeto debemos instanciarlo utilizando el constructor de la clase. Veamos la clase `Date` proporcionada por la Biblioteca de Clases de Java. Si queremos instanciar un objeto a partir de la clase `Date` tan sólo tendremos que utilizar el constructor seguido de una pareja de abrir-cerrar paréntesis:

```
Date fecha = new Date();
```

Con la anterior instrucción estamos creando un objeto fecha de tipo `Date`, que contendrá la fecha y hora actual del sistema.

La estructura de los constructores es similar a la de cualquier método, salvo que no tiene tipo de dato devuelto porque no devuelve ningún valor. Está formada por una cabecera y un cuerpo, que contiene la inicialización de atributos y resto de instrucciones del constructor.

El método constructor tiene las siguientes particularidades:

- **El constructor es invocado** automáticamente **en la creación** de un objeto, **y sólo esa vez**.
- **Los constructores no empiezan con minúscula**, como el resto de los métodos, ya que se llaman igual que la clase y las clases empiezan con letra mayúscula.
- **Puede haber varios** constructores para una clase.
- Como cualquier método, **el constructor puede tener parámetros** para definir qué valores dar a los atributos del objeto.
- **El constructor por defecto es aquél que no tiene argumentos o parámetros**. Cuando creamos un objeto llamando al nombre de la clase sin argumentos, estamos utilizando el constructor por defecto.
- **Es necesario que toda clase tenga al menos un constructor.** Si no definimos constructores para una clase, y sólo en ese caso, el compilador crea un constructor por defecto vacío, que inicializa los atributos a sus valores por defecto, según del

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

tipo que sean: `0` para los tipos numéricos, `false` para los `boolean` y `null` para los tipo carácter y las referencias. Dicho constructor lo que hace es llamar al constructor sin argumentos de la superclase (clase de la cual hereda); si la superclase no tiene constructor sin argumentos se produce un error de compilación.

> **Atención**
>
> Cuando definimos constructores personalizados, el constructor por defecto deja de existir, y si no definimos nosotros un constructor sin argumentos cuando intentemos utilizar el constructor por defecto nos dará un error de compilación.

El operador `this`

Los constructores y métodos de un objeto suelen utilizar el operador `this`. Este operador sirve para referirse a los atributos de un objeto cuando estamos dentro de él. Sobre todo se utiliza cuando existe ambigüedad entre el nombre de un parámetro y el nombre de un atributo, entonces en lugar del nombre del atributo solamente escribiremos `this.nombre_atributo`, y así no habrá duda de a qué elemento nos estamos refiriendo.

Métodos estáticos

Cuando trabajábamos con cadenas de caracteres utilizando la clase `String`, veíamos las operaciones que podíamos hacer con ellas: obtener longitud, comparar dos cadenas de caracteres, cambiar a mayúsculas o minúsculas, etc. Pues bien, sin saberlo estábamos utilizando métodos estáticos definidos por Java para la clase `String`. Pero ¿qué son los métodos estáticos? Veámoslo.

Los métodos estáticos son aquellos métodos definidos para una clase que se pueden usar directamente, sin necesidad de crear un objeto de dicha clase. También se llaman métodos de clase.

Para llamar a un método estático utilizaremos:

- **El nombre del método**, si lo llamamos desde la misma clase en la que se encuentra definido.
- **El nombre de la clase**, seguido por el operador punto (`.`) más el nombre del método estático, si lo llamamos desde una clase distinta a la que se encuentra definido:

```
Nombre_clase.nombre_metodo_estatico
```

- **El nombre del objeto**, seguido por el operador punto (`.`) más el nombre del método estático. Utilizaremos esta forma cuando tengamos un objeto instanciado de la clase en la que se encuentra definido el método estático, y no podamos utilizar la anterior:

```
nombre_objeto.nombre_metodo_no_estatico
```

Los métodos estáticos no afectan al estado de los objetos instanciados de la clase (variables instancia), y suelen utilizarse para realizar operaciones comunes a todos los objetos de la clase. Por ejemplo, si necesitamos contar el número de objetos instanciados de una clase, podríamos utilizar un método estático que fuera incrementando el valor de una variable entera de la clase conforme se van creando los objetos.

En la Biblioteca de Clases de Java existen muchas clases que contienen métodos estáticos. Pensemos en las clases que ya hemos utilizado en unidades anteriores, como hemos comentado la clase `String` con todas las operaciones que podíamos hacer con ella y con los objetos instanciados a partir de ella. O bien la clase `Math` para la conversión de tipos de datos. Todos ellos son métodos estáticos que la API de Java define para esas clases. Lo importante es que tengamos en cuenta que al tratarse de métodos estáticos, para utilizarlos no necesitamos crear un objeto de dichas clases.

Fijémonos en esta secuencia de instrucciones

```
//Creamos dos círculos de radio 100  en distintas posiciones
Circulo c1 = new Circulo(50,50,100);
Circulo c2 = new Circulo(80,80,100);
...
//Aumentamos el radio del primer círculo a 200
c1.setRadio(200);
```

y en esta otra

```
System.out.println(Math.sqrt(4));
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

En el primer ejemplo, `.setRadio(200)` va precedido por un objeto. La variable `c1` es un objeto de la clase Círculo, por tanto, la instrucción está modificando el radio de un círculo concreto, el que se encuentra en la posición (50,50) . El método `setRadio` es un método no estático. Los métodos no estáticos actúan siempre sobre algún objeto (el que figura a la izquierda del punto).

En el segundo ejemplo, en cambio, a la izquierda de `.sqrt(4)` no se ha puesto el nombre de un objeto, sino el de una clase, la clase `Math`. El método `sqrt` no está actuando sobre un objeto concreto: no tiene sentido hacerlo, solo pretendemos calcular la raíz cuadrada de `4`. `Sqrt` es un método estático. Los métodos estáticos se usan poniendo delante del punto el nombre de la clase en que se encuentran definidos.

Ejemplo completo: clase Pajaro

Consulta el Ejemplo05 para ver un ejemplo completo de clase con constructores, métodos y el uso de `this`.

3.1.6 Librerías de Objetos (Paquetes)

Conforme nuestros programas se van haciendo más grandes, el número de clases va creciendo. Meter todas las clases en único directorio no ayuda a que estén bien organizadas, lo mejor es hacer grupos de clases, de forma que todas las clases que estén relacionadas o traten sobre un mismo tema estén en el mismo grupo.

Un **paquete** de clases es una agrupación de clases que consideramos que están relacionadas entre sí o tratan de un tema común.

Recuerda

Las clases de un mismo paquete tienen un acceso privilegiado a los atributos y métodos de otras clases de dicho paquete. Es por ello por lo que se considera que los paquetes son también, en cierto modo, unidades de encapsulación y ocultación de información.

Java nos ayuda a organizar las clases en paquetes. En cada fichero `.java` que hagamos, al principio, podemos indicar a qué paquete pertenece la clase que hagamos en ese fichero.

Los paquetes se declaran utilizando la palabra clave `package` seguida del nombre del paquete.

Para establecer el paquete al que pertenece una clase hay que poner una sentencia de declaración como la siguiente al principio de la clase:

```
1  package nombre_de_Paquete;
```

Por ejemplo, si decidimos agrupar en un paquete `ejemplos` un programa llamado `Bienvenida`, pondríamos en nuestro fichero `Bienvenida.java` lo siguiente:

```
1  package ejemplos;
2
3  public class Bienvenida {
4      [...]
5  }
```

El código es exactamente igual que como hemos venido haciendo hasta ahora, solamente hemos añadido la línea `package ejemplos;` al principio.

Sentencia `import`

Cuando queremos utilizar una clase que está en un paquete distinto a la clase que estamos utilizando, se suele utilizar la sentencia `import`. Por ejemplo, si queremos utilizar la clase `Scanner` que está en el paquete `java.util` de la Biblioteca de Clases de Java, tendremos que utilizar esta sentencia:

```
1  import java.util.Scanner;
```

Se pueden importar todas las clases de un paquete, así:

```
1  import java.awt.*;
```

Esta sentencia debe aparecer al principio de la clase, justo después de la sentencia package, si ésta existiese.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

También podemos utilizar la clase sin sentencia `import`, en cuyo caso cada vez que queramos usarla debemos indicar su ruta completa:

```
java.util.Scanner teclado = new java.util.Scanner (System.in);
```

Hasta aquí todo correcto. Sin embargo, al trabajar con paquetes, Java nos obliga a organizar los directorios, compilar y ejecutar de cierta forma para que todo funcione adecuadamente.

Librerías Java

Cuando descargamos el entorno de compilación y ejecución de Java, obtenemos la API de Java. Como ya sabemos, se trata de un conjunto de bibliotecas que nos proporciona paquetes de clases útiles para nuestros programas. Utilizar las clases y métodos de la Biblioteca de Java nos va ayudar a reducir el tiempo de desarrollo considerablemente, por lo que es importante que aprendamos a consultarla y conozcamos las clases más utilizadas.

Ejemplo

```
import java.lang.System; // Se importa la clase System.
import java.awt.*;       // Se importa todas las clases del paquete awt;
```

Los paquetes más importantes que ofrece el lenguaje Java son:

Paquete o librería	Descripción
java.io	Contiene las clases que gestionan la entrada y salida, ya sea para manipular ficheros, leer o escribir en pantalla, en memoria, etc. Este paquete contiene por ejemplo la clase BufferedReader que se utiliza para la entrada por teclado.
java.lang	Contiene las clases básicas del lenguaje. Este paquete no es necesario importarlo, ya que es importado automáticamente por el entorno de ejecución. En este paquete se encuentra la clase Object, que sirve como raíz para la jerarquía de clases de Java, o la clase System que ya hemos utilizado en algunos ejemplos y que representa al sistema en el que se está ejecutando la aplicación. También podemos encontrar en este paquete las clases que "envuelven" los tipos primitivos de datos. Lo que proporciona una serie de métodos para cada tipo de dato de utilidad, como por ejemplo las conversiones de datos.
java.util	Biblioteca de clases de utilidad general para el programador. Este paquete contiene por ejemplo la clase Scanner utilizada para la entrada por teclado de diferentes tipos de datos, la clase Date, para el tratamiento de fechas, etc.
java.math	Contiene herramientas para manipulaciones matemáticas.
java.awt	Incluye las clases relacionadas con la construcción de interfaces de usuario, es decir, las que nos permiten construir ventanas, cajas de texto, botones, etc. Algunas de las clases que podemos encontrar en este paquete son Button, TextField, Frame, Label, etc.
java.swing	Contiene otro conjunto de clases para la construcción de interfaces avanzadas de usuario. Los componentes que se engloban dentro de este paquete se denominan componentes Swing, y suponen una alternativa mucho más potente que AWT para construir interfaces de usuario.
java.net	Conjunto de clases para la programación en la red local e Internet.
java.sql	Contiene las clases necesarias para programar en Java el acceso a las bases de datos.
java.security	Biblioteca de clases para implementar mecanismos de seguridad.

Como se puede comprobar Java ofrece una completa jerarquía de clases organizadas a través de paquetes.

3.1.7 Cadenas de caracteres. La clase `String`

Cadenas de caracteres

Hasta ahora hemos utilizado literales de cadenas de caracteres que, como sabemos, se ponen entre comillas dobles, como en la siguiente expresión

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
System.out.println("Hola");
```

Para almacenar cadenas de caracteres en variables se utiliza la clase `String`. `String` se encuentra definida en el paquete `java.lang`. *Recordemos que no es necesario importar este paquete para utilizar sus clases.*

La forma de `String` es la siguiente:

```
String variable = new String("texto");
```

Ejemplo:

```
String nombre = new String("Javier");
System.out.println("Mi nombre es " + nombre);
```

Sin embargo, debido a que es una clase que se utiliza ampliamente en los programas, Java permite una forma abreviada de crear objetos String:

```
String nombreVariable = "texto";
```

Ejemplo:

```
String nombre = "Javier";
System.out.println("Mi nombre es " + nombre);
```

Leer cadenas desde teclado

CLASE `Scanner`

Para leer cadenas de caracteres desde teclado podemos utilizar la clase `Scanner`. Ésta dispone de dos métodos para leer cadenas:

- `next()`: Lee desde la entrada estándar (teclado) una secuencia de caracteres hasta encontrar un delimitador (un espacio). Devuelve un `String`.
- `nextLine()`: Lee desde la entrada estándar (teclado) una secuencia de caracteres hasta encontrar un salto de línea. Devuelve un `String`.

Ejemplo:

```
Scanner tec = new Scanner(System.in);
//De lo que introduce el usuario, lee la 1º palabra.
String nombre = tec.next();
//Lee lo que introduce el usuario hasta que pulsa intro.
String nombreCompleto = tec.nextLine();
```

EJEMPLOS DE LA UD01 PERO UTILIZANDO `Scanner` (COMPATIBLE CON LOS IDE'S)

Scanner

Consulta el Ejemplo04 para ver el código completo de lectura por teclado con `Scanner`.

La clase `String`

Además de permitir almacenar cadenas de caracteres, `String` tiene métodos para realizar cálculos u operaciones con ellas.

Así por ejemplo, la clase tiene un método `toUpperCase()` que devuelve el `String` convertido a mayúsculas. El siguiente ejemplo ilustra su uso:

```
String nombre = "Javier";
System.out.println(nombre.toUpperCase()); // Se muestra JAVIER por pantalla
```

Accede a la documentación en línea de Java y estudia los siguientes métodos de la clase:

- `charAt`
- `indexOf`

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- `substring`
- `toLowerCase`
- `trim`

`printf` o `format`

El método `printf()` o `format()` (son sinónimos) utilizan unos códigos de conversión para indicar si el contenido a mostrar de qué tipo es. Estos códigos se caracterizan porque llevan delante el símbolo %, algunos de ellos son:

- `%c` : Escribe un carácter.
- `%s` : Escribe una cadena de texto.
- `%d` : Escribe un entero.
- `%f` : Escribe un número en punto flotante.
- `%e` : Escribe un número en punto flotante en notación científica.

Por ejemplo, si queremos escribir el número float `12345.1684` con el punto de los miles y sólo dos cifras decimales la orden sería:

```
System.out.printf("%,.2f\n", 12345.1684);
```

Esta orden mostraría el número `12.345,17` por pantalla.

Otro ejemplo seria:

```
System.out.format("El valor de la variable float es" +
     "%f, mientras que el valor del entero es %d" +
     "y el string contiene %s", variableFloat, variableInt, variableString);
```

Puedes investigar más sobre `printf` o `format` en este enlace

Salida de error

La salida de error está representada por el objeto `System.err`. No parece muy útil utilizar `out` y `err` si su destino es la misma pantalla, o al menos en el caso de la consola del sistema donde las dos salidas son representadas con el mismo color y no notamos diferencia alguna. En cambio en la consola de varios entornos integrados de desarrollo como NetBeans o Eclipse la salida de `err` se ve en un color diferente. Teniendo el siguiente código:

```
System.out.println("Salida estándar por pantalla");
System.err.println("Salida de error por pantalla");
```

Tanto NetBeans, Eclipse como IntelliJ Idea mostraran el mensaje `err` en color rojo.

Ejemplo completo: String, printf y System.err

Consulta el Ejemplo06 para ver un ejemplo completo con métodos de `String`, `printf` / `format` y `System.err`.

3.1.8 Ejemplos UD02

Descarga el código fuente completo de los ejemplos

Ejemplo01

Sintaxis de declaración de un método en Java.

```
public static void main (String[] args)
[ámbito] [static] tipoDevuelto nombreDelMetodo ([parámetros]){
    //Cuerpo del método (instrucciones)
    ...
}
```

⬆ Volver a teoría

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Ejemplo02

Estructura básica de una clase Java.

```java
//Paquete al que pertenece la clase
package NombreDePaquete;

//Paquetes que importa la clase
import ...

public class NombreDeLaClase {
    // Atributos de la clase
    ...
    // Métodos de la clase
    ...
}
```

⬆ Volver a teoría

Ejemplo03

Manipulación de objetos `Rectangle` de la biblioteca `java.awt`.

```java
/*
 * Muestra como se manipulan objetos en Java
 */
import java.awt.Rectangle;
public class Manipular {
    public static void main(String[] args) {
        // Instanciamos el objeto rect indicando posicion y dimensiones
        Rectangle rect = new Rectangle( 50, 50, 150, 150 );
        //Consultamos las coordenadas x e y del rectangulo
        System.out.println( "------- Coordenadas esquina superior izqda. -------");
        System.out.println("\tx = " + rect.x + "\n\ty = " + rect.y);
        // Consultamos las dimensiones (altura y anchura) del rectangulo
        System.out.println( "\n----------------- Dimensiones -------------------");
        System.out.println("\tAlto = " + rect.height );
        System.out.println( "\tAncho = " + rect.width);
        //Cambiar coordenadas del rectangulo
        rect.height=100;
        rect.width=100;
        rect.setSize(200, 200);
        System.out.println( "\n-- Nuevos valores de los atributos --");
        System.out.println("\tx = " + rect.x + "\n\ty = " + rect.y);
        System.out.println("\tAlto = " + rect.height );
        System.out.println( "\tAncho = " + rect.width);
    }
}
```

⬆ Volver a teoría

Ejemplo04

Lectura de datos desde teclado con `Scanner` (compatible con IDEs).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
import java.util.Scanner;

public class EjemploUD02 {

    public static void main(String[] args) {

        Scanner teclado = new Scanner(System.in);

        //Introducir texto desde teclado
        String texto;
        System.out.print("Introduce un texto: ");
        texto = teclado.nextLine();
        System.out.println("El texto introducido es: "+ texto);

        //Introducir un número entero desde teclado
        String texto2;
        int entero2;
        System.out.print("Introduce un número: ");
        texto2 = teclado.nextLine();
        entero2 = Integer.parseInt(texto2);
        System.out.println("El número introducido es:"+entero2);

        //Introducir un número decimal desde teclado
        String texto3;
        double doble3;
        System.out.print("Introduce un número decimal: ");
        texto3 = teclado.nextLine();
        doble3 = Double.parseDouble(texto3); // convertimos texto a doble
        System.out.println("Número decimal introducido es: "+doble3);
    }
}
```

⬆ Volver a teoría

Ejemplo05

Clase `Pajaro` completa con constructores y método `volar()`.

```java
public class Pajaro {
    //atributos/variables
    private String nombre;
    private int posX;
    private int posY;
    //constructores
    public Pajaro() {
    }
    public Pajaro(String nombre) {
        this.nombre = nombre;
    }
    public Pajaro(String nombre, int posX, int posY) {
        this.nombre = nombre;
        this.posX = posX;
        this.posY = posY;
    }

    //metodos
    public double volar(int posX, int posY) {
        double desplazamiento = Math.sqrt((posX * posX) + (posY * posY));
        this.posX = posX;
        this.posY = posY;
        return desplazamiento;
    }

    public static void main(String[] args) {
        //creamos el objeto con parámetros
        Pajaro pajaro1 = new Pajaro("WoodPecker", 50, 50);
        double d1 = pajaro1.volar(50, 50);
        System.out.println("El desplazamiento de " + pajaro1.nombre + " ha sido " + d1);

        Pajaro pajaro2 = new Pajaro();
        //damos nombre y cambiamos la posición de "Piolin" a mano
        pajaro2.nombre="Piolín";
        pajaro2.posX=30;
        pajaro2.posY=30;
        double d2 = pajaro2.volar(pajaro2.posX, pajaro2.posY);
        System.out.println("El desplazamiento de " + pajaro2.nombre + " ha sido " + d2);
    }
}
```

Salida:

```
El desplazamiento de WoodPecker ha sido 70.71067811865476
El desplazamiento de Piolín ha sido 42.42640687119285
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

⬆ Volver a teoría

Ejemplo06

Métodos de la clase `String`, `printf` / `format` y `System.err`.

```java
package UD02;

import java.util.Scanner;

public class EjemploUD02 {

    public static void main(String[] args) {

        Scanner teclado = new Scanner(System.in);

        //Introducir texto desde teclado
        String texto;
        System.out.print("Introduce un texto: ");
        texto = teclado.nextLine();
        System.out.println("El texto introducido es: " + texto);

        //Introducir un número entero desde teclado
        String texto2;
        int entero2;
        System.out.print("Introduce un número: ");
        texto2 = teclado.nextLine();
        entero2 = Integer.parseInt(texto2);
        System.out.println("El número introducido es:" + entero2);

        //Introducir un número decimal desde teclado
        String texto3;
        double doble3;
        System.out.print("Introduce un número decimal: ");
        texto3 = teclado.nextLine();
        doble3 = Double.parseDouble(texto3);
        System.out.println("Número decimal introducido es: " + doble3);

        System.out.println("La clase String");
        String nombre = "Javier "; //Observa que hay un espacio final
        System.out.println(nombre.toUpperCase()); //JAVIER
        System.out.println(nombre.charAt(4)); //e
        System.out.println(nombre.indexOf("i"));//3
        System.out.println(nombre.substring(0, 3));//Javi
        System.out.println(nombre.toLowerCase());//javier
        System.out.println(nombre.trim());//Javier sin espacios finales
        System.out.printf("%,.2f\n", 12345.1684);
        nombre.toUpperCase().substring(0,3).indexOf("I"); //3
        System.out.format("El valor de la variable float es %f"
                + ", mientras que el valor del entero es %d"
                + " y el string contiene %s", doble3, entero2, texto);

        System.err.println("Salida de error por pantalla");
    }
}
```

⬆ Volver a teoría

3.1.9 Resumen — Conceptos clave

Concepto	Definición
POO	Paradigma de programación basado en objetos que representan entidades del mundo real
Clase	Plantilla o modelo que define atributos y métodos comunes a un conjunto de objetos
Objeto	Instancia concreta de una clase, con identidad, estado y comportamiento propios
Atributo	Dato que almacena el estado de un objeto
Método	Operación que define el comportamiento de un objeto
Encapsulación	Mecanismo que oculta los detalles internos de un objeto
Constructor	Método especial que inicializa un objeto al crearlo

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

3.1.10 Autoevaluación

- [ ] Entiendo los conceptos básicos de la POO
- [ ] Sé crear una clase con atributos y métodos
- [ ] Puedo instanciar objetos y llamar a sus métodos
- [ ] Comprendo la diferencia entre parámetros y argumentos
- [ ] Sé utilizar constructores
- [ ] Reconozco la importancia de la encapsulación

3.1.11 Vídeos recomendados

Canal	Vídeo	Contenido
Programación ATS	Playlist completa del curso Java	Vídeos 1–9: introducción, objetos, clases, Scanner
Píldoras Informáticas	Curso Java 2026 — Estructuras principales	Vídeos 2–22: tipos, variables, Scanner, operadores
MoureDev	Curso Completo de Java desde Cero	Sección de objetos, clases y Strings
DiscoDurodeRoer	Variables y tipos	Variables en Java
DiscoDurodeRoer	Scanner — entrada de datos	Lectura por teclado con Scanner

27 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

3.2 Ejercicios de la UD02

3.2.1 Actividades

Cómo leer los recuadros «Para pasar los tests»

Cada ejercicio que se corrige automáticamente indica el nombre exacto de la clase (con su prefijo `_NN_`), los constructores y los métodos que comprueban los tests, si son `static` o no y qué tipo devuelven. Si un nombre, un tipo o un formato no coincide exactamente, el test falla y te dice qué esperaba.

1. (Temperatura) Crear una clase llamada Temperatura con dos métodos:
   - `celsiusToFahrenheit`. Convierte grados *Celsius* a *Fahrenheit*.

   \[F=(1,8*C)+32 \]
   - `fahrenheitToCelsius`. Convierte grados *Fahrenheit* a *Celsius*.

   \[C=\frac{F-32}{1,8} \]

   Para pasar los tests
   - Clase `_01_Temperatura`.
   - `public static double celsiusToFahrenheit(double celsius)`
   - `public static double fahrenheitToCelsius(double fahrenheit)`

2. (Moto) A partir de la siguiente clase:

```java
public class Moto {

    private int velocidad;

    public Moto() {
        velocidad=0;
    }
}
```

   Añade los siguientes métodos:
   - `int getVelocidad()`. Devuelve la velocidad del objeto moto.
   - `void acelera(int mas)`. Permite aumentar la velocidad del objeto moto.
   - `void frena(int menos)`. Permite reducir la velocidad del objeto moto.

   Para pasar los tests
   - Clase `_02_Moto`, con constructor sin parámetros.
   - Métodos de instancia (sin `static`): `int getVelocidad()`, `void acelera(int mas)` y `void frena(int menos)`.

3. (Rebajas) Crea una clase `Rebajas` con un método `descubrePorcentaje()` que descubra el descuento aplicado en un producto. El método recibe el precio original del producto y el rebajado y devuelve el porcentaje aplicado. Podemos calcular el descuento realizando la operación: $$ porcentajeDescuento = \frac{precioOriginal-precioRebajado}{precioOriginal} $$

   Para pasar los tests
   - Clase `_03_Rebajas`.
   - `public static double descubrePorcentaje(double precioOriginal, double precioRebajado)`
   - Devuelve el descuento **como fracción**, no multiplicado por 100: `descubrePorcentaje(150, 123)` devuelve `0.18`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

4. (Finanzas) Realiza una clase `Finanzas` que convierta dólares a euros y viceversa. Codifica los métodos `dolaresToEuros` y `eurosToDolares`. Prueba que dicha clase funciona correctamente haciendo conversiones entre euros y dólares. La clase tiene que tener:
   - Un constructor `Finanzas()` por defecto el cual establece el cambio Dólar-Euro en 1.36.
   - Un constructor `Finanzas(double cambio)`, el cual permitirá configurar el cambio Dólar-euro a una cantidad personalizada.

   El cambio Dólar-Euro indica **cuántos euros vale 1 dólar**. Con el cambio por defecto, 1 $ = 1,36 €; por tanto, para pasar de dólares a euros se multiplica por el cambio y para pasar de euros a dólares se divide.

   Para pasar los tests
   - Clase `_04_Finanzas`, con los constructores `_04_Finanzas()` y `_04_Finanzas(double cambio)`.
   - Métodos de instancia: `double dolaresToEuros(double dolares)` y `double eurosToDolares(double euros)`.
   - Con el cambio por defecto (1 $ = 1,36 €): `dolaresToEuros(5)` devuelve `6.8` y `eurosToDolares(5)` devuelve `3.68` (aprox.).

5. (MiNumero) Realiza una clase `MiNumero` que proporcione el doble, triple y cuádruple de un número (que puede contener decimales) proporcionado en su constructor (realiza un método para `doble`, otro para `triple` y otro para `cuadruple`).

   (Opcional, no hay puntos) Haz que la clase tenga un método `main` y comprueba los distintos métodos.

   Para pasar los tests
   - Clase `_05_MiNumero`, con constructor `_05_MiNumero(double numero)`.
   - Métodos de instancia: `double doble()`, `double triple()` y `double cuadruple()`.

6. (Numero) Realiza una clase `Numero` que almacene un número entero y tenga las siguientes características:
   - Constructor por defecto que inicializa a 0 el número interno.
   - Constructor que inicializa el número interno.
   - Método `anade` que permite sumarle un número al valor interno.
   - Método `resta` que resta un número al valor interno.
   - Método `getValor`. Devuelve el valor interno.
   - Método `getDoble`. Devuelve el doble del valor interno.
   - Método `getTriple`. Devuelve el triple del valor interno.
   - Método `setNumero`. Inicializa de nuevo el valor interno.

   Para pasar los tests
   - Clase `_06_Numero`, con los constructores `_06_Numero()` y `_06_Numero(int numero)`.
   - Métodos de instancia: `void anade(int n)`, `void resta(int n)`, `void setNumero(int n)`, `int getValor()`, `int getDoble()` y `int getTriple()`.

7. (Peso) Crea la clase `Peso`, la cual tendrá las siguientes características:
   - Deberá tener un atributo donde se almacene el peso de un objeto en kilogramos. En el constructor se le pasará el peso y la medida en la que se ha tomado ("Lb" para libras, "Li" para lingotes, "Oz" para onzas, "P" para peniques, "K" para kilos, "G" para gramos y "Q" para quintales).
   - Deberá de tener los siguientes métodos:
   - `getLibras`. Devuelve el peso en libras.
   - `getLingotes`. Devuelve el peso en lingotes.
   - `getPeso`. Devuelve el peso en la medida que se pase como parámetro ("Lb" para libras, "Li" para lingotes, "Oz" para onzas, "P" para peniques, "K" para kilos, "G" para gramos y "Q" para quintales).
   - Para la realización del ejercicio toma como referencia los siguientes datos:
   - 1 Libra = 16 onzas = 453 gramos.
   - 1 Lingote = 32,17 libras = 14,59 kg.
   - 1 Onza = 0,0625 libras = 28,35 gramos.
   - 1 Penique = 0,05 onzas = 1,55 gramos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- 1 Quintal = 100 libras = 43,3 kg.

(Opcional, no hay puntos) Crea además un método `main` para testear y verificar los métodos de esta clase.

> **Para pasar los tests**
>
> - Clase `_07_Peso`, con constructor `_07_Peso(double peso, String medida)`.
> - Métodos de instancia: `double getLibras()`, `double getLingotes()` y `double getPeso(String medida)`.
> - Compara la medida con `equals` (`medida.equals("Lb")`), no con `==`: el test usa textos que no son literales y con `==` no los reconoce.

8. (Millas) Crea una clase con un método `millasAMetros()` que toma como parámetro de entrada un valor en millas marinas y las convierte a metros. Una vez tengas este método escribe otro `millasAKilometros()` que realice la misma conversión, pero esta vez exprese el resultado en kilómetros.

   Nota: 1 milla marina equivale a 1852 metros.

> **Para pasar los tests**
>
> - Clase `_08_Millas`.
> - `public static double millasAMetros(double millas)` y `public static double millasAKilometros(double millas)`.

9. (Coche) Crea la clase `Coche` con dos constructores. Uno no toma parámetros y el otro sí. Los dos constructores inicializarán los atributos `marca` y `modelo` de la clase. El constructor por defecto (sin parametros) crea el coche "Ford" modelo "C-MAX".

   (Opcional, no hay puntos) Crea dos objetos (cada objeto llama a un constructor distinto) y verifica que todo funciona correctamente.

> **Para pasar los tests**
>
> - Clase `_09_Coche`, con los constructores `_09_Coche()` y `_09_Coche(String marca, String modelo)`.
> - Atributos `String marca` y `String modelo` (con esos nombres; pueden ser `private`). No hace falta ningún getter.

10. (Consumo) Implementa una clase `Consumo`, la cual forma parte del "ordenador de a bordo" de un coche y tiene las siguientes características:
    - Atributos:
      - kilometros.
      - litros. Litros de combustible consumido.
      - vmed. Velocidad media.
      - pgas. Precio de la gasolina.
    - Métodos:
      - `getTiempo`. Indicará el tiempo empleado en realizar el viaje.
      - `consumoMedio`. Consumo medio del vehículo (en litros cada 100 kilómetros).
      - `consumoEuros`. Consumo medio del vehículo (en euros cada 100 kilómetros).

    No olvides crear un constructor para la clase que establezca el valor de los atributos. Elige el tipo de datos más apropiado para cada atributo.

> **Para pasar los tests**
>
> - Clase `_10_Consumo`, con constructor `_10_Consumo(double kilometros, double litros, double vmed, double pgas)` (en ese orden).
> - Métodos de instancia: `double getTiempo()` (en horas), `double consumoMedio()` y `double consumoEuros()`.
> - Ejemplo: 100 km, 8 l, 50 km/h y 1,5 €/l → tiempo `2.0`, consumo medio `8.0`, consumo en euros `12.0`.

11. (ConsumoModificadores) Para la clase anterior implementa los siguientes métodos, los cuales podrán modificar los valores de los atributos de la clase:
    - `setKilometros`

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- `setLitros`
- `setVmed`
- `setPgas`

> **Para pasar los tests**
>
> - Clase `_11_ConsumoModificadores`: la del ejercicio anterior (mismo constructor y métodos) más los cuatro métodos de instancia `void setKilometros(double)`, `void setLitros(double)`, `void setVmed(double)` y `void setPgas(double)`.

12. (Restaurante) Un restaurante cuya especialidad son las patatas con carne nos pide diseñar un método (`calcularClientes`) con el que se pueda saber cuántos clientes pueden atender con la materia prima que tienen en el almacén. El método recibe la cantidad de patatas y carne en kilos y devuelve el número de clientes que puede atender el restaurante teniendo en cuenta que por cada tres personas, utilizan un dos kilos de patatas y un kilo de carne.

> **Para pasar los tests**
>
> - Clase `_12_Restaurante`.
> - `public static int calcularClientes(double kilosPatatas, double kilosCarne)`: primero las patatas y después la carne. El resultado se trunca (no se redondea).
> - Ejemplos: `calcularClientes(2, 1)` → `3`; `calcularClientes(3, 2)` → `4`; `calcularClientes(10, 2)` → `6`.

13. (RestauranteClase) Modifica el programa anterior creando una clase que permita almacenar los kilos de patatas y carne del restaurante. Implementa los siguientes métodos:
    - `public void addCarne(int x)`. Añade x kilos de carne a los ya existentes.
    - `public void addPatatas(int x)`. Añade x kilos de patatas a los ya existentes.
    - `public int getComensales()`. Devuelve el número de clientes que puede atender el restaurante (este es el método del ejercicio anterior).
    - `public double getKilosCarne()`. Devuelve los kilos de carne que hay en el almacén.
    - `public double getKilosPatatas()`. Devuelve los kilos de patatas que hay en el almacén.

> **Para pasar los tests**
>
> - Clase `_13_RestauranteClase`, con constructor sin parámetros (almacén vacío).
> - Todos los métodos de la lista son de instancia, con esos nombres y tipos exactos.

14. (Proveedor) Crear un clase llamada `Proveedor` con las siguientes propiedades:
    - `cif`
    - `nombreEmpresa`
    - `descripcion`
    - `sector`
    - `direccion`
    - `telefono`
    - `poblacion`
    - `codPostal`
    - `correo`

    Crear para la clase `Proveedor` los métodos:
    - Constructor que permite crear una instancia con los datos de un proveedor.
    - Métodos get (*getters*).
    - Métodos set (*setters*).
    - Método `verificaCorreo` que devuelve true si la dirección de correo contiene `@`.
    - Método `toString` que devuelva todos los datos del proveedor.

    Crear un método principal `main` ejecutable que:
    - Cree una instancia del objeto `Proveedor` llamado `proveedor`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- Cambie el sector del `proveedor`.
- Muestre el sector del `proveedor`.
- Verifique si el correo es válido.
- Muestre todos los datos del `proveedor`.

> **Para pasar los tests**
>
> - Clase `_14_Proveedor`, con constructor `_14_Proveedor(String cif, String nombreEmpresa, String descripcion, String sector, String direccion, long telefono, String poblacion, long codPostal, String correo)` (en ese orden).
> - Getters de instancia: `getCif()`, `getNombreEmpresa()`... (`telefono` y `codPostal` son `long`; el resto, `String`). Setters `setSector(String)` y `setCorreo(String)`, y `boolean verificaCorreo()`.
> - `toString()` devuelve exactamente este formato (ejemplo):
>
> ```
> _14_Proveedor{cif=B12345678, nombreEmpresa=Empresa Test, descripcion=Descripción de prueba, sector=Tecnología, direccion=Calle Test 123, telefono=91234567, poblacion=Madrid, codPostal=28001, correo=info@test.com}
> ```

15. (Producto) Crear una clase llamada `Producto` con las siguientes propiedades:
    - `codProducto`
    - `nombreProducto`
    - `descripcion`
    - `categoria`
    - `peso`
    - `precio`
    - `stock`

    Crear para la clase `Producto` los siguiente métodos:
    - `Producto`: Permite crear una instancia con los datos de un producto.
    - Getters y Setters para todas las propiedades.
    - `aumentaStock`: Permite aumentar el stock de unidades del producto. Se le pasa el dato de unidades que aumentamos.
    - `disminuyeStock`: Permite disminuir el stock de unidades del producto. Se le pasa el dato de unidades que disminuimos.
    - `ivaProducto`: Permite calcular el IVA aplicado al precio del producto. Se le pasa el dato del porcentaje de IVA.
    - `mostrarDatos`: Muestra los datos del producto. (No tiene test)

    Crear un método principal `main` ejecutable que:
    - Crear dos instancias de la clase `Producto` llamadas `productoHardware` y `productoSoftware`.
    - Mostrar los datos de los dos objetos `Producto` que hemos creado.
    - Aumenta el stock de unidades del `productoHardware` en 12 unidades.
    - Disminuir el stock de unidades del `productoSoftware` en 5 unidades.
    - Calcula el IVA de los dos objetos `Producto` que hemos creado.
    - Mostrar los datos de los dos objetos `Producto`, así como sus importes de IVA y los precios finales de cada una de las instancias.

    > **Para pasar los tests**
    >
    > - Clase `_15_Producto`, con constructor `_15_Producto(long codProducto, String nombreProducto, String descripcion, String categoria, double peso, double precio, long stock)` (en ese orden).
    > - Getters de instancia con esos tipos (`long getCodProducto()`, `double getPeso()`, `long getStock()`...).
    > - `void aumentaStock(long unidades)`, `void disminuyeStock(long unidades)` y `double ivaProducto(double porcentaje)`, que devuelve **el importe del IVA** (no el precio final): con precio 49.99 e IVA 21 devuelve `10.4979`.

16. (Cuenta) Crea una clase llamada `Cuenta` que tendrá los siguientes atributos: `titular` y `cantidad` (puede tener decimales).

    Al crear una instancia del objeto Cuenta, el titular será obligatorio y la cantidad es opcional. Crea dos constructores que cumplan lo anterior, es decir debemos crear dos métodos constructores con el mismo nombre que será el nombre del objeto.

    Crea sus métodos get, set y el método `mostrarDatos` que devuelva un `String` con los datos de la cuenta. Tendrá dos métodos especiales:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- `ingresar(double cantidad)`: se ingresa una cantidad a la cuenta, si la cantidad introducida es negativa, no se hará nada.
- `retirar(double cantidad)`: se retira una cantidad a la cuenta, si restando la cantidad actual a la que nos pasan es negativa, la cantidad de la cuenta pasa a ser 0 retirando el importe máximo en función de la cantidad disponible en el objeto.

Crear una método principal `main` ejecutable:

- Crear una instancia del objeto Cuenta llamada `cuentaParticular1` con el nombre del titular.
- Crear una instancia del objeto Cuenta llamada `cuentaEmpresa1` con el nombre del titular y una cantidad inicial de dinero.
- Mostrar el titular de la instancia `cuentaParticular1`.
- Mostrar el saldo de la instancia `cuentaEmpresa1`.
- Ingresar 1000 € en la instancia `cuentaParticular1`.
- Retirar 500 € en la instancia `cuentaEmpresa1`.
- Mostrar los datos de las dos instancias del objeto `Cuenta`.

> **Para pasar los tests**
>
> - Clase `_16_Cuenta`, con los constructores `_16_Cuenta(String titular)` (cantidad 0) y `_16_Cuenta(String titular, double cantidad)`.
> - Métodos de instancia: `String getTitular()`, `double getCantidad()`, `void setTitular(String)`, `void setCantidad(double)`, `void ingresar(double)`, `void retirar(double)` y `String mostrarDatos()`.
> - `mostrarDatos()` **devuelve** (no imprime) un `String` que incluye el titular y la cantidad.

17. (Libro) Crea una clase llamada `Libro` que guarde la información de cada uno de los libros de una biblioteca. La clase debe guardar las siguientes propiedades:

    - `título`
    - `autor`
    - `editorial`
    - `número de ejemplares totales`
    - `número de prestados`

    La clase contendrá los siguientes métodos:

    - Constructor por defecto.
    - Constructor con parámetros.
    - Métodos Setters/getters.
    - Método `prestamo` que incremente el atributo correspondiente cada vez que se realice un préstamo del libro. No se podrán prestar libros de los que no queden ejemplares disponibles para prestar. Devuelve `true` si se ha podido realizar la operación y `false` en caso contrario.
    - Método `devolucion` que decremente el atributo correspondiente cuando se produzca la devolución de un libro. No se podrán devolver libros que no se hayan prestado. Devuelve `true` si se ha podido realizar la operación y `false` en caso contrario.
    - Método `perdido` que decremente el atributo número de ejemplares por perdida de ejemplar. Como el ejemplar perdido estaba prestado, también decrementa el número de prestados. No se podrán perder libros que no tengan ejemplares o no se hayan prestado. Devuelve `true` si se ha podido realizar la operación y `false` en caso contrario.
    - Método `mostrarDatos` que devuelva un `String` con los datos del libro.

    Crear un método principal `main` ejecutable:

    - Crear una instancia del objeto libro `libroInformatica1` con los datos de un libro.
    - Consultar el título de la instancia `libroInformatica1`.
    - Cambiar la editorial de la instancia `libroInformatica1` por Anaya.
    - Realiza el préstamo de la instancia `libroInformatica1`.
    - Realiza otro préstamo de la instancia `libroInformatica1`.
    - Muestra los prestamos de la instancia `libroInformatica1`.
    - Realiza la devolución de la instancia `libroInformatica1`.
    - Muestra los prestamos de la instancia `libroInformatica1`.
    - Gestiona la pérdida de un ejemplar de la instancia `libroInformatica1`.
    - Muestra los ejemplares de la instancia `libroInformatica1`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- Muestra todos los datos de la instancia `libroInformatica1`.

> **Para pasar los tests**
>
> - Clase `_17_Libro`, con los constructores `_17_Libro()` y `_17_Libro(String titulo, String autor, String editorial, int ejemplaresTotales)` (los prestados empiezan en 0).
> - El constructor por defecto crea el libro `"Sin titulo"`, `"Sin autor"`, `"Sin editorial"`, con 1 ejemplar y 0 prestados.
> - Getters y setters de instancia: `getTitulo`, `getAutor`, `getEditorial`, `int getEjemplaresTotales()`, `int getEjemplaresPrestados()`, y `setTitulo`, `setAutor`, `setEditorial`, `setEjemplaresTotales(int)`.
> - `boolean prestamo()`, `boolean devolucion()` y `boolean perdido()`.
> - `mostrarDatos()` **devuelve** (no imprime) exactamente este formato:
>
> ```
> Titulo: Aprende Java, Autor: David Martínez, Editorial: Marcombo, Ejemplares Totales: 3, Ejemplares Prestados: 0
> ```

18. (Hospital) Crear una clase llamada `Hospital` con las siguientes propiedades y métodos:

    - Propiedades:
        - `codHospital`
        - `nombreHospital`
        - `direccion`
        - `telefono`
        - `poblacion`
        - `codPostal`
        - `habitacionesTotales`
        - `habitacionesOcupadas`
    - Métodos:
        - `Hospital`: Permite crear una instancia con los datos de un hospital.
        - Métodos get.
        - Métodos set.
        - Método `ingreso` que incrementa las habitaciones ocupadas. No puede realizarse el ingreso si las habitaciones ocupadas son iguales a las habitaciones totales del hospital. Devuelve `true` si se ha podido realizar el ingreso.
        - Método `alta` que decrementa las habitaciones ocupadas. No puede realizarse el alta las habitaciones ocupadas son 0. Devuelve `true` si se ha podido realizar el alta.
        - Método `toString` que devuelva todos los datos del hospital.
    - Crear un método principal `main` ejecutable que:
        - Cree una instancia de la clase `Hospital` llamada `hospitalRibera`.
        - Cambie el número de habitaciones de la instancia `hospitalRibera`.
        - Muestre el número de habitaciones de la instancia `hospitalRibera`.
        - Realiza un ingreso de la instancia `hospitalRibera`.
        - Muestra las habitaciones ocupadas de la instancia `hospitalRibera`.
        - Realiza un alta de la instancia `hospitalRibera`.
        - Muestra las habitaciones ocupadas de la instancia `hospitalRibera`.
        - Muestre todos los datos de la instancia `hospitalRibera`.

> **Para pasar los tests**
>
> - Clase `_18_Hospital`, con constructor `_18_Hospital(int codHospital, String nombreHospital, String direccion, long telefono, String poblacion, long codPostal, int habitacionesTotales, int habitacionesOcupadas)` (en ese orden).
> - Getters y setters de instancia con esos tipos, `boolean ingreso()` y `boolean alta()`.
> - `toString()` devuelve un `String` que contiene todos los datos del hospital.

19. (Medico) Crear un clase llamada `Medico` con las siguientes propiedades y métodos:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- Propiedades:
  - `codMedico`
  - `nombre`
  - `apellidos`
  - `dni`
  - `direccion`
  - `telefono`
  - `poblacion`
  - `codPostal`
  - `fechaNacimiento`
  - `especialidad`
  - `sueldo`
- Métodos:
  - Medico: Permite crear una instancia con los datos de un médico.
  - Métodos get. Recuperan datos de la instancia del objeto.
  - Métodos set. Asignan datos a la instancia del objeto.
  - `retencionMedico`: Permite calcular la retención aplicada al sueldo del médico. Se le pasa el dato del porcentaje de retención.
  - `toString`: Devuelve los datos del médico.
- Crear un método principal `main` ejecutable que:
  - Crear dos instancias de la clase `Medico` llamados `medicoDigestivo` y `medicoTraumatologo`.
  - Cambia el sueldo del `medicoTraumatologo`.
  - Muestra el sueldo del `medicoTraumatologo`.
  - Cambia el dni del `medicoDigestivo`.
  - Muestra el dni del `medicoDigestivo`.
  - Calcula la retención de las dos instancias de la clase `Medico` que hemos creado.
  - Mostrar los datos de las dos instancias de la clase `Medico` que hemos creado, así como las retenciones y los sueldos finales de cada una.

Para pasar los tests

- Clase `_19_Medico`, con constructor `_19_Medico(int codMedico, String nombre, String apellidos, String dni, String direccion, long telefono, String poblacion, long codPostal, String fechaNacimiento, String especialidad, double sueldo)` (en ese orden).
- Getters y setters de instancia con esos tipos.
- `double retencionMedico(double porcentaje)` devuelve sueldo × porcentaje / 100: con sueldo 3500 y 20 devuelve `700.0`.
- `toString()` devuelve un `String` que contiene, al menos, el nombre, los apellidos, el DNI y el sueldo.

27 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

3.3 Talleres

3.3.1 Taller UD02_01: Classroom 50

Requisitos previos

Necesitamos:

- Una cuenta de GitHub
- Estar unidos a la clase (Classroom): https://classroom50.org/IES-Eduardo-Primo-Marques/1dam-programacio-java/onboard
- Tener el IDE IntelliJ instalado en nuestro ordenador

Aceptar el Assignement (tarea para la UD02)

Aceptamos el *Assignement* (la tarea/ejercicio) a partir del link del profesor, en este caso: https://classroom50.org/IES-Eduardo-Primo-Marques/1dam-programacio-java/assignments/ud02

La página nos muestra el nombre de la tarea, la fecha de entrega si la tiene, y con qué cuenta estamos identificados. Pulsamos el botón **Accept assignment**.

Al aceptar, Classroom 50 crea automáticamente nuestro repositorio privado en GitHub. Cuando termine, pulsamos **Open repository** para ir a nuestro repositorio en GitHub (ese es el que necesitamos para el siguiente paso).

Preparar nuestra autenticación a GitHub mediante clave privada/pública (necesaria para usar ssh)

En agosto 2021 GitHub eliminó la autenticación por contraseña para operaciones Git en la línea de comandos. Desde el 13 de agosto de 2021, es OBLIGATORIO usar:

- Tokens de acceso personal (Personal Access Tokens - PAT) para HTTPS
- Claves SSH para conexiones SSH
- GitHub CLI con su propio sistema de autenticación

Recomendamos usar siempre SSH y no HTTPS. Con SSH, una vez configurada la clave, `git push` funciona sin que GitHub te pida usuario, contraseña ni tokens cada vez — es un paso único por ordenador y evita tener que lidiar con tokens que caducan.

PASO 1: GENERAR UNA NUEVA CLAVE SSH

1. **Abre la consola** o PowerShell
2. **Genera la clave SSH**:

```
ssh-keygen -t ed25519 -C "tu_email@ejemplo.com"
```

   Reemplaza `"tu_email@ejemplo.com"` con tu email de GitHub

   Para sistemas más antiguos, usa: `ssh-keygen -t rsa -b 4096 -C "tu_email@ejemplo.com"`

3. **Sigue las instrucciones**:

```
Enter file in which to save the key (C:\Users\tunombre/.ssh/id_ed25519):
```

Presiona **Enter** para aceptar la ubicación por defecto

1. **Establece una contraseña segura** (opcional pero recomendado):

```
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
```

2. Introduce una **contraseña segura** y confírmala.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

PASO 2: LOCALIZAR Y COPIAR LA CLAVE PÚBLICA

1. Navega al directorio `.ssh`: Deberías ver `id_ed25519` (clave privada) y `id_ed25519.pub` (clave pública)
2. Copia la clave pública `id_ed25519.pub`:

Abre el archivo en el Bloc de notas y copia el contenido del fichero

PASO 3: INICIAR EL AGENTE Y AGREGAR LA CLAVE PRIVADA (OPCIONAL)

1. Abre PowerShell (esta vez necesitaras hacerlo como Administrador)
2. **Verifica si el servicio SSH está instalado**:

```
Get-WindowsCapability -Online | Where-Object Name -like 'OpenSSH*'
```

1. Si falta algún componente, **instálalo**:

```
#Instalar cliente SSH
Add-WindowsCapability -Online -Name OpenSSH.Client~~~~0.0.1.0
#Instalar servidor SSH
Add-WindowsCapability -Online -Name OpenSSH.Server~~~~0.0.1.0
```

1. **Inicia el agente SSH**:

```
# Iniciar servicio
Start-Service ssh-agent

# Configurar para inicio automático
Set-Service -Name ssh-agent -StartupType Automatic
```

1. **Agrega tu clave SSH**:

```
ssh-add $env:USERPROFILE\.ssh\id_ed25519
```

PASO 4: CONFIGURAR LA CLAVE SSH EN GITHUB

1. **Inicia sesión en tu cuenta de GitHub** - Ve a github.com y accede a tu cuenta
2. **Accede a la configuración de SSH**: - Haz clic en tu **foto de perfil** (esquina superior derecha) - Selecciona **Settings** - En el menú lateral, haz clic en **SSH and GPG keys**
3. **Agrega una nueva clave SSH**: - Haz clic en el botón **New SSH key** o **Add SSH key**
4. **Completa los campos**: - **Title**: Un nombre descriptivo (ej: "Mi PC Windows") - **Key type**: Dejar como "Authentication Key" - **Key**: Pega la clave pública que copiaste anteriormente
5. **Guarda la clave**: - Haz clic en **Add SSH key** - Confirma tu contraseña de GitHub si es necesario

PASO 5: VERIFICAR LA CONEXIÓN SSH

1. **Prueba la conexión** en la terminal o PowerShell:

   Si ejecutas este comando:

   ```
   ssh -T git@github.com
   ```

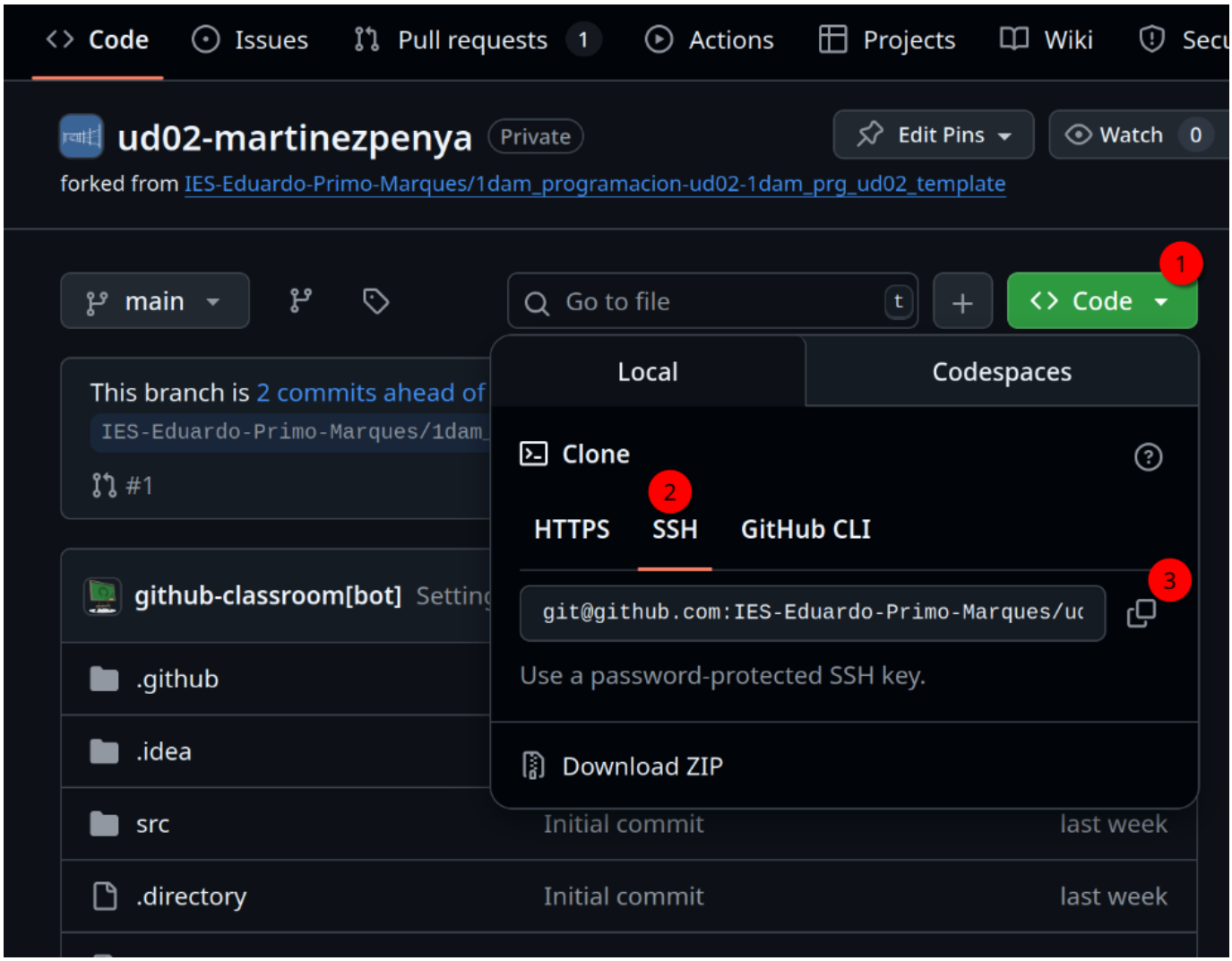
   Deberías ver algo parecido a esto (después de contestar `yes` a la pregunta, fíjate que aparece tu nombre de GitHub, en mi caso `martinezpenya`):

   ```
   The authenticity of host 'github.com (140.82.121.3)' can't be established.
   ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
   This key is not known by any other names.
   Are you sure you want to continue connecting (yes/no/[fingerprint])?yes
   Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
   Hi martinezpenya! You've successfully authenticated, but GitHub does not provide shell access.
   ```

Crear el proyecto en IntelliJ

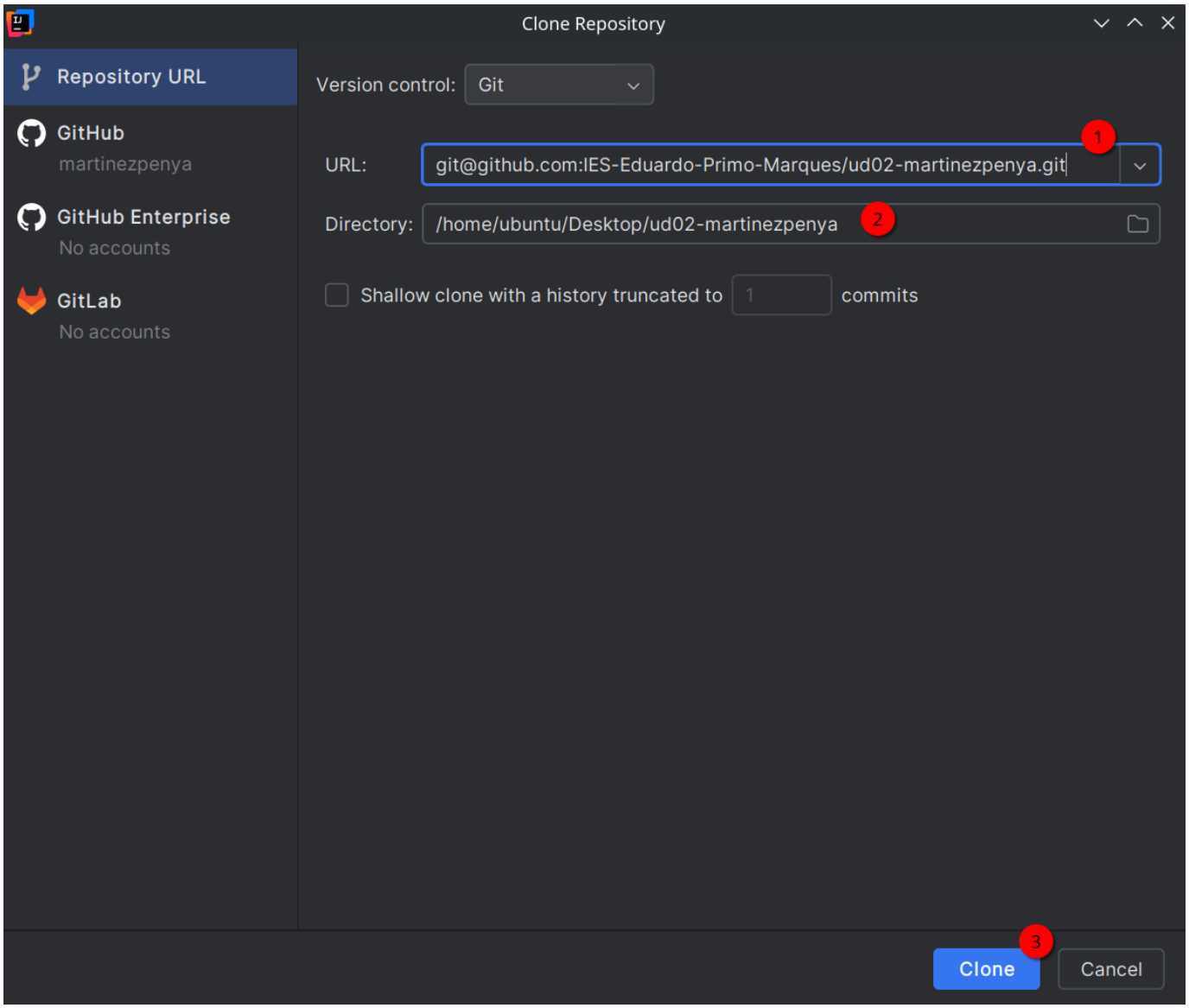
Ahora, en la página de nuestro repositorio en GitHub, pulsamos el botón verde **Code**, elegimos la pestaña **SSH** (no HTTPS) y copiamos esa URL:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



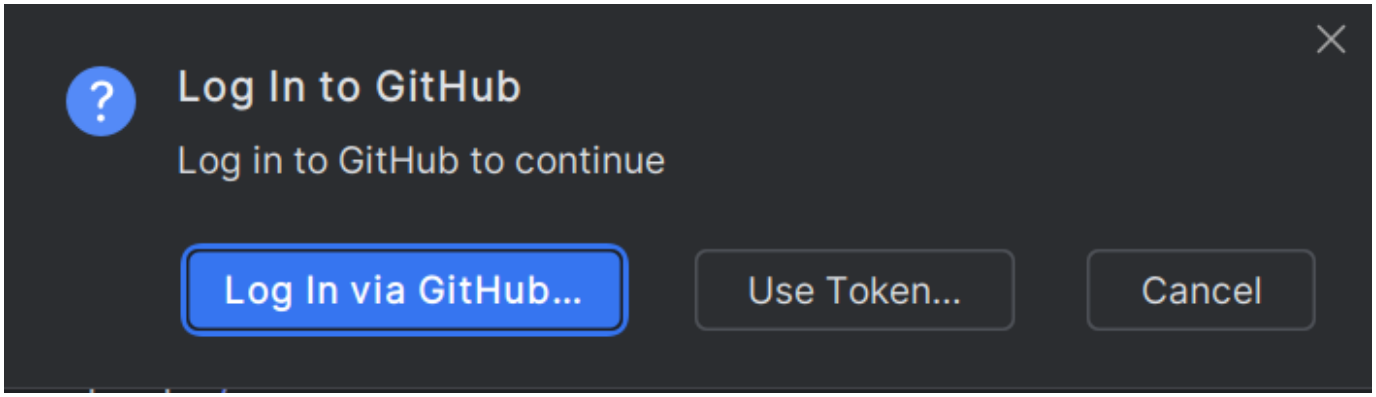
Abriremos IntelliJ y crearemos un nuevo proyecto (New Project from Version Control, o Project/Clone Repository) a partir de la url SSH que hemos copiado:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



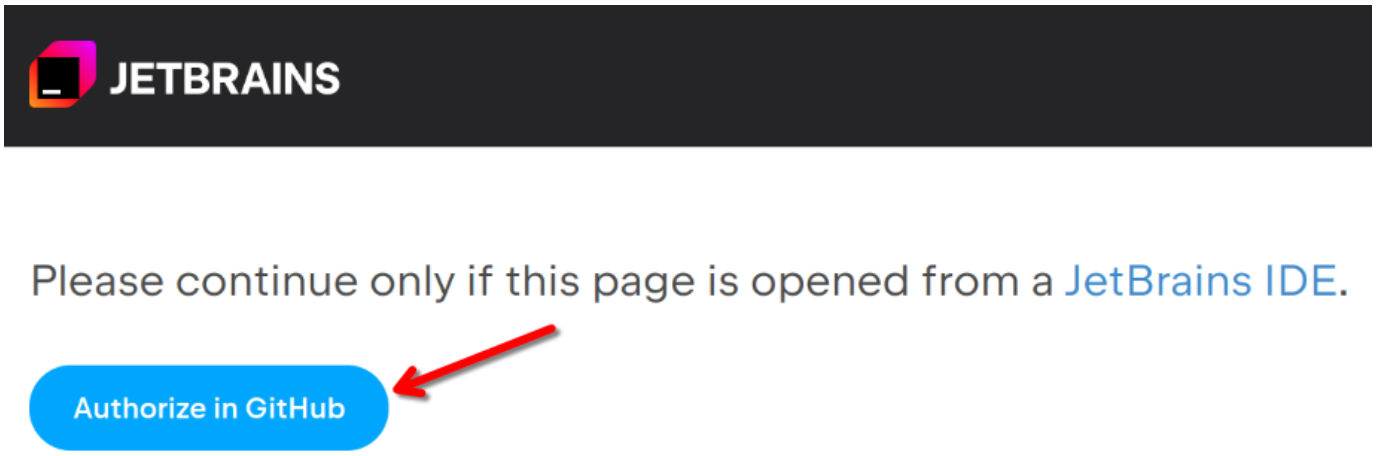
1. Pegamos la URL del paso anterior
2. Elegimos la ubicación de nuestro PC donde guardaremos el proyecto
3. Pulsamos el botón clonar

A continuación nos pedirá hacer Login con nuestra cuenta de GitHub:

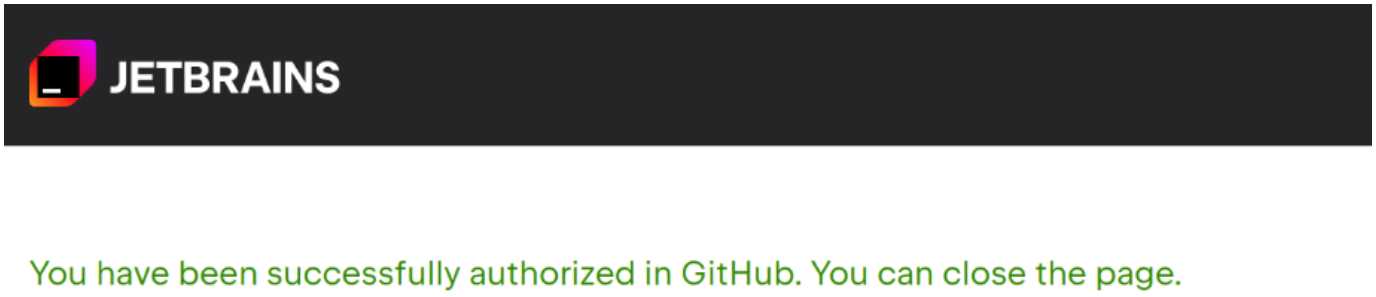


Y en nuestro navegador debemos Autorizar la vinculación de GitHub en el IDE de JetBrains:

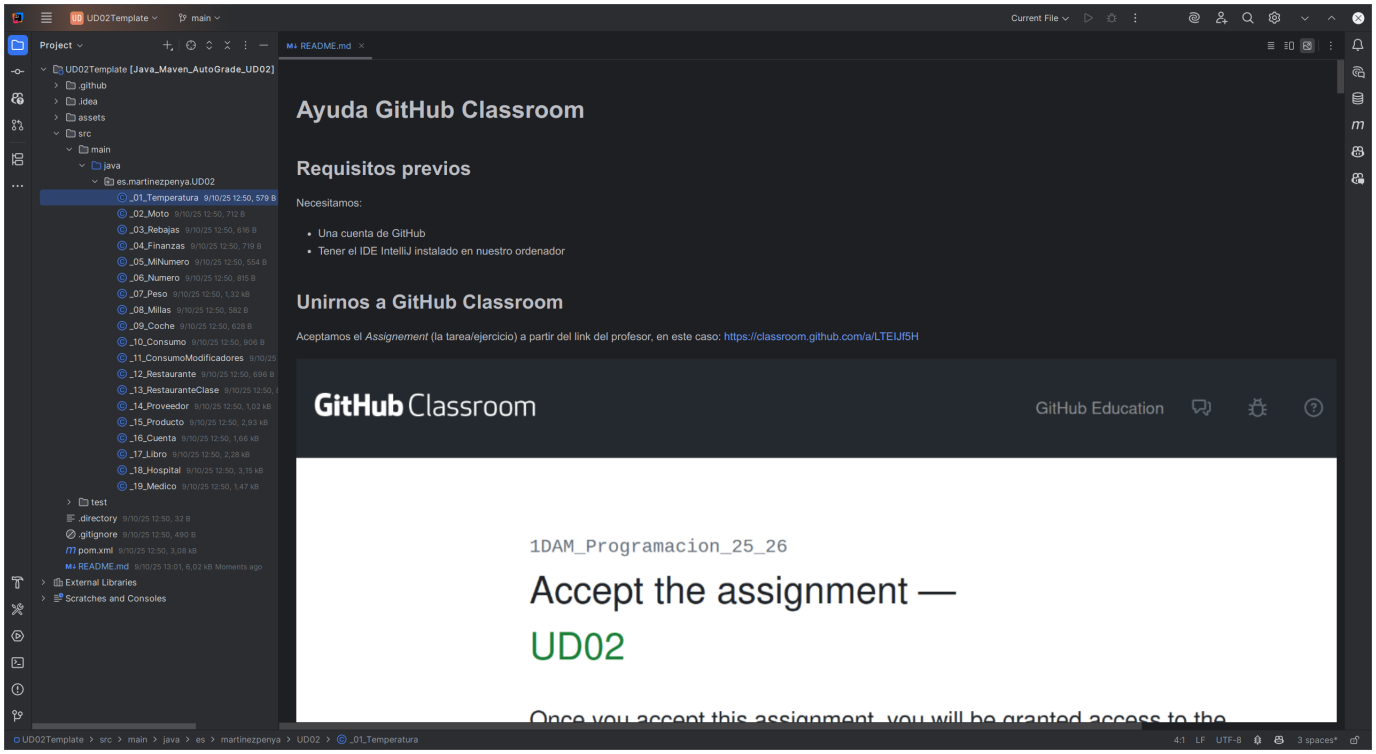
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Y cuando todo esté correcto aparecerá:



Y ahora en nuestro IDE IntelliJ tenemos:



© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Ahora en la carpeta `src` debemos buscar el enunciado del ejercicio, en este caso `src/main/java/es/martinezpenya/UD02/_01_Temperatura.java`

```java
package es.martinezpenya.UD02;

import java.util.Scanner;

/**
 *
 * @author David Martínez (https://wwww.martinezpenya.es | https://martinezpenya.es/1DAMProgramacion/)
 */

/*
ENUNCIADO (Puntos: 2)
    (Temperatura) Crear una clase llamada Temperatura con dos métodos:

   - `celsiusToFarenheit`. Convierte grados *Celsius* a *Farenheit*.
        $$
            F=(1,8*C)+32
        $$

   - `farenheitToCelsius`. Convierte grados *Farenheit* a *Celsius*.
        $$
            C=\frac{F-32}{1,8}
        $$

*/

public class _01_Temperatura {
/* TODO: Tu solucion aqui */
}
```

Debemos leer el enunciado y escribir nuestro código donde pone:

```
...
/* TODO: Tu solucion aqui */
...
```

Una vez comprobamos que el código funciona correctamente, vamos a comprobar si pasará los test impuestos por el profesor.

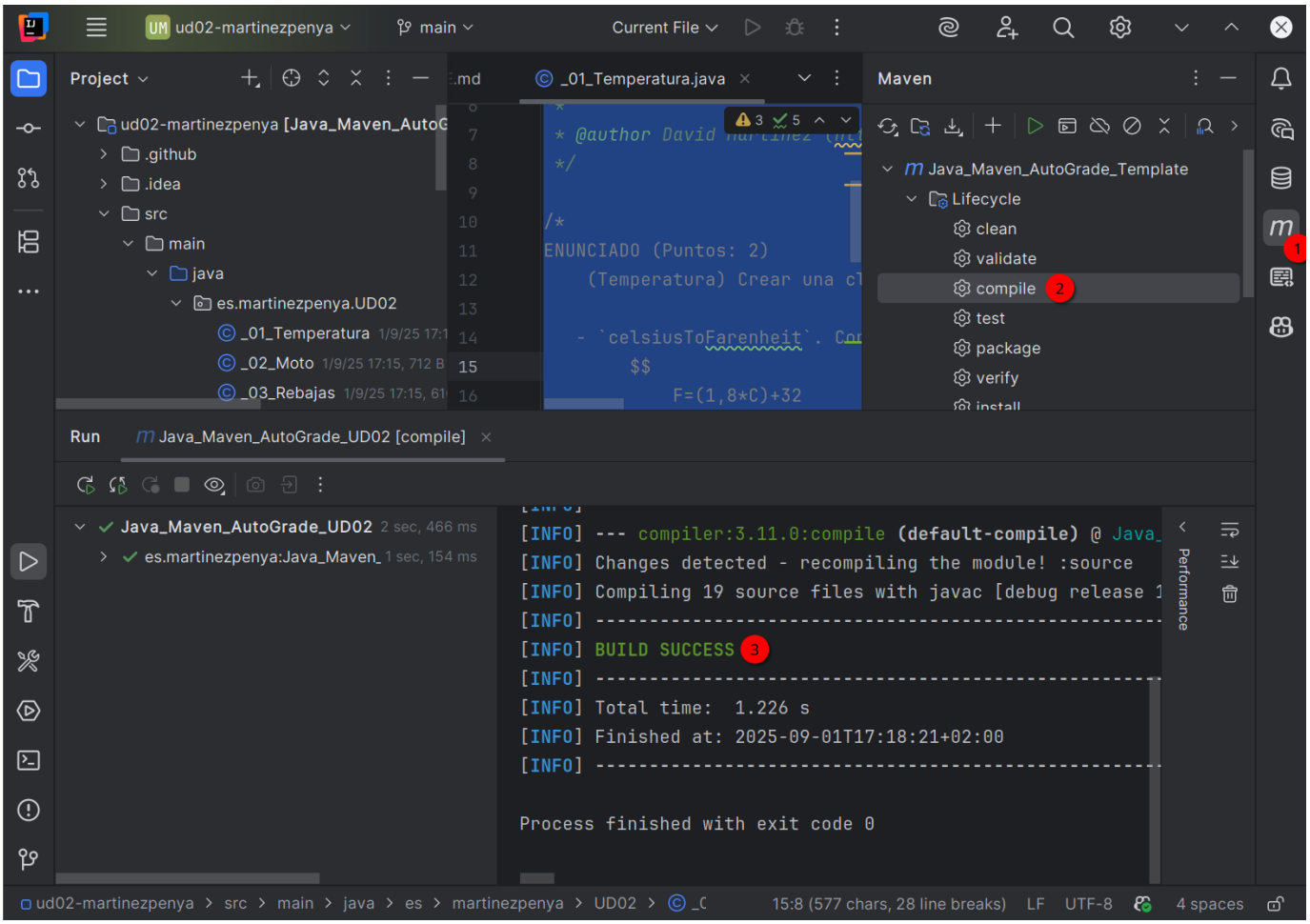
> **Importante**
>
> En las siguientes capturas se asume que hemos resuelto correctamente el primer ejercicio, por tanto pasamos 2 de los 69 tests que contiene la UD02.

Comprobar los tests (Maven)

Debemos realizar dos acciones:

1. Primero compilar el proyecto completo con Maven:

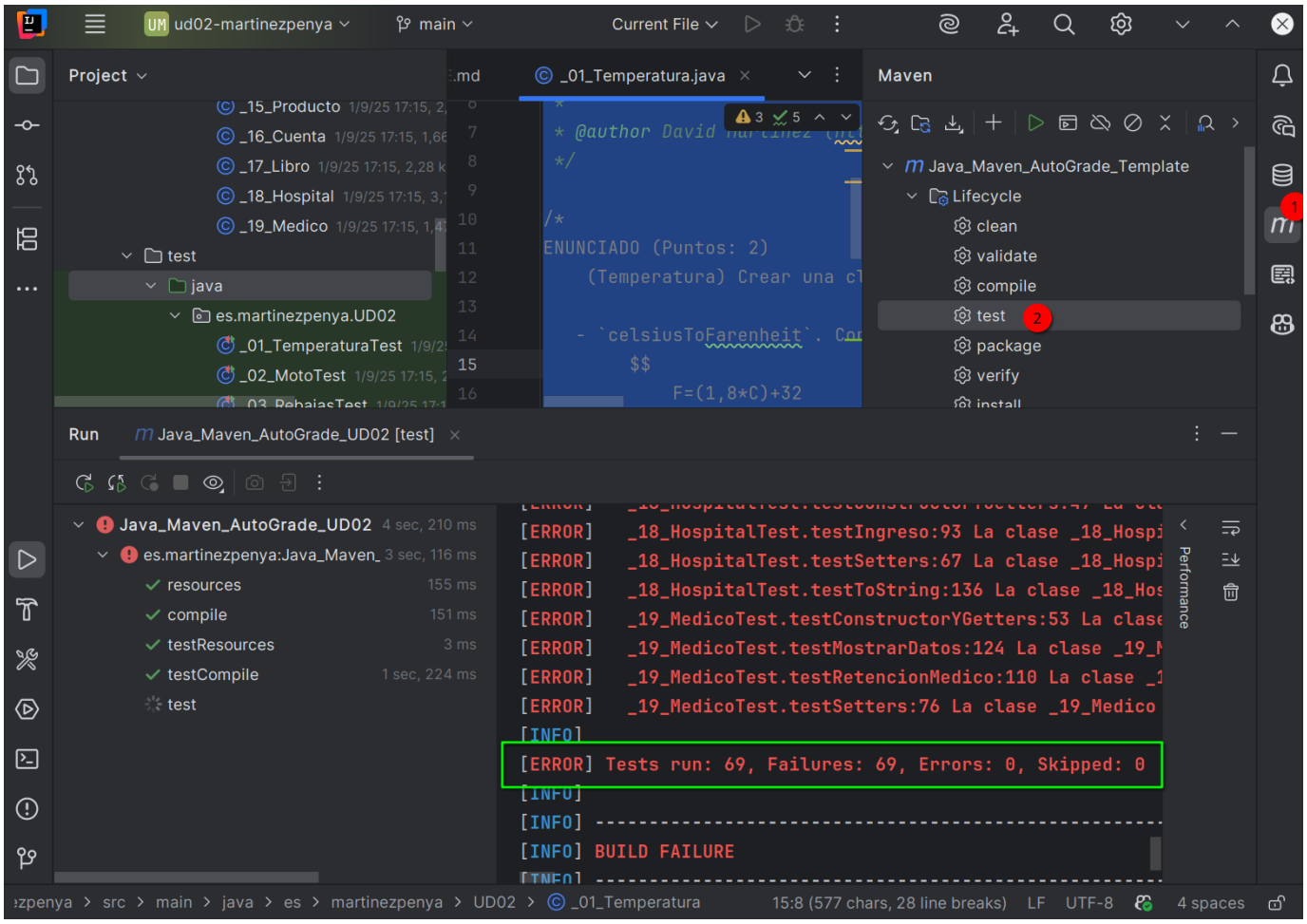
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Una vez comprobado que aparece **BUIL SUCCESS** (seguimos).

1. Lanzar la tarea test y comprobar que sea correcta:

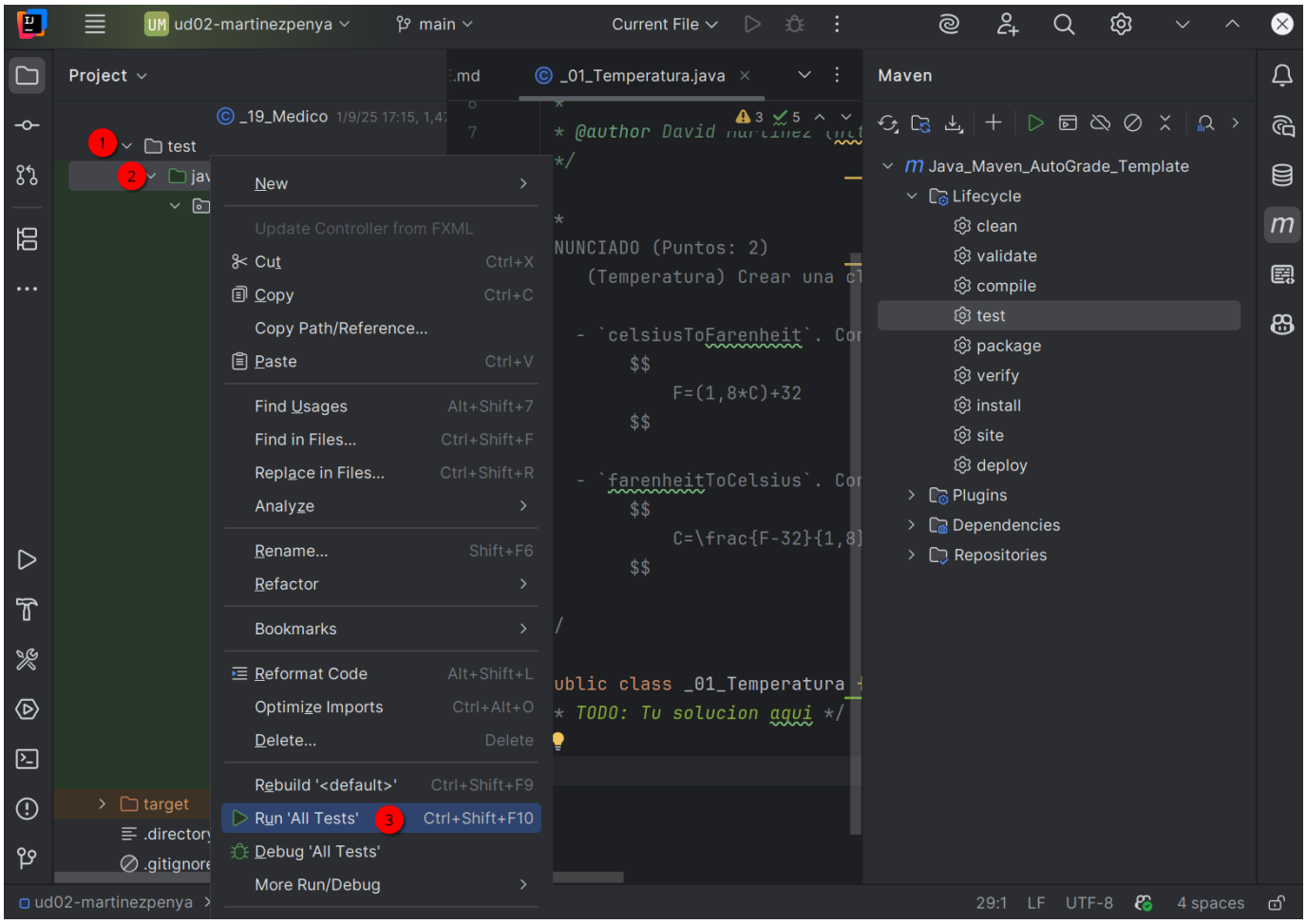
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



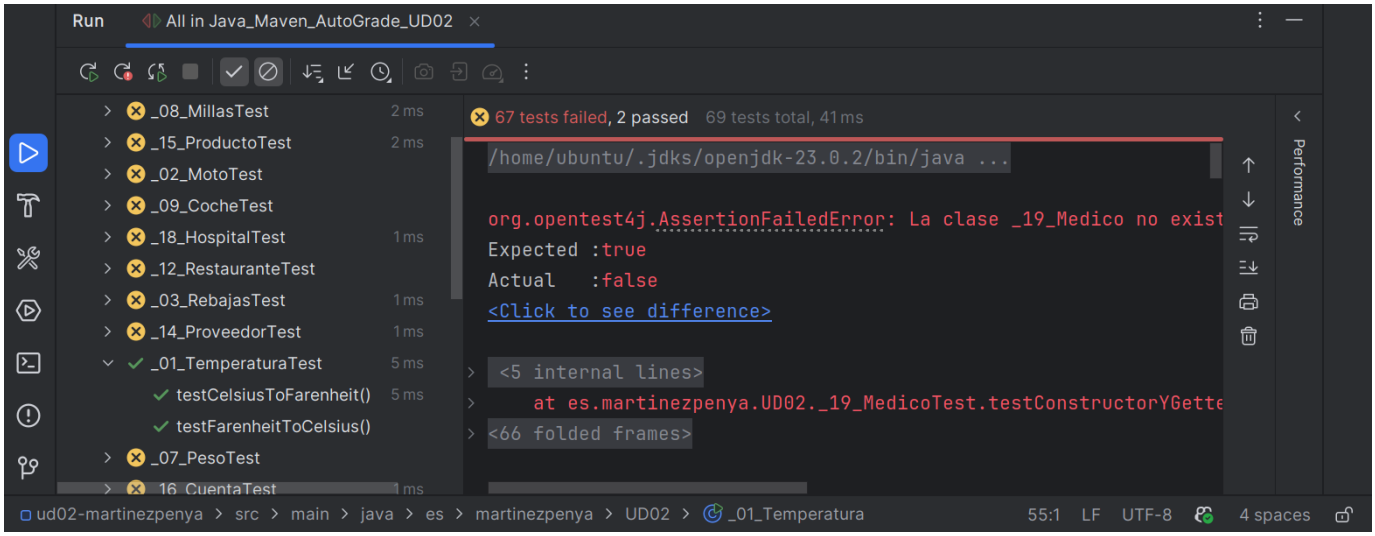
O mejor todavia, podemos ejecutar todos los test con información más visual:

1. Buscamos la carpeta `test` en el proyecto
2. Pulsamos el botón derecho del ratón sobre la carpeta `java`
3. Elegimos la opción "`Run 'All Tests'`"

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Y deberíamos ver un apartado similar en la parte inferior izquierda de nuestro IDE:



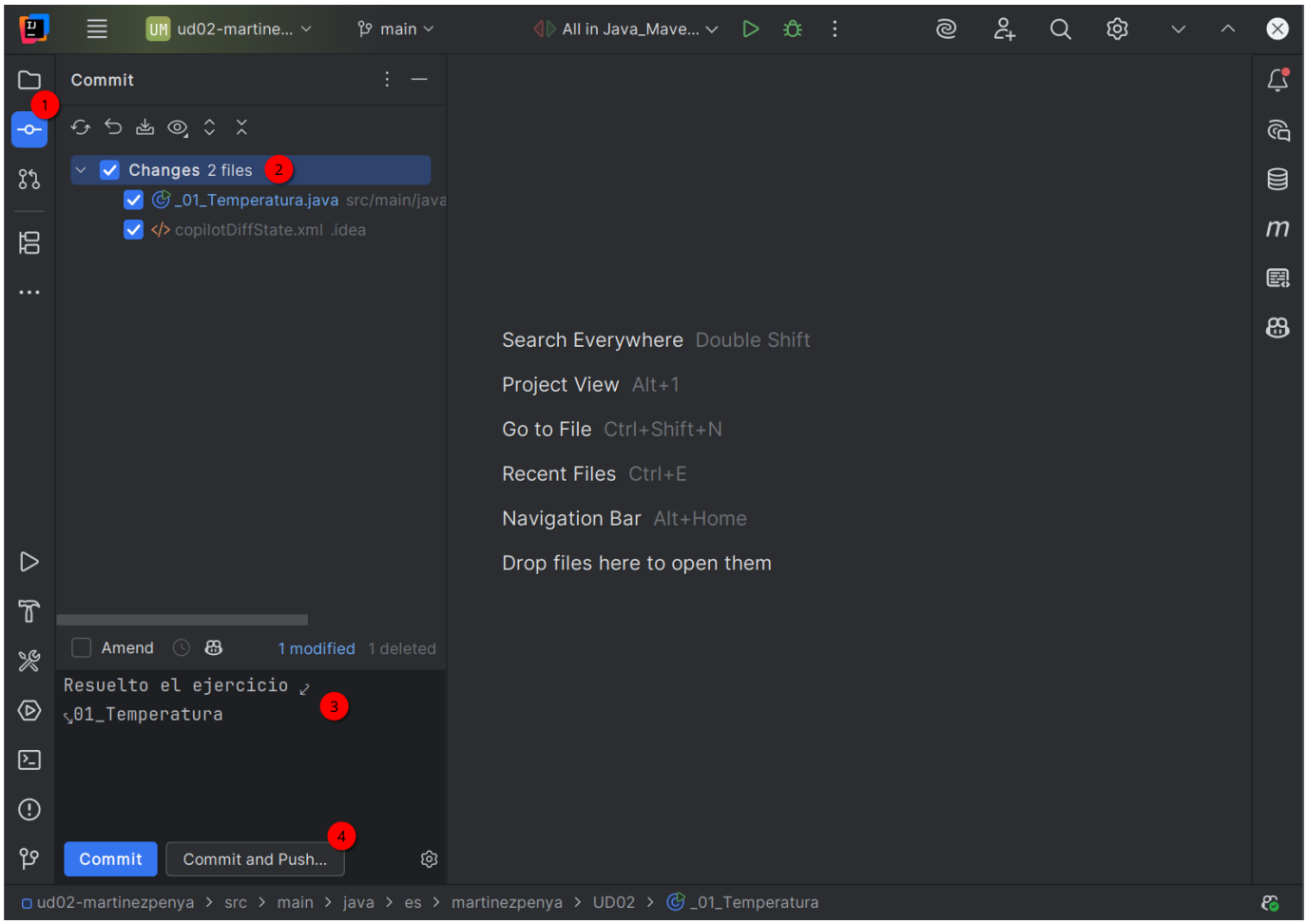
Debes tener en cuenta que:

1. Los tests no se ordenan alfabéticamente.
2. Si aparece un circulo amarillo con una X dentro, el test no pasa (aunque alguno de los tests internos funcionen)
3. Si el tick aparece en verde es que el test ha pasado.
4. Solo se conceden puntos en Classroom 50 si todos los test de la clase funcionan. En este caso solo la clase `_01_Temperatura` nos otorgará puntos

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Subir nuestra solución a Classroom 50

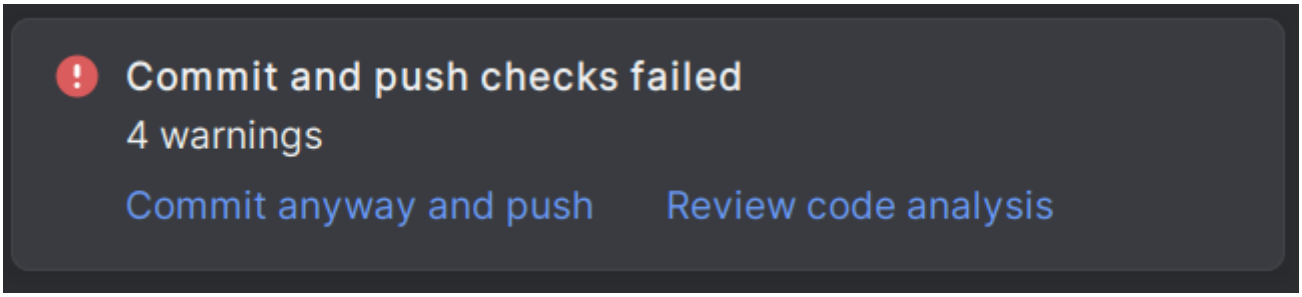
Una vez comprobado que pasamos los tests lo que queda es subir nuestra solución para que sea evaluada y valorada por el docente (Podemos repetir el proceso tantas veces como queramos, así que lo podemos ir haciendo segun vayamos realizando los ejercicios). No hay un botón de "entrega" aparte: cada `push` a `main` dispara la corrección automáticamente.



1. Pasamos al apartado de control de versiones
2. Elegimos los archivos que hemos cambiado y que queremos subir al repositorio para ser evaluados
3. Indicamos el mensaje del commit (recomiendo que los puedas identificar de alguna manera, indicando lo que has cambiado, un número de versión, etc.)
4. Finalmente pulsamos el botón `Commit adn Push`

Error común

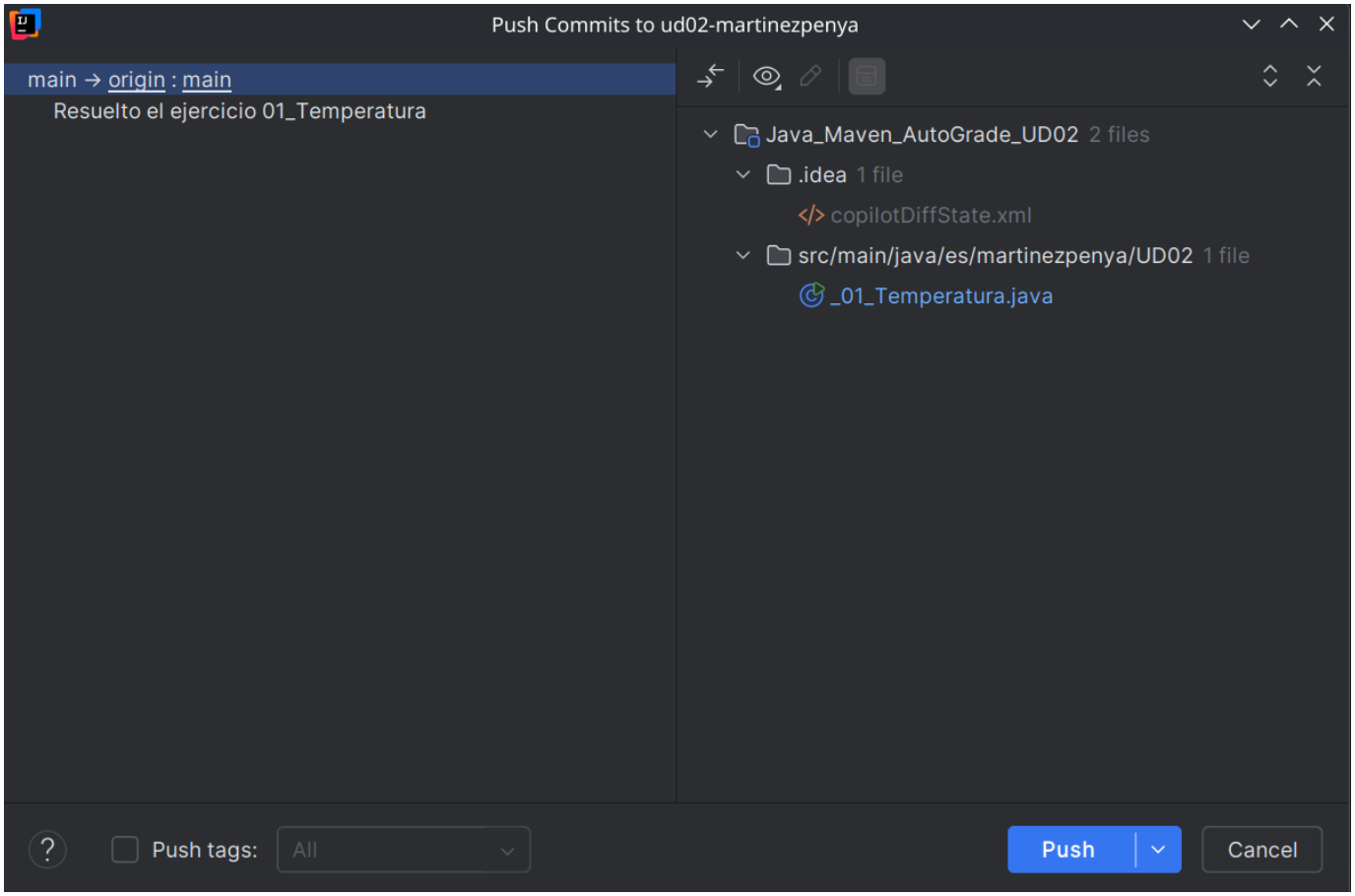
Si aparece este error:



Podemos elegir la opción `Commit anyway and push` si hemos pasado alguno de los test correctamente, o bien `Review code analysis` para ver porque IntelliJ ha detectado algun problema.

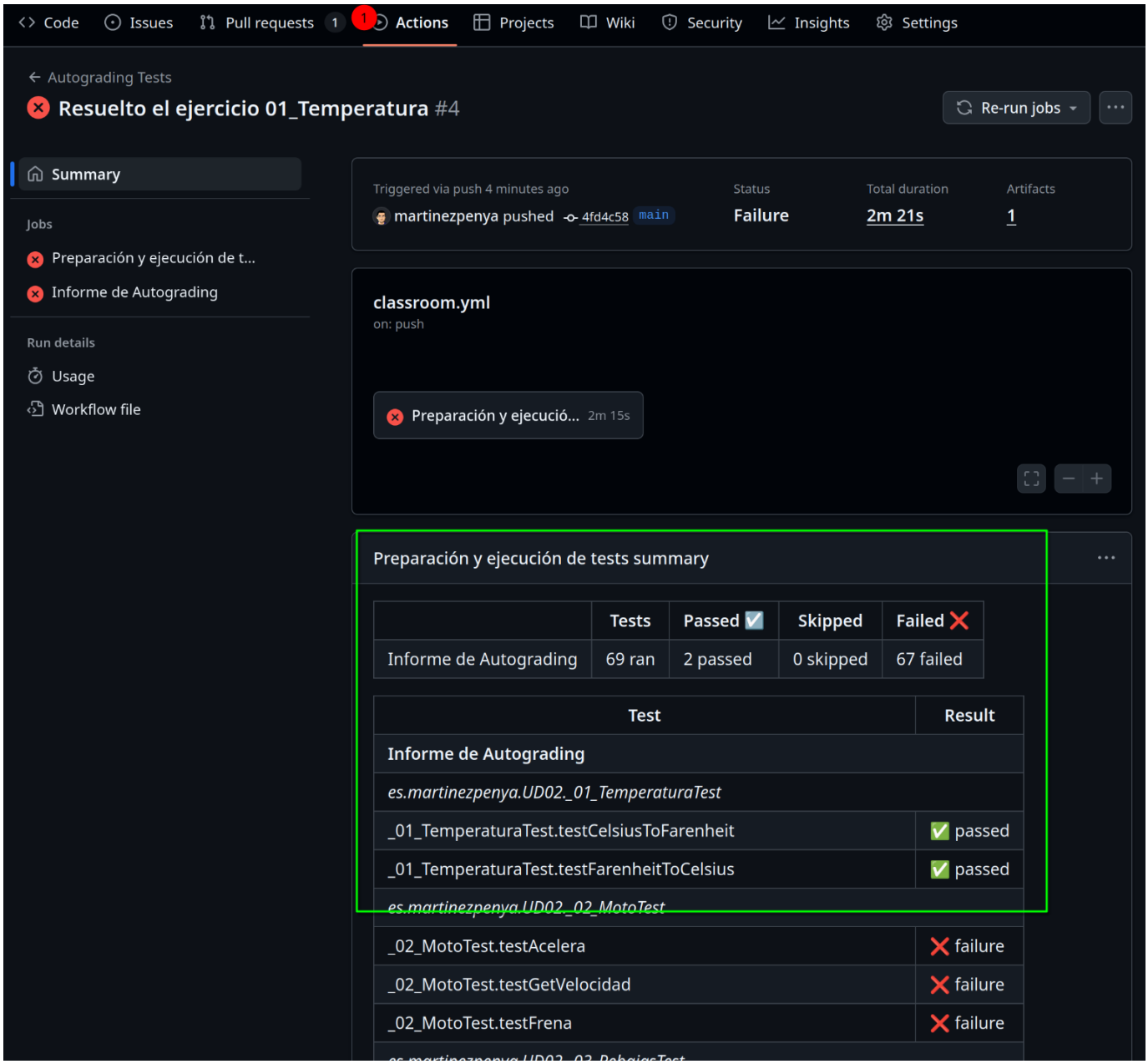
© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Por último aparece esta pantalla, y pulsamos directamente el botón `Push` :



También podemos ver que test hemos pasado satisfactoriamente en nuestro repositorio de Github:

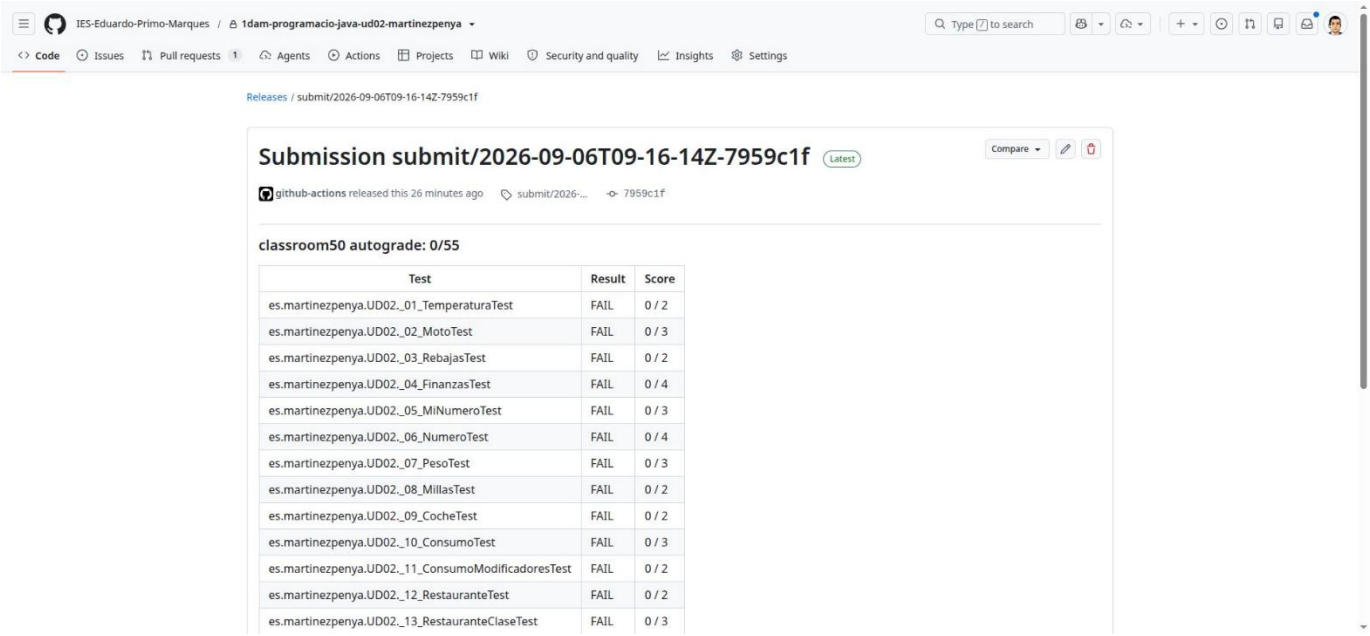
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Ver tu nota y el desglose

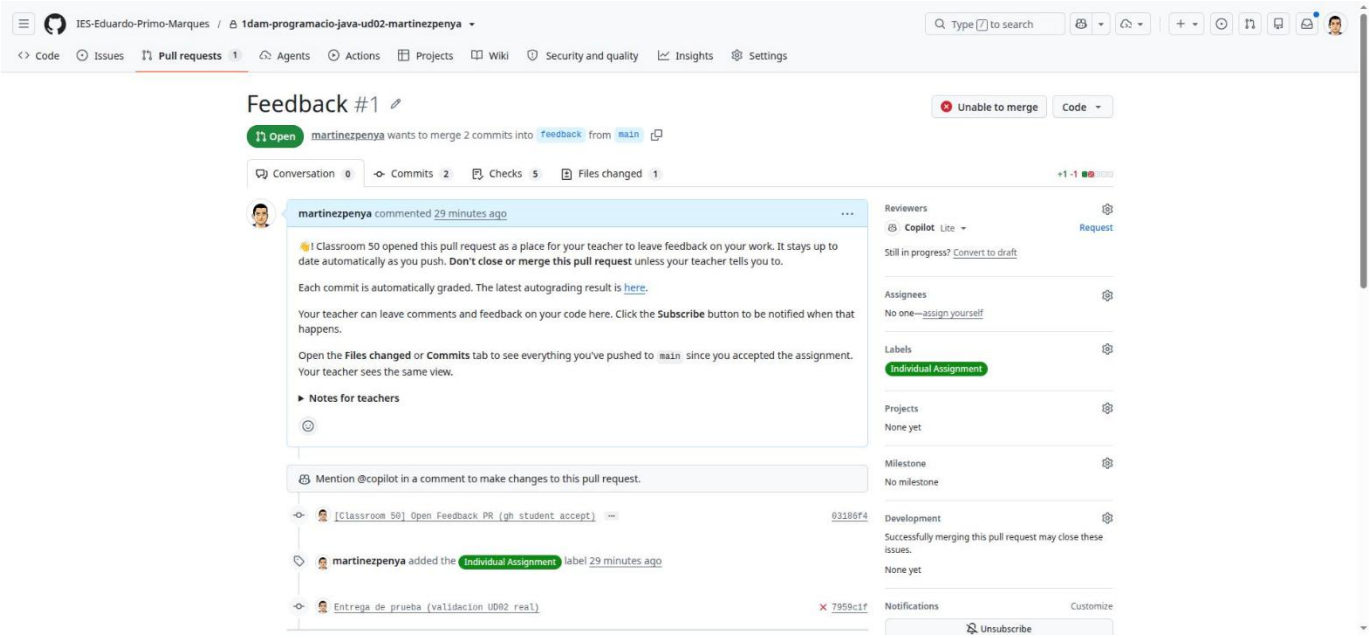
Unos segundos/minutos después de cada push se ejecuta la corrección con GitHub Actions. Cada entrega genera una **Release** en tu repositorio (pestaña *Releases*), con la puntuación total y el resultado de cada test:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Cada fila es una clase de ejercicio; solo se dan los puntos de una fila si **todos** los tests de esa clase pasan — no hay puntos parciales dentro de un mismo ejercicio.

Al aceptar la tarea también se abre automáticamente un Pull Request llamado **Feedback**, que dura todo el curso (no lo cierres ni lo fusiones). Es donde tu profesor puede dejarte comentarios sobre tu código:



OJO: No está permitido modificar archivos del proyecto que no estén en la carpeta `src/java`

Aunque aparentemente todo te funcionará el docente será notificado e invalidará los resultados de los test:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Tarea

Debes enviar tus soluciones a Classroom 50 y superar al menos la mitad de los tests, cuantos más tests superados, mejor nota tendrás en la tarea.

19 de julio de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

3.3.2 Taller UD02_02: Configurar OBS para grabar los exámenes

¿Por qué grabamos los exámenes?

En los exámenes prácticos del módulo grabarás tu pantalla mientras trabajas. La grabación muestra en todo momento **la hora del sistema (con segundos) y tu nombre**, y sirve para comprobar quién ha hecho el examen, cuándo y cómo.

Para grabar usaremos OBS Studio, un programa **libre, gratuito y de código abierto** que funciona en **Linux, Windows y macOS**. Tu profesor te da una plantilla ya configurada: solo tienes que importarla, escribir tu nombre y elegir dónde se guarda el vídeo.

Este taller sirve para dejarlo todo listo **antes** del primer examen y comprobar que funciona en tu portátil.

Antes de empezar

Necesitas:

- **OBS Studio** instalado. Si no lo tienes, descárgalo gratis desde obsproject.com.
- La plantilla `EXAMEN_OBS.zip`.

Descomprime el zip antes de continuar (clic derecho → *Extraer todo* en Windows, doble clic en macOS). Tendrás una carpeta con:

- `Escen_EXAMEN_EPM.json` → la colección de escenas (reloj y nombre)
- `Perf_EXAMEN_EPM` → la carpeta del perfil (ajustes de grabación)

> **Atención**
>
> No intentes importar directamente desde dentro del zip sin descomprimirlo: OBS no lo encontrará.

Paso 1: Instalar OBS e importar la plantilla

1. Instala OBS Studio si no lo tienes.
2. Abre OBS. En **macOS**, los menús **Perfil** y **Colección de escenas** están en la barra de menús de arriba del todo de la pantalla, no dentro de la ventana de OBS.
3. Importa el perfil: menú **Perfil → Importar** y selecciona la carpeta `Perf_EXAMEN_EPM`. Después, abre otra vez el menú **Perfil** y haz clic en **Perf_EXAMEN_EPM** para activarlo (importar no lo activa).
4. Importa la colección de escenas: menú **Colección de escenas → Importar**, pulsa el botón **…** para buscar el archivo `Escen_EXAMEN_EPM.json` y acepta. Después, abre otra vez el menú **Colección de escenas** y haz clic en **Escen_EXAMEN_EPM** para activarla.

> **Atención**
>
> En el paso 3 tienes que elegir **exactamente** la carpeta `Perf_EXAMEN_EPM`: la que tiene dentro el archivo `basic.ini`. Si eliges la carpeta que la contiene (la que se creó al descomprimir el zip, por ejemplo `EXAMEN_OBS`), OBS no da ningún error pero no importa nada, y en el menú **Perfil** solo seguirá apareciendo el perfil que ya tenías (normalmente **Sin título**). Si te pasa, repite la importación entrando en la carpeta del zip y seleccionando `Perf_EXAMEN_EPM`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

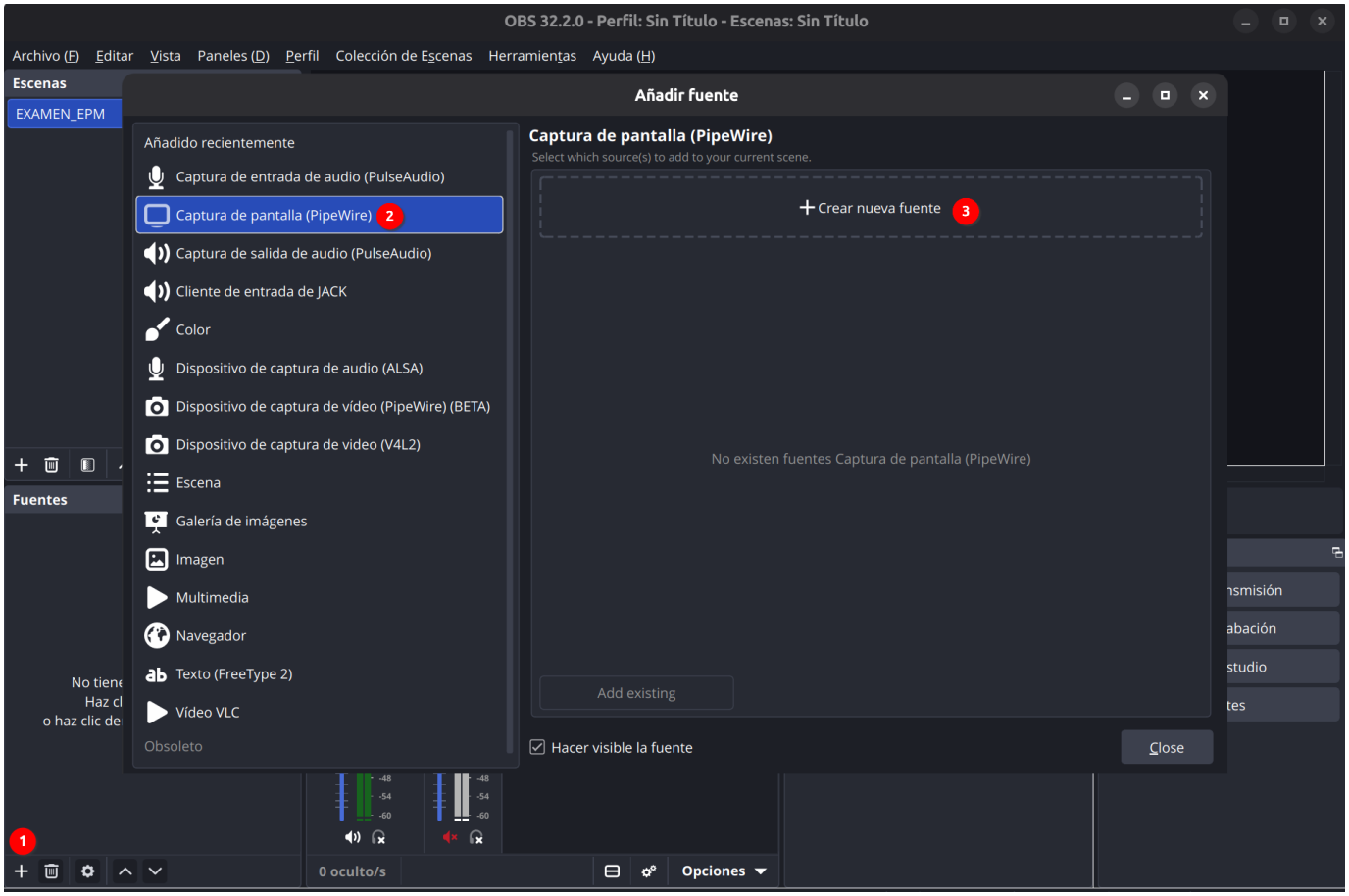
Si el perfil sigue sin aparecer

Cópialo a mano en la carpeta de perfiles de OBS:

1. Cierra OBS.
2. Abre la carpeta de perfiles de OBS:
   - **macOS**: en Finder, menú **Ir → Ir a la carpeta...**, pega `~/Library/Application Support/obs-studio/basic/profiles/` y pulsa Intro.
   - **Windows**: pulsa **Windows + R**, escribe `%APPDATA%\obs-studio\basic\profiles` y pulsa **Aceptar**.
   - **Linux**: abre `~/.config/obs-studio/basic/profiles/` (si instalaste OBS con Flatpak: `~/.var/app/com.obsproject.Studio/config/obs-studio/basic/profiles/`).
3. Si ya hay una carpeta `Perf_EXAMEN_EPM` (o con el nombre de la carpeta del zip, como `EXAMEN_OBS`) que dejaron los intentos fallidos, bórrala.
4. Copia ahí la carpeta `Perf_EXAMEN_EPM` del zip.
5. Abre OBS y actívalo desde el menú **Perfil → Perf_EXAMEN_EPM**.

La plantilla trae una fuente llamada **"Pantalla"** preparada para Linux. Si usas **Windows o macOS** (o Linux sin Wayland), al activar la colección aparecerá un aviso de que falta esa fuente. Es normal: se soluciona así.

1. Cierra el aviso.
2. En el panel **Fuentes**, selecciona **"Pantalla"** y bórrala con el botón de la papelera.
3. Pulsa el botón **+** y elige la captura de pantalla completa de tu sistema:



- Windows → **Captura de pantalla**
- macOS → **Captura de pantalla de macOS**
- Linux con Wayland → **Captura de pantalla (PipeWire)** (ya viene en la plantilla; OBS te pedirá elegir qué pantalla compartir)
- Linux con X11 → **Captura de pantalla (XSHM)**

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

1. Llámala **"Pantalla"** y acepta.
2. Asegúrate de que queda **la última** de la lista de fuentes, debajo de "Nombre" y "Reloj". Si queda arriba, tapará el nombre y el reloj: arrástrala hacia abajo o usa la flecha ↓.

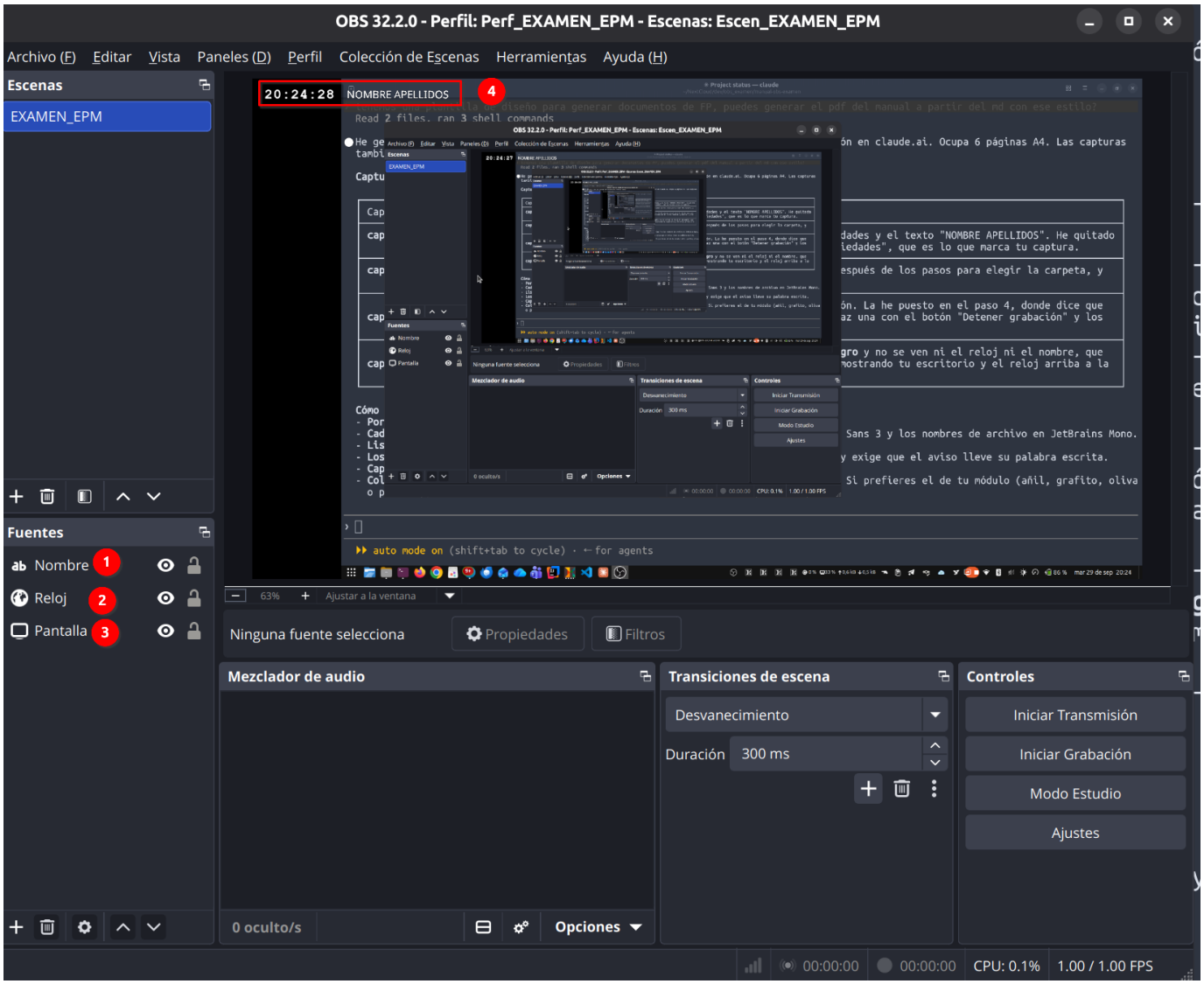
> **La captura debe ocupar toda la grabación**
>
> Uses el sistema que uses (también en Linux con Wayland, si te quedas con la fuente "Pantalla" que trae la plantilla), tu escritorio tiene que llenar **toda** la vista previa de OBS, sin bordes negros alrededor. Si se ve reducido dentro de un recuadro, haz clic derecho sobre la fuente **"Pantalla" → Transformar → Ajustar a la pantalla** (o selecciónala y pulsa **Ctrl+F**; en macOS, **Cmd+F**). Si tu pantalla no es panorámica (16:9), pueden quedar franjas finas a los lados o arriba y abajo: eso es normal.

> **Solo en macOS: permiso de grabación de pantalla**
>
> Al añadir la captura, macOS te pedirá permiso de **grabación de pantalla** para OBS. Sin ese permiso la captura sale en negro, así que dáselo:
>
> 1. Abre **Ajustes del Sistema → Privacidad y seguridad → Grabación de pantalla y audio del sistema** (en versiones antiguas de macOS se llama solo **Grabación de pantalla**).
> 2. Activa el interruptor de **OBS**.
> 3. macOS te pedirá **salir y volver a abrir** OBS: acepta. Al volver a abrirlo, comprueba que la fuente "Pantalla" ya muestra tu escritorio; si sigue en negro, haz doble clic sobre ella y elige tu pantalla en **Pantalla**.
>
> Si OBS te pide también permiso de micrófono, cámara o accesibilidad, no hace falta que lo des: para el examen solo se graba la pantalla. Haz esta configuración **antes del día del examen**, porque hay que reiniciar OBS después de dar el permiso.

Paso 2: Comprobar que el reloj aparece

La plantilla ya incluye una fuente de tipo **Navegador** llamada **"Reloj"**, con el reloj ya integrado (no necesitas ningún archivo aparte ni conexión a internet). No tienes que crearla ni tocarla, solo comprobar que aparece en la esquina de la pantalla en la vista previa:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

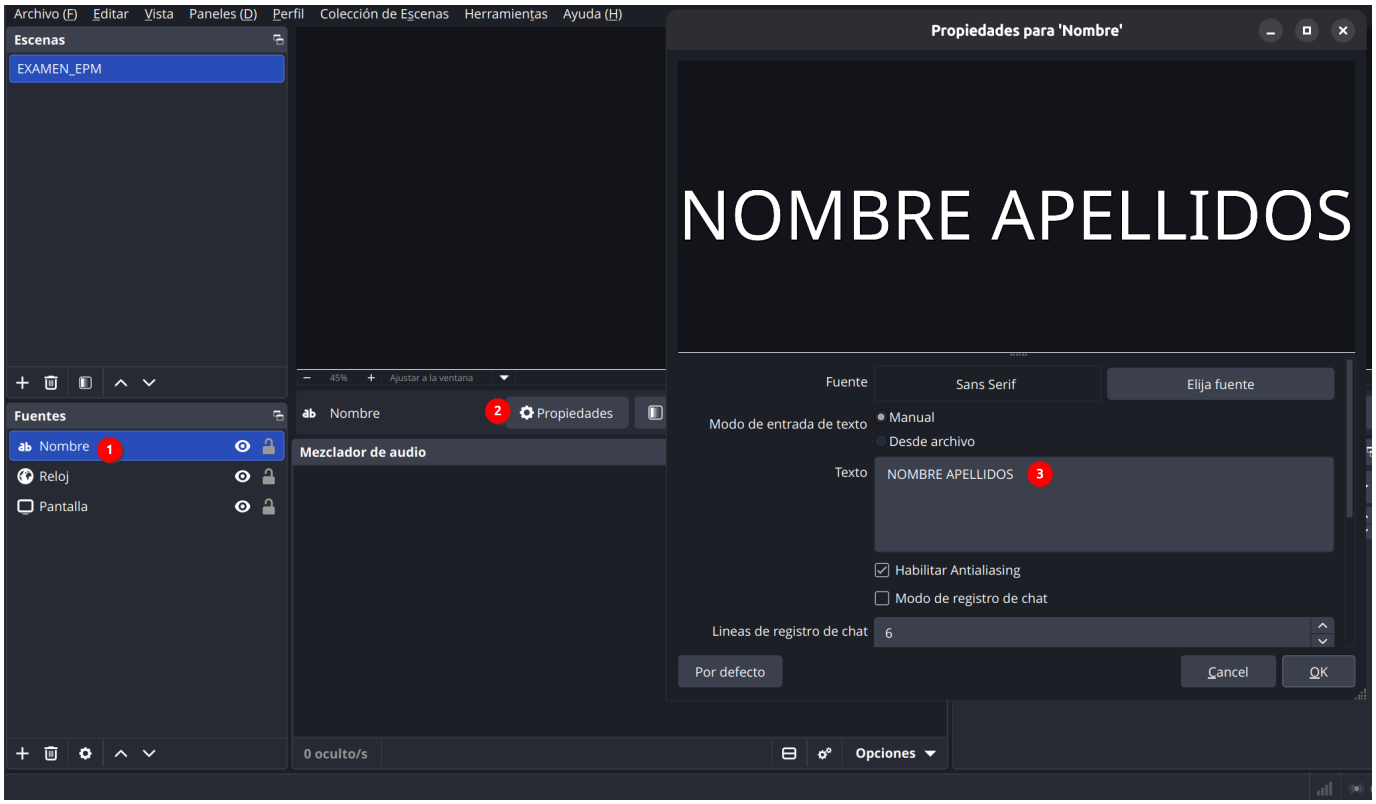


Si no la ves, comprueba que el ojo junto a la fuente "Reloj" en el panel **Fuentes** está activado. Si aun así no aparece, avisa a tu profesor.

Paso 3: Escribir tu nombre

1. En el panel **Fuentes**, busca la fuente de texto llamada **"Nombre"**.
2. Haz doble clic sobre ella (o selecciónala y pulsa **Propiedades**).
3. Borra el texto de ejemplo **"NOMBRE APELLIDOS"**.
4. Escribe tu nombre y apellidos reales.
5. Cierra la ventana de propiedades.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

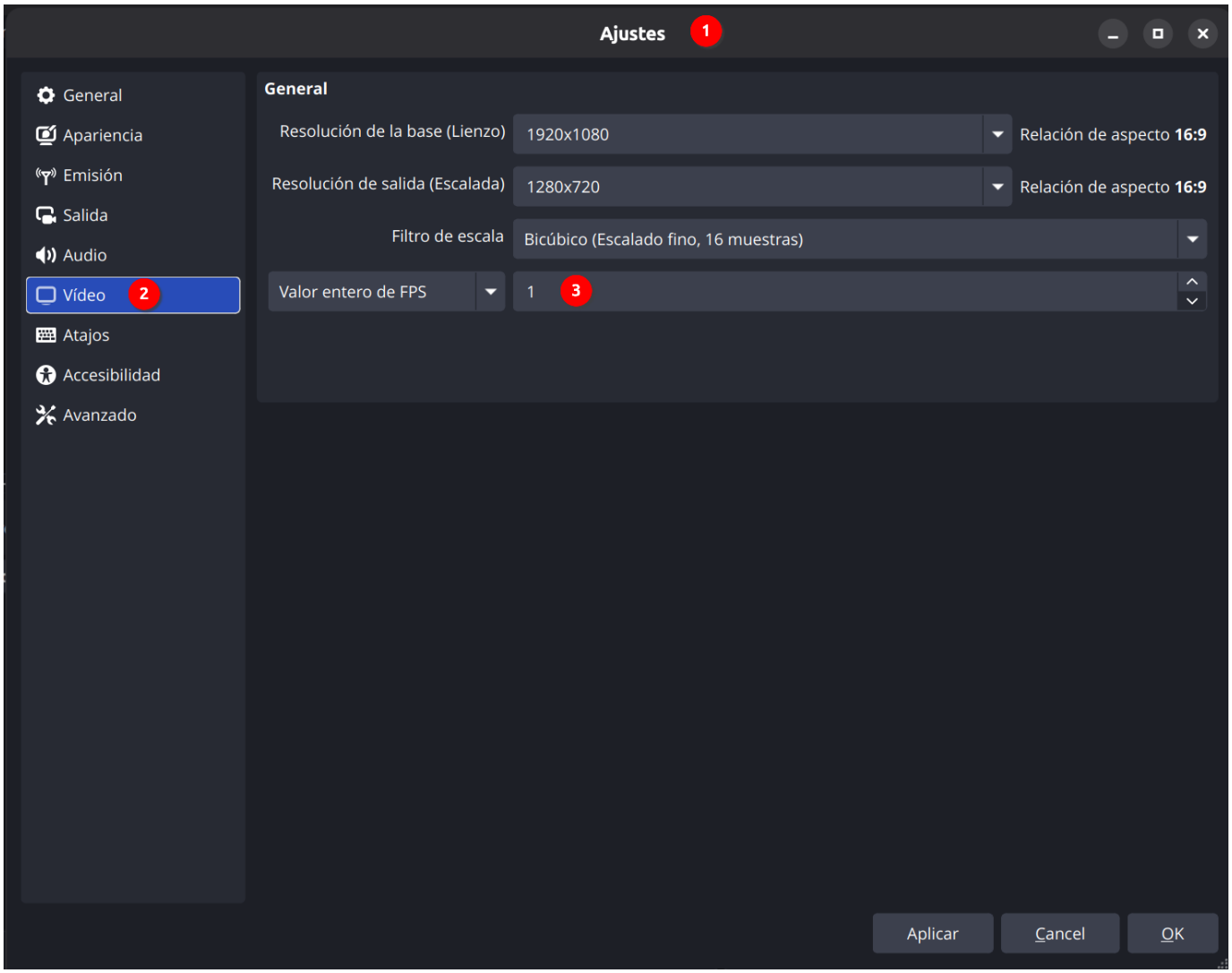


> **Atención**
>
> La fuente debe ser de tipo **Texto (FreeType 2)**. Si por error creas una nueva y tu sistema te ofrece **Texto (GDI+)**, no la uses: esa variante solo existe en Windows y en Mac/Linux no aparecerá.

Paso 4: Elegir dónde se guarda el vídeo

La resolución, los FPS (1) y el formato de grabación (MP4) ya vienen preconfigurados en el perfil que has importado. No cambies nada en **Ajustes → Vídeo**.

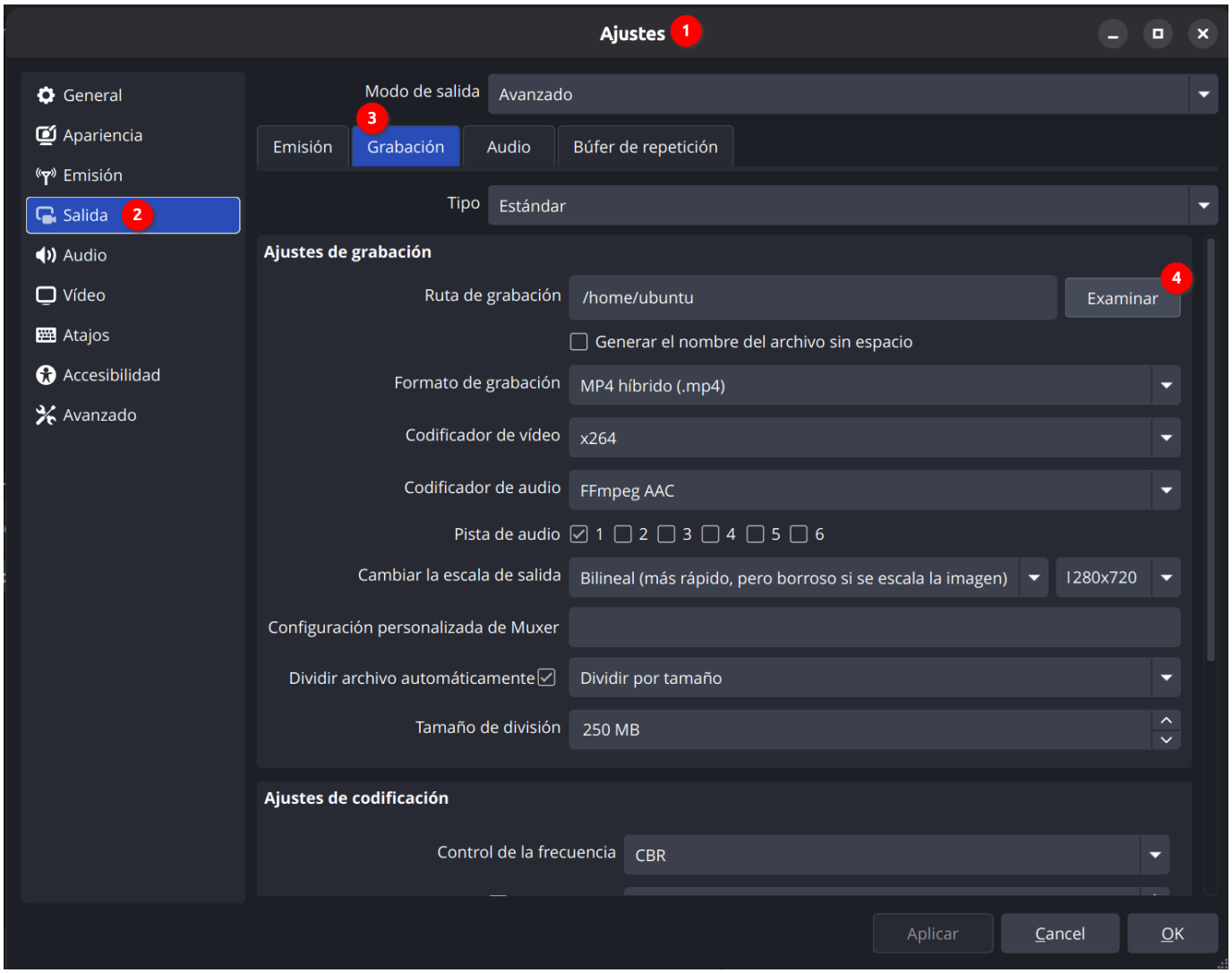
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Lo único que debes revisar es la carpeta donde se guardará la grabación:

1. Abre **Ajustes → Salida** y ve a la pestaña **Grabación**.
2. En **Ruta de grabación**, pulsa **Examinar** y elige tu carpeta **Vídeos**.
3. Pulsa **OK**. No toques ningún otro ajuste de esa pantalla.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Paso 5: Grabar

1. Comprueba en la vista previa de OBS que se ven **tu nombre** y **el reloj con los segundos** en la esquina de la pantalla, y que tu escritorio ocupa toda la vista previa, sin bordes negros alrededor.
2. Pulsa **Iniciar grabación**.
3. Trabaja con normalidad.
4. Al terminar, pulsa **Detener grabación**.
5. El vídeo queda en la carpeta que elegiste en el paso 4, con la fecha y la hora en el nombre del archivo.

Grabaciones largas

OBS divide automáticamente la grabación en varios archivos de unos 45 MB (unos 40 minutos de grabación cada uno), porque Aules no admite ficheros de más de 50 MB. En un examen de más de 40 minutos tendrás varios vídeos con la fecha y hora en el nombre: **hay que entregarlos todos**, no solo el último.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Resumen

Resumen

Importas y **activas** el perfil `Perf_EXAMEN_EPM` y la colección `Escen_EXAMEN_EPM`, cambias la fuente "Pantalla" por la de tu sistema si hace falta (siempre la última de la lista), escribes tu nombre en la fuente "Nombre" y eliges la carpeta de grabación. El resto ya viene configurado.

Tarea

Como tarea, se propone:

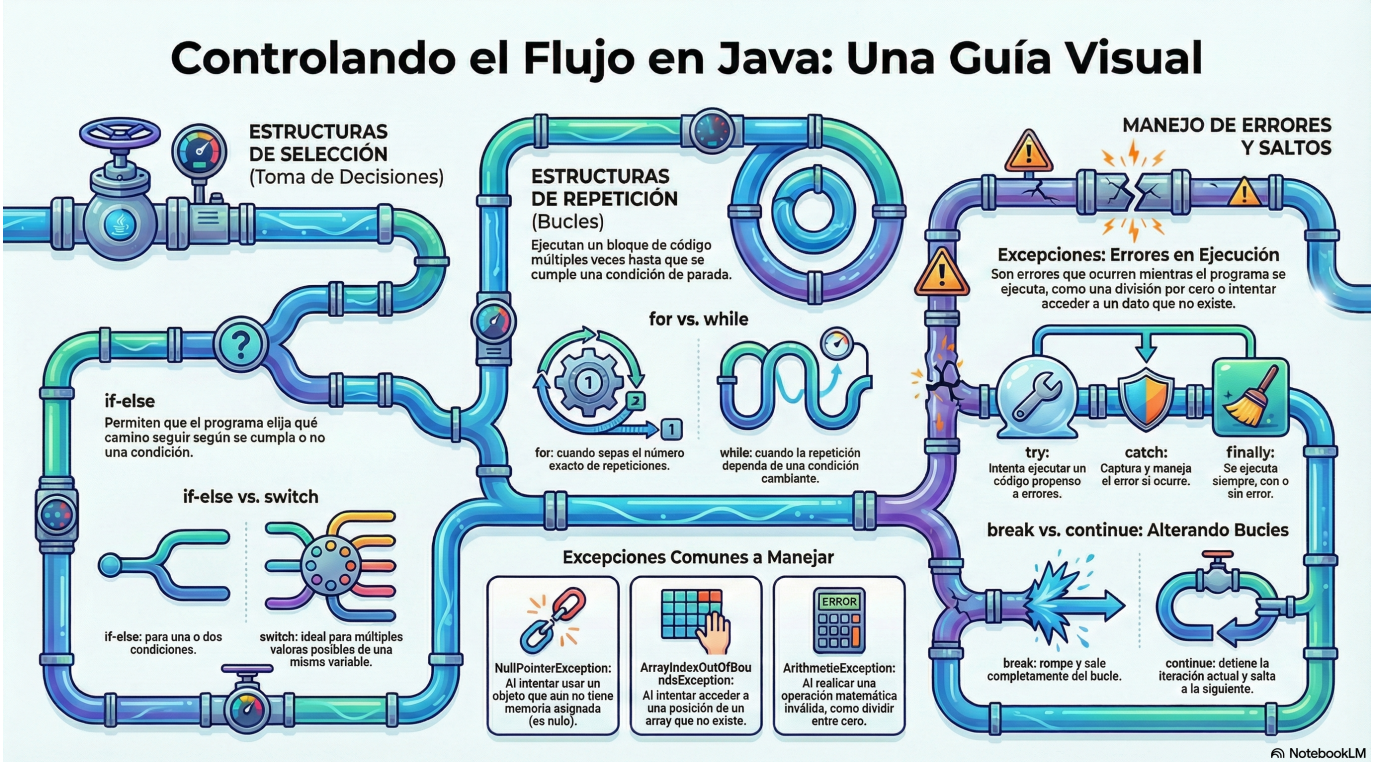
- Dejar OBS configurado en tu portátil siguiendo los pasos de este taller.
- Hacer una grabación corta de tu pantalla, de **al menos 10 segundos**.
- En el vídeo se debe ver en todo momento **el reloj con los segundos avanzando** y **tu nombre y apellidos** (no el texto de ejemplo "NOMBRE APELLIDOS").
- Tu escritorio debe ocupar **toda** la grabación, sin bordes negros alrededor.
- Comprobar que el vídeo se reproduce correctamente antes de entregarlo.
- **Subir a la plataforma AULES el archivo de vídeo (.mp4) generado por OBS.*

27 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

4. UD03

4.1 Estructuras de control y Excepciones



Al finalizar esta unidad serás capaz de...

- Utilizar estructuras de selección (if, switch) para tomar decisiones
- Emplear estructuras de iteración (for, while, do-while) para repetir acciones
- Conocer las sentencias de salto (break, continue)
- Manejar excepciones con try-catch-finally
- Lanzar excepciones personalizadas con throw
- Depurar programas para encontrar errores
- Documentar el código correctamente

4.1.1 Introducción

En unidades anteriores has podido aprender cuestiones básicas sobre el lenguaje JAVA: definición de variables, tipos de datos, asignación de valores, uso de literales, diferentes operadores que se pueden aplicar, conversiones de tipos, inserción de comentarios, etc. Posteriormente, nos sumergimos de lleno en el mundo de los objetos. Primero hemos conocido su filosofía, para más tarde ir recorriendo los conceptos y técnicas más importantes relacionadas con ellos: Propiedades, métodos, clases, declaración y uso de objetos, librerías, etc.

Vale, parece ser que tenemos los elementos suficientes para comenzar a generar programas escritos en JAVA, ¿Seguro?

Como habrás deducido, con lo que sabemos hasta ahora no es suficiente. Existen múltiples situaciones que nuestros programas deben representar y que requieren tomar ciertas decisiones, ofrecer diferentes alternativas o llevar a cabo determinadas operaciones repetitivamente para conseguir sus objetivos.

Si has programado alguna vez o tienes ciertos conocimientos básicos sobre lenguajes de programación, sabes que la gran mayoría de lenguajes poseen estructuras que permiten a los programadores controlar el flujo de la información de sus programas. Esto realmente es una ventaja para la persona que está aprendiendo un nuevo lenguaje, o tiene previsto aprender

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

más de uno, ya que estas estructuras suelen ser comunes a todos (con algunos cambios de sintaxis o conjunto de reglas que definen las secuencias correctas de los elementos de un lenguaje de programación.). Es decir, si conocías sentencias de control de flujo en otros lenguajes, lo que vamos a ver a lo largo de esta unidad te va a sonar bastante.

Para alguien que no ha programado nunca, un ejemplo sencillo le va a permitir entender qué es eso de las sentencias de control de flujo. Piensa en un fontanero (programador), principalmente trabaja con agua (datos) y se encarga de hacer que ésta fluya por donde él quiere (programa) a través de un conjunto de tuberías, codos, latiguillos, llaves de paso, etc. (sentencias de control de flujo).

Pues esas estructuras de control de flujo son las que estudiaremos, conoceremos su estructura, funcionamiento, cómo utilizarlas y dónde. A través de ellas, al construir nuestros programas podremos hacer que los datos (agua) fluyan por los caminos adecuados para representar la realidad del problema y obtener un resultado adecuado.

Los tipos de estructuras de programación que se emplean para el control del flujo de los datos son los siguientes:

- **Secuencia**: compuestas por \(0\), \(1\) o \(N\) sentencias que se ejecutan en el orden en que han sido escritas. Es la estructura más sencilla y sobre la que se construirán el resto de estructuras.
- **Selección**: es un tipo de sentencia especial de decisión y de un conjunto de secuencias de instrucciones asociadas a ella. Según la evaluación de la sentencia de decisión se generará un resultado (que suele ser verdadero o falso) y en función de éste, se ejecutarán una secuencia de instrucciones u otra. Las estructuras de selección podrán ser simples, compuestas y múltiples.
- **Iteración**: es un tipo de sentencia especial de decisión y una secuencia de instrucciones que pueden ser repetidas según el resultado de la evaluación de la sentencia de decisión. Es decir, la secuencia de instrucciones se ejecutará repetidamente si la sentencia de decisión arroja un valor correcto, en otro la estructura de repetición se detendrá.

Además de las sentencias típicas de control de flujo, en esta unidad haremos una revisión de las sentencias de salto, que aunque no son demasiado recomendables, es necesario conocerlas. Como nuestros programas podrán generar errores y situaciones especiales, echaremos un vistazo al manejo de excepciones en JAVA. Posteriormente, analizaremos la mejor manera de llevar a cabo las pruebas de nuestros programas y la depuración de los mismos. Y finalmente, aprenderemos a valorar y utilizar las herramientas de documentación de programas.

4.1.2 Sentencias y bloques

Este epígrafe lo utilizaremos para reafirmar cuestiones que son obvias y que en el transcurso de anteriores unidades se han dado por sabidas. Aunque, a veces, es conveniente recordar. Lo haremos como un conjunto de FAQ's:

- **¿Cómo se escribe un programa sencillo?** Si queremos que un programa sencillo realice instrucciones o sentencias para obtener un determinado resultado, es necesario colocar éstas una detrás de la otra, exactamente en el orden en que deben ejecutarse.
- **¿Podrían colocarse todas las sentencias una detrás de otra, separadas por puntos y comas en una misma línea?**, claro que sí, pero no es muy recomendable. Cada sentencia debe estar escrita en una línea, de esta manera tu código será mucho más legible y la localización de errores en tus programas será más sencilla y rápida. De hecho, cuando se utilizan herramientas de programación, los errores suelen asociarse a un número o números de línea.
- **¿Puede una misma sentencia ocupar varias líneas en el programa?**, sí. Existen sentencias que, por su tamaño, pueden generar varias líneas. Pero siempre finalizarán con un punto y coma.
- **¿En Java todas las sentencias se terminan con punto y coma?**, Efectivamente. Si detrás de una sentencia ha de venir otra, pondremos un punto y coma. Escribiendo la siguiente sentencia en una nueva línea. Pero en algunas ocasiones, sobre todo cuando utilizamos estructuras de control de flujo, detrás de la cabecera de una estructura de este tipo no debe colocarse punto y coma. No te preocupes, lo entenderás cuando analicemos cada una de ellas.
- **¿Qué es la sentencia nula en Java?** La sentencia nula es una línea que no contiene ninguna instrucción y en la que sólo existe un punto y coma. Como su nombre indica, esta sentencia no hace nada.
- **¿Qué es un bloque de sentencias?** Es un conjunto de sentencias que se encierra entre llaves y que se ejecutaría como si fuera una única orden. Sirve para agrupar sentencias y para clarificar el código. Los bloques de sentencias son utilizados en Java en la práctica totalidad de estructuras de control de flujo, clases, métodos, etc. La siguiente tabla muestra dos formas de construir un bloque de sentencias.

Bloque de sentencias 1	Bloque de sentencias 2

`{sentencia1; sentencia2; ...; sentenciaN;}`	`{`
	`sentencia1;`
	`sentencia2;`
	`...;`

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Bloque de sentencias 1	Bloque de sentencias 2
	`sentenciaN;` `}`

- **¿En un bloque de sentencias, éstas deben estar colocadas con un orden exacto?** En ciertos casos sí, aunque si al final de su ejecución se obtiene el mismo resultado, podrían ocupar diferentes posiciones en nuestro programa.

DEBES CONOCER Observa los tres archivos que te ofrecemos a continuación y compara su código fuente. Verás que los tres obtienen el mismo resultado, pero la organización de las sentencias que los componen es diferente entre ellos.

Consulta el Ejemplo01 donde se muestra la organización secuencial de sentencias.

En este primer archivo, las sentencias están colocadas en orden secuencial.

Consulta el Ejemplo02 donde se muestran las sentencias con declaración previa de variables.

En este segundo archivo, se declaran al principio las variables necesarias. En Java no es imprescindible hacerlo así, pero sí que antes de utilizar cualquier variable ésta debe estar previamente declarada. Aunque la declaración de dicha variable puede hacerse en cualquier lugar de nuestro programa.

Consulta el Ejemplo03 donde se muestra la organización en zonas diferenciadas.

En este tercer archivo, podrás apreciar que se ha organizado el código en las siguientes partes: declaración de variables, petición de datos de entrada, procesamiento de dichos datos y obtención de la salida. Este tipo de organización está más estandarizada y hace que nuestros programas ganen en legibilidad.

> **Atención**
>
> Construyas de una forma o de otra tus programas, debes tener en cuenta siempre en Java las siguientes premisas: - **Declara** cada variable antes de utilizarla. - **Inicializa** con un valor cada variable la primera vez que la utilices. No es recomendable usar variables no inicializadas en nuestros programas, pueden provocar errores o resultados imprevistos.

4.1.3 Estructuras de selección

¿Cómo conseguimos que nuestros programas puedan tomar decisiones? Para comenzar, lo haremos a través de las estructuras de selección. Estas estructuras constan de una sentencia especial de decisión y de un conjunto de secuencias de instrucciones.

El funcionamiento es sencillo, la sentencia de decisión será evaluada y ésta devolverá un valor (verdadero o falso), en función del valor devuelto se ejecutará una secuencia de instrucciones u otra.

Por ejemplo, si el valor de una variable es mayor o igual que 5 se imprime por pantalla la palabra APROBADO y si es menor, se imprime SUSPENSO. Para este ejemplo, la comprobación del valor de la variable será la sentencia especial de decisión. La impresión de la palabra APROBADO será una secuencia de instrucciones y la impresión de la palabra SUSPENSO será otra. Cada secuencia estará asociada a cada uno de los resultados que puede arrojar la evaluación de la sentencia especial de decisión. Las estructuras de selección se dividen en:

1. Estructuras de selección simples o estructura if.
2. Estructuras de selección compuesta o estructura ifelse.
3. Estructuras de selección basadas en el operador condicional.
4. Estructuras de selección múltiples o estructura switch.

A continuación, detallaremos las características y funcionamiento de cada una de ellas. Es importante que a través de los ejemplos que vamos a ver, puedas determinar en qué circunstancias utilizar cada una de estas estructuras. Aunque un mismo problema puede ser resuelto con diferentes estructuras e incluso, con diferentes combinaciones de éstas.

Estructura `if`, `if else`, `if else if`

La estructura `if` es una estructura de selección o estructura condicional, en la que se evalúa una expresión lógica o sentencia de decisión y en función del resultado, se ejecuta una sentencia o un bloque de éstas. La estructura `if` puede presentarse de las siguientes formas:

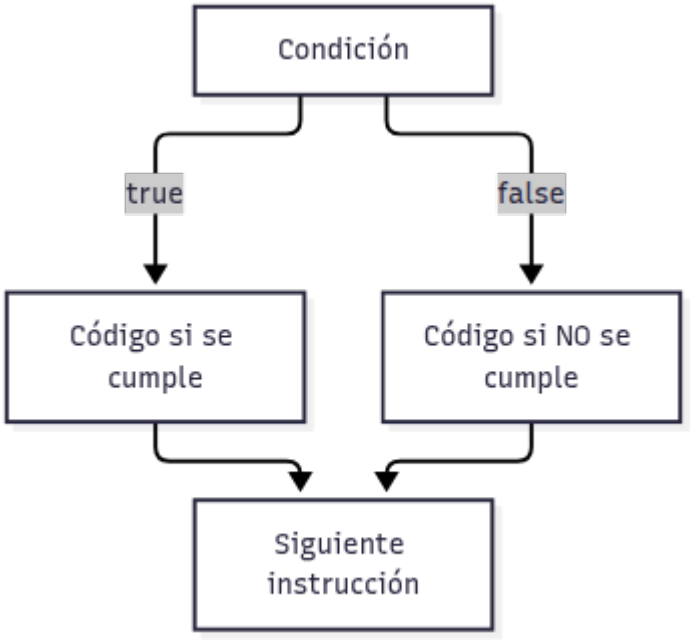
Estructura `if` simple:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
if (expresión-lógica)
    sentencia1;
```

```
if (expresión-lógica){
    sentencia1;
    sentencia2;
    ...;
    sentenciaN;
}
```

Si la evaluación de la expresión-lógica ofrece un resultado verdadero, se ejecuta la sentencia1 o bien el bloque de sentencias asociado. Si el resultado de dicha evaluación es falso, no se ejecutará ninguna instrucción asociada a la estructura condicional.



Estructura `if` de doble alternativa:

```
if (expresión-lógica)
    sentencia1;
else
    sentencia2;
sentencia3;
```

```
if (expresión-lógica){
    sentencia1;
    ...;
    sentenciaN;
} else {
    sentencia1;
    ...;
    sentenciaN;
}
```

Si la evaluación de la expresión-lógica ofrece un resultado verdadero, se ejecutará la primera sentencia o el primer bloque de sentencias. Si, por el contrario, la evaluación de la expresión-lógica ofrece un resultado falso, no se ejecutará la primera sentencia o el primer bloque y sí se ejecutará la segunda sentencia o el segundo bloque.

Ejemplo

Haciendo una interpretación cercana al pseudocódigo tendríamos que si se cumple la condición (expresión lógica), se ejecutará un conjunto de instrucciones y si no se cumple, se ejecutará otro conjunto de instrucciones.

Hay que tener en cuenta que la cláusula `else` de la sentencia `if` no es obligatoria. En algunos casos no necesitaremos utilizarla, pero sí se recomienda cuando es necesario llevar a cabo alguna acción en el caso de que la expresión lógica no se cumpla.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

En aquellos casos en los que no existe cláusula `else`, si la expresión lógica es falsa, simplemente se continuarán ejecutando las siguientes sentencias que aparezcan bajo la estructura condicional `if`.

Los condicionales `if` e `if-else` pueden anidarse, de tal forma que dentro de un bloque de sentencias puede incluirse otro `if` o `if-else`. El nivel de anidamiento queda a criterio del programador, pero si éste es demasiado profundo podría provocar problemas de eficiencia y legibilidad en el código. En otras ocasiones, un nivel de anidamiento excesivo puede denotar la necesidad de utilización de otras estructuras de selección más adecuadas.

Cuando se utiliza anidamiento de este tipo de estructuras, es necesario poner especial atención en saber a qué `if` está asociada una cláusula `else`. Normalmente, un `else` estará asociado con el `if` inmediatamente superior o más cercano que exista dentro del mismo bloque y que no se encuentre ya asociado a otro `else`.

Estructura `if else if`.

Esta estructura es una alternativa a la anidación de sentencias `if else` funciona de modo que si se cumple una condición ejecuta unas sentencias y el caso contrario comprueba otra condición ejecutando unas sentencias si se cumple y así sucesivamente. Veamos un ejemplo con `if` anidados:

```
if (condicion1) {
    sentencias1;
} else {
    if (condicion2) {
        sentencias2;
    } else {
        if (condicion3) {
            sentencias3;
        } else {
            sentencias4;
        }
    }
}
```

El mismo ejemplo usando `if else if` quedaría de este modo:

```
if (condicion1) {
    sentencias1;
} else if (condicion2) {
    sentencias2;
} else if (condicion3) {
    sentencias3;
} else {
    sentencias4;
}
```

Consulta el Ejemplo12 para ver un programa completo que calcula la nota de un examen combinando estructuras if-else anidadas.

Estructura `switch`

¿Qué podemos hacer cuando nuestro programa debe elegir entre más de dos alternativas?, una posible solución podría ser emplear estructuras if anidadas, aunque no siempre esta solución es la más eficiente. Cuando estamos ante estas situaciones podemos utilizar la estructura de selección múltiple switch. En la siguiente tabla se muestra tanto la sintaxis, como el funcionamiento de esta estructura.

Sintaxis switch:

```
switch (expresion) {
    case valor1:
        sentencia1_1;
        sentencia1_2;
        ....
        break;
    case valor2:
        ....
    case valorN:
        sentenciaN_1;
        sentenciaN_2;
        ....
        break;
    default:
    sentencias-default;
}
```

Condiciones:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- Donde expresión debe ser del tipo `char`, `byte`, `short` o `int`, y las constantes de cada `case` deben ser de este tipo o de un tipo compatible.
- La `expresion` debe ir entre paréntesis.
- Cada `case` llevará asociado un `valor` y se finalizará con dos puntos (`:`).
- El bloque de sentencias asociado a la cláusula `default` puede finalizar con una sentencia de ruptura `break` o no.

Funcionamiento:

- Las diferentes alternativas de esta estructura estarán precedidas de la cláusula case que se ejecutará cuando el valor asociado al case coincida con el valor obtenido al evaluar la expresión del switch.
- En las cláusulas case, no pueden indicarse expresiones condicionales, rangos de valores o listas de valores. (otros lenguajes de programación sí lo permiten). Habrá que asociar una cláusula case a cada uno de los valores que deban ser tenidos en cuenta.
- La cláusula default será utilizada para indicar un caso por defecto, las sentencias asociadas a la cláusula default se ejecutarán si ninguno de los valores indicados en las cláusulas case coincide con el resultado de la evaluación de la expresión de la estructura switch.
- La cláusula default puede no existir, y por tanto, si ningún case ha sido activado finalizaría el switch.
- Cada cláusula case puede llevar asociadas una o varias sentencias, sin necesidad de delimitar dichos bloques por medio de llaves.
- En el momento en el que el resultado de la evaluación de la expresión coincide con alguno de los valores asociados a las cláusulas case, se ejecutarán todas las instrucciones asociadas hasta la aparición de una sentencia break de ruptura. (la sentencia break se analizará en epígrafes posteriores)

EXPRESIONES SWITCH MEJORADAS

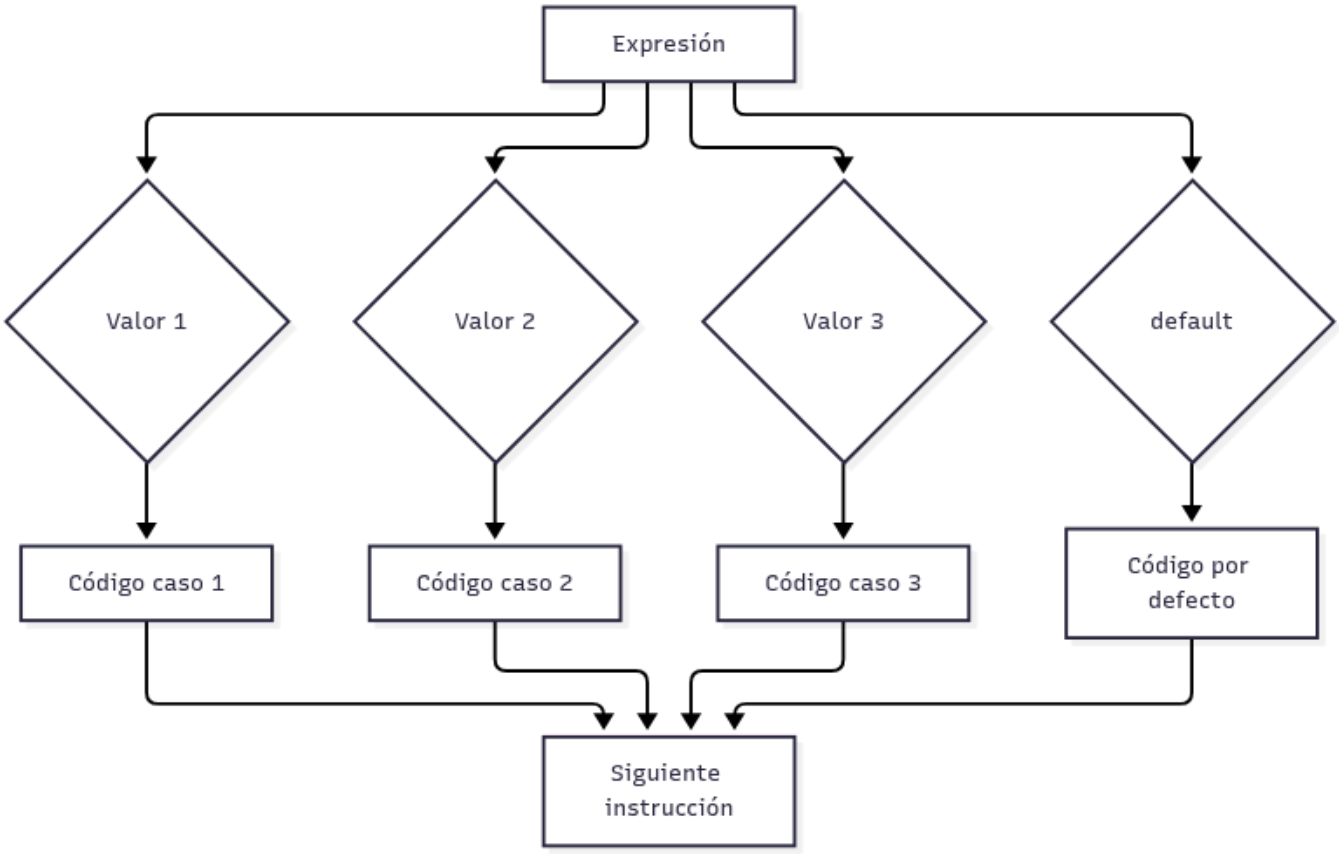
En las novedades de Java 12 se añadió la posibilidad de los switch fueran expresiones que retornan un valor en vez de sentencias y se evita el uso de la palabra reservada break.

Consulta el Ejemplo04 con la sintaxis de la sentencia Switch mejorada (Java 12).

En Java 13 en vez de únicamente el valor a retornar se permite crear bloques de sentencias para cada rama `case` y retornar el valor con la palabra reservada `yield`. En los bloques de sentencias puede haber algún cálculo más complejo que directamente retornar el valor deseado. Consulta el Ejemplo05 para ver la sintaxis completa.

En resumen, se ha de comparar el valor de una expresión con un conjunto de constantes, si el valor de la expresión coincide con algún valor de dichas constantes, se ejecutarán los bloques de instrucciones asociados a cada una de ellas. Si no existiese coincidencia, se ejecutarían una serie de instrucciones por defecto. Consulta el Ejemplo13 para ver un programa que calcula la nota de un examen utilizando la estructura `switch`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



4.1.4 Estructuras de repetición

Nuestros programas ya son capaces de controlar su ejecución teniendo en cuenta determinadas condiciones, pero aún hemos de aprender un conjunto de estructuras que nos permita repetir una secuencia de instrucciones determinada. La función de estas estructuras es repetir la ejecución de una serie de instrucciones teniendo en cuenta una condición.

A este tipo de estructuras se las denomina estructuras de repetición, estructuras repetitivas, bucles o estructuras iterativas. En Java existen cuatro clases de bucles:

- Bucle `for` (repite para)
- Bucle `for/in` (repite para cada), aka `for each`
- Bucle `while` (repite mientras)
- Bucle `do while` (repite hasta)

Los bucles `for` y `for/in` se consideran bucles controlados por contador. Por el contrario, los bucles `while` y `do...while` se consideran bucles controlados por sucesos.

La utilización de unos bucles u otros para solucionar un problema dependerá en gran medida de las siguientes preguntas:

- ¿Sabemos a priori cuántas veces necesitamos repetir un conjunto de instrucciones?
- ¿Sabemos si hemos de repetir un conjunto de instrucciones si una condición satisface un conjunto de valores?
- ¿Sabemos hasta cuándo debemos estar repitiendo un conjunto de instrucciones?
- ¿Sabemos si hemos de estar repitiendo un conjunto de instrucciones mientras se cumpla una condición?

Estas y otras preguntas tendrán su respuesta en cuanto analicemos cada una de estructuras repetitivas en detalle.

Definición

Estudia cada tipo de estructura repetitiva, conoce su funcionamiento y podrás llegar a la conclusión de que algunos de estos bucles son equivalentes entre sí. Un mismo problema, podrá ser resuelto empleando diferentes tipos de bucles y obtener los mismos resultados.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Estructura `for`

Hemos indicado anteriormente que el bucle `for` es un bucle controlado por contador. Este tipo de bucle tiene las siguientes características:

- Se ejecuta un número determinado de veces.
- Utiliza una variable contadora que controla las iteraciones del bucle.

En general, existen tres operaciones que se llevan a cabo en este tipo de bucles:

- Se inicializa la variable contadora.
- Se evalúa el valor de la variable contador, por medio de una comparación de su valor con el número de iteraciones especificado.
- Se modifica o actualiza el valor del contador a través de incrementos o decrementos de éste, en cada una de las iteraciones.

Ejemplo

La inicialización de la variable contadora debe realizase correctamente para garantizar que el bucle lleve a cabo, al menos, la primera repetición de su código interno. La condición de terminación del bucle debe variar en el interior del mismo, de no ser así, podemos caer en la creación de un bucle infinito. Cuestión que se debe evitar por todos los medios. Es necesario estudiar el número de veces que se repite el bucle, pues debe ajustarse al número de veces estipulado.

Sintaxis estructura `for` con una única sentencia:

```
for (inicialización; condición; iteración)
    sentencia;
```

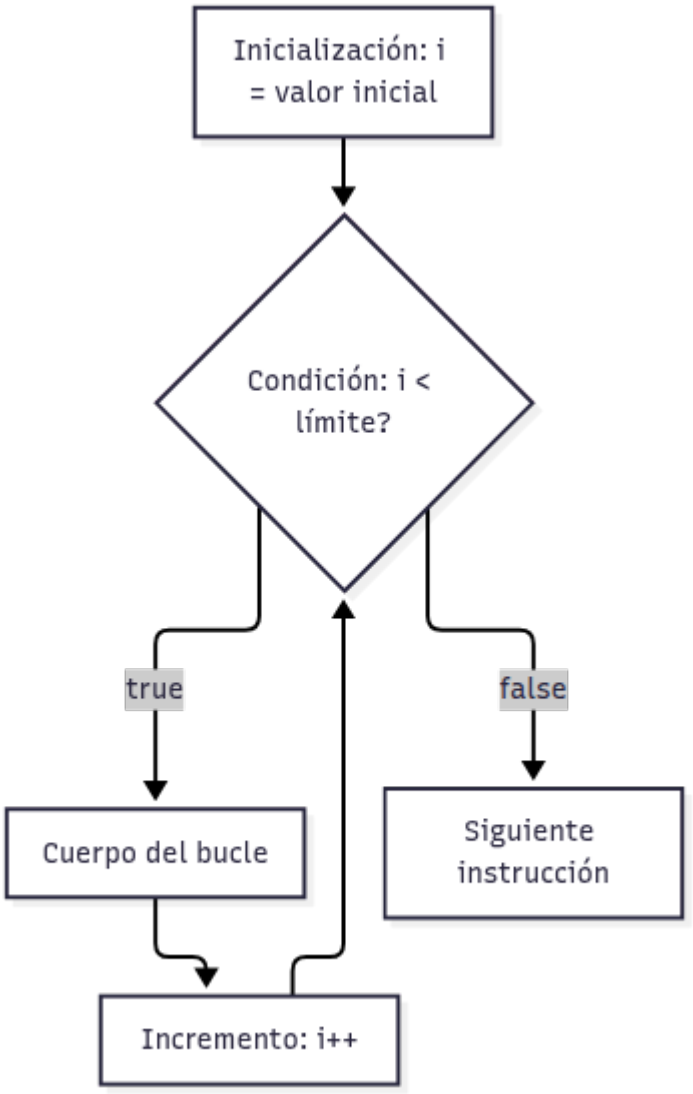
Sintaxis estructura `for` con un bloque de sentencias:

```
for (inicialización; condición; iteración) {
    sentencia1;
    sentencia2;
    ...
    sentenciaN;
}
```

Donde...:

- `inicialización` es una expresión en la que se inicializa una variable de control, que será la encargada de controlar el final del bucle.
- `condición` es una expresión que evaluará la variable de control. Mientras la condición sea falsa, el cuerpo del bucle estará repitiéndose. Cuando la condición se cumpla, terminará la ejecución del bucle.
- `iteración` indica la manera en la que la variable de control va cambiando en cada iteración del bucle. Podrá ser mediante incremento o decremento, y no solo de uno en uno.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Consulta el Ejemplo14 para ver un ejemplo completo del bucle `for` que imprime la tabla de multiplicar del siete.

Estructura `for` / `in`

Junto a la estructura `for`, `for` / `in` también se considera un bucle controlado por contador. Este bucle es una mejora incorporada en la versión 5.0 de Java.

Este tipo de bucles permite realizar recorridos sobre arrays y colecciones de objetos. Los arrays son colecciones de variables que tienen el mismo tipo y se referencian por un nombre común. Así mismo, las colecciones de objetos son objetos que se dice son iterables, o que se puede iterar sobre ellos.

Este bucle es nombrado también como bucle `for` mejorado, o bucle `foreach`. En otros lenguajes de programación existen bucles muy parecidos a este.

La sintaxis `for` / `in` es la siguiente:

```
for (declaración: expresión) {
    sentencia1;
    ...
    sentenciaN;
}
```

Donde...:

- `expresion` es un array o una colección de objetos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- `declaración` es la declaración de una variable cuyo tipo sea compatible con expresión. Normalmente, será el tipo y el nombre de la variable a declarar.

El funcionamiento consiste en que para cada elemento de la expresión, guarda el elemento en la variable declarada y haz las instrucciones contenidas en el bucle. Después, en cada una de las iteraciones del bucle tendremos en la variable declarada el elemento actual de la expresión. Por tanto, para el caso de los arrays y de las colecciones de objetos, se recorrerá desde el primer elemento que los forma hasta el último.

Observa el contenido del código representado en la siguiente imagen, puedes apreciar cómo se construye un bucle de este tipo y su utilización sobre un array.

Los bucles `for` / `in` permitirán al programador despreocuparse del número de veces que se ha de iterar, pero no sabremos en qué iteración nos encontramos salvo que se añada artificialmente alguna variable contadora que nos pueda ofrecer esta información.

> **Información**
>
> Esta estructura tomará sentido cuando avancemos en el curso y veamos los Arrays y las colecciones de Objetos.

Estructura `while`

El bucle `while` es la primera de las estructuras de repetición controladas por sucesos que vamos a estudiar. La utilización de este bucle responde al planteamiento de la siguiente pregunta: ¿Qué podemos hacer si lo único que sabemos es que se han de repetir un conjunto de instrucciones mientras se cumpla una determinada condición?.

La característica fundamental de este tipo de estructura repetitiva estriba en ser útil en aquellos casos en los que las instrucciones que forman el cuerpo del bucle podría ser necesario ejecutarlas o no. Es decir, en el bucle `while` siempre se evaluará la condición que lo controla, y si dicha condición es cierta, el cuerpo del bucle se ejecutará una vez, y se seguirá ejecutando mientras la condición sea cierta. Pero si en la evaluación inicial de la condición ésta no es verdadera, el cuerpo del bucle no se ejecutará.

> **Atención**
>
> Es imprescindible que en el interior del bucle `while` se realice alguna acción que modifique la condición que controla la ejecución del mismo, en caso contrario estaríamos ante un bucle infinito.

Sintaxis estructura `while` con una única sentencia:

```
while (condición)
    sentencia;
```

Sintaxis estructura `while` con un bloque de sentencias:

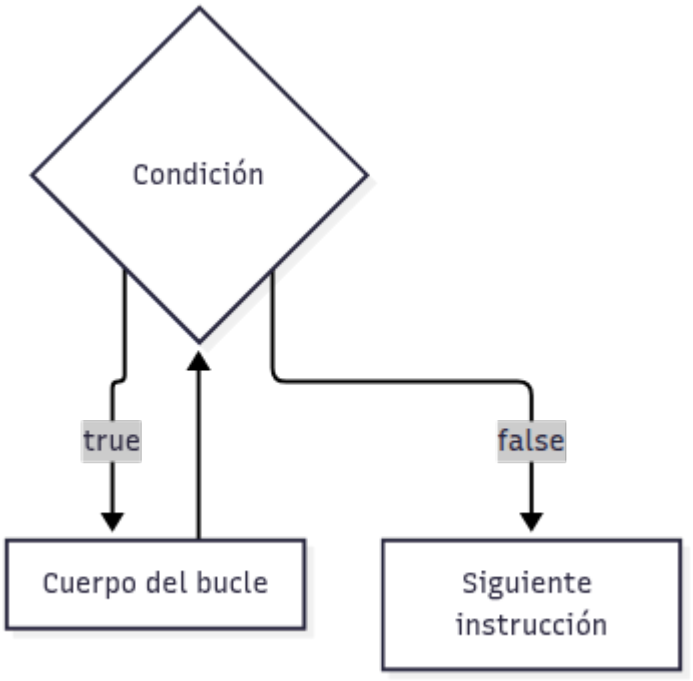
```
while (condición) {
    sentencia1;
    ...
    sentenciaN;
}
```

Funcionamiento: Mientras la condición sea cierta, el bucle se repetirá, ejecutando la/s instrucción/es de su interior.

En el momento en el que la condición no se cumpla, el control del flujo del programa pasará a la siguiente instrucción que exista justo detrás del bucle `while`.

La condición se evaluará siempre al principio, y podrá darse el caso de que las instrucciones contenidas en él no lleguen a ejecutarse nunca si no se satisface la condición de partida.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Consulta el Ejemplo15 para ver un ejemplo completo del bucle `while` que imprime la tabla de multiplicar del siete.

Estructura `do while`

La segunda de las estructuras repetitivas controladas por sucesos es `do while`. En este caso, la pregunta que nos planteamos es la siguiente: ¿Qué podemos hacer si lo único que sabemos es que se han de ejecutar, al menos una vez, un conjunto de instrucciones y seguir repitiéndose hasta que se cumpla una determinada condición?.

La característica fundamental de este tipo de estructura repetitiva estriba en ser útil en aquellos casos en los que las instrucciones que forman el cuerpo del bucle necesitan ser ejecutadas, al menos, una vez y repetir su ejecución hasta que la condición sea verdadera. Por tanto, en esta estructura repetitiva siempre se ejecuta el cuerpo del bucle una primera vez.

Es imprescindible que en el interior del bucle se realice alguna acción que modifique la condición que controla la ejecución del mismo, en caso contrario estaríamos ante un bucle infinito.

Sintaxis estructura `do-while` con una única sentencia:

```
do
    sentencia;
while (condición);
```

Sintaxis estructura `do-while` con un bloque de sentencias:

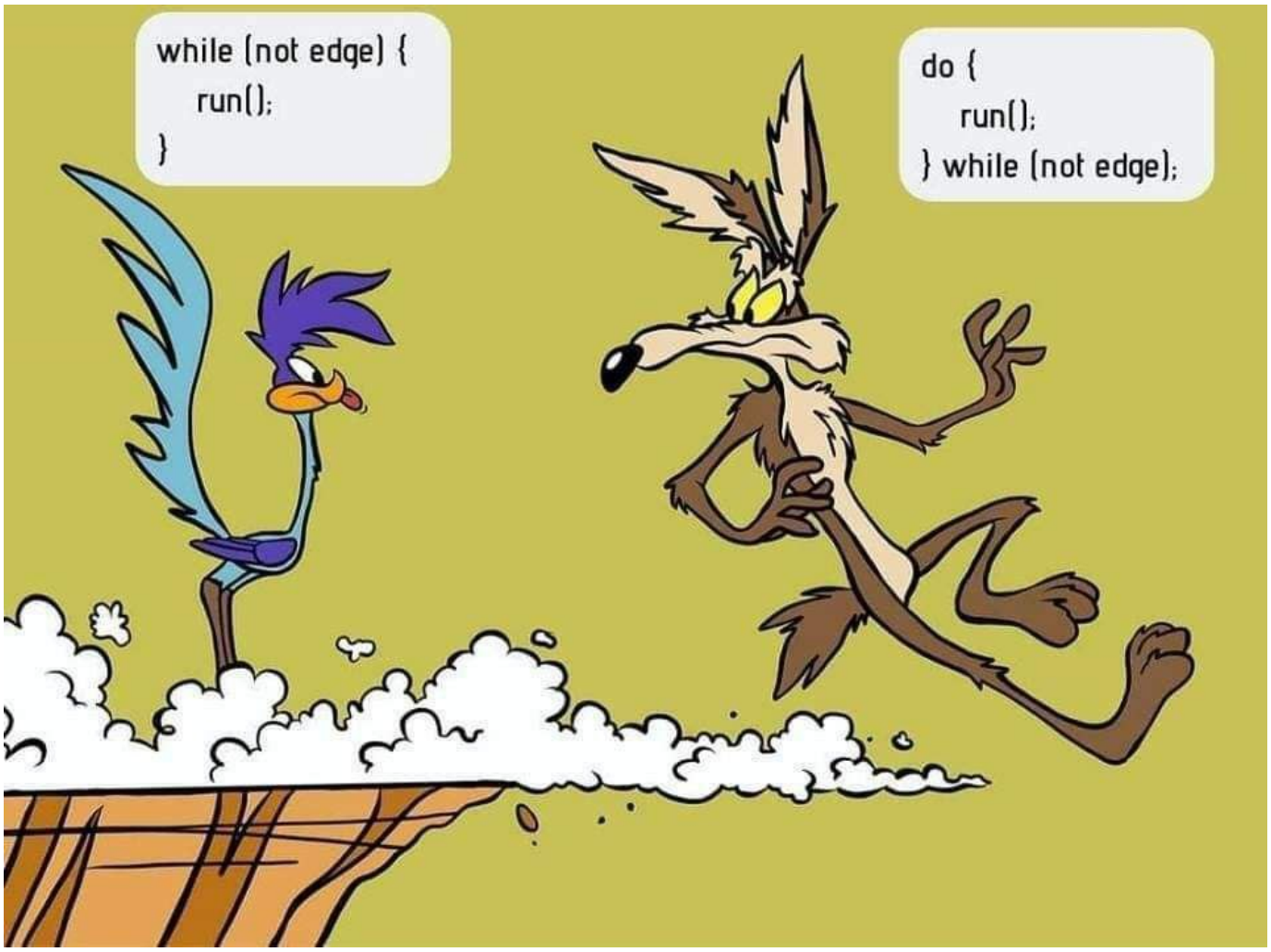
```
do {
    sentencia1;
    ...
    sentenciaN;
} while (condición);
```

Funcionamiento:

El cuerpo del bucle se ejecuta la primera vez, a continuación se evaluará la condición y, si ésta es falsa, el cuerpo el bucle volverá a repetirse. El bucle finalizará cuando la evaluación de la condición sea verdadera.

En ese momento el control del flujo del programa pasará a la siguiente instrucción que exista justo detrás del bucle do-while. La condición se evaluará siempre después de una primera ejecución del cuerpo del bucle, por lo que no se dará el caso de que las instrucciones contenidas en él no lleguen a ejecutarse nunca.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Consulta el Ejemplo16 para ver un ejemplo completo del bucle `do-while` que imprime la tabla de multiplicar del siete.

Bucle infinito

Uno de los errores más comunes al implementar cualquier tipo de bucle es que nunca pueda salir, es decir, el bucle se ejecuta durante un número infinito de veces.

Podemos provocarlo intencionadamente como en estos dos ejemplos equivalentes:

(NO RECOMENDABLE)

```
for(;;){
    //sentencias
}
```

```
while(true){
    //sentencias
}
```

O sucede cuando la condición falla por alguna razón, como en el Ejemplo06.

Otro inconveniente es que puede estar agregando algo en su objeto de colección a través de un bucle y puede **quedarse sin memoria**. Si intenta ejecutar el siguiente programa, después de un tiempo, se producirá una excepción de falta de memoria. En este ejemplo se hace uso de la colección ArrayList, pero de momento solo necesitamos saber que se comporta como un casillero al que vamos asignando elementos (que evidentemente ocupan memoria)

Consulta el Ejemplo07 que provoca una excepción de falta de memoria.

Salida:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
at java.util.Arrays.copyOf(Unknown Source)
at java.util.Arrays.copyOf(Unknown Source)
at java.util.ArrayList.grow(Unknown Source)
at java.util.ArrayList.ensureCapacityInternal(Unknown Source)
at java.util.ArrayList.add(Unknown Source)
at article.Integer1.main(Integer1.java:9)
```

4.1.5 Estructuras de salto

¿Saltar o no saltar? he ahí la cuestión. En la gran mayoría de libros de programación y publicaciones de Internet, siempre se nos recomienda que prescindamos de sentencias de salto incondicional, es más, se desaconseja su uso por provocar una mala estructuración del código y un incremento en la dificultad para el mantenimiento de los mismos. Pero Java incorpora ciertas sentencias o estructuras de salto que es necesario conocer y que pueden sernos útiles en algunas partes de nuestros programas.

Estas estructuras de salto corresponden a las sentencias `break`, `continue`, las etiquetas de salto y la sentencia `return`. Pasamos ahora a analizar su sintaxis y funcionamiento.

Sentencias `break` y `continue`

Se trata de dos instrucciones que permiten modificar el comportamiento de otras estructuras o sentencias de control, simplemente por el hecho de estar incluidas en algún punto de su secuencia de instrucciones.

La sentencia `break` incidirá sobre las estructuras de control `switch`, `while`, `for` y `do while` del siguiente modo:

- Si aparece una sentencia `break` dentro de la secuencia de instrucciones de cualquiera de las estructuras mencionadas anteriormente, dicha estructura terminará inmediatamente.
- Si aparece una sentencia `break` dentro de un bucle anidado sólo finalizará la sentencia de iteración más interna, el resto se ejecuta de forma normal.

Es decir, que `break` sirve para romper el flujo de control de un bucle, aunque no se haya cumplido la condición del bucle. Si colocamos un `break` dentro del código de un bucle, cuando se alcance el `break`, automáticamente se saldrá del bucle pasando a ejecutarse la siguiente instrucción inmediatamente después de él. Consulta el Ejemplo17 para ver un ejemplo de uso de `break`.

La sentencia `continue` incidirá sobre las sentencias o estructuras de control `while`, `for` y `do while` del siguiente modo:

- Si aparece una sentencia `continue` dentro de la secuencia de instrucciones de cualquiera de las sentencias anteriormente indicadas, dicha sentencia dará por terminada la iteración actual y se ejecuta una nueva iteración, evaluando de nuevo la expresión condicional del bucle.
- Si aparece en el interior de un bucle anidado solo afectará a la sentencia de iteración más interna, el resto se ejecutaría de forma normal.

Es decir, la sentencia `continue` forzará a que se ejecute la siguiente iteración del bucle, sin tener en cuenta las instrucciones que pudiera haber después del `continue`, y hasta el final del código del bucle. Consulta el Ejemplo18 para ver un ejemplo de uso de `continue`.

Etiquetas de salto

> **Atención**
>
> Los saltos incondicionales y en especial, saltos a una etiqueta son totalmente **desaconsejables**.

Java permite asociar etiquetas cuando se va a realizar un salto. De este modo puede conseguirse algo más de legibilidad en el código.

Las estructuras de salto `break` y `continue`, pueden tener asociadas etiquetas. Es a lo que se llama un `break` etiquetado o un `continue` etiquetado. Pero sólo se recomienda su uso cuando se hace necesario salir de bucles anidados hacia diferentes niveles. ¿Y cómo se crea un salto a una etiqueta? En primer lugar, crearemos la etiqueta mediante un identificador seguido de dos puntos (`:`). A continuación, se escriben las sentencias Java asociadas a dicha etiqueta encerradas entre llaves. Por así decirlo, la creación de una etiqueta es como fijar un punto de salto en el programa para poder saltar a él desde otro lugar de dicho programa.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

¿Cómo se lleva a cabo el salto? Es sencillo, en el lugar donde vayamos a colocar la sentencia `break` o `continue`, añadiremos detrás el identificador de la etiqueta. Con ello, conseguiremos que el salto se realice a un lugar determinado.

La sintaxis será:

```
1  break <etiqueta>;
```

> **Curiosidad**
>
> Quizá a aquellos/as que han programado en HTML les suene esta herramienta, ya que tiene cierta similitud con las anclas que pueden crearse en el interior de una página web, a las que nos llevará el hiperenlace o link que hayamos asociado. También para aquellos/as que han creado alguna vez archivos por lotes o archivos batch bajo MSDOS es probable que también les resulte familiar el uso de etiquetas, pues la sentencia GOTO que se utilizaba en este tipo de archivos, hacía saltar el flujo del programa al lugar donde se ubicaba la etiqueta que se indicara en dicha sentencia. Consulta el Ejemplo19 para ver un ejemplo de declaración y uso de etiquetas de salto.

`return`

Ya sabemos cómo modificar la ejecución de bucles y estructuras condicionales múltiples, pero ¿Podríamos modificar la ejecución de un método? ¿Es posible hacer que éstos detengan su ejecución antes de que finalice el código asociado a ellos?. Sí es posible, a través de la sentencia return podremos conseguirlo. La sentencia `return` puede utilizarse de dos formas:

- Para terminar la ejecución del método donde esté escrita, con lo que transferirá el control al punto desde el que se hizo la llamada al método, continuando el programa por la sentencia inmediatamente posterior.
- Para devolver o retornar un valor, siempre que junto a `return` se incluya una expresión de un tipo determinado. Por tanto, en el lugar donde se invocó al método se obtendrá el valor resultante de la evaluación de la expresión que acompañaba al método.

> **Importante**
>
> En general, una sentencia `return` suele aparecer al final de un método, de este modo el método tendrá una entrada y una salida. También es posible utilizar una sentencia `return` en cualquier punto de un método, con lo que éste finalizará en el lugar donde se encuentre dicho `return`. No será recomendable incluir más de un `return` en un método y por regla general, deberá ir al final del método como hemos comentado.

El valor de retorno es opcional, si lo hubiera debería de ser del mismo tipo o de un tipo compatible al tipo del valor de retorno definido en la cabecera del método, pudiendo ser desde un entero a un objeto creado por nosotros. Si no lo tuviera, el tipo de retorno sería `void`, y `return` serviría para salir del método sin necesidad de llegar a ejecutar todas las instrucciones que se encuentran después del `return`. Consulta el Ejemplo20 para ver un programa que obtiene la suma de dos números empleando `return`.

4.1.6 Excepciones

A lo largo de nuestro aprendizaje de Java nos hemos topado en alguna ocasión con errores, pero éstos suelen ser los que nos ha indicado el compilador. Un punto y coma por aquí, un nombre de variable incorrecto por allá, pueden hacer que nuestro compilador nos avise de estos descuidos.

Cuando los vemos, se corrigen y obtenemos nuestra clase compilada correctamente.

Pero, ¿Sólo existen este tipo de errores? ¿Podrían existir errores no sintácticos en nuestros programas?. Está claro que sí, un programa perfectamente compilado en el que no existen errores de sintaxis, puede generar otros tipos de errores que quizá aparezcan en tiempo de ejecución. A estos errores se les conoce como **excepciones**.

Aprenderemos a gestionar de manera adecuada estas excepciones y tendremos la oportunidad de utilizar el potente sistema de manejo de errores que Java incorpora. La potencia de este sistema de manejo de errores radica en:

1. Que el código que se encarga de manejar los errores, es perfectamente identificable en los programas. Este código puede estar separado del código que maneja la aplicación.
2. Que Java tiene una gran cantidad de errores estándar asociados a multitud de fallos comunes, como por ejemplo divisiones por cero, fallos de entrada de datos, etc. Al tener tantas excepciones localizadas, podemos gestionar de manera específica cada uno de los errores que se produzcan.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

En Java se pueden preparar los fragmentos de código que pueden provocar errores de ejecución para que si se produce una excepción, el flujo del programa es lanzado (`throw`) hacia ciertas zonas o rutinas que han sido creadas previamente por el programador y cuya finalidad será el tratamiento efectivo de dichas excepciones. Si no se captura la excepción, el programa se detendrá con toda probabilidad.

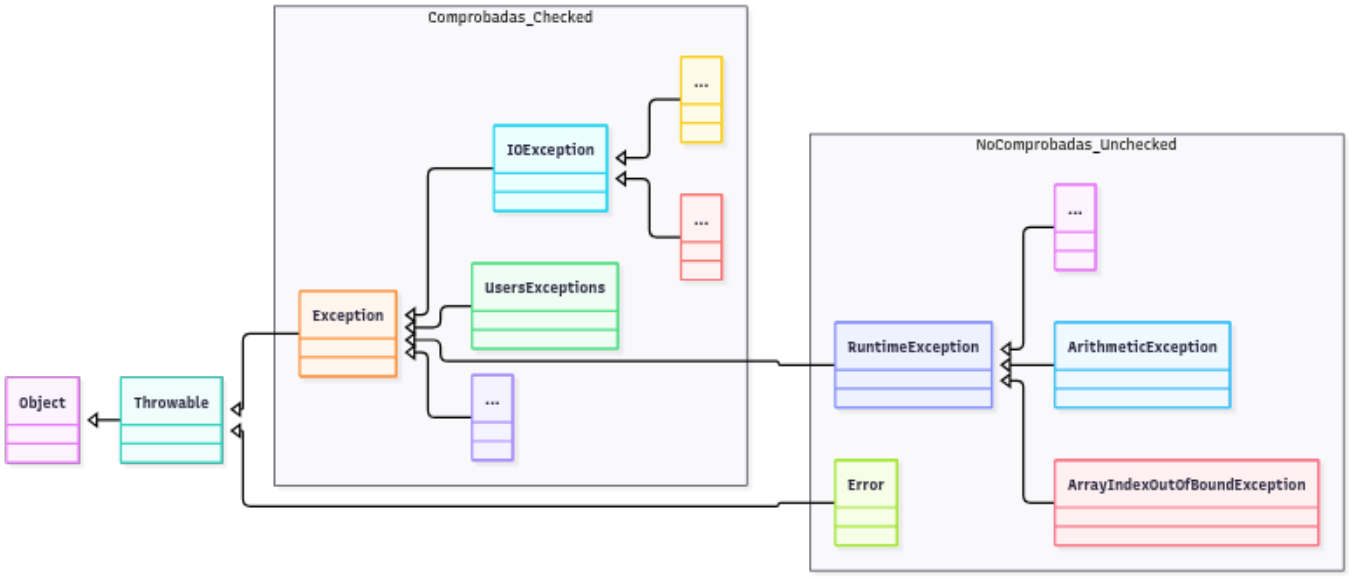
En Java, las excepciones están representadas por clases. El paquete `java.lang.Exception` y sus subpaquetes contienen todos los tipos de excepciones. Todas las excepciones derivarán de la clase `Throwable` , existiendo clases más específicas. Por debajo de la clase `Throwable` existen las clases `Error` y `Exception` . Error es una clase que se encargará de los errores que se produzcan en la máquina virtual, no en nuestros programas. Y la clase `Exception` será la que a nosotros nos interese conocer, pues gestiona los errores provocados en los programas.

Java lanzará una excepción en respuesta a una situación poco usual. Cuando se produce un error se genera un objeto asociado a esa excepción. Este objeto es de la clase `Exception` o de alguna de sus herederas. Este objeto se pasa al código que se ha definido para manejar la excepción. Dicho código puede manipular las propiedades del objeto `Exception` .

El programador también puede lanzar sus propias excepciones. Las excepciones en Java serán objetos de clases derivadas de la clase base `Exception` . Existe toda una jerarquía de clases derivada de la clase base `Exception` . Estas clases derivadas se ubican en dos grupos principales:

- Las excepciones en tiempo de ejecución, que ocurren cuando el programador no ha tenido cuidado al escribir su código.
- Las excepciones que indican que ha sucedido algo inesperado o fuera de control.

En la siguiente imagen te ofrecemos una aproximación a la jerarquía de las excepciones en Java.



Y aquí tenemos una lista de las más habituales con su explicación:

NOMBRE	DESCRIPCIÓN
FileNotFoundException	Lanza una excepción cuando el fichero no se encuentra.
ClassNotFoundException	Lanza una excepción cuando no existe la clase.
EOFException	Lanza una excepción cuando llega al final del fichero.
ArrayIndexOutOfBoundsException	Lanza una excepción cuando se accede a una posición de un array que no exista.
NumberFormatException	Lanza una excepción cuando se procesa un numero pero este es un dato alfanumérico.
NullPointerException	Lanza una excepción cuando intentando acceder a un miembro de un objeto para el que todavía no hemos reservado memoria.
IOException	Generaliza muchas excepciones anteriores. La ventaja es que no necesitamos controlar cada una de las excepciones.
Exception	Es la clase padre de IOException y de otras clases. Tiene la misma ventaja que IOException.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

NOMBRE	DESCRIPCIÓN
ArithmeticException	Se lanza por ejemplo, cuando intentamos dividir un número entre cero.

El manejo de excepciones

Como hemos comentado, siempre debemos controlar las excepciones que se puedan producir o de lo contrario nuestro software quedará expuesto a fallos. Las excepciones pueden tratarse de dos formas:

- **Interrupción**. En este caso se asume que el programa ha encontrado un error irrecuperable. La operación que dio lugar a la excepción se anula y se entiende que no hay manera de regresar al código que provocó la excepción. Es decir, la operación que dio originó el error, se anula.
- **Reanudación**. Se puede manejar el error y regresar de nuevo al código que provocó el error.

Java emplea la primera forma, pero puede simularse la segunda mediante la utilización de un bloque `try` en el interior de un `while`, que se repetirá hasta que el error deje de existir. Consulta el Ejemplo21 para ver cómo poner un `try-catch` dentro de un `do-while`.

Capturar una excepción

Para poder capturar excepciones, emplearemos la estructura de captura de excepciones `try-catch-finally`.

Básicamente, para capturar una excepción lo que haremos será declarar bloques de código donde es posible que ocurra una excepción. Esto lo haremos mediante un bloque `try` (intentar). Si ocurre una excepción dentro de estos bloques, se lanza una excepción. Estas excepciones lanzadas se pueden capturar por medio de bloques `catch`. Será dentro de este tipo de bloques donde se hará el manejo de las excepciones.

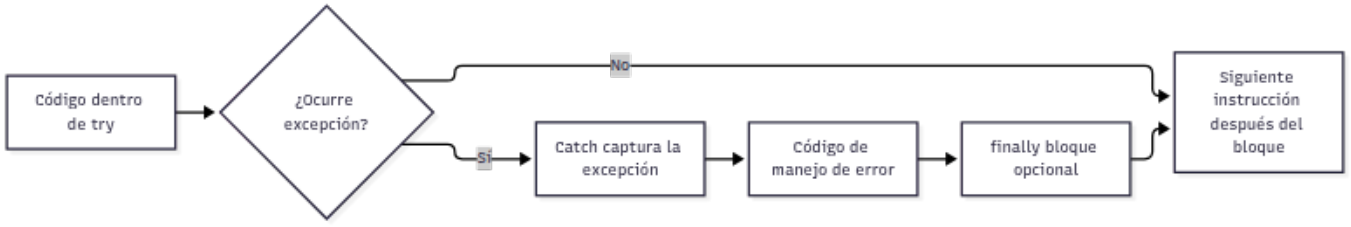
Sintaxis `try-catch`:

```
try {
    //código que puede generar excepciones;
} catch (Tipo_excepcion_1 objeto_excepcion) {
    //Manejo de excepción de Tipo_excepcion_1;
} catch (Tipo_excepcion_2 objeto_excepcion) {
    //Manejo de excepción de Tipo_excepcion_2;
}
...
finally {
    //instrucciones que se ejecutan siempre
}
```

En esta estructura, la parte `catch` puede repetirse tantas veces como excepciones diferentes se deseen capturar. La parte `finally` es opcional y, si aparece, solo podrá hacerlo una vez.

Cada `catch` maneja un tipo de excepción. Cuando se produce una excepción, se busca el `catch` que posea el manejador de excepción adecuado, será el que utilice el mismo tipo de excepción que se ha producido. Esto puede causar problemas si no se tiene cuidado, ya que la clase `Exception` es la superclase de todas las demás. Por lo que si se produjo, por ejemplo, una excepción de tipo `Aritmethic Exception` y el primer `catch` captura el tipo genérico `Exception`, será ese `catch` el que se ejecute y no los demás.

Por eso el último `catch` debe ser el que capture excepciones genéricas y los primeros deben ser los más específicos. Lógicamente si vamos a tratar a todas las excepciones (sean del tipo que sean) igual, entonces basta con un solo `catch` que capture objetos `Exception`.



© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Recuerda

En Java, cuando un bloque de código puede provocar una excepción pero no se maneja adecuadamente, se produce lo que se conoce como una "excepción no controlada" o "excepción no capturada". Cuando ocurre una excepción no controlada, Java sigue un conjunto de reglas específicas para manejarla:

1. **Propagación de excepciones**: Java busca en la pila de llamadas (el seguimiento de la ejecución del programa) para ver si el método actual maneja la excepción. Si el método actual no maneja la excepción, la excepción se "propaga" hacia arriba en la pila de llamadas. (Piensa en una burbuja de aire en el fondo del mar intentando buscar una salida)
2. **Búsqueda de un manejador de excepciones**: La excepción propagada continúa buscando un manejador de excepciones adecuado a medida que se retrocede a través de los métodos que llamaron al método actual. Si se encuentra un bloque `try-catch` que puede manejar la excepción, se ejecutará el código del bloque `catch` correspondiente.
3. **Si no se encuentra un manejador adecuado**: Si la excepción llega a la parte superior de la pila de llamadas y no se encuentra un manejador de excepciones adecuado, el programa se detendrá y se imprimirá un mensaje de error en la consola, que contiene información sobre la excepción, como su tipo, mensaje y seguimiento de pila (`stack trace`).

Delegación de excepciones con `throws`

¿Puede haber problemas con las excepciones al usar llamadas a métodos en nuestros programas? Efectivamente, si se produjese una excepción es necesario saber quién será el encargado de solucionarla. Puede ser que sea el propio método llamado o el código que hizo la llamada a dicho método.

Quizá pudiéramos pensar que debería ser el propio método el que se encargue de sus excepciones, aunque es posible hacer que la excepción sea resuelta por el código que hizo la llamada. Cuando un método utiliza una sentencia que puede generar una excepción, pero dicha excepción no es capturada y tratada por él, sino que se encarga su gestión a quién llamó al método, decimos que se ha producido delegación de excepciones.

Para establecer esta delegación, en la cabecera del método se declara el tipo de excepciones que puede generar y que deberán ser gestionadas por quien invoque a dicho método. Utilizaremos para ello la sentencia `throws` y tras esa palabra se indica qué excepciones puede provocar el código del método. Si ocurre una excepción en el método, el código abandona ese método y regresa al código desde el que se llamó al método. Allí se buscará el `catch` apropiado para esa excepción. Su sintaxis es la siguiente:

Sintaxis `throws`:

```
public class Delegacion_Excepciones {
    ...
    public int leeAnio(BufferedReader lector) throws IOException, NumberFormatException{
        String linea = teclado.readLine();
        return Integer.parseInt(linea);
    }
    ...
}
```

Donde `IOException` y `NumberFormatException`, serían dos posibles excepciones que el método `leeAnio` podría generar, pero que no gestiona. Por tanto, un método puede incluir en su cabecera un listado de excepciones que puede lanzar, separadas por comas. Consulta el Ejemplo22 para ver cómo se propaga una excepción hacia arriba en la pila de ejecución.

Crear y lanzar excepciones de usuario

Las excepciones de usuario son subclases de la clase `Exception` que podemos crear y lanzar en nuestros programas para avisar sobre determinadas situaciones.

CREAR UNA NUEVA EXCEPCIÓN

Para crear una nueva excepción tenemos que crear una clase derivada (subclase) de la clase `Exception`.

La clase `Exception` tiene dos constructores, uno sin parámetros y otro que acepta un `String` con un texto descriptivo de la excepción. Todas las excepciones de usuario las crearemos de la siguiente forma:

Constructores de Exception:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
class NombreExcepcion extends Exception {
    public NombreExcepcion(){
        super();
    }
    public NombreExcepcion(String msg){
        super(msg);
    }
}
```

LANZAR UNA EXCEPCIÓN

Las excepciones se lanzan mediante la instrucción `throw`. La sintaxis es:

Instrucción `throw`:

```java
throw new NombreExcepcion("Mensaje descriptivo de la situación inesperada");
```

Ya que se tratará de una excepción comprobada, en la cabecera del método que lanza la excepción habrá que propagarla. Consulta el Ejemplo23 para ver una excepción personalizada completa.

Excepciones `Checked` y `unChecked`

En Java, las excepciones se dividen en dos categorías principales: excepciones "checked" (comprobadas) y excepciones "unchecked" (no comprobadas).

1. **Excepciones Comprobadas (Checked Exceptions)**: - Las excepciones comprobadas son aquellas que el compilador obliga a manejar. Esto significa que, si un método puede lanzar una excepción comprobada, el programador está obligado a manejarla de alguna manera, ya sea mediante la declaración del método con `throws` o mediante el manejo directo con un bloque `try-catch`. - Ejemplos de excepciones comprobadas incluyen `IOException` y `SQLException`. - Estas excepciones suelen representar situaciones en las que un programa no puede continuar normalmente y se espera que el código las maneje de manera adecuada.

Consulta el Ejemplo08 con un ejemplo de excepción comprobada.

1. **Excepciones No Comprobadas (Unchecked Exceptions):** - Las excepciones no comprobadas son aquellas que el compilador no requiere que se manejen explícitamente. Normalmente, son subclases de `RuntimeException`. - Estas excepciones suelen deberse a errores de programación, como acceder a un índice fuera de los límites de un array (`ArrayIndexOutOfBoundsException`) o intentar convertir un objeto a un tipo incompatible (`ClassCastException`). - Aunque no se requiere que se manejen explícitamente, es buena práctica manejarlas para evitar que el programa termine abruptamente.

Consulta el Ejemplo09 con un ejemplo de excepción no comprobada.

¿COMO SÉ SI UNA EXCEPCIÓN ES DE UN TIPO O DE OTRO?

La principal diferencia radica en la obligación del compilador de manejar o declarar excepciones. Las excepciones comprobadas deben ser manejadas o declaradas en el código, mientras que las excepciones no comprobadas no tienen esta obligación y generalmente se deben a errores de programación.

En Java, puedes distinguir entre excepciones comprobadas y no comprobadas principalmente por el tipo de clase que heredan. Aquí hay algunas pautas generales:

1. **Excepciones Comprobadas (Checked Exceptions):** - Las excepciones comprobadas suelen ser subclases directas de la clase `Exception` (o alguna de sus subclases), pero no heredan de `RuntimeException` ni de sus subclases. - Ejemplos comunes incluyen `IOException`, `SQLException`, y cualquier excepción que herede directamente de `Exception` (pero no de `RuntimeException`).
2. **Excepciones No Comprobadas (Unchecked Exceptions):** - Las excepciones no comprobadas suelen ser subclases directas de la clase `RuntimeException`. - Ejemplos comunes incluyen `NullPointerException`, `ArrayIndexOutOfBoundsException`, y cualquier excepción que herede directamente de `RuntimeException`.

> ⚠ **Importante**
>
> Ten en cuenta que estas son pautas generales y puede haber excepciones personalizadas o situaciones específicas en las que estas reglas no se apliquen estrictamente. Para obtener información precisa sobre un tipo de excepción específico, puedes consultar la documentación de Java o examinar la jerarquía de clases y herencia de la excepción en cuestión.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

4.1.7 Aserciones (`Assertions`)

Una aserción (afirmación) permite probar la exactitud de cualquier suposición que se haya hecho en el programa. Una afirmación se logra utilizando la declaración de `assertion` en Java. Al ejecutar una aserción, se cree que es cierta. Si falla, JVM genera un error denominado `AssertionError`. Se utiliza principalmente con fines de prueba durante el desarrollo.

La declaración de afirmación se usa con una expresión booleana y se puede escribir de dos maneras diferentes.

Primera forma:

```
assert expression;
```

Segunda forma:

```
assert expression1 : expression2;
```

Consulta el Ejemplo10 con un ejemplo completo de assertions.

Salida sin assertions:

```
Introduce tu edad: 14
La edad del votante es de 14
```

Después de habilitar las assertions:

> **Saber más...**
>
> Puedes habilitar las assertions añadiendo los parámetros de la JVM en IntelliJ:
>
> - `-ea`: Enable Assertions (habilitar aserciones)
> - `-da`: Disable Assertions (deshabilitar aserciones, que es la opción por defecto)
>
> Puedes consultar este enlace para saber donde agregar estas opciones: https://stackoverflow.com/questions/68848158/java-assertions-in-intellij-idea-community

Salida:

```
Introduce tu edad: 14
Exception in thread "main" java.lang.AssertionError: No puede votar
    at UD03.P7_Assertions.main(P7_Assertions.java:11)
```

Consulta el Ejemplo11 con otro ejemplo de assertions.

Ejecución sin aserciones:

```
Probando Aserciones...
```

Y con aserciones:

```
Probando Aserciones...
Exception in thread "main" java.lang.AssertionError: Esto solo lo veremos si activamos las aserciones.
    at UD03.P7_Assertions2.main(P7_Assertions2.java:7)
```

¿Por qué utilizar aserciones?

Dondequiera que un programador quiera ver si sus suposiciones son erróneas o no.

- Para asegurarse de que un código que parece inalcanzable sea realmente inalcanzable.
- Para asegurarse de que las suposiciones escritas en los comentarios sean correctas.
- Para asegurarse de que no se alcance el caso default del switch.
- Para comprobar el estado del objeto.
- Al comienzo del método.
- Después de la invocación del método.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Aserción o Excepciones

Las aserciones se utilizan principalmente para comprobar situaciones lógicamente imposibles. Por ejemplo, se pueden utilizar para comprobar el estado que espera un código antes de empezar a ejecutarse o el estado después de que termine de ejecutarse. A diferencia del manejo normal de excepciones/errores, las aserciones generalmente están deshabilitadas en tiempo de ejecución.

¿Dónde utilizarlas?:

- Argumentos para los métodos privados. Los argumentos para los métodos privados los proporciona únicamente el código del desarrollador y es posible que este desee comprobar sus suposiciones sobre los argumentos.
- Casos condicionales.
- Condiciones al inicio de cualquier método.

¿Dónde **NO** utilizar aserciones?:

- Las aserciones no deben usarse para reemplazar mensajes de error
- Las aserciones no deben usarse para verificar argumentos en los métodos públicos, ya que pueden ser proporcionados por el usuario.
- Para manejar los errores proporcionados por los usuarios usaremos las excepciones.
- Las aserciones no deben usarse en argumentos de línea de comando.

4.1.8 Ejemplos UD03

{: #ejemplo01 }

Ejemplo01

Organización secuencial de sentencias.

```java
package organizacion_sentencias1;
/**
*
* Organización de sentencias secuencial
*/
public class Organizacion_sentencias_1 {
    public static void main(String[] args) {
        System.out.println ("Organización secuencial de sentencias");
        int dia=12;
        System.out.println ("El día es: " + dia);
        int mes=11;
        System.out.println ("El mes es: " + mes);
        int anio=2011;
        System.out.println ("El anio es: " + anio);
    }
}
```

⬆ Volver a teoría

Ejemplo02

Declaración previa de variables.

```java
package organizacion_sentencias2;
/**
*
* Organización de sentencias con declaración previa de variables
*/
public class Organizacion_sentencias_2 {
    public static void main(String[] args) {
        // Zona de declaración de variables
        int dia=10;
        int mes=11;
        int anio=2011;
        System.out.println ("Organización con declaración previa de variables");
        System.out.println ("El día es: " + dia);
        System.out.println ("El mes es: " + mes);
        System.out.println ("El año es: " + anio);
    }
}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

⬆ Volver a teoría

Ejemplo03

Organización del código en zonas diferenciadas.

```java
package organizacion_sentencias3;
/**
*
* Organización de sentencias en zonas diferenciadas
* según las operaciones que se realicen en el código
*/
public class Organizacion_sentencias_3 {
    public static void main(String[] args) {
        // Zona de declaración de variables
        int dia;
        int mes;
        int anio;
        String fecha;
        //Zona de inicialización o entrada de datos
        dia=10;
        mes=11;
        anio=2011;
        fecha="";
        //Zona de procesamiento
        fecha=dia+"/"+mes+"/"+anio;
        //Zona de salida
        System.out.println ("Organización con zonas diferenciadas en el código");
        System.out.println ("La fecha es: " + fecha);
    }
}
```

⬆ Volver a teoría

Ejemplo04

Sintaxis de la sentencia switch mejorada (Java 12).

```java
int entero = 5;

String numericString = switch (entero) {
    case 0 -> "cero";
    case 1, 3, 5, 7, 9 -> "impar";
    case 2, 4, 6, 8, 10 -> "par";
    default -> "error";
};
System.out.println(numericString); //impar
```

⬆ Volver a teoría

Ejemplo05

Sintaxis de la sentencia switch con yield (Java 13).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
int entero2 = 4;

String numericString2 = switch (entero2) {
    case 0 -> {
        String value = calculaCero();
        yield value;
    }
    case 1, 3, 5, 7, 9 -> {
        String value = calculaImpar();
        yield value;
    }

    case 2, 4, 6, 8, 10 -> {
        String value = calculaPar();
        yield value;
    }

    default -> {
        String value = calculaDefecto();
        yield value;
    }
};
System.out.println(numericString); //calculaPar()
```

⬆ Volver a teoría

Ejemplo06

Ilustración de bucles infinitos.

```java
//Programa Java para ilustrar varias trampas de bucles.
public class BucleInfinito{

    public static void main(String[] args)
    {
        // bucle infinito porque la condición no es apta
        // la condición; debería haber sido i>0.
        for (int i = 5; i != 0; i -= 2){
            System.out.println(i);
        }

        int x = 5;
        // bucle infinito porque la actualización
        // no se proporciona
        while (x == 5)
        {
            System.out.println("En el bucle");
        }
    }
}
```

⬆ Volver a teoría

Ejemplo07

Provocar una excepción OutOfMemoryError.

```java
//Programa Java para la excepción de falta de memoria.
import java.util.ArrayList;
public class HeapSpace
{
    public static void main(String[] args)
    {
        ArrayList<Integer> ar = new ArrayList<>();
        for (int i = 0; i < Integer.MAX_VALUE; i++)
        {
            ar.add(i);
        }
    }
}
```

⬆ Volver a teoría

Ejemplo08

Ejemplo de excepción comprobada (Checked Exception).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
import java.io.FileReader;
import java.io.FileNotFoundException;

public class EjemploCheckedException {
    public static void main(String[] args) {
        try {
            FileReader file = new FileReader("archivo.txt");
        } catch (FileNotFoundException e) {
            System.out.println("Archivo no encontrado: " + e.getMessage());
        }
    }
}
```

⬆ Volver a teoría

Ejemplo09

Ejemplo de excepción no comprobada (Unchecked Exception).

```java
public class EjemploUncheckedException {
    public static void main(String[] args) {
        int[] numeros = {1, 2, 3};
        System.out.println(numeros[4]);  // Esto lanzará ArrayIndexOutOfBoundsException
    }
}
```

⬆ Volver a teoría

Ejemplo10

Uso de aserciones para validar la edad.

```java
import java.util.Scanner;

public class P7_Assertions {
    // Programa Java para demostrar el uso de las assertions
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        System.out.print("Introduce tu edad: ");
        int age = entrada.nextInt();
        assert (age >= 18): "No puede votar";
        System.out.println("La edad del votante es de " + age);
    }
}
```

⬆ Volver a teoría

Ejemplo11

Otro ejemplo de aserciones.

```java
package UD03;

public class P7_Assertions2 {
    public static void main(String[] args) {
        System.out.println("Probando Aserciones...");
        assert true : "Nunca veremos esto.";
        assert false : "Esto solo lo veremos si activamos las aserciones.";
    }
}
```

⬆ Volver a teoría

Ejemplo12

Cálculo de la nota de un examen usando if/else.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Para completar la información que debes saber sobre las estructuras `if` e `if-else`, en este programa se calcula la nota de un examen de tipo test. Además de calcular el valor de la nota, se ofrece como salida la calificación no numérica combinando las diferentes estructuras condicionales aprendidas hasta ahora.

Presta especial atención a los comentarios incorporados en el código fuente, así como a la forma de combinar las estructuras condicionales y a las expresiones lógicas utilizadas en ellas.

```java
package UD03;
public class Sentencias_Condicionales {
    /*Vamos a realizar el cálculo de la nota de un examen
     * de tipo test. Para ello, tendremos en cuenta el número
     * total de pregunta, los aciertos y los errores. Dos errores
     * anulan una respuesta correcta.
     *
     * Finalmente, se muestra por pantalla la nota obtenida, así
     * como su calificación no numérica.
     *
     * La obtención de la calificación no numérica se ha realizado
     * combinando varias estructuras condicionales, mostrando expresiones
     * lógicas compuestas, así como anidamiento.
     */
    public static void main(String[] args) {
        // Declaración e inicialización de variables
        int num_aciertos = 12;
        int num_errores = 3;
        int num_preguntas = 20;
        float nota = 0;
        String calificacion = "";
        //Procesamiento de datos
        nota = ((num_aciertos - (num_errores / 2)) * 10) / num_preguntas;

        if (nota < 5) {
            calificacion = "INSUFICIENTE";
        } else {
            /* Cada expresión lógica de estos if está compuesta por dos
             * expresiones lógicas combinadas a través del operador Y o AND
             * que se representa con el símbolo &&. De tal manera, que para
             * que la expresión lógica se cumpla (sea verdadera) la variable
             * nota debe satisfacer ambas condiciones simultáneamente
             */
            if (nota >= 5 && nota < 6) {
                calificacion = "SUFICIENTE";
            } else if (nota >= 6 && nota < 7) {
                calificacion = "BIEN";
            } else if (nota >= 7 && nota < 9) {
                calificacion = "NOTABLE";
            } else if (nota >= 9 && nota <= 10) {
                calificacion = "SOBRESALIENTE";
            }
        }
        //Salida de información
        System.out.println("La nota obtenida es: " + nota);
        System.out.println("y la calificación obtenida es: " + calificacion);
    }
}
```

⬆ Volver a teoría

Ejemplo13

Cálculo de la nota de un examen usando switch.

Este programa resuelve el cálculo de un examen de tipo test utilizando la estructura `switch`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD03;

public class P3_2_condicional_switch {

    /*
     * Vamos a realizar el cálculo de la nota de un examen de tipo test. Para
     * ello, tendremos en cuenta el número total de preguntas, los aciertos y
     * los errores. Dos errores anulan una respuesta correcta.
     *
     * La nota que vamos a obtener será un número entero.
     *
     * Finalmente, se muestra por pantalla la nota obtenida, así como su
     * calificación no numérica.
     *
     * La obtención de la calificación no numérica se ha realizado utilizando la
     * estructura condicional múltiple o switch.
     *
     */
    public static void main(String[] args) {
        // Declaración e inicialización de variables
        int num_aciertos = 17;
        int num_errores = 3;
        int num_preguntas = 20;
        int nota = 0;
        String calificacion = "";
        //Procesamiento de datos
        nota = ((num_aciertos - (num_errores / 2)) * 10) / num_preguntas;
        switch (nota) {
            case 5:
                calificacion = "SUFICIENTE";
                break;
            case 6:
                calificacion = "BIEN";
                break;
            case 7:
                calificacion = "NOTABLE";
                break;
            case 8:
                calificacion = "NOTABLE";
                break;
            case 9:
                calificacion = "SOBRESALIENTE";
                break;
            case 10:
                calificacion = "SOBRESALIENTE";
                break;
            default:
                calificacion = "INSUFICIENTE";
        }
        //Salida de información
        System.out.println("La nota obtenida es: " + nota);
        System.out.println("y la calificación obtenida es: " + calificacion);
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
        //Expresiones switch mejoradas JAVA 12
        int entero = 5;

        String numericString = switch (entero) {
            case 0 -> "cero";
            case 1, 3, 5, 7, 9 -> "impar";
            case 2, 4, 6, 8, 10 -> "par";
            default -> "error";
        };
        System.out.println(numericString); //impar

        //Expresiones switch mejoradas JAVA 13

        int entero2 = 4;

        String numericString2 = switch (entero2) {
            case 0 -> {
                String value = calculaCero();
                yield value;
            }
            case 1, 3, 5, 7, 9 -> {
                String value = calculaImpar();
                yield value;
            }

            case 2, 4, 6, 8, 10 -> {
                String value = calculaPar();
                yield value;
            }

            default -> {
                String value = calculaDefecto();
                yield value;
            }
        };
        System.out.println(numericString); //calculaPar()
    }
    static String calculaCero() {return "";};
    static String calculaImpar() {return "";};
    static String calculaPar() {return "";};
    static String calculaDefecto() {return "";};
}
```

⬆ Volver a teoría

Ejemplo14

Tabla de multiplicar del siete con el bucle for.

Este ejemplo utiliza el bucle `for` para la impresión por pantalla de la tabla de multiplicar del siete. Lee atentamente los comentarios incluidos en el código, pues aclaran algunas cuestiones interesantes sobre este bucle.

```
package UD03;

public class Repetitiva_For {
    /* En este ejemplo se utiliza la estructura repetitiva for
     * para representar en pantalla la tabla de multiplicar del siete
     */
    public static void main(String[] args) {
        // Declaración e inicialización de variables
        int numero = 7;
        int contador;
        int resultado = 0;
        //Salida de información
        System.out.println("Tabla de multiplicar del " + numero);
        System.out.println("............................. ");
        //Utilizamos ahora el bucle for
        for (contador = 1; contador <= 10; contador++) {
            /* La cabecera del bucle incorpora la inicialización de la variable
             * de control, la condición de multiplicación hasta el 10 y el
             * incremento de dicha variable de uno en uno en cada iteración del
             * bucle.
             * En este caso contador++ incrementará en una unidad el valor de
             * dicha variable.
             */
            resultado = contador * numero;
            System.out.println(numero + " x " + contador + " = " + resultado);
            /* A través del operador + aplicado a cadenas de caracteres,
             * concatenamos los valores de las variables con las cadenas de
             * caracteres que necesitamos para representar correctamente la
             * salida de cada multiplicación.
             */
        }
    }
}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

⬆ Volver a teoría

Ejemplo15

Tabla de multiplicar del siete con el bucle while.

Este ejemplo utiliza el bucle `while` para la impresión por pantalla de la tabla de multiplicar del siete. Lee atentamente los comentarios incluidos en el código, pues aclaran algunas cuestiones interesantes sobre este bucle. Como podrás comprobar, el resultado es totalmente equivalente al obtenido utilizando el bucle `for`.

```java
package UD03;

public class Repetitiva_While {

    public static void main(String[] args) {
        // Declaración e inicialización de variables
        int numero = 7;
        int contador;
        int resultado = 0;
        //Salida de información
        System.out.println("Tabla de multiplicar del " + numero);
        System.out.println("............................ ");
        //Utilizamos ahora el bucle while
        contador = 1; //inicializamos la variable contadora
        while (contador <= 10){ //Establecemos la condición del bucle
            resultado = contador * numero;
            System.out.println(numero + " x " + contador + " = " + resultado);
            //Modificamos el valor de la variable contadora, para hacer que el
            //bucle pueda seguir iterando hasta llegar a finalizar
            contador++;
        }
    }
}
```

⬆ Volver a teoría

Ejemplo16

Tabla de multiplicar del siete con el bucle do-while.

En este ejemplo se utiliza el bucle `do-while` para la impresión por pantalla de la tabla de multiplicar del siete. Como podrás comprobar, el resultado es totalmente equivalente al obtenido utilizando el bucle `for` y el bucle `while`.

```java
package UD03;

public class Repetitiva_DoWhile {

    public static void main(String[] args) {
        // Declaración e inicialización de variables
        int numero = 7;
        int contador;
        int resultado = 0;

        //Salida de información
        System.out.println("Tabla de multiplicar del " + numero);
        System.out.println("............................ ");
        //Utilizamos ahora el bucle do-while
        contador = 1; //inicializamos la variable contadora
        do {
            resultado = contador * numero;
            System.out.println(numero + " x " + contador + " = " + resultado);
            //Modificamos el valor de la variable contadora, para hacer que el
            //bucle pueda seguir iterando hasta llegar a finalizar
            contador++;
        } while (contador <= 10); //Establecemos la condición del bucle
    }
}
```

⬆ Volver a teoría

Ejemplo17

Uso de break en un bucle for.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Ejemplo de uso de la estructura `break` dentro de un bucle `for` (aunque no es recomendable su uso).

```java
package UD03;

public class Sentencia_Break {

    public static void main(String[] args) {
        int contador;
        for (contador=1;contador<=10;contador++){
            if (contador==7)
                break;
            System.out.println("Valor: " + contador);
        }
        System.out.println("Fin del programa");
        /*
         * El bucle solo se ejecutará en 6 ocasiones, ya que cuando
         * la variable contador sea igual a 7 encontraremos un break que
         * romperá el flujo del bucle, transfiriéndonos a la sentencia que
         * imprime el mensaje de Fin del programa.
         */
    }
}
```

⬆ Volver a teoría

Ejemplo18

Uso de continue en un bucle for.

Ejemplo de uso de la estructura `continue` dentro de un bucle `for` (aunque no es recomendable su uso).

```java
package UD03;

public class Sentencia_Continue {

    public static void main(String[] args) {
        int contador;
        System.out.println("Imprimiendo los números pares que hay del 1 al 10...");
        for (contador = 1; contador <= 10; contador++) {
            if (contador % 2 != 0) {
                continue;
            }
            System.out.println(contador + " ");
        }
        System.out.println("
Fin del programa");
        /*
         * Las iteraciones del bucle que generarán la impresión de cada uno de
         * los números pares, serán aquellas en las que el resultado de calcular
         * el resto de la división entre 2 de cada valor de la variable
         * contador, sea igual a 0.
         */
    }
}
```

⬆ Volver a teoría

Ejemplo19

Declaración y uso de etiquetas de salto.

En este ejemplo se muestra la declaración y uso de etiquetas en un bucle. Como podrás apreciar, las sentencias asociadas a cada etiqueta están encerradas entre llaves para delimitar así su ámbito de acción.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD03;

public class EtiquetasSalto {

    public static void main(String[] args) {
        for (int i = 1; i < 3; i++) {
            bloque_uno:
            {
                bloque_dos:
                {
                    System.out.println("Iteración: " + i);
                    if (i == 1) {
                        break bloque_uno;
                    }
                    if (i == 2) {
                        break bloque_dos;
                    }
                }
                System.out.println("después del bloque dos");
            }
            System.out.println("después del bloque uno");
        }
        System.out.println("Fin del bucle");
    }
}
```

⬆ Volver a teoría

Ejemplo20

Suma de dos números empleando return.

Este programa obtiene la suma de dos números empleando un método que retorna el valor de la suma mediante `return`.

```java
package UD03;

import java.io.*;

public class Sentencia_Return {

    private static BufferedReader stdin = new BufferedReader(
            new InputStreamReader(System.in));

    public static int suma(int numero1, int numero2) {
        int resultado;
        resultado = numero1 + numero2;
        return resultado; //Mediante return devolvemos el resultado de la suma
    }

    public static void main(String[] args) throws IOException {
        //Declaración de variables
        String input; //Esta variable recibirá la entrada de teclado
        int primer_numero, segundo_numero; //Estas variables almacenarán los operandos
        // Solicitamos que el usuario introduzca dos números por consola
        System.out.print("Introduce el primer operando:");
        input = stdin.readLine(); //Leemos la entrada como cadena de caracteres
        primer_numero = Integer.parseInt(input); //Transformamos a entero lo introducido
        System.out.print("Introduce el segundo operando: ");
        input = stdin.readLine(); //Leemos la entrada como cadena de caracteres
        segundo_numero = Integer.parseInt(input); //Transformamos a entero lo introducido
        //Imprimimos los números introducidos
        System.out.println("Los operandos son: " + primer_numero + " y " + segundo_numero);
        System.out.println("obteniendo su suma... ");
        //Invocamos al método que realiza la suma, pasándole los parámetros adecuados
        System.out.println("La suma de ambos operandos es: " +
                suma(primer_numero, segundo_numero));
    }
}
```

⬆ Volver a teoría

Ejemplo21

Manejo de excepciones con try-catch dentro de un do-while.

Vamos a realizar un programa en Java en el que se solicite al usuario la introducción de un número por teclado comprendido entre el 0 y el 100. Utilizando manejo de excepciones, controlaremos la entrada de dicho número y volver a solicitarlo en caso de que ésta sea incorrecta.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD03;

import java.io.*;
import java.util.Scanner;

public class P6_1_Excepciones {

    public static void main(String[] args) {
        int numero = -1;
        int intentos = 0;
        String linea;
        Scanner teclado = new Scanner(System.in);
        do {
            try {
                System.out.print("Introduzca un número entre 0 y 100: ");
                linea = teclado.nextLine();
                numero = Integer.parseInt(linea);
            } catch (NumberFormatException e) {
                System.out.println("Debe introducir un número entre 0 y 100.");
            } catch (Exception e) {
                System.out.println("Error al leer del teclado.");
            } finally {
                intentos++;
            }
        } while (numero < 0 || numero > 100);
        System.out.println("El número introducido es: " + numero);
        System.out.println("Número de intentos: " + intentos);
    }
}
```

⬆ Volver a teoría

Ejemplo22

Propagación de excepciones hacia arriba en la pila.

Este ejemplo muestra cómo se propaga una excepción hacia arriba en la pila de ejecución en Java.

```java
package UD03;

import java.util.Scanner;

public class P6_2_PropagacionExcepciones {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        try {
            System.out.print("Introduzca un número entre 0 y 100: ");
            String linea = teclado.nextLine();
            int numero = Integer.parseInt(linea);
            metodoA(numero);
        } catch (Exception e) {
            System.out.println("Excepción atrapada en el método main: " + e.getMessage());
        }
    }

    public static void metodoA(int numero) {
        try {
            metodoB(numero);
        } catch (ArithmeticException e) {
            System.out.println("Excepción atrapada en el método A: " + e.getMessage());
        }
    }

    public static void metodoB(int divisor) {
        int resultado = 10 / divisor;
    }

}
```

⬆ Volver a teoría

Ejemplo23

Excepción personalizada EdadInvalidaException.

En este ejemplo se define una clase de excepción personalizada (en una clase interna), se lanza en un método y posteriormente se captura en el método `main`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
import java.util.Scanner;

public class ValidadorEdad {

    public static void validarEdad(int edad) throws EdadInvalidaException {
        // Validaciones específicas con mensajes personalizados
        if (edad < 0) {
            throw new EdadInvalidaException("La edad no puede ser negativa");
        } else if (edad > 150) {
            throw new EdadInvalidaException("La edad parece incorrecta (muy alta)");
        } else if (edad < 18) {
            throw new EdadInvalidaException("Acceso denegado: Menor de edad");
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Dime una edad para validar: ");
        int edadInput = sc.nextInt();
            try {
                System.out.println("Validando edad: " + edadInput);
                validarEdad(edadInput);
                //Si llegamos hasta aquí, la edad es válida
                System.out.println("Edad válida. Acceso permitido.");
            } catch (EdadInvalidaException e) {
                // Capturamos y manejamos nuestra excepción personalizada
                System.out.println("Excepción capturada: " + e.getMessage());
            } catch (Exception e) {
                // Capturamos cualquier otra excepción genérica
                System.out.println("Error inesperado: " + e.getMessage());
            }
        }
    }
    //Clase interna, ni public, ni static, ni private
    class EdadInvalidaException extends Exception {
        // Constructor que recibe un mensaje personalizado
        public EdadInvalidaException(String s) {
            //Llamamos al constructor de la clase base Exception y pasamos el mensaje que recibimos
            super(s);
        }
        // Constructor por defecto
        public EdadInvalidaException() {
            super("Edad inválida");
    }
}
```

⬆ Volver a teoría

4.1.9 Resumen — Conceptos clave

Concepto	Definición
Estructura secuencial	Sentencias ejecutadas en orden, una tras otra
Estructura de selección	Permite elegir entre distintos caminos según una condición
Estructura de iteración	Repite un bloque de código mientras se cumpla una condición
Excepción	Evento anómalo que interrumpe el flujo normal del programa
try-catch	Bloque que permite capturar y manejar excepciones
Depuración	Proceso de identificar y corregir errores en el código

4.1.10 Autoevaluación

- Sé utilizar if/else y switch correctamente
- Controlo los bucles for, while y do-while
- Comprendo cuándo usar break y continue
- Puedo capturar y manejar excepciones
- Sé lanzar excepciones con throw
- Utilizo el depurador del IDE para encontrar errores

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

4.1.11 Vídeos recomendados

Canal	Vídeo	Contenido
Programación ATS	Creación de clases y objetos (vídeo 61)	Clases, atributos, métodos, instanciación
Programación ATS	Playlist POO Java	Vídeos 61–72: constructores, sobrecarga (66), modificadores (67), encapsulamiento (68), static (70)
Píldoras Informáticas	Curso Java — POO básico	Vídeos 30–39 de la serie antigua
MoureDev	Curso Completo de Java desde Cero	Sección POO a partir de 05:26:26
makigas	Java: introducción a las excepciones	Conceptos básicos de excepciones
makigas	Java: throw y throws, usos y diferencias	Lanzar y delegar excepciones
BitBoss	Los 4 pilares de POO	Resumen de abstracción, encapsulación, herencia, polimorfismo (10 min)
EducaMadrid	POO Parte 1	Conceptos teóricos de POO

29 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

4.2 Ejercicios de la UD03

4.2.1 Retos

1. (Descuento) modifique el programa para que, en lugar de realizar un descuento del 8% si la compra es de 100 € o más, aplique una penalización de 2 € si el precio es inferior a 30 €.

```java
import java.util.Scanner;
//Un programa que calcula descuentos.

public class _1_Descuento{
    public static final float DESCUENTO= 8;
    public static final float COMPRA_MIN = 100;

    public static void main(String[] args) {
        Scanner lector = new Scanner(System.in);
        System.out.print("¿Cuál es el precio del producto, en euros?");
        float precio= lector.nextFloat();
        lector.nextLine();
        if (precio>= COMPRA_MIN) {
            float descuentoHecho= precio * DESCUENTO / 100;
            precio = precio - descuentoHecho;
        }
        System.out.println("El precio final a pagar es de "+ precio +" euros.");
    }
}
```

2. (Adivina) modifique el programa para que, en lugar de un único valor secreto, haya dos. Para ganar, basta con acertar uno de los dos. La condición lógica que necesitará ya no se puede resolver con una expresión compuesta por una única comparación. Será más compleja.

> **Atención**
>
> Para pasar satisfactoriamente los tests, la variable `VALOR_SECRETO` debe ser renombrada a `VALOR_SECRETO1`, y la nueva debe llamarse `VALOR_SECRETO2` (con un valor aleatorio entre 0 y 10). Las dos son constantes (`static final`).

```java
import java.util.Scanner;

public class Adivina{

    public static final int VALOR_SECRETO = 4;

    public static void main(String[] args) {
        Scanner lector = new Scanner(System.in);
        System.out.println("Empecemos el juego.");
        System.out.print("Adivina el valor entero, entre 0 y 10: ");
        int valorUsuario = lector.nextInt();
        lector.nextLine();
        if (VALOR_SECRETO == valorUsuario) {
            System.out.println("¡Exactamente! Era " + VALOR_SECRETO + ".");
        } else {
            System.out.println("¡Te has equivocado!");
        }
        System.out.println("Hemos terminado el juego.");
    }
}
```

3. (AdivinaCorrecto) modifique el ejemplo anterior (Adivina) para que comprueben que el valor que ha introducido el usuario se encuentra dentro del rango de valores correcto (entre 0 y 10).
4. (AdivinaControlErroresEntrada) aplique el mismo tipo de control sobre los datos de la entrada del ejemplo siguiente al ejercicio del reto 1.

> **Atención**
>
> Para pasar satisfactoriamente los tests, el mensaje de error cuando no se introduzca un entero debe contener la palabra "ERROR". El programa es el del reto 1 (Descuento): la penalización de 2 € se suma al precio

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
import java.util.Scanner;

public class AdivinaControlErroresEntrada{

    public static final int VALOR_SECRETO = 4;

    public static void main(String[] args) {
        Scanner lector = new Scanner(System.in);
        System.out.println("Empecemos el juego.");
        System.out.print("Adivina el valor entero, entre 0 y 10: ");
        boolean tipoCorrecto = lector.hasNextInt();
        if (tipoCorrecto) {
            //Se ha escrito un entero correctamente. Ya puede leerse.
            int valorUsuario = lector.nextInt();
            lector.nextLine();
            if (VALOR_SECRETO == valorUsuario) {
                System.out.println("Exacto! Era " + VALOR_SECRETO + ".");
            } else {
                System.out.println("Te has equivocado!");
            }
            System.out.println("Hemos terminado el juego.");
        } else {
            //No se ha escrito un entero.
            System.out.println("El valor introducido no es un entero.");
        }
    }
}
```

5. (Linea) Modifique el ejemplo para que primero pregunte al usuario cuántos caracteres "-" quiere escribir por pantalla, y entonces los escriba. Cuando pruebe el programa, no introduzca un número muy alto!

```java
//Un programa que escribe una línea con 100 caracteres '-'.

public class Linea {

    public static void main(String[] args) {
        //Inicializamos un contador

        int i = 0;
        //¿Ya hemos hecho esto 100 veces?
        while (i < 100) {
            System.out.print("-");
            //Lo hemos hecho una vez, sumamos 1 al contador

            i = i + 1;
        }
        //Forzamos un salto de línea
        System.out.println();
    }
}
```

6. (TablaMultiplicar) un contador tanto puede empezar a contar desde 0 e ir subiendo, como desde el final e ir disminuyendo como una cuenta atrás. Modifique este programa para que la tabla de multiplicar comience mostrando el valor para 10 y vaya bajando hasta el 1.

```java
import java.util.Scanner;
public class TablaMultiplicar{

    public static void main(String[] args) {
        Scanner lector = new Scanner(System.in);
        System.out.print("¿Qué tabla de multiplicar quieres? ");
        int tabla = lector.nextInt();
        lector.nextLine();
        int i = 1;
        while (i <= 10) {
            int resultado = tabla * i;
            System.out.println(tabla + " * " + i + " = " + resultado);
            i = i + 1;
        }
        System.out.println("Ésta ha sido la tabla del " + tabla);
    }
}
```

7. (Modulo) el uso de contadores y acumuladores no es excluyente, sino que puede ser complementario. Piense cómo se podría modificar el programa para calcular el resultado del módulo y la división entera a la vez. Recuerde que la división entera simplemente sería contar cuántas veces se ha podido restar el divisor.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
import java.util.Scanner;

public class Modulo{

    public static void main(String[] args) {
         Scanner lector = new Scanner(System.in);
        System.out.print("¿Cuál es el dividendo? ");
        int dividendo = lector.nextInt();
        lector.nextLine();
        System.out.print("¿Cuál es el divisor? ");
        int divisor = lector.nextInt();
        lector.nextLine();
        while (dividendo >= divisor) {
            dividendo = dividendo - divisor;
            System.out.println("Bucle: por ahora el dividendo vale " + dividendo + ".");
        }
        System.out.println("El resultado final es" + dividendo + ".");
    }
}
```

4.2.2 Ejercicios

`if else`

1. (MenorDeDos) Escribir un método (`menorDeDos`) que devuelva el menor de dos números enteros introducidos por teclado.
2. (MenorDeTres) Escribir dos métodos que devuelve el menor de tres números recibidos por parámetros. Haz dos versiones:
   - `menorDeTresV1` : utilizando los operadores de comparación (<, >, ==) y lógicos (&&,||, ...) necesarios
   - `menorDeTresV2` : sin utilizar ninguno de los operadores lógicos (habrá que usar sentencias if else anidadas)
3. (IntermedioDeTres) Escribir un método (`intermedioDeTres`) que devuelva el intermedio de tres números recibidos por parámetros.
4. (NotasTexto) Escribir un método (`notas2Texto`) que recibe un valor numérico (se supone entre 1 y 10 y puede contener decimales) y devuelva el literal correspondiente a dicha nota según ("insuficiente", "suficiente", "bien", "notable", "sobresaliente" y "matricula de honor").
5. (Division) Escribir un método (`division`) que reciba dos números enteros y devuelva un `String` con el resultado de la división (o el mensaje de error). Tener en cuenta que si dividimos un número por cero se producirá un error de ejecución y debemos evitarlo. Ejemplos de ejecución:

```
Vamos a dividir dos números.
Introduce el valor del dividendo: 15
Introduce el valor del divisor: 3
La división es: 5.0

Vamos a dividir dos números.
Introduce el valor del dividendo: 4
Introduce el valor del divisor: 0
No se puede realizar la división porque el divisor es 0
```

6. (Raiz) Se desea calcular la raíz cuadrada real de un número real cualquiera pedido inicialmente al usuario. Como dicha operación no está definida para los números negativos es necesario tratar, de algún modo, dicho posible error sin que el programa detenga su ejecución. Debes escribir un método `raiz` que recibe un entero y devuelve un `String` con el resultado (o el mensaje de error). Ejemplos de ejecución:

```
Vamos a calcular la raíz cuadrada de un número.
Introduce el valor del número: 81
La raíz es: 9.0

Vamos a calcular la raíz cuadrada de un número.
Introduce el valor del número: -52
No se puede realizar la raíz porque el número es negativo
```

7. (Hora12) Escribir un método (`formatoDoce`) que recibe dos enteros (hora y minutos) en notación de 24 horas y la devuelve como `String` en notación de 12 horas. Por ejemplo, si la entrada es 13 horas 45 minutos, la salida será " `01:45 PM` " (también vale " `1:45 PM` "). Las 12:00 son " `12:00 PM` " (mediodía). La hora y los minutos se leerán de teclado de forma separada, primero la hora y luego los minutos. Si la hora pasada al método no es correcta devolverá: "Error de conversión"

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
Vamos a convertir una hora a formato 12H.
Introduce el valor para la hora: 51
Introduce el valor para los minutos: 6
Error de conversión.

Vamos a convertir una hora a formato 12H.
Introduce el valor para la hora: 13
Introduce el valor para los minutos: 45
01:45 PM

Vamos a convertir una hora a formato 12H.
Introduce el valor para la hora: 4
Introduce el valor para los minutos: 53
04:53 AM
```

8. (Bisiesto) Escribir un método (`esBisiesto`) que devuelva si un año introducido por teclado es o no bisiesto. Un año es bisiesto si es múltiplo de 4 (por ejemplo 1984). Sin embargo, los años múltiplos de 100 no son bisiestos, salvo que sean múltiplos de 400, en cuyo caso si lo son (por ejemplo 1800 no es bisiesto y 2000 si lo es). Para hacer el programa, implementa un método dentro de la clase que reciba un año y devuelva true si el año es bisiesto y false en caso de que no lo sea.
9. (Fechas) Escribir un método (`imprimeMenor`) que reciba dos fechas (día, mes y año) (6 parámetros en total), que se suponen correctas, y le muestre la menor de ellas. La fecha se mostrará en formato dd/mm/año. Utiliza un método `mostrarFecha` , para mostrar la fecha por pantalla. La fecha se mostrará siempre con dos dígitos para el día, dos para el mes y cuatro para el año.

```
Vamos a ver que fecha es menor.
Introduce el valor para el Fecha1.
    - Dia: 4
    - Mes: 5
    - Año: 1999

Introduce el valor para el Fecha2.
    - Dia: 3
    - Mes: 5
    - Año: 1999

La fecha menor es:
03/05/1999
```

10. (DiasDelMes) Escribir un método (`diasMes`) que recibe el número de un mes (1 a 12) y devuelve el número de días que tiene el mes. Hacerlo utilizando sentencias `if else` (sin el operador ternario `? :`). Si el mes recibido no es correcto, debe devolver 0.

```
Vamos a ver cuantos días tiene un mes.
Introduce el valor para el mes: 8
Este mes tiene 31 días.

Vamos a ver cuantos días tiene un mes.
Introduce el valor para el mes: 54
Este mes tiene 0 días.
```

11. (NombreDelMes) Escribir un programa que lea de teclado el número de un mes (1 a 12) y visualice el nombre del més (enero, febrero, etc). Hacerlo utilizando sentencias `if else` . Para hacer un programa, implementa un método en la clase que reciba un número de mes y devuelva el nombre del mes
12. (Salario) Escribir un método (`salario`) que recibe las horas trabajadas por un empleado en una semana y calcula y devuelve su salario neto semanal, sabiendo que:
    - Las horas ordinarias se pagan a 6 €.
    - Las horas extraordinarias se pagan a 10 €.
    - Los impuestos a deducir son:
        - Un 2 % si el salario bruto semanal es menor o igual a 350 €
        - Un 10 % si el salario bruto semanal es superior a 350 €
    - La jornada semanal ordinaria son 40 horas. El resto de horas trabajadas se considerarán horas extra.
13. (Signo) Dados dos números enteros, num1 y num2, realizar un programa que escriba uno de los dos mensajes:
    - "el producto de los dos números es positivo o nulo" o bien
    - "el producto de los dos números es negativo".

    Resolverlo sin calcular el producto, sino teniendo en cuenta únicamente el signo de los números a multiplicar.
14. (Calculadora) Escribir un programa para simular una calculadora. Considera que los cálculos posibles son del tipo num1 operado num2, donde num1 y num2 son dos números reales cualesquiera y operador es una de entre: +, -, * y /. El programa pedirá al usuario en primer lugar el valor num1, a continuación el operador y finalmente el valor num2. Resolver utilizando instrucciones `if else`

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

15. (Comercio) Un comercio aplica un descuento del 8% por compras superiores a 40 euros. El descuento máximo será de 12 euros. Escribir un programa que solicite al usuario el importe de la compra y muestre un mensaje similar al siguiente:
    - Importe de la compra 100 €
    - Porcentaje de descuento aplicado: 8%
    - Descuento aplicado: 8 €
    - Cantidad a pagar: 92 €
16. (Editorial) Una compañía editorial dispone de 2 tipos de publicaciones: libros y revistas. El precio de cada pedido depende del número de elementos solicitados al cual se le aplica un determinado descuento, que es diferente para libros y para revistas. La siguiente tabla muestra los descuentos a aplicar en función del número de unidades y del tipo de producto:

Cantidad pedida	Libros	Revistas
Hasta 5 unidades	0 % de descuento	0 % de descuento
De 6 a 10 unidades	10 % de descuento	15 % de descuento
De 11 a 20 unidades	15 % de descuento	20 % de descuento
A partir de 20 unidades	20 % de descuento	25 % de descuento

    Escribe un método `calcularCoste` que, recibiendo el tipo de publicación (`String`), que puede ser "libro" o "revista", el precio individual (`double`) y el número de unidades solicitado (`int`), devuelva el coste del pedido (aplicando el descuento correspondiente). Escribe un programa en el que el usuario deba introducir cantidad y precio de revistas y cantidad y precio de libros que incluye un pedido, y muestre el coste del pedido.

    Si el tipo especificado es diferente de "libro" o "revista", el método devolverá -1.

    ```
    Vamos a calcular el coste del pedido.

    Introduce los datos respecto a LIBROS.
        Precio unitario: 12,3
        Cantidad: 4

    Introduce los datos respecto a REVISTAS.
        Precio unitario: 2,6
        Cantidad: 5

    El coste total de pedido será de 62,20€.
    ```

17. (Taxi) Se desea calcular el coste del trayecto realizado en taxi en función de los kilómetros recorridos en las carreras metropolitanas de Valencia. Según las tarifas vigentes para el 2012, el coste se calcula de la siguiente manera:
    - Días laborables en horario diurno (de 6:00 a antes de las 22:00h): 0.73 €/km.
    - Días laborables en horario nocturno: 0.84 €/km.
    - Sábados y domingos: 0.93 €/km.
    - Además, la tarifa mínima diurna es de 2.95€ y la mínima nocturna de 4€.

    Escribir un método (`costeTrayecto`) que recibe, en este orden:
    - La hora (`hora` y `minutos`) en que se realizó el trayecto.
    - El `día de la semana` (se supone que el usuario introduce un valor entre 1 para lunes y 7 para domingo)
    - Los `kilómetros` recorridos (pueden tener decimales).

    El método devuelve el coste del trayecto. Escribe un método `main` que solicite al usuario la introducción de los datos y llamando al método muestre el resultado:

    ```
    Vamos a calcular el coste del trayecto.
    Introduce la hora: 15
    Introduce los minutos: 20
    Introduce el dia de la semana (1 para lunes y 7 para domingo): 6
    Introduce los kilometros recorridos: 3,5

    El coste total del trayecto es de 3,26€.
    ```

18. (Nombre) Escribir un método (`coincidenPrimeraYUltima`) que recibe una cadena de texto. El método devolverá si la primera y la última letra del nombre coinciden o no.

    Haz que funcione aunque las letras tengan diferente CASE (pista: lowercase i uppercase).

    Escribe el método `main` que solicite al usuario un nombre de persona y llamando al método diga si coinciden o no, pruébalo con "Ana", "ana", "Angel", "Amanda" y "David":

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
Vamos a ver si la primera y la última letra de tu nombre son iguales.
Introduce un nombre: Ana
Si coinciden.

Vamos a ver si la primera y la última letra de tu nombre son iguales.
Introduce un nombre: Angel
No coinciden.
```

19. (Validar) Se desea implementar un programa que determine si dos datos `x` e `y` de entrada son válidos. Un par de datos es válido si es uno de los que aparecen en la siguiente tabla:

x :	**a**	**a**	**a**	**a**	**a**	**b**
y :	1	3	5	7	9	2

Se pide implementar un método (`esValido`) que recibe los dos valores (`char x` , `int y`), que devuelva si la combinación es válida o no.

Escribe también un método `main` que lea de teclado el valor de `x` y el valor de `y` , e indique por pantalla "VALIDOS" o "NO VALIDOS". Se pide hacerlo de forma que no se utilice ninguna estructura condicional (if, switch, operador ternario...), es decir, se calculará una expresión booleana que determine si `x` e `y` son válidos. Se procurará que la expresión booleana propuesta sea breve y concisa.

Bucles simples

1. (SencillosWhile) Crear una clase llamada `SencillosWhile` y crear en él métodos que realicen las siguientes tareas.
   a. (imparesHastaN) Recibe un nº entero `n` , devuelve los números impares que hay entre 1 y `n` . Por ejemplo, si `n` es 8 devolverá `"1 3 5 7 "`
   b. (nImpares) Recibe un nº entero `n` , devuelve los `n` primeros números impares. Por ejemplo, si `n` es 3 devolverá `"1 3 5"` (3 primeros impares)
   c. (cuentaAtras) Recibe un entero `n` , devuelve una cuenta atrás partiendo de `n` : `n` , `n-1` , ... Por ejemplo, si `n` es 5 devolverá `"5 4 3 2 1 0 "`
   d. (sumaNPrimeros) Recibe un entero `n` , devuelve la suma de los números entre 1 y `n` . Por ejemplo, si `n` es 5 devolverá `15`
   e. (mostrarDivisoresN) Recibe un entero `n` , devuelve todos sus divisores, incluidos el 1 y el mismo `n` . Por ejemplo, si `n` es 12 mostraría `"1 2 3 4 6 12 "`
   f. (sumaDivisoresN) Recibe un entero `n` , devuelve la suma de todos sus divisores, sin incluir al propio `n` . Por ejemplo, si `n` es 12 sumará 1, 2, 3, 4 y 6 = `16`

   Ejemplo de ejecución con todos los métodos:

```
Esta realiza varios While sencillos:
Introduce un número: 15
Imprimimos:
    Todos los números impares menores que 15: 1 3 5 7 9 11 13 15
    Los primeros 15 números impares: 1 3 5 7 9 11 13 15 17 19 21 23 25 27 29
    Cuenta atrás desde 15: 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
    La suma de los 15 primeros números resulta: 120
    Los divisores del número 15 son: 1 3 5 15
    La suma de todos los divisores del número 15 (sin él mismo) son: 9
```

2. (SencillosFor) Crear una clase llamada "SencillosFor" y crear en él los mismos métodos que en el ejercicio anterior, pero utilizando la sentencia `for` en lugar de `while` :

```
Esta realiza varios For sencillos:
Introduce un número: 15
Imprimimos:
    Todos los números impares menores que 15: 1 3 5 7 9 11 13 15
    Los primeros 15 números impares: 1 3 5 7 9 11 13 15 17 19 21 23 25 27 29
    Cuenta atrás desde 15: 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
    La suma de los 15 primeros números resulta: 120
    Los divisores del número 15 son: 1 3 5 15
    La suma de todos los divisores del número 15 (sin él mismo) son: 9
```

3. (PotenciasDe2) Crea un método (`potencias`) que recibe un entero `n` positivo y devuelve las `n` primeras potencias de 2. Es decir, 2^0, 2^1, 2^2, 2^3, ..., 2^n. Crea un método `main` que solicite un número al usuario y muestre por pantalla todas las potencias de 2. Soluciona el ejercicio sin utilizar `Math.pow` . Ten en cuenta que, por ejemplo, 2^3 = 1* 2 * 2 * 2 o que 2^4 = 1* 2 * 2 * 2 * 2:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
Dime n: 20
1 2 4 8 16 32 64 128 256 512 1024 2048 4096 8192 16384 32768 65536 131072 262144 524288 1048576
```

4. (Etapas) El ser humano pasa por una serie de etapas en su vida que, con carácter general se asocian a las edades que aparecen en la tabla siguiente.

Etapa	**Rango**
Infancia	Hasta los 10 años (incluidos)
Pubertad	De 11 a 14 años
Adolescencia	De 15 a 21 años
Adultez	De 22 a 54 años
Vejez	De 55 a 70 años
Ancianidad	A partir de los 71 años

   Escribe un método `main` en el que el usuario introduzca las edades de una serie de personas y calcule y muestre que porcentaje de personas que se encuentran en cada etapa. En primer lugar el programa pedirá el número de personas que participan en la muestra y a continuación solicitará la edad de cada una de ellas. El resultado será similar al siguiente:

```
¿Cuantas personas hay en la muestra?: 5
Introduce la edad de la persona 1 de 5 : 17
 --> adolescencia
Introduce la edad de la persona 2 de 5 : 21
 --> adolescencia
Introduce la edad de la persona 3 de 5 : 53
 --> adultez
Introduce la edad de la persona 4 de 5 : 62
 --> vejez
Introduce la edad de la persona 5 de 5 : 78
 --> ancianidad
Etapa       Número  %
Infancia:   0   0.0%
Pubertad:   0   0.0%
Adolescencia:   2   40.0%
Adultez:    1   20.0%
Vejez:      1   20.0%
Ancianidad:     1   20.0%

¿Cuantas personas hay en la muestra?: 3
Introduce la edad de la persona 1 de 3 : 1
 --> infancia
Introduce la edad de la persona 2 de 3 : 2
 --> infancia
Introduce la edad de la persona 3 de 3 : 25
 --> adultez
Etapa       Número  %
Infancia:   2   66.66666666666667%
Pubertad:   0   0.0%
Adolescencia:   0   0.0%
Adultez:    1   33.333333333333336%
Vejez:      0   0.0%
Ancianidad:     0   0.0%
```

5. (Primo) Escribir un método `esPrimo` que recibe un entero y devuelve si es primo o no. Escribe un método `main` en el que el usuario escriba un número entero y se le diga si se trata o no de un número primo, usando el método `esPrimo`. Recuerda que un nº primo es aquel que solo es divisible por 1 y por sí mismo (Es decir tiene SOLO y EXCLUSIVAMENTE dos divisores cuyo resto sea cero).

```
Este te dice si un número es primo:
Introduce un número: 15
El número es primo?: false

Este te dice si un número es primo:
Introduce un número: 7
El número es primo?: true
```

6. (Primos) Escribir un programa en el que el usuario escriba un número entero y se le diga todos los números primos entre 1 y el número introducido (Puedes usar el método `esPrimo` que has escrito en el ejercicio anterior.
7. (EsPrimoMejorada) Haz una nueva versión del programa del ejercicio anterior teniendo en cuenta lo siguiente:
   - El único número par que es primo es el 2.
   - Un número *n* no puede tener divisores mayores que n/2 (o mayores que Math.sqrt(n))

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

> **Atención**
>
> Escribe el método `esPrimo` en esta clase. Para pasar el test de rendimiento tiene que ser, al menos, 1,5 veces más rápido que probar todos los divisores de 2 a n-1.
>
> ¿Quieres ir mucho más allá? Compara cuánto tarda con `n/2` y con `Math.sqrt(n)` en un primo grande como 2147483647, y si tienes que listar muchos primos seguidos, investiga la *criba de Eratóstenes*.

8. (Divisores) Escribir un programa que muestre los tres primeros divisores de un número n introducido por el usuario. Por ejemplo, si el usuario introduce el número 45, el programa mostrará los divisores 1, 3 y 5. Ten en cuenta que la posibilidad de que el número n tenga menos de 3 divisores. Prueba qué pasa si el usuario pide, por ejemplo, los tres primeros divisores de 7.
9. (SumaSerie) Dado un número `n`, introducido por el usuario, calcula y muestra por pantalla la siguiente suma 1/1+½+⅓+ ⋯ + 1/`n`
10. (Cifras) Escribir un método (`numeroCifras`) que recibe un número entero cualquiera (positivo, negativo o cero) y devuelve cuantas cifras tiene. Pistas: ¿Cuantas cifras tiene el nº 25688? ¿Cuántas veces podemos dividir el nº 25688 por 10 hasta que se hace cero? Cuidado, el nº 0 tiene una cifra.

```
Vamos a decirte cuantas cifras tiene un número:
Introduce un número: 0
El número tiene 1 cifras.

Vamos a decirte cuantas cifras tiene un número:
Introduce un número: 25688
El número tiene 5 cifras.

Vamos a decirte cuantas cifras tiene un número:
Introduce un número: -14
El número tiene 2 cifras.
```

11. (Transportes) Una empresa de transportes cobra 30€ por cada bulto que transporta. Además, si el peso total de todos los bultos supera los 300 kilos, cobra 0.9€ por cada kg extra. Por último si el trasporte debe realizarse en sábado, cobra un plus de 60€. La empresa no realiza el pedido si hay que transportar más de 30 bultos, si el peso total supera los 1000 kg o si se solicita hacerlo en domingo. Realizar un método `main` que solicite el número de bultos, el día de la semana (valor entre 1 y 7) y el peso de cada uno de los bultos y muestre el coste del transporte en caso de que pueda realizarse o un mensaje adecuado en caso contrario:

```
¿Cuantos bultos debes enviar?: 3
¿Qué dia de la semana se realiza el envio (entre 1=lunes y 7=domingo)?: 7
No se puede realizar el envio por superar los 30 bultos o ser domingo

¿Cuantos bultos debes enviar?: 4
¿Qué dia de la semana se realiza el envio (entre 1=lunes y 7=domingo)?: 6
Cuanto pesa el bulto 1 de 4 : 25
Cuanto pesa el bulto 2 de 4 : 120
Cuanto pesa el bulto 3 de 4 : 35
Cuanto pesa el bulto 4 de 4 : 78
El coste del envio será de: 180.0

¿Cuantos bultos debes enviar?: 2
¿Qué dia de la semana se realiza el envio (entre 1=lunes y 7=domingo)?: 1
Cuanto pesa el bulto 1 de 2 : 700
Cuanto pesa el bulto 2 de 2 : 400
No se puede realizar el envio por superar los 1000Kg
```

12. (Containers) La capacidad de un buque que transporta containers está limitada tanto por la cantidad de containers como por el peso, pudiendo transportar un máximo de 100 containers y un máximo de 700 toneladas. Hacer un programa en el que se vaya introduciendo el peso de los containers (en toneladas) a medida que se cargan en el barco, hasta que se llegue al máximo de capacidad. Mostrar al final la cantidad de containers cargados y el peso total. En el momento en que se desee cargar un container que haga que la carga total supere las 700 toneladas, se dará por finalizada la carga, aunque pudieran existir containers menos pesados con posibilidad de ser cargados.
13. (Notas) Realizar un método `main` que permita introducir las notas de varios examenes de un alumno de un curso. El usuario irá introduciendo las notas una tras otra. Se considerará finalizado el proceso de introducción de notas cuando el usuario introduzca una nota negativa. Al final, el programa le dirá: - El número de notas introducidas. - El número de aprobados (mayor o igual a 5 puntos) - La nota media

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
Vamos a darte información sobre tus notas, introduce una nota tras otra. Para finalizar escribe una nota negativa:
Introduce la nota 1: 5,5
Introduce la nota 2: 6,7
Introduce la nota 3: 8,4
Introduce la nota 4: 3,2
Introduce la nota 5: 1,5
Introduce la nota 6: -3
Has introducido un total de 5 notas,
de las cuales has aprobado 3,
con una media de 4.
```

14. (NotasExtremas) Modificar el ejercicio anterior para que además calcule la nota máxima y la nota mínima.

Bucles anidados

1. (Edades) Programa que pida al usuario la edad de cinco personas. Si la suma de las edades es inferior a 200, el programa volverá a solicitar las 5 edades.

```
Introduce la edad de la persona 1: 80
Introduce la edad de la persona 2: 42
Introduce la edad de la persona 3: 15
Introduce la edad de la persona 4: 56
Introduce la edad de la persona 5: 34
```

```
Introduce la edad de la persona 1: 15
Introduce la edad de la persona 2: 12
Introduce la edad de la persona 3: 43
Introduce la edad de la persona 4: 5
Introduce la edad de la persona 5: 4
La suma de edades debe ser superior a 200

Introduce la edad de la persona 1: 48
Introduce la edad de la persona 2: 54
Introduce la edad de la persona 3: 35
Introduce la edad de la persona 4: 12
Introduce la edad de la persona 5: 89
```

2. (NotasPorAlumno) Programa que pida al usuario las notas de `A` alumnos en `S` asignaturas, alumno por alumno. `A` y `S` se definirán en el programa como `CONSTANTES`.

> **Atención**
>
> Para pasar satisfactoriamente los tests, la variable `A` debe ser 3 y `S` debe ser 5.

```
Alumno 1
    Introduce la nota de la asignatura 1: 5
    Introduce la nota de la asignatura 2: 4
    Introduce la nota de la asignatura 3: 3
    Introduce la nota de la asignatura 4: 5
    Introduce la nota de la asignatura 5: 0
Alumno 2
    Introduce la nota de la asignatura 1: 6
    Introduce la nota de la asignatura 2: 8
    Introduce la nota de la asignatura 3: 9
    Introduce la nota de la asignatura 4: 8
    Introduce la nota de la asignatura 5: 7
Alumno 3
    Introduce la nota de la asignatura 1: 0
    Introduce la nota de la asignatura 2: 0
    Introduce la nota de la asignatura 3: 2
    Introduce la nota de la asignatura 4: 4
    Introduce la nota de la asignatura 5: 5
```

3. (NotasPorAsignatura) Programa que pida al usuario las notas de `A` alumnos en `S` asignaturas, asignatura por asignatura. `A` y `S` se definirán en el programa como `CONSTANTES`.

```
Asignatura 1
Introduce nota del alumno 1:
Introduce nota del alumno 2:
...
Asignatura 2
Introduce nota del alumno 1:
...
```

4. (MediasPorAsignatura) Repite el ejercicio anterior haciendo que se muestre la media de cada asignatura

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
Asignatura 1
Introduce nota del alumno 1:
Introduce nota del alumno 2:
...
Media asignatura 1: 8.5 puntos

Asignatura 2
Introduce nota del alumno 1:
...
Media asignatura 2: 6.5 puntos
...
```

5. (TablaMult) Escribir un programa que permita al usuario introducir un número `N` e imprima la tabla de multiplicar (del 0 al 10) de todos los números entre 1 y `N`. Ejemplo: Si el usuario introduce en número 5, el programa imprimiria

```
Tabla del 1:
1 por 0, 0
1 por 1, 1
1 por 2, 2
...
1 por 10, 10

Tabla del 2:
2 por 0, 0
2 por 1, 2
....
2 por 10, 20

Tabla del 3:
...

Tabla del 5:
...
5 por 10, 50
```

6. (PrimosHastaN) Programa que solicite al usuario un numero `n` y muestre todos los números primos menores o iguales que `n`. (IGUAL AL 27!!)
7. (CombinarLetras2) Escribir un programa que muestre todas las palabras de dos letras que se pueden formar con los cuatro primeros caracteres del alfabeto en minúsculas ('a', 'b', 'c', 'd'):

```
aa
ab
ac
ad
ba
bb
bc
bd
...
da
db
dc
dd
```

8. (CombinarLetras3) Repite el ejercicio anterior mostrando palabras de tres letras

```
aaa
aab
...
ddc
ddd
```

9. (LetraALetra) Escribe un programa en el que se solicite al usuario un texto de forma repetida hasta que el usuario introduzca la cadena vacía. Con cada texto que introduzca el usuario se le mostrará carácter a carácter, cada carácter en una línea

```
Vamos a escribir textos letra a letra
Introduce texto: Hola
H
o
l
a
Introduce texto: Casa
C
a
s
a
Introduce texto:
Fin del programa
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

10. (DibujarFiguras1) Escribe una clase que contenga los métodos que se indican a continuación. En el método `main` solicita al usuario las dimensiones de las figuras necesarias en cada caso y llama al método correspondiente para que se muestre por pantalla

    a. `void dibRecAsteriscos (int ancho, int alto)` dibuja un rectángulo utilizando asteriscos, como el siguiente. En el ejemplo ancho es 7 y alto es 3

```
* * * * * * *
* * * * * * *
* * * * * * *
```

```
Vamos a dibujar figuras.
Introduce el ancho: 8
Introduce el alto: 4
* * * * * * * *
* * * * * * * *
* * * * * * * *
* * * * * * * *
```

    b. `void dibRecNumeros1 (int ancho, int alto)` dibuja un rectángulo utilizando números, como el siguiente. En el ejemplo ancho es 7 y alto es 3

```
1 2 3 4 5 6 7
1 2 3 4 5 6 7
1 2 3 4 5 6 7
```

    c. `void dibRecNumeros2 (int ancho, int alto)` dibuja un rectángulo utilizando números, como el siguiente. En el ejemplo ancho es 7 y alto es 3

```
7 6 5 4 3 2 1
7 6 5 4 3 2 1
7 6 5 4 3 2 1
```

    d. `void dibRecNumeros3 (int ancho, int alto)` dibuja un rectángulo utilizando números, como el siguiente. En el ejemplo ancho es 7 y alto es 3

```
01 02 03 04 05 06 07
08 09 10 11 12 13 14
15 16 17 18 19 20 21
```

    e. `void dibDiagonal (int ancho, int alto)` dibuja un rectángulo con ceros y unos. Los 1 están en las posiciones en las que fila y columna coinciden. En el ejemplo ancho es 7 y alto es 3

```
1 0 0 0 0 0 0
0 1 0 0 0 0 0
0 0 1 0 0 0 0
```

    f. `void dibRecLetras (int ancho, int alto)` dibuja un rectángulo letras sucesivas comenzando por la "a". En el ejemplo ancho es 7 y alto es 3

```
a a a a a a a
b b b b b b b
c c c c c c c
```

    g. `void dibRecLetras2 (int ancho, int alto)` dibuja un rectángulo letras sucesivas terminando por la "a". En el ejemplo ancho es 7 y alto es 3

```
c c c c c c c
b b b b b b b
a a a a a a a
```

    h. `void dibRecLetras3 (int ancho, int alto)` dibuja un rectángulo letras sucesivas comenzando por la "a". En el ejemplo ancho es 7 y alto es 3

```
a b c d e f g
h i j k l m n
o p q r s t u
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

11. (dibujarFiguras2) Escribe una clase que contenga los métodos que se indican a continuación. En el método main solicita al usuario las dimensiones de las figuras necesarias en cada caso y llama al método correspondiente para que se muestre por pantalla

    a. `void dibRectNumeros3 (int ancho, int alto)` dibuja un rectángulo utilizando números, como el siguiente. En el ejemplo ancho es 7 y alto es 3

    ```
    1 2 3 4 5 6 7 7 6 5 4 3 2 1
    1 2 3 4 5 6 7 7 6 5 4 3 2 1
    1 2 3 4 5 6 7 7 6 5 4 3 2 1
    ```

    b. `void dibRectAsteriscos1 (int ancho, int alto)` dibuja un rectángulo utilizando asteriscos (*) y espacios en blanco, como el siguiente. En el ejemplo ancho es 7 y alto es 3

    ```
    * * * * * * *
    * * * * * * *
    * * * * * * *
    ```

    c. `void dibRectAsteriscos2 (int ancho, int alto)` dibuja un rectángulo utilizando asteriscos (*), espacios en blanco y el carácter '+', como el siguiente. En el ejemplo ancho es 7 y alto es 3

    ```
    * + * + * + *
    * + * + * + *
    * + * + * + *
    ```

    d. `void dibRectAsteriscos3 (int ancho, int alto)` dibuja un rectángulo utilizando asteriscos (*) y espacios en blanco, como el siguiente. En el ejemplo ancho es 7 y alto es 3

    ```
    * * * * * * *
    *           *
    * * * * * * *
    ```

    e. `void dibTriangulo1 (int base)` dibuja un triángulo utilizando asteriscos (*) y espacios en blanco, como el siguiente. En el ejemplo base es 5

    ```
    *
    * *
    * * *
    * * * *
    * * * * *
    ```

    f. `void dibTriangulo2 (int altura)` dibuja un triángulo utilizando asteriscos (*) y espacios en blanco, como el siguiente. En el ejemplo altura es 5

    ```
            *
          * *
        * * *
      * * * *
    * * * * *
    ```

    g. `void dibTriangulo3 (int altura)` dibuja un triángulo utilizando asteriscos (*) y espacios en blanco, como el siguiente. En el ejemplo altura es 5

    ```
            *
          * * *
        * * * * *
      * * * * * * *
    * * * * * * * * *
    ```

`switch`

1. (NotasTexto2) Escribir un método (`nota2TextoSwitch`) que reciba la nota de un examen (valor numérico entero entre 1 y 10) y muestre el literal correspondiente a dicha nota según (insuficiente, suficiente, bien, notable, sobresaliente, matrícula de honor). Hacerlo utilizando la sentencias switch.
2. (DiasDelMes2) Escribir un método (`numeroDias`) que recibe un número de mes (1 a 12) y devuelva el número de días que tiene el mes. Resolver utilizando la sentencias switch. Si el mes es erroneo, devolverá 0.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
Vamos a calcular el número de dias que tiene un mes.
Introduce el número del mes (1-12): 8
31

Vamos a calcular el número de dias que tiene un mes.
Introduce el número del mes (1-12): 2
28

Vamos a calcular el número de dias que tiene un mes.
Introduce el número del mes (1-12): 15
0
```

3. (NombreDelMes2) Escribir un método (`nombreMes`) que recibe el número de un mes (1 a 12) y devuelve el nombre del mes ("enero", "febrero", etc). Resolver utilizando la sentencias switch. Si el mes no es válido, devolverá la cadena vacía "".
4. (Calculadora2) Escribir un método (`calculadora`) para simular una calculadora. Considera que los cálculos posibles son del tipo `num1, operador, num2` , donde `num1` y `num2` son dos números reales cualesquiera y `operador` es un `char` , uno de entre: `'+', '-', '*'` y `'/'` . El método recibirá los tres parámetros y devolverá el resultado de la operación. Resolver utilizando la sentencias switch. Si el `operador` no es ninguno de la lista, devuelve el `num1` .

en papel...

1. (Valor) ¿Qué valor se asignará a consumo en la sentencia `if` siguiente si velocidad es 120?

```
if (velocidad > 80)
    consumo = 10;
else if (velocidad > 100)
    consumo = 12;
else if (velocidad > 120)
    consumo = 15;
```

2. (Errores) Encuentra y corrige los errores de los siguientes fragmentos de programa.

   a. fragmento a

```
if (x > 25)
    y = x
else
    y = z;
```

   b. fragmento b

```
if (x<0)
    System.out.println("El valor de x es" +x);
    System.out.println ("x es negativo");
else
    System.out.println ("El valor de x es"+x);
    System.out.println ("x es positivo");
```

   c. fragmento c

```
if (x = 0) System.out.println ("x igual a cero");
else System.out.println ("x distinto de cero");
```

3. (SalidaExacta) Cuál es la salida exacta por pantalla del siguiente fragmento de programa

```
int x = 20;
System.out.println("Comenzamos");
if (x>= 20)
    if (x>50) System.out.println("Muy grande");
    else {
        if (x%2 != 0) System.out.println("Impar");
    }
else if (x<=20) System.out.println("Pequeño");
System.out.println("Terminamos");
```

4. (Descuentos) En una tienda, por liquidación, se aplican distintos descuentos en función del total de las compras realizadas:
   - Si total < 500 €, no se aplica descuento.
   - Si 500 € <= total <= 2000 €, se aplica un descuento del 30 %.
   - Si total > 2000 €, entonces se aplica un descuento del 50 %

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

¿Cuál de los siguientes fragmentos de programa asigna a la variable `desc` el descuento correcto? Indica "Si" o "NO" al lado de cada fragmento

a. fragmento a

```
double desc = 0.0;
if (total <= 500)
    if (total >= 2000) desc = 30.0;
    else desc = 50.0;
total = total * desc / 100.0;
```

b. fragmento b

```
double desc = 0.0;
if (total >= 500)
    if (total <= 2000) desc = 30.0;
    else desc = 50.0;
total = total * desc / 100.0;
```

c. fragmento c

```
double desc = 0.0;
if (total <= 2000){
    if (total >= 500) desc = 30.0;
    } else desc = 50.0;
total = total * desc / 100.0;
```

d. fragmento d

```
double desc = 0.0;
if (total > 500)
    if (total < 2000) desc = 30.0;
    else desc = 50.0;
total = total * desc /100.0;
```

5. (Salida) ¿Qué salida producirá el siguiente fragmento de programa si la variable entera platos vale 1? ¿Y si vale 3? ¿Y si vale 0?

```
switch (platos) {
    case 1: System.out.println("\nPrimer plato");
    case 2: System.out.println ("\nSegundo plato");
    case 3: System.out.println ("\nBebida");
            System.out.println ("\nPostre");
            break;
    default: System.out.println("\nCafé");
}
```

6. (ValorP) Dados tres enteros a, b y c, y un booleano p, el siguiente análisis por casos establece el valor de p en función de los valores de a, b y c:

```
si a > b entonces p = cierto;
si a < b entonces p = falso;
si a = b entonces
    si a > c entonces p = cierto;
    si a < c entonces p = falso;
    si a = c entonces p = falso;
```

Se pide la traducción de dicho análisis por casos a Java mediante:

- Una única instrucción `if` sin anidamientos.
- Una única instrucción, de la forma `p = ...`, que utilice el operador ternario.
- Una única instrucción, de la forma `p = ...` , sin sentencias `if` ni utilizar el operador ternario.

Trazas

Indica cual será la salida producida por los siguientes programas, teniendo en cuenta los datos de entrada:

1. (Traza1) **Datos de entrada: 2, 5**

   a.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
public static void main (String[] args){
    Scanner tec = new Scanner(System.in);
    int x,y,a;
    x = tec.nextInt();
    y = tec.nextInt();
    a = x+y;
    System.out.println(a);
}
```

b.

```
public static void main (String[] args){
    Scanner tec = new Scanner(System.in);
    int x,a;
    x = tec.nextInt();
    x = tec.nextInt();
    a= x+x;
    System.out.println(a);
}
```

c.

```
public static void main (String[] args){
    Scanner tec = new Scanner(System.in);
    int x,y,a;
    x = tec.nextInt();
    y = tec.nextInt();
    a = x+y;
    a = x*y;
    System.out.println(a);
}
```

d.

```
 public static void main (String[] args){
     Scanner tec = new Scanner(System.in);
     int x,y,a;
     x = tec.nextInt();
     y = tec.nextInt();
     a = x+y;
     System.out.println(a);
     a = x*y;
     System.out.println(a);
 }
```

e.

```
 public static void main (String[] args){
     Scanner tec = new Scanner(System.in);
     int x,y,a;
     x = tec.nextInt();
     y = tec.nextInt();
     a = x+y;
     a = a+x+y;
     a = a+a;
     System.out.println(a);
 }
```

f.

```
 public static void main (String[] args){
     Scanner tec = new Scanner(System.in);
     int x,y,a;
     x = tec.nextInt();
     y = tec.nextInt();
     a = x;
     a = doble(x);
     System.out.format ("%d%n%d%n%d",x,y,a);
 }
 public static int doble(int num){
     return 2*num;
 }
```

g.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
public static void main (String[] args) {
    Scanner tec = new Scanner(System.in);
    int x,y,a;
    x = tec.nextInt();
    y = tec.nextInt();
    a = x;
    doble(a);
    System.out.format("%d%n%d%n%d%n",x,y,a);
}
public static void doble(int x){
    x = 2*x;
}
```

h.

```
public static void main (String[] args){
    Scanner tec = new Scanner(System.in);
    int x,y,a;
    x = tec.nextInt();
    y = tec.nextInt();
    a = calcular(y,x);
    System.out.format("%d%n%d%n%d%n",x,y,a);
}
public static int calcular (int x, int y){
    return x-y;
}
```

i.

```
public static void main (String[] args){
    Scanner tec = new Scanner(System.in);
    int x,y,a;
    x = tec.nextInt();
    y = tec.nextInt();
    y = calcular(x);
    a = calcular(y);
    System.out.format("%d%n%d%n%d%n",x,y,a);
}
public static int calcular (int x){
    return x*x;
}
```

2. (Traza2) **Datos de entrada: 2, 5, 7**

```
public static void main (String[] args){
    int k,l,m,x,y,z;
    k = tec.nextInt();
    l = tec.nextInt();
    m = tec.nextInt();
    x = k+l;
    if (x != m) {
        y = k*l;
        z = 0;
    } else {
        y = 0;
        z = k-l;
    }
    if (z < 0) z = -z;
        System.out.format("%d%n%d%n%d%n",x,y,z);
}
```

3. (Traza3) **Datos de entrada: 2, 5, 7, 9, -9, -7, -5, -2**

a.

```
public static void main (String[] args){
    int x,y;
    x = 0;
    y = tec.nextInt();
    while(!(y<0)) {
        x+=-y;
        y = tec.nextInt();
        System.out.format("%d, %d",x,y);
    }
}
```

b.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
public static void main (String[] args){
    int x,y,z,a;
    x = y = z = a = 0;
    x = tec.nextInt();
    while(x>0) {
        if (y < z) y = tec.nextInt();
        else z= tec.nextInt();
        a = a-x+y*z;
        x = tec.nextInt();
        System.out.format("%d, %d, %d, %d",a,x,y,z);
    }
}
```

4. (Traza4) **Datos de entrada: 5, 5, 7, -5, -4, 2**

   a.

```
public static void main (String[] args){
    int x, y, a=0;
    x = 0;
    y = 99;
    while (x >= 0) {
        x = tec.nextInt();
        y = tec.nextInt();
        a = a + x*y;
    }
    System.out.println(a);
}
```

   b.

```
public static void main (String[] args){
    int x, y, a=0;
    x = 0;
    y = 99;
    while (x >= 0 && y >= 0) {
        x = tec.nextInt();
        y = tec.nextInt();
        a = a + x*y;
    }
    System.out.println(a);
}
```

   c.

```
public static void main (String[] args){
    int x, y, a=0;
    x = 0;
    y = 99;
    while (x >= 0 && y <= 0) {
        x = tec.nextInt();
        y = tec.nextInt();
        a = a + x*y;
    }
    System.out.println(a);
}
```

   d.

```
public static void main (String[] args){
    int x, y, a=0;
    x = 0;
    y = 99;
    while (x >= 0 || y >= 0) {
        x = tec.nextInt();
        y = tec.nextInt();
        a = a + x*y;
    }
    System.out.println(a);
}
```

5. (Traza5) **Datos de entrada: 5, 5, 7, -5, -4, 2**

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
public static void main(String[] args) {
    int x, y;
    x = 2;
    y = 3;
    while (x + y > 0) {
        x = tec.nextInt();
        y = tec.nextInt();
        x += y;
        y = x - y;
        System.out.format("%d, %d", x, y);
    }
}
```

6. (Traza6) **Datos de entrada: 2, 4, 7, 5, -6, -3, 6, 6**

   a.

```
public static void main (String[] args){
    int a,b;
    do{
        a = tec.nextInt();
        b = tec.nextInt();
        for (int i=a ; i<=b ; i++)
        System.out.println(i);
    } while (a!=b)
}
```

   b.

```
public static void main (String[] args){
    int a,b;
    a=5;
    b=5;
    do {
        for (int i=a ; i<=b ; i++)
        System.out.println(i);
        a = tec.nextInt();
        b = tec.nextInt();
    } while (a!=b);
}
```

7. (Traza7) **Datos de entrada: 3, 3, 5, 5, -3, -7, 2, 2**

```
public static void main (String[] args){
    int x,y;
    do {
        x = tec.nextInt();
        b = tec.nextInt();
    } while (x==y);
    if (x>y) {
        x=y;
        y=x;
    }
    System.out.format("%d %d %n",x,y);
}
```

8. (Traza8) **Datos de entrada: 3, 2, 1, 4**

```
public static void main (String[] args){
    int a=0,b;
    b = tec.nextInt();
    for(int i=1;i<=b,i++) a=(a+i)*i;
    System.out.println(a);
}
```

9. (Traza9) **Datos de entrada:** No aplica

```
public static void main (String[] args){
    int x,y;
    for (x=3;x>=1;x--){
        for(y=1;y<=x;y++) System.out.println(x);
        System.out.println();
    }
}
```

10. (Traza10) **Datos de entrada:** No aplica

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
public static void main (String[] args){
    int x,y;
    x=0;
    y=0;
    for (int i=1;i<=2;i++) {
        for (int j=1;j<=3;j++) x=(x+i)*j;
            y+=x;
    }
    System.out.println("%d %d %n",x,y);
}
```

11. (Traza11) **Datos de entrada: 4, 5, 6, 7, 8, 9**

```java
public static void main (String[] args){
    int x,y;
    do x = tec.nextInt();
    while (x<=5);
    y=0;
    for (int i=12;i>=x;i-=2) y+=(x*i);
    System.out.println(y);
}
```

Excepciones

1. (Edades) Escribe un programa que solicite al usuario la edad de cinco personas y calcule la media. La edad de una persona debe ser un valor entero comprendido en el rango [0,110]. Realiza tres versiones:
    a. (Edades_1) Si se introduce mal la edad de una persona se vuelve a pedir la edad de esa persona.

```
Introduce una edad entre 0 y 110 para la persona 1: 55
Introduce una edad entre 0 y 110 para la persona 2: 125
Introduce una edad entre 0 y 110 para la persona 2: -4
Introduce una edad entre 0 y 110 para la persona 2: 34
Introduce una edad entre 0 y 110 para la persona 3: 45
Introduce una edad entre 0 y 110 para la persona 4: 12
Introduce una edad entre 0 y 110 para la persona 5: 5
La media de las 5 edades introducidas es: 30,20
```

    a. (Edades_2) Si se introduce mal la edad de una persona, el programa muestra un mensaje de error, no calcula la media y termina.

```
Introduce una edad entre 0 y 110 para la persona 1: 55
Introduce una edad entre 0 y 110 para la persona 2: 125
java.lang.Exception: La edad introducida debe estar entre 0 y 110.
```

    a. (Edades_3) Si se introduce mal la edad de una persona, el programa vuelve a solicitar la edad de las cinco personas (comienza el proceso).
2. (PosicionLetra) Escribe los programas que se indican a continuación. Ejecuta cada programa haciendo que la entrada del usuario provoque una excepción. Anota el nombre de la excepción que se produce y cuál es la jerarquía de objetos de la que desciende:
    a. Programa que solicita dos números enteros (a y b) y muestra el resultado de su división (a/b).
        i. El usuario introduce 0 como valor de b.
        ii. El usuario introduce letras cuando el programa espera números enteros.
        iii. El usuario introduce un número real cuando el programa espera un entero.
    b. Programa que solicita al usuario su nombre y una posición dentro del nombre. Se muestra al usuario la letra del nombre cuya posición se ha indicado. Por ejemplo:

```
Introduce nombre: Javi
Introduce posición: 2
En la posición 2 de Javi está la letra a
```

        i. El usuario introduce una posición inválida.
3. (PosicionLetraMain) Repite el ejercicio anterior utilizando métodos y llamándolos desde el método `main`:
    a. Un método `dividir` que devuelva el cociente de dos números que recibe como parámetro
    b. Un método `letraNombre` que, dados un String `nombre` y un entero `pos`, devuelva el carácter del nombre que ocupa la posición indicada. Ejecuta los programas provocando errores (como en el ejercicio anterior) y observa los mensajes que se generan.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

4. (DividirArgs) Escribir un programa que divida dos números que se reciben en main en `args[0]` y `args[1]`.
   Ejemplo:

```
$ java dividir 10 5
10/5 es igual a 2
```

   Donde 10 y 5 son `args[0]` y `args[1]` respectivamente, es decir los parámetros con que llamamos al programa dividir.
5. (PorqueError) Justifica por qué se produce error en el siguiente fragmento de código

```
try {
    System.out.println("Introduce edad: ");
    int edad = tec.nextInt();
    if (edad >= 18) {
        System.out.println("Mayor edad");
    } else {
        System.out.println("Menor edad");
    }
    System.out.println("Introduce nif");
    String nif = tec.next();
    int numero = Integer.parseInt(nif.substring(0, nif.length() - 1));
    char letra = nif.charAt(nif.length() - 1);
    System.out.println("Numero: " + numero);
    System.out.println("Letra: " + letra);
} catch (Exception e){
    System.out.println("Debías introducir un número");
} catch (NumberFormatException e) {
    System.out.println("El nif es incorrecto");
}
```

6. (SalidaPantalla) Indica qué se mostrará por pantalla cuando se ejecute esta clase y por qué:

```
public class Uno {
      private static int metodo()  {
            int valor=0;
            try  {
                  valor = valor + 1;
                  valor = valor + Integer.parseInt("42") ;
                  valor = valor + 1;
                  System.out.println("Valor al final del try: " + valor);
            } catch(NumberFormatException e)  {
                  valor = valor + Integer.parseInt ("42");
                  System.out.println("Valor al final del catch: " + valor) ;
            }
            finally  {
                  valor = valor + 1;
                  System.out.println("Valor al final de finally: " + valor) ;
            }
            valor = valor + 1;
            System.out.println ("Valor antes del return: " + valor) ;
            return valor;
      }

      public static void main(String[] args)  {
            try {
                  System.out.println (metodo());
            }  catch (Exception e)  {
                  System.err.println("Excepcion en metodo()") ;
                  e.printStackTrace();
            }
      }
}
```

7. (SalidaPantalla2) Indica qué se mostrará por pantalla cuando se ejecute esta clase y por qué:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
public class Dos {
	private static int metodo()  {
		int valor=0;
		try   {
			valor = valor+1;
			valor = valor + Integer.parseInt("W");
			valor = valor + 1;
			System.out.println("Valor al final del try: " + valor);
		} catch(NumberFormatException e) {
			valor = valor + Integer.parseInt("42");
			System.out.println("Valor al final del catch: " + valor) ;
		} finally {
			valor = valor + 1;
			System.out.println("Valor al final de finally: " + valor) ;
		}
		valor = valor + 1;
		System.out.println ("Valor antes del return: " + valor) ;
		return valor ;
	}

	public static void main (String[] args)  {
		try  {
		    System .out.println(metodo());
		}  catch (Exception e) {
			System.err.println("Excepcion en metodo() ");
			e.printStackTrace();
		}
	}
}
```

8. (SalidaPantalla3) Indica qué se mostrará por pantalla cuando se ejecute esta clase y por qué:

```java
public class Tres {
	private static int metodo()  {
		int valor = 0;
		try {
			valor = valor +1;
			valor = valor + Integer.parseInt("W");
			valor = valor + 1;
			System.out.println("Valor al final del try : " + valor);
		} catch (NumberFormatException e) {
			valor = valor + Integer.parseInt("W");
			System.out.println("Valor al final del catch : " + valor);
		} finally {
			valor = valor + 1;
			System.out.println("Valor al final de finally: " + valor);
		}
		valor = valor + 1;
		System.out.println ("Valor antes del return: " + valor);
		return valor ;
	}

	public static void main (String[ ] args)
	{
		try {
			System.out.println(metodo ());
		} catch (Exception e) {
			System.err.println("Excepcion en metodo()") ;
			e.printStackTrace();
		}
	}
}
```

9. (SalidaPantalla4) Indica qué se mostrará por pantalla cuando se ejecute esta clase y por qué:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
import java.io.*;

public class Cuatro
{
      private static int metodo()  {
            int valor = 0;
            try {
                  valor = valor+1;
                  valor = valor + Integer.parseInt("W");
                  valor = valor + 1;
                  System.out.println("Valor al final del try : " + valor) ;
                  throw new IOException();
            } catch (IOException e)  {
                  valor = valor + Integer.parseInt("42");
                  System.out.println("Valor al final del catch : " + valor);
            } finally {
                  valor = valor + 1;
                  System.out.println("Valor al final de finally: " + valor);
            }
            valor = valor + 1;
            System.out.println ("Valor antes del return: " + valor) ;
            return valor ;
      }

      public static void main(String[] args)  {
            try {
                  System.out.println(metodo());
            } catch (Exception e) {
                  System.err.println("Excepcion en metodo()");
                  e.printStackTrace();
            }
      }
}
```

10. (SalidaPantalla5) Indica qué se mostrará por pantalla cuando se ejecute esta clase:
    a. Si se ejecuta con `java Cinco casa`
    b. Si se ejecuta con `java Cinco 0`
    c. Si se ejecuta con `java Cinco 7`

```java
public class Cinco {
   public static void main(String args[])  {
      try  {
            int a = Integer.parseInt(args[0]);
            System.out.println("a = " + a);
            int b=42/a;
            String c = "hola";
            char d = c.charAt(50);
      }  catch (ArithmeticException e) {
         System.out.println("div por 0: " + e);
      }  catch (IndexOutOfBoundsException e) {
         System.out.println("Índice del String fuera de límites: " + e);
      }  finally {
          System.out.println("Ejecución de finally");
      }
   }
}
```

11. (SalidaPantalla6) Indica cuál será la salida del siguiente programa y por qué

```java
public class Seis {
   public static void procA()  {
       try {
           System.out.println("dentro del procA"); 2
           throw new RuntimeException("demo"); 3
       } finally {
           System.out.println("Finally del procA"); 4
       }
    }

   public static void procB() {
       try  {
           System.out.println("dentro del procB"); 6
           return; 7
       } finally {
           System.out.println("finally del procB"); 8
       }
    }

    public static void main(String args[])  {
        try  {
            procA(); 1
        } catch(Exception e) {
            procB(); 5
        }
     }
}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

12. (SalidaPantalla7) Indica cuál será la salida del siguiente programa y por qué

```
public class Siete {
   public static void metodo() {
       try  {
           throw new NullPointerException("demo"); 2
       } catch (NullPointerException e) {
           System.out.println("capturada en método"); 3
           throw e; 4
       }
    }

    public static void main (String args[])  {
        try   {
            metodo(); 1
        }  catch(NullPointerException e)  {
           System.out.println("capturada en main " + e); 5
        }
    }
}
```

13. (DivisionPorCero) Crea un programa que intente dividir dos números enteros ingresados por el usuario y maneja la excepción de división por cero. Aquí tienes la explicación de porqué la división entre 0 no provoca excepciones para `double` y `float`.

```
Introduce el dividendo (entero): 85
Introduce el divisor (entero): 4
El resultado de 85/4 es: 21

Introduce el dividendo (entero): 5
Introduce el divisor (entero): 0
Error: División por cero no permitida.
```

14. (CalculadoraExcepcion) Crea una clase `CalculadoraExcepcion` con un método `dividir` que acepte dos números con decimales como argumentos, devuelva su resultado o lance una excepción personalizada (`DivisionPorCeroException`, que extiende `Exception`) si el divisor es cero. Captura la excepción en el método `main` y muestra un mensaje de error.

```
Introduce el dividendo: 15,5
Introduce el divisor: 4
El resultado de 15.5/4.0 es: 3.875

Introduce el dividendo: 3,5
Introduce el divisor: 0
Error: División por cero no permitida.
```

15. (EntradaNoNumerica) Escribe un programa que lea un número entero desde el teclado. Si el usuario ingresa algo que no es un número entero, maneja la excepción y muestra un mensaje de error.
16. (RangoNumerico) Escribe un programa que solicite al usuario ingresar un número entre 1 y 100. Si el número está fuera de ese rango, lanza una excepción personalizada (`NumeroFueraDeRangoException`, que extiende `Exception`) y muestra un mensaje de error.

```
Introduce un número entre 1 y 100: 5
Número válido: 5

Introduce un número entre 1 y 100: 500
Error: Número fuera de rango: 500
```

17. (NumeroNegativo) Crea un método (`sonPositivos`) que reciba dos números como argumentos y lance una excepción personalizada (`NumeroNegativoException`, que extiende `Exception`) si uno de los números es negativo. Captura esa excepción en el método principal y muestra un mensaje de error.

```
Introduce el primer número: 5
Introduce el segundo número: 4
Ambos números son positivos.

Introduce el primer número: -4
Introduce el segundo número: 5
Error: Uno de los números es negativo.
```

18. (LongitudCadena) Diseña un programa que lea una cadena de caracteres desde el teclado y, si la longitud de la cadena es mayor de 10 caracteres, lance una excepción personalizada. Captura esa excepción y muestra un mensaje de error.
19. (TemperaturaInvalida) Implementa una clase `ConversorTemperatura` que tenga un método para convertir grados Celsius a Fahrenheit. Si el valor en grados Celsius es inferior a -273.15, lanza una excepción personalizada. Captura la excepción y muestra un mensaje de error en el método principal.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

20. (EdadInvalida) Diseña una clase `ValidadorEdad` que tenga un método para validar si una persona tiene una edad válida (por ejemplo, entre 0 y 120 años). Si la edad no es válida, lanza una excepción personalizada y muestra un mensaje de error en el método principal.

Aserciones

1. (Aserciones1) A partir del siguiente fragmento de código, añade una linea debajo del comentario de la linea 4 que haga lo que se solicita:

```
1  class Main {
2      public static void main(String args[]) {
3          String[] finde = {"viernes", "sabado", "domingo"};
4          //Añade una aserción que compruebe que solo hay dos dias en el fin de semana.
5
6          System.out.println("Solo hay " + weekends.length + "  dias en el fin de semana");
7      }
8  }
```

2. (Aserciones2) Escribe un método llamado `validarEdad(int edad)` que acepte como parámetro la edad de una persona. Usa una aserción para verificar que la edad sea un valor positivo y menor que 150. Si la edad es negativa o extremadamente alta, la aserción debería fallar. Dispones del siguiente `main`:

```
1  public static void main(String[] args) {
2      // Caso que debe pasar la aserción
3      validarEdad(25);
4      // Caso que debe fallar la aserción (edad negativa)
5      validarEdad(-5);
6  }
```

3. (Aserciones3) Crea un método llamado `esPar(int numero)` que devuelva `true` si el número es par y `false` en caso contrario. Luego, escribe una aserción para verificar que el resultado es `true` cuando el número proporcionado es efectivamente par. Dispones del siguiente `main`:

```
1  public static void main(String[] args) {
2      // Caso que debe devolver true
3      assert esPar(4) : "El número 4 debería ser par";
4      // Caso que debe devolver false
5      assert !esPar(3) : "El número 3 no debería ser par";
6      // Hace saltar la excepción si las aserciones están activadas y alguna falla
7      assert esPar(5) : "Hará saltar la excepción";
8  }
```

4. (Aserciones4) Implementa un método llamado `dentroDeRango(int numero, int min, int max)` que devuelva `true` si el número está en el rango `[min, max]` y `false` en caso contrario. Usa aserciones para probar que el método devuelve `true` para un número dentro del rango y `false` para uno fuera.

```
1  // Ejemplo de uso:
2  assert dentroDeRango(5, 1, 10) : "El número 5 debería estar en el rango [1, 10]";
3  assert !dentroDeRango(15, 1, 10) : "El número 15 no debería estar en el rango [1, 10]";
```

4.2.3 Actividades

1. (TransformarBucle) Transforma el siguiente bucle for en un bucle while:

```
1  for (i=5; i<15; i++) {
2      System.out.println(i);
3  }
```

2. (NumerosPares) Escribe un método (`nPares`) que recibe un número `n` y que devuelve un `String` con los `n` primeros números pares. Escribe un método `main` que pida un número al usuario y use el método `nPares` para mostrarlos en pantalla.

```
1  Introduce un número entero positivo: 15
2  2 4 6 8 10 12 14 16 18 20 22 24 26 28 30
```

3. (Rango) Escribe un método (`rango`) que recibe dos enteros y que devuelve un `String` que comience con el primero y termine en el segundo. Se asume que el primero será menor que el segundo. Escribe un método `main` que pida los dos números al usuario y muestre el rango de números usando el método `rango`:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
Introduce el primer número del rango: 200
Introduce el segundo número del rango: 250
200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236
237 238 239 240 241 242 243 244 245 246 247 248 249 250
```

4. (TablasMultiplicar) Escribe un método (`tabla`) que recibe un número entero positivo y devuelve un String con la tabla de múltiplicar de dicho número con el siguiente formato:

```
Tabla del 2:
2 x 1 = 2
2 x 2 = 4
...
2 x 10 = 20
```

   Escribe un método main, que usando el método tabla imprima por pantalla las tablas del 1 al 10.

5. (SinMultiplos5) Programa que muestre los números del 1 al 100 sin mostrar los múltiplos de 5.

```
...
28
29
31
32
33
34
36
37
38
39
41
42
43
...
```

6. (CuadradoHastaNegativo) Leer un número y mostrar su cuadrado, repetir el proceso hasta que se introduzca un número negativo.
7. (PositivoNegativo) Leer un número e indicar si es positivo o negativo. El proceso se repetirá hasta que se introduzca un 0.
8. (ParImpar) Leer números hasta que se introduzca un 0. Para cada uno indicar si es par o impar.
9. (ContarNumeros) Pedir números hasta que se teclee uno negativo, y mostrar cuántos números se han introducido.
10. (AdivinarNumero) Realizar un juego para adivinar un número `X`. Para ello pedir un número `N`, y luego ir pidiendo números indicando "mayor" o "menor" según sea mayor o menor con respecto a `X`. El proceso termina cuando el usuario acierta.

```
Voy a pensar un número menor que 1000 para que lo adivines...

Introduce que número crees que he pensado: 500
El número que he pensado es menor que 500
Introduce que número crees que he pensado: 250
El número que he pensado es menor que 250
Introduce que número crees que he pensado: 125
El número que he pensado es mayor que 125
Introduce que número crees que he pensado: 187
El número que he pensado es mayor que 187
Introduce que número crees que he pensado: 210
El número que he pensado es mayor que 210
Introduce que número crees que he pensado: 230
El número que he pensado es mayor que 230
Introduce que número crees que he pensado: 240
El número que he pensado es mayor que 240
Introduce que número crees que he pensado: 245
Enhorabuena has acertado, el número que había pensado es el 245
```

11. (SumaNumeros) Pedir números hasta que se teclee un 0, mostrar la suma de todos los números introducidos.
12. (MediaNumeros) Pedir números hasta que se introduzca uno negativo, y calcular la media.
13. (NumerosHastaN) Pedir un número `N`, y mostrar todos los números del 1 al `N`.
14. (De100a0) Escribir todos los números del 100 al 0 de 7 en 7.
15. (Suma15Numeros) Pedir 15 números y escribir la suma total.
16. (ProductoImpares) Diseñar un programa que muestre el producto de los 10 primeros números impares.
17. (Factorial) Pedir un número y calcular su factorial (el factorial se representa con el simbolo `!`).

    Aquí tienes el factorial de los 5 primeros números enteros:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
1! = 1
2! = 2 * 1 = 2
3! = 3 * 2 * 1 = 6
4! = 4 * 3 * 2 * 1 = 24
5! = 5 * 4 * 3 * 2 * 1 = 120
```

Ejemplo de ejecución del programa:

```
Dime el número para calcular su factorial: 6
6! = 6 * 5 * 4 * 3 * 2 * 1 = 720
```

18. (MediaPosNeg) Pedir 10 números. Mostrar la media de los números positivos, la media de los números negativos y la cantidad de ceros.
19. (Sueldos1000) Pedir 10 sueldos. Mostrar su suma y cuantos hay mayores de 1000€.
20. (AlumnosEdadAltura) Dadas las edades y alturas de 5 alumnos, mostrar la edad y la estatura media, la cantidad de alumnos mayores de 18 años, y la cantidad de alumnos que miden más de 1.75.
21. (TablaMultiplicar) Pide un número (que debe estar entre 0 y 10) y mostrar la tabla de multiplicar de dicho número.
22. (AprobadosSuspensos) Dadas 6 notas, escribir la cantidad de alumnos aprobados y suspensos.
23. (SueldoMaximo) Pedir un número `N`, introducir `N` sueldos, y mostrar el sueldo máximo.
24. (HayNegativo) Pedir 10 números, y mostrar al final si se ha introducido alguno negativo.
25. (HaySuspenso) Pedir 5 calificaciones de alumnos y decir al final si hay algún suspenso.
26. (Multiplo3) Pedir 5 números e indicar si alguno es múltiplo de 3.
27. (SaludoHorario) Realiza un programa que pida una hora por teclado y que muestre luego buenos días, buenas tardes o buenas noches según la hora. Se utilizarán los tramos de 6 a 12, de 13 a 20 y de 21 a 5. respectivamente. Sólo se tienen en cuenta las horas, los minutos no se deben introducir por teclado.
28. (DiaSemana) Escribe un programa en que dado un número del 1 a 7 escriba el correspondiente nombre del día de la semana.
29. (SalarioHorasExtras) Escribe un programa que calcule el salario semanal de un trabajador teniendo en cuenta que las horas ordinarias (40 primeras horas de trabajo) se pagan a 12 euros la hora. A partir de la hora 41, se pagan a 16 euros la hora.
30. (MediaTresNotas) Realiza un programa que calcule la media de tres notas.
31. (BoletinNotas) Amplía el programa anterior para que diga la nota del boletín (insuficiente, suficiente, bien, notable o sobresaliente).
32. (Horoscopo) Escribe un programa que nos diga el horóscopo a partir del día y el mes de nacimiento.
33. (Cuestionario) Realiza un minicuestionario con 4 preguntas tipo test sobre las asignaturas que se imparten en el curso. Cada pregunta acertada sumará un punto. El programa mostrará al final la calificación obtenida.
34. (NotaProgramacion) Calcula la nota de un trimestre de la asignatura Programación. El programa pedirá las dos notas que ha sacado el alumno en los dos primeros controles. Si la media de los dos controles da un número mayor o igual a 5, el alumno está aprobado y se mostrará la media. En caso de que la media sea un número menor que 5, el alumno habrá tenido que hacer el examen de recuperación que se califica como apto o no apto, por tanto se debe preguntar al usuario ¿Cuál ha sido el resultado de la recuperación? (apto/no apto). Si el resultado de la recuperación es apto, la nota será un 5; en caso contrario, la nota será 1.

    Ejemplo 1:

    ```
    Nota del primer control: 7 Nota del segundo control: 10
    Tu nota de Programación es 8.5
    ```

    Ejemplo 2:

    ```
    Nota del primer control: 6 Nota del segundo control: 3
    ¿Cuál ha sido el resultado de la recuperación? (apto/no apto): apto
    Tu nota de Programación es 5
    ```

    Ejemplo 3:

    ```
    Nota del primer control: 6 Nota del segundo control: 3
    ¿Cuál ha sido el resultado de la recuperación? (apto/no apto): no apto
    Tu nota de Programación es 1
    ```

35. (Multiplos5For) Muestra los números múltiplos de 5 entre el 0 y el 100 utilizando un bucle `for`.
36. (Multiplos5While) Muestra los números múltiplos de 5 entre el 0 y el 100 utilizando un bucle `while`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

37. (Multiplos5DoWhile) Muestra los números múltiplos de 5 entre el 0 y el 100 utilizando un bucle `do while`.
38. (ContarAtrasFor) Muestra los números del 320 al 160, contando de 20 en 20 hacia atrás utilizando un bucle `for`.
39. (ContarAtrasWhile) Muestra los números del 320 al 160, contando de 20 en 20 hacia atrás utilizando un bucle `while`.
40. (ContarAtrasDoWhile) Muestra los números del 320 al 160, contando de 20 en 20 utilizando un bucle `do-while`.
41. (CajaFuerte) Realiza el control de acceso a una caja fuerte. La combinación será un número de 4 cifras. El programa nos pedirá la combinación para abrirla. Si no acertamos, se nos mostrará el mensaje "**Esa no es la combinación correcta**" y si acertamos se nos dirá "**La caja se ha abierto correctamente**". Tendremos cuatro oportunidades para abrir la caja fuerte, si lo sobrepasamos nos dirá "**Has sobrepasado el número de intentos permitido**".

    > ⚠ **Atención**
    >
    > Para pasar satisfactoriamente los tests, la combinación de apertura debe estar configurada en "1234"

42. (CuadradoCubo) Escribe un programa que muestre en tres columnas, el cuadrado y el cubo de los 5 primeros números enteros a partir de uno que se introduce por teclado.
43. (Potencia) Escribe un método (`potencia`) que reciba una base y un exponente (enteros positivos) y que calcule la potencia (sin usar `Math`). Escribe un método `main` para pedir los datos al usuario y usando el método `potencia` mostrar el resultado.

```
Vamos a calcular una potencia.
Introduce la BASE: 6
Introduce el EXPONENTE: 5
El resultado de elevar la BASE: 6 al EXPONENTE: 5 resulta en: 7776
```

44. (Suma100Siguientes) Realiza un programa que sume los 100 números siguientes a un número entero y positivo introducido por teclado. Se debe comprobar que el dato introducido es correcto (que es un número positivo).
45. (NumerosEntre7) Escribe un programa que imprima por pantalla los números enteros comprendidos entre dos números introducidos por teclado y validados como distintos, el programa debe empezar por el menor de los enteros introducidos e ir incrementando de 7 en 7.

```
Introduce dos números DIFERENTES!
Introduce el primer número: 8
Introduce el segundo número: 8
Los números no son DIFERENTES!

Introduce dos números DIFERENTES!
Introduce el primer número: 6
Introduce el segundo número: 50
6
13
20
27
34
41
48
```

46. (EstadisticasNumeros) Realiza un programa que vaya pidiendo números hasta que se introduzca un numero negativo y nos diga cuantos números se han introducido, la media de los impares y el mayor de los pares. El número negativo sólo se utiliza para indicar el final de la introducción de datos pero no se incluye en el cómputo.
47. (SumaHasta10000) Escribe un programa que permita ir introduciendo una serie indeterminada de números mientras su suma no supere el valor 10000. Cuando esto último ocurra, se debe mostrar el total acumulado, el contador de los números introducidos y la media.
48. (Multiplos3) Escribe un programa que muestre, cuente y sume los múltiplos de 3 que hay entre 1 y un número leído por teclado.
49. (PrecioFinal) Escribe un programa que calcule el precio final de un producto según su base imponible (precio antes de impuestos), el tipo de IVA aplicado (general, reducido o superreducido) y el código promocional. Los tipos de IVA general, reducido y superreducido son del 21%, 10% y 4% respectivamente. Los códigos promocionales pueden ser nopro, mitad, meno5 o 5porc que significan respectivamente que no se aplica promoción, el precio se reduce a la mitad, se descuentan 5 euros o se descuenta el 5%.

    Ejemplo:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
Introduzca la base imponible: 25
Introduzca el tipo de IVA (general, reducido o superreducido): reducido
Introduzca el código promocional (nopro, mitad, meno5 o 5porc): mitad
Base imponible 25.00
Cód. promo. (mitad): -12.50
IVA (10%) 1.25
Precio con IVA 13.75
TOTAL 13.75
```

50. (AnioBisiesto) Pedir un año e indicar si es bisiesto, teniendo en cuenta que son bisiestos todos los años divisibles por 4, excluyendo los que sean divisibles por 100, pero no los que sean divisibles por 400.

    En pseudocódigo se calcularía así:

```
SI ((año divisible por 4) Y ((año no divisible por 100) O (año divisible por 400)))ENTONCES
    es bisiesto
SINO
    no es bisiesto
FIN_SI
```

51. (NumeroALetras) Pedir un número de 20 a 99 y mostrarlo escrito. Por ejemplo, para 56 mostrar: cincuenta y seis.
52. (VehiculoIVA) Introducir datos de un vehículo (marca, modelo y precio). Devolver el precio con IVA del vehículo. Controlar con Excepciones que el precio del vehículo introducido son números y que el cálculo de Precio Final con IVA no devuelva error.
53. (NotaMediaAlumnos) Introducir códigos de alumnos, nombre y nota hasta que se introduzca un código de alumno negativo. Devolver la nota media de los alumnos la clase. Controlar con Excepciones que las notas introducidas son números y que si no se introducen alumnos el cálculo de la media no devuelva error.
54. (ImporteFinal) Crear una función o método llamado `impFinal`, que calcule el importe final de una compra. Los parámetros que se le pasarán a la función son el `precio` del producto, las `cantidad de unidades` compradas, el `porcentaje de iva` y el `porcentaje de descuento`. El método principal debe pedir por teclado el precio del producto, las unidades adquiridas, el porcentaje de IVA y el porcentaje de descuento y devolver el `Importe final` de la Factura.
55. (CapacidadDisco) Crear una función que calcule la capacidad de un disco. La capacidad se calcula multiplicando los Cabezales o pistas del disco por los Cilindros por los Sectores por Tamaño de Sector. El método principal debe pedir por teclado los Cabezales o Pistas del disco, los Cilindros, Sectores y Tamaño de Sector y devolver la Capacidad del disco en Gigabytes.

    Ejemplo

    Por ejemplo: Calcular la capacidad de un disco teniendo en cuenta que dispone de 10 Cabezales o Pistas, 65535 Cilindros, 1024 Sectores/pista y un Tamaño de 512 bytes/sector:

    Capacidad del disco = 10 * 65535 * 1024 * 512 = 343597383680 bytes

    343597383680 bytes / 1024 / 1024 / 1024 = 320 Gbytes

56. (MayorDeTres) Función que devuelva el mayor de tres números. El método principal debe pedir por teclado los tres números introducidos por el teclado. La función debe recibir como parámetros los tres números y devolver el mayor.

21 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

4.3 Talleres

4.3.1 Taller UD03_01: Classroom 50

Aceptar la tarea en Classroom 50

Aceptamos el *Assignement* (la tarea/ejercicio) a partir del link del profesor, en este caso: https://classroom50.org/IES-Eduardo-Primo-Marques/1dam-programacio-java/assignments/ud03

Si no recuerdas cómo aceptar la tarea, clonar el repositorio o abrirlo en IntelliJ, repasa la guía completa en el taller de UD02.

Tarea

Debes enviar tus soluciones a Classroom 50 y superar al menos la mitad de los tests, cuantos más tests superados, mejor nota tendrás en la tarea.

6 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

4.3.2 Taller UD03_02: Acepta el reto

Inscribete en Acepta el reto

Inscribete en la web www.aceptaelreto.com seleccionando nuestro instituto.

Tarea

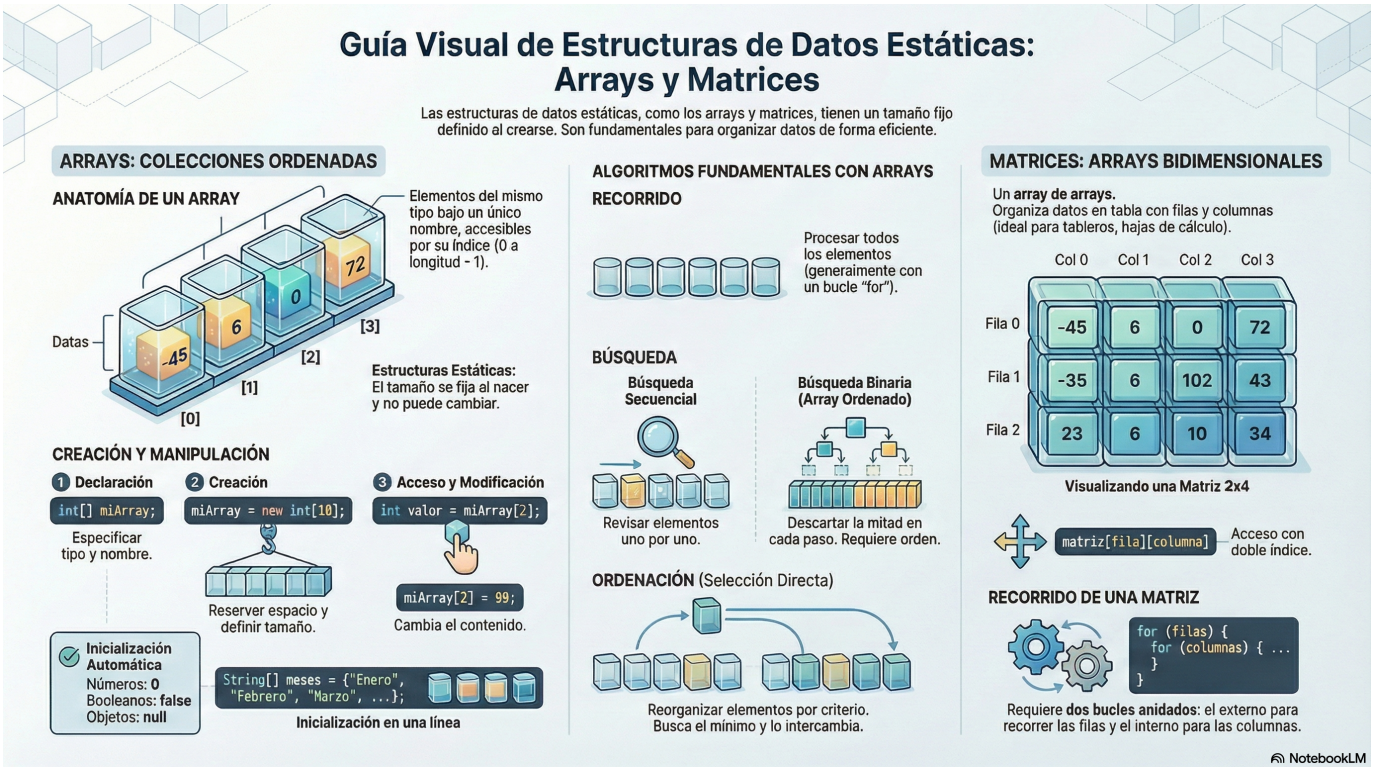
Haz una captura de pantalla de tu perfil una vez hecho login con tu usuario y envia la captura en un pdf al profesor.

21 de noviembre de 2025

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

5. UD04

5.1 Estructuras de datos: Arrays y matrices. Recursividad.



Al finalizar esta unidad serás capaz de...

- Declarar y utilizar arrays unidimensionales
- Recorrer arrays con bucles y for-each
- Trabajar con arrays multidimensionales (matrices)
- Conocer la clase Arrays y sus métodos útiles
- Comprender el concepto de recursividad
- Identificar el caso base y el caso recursivo
- Implementar funciones recursivas simples

5.1.1 Introducción

A menudo, para resolver problemas de programación, no basta con disponer de sentencias condicionales o iterativas como las que hemos visto (`if`, `switch`, `while`, `for`, ...).

También es necesario disponer de herramientas para organizar la información de forma adecuada: **las estructuras de datos**.

Los arrays son una estructura de datos fundamental, que está disponible en la mayoría de lenguajes de programación y que nos permitirá resolver problemas que, sin ellos, resultarían difíciles o tediosos de solucionar.

Imaginemos, por ejemplo, que queremos leer los datos de pluviosidad de cada uno de los 31 días de un mes. Posteriormente se desea mostrar la pluviosidad media del mes y en cuántos días las lluvias superaron la media.

Con las herramientas de que disponemos hasta ahora, nos veríamos obligados a declarar **31 variables** `double`, una para cada día, y a elaborar un largo programa que leyera los datos y contara cuales superan la media. Con el uso de arrays, problemas como este tienen una solución fácil y corta.

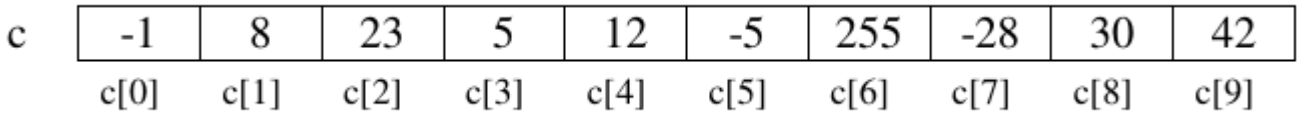
© 2025 David Martínez licensed under CC BY-NC-SA 4.0

5.1.2 Arrays

Un array es una colección de elementos del mismo tipo, que tienen un nombre o identificador común.

Se puede acceder a cada componente del array de forma individual para consultar o modificar su valor. El acceso a los componentes se realiza mediante un subíndice, que viene dado por la posición que ocupa el elemento dentro del array.

En la siguiente figura se muestra un array `c` de enteros:



> **Importante**
>
> El primer subíndice de un array es el cero. El último subíndice es la longitud del array menos uno.

El número de componentes de un array se establece inicialmente al crearlo y no es posible cambiarlo de tamaño. Es por esto que reciben el nombre de estructuras de datos estáticas.

Declaración y creación

Para poder utilizar un array hay que declararlo:

```
tipo nombreVariable[];
```

o

```
tipo[] nombreVariable;
```

En la declaración se establece el nombre de la variable y el tipo de los componentes. Por ejemplo:

```
double lluvia1[]; // lluvia1 es un array de double
double[] lluvia2;  // lluvia2 es un array de double <== Esta es la declaración recomendada, el [] siempre acompañando al tipo.
```

En la declaración anterior no se ha establecido el número de componentes. El número de componentes se indica en la creación, que se hace utilizando el operador `new`:

```
lluvia1 = new double[31];
```

Con esta instrucción se establece que el número de elementos del array `lluvia` son 31, reservando con ello el compilador espacio consecutivo para 31 componentes individuales de tipo `double`.

Las dos instrucciones anteriores se pueden unir en una sola:

```
// tipo[] nombreVariable = new tipo[numElementos];
double[] lluvia2 = new double[31];
```

El valor mediante el cual se define el número de elementos del array tiene que ser una expresión entera, pero no tiene por qué ser un literal como en el ejemplo anterior. El tamaño de un array se puede establecer durante la ejecución, como en el Ejemplo01.

En el Ejemplo01 encontrarás el código completo con todos los ejemplos de esta unidad sobre arrays, matrices, búsqueda y ordenación.

```
// usamos un array para almacenar las edades de un grupo de personas
// la variable numPersonas contiene el número de personas del grupo
// y se asigna en tiempo de ejecución
Scanner teclado = new Scanner(System.in);
System.out.print("Introduce cuantos elementos debe tener el array edad[]:");
int numPersonas = teclado.nextInt();
int[] edad = new int[numPersonas];
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Acceso a los componentes

Como ya hemos dicho, el acceso a los componentes del array se realiza mediante subíndices. La sintaxis para referirse a un componente del array es la siguiente:

```
nombreVariable[subíndice]
```

Tras declarar el array `lluvia`, se dispone de 31 componentes de tipo `double` numeradas desde la 0 a la 30 y accesibles mediante la notación: `lluvia[0]` (componente primera), `lluvia[1]` (componente segunda) y así sucesivamente hasta la última componente: `lluvia[30]`.

Con cada una de las componentes del array de `double` lluvia es posible efectuar todas las operaciones que podrían realizarse con variables individuales de tipo `double`, por ejemplo, dadas las declaraciones anteriores, las siguientes instrucciones serían válidas:

```
int[] edad = new int[3];
System.out.print("Introduce el dato para el componente 0: ");
edad[0] = teclado.nextInt(); //25
System.out.println("El componente [0] vale " + edad[0]);
edad[1] = edad[0] + 1;
edad[2] = edad[0] + edad[1];
edad[2]++;
System.out.println("El componente [1] vale " + edad[1]); //26
System.out.println("El componente [2] vale " + edad[2]); //52
```

Además, hay que tener en cuenta que el subíndice ha de ser una expresión entera, por lo que también son válidas expresiones como las siguientes:

```
int i;
...
edad[i] = edad[i + 1];
edad[i + 2] = edad[i];
```

Inicialización

Cuando creamos un array, Java inicializa automáticamente sus componentes:

- Con 0 cuando los componentes son de tipo numérico.
- Con false cuando los componentes son `boolean`.
- Con el carácter de ASCII 0, cuando los componentes son `char`.
- Con `null` cuando son objetos (`Strings`, etc)

Aun así, es probable que estos no sean los valores con los que queremos inicializar el array. Tenemos entonces dos posibilidades:

- Acceder individualmente a los componentes del array para darles valor:

```
int edad2[] = new int[10];
edad2[0] = 25;
edad2[1] = 10;
...
edad2[9] = 12;
```

- O inicializar el array en la declaración de la siguiente forma:

```
int edad3[] = {25,10,23,34,65,23,1,67,54,12};
```

> **Más información**
>
> Es decir, enumerando los valores con los que se quiere inicializar cada componente, encerrados entre llaves. De hacerlo así, no hay que crear el array con el operador `new`. Java crea el array con tantos componentes como valores hemos puesto entre llaves. Además no es necesario indicar el número de elementos del mismo.

Un ejemplo práctico

Ya hemos resuelto en temas anteriores el problema de devolver el nombre de un mes dado su número.

Vamos a resolverlo ahora ayudándonos de arrays:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
public static String nombreMes(int mes){
    String[] nombre = {"enero", "febrero", "marzo", "abril",
                       "mayo", "junio", "julio","agosto",
                       "septiembre", "octubre", "noviembre", "diciembre"};
    return nombre[mes-1];
}
```

El método define un array de `String` que se inicializa con los nombres de los doce meses. La primera componente del array (`nombre[0]`) se deja vacía, de forma que enero quede almacenado en `nombre[1]` . Devolver el nombre del mes indicado se reduce a devolver el componente del array cuyo número indica el parámetro mes: `nombre[mes]`

Arrays como parámetros. Paso de parámetros por referencia.

Hasta el momento sólo se ha considerado el paso de parámetros por valor; de manera que cualquier cambio que el método realice sobre los parámetros formales no modifica el valor que tiene el parámetro real con el que se llama al método. En java, todos los parámetros de tipo simple (`byte` , `short` , `int` , ...) se pasan por valor.

> **Importante**
>
> Por el contrario, los arrays no son variables de tipo primitivo, y como cualquier otro objeto, se pasa siempre por referencia.

En el paso de parámetros por referencia lo que se pasa en realidad al método es la dirección de la variable u objeto. Es por esto que el papel del parámetro formal es el de ser una referencia al parámetro real; la llamada al método no provoca la creación de una nueva variable. De esta forma, las modificaciones que el método pueda realizar sobre estos parámetros se realizan efectivamente sobre los parámetros reales. En este caso, ambos parámetros (formal y real) se pueden considerar como la misma variable con dos nombres, uno en el método llamante y otro en el llamado o invocado, pero hacen referencia a la misma posición de memoria.

En el siguiente ejemplo, la variable `a` , de tipo primitivo, no cambia de valor tras la llamada al método. Sin embargo la variable `v` , array de enteros, si se ve afectada por los cambios que se han realizado sobre ella en el método:

```
public static void main(String[] args){
    int a = 1;
    int[] v = {1,1,1};
    metodo(v,a); //Pasar un array como parámetro
    System.out.println(a); // Muesta 1
    System.out.println(v[0]); // Muestra 2
}

public static void metodo(int[] x, int y){ //recibir un array como parámetro
    x[0]++;
    y++;
}
```

> **Importante**
>
> Como podemos observar, para pasar un array a un método, simplemente usamos el nombre de la variable en la llamada. En la cabecera del método, sin embargo, tenemos que utilizar los corchetes `[]` para indicar que el parámetro es un array.

El atributo `length`

Todas las variables de tipo array tienen un atributo `length` que permite consultar el número de componentes del array. Su uso se realiza posponiendo `.length` al nombre de la variable:

```
double[] estatura = new double[25];
...
System.out.println(estatura.length); // Mostrará por pantalla: 25
```

`String[] args` en el `main`

El método `main` puede recibir argumentos desde la línea de comandos. Para ello, el método `main` recibe un parámetro (`String args[]`). Vemos que se trata de un array de `Strings` . El uso del atributo `length` nos permite comprobar si se ha llamado al programa de forma correcta o no. Veamos un ejemplo para saber si es Navidad. Se habrá llamado correctamente si el array `args` contiene dos componentes (día, mes):

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
public class EsNavidad {
    public static void main(String[] args) {
        if (args.length != 2) {
            System.out.println("ERROR:");
            System.out.println("Llame al programa de la siguiente forma:");
            System.out.println("java EsNavidad dia mes");
        } else {
            // args[0] es el día
            // args[1] es el mes
            if ((Integer.valueOf(args[0]) == 25) && (Integer.valueOf(args[1]) == 12)) {
                  System.out.println("ES NAVIDAD!");
            } else {
                System.out.println("No es navidad.");
            }
        }
    }
}
```

5.1.3 Problemas de recorrido, búsqueda y ordenación

Muchos de los problemas que se plantean cuando se utilizan arrays pueden clasificarse en tres grandes grupos de problemas genéricos: los que conllevan el recorrido de un array, los que suponen la búsqueda de un elemento que cumpla cierta característica dentro del array, y los que implican la ordenación de los elementos del array.

La importancia de este tipo de problemas proviene de que surgen, no sólo en el ámbito de los arrays, sino también en muchas otras organizaciones de datos de uso frecuente (como las listas, los ficheros, etc.). Las estrategias básicas de resolución que se verán a continuación son también extrapolables a esos otros ámbitos.

Problemas de recorrido

Se clasifican como problemas de recorrido aquellos que para su resolución exigen algún tratamiento de todos elementos del array. El orden para el tratamiento de estos elementos puede organizarse de muchas maneras: ascendentemente, descendentemente, ascendente y descendente de forma simultánea, etc.

En el siguiente ejemplo se muestra un método en java para devolver, a partir de un array que contiene la pluviosidad de cada uno de los días de un mes, la pluviosidad media de dicho mes. Para ello se recorren ascendentemente los componentes del array para ir sumándolos:

```java
public static double pluviosidadMediaAscendente(double[] lluvia){
    double suma = 0;
    //Recorremos el array ascendentemente
    for (int i = 0; i<lluvia.length; i++){
        suma += lluvia[i];
    }
    double media = suma / lluvia.length;
    return media;
}
```

La forma de recorrer el array ascendentemente es, como vemos, utilizar una variable entera (`i` en nuestro caso) que actúa como subíndice del array. Éste subíndice va tomando los valores `0`, `1`, ..., `lluvia.length-1` en el seno de un bucle, de manera que se accede a todos los componentes del array para sumarlos.

El mismo problema resuelto con un recorrido descendente sería como sigue:

```java
public static double pluviosidadMediaDescendente(double[] lluvia){
    double suma = 0;
    //Recorremos el array descendentemente
    for (int i = lluvia.length-1; i>=0; i--){
        suma += lluvia[i];
    }
    double media = suma / lluvia.length;
    return media;
}
```

También realizamos un recorrido para obtener la pluviosidad máxima del mes (la cantidad de lluvia más grande caída en un día), es decir, el elemento más grande del array:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
public static double pluviosidadMaxima(double[] lluvia){
    // Suponemos que la pluviosidad máxima se produjo el primer día
    double max = lluvia[0];
    //Recorremos el array desde la posición 1, comprobando si hay una pluviosidad mayor
    for (int i = 1; i<lluvia.length; i++)
        if(lluvia[i] > max){
            max = lluvia[i];
        }
    return max;
}
```

BUCLE FOR EACH (FOR-LOOP)

En el tema anterior vimos algún tipo de bucles que explicariamos cuandos los pudiesemos utilizar, en este grupo estan los bucles for each o for-loops. Aquí tenemos un ejemplo de recorrido de un array con la sintaxis que ya conocemos:

```java
int[] array = { 1, 2, 3, 4, 5, 6, 7, 8 };
for (int i = 0; i < array.length; i++) {
    System.out.print(array[i] + " ");
}
```

el anterior fragmento genera la siguiente salida:

```
1 2 3 4 5 6 7 8
```

Este mismo código se puede escribir de la siguiente manera:

```java
int[] array = { 1, 2, 3, 4, 5, 6, 7, 8 };
for (int i : array) { //mentalmente podemos traducir por:
    //"para cada entero "i" que encontremos en el array"
    System.out.print(i + " ");
}
```

la salida seguirá siendo la misma:

```
1 2 3 4 5 6 7 8
```

> **Cuidado**
>
> Con el segundo método no tenemos acceso a la posición o índice del array, este método no serviría para métodos en los que necesitamos conocer la posición o utilizarla de alguna manera. Traducimos el método de `pluviosidadMedia` con un bucle `loop`:

```java
public static double pluviosidadMediaLoop(double[] lluvia){
    double suma = 0;
    //Recorremos el array con el loop
    for (int i : lluvia){
        suma += i;
    }
    double media = suma / lluvia.length;
    return media;
}
```

Problemas de búsqueda

Se denominan problemas de búsqueda a aquellos que, de alguna manera, implican determinar si existe algún elemento del array que cumpla una propiedad dada. Con respecto a los problemas de recorrido presentan la diferencia de que no es siempre necesario tratar todos los elementos del array: el elemento buscado puede encontrarse inmediatamente, encontrarse tras haber recorrido todo el array, o incluso no encontrarse.

BÚSQUEDA ASCENDENTE

Consideremos, por ejemplo, el problema de encontrar cual fue el primer día del mes en que no llovió nada, es decir, el primer elemento del array con valor cero:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
//Devolveremos el subíndice del primer componente del array cuyo valor es cero.
// Si no hay ningún día sin lluvias devolveremos -1
public static int primerDiaSinLluvia1(double[] lluvia){
    int i=0 ;
    boolean encontrado = false ;
    while (i<lluvia.length && !encontrado){
        if (lluvia[i] == 0) encontrado = true ;
        else i++ ;
    }
    if (encontrado) return i ;
    else return -1 ;
}
```

Hemos utilizado el esquema de búsqueda: Definimos una variable `boolean` que indica si hemos encontrado o no lo que buscamos. El bucle se repite mientras no lleguemos al final del array y no hayamos encontrado un día sin lluvias.

También es posible una solución sin utilizar la variable `boolean`:

```
public static int primerDiaSinLluvia2(double[] lluvia){
    int i=0 ;
    while (i<lluvia.length && lluvia[i] != 0)
        i++;
    if (i == lluvia.length) return -1 ;
    else return i;
}
```

En este caso el subíndice `i` se incrementa mientras estemos dentro de los límites del array y no encontremos un día con lluvia `0`. Al finalizar el bucle hay que comprobar por cual de las dos razones finalizó: ¿Se encontró un día sin lluvias o se recorrió todo el array sin encontrar ninguno? En esta comprobación es importante no acceder al array si existe la posibilidad de que el subíndice esté fuera de los límites del array. La siguiente comprobación sería **incorrecta**:

```
if (lluvia[i] == 0) return i;
else return -1;
```

ya que, si se ha finalizado el bucle sin encontrar ningún día sin lluvia, `i` valdrá `lluvia.length`, que no es una posición válida del array, y al acceder a `lluvia[i]` se producirá la excepción `ArrayIndexOutOfBoundsException` (índice del array fuera de los límites)

Por otra parte, el mismo problema se puede resolver utilizando la sentencia `for`, como hemos hecho otras veces. Sin embargo la solución parece menos intuitiva porque el cuerpo del `for` quedaría vacío:

```
public static int primerDiaSinLluvia3(double[] lluvia){
    int i;
    for (i=0; i<lluvia.length && lluvia[i] != 0; i++) /*Nada*/ ;
    if (i == lluvia.length) return -1 ;
    else return i;
}
```

Otra opción más:

```
public static int primerDiaSinLluvia4(double[] lluvia){
    int i=0 ;
    while (i<lluvia.length){
        if (lluvia[i++] == 0) return i ;
    }
    return -1 ;
}
```

BÚSQUEDA DESCENDENTE

En los ejemplos de búsqueda anteriores hemos iniciado la búsqueda en el elemento cero y hemos ido ascendiendo hasta la última posición del array. A esto se le llama búsqueda ascendente.

Si queremos encontrar el último día del mes en que no llovió podemos realizar una búsqueda descendente, es decir, partiendo del último componente del array y decrementando progresivamente el subíndice hasta llegar a la posición cero o hasta encontrar lo buscado:

```
public static int ultimoDiaSinLluvia(double[] lluvia){
    int i=lluvia.length-1;
    boolean encontrado = false ;
    while (i>=0 && !encontrado){
        if (lluvia[i] == 0) encontrado = true ;
        else i-- ;
    }
    if (encontrado) return i ;
    else return -1 ;
}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

BÚSQUEDA EN UN ARRAY ORDENADO: BÚSQUEDA BINARIA

Suponga que una amiga apunta un número entre el 0 y el 99 en una hoja de papel y vosotros debéis adivinarlo. Cada vez que conteste, le dirá si el valor que ha dicho es mayor o menor que el que ha de adivinar. ¿Qué estrategia seguiría para lograrlo? Hay que pensar un algoritmo a seguir para resolver este problema.

Una aproximación muy ingenua podría ser ir diciendo todos los valores uno por uno, empezando por 0. Está claro que cuando llegue al 99 lo habréis adivinado. En el mejor caso, si había escrito el 0, acertará en la primera, mientras que en el peor caso, si había escrito el 99, necesitareis 100 intentos. Si estaba por medio, tal vez con 40-70 basta. Este sería un algoritmo eficaz (hace lo que tiene que hacer), pero no muy eficiente (lo hace de la mejor manera posible). Ir probando valores al azar en lugar de hacer esto tampoco mejora gran cosa el proceso, y viene a ser lo mismo.

Si alguna vez habeis jugado a este juego, lo que habreis hecho es ser un poco más astutos y empezar por algún valor del medio. En este caso, por ejemplo, podría ser el 50. Entonces, en caso de fallar, una vez está seguro de si el valor secreto es mayor o menor que su respuesta, en el intento siguiente probar un valor más alto o más bajo , e ir haciendo esto repetidas veces.

Generalmente, la mejor estrategia para adivinar un número secreto entre 0 y N sería primer probar N/2. Si no se ha acertado, entonces si el número secreto es más alto se intenta adivinar entre (N/2 + 1) y N. Si era más bajo, se intenta adivinar el valor entre 0 y N-1. Para cada caso, se vuelve a probar el valor que hay en el medio del nuevo intervalo. Y así sucesivamente, haciendo cada vez más pequeño el intervalo de búsqueda, hasta adivinarlo. En el caso de 100 valores, esto garantiza que, en el peor de los casos, en 7 intentos seguro que se adivina. Esto es una mejora muy grande respecto al primer algoritmo, donde hacían falta 100 intentos, y por tanto, este sería un algoritmo más eficiente. Concretamente, siempre se adivinará en $\log_2$ (N) intentos como máximo.

Si os fijáis, el ejemplo que se acaba de explicar, en realidad, no es más que un esquema de búsqueda en una secuencia de valores, como puede ser dentro de un array, partiendo de la condición que todos los elementos estén ordenados de menor a mayor. De hecho, hasta ahora, para hacer una búsqueda de un valor dentro de un array se ha usado el sistema "ingenuo", mirando una por una todas las posiciones. Pero si los elementos están ordenados previamente, se podría usar el sistema "astuto" para diseñar un algoritmo mucho más eficiente, y hasta cierto punto, más "inteligente".

El algoritmo basado en esta estrategia se conoce como **búsqueda binaria o dicotómica**.

Para ello iniciaremos la búsqueda en la posición central del array.

- Si el elemento central es el buscado habremos finalizado la búsqueda.
- Si el elemento central es mayor que el buscado, tendremos que continuar la búsqueda en la mitad izquierda del array ya que, al estar éste ordenado todos los elementos de la mitad derecha serán también mayores que el buscado.
- Si el elemento central es menor que el buscado, tendremos que continuar la búsqueda en la mitad derecha del array ya que, al estar éste ordenado todos los elementos de la mitad izquierda serán también menores que el buscado.

En un solo paso hemos descartado la mitad de los elementos del array. Para buscar el la mitad izquierda o en la mitad derecha utilizaremos el mismo criterio, es decir, iniciaremos la búsqueda en el elemento central de dicha mitad, y así sucesivamente hasta encontrar lo buscado o hasta que descubramos que no está.

Supongamos por ejemplo que, dado un array que contiene edades de personas, ordenadas de menor a mayor queremos averiguar si hay alguna persona de 36 años o no.

El siguiente método soluciona este problema realizando una búsqueda binaria:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
public static boolean hayAlguienDe36(int[] edad) {
    // Las variables izq y der marcarán el fragmento del array en el que
    // realizamos la búsqueda. Inicialmente buscamos en todo el array.
    final int NUMERO_BUSCADO = 36;
    int izq = 0;
    int der = edad.length - 1;
    boolean encontrado = false;
    while (izq <= der && !encontrado) {
        // Calculamos posición central del fragmento en el que buscamos
        int m = (izq + der) / 2;
        if (edad[m] == NUMERO_BUSCADO) // Hemos encontrado una persona de 36
        {
            encontrado = true;
        } else if (edad[m] > NUMERO_BUSCADO) {
            // El elemento central tiene más de 36.
            // Continuamos la búsqueda en la mitad izquierda. Es decir,
            // entre las posiciónes izq y m-1
            der = m - 1;
        } else {
            // El elemento central tiene menos de 36.
            // Continuamos la búsqueda en la mitad derecha. Es decir,
            // entre las posiciones m+1 y der
            izq = m + 1;
        } // del if
    } // del while
    return encontrado; // if (encontrado) return true; else return false;
}
```

La búsqueda finaliza cuando encontramos una persona con 36 años (`encontrado==true`) o cuando ya no es posible encontrarla, circunstancia que se produce cuando `izq` y `der` se cruzan (`izq>der`).

Problemas de ordenación

Con frecuencia necesitamos que los elementos de un array estén ordenados (por ejemplo para usar la búsqueda binaria).

Existen multitud de algoritmos que permiten ordenar los elementos de un array, entre los que hay soluciones **iterativas** y soluciones **recursivas**.

Entre los algoritmos **iterativos** tenemos, por ejemplo, el **método de la burbuja**, el **método de selección directa** y el **método de inserción directa**.

Entre los **recursivos**, son conocidos el algoritmo **mergesort** y el **quickSort**, que realizan la ordenación más rápidamente que los algoritmos iterativos que hemos nombrado.

Como ejemplo vamos a ver como se realiza la ordenación de un array de enteros utilizando el método de **selección directa**:

```java
public static void seleccionDirecta(int[] v) {
    for (int i = 0; i < v.length-1; i++) {
        //Localizamos elemento que tiene que ir en la posición i
        int posMin = i;
        //buscar el menor a la derecha
        for (int j = i + 1; j < v.length; j++) {
            if (v[j] < v[posMin]) {
                posMin = j;
            }
        }
        //al llegar aquí posMin tendrá la posición del elemento menor
        //Intercambiamos los elementos de las posiciones i y posMin
        //v[i]<=>v[posMin];
        int aux = v[posMin];
        v[posMin] = v[i];
        v[i] = aux;
    }
}
```

El método consiste en recorrer el array ascendentemente a partir de la posición cero.

En cada posición (`i`) localizamos el elemento que tiene que ocupar dicha posición cuando el array esté ordenado, es decir, el menor de los elementos que quedan a su derecha.

Cuando se ha determinado el menor se coloca en su posición realizando un intercambio con el elemento de la posición `i` . Con ello, el array queda ordenado hasta la posición `i` .

Y a modo de curiosidad os dejo por aquí el método de inserción directa:

1. Comenzamos considerando el primer elemento como la parte ordenada.
2. Luego, tomamos un elemento de la parte no ordenada y lo insertamos en la posición correcta dentro de la parte ordenada, desplazando los elementos mayores que él hacia la derecha.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

3. Repetimos este proceso hasta que todos los elementos estén en la parte ordenada.

```java
public static void insercionDirecta(int[] array) {
    for (int i = 1; i < array.length; i++) {
        int key = array[i];
        int j = i - 1;

        // Mover los elementos mayores que key hacia la derecha
        while (j >= 0 && array[j] > key) {
            array[j + 1] = array[j];
            j--;
        }

        // Insertar key en su posición correcta
        array[j + 1] = key;
    }
}
```

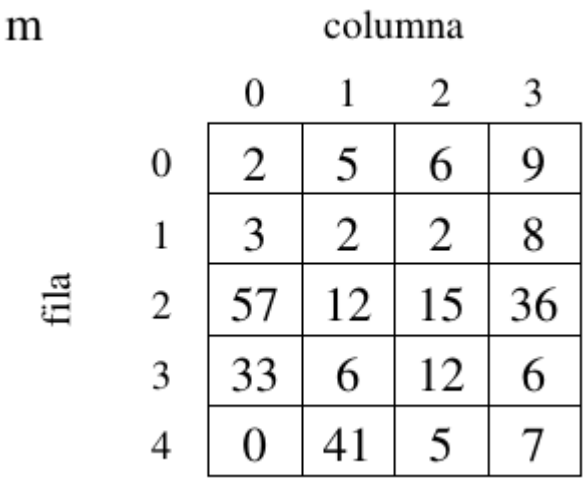
Más información

Ejemplos visuales de distintos métodos de ordenación, con distintos tipos de entradas: https://www.toptal.com/developers/sorting-algorithms Otro ejemplo de visualizador de algoritmos: http://algorithm-visualizer.org/

5.1.4 Arrays bidimensionales: matrices

Los arrays bidimensionales, también llamados matrices, son muy similares a los arrays que hemos visto hasta ahora: También son una colección de elementos del mismo tipo que se agrupan bajo un mismo nombre de variable. Sin embargo:

- Sus elementos están organizados en filas y columnas. Tienen, por tanto una altura y una anchura, y por ello se les llama bidimensionales.
- A cada componente de una matriz se accede mediante dos subíndices: el primero se refiere al número de fila y el segundo al número de columna. En la siguiente figura, `m[0][0]` es `2`, `m[0][3]` es `9`, `m[2][0]` es `57`



- Como vemos, filas y columnas se numeran a partir del `0`.

Si se quisiera extender el tratamiento el estudio de la pluviosidad, para abarcar no solo los días de un mes sino los de todo un año, se podría definir, por ejemplo, un array de 366 elementos, que mantuviera de forma correlativa los datos de pluviosidad de una zona día a día. Con ello, por ejemplo, el dato correspondiente al día 3 de febrero ocuparía la posición 34 del array, mientras que el correspondiente al 2 de julio ocuparía el 184.

Una aproximación más conveniente para la representación de estos datos consistiría en utilizar una matriz con 12 filas (una por mes) y 31 columnas (una por cada día del mes). Esto permitiría una descripción más ajustada a la realidad y, sobre todo, simplificaría los cálculos de la posición real de cada día en la estructura de datos. El elemento `[0][3]` correspondería, por ejemplo, a las lluvias del 4 de enero.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

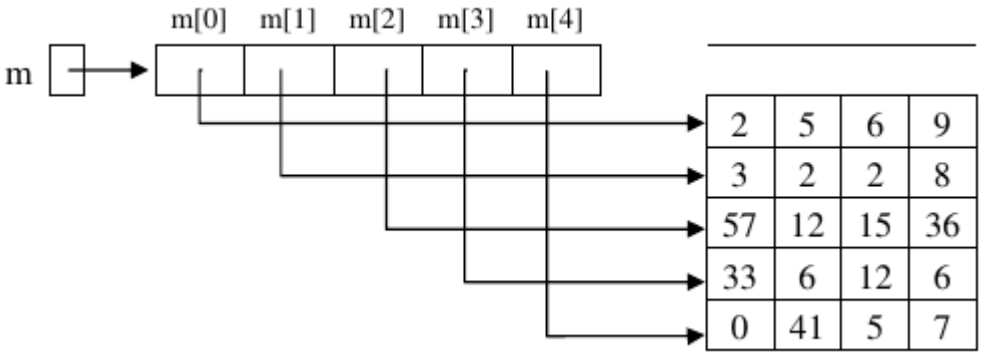
Matrices en Java

Definición

En Java, una matriz es, en realidad un array en el que cada componente es, a su vez, un array. Dicho de otra manera, una matriz de enteros es un array de arrays de enteros.

Esto, que no es igual en otros lenguajes de programación, tiene ciertas consecuencias en la declaración, creación y uso de las matrices en Java:

- Una matriz, en Java, puede tener distinto número de elementos en cada fila.
- La creación de la matriz se puede hacer en un solo paso o fila por fila.
- Si `m` es una matriz de enteros...
- `m[i][j]` es el entero de la fila `i`, columna `j`
- `m[i]` es un array de enteros.
- `m.length` es el número de filas de `m`.
- `m[i].length` es el número de columnas de la fila `i`
- Podríamos dibujar la matriz `m` del ejemplo anterior de una forma más cercana a cómo Java las representa internamente:



Declaración de matrices.

El código siguiente declara una matriz (array bidimensional) de elementos de tipo `double`, y la crea para que tenga `5` filas y `4` columnas (matriz de 5x4):

```
double[][] m1 = new double[5][4];
```

La siguiente declaración es equivalente a la anterior aunque en la práctica es menos utilizada a no ser que queramos que cada fila tenga un número distinto de elementos:

```
double[][] m2 = new double [5][];
m2[0] = new double[4];
m2[1] = new double[4];
m2[2] = new double[4];
m2[3] = new double[4];
m2[4] = new double[4];
```

Es posible inicializar cada uno de los subarrays con un tamaño diferente (aunque el tipo base elemental debe ser siempre el mismo para todos los componentes). Por ejemplo:

```
double[][] m3 = new double [5][];
m3[0] = new double[3];
m3[1] = new double[4];
m3[2] = new double[14];
m3[3] = new double[10];
m3[4] = new double[9];
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Inicialización.

La forma de inicializar una matriz de enteros de por ejemplo [4][3] seria:

```
int[][] m4 = {{7,2,4},{8,2,5},{9,4,3},{1,2,4}};

//aunque se entiende mejor de este modo:
int[][] m4 = {
               {7,2,4},
               {8,2,5},
               {9,4,3},
               {1,2,4}
             };
```

7	2	4
8	2	5
9	4	3
1	2	4

Recorrido

El recorrido se hace de forma similar al de un array aunque, dado que hay dos subíndices, será necesario utilizar dos bucles anidados: uno que se ocupe de recorrer las filas y otro que se ocupe de recorrer las columnas.

El siguiente fragmento de código recorre una matriz `m4` para imprimir sus elementos uno a uno.

```
//recorrido por filas
System.out.println("\nRecorrido por filas: ");
for (int f = 0; f < m4.length; f++) {
    for (int c = 0; c < m4[f].length; c++) {
        System.out.print(m4[f][c] + " ");
    }
    System.out.println("");
}
//Recorrido por filas:
//7 2 4
//8 2 5
//9 4 3
//1 2 4
```

El recorrido se ha hecho por filas, es decir, se imprimen todos los elementos de una fila y luego se pasa a la siguiente. Como habíamos indicado anteriormente, `m.length` representa el número de filas de `m`, mientras que `m[i].length` el número de columnas de la fila `i`.

También es posible hacer el recorrido por columnas: imprimir la columna 0, luego la 1, etc:

```
System.out.println("\nRecorrido por columnas: ");
int numFilas = m4.length;
int numColumnas = m4[0].length;
for (int c = 0; c < numColumnas; c++) {
    for (int f = 0; f < numFilas; f++) {
        System.out.print(m4[f][c] + " ");
    }
    System.out.println("");
}
//Recorrido por columnas:
//7 8 9 1
//2 2 4 2
//4 5 3 4
```

o, directamente ...

```
System.out.println("\nRecorrido por columnas versión 2: ");
for (int c = 0; c < m4[0].length; c++) {
    for (int f = 0; f < m4.length; f++) {
        System.out.print(m4[f][c] + " ");
    }
    System.out.println("");
}
//Recorrido por columnas versión 2:
//7 8 9 1
//2 2 4 2
//4 5 3 4
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

En este caso, para un funcionamiento correcto del recorrido sería necesario que todas las columnas tuvieran igual número de elementos, pues en el bucle externo, se toma como referencia para el número de columnas la longitud de `m[0]`, es decir el número de elementos de la primera fila.

5.1.5 Arrays multidimensionales

En el punto anterior hemos visto que podemos definir arrays cuyos elementos son a la vez arrays, obteniendo una estructura de datos a la que se accede mediante dos subíndices, que hemos llamado arrays bidimensionales o matrices.

Este *anidamiento* de estructuras se puede generalizar, de forma que podríamos construir arrays de más de dos dimensiones. En realidad Java no pone límite al número de subíndices de un array. Podríamos hacer declaraciones como las siguientes:

```
int[][][] notas = new int[10][5][3]; //Notas de 10 alum. en 5 asign. en 3 eval.
notas[2][3][1]=5;//El alumno 2, para la asignatura 3 de la primera evaluación ha sacado un 5
double[][][][][] w = new double [2][7][10][4][10];
```

Más información

Sin embargo, encontrar ejemplos en los que sean necesarios arrays de más de tres dimensiones es bastante raro, y aún cuando los encontramos solemos utilizar arrays de uno o dos subíndices porque nos resulta menos complejo manejarlos.

5.1.6 Recursividad

A la hora de crear programas complejos, uno de los aspectos que diferencia el buen programador del aficionado es su capacidad de hacer algoritmos eficientes. O sea, que sean capaces de resolver el problema planteado en el mínimo de pasos. En el caso de un programa, esto significa la necesidad de ejecutar el mínimo número de instrucciones posible. Ciertamente, si el resultado tiene que ser exactamente el mismo, siempre será mejor hacer una tarea en 10 pasos que en 20, intentando evitar pasos que en realidad son innecesarios. Por lo tanto, la etapa de diseño de un algoritmo es bastante importante y hay que pensar bien una estrategia eficiente. Ahora bien, normalmente, los algoritmos más eficientes también son más difíciles de pensar y codificar, ya que no siempre son evidentes.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Aplicación de la recursividad

A menudo encontrareis que explicar de palabra la idea general de una estrategia puede ser sencillo, pero traducirla instrucciones de Java ya no lo es tanto. Retomamos ahora el caso de la búsqueda dicotómica o binaria, dado que hay que ir repitiendo unos pasos en sucesivas iteraciones, está más o menos claro que el problema planteado para realizar búsquedas eficientes se basa en una estructura de repetición. Pero no se recorren todos los elementos y el índice no se incrementa uno a uno, sino que se va cambiando a valores muy diferentes para cada iteración. No es un caso evidente. Precisamente, este ejemplo no se ha elegido al azar, ya que es un caso en el que os puede ir bien aplicar un nuevo concepto que permite facilitar la definición de algoritmos complejos donde hay repeticiones.

Definición

La **recursividad** es una forma de describir un proceso para resolver un problema de manera que, a lo largo de esta descripción, se usa el proceso mismo que se está describiendo, pero aplicado a un caso más simple.

De hecho, tal vez sin darse cuenta de ello en, ya se ha usado recursividad para describir cómo resolver un problema. Para ver qué significa exactamente la definición formal apenas descrita, se repetirá el texto en cuestión, pero remarcando el aspecto recursivo de la descripción:

"*Generalmente, la mejor estrategia para adivinar un número secreto entre 0 y N sería primero probar N/2. Si no se ha acertado, entonces si el número secreto es más alto se intenta adivinar entre (N/2 + 1) y N. Si era más bajo, se intenta adivinar el valor entre 0 y N-1. Para cada caso, se vuelve a probar el valor que hay en el centro del nuevo intervalo. Y así sucesivamente, hasta adivinarlo.*"

O sea, **el proceso de adivinar un número se basa en el proceso de intentar adivinar un número!** Esto parece hacer trampas, ya es como usar la misma palabra que se quiere definir a su propia definición. Pero fíjese en un detalle muy importante. Los nuevos usos del proceso de "adivinar" son casos más simples, ya que primero se adivina entre N valores posibles, luego entre N/2 valores, después entre N/4, etc. Este hecho no es casual y de él depende poder definir un proceso recursivo de manera correcta.

Más ejemplos

Otro ejemplo de recursividad es la definición de las iniciales del sistema operativo GNU quieren decir "*GNU is Not Unix*"

Implementación de la recursividad

La implementación de la recursividad dentro del código fuente de un programa se hace a nivel de método.

Definición

Un **método recursivo** es aquel que, dentro de su bloque de instrucciones, tiene alguna invocación a él mismo.

El bloque de código de un método recursivo siempre se basa en una estructura de selección múltiple, donde cada rama es de alguno de los dos casos posibles descritos a continuación.

- Por un lado, en el **caso base**, que contiene un bloque instrucciones dentro de las cuales no hay ninguna llamada al método mismo. Se ejecuta cuando se considera que, a partir de los parámetros de entrada, el problema ya es suficientemente simple como para ser resuelto directamente. En el caso de la búsqueda, sería cuando la posición intermedia es exactamente el valor que se está buscando, o bien cuando ya se puede decidir que el elemento a buscar no existe.
- Por otra parte, existe el **caso recursivo**, que contiene un bloque de instrucciones dentro de las cuales hay una llamada al método mismo, dado que se considera que aún no se puede resolver el problema fácilmente. Ahora bien, los valores usados como parámetros de esta nueva llamada deben ser diferentes a los originales. Concretamente, serán unos valores que tiendan a acercarse al caso base. En el caso de la búsqueda, se corresponde a la búsqueda sobre la mitad de los valores originales, ya sea hacia la mitad inferior o superior.

Este es un caso en el que el intervalo de posiciones donde se hará la nueva búsqueda se va acercando al caso base, ya que tarde o temprano, llamada tras llamada, el espacio de búsqueda se irá reduciendo hasta que, o bien se encuentra el elemento, o queda claro que no está.

Dentro de la estructura de selección siempre debe haber al menos un caso base y uno recursivo. Normalmente, los algoritmos recursivos más sencillos tienen uno de cada. Es imprescindible que los casos recursivos siempre garanticen que sucesivas llamadas van aproximando los valores de los parámetros de entrada a algún caso base, ya que, de lo contrario, el programa nunca termina y se produce el mismo efecto que en un bucle infinito.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

CÁLCULO RECURSIVO DE LA OPERACIÓN FACTORIAL

Como ejemplo del funcionamiento de un método recursivo, se empezará con un caso sencillo. Se trata del cálculo de la llamada operación **factorial** de un valor entero positivo. Esta es unaria y se expresa con el operador exclamación (por ejemplo, 4!, 20!, 3!). El resultado de esta operación es la multiplicación de todos los valores desde el 1 hasta el indicado (7! = 1 * 2 * 3 * 4 * 5 * 6 * 7). Normalmente, la definición matemática de esta operación se hace de manera recursiva:

- `0! = 1` ⬅️ **caso base**
- `n! = n * (n - 1)!` ⬅️ **caso recursivo**

Así pues, tened en cuenta que el caso recursivo realiza un cálculo que depende de usar la propia definición de la operación, pero cuando lo hace es con un nuevo valor inferior al original, por lo que se garantiza que, en algún momento, se hará una llamada recursiva que desembocará en el caso base. Cuando esto ocurra, la cadena de llamadas recursivas acabará. Una manera de ver esto es desarrollando paso a paso esta definición:

```
4! = 4 * (4 - 1)! = 4 * (3)!
4 * 3! = 4 * (3 * (3-1)!) = 4 * 3 * (2)!
4 * 3 * 2! = 4 * 3 * (2 * (2-1)!) = 4 * 3 * 2 * (1)!
4 * 3 * 2 * 1! = 4 * 3 * 2 * (1 * (1 - 1)!) = 4 * 3 * 2 * 1 * (0)!
4 * 3 * 2 * 1 * 0! = 4 * 3 * 2 * 1 * (1) = 24
```

Su implementación en Java sería la que ves más abajo y también en el Ejemplo02. Ahora bien, en este código se han añadido algunas sentencias para escribir información por pantalla, de forma que se vea con más detalle cómo funciona un método recursivo. Veréis que, inicialmente, se llevan a cabo una serie de invocaciones del caso recursivo, uno tras otro, hasta que se llega a una llamada que ejecuta el caso base. Es a partir de entonces cuando, a medida que se van ejecutando las sentencias `return` del caso recursivo, realmente se va acumulando el cálculo. Otra forma de verlo es depurando el programa.

```java
package UD04;

public class Recursividad {

    public static void main(String[] args) {
        //factorial
        System.out.println(factorial(4));

        [...]
    }

    /**
     * Método recursivo que calcula el factorial
     */
    public static int factorial(int n) {
        if (n == 0) {
            //Caso base: Se sabe el resultado directamente
            System.out.println("Caso base: n es igual a 0");
            return 1;
        } else {
            //Caso recursivo: Para calcularlo hay que invocar al método recursivo
            //El valor del nuevo parámetro de entradad se ha de modificar, de
            //manera que se vaya acercando al caso base
            System.out.println("Caso recursivo " + (n - 1)
                    + ": Se invoca al factorial(" + (n - 1) + ")");
            int res = n * factorial(n - 1);
            System.out.println("  cuyo resultado es: " + res);
            return res;
        }
    }
[...]
```

La ejecución resultante es:

```
Caso recursivo 3: Se invoca al factorial(3)
Caso recursivo 2: Se invoca al factorial(2)
Caso recursivo 1: Se invoca al factorial(1)
Caso recursivo 0: Se invoca al factorial(0)
Caso base: n es igual a 0
  cuyo resultado es: 1
  cuyo resultado es: 2
  cuyo resultado es: 6
  cuyo resultado es: 24
24
```

CÁLCULO RECURSIVO DE LA BÚSQUEDA DICOTÓMICA

A continuación se muestra el código del algoritmo recursivo de búsqueda dicotómica o binaria sobre un array. Observad atentamente los comentarios, los cuales identifican los casos base y recursivos. En este caso, hay más de un caso base y recursivo.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD04;

public class Recursividad {

    public static void main(String[] args) {
        [...]
        //busqueda binaria recursiva
        int[] array = {2, 4, 6, 8, 10, 12, 14, 16, 18, 20};
        int buscaDieciocho = BusquedaBinaria(array, 0, array.length - 1, 18);
        int buscaCinco = BusquedaBinaria(array, 0, array.length - 1, 5);
        System.out.println("Busqueda del 18: " + buscaDieciocho);
        System.out.println("Busqueda del 5: " + buscaCinco);
        [...]
    }

    [...]
    public static int BusquedaBinaria(int[] array, int inicio, int fin, int valor) {
        if (inicio > fin) {
            //Caso base: No se ha encontrado el valor
            return -1;
        }
        //Se calcula la posición central entre los dos índices de búsqueda
        int pos = inicio + (fin - inicio) / 2;
        if (array[pos] > valor) {
            //Caso recursivo: Si el valor es menor que la posición que se ha
            //consultado, entonces hay que seguir buscando por la parte
            //"derecha" del array
            return BusquedaBinaria(array, inicio, pos - 1, valor);
        } else if (array[pos] < valor) {
            //Caso recursivo: Si el valor es mayor que la posición que se ha
            //consultado, entonces hay que seguir buscando por la parte
            //"izquierda" del array
            return BusquedaBinaria(array, pos + 1, fin, valor);
        } else {
            //caso base: Es igual, por tanto, se ha encontrado
            return pos;
        }
    }
    [...]
}
```

El resultado de la ejecución es:

```
Busqueda del 18: 8
Busqueda del 5: -1
```

Prácticamente cualquier problema que se puede resolver con un algoritmo recursivo también se puede resolver con sentencias de estructuras de repetición (de manera iterativa). Pero muy a menudo su implementación será mucho menos evidente y las interacciones entre instrucciones bastante más complejas que la opción recursiva (una vez se entiende este concepto, claro).

Desbordamiento de pila (stack overflow)

Las versiones recursivas de muchas rutinas pueden ejecutarse un poco más lentamente que sus equivalentes iterativos debido a la sobrecarga adicional de las llamadas a métodos adicionales. Demasiadas llamadas recursivas a un método podrían causar un **desbordamiento de la pila**.

Como el almacenamiento para los parámetros y las variables locales está en la pila y cada llamada nueva crea una nueva copia de estas variables, es posible que la pila se haya agotado. Si esto ocurre, el sistema de tiempo de ejecución (run-time) de Java causará una excepción. Sin embargo, probablemente no tendrás que preocuparte por esto a menos que una rutina recursiva se vuelva loca.

La principal ventaja de la recursividad es que algunos tipos de algoritmos se pueden implementar de forma más clara y más recursiva de lo que pueden ser iterativamente. Por ejemplo, el algoritmo de clasificación Quicksort es bastante difícil de implementar de forma iterativa. Además, algunos problemas, especialmente los relacionados con la **IA**, parecen prestarse a **soluciones recursivas**.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD04;

public class Recursividad {

    public static void main(String[] args) {
        [...]
        //desbordamiento de pila
        desbordamientoPila(10);
    }

    [...]

    public static int desbordamientoPila(int n) {
        // condición base incorrecta (esto provoca un desbordamiento de la pila).
        if (n == 100) {
            return 1;
        } else {
            return n * desbordamientoPila(n - 1);
        }
    }
}
```

En el ejemplo anterior si se llama a `desbordamientoPila(10)`, llamará a `desbordamientoPila (9)`, `desbordamientoPila(8)`, `desbordamientoPila(7)`, etc., pero el número nunca llegará a 100. Por lo tanto, no se alcanza la condición base. Si la memoria se agota con estos métodos en la pila, provocará un error de desbordamiento de pila (`java.lang.StackOverflowError`).

> **Importante**
>
> Al escribir métodos recursivos, debe tener una instrucción condicional, como un `if`, en algún lugar para forzar el retorno del método sin que se ejecute la llamada recursiva. Si no lo hace, una vez que llame al método, nunca retornará. Este tipo de error es muy común cuando se trabaja con recursividad.

5.1.7 Ejemplos UD04

Ejemplo01

Ejemplo completo con declaración, inicialización, recorrido, búsqueda y ordenación de arrays y matrices.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD04;

import java.util.Scanner;

public class EjemploUD04 {

    public static void main(String[] args) {
        //declaración
        double lluvia1[]; // lluvia1 es un array de double
        double[] lluvia2;  // lluvia2 es un array de double

        //instanciación
        lluvia1 = new double[31];

        //declaración + instanciación
        double lluvia3[] = new double[31];

        // usamos un array para almacenar las edades de un grupo de personas
        // la variable numPersonas contiene el número de personas del grupo
        // y se asigna en tiempo de ejecución
        Scanner teclado = new Scanner(System.in);
        System.out.print("Introduce cuantos elementos debe tener el array edad[]:");
        int numPersonas = teclado.nextInt();
        int edad[] = new int[numPersonas];

        //acceso a componentes
        System.out.print("Introduce el dato para el componente 0: ");
        edad[0] = teclado.nextInt();
        System.out.println("El componente [0] vale " + edad[0]);
        edad[1] = edad[0] + 1;
        edad[2] = edad[0] + edad[1];
        edad[2]++;
        System.out.println("El componente [1] vale " + edad[1]);
        System.out.println("El componente [2] vale " + edad[2]);

        //el índice también admite calculos/variables:
        int i = 3;
        edad[i] = edad[i + 1];
        edad[i + 2] = edad[i];

        //Inicialización
        int edad2[] = new int[10];
        edad2[0] = 25;
        edad2[1] = 10;
        edad2[9] = 12;
        //...

        int edad3[] = {25, 10, 23, 34, 65, 23, 1, 67, 54, 12};

        //Ejemplo práctico
        System.out.println(nombreMes(3)); //marzo

        //Paso de arrays como parámetros:
        int a = 1;
        int v[] = {1, 1, 1};
        metodo(v, a); //Pasar un array como parámetro
        System.out.println(a); // Muesta 1
        System.out.println(v[0]); // Muestra 2

        //atributo lenght
        double estatura[] = new double[25];
        System.out.println(estatura.length); // Mostrará por pantalla: 25

        //Array args[] del método main contiene los parámetros de entrada
        System.out.println(args[0]);//parámetro 1 de la línea de comandos
        System.out.println(args[1]);//parámetro 2 de la línea de comandos

        //busquedas y recorridos de arrays
        double pluviosidad[] = {5, 4, 0, 0, 0, 10, 0, 0, 0, 0, 2, 2, 3, 4, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 6, 0, 0};

        //recorrido ascendente
        System.out.println(pluviosidadMediaAscendente(pluviosidad)); //1.3
        //recorrido descendente
        System.out.println(pluviosidadMediaDescendente(pluviosidad)); //1.3
        //recorrido para máximo
        System.out.println(pluviosidadMaxima(pluviosidad)); //10.0
        //busqueda con while y boolean
        System.out.println(primerDiaSinLluvia1(pluviosidad)); //2
        //busqueda con while sin boolean
        System.out.println(primerDiaSinLluvia2(pluviosidad)); //2
        //busqueda con for
        System.out.println(primerDiaSinLluvia3(pluviosidad)); //2
        //busqueda descendente
        System.out.println(ultimoDiaSinLluvia(pluviosidad)); //31
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
        //busqueda en arrays ordenados (busqueda binaria)
        int buscarEdad[] = {15, 22, 33, 36, 41, 56, 71, 92};
        System.out.println(hayAlguienDe36(buscarEdad)); //true

        //ordenar arrays
        int desordenado[] = {62, 4, 25, 27, 32, 1, 80, 43, 22};
        seleccionDirecta(desordenado);
        //con el siguiente bucle recorremos el array ascendentemente y al
        //imprimirlo resulta en: 1 4 22 25 27 32 43 62 80
        for (int j = 0; j <= desordenado.length - 1; j++) {
            System.out.print(desordenado[j] + " ");
        }

        //arrays bidimensionales
        double m1[][] = new double[5][4];

        //con el mismo número de columnas
        double m2[][] = new double[5][];
        m2[0] = new double[4];
        m2[1] = new double[4];
        m2[2] = new double[4];
        m2[3] = new double[4];
        m2[4] = new double[4];

        //diferentes números de columnas
        double m3[][] = new double[5][];
        m3[0] = new double[3];
        m3[1] = new double[4];
        m3[2] = new double[14];
        m3[3] = new double[10];
        m3[4] = new double[9];

        int m4[][] = {
            {7, 2, 4},
            {8, 2, 5},
            {9, 4, 3},
            {1, 2, 4}
        };

        //recorrido por filas
        System.out.println("\nRecorrido por filas: ");
        for (int r = 0; r < m4.length; r++) {
            for (int s = 0; s < m4[r].length; s++) {
                System.out.print(m4[r][s] + " ");
            }
            System.out.println("");
        }
        //Recorrido por filas:
        //7 2 4
        //8 2 5
        //9 4 3
        //1 2 4

        System.out.println("\nRecorrido por columnas: ");
        int numFilas = m4.length;
        int numColumnas = m4[0].length;
        for (int j = 0; j < numColumnas; j++) {
            for (int k = 0; k < numFilas; k++) {
                System.out.print(m4[k][j] + " ");
            }
            System.out.println("");
        }
        //Recorrido por columnas:
        //7 8 9 1
        //2 2 4 2
        //4 5 3 4

        System.out.println("\nRecorrido por columnas versión 2: ");
        for (int j = 0; j < m4[0].length; j++) {
            for (int k = 0; k < m4.length; k++) {
                System.out.print(m4[k][j] + " ");
            }
            System.out.println("");
        }
        //Recorrido por columnas versión 2:
        //7 8 9 1
        //2 2 4 2
        //4 5 3 4

        //arrays multidimensionales:
        int notas[][][] = new int[10][5][3]; //Notas de 10 alum. en 5 asign. en 3 eval.
        double w[][][][][] = new double [2][7][10][4][10];
    }

    public static String nombreMes(int mes) {
        String nombre[] = {" ", "enero", "febrero", "marzo", "abril",
            "mayo", "junio", "julio", "agosto", "septiembre", "octubre",
            "noviembre", "diciembre"};
        return nombre[mes];
    }
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
    public static void metodo(int x[], int y) { //recibir un array como parámetro
        x[0]++;
        y++;
    }

    //recorremos ascendentemente el array para obtener la media
    public static double pluviosidadMediaAscendente(double lluvia[]) {
        double suma = 0;
        //Recorremos el array
        for (int i = 0; i < lluvia.length; i++) {
            suma += lluvia[i];
        }
        double media = suma / lluvia.length;
        return media;
    }

    //recorremos descendentemente el array para obtener la media
    public static double pluviosidadMediaDescendente(double lluvia[]) {
        double suma = 0;
        //Recorremos el array
        for (int i = lluvia.length - 1; i >= 0; i--) {
            suma += lluvia[i];
        }
        double media = suma / lluvia.length;
        return media;
    }

    //recorremos el array para encontrar el dia con más pluviosidad
    public static double pluviosidadMaxima(double lluvia[]) {
        // Suponemos el la pluviosidad máxima se produjo el primer día
        double max = lluvia[0];
        //Recorremos el array desde la posición 1, comprobando si hay una pluviosidad mayor
        for (int i = 1; i < lluvia.length; i++) {
            if (lluvia[i] > max) {
                max = lluvia[i];
            }
        }
        return max;
    }
    //Devolveremos el subíndice del primer componente del array cuyo valor es cero.
// Si no hay ningún día sin lluvias devolveremos -1

    public static int primerDiaSinLluvia1(double lluvia[]) {
        int i = 0;
        boolean encontrado = false;
        while (i < lluvia.length && !encontrado) {
            if (lluvia[i] == 0) {
                encontrado = true;
            } else {
                i++;
            }
        }
        if (encontrado) {
            return i;
        } else {
            return -1;
        }
    }

    public static int primerDiaSinLluvia2(double lluvia[]) {
        int i = 0;
        while (i < lluvia.length && lluvia[i] != 0) {
            i++;
        }
        if (i == lluvia.length) {
            return -1;
        } else {
            return i;
        }
    }

    public static int primerDiaSinLluvia3(double lluvia[]) {
        int i;
        for (i = 0; i < lluvia.length && lluvia[i] != 0; i++) /*Nada*/ ;
        if (i == lluvia.length) {
            return -1;
        } else {
            return i;
        }
    }
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
    public static int ultimoDiaSinLluvia(double lluvia[]) {
        int i = lluvia.length - 1;
        boolean encontrado = false;
        while (i >= 0 && !encontrado) {
            if (lluvia[i] == 0) {
                encontrado = true;
            } else {
                i--;
            }
        }
        if (encontrado) {
            return i;
        } else {
            return -1;
        }
    }

    public static boolean hayAlguienDe36(int edad[]) {
        // Las variables izq y der marcarán el fragmento del array en el que
        // realizamos la búsqueda. Inicialmente buscamos en todo el array.
        int izq = 0;
        int der = edad.length - 1;
        boolean encontrado = false;
        while (izq <= der && !encontrado) {
            // Calculamos posición central del fragmento en el que buscamos
            int m = (izq + der) / 2;
            if (edad[m] == 36) // Hemos encontrado una persona de 36
            {
                encontrado = true;
            } else if (edad[m] > 36) {
                // El elemento central tiene más de 36.
                // Continuamos la búsqueda en la mitad izquierda. Es decir,
                // entre las posiciónes izq y m-1
                der = m - 1;
            } else {
                // El elemento central tiene menos de 36.
                //Continuamos la búsqueda en la mitad derecha. Es decir,
                // entre las posiciones m+1 y der
                izq = m + 1;
            } // del if
        } // del while
        return encontrado; // if (encontrado) return true; else return false;
    }

    public static void seleccionDirecta(int v[]) {
        for (int i = 0; i < v.length - 1; i++) {
            //Localizamos elemento que tiene que ir en la posición i
            int posMin = i;
            for (int j = i + 1; j < v.length; j++) {
                if (v[j] < v[posMin]) {
                    posMin = j;
                }
            }
            //Intercambiamos los elementos de las posiciones i y posMin
            int aux = v[posMin];
            v[posMin] = v[i];
            v[i] = aux;
        }
    }
}
```

⬆ Volver a teoría

Ejemplo02

Ejemplo completo de recursividad: factorial, búsqueda binaria y desbordamiento de pila.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD04;

public class Recursividad {

    public static void main(String[] args) {
        //factorial
        System.out.println(factorial(4));

        //busqueda binaria recursiva
        int[] array = {2, 4, 6, 8, 10, 12, 14, 16, 18, 20};
        int buscaDieciocho = BusquedaBinaria(array, 0, array.length - 1, 18);
        int buscaCinco = BusquedaBinaria(array, 0, array.length - 1, 5);
        System.out.println("Busqueda del 18: " + buscaDieciocho);
        System.out.println("Busqueda del 5: " + buscaCinco);

        //desbordamiento de pila
        desbordamientoPila(9);
    }

    /**
     * Método recursivo que calcula el factorial
     */
    public static int factorial(int n) {
        if (n == 0) {
            //Caso base: Se sabe el resultat directamente
            System.out.println("Caso base: n es igual a 0");
            return 1;
        } else {
            //Caso recursivo: Para calcularlo hay que invocar al método recursivo
            //El valor del nuevo parámetro de entradad se ha de modificar, de
            //manera que se vaya acercando al caso base
            System.out.println("Caso recursivo " + (n - 1)
                    + ": Se invoca al factorial(" + (n - 1) + ")");
            int res = n * factorial(n - 1);
            System.out.println("  cuyo resultado es: " + res);
            return res;
        }
    }

    public static int BusquedaBinaria(int[] array, int inicio, int fin, int valor) {
        if (inicio > fin) {
            //Caso base: No se ha encontrado el valor
            return - 1;
        }
        //Es calcula la posició central entre els dos índexs de cerca
        int pos = inicio + (fin - inicio) / 2;
        if (array[pos] > valor) {
            //Caso recursivo: Si el valor es menor que la posición que se ha
            //consultado, entonces hay que seguir buscando por la parte
            //"derecha" del array
            return BusquedaBinaria(array, inicio, pos - 1, valor);
        } else if (array[pos] < valor) {
            //Caso recursivo: Si el valor es mayor que la posición que se ha
            //consultado, entonces hay que seguir buscando por la parte
            //"izquierda" del array
            return BusquedaBinaria(array, pos + 1, fin, valor);
        } else {
            //caso base: Es igual, por tanto, se ha encontrado
            return pos;
        }
    }

    public static int desbordamientoPila(int n) {
        // condición base incorrecta (esto provoca un desbordamiento de la pila).
        if (n == 100) {
            return 1;
        } else {
            return n * desbordamientoPila(n - 1);
        }
    }
}
```

⬆ Volver a teoría

5.1.8 Resumen — Conceptos clave

Concepto	Definición
Array	Estructura de datos que almacena múltiples valores del mismo tipo
Índice	Posición numérica que permite acceder a un elemento del array
Matriz	Array bidimensional (filas × columnas)
Recursividad	Técnica donde un método se llama a sí mismo para resolver un problema

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Concepto	Definición
Caso base	Condición que detiene la recursión

5.1.9 Autoevaluación

- [ ] Sé declarar e inicializar arrays
- [ ] Puedo recorrer arrays con bucles
- [ ] Trabajo con matrices de varias dimensiones
- [ ] Entiendo el concepto de recursividad
- [ ] Soy capaz de escribir funciones recursivas
- [ ] Distingo entre caso base y caso recursivo

5.1.10 Vídeos recomendados

Canal	Vídeo	Contenido
Píldoras Informáticas	Arrays I (vídeo 23)	Declaración y uso de arrays unidimensionales
Píldoras Informáticas	Arrays II (vídeo 24)	Recorrido y manipulación de arrays
Píldoras Informáticas	Arrays III — bidimensionales (vídeo 25)	Arrays de dos dimensiones
EducaMadrid	Arrays 1	Introducción a arrays
EducaMadrid	Arrays en Java — Parte 2	Ejercicio práctico con arrays
Programación ATS	Playlist completa POO Java	Vídeos 40–44: arrays y matrices

3 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

5.2 Anexo Cheatsheet Strings en Java

5.2.1 Introducción

Desde el punto de vista de la programación diaria, uno de los tipos de datos más importantes de Java es **String**. *String* define y admite cadenas de caracteres. En algunos otros lenguajes de programación, una cadena o string es una matriz o array de caracteres. Este no es el caso con Java. En Java, **los String son objetos**.

En realidad, has estado usando la clase String desde el comienzo del curso, pero no lo sabías. Cuando crea un literal de cadena, en realidad está creando un objeto String. Por ejemplo, en la declaración:

```
System.out.println("En Java, los String son objetos");
```

La clase String es bastante grande, y solo veremos una pequeña parte aquí.

5.2.2 Construyendo `String`

Puede construir un `String` igual que construye cualquier otro tipo de objeto: utilizando new y llamando al constructor *String*. Por ejemplo:

```
String str = new String("Hola");
```

Esto crea un objeto *String* llamado *str* que contiene la cadena de caracteres “Hola”. También puedes construir una *String* desde otro *String*. Por ejemplo:

```
String str = new String("Hola");
String str2 = new String(str);
```

Después de que esta secuencia se ejecuta, str2 también contendrá la cadena de caracteres “Hola”. Otra forma fácil de crear una cadena se muestra aquí:

```
String str = "Estoy aprendiendo sobre String en JavadesdeCero.";
```

En este caso, *str* se inicializa en la secuencia de caracteres “*Estoy aprendiendo sobre String en JavadesdeCero*.”. Una vez que haya creado un objeto String, puede usarlo en cualquier lugar que permita una cadena entrecomillada. Por ejemplo, puede usar un objeto String como argumento para *println()*, como se muestra en este ejemplo:

```
// Uso de String
class DemoString
{
    public static void main(String args[])
    {
        //Declaración de String de diferentes maneras
        String str1=new String("En Java, los String son objetos");
        String str2=new String("Se construyen de varias maneras");
        String str3=new String(str2);

        System.out.println(str1);
        System.out.println(str2);
        System.out.println(str3);

    }
}
```

La salida del programa se muestra a continuación:

```
En Java, los String son objetos
Se construyen de varias maneras
Se construyen de varias maneras
```

5.2.3 Operando con Métodos de la clase `String`

La clase String contiene varios métodos que operan en cadenas. Aquí se detallan todos los métodos:

- `int length()`: Devuelve la cantidad de caracteres del String.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
"Javadesdecero.es".length(); // retorna 16
```

- `Char charAt(int i)` : Devuelve el carácter en el índice *i*.

```
System.out.println("Javadesdecero.es".charAt(3)); // retorna 'a'
```

- `String substring(int i)` : Devuelve la subcadena del i-ésimo carácter de índice al final.

```
"Javadesdecero.es".substring(4); // retorna desdecero.es
```

- `String substring(int i, int j)` : Devuelve la subcadena del índice *i* a *j-1*.

```
"Javadesdecero.es".substring(4,9); // retorna desde
```

- `String concat(String str)` : Concatena la cadena especificada al final de esta cadena.

```
String s1 = "Java";
String s2 = "desdeCero;
String salida = s1.concat(s2); // retorna "JavadesdeCero"
```

- `int indexOf(String s)` : Devuelve el índice dentro de la cadena de la primera aparición de la cadena especificada.

```
String s = "Java desde Cero";
int salida = s.indexOf("Cero"); // retorna 11
```

- `int indexOf(String s, int i)` : Devuelve el índice dentro de la cadena de la primera aparición de la cadena especificada, comenzando en el índice especificado.

```
String s = "Java desde Cero";
int salida = s.indexOf('a',2); //retorna 3
```

- `int lastIndexOf(int ch)` : Devuelve el índice dentro de la cadena de la última aparición de la cadena especificada.

```
String s = "Java desde Cero";
int salida = s.lastIndexOf('a'); // retorna 3
```

- `boolean equals(Objeto otroObjeto)` : Compara este String con el objeto especificado.

```
Boolean salida = "Java".equals("Java"); // retorna true
Boolean salida = "Java".equals("java"); // retorna false
```

- `boolean equalsIgnoreCase(String otroString)` : Compares string to another string, ignoring case considerations.

```
Boolean salida= "Java".equalsIgnoreCase("Java"); // retorna true
Boolean salida = "Java".equalsIgnoreCase("java"); // retorna true
```

- `int compareTo(String otroString)` : Compara dos cadenas lexicográficamente.

```
int salida = s1.compareTo(s2); // donde s1 y s2 son strings que se comparan
/*
 Esto devuelve la diferencia s1-s2. Si:
    salida < 0 // s1 es menor que s2
    salida = 0 // s1 y s2 son iguales
    salida > 0 // s1 es mayor que s2
 */
```

- `int compareToIgnoreCase(String otroString)` : Compara dos cadenas lexicográficamente, ignorando las consideraciones *case*.

```
int salida = s1.compareToIgnoreCase(s2); // donde s1 y s2 son strings que se comparan
/*
 Esto devuelve la diferencia s1-s2. Si:
    salida < 0 // s1 es menor que s2
    salida = 0 // s1 y s2 son iguales
    salida > 0 // s1 es mayor que s2
*/
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

> **Más información**
>
> En este caso, no considerará el *case* de una letra (ignorará si está en mayúscula o minúscula).

- `String toLowerCase()`: Convierte todos los caracteres de String a minúsculas.

```
String palabra1 = "HoLa";
String palabra2 = palabra1.toLowerCase(); // retorna "hola"
```

- `String toUpperCase()`: Convierte todos los caracteres de String a mayúsculas.

```
String palabra1 = "HoLa";
String palabra2 = palabra1.toUpperCase(); // retorna "HOLA"
```

- `String trim()`: Devuelve la copia de la cadena, eliminando espacios en blanco en ambos extremos. No afecta los espacios en blanco en el medio.

```
String palabra1 = " Java desde Cero ";
String palabra2 = palabra1.trim(); // retorna "Java desde Cero"
```

- `String replace(char oldChar, char newChar)`: Devuelve una nueva cadena al reemplazar todas las ocurrencias de oldChar con newChar.

```
String palabra1 = "yavadesdecero";
String palabra2 = palabra1.replace('y' ,'j'); //retorna javadesdecero
```

> **Más información**
>
> palabra1 sigue siendo yavadesdecero y palabra2, javadesdecero

- `String replaceAll(String regex, String replacement)`: devuelve una cadena que reemplaza toda la secuencia de caracteres que coinciden con la expresión regular `regex` por la cadena de reemplazo `replacement`.

```
String str = "Ejemplo con espacios en blanco y tabs";
String str2 = str.replaceAll("\\s", ""); //retorna Ejemploconespaciosenblancoytabs
```

> **Más información**
>
> Otras expresiones regulares (entre otras muchísimas):
>
> - `\w` Cualquier cosa que sea un carácter de palabra
> - `\W` Cualquier cosa que no sea un carácter de palabra (incluida la puntuación, etc.)
> - `\s` Cualquier cosa que sea un carácter de espacio (incluido el espacio, los caracteres de tabulación, etc.)
> - `\S` Cualquier cosa que no sea un carácter de espacio (incluidas letras y números, así como puntuación, etc.)
>
> Debe escapar de la barra invertida si desea que `\s` alcance el motor de expresiones regulares, lo que da como resultado `\\s`).
>
> Más información sobre expresiones regulares en java: https://www.vogella.com/tutorials/JavaRegularExpressions/article.html

- `String[] split(String regex)`: Divide la cadena en un array de trozos, usando `regex` como separador. El separador no forma parte de ningún trozo.

```
String s = "manzana,pera,platano";
String[] frutas = s.split(","); // {"manzana", "pera", "platano"}
System.out.println(frutas[1]); // retorna pera
```

> **Más información**
>
> Igual que en `replaceAll()`, `regex` es una expresión regular: si el separador es un carácter especial de regex (por ejemplo, un punto `.`), hay que escaparlo (`"\\."`). Ver la información sobre expresiones regulares en el bloque anterior.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

5.2.4 Ejemplo de todos los métodos de `String`

```java
// Código Java para ilustrar diferentes constructores y métodos
// de la clase String.

class DemoMetodosString
{
    public static void main (String[] args)
    {
        String s= "JavadesdeCero";
        // o String s= new String ("JavadesdeCero");

        // Devuelve la cantidad de caracteres en la Cadena.
        System.out.println("String length = " + s.length());

        // Devuelve el carácter en el índice i.
        System.out.println("Character at 3rd position = "
                + s.charAt(3));

        // Devuelve la subcadena del carácter índice i-ésimo
        // al final de la cadena
        System.out.println("Substring " + s.substring(3));

        // Devuelve la subcadena del índice i a j-1.
        System.out.println("Substring  = " + s.substring(2,5));

        // Concatena string2 hasta el final de string1.
        String s1 = "Java";
        String s2 = "desdeCero";
        System.out.println("String concatenado  = " +
                s1.concat(s2));

        // Devuelve el índice dentro de la cadena de
        // la primera aparición de la cadena especificada.
        String s4 = "Java desde Cero";
        System.out.println("Índice de Cero: " +
                s4.indexOf("Cero"));

        // Devuelve el índice dentro de la cadena de
        // la primera aparición de la cadena especificada,
        // comenzando en el índice especificado.
        System.out.println("Índice de a  = " +
                s4.indexOf('a',3));

        // Comprobando la igualdad de cadenas
        Boolean out = "Java".equals("java");
        System.out.println("Comprobando la igualdad: " + out);
        out = "Java".equals("Java");
        System.out.println("Comprobando la igualdad: " + out);

        out = "Java".equalsIgnoreCase("jaVA ");
        System.out.println("Comprobando la igualdad: " + out);

        int out1 = s1.compareTo(s2);
        System.out.println("Si s1 = s2: " + out1);

        // Conversión de cases
        String palabra1 = "JavadesdeCero";
        System.out.println("Cambiando a minúsculas: " +
                palabra1.toLowerCase());

        // Conversión de cases
        String palabra2 = "JavadesdeCero";
        System.out.println("Cambiando a MAYÚSCULAS: " +
                palabra1.toUpperCase());

        // Recortando la palabra
        String word4 = " JavadesdeCero ";
        System.out.println("Recortando la palabra: " + word4.trim());

        // Reemplazar caracteres
        String str1 = "YavadesdeCero";
        System.out.println("String Original: " + str1);
        String str2 = "YavadesdeCero".replace('Y' ,'J') ;
        System.out.println("Reemplazando Y por J -> " + str2);

        // Reemplazar todos los carácteres
        String strAll = "Ejemplo con espacios en blanco y tabs";
        System.out.println("String Original: " + strAll);
        String strAll2 = strAll.replaceAll("\\s", "");
        System.out.println("Eliminando todos los espacios en blanco -> " + strAll2);

        // Dividir la cadena en un array de trozos
        String csv = "manzana,pera,platano";
        String[] frutas = csv.split(",");
        System.out.println("Fruta en la posición 1: " + frutas[1]);
    }
}
```

Salida:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
String length = 13
Character at 3rd position = a
Substring adesdeCero
Substring = vad
String concatenado = JavadesdeCero
Índice de Cero: 11
Índice de a = 3
Comprobando la igualdad: false
Comprobando la igualdad: true
Comprobando la igualdad: false
Si s1 = s2: -26
Cambiando a minúsculas: javadesdecero
Cambiando a MAYÚSCULAS: JAVADESDECERO
Recortando la palabra: JavadesdeCero
String Original: YavadesdeCero
Reemplazando Y por J -> JavadesdeCero
String Original: Ejemplo con espacios en blanco y tabs
Eliminando todos los espacios en blanco -> Ejemploconespaciosenblancoytabs
Fruta en la posición 1: pera
```

5.2.5 Arrays de `String`

Al igual que cualquier otro tipo de datos, los String se pueden ensamblar en arrays. Por ejemplo:

```
// Demostrando arrays de String
class StringArray
{
    public static void main (String[] args)
    {
       String str[]={"Java", "desde","Cero"};

        System.out.println("Array Original: ");
        for (String s : str)
            System.out.print(s+ "");
        System.out.println("\n");

        //Cambiando un String
        str[1]="Curso";
        str[2]="Online";

        System.out.println("Array Modificado: ");
        for (String s : str)
            System.out.print(s+ "");
        System.out.println("\n");
    }
}
```

Se muestra el resultado de este programa:

```
Array Original:
JavadesdeCero

Array Modificado:
JavaCursoOnline
```

5.2.6 Los `String` son inmutables

El contenido de un objeto String es inmutable. Es decir, una vez creada, la secuencia de caracteres que compone la cadena **no se puede modificar**. Esta restricción permite a Java implementar cadenas de manera más eficiente. Aunque esto probablemente suene como un serio inconveniente, no lo es.

Cuando necesite una cadena que sea una variación de una que ya existe, simplemente cree una nueva cadena que contenga los cambios deseados. Como los objetos String no utilizados se recolectan de forma automática, ni siquiera tiene que preocuparse por lo que sucede con las cadenas descartadas. Sin embargo, debe quedar claro que las variables de referencia de cadena pueden, por supuesto, cambiar el objeto al que hacen referencia. Es solo que el contenido de un objeto `String` específico no se puede cambiar después de haber sido creado.

Para comprender completamente por qué las cadenas inmutables no son un obstáculo, utilizaremos otro de los métodos de String: `substring()`. El método `substring()` devuelve una nueva cadena que contiene una parte especificada de la cadena invocadora. Como se fabrica un nuevo objeto String que contiene la subcadena, la cadena original no se altera y la regla de inmutabilidad permanece intacta. La forma de `substring( )` que vamos a utilizar se muestra aquí:

```
String substring(int beginIndex, int endIndex)
```

Aquí, `beginIndex` especifica el índice inicial, y `endIndex` especifica el punto de detención. Aquí hay un programa que demuestra el uso de `substring( )` y el principio de cadenas inmutables:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
// uso de substring()
class SubString {
    public static void main (String[] args){
        String str="Java desde Cero";

        //Construyendo un substring
        String substr=str.substring(5,15);

        System.out.println("str: "+str);
        System.out.println("substr: "+substr);
    }
}
```

Salida:

```
str: Java desde Cero
substr: desde Cero
```

Como puede ver, la cadena original `str` no se modifica, y `substr` contiene la subcadena.

5.2.7 `String` en Argumentos de Línea de Comandos

Ahora que conoce la clase String, puede comprender el parámetro *args* en main() que ha estado en cada programa mostrado hasta ahora. Muchos programas aceptan lo que se llaman **argumentos de línea de comandos**. Un argumento de línea de comandos es la información que sigue directamente el nombre del programa en la línea de comando cuando se ejecuta.

Para acceder a los argumentos de la línea de comandos dentro de un programa Java es bastante fácil: se almacenan como cadenas en la matriz String pasada a `main()`. Por ejemplo, el siguiente programa muestra todos los argumentos de línea de comandos con los que se llama:

```
// Mostrando Información de Línea de Comando
class DemoLC{
    public static void main (String[] args){
        System.out.println("Aquí se muestran "+ args.length
                           + " argumentos de línea de comando.");
        System.out.println("Estos son: ");
        for (int i=0; i<args.length; i++){
            System.out.println("arg["+i+"]: "+args);
        }
    }
}
```

Si `DemoLC` se ejecuta de esta manera,

```
java DemoLC uno dos tres
```

verá la siguiente salida:

```
Aquí se muestran 3 argumentos de línea de comando.
Estos son:
arg[0]: uno
arg[1]: dos
arg[2]: tres
```

Observe que el primer argumento se almacena en el índice 0, el segundo argumento se almacena en el índice 1, y así sucesivamente.

Para tener una idea de la forma en que se pueden usar los argumentos de la línea de comandos, considere el siguiente programa. Se necesita un argumento de línea de comandos que especifique el nombre de una persona. Luego busca a través de una matriz bidimensional de cadenas para ese nombre. Si encuentra una coincidencia, muestra el número de teléfono de esa persona.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
class Telefono {
    public static void main (String[] args){
        String numeros[][]={{ "Alex", "123-456"},
                            { "Juan", "145-478"},
                            { "Javier", "789-457"},
                            { "Maria", "784-554"}
                            };
        int i;
        if (args.length != 1){
            System.out.println("Ejecute así: java Telefono <nombre>");
        } else {
            for (i = 0; i < numeros.length; i++) {
                System.out.println(numeros[i][0] + ": " + numeros);
                break;
            }
            if (i == numeros.length){
                System.out.println("Nombre no encontrado.");
            }
        }
    }
}
```

Aquí hay una muestra de ejecución:

```
java Telefono Alex
Alex: 123-456
```

5.2.8 Concatenar cadenas en `Java`

Operador `+`

```
String s1 = "Hola,";
String s2 = "cómo estas?";
String s3 = s1 + s2;
System.out.println("String 1: " + s1);
System.out.println("String 2: " + s2);
System.out.println("Cadena resultante: " + s3);
```

salida:

```
String 1: Hola,
String 2: cómo estas?
Cadena resultante: Hola,cómo estas?
```

Método `concat()`

```
String s1 = "Hola,";
String s2 = "cómo estas?";
String s3 = s1.concat(s2);
System.out.println("String 1: " + s1);
System.out.println("String 2: " + s2);
System.out.println("Cadena resultante: " + s3);
```

salida:

```
String 1: Hola,
String 2: cómo estas?
Cadena resultante: Hola,cómo estas?
```

Método `append` de `StringBuilder`

```
String s3 = (new StringBuilder()).append("Hola,").append("cómo estas?").toString();
System.out.println("Cadena resultante: " + s3);
```

salida:

```
Cadena resultante: Hola,cómo estas?
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

¿Cuándo usar cada uno?

Si usas la concatenación de Strings en un bucle, por ejemplo algo similar a esto:

```
String s = "";
for (int i = 0; i < 100; i++) {
    s += ", " + i;
}
System.out.println(s);
```

Para el caso anterior, lo mejor seria utilitzar el método `append` de `StringBuilder` en lugar del operador `+` porque es mucho más rápido y consume menos memoria.

Versión con append:

```
StringBuilder s = new StringBuilder();
for (int i = 1; i < 100; i++) {
    s.append(", ").append(i);
}
System.out.println(s.toString());
```

Si solo tienes una sentencia similar a esta:

```
String s = "1, " + "2, " + "3, " + "4, " ...;
```

Entonces puedes usar el operador `+` sin problemas, porque el compilador usará `StringBuilder` automáticamente.

5.2.9 Formato de salida: `printf` y `String.format`

Java proporciona dos mecanismos equivalentes para formatear cadenas con valores insertados: `System.out.printf()` y `String.format()`. Ambos usan los mismos especificadores de formato.

`System.out.printf()`

Imprime directamente una cadena formateada por consola. El primer argumento es la cadena con los especificadores, y los siguientes son los valores a insertar:

```
System.out.printf("Hola %s, tienes %d años.%n", "Juan", 25);
// Salida: Hola Juan, tienes 25 años.
```

`String.format()`

Funciona igual que `printf` pero **devuelve** la cadena formateada en lugar de imprimirla:

```
String mensaje = String.format("Hola %s, tienes %d años.%n", "Juan", 25);
System.out.print(mensaje);
```

Especificadores de formato

Especificador	Tipo	Ejemplo
`%d`	Entero (int, long)	`%d` → `42`
`%f`	Decimal (float, double)	`%f` → `3,141593`
`%s`	Cadena (String, cualquier objeto)	`%s` → `Hola`
`%c`	Carácter (char)	`%c` → `A`
`%b`	Booleano (boolean)	`%b` → `true`
`%n`	Salto de línea	`%n`
`%%`	Literal %	`%%` → `%`

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Ancho de campo y alineación

Se puede especificar un ancho mínimo para cada valor. El valor se alinea a la derecha por defecto:

```
System.out.printf("%10s %10s %10s%n", "Nombre", "Edad", "Peso");
System.out.printf("%10s %10d %10.2f%n", "Ana", 25, 55.5);
System.out.printf("%10s %10d %10.2f%n", "Juan", 30, 72.3);
```

Salida:

```
    Nombre       Edad       Peso
       Ana         25      55,50
      Juan         30      72,30
```

Para alinear a la izquierda, se usa el guion (`-`):

```
System.out.printf("%-10s %-10s %-10s%n", "Nombre", "Edad", "Peso");
System.out.printf("%-10s %-10d %-10.2f%n", "Ana", 25, 55.5);
```

Salida:

```
Nombre     Edad       Peso
Ana        25         55,50
```

Precisión en decimales

Para controlar el número de decimales en un `double` o `float`:

```
double pi = Math.PI;
System.out.printf("Pi con 2 decimales: %.2f%n", pi);       // 3,14
System.out.printf("Pi con 4 decimales: %.4f%n", pi);       // 3,1416
System.out.printf("Pi con ancho 10 y 2 decimales: %10.2f%n", pi); // "      3,14"
```

Cero-padding en enteros

Para rellenar con ceros a la izquierda:

```
System.out.printf("%04d%n", 42);   // 0042
System.out.printf("%02d:%02d%n", 9, 5);  // 09:05
```

Ejemplo completo: tabla de cuadrados

```
System.out.printf("%-8s %-8s %-8s%n", "Número", "Cuadrado", "Cubo");
for (int i = 1; i <= 5; i++) {
    System.out.printf("%-8d %-8d %-8d%n", i, i * i, i * i * i);
}
```

Salida:

```
Número   Cuadrado Cubo
1        1        1
2        4        8
3        9        27
4        16       64
5        25       125
```

31 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

5.3 Ejercicios de la UD04

5.3.1 Arrays. Ejercicios de recorrido

1. (Estaturas) Escribir un programa que lea de teclado la estatura de 10 personas y las almacene en un array. Al finalizar la introducción de datos, se mostrarán al usuario los datos introducidos con el siguiente formato:

```
Persona 1: 1.85 m.
Persona 2: 1.53 m.
...
Persona 10: 1.23 m.
```

2. (Lluvias) Se dispone de un fichero, de nombre *lluviasEnero.txt*, que contiene 31 datos correspondientes a las lluvias caídas en el mes de enero del pasado año. Se desea analizar los datos del fichero para averiguar:
   - La lluvia total caída en el mes.
   - La cantidad media de lluvias del mes.
   - La cantidad más grande de lluvia caída en un solo día.
   - Cual fue el día que más llovió.
   - La cantidad más pequeña de lluvia caída en un solo día.
   - Cual fue el día que menos llovió.
   - En cuantos días no llovió nada.
   - En cuantos días la lluvia superó la media.
   - Si en la primera quincena del mes llovió más o menos que en la segunda.
   - En cuantos días la lluvia fue menor que la del día siguiente.

   Para resolver el problema se desarrollarán los siguientes métodos:

   a. `public static void leerArray (double v[], String nombreFichero)`, que rellena el array v con datos que se encuentran en el fichero especificado. El número de datos a leer vendrá determinado por el tamaño del array y no por la cantidad de datos que hay en el fichero.
   b. `public static double suma(double[] v)`, que devuelve la suma de los elementos del array v
   c. `public static double media(double v[])`, que devuelve la media de los elementos del array v. Se puede hacer uso del método del apartado anterior.
   d. `public static int contarMayorQueMedia(double v[])`, que devuelve la cantidad de elementos del array v que son mayores que la media. Se puede hacer uso del método del apartado anterior.
   e. `public static double maximo(double v[])`, que devuelve el valor más grande almacenado en el array v.
   f. `public static double minimo(double v[])`, que devuelve el valor más pequeño almacenado en el array v.
   g. `public static int posMaximo(double v[])`, que devuelve la posición del elemento más grande de v. Si éste se repite en el array es suficiente devolver la posición en que aparece por primera vez.
   h. `public static int posMinimo(double v[])`, que devuelve la posición del elemento más pequeño de v. Si éste se repite en el array es suficiente devolver la posición en que aparece por primera vez.
   i. `public static int contarApariciones(double v[], double x)`, que devuelve el número de veces que el valor *x* aparece en el array v.
   j. `public static double sumaParcial(double v[], int izq, int der)`, que devuelve la suma de los elementos del array v que están entre las posiciones *izq* y *der*.
   k. `public static int menoresQueElSiguiente(double v[])`, que devuelve el número de elementos de v que son menores que el elemento que tienen a continuación.

   Además dispones de un archivo `Lluvias.java` (incompleto), que el alumnado deberá completar. El resultado deberia ser similar a este:

```
La suma de las lluvias es 93,30 litros
La media de las lluvias es 3,01 litros
La máxima de las lluvias es 12,40 litros
La máxima de las lluvias fué el dia 17
La mínima de las lluvias es 0,00 litros
La mínima de las lluvias fué el dia 1
Ha habido un total de 16 dias sin lluvia
Ha habido un total de 11 dias en los que la lluvia ha superado la media
La segunda quincena ha llovido más que la otra
Ha habido 8 dias en los que ha llovido menos que el dia siguiente
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

3. (Dados) El lanzamiento de un dado es un experimento aleatorio en el que cada número tiene las mismas probabilidades de salir. Según esto, cuantas más veces lancemos el dado, más se igualarán las veces que aparece cada uno de los 6 números. Vamos a hacer un programa para comprobarlo.

   - Generaremos un número aleatorio entre 1 y 6 un número determinado de veces (por ejemplo 100.000). Para ello puedes usar el método `random` de la clase `Math`.
   - Tras cada lanzamiento incrementaremos un contador correspondiente a la cifra que ha salido. Para ello crearemos un array `veces` de 7 componentes, en el que el `veces[1]` servirá para contar las veces que sale un 1, `veces[2]` para contar las veces que sale un 2, etc. `veces[0]` no se usará.
   - Cada, por ejemplo, 1.000 lanzamientos mostraremos por pantalla las estadísticas que indican que porcentaje de veces ha aparecido cada número en los lanzamientos hechos hasta ese momento. Por ejemplo:

     ```
     Número de lanzamientos: 1000
     1: 18 %
     2: 14 %
     3: 21 %
     4: 10 %
     5: 18 %
     6: 19 %

     Número de lanzamientos: 2000
     ...
     ```

   - Para el número de lanzamientos (100.000 en el ejemplo) y para la frecuencia con que se muestran las estadísticas (1.000 en el ejemplo) utilizaremos dos **constantes** enteras, de nombre `LANZAMIENTOS` y `FRECUENCIA`, de esta forma podremos variar de forma cómoda el modo en que probamos el programa.

4. (Invertir) Diseñar un método `public static int[] invertirArray(int[] v)`, que dado un array `v` devuelva otro con los elementos en orden inverso. Es decir, el último en primera posición, el penúltimo en segunda, etc.

   Desde el método `main` crearemos e inicializaremos un array, llamaremos a `invertirArray` y mostraremos el array invertido.

   Implementa un método que imprima por pantalla un Array `public static void imprimirArray(int[] v)`, y así poder imprimir el Array v (con espacios entre cada elemento) aquí tienes el resultado de imprimir el array con los 10 primeros números:

   ```
   1 2 3 4 5 6 7 8 9 10
   ```

5. (SumasParciales) Se quiere diseñar un método `public static int[] sumaParcial(int[] v)`, que dado un array de enteros `v`, devuelva otro array de enteros t de forma que `t[i] = v[0] + v[1] + ...+ v[i]`. Es decir:

   ```
   t[0] = v[0]
   t[1] = v[0] + v[1]
   t[2] = v[0] + v[1] + v[2]
   ...
   t[10] = v[0] + v[1] + v[2] + ... + v[10]
   ```

   Desde el método main crearemos e inicializaremos un array, llamaremos a `sumaParcial` y mostraremos el array resultante.

   Ejemplo de salida, suponiendo que `v = {2,4,1,0,6}`:

   ```
   El valor del array con sumas parciales es:
   2 6 7 7 13
   ```

6. (Rotaciones) Rotar una posición a la derecha los elementos de un array consiste en mover cada elemento del array una posición a la derecha. El último elemento pasa a la posición 0 del array. Por ejemplo si rotamos a la derecha el array `{1,2,3,4}` obtendríamos `{4,1,2,3}`.

   - Diseñar un método `public static void rotarDerecha(int[] v)`, que dado un array de enteros rote sus elementos un posición a la derecha.
   - En el método `main` crearemos e inicializaremos un array y rotaremos sus elementos tantas veces como elementos tenga el array (mostrando cada vez su contenido), de forma que al final el array quedará en su estado original. Por ejemplo, si inicialmente el array contiene `{7,3,4,2}`, el programa mostrará

     ```
     Rotación 1: 2 7 3 4
     Rotación 2: 4 2 7 3
     Rotación 3: 3 4 2 7
     Rotación 4: 7 3 4 2
     ```

   - Diseña también un método (`rotarIzquierda`) para rotar a la izquierda y pruébalo de la misma forma.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

7. (DosArrays) Desarrolla los siguientes métodos en los que intervienen dos arrays y pruébalos desde el método `main`
   - `public static double[] sumaArraysIguales (double[] a, double[] b)` que dados dos arrays de `double` `a` y `b`, del mismo tamaño devuelva un array con la suma de los elementos de `a` y `b`, es decir, devolverá el array `{a[0]+b[0], a[1]+b[1], ....}`
   - `public static double[] sumaArrays(double[] a, double[] b)`. Repite el ejercicio anterior pero teniendo en cuenta que `a` y `b` podrían tener longitudes distintas. En tal caso el número de elementos del array resultante coincidirá con la longitud del array de mayor tamaño.

5.3.2 Arrays. Ejercicios de búsqueda

1. (Lluvias – continuación). Queremos incorporar al programa la siguiente información:
   - Cual fue el **primer** día del mes en que llovió exactamente 19 litros (si no hubo ninguno mostrar un mensaje por pantalla indicándolo)
   - Cual fue el **último** día del mes en que llovió exáctamente 8 litros (si no hubo ninguno mostrar un mensaje por pantalla indicándolo)

   Para ello desarrollarán los siguientes métodos:
   - `public static int posPrimero(double[] v, double x)`, que devuelve la posición de la primera aparición de `x` en el array `v`. Si `x` no está en `v` el método devolverá -1. El método realizará una búsqueda ascendente para proporcionar el resultado.
   - `public static int posUltimo(double[] v, double x)`, que devuelve la posición de la última aparición de `x` en el array `v`. Si `x` no está en `v` el método devolverá -1. El método realizará una búsqueda descendente para proporcionar el resultado.
2. (Tocayos) Disponemos de los nombres de dos grupos de personas (dos arrays de `String`). Dentro de cada grupo todas las personas tienen nombres distintos, pero queremos saber cuántas personas del primer grupo tienen algún tocayo en el segundo grupo, es decir, el mismo nombre que alguna persona del segundo grupo. Escribir un programa que resuelva el problema: en el `main`, inicializa los dos arrays con los nombres del ejemplo siguiente y diseña los métodos que consideres necesarios.

   Por ejemplo, si los nombres son {"miguel","**josé**","**ana**","maría"} y {"**Ana**", "luján", "juan", "**josé**", "pepa", "ángela", "sofía", "andrés", "bartolo"} , el programa mostraría:

```
josé tiene tocayo en el segundo grupo.
ana tiene tocayo en el segundo grupo.
TOTAL: 2 personas del primer grupo tienen tocayo.
```

   Optimiza el algoritmo para que no tenga en cuenta si se escribe el nombre en mayúsculas, minúsculas o cualquier combinación de mayúsculas y minúsculas.
3. (SumaDespuesImpar) Escribir un método (`sumaDespuesImpar`) que, dado un array de enteros, devuelva la suma de los elementos que aparecen tras el primer valor impar (la suma debe incluir el valor impar encontrado). Usar `main` para probar el método.

```
Para el array: {0,2,4,6,3,4,7,8,1}
El total a partir del primer impar es: 23
```

4. (HayPares) (En papel) Para determinar si existe algún valor par en un array se proponen varias soluciones. Indica cual/cuales son válidas para resolver el problema.

```java
public static boolean haypares1(int[] v) {

    int i = 0;

    while (i < v.length && v[i] % 2 != 0) {
        i++;
    }

    if (v[i] % 2 == 0) {
        return true;
    } else {
        return false;
    }

}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
public static boolean haypares2(int v[]) {

    int i = 0;

    while (i < v.length && v[i] % 2 != 0) {
        i++;
    }

    if (i < v.length) {
        return true;
    } else {
        return false;
    }

}
```

```java
public static boolean haypares3(int v[]) {

    int i = 0;

    while (v[i] % 2 != 0 && i < v.length) {
        i++;
    }

    if (i < v.length) {
        return true;
    } else {
        return false;
    }

}
```

```java
public static boolean haypares4(int v[]) {

    int i = 0;

    boolean encontrado = false;

    while (i <= v.length && !encontrado) {

        if (v[i] % 2 == 0) {
            encontrado = true;
        } else {
            i++;
        }

    }

    return encontrado;

}
```

```java
public static boolean haypares5(int v[]) {

    int i = 0;

    boolean encontrado = false;

    while (i < v.length && !encontrado) {

        if (v[i] % 2 == 0) {
            encontrado = true;
        }

        i++;

    }

    return encontrado;

}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
public static boolean haypares6(int v[]) {

    int i = 0;

    while (i < v.length) {

        if (v[i] % 2 == 0) {
            return true;
        } else {
            return false;
        }

    }

}
```

```java
public static boolean haypares7(int v[]) {

    int i = 0;

    while (i < v.length) {

        if (v[i] % 2 == 0) {
            return true;
        }

        i++;

    }

    return false;

}
```

5. (Capicúa) Escribir un método (`capicua`) para determinar si un array de palabras (`String`) es capicúa, esto es, si la primera y última palabra del array son la misma, la segunda y la penúltima palabras también lo son, y así sucesivamente. Escribir el método `main` para probar el método anterior.

```
El array: {"Ana", "Barcelona", "Casa", "David", "Elena", "David", "Casa", "Barcelona", "Ana"} es capicua?: true
El array: {"Amigo", "Besugo", "Cita", "Dia", "Enano", "Dia"} es capicua?: false
```

6. (Subsecuencia) Escribir un método (`subsecuencia`) que, dado un array, determine la posición de la primera subsecuencia del array que comprenda al menos tres números enteros consecutivos en posiciones consecutivas del array. De no existir dicha secuencia devolverá -1. Realiza al menos una prueba del método en el `main` .

   Por ejemplo: en el array {23, 8, 12, 6, 7, **9, 10, 11**, 2} hay 3 números consecutivos en tres posiciones consecutivas, a partir de la posición 6 humana (5 del array): {9,10,11}

```
En el array: {23, 8, 12, 6, 7, 9, 10, 11, 1}
Posición de la subsecuencia: 6
```

7. (MismosValores) Se desea comprobar si dos arrays de `double` contienen los mismos valores, aunque sea en orden distinto. Para ello se ha escrito el siguiente método, que aparece incompleto:

```java
public static boolean mismosValores(double[] v1, double[] v2) {
    boolean encontrado = false;
    int i = 0;
    while (i < v1.length && !encontrado) {
        boolean encontrado2 = false;
        int j = 0;
        while (j < v2.length && !encontrado2) {
            if (v1[?] == v2[?]) {
                encontrado2 = true;
                i++;
            } else {
                ?;
            }
        }
        if (encontrado2 == ?) {
            encontrado = true;
        }
    }
    return !encontrado;
}
```

   Completa el programa en los lugares donde aparece el símbolo **?**

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

5.3.3 Matrices

1. (Notas). Se dispone de una matriz (de doubles) que contiene las notas de una serie de alumnos en una serie de asignaturas. Cada fila corresponde a un alumno, mientras que cada columna corresponde a una asignatura. Desarrollar métodos para:
   a. Imprimir las notas alumno por alumno `imprimirXAlumno`.
   b. Imprimir las notas asignatura por asignatura `imprimirXAsignatura`.
   c. Imprimir la media de cada alumno `imprimirMediaXAlumno`.
   d. Imprimir la media de cada asignatura `imprimirMediaXAsignatura`.
   e. Indicar cual es la asignatura más fácil, es decir la de mayor nota media `indicarAsignaturaMasFacil`.
   f. ¿Hay algún alumno que suspenda todas las asignaturas? ¿Quién? `alumnosSuspendenTodas`
   g. ¿Hay alguna asignatura en la que suspendan todos los alumnos? ¿Cual es? `asignaturasSuspendenTodos`

   Generar la matriz (al menos 5x5) en el método main, rellenarla, y comprobar los métodos anteriores.

   Ejemplo de ejecución:

```
{{5.3, 6.8, 8.2, 3.4,  5.0, 3.0},
{0.0, 4.5, 3.0, 2.8,  4.9, 2.0},
{7.8, 8.8, 9.8, 8.9,  9.9, 1.2},
{6.5, 5.5, 4.5, 7.8, 10.0, 2.2},
{5.5, 6.6, 5.6, 6.7,  7.0, 3.3}}

--- IMPRIMIR POR ALUMNOS ---
Notas para el Alumno 0: 5,30 6,80 8,20 3,40 5,00 3,00
Notas para el Alumno 1: 0,00 4,50 3,00 2,80 4,90 2,00
Notas para el Alumno 2: 7,80 8,80 9,80 8,90 9,90 1,20
Notas para el Alumno 3: 6,50 5,50 4,50 7,80 10,00 2,20
Notas para el Alumno 4: 5,50 6,60 5,60 6,70 7,00 3,30

--- IMPRIMIR POR ASIGNATURAS ---
Notas para la Asignatura 0: 5,30 0,00 7,80 6,50 5,50
Notas para la Asignatura 1: 6,80 4,50 8,80 5,50 6,60
Notas para la Asignatura 2: 8,20 3,00 9,80 4,50 5,60
Notas para la Asignatura 3: 3,40 2,80 8,90 7,80 6,70
Notas para la Asignatura 4: 5,00 4,90 9,90 10,00 7,00
Notas para la Asignatura 5: 3,00 2,00 1,20 2,20 3,30

--- IMPRIMIR MEDIA POR ALUMNO ---
Media para el Alumno 0: 5,28
Media para el Alumno 1: 2,87
Media para el Alumno 2: 7,73
Media para el Alumno 3: 6,08
Media para el Alumno 4: 5,78

--- IMPRIMIR MEDIA POR ASIGNATURA ---
Media para la Asignatura 0: 5,02
Media para la Asignatura 1: 6,44
Media para la Asignatura 2: 6,22
Media para la Asignatura 3: 5,92
Media para la Asignatura 4: 7,36
Media para la Asignatura 5: 2,34

--- INDICAR LA ASIGNATURA MÁS FACIL---
4

--- ¿HAY ALGUN ALUMNO QUE SUSPENDA TODAS? ---
El alumno 1 las suspende todas

--- ¿HAY ALGUNA ASIGNATURA QUE SUSPENDAN TODOS? ---
La asignatura 5 la suspenden todos
```

2. (Ventas). Una empresa comercializa 10 productos para lo cual tiene 5 distribuidores.

   Los datos de ventas los tenemos almacenados en una matriz (de enteros) de 5 filas x 10 columnas, `ventas`, con el número de unidades de cada producto que ha vendido cada distribuidor. Cada fila corresponde a las ventas de un distribuidor (la primera fila, del primer distribuidor, etc.), mientras que cada columna corresponde a un producto :

ventas	**columna[0]**	**columna[1]**	**columna[2]**	**columna[3]**	**columna[4]**	**c**
fila[0]	100	25	33	89	23	9
fila[1]	28	765	65	77	987	5
...	...					
...						

   El array `precio`, de 10 elementos, contiene el precio en € de cada uno de los 10 productos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

precio[0]	precio[1]	precio[2]	...
125.2	234.4	453.9	...

Escribe el programa y los métodos necesarios para averiguar:

a. Distribuidor que más artículos ha vendido `int distribuidorMasVende(int[][] matriz)` .

b. El artículo que más se vende `int articuloMasVende(int[][] matriz)`.

c. Sabiendo que los distribuidores que realizan ventas superiores a 30.000€ cobran una comisión del 5% de las ventas y los que superan los 70.000€ una comisión del 8%, emite un informe de los distribuidores que cobran comisión, indicando nº de distribuidor, importe de las ventas, porcentaje de comisión e importe de la comisión en €
`void informe(int[][] ventas, double[] precio)`.

Ejemplo de ejecución

```
--- VENTAS DE DISTRIBUIDORES ---
Distribuidores (filas) y productos (columnas)
{
{5, 10, 15, 20, 25, 30, 35, 40, 45, 50},
{55, 45, 35, 25, 20, 10, 15, 25, 30, 50},
{25, 30, 35, 30, 25, 30, 35, 30, 25, 30},
{60, 59, 58, 57, 56, 55, 54, 53, 52, 51},
{10, 20, 30, 40, 50, 40, 30, 20, 10, 5}
}
--------------------------------
PRECIOS DE LOS PRODUCTOS
{55.0, 65.5, 85, 95.5, 55.5, 65, 85.5, 59, 510, 511}
--------------------------------
EL DISTRIBUIDOR QUE MÁS VENDE: 3
EL ARTÍCULO QUE MÁS SE VENDE: 9

--- INFORME DE VENTAS ---
El distribuidor número 1 ha vendido 61.305,00. Lo que supone un porcentaje de 5% que supone 3.065,25 €.
El distribuidor número 2 ha vendido 56.702,50. Lo que supone un porcentaje de 5% que supone 2.835,13 €.
El distribuidor número 3 ha vendido 45.360,00. Lo que supone un porcentaje de 5% que supone 2.268,00 €.
El distribuidor número 4 ha vendido 84.546,00. Lo que supone un porcentaje de 8% que supone 6.763,68 €.
El distribuidor número 5 ha vendido 25.005,00. Lo que no supone ninguna comisión.
```

3. (Utiles) Dada una matríz (de enteros) con el mismo número de filas y de columnas, diseñar los siguientes métodos:

- `public static void mostrarDiagonal(int[][] m)` que muestre por pantalla los elementos de la diagonal principal.
- `public static int filaDelMayor (int[][] m)`, que devuelva la fila en que se encuentra el mayor elemento de la matriz. (Si aparece más de una vez, el primero que aparezca de arriba a abajo)
- `public static void intercambiarFilas(int[][] m, int f1, int f2)`, que intercambie los elementos de las filas indicadas.
- Escribir un método `public static boolean esSimetrica (int[][] m)` que devuelva true si la matriz m es simétrica. Una matriz es simétrica si tiene el mismo número de filas que de columnas y además `m[i][j] = m[j][i]` para todo par de índices `i,j`.
- Escribir una versión mejorada, `public static boolean esSimetricaMejorada(int[][] m)`, que haga la misma comprobación recorriendo solo las posiciones que están por encima de la diagonal principal.

Atención

Todos los métodos se llaman y devuelven con los valores "reales" de las filas y columnas (0-based), pero los valores se muestra al usuario con los valores "amigables" (1-based)

Atajo

Puedes usar `Arrays.deepToString(m)` para imprimir las matrices

Por ejemplo, es simétrica:

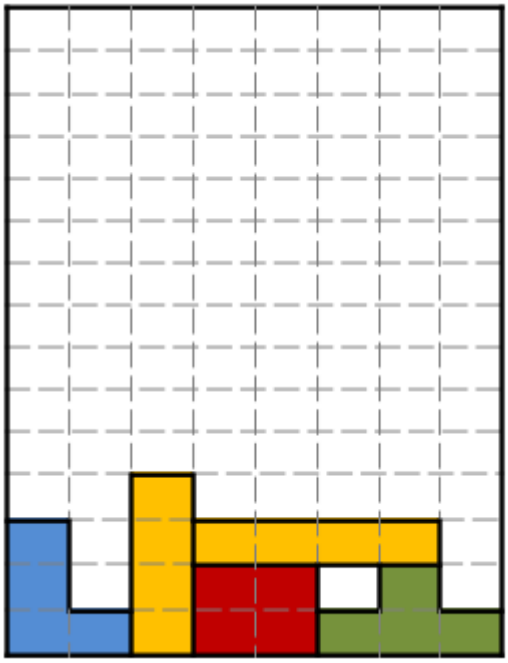
```
1 5 3
5 4 7
3 7 5
```

Ejemplo de ejecución

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
MATRIZ:
{{1, 5, 3},
{5, 4, 7},
{3, 7, 5}}
MOSTRAR DIAGONAL: 1 4 5
FILA DEL MAYOR: 2
ANTES DE INVERTIR LA FILA 1 Y LA 3
[[1, 5, 3], [5, 4, 7], [3, 7, 5]]
DESPUES DE INVERTIR LA FILA 1 Y LA 3
[[3, 7, 5], [5, 4, 7], [1, 5, 3]]
VOLVEMOS A INVERTIR LAS FILAS PARA CONSEGUIR LA MATRIZ ORIGINAL
[[1, 5, 3], [5, 4, 7], [3, 7, 5]]
LA MATRIZ ES SIMETRICA?: true
```

4. (Tetris) Supongamos que estamos desarrollando un Tetris en Java y para representar la partida utilizamos una matriz bidimensional de enteros 15 filas por 8 columnas. Se utiliza el valor 0 para indicar que la celda está vacía y un valor distinto de cero para las celdas que contienen parte de una pieza (distintos valores para distintos colores):



0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	2	0	0	0	0	0
1	0	2	2	2	2	2	0
1	0	2	4	4	0	3	0
1	1	2	4	4	3	3	3

Escribir un método que reciba la matriz y elimine las filas completas, haciendo caer las piezas que hay por encima de las celdas eliminadas tal y como se hace en el juego.

Métodos a implementar:

- `boolean esFilaLlena(int[] fila)` : devuelve true si la fila está llena (ningún cero)
- `void limpiarFilas(int[][] partida)` : Recorremos todas las filas de abajo hacia arriba, si la fila está llena: la eliminamos y bajamos todas las celdas superiores y volvemos a comprobar la fila donde estábamos. Sino seguimos hacia arriba.
- `void imprimePartida(int[][] partida)` : imprime la partida por consola

Ejemplo de ejecución1:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
Comienzo1
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 2, 0, 0, 0, 0, 0]
[1, 0, 2, 2, 2, 2, 2, 0]
[1, 0, 2, 4, 4, 0, 3, 0]
[1, 1, 2, 4, 4, 3, 3, 3]
Limpia1
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 2, 0, 0, 0, 0, 0]
[1, 0, 2, 2, 2, 2, 2, 0]
[1, 0, 2, 4, 4, 0, 3, 0]
```

Ejemplo de ejecución 2:

```
Comienzo2
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 2, 0, 0, 0, 0, 0]
[1, 0, 2, 2, 2, 2, 2, 0]
[1, 3, 2, 4, 4, 4, 3, 4]
[1, 1, 2, 4, 4, 3, 3, 3]
Limpia2
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 2, 0, 0, 0, 0, 0]
[1, 0, 2, 2, 2, 2, 2, 0]
```

Ejemplo de ejecución 3:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
Comienzo3
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[5, 5, 5, 5, 5, 4, 4, 4]
[0, 0, 0, 0, 4, 4, 0, 0]
[0, 0, 0, 0, 4, 4, 0, 0]
[0, 0, 0, 0, 4, 6, 0, 0]
[0, 0, 0, 0, 4, 7, 0, 0]
[0, 0, 0, 0, 4, 3, 0, 0]
[0, 0, 2, 0, 3, 2, 0, 0]
[1, 0, 2, 2, 2, 2, 2, 0]
[1, 3, 2, 4, 4, 4, 3, 4]
[1, 1, 2, 4, 4, 3, 3, 3]
Limpia3
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 4, 4, 0, 0]
[0, 0, 0, 0, 4, 4, 0, 0]
[0, 0, 0, 0, 4, 6, 0, 0]
[0, 0, 0, 0, 4, 7, 0, 0]
[0, 0, 0, 0, 4, 3, 0, 0]
[0, 0, 2, 0, 3, 2, 0, 0]
[1, 0, 2, 2, 2, 2, 2, 0]
```

5.3.4 Recursividad

1. (Palindromo) Implemente, tanto de forma recursiva (`boolean palindromoRecursivo(String frase)`) como de forma iterativa (`boolean palindromoIterativo(String frase)`), métodos que nos digan si una cadena de caracteres es simétrica (un palíndromo). Por ejemplo, "DABALEARROZALAZORRAELABAD" es un palíndromo.

   "La ruta nos aporto otro paso natural"

   "Nada, yo soy Adan"

   "A mama Roma le aviva el amor a papa y a papa Roma le aviva el amor a mama"

   "Ana, la tacaña catalana"

   "Yo hago yoga hoy"

   > **Importante**
   >
   > Recuerda que no se debe tener en cuenta si la letra está en mayúsculas o minúsculas, solo si es la misma, por tanto `Ana` también es un palíndromo.

   > **Reto**
   >
   > ¿Te atreves a implementar una solución que permita la entrada con espacios `PalindromoConEspacios` ? ¿Y permitiendo espacios y signos de puntuación `PalindromoConEspaciosYPuntuacion` ?"

2. (Voltear) Implemente, tanto de forma recursiva (`String recursiva(String frase)`) como de forma iterativa (`String iterativa(String frase)`), una función que le dé la vuelta a una cadena de caracteres.

   > **Importante**
   >
   > Obviamente, si la cadena es un palíndromo, la cadena y su inversa coincidirán.

   Ejemplo de ejecución:

```
Introduce una frase para darle la vuelta: David es el mejor
Iterativa: rojem le se divaD
Recursiva: rojem le se divaD
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

3. (Combinaciones) Implemente, tanto de forma recursiva como de forma iterativa, una función que permitan calcular el número de combinaciones de `n` elementos tomados de `m` en `m`.

   Realice dos versiones de la implementación iterativa, una aplicando la fórmula y otra utilizando una matriz auxiliar (en la que se vaya construyendo el triángulo de Pascal).

4. (MCD) Implemente, tanto de forma recursiva (`int recursiva(int m, int n)`) como de forma iterativa (`int iterativa(int m, int n)`), una función que nos devuelva el máximo común divisor de dos números enteros utilizando el algoritmo de Euclides.

```
ALGORITMO DE EUCLIDES MCD
Dados dos números enteros positivos m y n, tal que m > n, para encontrar su máximo común divisor (es decir, el mayor entero positivo que divide a
ambos):
- Dividir m por n para obtener el resto r (0 ≤ r < n)
- Si r = 0, el MCD es n.
- Si no, el máximo común divisor es MCD(n,r).
```

5. (MergeSort) La ordenación por mezcla (mergesort) es un método de ordenación que se basa en un principio muy simple: se ordenan las dos mitades de un vector y, una vez ordenadas, se mezclan. Escriba un programa que implemente este método de ordenación.
6. (Descomposiciones) Diseñe e implemente un algoritmo que imprima todas las posibles descomposiciones de un número natural como suma de números menores que él (sumas con más de un sumando).
7. (Determinante) Diseñe e implemente un método recursivo que nos permita obtener el determinante de una matriz cuadrada de dimensión `n`.
8. (CifrasYLetras) Diseñe e implemente un programa que juegue al juego de cifras de "Cifras y Letras ". El juego consiste en obtener, a partir de 6 números, un número lo más cercano posible a un número de tres cifras realizando operaciones aritméticas con los 6 números.
9. (OchoReinas) Problema de las 8 reinas: Se trata de buscar la forma de colocar 8 reinas en un tablero de ajedrez de forma que ninguna de ellas amenace ni se vea amenazada por otra reina.

```
 Algoritmo:
 - Colocar la reina i en la primera casilla válida de la fila i
 - Si una reina no puede llegar a colocarse en ninguna casilla, se vue lve atrás y se cambia la posición de la reina de la fila i-1
 - Intentar colocar las reinas restantes en las filas que quedan
```

10. (Laberinto) Salida de un laberinto: Se trata de encontrar un camino que nos permita salir de un laberinto definido en una matriz NxN. Para movernos por el laberinto, sólo podemos pasar de una casilla a otra que sea adyacente a la primera y no esté marcada como una casilla prohibida (esto es, las casillas prohibidas determinan las paredes que forman el laberinto).

```
Algoritmo:
- Se comienza en la casilla (0,0) y se termina en la casilla (N-1, N-1)
- Nos movemos a una celda adyacente si esto es posible.
- Cuando llegamos a una situación en la que no podemos realizar ningún movimiento que nos lleve a una celda que no hayamos visitado ya, retrocedemos
sobre nuestros pasos y buscamos un camino alternativo.
```

21 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

5.4 Talleres

5.4.1 Taller UD04_01: Classroom 50

Aceptar la tarea en Classroom 50

Aceptamos el *Assignement* (la tarea/ejercicio) a partir del link del profesor, en este caso: https://classroom50.org/IES-Eduardo-Primo-Marques/1dam-programacio-java/assignments/ud04

Si no recuerdas cómo aceptar la tarea, clonar el repositorio o abrirlo en IntelliJ, repasa la guía completa en el taller de UD02.

Tarea

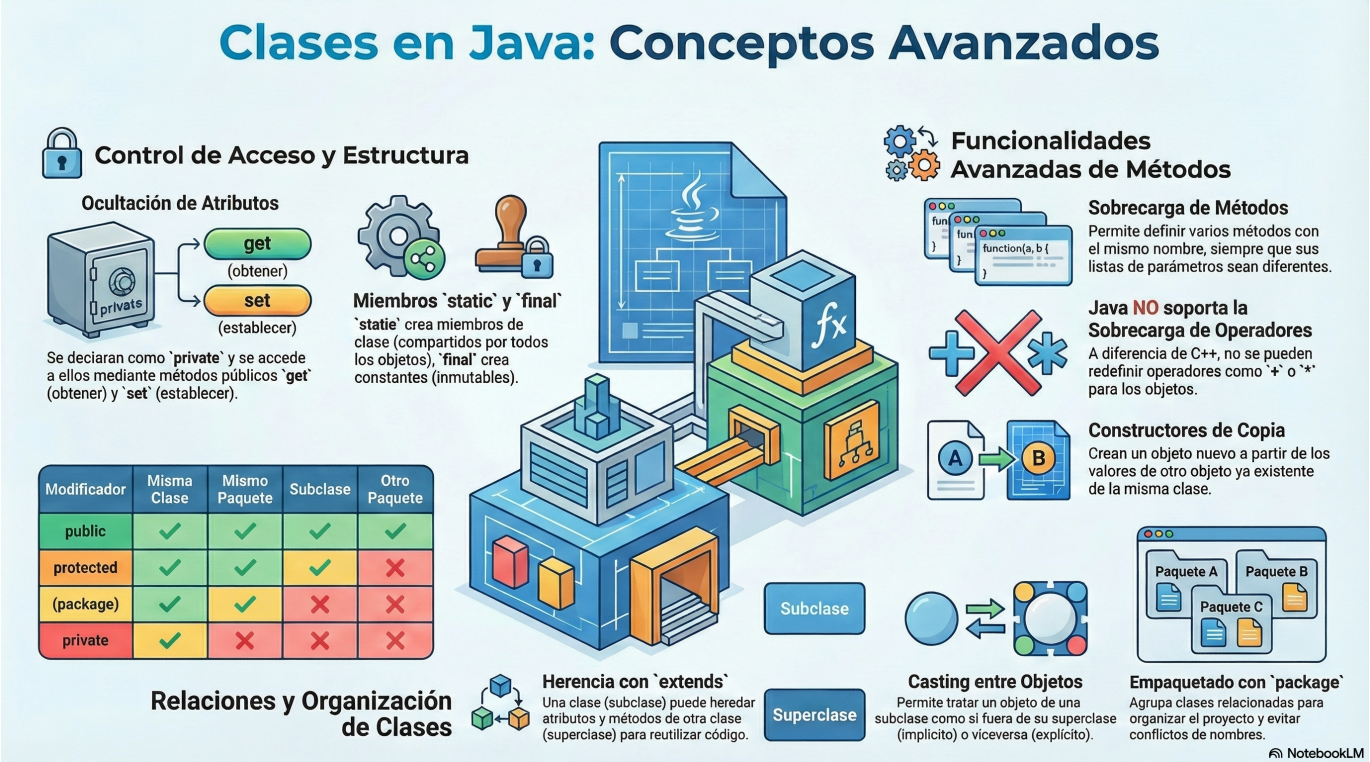
Debes enviar tus soluciones a Classroom 50 y superar al menos la mitad de los tests, cuantos más tests superados, mejor nota tendrás en la tarea.

6 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

6. UD05

6.1 Desarrollo de clases



Al finalizar esta unidad serás capaz de...

- Diseñar clases completas con atributos y métodos
- Aplicar modificadores de acceso (public, private, protected)
- Sobrecargar métodos y constructores
- Implementar la ocultación de atributos (getters/setters)
- Crear constructores de copia
- Definir clases anidadas e internas
- Comprender los fundamentos de la herencia
- Utilizar el empaquetado de clases (paquetes)

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Cómo estudiar esta unidad

Esta unidad retoma los conceptos de la Unidad 2: Utilización de Objetos y Clases y los amplía. Las secciones de repaso incluyen enlaces directos a UD02 para que puedas consultarlos si lo necesitas.

Si tienes claros los conceptos de la Unidad 2, ve directamente a las secciones marcadas como **[NUEVO]**, que contienen los contenidos que no se vieron en UD02:

- Modificadores de acceso
- Modificadores en la declaración de un método
- Sobrecarga de operadores
- Ocultación de atributos. Métodos de acceso
- Ocultación de métodos
- Constructores de copia
- Clases Anidadas, Clases Internas
- Introducción a la herencia
- Conversión entre objetos (Casting)
- Acceso a métodos de la superclase
- Empaquetado de clases

Si por el contrario tienes dudas o lagunas, esta es la última oportunidad de afianzar estos conceptos: a partir de aquí, objetos y clases serán parte del día a día. Los enlaces te llevarán directamente al contenido de UD02 que necesitas repasar.

6.1.1 Introducción

Esta unidad retoma los conceptos de POO vistos en la Unidad 2 y los desarrolla en profundidad. A continuación se resumen los conceptos fundamentales; si necesitas repasarlos con más detalle, sigue los enlaces a UD02.

Repaso del concepto de objeto

Un objeto es una entidad con **identidad** (único y distinguible), **estado** (valores de sus atributos) y **comportamiento** (métodos que puede ejecutar). Los objetos son instancias concretas de una clase.

Repaso

Si necesitas refrescar estos conceptos, revisa Objetos y Clases en UD02 y las características de la POO.

El concepto de clase

Una **clase** es una plantilla que define los atributos y métodos comunes a un conjunto de objetos. A partir de una clase se crean objetos (instancias).

Repaso

Revisa el concepto de clase en UD02 si tienes dudas.

6.1.2 Estructura y miembros de una clase

En unidades anteriores ya se indicó que para declarar una clase en Java se usa la palabra reservada `class`. En la declaración de una clase vas a encontrar:

- **Cabecera de la clase**. Compuesta por una serie de modificadores de acceso, la palabra reservada `class` y el nombre de la clase.
- **Cuerpo de la clase**. En él se especifican los distintos miembros de la clase: atributos y métodos. Es decir, el contenido de la clase.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
public class NombreDeLaClase [herencia] [interfaces]
{
    // Atributos de la clase
    ...
    ...
    ...
    // Métodos de la clase
    ...
    ...
    ...
}
```

Repaso

La sintaxis básica de declaración de una clase se explicó en Clases en UD02. Aquí nos centramos en los elementos avanzados de la cabecera y los modificadores.

Cabecera de una clase.

En general, la declaración de una clase puede incluir los siguientes elementos y en el siguiente orden:

1. Modificadores tales como `public`, `abstract` o `final`.
2. El nombre de la clase (con la primera letra de cada palabra en mayúsculas, por convenio).
3. El nombre de su clase padre (superclase), si es que se especifica, precedido por la palabra reservada `extends` ("extiende" o "hereda de").
4. Una lista separada por comas de interfaces que son implementadas por la clase, precedida por la palabra reservada `implements` ("implementa").
5. El cuerpo de la clase, encerrado entre llaves `{...}`.

La sintaxis completa de una cabecera (los cuatro primeros puntos) queda de la forma:

```
[modificadores] class <NombreClase> [extends <NombreSuperClase>][implements <NombreInterface1>][[implements<NombreInterface2>] ...] {
```

En el ejemplo anterior de la clase Punto teníamos la siguiente cabecera:

```
class Punto {
```

En este caso no hay modificadores, ni indicadores de herencia, ni implementación de interfaces. Tan solo la palabra reservada `class` y el nombre de la clase. Es lo mínimo que puede haber en la cabecera de una clase.

La herencia y las interfaces las verás más adelante. Vamos a ver ahora cuáles son los modificadores que se pueden indicar al crear la clase y qué efectos tienen. Los modificadores de clase son:

```
[public] [final | abstract]
```

Veamos qué significado tiene cada uno de ellos:

- Modificador `public`. Indica que la clase es visible (se pueden crear objetos de esa clase) desde cualquier otra clase. Es decir, desde cualquier otra parte del programa. Si no se especifica este modificador, la clase sólo podrá ser utilizada desde clases que estén en el mismo paquete. El concepto de paquete lo veremos más adelante. **Sólo puede haber una clase public (clase principal) en un archivo** `.java`. El resto de clases que se definan en ese archivo no serán públicas.
- Modificador `abstract`. Indica que la clase es abstracta. **Una clase abstracta no es instanciable**. Es decir, no es posible crear objetos de esa clase (habrá que utilizar clases que hereden de ella). En este momento es posible que te parezca que no tenga sentido que esto pueda suceder (si no puedes crear objetos de esa clase, ¿para qué la quieres?), pero puede resultar útil a la hora de crear una jerarquía de clases. Esto lo verás también más adelante al estudiar el concepto de herencia.
- Modificador `final`. Indica que no podrás crear clases que hereden de ella. También volverás a este modificador cuando estudies el concepto de herencia. **Los modificadores final y abstract son excluyentes (sólo se puede utilizar uno de ellos)**.

Todos estos modificadores y palabras reservadas las iremos viendo poco a poco, así que no te preocupes demasiado por intentar entender todas ellas en este momento.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

En el ejemplo anterior de la clase `Punto` tendríamos una clase que sería sólo visible (utilizable) desde el mismo paquete en el que se encuentra la clase (modificador de acceso por omisión o de paquete, o `package`). Desde fuera de ese paquete no sería visible o accesible. Para poder utilizarla desde cualquier parte del código del programa bastaría con añadir el atributo `public`:

```
public class Punto{
    ...
}
```

Cuerpo de una clase.

El cuerpo de una clase contiene la declaración de sus miembros:

- **Atributos**: datos que almacena cada objeto.
- **Métodos**: operaciones que se pueden realizar sobre el objeto.

Una clase no puede estar vacía: debe contener al menos un atributo o un método.

> **Repaso**
>
> Para más detalle sobre la estructura de una clase, consulta la Unidad 2.

Miembros estáticos o de clase

Los miembros declarados con `static` pertenecen a la clase, no a cada instancia. Se conocen como **miembros estáticos** o **miembros de clase**. Un atributo `static` es compartido por todos los objetos; un método `static` puede invocarse sin crear un objeto. Se estudiarán en detalle en las secciones de atributos estáticos y métodos estáticos.

6.1.3 Atributos

Los **atributos** (o variables miembro) son los datos que almacenan los objetos de una clase. Se declaran dentro de la clase y pueden ser de cualquier tipo (`int`, `double`, `String`, objetos, etc.).

```
[modificadores] <tipo> <nombreAtributo>;
```

> **Repaso**
>
> La declaración básica de atributos se explicó en Atributos en UD02. A continuación nos centramos en los modificadores que pueden acompañar a un atributo.

Dentro de la declaración de un atributo puedes encontrar tres partes:

- **Modificadores**. Son palabras reservadas que permiten modificar la utilización del atributo (indicar el control de acceso, si el atributo es constante, si se trata de un atributo de clase, etc.). Los iremos viendo uno a uno.
- **Tipo**. Indica el tipo del atributo. Puede tratarse de un tipo primitivo (`int`, `char`, `boolean`, `double` ...) o bien de uno referenciado (`objeto`, `array`, etc.).
- **Nombre**. Identificador único para el nombre del atributo. Por convenio se suelen utilizar las minúsculas. En caso de que se trate de un identificador que contenga varias palabras, a partir de la segunda palabra se suele poner la letra de cada palabra en mayúsculas. Por ejemplo: `primerValor`, `valor`, `puertaIzquierda`, `cuartoTrasero`, `equipoVecendor`, `sumaTotal`, `nombreCandidatoFinal`, etc. Cualquier identificador válido de Java será admitido como nombre de atributo válido, pero es importante seguir este convenio para facilitar la legibilidad del código (todos los programadores de Java lo utilizan).

Como puedes observar, los atributos de una clase también pueden contener modificadores en su declaración (como sucedía al declarar la propia clase). Estos modificadores permiten indicar cierto comportamiento de una tributo a la hora de utilizarlo. Entre los modificadores de un atributo podemos distinguir:

- **Modificadores de acceso**. Indican la forma de acceso al atributo desde otra clase. Son modificadores excluyentes entre sí. Sólo se puede poner uno.
- **Modificadores de contenido**. No son excluyentes. Pueden aparecer varios a la vez.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- **Otros modificadores**: `transient` y `volatile`. El primero se utiliza para indicar que un atributo es transitorio (no persistente) y el segundo es para indicar al compilador que no debe realizar optimizaciones sobre esa variable. Es más que probable que no necesites utilizarlos en este módulo.

Aquí tienes la sintaxis completa de la declaración de un atributo teniendo en cuenta la lista de todos los modificadores e indicando cuáles son incompatibles unos con otros:

```
[private | protected | public] [static] [final] [transient] [volatile] <tipo><nombreAtributo>;
```

Vamos a estudiar con detalle cada uno de ellos.

Modificadores de acceso [NUEVO]

Los modificadores de acceso disponibles en Java para un atributo son:

- **Modificador de acceso** `public`. Indica que cualquier clase (por muy ajena o lejana que sea) tiene acceso a ese atributo. No es muy habitual declarar atributos públicos (`public`).
- **Modificador de acceso** `protected`. En este caso se permitirá acceder al atributo desde cualquier subclase (lo verás más adelante al estudiar la herencia) de la clase en la que se encuentre declarado el atributo, y también desde las clases del mismo paquete.
- **Modificador de acceso por omisión** (o **de paquete**). Si no se indica ningún modificador de acceso en la declaración del atributo, se utilizará este tipo de acceso. Se permitirá el acceso a este atributo desde todas las clases que estén dentro del mismo paquete (`package`) que esta clase (la que contiene el atributo que se está declarando). No es necesario escribir ninguna palabra reservada. Si no se pone nada se supone se desea indicar este modo de acceso.
- **Modificador de acceso** `private`. Indica que sólo se puede acceder al atributo desde dentro de la propia clase. El atributo estará "oculto" para cualquier otra zona de código fuera de la clase en la que está declarado el atributo. Es lo opuesto a lo que permite `public`.

A continuación puedes observar un resumen de los distintos niveles accesibilidad que permite cada modificador:

modificador	Misma clase	Mismo paquete	Subclase	Otro paquete
`public`	✔	✔	✔	✔
`protected`	✔	✔	✔	❌
Sin modificador (`package`)	✔	✔	❌	❌
`private`	✔	❌	❌	❌

Recuerda

¡Recuerda que **los modificadores de acceso son excluyentes**! Sólo se puede utilizar uno de ellos en la declaración de un atributo.

Modificadores de contenido.

Los modificadores de contenido no son excluyentes (pueden aparecer varios para un mismo atributo). Son los siguientes:

- **Modificador** `static`. Hace que el atributo sea común para todos los objetos de una misma clase. Es decir, todos los objetos de la clase compartirán ese mismo atributo con el mismo valor. Es un caso de **miembro estático** o **miembro de clase**: un **atributo estático** o **atributo de clase** o **variable de clase**.
- **Modificador** `final`. Indica que el atributo es una constante. Su valor no podrá ser modificado a lo largo de la vida del objeto. Por convenio, el nombre de los atributos constantes (`final`) se escribe con todas las letras en mayúsculas.

En el siguiente apartado sobre atributos estáticos verás un ejemplo completo de un atributo estático (`static`). Veamos ahora un ejemplo de atributo constante (`final`).

Imagina que estás diseñando un conjunto de clases para trabajar con expresiones geométricas (figuras, superficies, volúmenes, etc.) y necesitas utilizar muy a menudo la constante pi con abundantes cifras significativas, por ejemplo, 3.14159265. Utilizar esa constante literal muy a menudo puede resultar tedioso además de poco operativo (imagina que el futuro hubiera que cambiar la cantidad de cifras significativas). La idea es declararla una sola vez, asociarle un nombre simbólico (un identificador) y utilizar ese identificador cada vez que se necesite la constante. En tal caso puede resultar muy útil declarar un atributo final con el valor

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

3.14159265 dentro de la clase en la que se considere oportuno utilizarla. El mejor identificador que podrías utilizar para ella será probablemente el propio nombre de la constante (y en mayúsculas, para seguir el convenio de nombres), es decir, `PI`.

Así podría quedar la declaración del atributo:

```
class ClaseGeometria {
    // Declaración de constantes
    public final double PI = 3.14159265;
    ...
```

Atributos estáticos.

Como ya has visto, el modificador `static` hace que el atributo sea común (el mismo) para todos los objetos de una misma clase. En este caso sí podría decirse que la existencia del atributo no depende de la existencia del objeto, sino de la propia clase y por tanto sólo habrá uno, independientemente del número de objetos que se creen. El atributo será siempre el mismo para todos los objetos y tendrá un valor único independientemente de cada objeto. Es más, aunque no exista ningún objeto de esa clase, el atributo sí existirá y podrá contener un valor (pues se trata de un atributo de la clase más que del objeto).

Uno de los ejemplos más habituales (y sencillos) de atributos estáticos o de clase es el de un contador que indica el número de objetos de esa clase que se han ido creando. Por ejemplo, en la clase de ejemplo `Punto` podrías incluir un atributo que fuera ese contador para llevar un registro del número de objetos de la clase `Punto` que se van construyendo durante la ejecución del programa.

Otro ejemplo de atributo estático (y en este caso también constante) que también se ha mencionado anteriormente al hablar de miembros estáticos era disponer de un atributo `nombre`, que contuviera un `String` con el nombre de la clase. Nuevamente ese atributo sólo tiene sentido para la clase, pues habrá de ser compartido por todos los objetos que sean de esa clase (es el nombre de la clase a la que pertenecen los objetos y por tanto siempre será la misma e igual para todos, no tiene sentido que cada objeto de tipo `Punto` almacene en su interior el nombre de la clase, eso lo debería hacer la propia clase).

```
class Punto {
    // Coordenadas del punto
    private int x, y;
    // Atributos de clase: cantidad de puntos creados hasta el momento
    public static cantidadPuntos;
```

Obviamente, para que esto funcione como estás pensando, también habrá que escribir el código necesario para que cada vez que se cree un objeto de la clase `Punto` se incremente el valor del atributo `cantidadPuntos`.

> **Más información**
>
> Volverás a este ejemplo para implementar esa otra parte cuando estudies los constructores.

6.1.4 Métodos

Los métodos definen el comportamiento de los objetos y constituyen su interfaz. A continuación se detallan los elementos avanzados de su declaración.

> **Repaso**
>
> La sintaxis básica de declaración de métodos se explicó en Métodos en UD02.

Cabecera de método.

La declaración de un método puede incluir los siguientes elementos:

1. **Modificadores** (como por ejemplo los ya vistos `public` o `private`, más algunos otros que irás conociendo poco a poco). No es obligatorio incluir modificadores en la declaración.
2. El **tipo devuelto** (o tipo de retorno), que consiste en el tipo de dato (primitivo o referencia) que el método devuelve tras ser ejecutado. Si eliges `void` como tipo devuelto, el método no devolverá ningún valor.
3. El **nombre** del método, aplicándose para los nombres el mismo convenio que para los atributos.
4. Una **lista de parámetros** separados por comas y entre paréntesis donde cada parámetro debe ir precedido por su tipo. Si el método no tiene parámetros la lista estará vacía y únicamente aparecerán los paréntesis.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

5. Una **lista de excepciones** que el método puede lanzar. Se utiliza la palabra reservada `throws` seguida de una lista de nombres de excepciones separadas por comas. No es obligatorio que un método incluya una lista de excepciones, aunque muchas veces será conveniente. En unidades anteriores ya has trabajado con el concepto de excepción y más adelante volverás a hacer uso de ellas.
6. El **cuerpo** del método, encerrado entre llaves. El cuerpo contendrá el código del método (una lista sentencias y estructuras de control en lenguaje Java) así como la posible declaración de variables locales.

La sintaxis general de la cabecera de un método podría entonces quedar así:

```
[private | protected | public] [static] [abstract] [final] [native] [synchronized] <tipo><nombreMétodo> ( [<lista_parametros>] ) [throws<lista_excepciones>]
```

Como sucede con todos los identificadores en Java (variables, clases, objetos, métodos, etc.), puede usarse cualquier identificador que cumpla las normas. Ahora bien, para mejorar la legibilidad del código, se ha establecido el siguiente convenio para nombrar los métodos: utilizar un verbo en minúscula o bien un nombre formado por varias palabras que comience por un verbo en minúscula, seguido por adjetivos, nombres, etc. los cuales sí aparecerán en mayúsculas.

Algunos ejemplos de métodos que siguen este convenio podrían ser: `ejecutar`, `romper`, `mover`, `subir`, `responder`, `obtenerX`, `establecerValor`, `estaVacio`, `estaLleno`, `moverFicha`, `subirPalanca`, `responderRapido`, `girarRuedaIzquierda`, `abrirPuertaDelantera`, `cambiarMarcha`, etc.

En el ejemplo de la clase `Punto`, puedes observar cómo los métodos `obtenerX` y `obtenerY` siguen el convenio de nombres para los métodos, devuelven en ambos casos un tipo `int`, su lista de parámetros es vacía (no tienen parámetros) y no lanzan ningún tipo de excepción:

- `java abstract int obtenerX()`
- `java int obtenerY()`

Modificadores en la declaración de un método [NUEVO]

En la declaración de un método también pueden aparecer modificadores (como en la declaración de la clase o delos atributos). Un método puede tener los siguientes tipos de modificadores:

- **Modificadores de acceso**. Son los mismos que en el caso de los atributos (por omisión o de paquete (`package`), `public`, `private` y `protected`) y tienen el mismo cometido (acceso al método sólo por parte de clases del mismo paquete, o por cualquier parte del programa, o sólo para la propia clase, o también para las subclases).
- **Modificadores de contenido**. Son también los mismos que en el caso de los atributos (`static` y `final`) aunque su significado no es el mismo.
- **Otros modificadores** (no son aplicables a los atributos, sólo a los métodos): `abstract`, `native`, `synchronized`.

Un método `static` es un método cuya implementación es igual para todos los objetos de la clase y sólo tendrá acceso a los atributos estáticos de la clase (dado que se trata de un método de clase y no de objeto, sólo podrá acceder a la información de clase y no la de un objeto en particular). Este tipo de métodos pueden ser llamados sin necesidad de tener un objeto de la clase instanciado.

En Java un ejemplo típico de métodos estáticos se encuentra en la clase `Math`, cuyos métodos son todos estáticos (`Math.abs`, `Math.sin`, `Math.cos`, etc.). Como habrás podido comprobar en este ejemplo, la llamada a métodos estáticos se hace normalmente usando el nombre de la propia clase y no el de una instancia (objeto), pues se trata realmente de un método de clase. En cualquier caso, los objetos también admiten la invocación de los métodos estáticos de su clase y funcionaría correctamente.

Un método `final` es un método que no permite ser sobrescrito por las clases descendientes de la clase a la quepertenece el método. Volverás a ver este modificador cuando estudies en detalle la herencia.

El modificador `native` es utilizado para señalar que un método ha sido implementado en código nativo (en un lenguaje que ha sido compilado a lenguaje máquina, como por ejemplo C o C++). En estos casos simplemente se indica la cabecera del método, pues no tiene cuerpo escrito en Java.

Un método `abstract` (método abstracto) es un método que no tiene implementación (el cuerpo está vacío). La implementación será realizada en las clases descendientes. Un método sólo puede ser declarado como `abstract` si se encuentra dentro de una clase `abstract`. También volverás a este modificador en unidades posteriores cuando trabajes con la herencia.

Por último, si un método ha sido declarado como `synchronized`, el entorno de ejecución obligará a que cuando un proceso esté ejecutando ese método, el resto de procesos que tengan que llamar a ese mismo método deberán esperar a que el otro proceso termine. Puede resultar útil si sabes que un determinado método va a poder ser llamado concurrentemente por varios procesos a la vez. Por ahora no lo vas a necesitar.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Dada la cantidad de modificadores que has visto hasta el momento y su posible aplicación en la declaración de clases, atributos o métodos, veamos un resumen de todos los que has visto y en qué casos pueden aplicarse:

modificador	Clase	Atributo	Método
Sin modificador (package)	✔	✔	✔
`public`	✔	✔	✔
`private`	❌	✔	✔
`protected`	✔	✔	✔
`static`	❌	✔	✔
`final`	✔	✔	✔
`synchronized`	❌	❌	✔
`native`	❌	❌	✔
`abstract`	✔	❌	✔

Parámetros en un método.

La lista de parámetros de un método se coloca tras el nombre del método. Esta lista estará constituida por pares de la forma `<tipoParametro> <nombreParametro>`. Cada uno de esos pares estará separado por comas y la lista completa estará encerrada entre paréntesis:

```
<tipo> nombreMetodo ( <tipo_1> <nombreParametro_1>, <tipo_2> <nombreParametro_2>, ..., <tipo_n><nombreParametro_n> )
```

Si la lista de parámetros es vacía, tan solo aparecerán los paréntesis:

```
<tipo> <nombreMetodo> ( )
```

A la hora de declarar un método, debes tener en cuenta:

- Puedes incluir cualquier cantidad de parámetros. Se trata de una decisión del programador, pudiendo ser incluso una lista vacía.
- Los parámetros podrán ser de cualquier tipo (tipos primitivos, referencias, objetos, arrays, etc.).
- No está permitido que el nombre de una variable local del método coincida con el nombre de un parámetro.
- No puede haber dos parámetros con el mismo nombre. Se produciría ambigüedad.
- Si el nombre de algún parámetro coincide con el nombre de un atributo de la clase, éste será ocultado por el parámetro. Es decir, al indicar ese nombre en el código del método estarás haciendo referencia al parámetro y no al atributo. Para poder acceder al atributo tendrás que hacer uso del operador de autorreferencia `this`, que verás un poco más adelante.
- En Java el paso de parámetros es siempre por valor, excepto en el caso de los tipos referenciados (por ejemplo los objetos) en cuyo caso se está pasando efectivamente una referencia. La referencia (el objeto en sí mismo) no podrá ser cambiada pero sí elementos de su interior (atributos) a través de sus métodos o por acceso directo si se trata de un miembro público.

Es posible utilizar una construcción especial llamada `varargs` (argumentos variables) que permite que un método pueda tener un número variable de parámetros. Para utilizar este mecanismo se colocan unos puntos suspensivos (tres puntos: `...`) después del tipo del cual se puede tener una lista variable de argumentos, un espacio en blanco y a continuación el nombre del parámetro que aglutinará la lista de argumentos variables.

```
<tipo><nombreMetodo> (<tipo> ... <nombre>)
```

Es posible además mezclar el uso de `varargs` con parámetros fijos. En tal caso, la lista de parámetros variables debe aparecer al final (y sólo puede aparecer una). En realidad se trata una manera transparente de pasar un `array` con un número variable de elementos para no tener que hacerlo manualmente. Dentro del método habrá que ir recorriendo el `array` para ir obteniendo cada uno de los elementos de la lista de argumentos variables.

Cuerpo de un método

El cuerpo de un método contiene las sentencias que implementan su lógica: declaración de variables locales, estructuras de control, y la sentencia `return` (si el método no es `void`).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Repaso

Para más detalle, consulta Parámetros y valores devueltos en UD02.

Sobrecarga de métodos

En principio podrías pensar que un método puede aparecer una sola vez en la declaración de una clase (no se debería repetir el mismo nombre para varios métodos). Pero no tiene porqué siempre suceder así. Es posible tener varias versiones de un mismo método (varios métodos con el mismo nombre) gracias a la sobrecarga de métodos.

El lenguaje Java soporta la característica conocida como sobrecarga de métodos. Ésta permite declarar en una misma clase varias versiones del mismo método con el mismo nombre. La forma que tendrá el compilador de distinguir entre varios métodos que tengan el mismo nombre será mediante la lista de parámetros del método: si el método tiene una lista de parámetros diferente, será considerado como un método diferente (aunque tenga el mismo nombre) y el analizador léxico no producirá un error de compilación al encontrar dos nombres de método iguales en la misma clase.

Imagínate que estás desarrollando una clase para escribir sobre un lienzo que permite utilizar diferentes tipografías en función del tipo de información que se va a escribir. Es probable que necesitemos un método diferente según se vaya a pintar un número entero (`int`), un número real (`double`) o una cadena de caracteres (`String`). Una primera opción podría ser definir un nombre de método diferente dependiendo de lo que se vaya a escribir en el lienzo. Por ejemplo:

- Método `pintarEntero (int entero)`.
- Método `pintarReal (double real)`.
- Método `pintarCadena (double String)`.
- Método `pintarEnteroCadena (int entero, String cadena)`.

Y así sucesivamente para todos los casos que desees contemplar...

La posibilidad que te ofrece la sobrecarga es utilizar un mismo nombre para todos esos métodos (dado que en el fondo hacen lo mismo: pintar). Pero para poder distinguir unos de otros será necesario que siempre exista alguna diferencia entre ellos en las listas de parámetros (bien en el número de parámetros, bien en el tipo de los parámetros). Volviendo al ejemplo anterior, podríamos utilizar un mismo nombre, por ejemplo `pintar`, para todos los métodos anteriores:

- Método `pintar (int entero)`.
- Método `pintar (double real)`.
- Método `pintar (double String)`.
- Método `pintar (int entero, String cadena)`.

En este caso el compilador no va a generar ningún error pues se cumplen las normas ya que unos métodos son perfectamente distinguibles de otros (a pesar de tener el mismo nombre) gracias a que tienen listas de parámetros diferentes.

Lo que sí habría producido un error de compilación habría sido por ejemplo incluir otro método pintar (`int` entero), pues es imposible distinguirlo de otro método con el mismo nombre y con la misma lista de parámetros (ya existe un método `pintar` con un único parámetro de tipo `int`).

También debes tener en cuenta que el tipo devuelto por el método no es considerado a la hora de identificar un método, así que un tipo devuelto diferente no es suficiente para distinguir un método de otro. Es decir, no podrías definir dos métodos exactamente iguales en nombre y lista de parámetros e intentar distinguirlos indicando un tipo devuelto diferente. El compilador producirá un error de duplicidad en el nombre del método y no te lo permitirá.

Recuerda

Es conveniente no abusar de sobrecarga de métodos y utilizarla con cierta moderación (cuando realmente puede beneficiar su uso), dado que podría hacer el código menos legible.

Sobrecarga de operadores. [NUEVO]

Del mismo modo que hemos visto la posibilidad de sobrecargar métodos (disponer de varias versiones de un método con el mismo nombre cambiando su lista de parámetros), podría plantearse también la opción de sobrecargar operadores del lenguaje tales como `+`, `-`, `*`, `( )`, `<`, `>`, etc. para darles otro significado dependiendo del tipo de objetos con los que vaya a operar.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

En algunos casos puede resultar útil para ayudar a mejorar la legibilidad del código, pues esos operadores resultan muy intuitivos y pueden dar una idea rápida de cuál es su funcionamiento.

Un típico ejemplo podría ser el de la sobrecarga de operadores aritméticos como la suma (+) o el producto (*) para operar con fracciones. Si se definen objetos de una clase `Fracción` (que contendrá los atributos `numerador` y `denominador`) podrían sobrecargarse los operadores aritméticos (habría que redefinir el operador suma (`+`) para la suma, el operador asterisco (`*`) para el producto, etc.) para esta clase y así podrían utilizarse para sumar o multiplicar objetos de tipo `Fraccion` mediante el algoritmo específico de suma o de producto del objeto `Fraccion` (pues esos operadores no están preparados en el lenguaje para operar con esos objetos).

En algunos lenguajes de programación como por ejemplo C++ o C# se permite la sobrecarga, pero no es algo soportado en todos los lenguajes. ¿Qué sucede en el caso concreto de Java?

Importante

El lenguaje Java **NO** soporta la sobrecarga de operadores.

En el ejemplo anterior de los objetos de tipo Fracción, habrá que declarar métodos en la clase `Fraccion` que se encarguen de realizar esas operaciones, pero no lo podremos hacer sobrecargando los operadores del lenguaje (los símbolos de la suma, resta, producto, etc.). Por ejemplo:

```
public Fraccion sumar (Fraccion sumando)
public Fraccion multiplicar (Fraccion multiplicando)
```

Y así sucesivamente...

Dado que en este módulo se está utilizando el lenguaje Java para aprender a programar, no podremos hacer uso de esta funcionalidad. Más adelante, cuando aprendas a programar en otros lenguajes, es posible que sí tengas la posibilidad de utilizar este recurso.

La referencia `this`.

La palabra reservada `this` consiste en una referencia al objeto actual. El uso de este operador puede resultar muy útil a la hora de evitar la ambigüedad que puede producirse entre el nombre de un parámetro de un método y el nombre de un atributo cuando ambos tienen el mismo identificador (mismo nombre). En tales casos el parámetro "oculta" al atributo y no tendríamos acceso directo a él (al escribir el identificador estaríamos haciendo referencia al parámetro y no al atributo). En estos casos la referencia `this` nos permite acceder a estos atributos ocultados por los parámetros.

Dado que `this` es una referencia a la propia clase en la que te encuentras en ese momento, puedes acceder a sus atributos mediante el operador punto (`.`) como sucede con cualquier otra clase u objeto. Por tanto, en lugar deponer el nombre del atributo (que estos casos haría referencia al parámetro), podrías escribir `this.nombreAtributo`, de manera que el compilador sabrá que te estás refiriendo al atributo y se eliminará la ambigüedad.

En el ejemplo de la clase `Punto`, podríamos utilizar la referencia `this` si el nombre del parámetro del método coincidiera con el del atributo que se desea modificar. Por ejemplo:

```
class Punto{
    private int x,y;

    void establecerX (int nuevaX){
        int x = 1; //<<<--- metodo
        this.x = 1; //<<<--- clase
        this.x=x;
        this.x=nuevaX;
    }
}
```

En este caso ha sido indispensable el uso de `this`, pues si no sería imposible saber en qué casos te estás refiriendo al parámetro `x` y en cuáles al atributo `x`. Para el compilador el identificador `x` será siempre el parámetro, pues ha "ocultado" al atributo.

Más información

En algunos casos puede resultar útil hacer uso de la referencia `this` aunque no sea necesario, pues puede ayudara mejorar la legibilidad del código.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Métodos estáticos.

Como ya has visto en ocasiones anteriores, un método estático es un método que puede ser usado directamente desde la clase, sin necesidad de tener que crear una instancia para poder utilizar al método. También son conocidos como **métodos de clase** (como sucedía con los atributos de clase), frente a los métodos de objeto (es necesario un objeto para poder disponer de ellos).

Los métodos estáticos no pueden manipular atributos de instancias (objetos) sino atributos estáticos (de clase) y suelen ser utilizados para realizar operaciones comunes a todos los objetos de la clase, más que para una instancia concreta.

Algunos ejemplos de operaciones que suelen realizarse desde métodos estáticos:

- **Acceso a atributos específicos de clase**: incremento o decremento de contadores internos de la clase (`node instancias`), acceso a un posible atributo de nombre de la clase, etc.
- **Operaciones genéricas relacionadas con la clase pero que no utilizan atributos de instancia**. Por ejemplo una clase `NIF` (o `DNI`) que permite trabajar con el `DNI` y la letra del `NIF` y que proporciona funciones adicionales para calcular la letra `NIF` de un número de `DNI` que se le pase como parámetro. Ese método puede ser interesante para ser usado desde fuera de la clase de manera independiente a la existencia de objetos de tipo `NIF` . Consulta el Ejemplo03 para una implementación completa.

En la biblioteca de Java es muy habitual encontrarse con clases que proporcionan métodos estáticos que pueden resultar muy útiles para cálculos auxiliares, conversiones de tipos, etc. Por ejemplo, la mayoría de las clases del paquete `java.lang` que representan tipos (`Integer` , `String` , `Float` , `Double` , `Boolean` , etc.) ofrecen métodos estáticos para hacer conversiones. Aquí tienes algunos ejemplos:

- `java static String valueOf(int i)`

Devuelve la representación en formato `String` (cadena) de un valor `int` . Se trata de un método que no tiene que ver nada en absoluto con instancias de concretas de `String` , sino de un método auxiliar que puede servir como herramienta para ser usada desde otras clases. Se utilizaría directamente con el nombre de la clase. Por ejemplo:

- `java String enteroCadena = String.valueOf(23).`
- `java static String valueOf(float f)`

Algo similar para un valor de tipo `float` . Ejemplo de uso:

- `java String floatCadena = String.valueOf(24.341)`
- `java static int parseInt(String s)`

En este caso se trata de un método estático de la clase `Integer` . Analiza la cadena pasada como parámetro y la transforma en un `int` . Ejemplo de uso:

- `java int cadenaEntero=Integer.parseInt ("-12")`

Todos los ejemplos anteriores son casos en los que se utiliza directamente la clase como una especie de caja de herramientas que contiene métodos que pueden ser utilizados desde cualquier parte, por eso suelen ser métodos públicos.

6.1.5 Encapsulación, control de acceso y visibilidad.

Dentro de la Programación Orientada a Objetos ya has visto que es muy importante el concepto de ocultación, la cual ha sido lograda gracias a la encapsulación de la información dentro de las clases. De esta manera una clase puede ocultar parte de su contenido o restringir el acceso a él para evitar que sea manipulado de manera inadecuada. Los modificadores de acceso en Java permiten especificar el ámbito de visibilidad de los miembros de una clase, proporcionando así un mecanismo de accesibilidad a varios niveles.

Acabas de estudiar que cuando se definen los miembros de una clase (atributos o métodos), e incluso la propia clase, se indica (aunque sea por omisión) un modificador de acceso. En función de la visibilidad que se desee que tengan los objetos o los miembros de esos objetos se elegirá alguno de los modificadores de acceso que has estudiado. Ahora que ya sabes cómo escribir una clase completa (declaración de la clase, declaración de sus atributos y declaración de sus métodos), vamos a hacer un repaso general de las opciones de visibilidad (control de acceso) que has estudiado.

Los modificadores de acceso determinan si una clase puede utilizar determinados miembros (acceder a atributos o invocar miembros) de otra clase. Existen dos niveles de control de acceso:

1. A nivel general (**nivel de clase**): visibilidad de la propia clase.
2. A **nivel de miembros**: especificación, miembro por miembro, de su nivel de visibilidad.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

En el caso de la clase, ya estudiaste que los niveles de visibilidad podían ser:

- Público (modificador `public`), en cuyo caso la clase era visible a cualquier otra clase (cualquier otro fragmento de código del programa).
- Privada al paquete (`package`)(sin modificador o modificador "por omisión"). En este caso, la clase sólo será visible a las demás clases del mismo paquete, pero no al resto del código del programa (otros paquetes).
- Protegido (`protected`), lo podrán ver las clases del mismo paquete y también las clases herederas.

En el caso de los miembros, disponías de otras dos posibilidades más de niveles de accesibilidad, teniendo un total de cuatro opciones a la hora de definir el control de acceso al miembro:

- Público (modificador `public`), igual que en el caso global de la clase y con el mismo significado (miembro visible desde cualquier parte del código).
- Del paquete (sin modificador), también con el mismo significado que en el caso de la clase (miembro visible sólo desde clases del mismo paquete, ni siquiera será visible desde una subclase salvo si ésta está en el mismo paquete).
- Privado (modificador `private`), donde sólo la propia clase tiene acceso al miembro.
- Protegido (modificador `protected`), lo podrán ver las clases del mismo paquete y también las clases herederas.

Ocultación de atributos. Métodos de acceso. [NUEVO]

Los atributos de una clase suelen ser declarados como privados a la clase o, como mucho, `protected` (accesibles también por clases heredadas), pero no como `public`. De esta manera puedes evitar que sean manipulados inadecuadamente (por ejemplo modificarlos sin ningún tipo de control) desde el exterior del objeto.

En estos casos lo que se suele hacer es declarar esos atributos como privados o protegidos y crear métodos públicos que permitan acceder a esos atributos. Si se trata de un atributo cuyo contenido puede ser observado pero no modificado directamente, puede implementarse un método de "obtención" del atributo (en inglés se les suele llamar método de tipo `get`) y si el atributo puede ser modificado, puedes también implementar otro método para la modificación o "establecimiento" del valor del atributo (en inglés se le suele llamar método de tipo `set`). Esto ya lo has visto en apartados anteriores.

Si recuerdas la clase `Punto` que hemos utilizado como ejemplo, ya hiciste algo así con los métodos de obtención y establecimiento de las coordenadas:

```
private int x, y;

// Métodos get
public int obtenerX() {
    return x;
}
public int obtenerY() {
    return y;
}
// Métodos set
public void establecerX(int x) {
    this.x= x;
}
public void establecerY(int y) {
    this.y= y;
}
```

Así, para poder obtener el valor del atributo `x` de un objeto de tipo `Punto` será necesario utilizar el `métodoobtenerX()` y no se podrá acceder directamente al atributo `x` del objeto. En algunos casos los programadores directamente utilizan nombres en inglés para nombrar a estos métodos:

```
getX(), getY(), setX(), setY(), getNombre, setNombre, getColor, etc.
```

También pueden darse casos en los que no interesa que pueda observarse directamente el valor de un atributo, sino un determinado procesamiento o cálculo que se haga con el atributo (pero no el valor original). Por ejemplo podrías tener un atributo `DNI` que almacene los 8 dígitos del `DNI` pero no la letra del `NIF` (pues se puede calcular a partir de los dígitos). El método de acceso para el `DNI` (método `getDNI`) podría proporcionar el `DNI` completo (es decir, el `NIF`, incluyendo la letra), mientras que la letra no es almacenada realmente en el atributo del objeto. Algo similar podría suceder con el dígito de control de una cuenta bancaria, que puede no ser almacenado en el objeto, pero sí calculado y devuelto cuando se nos pide el número de cuenta completo.

En otros casos puede interesar disponer de métodos de modificación de un atributo pero a través de un determinado procesamiento previo para por ejemplo poder controlar errores o valores inadecuados. Volviendo al ejemplo del `NIF`, un método para modificar un `DNI` (método `setDNI`) podría incluir la letra (`NIF` completo), de manera que así podría comprobarse si el número de `DNI` y la letra coinciden (es un `NIF` válido). En tal caso se almacenará el `DNI` y en caso contrario se producirá un

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

error de validación (por ejemplo lanzando una excepción). En cualquier caso, el `DNI` que se almacenara sería solamente el número y no la letra (pues la letra es calculable a partir del número de `DNI`).

Ocultación de métodos. [NUEVO]

Normalmente los métodos de una clase pertenecen a su interfaz y por tanto parece lógico que sean declarados como públicos. Pero también es cierto que pueden darse casos en los que exista la necesidad de disponer de algunos métodos privados a la clase. Se trata de métodos que realizan operaciones intermedias o auxiliares y que son utilizados por los métodos que sí forman parte de la interfaz. Ese tipo de métodos (de comprobación, de adaptación de formatos, de cálculos intermedios, etc.) suelen declararse como privados pues no son de interés (o no es apropiado que sean visibles) fuera del contexto del interior del objeto.

En el ejemplo anterior de objetos que contienen un `DNI`, será necesario calcular la letra correspondiente a un determinado número de `DNI` o comprobar si una determinada combinación de número y letra forman un `DNI` válido. Este tipo de cálculos y comprobaciones podrían ser implementados en métodos privados de la clase (o al menos como métodos protegidos).

```
public static boolean validarDNI(String dni){
    [...]
    char letra = LetraDNI(Long numeroDNI);
    [...]
}

??? static char LetraDNI (Long numero) {
    [...]
}
```

6.1.6 Utilización de los métodos y atributos de una clase

Una vez implementada una clase, se pueden crear objetos e interactuar con ellos. El ciclo de vida completo (declaración, instanciación, manipulación) se explicó en detalle en UD02.

> **Repaso**
>
> Consulta los siguientes apartados de la Unidad 2 según necesites: - Declaración de objetos - Instanciación (operador new) - Manipulación de objetos - Destrucción y recolector de basura

6.1.7 Constructores

Un **constructor** es un método especial con el mismo nombre de la clase que se ejecuta al crear un objeto. No devuelve ningún valor. Los constructores soportan sobrecarga.

> **Repaso**
>
> El concepto básico de constructor se explicó en Constructores en UD02.

Creación de constructores

Cuando se escribe una clase, se pueden definir uno o más constructores. En la definición se indica:

- El tipo de acceso (`public`, `private`, etc.).
- El nombre de la clase (obligatorio).
- La lista de parámetros.
- El cuerpo del constructor.

```
public Punto(int x, int y) {
    this.x = x;
    this.y = y;
}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

> **Recuerda**
>
> Si defines constructores personalizados, el constructor por defecto (sin parámetros) deja de ser generado por el compilador. Debes crearlo tú si lo necesitas.

Los constructores soportan la sobrecarga, igual que los métodos. Consulta los Ejemplo01 y Ejemplo02 para ver implementaciones completas:

```
public Rectangulo() { ... }
public Rectangulo(double x1, double y1, double x2, double y2) { ... }
public Rectangulo(double base, double altura) { ... }
```

Consulta el Ejemplo02 para ver una implementación con constructores sobrecargados y constructor de copia.

Utilización de constructores

```
Punto p1 = new Punto(10, 7);
Punto p2 = new Punto(); // constructor por defecto (si existe)
```

Creación de constructores.

Cuando se escribe el código de una clase normalmente se pretende que los objetos de esa clase se creen de una determinada manera. Para ello se definen uno o más constructores en la clase. En la definición de un constructor se indican:

- El tipo de acceso.
- El nombre de la clase (el nombre de un método constructor es siempre el nombre de la propia clase).
- La lista de parámetros que puede aceptar.
- Si lanza o no excepciones.
- El cuerpo del constructor (un bloque de código como el de cualquier método).

Como puedes observar, la estructura de los constructores es similar a la de cualquier método, con las excepciones de que no tiene tipo de dato devuelto (no devuelve ningún valor) y que el nombre del método constructor debe ser obligatoriamente el nombre de la clase.

> **Recuerda**
>
> Si defines constructores personalizados para una clase, el constructor por defecto (sin parámetros) para esa clase deja de ser generado por el compilador, de manera que tendrás que crearlo tú si quieres poder utilizarlo.
>
> Si se ha creado un constructor con parámetros y no se ha implementado el constructor por defecto, el intento de utilización del constructor por defecto producirá un error de compilación (el compilador no lo hará por nosotros).

Un ejemplo de constructor para la clase Punto podría ser:

```
public Punto(int x, int y) {
    this.x= x;
    this.y= y;
    cantidadPuntos++; // Suponiendo que tengamos un atributo estático cantidadPuntos
}
```

En este caso el constructor recibe dos parámetros. Además de reservar espacio para los atributos (de lo cual se encarga automáticamente Java), también asigna sendos valores iniciales a los atributos `x` e y. Por último incrementa un atributo (probablemente estático) llamado `cantidadPuntos`.

Utilización de constructores.

Una vez que dispongas de tus propios constructores personalizados, la forma de utilizarlos es igual que con el constructor por defecto (mediante la utilización de la palabra reservada `new`) pero teniendo en cuenta que si has declarado parámetros en tu método constructor, tendrás que llamar al constructor con algún valor para esos parámetros. Un ejemplo de utilización del constructor que has creado para la clase `Punto` en el apartado anterior podría ser:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
Punto p1;
p1= new Punto(10, 7);
```

En este caso no se estaría utilizando el constructor por defecto sino el constructor que acabas de implementar en el cual además de reservar memoria se asigna un valor a algunos de los atributos.

Constructores de copia. [NUEVO]

Una forma de iniciar un objeto podría ser mediante la copia de los valores de los atributos de otro objeto ya existente. Imagina que necesitas varios objetos iguales (con los mismos valores en sus atributos) y que ya tienes uno de ellos perfectamente configurado (sus atributos contienen los valores que tú necesitas). Estaría bien disponer de un constructor que hiciera copias idénticas de ese objeto.

Durante el proceso de creación de un objeto puedes generar objetos exactamente iguales(basados en la misma clase) que se distinguirán posteriormente porque podrán tener estados distintos (valores diferentes en los atributos). La idea es poder decirle a la clase que además de generar un objeto nuevo, que lo haga con los mismos valores que tenga otro objeto ya existente. Es decir, algo así como si pudieras clonar el objeto tantas veces como te haga falta. A este tipo de mecanismo se le suele llamar constructor copia o constructor de copia.

Un constructor copia es un método constructor como los que ya has utilizado pero con la particularidad de que recibe como parámetro una referencia al objeto cuyo contenido se desea copiar. Este método revisa cada uno de los atributos del objeto recibido como parámetro y se copian todos sus valores en los atributos del objeto que se está creando en ese momento en el método constructor.

Un ejemplo de constructor copia para la clase Punto podría ser:

```
public Punto(Punto p){
    this.x = p.obtenerX();
    this.y = p.obtenerY();
}
```

En este caso el constructor recibe como parámetro un objeto del mismo tipo que el que va a ser creado (clase `Punto`), inspecciona el valor de sus atributos (atributos `x` e `y`), y los reproduce en los atributos del objeto en proceso de construcción (`this`).

Un ejemplo de utilización de ese constructor podría ser:

```
Punto p1, p2;
p1 = new Punto (10, 7);
p2 = new Punto (p1);
```

En este caso el objeto `p2` se crea a partir de los valores del objeto `p1`.

Destrucción de objetos

Java gestiona la destrucción de objetos automáticamente mediante el **recolector de basura** (garbage collector). Los objetos que ya no tienen referencias son marcados para su eliminación.

Es posible implementar el método `finalize()` para liberar recursos antes de la destrucción:

```
protected void finalize() { ... }
```

> **Repaso**
>
> El ciclo de vida y la destrucción de objetos se explican en Destrucción en UD02.

6.1.8 Clases Anidadas, Clases Internas (*Inner Class*) [NUEVO]

Clase Anidada (Nested Class):

Una clase anidada es una clase estática definida dentro de otra clase. Se declara con el modificador `static`.

Asociación:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- No tiene una relación directa con las instancias de la clase externa.
- Se puede acceder a ella sin necesidad de crear una instancia de la clase externa.

Contexto:

- Solo puede acceder a los miembros `static` de la clase externa.
- No puede acceder a miembros de instancia (no estáticos) de la clase externa.

Uso común:

- Se utiliza cuando la clase anidada tiene un propósito independiente de las instancias de la clase externa.

Ejemplo:

```
public class Empresa {
[...]
    static class PlantillaCompletaException extends Exception {
        public PlantillaCompletaException() {
            super("La empresa tiene la plantilla completa.");
        }
    }
}
```

Así después podemos usarla en el `catch` accediendo directamente a la clase `Empresa` (ya que es `static`):

```
public class TestEmpresa {
    public static void main(String[] args) {
        [...]
        try {
            [...]
        } catch (Empresa.PlantillaCompletaException ex) {
            System.err.println(ex.getMessage());
        }
        [...]
    }
}
```

Clase Interna (Inner Class):

Una clase interna es una clase no estática definida dentro de otra clase. No se declara con `static`.

Asociación:

- Está directamente asociada con una instancia de la clase externa.
- Para instanciarla, primero necesitas una instancia de la clase externa.

Contexto:

- Puede acceder a **todos** los miembros (estáticos y no estáticos) de la clase externa, incluso si son privados.
- La clase interna tiene una referencia implícita a la instancia de la clase externa.

Uso común:

- Se utiliza cuando la clase interna necesita interactuar con los miembros de instancia de la clase externa.

Puedes ver la implementación completa con una clase `main` de prueba en el Ejemplo08.

Comparación Resumida:

Característica	Clase Anidada (Nested)	Clase Interna (Inner)
Es estática	Sí (`static`)	No (`non-static`)
Acceso a clase externa	Solo miembros `static`	Miembros estáticos y no estáticos
Asociación con clase externa	No está asociada a instancias	Está asociada a una instancia de la clase externa
Uso común	Lógica independiente de instancias de la clase externa	Lógica que necesita interactuar con la instancia externa
Instanciación		

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Característica	Clase Anidada (Nested)	Clase Interna (Inner)
	`OuterClass.NestedClass nested = new OuterClass.NestedClass();`	`OuterClass.InnerClass inner = outer.new InnerClass();`

Convenciones comunes sobre el número de clases en un archivo:

1. **Una clase pública por archivo:** Es una convención estándar en Java que cada archivo .java debe contener solo una clase pública, y el nombre del archivo debe coincidir exactamente con el nombre de la clase pública (incluyendo mayúsculas y minúsculas). Esto es obligatorio para que el compilador de Java pueda encontrar las clases correctamente.

   Ejemplo:

   ```java
   // Archivo: MiClase.java
   public class MiClase {
       // Código de la clase
   }
   ```

   2. **Clases no públicas en el mismo archivo:**

   Puedes incluir múltiples clases no públicas (con modificador default o package-private) en el mismo archivo, siempre que no haya más de una clase pública.

   Ejemplo:

   ```java
   // Archivo: ClasePrincipal.java
   public class ClasePrincipal {
       // Código de la clase pública
   }

   class ClaseAuxiliar {
       // Código de la clase auxiliar
   }
   ```

   Sin embargo, esto no es una práctica común en proyectos grandes, ya que puede dificultar el mantenimiento y la localización de clases.

2. **Separar clases auxiliares en archivos propios:**

Aunque no es obligatorio, es una buena práctica colocar cada clase en su propio archivo, incluso si no es pública. Esto facilita:

- La localización y lectura de la clase.
- El control de versiones y la fusión de cambios en sistemas de control de código fuente.
- La prueba unitaria y el mantenimiento.

1. **Uso de clases anidadas:**

Si una clase es relevante solo dentro del contexto de otra clase, considera usar una clase anidada o una clase interna en lugar de una clase separada.

Ejemplo:

````
```java
public class ClaseExterna {
    private static class ClaseAnidada {
        // Clase anidada
    }

    private class InnerClass {
        // Clase interna
    }
}
```
````

1. **Evitar clases no relacionadas en un solo archivo:**

Colocar múltiples clases no relacionadas en un solo archivo se considera una mala práctica, ya que:

- Puede hacer que el archivo sea difícil de leer.
- Introduce acoplamiento innecesario entre clases.
- Viola el principio de responsabilidad única (SRP).

1. **Clases utilitarias:**

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Para clases con métodos estáticos (por ejemplo, `Utils` o `Constants`), es común colocarlas en un único archivo, ya que suelen ser pequeñas y no tienen lógica compleja.

6.1.9 Introducción a la herencia. [NUEVO]

La herencia es uno de los conceptos fundamentales que introduce la programación orientada a objetos. La idea fundamental es permitir crear nuevas clases aprovechando las características (atributos y métodos) de otras clases ya creadas evitando así tener que volver a definir esas características (reutilización).

A una clase que hereda de otra se le llama subclase o clase hija y aquella de la que se hereda es conocida como superclase o clase padre. También se puede hablar en general de clases descendientes o clases ascendientes. Al heredar, la subclase adquiere todas las características (atributos y métodos) de su superclase, aunque algunas de ellas pueden ser sobrescritas o modificadas dentro de la subclase (a eso se le suele llamar **especialización**).

Una clase puede heredar de otra que a su vez ha podido heredar de una tercera y así sucesivamente. Esto significa que las clases van tomando todas las características de sus clases ascendientes (no sólo de su superclase o clase padre inmediata) a lo largo de toda la rama del árbol de la jerarquía de clases en la que se encuentre.

Imagina que quieres modelar el funcionamiento de algunos vehículos para trabajar con ellos en un programa de simulación. Lo primero que haces es pensar en una clase `Vehículo` que tendrá un conjunto de atributos (por ejemplo: posición actual, velocidad actual y velocidad máxima que puede alcanzar el vehículo) y de métodos (por ejemplo: detener, acelerar, frenar, establecer dirección, establecer sentido).

Dado que vas a trabajar con muchos tipos de vehículos, no tendrás suficiente con esas características, así que seguramente vas a necesitar nuevas clases que las incorporen. Pero las características básicas que has definido en la clase Vehículo van a ser compartidas por cualquier nuevo vehículo que vayas a modelar. Esto significa que si creas otra clase podrías heredar de `Vehículo` todas esos atributos y propiedades y tan solo tendrías que añadir las nuevas.

Si vas a trabajar con vehículos que se desplazan por tierra, agua y aire, tendrás que idear nuevas clases con características adicionales. Por ejemplo, podrías crear una clase `VehiculoTerrestre`, que herede las características de `Vehículo`, pero que también incorpore atributos como el número de ruedas o la altura de los bajos). A su vez, podría idearse una nueva clase que herede de `VehiculoTerrestre` y que incorpore nuevos atributos y métodos como, por ejemplo, una clase `Coche`. Y así sucesivamente con toda la jerarquía de clases heredadas que consideres oportunas para representar lo mejor posible el entorno y la información sobre la que van a trabajar tus programas.

Creación y utilización de clases heredadas.

¿Cómo se indica en Java que una clase hereda de otra? Para indicar que una clase hereda de otra es necesario utilizar la palabra reservada `extends` junto con el nombre de la clase de la que se quieren heredar sus características:

```
class<NombreClase> extends <nombreSuperClase> {
    ...
}
```

En el ejemplo anterior de los vehículos, la clase `VehiculoTerrestre` podría quedar así al ser declarada:

```
class VehiculoTerrestre extends Vehiculo {
    ...
}
```

Y en el caso de la clase `Coche`:

```
class Coche extends VehiculoTerrestre {
    ...
}
```

En unidades posteriores estudiarás detalladamente cómo crear una jerarquía de clases y qué relación existe entre la herencia y los distintos modificadores de clases, atributos y métodos. Por ahora es suficiente con que entiendas el concepto de herencia y sepas reconocer cuándo una clase hereda de otra (uso de la palabra reservada `extends`).

Puedes comprobar que en las bibliotecas proporcionadas por Java aparecen jerarquías bastante complejas de clases heredadas en las cuales se han ido aprovechando cada uno de los miembros de una clase base para ir construyendo las distintas clases derivadas añadiendo (y a veces modificando) poco a poco nueva funcionalidad.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Eso suele suceder en cualquier proyecto de software conforme se van a analizando, descomponiendo y modelando los datos con los que hay que trabajar. La idea es poder representar de una manera eficiente toda la información que es manipulada por el sistema que se desea automatizar. Una jerarquía de clases suele ser una buena forma de hacerlo.

En el caso de Java, cualquier clase con la que trabajes tendrá un ascendiente. Si en la declaración de clase no indicas la clase de la que se hereda (no se incluye un `extends`), el compilador considerará automáticamente que se hereda de la clase `Object`, que es la clase que se encuentra en el nivel superior de toda la jerarquía de clases en Java (y que es la única que no hereda de nadie).

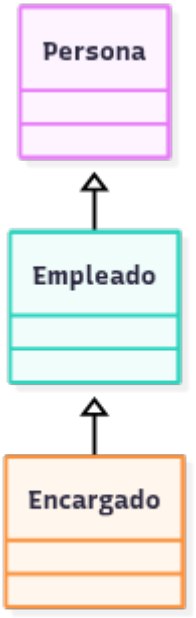
También irás viendo al estudiar distintos componentes de las bibliotecas de Java (por ejemplo en el caso de las interfaces gráficas) que para poder crear objetos basados en las clases proporcionadas por esas bibliotecas tendrás que crear tus propias clases que hereden de algunas de esas clases. Para ellos tendrás que hacer uso de la palabra reservada `extends`.

> **Recuerda**
>
> En Java todas las clases son descendientes (de manera explícita o implícita) de la clase `Object`.

6.1.10 Conversión entre objetos (Casting) [NUEVO]

La esencia de Casting permite convertir un dato de tipo primitivo en otro generalmente de más precisión.

Entre objetos es posible realizar el casting. Tenemos una clase persona con una subclase empleado y este a su vez una subclase encargado.



Si creamos una instancia de tipo persona y le asignamos un objeto de tipo empleado o encargado, al ser una subclase no existe ningún tipo de problema, ya que todo encargado o empleado es persona.

Por otro lado, si intentamos asignar valores a los atributos específicos de empleado o encargado nos encontramos con una pérdida de precisión puesto que no se pueden ejecutar todos los métodos de los que dispone un objeto de tipo empleado o encargado, ya que persona contiene menos métodos que la clase empleado o encargado. En este caso es necesario hacer un casting, sino el compilador dará error.

Ejemplo (Casting):

Si quieres ver la definición completa de la clase `Persona` dentro de su propio archivo, consulta el Ejemplo05.

La clase `Empleado` completa se muestra en el Ejemplo06.

Puedes ver la clase `Encargado` completa en el Ejemplo07.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD05;

public class Casting {

    public static void main(String[] args) {
        // Casting Implicito
        Persona encargadoCarniceria = new Encargado("Rosa Ramos", 1200, "Carniceria");

        // No tenemos disponibles los métodos de la clase Encargado:
        //EncargadaCarniceria.setSueldoBase(1200);
        //EncargadaCarniceria.setSeccion("Carniceria");
        //Pero al imprimir se imprime con el método más específico (luego lo vemos)
        System.out.println(encargadoCarniceria);

        // Casting Explicito
        Encargado miEncargado = (Encargado) encargadoCarniceria;
        //Tenemos disponibles los métodos de la clase Encargado:
        miEncargado.setSueldoBase(1200);
        miEncargado.setSeccion("Carniceria");
        //Al imprimir se imprime con el método más específico de nuevo.
        System.out.println(miEncargado);
    }
}
```

Las reglas a la hora de realizar casting es que:

- cuando se utiliza una clase más específicas (más abajo en la jerarquía) no hace falta casting. Es lo que llamamos **casting implícito**.
- cuando se utiliza una clase menos específica (más arriba en la jerarquía) hay que hacer un **casting explícito**.

¿Por qué se imprime con el método más especializado?

Debes entender que en realidad `encargadoCarniceria` es un `Encargado` que se *disfraza* de `Persona`, pero en realidad sus métodos son los especializados (el `toString()` más moderno sobrescribe al de sus padres. Recuerda que la anotación `@override` es opcional, y aunque no se indique el método sigue sobrescribiendo al de su padre)

Si por ejemplo usamos este fragmento:

```java
//Persona
Persona David = new Persona ("David");
System.out.println(David);
```

Se imprimirá con el método `toString()` de la clase `Persona` (sólo el nombre).

Y si hacemos un casting del objeto David a uno más genérico (Object) seguirá usando el método más especializado:

```java
//Object
Object oDavid = David;
System.out.println(oDavid);
```

6.1.11 Acceso a métodos de la superclase [NUEVO]

Para acceder a los métodos de la superclase se utiliza la sentencia `super`. La sentencia `this` permite acceder a los campos y métodos de la clase. La sentencia `super` permite acceder a los campos y métodos de la superclase. El uso de `super` lo hemos visto en las clases `Empleado` y `Encargado` anteriores:

```java
public class Persona {
    private String nombre;

    public Persona(String nombre){
        this.nombre=nombre;
    }
}
```

```java
public class Empleado extends Persona {
[...]
    public Empleado(String nombre, double sueldoBase) {
        super(nombre);
        this.sueldoBase = sueldoBase;
    }
[...]
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
public class Encargado extends Empleado {
[...]
    public Encargado(String nombre, double sueldoBase, String seccion) {
        super(nombre, sueldoBase);
        this.seccion = seccion;
    }
[...]
```

Podemos mostrar el nombre de la clase y el nombre de la clase de la que hereda con `getClass()` y `getSuperclass()`. Ejemplo:

```java
package UD05;

public class Anexo4SuperClase {

    public static void main(String[] args) {
        Empleado empleadoCarniceria = new Empleado("Rosa Ramos", 1200);
        // Muestra los datos del Empleado
        System.out.println(empleadoCarniceria instanceof Encargado); //false
        System.out.println(empleadoCarniceria.getClass()); //class Empleado
        System.out.println(empleadoCarniceria.getClass().getSuperclass()); //class Persona
    }
}
```

6.1.12 Empaquetado de clases [NUEVO]

Un **paquete** es un conjunto de clases relacionadas agrupadas bajo un mismo nombre, formando un nivel superior de encapsulación.

> **Repaso**
>
> Las sentencias `package` e `import` se introdujeron en Librerías de Objetos en UD02. Aquí se amplía la organización jerárquica de paquetes.

Jerarquía de paquetes.

Los paquetes en Java pueden organizarse jerárquicamente de manera similar a lo que puedes encontrar en la estructura de carpetas en un dispositivo de almacenamiento, donde:

- Las clases serían como los archivos.
- Cada paquete sería como una carpeta que contiene archivos (clases).
- Cada paquete puede además contener otros paquetes (como las carpetas que contienen carpetas).
- Para poder hacer referencia a una clase dentro de una estructura de paquetes, habrá que indicar la trayectoria completa desde el paquete raíz de la jerarquía hasta el paquete en el que se encuentra la clase, indicando por último el nombre de la clase (como el path absoluto de un archivo).

La estructura de paquetes en Java permite organizar y clasificar las clases, evitando conflictos de nombres y facilitando la ubicación de una clase dentro de una estructura jerárquica.

Por otro lado, la organización en paquetes permite también el control de acceso a miembros de las clases desde otras clases que estén en el mismo paquete gracias a los modificadores de acceso (recuerda que uno de los modificadores que viste era precisamente el de paquete).

Las clases que forman parte de la jerarquía de clases de Java se encuentran organizadas en diversos paquetes.

Todas las clases proporcionadas por Java en sus bibliotecas son miembros de distintos paquetes y se encuentran organizadas jerárquicamente. Dentro de cada paquete habrá un conjunto de clases con algún tipo de relación entre ellas. Se dice que todo ese conjunto de paquetes forman la API de Java. Por ejemplo las clases básicas del lenguaje se encuentran en el paquete `java.lang`, las clases de entrada/salida las podrás encontrar en el paquete `java.io` y en el paquete `java.math` podrás observar algunas clases para trabajar con números grandes y de gran precisión.

Utilización de los paquetes

Se accede a una clase mediante su nombre completo (ruta del paquete + nombre de clase) o mediante `import`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Repaso

El uso de la sentencia `import` se explica en Sentencia import en UD02.

Inclusión de una clase en un paquete.

Al principio de cada archivo `.java` se puede indicar a qué paquete pertenece mediante la palabra reservada `package` seguida del nombre del paquete:

```
package nombre_paquete;
```

Por ejemplo:

```
package paqueteEjemplo;
class claseEjemplo {
    ...
}
```

La sentencia `package` debe ser incluida en cada archivo fuente de cada clase que quieras incluir ese paquete. Si en un archivo fuente hay definidas más de una clase, todas esas clases formarán parte del paquete indicado en la sentencia `package`.

Si al comienzo de un archivo Java no se incluyen ninguna sentencia `package`, el compilador considerará que las clases de ese archivo formarán parte del paquete por omisión (un paquete sin nombre asociado al proyecto).

Para evitar la ambigüedad, dentro de un mismo paquete no puede haber dos clases con el mismo nombre, aunque sí pueden existir clases con el mismo nombre si están en paquetes diferentes. El compilador será capaz de distinguir una clase de otra gracias a que pertenecen a paquetes distintos.

Como ya has visto en unidades anteriores, el nombre de un archivo fuente en Java se construye utilizando el nombre de la clase pública que contiene junto con la extensión `.java`, pudiendo haber únicamente una clase pública por cada archivo fuente. El nombre de la clase debía coincidir (en mayúsculas y minúsculas) exactamente con el nombre del archivo en el que se encontraba definida.

Así, si por ejemplo tenías una clase `Punto` dentro de un archivo `Punto.java`, la compilación daría lugar a un archivo `Punto.class`.

En el caso de los paquetes, la correspondencia es a nivel de directorios o carpetas. Es decir, si la clase `Punto` se encuentra dentro del paquete `prog.figuras`, el archivo `Punto.java` debería encontrarse en la carpeta `prog\figuras`. Para que esto funcione correctamente el compilador ha de ser capaz de localizar todos los paquetes (tanto los estándar de Java como los definidos por otros programadores). Es decir, que el compilador debe tener conocimiento de dónde comienza la estructura de carpetas definida por los paquetes y en la cual se encuentran las clases. Para ello se utiliza el **ClassPath** cuyo funcionamiento habrás estudiado en las primeras unidades de este módulo. Se trata de una variable de entorno que contiene todas las rutas en las que comienzan las estructuras de directorios (distintas jerarquías posibles de paquetes) en las que están contenidas las clases.

Proceso de creación de un paquete.

Para crear un paquete en Java te recomendamos seguir los siguientes pasos:

1. **Poner un nombre al paquete**. Suele ser habitual utilizar el dominio de Internet de la empresa que ha creado el paquete. Por ejemplo, para el caso de `miempresa.com`, podría utilizarse un nombre de paquete `com.miempresa`.
2. **Crear una estructura jerárquica de carpetas equivalente a la estructura de subpaquetes**. La ruta de la raíz de esa estructura jerárquica deberá estar especificada en el **ClassPath** de Java.
3. **Especificar a qué paquete pertenecen la clase (o clases) del archivo** `.java` mediante el uso de la sentencia package tal y como has visto en el apartado anterior.

Este proceso ya lo has debido de llevar a cabo en unidades anteriores al compilar y ejecutar clases con paquetes. Estos pasos simplemente son para que te sirvan como recordatorio del procedimiento que debes seguir a ala hora de clasificar, jerarquizar y utilizar tus propias clases.

6.1.13 Ejercicios resueltos

Modificadores de acceso

Imagina que quieres escribir una clase que represente un rectángulo en el plano. Para ello has pensado en los siguientes atributos:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- Atributos `x1`, `y1`, que representan la coordenadas del vértice inferior izquierdo del rectángulo. Ambos de tipo `double` (números reales).
- Atributos `x2`, `y2`, que representan las coordenadas del vértice superior derecho del rectángulo. También de tipo `double` (números reales).

Con estos dos puntos (`x1`, `y1`) y (`x2`, `y2`) se puede definir perfectamente la ubicación de un rectángulo en el plano.

Escribe una clase que contenga todos esos atributos teniendo en cuenta que queremos que sea una clase visible desde cualquier parte del programa y que sus atributos sean también accesibles desde cualquier parte del código.

Respuesta: Dado que se trata de una clase que podrá usarse desde cualquier parte del programa, utilizaremos el modificador de acceso `public` para la clase:

```
public class Rectangulo
```

Los cuatro atributos que necesitamos también han de ser visibles desde cualquier parte, así que también se utilizará el modificador de acceso `public` para los atributos:

```
public double x1, y1; // Vértice inferior izquierdo
public double x2, y2; // Vértice superior derecho
```

De esta manera la clase completa quedaría:

```
public class Rectangulo {
    public double x1, y1; // Vértice inferior izquierdo
    public double x2, y2; // Vértice superior derecho
}
```

Atributos estáticos

Ampliar el ejercicio anterior del rectángulo incluyendo los siguientes atributos:

- Atributo `numRectangulos`, que almacena el número de objetos de tipo rectángulo creados hasta el momento.
- Atributo `nombre`, que almacena el nombre que se le quiera dar a cada rectángulo.
- Atributo `nombreFigura`, que almacena el nombre de la clase, es decir, "Rectángulo".
- Atributo `PI`, que contiene el nombre de la constante `PI` con una precisión de cuatro cifras decimales.

No se desea que los atributos `nombre` y `numRectangulos` puedan ser visibles desde fuera de la clase. Y además se desea que la clase sea accesible solamente desde su propio paquete.

Respuesta: Los atributos `numRectangulos`, `nombreFigura` y `PI` podrían ser estáticos pues se trata de valores más asociados a la propia clase que a cada uno de los objetos que se puedan ir creando. Además, en el caso de `PI` y `nombreFigura`, también podría ser un atributo `final`, pues se trata de valores únicos y constantes (3.1416 en el caso de `PI` y "Rectángulo" en el caso de `nombreFigura`).

Dado que no se desea que se tenga accesibilidad a los atributos nombre y `numRectangulos` desde fuera de la clase podría utilizarse el atributo `private` para cada uno de ellos.

Por último hay que tener en cuenta que se desea que la clase sólo sea accesible desde el interior del paquete al que pertenece, por tanto habrá que utilizar el modificador por omisión o de paquete. Esto es, no incluir ningún modificador de acceso en la cabecera de la clase.

Teniendo en cuenta todo lo anterior la clase podría quedar finalmente así:

```
class Rectangulo { // Sin modificador "public" para que sólo sea accesible desde el paquete
    // Atributos de clase
    private static int numRectangulos; // Número total de rectángulos creados
    public static final String NOMBREFIGURA = "Rectángulo"; // Nombre de la clase
    public static final double PI = 3.1416; // Constante PI

    // Atributos de objeto
    private String nombre; // Nombre del rectángulo
    public double x1, y1; // Vértice inferior izquierdo
    public double x2, y2; // Vértice superior derecho
}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Cuerpo de un método

Vamos a seguir ampliando la clase en la que se representa un rectángulo en el plano (clase `Rectangulo`). Para ello has pensado en los siguientes métodos públicos:

- Métodos `getNombre` y `setNombre`, que permiten el acceso y modificación del atributo nombre del rectángulo.
- Método `calcularSuperfice`, que calcula el área encerrada por el rectángulo.
- Método `calcularPerímetro`, que calcula la longitud del perímetro del rectángulo.
- Método `desplazar`, que mueve la ubicación del rectángulo en el plano en una cantidad `X` (para el `ejeX`) y otra cantidad `Y` (para el eje `Y`). Se trata simplemente de sumar el desplazamiento `X` a las coordenadas `x1` y `x2`, y el desplazamiento `Y` a las coordenadas `y1` e `y2`. Los parámetros de entrada de este método serán por tanto `X` e `Y`, de tipo `double`.
- Método `obtenerNumRectangulos`, que devuelve el número de rectángulos creados hasta el momento.

Incluye la implementación de cada uno de esos métodos en la clase `Rectangulo`.

Respuesta: En el caso del método `obtenerNombre()`, se trata simplemente de devolver el valor del atributo nombre:

```
public String obtenerNombre () {
    return nombre;
}
```

Para el implementar el método `establecerNombre` también es muy sencillo. Se trata de modificar el contenido del atributo nombre por el valor proporcionado a través de un parámetro de entrada:

```
public void establecerNombre (String nom) {
    nombre = nom;
}
```

Los métodos de cálculo de `superficie` y `perímetro` no van a recibir ningún parámetro de entrada, tan solo deben realizar cálculos a partir de los atributos contenidos en el objeto para obtener los resultados perseguidos. Encada caso habrá que aplicar la expresión matemática apropiada:

- En el caso de la `superficie`, habrá que calcular la longitud de la base y la altura del rectángulo a partir de las coordenadas de las esquinas inferior izquierda (`x1`, `y1`) y superior derecha (`x2`, `y2`) de la figura. La base sería la diferencia entre `x2` y `x1`, y la altura la diferencia entre `y2` e `y1`. A continuación tan solo tendrías que utilizar la consabida fórmula de "base por altura", es decir, una multiplicación.
- En el caso del `perímetro` habrá también que calcular la longitud de la base y de la altura del rectángulo y a continuación sumar dos veces la longitud de la base y dos veces la longitud de la altura.

En ambos casos el resultado final tendrá que ser devuelto a través de la sentencia `return`. También es aconsejable en ambos casos la utilización de variables locales para almacenar los cálculos intermedios (como la base o la altura).

```
public double calcularSuperficie () {
    double area, base, altura; // Variables locales
    // Cálculo de la base
    base = x2-x1;
    // Cálculo de la altura
    altura = y2-y1;
    // Cálculo del área
    area = base * altura;
    // Devolución del valor de retorno
    return area;
}

public double calcularPerimetro () {
    double perimetro, base, altura; // Variables locales
    // Cálculo de la base
    base = x2-x1;
    // Cálculo de la altura
    altura = y2-y1;
    // Cálculo del perímetro
    perimetro = 2*base + 2*altura;
    // Devolución del valor de retorno
    return perimetro;
}
```

En el caso del método `desplazar()`, se trata de modificar:

- Los contenidos de los atributos `x1` y `x2` sumándoles el parámetro `X`,
- Los contenidos de los atributos `y1` e `y2` sumándoles el parámetro `Y`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
public void desplazar (double X, double Y) {
    // Desplazamiento en el eje X
    x1 = x1 + X;
    x2 = x2 + X;
    // Desplazamiento en el eje X
    y1 = y1 + Y;
    y2 = y2 + Y;
}
```

En este caso no se devuelve ningún valor (tipo devuelto vacío: `void`).

Por último, el método `obtenerNumRectangulos` simplemente debe devolver el valor del atributo `numRectangulos`. En este caso es razonable plantearse que este método podría ser más bien un método de clase (estático) más que un método de objeto, pues en realidad es una característica de la clase más que algún objeto en particular. Para ello tan solo tendrías que utilizar el modificador de acceso `static`:

```
public static int obtenerNumRectangulos () {
    return numRectangulos;
}
```

Declaración de un objeto

Utilizando la clase `Rectangulo` implementada en ejercicios anteriores, indica como declararías tres objetos (variables) de esa clase llamados `r1`, `r2`, `r3`.

Respuesta Se trata simplemente de realizar una declaración de esas tres variables:

```
Rectangulo r1;
Rectangulo r2:
Rectangulo r3:
```

También podrías haber declarado los tres objetos en la misma sentencia de declaración:

```
Rectangulo r1, r2, r3;
```

Creación de un objeto

Ampliar el ejercicio anterior instanciando los objetos `r1`, `r2`, `r3` mediante el constructor por defecto.

Respuesta Habría que añadir simplemente una sentencia de creación o instanciación (llamada al constructor mediante el operador `new`) por cada objeto que se desee crear:

```
Rectangulo r1, r2, r3;
r1= new Rectangulo();
r2= new Rectangulo();
r3= new Rectangulo();
```

Manipulación de un objeto

Utilizar el ejemplo de los rectángulos para crear un rectángulo `r1`, asignarle los valores `x1`=0, `y1`=0, `x2`=10, `y2`=10, calcular su área y su perímetro y mostrarlos en pantalla.

Respuesta Se trata de declarar e instanciar el objeto `r1`, rellenar sus atributos de ubicación (coordenadas de las esquinas), e invocar a los métodos `calcularSuperficie()` y `calcularPerimetro()` utilizando el operador punto (`.`).

Por ejemplo:

```
Rectangulo r1= new Rectangulo ();
r1.x1= 0;
r1.y1= 0;
r1.x2= 10;
r1.y2= 10;
area= r1.calcularSuperficie ();
perímetro= r1.calcularPerimetro ();
```

Por último faltaría mostrar en pantalla la información calculada, podemos añadir esta y más pruebas en un método `main` de la propia clase, o en el `main` de otra clase del mismo paquete, como prefieras.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Utilización de constructores

Vamos a ampliar el ejemplo anterior creando una clase `RectanguloV2`, ampliando sus funcionalidades añadiéndole tres constructores:

1. **Un constructor sin parámetros** (para sustituir al constructor por defecto) que haga que los valores iniciales de las esquinas del rectángulo sean (0,0) y (1,1);
2. **Un constructor con cuatro parámetros**, `x1`, `y1`, `x2`, `y2`, que rellene los valores iniciales de los atributos del rectángulo con los valores proporcionados a través de los parámetros.
3. **Un constructor con dos parámetros**, base y altura, que cree un rectángulo donde el vértice inferior izquierdo esté ubicado en la posición (0,0) y que tenga una base y una altura tal y como indican los dos parámetros proporcionados.

Respuesta En el caso del primer constructor lo único que hay que hacer es "rellenar" los atributos `x1`, `y1`, `x2`, `y2` con los valores 0, 0, 1, 1:

```
public Rectangulov2 (){
    x1= 0.0;
    y1= 0.0;
    x2= 1.0;
    y2= 1.0;
}
```

Para el segundo constructor es suficiente con asignar a los atributos `x1`, `y1`, `x2`, `y2` los valores de los parámetros `x1`, `y1`, `x2`, `y2`. Tan solo hay que tener en cuenta que al tener los mismos nombres los parámetros del método que los atributos de la clase, estos últimos son ocultados por los primeros y para poder tener acceso a ellos tendrás que utilizar el operador de autorrerferencia `this`:

```
public Rectangulov2 (double x1, double y1, double x2, double y2){
    this.x1= x1;
    this.y1= y1;
    this.x2= x2;
    this.y2= y2;
}
```

En el caso del tercer constructor tendrás que inicializar el vértice (`x1`, `y1`) a (0,0) y el vértice (`x2`, `y2`) a (0 + base, 0 + altura), es decir a (base, altura):

```
public Rectangulov2 (double base, double altura) {
    this.x1= 0.0;
    this.y1= 0.0;
    this.x2= base;
    this.y2= altura;
}
```

Queda propuesto como ejercicio de ampliación la modificación de la clase `Rectangulo`, y ampliar el método `main` para usar las nuevas funcionalidades.

Referencia `this`

Añadir un método `obtenerNombrev2` de la clase `Rectangulov2` de ejercicios anteriores utilizando la referencia `this`.

Respuesta: Si utilizamos la referencia `this` en este método, entonces podremos utilizar como identificador del parámetro el mismo identificador que tiene el atributo (aunque no tiene porqué hacerse si no se desea):

```
public void establecerNombrev2 (String nombre) {
    this.nombre = nombre;
}
```

Constructores de copia

Añadir un constructor de copia al ejercicio de la clase `Rectangulov2` usando referencias `this`.

Respuesta Se trata de añadir un nuevo constructor además de los tres que ya habíamos creado:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
// Constructor copia
public Rectangulov2 (Rectangulov2 r) {
    this.x1= r.x1;
    this.y1= r.y1;
    this.x2= r.x2;
    this.y2= r.y2;
}
```

Para usar este constructor basta con haber creado anteriormente otro `Rectangulo` para utilizarlo como base de la copia. Por ejemplo:

```java
Rectangulo r1, r2;
r1= new Rectangulo (0,0,2,2);
r2= new Rectangulo (r1);
```

Ocultación de métodos

Vamos a intentar implementar una clase `DNI` que incluya todo lo que has visto hasta ahora. Se desea crear una clase que represente un DNI español y que tenga las siguientes características:

- La clase almacenará el número de `DNI` en un `int`, sin guardar la letra, pues se puede calcular a partir del número. Este atributo será privado a la clase. Formato del atributo: `private int numDNI`.
- Para acceder al `DNI` se dispondrá de dos métodos obtener (`get`), uno que proporcionará el número de `DNI` (sólo las cifras numéricas) y otro que devolverá el `NIF` completo (incluida la letra).

El formato del método será:

- `java public int obtenerDNI()`
- `java public String obtenerNIF()`
- Para modificar el `DNI` se dispondrá de dos métodos establecer (`set`), que permitirán modificar el `DNI`. Uno en el que habrá que proporcionar el `NIF` completo (número y letra). Y otro en el que únicamente será necesario proporcionar el `DNI` (las siete u ocho cifras). Si el `DNI`/`NIF` es incorrecto se debería lanzar algún tipo de excepción. El formato de los métodos (sobrecargados) será:
- `java public void establecer (String nif) throws ...`
- `java public void establecer (int dni) throws ...`
- La clase dispondrá de algunos métodos internos privados para calcular la letra de un número de `DNI` cualquiera, para comprobar si un `DNI` con su letra es válido, para extraer la letra de un `NIF`, etc. Aquellos métodos que no utilicen ninguna variable de objeto podrían declararse como estáticos (pertenecientes a la clase). Formato de los métodos:
- `java private static char calcularLetraNIF (int dni)`
- `java private boolean validarNIF (String nif)`
- `java private static char extraerLetraNIF (String nif)`
- `java private static int extraerNumeroNIF (String nif)`

Para calcular la letra `NIF` correspondiente a un número de `DNI` puedes consultar el artículo sobre el `NIF` de la Wikipedia

Respuesta:

Inténtalo por tu cuenta y cuando te quedes atascado tienes la solución en el Ejemplo03

6.1.14 Ejemplos UD05

Ejemplo01

Clase `Rectangulo` con miembros estáticos, de instancia y método `main` de prueba.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD05;

class Rectangulo {

    // Atributos de clase
    private static int numRectangulos; // Número total de rectangulos creados
    public static final String NOMBREFIGURA = "Rectángulo"; // Nombre de la clase
    public static final double PI = 3.1416; // Constante PI

    // Atributos de objeto
    private String nombre;// Nombre del rectángulo
    public double x1, y1;// Vértice inferior izquierdo
    public double x2, y2;// Vértice superior derecho

    // Método obtenerNombre
    public String obtenerNombre() {
        return nombre;
    }

    // Método establecerNombre
    public void establecerNombre(String nom) {
        nombre = nom;
    }

    // Método CalcularSuperficie
    public double CalcularSuperficie() {
        double area, base, altura;
        // Cálculo de la base
        base = x2 - x1;
        // Cálculo de la altura
        altura = y2 - y1;
        // Cálculo del área
        area = base * altura;
        // Devolución del valor de retorno
        return area;
    }

    // Método CalcularPerimetro
    public double CalcularPerimetro() {
        double perimetro, base, altura;
        // Cálculo de la base
        base = x2 - x1;
        // Cálculo de la altura
        altura = y2 - y1;
        // Cálculo del perímetro
        perimetro = 2 * base + 2 * altura;
        // Devolución del valor de retorno
        return perimetro;
    }

    // Método desplazar
    public void desplazar(double X, double Y) {
        // Desplazamiento en el eje X
        x1 = x1 + X;
        x2 = x2 + X;
        // Desplazamiento en el eje X
        y1 = y1 + Y;
        y2 = y2 + Y;
    }

    // Método obtenerNumRectangulos
    public static int obtenerNumRectangulos() {
        return numRectangulos;
    }

    public static void main(String[] args) {
        Rectangulo r1, r2;
        r1 = new Rectangulo();
        r2 = new Rectangulo();
        r1.x1 = 0;
        r1.y1 = 0;
        r1.x2 = 10;
        r1.y2 = 10;
        r1.establecerNombre("rectangulo1");
        System.out.printf("PRUEBA DE USO DE LA CLASE " + Rectangulo.NOMBREFIGURA + "\n");
        System.out.printf("----------------------------------\n\n");
        System.out.printf("r1.x1: %4.2f\nr1.y1: %4.2f\n", r1.x1, r1.y1);
        System.out.printf("r1.x2: %4.2f\nr1.y2: %4.2f\n", r1.x2, r1.y2);
        System.out.printf("Perimetro: %4.2f\nSuperficie: % 4.2f\n",
                r1.CalcularPerimetro(), r1.CalcularSuperficie());
        System.out.printf("Desplazamos X=3, Y=3\n");
        r1.desplazar(-3, 3);
        System.out.printf("r1.x1: %4.2f\nr1.y1: %4.2f\n", r1.x1, r1.y1);
        System.out.printf("r1.x2: %4.2f\nr1.y2: %4.2f\n", r1.x2, r1.y2);
    }
}
```

⬆ Volver a teoría

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Ejemplo02

Clase `Rectangulov2` con constructores sobrecargados, constructor de copia y uso de `this`.

```java
package UD05;

class Rectangulov2 {

    // Atributos de clase
    private static int numRectangulos; // Número total de rectangulos creados
    public static final String NOMBREFIGURA = "Rectángulov2"; // Nombre de la clase
    public static final double PI = 3.1416; // Constante PI

    // Atributos de objeto
    private String nombre;// Nombre del rectángulo
    public double x1, y1;// Vértice inferior izquierdo
    public double x2, y2;// Vértice superior derecho

    // Método obtenerNombre
    public String obtenerNombre() {
        return nombre;
    }

    // Método establecerNombre
    public void establecerNombre(String nom) {
        nombre = nom;
    }

    // Método CalcularSuperficie
    public double CalcularSuperficie() {
        double area, base, altura;
        // Cálculo de la base
        base = x2 - x1;
        // Cálculo de la altura
        altura = y2 - y1;
        // Cálculo del área
        area = base * altura;
        // Devolución del valor de retorno
        return area;
    }

    // Método CalcularPerimetro
    public double CalcularPerimetro() {
        double perimetro, base, altura;
        // Cálculo de la base
        base = x2 - x1;
        // Cálculo de la altura
        altura = y2 - y1;
        // Cálculo del perímetro
        perimetro = 2 * base + 2 * altura;
        // Devolución del valor de retorno
        return perimetro;
    }

    // Método desplazar
    public void desplazar(double X, double Y) {
        // Desplazamiento en el eje X
        x1 = x1 + X;
        x2 = x2 + X;
        // Desplazamiento en el eje X
        y1 = y1 + Y;
        y2 = y2 + Y;
    }

    // Método obtenerNumRectangulos
    public static int obtenerNumRectangulos() {
        return numRectangulos;
    }

    //Constructor por defecto
    public Rectangulov2() {
        x1 = 0.0;
        y1 = 0.0;
        x2 = 1.0;
        y2 = 1.0;
        numRectangulos++;
    }

    //constructor con los 4 vertices
    public Rectangulov2(double x1, double y1, double x2, double y2) {
        this.x1 = x1;
        this.y1 = y1;
        this.x2 = x2;
        this.y2 = y2;
        numRectangulos++;
    }
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
    //constructor con base y altura
    public Rectangulov2(double base, double altura) {
        this.x1 = 0.0;
        this.y1 = 0.0;
        this.x2 = base;
        this.y2 = altura;
        numRectangulos++;
    }

    //referencia this
    public void establecerNombrev2(String nombre) {
        this.nombre = nombre;
    }

    // Constructor copia
    public Rectangulov2(Rectangulov2 r) {
        this.nombre=r.nombre;//supongo que también quiero copiar el nombre
        this.x1 = r.x1;
        this.y1 = r.y1;
        this.x2 = r.x2;
        this.y2 = r.y2;
        numRectangulos++;
    }

    public static void main(String[] args) {
        Rectangulov2 r1;
        Rectangulov2 r2;
        Rectangulov2 r3;
        r1 = new Rectangulov2();
        r2 = new Rectangulov2(4, 4, 8, 8);
        r3 = new Rectangulov2(5, 5);
        r1.establecerNombrev2("defecto");
        r2.establecerNombrev2("4 vertices");
        r3.establecerNombrev2("base y altura");

        System.out.printf("PRUEBA DE USO DE LA CLASE " + Rectangulov2.NOMBREFIGURA + "\n");
        System.out.printf("-----------------------------------\n\n");
        System.out.printf("r1.x1: %4.2f\nr1.y1: %4.2f\n", r1.x1, r1.y1);
        System.out.printf("r1.x2: %4.2f\nr1.y2: %4.2f\n", r1.x2, r1.y2);

        //Usamos el constructor de copia para realizar una copia del rectangulo
        Rectangulov2 r4 = new Rectangulov2(r1);
        System.out.println("r4 es una copia de r1");

        System.out.printf("r4.x1: %4.2f\nr4.y1: %4.2f\n", r4.x1, r4.y1);
        System.out.printf("r4.x2: %4.2f\nr4.y2: %4.2f\n", r4.x2, r4.y2);
    }
}
```

⬆ Volver a teoría

Ejemplo03

Clase `DNI` con métodos estáticos de validación, cálculo de letra NIF y encapsulación.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
public class DNI {

    // Atributos estáticos
    // Cadena con las letras posibles del DNI ordenados para el cálculo de DNI
    private static final String LETRAS_DNI = "TRWAGMYFPDXBNJZSQVHLCKE";

    // Atributos de objeto
    private int numDNI;

    // Métodos
    public String obtenerNIF() {
        // Variables locales
        String cadenaNIF;
        // NIF con letra para devolver
        char letraNIF;
        // Letra del número de NIF calculado
        // Cálculo de la letra del NIF
        letraNIF = calcularLetraNIF(numDNI);
        // Construcción de la cadena del DNI: número + letra
        cadenaNIF = Integer.toString(numDNI) + String.valueOf(letraNIF);
        // Devolución del resultado
        return cadenaNIF;
    }

    public int obtenerDNI() {
        return numDNI;
    }

    public void establecer(String nif) throws Exception {
        if (DNI.validarNIF(nif)) { // Valor válido: lo almacenamos
            this.numDNI = DNI.extraerNumeroNIF(nif);
        } else { // Valor inválido: lanzamos una excepción
            throw new Exception("NIF inválido: " + nif);
        }
    }

    public void establecer(int dni) throws Exception {
        // Comprobación de rangos
        if (dni > 999999 && dni < 99999999) {
            this.numDNI = dni; // Valor válido: lo almacenamos
        } else { // Valor inválido: lanzamos una excepción
            throw new Exception("DNI inválido: " + String.valueOf(dni));
        }
    }

    private static char calcularLetraNIF(int dni) {
        char letra;
        // Cálculo de la letra NIF
        letra = LETRAS_DNI.charAt(dni % 23);
        // Devolución de la letra NIF
        return letra;
    }

    private static char extraerLetraNIF(String nif) {
        char letra = nif.charAt(nif.length() - 1);
        return letra;
    }

    private static int extraerNumeroNIF(String nif) {
        int numero = Integer.parseInt(nif.substring(0, nif.length() - 1));
        return numero;
    }

    private static boolean validarNIF(String nif) {
        boolean valido = true;
        // Suponemos el NIF válido mientras no se encuentre algún fallo
        char letra_calculada;
        char letra_leida;
        int dni_leido;
        if (nif == null) { // El parámetro debe ser un objeto no vacío
            valido = false;
        } else if (nif.length() < 8 || nif.length() > 9) {
            // La cadena debe estar entre 8(7+1) y 9(8+1) caracteres
            valido = false;
        } else {
            letra_leida = DNI.extraerLetraNIF(nif);
            // Extraemos la letra de NIF (letra)
            dni_leido = DNI.extraerNumeroNIF(nif); // Extraemos el número de DNI (int)
            letra_calculada = DNI.calcularLetraNIF(dni_leido);
            // Calculamos la letra de NIF a partir del número extraído
            if (letra_leida == letra_calculada) {
                // Comparamos la letra extraída con la calculada
                // Todas las comprobaciones han resultado válidas. El NIF es válido.
                valido = true;
            } else {
                valido = false;
            }
        }
        return valido;
    }
}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

⬆ Volver a teoría

Ejemplo04

Clase `Casting` que demuestra el casting implícito y explícito entre objetos de una jerarquía de herencia.

```java
package UD05;

public class Casting {

    public static void main(String[] args) {
        // Casting Implicito
        Persona encargadoCarniceria = new Encargado("Rosa Ramos", 1200,
                "Carniceria");

        // No tenemos disponibles los métodos de la clase Encargado:
        //EncargadaCarniceria.setSueldoBase(1200);
        //EncargadaCarniceria.setSeccion("Carniceria");
        //Pero al imprimir se imprime con el método más específico (luego lo vemos)
        System.out.println(encargadoCarniceria);

        // Casting Explicito
        Encargado miEncargado = (Encargado) encargadoCarniceria;
        //Tenemos disponibles los métodos de la clase Encargado:
        miEncargado.setSueldoBase(1200);
        miEncargado.setSeccion("Carniceria");
        //Al imprimir se imprime con el método más específico de nuevo.
        System.out.println(miEncargado);
    }
}
```

⬆ Volver a teoría

Ejemplo05

Clase base `Persona` con atributo `nombre`, getter, setter y método `toString`.

```java
package UD05;

// Clase Persona que solo dispone de nombre
public class Persona {

    String nombre;

    public Persona(String nombre) {
        this.nombre = nombre;
    }

    public void setNombre(String nom) {
        nombre = nom;
    }

    public String getNombre() {
        return nombre;
    }

    @Override
    public String toString() {
        return "Nombre: " + nombre;
    }
}
```

⬆ Volver a teoría

Ejemplo06

Clase derivada `Empleado` que hereda de `Persona` y añade el atributo `sueldoBase`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD05;

// Clase Empleado que hereda de Persona y añade atributo sueldoBase
public class Empleado extends Persona {

    double sueldoBase;

    public Empleado(String nombre, double sueldoBase) {
        super(nombre);
        this.sueldoBase = sueldoBase;
    }

    public double getSueldo() {
        return sueldoBase;
    }

    public void setSueldoBase(double sueldoBase) {
        this.sueldoBase = sueldoBase;
    }

    @Override
    public String toString() {
        return super.toString() + "\nSueldo Base: " + sueldoBase;
    }
}
```

⬆ Volver a teoría

Ejemplo07

Clase derivada `Encargado` que hereda de `Empleado` y añade el atributo `seccion`.

```java
package UD05;

// Clase Encargado que hereda de Empleado y añade atributo seccion
public class Encargado extends Empleado {

    String seccion;

    public Encargado(String nombre, double sueldoBase, String seccion) {
        super(nombre, sueldoBase);
        this.seccion = seccion;
    }

    public String getSeccion() {
        return seccion;
    }

    public void setSeccion(String seccion) {
        this.seccion = seccion;
    }

    @Override
    public String toString() {
        return super.toString() + "\nSección:" + seccion ;
    }
}
```

⬆ Volver a teoría

Ejemplo08

Clase `ClasesAnidadas` con clases internas `Monitor` y `Cpu` dentro de la clase `Pc`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD05;

class Pc {

    double precio;

    public String toString() {
        return "El precio del PC es " + this.precio;
    }

    class Monitor {

        String marca;

        public String toString() {
            return "El monitor es de la marca " + this.marca;
        }
    }

    class Cpu {

        String marca;

        public String toString() {
            return "La CPU es de la marca " + this.marca;
        }
    }
}

public class ClasesAnidadas {

    public static void main(String[] args) {
        Pc miPc = new Pc();
        Pc.Monitor miMonitor = miPc.new Monitor();
        Pc.Cpu miCpu = miPc.new Cpu();
        miPc.precio = 1250.75;
        miMonitor.marca = "Asus";
        miCpu.marca = "Acer";
        System.out.println(miPc); //El precio del PC es 1250.75
        System.out.println(miMonitor); //El monitor es de la marca Asus
        System.out.println(miCpu); //La CPU es de la marca Acer
    }
}
```

⬆ Volver a teoría

6.1.15 Resumen — Conceptos clave

Concepto	Definición
Modificador de acceso	Palabra clave que controla la visibilidad (public, private, protected)
Sobrecarga	Varios métodos con el mismo nombre pero diferentes parámetros
Getter/Setter	Métodos que permiten acceder y modificar atributos privados
Constructor de copia	Constructor que crea un objeto copiando los valores de otro
Clase interna	Clase definida dentro de otra clase
Paquete	Agrupación de clases relacionadas

6.1.16 Autoevaluación

- [ ] Diseño clases con atributos privados y métodos públicos
- [ ] Aplico correctamente los modificadores de acceso
- [ ] Sobrecargo métodos y constructores
- [ ] Implemento getters y setters
- [ ] Comprendo el concepto de clase interna
- [ ] Organizo mis clases en paquetes

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

6.1.17 Vídeos recomendados

Canal	Vídeo	Contenido
Programación ATS	Herencia en POO (vídeo 74)	Herencia entre clases
Programación ATS	Polimorfismo en POO (vídeo 77)	Polimorfismo y enlace dinámico
Programación ATS	Playlist POO Java	Vídeos 73–82: clases abstractas (76), interfaces, modificadores de acceso
Píldoras Informáticas	Herencia I (vídeo 40)	Introducción a la herencia
Píldoras Informáticas	Interfaces I–IV (vídeos 49–52)	Interfaces y clases internas
BitBoss	Los 4 pilares de POO	Abstracción, encapsulación, herencia, polimorfismo (10 min)
MoureDev	Curso Completo de Java desde Cero	Sección POO (herencia, interfaces, polimorfismo)

27 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

6.2 Anexo Wrappers

6.2.1 Wrappers (Envoltorios)

Los wrappers permiten "envolver" datos primitivos en objetos, también se llaman clases contenedoras. La diferencia entre un tipo primitivo y un wrapper es que este último es una clase y por tanto, cuando trabajamos con wrappers estamos trabajando con objetos.

> **Recuerda**
>
> Como son objetos debemos tener cuidado en el paso como parámetro en métodos ya que en el wrapper se realiza por referencia.

Una de las principales ventajas del uso de wrappers son la facilidad de conversión entre tipos primitivos y cadenas.

Hay una clase contenedora por cada uno de los tipos primitivos de Java. Los datos primitivos se escriben en minúsculas y los wrappers se escriben con la primera letra en mayúsculas.

Tipo primitivo	Wrapper asociado
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Char
boolean	Boolean

Cada clase wrapper tiene dos constructores, uno se le pasa por parámetro el dato de tipo primitivo y otro se le pasa un `String`.

Para wrapper `Integer`:

```
Integer(int)
Integer(String)
```

Ejemplo:

```
Integer i1 = new Integer(42);
Integer i2 = new Integer ("42");
Float f1 = new Float(3.14f);
Float f2 = new Float ("3.14f");
```

Antiguamente, una vez asignado un valor a un objeto o wrapper `Integer`, este no podía cambiarse. Actualmente e internamente se apoyan en variables y wrapers internos para poder variar el valor de un wrapper.

Ejemplo:

```
Integer y = new Integer(567);       //Crea el objeto
y++;                                //Lo desenvuelve, incrementa y lo vuelve a envolver
System.out.println("Valor: " + y); //Imprime el valor del Objeto y
```

Los wrapper disponen de una serie de métodos que permiten realizar funciones de conversión de datos. Por ejemplo, el wrapper `Integer` dispone de los siguientes métodos:

Método	Descripción
`Integer(int)` `Integer(String)`	Constructores

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Método	Descripción

`byteValue()`
`shortValue()`
`intValue()`
`longValue()`
`doubleValue()`
`floatValue()`
	Funciones de conversión con datos primitivos

`Integer decode(String)`
`Integer parseInt(String)`
`Integer parseInt(String, int)`
`Integer valueOf(String)`
`String toString()`
	Conversión a String

`String toBinaryString(int)`
`String toHexString(int)`
`String toOctalString(int)`
	Conversión a otros sistemas de numeración

`MAX_VALUE`, `MIN_VALUE`, `TYPE`	Constantes

Métodos `valueOf()`

El método `valueOf()` permite crear objetos wrapper y se le pasa un parámetro `String` y opcionalmente otro parámetro que indica la base en la que será representado el primer parámetro.

Ejemplo:

```
// Convierte el 101011 (base 2) a 43 y le asigna el valor al objeto Integer i3
Integer i3 = Integer.valueOf("101011", 2);
System.out.println(i3);

// Asigna 3.14 al objeto Float f3
Float f3 = Float.valueOf("3.14f");
System.out.println(f3);
```

Métodos `xxxValue()`.

Los métodos `xxxValue()` permiten convertir un wrapper en un dato de tipo primitivo y no necesitan argumentos.

Ejemplo:

```
Integer i4 = 120; // Crea un nuevo objeto wrapper
byte b = i4.byteValue(); // Convierte el valor de i4 a un primitivo byte
short s1 = i4.shortValue(); // Otro de los métodos de Integer
double d = i4.doubleValue(); // Otro de los métodos xxxValue de Integer
System.out.println(s1); // Muestra 120 como resultado

Float f4 = 3.14f; // Crea un nuevo objeto wrapper
short s2 = f4.shortValue(); // Convierte el valor de f4 en un primitivo short
System.out.println(s2); // El resultado es 3 (truncado, no redondeado)
```

Métodos `parseXxxx()`

Los métodos `parseXxxx()` permiten convertir un wrapper en un dato de tipo primitivo y le pasamos como parámetro el `String` con el valor que deseamos convertir y opcionalmente la base a la que convertiremos el valor (2, 8, 10 o 16).

Ejemplo:

```
double d4 = Double.parseDouble("3.14"); // Convierte un String a primitivo
System.out.println("d4 = " + d4);   // El resultado será d4 = 3.14
long l2 = Long.parseLong("101010", 2);  // un String binario a primitivo
System.out.println("l2 = " + l2);   // El resultado es l2 = 42
```

Métodos `toString()`

El método `toString()` permite retornar un `String` con el valor primitivo que se encuentra en el objeto contenedor. Se le pasa un parámetro que es el wrapper y opcionalmente para `Integer` y `Long` un parámetro con la base a la que convertiremos el valor (2, 8, 10 o 16).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Ejemplo:

```
Double d1 = new Double("3.14");
System.out.println("d1 = " + d1.toString() ); // El resultado es d = 3.14
String d2 = Double.toString(3.14); // d2 = "3.14"
System.out.println("d2 = " + d2); // El resultado es d = 3.14
String s3 = "hex = " + Long.toString(254, 16); // s3 = "hex = fe"
System.out.println("s3 = " + s3); // El resultado es s3 = hex = fe
```

Métodos `toXxxxxString()` (Binario, Hexadecimal y Octal)

Los métodos `toXxxxxString()` permiten a las clases contenedoras `Integer` y `Long` convertir números en base 10 a otras bases, retornando un `String` con el valor primitivo que se encuentra en el objeto contenedor.

Ejemplo:

```
String s4 = Integer.toHexString(254); // Convierte 254 a hex
System.out.println("254 es " + s4); // Resultado: "254 es fe"
String s5 = Long.toOctalString(254); // Convierte 254 a octal
System.out.println("254(oct) = " + s5); // Resultado: "254(oct) = 376"
```

Resumen

Para resumir, los métodos esenciales para las conversiones son:

- **`primitive xxxValue()`** – Para convertir de `Wrapper` a `primitive`
- **`primitive parseXxx(String)`** – Para convertir un `String` en `primitive`
- **`Wrapper valueOf(String)`** – Para convertir `String` en `Wrapper`

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

6.2.2 Ejemplo Anexo UD05

Anexo1Wrappers

```java
package es.martinezpenya.ejemplos.UD05;

public class Anexo1Wrappers {

    public static void main(String[] args) {

        // WRAPPERS
        //Integer i1 = new Integer(42); // Obsoleto (deprecated)
        Integer i1 = Integer.valueOf(42);
        //Integer i2 = new Integer("42");// Obsoleto (deprecated)
        Integer i2 = Integer.valueOf("42");
        //Float f1 = new Float(3.14f);// Obsoleto (deprecated)
        Float f1 = Float.valueOf(3.14f);
        //Float f2 = new Float("3.14f");// Obsoleto (deprecated)
        Float f2 = Float.valueOf("3.14f");

        Integer y = Integer.valueOf(567);      //Crea el objeto
        y++;                    //Lo desenvuelve, incrementa y lo vuelve a envolver
        System.out.println("Valor: " + y); //Imprime el valor del Objeto y

        // VALUEOF
        // Convierte el 101011 (base 2) a 43 y le asigna el valor al objeto Integer i1
        Integer i3 = Integer.valueOf("101011", 2);
        System.out.println(i3);

        // Asigna 3.14 al objeto Float f3
        Float f3 = Float.valueOf("3.14f");
        System.out.println(f3);

        // XXXVALUE
        Integer i4 = 120; // Crea un nuevo objeto wrapper
        byte b = i4.byteValue(); // Convierte el valor de i2 a un primitivo byte
        short s1 = i4.shortValue(); // Otro de los métodos de Integer
        double d = i4.doubleValue(); // Otro de los métodos xxxValue de Integer
        System.out.println(s1); // Muestra 120 como resultado

        Float f4 = 3.14f; // Crea un nuevo objeto wrapper
        short s2 = f4.shortValue(); // Convierte el valor de f2 en un primitivo short
        System.out.println(s2); // El resultado es 3 (truncado, no redondeado)

        // PARSEXXXX
        double d4 = Double.parseDouble("3.14"); // Convierte un String a primitivo
        System.out.println("d4 = " + d4);   // El resultado será d4 = 3.14
        long l2 = Long.parseLong("101010", 2);  // un String binario a primitivo
        System.out.println("l2 = " + l2);   // El resultado es L2 42

        // TOSTRING
        Double d1 = Double.valueOf("3.14");
        System.out.println("d1 = " + d1.toString()); // El resultado es d = 3.14
        String d2 = Double.toString(3.14); // d2 = "3.14"
        System.out.println("d2 = " + d2); // El resultado es d = 3.14
        String s3 = "hex = " + Long.toString(254, 16); // s = "hex = fe"
        System.out.println("s3 = " + s3); // El resultado es d = 3.14

        // TOXXXSTRING
        String s4 = Integer.toHexString(254); // Convierte 254 a hex
        System.out.println("254 es " + s4); // Resultado: "254 es fe"
        String s5 = Long.toOctalString(254); // Convierte 254 a octal
        System.out.println("254(oct) = " + s5); // Resultado: "254(oct) = 376"
    }
}
```

21 de julio de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

6.3 Anexo Fechas

6.3.1 Clase `Date`

La clase Date es una utilidad contenida en el paquete `java.util` y permiten trabajar con fechas y horas. La fechas y hora se almacenan en un entero de tipo `Long` que almacena los milisegundos transcurridos desde el 1 de Enero de de 1970 que se obtienen con `getTime()`. (Importamos `java.util.Date`).

Ejemplo:

```
Date fecha = new Date(2021, 9, 19);
System.out.println(fecha);          //Mon Sep 19 00:00:00 CEST 2021
System.out.println(fecha.getTime()); //61590146400000
```

Clase `GregorianCalendar`

Para utilizar fechas y horas se utiliza la clase `GregorianCalendar` que dispone de variable enteras como: `DAY_OF_WEEK`, `DAY_OF_MONTH`, `YEAR`, `MONTH`, `HOUR`, `MINUTE`, `SECOND`, `MILLISECOND`, `WEEK_OF_MONTH`, `WEEK_OF_YEAR`, ... (Importamos Clase `java.util.Calendar` y `java.util.GregorianCalendar`)

Ejemplo 1:

```
Calendar calendar = new GregorianCalendar(2021, 9, 19);
System.out.println(calendar.getTime()); //Sun Sep 19 00:00:00 CEST 2021
```

Ejemplo 2:

```
Date d = new Date();
GregorianCalendar c = new GregorianCalendar();
System.out.println("Fecha: "+d);  //Fecha: Thu Aug 19 20:06:14 CEST 2021
System.out.println("Info: "+c); //Info:
//java.util.GregorianCalendar[time=1629396374723,areFieldsSet=true
//,areAllFieldsSet=true
//,lenient=true,zone=sun.util.calendar.ZoneInfo[id="Europe/Madrid",offset=3600000
//,dstSavings=3600000,useDaylight=true,transitions=163
//,lastRule=java.util.SimpleTimeZone[id=Europe/Madrid,offset=3600000
//,dstSavings=3600000,useDaylight=true,startYear=0,startMode=2,startMonth=2
//,startDay=-1,startDayOfWeek=1,startTime=3600000,startTimeMod2.1e=2,endMode=2
//,endMonth=9,endDay=-1,endDayOfWeek=1,endTime=3600000,endTimeMode=2]]
//,firstDayOfWeek=2,minimalDaysInFirstWeek=4,ERA=1,YEAR=2021,MONTH=7,WEEK_OF_YEAR=33
//,WEEK_OF_MONTH=3,DAY_OF_MONTH=19,DAY_OF_YEAR=231,DAY_OF_WEEK=5
//,DAY_OF_WEEK_IN_MONTH=3,AM_PM=1,HOUR=8,HOUR_OF_DAY=20,MINUTE=6,SECOND=14
//,MILLISECOND=723,ZONE_OFFSET=3600000,DST_OFFSET=3600000]
c.setTime(d);
System.out.print(c.get(Calendar.DAY_OF_MONTH));//19
System.out.print("/");
System.out.print(c.get(Calendar.MONTH)+1); //9
System.out.print("/");
System.out.println(c.get(Calendar.YEAR)); //2022
```

Paquete `java.time`

El paquete `java.time` dispone de las clases `LocalDate`, `LocalTime`, `LocalDateTime`, `Duration` y `Period` para trabajar con fechas y horas.

Estas clases no tienen constructores públicos, y por tanto, no se puede usar `new` para crear objetos de estas clases. Necesitas usar sus métodos `static` para instanciarlas.

No es válido llamar directamente al constructor usando `new`, ya que no tienen un constructor público.

Ejemplo:

```
LocalDate d = new LocalDate(); //NO compila
```

`LocalDate`

`LocalDate` representa una fecha determinada. Haciendo uso del método `of()`, esta clase puede crear un `LocalDate` teniendo en cuenta el año, mes y día. Finalmente, para capturar el `LocalDate` actual se puede usar el método `now()`:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Ejemplo:

```
LocalDate date = LocalDate.of(1989, 11, 11); //1989-11-11
System.out.println(date.getYear()); //1989
System.out.println(date.getMonth()); //NOVEMBER
System.out.println(date.getDayOfMonth()); //11
date = LocalDate.now();
System.out.println(date); //2021-08-19
```

`LocalTime`

`LocalTime`, representa un tiempo determinado. Haciendo uso del método `of()`, esta clase puede crear un `LocalTime` teniendo en cuenta la hora, minuto, segundo y nanosegundo. Finalmente, para capturar el `LocalTime` actual se puede usar el método `now()`.

```
LocalTime time = LocalTime.of(5, 30, 45, 35); //05:30:45:35
System.out.println(time.getHour()); //5
System.out.println(time.getMinute()); //30
System.out.println(time.getSecond()); //45
System.out.println(time.getNano()); //35
time = LocalTime.now();
System.out.println(time); //20:13:53.118044
```

`LocalDateTime`

`LocalDateTime`, es una clase compuesta, la cual combina las clases anteriormente mencionadas `LocalDate` y `LocalTime`. Podemos construir un `LocalDateTime` haciendo uso de todos los campos (año, mes, día, hora, minuto, segundo, nanosegundo).

Ejemplo:

```
LocalDateTime dateTime = LocalDateTime.of(1989, 11, 11, 5, 30, 45, 35);
```

También, se puede crear un objeto `LocalDateTime` basado en los tipos `LocalDate` y `LocalTime`, haciendo uso del método `of()` (`LocalDate date`, `LocalTime time`):

Ejemplo:

```
LocalDate date = LocalDate.of(1989, 11, 11);
LocalTime time = LocalTime.of(5, 30, 45, 35);
LocalDateTime dateTime = LocalDateTime.of(date, time);
LocalDateTime dateTime = LocalDateTime.now();
```

`Duration`

`Duration`, hace referencia a la diferencia que existe entre dos objetos de tiempo. La duración denota la cantidad de tiempo en horas, minutos y segundos.

Ejemplo:

```
LocalTime localTime1 = LocalTime.of(12, 25);
LocalTime localTime2 = LocalTime.of(17, 35);
Duration duration1 = Duration.between(localTime1, localTime2);
System.out.println(duration1); //PT5H10M
System.out.println(duration1.toDays()); //0

LocalDateTime localDateTime1 = LocalDateTime.of(2016, Month.JULY, 18, 14, 13);
LocalDateTime localDateTime2 = LocalDateTime.of(2016, Month.JULY, 20, 12, 25);
Duration duration2 = Duration.between(localDateTime1, localDateTime2);
System.out.println(duration2); //PT46H12M
System.out.println(duration2.toDays()); //1
```

También, se puede crear `Duration` basado en los métodos `ofDays(long days)`, `ofHours(long hours)`, `ofMilis(long milis)`, `ofMinutes(long minutes)`, `ofNanos(long nanos)`, `ofSeconds(long seconds)`.

Ejemplo:

```
Duration duracion3 = Duration.ofDays(1);
System.out.println(duracion3); //PT24H
System.out.println(duracion3.toDays()); //1
```

`Period`

`Period`, hace referencia a la diferencia que existe entre dos fechas. Esta clase denota la cantidad de tiempo en años, meses y días.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
LocalDate localDate1 = LocalDate.of(2016, 7, 18);
LocalDate localDate2 = LocalDate.of(2016, 7, 20);
Period periodo1 = Period.between(localDate1, localDate2);
System.out.println(periodo1); //P2D
```

Se puede crear `Period` basado en el método `of(int years, int months, int days)`. En el siguiente ejemplo, se crea un período de 1 año 2 meses y 3 días:

```
Period periodo2 = Period.of(1, 2, 3);
System.out.println(periodo2); //P1Y2M3D
```

Se puede crear `Period` basado en los métodos `ofDays(int days)`, `ofMonths(int months)`, `ofWeeks(int weeks)`, `ofYears(int years)`.

Ejemplo:

```
Period periodo3 = Period.ofYears(1);
System.out.println(periodo3); //P1Y
```

`ChronoUnit`

Permite devolver el tiempo transcurrido entre dos fechas en diferentes formatos (`DAYS`, `MONTHS`, `YEARS`, `HOURS`, `MINUTES`, `SECONDS`, ...). Debemos importar la clase `java.time.temporal.ChronoUnit`;

Ejemplo:

```
LocalDate fechaInicio = LocalDate.of(2016, 7, 18);
LocalDate fechaFin = LocalDate.of(2016, 7, 20);
// Calculamos el tiempo transcurrido entre las dos fechas
// con la clase ChronoUnit y la unidad temporal en la que
// queremos que nos lo devuelva, en este caso DAYS.
long tiempo = ChronoUnit.DAYS.between(fechaInicio, fechaFin);
System.out.println(tiempo); //2
```

Introducir fecha como Cadena

Podemos introducir la fecha como una cadena con el formato que deseemos y posteriormente convertir a fecha con la sentencia `parse`. Debemos importar las clases `time` y `time.format`.

Ejemplo:

```
DateTimeFormatter formato = DateTimeFormatter.ofPattern("d/MM/u");
String fechaCadena = "16/08/2016";
LocalDate mifecha = LocalDate.parse(fechaCadena, formato);
System.out.println(formato.format(mifecha)); //16/08/2016
```

> **Ojo!**
>
> A partir de Java 8 `y` es para el año de la era (BC AD), y para el año debemos usar `u`

Más detalles sobre los formatos: https://docs.oracle.com/javase/8/docs/api/java/time/format/DateTimeFormatter.html

Manipulación

1. Manipulando `LocalDate`

Haciendo uso de los métodos `withYear(int year)`, `withMonth(int month)`, `withDayOfMonth(int dayOfMonth)`, `with(TemporalField field, long newValue)` se puede modificar el `LocalDate`.

Ejemplo:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
LocalDate date = LocalDate.of(2016, 7, 25);
LocalDate date1 = date.withYear(2017);
LocalDate date2 = date.withMonth(8);
LocalDate date3 = date.withDayOfMonth(27);
System.out.println(date);  //2016-07-25
System.out.println(date1); //2017-07-25
System.out.println(date2); //2016-08-25
System.out.println(date3); //2016-07-27
```

1. Manipulando `LocalTime`

Haciendo uso de los métodos `withHour(int hour)`, `withMinute(int minute)`, `withSecond(int second)`, `withNano(int nanoOfSecond)` se puede modificar el `LocalTime`.

Ejemplo:

```
LocalTime time = LocalTime.of(14, 30, 35);
LocalTime time1 = time.withHour(20);
LocalTime time2 = time.withMinute(25);
LocalTime time3 = time.withSecond(23);
LocalTime time4 = time.withNano(24);
System.out.println(time);  //14:30:35
System.out.println(time1); //20:30:35
System.out.println(time2); //14:25:35
System.out.println(time3); //14:30:23
System.out.println(time4); //14:30:35.000000024
```

1. Manipulando `LocalDateTime`

`LocalDateTime` provee los mismo métodos mencionados en las clases `LocalDate` y `LocalTime`.

Ejemplo:

```
LocalDateTime dateTime = LocalDateTime.of(2016, 7, 25, 22, 11, 30);
LocalDateTime dateTime1 = dateTime.withYear(2017);
LocalDateTime dateTime2 = dateTime.withMonth(8);
LocalDateTime dateTime3 = dateTime.withDayOfMonth(27);
LocalDateTime dateTime4 = dateTime.withHour(20);
LocalDateTime dateTime5 = dateTime.withMinute(25);
LocalDateTime dateTime6 = dateTime.withSecond(23);
LocalDateTime dateTime7 = dateTime.withNano(24);
System.out.println(dateTime);  //2016-07-25T22:11:30
System.out.println(dateTime1); //2017-07-25T22:11:30
System.out.println(dateTime2); //2016-08-25T22:11:30
System.out.println(dateTime3); //2016-07-27T22:11:30
System.out.println(dateTime4); //2016-07-25T20:11:30
System.out.println(dateTime5); //2016-07-25T22:25:30
System.out.println(dateTime6); //2016-07-25T22:11:23
System.out.println(dateTime7); //2016-07-25T22:11:30.000000024
```

Operaciones

OPERACIONES CON `LocalDate`

Realizar operaciones como suma o resta de días, meses, años, etc es muy fácil con la nueva `Date` API. Los siguientes métodos `plus(long amountToAdd, TemporalUnit unit)`, `minus(long amountToSubtract, TemporalUnit unit)` proveen una manera general de realizar estas operaciones. (Debemos importar la clase `java.time.temporal.ChronoUnit` para poder utilizar las unidades: `ChronoUnit.YEARS`, `ChronoUnit.MONTHS`, `ChronoUnit.DAYS`).

Ejemplo:

```
LocalDate date = LocalDate.of(2016, 7, 18);
LocalDate datePlusOneDay = date.plus(1, ChronoUnit.DAYS);
LocalDate dateMinusOneDay = date.minus(1, ChronoUnit.DAYS);
System.out.println(date);            // 2016-07-18
System.out.println(datePlusOneDay);  // 2016-07-19
System.out.println(dateMinusOneDay); // 2016-07-17
```

También se puede hacer cálculos basados en un `Period`. En el siguiente ejemplo, se crea un `Period` de 1 día para poder realizar los cálculos.

Ejemplo:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
LocalDate date = LocalDate.of(2016, 7, 18);
LocalDate datePlusOneDay = date.plus(Period.ofDays(1));
LocalDate dateMinusOneDay = date.minus(Period.ofDays(1));
System.out.println(date);            // 2016-07-18
System.out.println(datePlusOneDay);  // 2016-07-19
System.out.println(dateMinusOneDay); // 2016-07-17
```

Finalmente, haciendo uso de métodos explícitos como `plusDays(long daysToAdd)` y `minusDays(long daysToSubtract)` se puede indicar el valor a incrementar o reducir.

Ejemplo:

```
LocalDate date = LocalDate.of(2016, 7, 18);
LocalDate datePlusOneDay = date.plusDays(1);
LocalDate dateMinusOneDay = date.minusDays(1);
System.out.println(date);            // 2016-07-18
System.out.println(datePlusOneDay);  // 2016-07-19
System.out.println(dateMinusOneDay); // 2016-07-17
```

OPERACIONES CON `LocalTime`

La nueva `Date` API perimite realizar operaciones como suma y resta de horas, minutos, segundos, etc. Al igual que `LocalDate`, los siguientes métodos `plus(long amountToAdd, TemporalUnit unit)`, `minus(long amountToSubtract, TemporalUnit unit)` proveen una manera general de realizar estas operaciones.

(Debemos importar la clase `java.time.temporal.ChronoUnit` para poder utilizar las unidades: `ChronoUnit.HOURS`, `ChronoUnit.MINUTES`, `ChronoUnit.SECONDS`, `ChronoUnit.NANOS`).

Ejemplo:

```
LocalTime time = LocalTime.of(15, 30);
LocalTime timePlusOneHour = time.plus(1, ChronoUnit.HOURS);
LocalTime timeMinusOneHour = time.minus(1, ChronoUnit.HOURS);
System.out.println(time);              // 15:30
System.out.println(timePlusOneHour);   // 16:30
System.out.println(timeMinusOneHour);  // 14:30
```

También se puede hacer cálculos basados en un `Duration`. En el siguiente ejemplo, se crea un `Duration` de 1 hora para poder realizar los cálculos.

```
LocalTime time = LocalTime.of(15, 30);
LocalTime timePlusOneHour = time.plus(Duration.ofHours(1));
LocalTime timeMinusOneHour = time.minus(Duration.ofHours(1));
System.out.println(time);              // 15:30
System.out.println(timePlusOneHour);   // 16:30
System.out.println(timeMinusOneHour);  // 14:30
```

Finalmente, haciendo uso de métodos explícitos como `plusHours(long hoursToAdd)` y `minusHours(long hoursToSubtract)` se puede indicar el valor a incrementar o reducir.

Ejemplo:

```
LocalTime time = LocalTime.of(15, 30);
LocalTime timePlusOneHour = time.plusHours(1);
LocalTime timeMinusOneHour = time.minusHours(1);
System.out.println(time);              // 15:30
System.out.println(timePlusOneHour);   // 16:30
System.out.println(timeMinusOneHour);  // 14:30
```

OPERACIONES CON `LocalDateTime`

`LocalDateTime`, al ser una clase compuesta por `LocalDate` y `LocalTime` ofrece los mismos métodos para realizar operaciones.

(Debemos importar la clase `java.time.temporal.ChronoUnit` para poder utilizar las unidades: `ChronoUnit.YEARS`, `ChronoUnit.MONTHS`, `ChronoUnit.DAYS`, `ChronoUnit.HOURS`, `ChronoUnit.MINUTES`, `ChronoUnit.SECONDS`, `ChronoUnit.NANOS`).

Ejemplo:

```
LocalDateTime dateTime = LocalDateTime.of(2016, 7, 28, 14, 30);
LocalDateTime dateTime1 = dateTime.plus(1, ChronoUnit.DAYS).plus(1, ChronoUnit.HOURS); LocalDateTime dateTime2 = dateTime.minus(1,
ChronoUnit.DAYS).minus(1, ChronoUnit.HOURS);
System.out.println(dateTime);  // 2016-07-28T14:30
System.out.println(dateTime1); // 2016-07-29T15:30
System.out.println(dateTime2); // 2016-07-27T13:30
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

En el siguiente ejemplo, se hace uso de `Period` y `Duration`:

```
LocalDateTime dateTime = LocalDateTime.of(2016, 7, 28, 14, 30);
LocalDateTime dateTime1 = dateTime.plus(Period.ofDays(1)).plus(Duration.ofHours(1));
LocalDateTime dateTime2 = dateTime.minus(Period.ofDays(1)).minus(Duration.ofHours(1));
System.out.println(dateTime);  // 2016-07-28T14:30
System.out.println(dateTime1); // 2016-07-29T15:30
System.out.println(dateTime2); // 2016-07-27T13:30
```

Finalmente, haciendo uso de los métodos `plusX(long xToAdd)` o `minusX(long xToSubtract)`:

```
LocalDateTime dateTime = LocalDateTime.of(2016, 7, 28, 14, 30);
LocalDateTime dateTime1 = dateTime.plusDays(1).plusHours(1);
LocalDateTime dateTime2 = dateTime.minusDays(1).minusHours(1);
System.out.println(dateTime);  // 2016-07-28T14:30
System.out.println(dateTime1); // 2016-07-29T15:30
System.out.println(dateTime2); // 2016-07-27T13:30
```

Además, métodos como `isBefore`, `isAfter`, `isEqual` están disponibles para comparar las siguientes clases `LocalDate`, `LocalTime` y `LocalDateTime`.

Ejemplo:

```
LocalDate date1 = LocalDate.of(2016, 7, 28);
LocalDate date2 = LocalDate.of(2016, 7, 29);
boolean isBefore = date1.isBefore(date2); //true
boolean isAfter = date2.isAfter(date1); //true
boolean isEqual = date1.isEqual(date2); //false
```

Formatos

Cuando se trabaja con fechas, en ocasiones se requiere de un formato personalizado. Podemos usar el método `ofPattern(String pattern)`, para definir un formato en particular.

Para utilizar `DateTimeFormatter.ofPattern` debemos importar la clase con `import java.time.format.DateTimeFormatter;`

Ejemplo:

```
LocalDate mifecha = LocalDate.of(2016, 7, 25);
String fechaTexto=mifecha.format(DateTimeFormatter.ofPattern("eeee',' dd 'de' MMMM 'del' u"));
System.out.println("La fecha es: "+fechaTexto); // La fecha es: lunes, 25 de julio del 2016
```

El patrón del formato se realiza en función a la siguiente tabla de símbolos:

Símbolo	Descripción	Salida
y	Año	2004; 04
D	Día del Año	189
M	Mes del Año	7; 07; Jul; July; J
d	Día del Mes	10
w	Semana del Año	27
E	Día de la Semana	Tue; Tuesday; T
F	Semana del Mes	3
a	AM/PM	PM
K	Hora AM/PM (0-11)	0
H	Hora del día (0-23)	0
m	Minutos de la hora	30
s	Segundos del minuto	55
n	Nanosegundos del Segundo	987654321
''	Texto	'Día de la semana'

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

DÍA DE LA SEMANA

La función `getDayOfWeek()` devuelve un elemento del tipo `DayOfWeek` que corresponde el día de la semana de una fecha. Debemos importar la clase `java.time.DayOfWeek`.

Por ejemplo, el lunes será `DayOfWeek.MONDAY`.

Ejemplo:

```
LocalDate lafecha = LocalDate.of(2016, 7, 25);
if (lafecha.getDayOfWeek().equals(DayOfWeek.SATURDAY)) {
    System.out.println("La fecha es Sábado");
} else {
    System.out.println("La fecha NO es Sábado");
}
//La fecha NO es Sábado
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

6.3.2 Ejemplo Anexo UD05

Anexo2Date

```java
package UD05;

import java.util.Calendar;
import java.util.Date;
import java.util.GregorianCalendar;
import java.time.*;
import java.time.format.DateTimeFormatter;
import java.time.temporal.ChronoUnit;

public class Anexo2Date {

    public static void main(String[] args) {

        //Clase Date (java.util.Date)
        Date fecha = new Date(2021, 8, 19);
        System.out.println(fecha);          //Mon Sep 19 00:00:00 CEST 3921
        System.out.println(fecha.getTime()); //61590146400000

        //Clase GregorianCalendar (java.util.Calendar y java.util.GregorianCalendar)
        Calendar calendar = new GregorianCalendar(2021, 8, 19);
        System.out.println(calendar.getTime()); //Sun Sep 19 00:00:00 CEST 2021

        Date d = new Date();
        GregorianCalendar c = new GregorianCalendar();
        System.out.println("Fecha: " + d);  //Fecha: Thu Aug 19 20:06:14 CEST 2021
        System.out.println("Info: " + c); //Info: java.util.GregorianCalendar[time=1629396374723,
        //areFieldsSet=true,areAllFieldsSet=true,lenient=true,zone=sun.util.calendar.ZoneInfo
        //[id="Europe/Madrid",offset=3600000,dstSavings=3600000,useDaylight=true,transitions=163,
        //lastRule=java.util.SimpleTimeZone[id=Europe/Madrid,offset=3600000,dstSavings=3600000,
        //useDaylight=true,startYear=0,startMode=2,startMonth=2,startDay=-1,startDayOfWeek=1,
        //startTime=3600000,startTimeMode=2,endMode=2,endMonth=9,endDay=-1,endDayOfWeek=1,
        //endTime=3600000,endTimeMode=2]],firstDayOfWeek=2,minimalDaysInFirstWeek=4,ERA=1,
        //YEAR=2021,MONTH=7,WEEK_OF_YEAR=33,WEEK_OF_MONTH=3,DAY_OF_MONTH=19,DAY_OF_YEAR=231,
        //DAY_OF_WEEK=5,DAY_OF_WEEK_IN_MONTH=3,AM_PM=1,HOUR=8,HOUR_OF_DAY=20,MINUTE=6,SECOND=14,
        //MILLISECOND=723,ZONE_OFFSET=3600000,DST_OFFSET=3600000]
        c.setTime(d);
        System.out.print(c.get(Calendar.DAY_OF_MONTH));
        System.out.print("/");
        System.out.print(c.get(Calendar.MONTH) + 1);
        System.out.print("/");
        System.out.println(c.get(Calendar.YEAR) + 1); //19/8/2022

        //LocalDate, LocalTime, LocalDateTime, Duration y Period (java.time.*)
        //LocalDate d = new LocalDate(); //NO compila
        LocalDate date = LocalDate.of(1989, 11, 11); //1989-11-11
        System.out.println(date.getYear()); //1989
        System.out.println(date.getMonth()); //NOVEMBER
        System.out.println(date.getDayOfMonth()); //11
        date = LocalDate.now();
        System.out.println(date); //2021-08-19

        LocalTime time = LocalTime.of(5, 30, 45, 35); //05:30:45:35
        System.out.println(time.getHour()); //5
        System.out.println(time.getMinute()); //30
        System.out.println(time.getSecond()); //45
        System.out.println(time.getNano()); //35
        time = LocalTime.now();
        System.out.println(time); //20:13:53.118044

        LocalDateTime dateTime = LocalDateTime.of(1989, 11, 11, 5, 30, 45, 35);

        LocalDate date2 = LocalDate.of(1989, 11, 11);
        LocalTime time2 = LocalTime.of(5, 30, 45, 35);
        LocalDateTime dateTime1 = LocalDateTime.of(date, time);
        LocalDateTime dateTime2 = LocalDateTime.now();

        LocalTime localTime1 = LocalTime.of(12, 25);
        LocalTime localTime2 = LocalTime.of(17, 35);
        Duration duration1 = Duration.between(localTime1, localTime2);
        System.out.println(duration1); //PT5H10M
        System.out.println(duration1.toDays()); //0

        LocalDateTime localDateTime1 = LocalDateTime.of(2016, Month.JULY, 18, 14, 13);
        LocalDateTime localDateTime2 = LocalDateTime.of(2016, Month.JULY, 20, 12, 25);
        Duration duration2 = Duration.between(localDateTime1, localDateTime2);
        System.out.println(duration2); //PT46H12M
        System.out.println(duration2.toDays()); //1

        Duration duracion3 = Duration.ofDays(1);
        System.out.println(duracion3); //PT24H
        System.out.println(duracion3.toDays()); //1

        LocalDate localDate1 = LocalDate.of(2016, 7, 18);
        LocalDate localDate2 = LocalDate.of(2016, 7, 20);
        Period periodo1 = Period.between(localDate1, localDate2);
        System.out.println(periodo1); //P2D
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
        Period periodo2 = Period.of(1, 2, 3);
        System.out.println(periodo2); //P1Y2M3D

        Period periodo3 = Period.ofYears(1);
        System.out.println(periodo3); //P1Y

        //CHRONOUNIT (java.time.temporal.ChronoUnit)
        LocalDate fechaInicio = LocalDate.of(2016, 7, 18);
        LocalDate fechaFin = LocalDate.of(2016, 7, 20);
        // Calculamos el tiempo transcurrido entre las dos fechas
        // con la clase ChronoUnit y la unidad temporal en la que
        // queremos que nos lo devuelva, en este caso DAYS.
        long tiempo = ChronoUnit.DAYS.between(fechaInicio, fechaFin);
        System.out.println(tiempo); //2

        //Introducir fecha por teclado (java.time.format.DateTimeFormatter)
        DateTimeFormatter formato = DateTimeFormatter.ofPattern("d/MM/yyyy");
        String fechaCadena = "16/08/2016";
        LocalDate mifecha = LocalDate.parse(fechaCadena, formato);
        System.out.println(formato.format(mifecha)); //16/08/2016

        //Manipulación
        LocalDate fec = LocalDate.of(2016, 7, 25);
        LocalDate fec1 = fec.withYear(2017);
        LocalDate fec2 = fec.withMonth(8);
        LocalDate fec3 = fec.withDayOfMonth(27);
        System.out.println(date);  //2016-07-25
        System.out.println(fec1); //2017-07-25
        System.out.println(fec2); //2016-08-25
        System.out.println(fec3); //2016-07-27

        LocalTime tim = LocalTime.of(14, 30, 35);
        LocalTime tim1 = tim.withHour(20);
        LocalTime tim2 = tim.withMinute(25);
        LocalTime tim3 = tim.withSecond(23);
        LocalTime tim4 = tim.withNano(24);
        System.out.println(tim);  //14:30:35
        System.out.println(tim1); //20:30:35
        System.out.println(tim2); //14:25:35
        System.out.println(tim3); //14:30:23
        System.out.println(tim4); //14:30:35.000000024

        LocalDateTime dateTim = LocalDateTime.of(2016, 7, 25, 22, 11, 30);
        LocalDateTime dateTim1 = dateTim.withYear(2017);
        LocalDateTime dateTim2 = dateTim.withMonth(8);
        LocalDateTime dateTim3 = dateTim.withDayOfMonth(27);
        LocalDateTime dateTim4 = dateTim.withHour(20);
        LocalDateTime dateTim5 = dateTim.withMinute(25);
        LocalDateTime dateTim6 = dateTim.withSecond(23);
        LocalDateTime dateTim7 = dateTim.withNano(24);
        System.out.println(dateTim);  //2016-07-25T22:11:30
        System.out.println(dateTim1); //2017-07-25T22:11:30
        System.out.println(dateTim2); //2016-08-25T22:11:30
        System.out.println(dateTim3); //2016-07-27T22:11:30
        System.out.println(dateTim4); //2016-07-25T20:11:30
        System.out.println(dateTim5); //2016-07-25T22:25:30
        System.out.println(dateTim6); //2016-07-25T22:11:23
        System.out.println(dateTim7); //2016-07-25T22:11:30.000000024

        //OPERACIONES
        LocalDate date3 = LocalDate.of(2016, 7, 18);
        LocalDate date3PlusOneDay = date3.plus(1, ChronoUnit.DAYS);
        LocalDate date3MinusOneDay = date3.minus(1, ChronoUnit.DAYS);
        System.out.println(date3);        // 2016-07-18
        System.out.println(date3PlusOneDay);  // 2016-07-19
        System.out.println(date3MinusOneDay); // 2016-07-17

        LocalDate date4 = LocalDate.of(2016, 7, 18);
        LocalDate date4PlusOneDay = date4.plus(Period.ofDays(1));
        LocalDate date4MinusOneDay = date4.minus(Period.ofDays(1));
        System.out.println(date4);        // 2016-07-18
        System.out.println(date4PlusOneDay);  // 2016-07-19
        System.out.println(date4MinusOneDay); // 2016-07-17

        LocalDate date5 = LocalDate.of(2016, 7, 18);
        LocalDate date5PlusOneDay = date5.plusDays(1);
        LocalDate date5MinusOneDay = date5.minusDays(1);
        System.out.println(date5);        // 2016-07-18
        System.out.println(date5PlusOneDay);  // 2016-07-19
        System.out.println(date5MinusOneDay); // 2016-07-17

        LocalTime time3 = LocalTime.of(15, 30);
        LocalTime time3PlusOneHour = time3.plus(1, ChronoUnit.HOURS);
        LocalTime time3MinusOneHour = time3.minus(1, ChronoUnit.HOURS);
        System.out.println(time3);      // 15:30
        System.out.println(time3PlusOneHour);   // 16:30
        System.out.println(time3MinusOneHour);  // 14:30
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
        LocalTime time4 = LocalTime.of(15, 30);
        LocalTime time4PlusOneHour = time4.plus(Duration.ofHours(1));
        LocalTime time4MinusOneHour = time4.minus(Duration.ofHours(1));
        System.out.println(time4);      // 15:30
        System.out.println(time4PlusOneHour);   // 16:30
        System.out.println(time4MinusOneHour);  // 14:30

        LocalTime time5 = LocalTime.of(15, 30);
        LocalTime time5PlusOneHour = time5.plusHours(1);
        LocalTime time5MinusOneHour = time5.minusHours(1);
        System.out.println(time5);      // 15:30
        System.out.println(time5PlusOneHour);   // 16:30
        System.out.println(time5MinusOneHour);  // 14:30

        LocalDateTime dateTime3 = LocalDateTime.of(2016, 7, 28, 14, 30);
        LocalDateTime dateTime4 = dateTime3.plus(1, ChronoUnit.DAYS).plus(1, ChronoUnit.HOURS);
        LocalDateTime dateTime5 = dateTime3.minus(1, ChronoUnit.DAYS).minus(1, ChronoUnit.HOURS);
        System.out.println(dateTime3); // 2016-07-28T14:30
        System.out.println(dateTime4); // 2016-07-29T15:30
        System.out.println(dateTime5); // 2016-07-27T13:30

        LocalDateTime dateTime6 = LocalDateTime.of(2016, 7, 28, 14, 30);
        LocalDateTime dateTime7 = dateTime6.plus(Period.ofDays(1)).plus(Duration.ofHours(1));
        LocalDateTime dateTime8 = dateTime6.minus(Period.ofDays(1)).minus(Duration.ofHours(1));
        System.out.println(dateTime6); // 2016-07-28T14:30
        System.out.println(dateTime7); // 2016-07-29T15:30
        System.out.println(dateTime8); // 2016-07-27T13:30

        LocalDateTime dateTime9 = LocalDateTime.of(2016, 7, 28, 14, 30);
        LocalDateTime dateTime10 = dateTime9.plusDays(1).plusHours(1);
        LocalDateTime dateTime11 = dateTime9.minusDays(1).minusHours(1);
        System.out.println(dateTime9);  // 2016-07-28T14:30
        System.out.println(dateTime10); // 2016-07-29T15:30
        System.out.println(dateTime11); // 2016-07-27T13:30

        LocalDate dat1 = LocalDate.of(2016, 7, 28);
        LocalDate dat2 = LocalDate.of(2016, 7, 29);
        boolean isBefore = dat1.isBefore(dat2); //true
        boolean isAfter = date2.isAfter(dat1); //true
        boolean isEqual = dat1.isEqual(dat2); //false

        //Formatos (java.time.format.DateTimeFormatter)
        LocalDate mifecha2 = LocalDate.of(2016, 7, 25);
        String fechaTexto = mifecha2.format(DateTimeFormatter.
                                            ofPattern("eeee',' dd 'de' MMMM 'del' yyyy"));
        System.out.println("La fecha es: " +
                           fechaTexto); // La fecha es: lunes, 25 de julio del 2016

        //DAYOFWEEK
        LocalDate lafecha = LocalDate.of(2016, 7, 25);
        if (lafecha.getDayOfWeek().equals(DayOfWeek.SATURDAY)) {
            System.out.println("La fecha es Sábado");
        } else {
            System.out.println("La fecha NO es Sábado");
        }
        //La fecha NO es Sábado
    }
}
```

12 de enero de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

6.4 Ejercicios de la UD05

6.4.1 Ejercicios

Paquete: `UD05._1_1gestionEmpleados`

1. Una empresa quiere hacer una gestión informatizada básica de sus empleados. Para ello, de cada empleado le interesa:
   - Nombre (String)
   - DNI (String)
   - Año de ingreso (número entero)
   - Sueldo bruto anual (número real)
2. (Empleado) Diseñar una clase Java `Empleado`, que contenga los atributos (privados) que caracterizan a un empleado e implemente los métodos adecuados para:
   - Crear objetos de la clase: **Constructor** que reciba todos los datos del empleado a crear.
   - Consultar el valor de cada uno de sus atributos. (**Consultores** o **getters**)
   - `public int antiguedad()`. Devuelve el número de años transcurridos desde el ingreso del empleado en la empresa. Si el año de ingreso fuera posterior al de la fecha actual, devolverá 0. Para obtener el año actual puedes usar:
   - `java int anyoActual = Calendar.getInstance().get(Calendar.YEAR);`
   - `public void incrementarSueldo(double porcentaje)`. Incrementa el sueldo del empleado en un porcentaje dado (expresado como una cantidad real entre 0 y 100).
   - `public String toString()`. Devuelve un `String` con los datos del empleado, de la siguiente forma:

     ```
     Nombre: Juan González
     Dni: 545646556K
     Año de ingreso: 1998
     Sueldo bruto anual: 20000 €
     ```

   - `public boolean equals(Object o)`. Método para comprobar si dos empleados son iguales. Dos empleados se consideran iguales si tienen el mismo DNI.
   - `public int compareTo(Empleado o)`. Se considera menor o mayor el empleado que tiene menor o mayor DNI (el mismo criterio que al comparar dos strings).
   - Método estático `public static double calcularIRPF(double salario)`. Determina el % de IRPF que corresponde a un salario (mensual) determinado, según la siguiente tabla:

Desde salario (incluido)	Hasta salario (no incluido)	% IRPF
0	800	3
800	1000	10
1000	1500	15
1500	2100	20
2100	infinito	30

3. (TestEmpleado) Diseñar una clase Java `TestEmpleado` que permita probar la clase `Empleado` y sus métodos. Para ello se desarrollará el método `main` en el que:
   - Se crearán dos empleados utilizando los datos que introduzca el usuario.
   - Se incrementará el sueldo un 20 % al empleado que menos cobre.
   - Se incrementará el sueldo un 10% al empleando más antiguo.
   - Muestra el IRPF que correspondería a cada empleado.
   - Para comprobar que las operaciones se realizan correctamente, muestra los datos de los empleados tras cada operación.
4. (Empresa) Diseñar una clase `Empresa`, que permita almacenar el nombre de la empresa y la información de los empleados de la misma (máximo 10 empleados) en un array. Para ello, se utilizarán tres atributos: `nombre`, `plantilla` (array de empleados) y `numEmpleados` (número de empleados que tiene la empresa). En esta clase, se deben implementar los métodos:
   - `public Empresa (String nombre)`. Constructor de la clase. Crea la empresa con el nombre indicado y sin empleados.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- `public void contratar(Empleado e) throws PlantillaCompletaException`. Añade el empleado indicado a la plantilla de la empresa, siempre que quepa en el array. Si no cabe, se lanzará la excepción `PlantillaCompletaException`.
- `public void despedir(Empleado e) throws ElementoNoEncontradoException`. Elimina el empleado indicado de la plantilla. Si no existe en la empresa, se lanza `ElementoNoEncontradoException`.
- `public void subirTrienio (double porcentaje)` Subir el sueldo, en el porcentaje indicado, a todos los empleados cuya antigüedad sea exactamente tres años.
- `public String toString()`. Devuelve un `String` con el nombre de la empresa y la información de todos los empleados. La información de los distintos empleados debe estar separada por saltos de línea.

5. (TestEmpresa) Diseñar una clase Java `TestEmpresa` que permita probar la clase `Empresa` y sus métodos. Para ello, desarrolla el método `main` y en él ...:
   - Crea una empresa, de nombre "DAMCarlet".
   - Contrata a varios empleados (con el nombre, DNI, etc. que quieras).
   - Usa el método `subirTrienio` para subir un 10% el salario de los empleados que cumplen un trienio en el año actual.
   - Despide a alguno de los empleados.
   - Trata de despedir a algún empleado que no exista en la empresa.
   - Muestra los datos de la empresa siempre que sea necesario para comprobar que las operaciones se realizan de forma correcta.

Paquete: `UD05._1_2gestionHospital`

1. Se desea realizar una aplicación para gestionar el ingreso y el alta de pacientes de un hospital. Una de las clases que participará en la aplicación será la clase `Paciente`, que se detalla a continuación :
2. (Paciente) La clase `Paciente` permite representar un paciente mediante los atributos: `nombre` (cadena), `edad` (entero), `estado` (entero entre 1 -más grave- y 5 -menos grave-, 6 si está curado), y con las siguientes operaciones:
   - `public Paciente (String n, int e)`. Constructor de un objeto `Paciente` de nombre `n`, de `e` años y cuyo estado es un valor aleatorio entre 1 y 5.
   - `public Paciente (String n, int e, int estado)`. Constructor de un objeto `Paciente` con el estado indicado (útil para probar la clase).
   - `public int getEdad()`. Consultor que devuelve edad.
   - `public int getEstado()`. Consultor que devuelve estado.
   - `public void mejorar()`. Modificador que incrementa en uno el estado del paciente (mejora al paciente)
   - `public void empeorar()`. Modificador que decrementa en uno el estado del paciente (empeora al paciente)
   - `public String toString()`. Transforma el paciente en un `String`. Por ejemplo,

     ```
     Pepe Pérez 46 5
     ```

   - `public int compareTo(Paciente o)`. Permite comparar dos pacientes. Se considera menor el paciente más leve. A igual gravedad, se considera menor el paciente más joven. Ejemplo:
   - Teniendo a `David 40 3`, `Pepe 25 3` y `Juan 35 5`:

     ```
     David.compareTo(Juan) = 2
     Juan.compareTo(Pepe) = -2
     David.compareTo(Pepe) = 15
     ```

3. (TestPaciente) Diseñar una clase Java `TestPaciente` que permita probar la clase `Paciente` y sus métodos. Para ello se desarrollará el método `main` en el que:
   - Se crearán dos pacientes: "Antonio" de 20 años y "Miguel" de 30 años.
   - Imprimir el estado inicial de los dos pacientes.
   - Mostrar los datos del que se considere menor (según el criterio de `compareTo` de la clase `Paciente`).
   - Aplicar "mejoras" al paciente más grave hasta que los dos pacientes tengan el mismo estado.
   - Imprimir el estado final de los dos pacientes.
4. (Hospital) La clase **Hospital** contiene la información de las camas de un hospital, así como de los pacientes que las ocupan. Un Hospital tiene un número máximo de camas `MAXC` = 200 y para representarlas se utilizará un array (llamado `listaCamas`) de objetos de tipo Paciente junto con un atributo (`numLibres`) que indique el número de camas libres del hospital en un momento dado. El número de cada cama coincide con su posición en el array de pacientes (la posición 0 no se utiliza), de

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

manera que `listaCamas[i]` es el Paciente que ocupa la cama `i` o es `null` si la cama está libre. Las operaciones de esta clase son:

- `public Hospital()`. Constructor de un hospital. Cuando se crea un hospital, todas las camas están libres.
- `public int getNumLibres()`. Consultor del número de camas libres.
- `public boolean hayLibres()`. Devuelve true si en el hospital hay camas libres y devuelve false en caso contrario.
- `public int primeraLibre()`. Devuelve el número de la primera cama libre del array `listaCamas` si hay camas libres o devuelve un 0 si no las hay.
- `public void ingresarPaciente(String n, int e) throws HospitalLlenoException` Si hay camas libres, la primera de ellas (la de número menor) pasa a estar ocupada por el paciente de nombre `n` y edad `e`. Si no hay camas libres, lanza una excepción.
- La excepción se declara dentro de la clase `Hospital`, como clase anidada: `static class HospitalLlenoException extends Exception`. Fuera de `Hospital` se usa como `Hospital.HospitalLlenoException`.
- `private void darAltaPaciente(int i)`. La cama `i` del hospital pasa a estar libre. (Afectará al número de camas libres)
- `public void darAltas()`. Se mejora el estado (método `mejorar()` de `Paciente`) de cada uno de los pacientes del hospital y a aquellos pacientes sanos (cuyo estado es 6) se les da el alta médica (invocando al método `darAltaPaciente`).
- `public String toString()`. Devuelve un `String` con la información de las camas del hospital. Por ejemplo,

```
1 María Medina 30 4
2 Pepe Pérez 46 5
3 libre
4 Juan López 50 1
5 libre
6 ...
7 199 Andrés Sánchez 29 3
```

5. (GestorHospital) En la clase `GestorHospital` se probará el comportamiento de las clases anteriores. El programa deberá:
   - Crear un hospital.
   - Ingresar a cinco pacientes con los datos simulados introducidos directamente en el programa.
   - Realizar el proceso de `darAltas` mientras que el número de habitaciones libres del hospital no llegue a una cantidad (por ejemplo 198).
   - Mostrar los datos del hospital cuando se considere oportuno para comprobar la corrección de las operaciones que se hacen.

Paquete: `UD05._1_3contrarreloj`

1. Se quiere realizar una aplicación para registrar las posiciones y tiempos de llegada en una carrera ciclista contrarreloj.
2. (Corredor) La clase `Corredor` representa a un participante en la carrera. Sus atributos son el dorsal (entero), el nombre (String) y el tiempo en segundos (double) que le ha costado completar el recorrido. Los métodos con los que cuenta son:
   - `public Corredor(int d, String n)`. Constructor a partir del dorsal y el nombre. Por defecto el tiempo tardado es 0
   - `public double getTiempo()`. Devuelve el tiempo tardado por el corredor
   - `public int getDorsal()`. Devuelve el dorsal del corredor
   - `public String getNombre()`. Devuelve el nombre del corredor
   - `public void setTiempo(double t) throws IllegalArgumentException`. Establece el tiempo tardado por el corredor. Lanzará la excepción si el tiempo indicado es negativo.
   - `public void setTiempo(double t1, double t2) throws IllegalArgumentException`. Establece el tiempo tardado por el corredor. `t1` indica la hora de comienzo y `t2` la hora de finalización (expresadas en segundos). La diferencia en segundos entre los dos datos servirá para establecer el tiempo tardado por el `Corredor`. Lanzará la excepción si el tiempo resultante es negativo
   - `public String toString()`. Devuelve un String con los datos del corredor, de la forma:

```
1 (234) - Juan Ramirez - 2597 segundos
```

   - `public boolean equals(Object o)`. Devuelve true si los corredores tienen el mismo dorsal y false en caso contrario
   - `public int compareTo (Corredor o)`. Un corredor es menor que otro si tiene menor dorsal.
   - `public static int generarDorsal()`. Devuelve un número de dorsal generado secuencialmente. Para ello la clase hará uso de un atributo `static int siguienteDorsal` que incrementará cada vez que se genere un nuevo dorsal.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

3. (TestCorredor) Diseñar una clase Java `TestCorredor` que permita probar la clase Corredor y sus métodos. Para ello se desarrollará el método `main` en el que:
   - Se crearán dos corredores: El nombre lo indicará el usuario mientras que el dorsal se generará utilizando el método `generarDorsal()` de la clase.
   - Se establecerá el tiempo de llegada del primer corredor a 300 segundos y el del segundo a 400.
   - Se mostrarán los datos de ambos corredores (`toString`)
4. (ListaCorredores) La clase `ListaCorredores` permite representar a un conjunto de corredores. En la lista, como máximo habrá 200 corredores, aunque puede haber menos de ese número. Se utilizará un array, llamado lista, de 200 elementos junto con una propiedad `numCorredores` que permita saber cuantos corredores hay realmente. Métodos:
   - `public ListaCorredores()` . Constructor. Crea la lista de corredores, incicialmente vacía.
   - `public void anyadir(Corredor c) throws ElementoDuplicadoException` . Añade un corredor al final de la lista de corredores (pero lo más al principio posible del array), siempre y cuando el corredor no esté ya en la lista, en cuyo caso se lanzará `ElementoDuplicadoException`
   - `public void insertarOrdenado(Corredor c)` . Inserta un corredor en la posición adecuada de la lista de manera que esta se mantenga ordenada crecientemente por el tiempo de llegada. Para poder realizar la inserción debe averiguarse la posición que debe ocupar el nuevo elemento y, antes de añadirlo al array, desplazar el elemento que ocupa esa posición y todos los posteriores, una posición a la derecha.
   - `public Corredor quitar(int dorsal) throws ElementoNoEncontradoException` . Quita de la lista al corredor cuyo dorsal se indica. El array debe mantenerse compacto, es decir, todos los elementos posteriores al eliminado deben desplazarse una posición a la izquierda. El método devuelve el Corredor quitado de la lista. Si no se encuentra se lanza `ElementoNoEncontradoException` .
   - Las excepciones se declaran dentro de la clase `ListaCorredores` , como clases anidadas: `static class ElementoDuplicadoException extends Exception` y `static class ElementoNoEncontradoException extends Exception` . Fuera de la lista se usan como `ListaCorredores.ElementoDuplicadoException` , etc.
   - `public String toString()` Devuelve un `String` con la información de la lista de corredores. Los minutos apareceran formateados con 2 decimales. Por ejemplo:

```
Posición: 0
Dorsal: 234
Nombre: Juan Ramirez
Tiempo: 25.97 minutos

Posición: 1
Dorsal: 26
Nombre: José González
Tiempo: 29.70 minutos
```

5. (Clase `ContraReloj` , te la doy resuelta y comentada en tu repositorio: descoméntala cuando tengas `Corredor` y `ListaCorredores` . No tiene puntos) Programa que simula una contrarreloj. Para llevar el control de una carrera contrarreloj se mantienen dos listas de corredores (dos objetos de tipo `ListaCorredores`):
   - (`hanSalido`) Una con los que han salido, que tiene a los corredores por orden de salida. El atributo tiempo de estos corredores será 0. Para que los corredores se mantengan por orden de salida, se añadiran a la lista utilizando el método añadir.
   - (`hanLlegado`) Otra con los corredores que hay llegado a la meta. A medida que los corredores llegan a la meta se les extrae de la primera lista, se les asigna un tiempo y se les inserta ordenadamente en esta segunda lista.
   - En el método `main` realizar un programa que muestre un menú con las siguientes opciones:
     a. `Salida` : Para registrar que una corredor ha comenzado la contrarreloj y sale de la línea de salida. Solicita al usuario el nombre de un corredor y su dorsal, y lo añade a la lista de corredores que han salido.
     b. `Llegada` : Para registrar que un corredor ha llegado a la meta. Solicita al usuario el dorsal de un corredor y el tiempo de llegada (en segundos). Quita al corredor de la lista de corredores que `hanSalido` , le asigna el tiempo que ha tardado y lo inserta (ordenadamente) en la lista de corredores que `hanLlegado`
     c. `Clasificación` : Muestra la lista de corredores que `hanLlegado` . Dado que esta lista está ordenada por tiempo, mostrarla por pantalla nos da la clasificación.
     d. `Salir` : Sale del programa

Paquete: `UD05._1_4reservasLibreria`

1. Una librería quiere proporcionar a sus clientes el siguiente servicio:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Cuando un cliente pide un libro y la librería no lo tiene, el cliente puede hacer una reserva de manera que cuando lo reciban en la librería le avisen por teléfono.

De cada reserva se almacena:

- `Nif` del cliente (`String`)
- `Nombre` del cliente (`String`)
- `Teléfono` del cliente (`String`)
- `Código` del libro reservado. (`entero`)
- Numero de `ejemplares` (`entero`)

2. (Reserva) Diseñar la clase `Reserva` , de manera que contemple la información descrita e implementar:
   - `public Reserva(String nif, String nombre, String tel, int codigo, int ejemplares)` . Constructor que recibe todos los datos de la reserva.
   - `public Reserva(String nif, String nombre, String tel, int codigo)` . Constructor que recibe los datos del cliente y el código del libro. Establece el número de ejemplares a uno.
   - Consultores de todos los atributos.
   - `public void setEjemplares(int ejemplares)` . Modificador del número de ejemplares. Establece el número de ejemplares al valor indicado como parámetro.
   - `public String toString()` que devuelva un `String` con los datos de la reserva
   - `public boolean equals(Object o)` . Dos reservas son iguales si son del mismo cliente y reservan el mismo libro.
   - `public int compareTo(Object o)` . Es menor la reserva cuyo código de libro es menor. El parámetro es de tipo `Object` así que revisa si debes hacer alguna "adaptación".
3. (TestReservas) Diseñar una clase Java `TestReservas` que permita probar la clase `Reserva` y sus métodos. Para ello se desarrollará el método `main` en el que:
   - Se creen dos reservas con los datos que introduce el usuario. Las reservas no pueden ser iguales (equals). Si la segunda reserva es igual a la primera se pedirá de nuevo los datos de la segunda al usuario.
   - Se incremente en uno el número de ejemplares de ambas reservas.
   - Se muestre la menor y a continuación la mayor.
   - Mostrar el listado de reservas cada vez que consideres oportuno.
4. (ListaReservas) Diseñar una clase `ListaReservas` que implemente una lista de reservas. Como máximo puede haber 100 reservas en la lista. Se utilizará un array de Reservas que ocuparemos a partir de la posición 0 y un atributo que indique el número de reservas. Las reservas existentes ocuparán las primeras posiciones del array (sin espacios en blanco). Cada objeto `ListaReservas` es una lista independiente: el array y el contador son atributos de instancia (no `static`), y se crea con un constructor sin parámetros (`new ListaReservas()`), que deja la lista vacía. Implementar los siguientes métodos (de instancia):
   - `public void reservar (String nif, String nombre, String telefono, int libro, int ejemplares) throws ListaLlenaException, ElementoDuplicadoException` : Crea una reserva y la añade a la lista. Lanza `ElementoDuplicadoException` si la reserva ya estaba en la lista. Lanza `ListaLlenaException` si la lista de reservas está llena.
   - `public void cancelar (String nif, int libro) throws ElementoNoEncontradoException` . Dado un nif de cliente y un código de libro, anular la reserva correspondiente. Lanzar `ElementoNoEncontradoException` si la reserva no existe.
   - Las excepciones se declaran dentro de la clase `ListaReservas` , como clases anidadas: `static class ListaLlenaException extends Exception` , `static class ElementoDuplicadoException extends Exception` y `static class ElementoNoEncontradoException extends Exception` . Fuera de la lista se usan como `ListaReservas.ListaLlenaException` , etc.
   - `public String toString()` : Devuelve un `String` con los datos de todas las reservas de la lista.
   - `public int numEjemplaresReservadosLibro (int codigo)` : Devuelve el número de ejemplares que hay reservados en total de un libro determinado.
   - `public void reservasLibro (int codigo)` : Dado un código de libro, muestra el nombre y el teléfono de todos los clientes que han reservado el libro.
5. (GestionReservas) Realizar un programa `GestionReservas` que, utilizando un menú, permita:
   - Realizar reserva. Permite al usuario realizar una reserva.
   - Anular reserva: Se anula la reserva que indique el usuario (Nif de cliente y código de libro).
   - Pedido: El usuario introduce un código de libro y el programa muestra el nº de reservas que se han hecho del libro. Esta opción de menú le resultará útil al usuario para poder hacer el pedido de un libro determinado.
   - Recepción: Cuando el usuario recibe un libro quiere llamar por teléfono a los clientes que lo reservaron. Solicitar al usuario un código de libro y mostrar los datos (nombre y teléfono) de los clientes que lo tienen reservado.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Paquete: `UD05._1_5gestorCorreoElectronico`

1. Queremos realizar la parte de un programa de correo electrónico que gestiona la organización de los mensajes en distintas carpetas. Para ello desarrollaremos:
2. (Mensaje) La clase `Mensaje`. De un mensaje conocemos:
   - `Codigo (int)` Número que permite identificar a los mensajes.
   - `Emisor (String)`: email del emisor.
   - `Destinatario (String)`: email del destinatario.
   - `Asunto (String)`
   - `Texto (String)`

   Desarrollar los siguientes métodos:
   - Constructor que reciba todos los datos, excepto el código, que se generará automáticamente (nº consecutivo. Ayuda: utiliza una variable de clase (`static`))
   - Consultores de todos los atributos.
   - `public boolean equals(Object o)`. Dos mensajes son iguales si tienen el mismo código.
   - `public static boolean validarEMail(String email)`: Método estático que devuelve true o false indicando si la dirección de correo indicada es válida o no. Una dirección es válida si tiene la forma `direccion@subdominio.dominio`
   - `public String toString()`
3. (TestCorreo) Con la clase `TestCorreo` probaremos las clases y métodos desarrollados.
   - Crea varios mensajes con los datos que introduzca el usuario y muéstralos por pantalla.
   - Prueba el método `validarEMail` de la clase Mensaje con las direcciones siguientes (solo la primera es correcta):
     - `tuCorreo@gmail.com`
     - `tuCorreogmail.com`
     - `tuCorreo@gmail`
     - `tuCorreo.com@gmail`
4. (Carpeta) La clase `Carpeta`, cada carpeta tiene un nombre y una lista de Mensajes. Para ello usaremos un array con capacidad para 100 mensajes y un atributo que indique el número de mensajes que contiene la carpeta. Además se implementarán los siguientes métodos:
   - `public Carpeta(String nombre)`: Constructor. Dado un nombre, crea la carpeta sin mensajes.
   - `public void anyadir(Mensaje m)`: Añade a la carpeta el mensaje indicado.
   - `public void borrar(Mensaje m) throws ElementoNoEncontradoException`: Borra de la carpeta el mensaje indicado. Lanza la excepción si el mensaje no existe.
   - `public Mensaje buscar(int codigo) throws ElementoNoEncontradoException`: Busca el mensaje cuyo código se indica. Si lo encuentra devuelve el mensaje, en caso contrario lanza la excepción.
   - `public String toString()` que devuelva un `String` con el nombre de la carpeta y sus mensajes
   - `public static void moverMensaje(Carpeta origen, Carpeta destino, int codigo) throws ElementoNoEncontradoException`: Método estático. Recibe dos Carpetas de correo y un código de mensaje y mueve el mensaje indicado de una carpeta a otra. Para ello buscará el mensaje en la carpeta origen. Si existe lo eliminará y lo añadirá a la carpeta de destino. Si el mensaje indicado no está en la carpeta de origen lanza `ElementoNoEncontradoException`.
5. (TestCarpetas) Con la clase `TestCarpetas` probaremos las clases y métodos desarrollados:
   - Crea dos carpetas de correo de nombre `Mensajes recibidos` y `Mensajes eliminados` respectivamente.
   - Crea varios mensajes y añádelos a `Mensajes recibidos`.
   - Mueve el mensaje de código 1 desde la `Mensajes recibidos` a `Mensajes elimiminados`.
   - Muestra el contenido de las carpetas antes y después de cada operación (añadir, mover,...)

Paquete: `UD05._1_6juegoDeCartas`

1. Se está desarrollando una aplicación que usa una baraja de cartas. Para ello, se implementarán en Java las clases necesarias.
2. (Carta) Una de ellas es la clase `Carta` que permite representar una carta de la baraja española. La información requerida para identificar una `Carta` es:
   - su `palo` (oros, copas, espadas o bastos) y
   - su `valor` (un entero entre 1 y 12).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Para dicha clase, se pide:

- Definir 4 constantes, atributos de clase (estáticos) públicos enteros, para representar cada uno de los palos de la baraja (`OROS` será el valor 0, `COPAS` el 1, `ESPADAS` el 2 y `BASTOS` el 3).
- Definir los atributos (privados): `palo` y `valor`.
- Escribir dos constructores: uno para construir una carta de forma aleatoria (sin parámetros) y otro para construir una carta de acuerdo a dos datos: su palo y su valor (si los datos son incorrectos se lanzará `IllegalArgumentException`)
- Escribir los métodos `consultores` y `modificadores` de los valores de los atributos.
- Escribir un método `compareTo` para comprobar si la carta actual es menor que otra carta dada. El criterio de ordenación es por palos (el menor es oros, después copas, a continuación espadas y, finalmente, bastos) y dentro de cada palo por valor (1, 2, ..., 12).
- Escribir un método `equals` para comprobar la igualdad de dos cartas. Dos cartas son iguales si tienen el mismo palo y valor.
- Escribir un método `sigPalo` para devolver una nueva carta con el mismo valor que el de la carta actual pero del palo siguiente, según la ordenación anterior y sabiendo que el siguiente al palo bastos es oros.
- Escribir un método `toString` para transformar en `String` la carta actual, con el siguiente formato: "valor de palo"; por ejemplo, "4 de oros" o "1 de bastos" (sobrescritura del método `toString` de `Object`).

3. (JuegoCartas) Implementar una clase `JuegoCartas` con los métodos siguientes:

   - Un método de clase (estático) `public static int ganadora( Carta c1, Carta c2, int triunfo)` que dados dos objetos `Carta` y un número entero representando el palo de triunfo (o palo ganador), determine cuál es la carta ganadora. El método debe devolver 0 si las dos cartas son iguales. En caso contrario, devolverá -1 cuando la primera carta es la ganadora y 1 si la segunda carta es la ganadora.
   - Para determinar la carta ganadora se aplicarán las siguientes reglas:
     - Si las dos cartas son del mismo palo, la carta ganadora es el as (valor 1) y, en el resto de casos, la carta ganadora es la de valor más alto (por ejemplo, "1 de oros" gana a "7 de oros", "5 de copas" gana a "2 de copas", "11 de bastos" gana a "7 de bastos").
     - Si las dos cartas son de palos diferentes:
       - Si el palo de alguna carta es el palo de triunfo, dicha carta es la ganadora.
       - En otro caso, la primera carta siempre gana a la segunda.
       - Un método `main` en el que se debe:
     - Crear una `Carta` aleatoriamente y mostrar sus datos por pantalla.
     - Generar aleatoriamente un entero en el rango [0..3] representando el palo de triunfo, y mostrar por pantalla a qué palo corresponde.
     - Crear una `Carta` a partir de un palo y un valor dados (solicitados al usuario desde teclado), y mostrar sus datos por pantalla.
     - Mostrar por pantalla la carta ganadora (invocando al método del apartado anterior con el objeto `Carta` del usuario.

6.4.2 Actividades

1. (WrapperDouble) Introducir por teclado un valor de tipo `double`, convertirlo en Wrapper e imprimirlo.
2. (StringAEntero) Introducir por teclado un valor numérico en un `String` y convertirlo en entero e imprimirlo.
3. (StringAWrapper) Introducir por teclado un valor numérico entero en un `String` y convertirlo en un `Wrapper` e imprimirlo.
4. (OperacionesBinarias) Introducir por teclado dos valores numéricos enteros y la operación que queremos realizar (`suma`, `resta` o `multiplicación`). Realizar la operación y mostrar el resultado en `Binario`, `Hexadecimal` y `Octal`.

   Ejemplo de ejecución:

```
Introduce el primer valor numérico: 14
Introduce el segundo valor numérico: 4
Introduce la operación (suma, resta, multiplicacion): resta
 EL RESULTADO:
 en binario: 1010
 en octal: 12
 en hexadecimal: a
```

5. (SegundosDesde1970) Mostrar los segundos transcurridos desde el `1 de Enero de 1970 a las 0:00:00` hasta `hoy`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

6. (FormatosFechaHora) Mostrar la `fecha` y `hora` de hoy con los siguientes formatos (para todos los ejemplos se supone que hoy es 26 de agosto de 2021 a las 17 horas 16 minutos y 8 segundos, tu deberas mostrar la fecha y hora de tu sistema en el momento de ejecución):
    - a) `August 26, 2021, 5:16 pm`
    - b) `08.26.21`
    - c) `26, 8, 2021`
    - d) `20210826`
    - e) `05-16-08, 26-08-21`
    - f) `Thu Aug 26 17:16:08`
    - g) `17:16:08`
7. (ValidarFecha) Introducir un día, un mes y un año y verificar si es una fecha correcta.

```
Introduce un dia para la fecha: 29
Introduce un mes para la fecha: 2
Introduce un año para la fecha: 2022
 LA FECHA ES INCORRECTA

Introduce un dia para la fecha: 29
Introduce un mes para la fecha: 2
Introduce un año para la fecha: 2020
LA FECHA ES CORRECTA
```

8. (DiasEntreFechas) Introducir dos fechas e indicar los días transcurridos entre las dos fechas.

```
Introduce la fecha inicial con formato dd/mm/yyyy: 01/02/2021
Introduce la fecha final con formato dd/mm/yyyy: 15/03/2022
La fecha inicial es: 1/2/2021
La fecha final es: 15/3/2022
 Entre la fecha inicial y la final hay un periodo de: P1Y1M14D
dias: 14
meses: 1
años: 1
```

9. (PagosPlazos) Introducir una fecha y devolver las fecha de los pagos a 30, 60 y 90 días.
10. (CompararFechas) Introducir tres fechas e indicar la mayor y a menor.
11. (FechaActualComparacion) Introducir el día, mes, año. Crear una fecha a partir de los datos introducidos y comprobar e indicar si se trata de la fecha actual, si es una fecha pasada o una fecha futura.
12. (EdadEmpleado) Introducir una fecha de nacimiento de un empelado e indicar cuántos años tiene el empleado.
13. (ProductoCaducado) Introducir la fecha de caducidad de un producto e indicar si el producto está o no caducado. El valor por defecto será la fecha actual y solo se podrán introducir fechas del año en curso.
14. (FormatoFechaCeros) Mostrar una fecha con formato dd/mm/aaaa utilizando 0 delante de los días o meses de 1 dígito.
15. (FormatoFechaExtendido) Mostrar una fecha con formato `DiaSemana`, `DiaMes` de `Mes` del `Año` a las `horas:minutos:segundos`. Por ejemplo: Miercoles, 9 de Diciembre del 2015 a las 18:45:32
16. (SumarFechaFutura) Suma 10 años, 4 meses y 5 días a la fecha actual.

```
Hoy es: dijous, 03 de març del 2022
Dentro de 10 años, 4 meses y 5 dias será: dijous, 08 de juliol del 2032
```

17. (RestarFechaPasada) Resta 5 años, 11 meses y 18 días a la fecha actual.
18. (PagoHorasExtra) Introducir el número de horas trabajadas por un empleado y la fecha en las que las trabajo. Si el día fue sábado o domingo el precio hora trabajada es 20€ en caso contrario 15€. Calcula la cantidad de dinero que habrá que pagar al empleado por las horas trabajadas.
19. (CalculoNomina) Introducir la fecha inicial y final de una nómina y calcular lo que debe cobrar el empleado sabiendo que cada día trabajado recibe 55 € y tiene una retención del 12% sobre el sueldo.
20. (ClaseAlumno) Crear una clase `Alumno` con los atributos `codigo`, `nombre`, `apellidos`, `fecha_nacimiento`, `calificacion`. La fecha de nacimiento deberá introducirse como una fecha. Crear constructor, métodos `setter` y `getter` y `toString`. Crear una instancia con los siguientes valores `1`, `'Luis'`, `'Mas Ros'`, `05/10/1990`, `7.5`. Mostrar los datos del alumno además de su edad.

```
Alumno{codigo=1, nombre=Luis, apellidos=Mas Ros, fecha=1990-10-05, calificacion=7.5, edad= 31}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

21. (PlazoEntrega) Introducir la fecha de entrega de un documento (con formato `dd/mm/yyyy`) y nos diga si está dentro o fuera de plazo teniendo en cuenta que la fecha de entrega límite es la fecha actual.
22. (RetencionesTrabajadores) Introducir en un array `nombre`, `apellidos` y `sueldo` de varios trabajadores y la `fecha de alta` en la empresa. Las fechas deberán introducirse como fechas. Recorrer el array y mostrar para cada trabajador la retención que debe aplicarse sobre el sueldo teniendo en cuenta que los trabajadores incorporados antes de 1980 tienen una retención del 20%, los trabajadores con fecha entre 1980 y 2000 una retención del 15% y los trabajadores con fecha posterior al 2000 la retención que aplicaremos será el 5% del sueldo.
    - La clase del trabajador tiene el constructor `(String nombre, String apellidos, double sueldo, LocalDate fechaAlta)` y su `toString` muestra la retención.
    - El programa pregunta primero cuántos trabajadores hay y después, para cada uno, su nombre, apellidos, sueldo y fecha de alta (`dd/mm/yyyy`).
23. (MayorEdad) Realiza un método estático `public static boolean esMayorEdad(LocalDate fechaNacimiento)` que dada la `fecha de nacimiento` de una persona indique si es mayor de edad.
24. (ClaseConversor) Realiza una clase `Conversor` que tenga las siguientes características: Toma como parámetro en el constructor un valor entero. Tiene un método `getNumero` que dependiendo del parámetro devolverá el mismo número (String) en el siguiente sistema de numeración: `B Binario`, `H Hexadecimal`, `O Octal`. Si el sistema indicado no es ninguno de estos, devuelve `"Error"`. Realiza un método `main` en la clase para probar todo lo anterior.
25. (ConversorFechas) Realiza una clase `ConversorFechas` que tenga los siguientes métodos:
    - `String normalToAmericano(String fecha)`. Este método convierte una fecha en formato normal `dd/mm/yyyy` a formato americano `m/dd/yyyy`
    - `` `String americanoToNormal(String fecha) ``. Este método realiza el paso contrario, convierte fechas en formato americano a formato normal.

28 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

6.5 Talleres

6.5.1 Taller UD05_01: Classroom 50

Aceptar la tarea en Classroom 50

Aceptamos el *Assignement* (la tarea/ejercicio) a partir del link del profesor, en este caso: https://classroom50.org/IES-Eduardo-Primo-Marques/1dam-programacio-java/assignments/ud05

Si no recuerdas cómo aceptar la tarea, clonar el repositorio o abrirlo en IntelliJ, repasa la guía completa en el taller de UD02.

Tarea

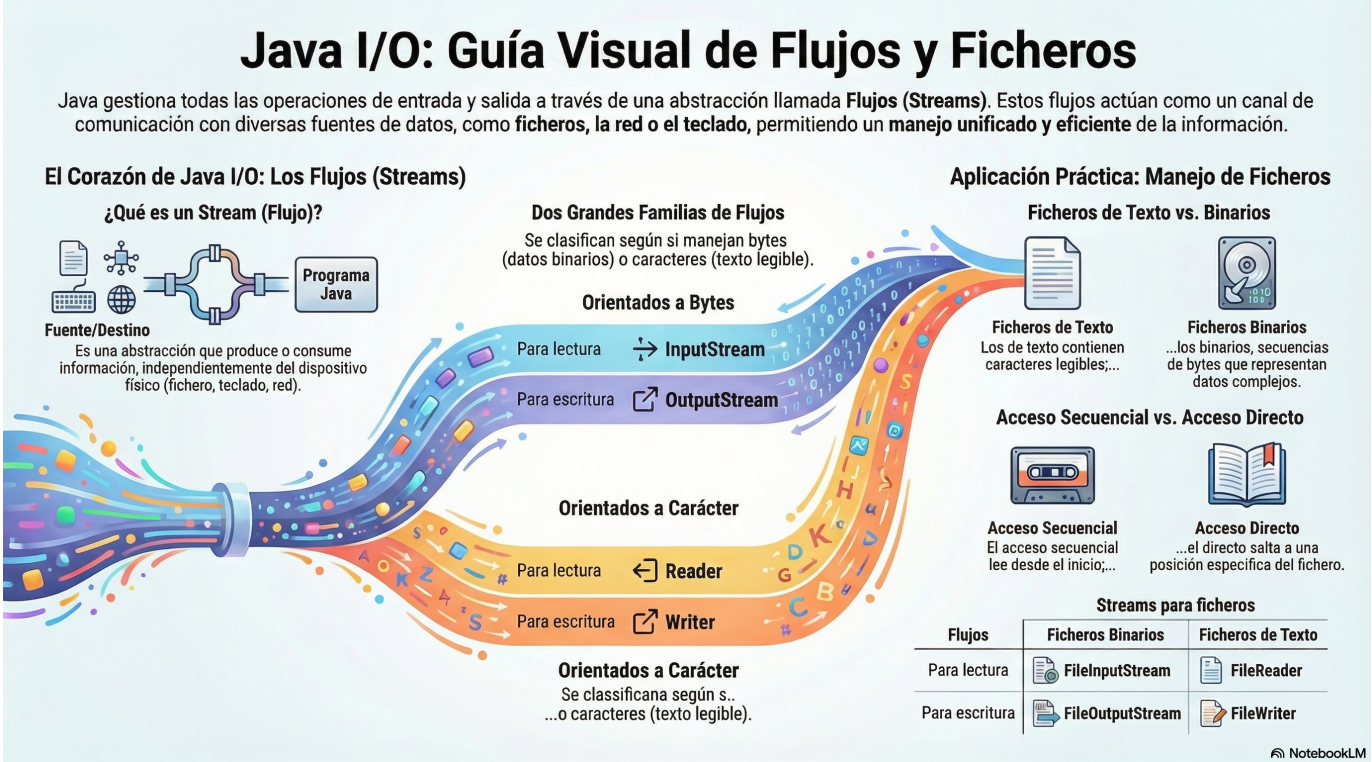
Debes enviar tus soluciones a Classroom 50 y superar al menos la mitad de los tests, cuantos más tests superados, mejor nota tendrás en la tarea.

6 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

7. UD06

7.1 Lectura y escritura de información



Al finalizar esta unidad serás capaz de...

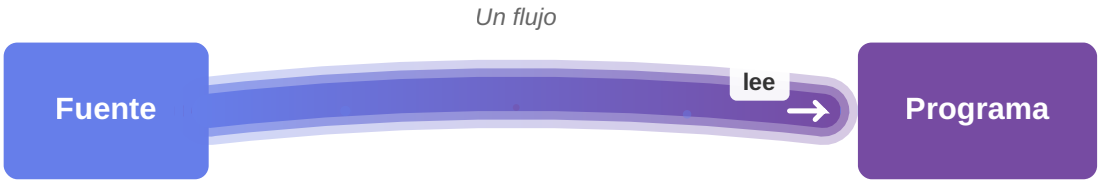
- Diferenciar entre flujos de bytes y de caracteres
- Leer y escribir ficheros de texto
- Leer y escribir ficheros binarios
- Serializar y deserializar objetos
- Utilizar las clases File, FileReader, FileWriter, BufferedReader
- Comprender el funcionamiento básico de los sockets
- Conocer el patrón CRUD y su aplicación

En Java, la interacción de nuestro programa con el mundo exterior—desde leer una pulsación de tecla hasta comunicarse a través de una red—se maneja de manera elegante y simplificada a través del concepto de **Streams (flujos)**. Imagina un *stream* como un conducto lógico universal: Java se encarga de la compleja tarea de vincular este conducto a cualquier dispositivo físico (teclado, monitor, ficheros, *sockets*), **liberándote** de tener que codificar para cada dispositivo individualmente. Estos canales se clasifican en flujos orientados a **bytes** (`InputStream` y `OutputStream`), perfectos para datos binarios, y flujos orientados a **caracteres** (`Reader` y `Writer`), esenciales para manejar la codificación Unicode y garantizar que tu aplicación sea internacionalizable. Más allá de la memoria volátil, veremos cómo asegurar que los datos persistan mediante el uso de **ficheros** (de texto o binarios). Finalmente, exploraremos potentes mecanismos como la **serialización**, que permite guardar y transmitir el estado completo de un objeto, y el uso de **sockets** para establecer la comunicación fundamental entre programas cliente y servidor.

7.1.1 Streams (Flujos)

Los programas Java realizan las operaciones de entrada y salida a través de lo que se denominan Streams (traducido, flujos).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Un stream es una abstracción de todo aquello que produzca o consuma información. Podemos ver a este stream como una entidad lógica que, por otra parte, se encontrará vinculado con un dispositivo físico. La eficacia de esta forma de implementación radica en que las operaciones de entrada y salida que el programador necesita manejar son las mismas independientemente del dispositivo con el que estemos actuando. Será Java quien se encargue de manejar el dispositivo concreto, ya se trate del teclado, el monitor, un sistema de ficheros o un socket de red, etc., liberando a nuestro código de tener que saber con quién está interactuando.

Clasificación de los Streams

En Java los Streams se materializan en un conjunto de clases y subclases, contenidas en el paquete `java.io`. Todas las clases para manejar streams parten, de cuatro clases abstractas:

- `InputStream`
- `OutputStream`
- `Reader`
- `Writer`

Streams	Orientados a bytes	Orientados a carácter
Para **lectura**	`InputStream`	`Reader`
Para **escritura**	`OutputStream`	`Writer`

STREAMS ORIENTADOS A BYTE (BYTE STREAMS)

Proporcionan un medio adecuado para el manejo de entradas y salidas de bytes y su uso lógicamente está orientado a la lectura y escritura de datos binarios. El tratamiento del flujo de bytes viene gobernado por dos clases abstractas que son `InputStream` y `OutputStream`.

Cada una de estas clases abstractas tiene varias subclases concretas que controlan las diferencias entre los distintos dispositivos de I/O que se pueden utilizar. Así mismo, estas dos clases son las que definen los métodos que sus subclases tendrán implementados y, de entre todas, destacan las operaciones `read()` y `write()` que leen y escriben bytes de datos respectivamente.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

STREAMS ORIENTADOS A CARACTER (CHARACTER STREAMS)

Proporciona un medio conveniente para el manejo de entradas y salidas de caracteres. Dichos flujos usan codificación Unicode y, por tanto, se pueden internacionalizar.

Una observación: Este es un modo que Java nos proporciona para manejar caracteres pero al nivel más bajo todas las operaciones de I/O son orientadas a byte. Al igual que la anterior el flujo de caracteres también viene gobernado por dos clases abstractas: `Reader` y `Writer`. Dichas clases manejan flujos de caracteres Unicode. Y también de ellas derivan subclases concretas que implementan los métodos definidos en ellas siendo los más destacados los métodos `read()` y `write()` que, en este caso, leen y escriben caracteres de datos respectivamente.

Como hemos comentado, tanto en `Reader` como en `InputStream` encontramos un método `read()`, en concreto `public int read()`. Observar la diferencia entre estos dos métodos nos ayudará a comprender la diferencia entre los flujos orientados a byte y los orientados a carácter.

método	devuelve
`int InputStream.read()`	valor entre 0 y 255
`int Reader.read()`	valor entre 0 y 65535

A pesar de llamarse igual, `InputStream.read()` devuelve el siguiente byte de datos leído del stream. El valor que devuelve está entre 0 y 255 (ó -1 si se ha llegado al final del stream). `Reader.read()`, sin embargo, devuelve un valor entre 0 y 65535 (ó -1), correspondiente al siguiente carácter simple leído del stream.

Stream estándar

Existen una serie de streams de uso común a los cuales se denomina streams estándar. El sistema se encarga de crear estos streams automáticamente.

- `System.in`
- Instancia de la clase `InputStream`: flujo de bytes de entrada
- Métodos:
  - `read()` permite leer un byte de la entrada como entero
  - `skip(n)` ignora `n` bytes de la entrada
  - `available()` número de bytes disponibles para leer en la entrada
- `System.out`
- Instancia de la clase `PrintStream`: flujo de bytes de salida
- Métodos
  - para impresión de datos: `print()`, `println()`
  - `flush()` vacía el buffer de salida escribiendo su contenido
- `System.err`
- Funcionamiento similar a `System.out`
- Se utiliza para enviar mensajes de error (por ejemplo a un fichero de log o a la consola)

> **Información**
>
> Por defecto, `System.in`, `System.out` y `System.err` se encuentran asociados a la consola (teclado y pantalla), pero es posible re-dirigirlos a otras fuentes o destinos, como por ejemplo a un fichero o una impresora.

Utilización de Streams

Para utilizar un stream hay que seguir una serie de pasos:

- Lectura
  - Abrir el stream asociado a una fuente de datos (creación del objeto stream)
    - Teclado
    - Fichero
    - Socket remoto

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

    - Mientras existan datos disponibles
        - Leer datos
    - Cerrar el stream (método close)
- Escritura
    - Abrir el stream asociado a una fuente de datos (creación del objeto stream)
        - Pantalla
        - Fichero
        - Socket local
    - Mientras existan datos disponibles
        - Escribir datos
    - Cerrar el stream (método close)

> **Importante**
>
> - Los Stream estándar ya se encarga el sistema de abrirlos y cerrarlos
> - Un fallo en cualquier punto del proceso produce una `IOException`

Consulta los ejemplos Ejemplo01

7.1.2 Ficheros

En ocasiones necesitamos que los datos que introduce el usuario o que produce un programa persistan cuando éste finaliza, es decir, que se conserven cuando el programa termina su ejecución. Para ello es necesario el uso de una base de datos o de ficheros, que permitan guardar los datos en un almacenamiento secundario como un pendrive, disco duro, DVD, etc. En esta unidad se abordan distintos aspectos relacionados con el almacenamiento en ficheros:

- Introducción a conceptos básicos como los de registro y campo.
- Clasificación de los ficheros según el contenido y forma de acceso.
- Operaciones básicas con ficheros de distinto tipo.

Registros y campos

Llamamos campo a un dato en particular almacenado en una base de datos o en un fichero. Un campo puede ser en nombre de un cliente, la fecha de nacimiento de un alumno, el número de teléfono de un comercio. Los campos pueden ser de distintos tipos: alfanuméricos, numéricos, fechas, etc.

La agrupación de uno o más campos forman un registro. Un registro de alumno podría consistir, por ejemplo, de los siguientes campos

1. Número de expediente.
2. Nombre y apellidos
3. Domicilio
4. Grupo al que pertenece.

Un fichero puede estar formado por registros, lo cual lo dotaría al archivo de estructura. En un fichero de alumnos tendríamos un registro por cada alumno. Los campos del registro serían cada uno de los datos que se almacena del alumno: Nº expediente, nombre, etc ...

En Java no existen específicamente los conceptos de campo y registro. Lo más similar que conocemos son las clases (similares a un registro) y, dentro de las clases, los atributos (similares a campos).

Tampoco en Java los ficheros están formados por registros. Java considera los archivos simplemente como flujos secuenciales de bytes. Cuando se abre un fichero se asocia a él un flujo (stream) a través del cual se lee o escribe en el fichero.

Fichero:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
65255
José Mateo Ruiz
C/ Paz, …
56488
Ángela Lopez Villa
Av. Blas..
24645
Armando García Ledesma
C/ Tuej …
54654
Tomás Ferrando Tamarit
C/ Poeta …
```

```
65255;José Mateo Ruiz;C/ Paz, …
56488;Ángela Lopez Villa;Av. Blas..
24645;Armando García Ledesma;C/ Tuej …
54654;Tomás Ferrando Tamarit;C/ Poeta …
```

Registro:

```
24645
Armando García Ledesma
C/ Tuej …
```

Campo:

```
Armando García Ledesma
```

Ficheros de texto vs ficheros binarios.

Desde un punto de vista a muy bajo nivel, un fichero es un conjunto de bits almacenados en memoria secundaria, accesibles a través de una ruta y un nombre de archivo.

Este punto de vista a bajo nivel es demasiado simple, pues cuando se recupera y trata la información que contiene el fichero, esos bits se agrupan en unidades mayores que las dotan de significado. Así, dependiendo de cuál es el contenido del fichero (de cómo se interpretan los bits que contiene el fichero), podemos distinguir dos tipos de ficheros:

- Ficheros de texto (o de caracteres)
- Ficheros binarios (o de bytes)

Un fichero de texto está formado únicamente por caracteres. Los bits que contiene se interpretan atendiendo a una tabla de caracteres, ya sea ASCII o Unicode. Este tipo de ficheros se pueden abrir con un editor de texto plano y son, en general, legibles. Por ejemplo, los ficheros .java que contienen los programas que elaboramos, son ficheros de texto.

Por otro lado, los ficheros binarios contienen secuencias de bytes que se agrupan para representar otro tipo de información: números, sonidos, imágenes, etc. Un fichero binario se puede abrir también con un editor de texto plano pero, en este caso, el contenido será ininteligible. Existen muchos ejemplos de ficheros binarios: el archivo .exe que contiene la versión ejecutable de un programa es un fichero binario.

Las operaciones de lectura/escritura que utilizamos al acceder desde un programa a un fichero de texto están orientadas al carácter: leer o escribir un carácter, una secuencia de caracteres, una línea de texto, etc. En cambio las operaciones de lectura/escritura en ficheros binarios están orientadas a byte: se leen o escriben datos binarios, como enteros, bytes, double, etc.

Acceso secuencial vs acceso directo.

Existen dos maneras de acceder a la información que contiene un fichero:

- Acceso secuencial
- Acceso directo (o aleatorio)

Con acceso secuencial, para poder leer el byte que se encuentra en determinada posición del archivo es necesario leer, previamente, todos los bytes anteriores. Al escribir, los datos se sitúan en el archivo uno a continuación del otro, en el mismo orden en que se introducen. Es decir, la nueva información se coloca en el archivo a continuación de la que ya hay. No es posible realizar modificaciones de los datos existentes, tan solo añadir al final.

Sin embargo, con el acceso directo, es posible acceder a determinada posición (dirección) del fichero de manera directa y, posteriormente, hacer la operación de lectura o escritura deseada.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

No siempre es necesario realizar un acceso directo a un archivo. En muchas ocasiones el procesamiento que realizamos de sus datos consiste en la escritura o lectura de todo el archivo siguiendo el orden en que se encuentran. Para ello basta con un acceso secuencial.

Streams para trabajar con ficheros.

Para trabajar con ficheros disponemos de las siguientes clases:

Streams para ficheros	Ficheros binarios	Ficheros de texto
Para **lectura**	`FileInputStream`	`FileReader`
Para **escritura**	`FileOutputStream`	`FileWriter`

- `FileReader` proporciona operaciones para leer de un fichero uno o varios caracteres
- `FileWriter` permite escribir en un fichero uno o varios caracteres o un String.
- `FileInputStream` permite leer bytes de un fichero.
- `FileOutputStream` permite escribir bytes de un fichero.

> **Información**
>
> Consulta en la documentación los distintos constructores disponibles para estas clases.

Observa el Ejemplo02 para ver la creación de un fichero de texto.

Observa el Ejemplo03 para ver cómo sobrescribir un fichero añadiendo contenido.

LECTURA Y ESCRITURA DE INFORMACIÓN ESTRUCTURADA.

Si observamos la documentación de las clases `FileInputStream` y `FileOutputStream` veremos que las operaciones de lectura y escritura son muy básicas y permiten únicamente leer o escribir uno o varios bytes. Es decir, son operaciones de muy bajo nivel. Si lo que queremos es escribir información binaria más compleja, como por ejemplo un dato de tipo `double` o `boolean` o `int`, tendríamos que hacerlo a través de un stream que permitiese ese tipo de operaciones y asociarlo al `FileOutputStream` o `FileOutputStream`.

Podríamos, por ejemplo, asociar un `DataInputStream` a un `FileInputStream` para leer del fichero un dato de tipo `int`.

En ejemplos posteriores se ilustrará cómo asociar un stream a un `File…Stream`.

Observa el Ejemplo04 para lectura secuencial de un fichero de texto.

Observa el Ejemplo05 para escritura secuencial en un fichero de texto.

Observa el Ejemplo07 para lectura y escritura de ficheros binarios.

Ficheros con buffering.

Cualquier operación que implique acceder a memoria externa es muy costosa, por lo que es interesante intentar reducir al máximo las operaciones de lectura/escritura que realizamos sobre los ficheros, haciendo que cada operación lea o escriba muchos caracteres. Además, eso también permite operaciones de más alto nivel, como la de leer una línea completa y devolverla en forma de cadena.

> **Referencia**
>
> En el libro Head First Java, describe los buffers de la siguiente forma: "*Si no hubiera buffers, sería como comprar sin un carrito: debería llevar los productos uno a uno hasta la caja. Los buffers te dan un lugar en el que dejar temporalmente las cosas hasta que está lleno. Por ello has de hacer menos viajes cuando usas el carrito.*"

Las clases `BufferedReader`, `BufferedWriter`, `BufferedInputStream` y `BufferedOutputStream` permiten realizar buffering. Situadas "por delante" de un stream de fichero acumulan las operaciones de lectura y escritura y cuando hay suficiente información se llevan finalmente al fichero.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

> **Atención**
>
> Recuerda la importancia de cerrar los flujos para asegurarte que se vacía el buffer.

Observa el Ejemplo06 para el uso de buffers en la lectura y escritura de ficheros.

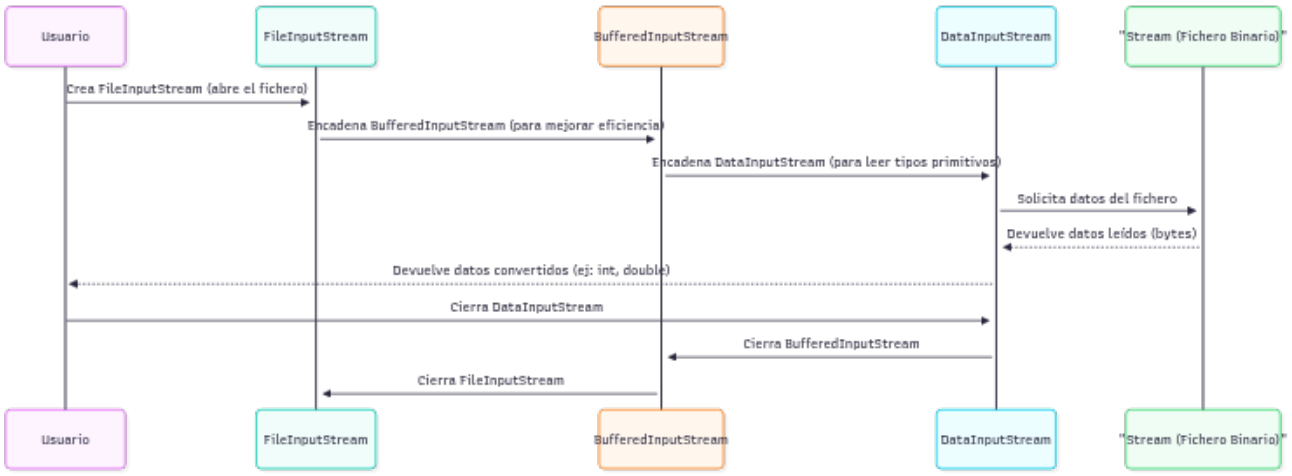
Combinación de Streams

En muchas ocasiones, una sola clase de las vistas no nos da la funcionalidad necesaria para poder hacer la tarea que se requiere. En tales casos es necesario combinar varios Streams de manera que unos actúan como origen de información de los otros, o unos escriben sobre los otros.

En este caso tendríamos que combinar tres clases:

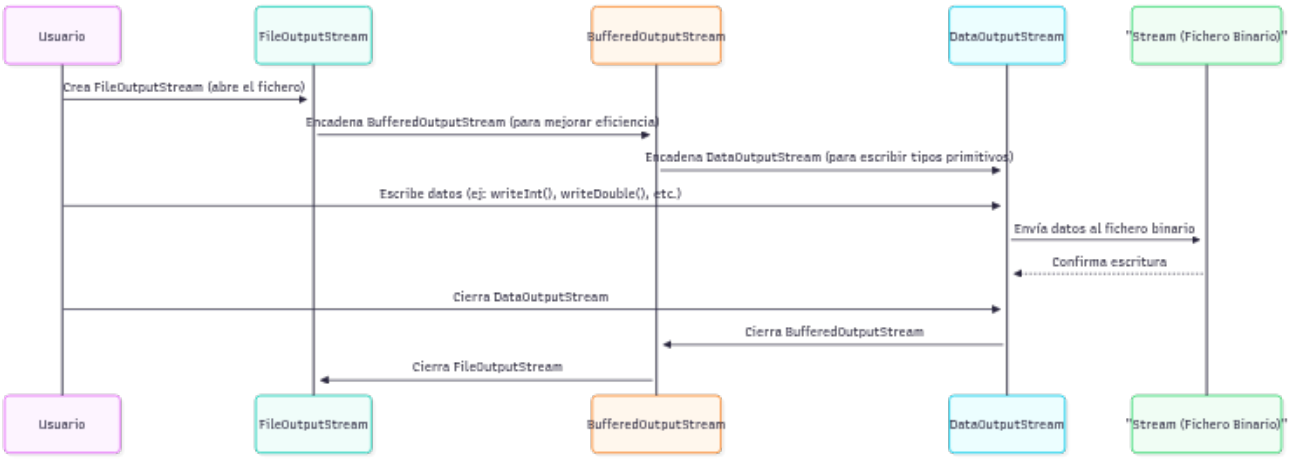
PARA LA ENTRADA/LECTURA:

```
1  FileInputStream fis = new FileInputStream("fichero.bin");
2  BufferedInputStream bis = new BufferedInputStream(fis);
3  DataInputStream dis = new DataInputStream(bis);
4  //o bien
5  DataInputStream dis = new DataInputStream(new BufferedInputStream(new FileInputStream("fichero.bin")));
```



PARA LA SALIDA/ESCRITURA

```
1  FileOutputStream fos = new FileOutputStream("fichero.bin");
2  BufferedOutputStream bos = new BufferedOutputStream(fos);
3  DataOutputStream dos = new DataOutputStream(bos);
4  //o bien
5  DataInputStream dos = new DataInputStream(new BufferedInputStream(new FileInputStream("fichero.bin")));
```



© 2025 David Martínez licensed under CC BY-NC-SA 4.0

`try` vs `try(with resources)`

En ocasiones el propio IDE nos sugiere que usemos el bloque `try with resources` en lugar de un simple `try`, así una sentencia como esta:

```
FileReader fr = new FileReader(path);
BufferedReader br = new BufferedReader(fr);
try {
    return br.readLine();
} finally {
    br.close();
    fr.close();
}
```

Acaba convertida en algo parecido a esta:

```
static String readFirstLineFromFile(String path) throws IOException {
    try (FileReader fr = new FileReader(path);
         BufferedReader br = new BufferedReader(fr)) {
        return br.readLine();
    }
}
```

> **Recomendación**
>
> La principal diferencia es que hasta java 7 sólo se podía hacer como en la primera versión. Además en la segunda versión nos "ahorramos" tener que cerrar los recursos, puesto que lo realizará automáticamente en caso de que se produzca algún error evitando así el enmascaramiento de excepciones. Por tanto, sigue siendo necesario cerrar el stream por ejemplo al usar un Buffer para que se vacíe totalmente en el fichero de destino.

7.1.3 Serialización

Java facilita el almacenamiento y transmisión del estado de un objeto mediante un mecanismo conocido con el nombre de serialización.

La serialización de un objeto consiste en generar una secuencia de bytes lista para su almacenamiento o transmisión. Después, mediante la deserialización, el estado original del objeto se puede reconstruir.

Para que un objeto sea serializable, ha de implementar la interfaz `java.io.Serializable` (que lo único que hace es marcar el objeto como serializable, sin que tengamos que implementar ningún método).

```
import java.io.*;

public class Persona implements Serializable {

    private String nombre;
    transient private int edad; //No se guardará al serializar
    private double salario;
    private Persona tutor;
[...]
```

Para que un objeto sea serializable, todas sus variables de instancia han de ser serializables.

Todos los tipos primitivos en Java son serializables por defecto (igual que los arrays y otros muchos tipos estándar).

Cuando queremos evitar que cualquier campo persista en un archivo, lo marcamos como transitorio (`transient`). No podemos marcar ningún método transitorio, solo campos.

> **Importante**
>
> El fichero con los objetos serializados almacena los datos en un formato propio de Java, por lo que no se puede leer fácilmente con un simple editor de texto (ni editar).

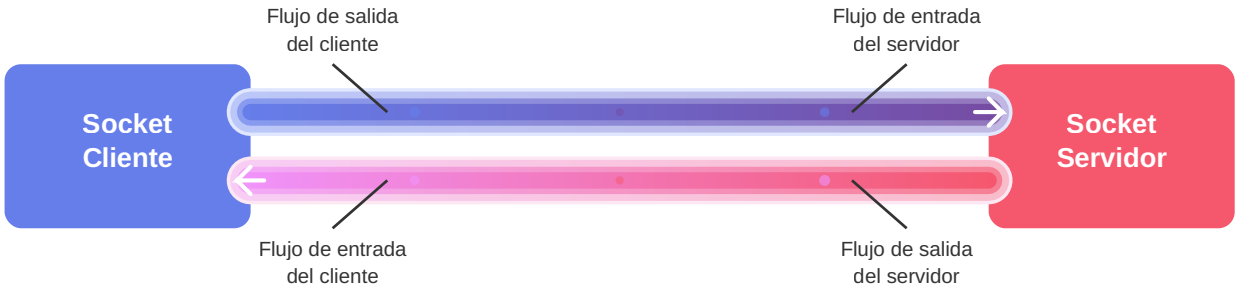
Observa el Ejemplo08 sobre serialización de objetos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

7.1.4 Sockets

Los sockets son un mecanismo que nos permite establecer un enlace entre dos programas que se ejecutan independientes el uno del otro (generalmente un programa cliente y un programa servidor) Java por medio de la librería `java.net` nos provee dos clases: `Socket` para implementar la conexión desde el lado del cliente y `ServerSocket` que nos permitirá manipular la conexión desde el lado del servidor.

Cabe resaltar que tanto el cliente como el servidor no necesariamente deben estar implementados en Java, solo deben conocer sus direcciones IP y el puerto por el cual se comunicarán.



Observa el Ejemplo09 sobre comunicación mediante sockets.

7.1.5 Manejo de ficheros y carpetas (`File`)

La clase `File` es una representación abstracta ficheros y carpetas. Cuando creamos en Java un objeto de la clase `File` en representación de un fichero o carpeta concretos, no creamos el fichero al que se representa. Es decir, el objeto `File` representa al archivo o carpeta de disco, pero no es el archivo o carpeta de disco.

La clase `File` dispone de métodos que permiten realizar determinadas operaciones sobre los ficheros. Podríamos, por ejemplo, crear un objeto de tipo `File` que represente a `c:\datos\libros.txt` y, a través de ese objeto `File`, realizar consultas relativas al fichero `libros.txt`, como su tamaño, atributos, etc, o realizar operaciones sobre él: borrarlo, renombrarlo, ...

Constructores

La clase `File` tiene varios constructores, que permiten referirse, de varias formas, al archivo que queremos representar:

Método	Descripción
`public File (String ruta)`	Crea el objeto `File` a partir de la ruta indicada. Si se trata de un archivo tendrá que indicar la ruta y el nombre.
`public File (String ruta, String nombre)`	Permite indicar de forma separada la ruta del archivo y su nombre
`public File (File ruta, String nombre)`	Permite indicar de forma separada la ruta del archivo y su nombre. En este caso la ruta está representada por otro objeto File.
`public File (URI uri)`	Crea el objeto File a partir de un objeto URI (Uniform Resource Identifier). Un URI permite representar un elemento siguiendo una sintaxis concreta, un estándar.

Métodos

Aquí exponemos algunos métodos interesantes. Hay otros que puedes consultar en la documentación de Java.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

	Relacionados con el nombre del fichero
`String getName()`	Devuelve el nombre del fichero o directorio al que representa el objeto. (Solo el nombre, sin la ruta)
`String getPath()`	Devuelve la ruta del fichero o directorio. La ruta obtenida es dependiente del sistema, es decir, contendrá el carácter de separación de directorios que esté establecido por defecto. Este separador está definido en public `static final String separator`
`String getAbsolutePath()`	Devuelve la ruta absoluta del fichero o directorio.
`String getParent()`	Devuelve la ruta del directorio en que se encuentra el fichero o directorio representado. Devuelve null si no hay directorio padre.

	Para hacer comprobaciones

`boolean exists()`
`boolean canWrite()`
`boolean canRead()`
`boolean isFile()`
`boolean isDirectory()`
	Permiten averiguar, respectivamente, si el fichero existe, si se puede escribir en el, si se puede leer de él, si se trata de un fichero o si se trata de un directorio

	Obtener información de un fichero
`long length`	Devuelve el tamaño en bytes del archivo. El resultado es indefinido si se consulta sobre un directorio o una unidad.
`long lastModified`	Devuelve la fecha de la última modificación del archivo. Devuelve el número de milisegundos transcurridos desde el 1 de enero de 1970

	Para trabajar con directorios
`boolean mkdir()`	Crea el directorio al cual representa el objeto File.
`boolean mkdirs()`	Crea el directorio al cual representa el objeto File, incluyendo todos aquellos que sean necesarios y no existan.
`String[] list()`	Devuelve un array de `Strings` con los nombres de los ficheros y directorios que contiene el directorio al que representa el objeto File.
`String[] list(FileNameFilter filtro)`	Devuelve un array de `Strings` con los nombres de los ficheros y directorios que contiene el directorio al que representa el objeto File y que cumplen con determinado filtro.
`public File[] listFiles()`	Devuelve un array de objetos `File` que representan a los archivos y carpetas contenidos en el directorio al que se refiere el objeto File.

	Para hacer cambios
`boolean renameTo(File nuevoNombre)`	Permite renombrar un archivo. Hay que tener en cuenta que la operación puede fracasar por muchas razones, y que será dependiente del sistema: Que no se pueda mover el fichero de un lugar a otro, que ya exista un fichero que coincide con el nuevo, etc. El método devuelve true solo si la operación se ha realizado con éxito. Existe un método move en la clase Files para mover archivos de una forma independiente del sistema.
`boolean delete()`	Elimina el archivo o la carpeta a la que representa el objeto File. Si se trata de una carpeta tendrá que estar vacía. Devuelve true si la operación tiene éxito.
`boolean createNewFile()`	Crea un archivo vacío. Devuelve true si la operación se realiza con éxito.
`File createTempFile(String prefijo, String sufijo)`	Crea un archivo vacío en la carpeta de archivos temporales. El nombre llevará el prefijo y sufijo indicados. Devuelve el objeto File que representa al nuevo archivo.

Observa el Ejemplo10 sobre manejo de ficheros y carpetas con la clase File.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

7.1.6 Ejemplos UD06

Ejemplo01

Flujo de entrada y salida estándar

Veamos un ejemplo en el que se lee por teclado hasta pulsar la tecla de retorno, en ese momento el programa acabará imprimiendo por la salida estándar la cadena leída.

Para ir construyendo la cadena con los caracteres leídos podríamos usar la clase `StringBuffer` o la `StringBuilder`. La clase `StringBuffer` permite almacenar cadenas que cambiarán en la ejecución del programa. `StringBuilder` es similar, pero no es síncrona. De este modo, para la mayoría de las aplicaciones, donde se ejecuta un solo hilo, supone una mejora de rendimiento sobre `StringBuffer`.

El proceso de lectura ha de estar en un bloque `try..catch`.

```java
package UD06.P1_Flujos;

import java.io.IOException;

public class P1_1_FlujoEstandarEntrada {

    public static void main(String[] args) {
        // Cadena donde iremos almacenando los caracteres que se escriban
        StringBuilder str = new StringBuilder();
        char c;
        // Por si ocurre una excepción ponemos el bloque try-cath
        try {
            // Mientras la entrada de terclado no sea Intro
            while ((c = (char) System.in.read()) != '\n') {
                // Añadir el character leído a la cadena str
                str.append(c);
            }
        } catch (IOException ex) {
            System.out.println(ex.getMessage());
        }
        // Escribir la cadena que se ha ido tecleando
        System.out.println("Cadena introducida: " + str);
    }
}
```

```java
package UD06.P1_Flujos;

import java.io.BufferedReader;
import java.io.FileWriter;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;

public class P1_2_FlujoEstandarSalida {

    public static void main(String[] args) {
        // Por si ocurre una excepción ponemos el bloque try-cath
        try {
            PrintWriter out = null;
            out = new PrintWriter(new FileWriter("test/salida.txt", true));
            BufferedReader br = new BufferedReader(
                    new InputStreamReader(System.in));
            String s;
            while (!(s = br.readLine()).equals("salir")) {
                out.println(s);
            }
            out.close();
        } catch (IOException ex) {
            System.out.println(ex.getMessage());
        }
    }
}
```

⬆ Volver a teoría

Ejemplo02

Creación de un fichero de texto

En el siguiente ejemplo vemos como crear un fichero de texto y escribir una frase en el.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD06.P2_Ficheros;

import java.io.*;

public class P2_1_CrearFichero {

    public static void main(String[] args) {
        FileWriter f = null;
        try {
            f = new FileWriter("texto.txt");
            f.write("Este texto se escribe en el fichero\n\r");
        } catch (IOException e) {
            System.out.println("Problema al abrir o escribir ");
        } finally {
            if (f != null) {
                try {
                    f.close();
                } catch (IOException e) {
                    System.out.println("Problema al cerrar el fichero");
                }
            }
        }
    }
}
```

La creación del `FileWriter` puede provocar `IOException`, lo mismo que el método `write`. Por ello las instrucciones se encuentran en un bloque `try-catch`.

Al finalizar su uso, y tan pronto como sea posible, hay que cerrar los streams (`close`) .

⬆ Volver a teoría

Ejemplo03

Sobrescritura de un fichero añadiendo contenido

Es muy importante tener en cuenta que cuando se crea un `FileWriter` o un `FileOutputStream` y se escribe en él ...

- ... si el fichero no existe se crea
- ... si el fichero existe, **su contenido se reemplaza** por el nuevo. El contenido previo que tuviera el fichero se pierde.

Es posible escribir en un fichero indicando que la información se añada a la que ya hay y no se reescriba el fichero. Para ello usaremos el constructor que recibe el parámetro `append` y lo pasaremos a true.

El siguiente ejemplo muestra como añadir una línea al final de un fichero de texto.

```java
package UD06.P2_Ficheros;

import java.io.*;

public class P2_2_SobrescribirFichero {

    public static void main(String[] args) {
        try (FileWriter f = new FileWriter("david/enero/texto.txt", true);) {
            f.write("Este texto se añade en el fichero\n\r");

        } catch (IOException e) {
            System.out.println("Problema al abrir o escribir ");
        }
    }
}
```

En este ejemplo se ha utilizado la nueva sintaxis disponible para los bloques `try-catch`: lo que se denomina "try with resources". Esta sintaxis permite crear un objeto en la cabecera del bloque `try`. El objeto creado se cerrará automáticamente al finalizar. El objeto debe pertenecer al interface `Closeable`, es decir, debe tener método `close()`.

⬆ Volver a teoría

Ejemplo04

Lectura secuencial de un fichero de texto

Como hemos comentado anteriormente el acceso secuencial a un fichero supone que para acceder a un byte es necesario leer previamente los anteriores. Suele utilizarse este tipo de acceso cuando es necesario leer un archivo de principio a fin.

Vamos a ver una serie de ejemplos que muestren cómo leer y escribir secuencialmente un fichero.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Leer un fichero de texto y mostrar el número de vocales que contiene.

```java
package UD06.P2_Ficheros;

import java.io.*;

public class P2_3_LecturaSecuencialTexto {

    final static String VOCALES = "AEIOUaeiou";

    public static void main(String[] args) {
        try (FileReader f = new FileReader(new File("texto.txt"));) {
            int contadorVocales = 0;
            int caracter;
            while ((caracter = f.read()) != -1) {
                char letra = (char) caracter;
                if (VOCALES.indexOf(letra) != -1) {
                    contadorVocales++;
                }
            }
            System.out.println("Numero de vocales: " + contadorVocales);
        } catch (FileNotFoundException e) {
            System.out.println("Probrema al abrir el fichero");
        } catch (IOException e) {
            System.out.println("Problema al leer");
        }
    }
}
```

Observa que:

- Para leer el fichero de texto usamos un `InputReader`.
- Al crear el stream (`InputReader`) es posible indicar un objeto de tipo `File`
- La operación `read()` devuelve un entero. Para obtener el carácter correspondiente tenemos que hacer una conversión explícita de tipos.
- La operación `read()` devuelve -1 cuando no queda información que leer del stream.
- La guarda del bucle `while` combina una asignación con una comparación. En primer lugar se realiza la asignación y luego se compara carácter con -1.
- `FileNotFoundException` sucede cuando el fichero no se puede abrir (no existe, permiso denegado, etc), mientras que `IOException` se lanzará si falla la operación `read()`

⬆ Volver a teoría

Ejemplo05

Escritura secuencial en un fichero de texto

Dada una cadena escribirla en un fichero en orden inverso:

```java
package UD06.P2_Ficheros;

import java.io.*;

public class P2_4_EscrituraSecuencialTexto {

    final static String CADENA = "En un lugar de la mancha...";

    public static void main(String[] args) {
        try (FileWriter f = new FileWriter(new File("texto.txt"));) {
            for (int i = CADENA.length() - 1; i >= 0; i--) {
                f.write(CADENA.charAt(i));
            }
            System.out.println("FIN");
        } catch (FileNotFoundException e) {
            System.out.println("Probrema al abrir el fichero");
        } catch (IOException e) {
            System.out.println("Problema al escribir");

        }
    }
}
```

Observa que:

- Para escribir el fichero de texto usamos un `FileWriter`.
- Tal y como se ha creado el stream, el fichero (si ya existe) se sobreescribirá.
- El manejo de excepciones es como el del caso previo.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

⬆ Volver a teoría

Ejemplo06

Uso de buffers para leer y escribir ficheros

En el siguiente codigo se usan buffers para leer líneas de un fichero y escribirlas en otro convertidas a mayúsculas

```java
package UD06.P2_Ficheros;

import java.io.*;

public class P2_5_Buffers {

    final static String ENTRADA = "texto.txt";
    final static String SALIDA = "textoMayusculas.txt";

    public static void main(String[] args) {
        try (BufferedReader fe = new BufferedReader(new FileReader(ENTRADA));
             BufferedWriter fs = new BufferedWriter(new FileWriter(SALIDA))){
            String linea;
            while ((linea = fe.readLine()) != null) {
                fs.write(linea.toUpperCase());
                fs.newLine();
            }
            System.out.println("FIN");
        } catch (FileNotFoundException e) {
            System.out.println("Probrema al abrir el fichero");
        } catch (IOException e) {
            System.out.println("Problema al leer o escribir");
        }
    }
}
```

Observa que:

- Usamos buffers tanto para leer como para escribir. Esto permite minimizar los accesos a disco.
- Los buffers quedan asociados a un `FileReader` y `FileWriter` respectivamente. Realizamos las operaciones de lectura/escritura sobre las clases `Buffered…` y cuando es necesario la clase accede internamente al stream que maneja el fichero.
- Es necesario escribir explícitamente los saltos de línea. Esto se hace mediante el método `newLine()`. `newLine()` permite añadir un salto de línea sin preocuparnos de cuál es el carácter de salto de línea. El salto de línea es distinto en distintos sistemas: en unos es `\n`, en otros `\r`, en otros `\n\r`, …
- `BufferedReader` dispone de un método para leer líneas completas (`readLine()`). Cuando se llega al final del fichero este método devuelve `null`.
- Fíjate como el bloque `try with resources` creamos varios objetos. Si la creación de cualquiera de ellos falla, se cerrarán todos los stream que se han abierto.

⬆ Volver a teoría

Ejemplo07

Lectura y escritura de ficheros binarios

Ya hemos visto que con `FileInputStream` y `FileOutputStream` se puede leer y escribir bytes de información de/a un archivo.

Sin embargo esto puede no ser suficiente cuando la información que tenemos que leer o escribir es más compleja y los bytes se agrupan para representar distintos tipos de datos.

Imaginemos por ejemplo que queremos guardar en un fichero “jugadores.dat”, el año de nacimiento y la estatura de cinco jugadores de baloncesto:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD06.P2_Ficheros;

import java.io.*;
import java.util.Scanner;

public class P2_6_EscrituraSecuencialBinario {

    public static void main(String[] args) {
        Scanner tec = new Scanner(System.in);
        try (DataOutputStream fs = new DataOutputStream(
                        new BufferedOutputStream(
                                new FileOutputStream("jugadores.dat")));) {
            for (int i = 1; i <= 5; i++) {
                //Pedimos datos al usuario
                System.out.println(" ---- Jugador " + i + " -----");
                System.out.print("Nombre: ");
                String nombre = tec.nextLine();

                System.out.print("Nacimiento: ");
                int anyo = tec.nextInt();

                System.out.print("Estatura: ");
                double est = tec.nextDouble();
                //Vaciar salto linea
                tec.nextLine();

                //Volcamos información al fichero
                fs.writeUTF(nombre);
                fs.writeInt(anyo);
                fs.writeDouble(est);
            }
        } catch (FileNotFoundException e) {
            System.out.println("Probrema al abrir el fichero");
        } catch (IOException e) {
            System.out.println("Problema al leer o escribir");
        }
    }
}
```

Observa que:

- Para escribir información binaria usamos un `DataInputStream` asociado al stream. La clase tiene métodos para escribir `int`, `byte`, `double`, `boolean`, etc.
- Además, como hemos hecho en ejemplos previos, usamos un buffer. Fíjate como en el constructor se enlazan unas clases con otras.
- A pesar de que en Java los ficheros son secuencias de bytes, estamos dotando al fichero de cierta estructura: primero aparece el nombre, luego el año y finalmente la estatura. Cada uno de estos tres datos constituirían un registro de formado por tres campos. Para poder recuperar información de un fichero binario es necesario conocer cómo se estructura ésta dentro del fichero.

```java
package UD06.P2_Ficheros;

import java.io.*;
import java.util.Scanner;

public class P2_7_LecturaSecuencialBinario {

    public static void main(String[] args) {
        Scanner tec = new Scanner(System.in);
        try (DataInputStream fe = new DataInputStream(new BufferedInputStream(
                 new FileInputStream("jugadores.dat")));) {
            while (true) {
                //Leemos nombre
                System.out.println(fe.readUTF());
                //leemos y desechamos resto de datos
                fe.readInt();
                fe.readDouble();
            }
        } catch (EOFException e) {
            //Se lanzará cuando se llegue al final del fichero
        } catch (FileNotFoundException e) {
            System.out.println("Probrema al abrir el fichero");
        } catch (IOException e) {
            System.out.println("Problema al leer o escribir");
        }
    }
}
```

Observa que:

- A pesar de que necesitamos solamente el nombre de cada jugador, es necesario leer también el año y la estatura. No es posible acceder al nombre del segundo jugador sin leer previamente todos los datos del primer jugador.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- La lectura se hace a través de un bucle infinito (`while (true)`), que finalizará cuando se llegue el final del fichero y al leer de nuevo se produzca la excepción `EOFException`

⬆ Volver a teoría

Ejemplo08

Serialización de objetos

En el siguiente ejemplo usaremos una clase persona que definiremos de la siguiente manera

```
package UD06.P3_Serializacion;

import java.io.*;

public class Persona implements Serializable {

    private String nombre;
    transient private int edad; //No se guardará al serializar
    private double salario;
    private Persona tutor;

    public Persona(String nom, double salari) {
        this.nombre = nom;
        this.salario = salari;
        edad = 0;
        tutor = null;
    }

    public String getNombre() {
        return nombre;
    }

    public int getEdad() {
        return edad;
    }

    public double getSalario() {
        return salario;
    }

    public Persona getTutor() {
        return tutor;
    }

    public void incrementaEdad() {
        edad++;
    }

    public void asignaTutor(Persona p) {
        tutor = p;
    }
}
```

Ahora detallamos la clase para serializar o guardar la información en un archivo:

```
package UD06.P3_Serializacion;

import java.io.*;

public class Guardar {

    public static void main(String args[]) {
        ObjectOutputStream salida;
        Persona p1, p2, p3, p4;

        p1 = new Persona("Vicent", 1200.0);
        p2 = new Persona("Mireia", 1800.0);
        p3 = new Persona("Josep", 2100.0);
        p4 = new Persona("Marta", 850.0);

        p1.asignaTutor(p2);
        p2.asignaTutor(p3);
        p3.asignaTutor(p4);

        try {
            salida = new ObjectOutputStream(new FileOutputStream("empleats.ser"));
            salida.writeObject(p1);
            salida.close();
        } catch (IOException e) {
            System.out.println("Algún problema guardando a disco.");
        }
    }
}
```

Y por último la clase para Leer la información una vez guardada:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD06.P3_Serializacion;

import java.io.*;

public class Leer {

    public static void main(String args[]) {
        ObjectInputStream entrada;
        Persona p1, p2, p3, p4;

        try {
            entrada = new ObjectInputStream(new FileInputStream("empleats.ser"));
            p1 = (Persona) entrada.readObject();
            entrada.close();

            p2 = p1.getTutor();
            p3 = p2.getTutor();
            p4 = p3.getTutor();

            System.out.println(p4.getNombre());
            System.out.println(p4.getEdad());
            System.out.println(p4.getSalario());

        } catch (ClassNotFoundException e) {
            System.out.println("Algun problema con las clases definidas.");
        } catch (IOException e) {
            System.out.println("Algun problema leyendo de disco.");
        }
    }
}
```

⬆ Volver a teoría

Ejemplo09

Comunicación mediante sockets

Para nuestro ejemplo de sockets implementaremos ambos (cliente y servidor) usando Java y se comunicarán usando el puerto 11000 (es bueno elegir los puertos en el rango de 1024 hasta 65535).

La secuencia de eventos en nuestro ejemplo será:

- El servidor creará el socket y esperará a que el cliente se conecte o lo detengamos.
- Por otro lado, el cliente abrirá la conexión con el servidor y le enviará una frase en minúsculas que escribirá el usuario y la enviará al servidor.
- Una vez recibida la frase en minúsculas, el servidor la convertirá en mayúsculas, la devolverá al cliente.
- El cliente mostrará la frase en mayúsculas recibida desde el servidor, e irá añadiendo el texto recibido acumulando las frases enviadas.
- Cuando en un envio reciba la palabra EXIT (no importa las mayúsculas) devolverá frase en mayúsculas y cerrará la conexión.
- Si la palabra recibida no es EXIT, el servidor quedará a la espera de una nueva conexión de otro cliente.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
import java.io.*;
import java.net.*;
import java.util.Enumeration;

public class ServidorSocket {

    private static final int PORT=11000;

    private static void mostrarIPs(StringBuilder sb) {
        try {
            Enumeration Interfaces = NetworkInterface.getNetworkInterfaces();
            while (Interfaces.hasMoreElements()) {
                NetworkInterface Interface = (NetworkInterface) Interfaces.nextElement();
                Enumeration Addresses = Interface.getInetAddresses();
                while (Addresses.hasMoreElements()) {
                    InetAddress Address = (InetAddress) Addresses.nextElement();
                    sb.append("\n\t").append(Address.getHostAddress()).append(":").append(PORT);
                }
            }
        } catch (SocketException ex) {
            System.err.println("Error. Al intentar obtener las interfaces de red.");
        }
    }

    public static <serverSocket> void main(String[] args) throws IOException, ClassNotFoundException {
        String FraseClient;
        String FraseMajuscules;
        Socket clientSocket;
        ObjectInputStream entrada;
        ObjectOutputStream eixida;

        try (ServerSocket serverSocket = new ServerSocket(PORT)) {

            StringBuilder sb = new StringBuilder();
            sb.append("Server iniciado y escuchando en la ip y puerto: ");
            mostrarIPs(sb);
            System.out.println(sb.toString());

            while (true) {
                clientSocket = serverSocket.accept();
                entrada = new ObjectInputStream(clientSocket.getInputStream());
                FraseClient = (String) entrada.readObject();

                System.out.println("La frase recibida es: " + FraseClient);

                eixida = new ObjectOutputStream(clientSocket.getOutputStream());
                FraseMajuscules = FraseClient.toUpperCase();

                System.out.println("El server devuelve la frase: " + FraseMajuscules);

                eixida.writeObject(FraseMajuscules);
                clientSocket.close();

                System.out.println("Server esperando una nueva conexión...");
            }
        } catch (ClassNotFoundException ex) {
            System.err.println("Error. Clase no encontrada");
        } catch (IOException ex) {
            System.err.println("Error. De entrada salida." + ex.toString());
        }
    }
}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class ClienteSocket{

    private static final int PORT = 11000;

    //private static final String DNSAWS = "ec2-44-200-177-74.compute-1.amazonaws.com";
    private static final String DNSAWS = "127.0.0.1";

    public static <string> void main(String[] args) throws IOException, ClassNotFoundException {
        ObjectInputStream entrada;
        ObjectOutputStream eixida;
        String frase;
        StringBuilder fraseMayuscula = new StringBuilder();
        String aux = "";
        int iter = 0;

        do {
            try (Socket socket = new Socket(DNSAWS, PORT)) {
                eixida = new ObjectOutputStream(socket.getOutputStream());

                if (iter == 0) {
                    System.out.println("Introduce la frase a enviar en minúsculas");
                    iter++;
                } else {
                    System.out.println("Muy bonito, ahora introduce otra o \"exit\"/\"EXIT\" para salir");
                }

                Scanner in = new Scanner(System.in);

                frase = in.nextLine();

                System.out.println("Se envia la frase " + frase);

                eixida.writeObject(frase);
                entrada = new ObjectInputStream(socket.getInputStream());
                aux = (String) entrada.readObject().toString();

                if (fraseMayuscula.toString().isBlank()) {
                    fraseMayuscula.append(aux);
                } else {
                    fraseMayuscula.append(" ").append(aux);
                }

                System.out.println("La frase recibida es: " + fraseMayuscula);

            } catch (Exception ex){
                System.err.println("Error:" + ex.toString());
                System.exit (-1);
            }
        } while (!aux.equals("EXIT"));

    }
}
```

⬆ Volver a teoría

Ejemplo10

Manejo de ficheros y carpetas con la clase File

Veamos ahora un ejemplo para mostrar información y contenido de una carpeta:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD06.P5_Manejo;

import java.io.*;
import java.util.*;

public class P5_1_Manejo {

    public static void main(String[] args) {

        Scanner tec = new Scanner(System.in);
        System.out.println("Introduce ruta absoluta de una carpeta");
        String nombreCarpeta = tec.nextLine();
        //Creamos objeto File para representar a la carpeta
        File car = new File(nombreCarpeta);
        //Comprobamos si existe
        if (car.exists()) {
            //¿Es una carpeta?
            if (car.isDirectory()) {
                if (car.canRead()) {
                    System.out.println("Lectura permitida");
                } else {
                    System.out.println("Lectura no permitida");
                }

                if (car.canWrite()) {
                    System.out.println("Escritura permitida");
                } else {
                    System.out.println("Escritura no permitida");
                }

                if (car.isHidden()) {
                    System.out.println("Carpeta oculta");
                } else {
                    System.out.println("Carpeta visible");
                }

                System.out.println("---- Contenido de la carpeta ----");
                File[] contenido = car.listFiles();
                for (File f : contenido) {
                    System.out.println(f.getName());
                }
            } else {
                System.out.println(car.getAbsolutePath() + " No es una carpeta");
            }
        } else {
            System.out.println(
                    "No existe la carpeta/archivo " + car.getAbsolutePath());
        }
    }
}
```

⬆ Volver a teoría

7.1.7 Resumen — Conceptos clave

Concepto	Definición
Stream	Flujo de datos que conecta el programa con una fuente o destino
Byte stream	Flujo que maneja datos binarios (InputStream, OutputStream)
Char stream	Flujo que maneja texto (Reader, Writer)
Serialización	Proceso de convertir un objeto en una secuencia de bytes
Fichero	Almacenamiento persistente de datos en disco
Socket	Punto final de una conexión de red entre dos programas

7.1.8 Autoevaluación

- Diferencio entre flujos de bytes y de caracteres
- Leo y escribo ficheros de texto
- Serializo y deserializo objetos
- Utilizzo BufferedReader para entrada eficiente
- Comprendo el concepto de socket a nivel básico

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

7.1.9 Vídeos recomendados

Canal	Vídeo	Contenido
Píldoras Informáticas	Streams I — Accediendo a ficheros (vídeo 152)	Lectura de ficheros con streams
Píldoras Informáticas	Clase File I (vídeo 159)	Manipulación de archivos y directorios
DiscoDurodeRoer	Clase File	La clase File en Java
DiscoDurodeRoer	Path, Paths y Files (NIO)	java.nio — nueva API de ficheros
DiscoDurodeRoer	FileWriter	Escritura de ficheros de texto
DiscoDurodeRoer	FileReader	Lectura de ficheros de texto
DiscoDurodeRoer	Try-with-resources	Gestión automática de recursos
DiscoDurodeRoer	ObjectOutputStream / ObjectInputStream	Serialización de objetos
makigas	Serie Java IO	8 vídeos: streams, FileOutputStream, FileInputStream, try-with-resources, Object streams
EducaMadrid	La clase File de Java	Clase File: crear, listar, borrar archivos
EducaMadrid	Ficheros_1 — Introducción	Acceso a ficheros de texto: FileReader, FileWriter
Programación ATS	Playlist POO Java	Vídeos 95+: ficheros y streams

21 de julio de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

7.2 Comparativa CRUD

Tenemos un array: **lista**

y una variable con la ocupación: **ocupacion** (empieza en cero y se va incrementando)

y otra variable **MAX_DISPONIBLE**: con el total máximo admitido.

Métodos	Con huecos ⭕	Sin huecos 🔴 No ordenado 🔄	Sin huecos 🔴 Ordenado ⬇️
C **añadir** **(create)** ➕	Si (`ocupacion` < `MAX_DISPONIBLE`) recorro toda la `lista` si encuentro un hueco(null) **inserto** el elemento y `ocupacion++` eficiente 🌕⚫⚫ | facilImplentar 🌕🌕⚫	Si (`ocupacion` < `MAX_DISPONIBLE`) **inserto** el elemento en `lista[ocupacion++]` eficiente 🌕🌕🌕| facilImplentar 🌕🌕🌕	Si (`ocupacion` < `MAX_DISPONIBLE`) **insertoOrdenado** el elemento en `lista` y `ocupacion++` eficiente 🌕🌕⚫| facilImplentar 🌕⚫⚫
R **leer** **(read)** 🔎	Si (`ocupacion` >0) Recorro toda la `lista` si encuentro el elemento (equals) lo **devuelvo** si no **devuelvo una excepción** de `NoEncontrado` eficiente 🌕⚫⚫| facilImplementar 🌕🌕🌕	Si (`ocupacion` >0) Recorro la `lista` hasta `ocupacion-1` si encuentro el elemento (equals) lo **devuelvo** si no **devuelvo una excepcion** de `NoEncontrado` eficente 🌕🌕⚫| facilImplementar 🌕🌕🌕	Si (`ocupacion` >0) Usar un metodo de **buscarOrdenado** **devolver** el `elemento` o la `Excepcion` si no existe eficiente 🌕🌕🌕| facilImplentar 🌕⚫⚫
U **modificar** **(update)** ✏️	Si (`ocupacion` >0) Recorro toda la `lista` si encuentro el elemento (equals) lo **modifico** **devuelvo una excepción** de `NoEncontrado` eficiente 🌕⚫⚫| facilImplentar 🌕🌕🌕	Si (`ocupacion` >0) Recorro la `lista` hasta `ocupacion -1` si encuentro el elemento (equals) lo **modifico** **devuelvo una excepcion** de `NoEncontrado` eficente 🌕🌕⚫| facilImplentar 🌕🌕🌕	Si (`ocupacion` >0) Usar un metodo de `buscarOrdenado` **modificar** el elemento o la `Excepcion` si no existe eficiente 🌕🌕🌕| facilImplentar 🌕⚫⚫

D
borrar
(delete)
✖️
	Si (`ocupacion` >0)
	Recorro toda la `lista`
	si encuentro el elemento (equals)
	lo **borro** y `ocupacion--`
	devuelvo una excepción de `NoEncontrado`
	
	eficiente 🌕⚫⚫|
	facilImplentar 🌕🌕🌕
		Si (`ocupacion` >0)
		Recorro la **lista** hasta `ocupacion-1`
		si encuentro el elemento (equals)
		lo **borro**
		`ocupacion--`
		//desplazo la lista hacia la izquierda
		recorro(`i` **)** des la posicion actual hasta `ocupacion -1`
		`lista[i] = lista[i+1]`
		sino **devuelvo una excepción** de `NoEncontrado`
		
		eficente 🌕🌕⚫| facilImplentar
		🌕🌕⚫
			Si (`ocupacion` >0)
			Usar un metodo de `buscarOrdenado`
			si encuentro el elemento
			lo **borro**
			`ocupacion--`
			//desplazo la lista hacia la izquierda
			sino **devuelvo una excepción** de `NoEncontrado`
			
			eficiente 🌕🌕🌕|
			facilImplentar 🌕⚫⚫

6 de diciembre de 2025

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

7.3 Ejercicios de la UD06

7.3.1 Paquetes Completos

Paquete: `UD06._01_1gestorVuelos`

> **Excepciones y ficheros**
>
> Las excepciones propias se declaran dentro de la clase que las lanza, como clases anidadas: `static class VueloCompletoException extends Exception` dentro de `Vuelo` y `static class ElementoNoEncontradoException extends Exception` dentro de `Compania`. El fichero de vuelos de una compañía está en la carpeta `files/_01_1gestorVuelos/` del proyecto (por ejemplo, `files/_01_1gestorVuelos/Iberia.txt`).

Se desea realizar una aplicación `GestorVuelos` para gestionar la reserva y cancelación de vuelos en una agencia de viajes. Dicha agencia trabaja únicamente con la compañía aérea Iberia, que ofrece vuelos desde/hacia varias ciudades de Europa. Se deben definir las clases que siguen, teniendo en cuenta que sus atributos serán privados y sus métodos sólo los que se indican en cada clase.

1. (Vuelo) Implementación de la clase `Vuelo`, que permite representar un vuelo mediante los atributos:
   - `identificador` (`String`)
   - `origen` (`String`)
   - `destino` (`String`)
   - `hSalida` (un tipo que te permita controlar la hora, no es un `String` ni un `int`, etc.)
   - `hLlegada` (un tipo que te permita controlar la hora, no es un `String` ni un `int`, etc.)
   - Además, cada vuelo dispone de 50 asientos, es decir, pueden viajar, como mucho, 50 pasajeros en cada vuelo. Para representarlos, se hará uso de `asiento`, un array de `String` (nombres de los pasajeros) junto con un atributo `numP` que indique el número actual de asientos reservados. Si el asiento `i` está reservado, `asiento[i]` contendrá el nombre del pasajero que lo ha reservado. Si no lo está, `asiento[i]` será `null`. En el array `asiento`, las posiciones impares pertenecen a asientos de ventanilla y las posiciones pares, a asientos de pasillo (la posición 0 no se utilizará).

   En esta clase, se deben implementar los siguientes métodos:

   - `public Vuelo(String id, String orig, String dest, LocalTime hsal, LocalTime hlleg)`. **Constructor** que crea un vuelo con identificador, ciudad de origen, ciudad de destino, hora de salida y hora de llegada indicados en los respectivos parámetros, y sin pasajeros.
   - `public String getIdentificador()`. Devuelve el `identificador`
   - `public String getOrigen()`. Devuelve `origen`.
   - `public String getDestino()`. Devuelve `destino`.
   - `public boolean hayLibres()`. Devuelve `true` si quedan asientos libres y `false` si no quedan.
   - `public boolean equals(Object o)`. Dos vuelos son iguales si tienen el mismo identificador.
   - `public int reservarAsiento(String pas, char pref) throws VueloCompletoException`. Si el vuelo ya está completo se lanza una excepción. Si no está completo, se reserva al pasajero `pas` el primer asiento libre en `pref`. El carácter `pref` será `'P'` o `'V'` en función de que el pasajero desee un asiento de pasillo o de ventanilla. En caso de que no quede ningún asiento libre en la preferencia indicada (`pref`), se reservará el primer asiento libre de la otra preferencia. El método devolverá el número de asiento que se le ha reservado. Este método hace uso del método privado `asientoLibre`, que se explica a continuación.
   - `private int asientoLibre(char pref)`. Dado un tipo de asiento `pref` (pasillo `'P'` o ventanilla `'V'`), devuelve el primer asiento libre (el de menor numero) que encuentre de ese tipo. O devuelve `0` si no quedan asientos libres de tipo `pref`.
   - `public void cancelarReserva(int numasiento)`. Se cancela la reserva del asiento `numasiento`.
   - `public String toString()`. Devuelve una `String` con los datos del vuelo y los nombres de los pasajeros, con el siguiente formato:

```
IB101 Valencia París 19:05:00 21:00:00
Pasajeros:
Asiento 1: Sonia Dominguez
…
Asiento 23: Fernando Romero
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

2. (TestVuelo) Diseñar e implementar una clase Java `TestVuelo` que permita probar la clase `Vuelo` y sus métodos. Para ello se desarrollará el método `main` en el que:
   - Se cree el vuelo IB101 de Valencia a París, que sale a las 19:05 y llega a las 21:00
   - Reservar:
     - Un asiento de ventanilla a "Miguel Fernández"
     - Un asiento de ventanilla a "Ana Folgado"
     - Un asiento de pasillo a "David Más"
   - Mostrar el vuelo por pantalla
   - Cancelar la reserva del asiento que indique el usuario.
   - Mostrar de nuevo el vuelo por pantalla
3. (Compania) Implementación de la clase `Compania` para representar todos los vuelos de una compañía aérea. Una Compañía tiene un nombre y puede ofrecer, como mucho, 10 vuelos distintos. Para representarlos se utilizará `listaVuelos`, un array de objetos `Vuelo` junto con un atributo `numVuelos` que indique el número de vuelos que la compañía ofrece en un momento dado. Las operaciones de esta clase son:
   - `public Compania(String n) throws FileNotFoundException`. Constructor de una compañía de nombre `n`. Cuando se crea una compañía, se invoca al método privado `leeVuelos()` para cargar la información de vuelos desde un fichero. Si el fichero no existe, se propaga la excepción `FileNotFoundException`
   - `private void leeVuelos() throws FileNotFoundException`. Lee desde un fichero toda la información de los vuelos que ofrece la compañía y los va almacenando en el array de vuelos `listaVuelos`. El nombre del fichero coincide con el nombre de la compañía y tiene extensión `.txt`. La información de cada vuelo se estructura en el fichero como sigue:

     ```
     <Identificador>
     <Origen>
     <Destino>
     <Hora de salida>
     <Minuto de salida>
     <Hora de llegada>
     <Minuto de llegada>
     ...
     ...
     ```

     Si el fichero no existe, se propaga la excepción `FileNotFoundException`.
   - `public Vuelo buscarVuelo(String id) throws ElementoNoEncontradoException`. Dado un identificador de vuelo `id`, busca dicho vuelo en el array de vuelos `listaVuelos`. Si lo encuentra, lo devuelve. Si no, lanza `ElementoNoEncontradoException`.
   - `public void mostrarVuelosIncompletos(String o, String d)`. Muestra por pantalla los vuelos con origen `o` y destino `d`, y que tengan asientos libres. Por ejemplo, vuelos con asientos libres de la compañía Iberia con origen Milán y destino Valencia:

     ```
     Iberia IB201 Milán Valencia 14:25:00 16:20:00
     Iberia IB202 Milán Valencia 21:40:00 23:35:00
     ```

4. (GestorVuelos) En la clase `GestorVuelos` se probará el comportamiento de las clases anteriores. En esta clase se debe implementar el método `main` en el que, por simplificar, se pide únicamente:
   - la creación de la compañía aérea `Iberia`. Se dispone de un fichero de texto "`Iberia.txt`", con la información de los vuelos que ofrece.
   - Reserva de un asiento de ventanilla en un vuelo de Valencia a París por parte de Manuel Soler Roca. Para ello:
     - Mostraremos vuelos con origen Valencia y destino París, que no estén completos.
     - Pediremos al usuario el identificador del vuelo en que quiere hacer la reserva.
     - Buscaremos el vuelo que tiene el identificador indicado. Si existe realizaremos la reserva y mostraremos un mensaje por pantalla. En caso contrario mostraremos un mensaje de error por pantalla.

Paquete: `UD06._01_2maquinaExpendedora`

> **Excepciones y ficheros**
>
> Las excepciones propias se declaran dentro de la clase que las lanza, como clases anidadas (`static class NoHayCambioException extends Exception`, y lo mismo para `NoHayProductoException` y `CreditoInsuficienteException`), tanto en `Expendedora` como en `ExpendedoraSurtido`. El fichero `productos.txt` está en la carpeta `files/_01_2maquinaExpendedora/` del proyecto.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Se desea simular el funcionamiento de una máquina expendedora. Se trata de una expendedora sencilla que, por el momento, será capaz de dispensar únicamente un producto.

Su funcionamiento, a grandes rasgos, es el siguiente:

1. El cliente introduce dinero en la máquina. Al dinero introducido lo llamaremos `credito`.
2. Selecciona el producto que quiere comprar (ya hemos comentado que por el momento habrá un solo producto).
3. Si hay stock del producto seleccionado, la máquina dispensa el artículo elegido y devuelve el importe sobrante (diferencia entre el crédito introducido y el precio del producto).

Durante el proceso se pueden producir diversas incidencias, como por ejemplo, que el cliente no haya introducido suficiente crédito para comprar el producto, que no quede producto o que no haya cambio suficiente para la devolución. La máquina también da la posibilidad de solicitar la devolución del crédito sin realizar la compra.

1. (Expendedora) Diseñar la clase `Expendedora` (proyecto `Expendedora`) con los atributos y métodos que se describen a continuación.
    - Atributos (privados)
        - `credito`: Cantidad de dinero (en euros) introducida por el cliente.
        - `stock`: Número de unidades que quedan en la máquina disponibles para la venta. Se reducirá con cada nueva venta.
        - `precio`: Precio del único artículo que dispensa la máquina (en euros).
        - `cambio`: Cambio del que dispone la máquina. El cambio disponible se reduce cada vez que se devuelve al cliente la diferencia entre el crédito introducido y el precio del producto comprado. El cambio nunca se ve incrementado por las compras de los clientes.
        - `recaudación`: Representa la suma de las ventas realizadas por la máquina (en euros). Se ve incrementada con cada nueva compra.
    - Métodos:
        - Constructor: `public Expendedora (double cambio, int stock, double precio)`. Crea la expendedora inicializando los atributos cambio, stock y precio con los valores indicados en los parámetros). El crédito y la recaudación serán cero.
        - Consultores:
            - Métodos consultores para los atributos crédito, cambio, y recaudación
            - Los consultores para el stock y el precio los haremos previendo que en el futuro la máquina pueda expender más de un tipo de producto. Para consultar el stock y el precio se indicará como parámetro el número de producto que se quiere consultar aunque, por el momento se ignorará el valor de dicho atributo.
            - `public getStock (int producto)` Devuelve el stock disponible del producto indicado. En esta versión simplificada se devolverá el valor del atributo stock, sea cual sea el valor de producto.
            - `public getPrecio (int producto)` Devuelve el precio del producto indicado. En esta versión simplificada se devolverá el valor del atributo precio, sea cual sea el valor de producto.
        - Modificadores: Para simplificar, consideramos que los atributos de la máquina solo van a cambiar por operaciones derivadas de su funcionamiento, por lo que no proporcionamos modificadores públicos
        - Otros métodos:
            - `public String toString()` Devuelve un `String` de la forma:

```
Credito: 3.00 euros
Cambio: 12.73 euros
Stock: 12 unidades
Recaudación: 127.87 euros
```

            - `public void introducirDinero(double importe)` Representa la operación mediante la cual el cliente añade dinero (crédito) a la máquina. Esta operación incrementa el crédito introducido por el cliente en el importe indicado como parámetro.
            - `public double solicitarDevolucion()` Representa la operación mediante la cual el cliente solicita la devolución del crédito introducido sin realizar la compra. El método devuelve la cantidad de dinero que se devuelve al cliente.
            - `public double comprarProducto(int producto) throws NoHayCambioException, NoHayProductoException, CreditoInsuficienteException`. Representa la operación mediante la cual el cliente selecciona un producto para su compra. El método devuelve la cantidad de dinero que se devuelve al cliente.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Si no se produce ninguna situación inesperada, se reduce el stock del producto, se devuelve el cambio, se pone el crédito a cero y se incrementa la recaudación.

Si la venta no es posible se lanzará la excepción correspondiente a la situación que impide completar la venta.

2. (Producto) La clase `Producto` permite representar uno de los artículos de los que vende una máquina expendedora. Para ello utilizaremos tres atributos privados `nombre` (`String`), `precio` (`double`) y `stock` (`int`), y los siguientes métodos:
   - `public Producto(String nombre, double precio, int stock)` Constructor que inicializa el producto con los parámetros indicados
   - Consultores de los tres atributos: `getNombre`, `getPrecio` y `getStock`
   - `public int decrementarStock()`: Decrementa en 1 el stock del producto y devuelve el stock resultante.
3. (TestExpendedora) La clase `TestExpendedora` sirve para provar los métodos desarrollados en las clases `Expendedora` y `Producto`.
   - Crea un Objeto de tipo `Expendedora` e inicializalo con: 12 unidades de stock, 5 euros de cambio y un precio de 3.75 euros. Muestra por pantalla su estado actual.
   - Simula la introducción por parte del cliente de un billete de 5 euros y muestra el estado de la máquina `Expendedora`.
   - Simula la compra de un `producto` y muestra la cantidad devuelta.
   - Simula la introducción de una moneda de 2 euros y solicita la devolución sin realizar ninguna compra y muestra la cantidad devuelta.
   - Intenta realizar una compra sin tener suficiente crédito y gestiona la excepción.
   - Crea otro objeto de tipo `Expendedora` que inicialmente tenga 0 unidades de stock (el resto de valores a tu gusto), simula la compra de un producto teniendo suficiente crédito y cambio. Gestiona la excepción.
   - Crea un último objeto de tipo `Expendedora` que inicialmente tenga 0 euros de cambio (el resto de valores a tu gusto), simula la compra de un producto para el que la máquina tenga que devolver algún importe, gestiona la excepción.
   - Muestra las recaudaciones para las 3 máquinas expendedoras.
4. (Surtido) La clase `Surtido` representa una colección de productos. Para ello se usará un atributo `listaProductos`, array de `Productos`. El array se rellenará con los datos de productos extraidos de un fichero de texto y, una vez creado el surtido no será posible añadir o quitar productos. Así, el array de productos estará siempre completo y no es necesario ningún atributo que indique cuantos productos hay en el array.

   Se implementarán los siguientes métodos:

   - `public Surtido() throws FileNotFoundException` Crea el surtido con los datos de los productos que se encuentran en el fichero `productos.txt`. El fichero tiene el siguiente formato:

     ```
     <nº de productos>
     <nombre de producto> <precio> <stock>
     <nombre de producto> <precio> <stock>
     <nombre de producto> <precio> <stock>
     ...
     ```

   Como vemos, la primera línea del fichero indica el número de productos que contiene el surtido. Este dato lo usaremos para dar al array de productos el tamaño adecuado. Ten en cuenta que la excepción `FileNotFoundException` hereda de `IOException`.

   - `public int numProductos()` Devuelve el número de productos que componen el surtido
   - `public Producto getProducto(int numProducto)`: Devuelve el producto que ocupa la posición `numProducto` del surtido. La primera posición válida es la `1`. La posición `0` no se utiliza.
   - `public String[] getNombresProductos()` Devuelve un array con los nombres de los productos. La posición `0` del array no se utilizará (será `null`)
5. (ExpendedoraSurtido) Crea una copia de la clase `Expendedora` y llámala `ExpendedoraSurtido`, con el constructor `public ExpendedoraSurtido(double cambio)`. Añadir los atributos y hacer los cambios necesarios en la clase para que sea capaz de dispensar varios productos usando la nueva clase `Surtido`. Por ejemplo ya no tienen sentido los atributos stock y precio ya que pertenecen al `Surtido`. Modifica también el método `public String toString()`, para que muestre por pantalla el listado de productos con su nombre, precio y stock para mostrar al cliente que productos puede elegir. El código del producto coincidirá con su posición al leer el surtido.
6. (TestExpendedoraSurtido) Crea una copia de la clase `TestExpendedora` y nómbrala `TestExpendedoraSurtido` para adaptarla a los cambios hechos en la clase `Expendedora` y usando la nueva posibilidad de comprar diferentes productos y usando solamente un único objeto `Expendedora`. Al final en lugar de mostrar la recaudación de las 3 máquinas expendedoras, muestra solo la de la única que hay, que muestra el surtido de esta manera:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
Prod.    Stock   Precio
CocaCola 18  1.5
Snickers 15  1.2
PotatoChips  25  1.0
Water    30  0.8
HamSandwich  10  2.5
```

7.3.2 Los flujos estándar

1. (clase `LeeNombre`) Escribir un programa que solicite al usuario su nombre y, utilizando directamente `System.in`, lo lea de teclado y muestre por pantalla un mensaje del estilo "Su nombre es Miguel". Recuerda que `System.in` es un objeto de tipo `InputStream`. La clase `InputStream` permite **leer bytes** utilizando el método `read()`. Será tarea nuestra ir construyendo un `String` a partir de los bytes leídos. Prueba el programa de manera que el usuario incluya en su nombre algún carácter "extraño", por ejemplo el símbolo "€" ¿Funciona bien el programa? ¿Por qué?
2. (clase `LeeEdad`) Escribir un programa que solicite al usuario su edad y, utilizando directamente `System.in`, la lea de teclado y muestre por pantalla un mensaje del estilo "Su edad es 32 años". En este caso, será tarea nuestra construir un String a partir de los bytes leídos y transformarlo posteriormente en un entero.
3. (clase `CambiarEstandar`). La salida estándar (`System.out`) y la salida de errores (`System.err`) están asociadas por defecto con la pantalla. Se puede cambiar este comportamiento por defecto utilizando los métodos `System.setout` y `System.seterr` respectivamente. Investiga un poco cómo se utilizan, escribe un programa que asocie la salida estándar a al fichero `salida.txt` y la salida de errores al fichero `errores.txt` y, a continuación, escriba algún mensaje en cada uno de las salidas, por ejemplo `System.out.println("El resultado es 20");` y `System.err.println("ERROR: Elemento no encontrado");`
4. (`SumarEdades`) - Escribir método `void sumaEdades()` que lea de teclado las edades de una serie de personas y muestre cuanto suman. El método finalizará cuando el usuario introduzca una edad negativa. - Escribir un método `main` que llame al método anterior para probarlo. - Modificar el método `main` de forma que, antes de llamar al método `sumaEdades`, se cambie la entrada estándar para que tome los datos del fichero `edades.txt` en lugar de leerlos de teclado.

7.3.3 `InputStreamReader`

1. (`leerByte`) `System.in` (`InputStream`) está orientado a lectura de bytes. Escribe un programa que lea un byte de teclado y muestre su valor (int) por pantalla. Pruébalo con un carácter "extraño", por ejemplo '€'.
2. (`leerCaracter`) `InputStreamReader` (`StreamReader`) está orientado a carácteres. Escribe un programa que lea un carácter de teclado usando un `InputStreamReader` y muestre su valor (`int`) por pantalla. Pruébalo con un carácter "extraño", por ejemplo '€'. ¿Se obtiene el mismo resultado que en el ejercicio anterior?.

7.3.4 Entrada "orientada a líneas".

En los ejercicios anteriores, las limitaciones de la clase utilizada (`InputStream`), nos obliga a incluir en el programa instrucciones que detecten que el usuario ha terminado su entrada (ha pulsado **INTRO**). La clase `BuffereReader` dispone del método `readLine()`, capaz de leer una línea completa (la propia instrucción detecta el final de la línea) y devolver un `String`.

7.- Repite el ejercicio 1 utilizando un `BufferedReader` asociado a la entrada estándar. La clase `BufferedReader`, está orientada a leer caracteres en lugar de bytes. ¿Qué ocurre ahora si el usuario introduce un carácter "extraño" en su nombre?

8.- Repite el ejercicio 2 utilizando un `BufferedReader` asociado a la entrada estándar.

7.3.5 Lectura/escritura en ficheros

1. (EscribirFichero1) Escribe un programa que, usando las clases `FileOutputStream` y `FileInputStream`, (puedes usar más a parte de estas dos)
   - escriba tu nombre y altura en un fichero (`files/_05_LE_Ficheros/nombre.txt`).
   - lea el fichero creado y lo muestre por pantalla.
   - Si abrimos el fichero creado con un editor de textos, ¿su contenido es legible?
2. (EscribirFichero2) Repetir el ejercicio anterior utilizando las clases `FileReader` y `FileWriter` para generar el fichero `files/_05_LE_Ficheros/nombre2.txt`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

7.3.6 Uso de buffers

Los buffers hacen que las operaciones de lectura-escritura se realicen inicialmente en memoria y, cuando los buffers correspondientes están vacíos/llenos, se hagan definitivamente sobre el dispositivo.

1. (TestVelocidadBuffer) Vamos a probar la diferencia de tiempo que conlleva escribir datos a un fichero directamente o hacerlo a través de un buffer. Para ello, crea un fichero de 1 Mb (1000000 de bytes aprox.) usando un la clase `FileWriter` y mide el tiempo que tarda en crearlo. Posteriormente, crea un fichero de exactamente el mismo tamaño utilizando un `BufferedWriter` y mide el tiempo que tarda. ¿Hay diferencia?. Para medir el tiempo puedes utilizar `System.currentTimeMillis()`, inmediatamente antes y después de crear el fichero y restar los valores obtenidos.
2. (TestVelocidadBuffer2) Modifica el programa `TestVelocidadBuffer` para probar cómo afecta a la escritura con buffer la ejecución de la instrucción `flush()`. Esta instrucción fuerza el volcado del buffer a disco. ¿Disminuye la velocidad si tras cada operación de escritura ejecutamos flush()?
3. (TestVelocidadBuffer3) Modifica el programa `TestVelocidadBuffer` para probar cómo a la velocidad el tamaño del buffer. La clase `BufferedWriter` tiene un constructor que permite indicar el tamaño del buffer. Prueba con distintos valores.

7.3.7 Streams para información binaria

1. (Personas) Escribe un programa que, utilizando entre otras la clase `DataOutputStream`, almacene en un fichero llamado `personas.dat` (en la carpeta `files/_07_StreamsParaInformacionBinaria/`) la información relativa a una serie de personas que va introduciendo el usuario desde teclado (se guardan en este orden):
   - `Nombre` (String)
   - `Edad` (entero)
   - `Peso` (double)
   - `Estatura` (double)

   La entrada del usuario terminará cuando se introduzca un nombre vacío.

   Nota: Utiliza la clase `Scanner` para leer desde teclado y los métodos `writeDouble`, `writeInt` y `writeUTF` de la clase `DataOutput/InputStream` para escribir en el fichero)

   Al finalizar el programa, abre el fichero resultante con un editor de texto (notepad o wordpad) ¿La información que contiene es legible?.
2. (AñadirPersonas) Modifica el programa anterior para que el usuario, al comienzo del programa, pueda elegir si quiere añadir datos al fichero o sobre escribir la información que contiene.

> ⚠ **Recomendación para Test**
>
> La pregunta debería ser del estilo de "¿Deseas añadir más datos? si eliges NO borraras el contenido anterior. ('S' para si o 'N' para no)"
>
> El test usará los carácteres S o N para probar en ese sentido.

1. (MostrarPersonas) Realizar un programa que lea la información del fichero `personas.dat` y la muestre por pantalla. Para determinar que no quedan más datos en el fichero podemos capturar la excepción `EOFException`

> ⚠ **Recomendación para Test**
>
> La salida esperada debe ser similar a:
>
> ```
> Registro 1: David 35 85,00 1,87
> Registro 2: Maria 23 52,00 1,67
> Registro 3: Pepe 46 67,00 1,80
> Fichero leido completamente
> ```

1. (CalculosPersonas) Realizar un programa, similar al anterior, que lea la información del fichero `personas.dat` y muestre por pantalla la estatura que tienen de media las personas cuya edad está entre 20 y 30 años.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Recomendación para Test

La salida esperada debe ser similar a:

```
Información sobre las personas de entre 20 y 30 años:
La media de estatura es: 1,67
```

7.3.8 Streams de objetos. Serialización.

1. (GuardaLibros) Paquete `GuardaLeeLibros`
   - (`Autor`) Crea la clase `Autor`, con los atributos **nombre**, **año de nacimiento** y **nacionalidad**. Incorpora un **constructor** `Autor(String nombre, int anyoNacimiento, String nacionalidad)` y el método `toString()`.
   - (`Libro`) Crea la clase `Libro`, con los atributos **titulo**, **año de edición** y **autor** (Objeto de la clase `Autor`). Incorpora un **constructor** `Libro(String titulo, int anyoEdicion, Autor autor)` y el método `toString()`, que incluye los datos de su autor.
   - Escribe un programa `GuardaLibros` que cree tres libros y los almacene en el fichero `files/_08_Serializacion/biblioteca.obj`, cada libro con su propio `writeObject`.
   - Las clases (también `Autor`) deberán implementar el interfaz `Serializable`.
2. (LeeLibros) Paquete `GuardaLeeLibros`
   - Escribe un programa (`LeeLibros`) que lea los objetos del fichero `biblioteca.obj` y los muestre por pantalla (Libros y Autores).

7.3.9 Sockets

1. (Zodiaco) Programar un Servidor que reciba una fecha (previamente validada por el cliente) y nos diga cual es nuestro signo del zodíaco occidental y el animal que corresponde en el zodíaco oriental (animales).

Test

Las clases son `TCPServidor` y `TCPClient`. El servidor está en `localhost` y el puerto se recibe como primer argumento del `main` (`args[0]`); si no hay argumentos, se usa el 6000. El cliente pide la fecha con formato `dd/mm/yyyy` y la envía al servidor como un `LocalDate` (con `ObjectOutputStream`). El servidor responde con dos `String`: el signo del zodíaco occidental y el animal del zodíaco oriental, que se calcula con el resto de dividir el año entre 12 (0 mono, 1 gallo, 2 perro, 3 cerdo, 4 rata, 5 buey, 6 tigre, 7 conejo, 8 dragón, 9 serpiente, 10 caballo, 11 oveja).

7.3.10 Más ejercicios (Lionel)

Importante

Para probar algunos de estos ejercicios debes utilizar el archivo `Documentos.zip`. Descárgalo del aula virtual y descomprímelo en la carpeta de cada proyecto que crees.

Para superar los tests, además, se debe mantener la ruta de la carpeta Documentos en `files/_10_MasEjercicios/Documentos` dentro del proyecto de IntelliJ, como algunos ejercicios borraran y moveran contenido, podrás descomprimir la carpeta `Documentos` a partir del archivo Documentos.zip que se incluye con el proyecto.

1. **Mostrar información de ficheros**

   Implementa un programa que pida al usuario introducir por teclado una ruta del sistema de archivos (por ejemplo, `C:/Windows` o `Documentos`) y muestre información sobre dicha ruta (ver función más abajo). El proceso se repetirá una y otra vez hasta que el usuario introduzca una ruta vacía (tecla intro). Deberá manejar las posibles excepciones.

   Necesitarás crear la función `void muestraInfoRuta(File ruta)` que dada una ruta de tipo `File` haga lo siguiente:

   - Si es un archivo, mostrará por pantalla el nombre del archivo.
   - Si es un directorio, mostrará por pantalla la lista de directorios y archivos que contiene (sus nombres). Deberá mostrar primero los directorios y luego los archivos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- En cualquier caso, añade delante del nombre la etiqueta `[*]` o `[A]` para indicar si es un directorio o un archivo respectivamente.
- Si el path no existe lanzará un `FileNotFoundException`.

2. **Mostrar información de ficheros (v2)**

   Partiendo de una copia del programa anterior, modifica la función `muestraInfoRuta`:

   - En el caso de un directorio, mostrará la lista de directorios y archivos en orden alfabético. Es decir, primero los directorios en orden alfabético y luego los archivos en orden alfabético.
   - Añade un segundo argumento `boolean info` que cuando sea `true` mostrará, junto a la información de cada directorio o archivo, su tamaño en bytes y la fecha de la última modificación. Cuando `info` sea `false` mostrará la información como en el ejercicio anterior.

3. **Renombrando directorios y ficheros**

   Implementa un programa que haga lo siguiente:

   - Cambiar el nombre de la carpeta `Documentos` a `DOCS`, el de la carpeta `Fotografias` a `FOTOS` y el de la carpeta `Libros` a `LECTURAS`
   - Cambiar el nombre de todos los archivos de las carpetas `FOTOS` y `LECTURAS` quitándole la extensión, con un método `quitarExtensionArchivos(File carpeta)`. Por ejemplo, `astronauta.jpg` pasará a llamarse `astronauta`.

4. **Creando (y moviendo) carpetas**

   Implementa un programa que cree, dentro de `Documentos`, dos nuevas carpetas: `MisCosas` y `Alfabeto`. Mueve las carpetas `Fotografias` y `Libros` dentro de `MisCosas`. Luego crea dentro de `Alfabeto` una carpeta por cada letra del alfabeto (en mayúsculas): `A`, `B`, `C`... `Z`. Te serán de ayuda los códigos numéricos ASCII: https://elcodigoascii.com.ar

5. **Borrando archivos**

   Implementa un programa con una función `boolean borrarTodo(File f)` que borre `f`: Si no existe lanzará una excepción. Si es un archivo lo borrará. Si es un directorio, borrará primero sus archivos y luego el propio directorio (recuerda que para poder borrar un directorio debe estar vacío). Devolverá `true` si pudo borrar el `File f` (`false` si no fué posible).

   Prueba la función borrando las carpetas: `Documentos/Fotografias`, `Documentos/Libros` y `Documentos` (es decir, tres llamadas a la función, en ese orden).

   Super extra challenge: Esta función, tal y como está definida, no borrará las subcarpetas que estén dentro de una carpeta (para ello habría que borrar primero el contenido de dichas subcarpetas). ¿Se te ocurre cómo podría hacerse? Resuélvelo en la clase `Ejercicio_25_Extra`, con una función `borrarTodo(File f)` que también borre las subcarpetas y su contenido.

6. **Máximo y mínimo**

   Implementa un programa que muestre por pantalla los valores máximos y mínimos del archivo `numeros.txt`.

7. **Notas de alumnos**

   El archivo `alumnos_notas.txt` contiene una lista de 10 alumnos y las notas que han obtenido en cada asignatura. El número de asignaturas de cada alumno es variable. Implementa un programa que muestre por pantalla la nota media de cada alumno junto a su nombre y apellido, ordenado por nota media de mayor a menor.

```
LISTADO DE NOTAS MEDIAS DE LOS ALUMNOS
--------------------------------------
7,40 Toni Harper
6,50 Patrick Santos
6,25 Michele Poole
6,00 Troy Walters
5,83 Joanna Rogers
5,75 Ron Garza
5,43 Malcolm Lindsey
5,00 Gilbert Santiago
4,50 Gabriel Moreno
4,00 Vivian Chambers
```

8. **Ordenando archivos**

   Implementa un programa que pida al usuario un nombre de archivo `A` para lectura y otro nombre de archivo `B` para escritura. Leerá el contenido del archivo `A` (por ejemplo `usa_personas.txt`) y lo escribirá ordenado alfabéticamente en B (por ejemplo `usa_personas_sorted.txt`).

9. **Nombre y apellidos**

   Implementa un programa que genere aleatoriamente nombres de persona (combinando nombres y apellidos de `usa_nombres.txt` y `usa_apellidos.txt`). Se le pedirá al usuario cuántos nombres de persona desea generar y a qué archivo **añadirlos** (por ejemplo `usa_personas.txt`).

10. **Diccionario**

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Implementa un programa que cree la carpeta `Diccionario` con tantos archivos como letras del abecedario (`A.txt`, `B.txt`... `Z.txt`). Introducirá en cada archivo las palabras de `diccionario.txt` que comiencen por dicha letra.

11. **Búsqueda en PI**

    Implementa un programa que pida al usuario un número de cualquier longitud, como por ejemplo "1234", y le diga al usuario si dicho número aparece en el primer millón de decimales del nº pi (están en el archivo `pi-million.txt`). No está permitido utilizar ninguna librería ni clase ni método que realice la búsqueda. Debes implementar el algoritmo de búsqueda tú.

12. **Estadísticas**

    Implementa un programa que lea un documento de texto y muestre por pantalla algunos datos estadísticos: nº de líneas, nº de palabras, nº de caracteres y cuáles son las 10 palabras más comunes (y cuántas veces aparecen) `public static void muestraPalabrasMasComunes(Hashtable<String, Integer> t)`. Prueba el programa con los archivos de la carpeta `Libros`.

    NOTA: *Para llevar la cuenta de cuántas veces aparece cada palabra puedes utilizar una Colección que almacene pares de clave/valor. Por ejemplo {"elefante", 5} o {"casa", 10} son pares que asocian una palabra (clave) con un nº entero (valor).*

```
ESTADÍSTICAS DE LIBROS
----------------------

Libro: quijote_cervantes.txt
Lineas totales: 37861
Número de palabras: 393764
Número de carácteres: 1723734
Las 10 palabras más comunes son:
que=19429
de=17988
y=15894
la=10200
a=9575
=9504
el=7957
en=7898
no=5611
se=4690

Libro: vida_unamuno.txt
Lineas totales: 10309
Número de palabras: 106585
Número de carácteres: 482163
Las 10 palabras más comunes son:
de=5355
la=4151
que=4056
y=3197
=2951
el=2679
en=2380
a=2096
es=1857
no=1671

Libro: lazarillo.txt
Lineas totales: 2504
Número de palabras: 23860
Número de carácteres: 102364
Las 10 palabras más comunes son:
y=1020
que=875
de=754
=695
a=525
la=524
el=449
en=410
no=325
con=270
....
```

7.3.11 Aún más ejercicios

1. (`CuentaLineas`) Escribe un programa que, sin utilizar la clase `Scanner` para contar las lineas, muestre el número de lineas que contiene un fichero de texto. El nombre del fichero se solicitará al usuario al comienzo de la ejecución (para esto si puedes usar la clase `Scanner`).
2. (`CuentaPalabras`) Escribe un programa que, sin utilizar la clase `Scanner` para contar las lineas, muestre el número de palabras que contiene un fichero de texto. El nombre del fichero se solicitará al usuario al comienzo de la ejecución (para esto si puedes usar la clase `Scanner`).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

> **Consejo**
>
> Lee el fichero, línea a línea y utiliza la clase `StringTokenizer` o bien el método `split` de la clase `String` para averiguar el nº de palabras.

3. (`Censura`) Escribir un programa que sustituya por otras, ciertas palabras de un fichero de texto. Para ello, se desarrollará y llamará al método `void aplicaCensura(String entrada, String censura, String salida),` que lee de un fichero de entrada y mediante un fichero de censura, crea el correspondiente fichero modificado. Por ejemplo:

   Fichero de entrada:

   ```
   En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que vivía un hidalgo de los de lanza en astillero
   ```

   Fichero de censura:

   ```
   lugar sitio
   quiero debo
   hidalgo noble
   ```

   Fichero de salida:

   ```
   En un sitio de la Mancha, de cuyo nombre no debo acordarme, no ha mucho tiempo que vivía un noble de los de lanza en astillero
   ```

   > **Consejo**
   >
   > Valora la posibilidad de cargar el fichero de censura en un mapa o par clave, valor.

4. (`Concatenar1`) Escribe un programa que dados dos ficheros de texto `f1` y `f2` (introducidos por el usuario por teclado) confeccione un tercer fichero `f3` cuyo contenido sea el de `f1` y a continuación el de `f2` .
5. (`Concatenar2`) Escribe un programa que dados dos ficheros de texto `f1` y `f2` (introducidos por el usuario por teclado) , añada al final de `f1` el contenido de `f2` . Es decir, como el ejercicio anterior, pero sin producir un nuevo fichero.
6. (`Iguales`) Escribir un programa que compruebe si el contenido de dos ficheros es idéntico. Puesto que no sabemos de qué tipo de ficheros se trata, (de texto, binarios, ...) habrá que hacer una comparación byte por byte.
7. (GenerarMedia) Escribe los siguientes métodos y programa:
   - Método `void generar()` que genere 20 números aleatorios enteros entre 1 y 100 y los muestre por pantalla.
   - Método `void media()` que lea de teclado 20 números enteros y calcule su media.
   - Programa que, modificando la entrada y la salida estándar, llame a `generar()` para que los datos se graben en un fichero y a continuación llame a `media()` de manera que se tomen los datos del fichero generado.
8. (`Notas`) Escribir un programa que almacene en un fichero binario (`notas.dat`) las notas de 20 alumnos. El programa tendrá el siguiente funcionamiento:
   - En el fichero se guardarán como máximo 20 notas, pero se pueden guardar menos. El proceso de introducción de notas (y en consecuencia, el programa) finalizará cuando el usuario introduzca una nota no válida (menor que cero o mayor que 10).
   - Si, al comenzar la ejecución, el fichero ya contiene notas, se indicará al usuario cuántas faltan por añadir y las notas que introduzca el usuario se añadirán a continuación de las que hay.
   - Si, al comenzar la ejecución, el fichero ya contiene 20 notas, se le preguntará al usuario si desea sobrescribirlas. En caso afirmativo las notas que introduzca sustituirán a las que hay y en caso negativo el fichero no se modificará.

28 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

7.4 Talleres

7.4.1 Taller UD06_01: Classroom 50

Aceptar la tarea en Classroom 50

Aceptamos el *Assignement* (la tarea/ejercicio) a partir del link del profesor, en este caso: https://classroom50.org/IES-Eduardo-Primo-Marques/1dam-programacio-java/assignments/ud06

Si no recuerdas cómo aceptar la tarea, clonar el repositorio o abrirlo en IntelliJ, repasa la guía completa en el taller de UD02.

Tarea

Debes enviar tus soluciones a Classroom 50 y superar al menos la mitad de los tests, cuantos más tests superados, mejor nota tendrás en la tarea.

6 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

7.4.2 Taller UD06_T02: Sockets en la nube (AWS)

La intención de este documento es la de dar una perspectiva más realista del uso de sockets, ya que en lugar de usar la misma máquina del alumno como cliente y servidor, vamos a desplegar el servidor del socket en una máquina alojada en la nube de Amazon (AWS).

Requisitos

Para realizar esta práctica guiada necesitamos:

- Acceso al Learner Lab proporcionado por el profesor. (https://awsacademy.instructure.com)
- Conocimientos sobre los sockets, IP's y puertos.
- Un dispositivo local con capacidad de ejecutar un cliente de socket, con acceso a los puertos e Ip's de AWS (Ojo con la red de conselleria)

Guía paso a paso

PREPARAR EL ENTORNO DE LA NUBE

Iniciar Laboratorio

Lo primero que necesitamos es arrancar el laboratorio, para ello Accedemos al LMS del awsacademy, buscamos el Curso facilitado por el docente, accedemos a sus contenidos y a continuación al Learner Lab. (Si es la primera vez que accedemos debemos aceptar los términos de uso).

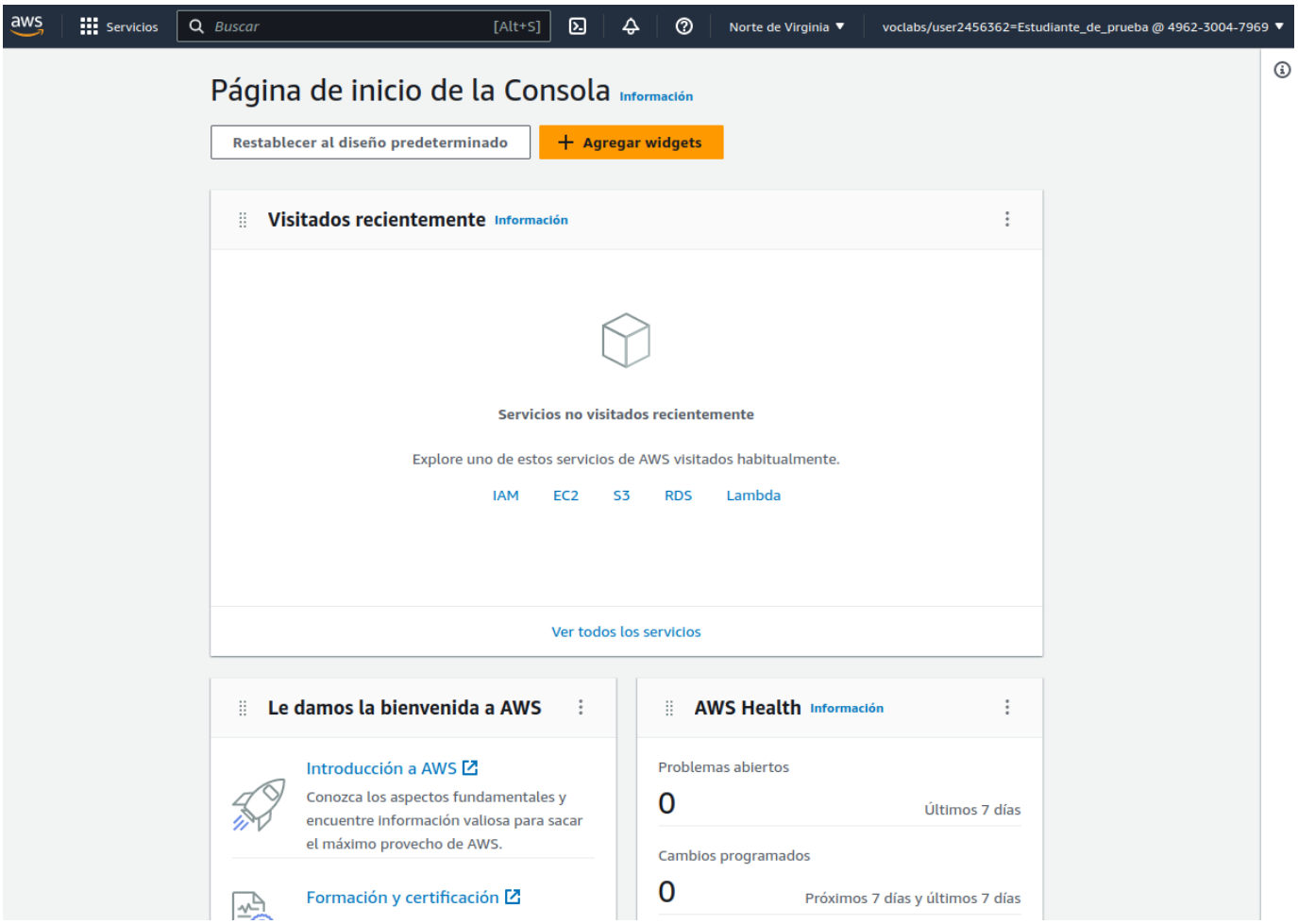
Inicialmente el laboratorio está en rojo:

Elegimos la opción `Start Lab` y esperamos a que aparezca el laboratorio en verde:

Por defecto el Learner Lab nos proporciona $_{100}$ 50 dolares de saldo, y un tiempo de 4 horas, tras el cual se detendrán la mayoría de servicios que tengamos en marcha. Pero mientras quede saldo podemos volver a iniciar el Laboratorio y dispondremos de 4 horas más.

Una vez aparece en verde podemos hacer click sobre las letras AWS y aparecerá el Dashboard de AWS (debemos permitir las ventanas emergentes):

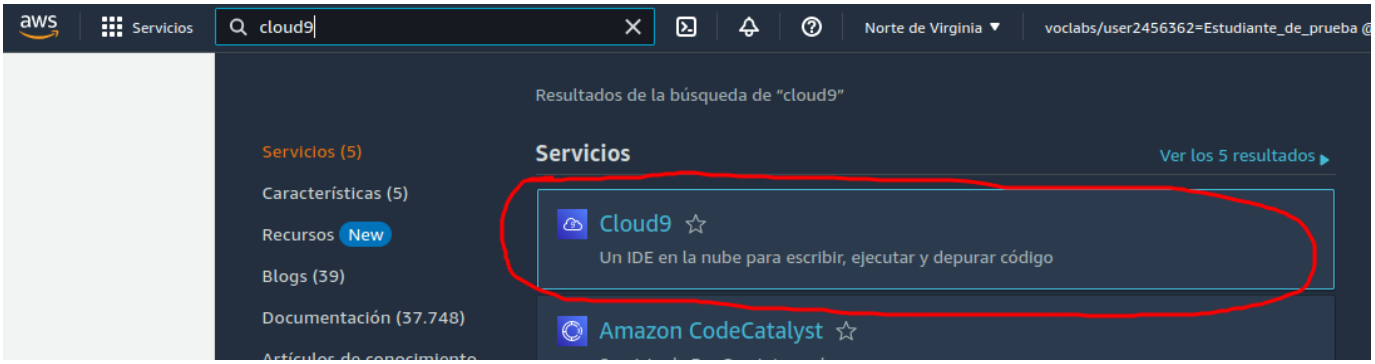
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Crear un entorno Cloud9

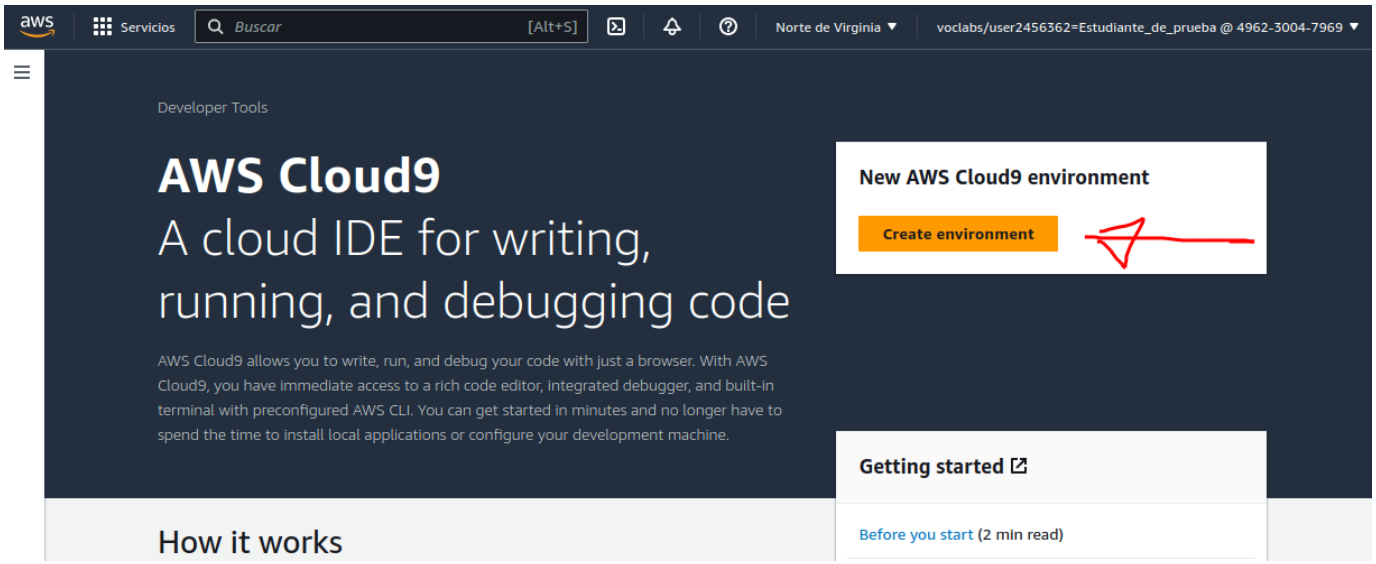
Cloud9 es un entorno de desarrollo en la nube que proporciona AWS asociado a una instáncia EC2 (máquina virtual en la nube).

El primer paso es crear este entorno, para ello buscamos cloud9 en la parte superior del Dashboard:

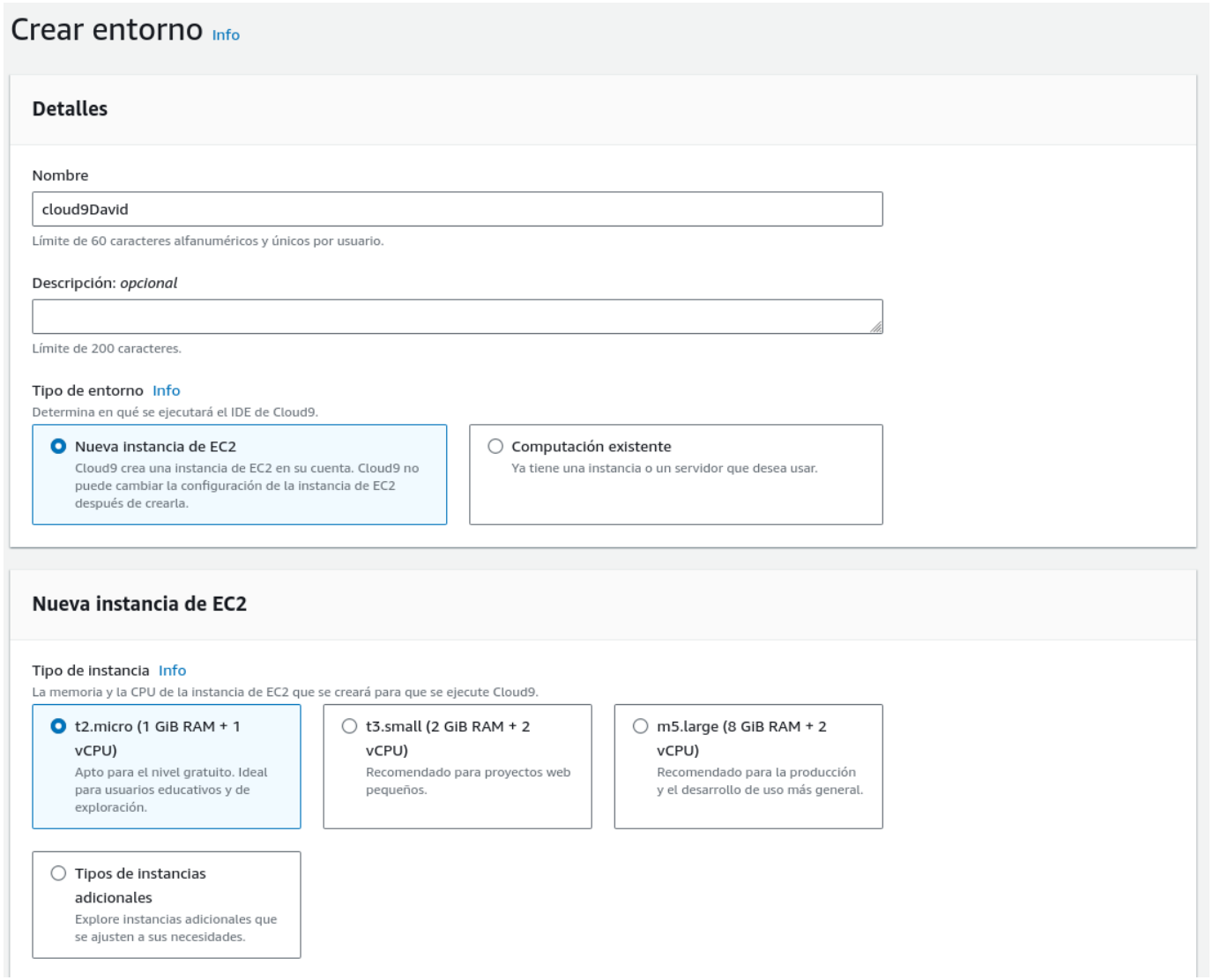


A continuación seleccionamos `Create environment`:

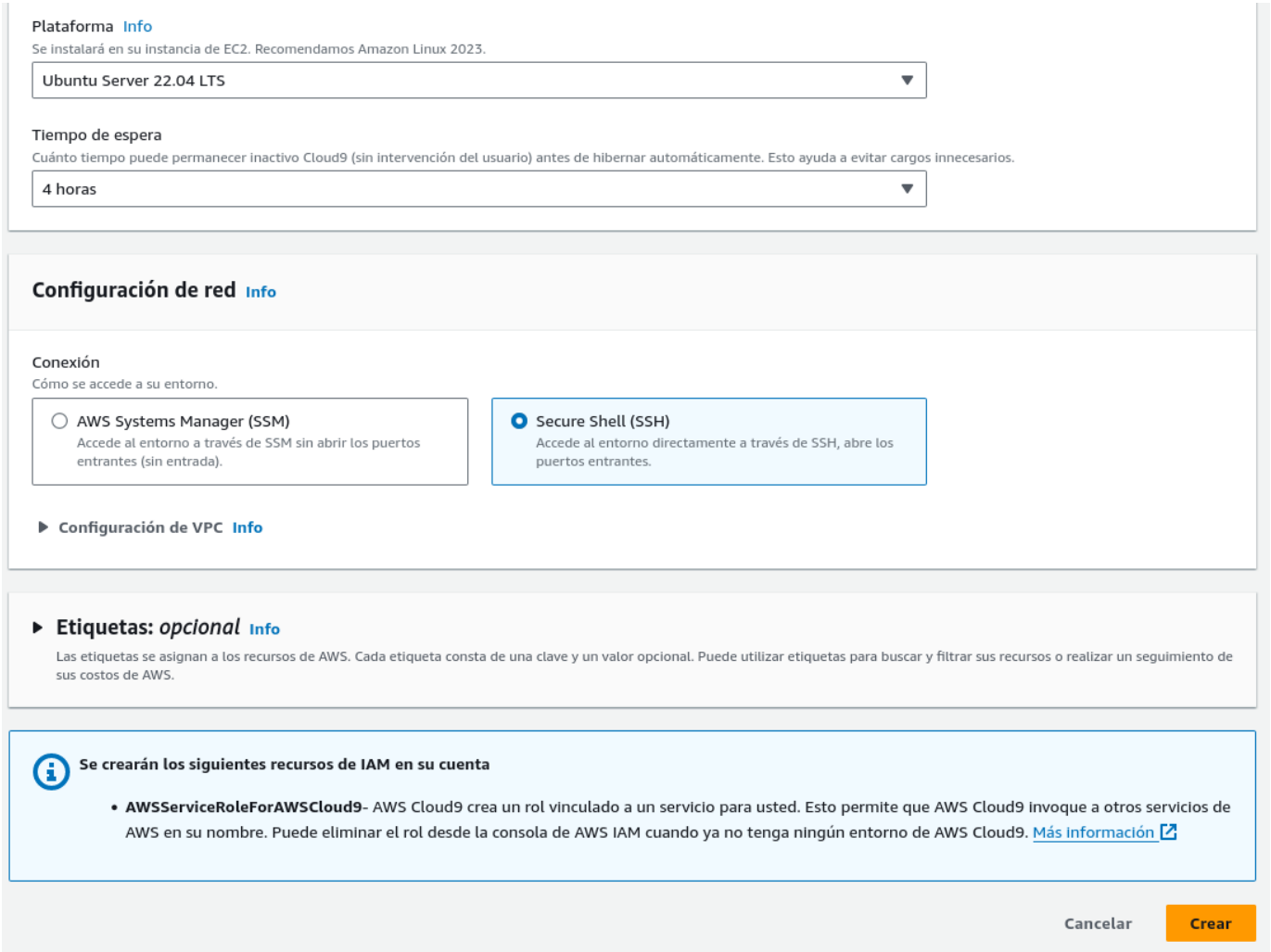
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



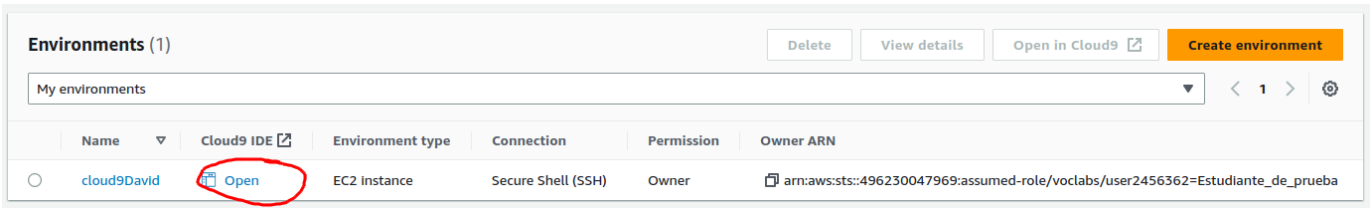
En la siguiente ventana debemos especificar el **nombre** (`Name`), cambiaremos la **plataforma** a `Ubuntu Server 18.04 LTS` , también podemos ampliar el tiempo de Timeout para no tener problemas a `4 horas` i por último dentro de los **Network settings** elegiremos la conexión por `SSH` , el resto de opciones se quedan por defecto y pulsamos el botón naranja del final `Create` .



© 2025 David Martínez licensed under CC BY-NC-SA 4.0

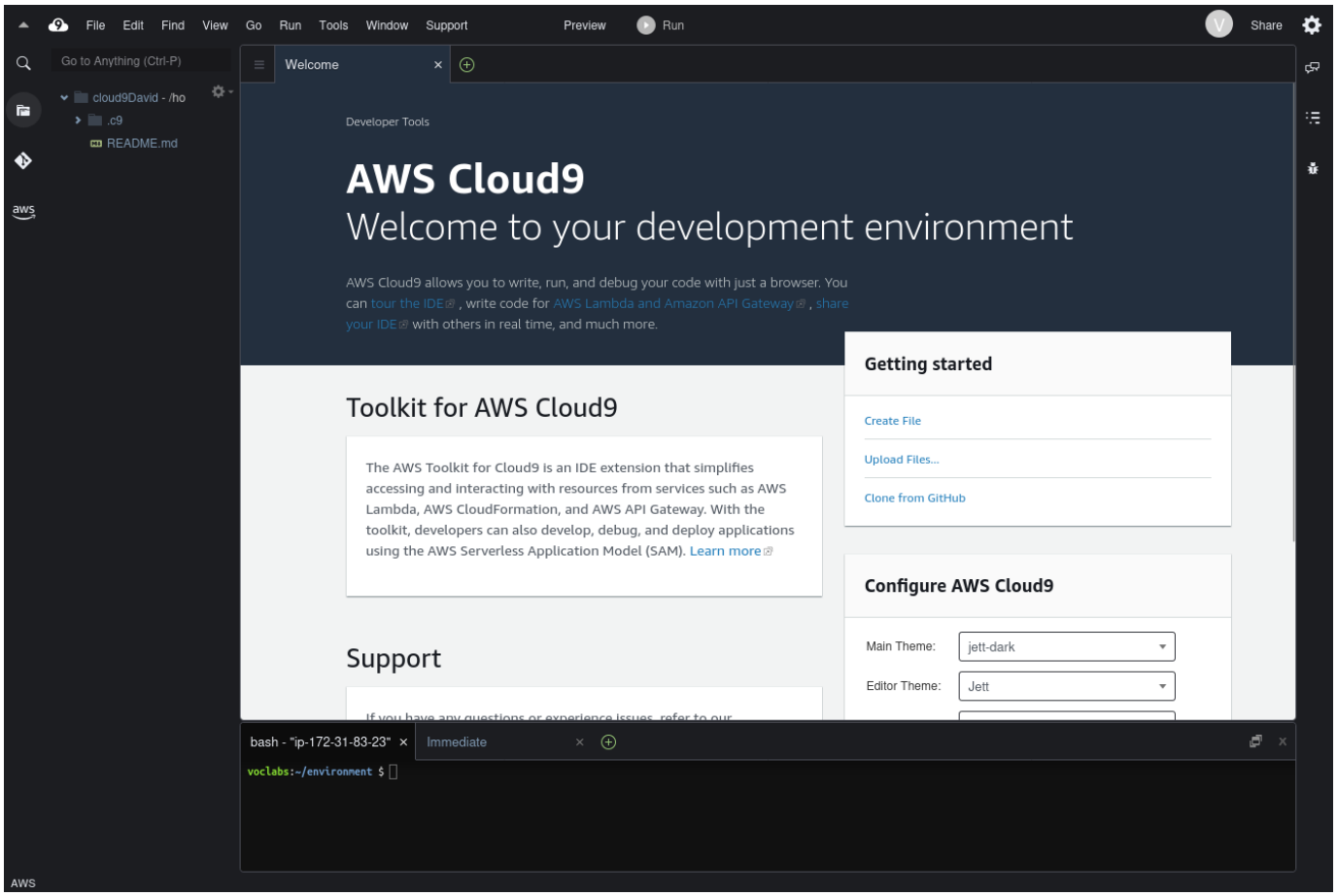


Si todo ha ido bien podemos seleccionar el botón `Open`:



Y deberíamos ver algo parecido a esto:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



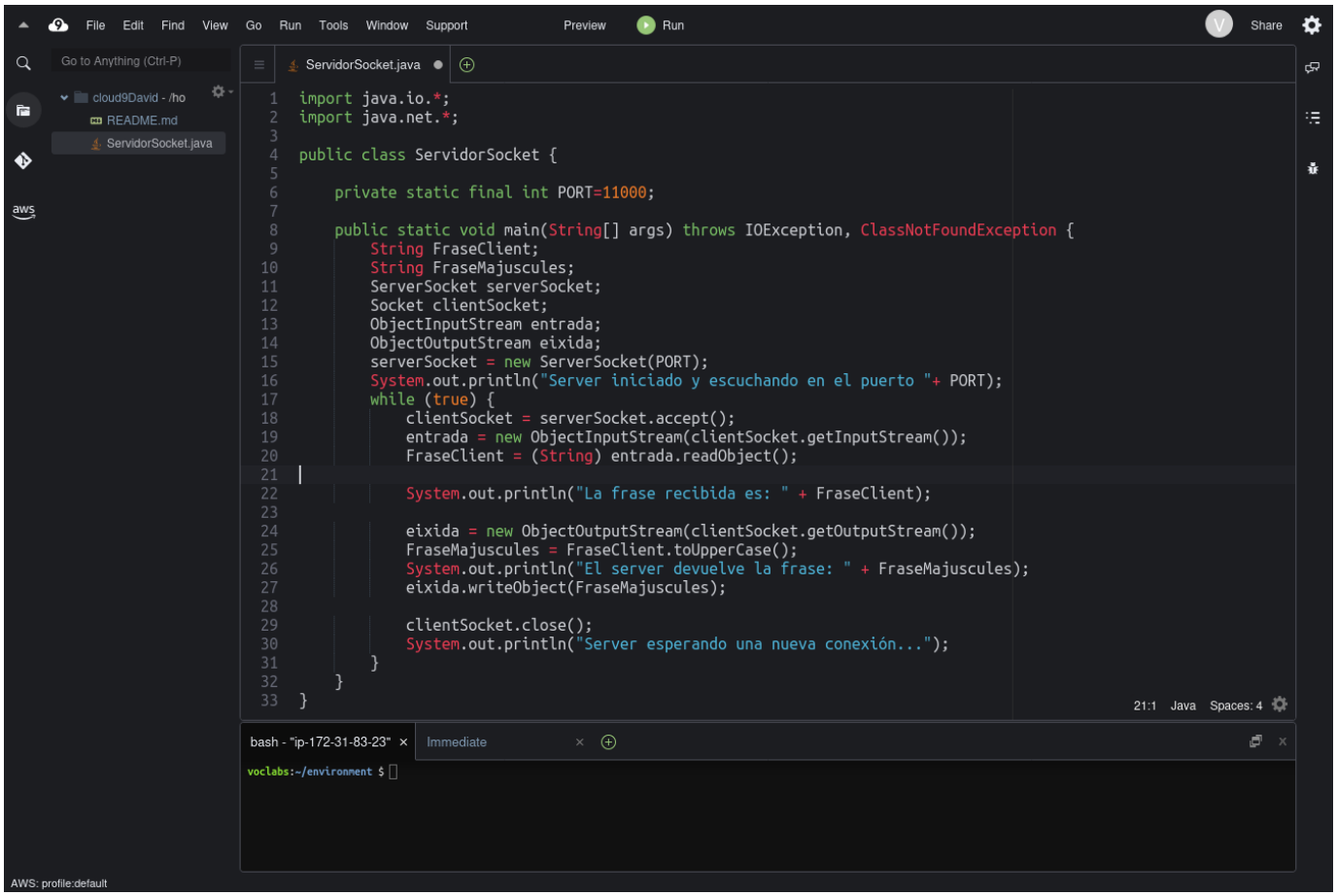
Creación del servidor de Sockets

Primero cerraremos la ventana de bienvenida, a continuación creamos un nuevo fichero, por ejemplo `ServidorSocket.java` con el siguiente código java:

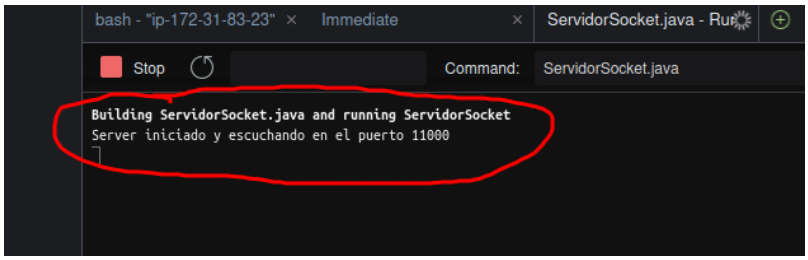
```java
import java.io.*;
import java.net.*;

public class ServidorSocket {

    private static final int PORT=11000;

    public static void main(String[] args) throws IOException, ClassNotFoundException {
        String FraseClient;
        String FraseMajuscules;
        ServerSocket serverSocket;
        Socket clientSocket;
        ObjectInputStream entrada;
        ObjectOutputStream eixida;
        serverSocket = new ServerSocket(PORT);
        System.out.println("Server iniciado y escuchando en el puerto "+ PORT);
        while (true) {
            clientSocket = serverSocket.accept();
            entrada = new ObjectInputStream(clientSocket.getInputStream());
            FraseClient = (String) entrada.readObject();

            System.out.println("La frase recibida es: " + FraseClient);

            eixida = new ObjectOutputStream(clientSocket.getOutputStream());
            FraseMajuscules = FraseClient.toUpperCase();
            System.out.println("El server devuelve la frase: " + FraseMajuscules);
            eixida.writeObject(FraseMajuscules);

            clientSocket.close();
            System.out.println("Server esperando una nueva conexión...");
        }
    }
}
```

Debería quedar algo así:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

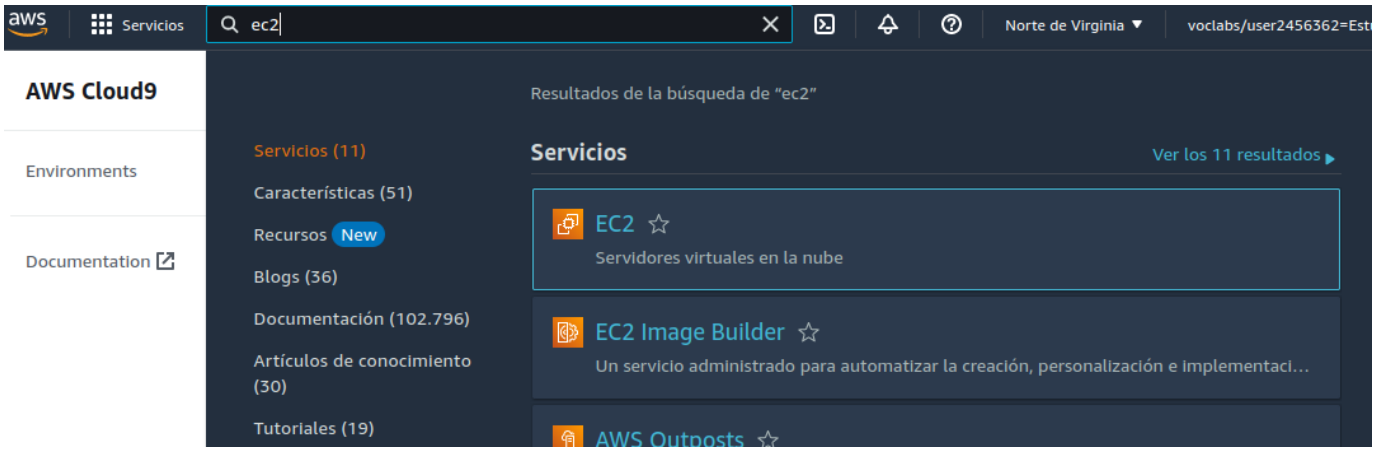


Y si iniciamos el servidor:



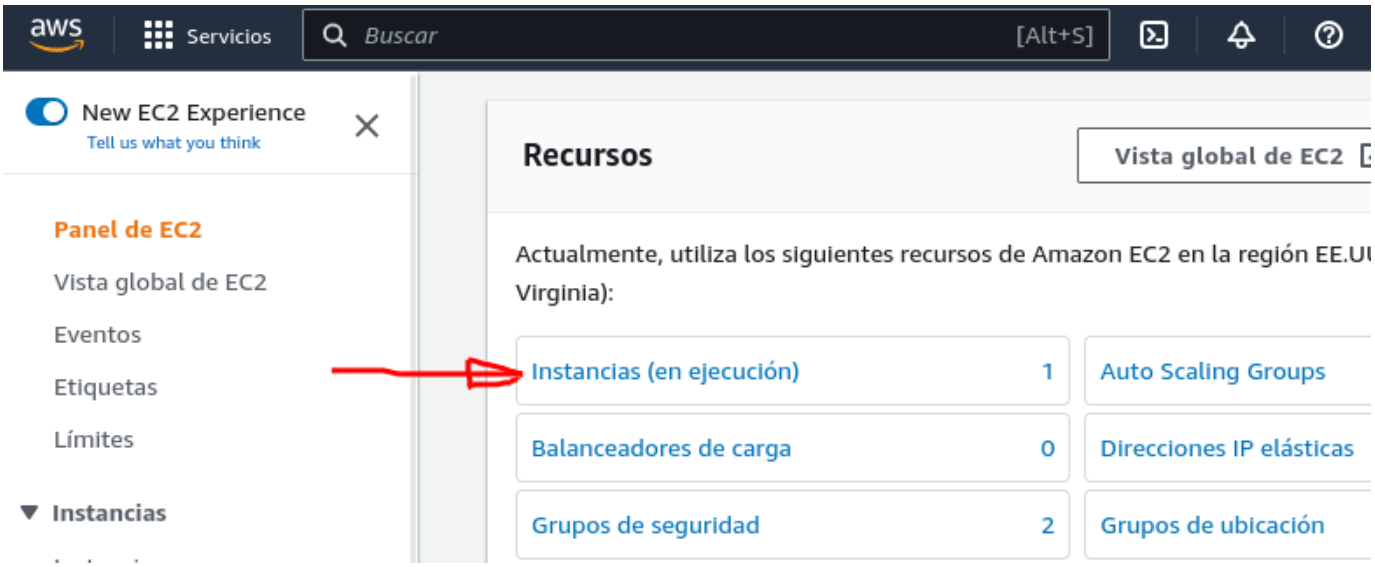
Abrir el puerto en la instancia EC2 del cloud9

Ahora debemos volver a la pestaña donde tenemos el Dashboard de AWS y buscar EC2 (donde antes buscamos cloud9):

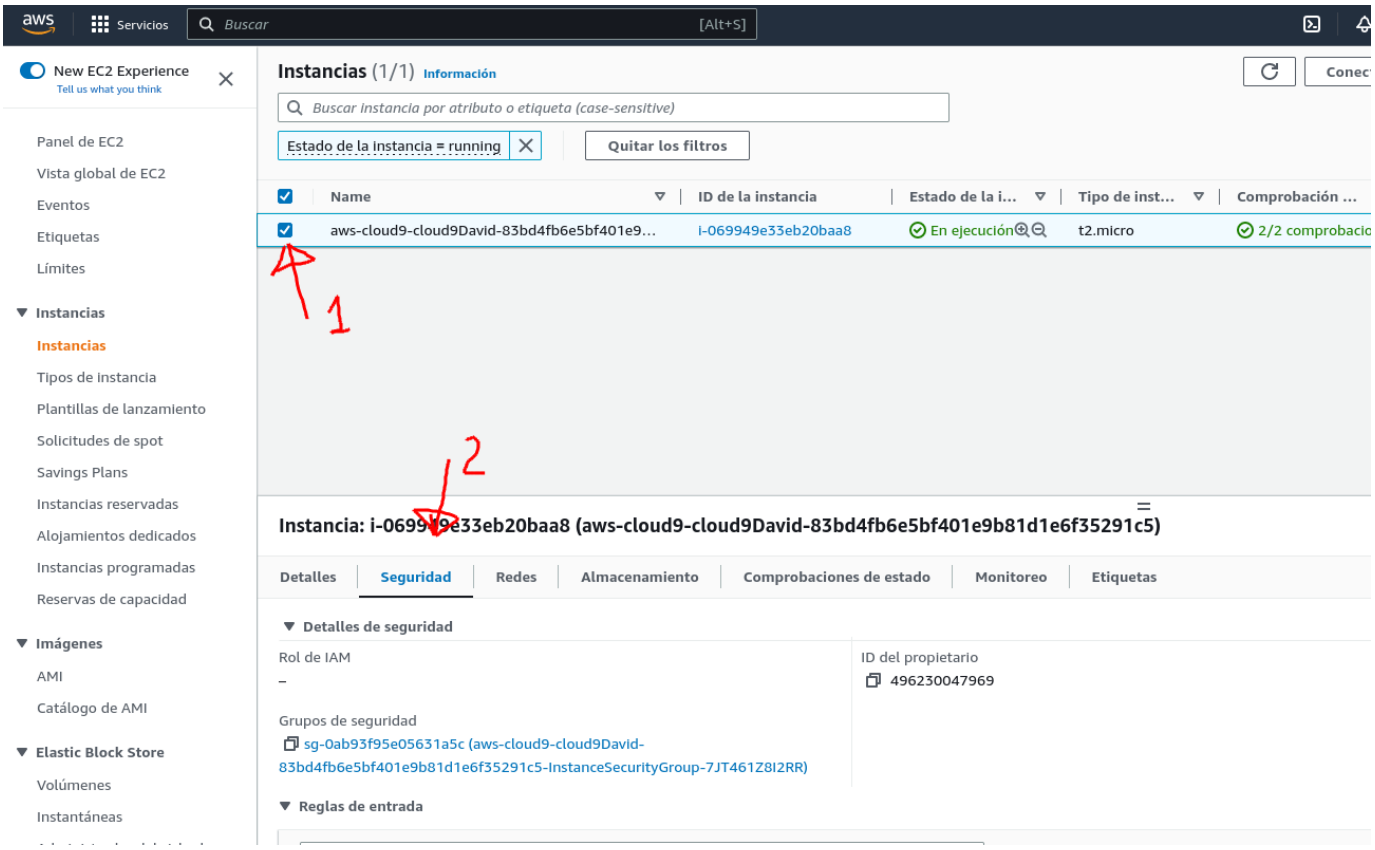


Una vez abierto elegimos la opción `Instancias (en ejecución)`:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



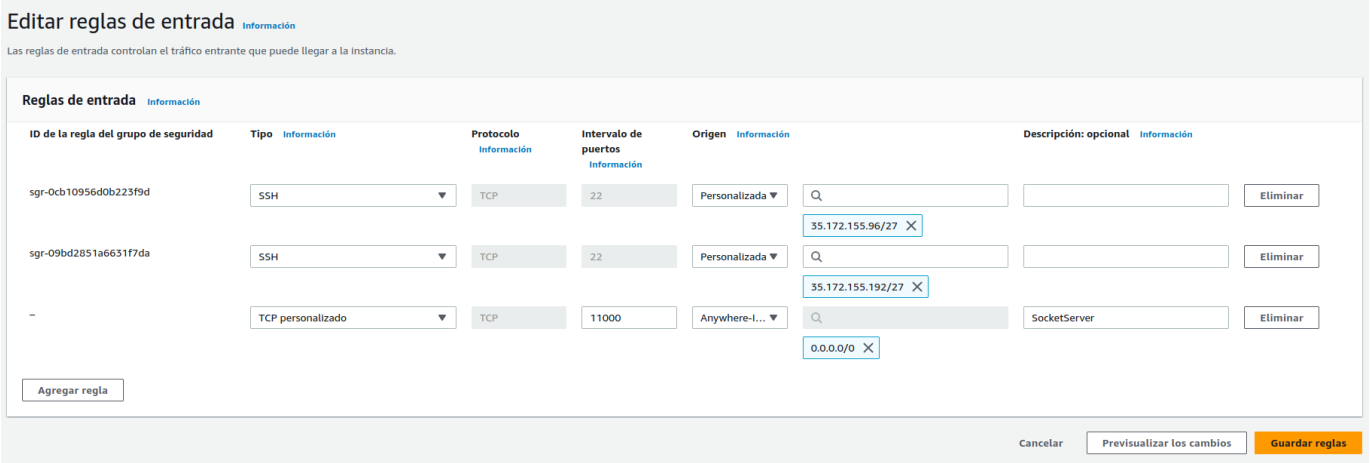
Deberíamos tener al menos una Instancia, si tenemos más de una debemos buscar la que contenga el nombre de nuestra instancia cloud9, debemos marcar el check que tiene justo delante del nombre y a continuación elegir la pestaña `Seguridad`:



Si nos fijamos en las reglas de entrada del grupo de seguridad, solo tiene habilitada la entrada para el puerto 22 (SSH), a continuación hacemos click sobre el enlace del Grupo de seguridad:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

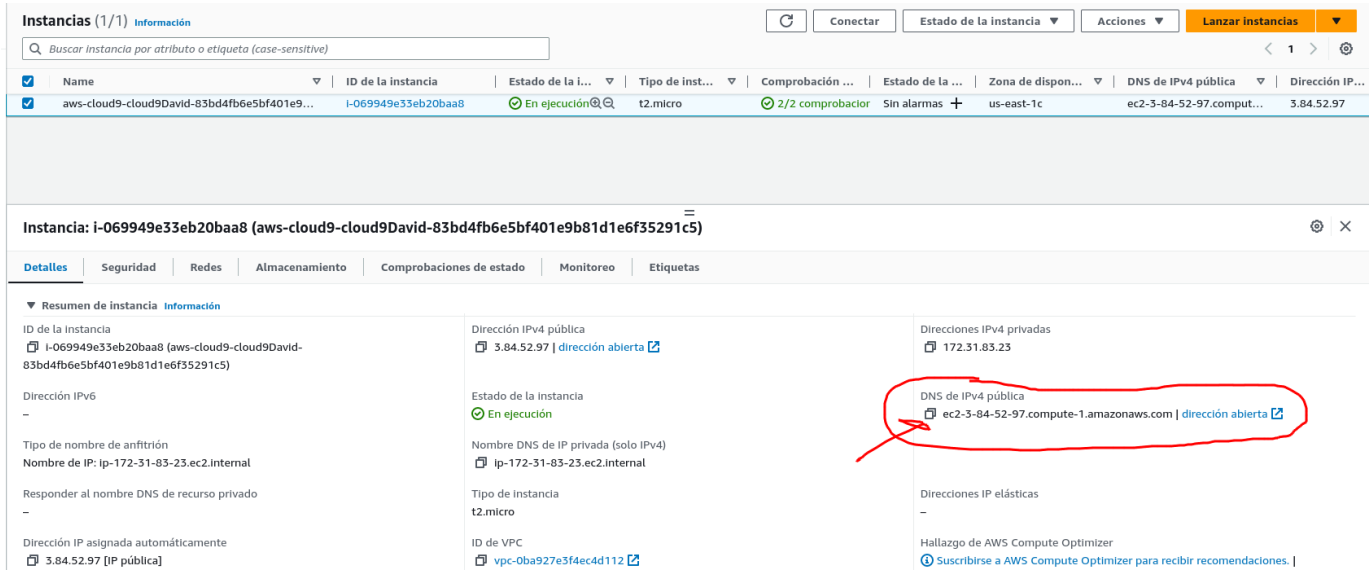
Y añadiremos el puerto 11000 (o el que hayamos elegido para nuestro servidor) a las reglas de entrada, elegimos el botón `Editar reglas de entrada`, a continuación `Agregar regla` Elegimos `TCP Personalizado`, puerto 11000 y `AnywhereIpv4` y añadimos una descripción si lo deseamos:



Una vez hecho esto si volvemos a la pestaña Seguridad de nuestra instancia EC2 veremos la regla añadida.

Dirección pública de la EC2

Necesitamos saber la DNS de IPv4 pública de nuestra instancia EC2 para acceder desde el cliente, marcamos el check de nuestra instancia y accedemos a la primera pestaña `Detalles`, y nos fijamos en la parte derecha y pulsaremos el botón de copiar y guardaremos esta información para más adelante:



© 2025 David Martínez licensed under CC BY-NC-SA 4.0

TAREA 2: PREPARAR EL CLIENTE LOCAL

En nuestro IDE preferido creamos un nuevo archivo `ClienteSocket.java` con el siguiente código:

```java
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class ClienteSocket {

    private static final String DNSAWS = "ec2-3-84-52-97.compute-1.amazonaws.com";

    public static void main(String[] args) throws IOException, ClassNotFoundException {
        Socket socket;
        ObjectInputStream entrada;
        ObjectOutputStream eixida;
        String frase;

        socket = new Socket(DNSAWS, 11000);
        eixida = new ObjectOutputStream(socket.getOutputStream());

        System.out.println("Introduce la frase a enviar en minúsculas");
        Scanner in = new Scanner(System.in);
        frase = in.nextLine();
        System.out.println("Se envia la frase " + frase);
        eixida.writeObject(frase);

        entrada = new ObjectInputStream(socket.getInputStream());
        System.out.println(
                "La frase recibida es: " + (String) entrada.readObject());
        socket.close();
    }
}
```

> **Recuerda**
>
> Recuerda cambiar la constante `DNSAWS` por el `String` que corresponde con la dirección DNS IPv4 de tu instancia EC2 obtenida en el punto 3.1.5.

EJECUCIÓN DE PRUEBA

Desde el punto de vista del cliente

Una vez ejecutado el cliente debe aparecer algo similar a esto:

```
Introduce la frase a enviar en minúsculas
```

Escribimos nuestra frase, y al pular INTRO obtenemos el siguiente resultado:

```
Introduce la frase a enviar en minúsculas
esta frase está en minúsculas
Se envia la frase esta frase está en minúsculas
La frase recibida es: ESTA FRASE ESTÁ EN MINÚSCULAS
```

Desde el punto de vista del servidor

La consola de salida del servidor por su parte debe haber registrado la conexión del cliente, la recepción de la frase, y la frase devuelta:

```
Server iniciado y escuchando en el puerto 11000
La frase recibida es: esta frase está en minúsculas
El server devuelve la frase: ESTA FRASE ESTÁ EN MINÚSCULAS
Server esperando una nueva conexión...
```

Tarea

Genera un pequeño documento con explicaciones donde muestres una captura de pantalla donde se vea el servidor después de haber esperado la conexión, haber recibido los datos del cliente y devuelto la información procesada. Adjunta también explicaciones y la captura del mismo ejemplo desde el punto de vista del cliente. El ejemplo debe incluir tu nombre y apellidos en la información enviada y recibida.

Convierte el documento en pdf y adjúntalo a la tarea de Aules de la UD06.

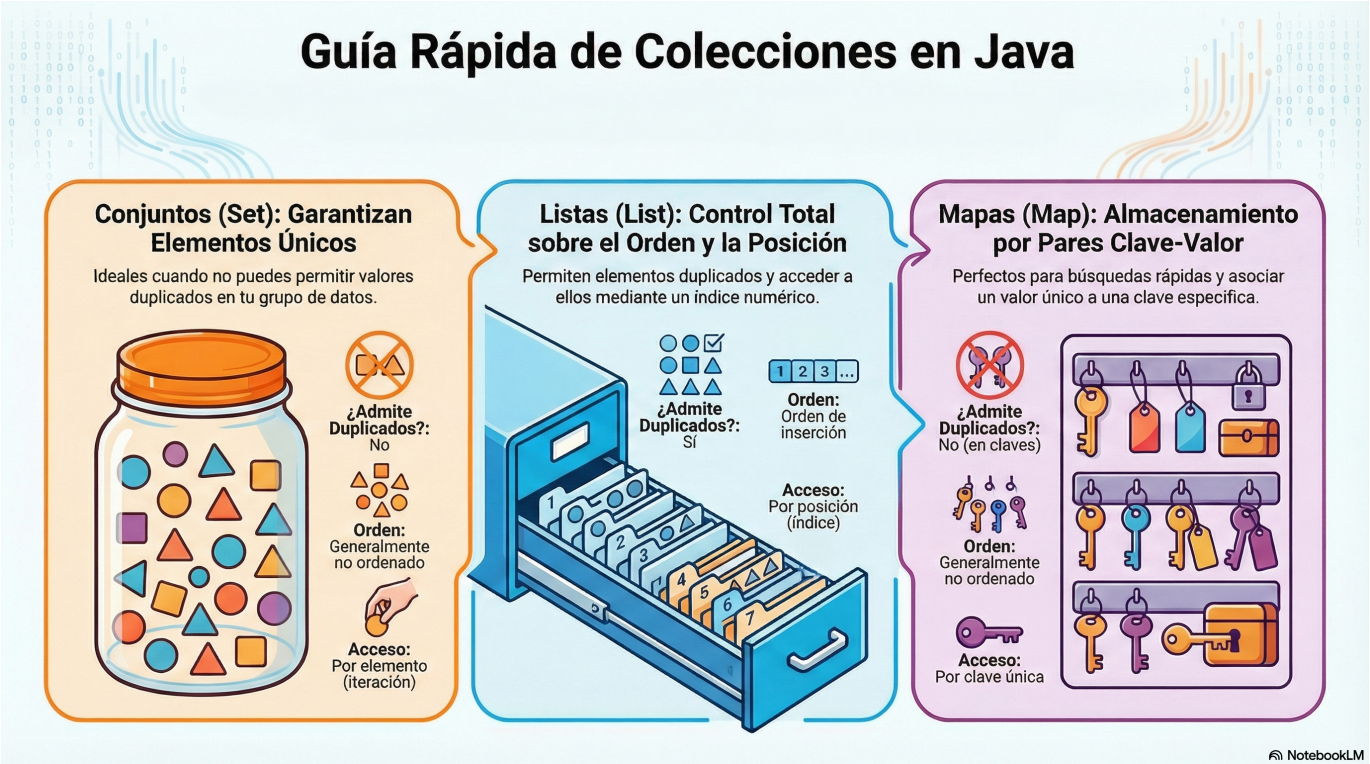
© 2025 David Martínez licensed under CC BY-NC-SA 4.0

21 de julio de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

8. UD07

8.1 Colecciones



Al finalizar esta unidad serás capaz de...

- Conocer el framework de colecciones de Java
- Utilizar las interfaces List, Set y Map
- Elegir la implementación adecuada (ArrayList, HashSet, HashMap, etc.)
- Recorrer colecciones con iteradores y for-each
- Aplicar expresiones lambda para operaciones funcionales
- Utilizar Streams para procesar datos
- Implementar la interfaz Comparable y Comparator

8.1.1 Introducción

Cuando el volumen de datos a manejar por una aplicación es elevado, no basta con utilizar variables. Manejar los datos de un único pedido en una aplicación puede ser relativamente sencillo, pues un pedido está compuesto por una serie de datos y eso simplemente se traduce en varias variables.

Pero, ¿qué ocurre cuando en una aplicación tenemos que gestionar varios pedidos a la vez? Lo mismo ocurre en otros casos. Para poder realizar ciertas aplicaciones se necesita poder manejar datos que van más allá de meros datos simples (números y letras). A veces, los datos que tiene que manejar la aplicación son datos compuestos, es decir, datos que están compuestos a su vez de varios datos más simples. Por ejemplo, un pedido está compuesto por varios datos, los datos podrían ser el cliente que hace el pedido, la dirección de entrega, la fecha requerida de entrega y los artículos del pedido.

Los datos compuestos son un tipo de estructura de datos, y en realidad ya los has manejado. Las clases son un ejemplo de estructuras de datos que permiten almacenar datos compuestos, y el objeto en sí, la instancia de una clase, sería el dato compuesto. Pero, a veces, los datos tienen estructuras aún más complejas, y son necesarias soluciones adicionales.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Aquí podrás aprender esas soluciones adicionales. Esas soluciones consisten básicamente en la capacidad de poder manejar varios datos del mismo o diferente tipo de forma dinámica y flexible.

8.1.2 Estructuras de almacenamiento

¿Cómo almacenarías en memoria un listado de números del que tienes que extraer el valor máximo?

Seguro que te resultaría fácil. Pero, ¿y si el listado de números no tiene un tamaño fijo, sino que puede variar en tamaño de forma dinámica? Entonces la cosa se complica.

Un listado de números que aumenta o decrece en tamaño es una de las cosas que aprenderás a utilizar aquí, utilizando estructuras de datos.

Pasaremos por alto las clases y los objetos, pues ya los has visto con anterioridad, pero debes saber que las clases en sí mismas son la evolución de un tipo de estructuras de datos conocidas como datos compuestos (también llamadas registros). Las clases, además de aportar la ventaja de agrupar datos relacionados entre sí en una misma estructura (característica aportada por los datos compuestos), permiten agregar métodos que manejen dichos datos, ofreciendo una herramienta de programación sin igual. Pero todo esto ya lo sabías.

Las estructuras de almacenamiento, en general, se pueden clasificar de varias formas. Por ejemplo, atendiendo a **si pueden almacenar datos de diferente tipo, o si solo pueden almacenar datos de un solo tipo**, se pueden distinguir:

- **Estructuras con capacidad de almacenar varios datos del mismo tipo**: varios números, varios caracteres, etc. Ejemplos de estas estructuras son los arrays, las cadenas de caracteres, las listas y los conjuntos.
- **Estructuras con capacidad de almacenar varios datos de distinto tipo**: números, fechas, cadenas de caracteres, etc., todo junto dentro de una misma estructura. Ejemplos de este tipo de estructuras son las clases.

Otra forma de clasificar las estructuras de almacenamiento va **en función de si pueden o no cambiar de tamaño** de forma dinámica:

- **Estructuras cuyo tamaño se establece en el momento de la creación o definición y su tamaño no puede variar después**. Ejemplos de estas estructuras son los arrays y las matrices (arrays multimensionales).
- **Estructuras cuyo tamaño es variable (conocidas como estructuras dinámicas). Su tamaño crece o decrece según las necesidades de forma dinámica**. Es el caso de las listas, árboles, conjuntos y, como veremos también, el caso de algunos tipos de cadenas de caracteres.

Por último, atendiendo a **la forma en la que los datos se ordenan** dentro de la estructura, podemos diferenciar varios tipos de estructuras:

- **Estructuras que no se ordenan de por sí**, y debe ser el programador el encargado de ordenar los datos si fuera necesario. Un ejemplo de estas estructuras son los arrays.
- **Estructuras ordenadas**. Se trata de estructuras que al incorporar un dato nuevo a todos los datos existentes, este se almacena en una posición concreta que irá en función del orden. El orden establecido en la estructura puede variar dependiendo de las necesidades del programa: alfabético, orden numérico de mayor a menor, momento de inserción, etc.

Todavía no conoces mucho de las estructuras, y probablemente todo te suena raro y extraño. No te preocupes, poco a poco irás descubriéndolas. Verás que son sencillas de utilizar y muy cómodas.

8.1.3 Clases y métodos genéricos

¿Crees qué el código es más legible al utilizar genéricos o qué se complica? La verdad es que al principio cuesta, pero después, el código se entiende mejor que si se empieza a a insertar conversiones de tipo.

Las clases genéricas son equivalentes a los métodos genéricos pero a nivel de clase, permiten definir un parámetro de tipo o genérico que se podrá usar a lo largo de toda la clase, facilitando así crear clases genéricas que son capaces de trabajar con diferentes tipos de datos base. Para crear una clase genérica se especifican los parámetros de e tipo al lado del nombre e de la clase:

```
1   public class Util<T> {
2      T t1;
3      public void invertir(T[] array) {
4          for (int i = 0; i < array.length / 2; i++) {
5              t1 = array[i];
6              array[i] = array[array.length - i - 1];
7              array[array.length - i - 1] = t1;
8          }
9      }
10  }
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

En el ejemplo anterior, la clase `Util` contiene el método `invertir` cuya función es invertir el orden de los elementos de cualquier `array`, sea del tipo que sea. Para usar esa clase genérica hay que crear un objeto o instancia de dicha clase especificando el tipo base entre los símbolos menor que ("`<`") y mayor que ("`>`"), justo detrás del nombre e de la clase. Veamos un ejemplo:

```
Integer[] numeros={0,1,2,3,4,5,6,7,8,9};
Util<Integer> u= new Util<Integer>();
u.invertir(numeros);
for (int i=0;i<numeros.length;i++){
    System.out.println(numeros[i]);
}
```

Como puedes observar, el uso de genéricos es sencillo, tanto a nivel de clase como a nivel de método.

Simplemente, a la hora de crear una instancia de una clase genérica, hay que especificar el tipo, tanto en la definición (`Util<Integer> u`) como en la creación (`new Util<Integer>()`).

Los genéricos los vamos a usar ampliamente a partir de ahora, aplicados a un montón de clases genéricas que tiene Java y que son de gran utilidad, por lo que es conveniente que aprendas bien a usar una clase genérica.

Atención

Los parámetros de tipo de las clases genéricas solo pueden ser clases, no pueden ser jamás tipos de datos primitivos como `int`, `short`, `double`, etc. En su lugar, debemos usar sus clases envoltorio (wrappers) `Integer`, `Short`, `Double`, etc.

Todavía hay un montón de cosas más sobre los métodos y las clases genéricas que deberías saber. A continuación se muestran algunos usos interesantes de los genéricos:

- Dos o más parámetros de tipo (I):

```
public class Util<T,M>{
    public static <T,M> int sumaDeLongitudes (T[] a, M[] b){
        return a.length+b.length;
    }
}
```

Información

Si el método genérico necesita tener dos o más parámetros genéricos, podemos indicarlo separándolos por comas. En el ejemplo anterior se suman las longitudes de dos arrays que no tienen que ser del mismo tipo.

- Dos o más parámetros de tipo (II):

```
Integer[] a1={0,1,2,3,4};
Double[] a2={0d,1d,2d,3d,4d};
int resultado=Util.<Integer,Double>sumaDeLongitudes(a1,a2);
System.out.println(resultado);
```

Información

Usar un método o una clase con dos o más parámetros genéricos es sencillo, a la hora de invocar al método o crear la clase, se indican los tipos base separados por coma.

- Dos o más parámetros de tipo (III):

```
public class Terna <A,B,C>{
    A a;
    B b;
    C c;
    public terna(A a, B b, C c){
        this.a=a;
        this.b=b;
        this.c=c;
    }
    public A getA(){return a;}
    public B getB(){return b;}
    public C getC(){return c;}
}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

> **Información**
>
> Si una clase genérica necesita tener dos o más parámetros genéricos, podemos indicarlo separándolos por comas. En el ejemplo anterior se muestra una clase que almacena una terna de elementos de diferente tipo base que están relacionados entre sí.

- Métodos con tipos adicionales:

```
class Util<A,B>{
    A a;
    Util (A a){
        this.a=a;
    }
    public <B> void Salida(B b){
        System.out.println(a.toString() + b.toString());
    }
}
```

> **Información**
>
> Una clase genérica puede tener unos parámetros genéricos, pero si en uno de sus métodos necesitamos otros parámetros genéricos distintos, no hay problema, podemos combinarlos.

- Inferencia de tipos (I):

```
Integer[] a1={0,1,2,3,4};
Double[] a2={0d,1d,2d,3d,4d};
util.<Integer,Double>sumaDeLongitudes(a1,a2);
util.sumaDeLongitudes(a1,a2);
```

> **Información**
>
> No siempre es necesario indicar los tipos a la hora de instanciar un método genérico. A partir de Java 7, es capaz de determinar los tipos a partir de los parámetros. Las dos expresiones de arriba serian válidas y funcionarían. Si no es capa de inferirlos, nos dará un error a la hora de compilar.

- Inferencia de tipos (II):

```
Integer a1=0;
Double d1=1.3d;
Float f1=1.4f;
Terna <Integer,Double,Float> t=new Terna<>(a1,d1,f1);
```

> **Información**
>
> A partir de Java 7 es posible usar el operador diamante <> para simplificar la instanciación o creación de nuevos objetos a partir de clases genéricas. **Cuidado, esto solo es posible a partir de Java 7**.

- Limitación de tipos

```
public class Util {
    public static <T extends Number> Double sumar (T t1, T t2){
        return new Double(t1.doubleValue() + t2.doubleValue());
    }
}
```

> **Información**
>
> Se pueden limitar el conjunto de tipos que se pueden usar con una clase o método genérico usando el operador `extends`. El operador `extends` permite indicar que la clase que se pasa como parámetro genérico tiene que derivar de una clase específica. En el ejemplo, no se admitirá ninguna clase que no derive de `Number`, pudiendo así realizar operaciones matemáticas.

- Paso de clases genéricas por parámetro

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
public class Ejemplo <A> {
    public A a;
}
...
void test (Ejemplo<Integer> e) {
    ...
}
```

Información

Cuando un método tiene como parámetro una clase genérica (como en el caso del método test del ejemplo), se puede especificar cual debe ser el tipo base usado en la instancia de la clase genérica que se le pasa como argumento. Esto permite, entre otras cosas, crear diferentes versiones de un mismo método (sobrecarga), dependiendo del tipo base usado en la instancia de la clase genérica se ejecutará una versión u otra.

- Paso de clases genéricas por parámetro. Wildcards. (I)

```
public class Ejemplo <A> {
    public A a;
}
...
void test (Ejemplo<?> e) {
    ...
}
```

Información

Cuando un método admite como parámetro una clase genérica en la que no importa el tipo de objeto sobre la que se ha creado, podemos usar el interrogante para indicar "*cualquier tipo*".

- Paso de clases genéricas por parámetro. Wildcards. (II)

```
public class Ejemplo <A> {
    public A a;
}
...
void test (Ejemplo<? extends Number> e) {
    ...
}
```

Información

También es posible limitar el conjunto de tipos que una clase genérica puede usar, a través del operador `extends`. El ejemplo anterior es como decir "*cualquier tipo que derive de Number*"

8.1.4 Colecciones

Introducción

¿Qué consideras una colección? Pues seguramente al pensar en el término se te viene a la cabeza una colección de libros o algo parecido, y la idea no va muy desencaminada. **Una colección a nivel de software es un grupo de elementos almacenados de forma conjunta en una misma estructura**. Eso son las colecciones.

Las colecciones definen un conjunto de interfaces, clases genéricas y algoritmos que permiten manejar grupos de objetos, todo ello enfocado a potenciar la reusabilidad del software y facilitar las tareas de programación. Te parecerá increíble el tiempo que se ahorra empleando colecciones y cómo se reduce la complejidad del software usándolas adecuadamente. Las colecciones permiten almacenar y manipular grupos de objetos que, a priori, están relacionados entre sí (aunque no es obligatorio que estén relacionados, lo lógico es que si se almacenan juntos es porque tienen alguna relación entre sí), pudiendo trabajar con cualquier tipo de objeto (de ahí que se empleen los genéricos en las colecciones).

Además las colecciones permiten realizar algunas operaciones útiles sobre los elementos almacenados, tales como búsqueda u ordenación. En algunos casos es necesario que los objetos almacenados cumplan algunas condiciones (que implementen algunas interfaces), para poder hacer uso de estos algoritmos.

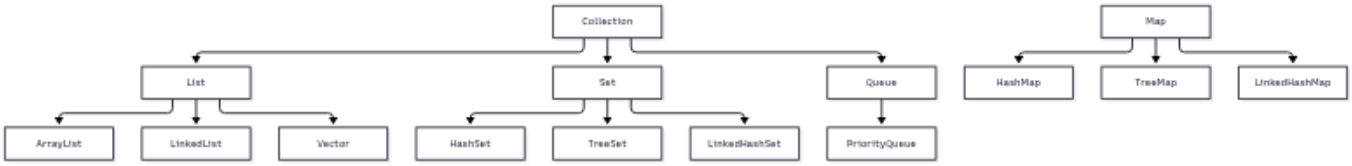
© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Las colecciones son en general elementos de programación que están disponibles en muchos lenguajes de programación. En algunos lenguajes de programación su uso es algo más complejo (como es el caso de C++), pero en Java su uso es bastante sencillo.

Las colecciones en Java parten de una serie de interfaces básicas. Cada interfaz define un modelo de colección y las operaciones que se pueden llevar a cabo sobre los datos almacenados, por lo que es necesario conocerlas. La interfaz inicial, a través de la cual se han construido el resto de colecciones, es la interfaz `java.util.Collection`, que define las operaciones comunes a todas las colecciones derivadas. A continuación se muestran las operaciones más importantes definidas por esta interfaz, ten en cuenta que `Collection` es una interfaz genérica donde `<E>` es el parámetro de tipo (podría ser cualquier clase):

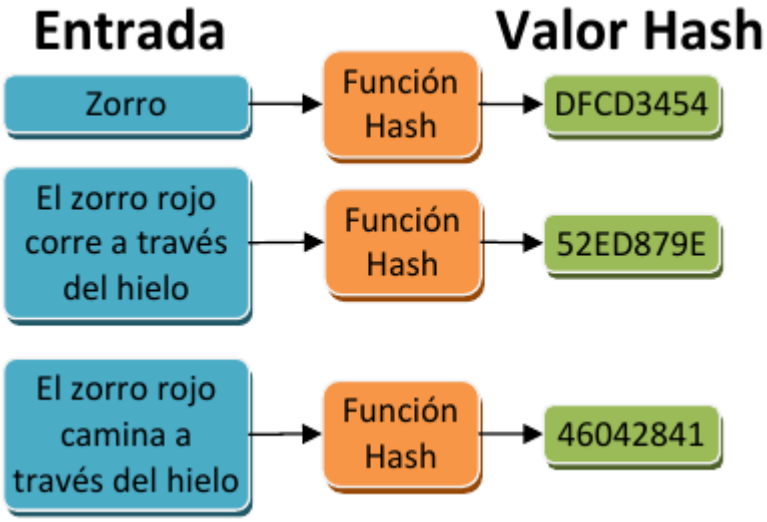
- **Método** `int size()` : retorna el número de elementos de la colección.
- **Método** `boolean isEmpty()` : retornará verdadero si la colección está vacía.
- **Método** `boolean contains (Object element)` : retornará verdadero si la colección tiene el elemento pasado como parámetro.
- **Método** `boolean add(E element)` : permitirá añadir elementos a la colección.
- **Método** `boolean remove(Object element)` : permitirá eliminar elementos de la colección.
- **Método** `Iterator<E> iterator()` : permitirá crear un iterador para recorrer los elementos de la colección. Esto se ve más adelante, no te preocupes.
- **Método** `Object[] toArray()` : permite pasar la colección a un array de objetos tipo Object.
- **Método** `boolean containsAll(Collection<?> c)` : permite comprobar si una colección contiene los elementos existentes en otra colección, si es así, retorna verdadero.
- **Método** `boolean addAll(Collection<?> extends E> c)` : permite añadir todos los elementos de una colección a otra colección, siempre que sean del mismo tipo (o deriven del mismo tipo base).
- **Método** `boolean removeAll(Collection<?> c)` : si los elementos de la colección pasada como parámetro están en nuestra colección, se eliminan, el resto se quedan.
- **Método** `boolean retainAll(Collection<?> c)` : si los elementos de la colección pasada como parámetro están en nuestra colección, se dejan, el resto se eliminan.
- **Método** `void clear()` : vaciar la colección.

Más adelante veremos cómo se usan estos métodos, será cuando veamos las implementaciones (clases genéricas que implementan alguna de las interfaces derivadas de la interfaz `Collection`).



Conjuntos (sets)

¿Con qué relacionarías los conjuntos? Seguro que con las matemáticas. Los conjuntos son un tipo de colección que no admite duplicados, derivados del concepto matemático de conjunto.



© 2025 David Martínez licensed under CC BY-NC-SA 4.0

La interfaz `java.util.Set` define cómo deben ser los conjuntos, y implementa la interfaz `Collection`, aunque no añade ninguna operación nueva. Las implementaciones (clases genéricas que implementan la interfaz `Set`) más usadas son las siguientes:

- `java.util.HashSet`. Conjunto que almacena los objetos usando tablas hash (estructura de datos formada básicamente por un array donde la posición de los datos va determinada por una función hash, permitiendo localizar la información de forma extraordinariamente rápida. Los datos están ordenados en la tabla en base a un resumen numérico de los mismos (en hexadecimal generalmente) obtenido a partir de un algoritmo para cálculo de resúmenes, denominadas funciones hash. El resumen no tiene significado para un ser humano, se trata simplemente de un mecanismo para obtener un número asociado a un conjunto de datos. El inconveniente de estas tablas es que los datos se ordenan por el resumen obtenido, y no por el valor almacenado. El resumen, de un buen algoritmo hash, no se parece en nada al contenido almacenado) lo cual acelera enormemente el acceso a los objetos almacenados.

Inconvenientes: necesitan bastante memoria y no almacenan los objetos de forma ordenada (al contrario, pueden aparecer completamente desordenados).

- `java.util.LinkedHashSet`. Conjunto que almacena objetos combinando tablas hash, para un acceso rápido a los datos, y listas enlazadas (estructura de datos que almacena los objetos enlazándolos entre sí a través de un apuntador de memoria o puntero, manteniendo un orden, que generalmente es el del momento de inserción, pero que puede ser otro. Cada dato se almacena en una estructura llamada nodo en la que existe un campo, generalmente llamado siguiente, que contiene la dirección de memoria del siguiente nodo (con el siguiente dato)) para conservar el orden. El orden de almacenamiento es el de inserción, por lo que se puede decir que es una estructura ordenada a medias.
- **Inconvenientes**: necesitan bastante memoria y es algo más lenta que `HashSet`.
- `java.util.TreeSet`. Conjunto que almacena los objetos usando unas estructuras conocidas como árboles rojo-negro. Son más lentas que los dos tipos anteriores. pero tienen una gran ventaja: los datos almacenados se ordenan por valor. Es decir, que aunque se inserten los elementos de forma desordenada, internamente se ordenan dependiendo del valor de cada uno.

Poco a poco, iremos viendo que son las listas enlazadas y los árboles (no profundizaremos en los árboles rojo-negro, pero si veremos las estructuras tipo árbol en general). Veamos un ejemplo de uso básico de la estructura `HashSet` y después, profundizaremos en los `LinkedHashSet` y los `TreeSet`.

Para crear un conjunto, simplemente creamos el `HashSet` indicando el tipo de objeto que va a almacenar, dado que es una clase genérica que puede trabajar con cualquier tipo de dato debemos crearlo como sigue (no olvides hacer la importación de `java.util.HashSet` primero):

```
HashSet<Integer> conjunto=new HashSet<Integer>();
HashSet<Integer> conjunto=new HashSet<>(); //a partir de Java 7
```

Después podremos ir almacenando objetos dentro del conjunto usando el método `add` (definido por la interfaz `Set`). Los objetos que se pueden insertar serán siempre del tipo especificado al crear el conjunto:

```
Integer n=new Integer(10);
if (!conjunto.add(n)){
    System.out.println("Número ya en la lista.");
}
```

Si el elemento ya está en el conjunto, el método `add` retornará `false` indicando que no se pueden insertar duplicados. Si todo va bien, retornará `true`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

> ⚠ **HashSet y HashMap con tus propias clases: `equals` y `hashCode`**
>
> Para buscar un objeto (con `contains`, `add` o `get`), un `HashSet` o un `HashMap` hace dos pasos:
>
> 1. Calcula el `hashCode()` del objeto para saber en qué «cajón» de la tabla buscar.
> 2. Solo dentro de ese cajón compara con `equals()`.
>
> Si tu clase redefine `equals` pero no `hashCode`, usa el `hashCode` de `Object`, que es distinto para cada objeto creado con `new`. Dos objetos «iguales» acaban en cajones distintos y no se encuentran:
>
> ```
> Set<Libro> catalogo = new HashSet<>();
> catalogo.add(new Libro("El Quijote", "Cervantes", "E1"));
> catalogo.contains(new Libro("El Quijote", "Cervantes", "E1")); // false si Libro no tiene hashCode
> ```
>
> Soluciones:
>
> - **Redefinir `equals()` y `hashCode()` juntos y con los mismos atributos** (en IntelliJ: *Code → Generate → equals() and hashCode()*). Es lo recomendado: dos objetos iguales según `equals` deben tener el mismo `hashCode`.
> - Usar como clave un tipo que ya trae su `hashCode`, por ejemplo el título: `HashMap<String, Libro>`.
> - Usar otra colección: `ArrayList` o `LinkedList` (buscan solo con `equals`) o `TreeSet` (usa `compareTo`).

ACCESO

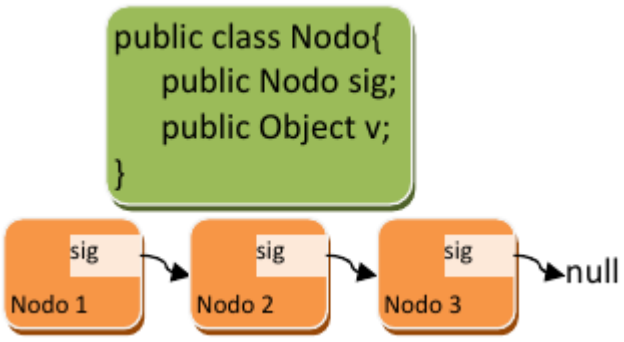
Y ahora te preguntarás, ¿cómo accedo a los elementos almacenados en un conjunto? Para obtener los elementos almacenados en un conjunto hay que usar iteradores, que permiten obtener los elementos del conjunto uno a uno de forma secuencial (no hay otra forma de acceder a los elementos de un conjunto, es su inconveniente). Los iteradores se ven en mayor profundidad más adelante, de momento, vamos a usar iteradores de forma transparente, a través de una estructura *for* especial, denominada bucle "for-each" o bucle "para cada". En el siguiente código se usa un bucle *for-each*, en él la variable i va tomando todos los valores almacenados en el conjunto hasta que llega al último:

```
for (Integer i: conjunto) {
    System.out.println("Elemento almacenado:"+i);
}
```

Como ves la estructura `for-each` es muy sencilla: la palabra `for` seguida de "(`tipo variable:colección`)" y el cuerpo del bucle; `tipo` es el tipo del objeto sobre el que se ha creado la colección, `variable` pues es la variable donde se almacenará cada elemento de la colección y `coleccion` la colección en sí. Los bucles `for-each` se pueden usar para todas las colecciones.

`LinkedHashSet` Y `TreeSet`

¿En qué se diferencian las estructuras `LinkedHashSet` y `TreeSet` de la estructura `HashSet`? Ya se comento antes, y es básicamente en su funcionamiento interno.



La estructura `LinkedHashSet` es una estructura que internamente funciona como una lista enlazada, aunque usa también tablas hash para poder acceder rápidamente a los elementos. Una lista enlazada es una estructura similar a la representada en la imagen de la derecha, la cual está compuesta por nodos (elementos que forman la lista) que van enlazándose entre sí. Un nodo contiene dos cosas: el dato u objeto almacenado en la lista y el siguiente nodo de la lista. Si no hay siguiente nodo, se indica poniendo nulo (null) en la variable que contiene el siguiente nodo.

Las listas enlazadas tienen un montón de operaciones asociadas en las que no vamos a profundizar: eliminación de un nodo de la lista, inserción de un nodo al final, al principio o entre dos nodos, etc.

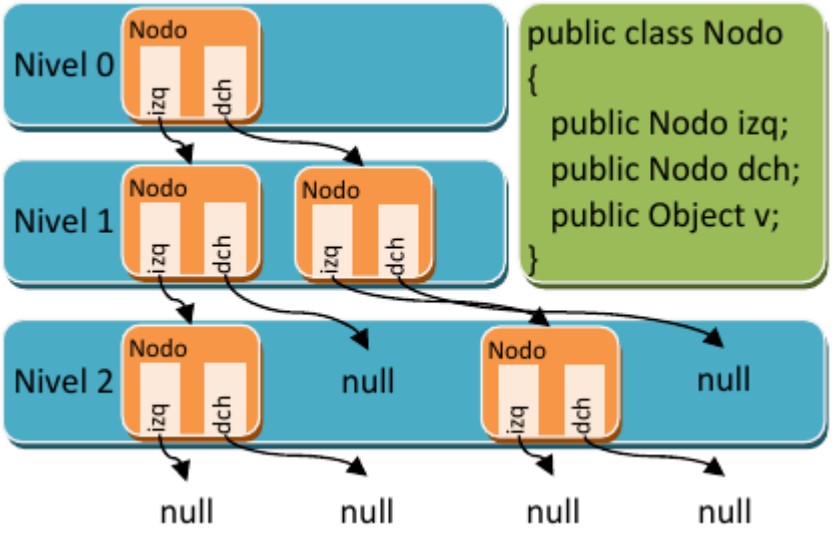
Gracias a las colecciones podremos utilizar listas enlazadas sin tener que complicarnos en detalles de programación.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

La estructura `TreeSet`, en cambio, utiliza internamente árboles. Los árboles son como las listas pero mucho más complejos. En vez de tener un único elemento siguiente, pueden tener dos o más elementos siguientes, formando estructuras organizadas y jerárquicas.

Los nodos se diferencian en dos tipos: nodos padre y nodos hijo; un nodo padre puede tener varios nodos hijo asociados (depende del tipo de árbol), dando lugar a una estructura que parece un árbol invertido (de ahí su nombre).

En la figura de abajo se puede apreciar un árbol donde cada nodo puede tener dos hijos, denominados izquierdo (izq) y derecho (dch). Puesto que un nodo hijo puede también ser padre a su vez, los árboles se suelen visualizar para su estudio por niveles para entenderlos mejor, donde cada nivel contiene hijos de los nodos del nivel anterior, excepto el primer nivel (que no tiene padre).



Los árboles son estructuras complejas de manejar y que permiten operaciones muy sofisticadas. Los árboles usados en los `TreeSet`, los árboles rojo-negro, son árboles auto-ordenados, es decir, que al insertar un elemento, este queda ordenado por su valor de forma que al recorrer el árbol, pasando por todos los nodos, los elementos salen ordenados. El ejemplo mostrado en la imagen es simplemente un árbol binario, el más simple de todos.

Nuevamente, no se va a profundizar en las operaciones que se pueden realizar en un árbol a nivel interno (inserción de nodos, eliminación de nodos, búsqueda de un valor, etc.). Nos aprovecharemos de las colecciones para hacer uso de su potencial. En la siguiente tabla tienes un uso comparado de `TreeSet` y `LinkedHashSet`. Su creación es similar a como se hace con `HashSet`, simplemente sustituyendo el nombre de la clase `HashSet` por una de las otras. Ni `TreeSet`, ni `LinkedHashSet` admiten duplicados, y se usan los mismos métodos ya vistos antes, los existentes en la interfaz `Set` (que es la interfaz que implementan).

- Conjunto `TreeSet` (Ejemplo01):

Resultado mostrado por pantalla (el resultado sale ordenado por valor):

```
1 3 4 99
```

- Conjunto `LinkedHashSet` (Ejemplo02):

Resultado mostrado por pantalla (los valores salen ordenados según el momento de inserción en el conjunto):

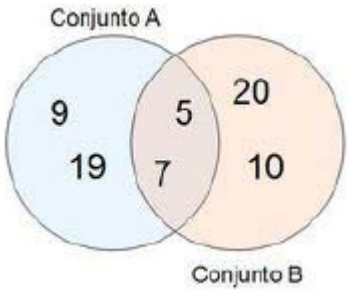
```
4 3 1 99
```

OPERAR CON ELEMENTOS

¿Cómo podría copiar los elementos de un conjunto de uno a otro? ¿Hay que usar un bucle for y recorrer toda la lista para ello? ¡Qué va! Para facilitar esta tarea, los conjuntos, y las colecciones en general, facilitan un montón de operaciones para poder combinar los datos de varias colecciones. Ya se vieron en un apartado anterior, aquí simplemente vamos poner un ejemplo de su uso.

Partimos del siguiente ejemplo, en el que hay dos colecciones de diferente tipo, cada una con 4 números enteros:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

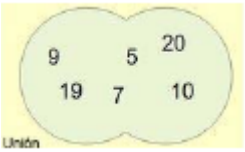



```
TreeSet<Integer> A= new TreeSet<Integer>();
A.add(9); A.add(19); A.add(5); A.add(7); // Elementos del conjunto A: 9, 19, 5 y 7
LinkedHashSet<Integer> B= new LinkedHashSet<Integer>();
B.add(10); B.add(20); B.add(5); B.add(7); // Elementos del conjunto B: 10, 20, 5 y 7
```

En el ejemplo anterior, el literal de número se convierte automáticamente a la clase envoltorio (wrapper) `Integer` sin tener que hacer nada, lo cual es una ventaja. Veamos las formas de combinar ambas colecciones:

- **Unión**. Añadir todos los elementos del conjunto B en el conjunto A.

```
A.addAll(B)
```



Todos los del conjunto A, añadiendo los del B, pero sin repetir los que ya están:

```
5, 7, 9, 10, 19 y 20.
```

- **Diferencia**. Eliminar los elementos del conjunto B que puedan estar en el conjunto A.

```
A.removeAll(B)
```

Todos los elementos del conjunto A, que no estén en el conjunto B:

```
9, 19.
```

- **Intersección**. Retiene los elementos comunes a ambos conjuntos.

```
A.retainAll(B)
```

Todos los elementos del conjunto A, que también están en el conjunto B:

```
5 y 7.
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Recuerda

Estas operaciones son comunes a todas las colecciones.

Consulta el Ejemplo03

ORDENACIÓN

Por defecto, los `TreeSet` ordenan sus elementos de forma ascendente, pero, ¿se podría cambiar el orden de ordenación? Los `TreeSet` tienen un conjunto de operaciones adicionales, además de las que incluye por el hecho de ser un conjunto, que permite entre otras cosas, cambiar la forma de ordenar los elementos. Esto es especialmente útil cuando el tipo de objeto que se almacena no es un simple número, sino algo más complejo (un artículo por ejemplo). `TreeSet` es capaz de ordenar tipos básicos (números, cadenas y fechas) pero otro tipo de objetos no puede ordenarlos con tanta facilidad.

Para indicar a un `TreeSet` cómo tiene que ordenar los elementos, debemos decirle cuándo un elemento va antes o después que otro, y cuándo son iguales. Para ello, utilizamos la interfaz genérica `java.util.Comparator` , usada en general en algoritmos de ordenación, como veremos más adelante.

Se trata de crear una clase que implemente dicha interfaz, así de fácil. Dicha interfaz requiere de un único método que debe calcular si un objeto pasado por parámetro es mayor, menor o igual que otro del mismo tipo. Veamos un ejemplo general de cómo implementar un comparador para una hipotética clase `Objeto`:

```
1  class ComparadorDeObjetos implements Comparator<Objeto> {
2      public int compare(Objeto o1, Objeto o2) { ... }
3  }
```

La interfaz `Comparator` obliga a implementar un único método, es el método `compare` , el cual tiene dos parámetros: los dos elementos a comparar. Las reglas son sencillas, a la hora de personalizar dicho método:

- Si el primer objeto (o1) es menor que el segundo (o2), debe retornar un número entero negativo.
- Si el primer objeto (o1) es mayor que el segundo (o2), debe retornar un número entero positivo.
- Si ambos son iguales, debe retornar 0.

A veces, cuando el orden que deben tener los elementos es diferente al orden real (por ejemplo cuando ordenamos los números en orden inverso), la definición de antes puede ser un poco liosa, así que es recomendable en tales casos pensar de la siguiente forma:

- Si el primer objeto (o1) debe ir antes que el segundo objeto (o2), retornar entero negativo.
- Si el primer objeto (o1) debe ir después que el segundo objeto (o2), retornar entero positivo.
- Si ambos son iguales, debe retornar 0.

Una vez creado el comparador simplemente tenemos que pasarlo como parámetro en el momento de la creación al `TreeSet` , y los datos internamente mantendrán dicha ordenación:

```
1  TreeSet<Objeto> ts=new TreeSet<>(new ComparadorDeObjetos());
```

Hay otra manera de definir esta ordenación, pero lo estudiaremos más a fondo en el punto Comparadores

Para entender mejor los Sets revisa el Ejemplo04

Listas

¿En qué se diferencia una lista de un conjunto? Las listas son elementos de programación un poco más avanzados que los conjuntos. Su ventaja es que amplían el conjunto de operaciones de las colecciones añadiendo operaciones extra, veamos algunas de ellas:

- Las listas si **pueden almacenar duplicados**, si no queremos duplicados, hay que verificar manualmente que el elemento no esté en la lista antes de su inserción.
- **Acceso posicional**. Podemos acceder a un elemento indicando su posición en la lista.
- **Búsqueda**. Es posible buscar elementos en la lista y obtener su posición. En los conjuntos, al ser colecciones sin aportar nada nuevo, solo se podía comprobar si un conjunto contenía o no un elemento de la lista, retornando verdadero o falso. Las listas mejoran este aspecto.
- **Extracción de sublistas**. Es posible obtener una lista que contenga solo una parte de los elementos de forma muy sencilla.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

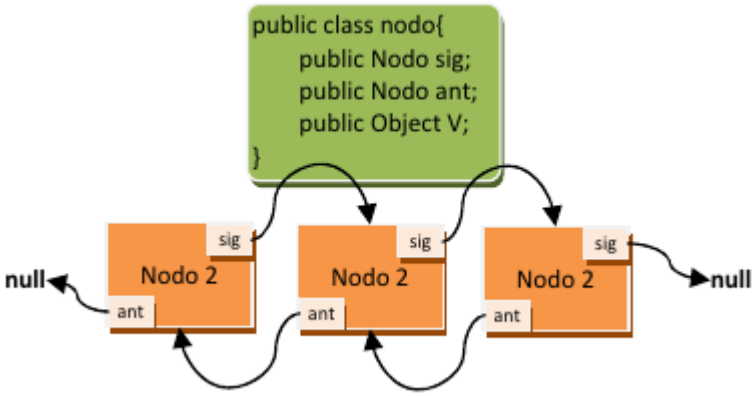
En Java, para las listas se dispone de una interfaz llamada `java.util.List`, y dos implementaciones (`java.util.LinkedList` y `java.util.ArrayList`), con diferencias significativas entre ellas. Los métodos de la interfaz `List`, que obviamente estarán en todas las implementaciones, y que permiten las operaciones anteriores son:

- **`E get(int index)`**. El método `get` permite obtener un elemento partiendo de su posición (index).
- **`E set(int index, E element)`**. El método `set` permite cambiar el elemento almacenado en una posición de la lista (index), por otro (element).
- **`void add(int index, E element)`** . Se añade otra versión del método `add`, en la cual se puede insertar un elemento (element) en la lista en una posición concreta (index), desplazando los existentes.
- **`E remove(int index)`**. Se añade otra versión del método `remove`, esta versión permite eliminar un elemento indicando su posición en la lista.
- **`boolean addAll(int index, Collection<? extends E> c)`**. Se añade otra versión del método `addAll` , que permite insertar una colección pasada por parámetro en una posición de la lista, desplazando el resto de elementos.
- **`int indexOf(Object o)`**. El método `indexOf` permite conocer la posición (índice) de un elemento, si dicho elemento no está en la lista retornará `-1`.
- **`int lastIndexOf(Object o)`**. El método `lastIndexOf` nos permite obtener la última ocurrencia del objeto en la lista (dado que la lista si puede almacenar duplicados).
- **`List<E> subList(int from, int to)`**. El método `subList` genera una sublista (una vista parcial de la lista) con los elementos comprendidos entre la posición inicial (incluida) y la posición final (no incluida).

Ten en cuenta que los elementos de una lista empiezan a numerarse por 0. Es decir, que el primer elemento de la lista es el 0. Ten en cuenta también que `List` es una interfaz genérica, por lo que `<E>` corresponde con el tipo base usado como parámetro genérico al crear la lista.

USO

Y, ¿cómo se usan las listas? Pues para usar una lista haremos uso de sus implementaciones `LinkedList` y `ArrayList`. Veamos un ejemplo de su uso y después obtendrás respuesta a esta pregunta.



Supongo que intuirás como se usan, pero nunca viene mal un ejemplo sencillo, que nos aclare las ideas. El siguiente ejemplo muestra como usar un `LinkedList` pero valdría también para `ArrayList` (no olvides importar las clases `java.util.LinkedList` y `java.util.ArrayList` según sea necesario). En este ejemplo se usan los métodos de acceso posicional a la lista:

```
1  LinkedList<Integer> t=new LinkedList<>(); // Declaración y creación del LinkedList de enteros.
2  t.add(1); // Añade un elemento al final de la lista.
3  t.add(3); // Añade otro elemento al final de la lista.
4  t.add(1,2); // Añade en la posición 1 el elemento 2.
5  t.add(t.get(1)+t.get(2)); // Suma los valores contenidos en la posición 1 y 2, y lo agrega al final.
6  t.remove(0); // Elimina el primer elementos de la lista.
```

En el ejemplo anterior, se realizan muchas operaciones, ¿cuál será el contenido de la lista al final? Pues será 2, 3 y 5. En el ejemplo cabe destacar el uso del bucle `for-each` , recuerda que se puede usar en cualquier colección.

Veamos otro ejemplo, esta vez con `ArrayList`, de cómo obtener la posición de un elemento en la lista:

```
1  ArrayList<Integer> al=new ArrayList<>(); // Declaración y creación del ArrayList de enteros.
2  al.add(10); al.add(11); // Añadimos dos elementos a la lista.
3  al.set(al.indexOf(11), 12); // Sustituimos el 11 por el 12, primero lo buscamos y luego lo reemplazamos.
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

En el ejemplo anterior, se emplea tanto el método `indexOf` para obtener la posición de un elemento, como el método `set` para reemplazar el valor en una posición, una combinación muy habitual. El ejemplo anterior generará un `ArrayList` que contendrá dos números, el 10 y el 12. Veamos ahora un ejemplo algo más difícil:

```
1  al.addAll(0, t.subList(1, t.size()));
```

> **Información**
>
> `subList` ➡️ *Returns a view of the portion of this list between the specified* `fromIndex`, *inclusive, and* `toIndex`, *exclusive.* (API de Java)

Este ejemplo es especial porque usa sublistas. Se usa el método `size` para obtener el tamaño de la lista. Después el método `subList` para extraer una sublista de la lista (que incluía en origen los números 2, 3 y 5), desde la posición 1 hasta el final de la lista (lo cual dejaría fuera al primer elemento). Y por último, se usa el método `addAll` para añadir todos los elementos de la sublista al `ArrayList` anterior. Y quedaria:

```
1  3, 5, 10 y 12.
```

Debes saber que las operaciones aplicadas a una sublista repercuten sobre la lista original. Por ejemplo, si ejecutamos el método `clear` sobre una sublista, se borrarán todos los elementos de la sublista, pero también se borrarán dichos elementos de la lista original:

```
1  ArrayList<Integer> alAux=new ArrayList<>();
2  alAux = al.subList(0, 2);
3  alAux.clear();
4
5  al.subList(0, 2).clear();
```

Lo mismo ocurre al añadir un elemento, se añade en la sublista y en la lista original.

Puedes consultar el código en el Ejemplo05

`LinkedList` Y `ArrayList`

¿Y en qué se diferencia un `LinkedList` de un `ArrayList` ?

Los **`LinkedList`** utilizan listas doblemente enlazadas, que son listas enlazadas (como se vio en un apartado anterior), pero que permiten ir hacia atrás en la lista de elementos. Los elementos de la lista se encapsulan en los llamados nodos.

Los nodos van enlazados unos a otros para no perder el orden y no limitar el tamaño de almacenamiento. Tener un doble enlace significa que en cada nodo se almacena la información de cuál es el siguiente nodo y además, de cuál es el nodo anterior. Si un nodo no tiene nodo siguiente o nodo anterior, se almacena null o nulo para ambos casos.

No es el caso de los **`ArrayList`**. Estos se implementan utilizando arrays que se van redimensionando conforme se necesita más espacio o menos. La redimensión es transparente a nosotros, no nos enteramos cuando se produce, pero eso redunda en una diferencia de rendimiento notable dependiendo del uso. Los **ArrayList** son más rápidos en cuanto a acceso a los elementos, acceder a un elemento según su posición es más rápido en un array que en una lista doblemente enlazada (hay que recorrer la lista). En cambio, eliminar un elemento implica muchas más operaciones en un array que en una lista enlazada de cualquier tipo.

¿Y esto que quiere decir? Que si se van a realizar muchas operaciones de eliminación de elementos sobre la lista, conviene usar una lista enlazada (`LinkedList`), pero si no se van a realizar muchas eliminaciones, sino que solamente se van a insertar y consultar elementos por posición, conviene usar una lista basada en arrays redimensionados (`ArrayList`).

`LinkedList` tiene otras ventajas que nos puede llevar a su uso. Implementa las interfaces `java.util.Queue` y `java.util.Deque` . Dichas interfaces permiten hacer uso de las listas como si fueran una cola de prioridad o una pila, respectivamente.

Las colas, también conocidas como colas de prioridad, son una lista pero que aportan métodos para trabajar de forma diferente. ¿Tú sabes lo que es hacer cola para que te atiendan en una ventanilla?

Pues igual. Se trata de que el que primero que llega es el primero en ser atendido (`FIFO` en inglés). Simplemente se aportan tres métodos nuevos: meter en el final de la lista (`add` y `offer`), sacar y eliminar el elemento más antiguo (`poll`), y examinar el elemento al principio de la lista sin eliminarlo (`peek`). Dichos métodos están disponibles en las listas enlazadas `LinkedList` :

- **`boolean add(E e)`** y **`boolean offer(E e)`**, retornarán true si se ha podido insertar el elemento al final de la `LinkedList`.
- **`E poll()`** retornará el primer elemento de la `LinkedList` y lo eliminará de la misma. Al insertar al final, los elementos más antiguos siempre están al principio. Retornará null si la lista está vacía.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- `E peek()` retornará el primer elemento de la `LinkedList` pero no lo eliminará, permite examinarlo. Retornará null si la lista está vacía.

Las pilas, mucho menos usadas, son todo lo contrario a las listas. Una pila es igual que una montaña de hojas en blanco, para añadir hojas nuevas se ponen encima del resto, y para retirar una se coge la primera que hay, encima de todas. En las pilas el último en llegar es el primero en ser atendido. Para ello se proveen de tres métodos: meter al principio de la pila (`push`), sacar y eliminar del principio de la pila (`pop`), y examinar el primer elemento de la pila (`peek` , igual que si usara la lista como una cola). Las pilas se usan menos y haremos menos hincapié en ellas. Simplemente ten en mente que, tanto las colas como las pilas, son una lista enlazada sobre la que se hacen operaciones especiales.

A TENER EN CUENTA

A la hora de usar las listas, hay que tener en cuenta un par de detalles, ¿sabes cuáles? Es sencillo, pero importante.

No es lo mismo usar las colecciones (listas y conjuntos) con objetos inmutables (`Strings` , `Integer` , etc.) que con objetos mutables. Los objetos inmutables no pueden ser modificados después de su creación, por lo que cuando se incorporan a la lista, a través de los métodos `add` , se pasan por copia (es decir, se realiza una copia de los mismos). En cambio los objetos mutables (como las clases que tú puedes crear), no se copian, y eso puede producir efectos no deseados.

Imagínate la siguiente clase, que contiene un número:

```
class Test {
    public Integer num;
    Test (int num) {
        this.num=new Integer(num);
    }
}
```

La clase de antes es mutable, por lo que no se pasa por copia a la lista. Ahora imagina el siguiente código en el que se crea una lista que usa este tipo de objeto, y en el que se insertan dos objetos:

```
Test p1=new Test(11); // Se crea un objeto Test donde el entero que contiene vale 11.
Test p2=new Test(12); // Se crea otro objeto Test donde el entero que contiene vale 12.
LinkedList<Test> lista=new LinkedList<Test>(); // Creamos una lista enlazada para objetos tipo Test.
lista.add(p1); // Añadimos el primero objeto test.
lista.add(p2); // Añadimos el segundo objeto test.
for (Test p:lista){
    System.out.println(p.num); // Mostramos la lista de objetos.
}
```

¿Qué mostraría por pantalla el código anterior? Simplemente mostraría los números 11 y 12. Ahora bien, ¿qué pasa si modificamos el valor de uno de los números de los objetos test? ¿Qué se mostrará al ejecutar el siguiente código?

```
p1.num=44;
for (Test p:lista) System.out.println(p.num);
```

El resultado de ejecutar el código anterior es que se muestran los números 44 y 12. El número ha sido modificado y no hemos tenido que volver a insertar el elemento en la lista para que en la lista se cambie también. Esto es porque en la lista no se almacena una copia del objeto Test, sino un apuntador a dicho objeto (solo hay una copia del objeto a la que se hace referencia desde distintos lugares).

Información

"*Controlar la complejidad es la esencia de la programación.*" **Brian Kernighan**

Consulta el Ejemplo06

Conjuntos de pares [clave/valor] (Diccionario)

¿Cómo almacenarías los datos de un diccionario? Tenemos por un lado cada palabra y por otro su significado. Para resolver este problema existen precisamente los arrays asociativos . Un tipo de array asociativo son los mapas o diccionarios, que permiten almacenar pares de valores conocidos como clave y valor. La clave se utiliza para acceder al valor, como una entrada de un diccionario permite acceder a su definición.

En Java existe la interfaz `java.util.Map` que define los métodos que deben tener los mapas, y existen tres implementaciones principales de dicha interfaz: `java.util.HashMap` , `java.util.TreeMap` y `java.util.LinkedHashMap` . ¿Te suenan? Claro que si. Cada una de ellas, respectivamente, tiene características similares a `HashSet` , `TreeSet` y `LinkedHashSet` , tanto en funcionamiento interno como en rendimiento.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Los mapas utilizan clases genéricas para dar extensibilidad y flexibilidad, y permiten definir un tipo base para la clave, y otro tipo diferente para el valor. Veamos un ejemplo de como crear un mapa, que es extensible a los otros dos tipos de mapas:

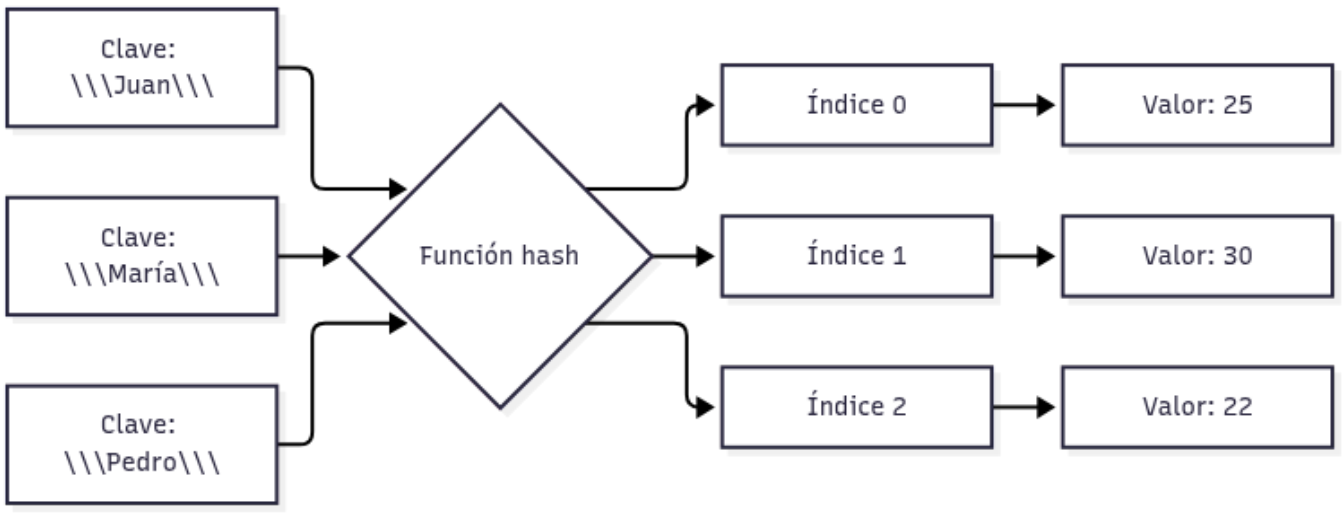
```
HashMap<String,Integer> t = new HashMap<>();
```

El mapa anterior permite usar cadenas como llaves y almacenar de forma asociada a cada llave, un número entero. Veamos los métodos principales de la interfaz `Map`, disponibles en todas las implementaciones. En los ejemplos, `V` es el tipo base usado para el valor (`Value`) y `K` el tipo base usado para la llave (`Key`):

Las claves de un `HashMap` se buscan igual que los elementos de un `HashSet`: si la clave es un objeto de tu propia clase, necesita `equals` y `hashCode` (revisa el aviso del apartado de conjuntos).

Método.	Descripción.
`V put(K key, V value);`	Inserta un par de objetos llave (key) y valor (value) en el mapa. Si la llave ya existe en el mapa, entonces retornará el valor asociado que tenía antes, si la llave no existía, entonces retornará null.
`V get(Object key);`	Obtiene el valor asociado a una llave ya almacenada en el mapa. Si no existe la llave, retornará null.
`V remove(Object key);`	Elimina la llave y el valor asociado. Retorna el valor asociado a la llave, por si lo queremos utilizar para algo, o null, si la llave no existe.
`boolean containsKey(Object key);`	Retornará true si el mapa tiene almacenada la llave pasada por parámetro, false en cualquier otro caso.
`boolean containsValue(Object value);`	Retornará true si el mapa tiene almacenado el valor pasado por parámetro, false en cualquier otro caso.
`int size();`	Retornará el número de pares llave y valor almacenado en el mapa.
`boolean isEmpty();`	Retornará true si el mapa está vacío, false en cualquier otro caso.
`void clear();`	Vacía el mapa.

Revisa el Ejemplo07



RECORRIDO CON `keySet` O `entrySet`

```
// Recorrido por claves
for (String clave:divisas.keySet()){
    System.out.format("%s --> %.02f\n", clave, divisas.get(clave));
}
// Recomendado cuando solo necesito las claves (aunque a partir de la clave puedes recuperar el valor)

// Recorrido por entries (pares clave/valor)
for (Map.Entry<String, Double> entry : divisas.entrySet()){
    System.out.format("%s --> %.02f\n", entry.getKey(), entry.getValue());
}
// Recomendado cuando necesito la clave y el valor
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

8.1.5 Iteradores

¿Qué son los iteradores realmente? Son un mecanismo que nos permite recorrer todos los elementos de una colección de forma sencilla, de forma secuencial, y de forma segura. Los mapas, como no derivan de la interfaz `Collection` realmente, no tienen iteradores, pero como veremos, existe un truco interesante.

Los iteradores permiten recorrer las colecciones de dos formas: bucles `for-each` (existentes en Java a partir de la versión 1.5) y a través de un bucle normal creando un iterador. Como los bucles `for-each` ya los hemos visto antes (y ha quedado patente su simplicidad), nos vamos a centrar en el otro método, especialmente útil en versiones antiguas de Java. Ahora la pregunta es, ¿cómo se crea un iterador? Pues invocando el método "`iterator()`" de cualquier colección.

Veamos un ejemplo (en el ejemplo `t` es una colección cualquiera):

```
1  Iterator<Integer> it=t.iterator();
```

Fíjate que se ha especificado un parámetro para el tipo de dato genérico en el iterador (poniendo `<Integer>` después de `Iterator`). Esto es porque los iteradores son también clases genéricas, y es necesario especificar el tipo base que contendrá el iterador. Sino se especifica el tipo base del iterador, igualmente nos permitiría recorrer la colección, pero retornará objetos tipo Object (clase de la que derivan todas las clases), con lo que nos veremos obligados a forzar la conversión de tipo.

Para recorrer y gestionar la colección, el iterador ofrece tres métodos básicos:

- `boolean hasNext()`. Retornará true si le quedan más elementos a la colección por visitar. False en caso contrario.
- `E next()`. Retornará el siguiente elemento de la colección, si no existe siguiente elemento, lanzará una excepción (`NoSuchElementException` para ser exactos), con lo que conviene chequear primero si el siguiente elemento existe.
- `remove()`. Elimina de la colección el último elemento retornado en la última invocación de next (no es necesario pasarselo por parámetro). Cuidado, si next no ha sido invocado todavía, saltará una incomoda excepción.

¿Cómo recorreríamos una colección con estos métodos? Pues de una forma muy sencilla, un simple bucle mientras (`while`) con la condición `hasNext()` nos permite hacerlo:

```
1  while (it.hasNext()) // Mientras haya siguiente elemento, seguiremos en el bucle.
2  {
3      Integer n=it.next(); // Escogemos el siguiente elemento.
4      if (n%2==0) it.remove(); //Si es par, eliminamos el elemento de la lista.
5  }
```

¿Qué elementos contendría la lista después de ejecutar el bucle? Efectivamente, solo los números impares.

Información

Las listas permiten acceso posicional a través de los métodos `get` y `set`, y acceso secuencial a través de iteradores, ¿cuál es para tí la forma más cómoda de recorrer todos los elementos? ¿Un acceso posicional a través un bucle `for (i=0;i<lista.size();i++)` o un acceso secuencial usando un bucle `while (iterador.hasNext())`?

¿Qué inconvenientes tiene usar los iteradores sin especificar el tipo de objeto? En el siguiente ejemplo, se genera una lista con los números del 0 al 10. De la lista, se eliminan aquellos que son pares y solo se dejan los impares. En el primer ejemplo se especifica el tipo de objeto del iterador, en el segundo ejemplo no, observa el uso de la conversión de tipos en la línea 7.

Ejemplo indicando el tipo de objeto de iterador.

```
1  ArrayList<Integer> lista=new ArrayList<>();
2  for (int i=0;i<10;i++){
3      lista.add(i);
4  }
5  //lista: [0,1,2,3,4,5,6,7,8,9]
6  Iterator<Integer> it=lista.iterator();
7  while (it.hasNext()) {
8      Integer n=it.next();
9      if (n%2==0){
10         it.remove();
11     }
12 }
13 //lista: [1,3,5,7,9]
```

Ejemplo no indicando el tipo de objeto del iterador,

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
ArrayList <Integer> lista=new ArrayList<Integer>();
for (int i=0;i<10;i++){
    lista.add(i);
}
Iterator it=lista.iterator();
while (it.hasNext()) {
    Integer n=(Integer)it.next();
    if (n%2==0){
        it.remove();
    }
}
```

Un iterador es seguro porque esta pensado para no sobrepasar los límites de la colección, ocultando operaciones más complicadas que pueden repercutir en errores de software. Pero realmente se convierte en inseguro cuando es necesario hacer la operación de conversión de tipos. Si la colección no contiene los objetos esperados, al intentar hacer la conversión, saltará una incómoda excepción.

Usar genéricos aporta grandes ventajas, pero usándolos adecuadamente.

Para recorrer los mapas con iteradores, hay que hacer un pequeño truco. Usamos el método `entrySet` que ofrecen los mapas para generar un conjunto con las entradas (pares de llave-valor), o bien, el método `keySet` para generar un conjunto con las llaves existentes en el mapa. Veamos como sería para el segundo caso, el más sencillo:

```
HashMap<Integer,Integer> mapa=new HashMap<>();
for (int i=0;i<10;i++){
    mapa.put(i, i); // Insertamos datos de prueba en el mapa.
}
for (Integer llave:mapa.keySet()){
    // Recorremos el conjunto generado por keySet, contendrá las llaves.
    Integer valor=mapa.get(llave); //Para cada llave, accedemos a su valor si es necesario.
}
```

Lo único que tienes que tener en cuenta es que el conjunto generado por `keySet` no tendrá obviamente el método `add` para añadir elementos al mismo, dado que eso tendrás que hacerlo a través del mapa.

Atención

Si usas iteradores, y piensas eliminar elementos de la colección (e incluso de un mapa), debes usar el método `remove` del iterador y no el de la colección. Si eliminas los elementos utilizando el método `remove` de la colección, mientras estás dentro de un bucle de iteración, o dentro de un bucle `for-each`, los fallos que pueden producirse en tu programa son impredecibles. ¿Logras adivinar porqué se pueden producir dichos problemas?

Los problemas son debidos a que el método `remove` del iterador elimina el elemento de dos sitios: de la colección y del iterador en sí (que mantiene interiormente información del orden de los elementos). Si usas el método `remove` de la colección, la información solo se elimina de un lugar, de la colección.

Consulta el Ejemplo08 para ver un ejemplo de iteradores.

Consulta el Ejemplo09 para ver el mismo ejemplo resuelto con un iterador.

8.1.6 Comparadores

En Java hay dos mecanismos para cambiar la forma en la que los elementos se ordenan. Imagina que tienes los artículos almacenados en una lista llamada `articulos`, y que cada artículo se almacena en la siguiente clase `Articulo` (fíjate que el código de artículo es una cadena y no un número):

```
class Articulo {
    public String codArticulo; // Código de artículo
    public String descripcion; // Descripción del artículo.
    public int cantidad; // Cantidad a proveer del artículo.
}
```

La primera forma de ordenar consiste en crear una clase que implemente la interfaz `java.util.Comparator`, y por ende, el método `compare` definido en dicha interfaz. Esto se explicó en el apartado de conjuntos, al explicar el `TreeSet`, así que no vamos a profundizar en ello. No obstante, el comparador para ese caso podría ser así:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
class comparadorArticulos implements Comparator<Articulo>{
    @Override
    public int compare( Articulo o1, Articulo o2) {
        return o1.codArticulo.compareTo(o2.codArticulo);
    }
}
```

Una vez creada esta clase, ordenar los elementos es muy sencillo, simplemente se pasa como segundo parámetro del método `sort` una instancia del comparador creado:

```
Collections.sort(coleccionArticulos, new comparadorArticulos());
```

La segunda forma es quizás más sencilla cuando se trata de objetos cuya ordenación no existe de forma natural, pero requiere modificar la clase `Articulo`. Consiste en hacer que los objetos que se meten en la lista o array implementen la interfaz `java.util.Comparable`. Todos los objetos que implementan la interfaz `Comparable` son "ordenables" y se puede invocar el método `sort` sin indicar un comparador para ordenarlos. La interfaz `comparable` solo requiere implementar el método `compareTo`:

```
class Articulo implements Comparable<Articulo>{
    public String codArticulo;
    public String descripcion;
    public int cantidad;

    @Override
    public int compareTo(Articulo o) {
        return codArticulo.compareTo(o.codArticulo);
    }
}
```

Del ejemplo anterior se pueden denotar dos cosas importantes: que la interfaz `Comparable` es genérica y que para que funcione sin problemas es conveniente indicar el tipo base sobre el que se permite la comparación (en este caso, el objeto `Articulo` debe compararse consigo mismo), y que el método `compareTo` solo admite un parámetro, dado que comparará el objeto con el que se pasa por parámetro.

El funcionamiento del método `compareTo` es el mismo que el método `compare` de la interfaz `Comparator`: si la clase que se pasa por parámetro es igual al objeto, se tendría que retornar 0; si es menor o anterior, se debería retornar un número menor que cero; si es mayor o posterior, se debería retornar un número mayor que 0.

Ordenar ahora la lista de artículos es sencillo, fíjate que fácil: `Collections.sort(coleccionArticulos);`

Consulta el Ejemplo10 para ver el uso de Comparator.

Consulta el Ejemplo11 para ver el uso de Comparable.

8.1.7 Extras

¿Qué más ofrece las clases `java.util.Collections` y `java.util.Arrays` de Java? Una vez vista la ordenación, quizás lo más complicado, veamos algunas operaciones adicionales. En los ejemplos, la variable `array` es un array y la variable `lista` es una lista de cualquier tipo de elemento:

Operación	Descripción	Ejemplos
Desordenar una lista.	Desordena una lista, este método no está disponible para arrays.	`Collections.shuffle (lista);`
Rellenar una lista o array.	Rellena una lista o array copiando el mismo valor en todos los elementos del array o lista. Útil para reiniciar una lista o array.	`Collections.fill (lista,elemento);` `Arrays.fill (array,elemento);`
Búsqueda binaria.	Permite realizar búsquedas rápidas en un una lista o array ordenados. Es necesario que la lista o array estén ordenados, sino lo están, la búsqueda no tendrá éxito.	`Collections.binarySearch(lista,elemento);` `Arrays.binarySearch(array, elemento);`

Convertir un array a lista.	Permite rápidamente convertir un array a una lista de elementos, extremadamente útil. No se especifica el tipo de lista retornado (no es `ArrayList` ni `LinkedList`), solo se especifica que retorna una lista que implementa la interfaz `java.util.List`.	`List lista = Arrays.asList(array);`
		Si el tipo de dato almacenado en el array es conocido (`Integer` por ejemplo), es conveniente especificar el tipo de objeto de la lista:
		`List<Integer> lista = Arrays.asList(array);`

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Operación	Descripción	Ejemplos

Convertir una lista a array.	Permite convertir una lista en array. Esto se puede realizar en todas las colecciones, y no es un método de la clase `Collections`, sino propio de la interfaz `Collection`. Es conveniente que sepas de su existencia.	Para este ejemplo, supondremos que los elementos de la lista son números, dado que hay que crear un array del tipo almacenado en la lista, y del tamaño de la lista:
		`Integer[] array=new Integer[lista.size()];`
		`lista.toArray(array);`

Dar la vuelta.	Da la vuelta a una lista, poniéndola en orden inverso al que tiene.	`Collections.reverse(lista);`
Imprimir un array o lista		lista.toString() Arrays.toString(array)

Otra operación que ya se ha visto en algún ejemplo anterior es la de dividir una cadena en partes. Cuando una cadena está formada internamente por trozos de texto claramente delimitados por un separador (una coma, un punto y coma o cualquier otro), es posible dividir la cadena y obtener cada uno de los trozos de texto por separado en un array de cadenas.

Para poder realizar esta operación, usaremos el método `split` de la clase `String` . El delimitador o separador es una expresión regular, único argumento del método `split`, y puede ser obviamente todo lo complejo que sea necesario:

```
String texto="Z,B,A,X,M,O,P,U";
String[] partes=texto.split(",");
//partes={"Z", "B", "A", "X", "M", "O", "P", "U"}
Arrays.sort(partes);//lo ordenamos
//partes={"A", "B", "M", "O", "P", "U", "X", "Z"}
```

En el ejemplo anterior la cadena texto contiene una serie de letras separadas por comas. La cadena se ha dividido con el método `split` , y se ha guardado cada carácter por separado en un `array`. Después se ha ordenado el `array`. ¡Increíble lo que se puede llegar a hacer con solo tres líneas de código!

8.1.8 Programación funcional

¿Qué es la programación funcional?

Paradigma de programación **declarativo**, no imperativo, se dice cómo es el problema a resolver, en lugar de los pasos a seguir para resolverlo.

La mayoría de lenguajes populares actuales no se pueden considerar funcionales, ni puros ni híbridos, pero han adaptado su sintaxis y funcionalidad para ofrecer parte de este paradigma.

Características principales

Transparencia referencial: la salida de una función debe depender sólo de sus argumentos. Si la llamamos varias veces con los mismos argumentos, debe producir siempre el mismo resultado.

Inmutabilidad de los datos: los datos deben ser inmutables para evitar posibles efectos colaterales.

Composición de funciones: las funciones se tratan como datos, de modo que la salida de una función se puede tomar como entrada para la siguiente.

Funciones de primer orden: funciones que permiten tener otras funciones como parámetros, a modo de callbacks.

Información

Si llamamos repetidamente a esta función con el parámetro 1, cada vez producirá un resultado distinto (3, 4, 5...)

```
class Prueba{
    static int valorExterno = 1;

    static int unaFuncion(int parametro){
        valorExterno++;
        return valorExterno + parametro;
    }
}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Imperativo vs Declarativo

Queremos obtener una sublista con los mayores de edad de entre una lista de personas:

Imperativo:

```
List<Persona> adultos = new ArrayList<>();
for (int i = 0; i < personas.size(); i++){
    if (personas.get(i).getEdad() >= 18)
        adultos.add(personas.get(i));
}
```

Declarativo

```
List<Persona> adultos = personas.stream().filter(p -> p.getEdad() >= 18).collect(Collectors.toList());
```

Se puede observar que el ejemplo declarativo es más compacto, y menos propenso a errores. (Además sirve de ejemplo a la composición de funciones)

Funciones Lambda

Son expresiones breves que simplifican la implementación de elementos más costosos en cuanto a líneas de código. También se las conoce como funciones anónimas, no necesitan una clase/nombre. En java se pueden aplicar a la implementación de interfaces, aunque tienen más utilidades prácticas. En algunos lenguajes se les suele denominar "funciones flecha" (arrow functions) ya que en su sintaxis es característica una flecha, que separa la cabecera de la función de su cuerpo.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Comparaciones

API del método `List.sort` de Java:

```
default void sort(Comparator<? super E> c)
```

La interfaz `Comparator` pide implementar un método `compare`, que recibe dos datos del tipo a tratar (T), y devuelve un entero indicando si el primero es menor, mayor, o son iguales (de forma similar al método `compareTo` de la interfaz `Comparable`.)

```
int compare (T o1, T o2)
```

Imaginemos una clase `Persona`:

```
class Persona{
    private String nombre;
    private int edad;
    ...
}
```

Y un ArrayList `personas` formada por objetos de tipo `Persona`:

```
...
ArrayList<Persona> personas = new ArrayList<>();
personas.add(new Persona("Nacho", 52));
personas.add(new Persona("David", 47));
personas.add(new Persona("Pepe", 42));
personas.add(new Persona("Maria", 22));
personas.add(new Persona("Marta", 4));
...
```

Ahora queremos ordenar el `ArrayList` de `personas` de mayor a menor edad usando...

Implementación "tradicional" java: `Comparator` o `Comparable`

```
...
   class ComparadorPersona implements Comparator <Persona>{
       @Override
       public int compare(Persona p1, Persona p2){
           return p2.getEdad() - p1.getEdad();
       }
   }
...
```

```
...
personas.sort(new ComparadorPersona());
for (int i = 0; i < personas.size(); i++){
    System.out.println(personas.get(i));
}
...
```

Sin embargo, implementado con funciones Lambda seria...

```
...
personas.sort((p1, p2) -> p2.getEdad() - p1.getEdad());
for (int i = 0; i < personas.size(); i++){
    System.out.println(personas.get(i));
}
...
```

ESTRUCTURA DE UNA EXPRESIÓN LAMBDA

`(lista de parametros) -> {cuerpo de la función a implementar}`

- El **operador lambda** (`->`) separa la declaración de parámetros de la declaración del cuerpo de la función.
- Los **parámetros** del lado izquierdo de la flecha se pueden omitir si sólo hay un parámetro. Cuando no se tienen parámetros, o cuando se tienen dos o más, es necesario utilizar paréntesis.
- El **cuerpo de la función** son las llaves de la parte derecha se pueden omitir si la única operación a realizar es un simple `return`.

FUNCIONES LAMBDA (LAS UTILIZAREMOS A FONDO CON LAS `Interfaces`)

```
z -> z + 2 //un sólo parámetro
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
() -> System.out.println("Mensaje 1") //sin parámetros
```

```
(int longitud, int altura) -> { return altura * longitud; } //dos parámetros
```

```
(String x) -> {
 String retorno = x;
 retorno = retorno.concat("****");
 return retorno;
} //un bloque de código más elaborado
```

Gestión de colecciones con streams en Java

Desde Java 8, permiten procesar grandes cantidades de datos aprovechando la paralelización que permita el sistema. No modifican la colección original, sino que crean copias.

Dos tipos de operaciones

- **Intermedias**: devuelven otro stream resultado de procesar el anterior de algún modo (filtrado, mapeo), para ir enlazando operaciones
- **Finales**: cierran el stream devolviendo algún resultado (colección resultante, cálculo numérico, etc).

Muchas de estas operaciones tienen como parámetro una interfaz, que puede implementarse muy brevemente empleando expresiones lambda

FILTRADO

El método `filter` es una operación intermedia que permite quedarnos con los datos de una colección que cumplan el criterio indicado como parámetro. `filter` recibe como parámetro una interfaz `Predicate`, cuyo método test recibe como parámetro un objeto y devuelve si ese objeto cumple o no una determinada condición.

```
[...]
Stream<Persona> adultos = personas.stream().filter(p -> p.getEdad() >= 18);
//La función lambda se podría traducir como: "Aquellas personas 'p' de la colección cuya edad sea mayor o igual que 18 años"
```

MAPEO

El método `map` es una operación intermedia que permite transformar la colección original para quedarnos con cierta parte de la información o crear otros datos. `map` recibe como parámetro una interfaz `Function`, cuyo método `apply` recibe como parámetro un objeto y devuelve otro objeto diferente, normalmente derivado del parámetro.

```
[...]
Stream<Integer> edades = personas.stream().map(p -> p.getEdad());
//La función lambda hace que se añadan al stream de enteros las edades de las personas 'p' de la colección personas.
```

COMBINAR

Se pueden combinar operaciones intermedias (composición de funciones) para producir resultados más complejos. Por ejemplo, las edades de las personas adultas.

```
[...]
Stream<Integer> edadesAdultos = personas.stream()
    .filter(p -> p.getEdad() >= 18).map (p -> p.getEdad());
//Añadiriamos al stream solamente las edades, de aquellas personas que son mayores de edad.
```

ORDENAR

El método `sorted` es una operación intermedia que permite ordenar los elementos de una colección según cierto criterio. Por ejemplo, ordenar las personas adultas por edad. `sorted` recibe como parámetro una interfaz `Comparator`, que ya conocemos.

```
Stream<Persona> personasOrdenadas = personas.stream()
    .filter(p -> p.getEdad() >= 18)
    .sorted((p1, p2) -> p1.getEdad() - p2.getEdad());
//Para cada pareja de personas p1 y p2, ordénalas en funcion de la resta de la edad de p1 menos la edad de p2 (lo que haciamos en el compareTo)
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

COLECCIÓN

El método `collect` es una operación final que permite obtener algún tipo de colección a partir de los datos procesados por las operaciones intermedias. Por ejemplo, una lista con las edades de las personas adultas.

```
List<Integer> edadesAdultos = personas.stream().filter(p -> p.getEdad() >= 18).map(p -> p.getEdad()).collect(Collectors.toList());
//similar a ejemplos anteriores, pero esta vez obtenemos una lista de enteros, en lugar de un stream.
```

El método `collect` también permite obtener una cadena de texto que una los elementos resultantes, a través de un separador común. En la función `Collectors.joining` se puede indicar también un prefijo y un sufijo para el texto.

```
String nombresAdultos = personas.stream().filter(p -> p.getEdad() >= 18)
    .map(p -> p.getNombre())
    .collect(Collectors.joining(", ","Adultos: ",""));
//genera una lista de nombres de personas, con un prefijo, separado y sufijo.
```

forEach

El método `forEach` permite recorrer cada elemento del stream resultante, y hacer lo que se necesite con él. Por ejemplo, sacar por pantalla en líneas separadas los nombres de las personas adultas.

```
personas.stream().filter(p -> p.getEdad() >= 18)
    .map(p -> p.getNombre()).forEach(p -> System.out.println(p));
```

MEDIA ARITMÉTICA

El método average permite, junto con la operación intermedia mapToInt, obtener una media de un stream que haya producido una colección resultante numérica. Por ejemplo, la media de edades de las personas adultas.

```
double mediaAdultos = personas.stream().filter(p -> p.getEdad() >= 18)
    .mapToInt(p -> p.getEdad()).average().getAsDouble();
```

Observa el Ejemplo12 para ver la diferencia entre programación imperativa y funcional con Stream.

Observa el Ejemplo13 para ver ordenación con Comparator vs. expresión lambda.

Observa el Ejemplo14 para ver operaciones funcionales con Stream: filtrado, mapeo, ordenación y combinación.

8.1.9 Ejemplos UD07

Ejemplo01

Uso básico de `TreeSet`: conjunto ordenado de enteros.

```
package UD07.P2_2_Sets;

import java.util.TreeSet;

public class Ejemplo01 {

    public static void main(String[] args) {
        TreeSet<Integer> t = new TreeSet<>();
        t.add(4);
        t.add(3);
        t.add(1);
        t.add(99);
        for (Integer i : t) {
            System.out.print(i + " ");
        }
}
```

⬆ Volver a teoría

Ejemplo02

Uso básico de `LinkedHashSet`: conjunto que mantiene el orden de inserción.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD07.P2_2_Sets;

import java.util.LinkedHashSet;

public class Ejemplo02 {

    public static void main(String[] args) {
        LinkedHashSet<Integer> t = new LinkedHashSet<>();
        t.add(4);
        t.add(3);
        t.add(1);
        t.add(99);
        for (Integer i : t) {
            System.out.print(i + " ");
        }
}
```

⬆ Volver a teoría

Ejemplo03

Operaciones sobre conjuntos: `addAll`, `removeAll` y `retainAll`.

```java
package UD07.P2_2_Sets;

import java.util.Collection;
import java.util.LinkedHashSet;
import java.util.TreeSet;

/**
 *
 * @author David Martínez (wwww.martinezpenya.es|ieseduardoprimo.es)
 */
public class Ejemplo03 {

    private static void imprimirColeccion(Collection<?> c) {
        for (Object i : c) {
            System.out.print(i.toString() + " ");
        }
        System.out.println("");
    }

    public static void main(String[] args) {
        TreeSet<Integer> conjuntoA = new TreeSet<>();
        conjuntoA.add(9);
        conjuntoA.add(19);
        conjuntoA.add(5);
        conjuntoA.add(7); // Elementos del conjunto A: 9, 19, 5 y 7
        LinkedHashSet<Integer> conjuntoB = new LinkedHashSet<>();
        conjuntoB.add(10);
        conjuntoB.add(20);
        conjuntoB.add(5);
        conjuntoB.add(7); // Elementos del conjunto B: 10, 20, 5 y 7

        conjuntoA.addAll(conjuntoB);
        imprimirColeccion(conjuntoA); //5 7 9 10 19 20

        conjuntoA.removeAll(conjuntoB);
        imprimirColeccion(conjuntoA); //9 19

        //recolocamos todo como al principio
        conjuntoA.add(5);
        conjuntoA.add(7);
        conjuntoB.add(10);
        conjuntoB.add(20);
        conjuntoB.add(5);
        conjuntoB.add(7);

        conjuntoA.retainAll(conjuntoB);
        imprimirColeccion(conjuntoA); //5 7

    }
}
```

⬆ Volver a teoría

Ejemplo04

Realiza un pequeño programa que pregunte al usuario 5 números diferentes (almacenándolos en un `HashSet`), y que después calcule la suma de los mismos (usando un bucle `for-each`).

Respuesta:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Una solución posible podría ser la siguiente. Fíjate en la solución y verás que el uso de conjuntos ha simplificado enormemente el ejercicio, permitiendo al programador o la programadora centrarse en otros aspectos:

```java
package UD07.P2_HashSet;

import java.util.HashSet;
import java.util.Scanner;

public class EjemploHashSet {

    public static void main(String[] args) {
        HashSet<Integer> conjunto = new HashSet<Integer>();
        Scanner teclado = new Scanner(System.in);
        int numero;
        do {
            try {
                System.out.print("Introduce un número " + (conjunto.size() + 1) + ": ");
                numero = teclado.nextInt();
                if (!conjunto.add(numero)) {
                    System.out.println("Número ya en la lista. Debes introducir otro.");
                }
            } catch (NumberFormatException e) {
                System.out.println("Número erróneo.");
            }
        } while (conjunto.size() < 5);
        // Calcular la suma
        Integer suma = 0;
        for (Integer i : conjunto) {
            suma = suma + i;
        }
        System.out.println("La suma es: " + suma);
    }
}
```

⬆ Volver a teoría

Ejemplo05

Operaciones con `LinkedList` y `ArrayList`: inserción, eliminación y uso de `subList`.

```java
package UD07.P2_3_Listas;

import java.util.ArrayList;
import java.util.Collection;
import java.util.LinkedList;

public class Ejemplo05 {

    private static void imprimirColeccion(Collection<?> c) {
        for (Object i : c) {
            System.out.print(i.toString() + " ");
        }
        System.out.println("");
    }
    public static void main(String[] args) {
        LinkedList<Integer> t = new LinkedList<>(); // Declaración y creación del LinkedList de enteros.
        t.add(1); // Añade un elemento al final de la lista.
        t.add(3); // Añade otro elemento al final de la lista.
        t.add(1, 2); // Añade en la posición 1 el elemento 2.
        t.add(t.get(1) + t.get(2)); // Suma los valores contenidos en la posición 1 y 2, y lo agrega al final.
        t.remove(0); // Elimina el primer elementos de la lista.
        imprimirColeccion(t); //2 3 5

        ArrayList<Integer> al = new ArrayList<>(); // Declaración y creación del ArrayList de enteros.
        al.add(10);
        al.add(11); // Añadimos dos elementos a la lista.
        al.set(al.indexOf(11), 12); // Sustituimos el 11 por el 12, primero lo buscamos y luego lo reemplazamos.

        al.addAll(0, t.subList(1, t.size()));
        imprimirColeccion(al); //3 5 10 12

        al.subList(0, 2).clear();
        imprimirColeccion(al); //10 12
    }
}
```

⬆ Volver a teoría

Ejemplo06

Tenemos la clase `Producto` con:

- Dos atributos: nombre (`String`) y cantidad (`int`).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- Un constructor con parámetros.
- Un constructor sin parámetros.
- Métodos `get` y `set` asociados a los atributos.

`Producto.java`

```java
package UD07.P2_3_Listas;

public class Producto {

    //Atributos
    private String nombre;
    private int cantidad;

    //Métodos
    //Constructor con parámetros donde asignamos el valor dado a los atributos
    public Producto(String nombre, int cantidad) {
        this.nombre = nombre;
        this.cantidad = cantidad;
    }

    //Constructor sin parámetros donde inicializamos los atributos
    public Producto() {
        //La palabra reservada null se utiliza para inicializar los objetos,
        //indicando que el puntero del objeto no apunta a ninguna dirección
        //de memoria. No hay que olvidar que String es una clase.
        this.nombre = null;
        this.cantidad = 0;
    }

    //Metodo get y set
    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public int getCantidad() {
        return cantidad;
    }

    public void setCantidad(int cantidad) {
        this.cantidad = cantidad;
    }
}
```

En el programa principal creamos una lista de productos y realizamos operaciones sobre ella:

`Ejemplo06.java`

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD07.P2_3_Listas;

import java.util.ArrayList;

public class Ejemplo06 {

    public static void main(String[] args) {

        //Definimos 5 instancias de la clase Producto
        Producto p1 = new Producto("Pan", 6);
        Producto p2 = new Producto("Leche", 2);
        Producto p3 = new Producto("Manzanas", 5);
        Producto p4 = new Producto("Brocoli", 2);
        Producto p5 = new Producto("Carne", 2);

        //Definir un ArrayList
        ArrayList<Producto> lista = new ArrayList<>();

        //Colocar instancias de producto en ArrayList
        lista.add(p1);
        lista.add(p2);
        lista.add(p3);
        lista.add(p4);

        //Añadimos "Carne" en la posición 1 de la lista
        lista.add(1, p5);

        //Añadimos "Carne" en la última posición
        lista.add(p5);

        //Imprimir el contenido del ArrayList
        System.out.println(" - Lista con " + lista.size() + " elementos");

        for (Producto p : lista) {
            System.out.println(p.getNombre() + " : " + p.getCantidad());
        }

        p5.setCantidad(99); //cambiamos la cantidad al producto, cambiará la lista?

        ((Producto)lista.get(1)).setCantidad(66); //

        System.out.println(p5.getCantidad());

        //Imprimir el contenido del ArrayList
        System.out.println(" - Lista con " + lista.size() + " elementos");

        for (Producto p : lista) {
            System.out.println(p.getNombre() + " : " + p.getCantidad());
        }

        //Eliminar todos los valores del ArrayList
        lista.clear();
        System.out.println(" - Lista final con " + lista.size() + " elementos");
    }
}
```

⬆ Volver a teoría

Ejemplo07

Uso básico de `HashMap`: inserción, búsqueda por clave y valor, eliminación y recorrido.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD07.P2_4_Maps;

import java.util.HashMap;

public class Ejemplo07 {

    public static void main(String[] args) {
        HashMap<String, Integer> hashMap = new HashMap<>();
        //Insertamos un solo elemento A con valor 1
        hashMap.put("A", 1);

        //Busqueda por clave
        if (hashMap.containsKey("A")) {
            System.out.printf("Contiene la clave A. Su valor es: %d\n", hashMap.get("A"));
        }

        //Busqueda por valor
        if (hashMap.containsValue(0)) {
            System.out.println("Contiene el valor 0");
        }

        //Eliminar el elemento con clave A
        hashMap.remove("A");

        //Ahora añadimos varios elementos para imprimirlos
        hashMap.put("A", 1);
        hashMap.put("E", 12);
        hashMap.put("I", 15);
        hashMap.put("O", 0);
        hashMap.put("U", 0);
        //Recorremos el mapa y lo imprimimos
        for (HashMap.Entry<String, Integer> entry : hashMap.entrySet()) {
            System.out.printf("Clave: %s. Valor: %d\n", entry.getKey(), entry.getValue());
        }
    }
}
```

⬆ Volver a teoría

Ejemplo08

Ejemplo que crea, rellena y recorre un `ArrayList` de dos formas diferentes. Cabe destacar que, por defecto, el método `System.out.println()` invoca al método `toString()` de los elementos que se le pasen como argumento, por lo que realmente no es necesario utilizar `toString()` dentro de `println()`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD07.P3.Iterators;

import java.util.ArrayList;
import java.util.Iterator;

public class Ejemplo08 {

    public static void main(String[] args) {
        //creamos la lista
        ArrayList l = new ArrayList();

        //Añadimos elementos al final de la lista
        l.add("uno");
        l.add("dos");
        l.add("tres");
        l.add("cuatro");

        //Añadimos el elemento en la posición 2
        l.add(2, "dos2");

        System.out.println(l.size()); //devuelve 5
        System.out.println(l.get(0)); //devuelve uno
        System.out.println(l.get(1)); //devuelve dos
        System.out.println(l.get(2)); //devuelve dos2
        System.out.println(l.get(3)); //devuelve tres
        System.out.println(l.get(4)); //devuelve cuatro

        //Recorremos la lista con un for y mostramos su contenido
        for (int i = 0; i < l.size(); i++) {
            System.out.print(l.get(i));
        }//imprime: unodosdos2trescuatro

        System.out.println();

        //Recorremos la lista con un Iterador
        //creamos el iterador
        Iterator it = l.iterator();

        //mientras haya elementos
        while (it.hasNext()) {
            System.out.print(it.next()); //obtengo el siguiente elemento
        }//imprime; unodosdos2trescuatro

        System.out.println();

        for (Object s : l) {
            System.out.print(s);
        }//imprime; unodosdos2trescuatro
    }
}
```

⬆ Volver a teoría

Ejemplo09

Recorrido de una lista con `Iterator` y modificación de objetos mutables dentro de la colección.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD07.P3.Iterators;

import UD07.P2_3_Listas.Producto;
import java.util.ArrayList;
import java.util.Iterator;

public class Ejemplo09 {

    public static void main(String[] args) {

        //Definimos 5 instancias de la clase Producto
        Producto p1 = new Producto("Pan", 6);
        Producto p2 = new Producto("Leche", 2);
        Producto p3 = new Producto("Manzanas", 5);
        Producto p4 = new Producto("Brocoli", 2);
        Producto p5 = new Producto("Carne", 2);

        //Definir un ArrayList
        ArrayList<Producto> lista = new ArrayList<>();

        //Colocar instancias de producto en ArrayList
        lista.add(p1);
        lista.add(p2);
        lista.add(p3);
        lista.add(p4);

        //Añadimos "Carne" en la posición 1 de la lista
        lista.add(1, p5);

        //Añadimos "Carne" en la última posición
        lista.add(p5);

        //Imprimir el contenido del ArrayList
        System.out.println(" - Lista con " + lista.size() + " elementos");

        //Definir Iterator para extraer/imprimir valores
        //si queremos utilizar un for con el iterador no hace falta poner el incremento
        for (Iterator<Producto> it = lista.iterator(); it.hasNext();) {
            Producto p = it.next();
            System.out.println(p.getNombre() + " : " + p.getCantidad());
        }

        p5.setCantidad(99); //cambiamos la cantidad al producto, cambiará la lista?

        ((Producto)lista.get(1)).setCantidad(66); //

        System.out.println(p5.getCantidad());

        //Imprimir el contenido del ArrayList
        System.out.println(" - Lista con " + lista.size() + " elementos");

        //Definir Iterator para extraer/imprimir valores
        //si queremos utilizar un for con el iterador no hace falta poner el incremento
        for (Iterator<Producto> it = lista.iterator(); it.hasNext();) {
            Producto p = it.next();
            System.out.println(p.getNombre() + " : " + p.getCantidad());
        }

        //Eliminar todos los valores del ArrayList
        lista.clear();
        System.out.println(" - Lista final con " + lista.size() + " elementos");
    }
}
```

⬆ Volver a teoría

Ejemplo10

Ejercicio resuelto `Comparator1`. Imagínate que Objeto es una clase como la siguiente:

```java
package UD07.P4.Comparator1;

public class Objeto {

    public int a;
    public int b;

    public Objeto(int a, int b) {
        this.a = a;
        this.b = b;
    }

    @Override
    public String toString() {
        return "Objeto{" + "a=" + a + ", b=" + b + '}';
    }

}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Imagina que ahora, al añadirlos en un `TreeSet`, estos se tienen que ordenar de forma que la suma de sus atributos (**a** y **b**) sea descendente, ¿como sería el comparador?

Respuesta

Una de las posibles soluciones a este problema podría ser la siguiente:

```java
package UD07.P4.Comparator1;

import java.util.Comparator;

class ComparadorDeObjetos implements Comparator<Objeto> {

    @Override
    public int compare(Objeto o1, Objeto o2) {
        int sumao1 = o1.a + o1.b;
        int sumao2 = o2.a + o2.b;
        if (sumao1 < sumao2) {
            return -1;
        } else if (sumao1 > sumao2) {
            return 1;
        } else {
            return 0;
        }
    }
}
```

Y para usarlo tendriamos:

```java
package UD07.P4.Comparator1;

import java.util.TreeSet;

public class Principal {

    public static void main(String[] args) {
        TreeSet<Objeto> ts = new TreeSet<Objeto>(new ComparadorDeObjetos());

        Objeto o1= new Objeto(0, 1);
        ts.add(o1);

        ts.add(new Objeto(1, 2));
        ts.add(new Objeto(4, 5));
        ts.add(new Objeto(2, 3));

        for (Objeto o : ts) {
            System.out.println(o);
        }
    }
}
```

Observa que la salida muestra los elementos correctamente ordenados, aunque se insertaron de manera "aleatoria":

```
Objeto{a=4, b=5}
Objeto{a=2, b=3}
Objeto{a=1, b=2}
Objeto{a=0, b=1}
```

⬆ Volver a teoría

Ejemplo11

Ejercicio resuelto `Comparator2`. Ahora convertiremos la clase `Objeto` para que directamente implemente la interfaz `Comparable`:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD07.P4.Comparator2;

public class Objeto implements Comparable<Objeto> {

    public int a;
    public int b;

    public Objeto(int a, int b) {
        this.a = a;
        this.b = b;
    }

    @Override
    public String toString() {
        return "Objeto{" + "a=" + a + ", b=" + b + '}';
    }

    @Override
    public int compareTo(Objeto t) {
        int sumao1 = this.a + this.b;
        int sumao2 = t.a + t.b;
        if (sumao1 < sumao2) {
            return -1;
        } else if (sumao1 > sumao2) {
            return 1;
        } else {
            return 0;
        }
    }

}
```

Y lo usamos directamente en la clase `Principal`:

```java
package UD07.P4.Comparator2;

import java.util.TreeSet;

public class Principal {

    public static void main(String[] args) {
        TreeSet<Objeto> ts = new TreeSet<Objeto>();

        ts.add(new Objeto(0, 1));
        ts.add(new Objeto(1, 2));
        ts.add(new Objeto(4, 5));
        ts.add(new Objeto(2, 3));

        for (Objeto o : ts) {
            System.out.println(o);
        }
    }
}
```

Fíjate que la salida sigue mostrando los elementos correctamente ordenados, aunque se insertaron de manera "aleatoria":

```
Objeto{a=4, b=5}
Objeto{a=2, b=3}
Objeto{a=1, b=2}
Objeto{a=0, b=1}
```

⬆ Volver a teoría

Ejemplo12

Programación imperativa vs. funcional con `Stream`: filtrado de personas mayores de edad.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD07.P6_Funcional;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

class Persona{

    private String nombre;
    private int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    public String getNombre() {
        return nombre;
    }

    public int getEdad() {
        return edad;
    }

    public String toString(){
        return nombre + " " + edad;
    }
}

public class P6_2_ImperativosFuncional {
    public static void main(String[] args) {

        List<Persona> personas = new ArrayList<>();

        personas.add(new Persona("Nacho", 52));
        personas.add(new Persona("David", 47));
        personas.add(new Persona("Pepe", 42));
        personas.add(new Persona("Maria", 22));
        personas.add(new Persona("Marta", 4));

        //IMPERATIVO
        List<Persona> adultosImperativo = new ArrayList<>();
        for (int i = 0; i < personas.size(); i++) {
            if (personas.get(i).getEdad() >= 18)
                adultosImperativo.add(personas.get(i));
        }
        for (Persona p : adultosImperativo) {
            System.out.println(p.getNombre() + " " + p.getEdad());
        }

        //FUNCIONAL
        List<Persona> adultosDeclarativo = personas.stream().filter(p -> p.getEdad() >= 18).collect(Collectors.toList());
        for (Persona p : adultosDeclarativo) {
            System.out.println(p.getNombre() + " " + p.getEdad());
        }
    }
}
```

⬆ Volver a teoría

Ejemplo13

Ordenación con `Comparator` vs. expresión lambda.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD07.P6_Funcional;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
class ComparadorPersona implements Comparator<Persona> {
    @Override
    public int compare(Persona p1, Persona p2){
        return p2.getEdad() - p1.getEdad();
    }
}
public class P6_3_Lambda {
    public static void main(String[] args) {

        List<Persona> personas = new ArrayList<>();

        personas.add(new Persona("Nacho", 52));
        personas.add(new Persona("David", 47));
        personas.add(new Persona("Pepe", 42));
        personas.add(new Persona("Maria", 22));
        personas.add(new Persona("Marta", 4));

        //IMPERATIVO
        personas.sort(new ComparadorPersona());
        for (int i = 0; i < personas.size(); i++){
            System.out.println(personas.get(i).toString());
        }

        //FUNCIONAL
        personas.sort((p1, p2) -> p2.getEdad() - p1.getEdad());
        for (int i = 0; i < personas.size(); i++){
            System.out.println(personas.get(i).toString());
        }
    }
}
```

⬆ Volver a teoría

Ejemplo14

Operaciones funcionales con `Stream`: filtrado, mapeo, ordenación y combinación.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD07.P6_Funcional;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class P6_4_ColeccionesStreams {

    public static void main(String[] args) {
        ArrayList<Persona> personas = new ArrayList<>();
        personas.add(new Persona("Nacho", 52));
        personas.add(new Persona("David", 47));
        personas.add(new Persona("Pepe", 42));
        personas.add(new Persona("Maria", 22));
        personas.add(new Persona("Marta", 4));

        // FILTRADO
        System.out.println("FILTRADO");
        Stream<Persona> adultos = personas.stream().filter(p -> p.getEdad() >= 18);
        //La función lambda se podría traducir como: "Aquellas personas 'p' de la
        // colección cuya edad sea mayor o igual que 18 años".
        adultos.forEach(p -> System.out.println(p.getNombre() + " " + p.getEdad()));

        // MAPEO
        System.out.println("MAPEO");
        Stream<Integer> edades = personas.stream().map(p -> p.getEdad());
        //La función lambda hace que se añadan al stream de enteros las edades de las
        // personas 'p' de la colección personas.
        edades.forEach(e -> System.out.println(e));

        // COMBINAR
        System.out.println("COMBINAR");
        Stream<Integer> edadesAdultos = personas.stream().
                filter(p -> p.getEdad() >= 18).map(p -> p.getEdad());
        //Añadiríamos al stream solamente las edades, de aquellas personas que son mayores de edad.
        edadesAdultos.forEach(e -> System.out.println(e));

        // ORDENAR
        System.out.println("ORDENAR");
        Stream<Persona> personasOrdenadas = personas.stream()
                .filter(p -> p.getEdad() >= 18)
                .sorted((p1, p2) -> p1.getEdad() - p2.getEdad());
        //Para cada pareja de personas p1 y p2, ordénalas en función de la resta
        // de la edad de p1 menos la edad de p2 (lo que hacíamos en el compareTo)
        personasOrdenadas.forEach(p -> System.out.println(p.getNombre() + " " + p.getEdad()));

        // COLECCIÓN
        System.out.println("COLECCIÓN");
        List<Integer> edadesAdultos2 = personas.stream()
                .filter(p -> p.getEdad() >= 18)
                .map(p -> p.getEdad()).collect(Collectors.toList());
        //similar a ejemplos anteriores, pero esta vez obtenemos una lista de enteros,
        // en lugar de un stream.
        System.out.println("Lista: " + edadesAdultos2);

        String nombresAdultos = personas.stream().filter(p -> p.getEdad() >= 18)
                .map(p -> p.getNombre())
                .collect(Collectors.joining(", ","Adultos: ",""));
        //genera una lista de nombres de personas, con un prefijo, separado y sufijo.
        System.out.println("String: " + nombresAdultos);

        // FOREACH
        System.out.println("FOREACH");
        personas.stream().filter(p -> p.getEdad() >= 18)
                .map(p -> p.getNombre()).forEach(p -> System.out.println(p));

        // MEDIA
        System.out.println("MEDIA");
        double mediaAdultos = personas.stream().filter(p -> p.getEdad() >= 18)
                .mapToInt(p -> p.getEdad()).average().getAsDouble();
        System.out.println("Media adultos: " + mediaAdultos);
    }
}
```

⬆ Volver a teoría

8.1.10 Resumen — Conceptos clave

Concepto	Definición
Colección	Estructura de datos que almacena y organiza objetos
List	Colección ordenada que permite elementos duplicados
Set	Colección que NO permite elementos duplicados

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Concepto	Definición
Map	Estructura que asocia claves únicas a valores
Iterador	Objeto que permite recorrer una colección
Lambda	Función anónima que permite programación funcional
Stream	Secuencia de elementos sobre la que se aplican operaciones

8.1.11 Autoevaluación

- [ ] Sé elegir entre List, Set y Map según la necesidad
- [ ] Utilizo ArrayList, HashSet y HashMap correctamente
- [ ] Recorro colecciones con iteradores y for-each
- [ ] Entiendo y aplico expresiones lambda
- [ ] Utilizo Streams para filtrar, mapear y reducir
- [ ] Implemento Comparable y Comparator

8.1.12 Vídeos recomendados

Canal	Vídeo	Contenido
EducaMadrid	Colecciones_1	ArrayList, List, LinkedList, diferencias con arrays
Programación ATS	Playlist POO Java	Vídeos sobre colecciones, ArrayList, HashMap
Píldoras Informáticas	Curso Java — Colecciones	Vídeos 80–90: colecciones y programación genérica
MoureDev	Curso Completo de Java desde Cero	Sección de colecciones

28 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

8.2 Ejercicios de la UD07

8.2.1 Ejercicios

1. (`package Varios`) Diseñar la clase `Varios` con los siguientes métodos **estáticos** que se harán apoyándose en alguna clase de las vistas al estudiar las colecciones de Java:
   - `int[] quitarDuplicados (int[] v)`, que dado un array de enteros devuelva otro array con los mismos valores que el original pero sin duplicados.
   - `int[] union1(int[] v1, int[] v2)`, que dados dos arrays v1 y v2 devuelva otro array con los elementos que están en v1 o que están en v2, sin que ningún elemento se repita.
   - `int[] union2(int v1[], int v2[])`, que dados dos arrays v1 y v2 devuelva otro array con los elementos que están en v1 o que están en v2. En este caso, si hay elementos duplicados se mantendrán.
   - `int[] interseccion(int v1[], int v2[])`, que dados dos arrays v1 y v2 devuelva otro array con los elementos que aparecen en los dos arrays. Cada elemento común aparecerá una sola vez en el resultado.
   - `int[] diferencia1 (int v1[], int v2[])`, que dados dos arrays v1 y v2 devuelva otro array con los elementos de v1 que no están en v2. En caso de haber elementos duplicados en v1 estos se mantendrán en el resultado.
   - `int[] diferencia2 (int v1[], int v2[])`, que dados dos arrays v1 y v2 devuelva otro array con los elementos de v1 que no están en v2. El array resultante no tendrá elementos duplicados.
2. (`package Biblioteca`) Se quiere hacer una aplicación en la que los usuarios van a hacer búsquedas de libros para saber si se encuentran en los fondos de la biblioteca. El funcionamiento básico sería algo así: Al iniciarse la aplicación todo el catálogo de libros se cargaría en memoria y a partir de ese momento los usuarios pueden realizar búsquedas por título, que interesa que sean lo más rápidas posibles. Nunca se insertan nuevos libros durante la ejecución de la aplicación.
   - Diseña la clase `Libro` con los métodos que consideres oportunos y los siguientes atributos:
     - `Titulo` (String): Es el dato que identifica al libro.
     - `Autor` (String): Autor del libro.
     - `Estantería` (String): Estantería de la biblioteca en la que se encuentra el libro.

     Con el constructor `public Libro(String titulo, String autor, String estanteria)` y los consultores `getTitulo()`, `getAutor()` y `getEstanteria()`.

     Si guardas los libros en un `HashSet`, o los usas como clave de un `HashMap`, revisa en los apuntes el aviso sobre `equals` y `hashCode`.
   - Diseña la clase `CatalogoLibros` como una colección de libros (llamada `catalogo`). Utiliza el tipo de colección que crees que más se ajusta a los requisitos de la aplicación justificando la elección. Implementa los siguientes métodos:
   - `public CatalogoLibros(Libro v[])`: Constructor. Para simplificar, inicializa el catálogo y lo rellena con los libros del `array v`, en lugar de obtenerlos de un fichero.
   - `public String buscar(Libro l)`: Dado un libro, lo busca en el Catálogo y devuelve la estantería en la que se encuentra el libro o null si el libro no está en el Catálogo.
3. (`package Academia`) Se quiere diseñar una clase `Academia`. De una Academia se conoce su nombre, dirección y las Aulas que tiene (Necesitas también generar la clase `Aula`).

   Definir la clase `Academia` utilizando una **Collection** para almacenar las aulas. El tipo de colección a utilizar se decidirá teniendo en cuenta que éstas se quieren mantener ordenadas según el criterio del método `compareTo` de la clase `Aula`. Implementar los atributos, el constructor, y los siguientes métodos:
   - `void ampliar (Aula a)`, que añade un aula a la academia.
   - `void quitar (Aula a)`, que elimina un aula de la academia.
   - `int getNumAulas()`, que devuelva el número de aulas que tiene.
   - `toString()`, que muestre todas las aulas de la academia.
4. (`paquete ListaEspera`) Deseamos mantener la lista de espera de pacientes de un hospital.

   Se quiere poder:
   - Añadir pacientes a la lista.
   - Mantener los pacientes por orden de inserción en la lista de espera.
   - Obtener el paciente más prioritario (al que hay que atender de la lista de espera), es decir, el que más tiempo lleva esperando, el que antes entró en el hospital.

   Pasos a seguir:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

a. Diseña la clase `Paciente` con los atributos nombre y gravedad. La gravedad es un valor aleatorio (entre 1: más grave y 5: menos grave) generado al crear el paciente.

b. Realiza la implementación de la clase `ListaEspera` con la estructura de datos elegida. Añade los métodos para insertar un paciente, obtener de la lista de espera al más prioritario y eliminar de la lista de espera al más prioritario. ¿Qué tipo de estructura de datos utilizaríamos para almacenar los pacientes, un `List`, `Set` o `Map`? Justifica la respuesta (puntos a favor de la que elijes y en contra de las que descartas).

c. Implementa una `ListaEsperaPorGravedad`, en la que se atienda primero a los pacientes más graves, independientemente de si llegaron antes o después.

5. (**`paquete DiccionarioIngEsp`**)

   Diseñar la clase `DiccionarioBilingüe` para almacenar pares formados por:

   a. Palabra en castellano.

   b. Colección de traducciones a ingles.

   La clase dispondrá de los siguientes métodos:

   - Constructor: crea el diccionario vacio.
   - `anyadirTraduccion(String cast, String ingl)` : Añade la pareja (cast, ingl) al diccionario de forma que:
     - Si la palabra `cast` no estaba en el diccionario la añade, junto con su traducción.
     - Si la palabra `cast` estaba ya en el diccionario pero no aparecía como traducción la palabra `ingl`, añade `ingl` a su colección de traducciones.
     - Si la palabra `cast` estaba y la traducción `ingl` también, no se realizarán cambios.
     - El método devuelve `true` si se han realizado cambios en el diccionario y `false` en caso contrario.
   - `QuitarTraduccion(String cast, String ingl)` : Quita la traducción `ingl` a la palabra `cast`. Si la palabra en castellano se queda sin traducciones, se elimina del dicicionario. Si se han producido cambios se devuelve `true` y en caso contrario `false`.
   - `traduccionesDe(String cast)` : Devuelve una colección con las traducciones de la palabra indicada o null si la palabra no está en el diccionario.
   - `toString()` : Devuelve un String con las palabras del diccionario y sus traducciones.
   - (clase `RepeticionPalabras`) Escribe un programa que abra el fichero de texto que indique el usuario y muestre cuántas veces se repite cada palabra que contiene. Ayudarse de un `Map`. ¿Se podría resolver con un `Set`? ¿Y con un `List`?
   - (clase `TraductorSimple`) Escribir un programa que solicite al usuario una frase y la muestre traducida a inglés, palabra a palabra. Para ello, se dispone de un fichero `palabras.txt` que contiene parejas (palabra en español, palabra en inglés) separadas por un tabulador. Cada pareja se encuentra en una línea del fichero. El proceso será el siguiente:
   - Leer el fichero y cargar sus datos en una estructura de datos adecuada. Tener en cuenta que nos interesará buscar una palabra en castellano y obtener su correspondencia en inglés.
   - Solicitar al usuario una frase. Traducir, usando la estructura de datos anterior, cada palabra de la frase y formar con ellas la frase traducida.
   - Mostrar la frase traducida al usuario.

6. (**`paquete ListaAdmitidos`**) Una serie de personas han solicitado realizar un curso de inglés. De las que han sido admitidas se quiere almacenar su nif, su nombre y su nivel en un `HashSet`. En el `HashSet` se almacenarán objetos de la clase Inscripción

   - Implementa la clase `Incripción` para representar el nif, nombre y nivel de un solicitante. Además de los atributos implementa aquellos métodos que consideres necesarios.
   - Escribe un programa (clase `ComprobarAdmision`) que
     - Defina un `HashSet` de Inscripciones, llamado `admitidas`.
     - Añada varias inscripciones (invéntate los datos).
     - Permita al usuario introducir un dni para comprobar si la persona indicada ha sido admitida. Indicarle si aparece o no en la lista y, en caso afirmativo, mostrar el nombre y el nivel en que ha sido admitido.

7. (**`clase PalabrasOrdenadas`**) Escribe un programa que, dado un fichero de texto cuya ubicación indica el usuario, muestre sus palabras ordenadas ascendentemente y, después, descendentemente. Cada palabra se mostrará una sola vez, aunque en el texto aparezca varias.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

8.2.2 Actividades

1. **Actividad 1**. Realizar las siguientes actividades relacionadas con `ArrayList`.
   - Crear un `ArrayList` de enteros llamado misNumeros.
   - Añadir los valores 1, 6, 3, 2, 0, 4, 5.
   - Mostrar los datos del `ArrayList`.
   - Mostrar el valor de la posición 5.
   - Añadir el valor 8 en la posición 4.
   - Cambiar el valor de la posición 1 por 9.
   - Eliminar el valor 5. (`misNumeros.remove(new Integer(5))`)
   - Eliminar el valor de la posición 3.
   - Recorrer el array con un bucle for.
   - Recorrer el array con un bucle `Iterator`.
   - Comprobar si existe el elemento 0.
   - Comprobar si existe el elemento 7.
   - Clonar el `ArrayList misNumeros` en otro llamado `copiaArrayList`.
   - Añadir el elemento 9.
   - Mostrar la posición de la primera ocurrencia del elemento 9.
   - Mostrar la posición de la última ocurrencia del elemento 9.
   - Borrar todos los elementos del `ArrayList copiaArrayList`.
   - Comprobar si el `ArrayList copiaArrayList` está vacio.
   - Convertir el `ArrayList misNumeros` en un `Array` y recorrerlo con un bucle mejorado.

   Muestra por pantalla el resultado de cada paso (los `ArrayList` con `System.out.println` y las comprobaciones como `true` / `false`).
2. **Actividad 2**. Un cine precisa una aplicación para controlar las personas de la cola para los estrenos de películas. Debemos crear una lista con la edad de las personas de la cola y tendremos que calcular la entrada según la edad de la persona (mínimo 5 años). Para la edad de la persona se generan aleatoriamente números entre 5 y 60 años. Al final, deberemos mostrar la cantidad total recaudada. El número de personas de la cola se elige al azar entre 0 y 50.

   La lista de precios se basa en la siguiente tabla.

EDAD	PRECIO
Entre 5 y 10 años	5 €
Entre 11 y 17 años	7.5 €
18 años o más	9.5 €

   Como comprobación imprime el número de personas, el precio total y la lista de edades. Por ejemplo:

   ```
   Hay un total de 6 personas en la cola.
   El precio total es de 57,00 euros
   [18, 36, 50, 35, 28, 55]
   ```

3. **Actividad 3**. Un supermercado nos pide que hagamos una aplicación que almacene los productos comprados. La aplicación debe almacenar `Productos` (clase `Actividad3Producto`) y cada producto al crearse contiene una `cantidad`, un `precio` (generados aleatoriamente). El nombre del producto será básico (producto1, producto2, producto3, etc.). Calcular el precio total de una lista de entre 1 y 10 productos (aleatorio). Mostrar un ticket con todo lo vendido y el precio final. Para numerar los nombres, `Actividad3Producto` usa un atributo de clase `static int contadorProducto`, y el precio de cada línea lo calcula el método `public double getPrecioLinea()` (cantidad × precio).

   ```
   Producto1   3   47,95   143,84
   Producto2   7   84,37   590,62
   Producto3   7   33,43   234,04
   Producto4   2   95,42   190,84
   Producto5   10  53,50   534,96
   Producto6   3   26,21   78,62
   El precio final del ticket es de 1772,92 euros
   ```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

4. **Actividad 4**. Desarrollar un sistema de gestión de pacientes. Tendremos un archivador dónde iremos guardando todas las fichas de los pacientes. Las fichas (`Actividad4Paciente`) contienen la siguiente información: `nombre` , `apellidos` y `edad` (constructor `public Actividad4Paciente(String nombre, String apellidos, int edad)`). El listado se hace con un método `public static void listado(Collection<Actividad4Paciente> c)` . La clase `Actividad4` tiene un método `main` en el que se crea un archivador, dos o tres fichas que se guardarán en el archivador, se listará el contenido, se recorrerá el archivador borrando todas las fichas que cumplan dos condiciones (nombre "David" o menor de 10 años, por ejemplo) y por último se eliminará alguna ficha (directamente) y se volverá a listar su contenido.

```
LISTADO DE PACIENTES
====================
{nombre=David, apellidos=Martinez Peña, edad=35}
{nombre=Elena, apellidos=Garcia Perez, edad=15}
{nombre=Marc, apellidos=Redondo Perez, edad=5}
{nombre=Pepe, apellidos=Perez Peña, edad=16}

LISTADO DE PACIENTES
====================
{nombre=Elena, apellidos=Garcia Perez, edad=15}
{nombre=Pepe, apellidos=Perez Peña, edad=16}

LISTADO DE PACIENTES
====================
{nombre=Elena, apellidos=Garcia Perez, edad=15}
```

5. **Actividad 5**. Crear una estructura `Map` llamada divisas, que almacene pares de moneda y valor al cambio en euros. Por ejemplo Dólar: 0,81€
   - Añadir los siguientes pares moneda/valor al Map divisas:

Moneda	Valor en €
Dólar Americano	0.81
Franco Suizo	0.85
Libra Esterlina	1.14
Corona Danesa	0.13
Peso Mexicano	0.04
Dólar Singapur	0.62
Real Brasil	0.24

   - Mostrar el valor de la Libra Esterlina.
   - Mostrar todas las divisas con las que se opera y su valor.
   - Indicar el número de divisas del Map.
   - Eliminar la divisa Real Brasil y mostrar los datos del Map.
   - Mostrar si existe la divisa Peso Mexicano.
   - Mostrar si existe la divisa Euro.
   - Mostrar si existe el valor al cambio 0.85 €.
   - Mostrar si existe el valor al cambio 0.33 €.
   - Indicar si el Map divisas está vacío.
   - Borra todos los componentes del Map divisas.
   - Volver a indicar si el Map divisas está vacío.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
1.14
Franco Suizo --> 0,85
Corona Danesa --> 0,13
Peso Mexicano --> 0,04
Dólar Americano --> 0,81
Libra Esterlina --> 1,14
Real Brasil --> 0,24
Dólar Singapur --> 0,62
7
Franco Suizo --> 0,85
Corona Danesa --> 0,13
Peso Mexicano --> 0,04
Dólar Americano --> 0,81
Libra Esterlina --> 1,14
Dólar Singapur --> 0,62
SI existe Peso Mexicano
NO existe Euro €
SI existe el valor 0.85
NO existe el valor 0.33
NO está vacio
Está vacio
```

8.2.3 Ejercicios Genericidad

1. (Genericos) Crear una clase Genericos (proyecto Genéricos, paquete Genericos) que incorpore los métodos genéricos que se indican a continuación. Los métodos creados serán *public static*. En el proyecto se creará además la clase o clases necesarias para probar los métodos desarrollados.
   a. `Object minimo (Object o1, Object o2)`, que devuelva el mínimo de dos objetos cualesquiera (que se suponen del mismo tipo). Una vez desarrollado, prueba el método para obtener el mínimo de dos objetos Integer. Pruébalo también para obtener el mínimo entre un Objeto Integer y un objeto String. En éste último caso, el programa ¿da error de ejecución? Si es así, explica por qué.
   b. `Object maximo (Object o1, Object o2)`, que devuelva el maximo de dos objetos cualesquiera (que se suponen del mismo tipo).
   c. `Object minimo (Object v[])`, que devuelva el mínimo de un array de objetos cualesquiera (que se suponen del mismo tipo). Al respecto de éste último comentario, ¿Se puede poner en un array de Object objetos de distinto tipo, como por ejemplo Strings, Integer, ...? En caso afirmativo, ¿funcionaría el método desarrollado con un array construido así?
   d. `Object maximo (Object v[])`, que devuelva el maximo de un array de objetos cualesquiera (que se suponen del mismo tipo).
   e. `int numVeces(Object v[], Object x)` que devuelva el el numero de apariciones del objeto x en el array v.
   f. `int numVecesOrdenado(Object v[], Object x)` que devuelva el el numero de apariciones del objeto x en el array v **ordenado ascendentemente**.
   g. `int mayores(Object v[], Object x)` que, dado un array de Object v y un Object x devuelva el número de elementos de v que son mayores que x.
   h. `int mayoresOrdenado(Object v[], Object x)` que, dado un array de Object v **ordenado ascendentemente** y un Object x devuelva el número de elementos de v que son mayores que x.
   i. `int menores(Object v[], Object x)` que, dado un array de Object v y un Object x devuelva el número de elementos de v que son menores que x.
   j. `int menoresOrdenado(Object v[], Object x)` que, dado un array de Object v **ordenado ascendentemente** y un Object x devuelva el número de elementos de v que son menores que x.
   k. `boolean estaEn(Object v[], Object x)` que devuelva true si el Objeto x está en el array v.
   l. `boolean estaEnOrdenado(Object v[], Object x)` que devuelva true si el Objeto x está en el array v, **ordenado ascendentemente**.
   m. `int posiciónDe(Object v[], Object x)`, que devuelva la posición que ocupa x dentro del array v, o -1 si x no está en v.
   n. `int posicionDeOrdenado(Object v[], Object x)`, que devuelva la posición que ocupa x dentro del array v **ordenado ascendentemente**, o -1 si x no está en v.
   o. `boolean estaOrdenado(Object v[])`, que devuelva true si el array está ordenado ascendentemente.
2. (**`paquete Nevera`**) Se quiere crear una aplicación que controla una nevera inteligente de última generación. Los alimentos que contiene la nevera se van a representar como objetos de la clase `Alimento` y la clase `NeveraInteligente`, tiene un array de Alimentos entre sus atributos privados.

   Se pide implementar la clase **Alimento** teniendo en cuenta que uno de los métodos de NeveraInteligente necesitará ordenar **por calorías** los Alimentos que contiene la nevera utilizando un método de Ordenación genérico. El diseño de la clase Alimento ha de incluir, por tanto, determinados elementos que lo permitan. La clase Alimento tendrá únicamente dos atributos (privados): nombre y calorias.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

3. (`paquete Academia`) Se quiere diseñar una clase *Academia* . De una Academia se conoce su nombre, dirección y las *Aulas* que tiene (*Aula* es una clase implementada en un ejercicio de herencia que ya hicimos).
   a. Definir la clase `Academia` utilizando una colección (que permita ordenación) para almacenar las aulas.: Implementar los atributos, el constructor, y los siguientes métodos: - `void ampliar (Aula a)` , que añade un aula a la academia. - `void quitar (Aula a)` , que elimina un aula de la academia. - `int getNumAulas()` , que devuelva el número de aulas que tiene. - método `toString()`
   b. Realiza en la clase `Aula` los cambios necesarios para que se pueda ordenar las aulas de la Academia usando un método genérico de ordenación. El orden sería creciente por capacidad del aula., y a igual capacidad primero las aulas de mayor superficie
   c. Añade a la clase `Academia` un método `ordenar` que ordene las aulas con el criterio especificado. Para realizar la ordenación se llamará a un método de ordenación.
4. (`paquete Conjuntos`)
   a. Diseñar un **interface *Conjunto*** para modelizar conjuntos de elementos. Diseñar (solo la cabecera) de los siguientes métodos de la clase conjunto (prestar atención a si los métodos deben ser *static* o no):
      - `Añadir` , que añade un elemento al conjunto, provocando la excepción `ElementoDuplicado` si el elemento ya estaba en el conjunto.
      - `Quitar` , que elimina el elemento indicado al conjunto. Provoca `ElementoNoEncontrado` si el elemento indicado no estaba en el conjunto.
      - `Intersección` ¸ que dados dos conjuntos que recibe como parámetro devuelve un tercer conjunto que es la intersección de los dos dados.
      - `Pertenece` ¸ que dado un elemento devuelve si este pertenece o no al conjunto.
   b. Diseñar una clase `ConjuntoArray` que implemente el interface `Conjunto` . Esta clase implementará los métodos del interface `Conjunto` . Para ello utilizará un array `Object elementos[]` y un `int numElementos` , de manera que los elementos del conjunto se mantendrán almacenados en el array. Además de los métodos del interface habrá que crear un constructor para la clase y también vendrá bien tener un método `toString` para poder probarla.

   NOTA: También lo puedes implementar con una Colección de las vistas en el tema anterior.

8.2.4 Programación funcional. Funciones Lambda.

1. (Libros) Supongamos que tenemos implementada una clase `Libro` que tiene los atributos de `titulo` , `autor` y `precio` , con el correspondiente constructor y sus getters. Disponemos además de una lista de libros en una variable `libros` . Se pide implementar un método que muestre los títulos de los libros por pantalla, ordenados de menor a mayor.

> **Importante**
>
> El constructor es `Libro(String titulo, String autor, double precio)` . Para pasar los tests, la clase `TestLibro` tiene que usar esta lista de libros:
>
> ```
> libros.add(new Libro("El juego de Ender", "Orson Scott Card", 8.95));
> libros.add(new Libro("La tabla de Flandes", "Arturo Pérez Reverte", 7.90));
> libros.add(new Libro("La historia interminable", "Michael Ende", 6.50));
> libros.add(new Libro("El señor de los anillos", "J.R.R. Tolkien", 14.25));
> ```

2. (Recetas) Disponemos de una clase llamada `Receta` que almacena los datos de una receta de cocina: su nombre, su categoría (carne, pasta...) y sus calorías, incluyendo su constructor `Receta(String nombre, String categoria, int calorias)` y getters. Supongamos que tenemos una lista de recetas llamada `recetas` . Se pide implementar las siguientes consultas en Java:
   - Recetas de menos de 500 calorías
   - Nombres de las recetas de "carne", ordenadas alfabéticamente
   - Media de calorías de las recetas de "verduras"
   - Cuántas recetas hay de más de 800 calorías

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

> **Importante**
>
> Para pasar satisfactoriamente los tests necesitaras que la colección de recetas contenga estas recetas:

```
recetas.add(new Receta("Ensalada de aguacate", "verduras", 250));
recetas.add(new Receta("Guisado de pollo", "carne", 530));
recetas.add(new Receta("Espaguetis boloñesa", "pasta", 480));
recetas.add(new Receta("Ratatouille", "verduras", 280));
recetas.add(new Receta("Cochinillo asado", "carne", 870));
recetas.add(new Receta("Ensalada de pasta", "pasta", 325));
```

3. (Numeros) Usando los streams de Java, que están muy relacionados con las expresiones lambda, debes crear una lista de 1000 números enteros aleatorios entre -5000 y 5000 y a partir de dicha lista vamos a imprimir por pantalla:
   - El máximo de los números pares.
   - El mínimo de los números múltiplos de 3.
   - El total de números negativos.
   - El total de números primos.
   - El máximo número primo (recuerda que el 0 y el 1 no son primos).

   Crea la lista de tres formas distintas, en `listaNumeros`, `listaNumerosv2` y `listaNumerosv3`, y muestra las tres completas. Los cálculos se hacen sobre una de ellas.

> **Importante**
>
> Para pasar los tests debes generar una salida similar a esta:

```
listaNumeros: [-579, -2887, 1443, -4055, 2871, -1416, 4804, 2668, -4048, 1834, -3107, 4901, 4716, -1410, ...
listaNumerosv2: [3120, -1877, 402, ...
listaNumerosv3: [-4410, 2765, -38, ...
Máximo de los números pares: 4934
Mínimo de los múltiplos de tres:-4992
Total de números negativos: 513
Listado de números primos: [3989, 2797, 4271, 1627, 131, 3517, 2609, 971, 4283, 643, 2789, 337, 191, 3301, 2383, ...
Total de números primos: 69
Máximo número primo: 4999
```

4. (GranjaDAM) Paquete `GranjaDAM`, crea una clase `Animal` con los siguientes atributos, constructor con todos los atributos, getters y setters:
   - String especie
   - String genero
   - String raza
   - String alimentacion
   - float peso
   - String color
   - boolean tienePelo

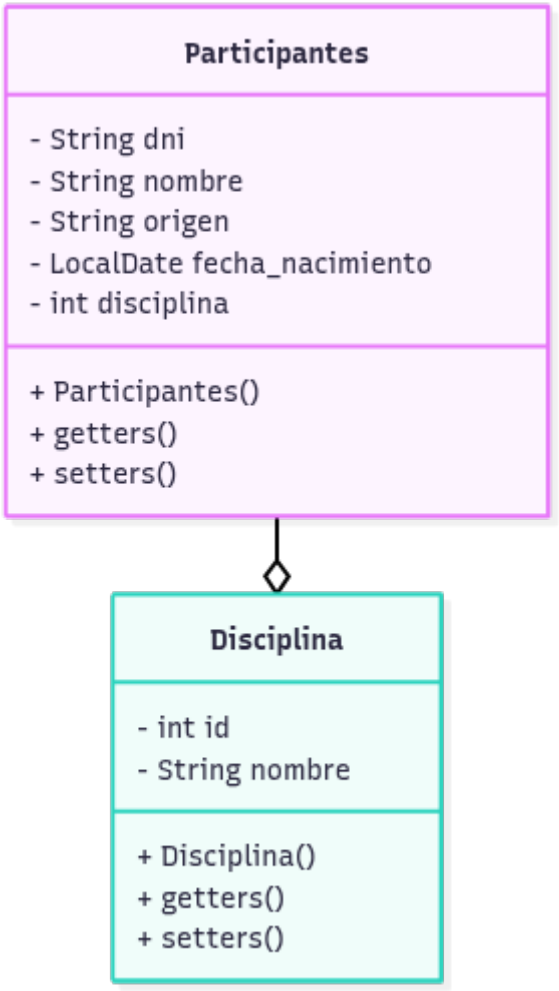
   Crea una clase `Granja` la cual contendrá un `List<Animal>` con los animales que posee. Crea también los métodos siguientes:
   - `void nuevoAnimal (Animal a)`
   - `int contarAnimalesDeRaza(String raza)`
   - `Map<String, Integer> contarAnimalesPorGenero()`
   - `List<String> especiesConPeso (float pesoMin, float pesoMax)`
   - `Map<String, List<Animal>> agruparPorAlimentacion ()`
   - `DoubleSummaryStatistics resumenPesoConPelo (boolean tienePelo)`
5. (Clase) Paquete `Clase`. Crear una clase `Alumno` con los atributos típicos de un alumno (nombre, fecha de nacimiento, dni, nia, teléfono, email, nota) y los correspondientes getters y setters (y constructor).

   Crea una clase `Aula` la cual contendrá un `List<Alumno>` con los alumnos que van a un aula determinada. Crea también los métodos siguientes:
   - `int contarAlumnosMayoresQue (int edad)`
   - `float edadMedia ()`
   - `List<String> nombreSuspendidos ()`

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- `void listadoPorNiaDescendente ()`
- `boolean encontarAlumnosDelMismoMesQue (Alumno a)`

6. (Olimpiada) Paquete `Olimpiada`. A partir de las siguientes clases...



se pide:

- Obtener la lista de todos los participantes junto con el nombre de la disciplina en la que participarán (los participantes de una misma disciplina se tienen que mostrar juntos).
- Obtener una lista de todas las disciplinas que no tengan participantes.
- Calcular la edad media de todos los participantes agrupada por disciplinas.
- Obtener las disciplinas con los participantes más joven y más viejo.

28 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

8.3 Talleres

8.3.1 Taller UD07_01: Classroom 50

Aceptar la tarea en Classroom 50

Aceptamos el *Assignement* (la tarea/ejercicio) a partir del link del profesor, en este caso: https://classroom50.org/IES-Eduardo-Primo-Marques/1dam-programacio-java/assignments/ud07

Si no recuerdas cómo aceptar la tarea, clonar el repositorio o abrirlo en IntelliJ, repasa la guía completa en el taller de UD02.

Tarea

Debes enviar tus soluciones a Classroom 50 y superar al menos la mitad de los tests, cuantos más tests superados, mejor nota tendrás en la tarea.

6 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

9. UD08

9.1 Composición, Herencia y Polimorfismo

Al finalizar esta unidad serás capaz de...

- Distinguir entre composición y herencia
- Crear jerarquías de clases mediante herencia
- Sobrescribir métodos de la superclase
- Aplicar el polimorfismo en el diseño de clases
- Utilizar clases abstractas y métodos abstractos
- Definir e implementar interfaces
- Diferenciar entre interfaz y clase abstracta
- Realizar conversiones entre objetos (casting)

9.1.1 Relaciones entre clases.

Cuando estudiaste el concepto de clase, ésta fue descrita como una especie de mecanismo de definición (plantillas), en el que se basaría el entorno de ejecución a la hora de construir un objeto: un mecanismo de definición de objetos.

Por tanto, a la hora de diseñar un conjunto de clases para modelar el conjunto de información cuyo tratamiento se desea automatizar, es importante establecer apropiadamente las posibles relaciones que puedan existir entre unas clases y otras.

En algunos casos es posible que no exista relación alguna entre unas clases y otras, pero lo más habitual es que sí exista: una clase puede ser una especialización (Relación entre dos clases donde una de ellas (la subclase) es una versión más especializada que la otra (la superclase), compartiendo características en común pero añadiendo ciertas características específicas que la especializan. El punto de vista inverso sería la generalización) de otra, o bien una generalización (Relación entre dos clases donde una de ellas (la superclase) es una versión más genérica que la otra (la subclase), compartiendo características en común pero sin las propiedades específicas que caracterizan a la subclase. El punto de vista inverso sería la especialización), o una clase

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

contiene en su interior objetos de otra, o una clase utiliza a otra, etc. Es decir, que entre unas clases y otras habrá que definir cuál es su relación (si es que existe alguna).

Se pueden distinguir diversos tipos de relaciones entre clases:

- **Clientela**. Cuando una clase utiliza objetos de otra clase (por ejemplo al pasarlos como parámetros a través de un método).
- **Composición**. Cuando alguno de los atributos de una clase es un objeto de otra clase.
- **Anidamiento**. Cuando se definen clases en el interior de otra clase.
- **Herencia**. Cuando una clase comparte determinadas características con otra (clase base), añadiéndole alguna funcionalidad específica (especialización).

La relación de **clientela** la llevas utilizando desde que has empezado a programar en Java, pues desde tu clase principal (clase con método `main`) has estado declarando, creando y utilizando objetos de otras clases. Por ejemplo: si utilizas un objeto `String` dentro de la clase principal de tu programa, éste será cliente de la clase `String` (como sucederá con prácticamente cualquier programa que se escriba en Java). Es la relación fundamental y más habitual entre clases (la utilización de unas clases por parte de otras) y, por supuesto, la que más vas a utilizar tú también, de hecho, ya la has estado utilizando y lo seguirás haciendo.

La relación de **composición** es posible que ya la hayas tenido en cuenta si has definido clases que contenían (tenían como atributos) otros objetos en su interior, lo cual es bastante habitual. Por ejemplo, si escribes una clase donde alguno de sus atributos es un objeto de tipo `String` , ya se está produciendo una relación de tipo composición (tu clase "tiene" un `String` , es decir, está compuesta por un objeto `String` y por algunos elementos más).

La relación de **anidamiento** (o anidación) es quizá menos habitual, pues implica declarar unas clases dentro de otras (clases internas o anidadas). En algunos casos puede resultar útil para tener un nivel más de encapsulamiento (ocultamiento del estado de un objeto, de sus datos miembro o atributos) de manera que sólo se puede cambiar mediante las operaciones (métodos) definidas para ese objeto. Cada objeto está aislado del exterior de manera que se protegen los datos contra su modificación por quien no tenga derecho a acceder a ellos, eliminando efectos secundarios y colaterales no deseados. Este modo de proceder permite que el usuario de una clase pueda obviar la implementación de los métodos y propiedades para concentrarse sólo en cómo usarlos. Por otro lado se evita que el usuario pueda cambiar su estado de manera imprevista e incontrolada) y ocultación de información (efecto que se consigue gracias a la encapsulación: se evita la visibilidad de determinados miembros de una clase al resto del código del programa para de ese modo comunicarse con los objetos de la clase únicamente a través de su interfaz/ métodos).

En el caso de la relación de **herencia** también la has visto ya, pues seguro que has utilizado unas clases que derivaban de otras, sobre todo, en el caso de los objetos que forman parte de las interfaces gráficas. Lo más probable es que hayas tenido que declarar clases que derivaban de algún componente gráfico (`JFrame` , `JDialog` , etc.).

Podría decirse que tanto la composición como la anidación son casos particulares de clientela, pues en realidad en todos esos casos una clase está haciendo uso de otra (al contener atributos que son objetos de la otra clase, al definir clases dentro de otras clases, al utilizar objetos en el paso de parámetros, al declarar variables locales utilizando otras clases, etc.).

A lo largo de la unidad, irás viendo distintas posibilidades de implementación de clases haciendo uso de todas estas relaciones, centrándonos especialmente en el caso de la herencia, que es la que permite establecer las relaciones más complejas.

Composición

Cuando en un sistema de información, una determinada entidad `A` contiene a otra `B` como una de sus partes, se suele decir que se está produciendo una relación de composición. Es decir, el objeto de la clase `A` contiene a uno o varios objetos de la clase `B` .

Por ejemplo, si describes una entidad `País` compuesta por una serie de atributos, entre los cuales se encuentra una lista de comunidades autónomas, podrías decir que los objetos de la clase `País` contienen varios objetos de la clase `ComunidadAutonoma` . Por otro lado, los objetos de la clase `ComunidadAutonoma` podrían contener como atributos objetos de la clase `Provincia` , la cual a su vez también podría contener objetos de la clase `Municipio` .

Como puedes observar, la composición puede encadenarse todas las veces que sea necesario hasta llegar a objetos básicos del lenguaje o hasta tipos primitivos que ya no contendrán otros objetos en su interior. Ésta es la forma más habitual de definir clases: mediante otras clases ya definidas anteriormente. Es una manera eficiente y sencilla de gestionar la reutilización de todo el código ya escrito. Si se definen clases que describen entidades distinguibles y con funciones claramente definidas, podrán utilizarse cada vez que haya que representar objetos similares dentro de otras clases.

Definición

La composición se da cuando una clase contiene algún atributo que es una referencia a un objeto de otra clase.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Una forma sencilla de plantearte si la relación que existe entre dos clases A y B es de composición podría ser mediante la expresión idiomática "tiene un": "la clase A tiene uno o varios objetos de la clase B", o visto de otro modo: "Objetos de la clase B pueden formar parte de la clase A".

Algunos ejemplos de composición podrían ser:

- Un coche tiene un motor y tiene cuatro ruedas.
- Una persona tiene un nombre, una fecha de nacimiento, una cuenta bancaria asociada para ingresar la nómina, etc.
- Un cocodrilo bajo investigación científica que tiene un número de dientes determinado, una edad, unas coordenadas de ubicación geográfica (medidas con GPS), etc.

Recuperando algunos de los ejemplos de clases que has utilizado en otras unidades:

- Una clase `Rectangulo` podría contener en su interior dos objetos de la clase `Punto` para almacenar los vértices inferior izquierdo y superior derecho.
- Una clase `Empleado` podría contener en su interior un objeto de la clase `DNI` para almacenar su DNI/NIF, y otro objeto de la clase `CuentaBancaria` para guardar la cuenta en la que se realizan los ingresos en nómina.

¿Podría decirse que la relación que existe entre la clase `Ave` y la clase `Loro` es una relación de composición?

No. Aunque claramente existe algún tipo de relación entre ambas, no parece que sea la de composición. No parece que se cumpla la expresión "tiene un": "Un loro tiene un ave". Se cumpliría más bien una expresión del tipo "es un": "Un loro es un ave". Algunos objetos que cumplirían la relación de composición podrían ser `Pico` o `Alas`, pues "un loro tiene un pico y dos alas", del mismo modo que "un ave tiene pico y dos alas". Este tipo de relación parece más de herencia (un loro es un tipo de ave).

Herencia

El mecanismo que permite crear clases basándose en otras que ya existen es conocido como herencia. Como ya has visto en unidades anteriores, Java implementa la herencia mediante la utilización de la palabra reservada extends.

El concepto de herencia es algo bastante simple y sin embargo muy potente: cuando se desea definir una nueva clase y ya existen clases que, de alguna manera, implementan parte de la funcionalidad que se necesita, es posible crear una nueva clase derivada de la que ya tienes. Al hacer esto se posibilita la reutilización de todos los atributos y métodos de la clase que se ha utilizado como base (clase padre o superclase), sin la necesidad de tener que escribirlos de nuevo.

Una subclase hereda todos los miembros de su clase padre (atributos, métodos y clases internas). Los constructores no se heredan, aunque se pueden invocar desde la subclase.

Algunos ejemplos de herencia podrían ser:

- Un coche es un vehículo (heredará atributos como la velocidad máxima o métodos como parar y arrancar).
- Un empleado es una persona (heredará atributos como el nombre o la fecha de nacimiento).
- Un rectángulo es una figura geométrica en el plano (heredará métodos como el cálculo de la superficie o de su perímetro).
- Un cocodrilo es un reptil (heredará atributos como por ejemplo el número de dientes).

En este caso la expresión idiomática que puedes usar para plantearte si el tipo de relación entre dos clases A y B es de herencia podría ser "es un": "la clase A es un tipo específico de la clase B" (especialización), o visto de otro modo: "la clase B es un caso general de la clase A" (generalización).

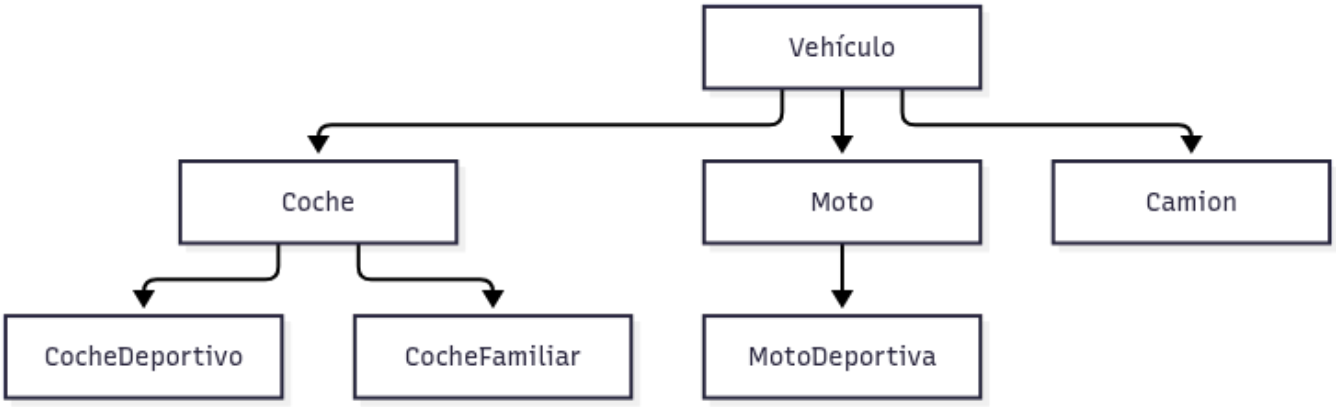
Recuperando algunos ejemplos de clases que ya has utilizado en otras unidades:

- Una ventana en una aplicación gráfica puede ser una clase que herede de `JFrame` (componente `Swing`: `javax.swing.JFrame`), de esta manera esa clase será un marco que dispondrá de todos los métodos y atributos de `JFrame` mas aquéllos que tú decidas incorporarle al rellenarlo de componentes gráficos.
- Una caja de diálogo puede ser un tipo de `JDialog` (otro componente `Swing`: `javax.swing.JDialog`).

En Java, la clase `Object` (dentro del paquete `java.lang`) define e implementa el comportamiento común a todas las clases (incluidas aquellas que tú escribas). Como recordarás, ya se dijo que en Java cualquier clase deriva en última instancia de la clase `Object`.

Todas las clases tienen una clase padre, que a su vez también posee una superclase, y así sucesivamente hasta llegar a la clase `Object`. De esta manera, se construye lo que habitualmente se conoce como una jerarquía de clases, que en el caso de Java tendría a la clase `Object` en la raíz.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Atención

Cuando escribas una clase en Java, puedes hacer que herede de una determinada clase padre (mediante el uso de `extends`) o bien no indicar ninguna herencia. En tal caso, aunque no indiques explícitamente ningún tipo de herencia, el compilador asumirá entonces de manera implícita que tu clase hereda de la clase `Object`, que define e implementa el comportamiento común a todas las clases.

¿Herencia o composición?

Cuando escribas tus propias clases, debes intentar tener claro en qué casos utilizar la composición y cuándo la herencia:

- **Composición**: cuando una clase está formada por objetos de otras clases. En estos casos se incluyen objetos de esas clases, pero no necesariamente se comparten características con ellos (no se heredan características de esos objetos, sino que directamente se utilizarán sus atributos y sus métodos). Esos objetos incluidos no son más que atributos miembros de la clase que se está definiendo.
- **Herencia**: cuando una clase cumple todas las características de otra. En estos casos la clase derivada es una especialización (o particularización, extensión o restricción) de la clase base. Desde otro punto de vista se diría que la clase base es una generalización de las clases derivadas.

Por ejemplo, imagina que dispones de una clase Punto (ya la has utilizado en otras ocasiones) y decides definir una nueva clase llamada Círculo. Dado que un punto tiene como atributos sus coordenadas en plano (x1, y1), decides que es buena idea aprovechar esa información e incorporarla en la clase Circulo que estás escribiendo. Para ello utilizas la herencia, de manera que al derivar la clase Círculo de la clase Punto, tendrás disponibles los atributos x1 e y1. Ahora solo faltaría añadirle algunos atributos y métodos más como por ejemplo el radio del círculo, el cálculo de su área y su perímetro, etc.

En principio parece que la idea pueda funcionar pero es posible que más adelante, si continúas construyendo una jerarquía de clases, observes que puedas llegar a conclusiones incongruentes al suponer que un círculo es una especialización de un punto (un tipo de punto). ¿Todas aquellas figuras que contengan uno o varios puntos deberían ser tipos de punto? ¿Y si tienes varios puntos? ¿Cómo accedes a ellos? ¿Un rectángulo también tiene sentido que herede de un punto? No parece muy buena idea.

Parece que en este caso habría resultado mejor establecer una relación de composición. Analízalo detenidamente: ¿cuál de estas dos situaciones te suena mejor? 1. "Un círculo es un punto (su centro)", y por tanto heredará las coordenadas x1 e y1 que tiene todo punto. Además tendrá otras características específicas como el radio o métodos como el cálculo de la longitud de su perímetro o de su área. 2. "Un círculo tiene un punto (su centro)", junto con algunos atributos más como por ejemplo el radio. También tendrá métodos para el cálculo de su área o de la longitud de su perímetro.

Parece que en este caso la composición refleja con mayor fidelidad la relación que existe entre ambas clases. Normalmente suele ser suficiente con plantearse las preguntas "¿A es un tipo de B?" o "¿A contiene elementos de tipo B?".

9.1.2 Composición

Sintaxis de la composición.

Para indicar que una clase contiene objetos de otra clase no es necesaria ninguna sintaxis especial. Cada uno de esos objetos no es más que un atributo y, por tanto, debe ser declarado como tal:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
class <nombreClase> {
    [modificadores] <NombreClase1> nombreAtributo1;
    [modificadores] <NombreClase2> nombreAtributo2;
    <NombreClase3>[] listado;
    ...
}
```

En unidades anteriores has trabajado con la clase `Punto`, que definía las coordenadas de un punto en el plano, y con la clase `Rectangulo`, que definía una figura de tipo rectángulo también en el plano a partir de dos de sus vértices (inferior izquierdo y superior derecho). Tal y como hemos formalizado ahora los tipos de relaciones entre clases, parece bastante claro que aquí tendrías un caso de composición: "un rectángulo contiene puntos". Por tanto, podrías ahora redefinir los atributos de la clase `Rectangulo` (cuatro números reales) como dos objetos de tipo Punto:

```
class Rectangulo {
    private Punto vertice1;
    private Punto vertice2;
    ...
}
```

Ahora los métodos de esta clase deberán tener en cuenta que ya no hay cuatro atributos de tipo `double`, sino dos atributos de tipo `Punto` (cada uno de los cuales contendrá en su interior dos atributos de tipo `double`).

Revisa el Ejemplo01 para ver composición de clases en `Rectangulo`.

Uso de la composición (I). Preservación de la ocultación.

Como ya has observado, la relación de composición no tiene más misterio a la hora de implementarse que simplemente declarar atributos de las clases que necesites dentro de la clase que estés diseñando.

Ahora bien, cuando escribas clases que contienen objetos de otras clases (lo cual será lo más habitual) deberás tener un poco de precaución con aquellos métodos que devuelvan información acerca de los atributos de la clase (métodos "consultores" o de tipo get).

Como ya viste en la unidad dedicada a la creación de clases, lo normal suele ser declarar los atributos como privados (o protegidos, como veremos un poco más adelante) para ocultarlos a los posibles clientes de la clase (otros objetos que en el futuro harán uso de la clase). Para que otros objetos puedan acceder a la información contenida en los atributos, o al menos a una parte de ella, deberán hacerlo a través de métodos que sirvan de interfaz, de manera que sólo se podrá tener acceso a aquella información que el creador de la clase haya considerado oportuna. Del mismo modo, los atributos solamente serán modificados desde los métodos de la clase, que decidirán cómo y bajo qué circunstancias deben realizarse esas modificaciones. Con esa metodología de acceso se tenía perfectamente separada la parte de manipulación interna de los atributos de la interfaz con el exterior.

Hasta ahora los métodos de tipo get devolvían tipos primitivos, es decir, copias del contenido (a veces con algún tipo de modificación o de formato) que había almacenado en los atributos, pero los atributos seguían "a salvo" como elementos privados de la clase. Pero, a partir de este momento, al tener objetos dentro de las clases y no sólo tipos primitivos, es posible que en un determinado momento interese devolver un objeto completo.

Ahora bien, cuando vayas a devolver un objeto habrás de obrar con mucha precaución. Si en un método de la clase devuelves directamente un objeto que es un atributo, estarás ofreciendo directamente una referencia a un objeto atributo que probablemente has definido como privado. ¡De esta forma estás volviendo a hacer público un atributo que inicialmente era privado!

Para evitar ese tipo de situaciones (ofrecer al exterior referencias a objetos privados) puedes optar por diversas alternativas, procurando siempre evitar la devolución directa de un atributo que sea un objeto:

- Una opción podría ser devolver siempre tipos primitivos.
- Dado que esto no siempre es posible, o como mínimo poco práctico, otra posibilidad es crear un nuevo objeto que sea una copia del atributo que quieres devolver y utilizar ese objeto como valor de retorno. Es decir, crear una copia del objeto especialmente para devolverlo. De esta manera, el código cliente de ese método podrá manipular a su antojo ese nuevo objeto, pues no será una referencia al atributo original, sino un nuevo objeto con el mismo contenido.

Por último, debes tener en cuenta que es posible que en algunos casos sí se necesite realmente la referencia al atributo original (algo muy habitual en el caso de atributos estáticos). En tales casos, no habrá problema en devolver directamente el atributo para que el código llamante (cliente) haga el uso que estime oportuno de él.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Devolución de objetos

Debes evitar por todos los medios la devolución de un atributo que sea un objeto (estarías dando directamente una referencia al atributo, visible y manipulable desde fuera), salvo que se trate de un caso en el que deba ser así.

Para entender estas situaciones un poco mejor, podemos volver a la clase `Rectangulo` y observar sus nuevos métodos de tipo get.

Revisa el Ejemplo02 para ver métodos get con composición.

Uso de la composición (II). Llamadas a constructores.

Otro factor que debes considerar, a la hora de escribir clases que contengan como atributos objetos de otras clases, es su comportamiento a la hora de instanciarse. Durante el proceso de creación de un objeto (constructor) de la clase contenedora habrá que tener en cuenta también la creación (llamadas a constructores) de aquellos objetos que son contenidos.

Atención

El constructor de la clase contenedora debe invocar a los constructores de las clases de los objetos contenidos.

En este caso hay que tener cuidado con las referencias a objetos que se pasan como parámetros para rellenar el contenido de los atributos. Es conveniente hacer una copia de esos objetos y utilizar esas copias para los atributos pues si se utiliza la referencia que se ha pasado como parámetro, el código cliente de la clase podría tener acceso a ella sin necesidad de pasar por la interfaz de la clase (volveríamos a dejar abierta una puerta pública a algo que quizá sea privado).

Además, si el objeto parámetro que se pasó al constructor formaba parte de otro objeto, esto podría ocasionar un desagradable efecto colateral si esos objetos son modificados en el futuro desde el código cliente de la clase, ya que no sabes de dónde provienen esos objetos, si fueron creados especialmente para ser usados por el nuevo objeto creado o si pertenecen a otro objeto que podría modificarlos más tarde. Es decir, correrías el riesgo de estar "compartiendo" esos objetos con otras partes del código, sin ningún tipo de control de acceso y con las nefastas consecuencias que eso podría tener: cualquier cambio de ese objeto afectaría a partes del programa supuestamente independientes, que entienden ese objeto como suyo.

Atención

En el fondo los objetos no son más que variables de tipo referencia a la zona de memoria en la que se encuentra toda la información del objeto en sí mismo. Esto es, puedes tener un único objeto y múltiples referencias a él. Pero sólo se trata de un objeto, y cualquier modificación desde una de sus referencias afectaría a todas las demás, pues estamos hablando del mismo objeto.

Recuerda también que sólo se crean objetos cuando se llama a un constructor (uso de new). Si realizas asignaciones o pasos de parámetros, no se están copiando o pasando copias de los objetos, sino simplemente de las referencias, y por tanto se tratará siempre del mismo objeto.

Se trata de un efecto similar al que sucedía en los métodos de tipo get, pero en este caso en sentido contrario (en lugar de que nuestra clase "regale" al exterior uno de sus atributos objeto mediante una referencia, en esta ocasión se "adueña" de un parámetro objeto que probablemente pertenezca a otro objeto y que es posible que el futuro haga uso de él).

Para entender mejor estos posibles efectos podemos continuar con el ejemplo de la clase `Rectangulo` que contiene en su interior dos objetos de la clase `Punto`. En los constructores del rectángulo habrá que incluir todo lo necesario para crear dos instancias de la clase `Punto` evitando las referencias a parámetros (haciendo copias).

Revisa el Ejemplo03 para ver constructores con composición.

Clases anidadas o internas.

En algunos lenguajes, es posible definir una clase dentro de otra clase (clases internas):

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
class ClaseContenedora {
    // Cuerpo de la clase
    ...
    class ClaseInterna {
    // Cuerpo de la clase interna
    ...
    }
}
```

Se pueden distinguir varios tipos de clases internas:

- Clases internas estáticas (o clases anidadas), declaradas con el modificador `static`.
- Clases internas miembro, conocidas habitualmente como clases internas. Declaradas al máximo nivel de la clase contenedora y no estáticas.
- Clases internas locales, que se declaran en el interior de un bloque de código (normalmente dentro de un método).
- Clases anónimas, similares a las internas locales, pero sin nombre (sólo existirá un objeto de ellas y, al no tener nombre, no tendrán constructores). Se suelen usar en la gestión de eventos en los interfaces gráficos.
- Aquí tienes algunos ejemplos:

```
class ClaseContenedora {
 ...
 static class ClaseAnidadaEstatica {
     ...
 }
 class ClaseInterna {
    ...
    }
}
```

Las clases anidadas, como miembros de una clase que son (miembros de `ClaseContenedora`), pueden ser declaradas con los modificadores `public`, `protected`, `private` o de `paquete`, como el resto de miembros.

Las clases internas (no estáticas) tienen acceso a otros miembros de la clase dentro de la que está definida aunque sean privados (se trata en cierto modo de un miembro más de la clase), mientras que las anidadas (estáticas) no.

Las clases internas se utilizan en algunos casos para:

- Agrupar clases que sólo tiene sentido que existan en el entorno de la clase en la que han sido definidas, de manera que se oculta su existencia al resto del código.
- Incrementar el nivel de encapsulación y ocultamiento.
- Proporcionar un código fuente más legible y fácil de mantener (el código de las clases internas y anidadas está más cerca de donde es usado).

En Java es posible definir clases internas y anidadas, permitiendo todas esas posibilidades. Aunque para lo ejemplos con los que vas a trabajar no las vas a necesitar por ahora.

9.1.3 Herencia

Como ya has estudiado, la herencia es el mecanismo que permite definir una nueva clase a partir de otra, pudiendo añadir nuevas características, sin tener que volver a escribir todo el código de la clase base.

La clase de la que se hereda suele ser llamada clase base, clase padre o superclase (de la que hereda otra clase. Se heredarán todas aquellas características que la clase padre permita). A la clase que hereda se le suele llamar clase hija, clase derivada o subclase (que hereda de otra clase. Se heredan todas aquellas características que la clase padre permita).

Una clase derivada puede ser a su vez clase padre de otra que herede de ella y así sucesivamente dando lugar a una jerarquía de clases, excepto aquellas que estén en la parte de arriba de la jerarquía (sólo serán clases padre) o en la parte de abajo (sólo serán clases hijas).

Una clase hija no tiene acceso a los miembros privados de su clase padre, tan solo a los públicos (como cualquier parte del código tendría) y los protegidos (a los que sólo tienen acceso las clases derivadas y las del mismo paquete). Aquellos miembros que sean privados en la clase base también habrán sido heredados, pero el acceso a ellos estará restringido al propio funcionamiento de la superclase y sólo se podrá acceder a ellos si la superclase ha dejado algún medio indirecto para hacerlo (por ejemplo a través de algún método).

Todos los miembros de la superclase, tanto atributos como métodos, son heredados por la subclase. Algunos de estos miembros heredados podrán ser redefinidos o sobrescritos (overriden) y también podrán añadirse nuevos miembros. De alguna manera podría decirse que estás "ampliando" la clase base con características adicionales o modificando algunas de ellas (proceso de especialización).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

> **Recuerda**
>
> Una clase derivada extiende la funcionalidad de la clase base sin tener que volver a escribir el código de la clase base.

Sintaxis de la herencia.

En Java la herencia se indica mediante la palabra reservada `extends`:

```
[modificador] class ClasePadre {
    // Cuerpo de la clase
    ...
}

[modificador] class ClaseHija extends ClasePadre {
    // Cuerpo de la clase
    ...
}
```

Imagina que tienes una clase `Persona` que contiene atributos como `nombre`, `apellidos` y `fecha de nacimiento`:

```
public class Persona {
    String nombre;
    String apellidos;
    LocalDate fechaNacim;
    ...
}
```

Es posible que, más adelante, necesites una clase `Alumno` que compartirá esos atributos (dado que todo alumno es una persona, pero con algunas características específicas que lo especializan). En tal caso tendrías la posibilidad de crear una clase `Alumno` que repitiera todos esos atributos o bien heredar de la clase `Persona`:

```
public class Alumno extends Persona {
    String grupo;
    double notaMedia;
    ...
}
```

A partir de ahora, un objeto de la clase `Alumno` contendrá los atributos `grupo` y `notaMedia` (propios de la clase `Alumno`), pero también `nombre`, `apellidos` y `fechaNacim` (propios de su clase base `Persona` y que por tanto ha heredado).

Revisa el Ejemplo04 para ver herencia con la clase `Profesor`.

Acceso a miembros heredados.

Como ya has visto anteriormente, no es posible acceder a miembros privados de una superclase. Para poder acceder a ellos podrías pensar en hacerlos públicos, pero entonces estarías dando la opción de acceder a ellos a cualquier objeto externo y es probable que tampoco sea eso lo deseable. Para ello se inventó el modificador protected (protegido) que permite el acceso desde clases heredadas, pero no desde fuera de las clases (estrictamente hablando, desde fuera del paquete), que serían como miembros privados.

En la unidad dedicada a la utilización de clases ya estudiaste los posibles modificadores de acceso que podía tener un miembro: sin modificador (acceso de paquete), público, privado o protegido.

Aquí tienes de nuevo el resumen:

modificador	Misma clase	Mismo paquete	Subclase	Otro paquete
`public`	✔	✔	✔	✔
`protected`	✔	✔	✔	❌
Sin modificador (`package`)	✔	✔	❌	❌
`private`	✔	❌	❌	❌

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Recuerda

¡Recuerda que **los modificadores de acceso son excluyentes**! Sólo se puede utilizar uno de ellos en la declaración de un atributo.

Si en el ejemplo anterior de la clase `Persona` se hubieran definido sus atributos como private:

```
public class Persona {
    private String nombre;
    private String apellidos;
    ...
}
```

Al definir la clase `Alumno` como heredera de `Persona`, no habrías tenido acceso a esos atributos, pudiendo ocasionar un grave problema de operatividad al intentar manipular esa información. Por tanto, en estos casos lo más recomendable habría sido declarar esos atributos como `protected` o bien sin modificador (para que también tengan acceso a ellos otras clases del mismo paquete, si es que se considera oportuno):

```
public class Persona {
    protected String nombre;
    protected String apellidos;
    ...
}
```

private vs protected

Sólo en aquellos casos en los que se desea explícitamente que un miembro de una clase no pueda ser accesible desde una clase derivada debería utilizarse el modificador `private`. En el resto de casos es recomendable utilizar `protected`, o bien no indicar modificador (acceso a nivel de `paquete`).

Revisa el Ejemplo05 para ver el modificador `protected`.

Utilización de miembros heredados (I). Atributos.

Los atributos heredados por una clase son, a efectos prácticos, iguales que aquellos que sean definidos específicamente en la nueva clase derivada.

En el ejemplo anterior la clase `Persona` disponía de tres atributos y la clase `Alumno`, que heredaba de ella, añadía dos atributos más. Desde un punto de vista funcional podrías considerar que la clase `Alumno` tiene cinco atributos: tres por ser `Persona` (nombre, apellidos, fecha de nacimiento) y otros dos más por ser `Alumno` (grupo y nota media).

Revisa el Ejemplo06 para ver getters y setters en herencia.

Utilización de miembros heredados (II). Métodos.

Del mismo modo que se heredan los atributos, también se heredan los métodos, convirtiéndose a partir de ese momento en otros métodos más de la clase derivada, junto a los que hayan sido definidos específicamente.

En el ejemplo de la clase `Persona`, si dispusiéramos de métodos get y set para cada uno de sus tres atributos (`nombre`, `apellidos`, `fechaNacim`), tendrías seis métodos que podrían ser heredados por sus clases derivadas. Podrías decir entonces que la clase Alumno, derivada de Persona, tiene diez métodos:

- Seis por ser Persona (`getNombre`, `getApellidos`, `getFechaNacim`, `setNombre`, `setApellidos`, `setFechaNacim`).
- Oros cuatro más por ser Alumno (`getGrupo`, `setGrupo`, `getNotaMedia`, `setNotaMedia`).

Sin embargo, sólo tendrías que definir esos cuatro últimos (los específicos) pues los genéricos ya los has heredado de la superclase.

Revisa el Ejemplo07 para ver getters y setters heredados.

Redefinición de métodos heredados.

Una clase puede redefinir algunos de los métodos que ha heredado de su clase base. En tal caso, el nuevo método (especializado) sustituye al heredado. Este procedimiento también es conocido como de sobrescritura de métodos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

En cualquier caso, aunque un método sea sobrescrito o redefinido, aún es posible acceder a él a través de la referencia super, aunque sólo se podrá acceder a métodos de la clase padre y no a métodos de clases superiores en la jerarquía de herencia.

Los métodos redefinidos pueden ampliar su accesibilidad con respecto a la que ofrezca el método original de la superclase, pero nunca restringirla. Por ejemplo, si un método es declarado como protected o de paquete en la clase base, podría ser redefinido como public en una clase derivada. Los métodos estáticos o de clase no pueden ser sobrescritos. Los originales de la clase base permanecen inalterables a través de toda la jerarquía de herencia.

En el ejemplo de la clase Alumno, podrían redefinirse algunos de los métodos heredados. Por ejemplo, imagina que el método getApellidos devuelva la cadena "Alumno:" junto con los apellidos del alumno. En tal caso habría que rescribir ese método para realizara esa modificación:

```
public String getApellidos () {
    return "Alumno: " + apellidos;
}
```

Cuando sobrescribas un método heredado en Java puedes incluir la anotación `@Override`. Esto indicará al compilador que tu intención es sobrescribir el método de la clase padre. De este modo, si te equivocas (por ejemplo, al escribir el nombre del método) y no lo estás realmente sobrescribiendo, el compilador producirá un error y así podrás darte cuenta del fallo. En cualquier caso, no es necesario indicar `@Override`, pero puede resultar de ayuda a la hora de localizar este tipo de errores (crees que has sobrescrito un método heredado y al confundirte en una letra estás realmente creando un nuevo método diferente). En el caso del ejemplo anterior quedaría:

```
@Override
public String getApellidos ()
```

Revisa el Ejemplo08 para ver redefinición de métodos.

Ampliación de métodos heredados.

Hasta ahora, has visto que para redefinir o sustituir un método de una superclase es suficiente con crear otro método en la subclase que tenga el mismo nombre que el método que se desea sobrescribir. Pero, en otras ocasiones, puede que lo que necesites no sea sustituir completamente el comportamiento del método de la superclase, sino simplemente ampliarlo.

Para poder hacer esto necesitas poder preservar el comportamiento antiguo (el de la superclase) y añadir el nuevo (el de la subclase). Para ello, puedes invocar desde el método "ampliador" de la clase derivada al método "ampliado" de la clase superior (teniendo ambos métodos el mismo nombre). ¿Cómo se puede conseguir eso? Puedes hacerlo mediante el uso de la referencia super.

La palabra reservada super es una referencia a la clase padre de la clase en la que te encuentres en cada momento (es algo similar a this, que representaba una referencia a la clase actual). De esta manera, podrías invocar a cualquier método de tu superclase (si es que se tiene acceso a él).

Por ejemplo, imagina que la clase Persona dispone de un método que permite mostrar el contenido de algunos datos personales de los objetos de este tipo (nombre, apellidos, etc.). Por otro lado, la clase Alumno también necesita un método similar, pero que muestre también su información especializada (grupo, nota media, etc.). ¿Cómo podrías aprovechar el método de la superclase para no tener que volver a escribir su contenido en la subclase?

Podría hacerse de una manera tan sencilla como la siguiente:

Este tipo de ampliaciones de métodos resultan especialmente útiles por ejemplo en el caso de los constructores, donde se podría ir llamando a los constructores de cada superclase encadenadamente hasta el constructor de la clase en la cúspide de la jerarquía (el constructor de la clase `Object`).

Revisa el Ejemplo09 para ver ampliación de métodos con `super`.

Constructores y herencia.

Recuerda que cuando estudiaste los constructores viste que un constructor de una clase puede llamar a otro constructor de la misma clase, previamente definido, a través de la referencia this. En estos casos, la utilización de this sólo podía hacerse en la primera línea de código del constructor.

Como ya has visto, un constructor de una clase derivada puede hacer algo parecido para llamar al constructor de su clase base mediante el uso de la palabra super. De esta manera, el constructor de una clase derivada puede llamar primero al constructor de su superclase para que inicialice los atributos heredados y posteriormente se inicializarán los atributos específicos de la clase:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

los no heredados. Nuevamente, esta llamada también debe ser la primera sentencia de un constructor (con la única excepción de que exista una llamada a otro constructor de la clase mediante this).

Si no se incluye una llamada a `super()` dentro del constructor, el compilador incluye automáticamente una llamada al constructor por defecto de clase base (llamada a `super()`). Esto da lugar a una llamada en cadena de constructores de superclase hasta llegar a la clase más alta de la jerarquía (que en Java es la clase `Object`).

En el caso del constructor por defecto (el que crea el compilador si el programador no ha escrito ninguno), el compilador añade lo primero de todo, antes de la inicialización de los atributos a sus valores por defecto, una llamada al constructor de la clase base mediante la referencia super.

A la hora de destruir un objeto (método `finalize`) es importante llamar a los finalizadores en el orden inverso a como fueron llamados los constructores (primero se liberan los recursos de la clase derivada y después los de la clase base mediante la llamada `super.finalize()`).

Si la clase Persona tuviera un constructor de este tipo:

```
public Persona (String nombre, String apellidos, LocalDate fechaNacim) {
    this.nombe= nombre;
    this.apellidos= apellidos;
    this.fechaNacim= new LocalDate (fechaNacim);
}
```

Podrías llamarlo desde un constructor de una clase derivada (por ejemplo Alumno) de la siguiente forma:

```
public Alumno (String nombre, String apellidos, LocalDate fechaNacim, String grupo, double notaMedia) {
    super (nombre, apellidos, fechaNacim);
    this.grupo= grupo;
    this.notaMedia= notaMedia;
}
```

En realidad se trata de otro recurso más para optimizar la reutilización de código, en este caso el del constructor, que aunque no es heredado, sí puedes invocarlo para no tener que reescribirlo.

Revisa el Ejemplo10 para ver constructores y llamada a `super`.

Creación y utilización de clases derivadas.

Ya has visto cómo crear una clase derivada, cómo acceder a los miembros heredados de las clases superiores, cómo redefinir algunos de ellos e incluso cómo invocar a un constructor de la superclase. Ahora se trata de poner en práctica todo lo que has aprendido para que puedas crear tus propias jerarquías de clases, o basarte en clases que ya existan en Java para heredar de ellas, y las utilices de manera adecuada para que tus aplicaciones sean más fáciles de escribir y mantener.

> **Herencia = menos código**
>
> La idea de la herencia no es complicar los programas, sino todo lo contrario: simplificarlos al máximo. Procurar que haya que escribir la menor cantidad posible de código repetitivo e intentar facilitar en lo posible la realización de cambios (bien para corregir errores bien para incrementar la funcionalidad).

La clase `Object` en Java.

Todas las clases en Java son descendentes (directos o indirectos) de la clase Object. Esta clase define los estados y comportamientos básicos que deben tener todos los objetos. Entre estos comportamientos, se encuentran:

- La posibilidad de compararse.
- La capacidad de convertirse a cadenas.
- La habilidad de devolver la clase del objeto.

Entre los métodos que incorpora la clase `Object` y que por tanto hereda cualquier clase en Java tienes:

Principales métodos de la clase `Object`:

Método	Descripción
`Object()`	Constructor.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Método	Descripción
`clone()`	Método clonador: crea y devuelve una copia del objeto ("clona" el objeto).
`boolean equals(Object obj)`	Indica si el objeto pasado como parámetro es igual a este objeto.
`void finalize()`	Método llamado por el recolector de basura cuando éste considera que no queda ninguna referencia a este objeto en el entorno de ejecución.
`int hashCode()`	Devuelve un código hash para el objeto.
`toString()`	Devuelve una representación del objeto en forma de String.

La clase `Object` representa la superclase que se encuentra en la cúspide de la jerarquía de herencia en Java. Cualquier clase (incluso las que tú implementes) acaban heredando de ella.

Herencia múltiple.

En determinados casos podrías considerar la posibilidad de que se necesite heredar de más de una clase, para así disponer de los miembros de dos (o más) clases disjuntas (que no derivan una de la otra). La herencia múltiple permite hacer eso: recoger las distintas características (atributos y métodos) de clases diferentes formando una nueva clase derivada de varias clases base.

El problema en estos casos es la posibilidad que existe de que se produzcan ambigüedades, así, si tuviéramos miembros con el mismo identificador en clases base diferentes, en tal caso, ¿qué miembro se hereda? Para evitar esto, los compiladores suelen solicitar que ante casos de ambigüedad, se especifique de manera explícita la clase de la cual se quiere utilizar un determinado miembro que pueda ser ambiguo.

Ahora bien, la posibilidad de herencia múltiple no está disponible en todos los lenguajes orientados a objetos, ¿lo estará en Java? La respuesta es **negativa**.

Herencia múltiple

En Java **no existe** la herencia múltiple de clases.

9.1.4 Clases Abstractas

En determinadas ocasiones, es posible que necesites definir una clase que represente un concepto lo suficientemente abstracto como para que nunca vayan a existir instancias de ella (objetos). ¿Tendría eso sentido? ¿Qué utilidad podría tener?

Imagina una aplicación para un centro educativo que utilice las clases de ejemplo Alumno y Profesor, ambas subclases de Persona. Es más que probable que esa aplicación nunca llegue a necesitar objetos de la clase Persona, pues serían demasiado genéricos como para poder ser utilizados (no contendrían suficiente información específica). Podrías llegar entonces a la conclusión de que la clase Persona ha resultado de utilidad como clase base para construir otras clases que hereden de ella, pero no como una clase instanciable de la cual vayan a existir objetos. A este tipo de clases se les llama clases abstractas.

Clases abstractas

En algunos casos puede resultar útil disponer de clases que nunca serán instanciadas, sino que proporcionan un marco o modelo a seguir por sus clases derivadas dentro de una jerarquía de herencia. Son las clases abstractas.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

La posibilidad de declarar clases abstractas es una de las características más útiles de los lenguajes orientados a objetos, pues permiten dar unas líneas generales de cómo es una clase sin tener que implementar todos sus métodos o implementando solamente algunos de ellos. Esto resulta especialmente útil cuando las distintas clases derivadas deban proporcionar los mismos métodos indicados en la clase base abstracta, pero su implementación sea específica para cada subclase.

Imagina que estás trabajando en un entorno de manipulación de objetos gráficos y necesitas trabajar con líneas, círculos, rectángulos, etc. Estos objetos tendrán en común algunos atributos que representen su estado (ubicación, color del contorno, color de relleno, etc.) y algunos métodos que modelen su comportamiento (dibujar, rellenar con un color, escalar, desplazar, rotar, etc.). Algunos de ellos serán comunes para todos ellos (por ejemplo la ubicación o el desplazamiento) y sin embargo otros (como por ejemplo dibujar) necesitarán una implementación específica dependiendo del tipo de objeto. Pero, en cualquier caso, todos ellos necesitan esos métodos (tanto un círculo como un rectángulo necesitan el método dibujar, aunque se lleven a cabo de manera diferente). En este caso resultaría muy útil disponer de una clase abstracta objeto gráfico donde se definirían las líneas generales (algunos atributos concretos comunes, algunos métodos concretos comunes implementados y algunos métodos genéricos comunes sin implementar) de un objeto gráfico y más adelante, según se vayan definiendo clases especializadas (líneas, círculos, rectángulos), se irán concretando en cada subclase aquellos métodos que se dejaron sin implementar en la clase abstracta.

Declaración de una clase abstracta.

Ya has visto que una clase abstracta es una clase que no se puede instanciar, es decir, que no se pueden crear objetos a partir de ella. La idea es permitir que otras clases deriven de ella, proporcionando un modelo genérico y algunos métodos de utilidad general. Las clases abstractas se declaran mediante el modificador `abstract`:

```
[modificador_acceso] abstract class nombreClase [herencia] [interfaces] {
    ...
}
```

> **Métodos abstractos**
>
> Una clase puede contener en su interior métodos declarados como `abstract` (métodos para los cuales sólo se indica la cabecera, pero no se proporciona su implementación). En tal caso, la clase tendrá que ser necesariamente también `abstract`. Esos métodos tendrán que ser posteriormente implementados en sus clases derivadas.

Por otro lado, una clase también puede contener métodos totalmente implementados (no abstractos), los cuales serán heredados por sus clases derivadas y podrán ser utilizados sin necesidad de definirlos (pues ya están implementados).

Cuando trabajes con clases abstractas debes tener en cuenta:

- Una clase abstracta sólo puede usarse para crear nuevas clases derivadas. No se puede hacer un new de una clase abstracta. Se produciría un error de compilación.
- Una clase abstracta puede contener métodos totalmente definidos (no abstractos) y métodos sin definir (métodos abstractos).

Revisa el Ejemplo11 para ver clases abstractas.

Métodos abstractos.

Un método abstracto es un método declarado en una clase para el cual esa clase no proporciona la implementación. Si una clase dispone de al menos un método abstracto se dice que es una clase abstracta. Toda clase que herede (sea subclase) de una clase abstracta debe implementar todos los métodos abstractos de su superclase o bien volverlos a declarar como abstractos (y por tanto también sería abstracta). Para declarar un método abstracto en Java se utiliza el modificador `abstract`) es un método cuya implementación no se define, sino que se declara únicamente su interfaz (cabecera) para que su cuerpo sea implementado más adelante en una clase derivada.

Un método se declara como abstracto mediante el uso del modificador `abstract` (como en las clases abstractas):

```
[modificador_acceso] abstract <tipo> <nombreMetodo> ([parámetros]) [excepciones];
```

Estos métodos tendrán que ser obligatoriamente redefinidos (en realidad "definidos", pues aún no tienen contenido) en las clases derivadas. Si en una clase derivada se deja algún método abstracto sin implementar, esa clase derivada será también una clase abstracta.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Clase abstracta?

Cuando una clase contiene un método abstracto tiene que declararse como abstracta obligatoriamente.

Imagina que tienes una clase `Empleado` genérica para diversos tipos de empleado y tres clases derivadas: `EmpleadoFijo` (tiene un salario fijo más ciertos complementos), `EmpleadoTemporal` (salario fijo más otros complementos diferentes) y `EmpleadoComercial` (una parte de salario fijo y unas comisiones por cada operación). La clase `Empleado` podría contener un método abstracto `calcularNomina`, pues sabes que se método será necesario para cualquier tipo de empleado (todo empleado cobra una nómina). Sin embargo el cálculo en sí de la nómina será diferente si se trata de un empleado fijo, un empleado temporal o un empleado comercial, y será dentro de las clases especializadas de `Empleado` (`EmpleadoFijo`, `EmpleadoTemporal`, `EmpleadoComercial`) donde se implementen de manera específica el cálculo de las mismas.

Debes tener en cuenta al trabajar con métodos abstractos:

- Un método abstracto implica que la clase a la que pertenece tiene que ser abstracta, pero eso no significa que todos los métodos de esa clase tengan que ser abstractos.
- Un método abstracto no puede ser privado (no se podría implementar, dado que las clases derivadas no tendrían acceso a él).
- Los métodos abstractos no pueden ser estáticos, pues los métodos estáticos no pueden ser redefinidos (y los métodos abstractos necesitan ser redefinidos).

Revisa el Ejemplo12 para ver métodos abstractos.

Clases y métodos finales.

En unidades anteriores has visto el modificador `final`, aunque sólo lo has utilizado por ahora para atributos y variables (por ejemplo para declarar atributos constantes, que una vez que toman un valor ya no pueden ser modificados). Pero este modificador también puede ser utilizado con clases y con métodos (con un comportamiento que no es exactamente igual, aunque puede encontrarse cierta analogía: **no se permite heredar o no se permite redefinir**).

Una clase declarada como `final` no puede ser heredada, es decir, no puede tener clases derivadas. La jerarquía de clases a la que pertenece acaba en ella (no tendrá clases hijas):

```
[modificador_acceso] final class nombreClase [herencia] [interfaces]
```

Un método también puede ser declarado como `final`, en tal caso, ese método no podrá ser redefinido en una clase derivada:

```
[modificador_acceso] final <tipo> <nombreMetodo> ([parámetros]) [excepciones]
```

Si intentas redefinir un método `final` en una subclase se producirá un error de compilación.

Distintos contextos en los que puede aparecer el modificador `final`:

Lugar	Función
Como modificador de clase.	La clase no puede tener subclases.
Como modificador de atributo.	El atributo no podrá ser modificado una vez que tome un valor. Sirve para definir constantes.
Como modificador al declarar un método	El método no podrá ser redefinido en una clase derivada.
Como modificador al declarar una variable referencia.	Una vez que la variable tome un valor referencia (un objeto), no se podrá cambiar. La variable siempre apuntará al mismo objeto, lo cual no quiere decir que ese objeto no pueda ser modificado internamente a través de sus métodos. Pero la variable no podrá apuntar a otro objeto diferente.
Como modificador en un parámetro de un método	El valor del parámetro (ya sea un tipo primitivo o una referencia) no podrá modificarse dentro del código del método.

Veamos un ejemplo de cada posibilidad: 1. Modificador de una clase.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
public final class ClaseSinDescendencia { // Clase "no heredable"
 ...
}
```

1. Modificador de un atributo.

```
public class ClaseEjemplo {
    // Valor constante conocido en tiempo de compilación
    final double PI= 3.14159265;

    // Valor constante conocido solamente en tiempo de ejecución
    final int SEMILLA= (int) Math.random()*10+1;
    ...
}
```

1. Modificador de un método.

```
public final metodoNoRedefinible (int parametro1) { // Método "no redefinible"
 ...
}
```

1. Modificador en una variable referencia.

```
// Referencia constante: siempre se apuntará al mismo objeto Alumno recién creado, aunque este objeto pueda sufrir modificaciones.
final Alumno PRIMER_ALUMNO= new Alumno ("Pepe", "Torres", 9.55); // Ref. constante

// Si la variable no es una referencia (tipo primitivo), sería una constante más (como un atributo constante).
final int NUMERO_DIEZ= 10; // Valor constante (dentro del ámbito de vida de la variable)
```

1. Modificador en un parámetro de un método.

```
void metodoConParametrosFijos (final int par1, final int par2) {
 // Los parámetros "par1" y "par2" no podrán sufrir modificaciones aquí dentro
 ...
}
```

9.1.5 Interfaces

Has visto cómo la herencia permite definir especializaciones (o extensiones) de una clase base que ya existe sin tener que volver a repetir de todo el código de ésta. Este mecanismo da la oportunidad de que la nueva clase especializada (o extendida) disponga de toda la interfaz que tiene su clase base.

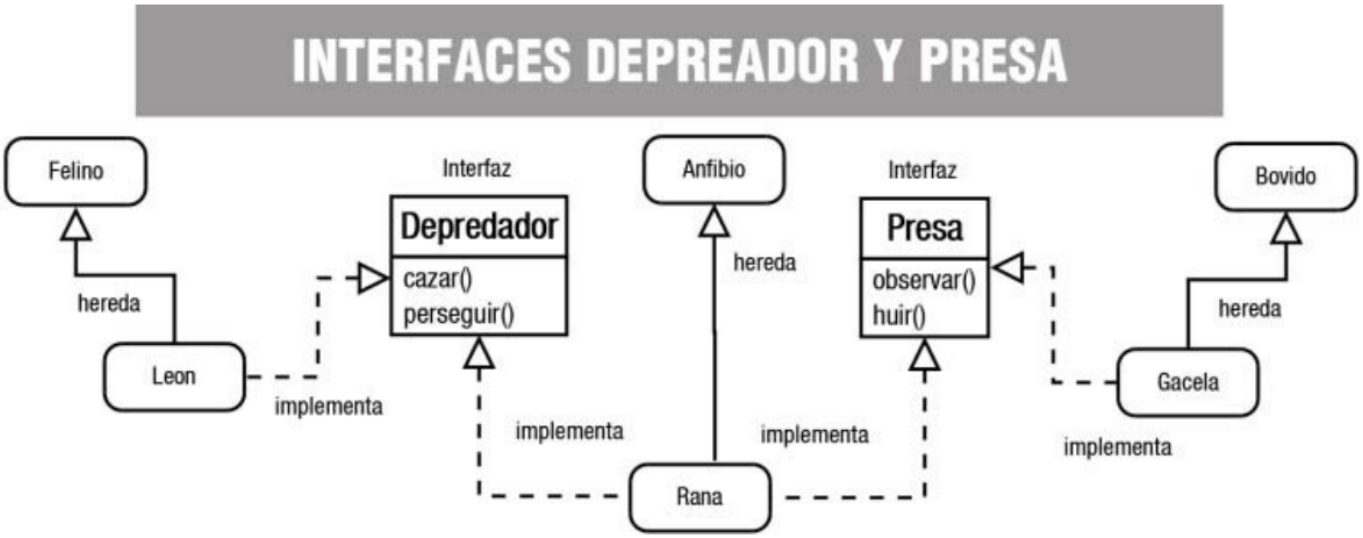
También has estudiado cómo los métodos abstractos permiten establecer una interfaz para marcar las líneas generales de un comportamiento común de superclase que deberían compartir de todas las subclases.

Si llevamos al límite esta idea de interfaz, podrías llegar a tener una clase abstracta donde todos sus métodos fueran abstractos. De este modo estarías dando únicamente el marco de comportamiento, sin ningún método implementado, de las posibles subclases que heredarán de esa clase abstracta. La idea de una interfaz (o interface) es precisamente ésa: disponer de un mecanismo que permita especificar cuál debe ser el comportamiento que deben tener todos los objetos que formen parte de una determinada clasificación (no necesariamente jerárquica).

Una interfaz consiste principalmente en una lista de declaraciones de métodos sin implementar, que caracterizan un determinado comportamiento. Si se desea que una clase tenga ese comportamiento, tendrá que implementar esos métodos establecidos en la interfaz. En este caso no se trata de una relación de herencia (la clase A es una especialización de la clase B, o la subclase A es del tipo de la superclase B), sino más bien una relación "de implementación de comportamientos" (la clase A implementa los métodos establecidos en la interfaz B, o los comportamientos indicados por B son llevados a cabo por A; pero no que A sea de clase B).

Imagina que estás diseñando una aplicación que trabaja con clases que representan distintos tipos de animales. Algunas de las acciones que quieres que lleven a cabo están relacionadas con el hecho de que algunos animales sean depredadores (por ejemplo: observar una presa, perseguirla, comérsela, etc.) o sean presas (observar, huir, esconderse, etc.). Si creas la clase `León`, esta clase podría implementar una interfaz `Depredador`, mientras que otras clases como `Gacela` implementarían las acciones de la interfaz `Presa`. Por otro lado, podrías tener también el caso de la clase `Rana`, que implementaría las acciones de la interfaz `Depredador` (pues es cazador de pequeños insectos), pero también la de `Presa` (pues puede ser cazado y necesita las acciones necesarias para protegerse).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Concepto de interfaz.

Una interfaz en Java consiste esencialmente en una lista de declaraciones de métodos sin implementar, junto con un conjunto de constantes.

Estos métodos sin implementar indican un comportamiento, un tipo de conducta, aunque no especifican cómo será ese comportamiento (implementación), pues eso dependerá de las características específicas de cada clase que decida implementar esa interfaz. Podría decirse que una interfaz se encarga de establecer qué comportamientos hay que tener (qué métodos), pero no dice nada de cómo deben llevarse a cabo esos comportamientos (implementación). Se indica sólo la forma, no la implementación.

En cierto modo podrías imaginar el concepto de interfaz como un guión que dice: "éste es el protocolo de comunicación que deben presentar todas las clases que implementen esta interfaz". Se proporciona una lista de métodos públicos y, si quieres dotar a tu clase de esa interfaz, tendrás que definir todos y cada uno de esos métodos públicos.

En conclusión: una interfaz se encarga de establecer unas líneas generales sobre los comportamientos (métodos) que deberían tener los objetos de toda clase que implemente esa interfaz, es decir, que no indican lo que el objeto es (de eso se encarga la clase y sus superclases), sino acciones (capacidades) que el objeto debería ser capaz de realizar. Es por esto que el nombre de muchas interfaces en Java termina con sufijos del tipo "-able", "-or", "-ente" y cosas del estilo, que significan algo así como capacidad o habilidad para hacer o ser receptores de algo (configurable, serializable, modificable, clonable, ejecutable, administrador, servidor, buscador, etc.), dando así la idea de que se tiene la capacidad de llevar a cabo el conjunto de acciones especificadas en la interfaz.

Imagínate por ejemplo la clase `Coche`, subclase de `Vehículo`. Los coches son vehículos a motor, lo cual implica una serie de acciones como, por ejemplo, arrancar el motor o detener el motor. Esa acción no la puedes heredar de `Vehículo`, pues no todos los vehículos tienen porqué ser a motor (piensa por ejemplo en una clase `Bicicleta`), y no puedes heredar de otra clase pues ya heredas de `Vehículo`. Una solución podría ser crear una interfaz `Arrancable`, que proporcione los métodos típicos de un objeto a motor (no necesariamente vehículos). De este modo la clase `Coche` sigue siendo subclase de `Vehículo`, pero también implementaría los comportamientos de la interfaz `Arrancable`, los cuales podrían ser también implementados por otras clases, hereden o no de `Vehículo` (por ejemplo una clase `Motocicleta` o bien una clase `Motosierra`). La clase `Coche` implementará su método arrancar de una manera, la clase `Motocicleta` lo hará de otra (aunque bastante parecida) y la clase `Motosierra` de otra forma probablemente muy diferente, pero todos tendrán su propia versión del método arrancar como parte de la interfaz `Arrancable`.

Según esta concepción, podrías hacerte la siguiente pregunta: ¿podrá una clase implementar varias interfaces? La respuesta en este caso sí es afirmativa.

> ⚠ **Importante**
>
> Una clase puede adoptar distintos modelos de comportamiento establecidos en diferentes interfaces. **Es decir una clase puede implementar varias interfaces.**

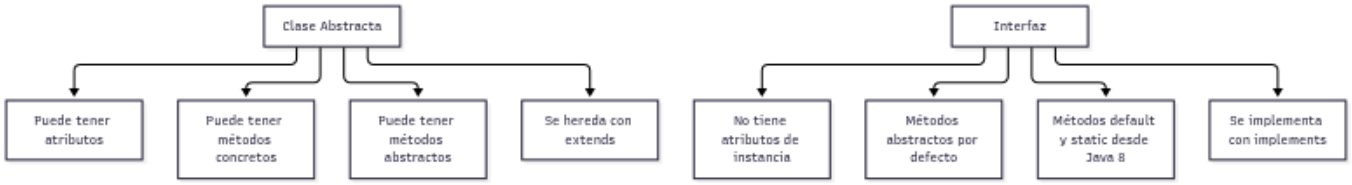
© 2025 David Martínez licensed under CC BY-NC-SA 4.0

¿CLASE ABSTRACTA O INTERFAZ?

Observando el concepto de interfaz que se acaba de proponer, podría caerse en la tentación de pensar que es prácticamente lo mismo que una clase abstracta en la que todos sus métodos sean abstractos.

Es cierto que en ese sentido existe un gran parecido formal entre una clase abstracta y una interfaz, pudiéndose en ocasiones utilizar indistintamente una u otra para obtener un mismo fin. Pero, a pesar de ese gran parecido, existen algunas diferencias, no sólo formales, sino también conceptuales, muy importantes:

- Una clase no puede heredar de varias clases, aunque sean abstractas (herencia múltiple). Sin embargo sí puede implementar una o varias interfaces y además seguir heredando de una clase.
- Una interfaz no puede definir métodos (no implementa su contenido), tan solo los declara o enumera.
- Una interfaz puede hacer que dos clases tengan un mismo comportamiento independientemente de sus ubicaciones en una determinada jerarquía de clases (no tienen que heredar las dos de una misma superclase, pues no siempre es posible según la naturaleza y propiedades de cada clase).
- Una interfaz permite establecer un comportamiento de clase sin apenas dar detalles, pues esos detalles aún no son conocidos (dependerán del modo en que cada clase decida implementar la interfaz).
- Las interfaces tienen su propia jerarquía, diferente e independiente de la jerarquía de clases.



De todo esto puede deducirse que una clase abstracta proporciona una interfaz disponible sólo a través de la herencia. Sólo quien herede de esa clase abstracta dispondrá de esa interfaz. Si una clase no pertenece a esa misma jerarquía (no hereda de ella) no podrá tener esa interfaz. Eso significa que para poder disponer de la interfaz podrías:

1. Volver a escribirla para esa jerarquía de clases. Lo cual no parece una buena solución.
2. Hacer que la clase herede de la superclase que proporciona la interfaz que te interesa, sacándola de su jerarquía original y convirtiéndola en clase derivada de algo de lo que conceptualmente no debería ser una subclase. Es decir, estarías forzando una relación "es un" cuando en realidad lo más probable es que esa relación no exista. Tampoco parece la mejor forma de resolver el problema.

Sin embargo, una interfaz sí puede ser implementada por cualquier clase, permitiendo que clases que no tengan ninguna relación entre sí (pertenecen a distintas jerarquías) puedan compartir un determinado comportamiento (una interfaz) sin tener que forzar una relación de herencia que no existe entre ellas.

A partir de ahora podemos hablar de otra posible relación entre clases: la de compartir un determinado comportamiento (interfaz). Dos clases podrían tener en común un determinado conjunto de comportamientos sin que necesariamente exista una relación jerárquica entre ellas. Tan solo cuando haya realmente una relación de tipo "es un" se producirá herencia.

¿Interfaz o clase abstracta?

Si sólo vas a proporcionar una lista de métodos abstractos (interfaz), sin definiciones de métodos ni atributos de objeto, suele ser recomendable definir una interfaz antes que clase abstracta. Es más, cuando vayas a definir una supuesta clase base, puedes comenzar declarándola como interfaz y sólo cuando veas que necesitas definir métodos o variables miembro, puedes entonces convertirla en clase abstracta (no instanciable) o incluso en una clase instanciable.

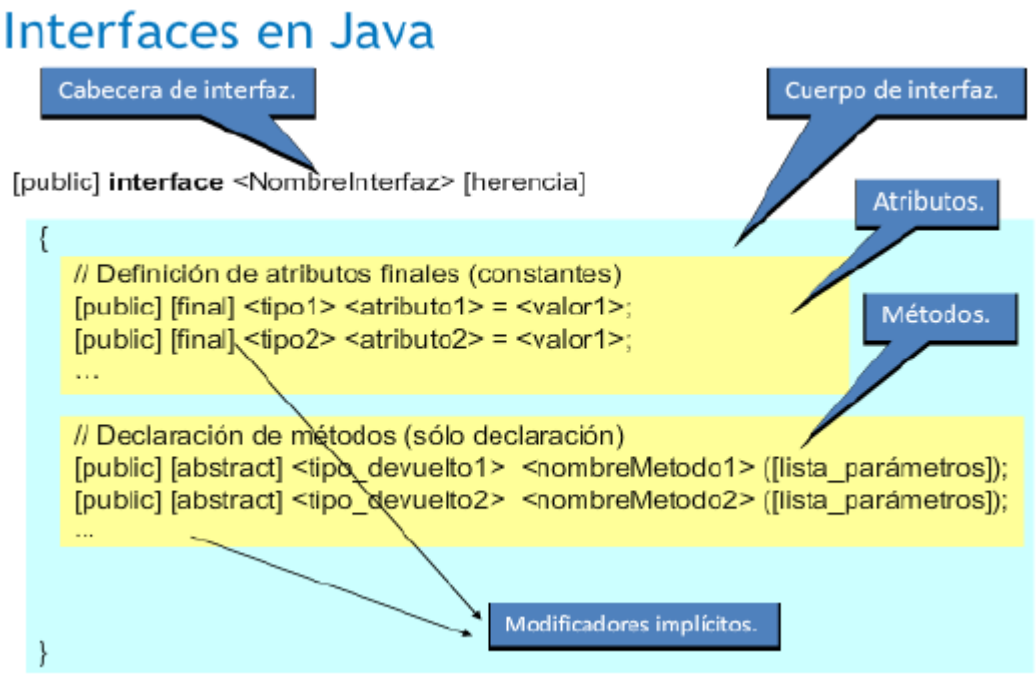
Definición de interfaces.

La declaración de una interfaz en Java es similar a la declaración de una clase, aunque con algunas variaciones:

- Se utiliza la palabra reservada `interface` en lugar de `class`.
- Puede utilizarse el modificador `public`. Si incluye este modificador la interfaz debe tener el mismo nombre que el archivo `.java` en el que se encuentra (exactamente igual que sucedía con las clases). Si no se indica el modificador `public`, el acceso será por omisión o "de paquete" (como sucedía con las clases).
- Todos los miembros de la interfaz (atributos y métodos) son `public` de manera implícita. No es necesario indicar el modificador `public`, aunque puede hacerse.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- Todos los atributos son de tipo `final` y `public` (tampoco es necesario especificarlo), es decir, constantes y públicos. Hay que darles un valor inicial.
- Todos los métodos son abstractos también de manera implícita (tampoco hay que indicarlo). No tienen cuerpo, tan solo la cabecera.



Como puedes observar, una interfaz consiste esencialmente en una lista de atributos finales (constantes) y métodos abstractos (sin implementar). Su sintaxis quedaría entonces:

```
[public] interface <NombreInterfaz> {
    [public] [final] <tipo1> <atributo1>=<valor1>;
    [public] [final] <tipo2> <atributo2>=<valor2>;
    ...
    [public] [abstract] <tipo_devuelto1> <nombreMetodo1> ([lista_parámetros]);
    [public] [abstract] <tipo_devuelto2> <nombreMetodo2> ([lista_parámetros]);
    ...
}
```

Si te fijas, la declaración de los métodos termina en punto y coma, pues no tienen cuerpo, al igual que sucede con los métodos abstractos de las clases abstractas. El ejemplo de la interfaz `Depredador` que hemos visto antes podría quedar entonces así:

```
public interface Depredador {
    void perseguir (Animal presa);
    void cazar (Animal presa);
    ...
}
```

Serán las clases que implementen esta interfaz (`León` , `Leopardo` , `Cocodrilo` , `Rana` , `Lagarto` , `Hombre` , etc.) las que definan cada uno de los métodos por dentro.

Revisa el Ejemplo13 para ver la creación de una interfaz.

Implementación de interfaces.

Como ya has visto, todas las clases que implementan una determinada interfaz están obligadas a proporcionar una definición (implementación) de los métodos de esa interfaz, adoptando el modelo de comportamiento propuesto por ésta.

Dada una interfaz, cualquier clase puede especificar dicha interfaz mediante el mecanismo denominado implementación de interfaces. Para ello se utiliza la palabra reservada implements:

```
class NombreClase implements NombreInterfaz {
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

De esta manera, la clase está diciendo algo así como "la interfaz indica los métodos que debo implementar, pero voy a ser yo (la clase) quien los implemente".

Es posible indicar varios nombres de interfaces separándolos por comas:

```
class NombreClase implements NombreInterfaz1, NombreInterfaz2,... {
```

Cuando una clase implementa una interfaz, tiene que redefinir sus métodos nuevamente con acceso público. Con otro tipo de acceso se producirá un error de compilación. Es decir, que del mismo modo que no se podían restringir permisos de acceso en la herencia de clases, tampoco se puede hacer en la implementación de interfaces.

Una vez implementada una interfaz en una clase, los métodos de esa interfaz tienen exactamente el mismo tratamiento que cualquier otro método, sin ninguna diferencia, pudiendo ser invocados, heredados, redefinidos, etc.

En el ejemplo de los depredadores, al definir la clase León, habría que indicar que implementa la interfaz Depredador:

```
class Leon implements Depredador {
```

En realidad la definición completa de la clase `Leon` debería ser:

```
class Leon extends Felino implements Depredador {
```

El orden importa

El orden de `extends` e `implements` es importante, primero se define la herencia y a continuación la interfaces que implementa.

Y en su interior habría que implementar aquellos métodos que contenga la interfaz:

```
void perseguir (Animal presa) {
    // Implementación del método localizar para un león
    ...
}
```

En el caso de clases que pudieran ser a la vez `Depredador` y `Presa`, tendrían que implementar ambas interfaces, como podría suceder con la clase Rana:

```
class Rana implements Depredador, Presa {
```

Que de manera completa quedaría:

```
class Rana extends Anfibio implements Depredador, Presa {
```

Y en su interior habría que implementar aquellos métodos que contengan ambas interfaces, tanto las de Depredador (localizar, cazar, etc.) como las de Presa (observar, huir, etc.).

Revisa el Ejemplo14 para ver implementación de interfaces.

Revisa el Ejemplo15 para ver una clase que implementa varias interfaces.

UN EJEMPLO DE IMPLEMENTACIÓN DE INTERFACES: LA INTERFAZ SERIES.

En la forma tradicional de una interfaz, los métodos se declaran utilizando solo su tipo de devolución y firma. Son, esencialmente, métodos abstractos. Por lo tanto, cada clase que incluye dicha interfaz debe implementar todos sus métodos.

Recuerda

En una interfaz, los métodos son implícitamente públicos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

> **Recuerda**
>
> **Las variables declaradas en una interfaz no son variables de instancia**. En cambio, son implícitamente `public`, `final`, y `static`, y deben inicializarse. Por lo tanto, son esencialmente **constantes**.

Aquí hay un ejemplo de una definición de interfaz. Especifica la interfaz a una clase que genera una serie de números.

Consulta el Ejemplo19 para ver el código completo de la interfaz `Series` y sus implementaciones `DeDos`, `SeriesDemo` y `DeTres`.

Esta interfaz se declara pública para que pueda ser implementada por código en cualquier paquete.

Los métodos que implementan una interfaz deben declararse públicos. Además, el tipo del método de implementación debe coincidir exactamente con el tipo especificado en la definición de la interfaz.

Como se explicó, cualquier cantidad de clases puede implementar una interfaz.

Simulación de la herencia múltiple mediante el uso de interfaces.

Una interfaz no tiene espacio de almacenamiento asociado (no se van a declarar objetos de un tipo de interfaz), es decir, no tiene implementación.

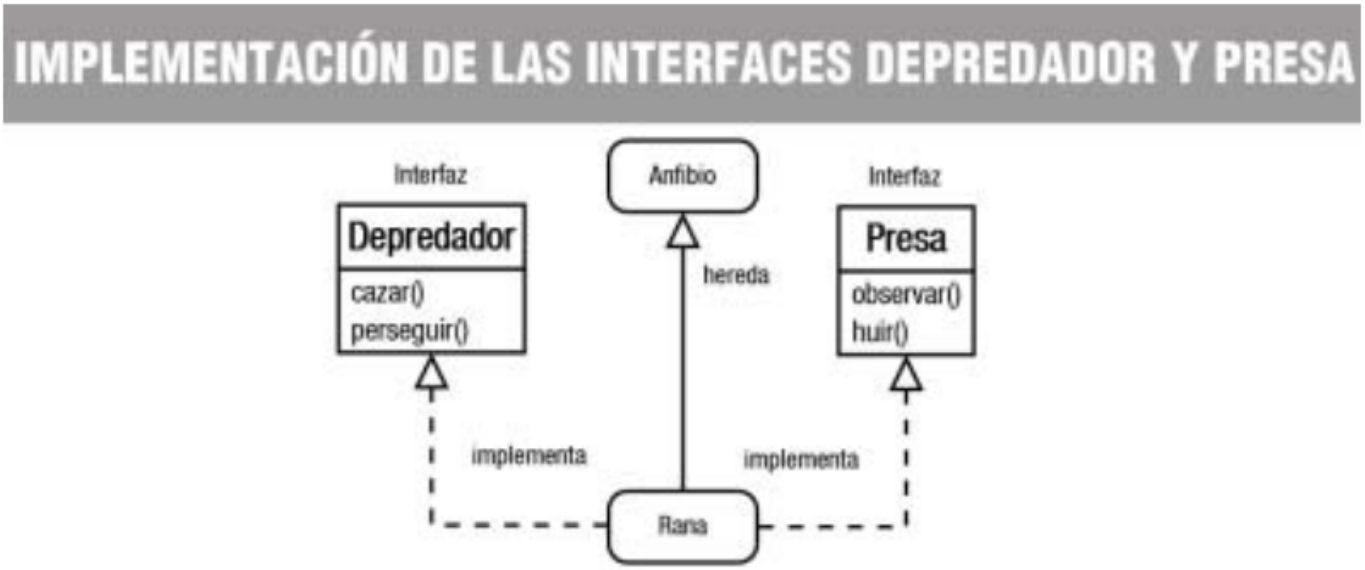
En algunas ocasiones es posible que interese representar la situación de que "una clase X es de tipo A, de tipo B, y de tipo C", siendo A, B, C clases disjuntas (no heredan unas de otras). Hemos visto que sería un caso de herencia múltiple que Java no permite.

Para poder simular algo así, podrías definir tres interfaces A, B, C que indiquen los comportamientos (métodos) que se deberían tener según se pertenezca a una supuesta clase A, B, o C, pero sin implementar ningún método concreto ni atributos de objeto (sólo interfaz).

De esta manera la clase X podría a la vez:

1. Implementar las interfaces A, B, C, que la dotarían de los comportamientos que deseaba heredar de las clases A, B, C.
2. Heredar de otra clase Y, que le proporcionaría determinadas características dentro de su taxonomía o jerarquía de objeto (atributos, métodos implementados y métodos abstractos).

En el ejemplo que hemos visto de las interfaces `Depredador` y `Presa`, tendrías un ejemplo de esto: la clase `Rana`, que es subclase de `Anfibio`, implementa una serie de comportamientos propios de un `Depredador` y, a la vez, otros más propios de una `Presa`. Esos comportamientos (métodos) no forman parte de la superclase `Anfibio`, sino de las interfaces. Si se decide que la clase `Rana` debe de llevar a cabo algunos otros comportamientos adicionales, podrían añadirse a una nueva interfaz y la clase `Rana` implementaría una tercera interfaz.



De este modo, con el mecanismo "**una herencia pero varias interfaces**", podrían conseguirse resultados similares a los obtenidos con la herencia múltiple.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Ahora bien, del mismo modo que sucedía con la herencia múltiple, puede darse el problema de la colisión de nombres al implementar dos interfaces que tengan un método con el mismo identificador. En tal caso puede suceder lo siguiente:

- Si los dos métodos tienen diferentes parámetros no habrá problema aunque tengan el mismo nombre pues se realiza una sobrecarga de métodos.
- Si los dos métodos tienen un valor de retorno de un tipo diferente, se producirá un error de compilación (al igual que sucede en la sobrecarga cuando la única diferencia entre dos métodos es ésa).

Si los dos métodos son exactamente iguales en identificador, parámetros y tipo devuelto, entonces solamente se podrá implementar uno de los dos métodos. En realidad se trata de un solo método pues ambos tienen la misma interfaz (mismo identificador, mismos parámetros y mismo tipo devuelto).

> **¡Cuidado!**
>
> La utilización de nombres idénticos en diferentes interfaces que pueden ser implementadas a la vez por una misma clase puede causar, además del problema de la colisión de nombres, dificultades de legibilidad en el código, pudiendo dar lugar a confusiones. Si es posible intenta evitar que se produzcan este tipo de situaciones.

Herencia de interfaces.

Las interfaces, al igual que las clases, también permiten la herencia. Para indicar que una interfaz hereda de otra se indica nuevamente con la palabra reservada `extends`. **Pero en este caso sí se permite la herencia múltiple de interfaces.** Si se hereda de más de una interfaz se indica con la lista de interfaces separadas por comas.

Por ejemplo, dadas las interfaces `InterfazUno` e `InterfazDos`:

```
public interface InterfazUno {
    // Métodos y constantes de la interfaz Uno
}

public interface InterfazDos {
    // Métodos y constantes de la interfaz Dos
}
```

Podría definirse una nueva interfaz que heredara de ambas:

```
public interface InterfazCompleja extends InterfazUno, InterfazDos {
    // Métodos y constantes de la interfaz compleja
}
```

Revisa el Ejemplo16 para ver herencia de interfaces.

Revisa el Ejemplo17 para ver diseño de interfaz e implementación.

Revisa el Ejemplo18 para ver un ejemplo completo de herencia de interfaces con constantes.

Funciones Lambda

Tal y como vimos en la unidad anterior, la implementación de los métodos de interfaces es muy susceptible de serlo a través de funciones lambda.

Imaginemos una clase `Persona`:

```
class Persona{
    private String nombre;
    private int edad;
    ...
}
```

Y un ArrayList `personas` formada por objetos de tipo `Persona`:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
...
ArrayList<Persona> personas = new ArrayList<>();
personas.add(new Persona("Nacho", 52));
personas.add(new Persona("David", 47));
personas.add(new Persona("Pepe", 42));
personas.add(new Persona("Maria", 22));
personas.add(new Persona("Marta", 4));
...
```

Ahora queremos ordenar el `ArrayList` de `personas` de mayor a menor edad usando... Implementación "tradicional" java: `Comparator` o `Comparable`

```
...
class ComparadorPersona implements Comparator <Persona>{
    @Override
    public int compare(Persona p1, Persona p2){
        return p2.getEdad() - p1.getEdad();
    }
}
...
```

```
...
personas.sort(new ComparadorPersona());
for (int i = 0; i < personas.size(); i++){
    System.out.println(personas.get(i));
}
...
```

Sin embargo, implementado con funciones Lambda seria...

```
...
personas.sort((p1, p2) -> p2.getEdad() - p1.getEdad());
for (int i = 0; i < personas.size(); i++){
    System.out.println(personas.get(i));
}
...
```

9.1.6 Polimorfismo

El polimorfismo es otro de los grandes pilares sobre los que se sustenta la Programación Orientada a Objetos (junto con la encapsulación y la herencia). Se trata nuevamente de otra forma más de establecer diferencias entre interfaz e implementación, es decir, entre el qué y el cómo.

La **encapsulación** te ha permitido agrupar características (atributos) y comportamientos (métodos) dentro de una misma unidad (clase), pudiendo darles un mayor o menor componente de visibilidad, y permitiendo separar al máximo posible la interfaz de la implementación.

Por otro lado la **herencia** te ha proporcionado la posibilidad de tratar a los objetos como pertenecientes a una jerarquía de clases. Esta capacidad va a ser fundamental a la hora de poder manipular muchos posibles objetos de clases diferentes como si fueran de la misma clase (polimorfismo).

El **polimorfismo** te va a permitir mejorar la organización y la legibilidad del código así como la posibilidad de desarrollar aplicaciones que sean más fáciles de ampliar a la hora de incorporar nuevas funcionalidades. Si la implementación y la utilización de las clases es lo suficientemente genérica y extensible será más sencillo poder volver a este código para incluir nuevos requerimientos.

Concepto de polimorfismo.

El polimorfismo consiste en la capacidad de poder utilizar una referencia a un objeto de una determinada clase como si fuera de otra clase (en concreto una subclase). Es una manera de decir que una clase podría tener varias (poli) formas (morfismo).

Un método "polimórfico" ofrece la posibilidad de ser distinguido (saber a qué clase pertenece) en tiempo de ejecución en lugar de en tiempo de compilación. Para poder hacer algo así es necesario utilizar métodos que pertenecen a una superclase y que en cada subclase se implementan de una forma en particular. En tiempo de compilación se invocará al método sin saber exactamente si será el de una subclase u otra (pues se está invocando al de la superclase). Sólo en tiempo de ejecución (una vez instanciada una u otra subclase) se conocerá realmente qué método (de qué subclase) es el que finalmente va a ser invocado.

Esta forma de trabajar te va a permitir hasta cierto punto "desentenderte" del tipo de objeto específico (subclase) para centrarte en el tipo de objeto genérico (superclase). De este modo podrás manipular objetos hasta cierto punto "desconocidos" en tiempo de compilación y que sólo durante la ejecución del programa se sabrá exactamente de qué tipo de objeto (subclase) se trata.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Polimorfismo

El polimorfismo ofrece la posibilidad de que toda referencia a un objeto de una superclase pueda tomar la forma de una referencia a un objeto de una de sus subclases. Esto te va a permitir escribir programas que procesen objetos de clases que formen parte de la misma jerarquía como si todos fueran objetos de sus superclases.

Más polimorfismo

El polimorfismo puede llevarse a cabo tanto con superclases (abstractas o no) como con interfaces.

Dada una superclase X, con un método m, y dos subclases A y B, que redefinen ese método m, podrías declarar un objeto O de tipo X que durante la ejecución podrá ser de tipo A o de tipo B (algo desconocido en tiempo de compilación). Esto significa que al invocarse el método m de X (superclase), se estará en realidad invocando al método m de A o de B (alguna de sus subclases). Por ejemplo:

```
// Declaración de una referencia a un objeto de tipo X
ClaseX obj; // Objeto de tipo X (superclase)
...

// Zona del programa donde se instancia un objeto de tipo A (subclase) y se le asigna a la referencia obj.
// La variable obj adquiere la forma de la subclase A.
obj = new ClaseA();
...

// Otra zona del programa.
// Aquí se instancia un objeto de tipo B (subclase) y se le asigna a la referencia obj.
// La variable obj adquiere la forma de la subclase B.
obj = new ClaseB();
...

// Zona donde se utiliza el método m sin saber realmente qué subclase se está utilizando.
// (Sólo se sabrá durante la ejecución del programa)

obj.m()
 // Llamada al método m (sin saber si será el método m de A o de B).
...
```

Imagina que estás trabajando con las clases `Alumno` y `Profesor` y que en determinada zona del código podrías tener objetos, tanto de un tipo como de otro, pero eso sólo se sabrá según vaya discurriendo la ejecución del programa. En algunos casos, es posible que un determinado objeto pudiera ser de la clase `Alumno` y en otros de la clase `Profesor`, pero en cualquier caso serán objetos de la clase `Persona`. Eso significa que la llamada a un método de la clase `Persona` (por ejemplo `devolverContenidoString`) en realidad será en unos casos a un método (con el mismo nombre) de la clase `Alumno` y, en otros, a un método (con el mismo nombre también) de la clase `Profesor`. Esto será posible hacerlo gracias a la ligadura dinámica.

Ligadura dinámica.

La conexión que tiene lugar durante una llamada a un método suele ser llamada ligadura (conexión o vinculación que tiene lugar durante una llamada a un método para saber qué código debe ser ejecutado. Puede ser estática o dinámica, vinculación o enlace (en inglés binding). Si esta vinculación se lleva a cabo durante el proceso de compilación, se le suele llamar ligadura estática (la vinculación que se produce en la llamada a un método con la clase a la que pertenece ese método se realiza en tiempo de compilación. Es decir, que antes de generar el código ejecutable se conoce exactamente el método (a qué clase pertenece) que será llamado. También conocido como vinculación temprana). En los lenguajes tradicionales, no orientados a objetos, ésta es la única forma de poder resolver la ligadura (en tiempo de compilación). Sin embargo, en los lenguajes orientados a objetos existe otra posibilidad: la ligadura dinámica (la vinculación que se produce en la llamada a un método con la clase a la que pertenece ese método se realiza en tiempo de ejecución. Es decir, que al generar el código ejecutable no se conoce exactamente el método (a qué clase pertenece) que será llamado. Sólo se sabrá cuando el programa esté en ejecución. También conocida como vinculación tardía, enlace tardío o late binding.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

La ligadura dinámica hace posible que sea el tipo de objeto instanciado (obtenido mediante el constructor finalmente utilizado para crear el objeto) y no el tipo de la referencia (el tipo indicado en la declaración de la variable que apuntará al objeto) lo que determine qué versión del método va a ser invocada. El tipo de objeto al que apunta la variable de tipo referencia sólo podrá ser conocido durante la ejecución del programa y por eso el polimorfismo necesita la ligadura dinámica.

En el ejemplo anterior de la clase X y sus subclases A y B, la llamada al método m sólo puede resolverse mediante ligadura dinámica, pues es imposible saber en tiempo de compilación si el método m que debe ser invocado será el definido en la subclase A o el definido en la subclase B:

```
1  //Llamada al método m (sin saber si será el método m de A o de B).
2  obj.m() // Esta llamada será resuelta en tiempo de ejecución (ligadura dinámica)
```

Revisa el Ejemplo20 para ver polimorfismo con instrumentos.

Limitaciones de la ligadura dinámica.

Como has podido comprobar, el polimorfismo se basa en la utilización de referencias de un tipo más "amplio" (superclases) que los objetos a los que luego realmente van a apuntar (subclases). Ahora bien, existe una importante restricción en el uso de esta capacidad, pues el tipo de referencia limita cuáles son los métodos que se pueden utilizar y los atributos a los que se pueden acceder.

> **Importante**
>
> No se puede acceder a los miembros específicos de una subclase a través de una referencia a una superclase. Sólo se pueden utilizar los miembros declarados en la superclase, aunque la definición que finalmente se utilice en su ejecución sea la de la subclase.

Veamos un ejemplo: si dispones de una clase `Profesor` que es subclase de `Persona` y declaras una variable como referencia un objeto de tipo `Persona`. Aunque más tarde esa variable haga referencia a un objeto de tipo `Profesor` (subclase), los miembros a los que podrás acceder sin que el compilador produzca un error serán los miembros de `Profesor` que hayan sido heredados de `Persona` (superclase). De este modo, se garantiza que los métodos que se intenten llamar van a existir cualquiera que sea la subclase de `Persona` a la que se apunte desde esa referencia.

En el ejemplo de las clases `Persona`, `Profesor` y `Alumno`, el polimorfismo nos permitiría declarar variables de tipo `Persona` y más tarde hacer con ellas referencia a objetos de tipo `Profesor` o `Alumno`, pero no deberíamos intentar acceder con esa variable a métodos que sean específicos de la clase `Profesor` o de la clase `Alumno`, tan solo a métodos que sabemos que van a existir seguro en ambos tipos de objetos (métodos de la superclase `Persona`).

Revisa el Ejemplo21 para ver polimorfismo con `Persona`, `Alumno` y `Profesor`.

Interfaces y polimorfismo.

Es posible también llevar a cabo el polimorfismo mediante el uso de interfaces. Un objeto puede tener una referencia cuyo tipo sea una interfaz, pero para que el compilador te lo permita, la clase cuyo constructor se utilice para crear el objeto deberá implementar esa interfaz (bien por si misma o bien porque la implemente alguna superclase). Un objeto cuya referencia sea de tipo interfaz sólo puede utilizar aquellos métodos definidos en la interfaz, es decir, que no podrán utilizarse los atributos y métodos específicos de su clase, tan solo los de la interfaz.

Las referencias de tipo interfaz permiten unificar de una manera bastante estricta la forma de utilizarse de objetos que pertenezcan a clases muy diferentes (pero que todas ellas implementan la misma interfaz). De este modo podrías hacer referencia a diferentes objetos que no tienen ninguna relación jerárquica entre sí utilizando la misma variable (referencia a la interfaz). Lo único que los distintos objetos tendrían en común es que implementan la misma interfaz.

> **Recuerda**
>
> En este caso sólo podrás llamar a los métodos de la interfaz y no a los específicos de las clases.

Por ejemplo, si tenías una variable de tipo referencia a la interfaz `Arrancable`, podrías instanciar objetos de tipo `Coche` o `Motosierra` y asignarlos a esa referencia (teniendo en cuenta que ambas clases no tienen una relación de herencia). Sin embargo, tan solo podrás usar en ambos casos los métodos y los atributos de la interfaz `Arrancable` (por ejemplo arrancar) y no los de `Coche` o los de `Motosierra` (sólo los genéricos, nunca los específicos).

En el caso de las clases `Persona`, `Alumno` y `Profesor`, podrías declarar, por ejemplo, variables del tipo `Imprimible`:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
Imprimible obj; // Imprimible es una interfaz y no una clase
```

Con este tipo de referencia podrías luego apuntar a objetos tanto de tipo Profesor como de tipo Alumno, pues ambos implementan la interfaz `Imprimible`:

```
// En algunas circunstancias podría suceder esto:
obj= new Alumno (nombre, apellidos, fecha, grupo, nota); // Polimorfismo con interfaces
...

// En otras circunstancias podría suceder esto:
obj= new Profesor (nombre, apellidos, fecha, especialidad, salario); // Polimorfismo con interfaces
...
```

Y más adelante hacer uso de la ligadura dinámica:

```
// Llamadas sólo a métodos de la interfaz
String contenido;
contenido= obj.devolverContenidoString(); // Ligadura dinámica con interfaces
```

Conversión de objetos.

Como ya has visto, en principio no se puede acceder a los miembros específicos de una subclase a través de una referencia a una superclase. Si deseas tener acceso a todos los métodos y atributos específicos del objeto subclase tendrás que realizar una conversión explícita (casting) que convierta la referencia más general (superclase) en la del tipo específico del objeto (subclase).

Para que puedas realizar conversiones entre distintas clases es obligatorio que exista una relación de herencia entre ellas (una debe ser clase derivada de la otra). Se realizará una conversión implícita o automática de subclase a superclase siempre que sea necesario, pues un objeto de tipo subclase siempre contendrá toda la información necesaria para ser considerado un objeto de la superclase.

Ahora bien, la conversión en sentido contrario (de superclase a subclase) debe hacerse de forma explícita y según el caso podría dar lugar a errores por falta de información (atributos) o de métodos. En tales casos se produce una excepción de tipo `ClassCastException`.

Por ejemplo, imagina que tienes una clase `Animal` y una clase `Besugo`, subclase de `Animal`:

```
class Animal {
    public String nombre;
}
class Besugo extends Animal {
    public double peso;
}
```

A continuación declaras una variable referencia a la clase `Animal` (superclase) pero sin embargo le asignas una referencia a un objeto de la clase `Besugo` (subclase) haciendo uso del polimorfismo:

```
Animal obj; // Referencia a objetos de la clase Animal
obj= new Besugo(); // Referencia a objetos clase Animal, pero apunta realmente a objeto clase Besugo (polimorfismo)
```

El objeto que acabas de crear como instancia de la clase `Besugo` (subclase de `Animal`) contiene más información que la que la referencia `obj` te permite en principio acceder sin que el compilador genere un error (pues es de clase `Animal`). En concreto los objetos de la clase `Besugo` disponen de `nombre` y `peso`, mientras que los objetos de la clase `Animal` sólo de `nombre`. Para acceder a esa información adicional de la clase especializada (`peso`) tendrás que realizar una conversión explícita (casting):

```
// Casting del tipo Animal al tipo Besugo (funcionará bien porque el objeto es realmente del tipo B)
System.out.printf ("obj.peso=%f\n", ((Besugo) obj).peso);
```

Sin embargo si se hubiera tratado de una instancia de la clase `Animal` y hubieras intentado acceder al miembro `peso`, se habría producido una excepción de tipo `ClassCastException`:

```
Animal obj; // Referencia a objetos de la clase Animal
obj= new Animal (); // Referencia a objetos de la clase Animal, y apunta realmente a un objeto de la clase Animal

// Casting del tipo Animal al tipo Besugo (puede dar problemas porque el objeto es realmente del tipo Animal):
// Funciona (la clase Animal tiene nombre)
System.out.printf ("obj.nombre=%s\n", ((Besugo) obj).nombre);

// ¡Error en ejecución! (la clase Animal no tiene peso). Producirá una ClassCastException.
System.out.printf ("obj.peso=%f\n", ((Besugo) obj).peso);
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

9.1.7 Ejemplos UD08

Ejemplo01

Composición de clases: refactorización de `Rectangulo` con `Punto`

Intenta rescribir los siguientes los métodos de la clase `Rectangulo` teniendo en cuenta ahora su nueva estructura de atributos (dos objetos de la clase `Punto`, en lugar de cuatro elementos de tipo `double`):

1. Método `calcularSuperfice`, que calcula y devuelve el área de la superficie encerrada por la figura.
2. Método `calcularPerimetro`, que calcula y devuelve la longitud del perímetro de la figura.

En ambos casos la interfaz no se ve modificada en absoluto (desde fuera su funcionamiento es el mismo), pero internamente deberás tener en cuenta que ya no existen los atributos `x1`, `y1`, `x2`, `y2`, de tipo `double`, sino los atributos `vertice1` y `vertice2` de tipo `Punto`.

Clase `Punto`:

```
package UD08._01_Ejemplo_2_1;

    public class Punto {
    private double x;
    private double y;

    public Punto(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getX() {
        return x;
    }

    public void setX(double x) {
        this.x = x;
    }

    public double getY() {
        return y;
    }

    public void setY(double y) {
        this.y = y;
    }
}
```

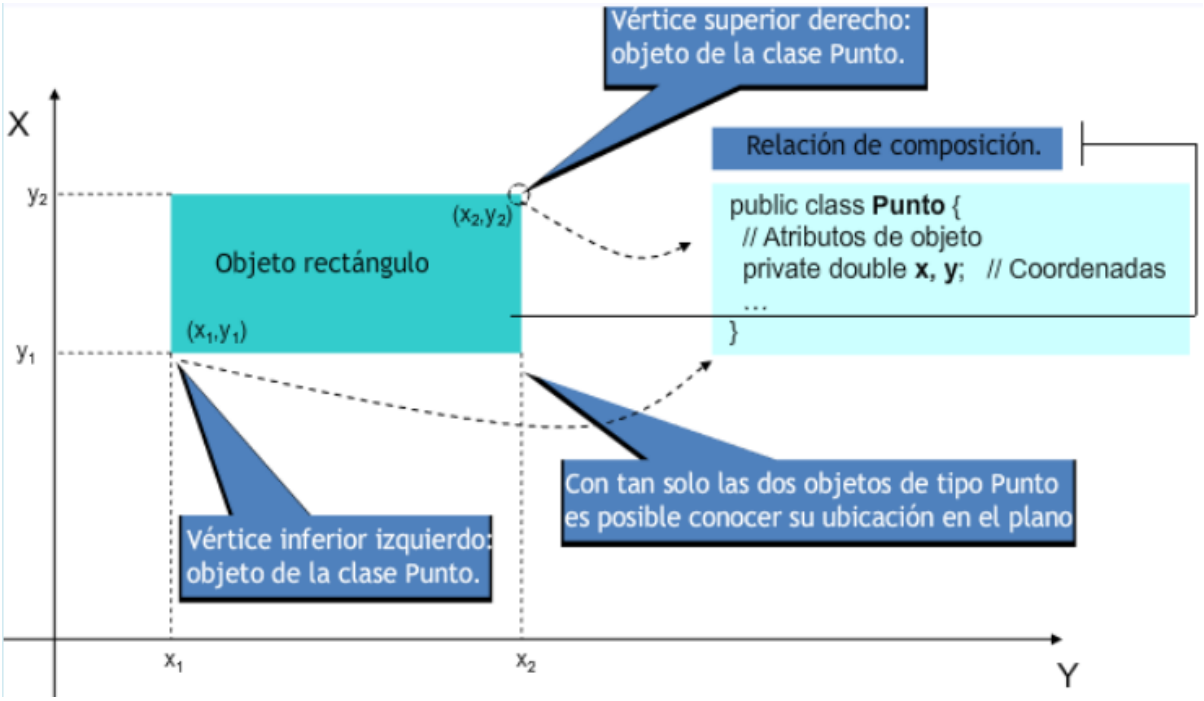
Clase `Rectangulo`:

```
package UD08._01_Ejemplo_2_1;

class Rectangulo {

    private Punto vertice1;
    private Punto vertice2;

    public double calcularSuperficie() {
        double area, base, altura; // Variables locales
        base = vertice2.getX() - vertice1.getX(); // Antes era x2 - x1
        altura = vertice2.getY() - vertice1.getY(); // Antes era y2 - y1
        area = base * altura;
        return area;
    }

    public double CalcularPerimetro() {
        double perimetro, base, altura; // Variables locales
        base = vertice2.getX() - vertice1.getX(); // Antes era x2 - x1
        altura = vertice2.getY() - vertice1.getY(); // Antes era y2 - y1
        perimetro = 2 * base + 2 * altura;
        return perimetro;
    }
}
```

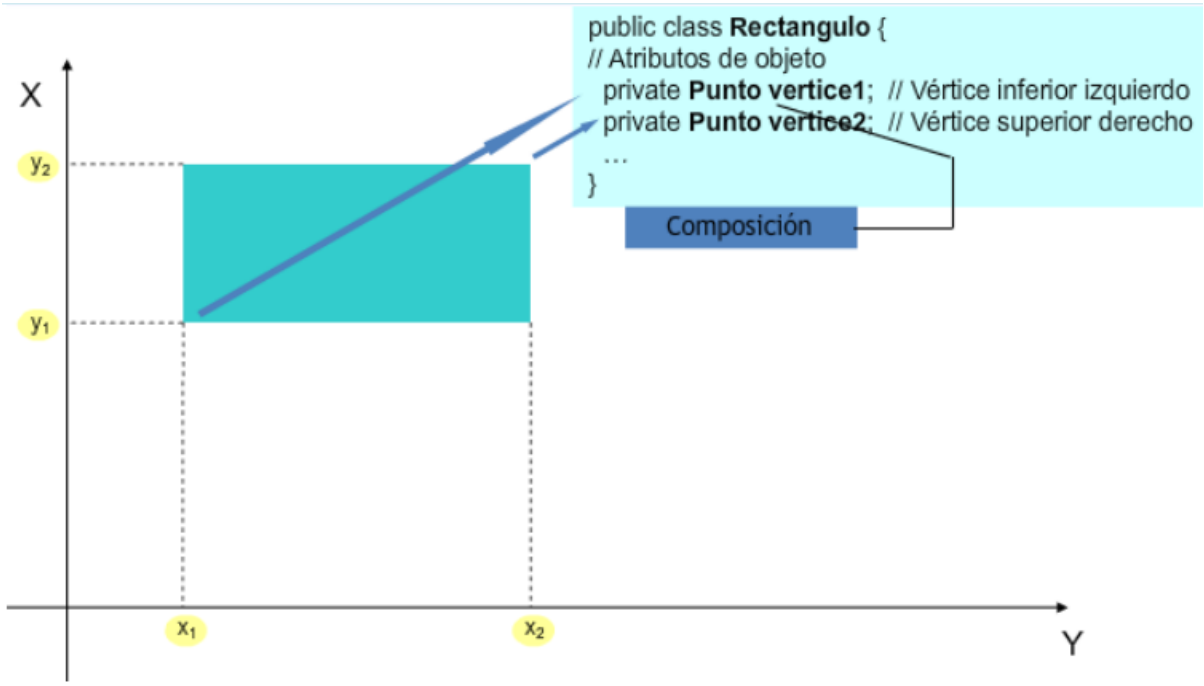
En la siguiente presentación puedes observar detalladamente el proceso completo de elaboración de la clase `Rectangulo` haciendo uso de la clase `Punto`:

1. Objetos de tipo `Rectangulo` compuesto por objetos de tipo `Punto`

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

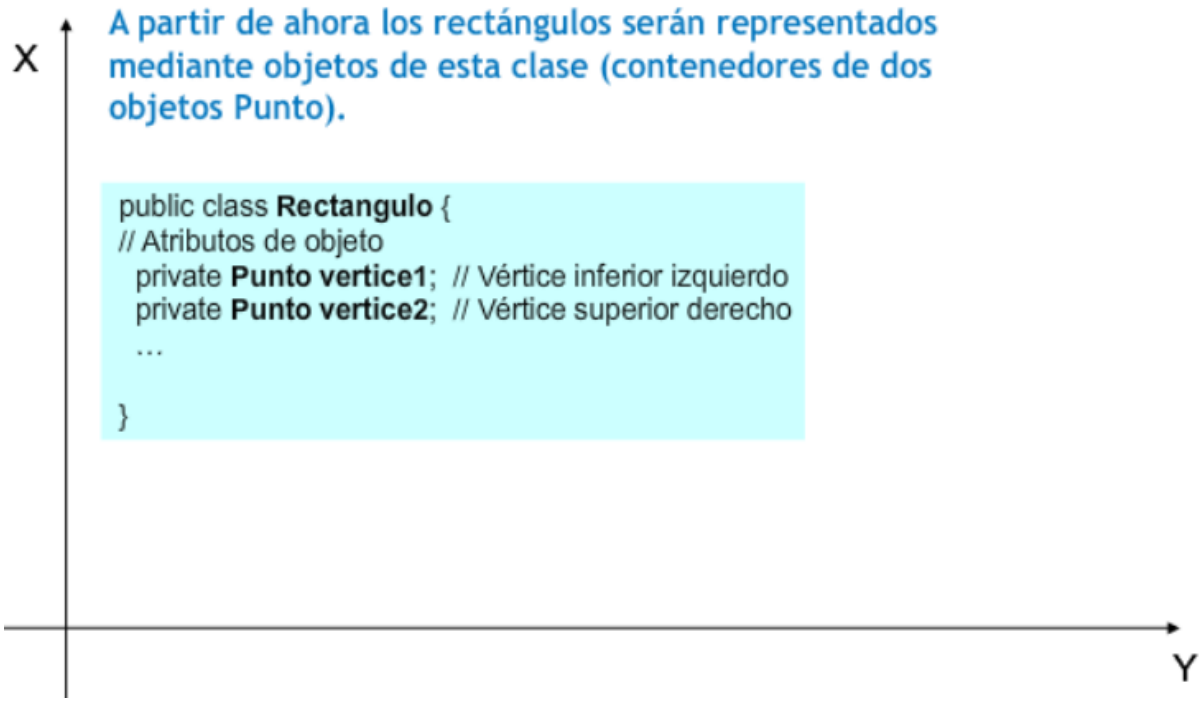


1. Atributos de un objeto `Rectangulo` compuesto por objetos de tipo `Punto`

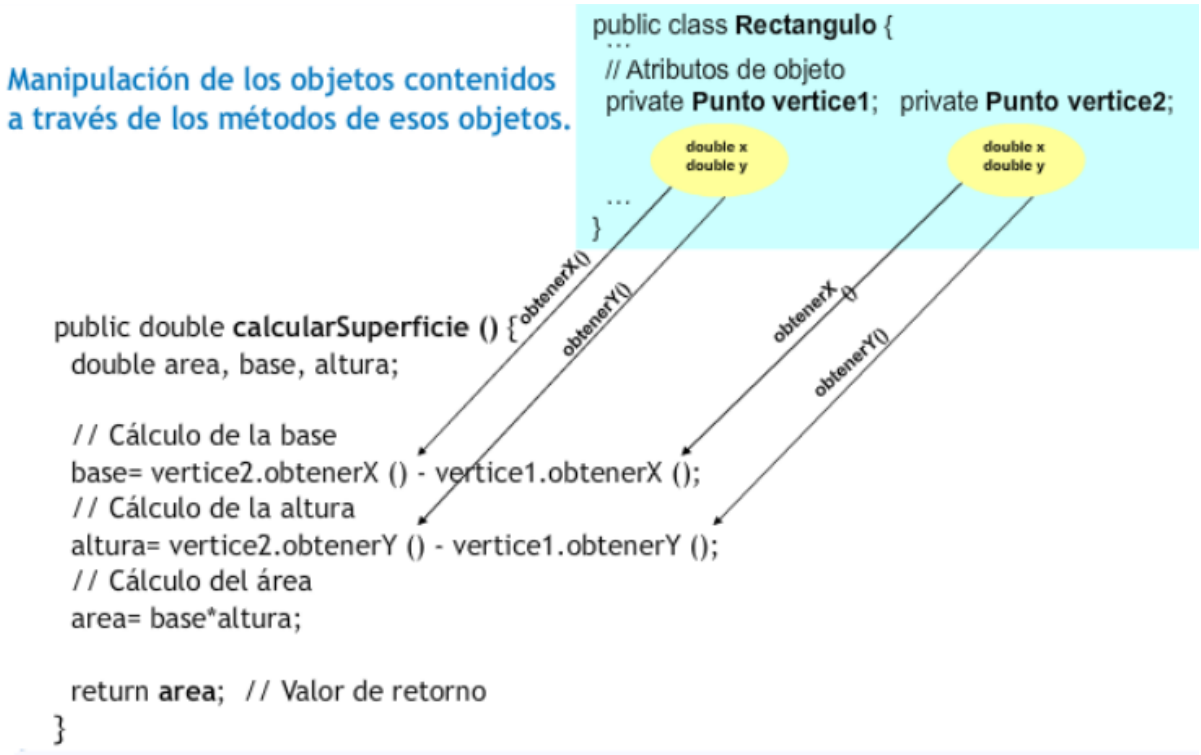


1. Clase `Rectangulo`

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



1. Método `calcularSuperficie`



⬆ Volver a teoría

Ejemplo02

Composición de clases: métodos de obtención de vértices

Dada la clase `Rectangulo`, escribe sus nuevos métodos `obtenerVertice1` y `obtenerVertice2` para que devuelvan los vértices inferior izquierdo y superior derecho del rectángulo (objetos de tipo `Punto`), teniendo en cuenta su nueva estructura de atributos (dos objetos de la clase `Punto`, en lugar de cuatro elementos de tipo `double`):

Los métodos de obtención de vértices devolverán objetos de la clase Punto:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
public Punto obtenerVertice1 (){
    return vertice1;
}

public Punto obtenerVertice2 (){
 return vertice2;
}
```

Esto funcionaría perfectamente, pero deberías tener cuidado con este tipo de métodos que devuelven directamente una referencia a un objeto atributo que probablemente has definido como privado. Estás de alguna manera haciendo público un atributo que fue declarado como privado.

Para evitar que esto suceda bastaría con crear un nuevo objeto que fuera una copia del atributo que se desea devolver (en este caso un objeto de la clase `Punto`).

Aquí tienes la solución para la nueva clase `Rectangulo`:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD08._02_Ejemplo_2_2;

class Rectangulo {

    private Punto vertice1;
    private Punto vertice2;

    public double calcularSuperficie() {
        double area, base, altura; // Variables locales
        base = vertice2.getX() - vertice1.getX(); // Antes era x2 - x1
        altura = vertice2.getY() - vertice1.getY(); // Antes era y2 - y1
        area = base * altura;
        return area;
    }

    public double CalcularPerimetro() {
        double perimetro, base, altura; // Variables locales
        base = vertice2.getX() - vertice1.getX(); // Antes era x2 - x1
        altura = vertice2.getY() - vertice1.getY(); // Antes era y2 - y1
        perimetro = 2 * base + 2 * altura;
        return perimetro;
    }

    /*
     * Así no!
     *
     *public Punto obtenerVertice1mal() {
     *    return vertice1;
     *}
     *
     *public Punto obtenerVertice2mal() {
     *    return vertice2;
     *}
     */

    //Mejor de este modo
    public Punto obtenerVertice1() {
        // Creación de un nuevo punto extrayendo sus atributos
        double x, y;
        Punto p;
        x = this.vertice1.getX();
        y = this.vertice1.getY();
        p = new Punto(x, y);
        return p;
    }

    //O mejor así:
    public Punto obtenerVertice2() {
        // Utilizando el constructor copia de Punto (si es que está definido)
        //Punto p;
        //p = new Punto(this.vertice2); // Uso del constructor copia
        //return p;

        //o más corto:
        return new Punto(this.vertice2);
    }

    public Rectangulo(Punto vertice1, Punto vertice2) {
        this.vertice1 = vertice1;
        this.vertice2 = vertice2;
    }

    public static void main(String[] args) {
        Punto puntoA = new Punto(0, 0);
        Punto puntoB = new Punto(5, 5);

        Rectangulo rectA = new Rectangulo(puntoA, puntoB);
        System.out.println("Perímetro del rectanculo A: " + rectA.CalcularPerimetro());//20

        puntoA.setX(4);
        puntoA.setY(4);

        Rectangulo rectB = new Rectangulo(puntoA, puntoB);
        System.out.println("Creo un nuevo rectangulo, pero cambia el Perímetro del anterior");
        System.out.println("Perímetro del rectanculo A: " + rectA.CalcularPerimetro());//20
        System.out.println("Perímetro del rectanculo B: " + rectB.CalcularPerimetro());//4

    }
}
```

De esta manera, se devuelve un punto totalmente nuevo que podrá ser manipulado sin ningún temor por parte del código cliente de la clase pues es una copia para él.

⬆ Volver a teoría

Ejemplo03

Constructores de `Rectangulo` con composición de `Punto`

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Intenta rescribir los constructores de la clase `Rectangulo` teniendo en cuenta ahora su nueva estructura de atributos (dos objetos de la clase `Punto`, en lugar de cuatro elementos de tipo `double`): 1. Un constructor sin parámetros (para sustituir al constructor por defecto) que haga que los valores iniciales de las esquinas del rectángulo sean (0,0) y (1,1). 2. Un constructor con cuatro parámetros, `x1`, `y1`, `x2`, `y2`, que cree un rectángulo con los vértices (`x1`, `y1`) y (`x2`, `y2`). 3. Un constructor con dos parámetros, `punto1`, `punto2`, que rellene los valores iniciales de los atributos del rectángulo con los valores proporcionados a través de los parámetros. 4. Un constructor con dos parámetros, `base` y `altura`, que cree un rectángulo donde el vértice inferior derecho esté ubicado en la posición (0,0) y que tenga una base y una altura tal y como indican los dos parámetros proporcionados. 5. Un constructor copia.

POSIBLE SOLUCIÓN

Durante el proceso de creación de un objeto (constructor) de la clase contenedora (en este caso `Rectangulo`) hay que tener en cuenta también la creación (llamada a constructores) de aquellos objetos que son contenidos (en este caso objetos de la clase Punto).

En el caso del primer constructor, habrá que crear dos puntos con las coordenadas (0,0) y (1,1) y asignarlos a los atributos correspondientes (`vertice1` y `vertice2`):

```
public Rectangulo (){
    this.vertice1= new Punto (0,0);
    this.vertice2= new Punto (1,1);
}
```

Para el segundo constructor habrá que crear dos puntos con las coordenadas x1, y1, x2, y2 que han sido pasadas como parámetros:

```
public Rectangulo (double x1, double y1, double x2, double y2){
    this.vertice1= new Punto (x1, y1);
    this.vertice2= new Punto (x2, y2);
}
```

En el caso del tercer constructor puedes utilizar directamente los dos puntos que se pasan como parámetros para construir los vértices del rectángulo.

Ahora bien, esto podría ocasionar un efecto colateral no deseado si esos objetos de tipo Punto son modificados en el futuro desde el código cliente del constructor (no sabes si esos puntos fueron creados especialmente para ser usados por el rectángulo o si pertenecen a otro objeto que podría modificarlos más tarde).

Por tanto, para este caso quizá fuera recomendable crear dos nuevos puntos a imagen y semejanza de los puntos que se han pasado como parámetros. Para ello tendrías dos opciones:

1. Llamar al constructor de la clase Punto con los valores de los atributos (x, y).
2. Llamar al constructor copia de la clase Punto, si es que se dispone de él.

Aquí tienes las dos posibles versiones:

Constructor que "extrae" los atributos de los parámetros y crea nuevos objetos:

```
public Rectangulo (Punto vertice1, Punto vertice2) {
    this.vertice1= vertice1;
    this.vertice2= vertice2;
}
```

Constructor que crea los nuevos objetos mediante el constructor copia de los parámetros:

```
public Rectangulo(Punto vertice1, Punto vertice2) {
    this.vertice1 = new Punto(vertice1.getX(), vertice1.getY());
    this.vertice2 = new Punto(vertice2.getX(), vertice2.getY());
}
```

En este segundo caso puedes observar la utilidad de los constructores de copia a la hora de tener que clonar objetos (algo muy habitual en las inicializaciones).

```
public Rectangulo (Punto vertice1, Punto vertice2) {
    this.vertice1= new Punto (vertice1 );
    this.vertice2= new Punto (vertice2 );
}
```

Para el caso del constructor que recibe como parámetros la base y la altura, habrá que crear sendos vértices con valores (0,0) y (0 + base, 0 + altura), o lo que es lo mismo: (0,0) y (base, altura).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
public Rectangulo(double base, double altura) {
    this.vertice1 = new Punto(0,0);
    this.vertice2 = new Punto(base, altura);
}
```

Quedaría finalmente por implementar el constructor copia:

```
public Rectangulo (Rectangulo r) {
    this.vertice1= new Punto (r.obtenerVertice1() );
    this.vertice2= new Punto (r.obtenerVertice2() );
}
```

En este caso nuevamente volvemos a clonar los atributos `vertice1` y `vertice2` del objeto `r` que se ha pasado como parámetro para evitar tener que compartir esos atributos en los dos rectángulos. Así ahora el método main que comprueba la clase Rectanclo funciona correctamente:

```
public static void main(String[] args) {
    Punto puntoA = new Punto(0, 0);
    Punto puntoB = new Punto(5, 5);

    Rectangulo rectA = new Rectangulo(puntoA, puntoB);
    System.out.println("Perímetro del rectanculo A: " + rectA.CalcularPerimetro());//20

    puntoA.setX(4);
    puntoA.setY(4);

    Rectangulo rectB = new Rectangulo(puntoA, puntoB);
    System.out.println("Creo un nuevo rectangulo, pero cambia el Perímetro del anterior");
    System.out.println("Perímetro del rectanculo A: " + rectA.CalcularPerimetro());//20
    System.out.println("Perímetro del rectanculo B: " + rectB.CalcularPerimetro());//4
}
```

⬆ Volver a teoría

Ejemplo04

Herencia: creación de la clase `Profesor` a partir de `Persona`

Imagina que también necesitas una clase Profesor, que contará con atributos como nombre, apellidos, fecha de nacimiento, salario y especialidad. ¿Cómo crearías esa nueva clase y qué atributos le añadirías?

Está claro que un Profesor es otra especialización de Persona, al igual que lo era Alumno, así que podrías crear otra clase derivada de Persona y así aprovechar los atributos genéricos (nombre, apellidos, fecha de nacimiento) que posee todo objeto de tipo Persona. Tan solo faltaría añadirle sus atributos específicos (salario y especialidad):

```
public class Profesor extends Persona {
    String especialidad;
    double salario;
    ...
}
```

⬆ Volver a teoría

Ejemplo05

Herencia: uso de `protected` en atributos

Reescribe las clases `Alumno` y `Profesor` utilizando el modificador protected para sus atributos del mismo modo que se ha hecho para su superclase `Persona` 1. Clase `Alumno`. Se trata simplemente de añadir el modificador de acceso protected a los nuevos atributos que añade la clase.

```
public class Alumno extends Persona {
    protected String grupo;
    protected double notaMedia;
    ...
}
```

1. Clase `Profesor`. Exactamente igual que en la clase `Alumno`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
public class Profesor extends Persona {
    protected String especialidad;
    protected double salario;
    ...
}
```

⬆ Volver a teoría

Ejemplo06

Getters y setters en herencia

Dadas las clases Alumno y Profesor que has utilizado anteriormente, implementa métodos get y set en las clases Alumno y Profesor para trabajar con sus cinco atributos (tres heredados más dos específicos).

POSIBLE SOLUCIÓN

1. Clase `Alumno`. Se trata de heredar de la clase Persona y por tanto utilizar con normalidad sus atributos heredados como si pertenecieran a la propia clase (de hecho se puede considerar que le pertenecen, dado que los ha heredado).

```
package UD08._06_Ejemplo_3_3;

import java.time.LocalDate;

public class Alumno extends Persona {
    protected String grupo;
    protected double notaMedia;

    // Método getNombre
    public String getNombre() {
        return nombre;
    }

    // Método getApellidos
    public String getApellidos() {
        return apellidos;
    }

    // Método getFechaNacim
    public LocalDate getFechaNacim() {
        return this.fechaNacim;
    }

    // Método getGrupo
    public String getGrupo() {
        return grupo;
    }

    // Método getNotaMedia
    public double getNotaMedia() {
        return notaMedia;
    }

    // Método setNombre
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    // Método setApellidos
    public void setApellidos(String apellidos) {
        this.apellidos = apellidos;
    }

    // Método setFechaNacim
    public void setFechaNacim(LocalDate fechaNacim) {
        this.fechaNacim = fechaNacim;
    }

    // Método setGrupo
    public void setGrupo(String grupo) {
        this.grupo = grupo;
    }

    // Método setNotaMedia
    public void setNotaMedia(double notaMedia) {
        this.notaMedia = notaMedia;
    }
}
```

Si te fijas, puedes utilizar sin problema la referencia `this` a la propia clase con esos atributos heredados, pues pertenecen a la clase: `this.nombre`, `this.apellidos`, etc.

1. Clase `Profesor`. Seguimos exactamente el mismo procedimiento que con la clase Alumno.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD08._06_Ejemplo_3_3;

import java.time.LocalDate;

public class Profesor extends Persona {
    String especialidad;
    double salario;

    // Método getNombre
    public String getNombre() {
        return nombre;
    }

    // Método getApellidos
    public String getApellidos() {
        return apellidos;
    }

    // Método getFechaNacim
    public LocalDate getFechaNacim() {
        return this.fechaNacim;
    }

    // Método getEspecialidad
    public String getEspecialidad() {
        return especialidad;
    }

    // Método getSalario
    public double getSalario() {
        return salario;
    }

    // Método setNombre
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    // Método setApellidos
    public void setApellidos(String apellidos) {
        this.apellidos = apellidos;
    }

    // Método setFechaNacim
    public void setFechaNacim(LocalDate fechaNacim) {
        this.fechaNacim = fechaNacim;
    }

    // Método setSalario
    public void setSalario(double salario) {
        this.salario = salario;
    }

    // Método setESpecialidad
    public void setESpecialidad(String especialidad) {
        this.especialidad = especialidad;
    }
}
```

Una conclusión que puedes extraer de este código es que has tenido que escribir los métodos `get` y `set` para los tres atributos heredados, pero ¿no habría sido posible definir esos seis métodos en la clase base y así estas dos clases derivadas hubieran también heredado esos métodos? La respuesta es afirmativa y de hecho es como lo vas a hacer a partir de ahora. De esa manera te habrías evitado tener que escribir seis métodos en la clase `Alumno` y otros seis en la clase `Profesor`.

Recuerda

Así que recuerda: **se pueden heredar tanto los atributos como los métodos.**

Aquí tienes un ejemplo de cómo podrías haber definido la clase Persona para que luego se hubieran podido heredar de ella sus métodos (y no sólo sus atributos):

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD08._06_Ejemplo_3_3;

import java.time.LocalDate;

public class Persona {
    protected String nombre;
    protected String apellidos;
    protected LocalDate fechaNacim;

    // Método getNombre
    public String getNombre() {
        return nombre;
    }

    // Método getApellidos
    public String getApellidos() {
        return apellidos;
    }

    // Método getFechaNacim
    public LocalDate getFechaNacim() {
        return this.fechaNacim;
    }

    // Método setNombre
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    // Método setApellidos
    public void setApellidos(String apellidos) {
        this.apellidos = apellidos;
    }

    // Método setFechaNacim
    public void setFechaNacim(LocalDate fechaNacim) {
        this.fechaNacim = fechaNacim;
    }
}
```

⬆ Volver a teoría

Ejemplo07

Getters y setters heredados

Dadas las clases `Persona`, `Alumno` y `Profesor` que has utilizado anteriormente, implementa métodos `get` y `set` en la clase `Persona` para trabajar con sus tres atributos y en las clases `Alumno` y `Profesor` para manipular sus cinco atributos (tres heredados más dos específicos), teniendo en cuenta que los métodos que ya hayas definido para `Persona` van a ser heredados en `Alumno` y en `Profesor`.

1. Clase `Persona`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD08._07_Ejemplo_3_3_1;

import java.time.LocalDate;

public class Persona {
    protected String nombre;
    protected String apellidos;
    protected LocalDate fechaNacim;

    // Método getNombre
    public String getNombre() {
        return nombre;
    }

    // Método getApellidos
    public String getApellidos() {
        return apellidos;
    }

    // Método getFechaNacim
    public LocalDate getFechaNacim() {
        return this.fechaNacim;
    }

    // Método setNombre
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    // Método setApellidos
    public void setApellidos(String apellidos) {
        this.apellidos = apellidos;
    }

    // Método setFechaNacim
    public void setFechaNacim(LocalDate fechaNacim) {
        this.fechaNacim = fechaNacim;
    }
}
```

1. Clase `Alumno`. Al heredar de la clase Persona tan solo es necesario escribir métodos para los nuevos atributos (métodos especializados de acceso a los atributos especializados), pues los métodos genéricos (de acceso a los atributos genéricos) ya forman parte de la clase al haberlos heredado.

```java
package UD08._07_Ejemplo_3_3_1;

public class Alumno extends Persona {
    protected String grupo;
    protected double notaMedia;

    // Método getGrupo
    public String getGrupo() {
        return grupo;
    }

    // Método getNotaMedia
    public double getNotaMedia() {
        return notaMedia;
    }

    // Método setGrupo
    public void setGrupo(String grupo) {
        this.grupo = grupo;
    }

    // Método setNotaMedia
    public void setNotaMedia(double notaMedia) {
        this.notaMedia = notaMedia;
    }
}
```

Aquí tienes una demostración práctica de cómo la herencia permite una reutilización eficiente del código, evitando tener que repetir atributos y métodos. Sólo has tenido que escribir cuatro métodos en lugar de diez.

1. Clase `Profesor`. Seguimos exactamente el mismo procedimiento que con la clase Alumno.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD08._07_Ejemplo_3_3_1;

public class Profesor extends Persona {
    String especialidad;
    double salario;

    // Método getEspecialidad
    public String getEspecialidad() {
        return especialidad;
    }

    // Método getSalario
    public double getSalario() {
        return salario;
    }

    // Método setSalario
    public void setSalario(double salario) {
        this.salario = salario;
    }

    // Método setEspecialidad
    public void setESpecialidad(String especialidad) {
        this.especialidad = especialidad;
    }
}
```

⬆ Volver a teoría

Ejemplo08

Redefinición del método `getNombre`

Dadas las clases `Persona`, `Alumno` y `Profesor` que has utilizado anteriormente, redefine el método `getNombre` para que devuelva la cadena "Alumno: ", junto con el nombre del alumno, si se trata de un objeto de la clase Alumno o bien "Profesor ", junto con el nombre del profesor, si se trata de un objeto de la clase Profesor.

1. Clase `Alumno`. Al heredar de la clase Persona tan solo es necesario escribir métodos para los nuevos atributos (métodos especializados de acceso a los atributos especializados), pues los métodos genéricos (de acceso a los atributos genéricos) ya forman parte de la clase al haberlos heredado. Esos son los métodos que se implementaron en el ejercicio anterior (`getGrupo`, `setGrupo`, etc.).

Ahora bien, hay que escribir otro método más, pues tienes que redefinir el método `getNombre` para que tenga un comportamiento un poco diferente al `getNombre` que se hereda de la clase base `Persona`:

```java
// Método getNombre
@Override
public String getNombre (){
    return "Alumno: " + this.nombre;
}
```

En este caso podría decirse que se "renuncia" al método heredado para redefinirlo con un comportamiento más especializado y acorde con la clase derivada.

1. Clase `Profesor`. Seguimos exactamente el mismo procedimiento que con la clase Alumno (redefinición del método `getNombre`).

```java
// Método getNombre
@Override
public String getNombre() {
    return "Profesor: " + this.nombre;
}
```

⬆ Volver a teoría

Ejemplo09

Método `mostrar` con llamada a `super`

Dadas las clases `Persona`, `Alumno` y `Profesor`, define un método mostrar para la clase Persona, que muestre el contenido de los atributos (datos personales) de un objeto de la clase Persona. A continuación, define sendos métodos mostrar especializados para las clases Alumno y Profesor que "amplíen" la funcionalidad del método mostrar original de la clase Persona. 1. Método mostrar de la clase `Persona`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
public void mostrar () {
    SimpleDateFormat formatoFecha = new SimpleDateFormat("dd/MM/yyyy");
    String Stringfecha= formatoFecha.format(this.fechaNacim.getTime());

    System.out.printf ("Nombre: %s\n", this.nombre);
    System.out.printf ("Apellidos: %s\n", this.apellidos);
    System.out.printf ("Fecha de nacimiento: %s\n", Stringfecha);
}
```

1. Método mostrar de la clase `Profesor`. Llamamos al método mostrar de su clase padre (`Persona`) y luego añadimos la funcionalidad específica para la subclase `Profesor`: ```java public void mostrar () { super.mostrar (); // Llamada al método "mostrar" de la superclase

   // A continuación mostramos la información "especializada" de esta subclase System.out.printf ("Especialidad: %s\n", this.especialidad); System.out.printf ("Salario: %7.2f euros\n", this.salario); } ```
2. Método mostrar de la clase `Alumno`. Llamamos al método mostrar de su clase padre (`Persona`) y luego añadimos la funcionalidad específica para la subclase `Alumno`:

```java
public void mostrar () {
    super.mostrar ();
    // A continuación mostramos la información "especializada" de esta subclase
    System.out.printf ("Grupo: %s\n", this.grupo);
    System.out.printf ("Nota media: %5.2f\n", this.notaMedia);
}
```

⬆ Volver a teoría

Ejemplo10

Constructor con llamada a `super`

Escribe un constructor para la clase `Profesor` que realice una llamada al constructor de su clase base para inicializar sus atributos heredados. Los atributos específicos (no heredados) sí deberán ser inicializados en el propio constructor de la clase `Profesor`.

```java
public Profesor (String nombre, String apellidos, GregorianCalendar fechaNacim, String especialidad, double salario){
    super (nombre, apellidos, fechaNacim);
    this.especialidad= especialidad;
    this.salario= salario;
}
```

Puedes hacer lo mismo para la clase `Alumno`.

```java
public Alumno(String nombre, String apellidos, LocalDate fechaNacim, String grupo, double notaMedia) {
     super(nombre, apellidos, fechaNacim);
     this.grupo = grupo;
     this.notaMedia = notaMedia;
}
```

⬆ Volver a teoría

Ejemplo11

Clase abstracta `Persona`

Basándote en la jerarquía de clases de ejemplo (`Persona`, `Alumno`, `Profesor`), que ya has utilizado en otras ocasiones, modifica lo que consideres oportuno para que `Persona` sea, a partir de ahora, una clase abstracta (no instanciable) y las otras dos clases sigan siendo clases derivadas de ella, pero sí instanciables.

En este caso lo único que habría que hacer es añadir el modificador `abstract` a la clase `Persona`. El resto de la clase permanecería igual y las clases Alumno y Profesor no tendrían porqué sufrir ninguna modificación.

```java
public abstract class Persona {
    protected String nombre;
    protected String apellidos;
    protected LocalDate fechaNacim;
    ...
}
```

A partir de ahora no podrán existir objetos de la clase `Persona`. El compilador generaría un error.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Localiza en la API de Java algún ejemplo de clase abstracta.

Existen una gran cantidad de clases abstractas en la API de Java. Aquí tienes un par de ejemplos:

- La clase `AbstractList`:

```
public abstract class AbstractList<E> extends AbstractCollection<E> implements List<E>
```

  De la que heredan clases instanciable como `Vector` o `ArrayList`.

- La clase `AbstractSequentialList`:

```
public abstract class AbstractSequentialList<E> extends AbstractList<E>
```

Que hereda de `AbstractList` y de la que hereda la clase `LinkedList`

⬆ Volver a teoría

Ejemplo12

Método abstracto `mostrar` en `Persona`

Basándote en la jerarquía de clases `Persona`, `Alumno`, `Profesor`, crea un método abstracto llamado `mostrar` para la clase `Persona`. Dependiendo del tipo de persona (`alumno` o `profesor`) el método mostrar tendrá que mostrar unos u otros datos personales (habrá que hacer implementaciones específicas en cada clase derivada).

Una vez hecho esto, implementa completamente las tres clases (con todos sus atributos y métodos) y utilízalas en un pequeño programa de ejemplo que cree un objeto de tipo `Alumno` y otro de tipo `Profesor`, los rellene con información y muestre esa información en la pantalla a través del método mostrar.

Dado que el método mostrar no va a ser implementado en la clase Persona, será declarado como abstracto y no se incluirá su implementación:

```
protected abstract void mostrar ();
```

Recuerda que el simple hecho de que la clase `Persona` contenga un método abstracto hace que la clase sea abstracta (y deberá indicarse como tal en su declaración):

```
public abstract class Persona {
 ...
```

En el caso de la clase `Alumno` habrá que hacer una implementación específica del método mostrar y lo mismo para el caso de la clase `Profesor`.

1. Método mostrar para la clase `Alumno`.

```
@Override
public void mostrar() {
    DateTimeFormatter formato = DateTimeFormatter.ofPattern("d/MM/yyyy");
    String stringfecha = formato.format(this.fechaNacim);
    System.out.printf("Nombre: %s\n", this.nombre);
    System.out.printf("Apellidos: %s\n", this.apellidos);
    System.out.printf("Fecha de nacimiento: %s\n", stringfecha);
    // A continuación mostramos la información "especializada" de esta subclase
    System.out.printf("Grupo: %s\n", this.grupo);
    System.out.printf("Nota media: %5.2f\n", this.notaMedia);
}
```

1. Método mostrar para la clase `Profesor`.

```
@Override
public void mostrar() {
    DateTimeFormatter formato = DateTimeFormatter.ofPattern("d/MM/yyyy");
    String stringfecha = formato.format(this.fechaNacim);
    System.out.printf("Nombre: %s\n", this.nombre);
    System.out.printf("Apellidos: %s\n", this.apellidos);
    System.out.printf("Fecha de nacimiento: %s\n", stringfecha);
    // A continuación mostramos la información "especializada" de esta subclase
    System.out.printf("Especialidad: %s\n", this.especialidad);
    System.out.printf("Salario: %7.2f euros\n", this.salario);
}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

1. Programa de ejemplo de uso.

Un pequeño programa de ejemplo de uso del método mostrar en estas dos clases podría ser:

```java
package UD08._12_Ejemplo_4_2;

import java.time.LocalDate;

public class EjemploUso {
    public static void main(String[] args) {
        // Declaración de objetos
        Alumno alumno;
        Profesor profesor;
        // Creación de objetos (llamada a constructores)
        alumno = new Alumno("Juan", "Torres", LocalDate.of(1990, 10, 6), "1DAW", 7.5);
        profesor = new Profesor("Antonio", "Campos", LocalDate.of(1970, 8, 15), "Informatica", 2000);
        // Utilización del método mostrar
        alumno.mostrar();
        profesor.mostrar();
    }
}
```

La salida debe ser algo parecido a esto:

```
Nombre: Juan
Apellidos: Torres
Fecha de nacimiento: 6/10/1990
Grupo: 1DAW
Nota media: 7,50
Nombre: Antonio
Apellidos: Campos
Fecha de nacimiento: 15/08/1970
Especialidad: Informatica
Salario: 2000,00 euros
```

⬆ Volver a teoría

Ejemplo13

Creación de la interfaz `Imprimible`

Crea una interfaz en Java cuyo nombre sea `Imprimible` y que contenga algunos métodos útiles para mostrar el contenido de una clase:

1. Método `devolverContenidoString`, que crea un `String` con una representación de todo el contenido público (o que se decida que deba ser mostrado) del objeto y lo devuelve. El formato será una lista de pares "nombre=valor" de cada atributo separado por comas y la lista completa encerrada entre llaves: "`{<nombre_atributo_1>=<valor_atributo_1>, ..., <nombre_atributo_n>=<valor_atributo_n>}`".
2. Método `devolverContenidoArrayList`, que crea un `ArrayList` de `String` con una representación de todo el contenido público (o que se decida que deba ser mostrado) del objeto y lo devuelve.
3. Método `devolverContenidoHashMap`, similar al anterior, pero en lugar devolver en un `ArrayList` los valores de los atributos, se devuelve en una `HashMap` en forma de pares (`nombre`, `valor`).

Se trata simplemente de declarar la interfaz e incluir en su interior esos tres métodos:

```java
package UD08._13_Ejemplo_5_2;

import java.util.ArrayList;
import java.util.HashMap;

public interface Imprimible {
    String devolverContenidoString();
    ArrayList devolverContenidoArrayList();
    HashMap devolverContenidoHashMap();
}
```

El cómo se implementarán cada uno de esos métodos dependerá exclusivamente de cada clase que decida implementar esta interfaz.

⬆ Volver a teoría

Ejemplo14

Implementación de `Imprimible` en `Alumno` y `Profesor`

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Haz que las clases `Alumno` y `Profesor` implementen la interfaz `Imprimible` que se ha escrito en el ejercicio anterior.

La primera opción que se te puede ocurrir es pensar que en ambas clases habrá que indicar que implementan la interfaz `Imprimible` y por tanto definir los métodos que ésta incluye: `devolverContenidoString`, `devolverContenidoHashMap` y `devolverContenidoArrayList`.

Si las clases `Alumno` y `Profesor` no heredaran de la misma clase habría que hacerlo obligatoriamente así, pues no comparten superclase y precisamente para eso sirven las interfaces: para implementar determinados comportamientos que no pertenecen a la estructura jerárquica de herencia en la que se encuentra una clase (de esta manera, clases que no tienen ninguna relación de herencia podrían compartir interfaz).

Pero en este caso podríamos aprovechar que ambas clases sí son subclases de una misma superclase (heredan de la misma) y hacer que la interfaz `Imprimible` sea implementada directamente por la superclase (`Persona`) y de este modo ahorrarnos bastante código. Así no haría falta indicar explícitamente que `Alumno` y `Profesor` implementan la interfaz `Imprimible`, pues lo estarán haciendo de forma implícita al heredar de una clase que ya ha implementado esa interfaz (la clase `Persona`, que es padre de ambas).

Una vez que los métodos de la interfaz estén implementados en la clase `Persona`, tan solo habrá que redefinir o ampliar los métodos de la interfaz para que se adapten a cada clase hija específica (`Alumno` o `Profesor`), ahorrándonos tener que escribir varias veces la parte de código que obtiene los atributos genéricos de la clase `Persona`.

1. Clase `Persona`.

Indicamos que se va a implementar la interfaz `Imprimible`:

```
public abstract class Persona implements Imprimible {
...
```

Definimos el método `devolverContenidoHashMap` a la manera de como debe ser implementado para la clase Persona. Podría quedar, por ejemplo, así:

```
@Override
public HashMap devolverContenidoHashMap() {
    // Creamos la HashMap que va a ser devuelta
    HashMap contenido = new HashMap();
    // Añadimos los atributos de la clase
    DateTimeFormatter formato = DateTimeFormatter.ofPattern("d/MM/yyyy");
    String stringFecha = formato.format(this.fechaNacim);
    contenido.put("nombre", this.nombre);
    contenido.put("apellidos", this.apellidos);
    contenido.put("fechaNacim", stringFecha);
    // Devolvemos la HashMap
    return contenido;
}
```

Del mismo modo, definimos también el método `devolverContenidoArrayList`:

```
@Override
public ArrayList devolverContenidoArrayList() {
    // Creamos la ArrayList que va a ser devuelta
    ArrayList contenido = new ArrayList();
    // Añadimos los atributos de la clase
    DateTimeFormatter formato = DateTimeFormatter.ofPattern("d/MM/yyyy");
    String stringFecha = formato.format(this.fechaNacim);
    contenido.add(this.nombre);
    contenido.add(this.apellidos);
    contenido.add(stringFecha);
    // Devolvemos la ArrayList
    return contenido;
}
```

Y por último el método `devolverContenidoString`:

```
@Override
public String devolverContenidoString() {
    DateTimeFormatter formato = DateTimeFormatter.ofPattern("d/MM/yyyy");
    String stringFecha = formato.format(this.fechaNacim);
    String contenido = "{" + this.nombre + ", " + this.apellidos + ", " + stringFecha + "}";
    return contenido;
}
```

1. Clase `Alumno`.

Esta clase hereda de la clase Persona, de manera que heredará los tres métodos anteriores. Tan solo habrá que redefinirlos para que, aprovechando el código ya escrito en la superclase, se añada la funcionalidad específica que aporta esta subclase.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
public class Alumno extends Persona {
...
```

Como puedes observar no ha sido necesario incluir el `implements Imprimible`, pues el `extends Persona` lo lleva implícito dado que `Persona` ya implementaba ese interfaz. Lo que haremos entonces será llamar al método que estamos redefiniendo utilizando la referencia a la superclase `super`.

El método `devolverContenidoHashMap` podría quedar, por ejemplo, así:

```
@Override
public HashMap devolverContenidoHashMap() {
    // Llamada al método de la superclase
    HashMap contenido = super.devolverContenidoHashMap();
    // Añadimos los atributos específicos de la clase
    contenido.put("grupo", this.grupo);
    contenido.put("notaMedia", this.notaMedia);
    // Devolvemos la HashMap rellena
    return contenido;
}
```

1. Clase `Profesor`. En este caso habría que proceder exactamente de la misma manera que con la clase Alumno: redefiniendo los métodos de la interfaz `Imprimible` para añadir la funcionalidad específica que aporta esta subclase, en este caso mostraremos la redifinición del método `devolverContenidoArrayList()`:

   ```
   @Override
   public ArrayList devolverContenidoArrayList() {
       // Llamada al método de la superclase
       ArrayList contenido = super.devolverContenidoArrayList();
       // Añadimos los atributos específicos de la clase
       contenido.add(this.especialidad);
       contenido.add(this.salario);
       // Devolvemos la ArrayList
       return contenido;
   }
   ```

   y la redefinición del método `devolverContenidoString()`:

   ```
   @Override
   public String devolverContenidoString() {
       // Llamada al método de la superclase
       String contenido = super.devolverContenidoString();
       //Eliminamos el último carácter, que contiene una llave de cierre.
       contenido = contenido.substring(0, contenido.length() - 1);
       contenido = contenido + ", " + this.especialidad + ", " + this.salario + "}";
       // Devolvemos el String creado.
       return contenido;
   }
   ```

⬆ Volver a teoría

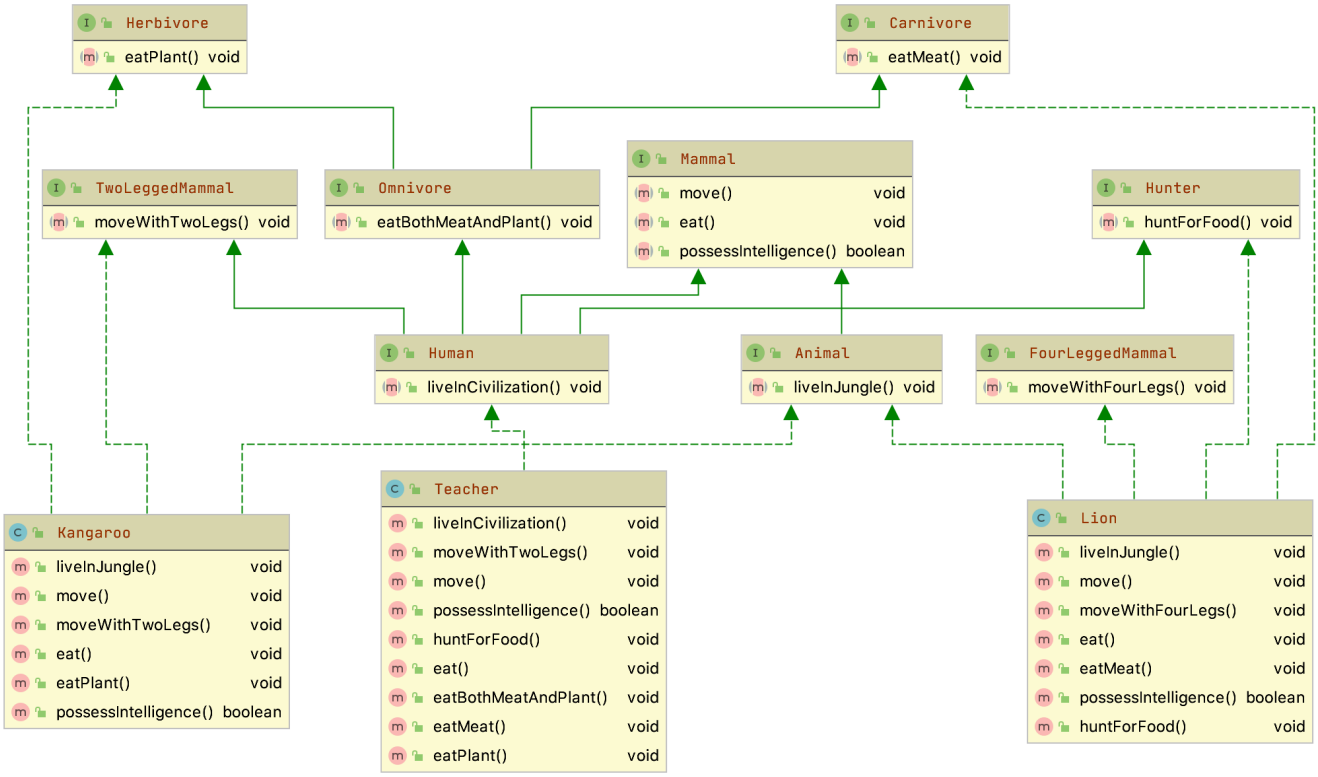
Ejemplo15

Una clase que implementa varias interfaces

¿Puede una clase implementar varias interfaces diferentes a la vez?

Observa el siguiente esquema UML:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Las clases `Kangaroo` y `Lion` implementan varias clases:

- `Kangaroo`: `Herbivore`, `TwoLeggedMammal` y `Animal`
- `Lion`: `Animal`, `FourLeggedMammal`, `Hunter` y `Carnivore`

⬆ Volver a teoría

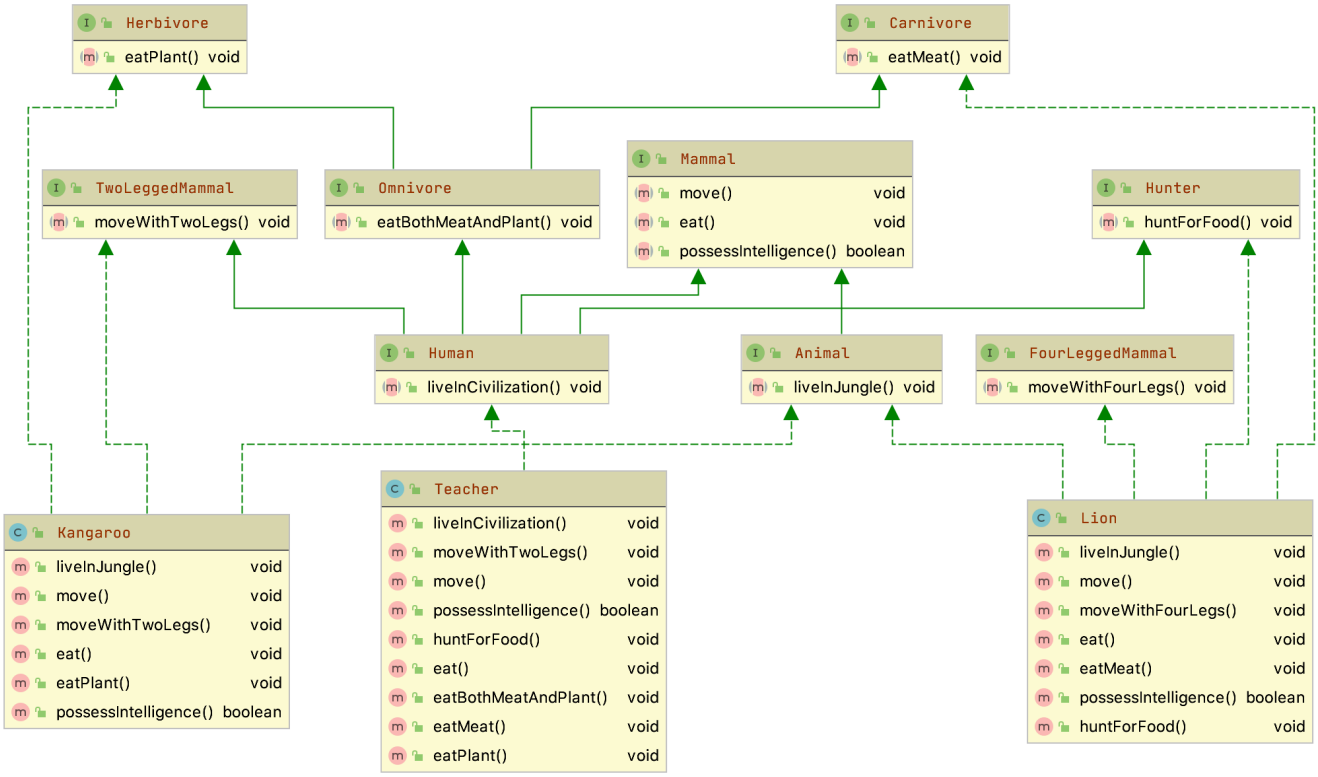
Ejemplo16

Herencia de interfaces

¿Puede una interfaz heredar de varias interfaces diferentes a la vez?

Observa el siguiente esquema UML:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Las interfaces `Human` y `Omnivore` heredan de varias interfaces:

- `Human`: de `TwoLeggedMammal`, `Omnivore`, `Mammal` y `Hunter`
- `Omnivore`: `Herbivore` y `Carnivore`

⬆ Volver a teoría

Ejemplo17

Diseño de interfaz e implementación en una clase

Supongamos una situación en la que nos interesa dejar constancia de que ciertas clases deben implementar una funcionalidad teórica determinada, diferente en cada clase afectada. Estamos hablando, pues, de la definición de un método teórico que algunas clases deberán implementar.

Un ejemplo real puede ser el método `calculoImporteJubilacion()` aplicable, de manera diferente, a muchas tipologías de trabajadores y, por tanto, podríamos pensar en diseñar una clase `Trabajador` en que uno de sus métodos fuera `calculoImporteJubilacion()`. Esta solución es válida si estamos diseñando una jerarquía de clases a partir de la clase `Trabajador` de la que cuelguen las clases correspondientes a las diferentes tipologías de trabajadores (metalúrgicos, hostelería, informáticos, profesores...). Además, disponemos del concepto de clase abstracta que cada subclase implemente obligatoriamente el método `calculoImporteJubilacion()`.

Pero, ¿y si resulta que ya tenemos las clases `Profesor`, `Informatico`, `Hostelero` en otras jerarquías de clases? La solución consiste en hacer que estas clases derivaran de la clase `Trabajador`, sin abandonar la derivación que pudieran tener, sería factible en lenguajes orientados a objetos que soportaran la herencia múltiple, pero esto no es factible en el lenguaje Java.

Para superar esta limitación, Java proporciona las interfaces.

Definición

Una interfaz es una **maqueta** contenedora de una lista de métodos abstractos y datos miembro (de tipos primitivos o de clases). Los atributos, si existen, son implícitamente consideradas `static` y `final`. Los métodos, si existen, son implícitamente considerados `public`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Para entender en qué nos pueden ayudar las interface, necesitamos saber:

- Una interfaz puede ser implementada por múltiples clases, de manera similar a como una clase puede ser superclase de múltiples clases.
- Las clases que implementan una interfaz están obligadas a sobrescribir todos los métodos definidos en la interfaz. Si la definición de alguno de los métodos a sobrescribir coincide con la definición de algún método heredado, este desaparece de la clase.
- Una clase puede implementar múltiples interfaces, a diferencia de la derivación, que sólo se permite una única clase base.
- Una interfaz introduce un nuevo tipo de dato, por la que nunca habrá ninguna instancia, pero sí objetos usuarios de la interfaz (objetos de las clases que implementan la interfaz). Todas las clases que implementan una interfaz son compatibles con el tipo introducido por la interfaz.
- Una interfaz no proporciona ninguna funcionalidad a un objeto (ya que la clase que implementa la interfaz es la que debe definir la funcionalidad de todos los métodos), pero en cambio proporciona la posibilidad de formar parte de la funcionalidad de otros objetos (pasándola por parámetro en métodos de otras clases).
- La existencia de las interfaces posibilita la existencia de una jerarquía de tipo (que no debe confundirse con la jerarquía de clases) que permite la herencia múltiple.
- Una interfaz no se puede instanciar, pero sí se puede hacer referencia.

Importante

Así, si `I` es una interfaz y `C` es una clase que implementa la interfaz, se pueden declarar referencias al tipo `I` que apunten objetos de `C`:

```
I obj = new C (<parámetros>);
```

Las interfaces pueden heredar de otras interfaces y, a diferencia de la derivación de clases, pueden heredar de más de una interfaz.

Así, si diseñamos la interfaz `Trabajador`, podemos hacer que las clases ya existentes (`Profesor`, `Informatico`, `Hostelero` ...) la implementen y, por tanto, los objetos de estas clases, además de ser objetos de las superclases respectivas, pasan a ser considerados objetos usuarios del tipo `Trabajador`. Con esta actuación nos veremos obligados a implementar el método `calculoImporteJubilacion()` a todas las clases que implementen la interfaz.

Alguien no experimentado en la gestión de interfaces puede pensar: ¿por qué tanto revuelo con las interfaces si hubiéramos podido diseñar directamente un método llamado `calculoImporteJubilacion()` en las clases afectadas sin necesidad de definir ninguna interfaz?

La respuesta radica en el hecho de que la declaración de la interfaz lleva implícita la declaración del tipo `Trabajador` y, por tanto, podremos utilizar los objetos de todas las clases que implementen la interfaz en cualquier método de cualquier clase que tenga algún argumento referencia al tipo `Trabajador` como, por ejemplo, en un hipotético método de una hipotética clase llamada Hacienda:

```
public void enviarBorradorIRPF(Trabajador t) {...}
```

Por el hecho de existir la interfaz `Trabajador`, todos los objetos de las clases que la implementan (`Profesor`, `Informatico`, `Hostelero` ...) se pueden pasar como parámetro en las llamadas al método `enviarBorradorIRPF(Trabajador t)`.

La sintaxis para declarar una interfaz es:

```
[public] interface <NombreInterfaz> [extends <Nombreinterfaz1>, <Nombreinterfaz2>...] {
    <CuerpoInterfaz>
}
```

Las interfaces también se pueden asignar a un paquete. La inexistencia del modificador de acceso público hace que la interfaz sea accesible a nivel del paquete.

Para los nombres de las interfaces, se aconseja seguir el mismo criterio que para los nombres de las clases.

Interfaces en cursiva

En la documentación de Java, las interfaces se identifican rápidamente entre las clases porque están en cursiva.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

El cuerpo de la interfaz es la lista de métodos y/o constantes que contiene la interfaz. Para las constantes no hay que indicar que son `static` y `final` y para los métodos no hay que indicar que son `public`. Estas características se asignan implícitamente.

La sintaxis para declarar una clase que implemente una o más interfaces es:

```
[final] [public] class <NombreClase> [extends <NombreClaseBase>] implements <NombreInterfaz1>, <NomInterfaz2>... {
    <CuerpoDeLaClase>
}
```

Los métodos de las interfaces a implementar en la clase deben ser obligatoriamente de acceso público.

Por último, comentar que, como por definición todos los datos miembro que se definen en una interfaz son `static` y `final`, y dado que las interfaz no se pueden instanciar, también resultan una buena herramienta para implantar grupos de constantes.

Así, por ejemplo:

```
public interface DiasSemana {
    int LUNES = 1, MARTES=2, MIERCOLES=3, JUEVES=4;
    int VIERNES=5, SABADO=6, DOMINGO=7;
    String[] NOMBRES_DIAS = {"", "lunes", "martes", "miércoles", "jueves", "viernes", "sábado", "domingo"};
}
```

Esta definición nos permite utilizar las constantes declaradas en cualquier clase que implemente la interfaz, de manera tan simple como:

```
System.out.println (DiasSemana.NOMBRES_DIAS[LUNES]);
```

⬆ Volver a teoría

Ejemplo18

Diseño de interfaz e implementación con herencia de interfaces.

Se presentan un par de interfaces que incorporan datos (de tipo primitivo y de referencia en clase) y métodos y una clase que las implementa. En la declaración de la clase se ve que sólo implementa la interfaz `B`, pero como esta interfaz deriva de la interfaz `A` resulta que la clase está implementando las dos interfaces.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD08._17_Ejemplo_5_6;

import java.util.Date;

interface A {
    Date ULTIMA_CREACION = new Date(0, 0, 1);
    void metodoA();
}

interface B extends A {
    int VALOR_B = 20;
    // 1 -1 -1900
    void metodoB();
}

public class Anexo5Interfaces implements B {
    private long b;
    private Date fechaCreacion = new Date();

    public Anexo5Interfaces(int factor) {
        b = VALOR_B * factor;
        ULTIMA_CREACION.setTime(fechaCreacion.getTime());
    }

    @Override
    public void metodoA() {
        System.out.println("En metodoA, ULTIMA_CREACION = " + ULTIMA_CREACION);
    }

    @Override
    public void metodoB() {
        System.out.println("En metodoB, b = " + b);
    }

    public static void main(String args[]) {
        System.out.println("Inicialmente, ULTIMA_CREACION = " + ULTIMA_CREACION);
        Anexo5Interfaces obj = new Anexo5Interfaces(5);
        obj.metodoA();
        obj.metodoB();
        A pa = obj;
        B pb = obj;
    }
}
```

```
Inicialmente, ULTIMA_CREACION = Mon Jan 01 00:00:00 CET 1900
En metodoA, ULTIMA_CREACION = Thu Aug 26 16:09:47 CEST 2021
En metodoB, b = 100
```

El ejemplo sirve para ilustrar algunos puntos:

- Comprobamos que los datos miembro de las interfaces son `static`, ya que en el método `main()` hacemos referencia al dato miembro `ULTIMA_CREACION` sin indicar ningún objeto de la clase.
- Si hubiéramos intentado modificar los datos `VALOR_B` o `ULTIMA_CREACION` no habríamos podido porque es final, pero en cambio sí podemos modificar el contenido del objeto `Date` apuntado por `ULTIMA_CREACION`, que corresponde al momento temporal de la última creación de un objeto ya cada nueva creación se actualiza su contenido.
- En las dos últimas instrucciones del método `main()` vemos que podemos declarar variables `pa` y `pb` de las interfaces y utilizarlas para hacer referencia a objetos de la clase `EjemploInterfaz()`.

⬆ Volver a teoría

Ejemplo19

Interfaz `Series` con implementaciones `DeDos` y `DeTres`.

Consulta el código completo de la interfaz Series y sus implementaciones `DeDos`, `SeriesDemo` y `DeTres`.

```java
public interface Series {
    int getSiguiente(); //Retorna el siguiente número de la serie
    void reiniciar(); //Reinicia
    void setComenzar(int x); //Establece un valor inicial
}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
class DeDos implements Series {
    int iniciar;
    int valor;

    DeDos(){
        iniciar=0;
        valor=0;
    }

    public int getSiguiente() {
        valor+=2;
        return valor;
    }

    public void reiniciar() {
        valor=iniciar;
    }

    public void setComenzar(int x) {
        iniciar=x;
        valor=x;
    }
}
```

```
class SeriesDemo {
    public static void main(String[] args) {
        DeDos ob=new DeDos();
        for (int i=0;i<5;i++){
            System.out.println("Siguiente valor es: "+ob.getSiguiente());
        }
        System.out.println("\nReiniciando");
        ob.reiniciar();
        for (int i=0;i<5;i++){
            System.out.println("Siguiente valor es: "+ob.getSiguiente());
        }
        System.out.println("\nIniciando en 100");
        ob.setComenzar(100);
        for (int i=0;i<5;i++){
            System.out.println("Siguiente valor es: "+ob.getSiguiente());
        }
    }
}
```

Las clases que implementan interfaces pueden definir miembros adicionales propios. Por ejemplo, la siguiente versión de `DeDos` agrega el método `getAnterior()`:

```
class DeDos implements Series {
    int iniciar;
    int valor;
    int anterior;
    DeDos(){
        iniciar=0;
        valor=0;
    }
    public int getSiguiente() {
        anterior=valor;
        valor+=2;
        return valor;
    }
    public void reiniciar() {
        valor=iniciar;
        anterior=valor-2;
    }
    public void setComenzar(int x) {
        iniciar=x;
        valor=x;
        anterior=x-2;
    }
    //Añadiendo un método que no está definido en Series
    int getAnterior(){
        return anterior;
    }
}
```

Cualquier cantidad de clases puede implementar una interfaz. Por ejemplo, `DeTres` genera una serie de múltiplos de tres:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
public class DeTres implements Series{
    int iniciar;
    int valor;
    DeTres(){
        iniciar=0;
        valor=0;
    }
    public int getSiguiente() {
        valor+=3;
        return valor;
    }
    public void reiniciar() {
        valor=iniciar;
    }
    public void setComenzar(int x) {
        iniciar=x;
        valor=x;
    }
}
```

⬆ Volver a teoría

Ejemplo20

Polimorfismo: `Instrumento`, `Piano` y `Flauta`

Imagínate una clase que represente a instrumento musical genérico (`Instrumento`) y dos subclases que representen tipos de instrumentos específicos (por ejemplo `Flauta` y `Piano`). Todas las clases tendrán un método `tocarNota` , que será específico para cada subclase.

Haz un pequeño programa de ejemplo en Java que utilice el polimorfismo (referencias a la superclase que se convierten en instancias específicas de subclases) y la ligadura dinámica (llamadas a un método que aún no están resueltas en tiempo de compilación) con estas clases que representan instrumentos musicales. Puedes implementar el método `tocarNota` mediante la escritura de un mensaje en pantalla.

La clase Instrumento podría tener un único método (`tocarNota`):

```java
package UD08._18_Ejemplo_6_2;

public abstract class Instrumento {
    public void tocarNota(String nota) {
        System.out.format("Instrumento: tocar nota %s.\n", nota);
    }
}
```

En el caso de las clases `Piano` y `Flauta` puede ser similar, heredando de Instrumento y redefiniendo el método tocarNota:

```java
package UD08._18_Ejemplo_6_2;

public class Flauta extends Instrumento {
    @Override
    public void tocarNota(String nota) {
        System.out.format("Flauta: tocar nota %s.\n", nota);
    }
}
```

```java
package UD08._18_Ejemplo_6_2;

public class Piano extends Instrumento {
    @Override
    public void tocarNota(String nota) {
        System.out.format("Piano: tocar nota %s.\n", nota);
    }
}
```

Creamos una clase para comprobar su funcionamiento `EjemploUso` :

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD08._18_Ejemplo_6_2;

import java.util.Scanner;

public class EjemploUso {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        System.out.print("Deseas un Piano o una Flauta (p o f)?: ");
        char respuesta = teclado.nextLine().toLowerCase().charAt(0);

        Instrumento instrumento1; // Ejemplo de objeto polimórfico (podrá ser Piano o Flauta)
        if (respuesta == 'p') {
            // Ejemplo de objeto polimórfico (en este caso va adquirir forma de Piano)
            instrumento1 = new Piano();
        } else {
            // Ejemplo de objeto polimórfico (en este caso va adquirir forma de Flauta)
            instrumento1 = new Flauta();
        }
        // Interpretamos una nota con el objeto instrumento1
        // No sabemos si se ejecutará el método tocarNota de Piano o de Flauta
        // (dependerá de la ejecución)
        instrumento1.tocarNota("do"); // Ejemplo de ligadura dinámica (tiempo de ejecución)
    }
}
```

A la hora de declarar una referencia a un objeto de tipo instrumento, utilizamos la superclase (Instrumento):

```java
Instrumento instrumento1; // Ejemplo de objeto polimórfico (podrá ser Piano o Flauta)
```

Sin embargo, a la hora de instanciar el objeto, utilizamos el constructor de alguna de sus subclases (Piano, Flauta, etc.):

```java
if (respuesta == 'p') {
    // Ejemplo de objeto polimórfico (en este caso va adquirir forma de Piano)
    instrumento1 = new Piano();
} else {
    // Ejemplo de objeto polimórfico (en este caso va adquirir forma de Flauta)
    instrumento1 = new Flauta();
}
```

Finalmente, a la hora de invocar el método `tocarNota`, no sabremos a qué versión (de qué subclase) de `tocarNota` se estará llamando, pues dependerá del tipo de objeto (subclase) que se haya instanciado. Se estará utilizando por tanto la ligadura dinámica:

```java
// Interpretamos una nota con el objeto instrumento1
// No sabemos si se ejecutará el método tocarNota de Piano o de Flauta
// (dependerá de la ejecución)
instrumento1.tocarNota("do"); // Ejemplo de ligadura dinámica (tiempo de ejecución)
```

⬆ Volver a teoría

Ejemplo21

Polimorfismo con `Persona`, `Alumno` y `Profesor`

Haz un pequeño programa en Java en el que se declare una variable de tipo `Persona`, se pidan algunos datos sobre esa persona (nombre, apellidos y si es alumno o si es profesor), y se muestren nuevamente esos datos en pantalla, teniendo en cuenta que esa variable no puede ser instanciada como un objeto de tipo `Persona` (es una clase abstracta) y que tendrás que instanciarla como `Alumno` o como `Profesor`. Recuerda que para poder recuperar sus datos necesitarás hacer uso de la ligadura dinámica y que tan solo deberías acceder a métodos que sean de la superclase.

Si tuviéramos diferentes variables referencia a objetos de las clases `Alumno` y `Profesor` tendrías algo así:

```java
Alumno obj1;
Profesor obj2;
...
// Si se dan ciertas condiciones el objeto será de tipo Alumno y lo tendrás en obj1
System.out.printf ("Nombre: %s\n", obj1.getNombre());
// Si se dan otras condiciones el objeto será de tipo Profesor y lo tendrás en obj2
System.out.printf ("Nombre: %s\n", obj2.getNombre());
```

Pero si pudieras tratar de una manera más genérica la situación, podrías intentar algo así:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
Persona obj;
// Si se dan ciertas condiciones el objeto será de tipo Alumno y por tanto lo instanciarás como tal
obj = new Alumno (<parámetros>);
// Si se otras condiciones el objeto será de tipo Profesor y por tanto lo instanciarás como tal
obj = new Profesor (<parámetros>);
```

De esta manera la variable obj podría contener una referencia a un objeto de la superclase `Persona` de subclase `Alumno` o bien de subclase `Profesor` (polimorfismo).

Esto significa que independientemente del tipo de subclase que sea (`Alumno` o `Profesor`), podrás invocar a métodos de la superclase `Persona` y durante la ejecución se resolverán como métodos de alguna de sus subclases:

```
//En tiempo de compilación no se sabrá de qué subclase de Persona será obj.
//Habrá que esperar la ejecución para que el entorno lo sepa e invoque al método adecuado.
System.out.format("Contenido del objeto: %s\n", obj.devolverContenidoString());
```

Por último recuerda que debes de proporcionar constructores a las subclases `Alumno` y `Profesor` que sean "compatibles" con algunos de los constructores de la superclase `Persona`, pues al llamar a un constructor de una subclase, su formato debe coincidir con el de algún constructor de la superclase (como debe suceder en general con cualquier método que sea invocado utilizando la ligadura dinámica).

Constructor "compatible" para `Alumno`:

```
public Alumno(String nombre, String apellidos, LocalDate fechaNacim) {
    super(nombre, apellidos, fechaNacim);
}
```

y el constructor "compatible" para `Profesor`:

```
public Profesor(String nombre, String apellidos, LocalDate fechaNacim) {
    super(nombre, apellidos, fechaNacim);
}
```

Aquí tienes el ejemplo completo de la clase `EjemploUso`:

```
package UD08._19_Ejemplo_6_3;

import java.time.LocalDate;
import java.util.Scanner;

public class EjemploUso {
    public static void main(String[] args) {
        Persona obj;
        Scanner teclado = new Scanner(System.in);
        System.out.print("Deseas crear un Profesor o un Alumno ('p' o 'a')?: ");
        char respuesta = teclado.nextLine().toLowerCase().charAt(0);

        if (respuesta == 'a') {
            // Ejemplo de objeto polimórfico (en este caso va adquirir forma de Alumno)
            obj = new Alumno("Alumno", "Apellidos", LocalDate.of(1977, 3, 8));
        } else {
            // Ejemplo de objeto polimórfico (en este caso va adquirir forma de Profesor)
            obj = new Profesor("Profe", "Apellidos", LocalDate.of(1977, 3, 8));
        }
        System.out.format("Contenido del objeto: %s\n", obj.devolverContenidoString());
    }
}
```

⬆ Volver a teoría

9.1.8 Resumen — Conceptos clave

Concepto	Definición
Composición	Relación «tiene un» entre clases (un todo contiene partes)
Herencia	Relación «es un» que permite reutilizar código de una superclase
Superclase	Clase de la que se heredan atributos y métodos
Subclase	Clase que hereda de otra
Polimorfismo	Capacidad de un objeto de comportarse de múltiples formas

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Concepto	Definición
Clase abstracta	Clase que no puede instanciarse, solo ser heredada
Interfaz	Contrato que define métodos que deben implementarse

9.1.9 Autoevaluación

- [ ] Diferencio entre composición y herencia
- [ ] Creo jerarquías de herencia correctamente
- [ ] Aplico la sobrescritura de métodos (@Override)
- [ ] Utilizo el polimorfismo en mis programas
- [ ] Diseño clases abstractas y métodos abstractos
- [ ] Implemento interfaces
- [ ] Sé cuándo usar una interfaz y cuándo una clase abstracta

9.1.10 Vídeos recomendados

Canal	Vídeo	Contenido
Programación ATS	Playlist POO Java	Vídeos 50–55: excepciones, try-catch, throws
Píldoras Informáticas	Curso Java — Excepciones	Vídeos 58–62: try-catch, finally, throw vs throws
MoureDev	Curso Completo de Java desde Cero	Sección de excepciones
makigas	Java: introducción a las excepciones	Conceptos básicos de excepciones
makigas	Java: throw y throws, usos y diferencias	Lanzar y delegar excepciones

21 de julio de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

9.2 Ejercicios de la UD08

9.2.1 Ejercicios Herencia

1. (Aulas) Diseñar una jerarquía de clases para modelizar las **aulas de un centro de estudios**.

   De un `Aula` se conoce el `código` (numérico), la `longitud` y la `anchura`. Se desea un método que devuelva la capacidad del aula sabiendo que esta se calcula a partir de la superficie a razón de 1 alumnos por cada 1.4 metros cuadrados de superficie.

   Además de las aulas, digamos normales, existen aulas de informática y aulas de música. En las aulas de música se necesita conocer si tienen o no piano. De las aulas de informática se conoce el número de ordenadores y su capacidad no se calcula en función de la superficie, sino a razón de dos alumnos por ordenador.

   Implementar el método `toString` de cada una de las clases diseñadas para que devuelva:

   - En las aulas normales, el `código` y la `superficie` y la `capacidad`.
   - En las aulas de música e informática el texto irá precedido por "Aula de música" o "Aula de informática", según corresponda.

2. (VideoJuegos) Un salón de **VideoJuegos** dispone de ordenadores en los que los clientes pueden jugar. Además de jugar en el establecimiento, la empresa alquila y vende juegos.

   a. Diseñar la clase `Juego` siguiendo las siguientes especificaciones:

      - Atributos protected: `titulo` (String), `fabricante` (String), `anyo` (int).
      - Constructor `public Juego(String t, String f, int a)`
      - Consultores de todos los atributos
      - `public String toString()`, que devuelve un String con los datos del Juego
      - `public boolean equals(Object o)`: Dos juegos son iguales si tienen el mismo título, fabricante y año.
      - `public int compareTo(Object o)`: Un juego es menor que otro si su título es menor. A igual título, si su fabricante es menor. A igual título y fabricante, si su año es menor.

   b. Diseñar las clases `JuegoEnAlquiler` y `JuegoEnVenta` (y otras si se considera oportuno), sabiendo que, además de los atributos descritos anteriormente, tienen.

      - `precio`
      - `nº de copias disponibles`
      - `JuegoEnAlquiler`
        - tiene un atributo que indica el número de días que se alquila. (Por el precio indicado, hay juegos que se alquilan por un dia, otros por 2, etc...)
        - Constructor que recibe todos sus datos
        - tiene un método `alquilar` que decrementa el número de copias disponibles.
        - tiene un método `devolver` que incrementa el número de copias disponibles.
        - `toString()` devuelve todos los datos del `JuegoEnAlquiler`
      - `JuegoEnVenta`
        - `Constructor` que recibe todos sus datos
        - tiene un método `vender`, que decrementa el número de copias disponibles.
        - `toString()` devuelve todos los datos del `JuegoEnVenta`

   > **Para pasar los tests**
   >
   > - Constructores: `JuegoEnAlquiler(titulo, fabricante, anyo, precio, copias, dias)` y `JuegoEnVenta(titulo, fabricante, anyo, precio, copias)`.
   > - Un consultor de las copias disponibles cuyo nombre empiece por `getCopias`.
   > - El formato del `toString()` lo eliges tú, pero tiene que incluir todos los datos.

3. (FNMT) La **Fabrica Nacional de Moneda y Timbre** quiere almacenar cierta información técnica del dinero (billetes y monedas) que emite. En concreto, le interesa:

   - `Valor`: Valor de la moneda o billete, en euros. (`double`)

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- `Año de emisión`: Año en que fué emitida la moneda o billete. (`int`)
- De las monedas,
  - `Diámetro`: Diámetro de la moneda, en milímetros. (`double`)
  - `Peso`: Peso de la moneda, en gramos (`double`)
- De los billetes.
  - `Altura del billete`, en mm (`double`)
  - `Anchura del billete`, en mm (`double`).

a. Diseñar la clase `Dinero` y sus subclases `Moneda` y `Billete`, desarrollando:

- Constructores que reciban los datos necesarios para inicializar los atributos de la clase correspondiente
- `equals`: Dos monedas o billetes son iguales si tienen el mismo año de emisión y valor.
- `compareTo`: Es menor (mayor) el de menor (mayor) año, a igual año es menor (mayor) el de menor (mayor) valor.
- `toString`: Que muestre todos los datos del billete o moneda. Los billetes irán precedidos por el texto "BILLETE" y las monedas por el texto "MONEDA"

b. Diseñar la clase `TestDinero` para probar las clases desarrolladas: Crear objetos de las clases `Moneda` y `Billete` y mostrarlos por pantalla.

> **Para pasar los tests**
>
> Constructores: `Moneda(diametro, peso, valor, anyoEmision)` y `Billete(anchura, altura, valor, anyoEmision)`.

4. (Televisores) Un **centro comercial** quiere mostrar cierta información sobre los televisores que vende. Los televisores pueden ser de dos tipos: de tubo o LCD. En concreto, de cada televisor le interesa mostrar
   - Marca (`String`)
   - Modelo (`String`)
   - Precio en euros
   - Pulgadas de la pantalla (`double`).
   - Resolución: La resolución se mide de forma distinta en los televisores de tubo que en los televisores LCD.
     - En los TV de tubo se mide en lineas.
     - En los TV LCD se mide pixels horizontales x pixels verticales.

   a. Diseñar la clase `Televisor` con los atributos y métodos comunes a los dos tipos de televisores y sus subclases `TVTubo` y `TVLCD` con los atributos y métodos que sea necesario:

   - Constructor de cada clase que permita inicializar todos los datos de la clase.
   - `equals`: Dos televisiones son iguales si son de la misma marca y modelo.
   - `compareTo`: Se considera menor (mayor) la de menor (mayor) marca. A igual marca, menor (mayor) la de menor (mayor) modelo.
   - `public String resulucion()`: Devuelve un texto con la resolución del televisor, como por ejemplo "420 lineas" o "800 x 600 pixels" dependiendo del tipo de televisor.
   - `public String toString()`: Devuelve un texto con la marca, modelo, precio, pulgadas y resolución.

   b. Diseñar la clase `TestTV` para probar las clases diseñadas. Crear algunos objetos de las clases `TVTubo` y `TVLCD` y mostrarlos por pantalla.

5. (Verdadero o falso) De cada pareja de afirmaciones **indica cual es la verdadera**:

   a. Pareja 1

   - Se dice que instanciamos una clase cuando creamos objetos de dicha clase.
   - Se dice que instanciamos una clase cuando creamos una subclase de dicha clase.

   b. Pareja 2

   - Si una clase es abstracta no se puede instanciar.
   - Si una clase es abstracta no se puede heredar de ella.

   c. Pareja 3

   - Una clase abstracta tiene que tener métodos abstractos.
   - Una clase puede ser abstracta y no tener métodos abstractos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

d. Pareja 4

- Si una clase tiene métodos abstractos tiene que ser abstracta.
- Una clase puede tener métodos abstractos y no ser abstracta.

e. Pareja 5

- Si una clase es abstracta sus subclases no pueden ser abstractas.
- Una clase abstracta puede tener subclases que también sean abstractas.

f. Pareja 6

- Si un método es abstracto en una clase, tiene que ser no abstracto en la subclase, o bien, la subclase tiene que ser también abstracta.
- Si un método es abstracto en una clase, no puede ser abstracto en las subclases.

g. Pareja 7

- Si un método se define final se tiene que reescribir en las subclases.
- Si un método se define final no se puede reescribir en las subclases.

h. Pareja 8

- Una clase puede tener un método final y no ser una clase final.
- Si una clase tiene un método final tiene que ser una clase final.

i. Pareja 9

- Si una clase se define final no se pueden definir subclases de ella.
- Si una clase se define final no se puede instanciar.

j. Pareja 10

- Un método definido final y abstract resultaría inútil, puesto que nunca se podría implementar en las subclases.
- Un método definido final y abstract podría resultar útil.

k. Pareja 11

- Una clase definida final y abstract resultaría inútil, puesto que no se podría instanciar ni heredar de ella.
- Una clase definida final y abstract podría resultar útil.

6. (Cuestiones) Dadas las siguientes **definiciones de clases**:

```java
public class Persona {
    private String nombre;
    private int edad;

    public Persona (){
        this.nombre = "";
        this.edad = 0;
    }
    public Persona(String n, int e){
        this.nombre = n;
        this.edad = e;
    }
    public String toString(){
        return "Nombre: " + nombre + "Edad " + edad;
    }
    public final String getNombre (){
        return nombre;
    }
    public final int getEdad(){
        return edad;
    }
}
```

```java
class Estudiante extends Persona {
    private double creditos;

    public Estudiante(String n, int e, double c){
        super(n,e);
        this.creditos = c;
    }
    public String toString(){
        return super.toString() + "\nCreditos: "+ creditos;
    }
}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
class Empleado extends Persona {
    private double salario;

    public Empleado(String n, int e, double s){
        super(n,e);
        this.salario = s;
    }
    public String toString(){
        return "Nombre: "+ this.nombre +
        "\nSalario: "+ this.salario;
    }
}
```

```java
class Test{
    public static void main(String[] args) {
        Estudiante e = new Estudiante("pepe",18,100);
        System.out.println(e.toString());
    }
}
```

Responde a las siguientes cuestiones justificando las respuestas.

a. ¿Es necesario el uso de `this` en el contructor de la clase `Estudiante`?
b. ¿Es necesario el uso de `super` en el método `toString` de la clase `Estudiante`?
c. Si quitásemos el constructor de la clase `Estudiante` ¿daría un error de compilación la clase `Estudiante`?
d. En el método `toString` de la clase `Empleado` ¿por qué es incorrecto el acceso que se hace al atributo `nombre`? ¿Cómo se tendría que definir `nombre` en la clase `Persona` para evitar el error?
e. ¿Qué consecuencia tiene que algunos métodos de la clase `Persona` se hayan definido `final`?
f. Si el método `toString` no se hubiera definido en ninguna de las tres clases ¿daría error el `sout` del método `main`?

9.2.2 Ejercicios Polimorfismo

1. (Obreros) Dada la siguiente **jerarquía de clases**:

```java
public interface Montador{
    void montar(String x);
    void desmontar(String x);
}

public class Obrero{
    public Obrero(){System.out.println("Se crea Obrero");}
    public void saludar(){System.out.println("Hola, soy Obrero");}
    ...
}

public class Carpintero extends Obrero implements Montador {
    public Carpintero(){System.out.println("Se crea Carpintero");}
    public void montar(String x) {System.out.println("Montando " + x);}
    public void desmontar(String x) {System.out.println("Desmontando " + x);}
    public void clavar() {...}
}

public class Albanyil extends Obrero {
    public Albanyil() {
        super();
        System.out.println("Se crea Albañil");
    }
    public void levantarMuro(){
        System.out.println("Levantando muro …");
    }
}
```

Indicar **qué líneas** del siguiente fragmento de programa **producirán errores de compilación**,

```java
public static void main(String[] args){
    Montador m1 = new Carpintero();
    Montador m2 = new Albanyil();
    Obrero o1 = new Carpintero();
    Obrero o2 = new Albanyil();
    o1.montar("Mesa");
    o2.levantarMuro();
    m1.saludar();
    m1.montar("Silla");
    ((Albanyil)o2).levantarMuro();
    ((Albanyil)o1).levantarMuro();
}
```

Una vez eliminadas las líneas con error, indicar **cuál sería la salida** por pantalla del programa.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

¿**Sería correcta** la instrucción siguiente?

```
Albanyil a = new Albanyil();
System.out.println(a.toString());
```

Escribe el programa en el `main` de la clase `Test` y las respuestas en comentarios.

> **Para pasar los tests**
>
> Deja **comentadas** (`//`) las líneas que dan error de compilación y escribe al lado por qué fallan. Si alguna se queda sin comentar, no compila ningún ejercicio de la unidad y la nota es 0.

2. (CuestionHerencia) Las clases siguientes implementan una **jerarquía de herencia**

```
class Base {
    String metodo1() {return "Base.metodo1()";}
    String metodo2(String s) {return "Base.metodo1(" + s + ")";}
}

public interface TipoI{
    String metodoIn2(String s);
    String metodoIn3();
}

class Derivada extends Base implements TipoI{
    public String metodoIn2(String s) {return "Derivada.metodoIn2()";}
    public String metodoIn3() {return "Derivada.metodoIn3()";}
    String metodo1() {return "Derivada.metodo1()";}
}

class Derivada2 extends Derivada{
    String metodo2 (String s) {return "Derivada2.metodo2(" + s + ")";}
    String metodo4() {return "Derivada2.metodo4()";}
}
```

Sea la clase `CuestionHerencia` que usa las anteriores:

```
public class CuestionHerencia{
    public static void main (String a[]){
        String tmp;
        Derivada derivada;
        Derivada2 derivada2;
        Base base;
        derivada2 = new Derivada2();
        base = derivada2;
        tmp = derivada2.metodo1();
        System.out.println("1.-"+tmp);
        tmp = derivada2.metodoIn2("EDA!!");
        System.out.println("2.-"+tmp);
        tmp = base.metodo1();
        System.out.println("3.-"+tmp);
        tmp = base.metodo2("EDA!!");
        System.out.println("4.-"+tmp);
        tmp = derivada2.metodoIn3();
        System.out.println("5.-"+tmp);
        tmp = derivada2.metodo4();
        System.out.println("6.-"+tmp);
        tmp = base.metodo3();
        System.out.println("7.-"+tmp);
        derivada = new Derivada();
        derivada2 = new Derivada2();
        base = new Base();
        Distinta ref = new Distinta();
        tmp = ref.prueba(derivada2);
        System.out.println("8.-"+tmp);
        tmp = ref.prueba(derivada);
        System.out.println("9.-"+tmp);
        tmp = ref.prueba(base);
        System.out.println("10.-"+tmp);
    }
}
```

Señalar los errores existentes.

Una vez corregido el programa, **escribir la salida** por pantalla resultado de su ejecución.

Escribe el programa corregido en el `main` de `CuestionHerencia` y las respuestas en comentarios.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

> **Para pasar los tests**
>
> Deja **comentadas** (`//`) las líneas que dan error de compilación y escribe al lado por qué fallan. Si alguna se queda sin comentar, no compila ningún ejercicio de la unidad y la nota es 0.

9.2.3 Actividades

1. (Facultad) Realizar una aplicación para la gestión de la información de las personas vinculadas a una `Facultad`, que se pueden clasificar en tres tipos: estudiantes, profesores y personal de servicio. A continuación, se detalla qué tipo de información debe gestionar esta aplicación:
   - Por cada `Personas`, se debe conocer, al menos, su `nombre` y `apellidos`, su número de identificación (`id`) y su `estadoCivil`.
   - Con respecto a los `Empleados`, sean del tipo que sean, hay que saber su año de incorporación (`anyoIncorporacion`) a la facultad y qué número de `despacho` tienen asignado.
   - En cuanto a los `Alumnos`, se requiere almacenar el `curso` en el que están matriculados.
   - Por lo que se refiere a los `Profesores`, es necesario gestionar a qué `departamento` pertenecen ("lenguajes", "matemáticas", "arquitectura", ...).
   - Sobre el `PersonalServicio`, hay que conocer a qué `sección` están asignados ("biblioteca", "decanato", "secretaría", ...).

   El ejercicio consiste, en primer lugar, en definir la jerarquía de clases de esta aplicación. A continuación, debe programar las clases definidas en las que, además de los constructores, hay que desarrollar los métodos correspondientes a las siguientes acciones: - Cambio del estado civil de una persona. - Reasignación de despacho a un empleado. - Matriculación de un estudiante en un nuevo curso. - Cambio de departamento de un profesor. - Traslado de sección de un empleado del personal de servicio. - Imprimir toda la información de cada tipo de individuo.

   En el método `main` crear un array de `Personas`. Crear diferentes instancias de las subclases e insertarlas en el array. Probar los diferentes métodos desarrollados.

   > **Para pasar los tests**
   >
   > - Constructores con todos los datos, en este orden: `Alumnos(nombre, apellidos, id, estadoCivil, curso)`, `Profesores(nombre, apellidos, id, estadoCivil, anyoIncorporacion, despacho, departamento)` y `PersonalServicio(nombre, apellidos, id, estadoCivil, anyoIncorporacion, despacho, seccion)`.
   > - El nombre de cada método de las acciones contiene el dato que cambia (`estadoCivil`, `despacho`, `curso`, `departamento`, `seccion`): por ejemplo, `setDespacho` o `cambiarDespacho`.
   > - La información se muestra con `toString()` o con un método `mostrar()` o `imprimir()`.

2. (Encargado) Crea una clase `Empleado` y una subclase `Encargado`. Los encargados reciben un 10% más de sueldo base que un empleado normal aunque realicen el mismo trabajo. Implementa dichas clases en el paquete objetos y sobrescribe el método `getSueldo()` para ambas clases.
3. (Sorteo) Crear la clase `Dado`, la cual desciende de la clase `Sorteo`. La clase `Dado`, en la llamada `lanzar()` mostrará un número aleatorio del 1 al 6. Crear la clase `Moneda`, la cual desciende de la clase `Sorteo`. Esta clase en la llamada al método `lanzar()` mostrará las palabras cara o cruz. Realizar una clase con un método `main` que compruebe todo lo realizado.

   > **Para pasar los tests**
   >
   > `Dado` y `Moneda` tienen un constructor sin parámetros. `lanzar()` puede mostrar el resultado o devolverlo.

   > **Extra: Sorteo con genéricos**
   >
   > ¿Tiene sentido crear un objeto `Sorteo` que no sea ni un dado ni una moneda? Piensa qué implica tu respuesta para la clase `Sorteo` y para su método `lanzar`. Después, intenta convertir `Sorteo` en una clase con genéricos, `Sorteo<E, T>`: ¿cómo sería entonces el método `lanzar` ?

4. (Huevo) Realiza una clase `Huevo` con un atributo `tamaño` (`S`, `M`, `L`, `XL`) con el método `toString`. La clase `Huevo` está compuesta por dos clases internas, una `Clara` y otra `Yema`. Ambas clases tiene un atributo `color` y el método `toString`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Realiza un método `main` en el que se cree un objeto de tipo `Huevo`, `Clara` y `Yema`. Se le asigne valor a sus atributos y se muestren dichos valores.

> **Para pasar los tests**
>
> Constructores: `Huevo(tamaño)`, `Clara(color)` y `Yema(color)`.

9.2.4 Ejercicios Lionel

Astros

Define una jerarquía de clases que permita almacenar datos sobre los planetas y satélites (lunas) que forman parte del sistema solar.

Algunos atributos que necesitaremos almacenar son:

- nombre
- radioEcuatorial
- rotacionEje
- masa
- temperaturaMedia
- gravedad
- etc.

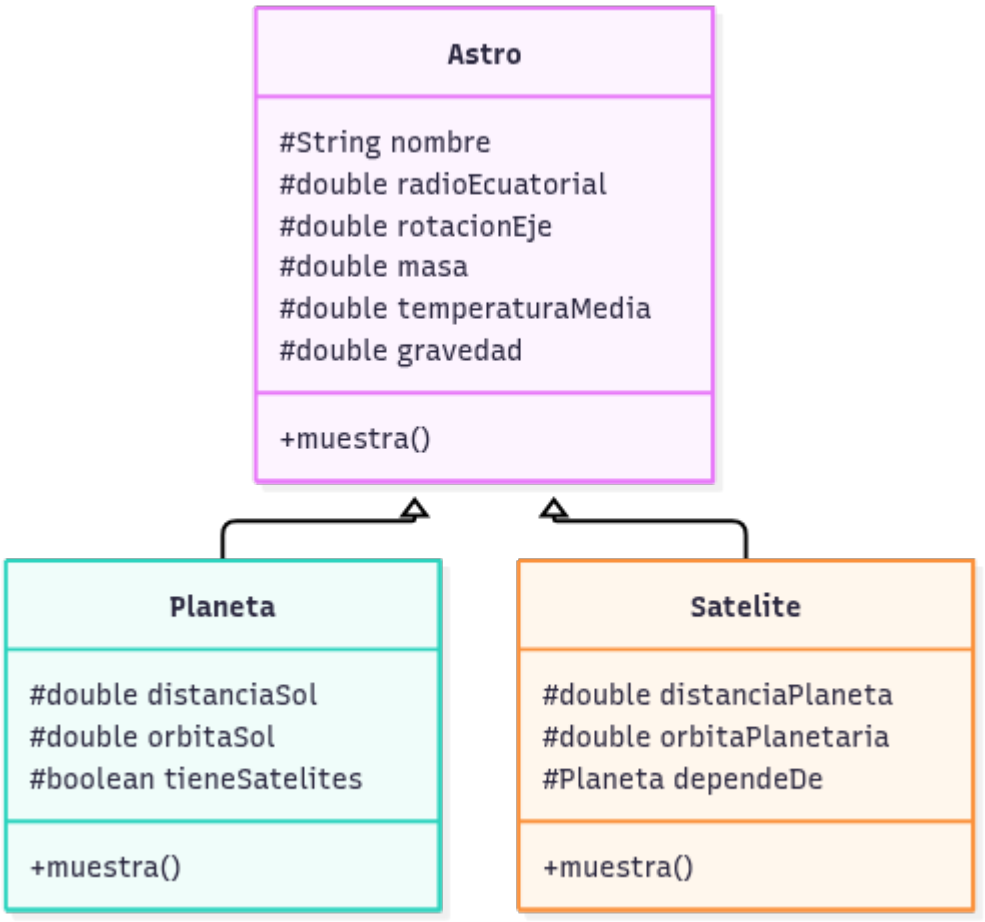
Define las clases necesarias conteniendo:

- Constructores.
- Métodos para recuperar y almacenas atributos.
- Método para mostrar la información del objeto.

Define un método, que dado un objeto del sistema solar (planeta o satélite), imprima toda la información que se dispone sobre el mismo (además de su lista de satélites si los tuviera).

El diagrama UML sería:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Una posible solución sería crear una lista de objetos, insertar los planetas y satélites (directamente mediante código o solicitándolos por pantalla) y luego mostrar un pequeño menú que permita al usuario imprimir la información del astro que elija.

Para pasar los tests

- Constructores con los datos en el orden del diagrama: `Planeta(nombre, radioEcuatorial, rotacionEje, masa, temperaturaMedia, gravedad, distanciaSol, orbitaSol)` y `Satelite(nombre, radioEcuatorial, rotacionEje, masa, temperaturaMedia, gravedad, distanciaPlaneta, orbitaPlanetaria, planeta)`, donde `planeta` es el `Planeta` del que depende.
- `muestra()` puede imprimir la información o devolverla.

Mascotas

Implementa una clase llamada **Inventario** que utilizaremos para almacenar referencias a todos los animales existentes en una tienda de mascotas.

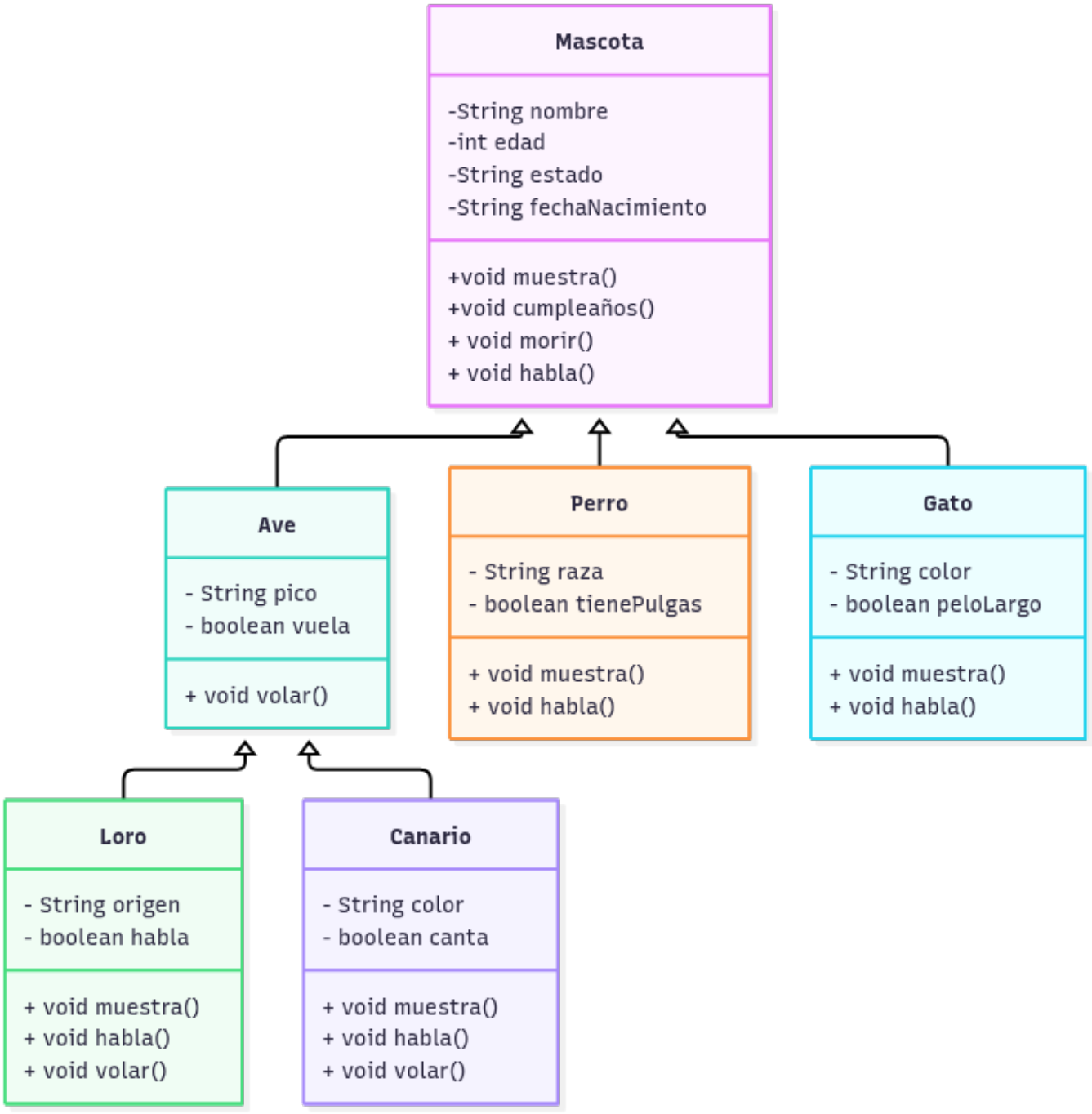
Esta clase debe cumplir con los siguientes requisitos:

- En la tienda existirán 4 tipos de animales: perros, gatos, loros y canarios.
- Los animales deben almacenarse en un `ArrayList` privado dentro de la clase **Inventario**.
  - La clase debe permitir realizar las siguientes acciones:
    - Mostrar la lista de animales (solo tipo y nombre, 1 línea por animal).
    - Mostrar todos los datos de un animal concreto.
    - Mostrar todos los datos de todos los animales.
    - Insertar animales en el inventario.
    - Eliminar animales del inventario.
    - Vaciar el inventario.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Implementa las demás clases necesarias para usar la clase `Inventario`.

El diagrama UML sería:



Para pasar los tests

- Constructores con todos los datos en el orden del diagrama: primero los de `Mascota`, después los de `Ave` (si es un ave) y al final los suyos. Por ejemplo, `Perro(nombre, edad, estado, fechaNacimiento, raza, tienePulgas)` o `Loro(nombre, edad, estado, fechaNacimiento, pico, vuela, origen, habla)`.
- Métodos de `Inventario`: `insertarAnimal`, `eliminarAnimal`, `vaciarInventario` y `mostrarListaAnimales`.

Banco

Vamos a hacer una aplicación que simule el funcionamiento de un banco.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Crea una clase **CuentaBancaria** con los atributos: **iban** y **saldo**. Implementa métodos para:

- Consultar los atributos.
- Ingresar dinero.
- Retirar dinero.
- Traspasar dinero de una cuenta a otra.

Para los tres últimos métodos puede utilizarse internamente un método privado más general llamado **añadir(...)** que añada una cantidad (positiva o negativa) al saldo.

También habrá un atributo común a todas las instancias llamado **interesAnualBasico**, que en principio puede ser constante: un **2,5 %**.

Cada tipo de cuenta tiene que tener un método **calcularIntereses()**, que calcula los intereses de un año de forma distinta según el tipo de cuenta.

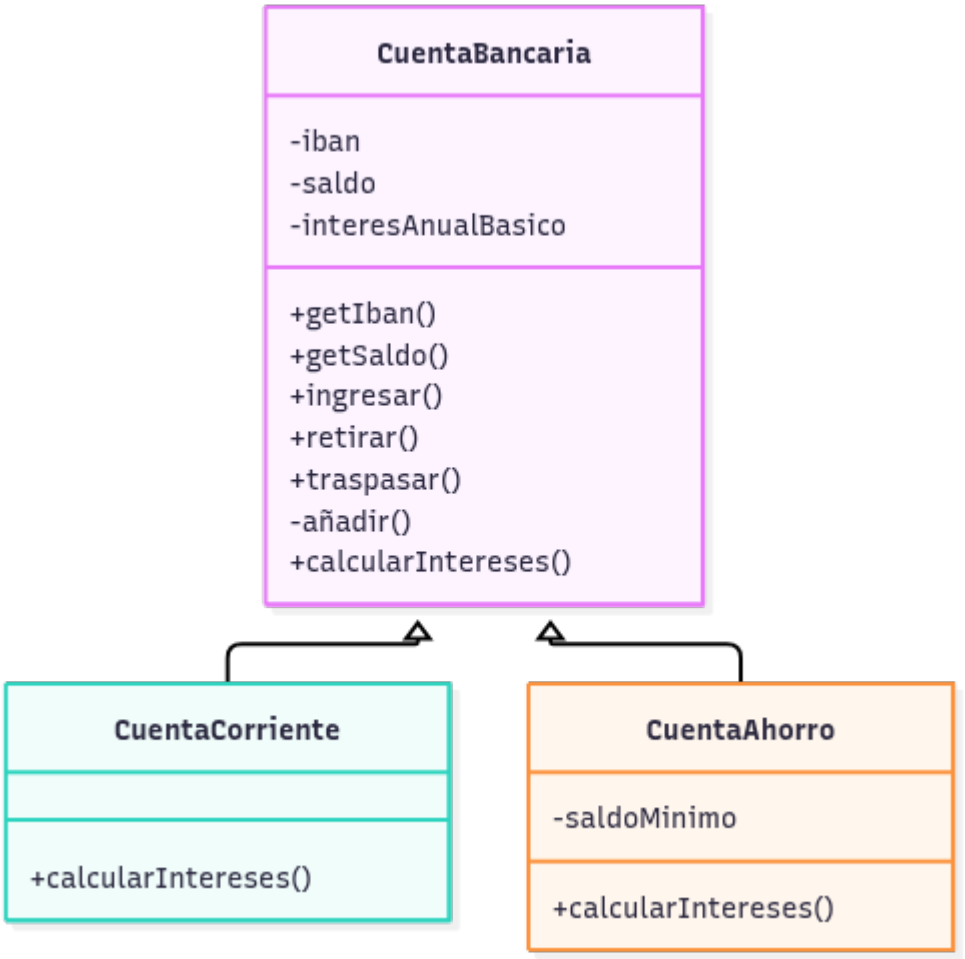
También puede ser útil implementar un método para mostrar los datos de la cuenta.

De esta clase heredarán dos subclases: **CuentaCorriente** y **CuentaAhorro**. La diferencia entre ambas será la manera de calcular los intereses:

- A la primera se le incrementará el saldo teniendo en cuenta el interés anual básico.
- La segunda tendrá una constante de clase llamada **saldoMinimo** (1000 €). Si no se llega a este saldo el interés será la mitad del interés básico. Si se supera el saldo mínimo el interés aplicado será el doble del interés anual básico.

Implementa una clase principal con función main para probar el funcionamiento de las tres clases: Crea varias cuentas bancarias de distintos tipos, pueden estar en un ArrayList si lo deseas; prueba a realizar ingresos, retiradas y transferencias; calcula los intereses y muéstralos por pantalla; etc.

El diagrama UML sería:



© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Para pasar los tests

- Constructores: `CuentaCorriente(iban, saldo)` y `CuentaAhorro(iban, saldo)`.
- Métodos `getSaldo()`, `ingresar(cantidad)`, `retirar(cantidad)` y `traspasar`, que mueve dinero de una cuenta a otra (los parámetros los decides tú).
- `calcularIntereses()` puede sumar los intereses al saldo o devolverlos.

Empresa y empleados

Vamos a implementar dos clases que permitan gestionar datos de empresas y sus empleados.

Los **empleados** tienen las siguientes características:

- Un empleado tiene nombre, DNI, sueldo bruto (mensual), edad, teléfono y dirección.
- El nombre y DNI de un empleado no pueden variar.
- Es obligatorio que todos los empleados tengan al menos definido su nombre, DNI y el sueldo bruto. Los demás datos no son obligatorios.
- Será necesario un método para imprimir por pantalla la información de un empleado.
- Será necesario un método para calcular el sueldo neto de un empleado. El sueldo neto se calcula descontando del sueldo bruto un porcentaje que depende del IRPF. El porcentaje del IRPF depende del sueldo bruto anual del empleado (sueldo bruto x 12 pagas).

Sueldo bruto anual	IRPF
Inferior a 12.000 €	20%
De 12.000 a 25.000 €	30%
Más de 25.000 €	40%

Por ejemplo, un empleado con un sueldo bruto anual de 17.000 € tendrá un 30% de IRPF. Para calcular su sueldo neto mensual se descontará un 30% a su sueldo bruto mensual.

Las **empresas** tienen las siguientes características:

- Una empresa tiene nombre y CIF (datos que no pueden variar), además de teléfono, dirección y empleados. Cuando se crea una nueva empresa esta carece de empleados.
- Serán necesarios métodos para:
  - Añadir y eliminar empleados a la empresa.
  - Mostrar por pantalla la información de todos los empleados.
  - Mostrar por pantalla el DNI, sueldo bruto y neto de todos los empleados.
  - Calcular la suma total de sueldos brutos de todos los empleados.
  - Calcular la suma total de sueldos netos de todos los empleados.

Implementa las clases Empleado y Empresa con los atributos oportunos, un constructor, los getters/setters oportunos y los métodos indicados. Puedes añadir más métodos si lo ves necesario. Estas clases no deben realizar ningún tipo de entrada por teclado.

Implementa también una clase Programa con una función main para realizar pruebas: Crear una o varias empresas, crear empleados, añadir y eliminar empleados a las empresas, listar todos los empleados, mostrar el total de sueldos brutos y netos, etc.

Información

El IRPF realmente es más complejo pero se ha simplificado para no complicar demasiado este ejercicio.

Vehículos

Es muy aconsejable hacer el diseño UML antes de empezar a programar.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Debes crear varias clases para un software de una empresa de transporte. Implementa la jerarquía de clases necesaria para cumplir los siguientes criterios:

- Los vehículos de la empresa de transporte pueden ser terrestres, acuáticos y aéreos. Los vehículos terrestres pueden ser coches y motos. Los vehículos acuáticos pueden ser barcos y submarinos. Los vehículos aéreos pueden ser aviones y helicópteros.
- Todos los vehículos tienen matrícula y modelo (datos que no pueden cambiar). La matrícula de los coches terrestres deben estar formadas por 4 números y 3 letras. La de los vehículos acuáticos por entre 3 y 10 letras. La de los vehículos aéreos por 4 letras y 6 números.
- Los vehículos terrestres tienen un número de ruedas (dato que no puede cambiar).
- Los vehículos acuáticos tienen eslora (dato que no puede cambiar).
- Los vehículos aéreos tienen un número de asientos (dato que no puede cambiar).
- Los coches pueden tener aire acondicionado o no tenerlo.
- Las motos tienen un color.
- Los barcos pueden tener motor o no tenerlo.
- Los submarinos tienen una profundidad máxima.
- Los aviones tienen un tiempo máximo de vuelo.
- Los helicópteros tienen un número de hélices.
- No se permiten vehículos genéricos, es decir, no se deben poder instanciar objetos que sean vehículos sin más. Pero debe ser posible instanciar vehículos terrestres, acuáticos o aéreos genéricos (es decir, que no sean coches, motos, barcos, submarinos, aviones o helicópteros).
- El diseño debe obligar a que todas las clases de vehículos tengan un método imprimir() que imprima por pantalla la información del vehículo en una sola línea.

Implementa todas las clases necesarias con: atributos, constructor con parámetros, getters, setters (solo de los datos que pueden cambiar) y el método imprimir. Utiliza **abstracción** y **herencia** de la forma más apropiada.

Implementa también una clase Programa para hacer algunas pruebas: Instancia varios vehículos de todo tipo (coches, motos, barcos, submarinos, aviones y helicópteros) así como vehículos genericos (terrestres, acuáticos y aéreos). Crea un ArrayList y añade todos los vehículos. Recorre la lista y llama al método imprimir de todos los vehículos.

Figuras

Implementa una **interface** llamada **iFigura2D** que declare los métodos:

- `double perimetro()`: Para devolver el perímetro de la figura
- `double area()`: Para devolver el área de la figura
- `void escalar(double escala)`: Para escalar la figura (aumentar o disminuir su tamaño). Solo hay que multiplicar los atributos de la figura por la escala (> 0).
- `void imprimir()`: Para mostrar la información de la figura (atributos, perímetro y área) en una sola línea.

Existen 4 tipos de figuras.

- **Cuadrado**: Sus cuatro lados son iguales.
- **Rectángulo**: Tiene ancho y alto.
- **Triángulo**: Es un triángulo rectángulo: su ancho y su alto son los catetos.
- **Círculo**: Tiene radio.

Crea las 4 clases de figuras de modo que implementen la interface iFigura2D. Define sus métodos.

Crea una clase ProgramaFiguras con un main en el que realizar las siguientes pruebas:

1. Crea un ArrayList figuras.
2. Añade figuras de varios tipos.
3. Muestra la información de todas las figuras.
4. Escala todas las figuras con escala = 2.
5. Muestra de nuevo la información de todas las figuras.
6. Escala todas las figuras con escala = 0.1.
7. Muestra de nuevo la información de todas las figuras.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Para pasar los tests

Constructores: `Cuadrado(lado)`, `Rectangulo(ancho, alto)`, `Triangulo(ancho, alto)` y `Circulo(radio)`.

29 de octubre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

9.3 Talleres

9.3.1 Taller UD08_01: Classroom 50

Aceptar la tarea en Classroom 50

Aceptamos el *Assignement* (la tarea/ejercicio) a partir del link del profesor, en este caso: https://classroom50.org/IES-Eduardo-Primo-Marques/1dam-programacio-java/assignments/ud08

Si no recuerdas cómo aceptar la tarea, clonar el repositorio o abrirlo en IntelliJ, repasa la guía completa en el taller de UD02.

Tarea

Debes enviar tus soluciones a Classroom 50 y superar al menos la mitad de los tests, cuantos más tests superados, mejor nota tendrás en la tarea.

6 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

9.3.2 Taller UD08_02: Librerias Maven vs Jar

Introducción

En el desarrollo de software, es común que nuestros proyectos requieran funcionalidades específicas que no están incluidas en el lenguaje de programación o en las bibliotecas estándar. Por ejemplo, podríamos necesitar trabajar con archivos JSON, conectarnos a una base de datos, o realizar operaciones matemáticas complejas. En lugar de reinventar la rueda y escribir todo el código desde cero, podemos aprovechar bibliotecas externas que ya han sido desarrolladas y probadas por la comunidad.

Estas bibliotecas nos permiten ahorrar tiempo, reducir errores y enfocarnos en la lógica específica de nuestra aplicación. Sin embargo, para utilizar estas bibliotecas, es necesario agregarlas a nuestro proyecto. Aquí es donde entra en juego la decisión de si añadir manualmente un archivo JAR o utilizar una herramienta de gestión de dependencias como Maven.

Ambas opciones tienen sus ventajas y desventajas, y entender cuándo y cómo usarlas es fundamental para un desarrollo eficiente y mantenible.

Añadir un `JAR` Manualmente:

Cuando añades un JAR manualmente, estás descargando el archivo JAR de la biblioteca que necesitas y lo agregas directamente a tu proyecto. Esto implica que tú eres responsable de gestionar las dependencias, incluyendo la descarga de versiones compatibles y la resolución de conflictos entre bibliotecas.

- **Proceso en IntelliJ IDEA:**
  a. Descargas el JAR de la biblioteca que necesitas.
  b. En IntelliJ, vas a `File > Project Structure > Libraries`.
  c. Haces clic en el botón `+` y seleccionas "Java".
  d. Navegas hasta la ubicación del JAR y lo agregas.
  e. Asegúrate de que el JAR esté en el classpath de tu proyecto.
- **Ventajas:**
  - Control total sobre las bibliotecas que se utilizan.
- **Desventajas:**
  - Gestión manual de dependencias.
  - Dificultad para mantener actualizadas las bibliotecas.
  - Posibles conflictos de versiones.

Usar `Maven`:

Maven es una herramienta de gestión y comprensión de proyectos, principalmente utilizada en proyectos Java. Su principal función es gestionar las dependencias (librerías y otros componentes que tu proyecto necesita), construir el proyecto y gestionar la configuración del ciclo de vida del desarrollo. Maven utiliza un archivo de configuración denominado `pom.xml` (Project Object Model), donde se especifican las dependencias y otros detalles del proyecto.

- **Proceso en IntelliJ IDEA:**
  a. Creas un proyecto Maven en IntelliJ.
  b. En el archivo `pom.xml`, defines las dependencias que necesitas.
  c. IntelliJ automáticamente descarga las dependencias y las añade al classpath del proyecto.
- **Ventajas:**
  - **Gestión Automática de Dependencias:** Maven descarga automáticamente las dependencias del proyecto desde repositorios remotos y las incluye en el proyecto, evitando la necesidad de descargar y agregar manualmente los archivos JAR.
  - **Estandarización del Proyecto:** Proporciona una estructura estándar para los proyectos, lo que facilita la organización y el mantenimiento.
  - **Reproducibilidad:** El uso de `pom.xml` permite que cualquier desarrollador que clone el repositorio tenga exactamente las mismas versiones de las dependencias, asegurando que el proyecto se construya de manera consistente en diferentes entornos.
  - **Integración con IDEs:** Herramientas como IntelliJ IDEA tienen soporte nativo para Maven, lo que facilita la configuración y gestión del proyecto dentro del IDE.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

    - **Gestión del Ciclo de Vida del Proyecto:** Maven puede automatizar tareas como compilación, pruebas, empaquetado y despliegue, facilitando la integración continua.
- **Desventajas:**
    - Curva de aprendizaje inicial.
    - Dependencia de la disponibilidad de los repositorios.

Comparativa

1. Gestión de Dependencias: en **Maven** las dependencias se especifican en el archivo `pom.xml`. Maven descarga las dependencias automáticamente desde los repositorios configurados. Mientras que en **Manual** las dependencias se descargan manualmente y se agregan al proyecto, lo que puede llevar a inconsistencias y errores si diferentes desarrolladores utilizan versiones distintas de las librerías.

2. Actualización de Dependencias: en **Maven** actualizar una dependencia es tan simple como cambiar la versión en el `pom.xml`. Maven se encarga de descargar la nueva versión. Por contra, en **Manual** requiere descargar la nueva versión manualmente y reemplazar los archivos JAR antiguos en el proyecto.

3. Estructura del Proyecto: en **Maven** proporciona una estructura de proyecto estandarizada, lo que facilita la comprensión y la navegación del proyecto. Mientras que en **Manual** la estructura del proyecto puede variar según el desarrollador, lo que puede llevar a confusión y desorganización.

4. Repositorios de Dependencias: en **Maven** utiliza repositorios remotos (como Maven Central) para buscar y descargar dependencias. También permite configurar repositorios internos. Pero en **Manual** las dependencias deben ser gestionadas y almacenadas manualmente, lo que puede ser engorroso y propenso a errores.

5. Integración con Herramientas de Construcción: Maven se integra fácilmente con herramientas de construcción y CI/CD, permitiendo la automatización de tareas como compilación, pruebas y despliegue. Por otro lado en **Manual** se requieren scripts personalizados y configuración adicional para integrar tareas de construcción y despliegue.

Actividades

EJERCICIO 1: AÑADIR UN JAR MANUALMENTE

Debes descargar el JAR de una biblioteca común, como `Gson` (para manejo de JSON), y añadirlo manualmente a un proyecto en IntelliJ.

1. **Pasos a seguir:**
    - Descargar el JAR de Gson desde aquí.
    - Crear un nuevo proyecto en IntelliJ.
    - Añadir el JAR de Gson al proyecto como se describió anteriormente.
    - Escribir un programa simple que convierta un objeto Java a JSON usando Gson.
2. **Código de Ejemplo:**

```java
import com.google.gson.Gson;

class Persona {
    String nombre;
    int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
     }
}

public class Main {
    public static void main(String[] args) {
        Gson gson = new Gson();
        String json = gson.toJson(new Persona("David", 35));
        System.out.println(json);
    }
}
```

EJERCICIO 2: USAR MAVEN

Ahora debes crear un proyecto `Maven` en IntelliJ y añadir la dependencia de `Gson` a través del `pom.xml`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

1. **Pasos:**
   - Crear un nuevo proyecto Maven en IntelliJ.
   - Añadir la dependencia de Gson en el `pom.xml`.
   - Escribir el mismo programa que convierte un objeto Java a JSON usando Gson.
2. **Código de Ejemplo en `pom.xml`:**

```xml
<dependencies>
    <dependency>
        <groupId>com.google.code.gson</groupId>
        <artifactId>gson</artifactId>
        <version>2.10.1</version>
    </dependency>
</dependencies>
```

3. **Código de Ejemplo en Java:**

```java
import com.google.gson.Gson;

class Persona {
    String nombre;
    int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }
}

public class Main {
    public static void main(String[] args) {
        Gson gson = new Gson();
        String json = gson.toJson(new Persona("David", 35));
        System.out.println(json);
    }
}
```

Tarea

Genera dos proyectos, uno con JAR y otro con MAVEN (Ejercicio1 y Ejercicio2), cambia el nombre de la persona y pon el tuyo, y también tu edad. Genera un zip con los dos proyectos de IntelliJ. Envía el archivo zip a la tarea de Aules.

16 de febrero de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

9.3.3 Taller UD08_T03: Introducción a JSON y YAML

¿Qué es JSON?

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos que es fácil de leer y escribir para los humanos, y fácil de parsear y generar para las máquinas. Es un formato basado en texto que utiliza una estructura de pares clave-valor, similar a los objetos en JavaScript.

¿Qué es YAML?

YAML (YAML Ain't Markup Language) es un formato de serialización de datos legible por humanos que se utiliza comúnmente para archivos de configuración y en aplicaciones donde los datos deben ser almacenados o transmitidos. YAML es más expresivo que JSON y permite comentarios, lo que lo hace más adecuado para configuraciones complejas.

Comparación entre JSON y YAML:

Característica	JSON	YAML
Legibilidad	Buena, pero menos expresivo	Muy buena, más expresivo
Comentarios	No soporta comentarios	Soporta comentarios
Estructura	Basado en pares clave-valor	Basado en indentación
Uso común	APIs, intercambio de datos	Configuraciones, DevOps
Complejidad	Más simple	Más flexible y complejo

Generación y Carga de JSON y YAML en Java

Para trabajar con `JSON` y `YAML` en Java, utilizaremos las bibliotecas **Jackson**.

> **Porqué `Jackson` y no `Gson` ?**
>
> Tanto **Jackson** como **Gson** son bibliotecas populares en Java para trabajar con **JSON** (serialización y deserialización). Ambas son ampliamente utilizadas, pero tienen diferencias en términos de rendimiento, funcionalidades, flexibilidad y facilidad de uso. - **Jackson**, desarrollada por **FasterXML**, es más potente, flexible y rápido, pero tiene una curva de aprendizaje más pronunciada. Es ideal para proyectos complejos o cuando necesitas trabajar con múltiples formatos (YAML, CSV, etc). - **Gson**, desarrollada por Google, es más sencilla y fácil de usar, pero menos flexible y potente. Es ideal para proyectos pequeños o cuando necesitas una solución rápida y ligera.

Para trabajar con `JSON` y `YAML` en Java usando **Jackson**, asegúrate de agregar las dependencias correctas en tu `pom.xml`. Jackson es una biblioteca poderosa y flexible que te permite manejar ambos formatos de manera eficiente.

Dependencias Maven:

```xml
<dependencies>
    <!-- https://mvnrepository.com/artifact/com.fasterxml.jackson.core/jackson-databind -->
    <!-- Dependencia para trabajar con JSON -->
    <dependency>
        <groupId>com.fasterxml.jackson.core</groupId>
        <artifactId>jackson-databind</artifactId>
        <version>2.15.2</version> <!-- Usa la versión más reciente -->
    </dependency>
    <!-- https://mvnrepository.com/artifact/com.fasterxml.jackson.dataformat/jackson-dataformat-yaml -->
    <!-- Dependencia para trabajar con YAML -->
    <dependency>
        <groupId>com.fasterxml.jackson.dataformat</groupId>
        <artifactId>jackson-dataformat-yaml</artifactId>
        <version>2.15.2</version> <!-- Usa la versión más reciente -->
    </dependency>
</dependencies>
```

Ejemplo Simple: Generación y Carga de JSON y YAML

Clase `Libro`:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
public class Libro {
    private String titulo;
    private String autor;
    private int anyoPublicacion;

    //Constructor por defecto, necesario para Jackson
    public Libro() {
    }

    // Constructor, getters y setters
    public Libro(String titulo, String autor, int anyoPublicacion) {
        this.titulo = titulo;
        this.autor = autor;
        this.anyoPublicacion = anyoPublicacion;
    }

    public String getTitulo() {
        return titulo;
    }

    public void setTitulo(String titulo) {
        this.titulo = titulo;
    }

    public String getAutor() {
        return autor;
    }

    public void setAutor(String autor) {
        this.autor = autor;
    }

    public int getAnyoPublicacion() {
        return anyoPublicacion;
    }

    public void setAnyoPublicacion(int anyoPublicacion) {
        this.anyoPublicacion = anyoPublicacion;
    }

    @Override
    public String toString() {
        return "Libro{" +
                "titulo='" + titulo + '\'' +
                ", autor='" + autor + '\'' +
                ", año Publicacion=" + anyoPublicacion +
                '}';
    }
}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.dataformat.yaml.YAMLFactory;

import java.io.File;
import java.io.IOException;
import java.util.ArrayList;

public class TestLibro {
    public static void main(String[] args) {
        // Crear una lista de libros
        ArrayList<Libro> libros = new ArrayList<>();
        libros.add(new Libro("El Principito", "Antoine de Saint-Exupéry", 1943));
        libros.add(new Libro("Cien Años de Soledad", "Gabriel García Márquez", 1967));
        libros.add(new Libro("1984", "George Orwell", 1949));

        // Generar JSON con Jackson
        ObjectMapper jsonMapper = new ObjectMapper(); // ObjectMapper de Jackson: permite leer y escribir JSON
        try {
            jsonMapper.writeValue(new File("libros.json"), libros);
        } catch (IOException ex) {
            System.out.println("Error al generar el archivo JSON.");
        }
        System.out.println("Archivo JSON generado con éxito.");

        // Cargar JSON con Jackson
        ArrayList<Libro> librosCargadosJson = null;
        try {
            librosCargadosJson = jsonMapper.readValue(new File("libros.json"), jsonMapper.getTypeFactory().constructCollectionType(ArrayList.class, Libr
o.class));
        } catch (IOException ex) {
            System.out.println("Error al cargar el archivo JSON.");
        }
        System.out.println("Libros cargados desde JSON:");
        if (librosCargadosJson != null) {
            for (Libro libro : librosCargadosJson) {
                System.out.println(libro);
            }
        } else {
            System.out.println("No se han cargado libros del JSON.");
        }

        // Generar YAML con Jackson
        ObjectMapper yamlMapper = new ObjectMapper(new YAMLFactory());
        try {
            yamlMapper.writeValue(new File("libros.yaml"), libros);
        } catch (IOException ex) {
            System.out.println("Error al generar el archivo YAML.");
        }
        System.out.println("Archivo YAML generado con éxito.");

        // Cargar YAML con Jackson
        ArrayList<Libro> librosCargadosYaml = null;
        try {
            librosCargadosYaml = yamlMapper.readValue(new File("libros.yaml"), jsonMapper.getTypeFactory().constructCollectionType(ArrayList.class, Libr
o.class));
        } catch (IOException ex) {
            System.out.println("Error al cargar el archivo YAML.");
        }
        System.out.println("Libros cargados desde YAML:");
        if (librosCargadosYaml != null) {
            for (Libro libro : librosCargadosYaml) {
                System.out.println(libro);
            }
        } else {
            System.out.println("No se han cargado libros del YAML.");
        }
    }
}
```

Resultado Esperado:

Archivo `libros.json`:

```json
[
  {
    "titulo": "El Principito",
    "autor": "Antoine de Saint-Exupéry",
    "anyoPublicacion": 1943
  },
  {
    "titulo": "Cien Años de Soledad",
    "autor": "Gabriel García Márquez",
    "anyoPublicacion": 1967
  },
  {
    "titulo": "1984",
    "autor": "George Orwell",
    "anyoPublicacion": 1949
  }
]
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Archivo `libros.yaml`:

```yaml
---
- titulo: "El Principito"
  autor: "Antoine de Saint-Exupéry"
  anyoPublicacion: 1943
- titulo: "Cien Años de Soledad"
  autor: "Gabriel García Márquez"
  anyoPublicacion: 1967
- titulo: "1984"
  autor: "George Orwell"
  anyoPublicacion: 1949
```

Salida en Consola:

```
Archivo JSON generado con éxito.
Libros cargados desde JSON:
Libro{titulo='El Principito', autor='Antoine de Saint-Exupéry', año Publicacion=1943}
Libro{titulo='Cien Años de Soledad', autor='Gabriel García Márquez', año Publicacion=1967}
Libro{titulo='1984', autor='George Orwell', año Publicacion=1949}
Archivo YAML generado con éxito.
Libros cargados desde YAML:
Libro{titulo='El Principito', autor='Antoine de Saint-Exupéry', año Publicacion=1943}
Libro{titulo='Cien Años de Soledad', autor='Gabriel García Márquez', año Publicacion=1967}
Libro{titulo='1984', autor='George Orwell', año Publicacion=1949}
```

Tarea

Genera un nuevo proyecto, **crea una clase** `Producto` con los siguientes atributos: - `nombre` (String) - `precio` (double) - `stock` (int)

Crear otra clase `TestProducto` **con una lista de objetos** `Producto` con al menos 3 productos.

Genera y guarda la lista en un archivo JSON (`productos.json`) y en un archivo YAML (`productos.yaml`).

Carga y muestra los datos desde ambos archivos.

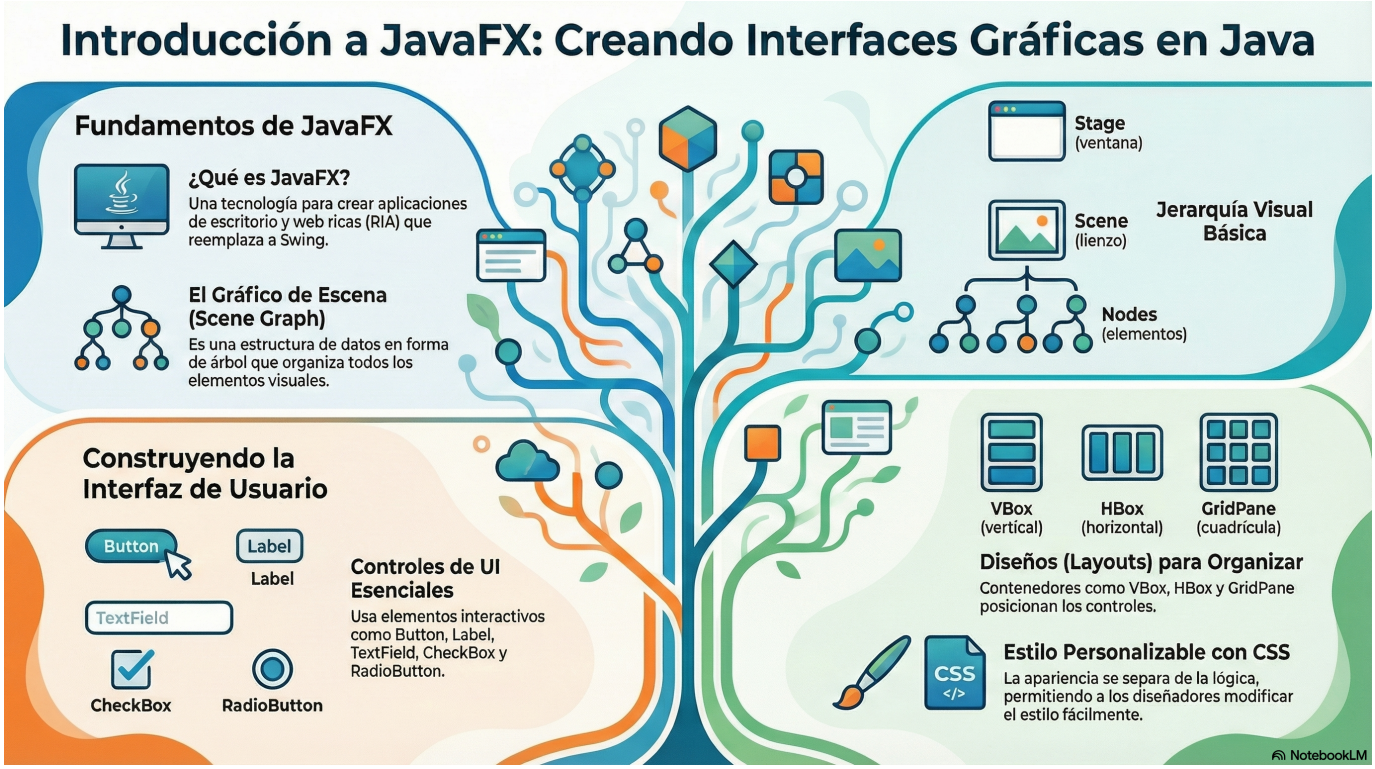
Genera un zip con el proyecto de IntelliJ. Envía el archivo zip a la tarea de Aules.

16 de febrero de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

10. UD09

10.1 Interfaz gráfica



Al finalizar esta unidad serás capaz de...

- Conocer la arquitectura de JavaFX (Stage, Scene, Node)
- Crear interfaces gráficas básicas con JavaFX
- Utilizar contenedores de layout (StackPane, VBox, HBox, etc.)
- Manejar eventos de usuario (botones, teclado, ratón)
- Diseñar interfaces con Scene Builder y FXML
- Separar la lógica de negocio de la interfaz
- Crear proyectos JavaFX con Maven

10.1.1 Introducción

JavaFX fue desarrollado por Chris Oliver. Inicialmente, el proyecto se denominó Form Follows Functions (F3). Está destinado a proporcionar las funcionalidades más ricas para el desarrollo de aplicaciones GUI. Posteriormente, Sun Micro-systems adquirió el proyecto F3 como JavaFX en junio de 2005.

Sun Micro-systems lo anuncia oficialmente en 2007 en la Conferencia W3. En octubre de 2008, se lanzó JavaFX 1.0. En 2009, la corporación ORACLE adquiere Sun Micro-Systems y lanzó JavaFX 1.2. la última versión de JavaFX es JavaFX 23.

JavaFX es una tecnología que nos permite crear aplicaciones de escritorio RIA (Ritch Internet Applications), esto es, aplicaciones web que tienen las características y capacidades de aplicaciones de escritorio, incluyendo aplicaciones multimedia interactivas que pueden ejecutarse en una amplia variedad de dispositivos. JavaFX está destinado a reemplazar a Swing como la biblioteca de GUI estándar para Java SE.

La biblioteca de JavaFX está escrita como una API de Java, las aplicaciones JavaFX pueden hacer referencia a APIs de código de cualquier biblioteca Java. Por ejemplo, las aplicaciones JavaFX pueden utilizar las bibliotecas de API de Java para acceder a las capacidades del sistema nativas y conectarse a aplicaciones de middleware basadas en servidor.

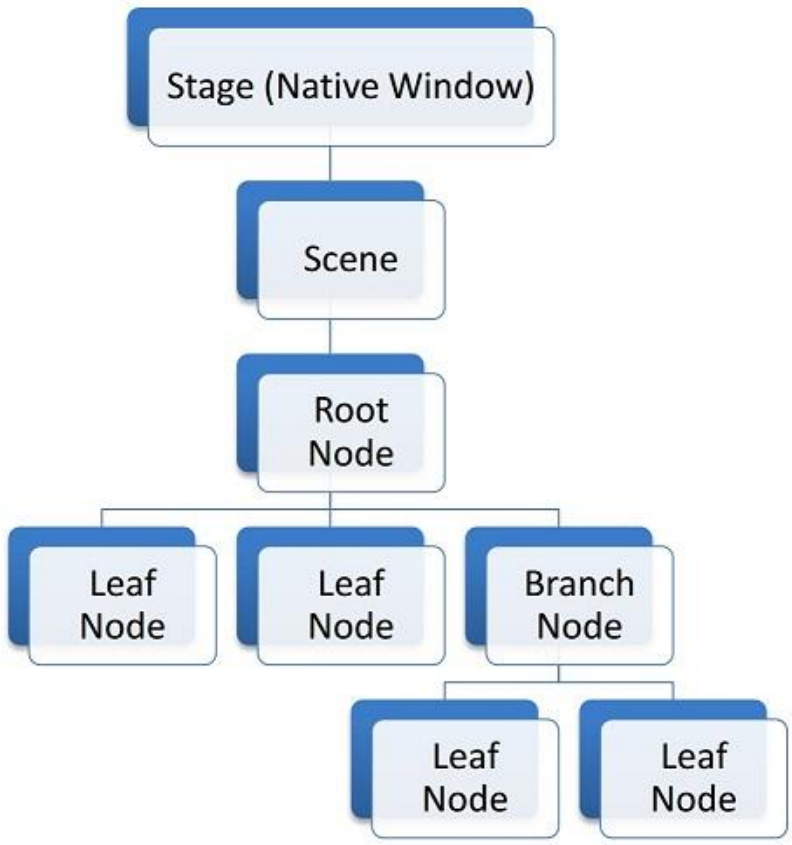
© 2025 David Martínez licensed under CC BY-NC-SA 4.0

La apariencia de las aplicaciones JavaFX se pueden personalizar. Las Hojas de Estilo en Cascada (CSS) separan la apariencia y estilo de la lógica de la aplicación para que los desarrolladores puedan concentrarse en el código. Los diseñadores gráficos pueden personalizar fácilmente el aspecto y el estilo de la aplicación a través de CSS. Si se tiene un diseño de fondo de la web, o si se desea separar la interfaz de usuario (UI) y la lógica de servidor, entonces, se pueden desarrollar los aspectos de la presentación de la interfaz de usuario en el lenguaje de scripting FXML y utilizar el código de Java para la aplicación lógica. Si se prefiere diseñar interfaces de usuario sin necesidad de escribir código, entonces, utilizaremos JavaFXSceneBuilder . Al diseñar la interfaz de usuario con javaFX Scene Builder el crea código de marcado FXML que puede ser portado a un entorno de desarrollo integrado (IDE) de forma que los desarrolladores pueden añadir la lógica de negocio.

10.1.2 Gráfico de escena

Descripción general

Un gráfico de escena es una estructura de datos de árbol que organiza (y agrupa) objetos gráficos para una representación lógica más sencilla. También permite que el motor de gráficos represente los objetos de la manera más eficiente al omitir total o parcialmente los objetos que no se verán en la imagen final. La siguiente figura muestra un ejemplo de la arquitectura del gráfico de escena JavaFX.



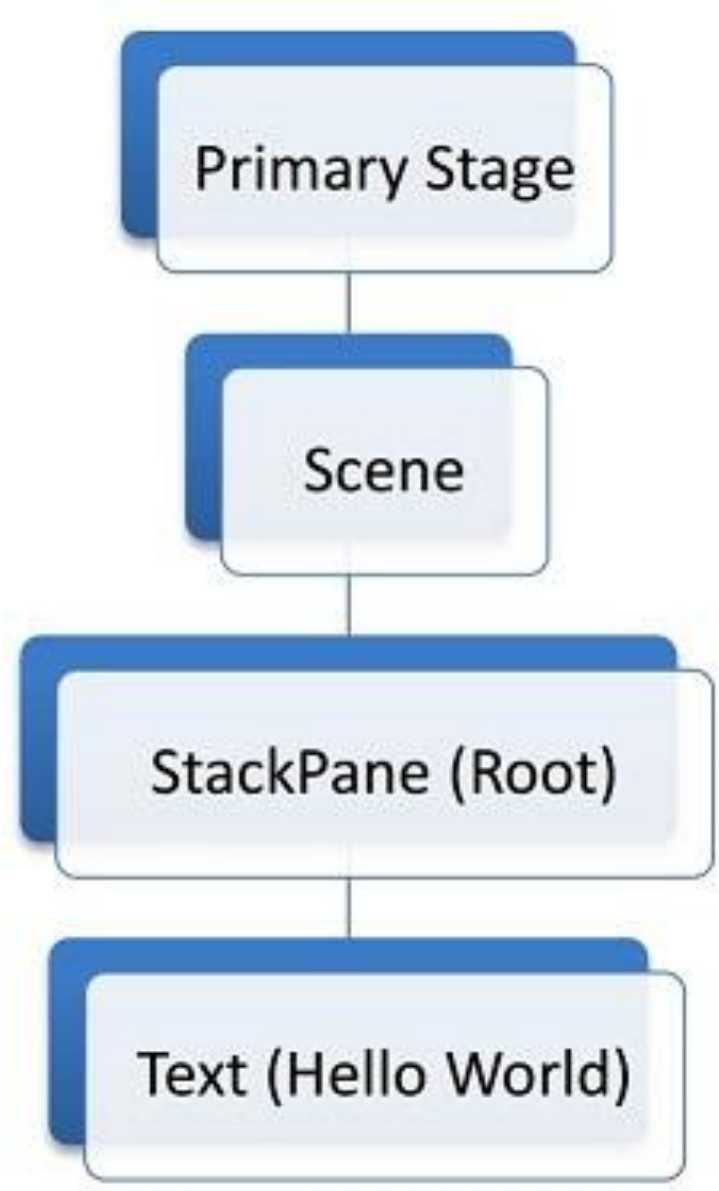
En la parte superior de la arquitectura hay un `Stage`. Una etapa es una representación JavaFX de una ventana de sistema operativo nativo. En un momento dado, un escenario puede tener un solo `Scene` adjunto. Una escena es un contenedor para el gráfico de escena JavaFX.

Todos los elementos en el gráfico de escena JavaFX se representan como `Node` objetos. Hay tres tipos de nudos: raíz, rama y hoja. El nodo raíz es el único nodo que no tiene un padre y está contenido directamente en una escena, que se puede ver en la figura anterior. La diferencia entre una rama y una hoja es que un nodo hoja no tiene hijos.

En el gráfico de escena, los nodos secundarios comparten muchas propiedades de un nodo principal. Por ejemplo, una transformación o un evento aplicado a un nodo padre también se aplicará recursivamente a sus hijos. Como tal, una jerarquía compleja de nodos se puede ver como un solo nodo para simplificar el modelo de programación. Exploraremos transformaciones y eventos en secciones posteriores.

En la siguiente figura se puede ver un ejemplo de un gráfico de escena "Hola Mundo".

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

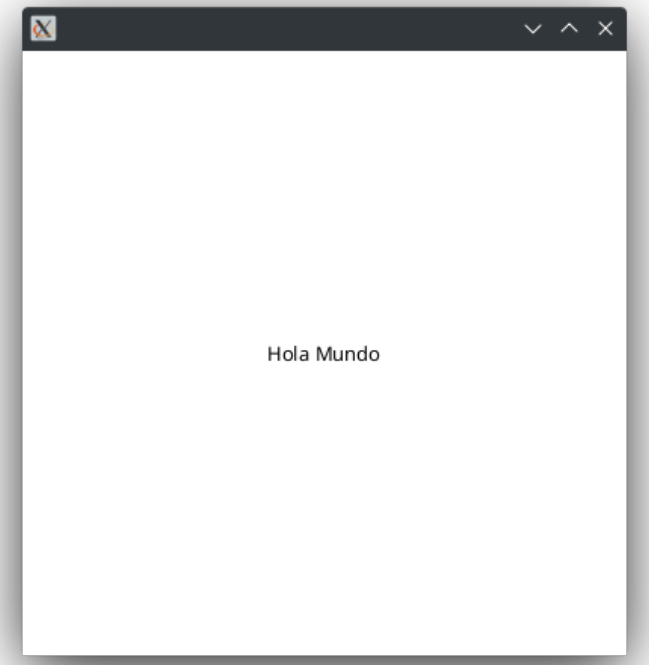


Una posible implementación que producirá un gráfico de escena que coincida con la figura anterior es la siguiente.

Consulta el código completo de E01_HolaMundo.

El resultado de ejecutar el código se ve en la siguiente figura.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Notas importantes:

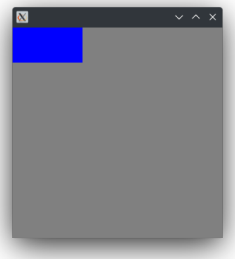
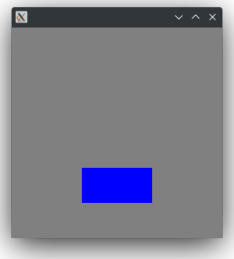
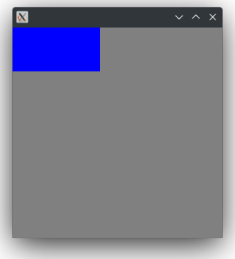
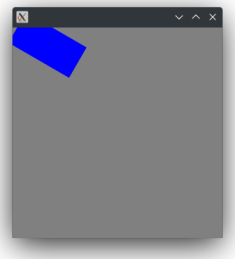
- Un nodo puede tener como máximo 1 padre.
- Un nodo en el gráfico de escena "activo" (adjunto a una escena actualmente visible) solo se puede modificar desde el subproceso de la aplicación JavaFX.

Transformaciones

Usaremos la siguiente aplicación como ejemplo para demostrar las 3 transformaciones más comunes.

Consulta el código completo de E02_TransformApp.

Ejecutar la aplicación dará como resultado la primera imagen (y si descomentamos por grupos, las siguientes).

Sin transformaciones	Traslación	Escalado	Rotación
			

En JavaFX, puede ocurrir una transformación simple en uno de los 3 ejes: X, Y o Z. La aplicación de ejemplo está en 2D, por lo que solo consideraremos los ejes X e Y.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

TRASLACIÓN

En JavaFX y gráficos por computadora, `translate` significa moverse. Podemos trasladar nuestra caja en 100 píxeles en el eje X y 200 píxeles en el eje Y.

```
box.setTranslateX(100);
box.setTranslateY(200);
```

ESCALA

Puede aplicar la escala para hacer un nodo más grande o más pequeño. El valor de escala es una relación. Por defecto, un nodo tiene un valor de escala de 1 (100%) en cada eje. Podemos agrandar nuestra caja aplicando una escala de 1.5 en los ejes X e Y.

```
box.setScaleX(1.5);
box.setScaleY(1.5);
```

ROTACIÓN

La rotación de un nodo determina el ángulo en el que se representa el nodo. En 2D el único eje de rotación sensible es el eje Z. Giremos la caja 30 grados.

```
box.setRotate(30);
```

Manejo de eventos

Un evento notifica que ha ocurrido algo importante. Los eventos suelen ser lo "primitivo" de un sistema de eventos (también conocido como bus de eventos). Generalmente, un sistema de eventos tiene las siguientes 3 responsabilidades:

- `fire` (desencadenar) un evento,
- notificar `listeners` (a las partes interesadas) sobre el evento y
- `handle` (procesar) el evento.

El mecanismo de notificación de eventos lo realiza la plataforma JavaFX automáticamente. Por lo tanto, solo consideraremos cómo disparar (`fire`) eventos, escuchar (`listen`) eventos y cómo manejarlos (`handle`).

Primero, vamos a crear un evento personalizado.

Consulta el código completo de E03_EventoUsuario.

Dado que los tipos de eventos son fijos, generalmente se crean dentro del mismo archivo de origen que el evento. Podemos ver que hay 2 tipos específicos de eventos: `LOGIN_SUCCEEDED` y `LOGIN_FAILED` . Podemos escuchar estos tipos específicos de eventos:

```
Node node = ...
node.addEventHandler(UserEvent.LOGIN_SUCCEEDED, event -> {
    // handle event
});
```

Alternativamente, podemos manejar cualquier `UserEvent` :

```
Node node = ...
node.addEventHandler(UserEvent.ANY, event -> {
    // handle event
});
```

Finalmente, podemos construir y disparar nuestros propios eventos:

```
UserEvent event = new UserEvent(UserEvent.LOGIN_SUCCEEDED);
Node node = ...
node.fireEvent(event);
```

Por ejemplo, `LOGIN_SUCCEEDED` o `LOGIN_FAILED` podría activarse cuando un usuario intenta iniciar sesión en una aplicación. Según el resultado del inicio de sesión, podemos permitir que el usuario acceda a la aplicación o bloquearlo. Si bien se puede lograr la misma funcionalidad con una `if` declaración simple, hay una ventaja significativa de un sistema de eventos. Los sistemas de eventos se diseñaron para permitir la comunicación entre varios módulos (subsistemas) en una aplicación sin acoplarlos estrechamente. Como tal, un sistema de audio puede reproducir un sonido cuando el usuario inicia sesión. Por lo tanto, mantiene todo el código relacionado con el audio en su propio módulo. Sin embargo, no profundizaremos en los estilos arquitectónicos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

EVENTOS DE ENTRADA

Los eventos de teclado y ratón son los tipos de eventos más comunes utilizados en JavaFX. Cada `Node` proporciona los llamados "métodos de conveniencia" para manejar estos eventos. Por ejemplo, podemos ejecutar algún código cuando se presiona un botón:

```
Button button = ...
button.setOnAction(event -> {
    // button was pressed
});
```

Para mayor flexibilidad también podemos usar lo siguiente:

```
Button button = ...
button.setOnMouseEntered(e -> ...);
button.setOnMouseExited(e -> ...);
button.setOnMousePressed(e -> ...);
button.setOnMouseReleased(e -> ...);
```

El objeto `e` anterior es de tipo `MouseEvent` y se puede consultar para obtener información diversa sobre el evento, por ejemplo, `x` posiciones `y`, número de clics, etc. Finalmente, podemos hacer lo mismo con las teclas:

```
Button button = ...
button.setOnKeyPressed(e -> ...);
button.setOnKeyReleased(e -> ...);
```

El objeto `e` aquí es de tipo `KeyEvent` y lleva información sobre el código de la tecla, que luego se puede asignar a una tecla física real en el teclado.

Veremos más ejemplos más adelante en este tema.

10.1.3 Controles de la interfaz de usuario

`Label`

La clase `Label` que reside en el paquete `javafx.scene.control` de la API de JavaFX se puede usar para mostrar un elemento de texto.

La etiqueta de la izquierda es un elemento de texto de color azul, que se auto-ajusta si cambiamos el tamaño de la ventana, la etiqueta del centro representa texto girado y la etiqueta de la derecha representa un texto con imagen, que además aumenta su tamaño cuando pasamos por encima.

La API de JavaFX proporciona tres constructores de la clase `Label` para crear etiquetas en su aplicación.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
//Creamos la etiqueta vacia
Label label1 = new Label();

//Creamos la etiqueta con texto
Label label2 = new Label("Etiqueta2");

//Creamos la etiqueta con imágen
Image image = new Image("UD09/label.png");
Label label3 = new Label("Search", new ImageView(image));
```

Una vez que haya creado una etiqueta, puede cambiar sus propiedades con los métodos;

- `setText(String text)` especifica el texto para la etiqueta.
- `setGraphic(Node graphic)` : especifica el icono gráfico,
- `setTextFill` especifica el color para pintar el elemento de texto de la etiqueta.
- `setGraphicTextGap` para establecer el espacio entre ellos.
- `setWrapText` para indicar si debe autoajustarse (`true`) o no (`false`)
- `setTextAlignment` puede variar la posición del contenido de la etiqueta dentro de su área de diseño
- `setContentDisplay` puede definir la posición del gráfico en relación con el texto aplicando el método y especificando una de las siguientes constantes `ContentDisplay` : `LEFT` , `RIGHT` , `CENTER` , `TOP` , `BOTTOM` .

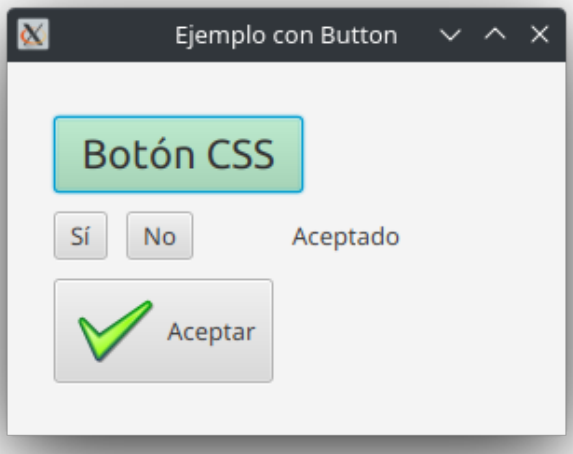
Cuando se activa el evento `MOUSE_ENTERED` en la etiqueta, se establece el factor de escala de 1,5 para los métodos `setScaleX` y `setScaleY` . Cuando un usuario mueve el cursor del ratón fuera de la etiqueta y ocurre el evento `MOUSE_EXITED` , el factor de escala se establece en 1.0 y la etiqueta se representa en su tamaño original.

```
label3.setOnMouseEntered((MouseEvent event) -> {
    label3.setScaleX(1.5);
    label3.setScaleY(1.5);
});

label3.setOnMouseExited((MouseEvent event) -> {
    label3.setScaleX(1);
    label3.setScaleY(1);
});
```

Consulta el código completo de E05_Label.

`Button`

La clase `Button` disponible a través de la API de JavaFX permite a los desarrolladores procesar una acción cuando un usuario hace clic en un botón. La clase `Button` es una extensión de la clase `Labeled` . Puede mostrar texto, una imagen o ambos.



© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Puede crear un control `Button` en una aplicación JavaFX usando tres constructores de la clase Button como se muestra a continuación

```
//Creamos el botón vacio
Button button1 = new Button();
//Creamos el botón con texto
Button button2 = new Button("Sí");
//Creamos el botón con texto e imágen
Image image = new Image("UD09/ok.png");
Button button4 = new Button("Aceptar", new ImageView(image));
```

La clase `Button` puede usar los siguientes métodos:

- `setText(String text)`: especifica el título de texto para el botón
- `setGraphic(Node graphic)`: especifica el icono gráfico
- `setOnAction`: La función principal de cada botón es producir una acción cuando se hace clic en él. En nuestro ejemplo cambiar el texto de un label.

```
button2.setOnAction((ActionEvent e) -> {
    label.setText("Aceptado");
});
```

Vemos cómo procesar un `ActionEvent`, de modo que cuando un usuario presiona button2 el título de texto de la etiqueta label se establece en "Aceptado".

Puede usar la clase `Button` para establecer tantos métodos de manejo de eventos como necesite para causar el comportamiento específico o aplicar efectos visuales.

Debido a que la clase `Button` amplía la clase `Node`, puede aplicar cualquiera de los efectos del paquete `javafx.scene.effect` para mejorar la apariencia visual del botón. En este caso añadimos una sobra al botón con imagen cuando pasamos el ratón sobre el.

```
//Creamos el estilo de sombra
DropShadow shadow = new DropShadow();
//Añadimos la sombra cuando pasamos sobre el botón
button4.addEventHandler(MouseEvent.MOUSE_ENTERED, (MouseEvent e) -> {
    button4.setEffect(shadow);
});

//Eliminamos la sombra al salir del botón
button4.addEventHandler(MouseEvent.MOUSE_EXITED, (MouseEvent e) -> {
    button4.setEffect(null);
});
```

El siguiente paso para mejorar la apariencia visual de un botón es aplicar estilos CSS definidos por la clase `Skin`. Usar CSS en aplicaciones JavaFX 2 es similar a usar CSS en HTML, porque cada caso se basa en la misma especificación de CSS.

Puede definir estilos en un archivo CSS separado y habilitarlos en la aplicación usando el método `getStyleClass`.

El fichero style.css:

```
.button1{
    -fx-font: 22 ubuntu;
    -fx-base: #b6e7c9;
}
```

La propiedad `-fx-font` establece el nombre y el tamaño de la fuente para el button1. La propiedad `-fx-base` anula el color predeterminado aplicado al botón.

Y desde java usamos:

```
//Asociamos el fichero css a nuestra escena
scene.getStylesheets().add("UD09/style.css");
[...]
//establecemos la clase correspondiente del css
button1.getStyleClass().add("button1");
```

Como resultado, el button1 es de color verde claro con un tamaño de texto mayor.

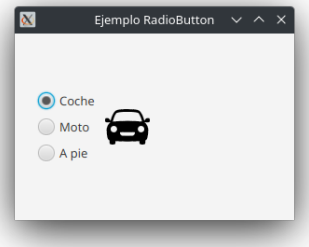
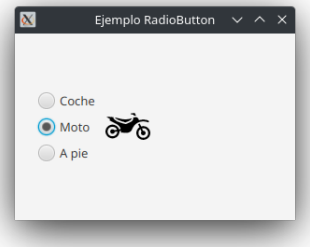
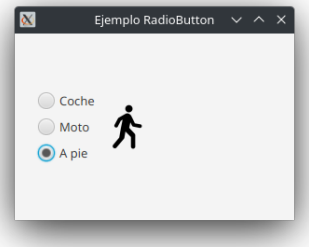
Consulta el código completo de E06_Button.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

`RadioButton`

Un control `RadioButton` se puede seleccionar o deseleccionar. Por lo general, se combinan en un grupo donde solo se puede seleccionar un botón a la vez.

Abajo se muestran tres capturas de pantalla del ejemplo `RadioButton`, en las que se agregan tres botones de opción a un grupo.

Opción 1	Opción 2	Opción 3
		

La clase `RadioButton` disponible en el paquete `javafx.scene.control` del SDK de JavaFX proporciona dos constructores con los que puede crear un botón de opción.

```
1  //Creamos el botón vacio
2  RadioButton rButton1 = new RadioButton();
3  //añadimos texto una vez creado
4  rButton1.setText("Coche");
5  //Creamos los RadioButton con texto
6  RadioButton rButton2 = new RadioButton("Moto");
```

Puede seleccionar explícitamente un botón de radio utilizando el método `setSelected` y especificando su valor como `true`. Si necesita verificar si un usuario seleccionó un botón de radio en particular, aplique el método `isSelected`.

Debido a que la clase `RadioButton` es una extensión de la clase `Labeled`, puede especificar no solo una leyenda de texto, sino también una imagen. Utilice el método `setGraphic` para especificar una imagen.

```
1  //Añadimos las imágenes a los Radio Button
2  ImageView imageCoche = new ImageView("UD09/coche.png");
3  rButton1.setGraphic(imageCoche);
```

Los `RadioButton` se utilizan normalmente en un grupo para presentar varias opciones mutuamente excluyentes. El objeto `ToggleGroup` proporciona referencias a todos los botones de radio asociados con él y los administra para que solo se pueda seleccionar uno de los botones de radio a la vez. A continuación se muestra como se crea un grupo de alternancia y especifica qué botón debe seleccionarse cuando se inicia la aplicación.

```
1  //Creamos el grupo de alternancia
2  final ToggleGroup grupo = new ToggleGroup();
3  rButton1.setToggleGroup(grupo);
4  rButton1.setSelected(true); //si queremos que la primera opción este marcada por defecto
5  rButton2.setToggleGroup(grupo);
6  rButton3.setToggleGroup(grupo);
```

Normalmente, la aplicación realiza una acción cuando se selecciona uno de los botones de opción del grupo. En este caso cambiará la imágen que acompaña al grupo de alternancia.

```
1  //Añadimos una imágen que cambiara al cambiar la selección
2  ImageView image = new ImageView();
3  grid.add(image, 1, 0, 1, 3);
4
5  //añadimos el listener al grupo para que capture el evento cuando se cambie la selección
6  grupo.selectedToggleProperty().addListener(
7          (ObservableValue<? extends Toggle> ov, Toggle old_toggle, Toggle new_toggle) -> {
8              if (grupo.getSelectedToggle() != null) {
9                  image.setImage(new Image("UD09/" + grupo.getSelectedToggle().getUserData().toString() + ".png"));
10             }
11         });
```

Los datos de usuario se asignaron para cada botón de opción. El objeto `ChangeListener<Toggle>` verifica un conmutador seleccionado en el grupo. Utiliza el método `getSelectedToggle` para identificar qué botón de opción está actualmente

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

seleccionado y extrae sus datos de usuario llamando al método `getUserData`. Luego, los datos del usuario se aplican para construir un nombre de archivo de imagen para cargar.

Por ejemplo, cuando se selecciona `rButton3`, el método `getSelectedToggle` devuelve "rButton3" y el método `getUserData` devuelve "coche" Por lo tanto, la imágen será "UD09/coche.png".

Consulta el código completo de E07_RadioButton.

CheckBox

Aunque las casillas de verificación se parecen a los `RadioButton`, no se pueden combinar en grupos de alternancia.

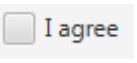
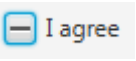
Más abajo se muestra una captura de pantalla de una aplicación en la que se usan tres casillas de verificación para habilitar o desactivar iconos en la barra de herramientas de una aplicación.

A continuación se crean tres casillas de verificación simples.

```
//Creamos el CheckBox vacio
CheckBox check1 = new CheckBox();
//añadimos texto una vez creado
check1.setText("Coche");

//Creamos los CheckBox con texto
CheckBox check2 = new CheckBox("Moto");

//Hademos aparezca marcado por defecto
CheckBox check3 = new CheckBox("A pie");
check3.setSelected(true);
```

Una vez que haya creado una casilla de verificación, puede modificarla utilizando los métodos disponibles a través de las API de JavaFX. Por ejemplo el método `setText` define el título de texto de la casilla de verificación `check1`. El método `setSelected` se establece en `true` para que la casilla de verificación `check3` se seleccione cuando se inicie la aplicación.

La casilla de verificación puede estar definida o indefinida. Cuando está definido, puede seleccionarlo o deseleccionarlo. Sin embargo, cuando la casilla de verificación está indefinida, no se puede seleccionar ni deseleccionar. Se usa una combinación de los métodos `setSelected` y `setIndeterminate` de la clase `CheckBox` para especificar el estado de la casilla de verificación. En la tabla se muestran tres estados de una casilla de verificación en función de sus propiedades `INDETERMINADO` y `SELECCIONADO`.

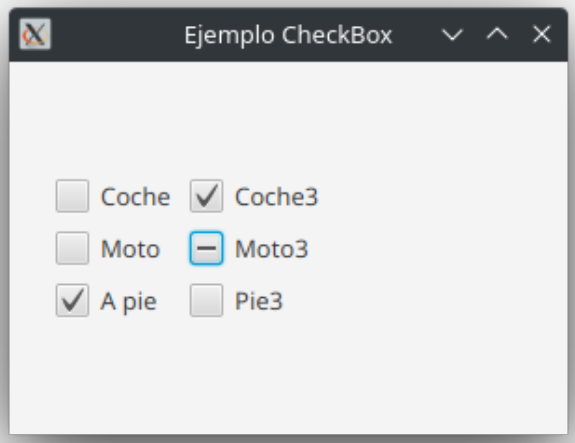
Valores de propiedad	Apariencia de la casilla de verificación
`INDETERMINADO` = **falso** `SELECCIONADO` = **falso**	
`INDETERMINADO` =**falso** `SELECCIONADO` = **verdadero**	
`INDETERMINADO` = **verdadero** `SELECCIONADO` = **verdadero/falso**	

Es posible que deba habilitar tres estados para las casillas de verificación en su aplicación cuando representan elementos de la interfaz de usuario que pueden estar en estados mixtos, por ejemplo, "Sí", "No", "No aplicable". La propiedad `allowIndeterminate` del objeto `CheckBox` determina si la casilla de verificación debe pasar por los tres estados: seleccionada, deseleccionada e indefinida. Si la variable es "verdadera", el control recorrerá los tres estados. Si es `falso`, el control recorrerá los estados seleccionado y deseleccionado. La aplicación descrita en la siguiente sección construye tres casillas de verificación y habilita solo dos estados para ellas.

```
//Ahora crearemos los 3 checkboxes en un bucle y tendran 3 estados
final String[] nombres = new String[]{"Coche3", "Moto3", "Pie3"};
final CheckBox[] checkBox = new CheckBox[nombres.length];

for (int i = 0; i < nombres.length; i++) {
    final CheckBox cb = checkBox[i] = new CheckBox(nombres[i]);
    cb.setAllowIndeterminate(true);
    grid.add(cb, 1, i);
}
```

En la siguiente imagen se puede observar como la columna derecha de checkbox permite los 3 estados:

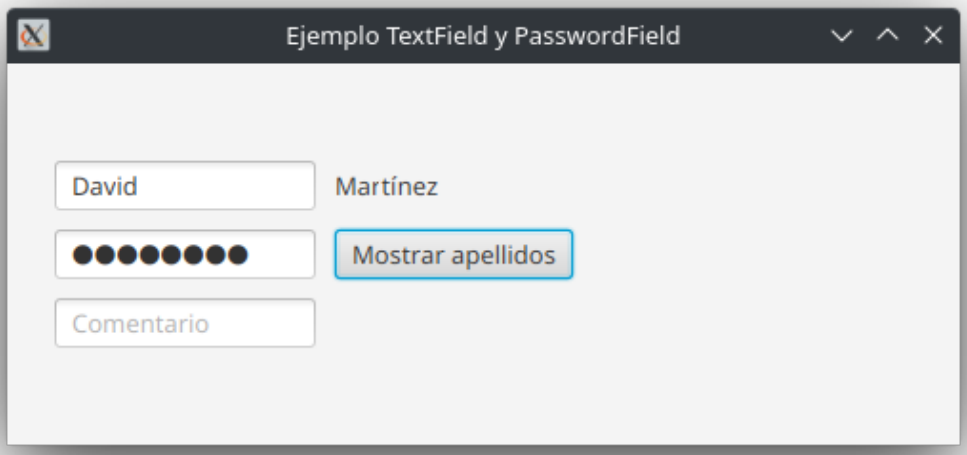
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Consulta el código completo de E08_CheckBox.

`TextField` y `PasswordField`

Ahora veremos los componentes para campos de texto, la clase `TextField` implementa un control de interfaz de usuario que acepta y muestra la entrada de texto. Junto con otro control de entrada de texto, `PasswordField`, esta clase amplía la clase `TextInput`, una superclase para todos los controles de texto disponibles a través de la API de JavaFX.



Puede aplicar el método `setPrefColumnCount` de la clase `TextInput` para establecer el tamaño del campo de texto, definido como el número máximo de caracteres que puede mostrar a la vez.

```
1  //Creamos el TextkField vacio
2  TextField textField1 = new TextField();
3  //Establecemos el número de caracteres que mostrará por defecto
4  textField1.setPrefColumnCount(10);
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Puede crear un campo de texto campo con un dato de texto particular en él. Para crear un campo de texto con el texto predefinido:

```
TextField tFComentario = new TextField();
//Establecemos el contenido por defecto del campo de texto
tFComentario.setText("Comentario por defecto");
//Seria lo mismo que haber creado el TextField de esta manera:
TextField tFComentario2 = new TextField("Comentario por defecto");
```

En lugar de añadir etiquetas para acompañar a los campos de texto en este fragmento de código se han añadido los subtítulos, que notifican a los usuarios qué tipo de datos deben ingresar en los campos de texto. El método `setPromptText` define la cadena que aparece en el campo de texto cuando se inicia la aplicación.

```
//definimos setPromptText para que indique la información que espera el campo
tFNombre.setPromptText("Nombre");
```

Puede obtener el valor de un campo de texto en cualquier momento llamando al método `getText`. Aquí por ejemplo hacemos que se muestre el campo oculto en un label:

```
Button btnMostraTexto = new Button("Mostrar apellidos");
Label label = new Label();
btnMostraTexto.setOnAction((ActionEvent e) -> {
    label.setText(tFApellidos.getText());
});
```

Revise algunos métodos útiles que puede usar con los campos de texto.

- `copy()` – transfiere el rango actualmente seleccionado en el texto al portapapeles, dejando la selección actual.
- `cut()` – transfiere el rango actualmente seleccionado en el texto al portapapeles, eliminando la selección actual.
- `selectAll()` - selecciona todo el texto en la entrada de texto.
- `pegar()` – transfiere el contenido del portapapeles a este texto, reemplazando la selección actual.

Consulta el código completo de E09_TextBox.

Mucho más

https://docs.oracle.com/javase/8/javafx/user-interface-tutorial/ui_controls.htm

10.1.4 Diseño (Layouts)

VBox y HBox

El diseño en JavaFX comienza con la selección de los controles de contenedor correctos. Los dos controles de diseño que uso con más frecuencia son `VBox` y `HBox`.

- `VBox` es un contenedor que organiza a sus hijos en una pila vertical.
- `HBox` ordena a sus hijos en una fila horizontal.

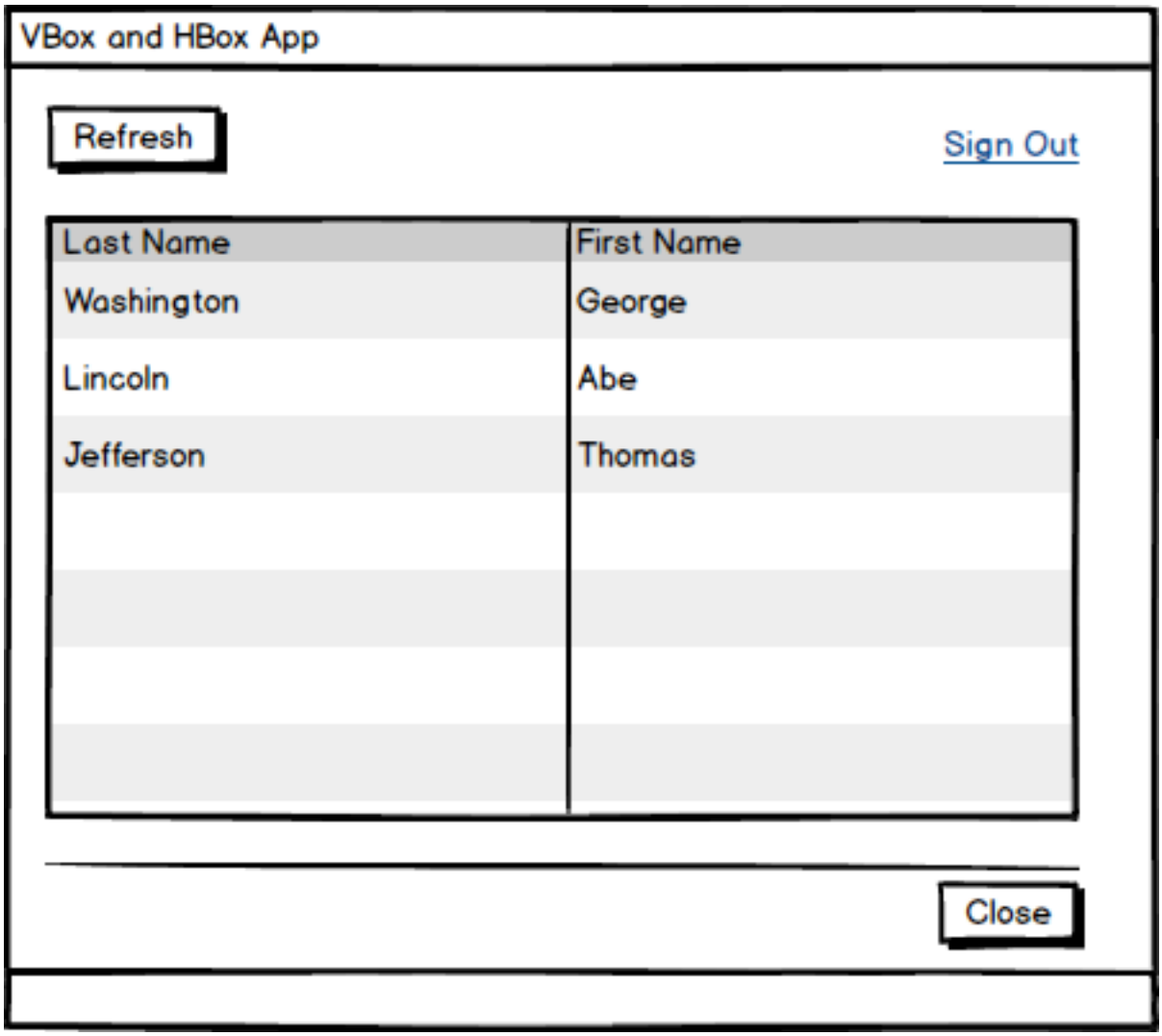
El poder de estos dos controles proviene de envolverlos y establecer algunas propiedades clave: `alineación`, `hgrow` y `vgrow`.

Este artículo demostrará estos controles a través de un proyecto de ejemplo. Una maqueta del proyecto muestra una interfaz de usuario con lo siguiente:

- Una fila de controles superiores que contiene *Actualizar* `Button` y *Cerrar sesión* `Hyperlink`,
- Una *Tabla* `TableView` que crecerá para ocupar el espacio vertical adicional, y
- Un `Button` *Cerrar*.

La interfaz de usuario también presenta un `Separator` panel que divide la parte superior de la pantalla con lo que puede convertirse en un panel inferior estándar (Guardar `Button`, Cancelar `Button`, etc.) para la aplicación.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



ESTRUCTURA

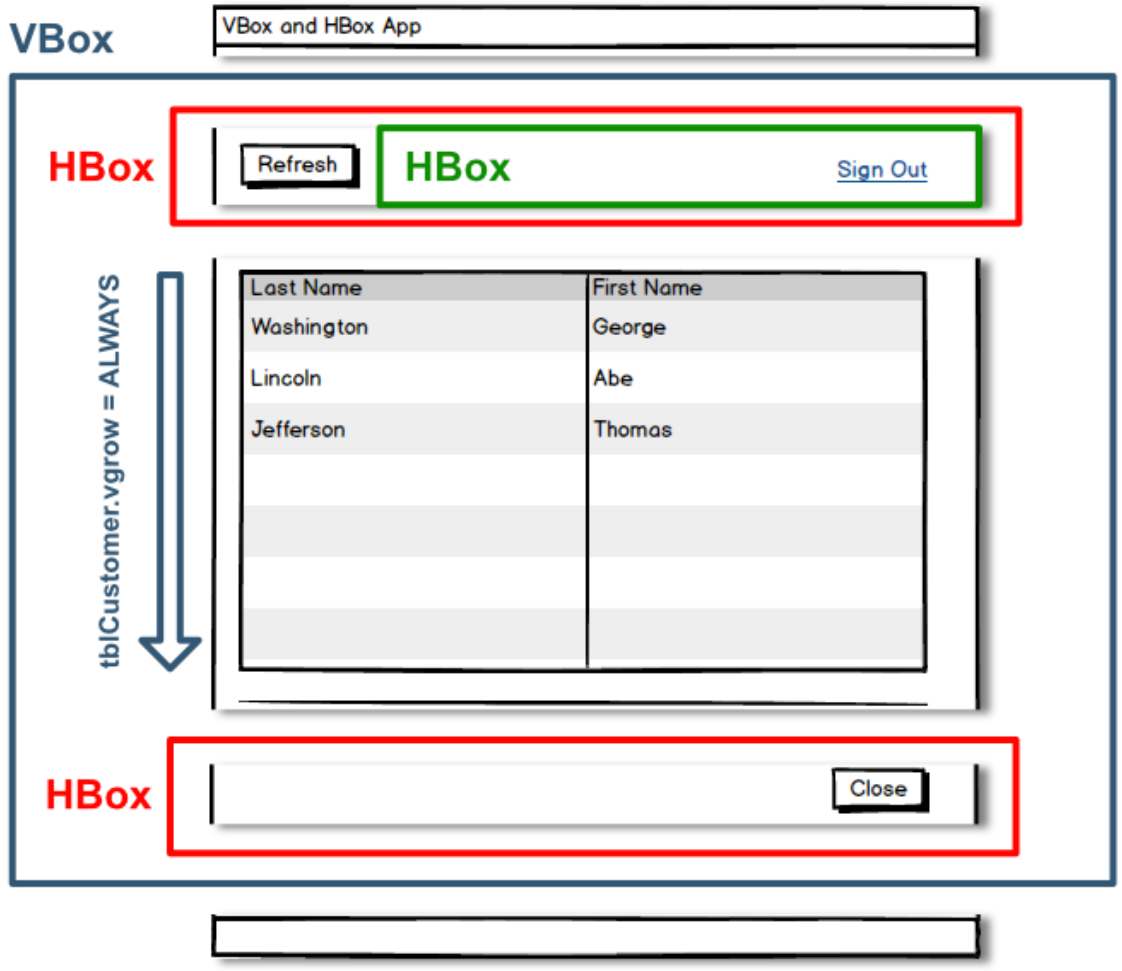
`VBox` será el contenedor más externo "vbox". Este será el `Parent` proporcionado a la Escena. El simple hecho de colocar los controles de la interfaz de usuario en este `VBox` permitirá que los controles, sobre todo el `TableView` , se estiren para adaptarse al espacio horizontal disponible. Los controles superiores, *Actualizar* `Button` y *Cerrar sesión* `Hyperlink` , están envueltos en un contenedor `HBox` . Del mismo modo, se envolverá el `Button` inferior *Cerrar* en un `HBox` , lo que permite botones adicionales.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
VBox vbox = new VBox();

        HBox controlesArriba = new HBox();
        VBox.setMargin( controlesArriba, new Insets(10.0d) );
        controlesArriba.setAlignment( Pos.BOTTOM_LEFT );

        Button btnActualizar = new Button("Actualizar");

        HBox controlesArribaDerecha = new HBox();
        HBox.setHgrow(controlesArribaDerecha, Priority.ALWAYS );
        controlesArribaDerecha.setAlignment( Pos.BOTTOM_RIGHT );
        Hyperlink lnkCerrarSesion = new Hyperlink("Cerrar sesión");
        controlesArribaDerecha.getChildren().add( lnkCerrarSesion );

        controlesArriba.getChildren().addAll( btnActualizar, controlesArribaDerecha );

        TableView<Cliente> tblClientes = new TableView<>();
        tblClientes.setColumnResizePolicy(TableView.CONSTRAINED_RESIZE_POLICY);
        VBox.setMargin( tblClientes, new Insets(0.0d, 10.0d, 10.0d, 10.0d) );
        VBox.setVgrow( tblClientes, Priority.ALWAYS );

        TableColumn<Cliente, String> columnaApellidos = new TableColumn<>("Apellidos");
        columnaApellidos.setCellValueFactory(new PropertyValueFactory<>("apellidos"));

        TableColumn<Cliente, String> columnaNombre = new TableColumn<>("Nombre");
        columnaNombre.setCellValueFactory(new PropertyValueFactory<>("nombre"));

        tblClientes.getColumns().addAll( columnaApellidos, columnaNombre );

        Separator sep = new Separator();

        HBox controlesAbajo = new HBox();
        controlesAbajo.setAlignment(Pos.BOTTOM_RIGHT );
        VBox.setMargin( controlesAbajo, new Insets(10.0d) );

        Button btnCerrar = new Button("Cerrar");

        controlesAbajo.getChildren().add( btnCerrar );

        vbox.getChildren().addAll(
                controlesArriba,
                tblClientes,
                sep,
                controlesAbajo
        );

        Scene scene = new Scene(vbox );

        primaryStage.setScene( scene );
        primaryStage.setWidth( 800 );
        primaryStage.setHeight( 600 );
        primaryStage.setTitle("Aplicación con VBox y HBox");
        primaryStage.setOnShown( (evt) -> loadTable(tblClientes) );
        primaryStage.show();
```

Esta imagen muestra la maqueta desglosada por contenedor. El padre `VBox` es el rectángulo azul más externo. Los HBoxes son los rectángulos interiores (rojo y verde).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



ALINEACIÓN Y HGROW

El `Button` *Actualizar* está alineado a la izquierda mientras que el `Hyperlink` *Cerrar sesión* está alineado a la derecha. Esto se logra usando dos HBoxes. `controlesArriba` es un `HBox` que contiene el `Button` *Actualizar* y también contiene un `HBox` con el `Hyperlink` *Cerrar sesión*. A medida que la pantalla se hace más ancha, el `Hyperlink` *Cerrar sesión* se desplazará hacia la derecha, mientras que el `Button` *Actualizar* mantendrá su alineación izquierda.

La alineación es la propiedad que le dice a un contenedor dónde colocar un control. `controlesArriba` establece la alineación en `BOTTOM_LEFT`. `controlesArribaDerecha` establece la alineación con `BOTTOM_RIGHT`. `BOTTOM` se asegura de que la línea de base del texto *Actualizar* coincida con la línea de base del texto *Cerrar sesión*.

Para que el `Hyperlink` *Cerrar sesión* se mueva hacia la derecha cuando la pantalla se ensancha, es necesario `Priority.ALWAYS`. Esta es una señal para que JavaFX amplíe `controlesArribaDerecha`. De lo contrario, `controlesArriba` mantendrá el espacio y `controlesArribaDerecha` aparecerá a la izquierda. El `Hyperlink` *Cerrar sesión* todavía estaría alineado a la derecha pero en un contenedor más estrecho.

Tenga en cuenta que `setHgrow()` es un método estático y no se invoca el `HBox` `controlesArriba` ni en sí mismo, `controlesArribaDerecha`. Esta es una faceta de la API de JavaFX que puede resultar confusa porque la mayoría de las API establece propiedades a través de setters en objetos.

```
1  controlesArriba.setAlignment( Pos.BOTTOM_LEFT );
2  HBox.setHgrow(controlesArribaDerecha, Priority.ALWAYS ); //en lugar de controlesArribaDerecha.setHgrow(Priority.ALWAYS);
3  controlesArribaDerecha.setAlignment( Pos.BOTTOM_RIGHT );
```

El `Button` `Cerrar` se envuelve en un `HBox` y se posiciona usando la prioridad `BOTTOM_RIGHT`.

```
1  controlesAbajo.setAlignment(Pos.BOTTOM_RIGHT );
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

CRECER

Dado que el contenedor más externo es `VBox`, el hijo `TableView` se expandirá para ocupar espacio horizontal adicional cuando se amplíe la ventana. Sin embargo, cambiar el tamaño vertical de la ventana producirá un espacio en la parte inferior de la pantalla. `VBox` no cambia automáticamente el tamaño de ninguno de sus elementos secundarios . Al igual que con el `HBox` `controlesArribaDerecha`, se puede configurar un indicador de crecimiento. En el caso del `HBox`, se trataba de una instrucción de cambio de tamaño horizontal `setHgrow()`. Para el contenedor `VBox` `TableView`, será `setVgrow()`.

```
1  VBox.setVgrow( tblClientes, Priority.ALWAYS ); //también estático
```

MARGEN

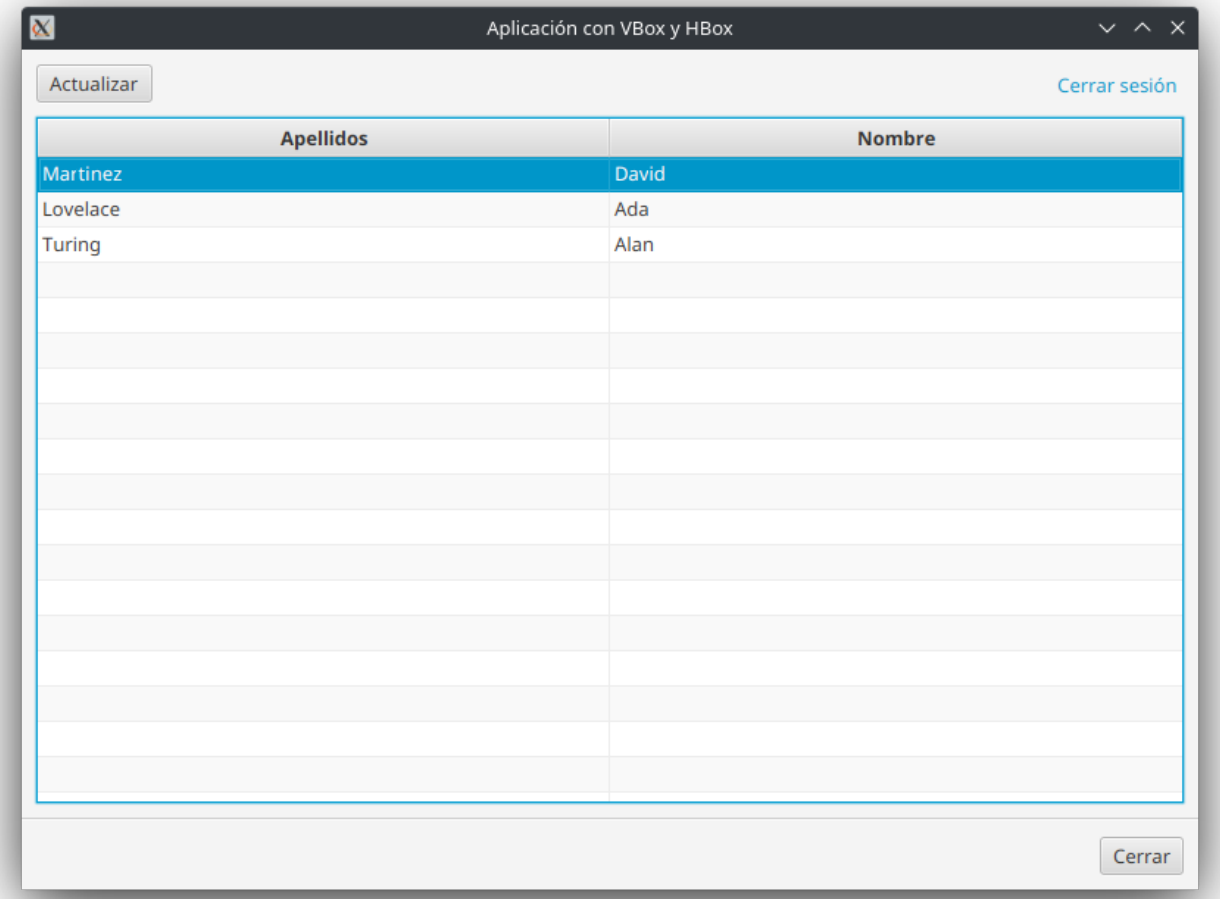
Hay algunas formas de espaciar los controles de la interfaz de usuario. Nosotros usaremos la propiedad `margin` en varios de los contenedores para agregar espacios en blanco alrededor de los controles. Estos se configuran individualmente en lugar de usar un espacio en el `VBox` para que el *Separador* abarque todo el ancho.

```
1  VBox.setMargin(controlesArriba, new Insets(10.0d));
2  VBox.setMargin(tblClientes, new Insets(0.0d, 10.0d, 10.0d, 10.0d));
3  VBox.setMargin(controlesAbajo, new Insets(10.0d));
```

El `Insets` usado por `tblClientes` omite cualquier espacio superior para mantener el espacio uniforme. JavaFX no consolida los espacios en blanco como en el diseño web. Si el Recuadro superior se estableciera en `10.0d` para el `tblClientes`, la distancia entre los controles superiores y el `TableView` sería el doble de ancha que la distancia entre cualquiera de los otros controles.

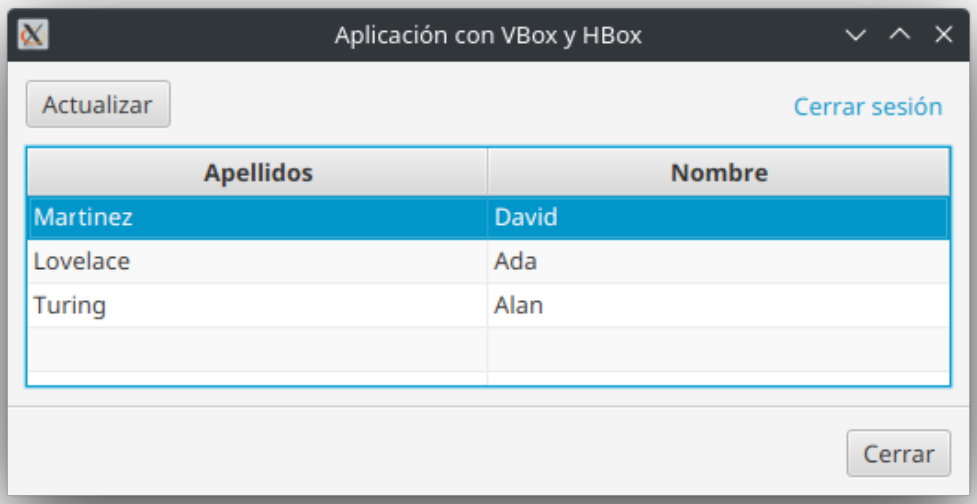
Tenga en cuenta que estos son métodos estáticos como el `Priority`.

Esta imagen muestra la aplicación cuando se ejecuta en su tamaño inicial de 800x600.



© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Esta imagen muestra la aplicación redimensionada a un alto y ancho más pequeños.



SELECCIONE LOS CONTENEDORES CORRECTOS

La filosofía del diseño de JavaFX es la misma que la filosofía de Swing. Seleccione el contenedor adecuado para la tarea en cuestión. Aquí hemos mostrado los dos contenedores más versátiles: `VBox` y `HBox`. Al establecer propiedades como alineación, `hgrow` y `vgrow`, puede crear diseños increíblemente complejos mediante el anidamiento. Estos son los contenedores que más uso y, a menudo, son los únicos contenedores que necesitaras.

CÓDIGO COMPLETO

El código se puede probar con un par de archivos .java. Hay un POJO para la clase `Cliente` utilizado por el `TableView`

Consulta el código completo de Cliente.

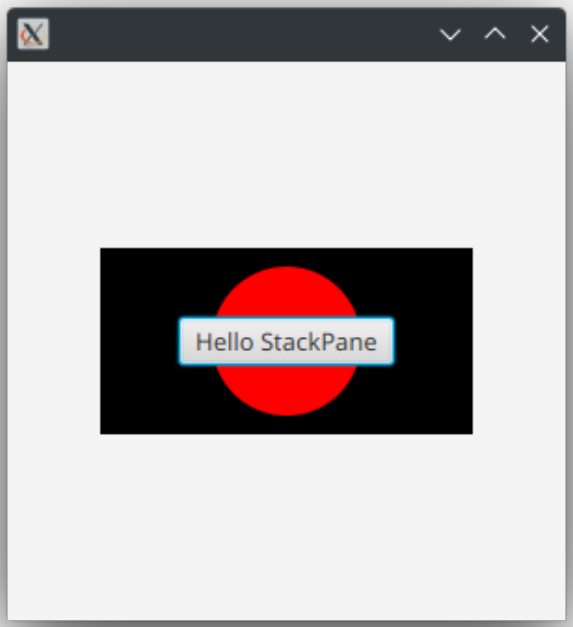
y la subclase JavaFX completa y principal:

Consulta el código completo de VBoxAndHBoxApp.

StackPane

El layout `StackPane` coloca a sus hijos uno encima de otro. El último `Node` agregado es el más alto. Por defecto `StackPane` alineará los hijos usando `Pos.CENTER`, como se puede ver en la siguiente imagen, donde están los 3 hijos (en orden de creación): `Rectangle`, `Circle` y `Button`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

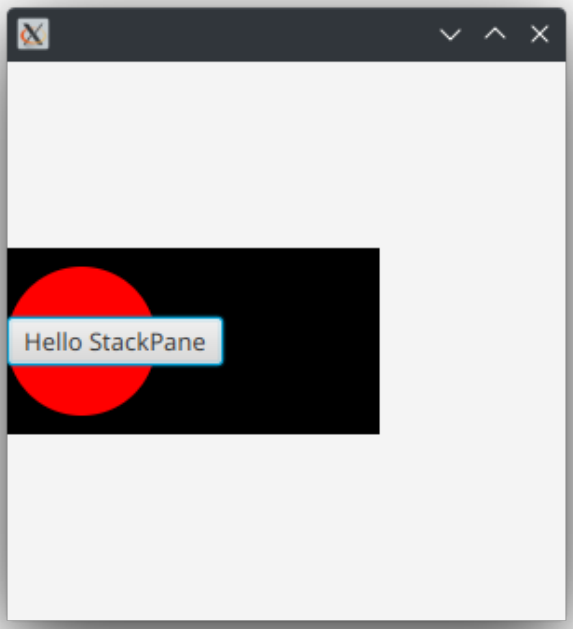


Esta imagen fue producida por el siguiente fragmento:

Consulta el código completo de StackPaneApp.

Podemos cambiar la alineación predeterminada descomentando la linea `pane.setAlignment(Pos.CENTER_LEFT);` para producir el siguiente efecto:

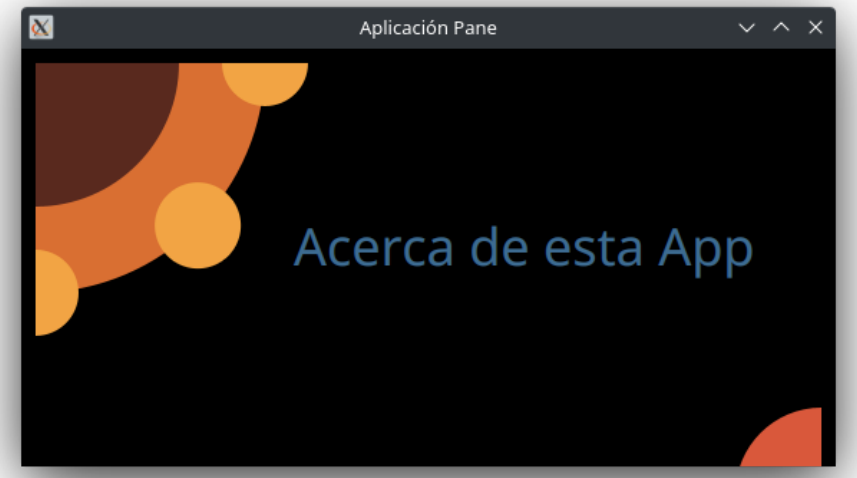
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Posicionamiento absoluto con panel (`Pane`)

Contenedores como `VBox` o `BorderPane` alinear y distribuir a sus hijos. La superclase `Pane` también es un contenedor, pero no impone un orden a sus hijos. Los hijos se posicionan a sí mismos a través de propiedades como `x`, `centerX` y `layoutX`. Esto se llama posicionamiento absoluto y es una técnica para colocar un `Shape` o un `Node` en un lugar determinado de la pantalla.

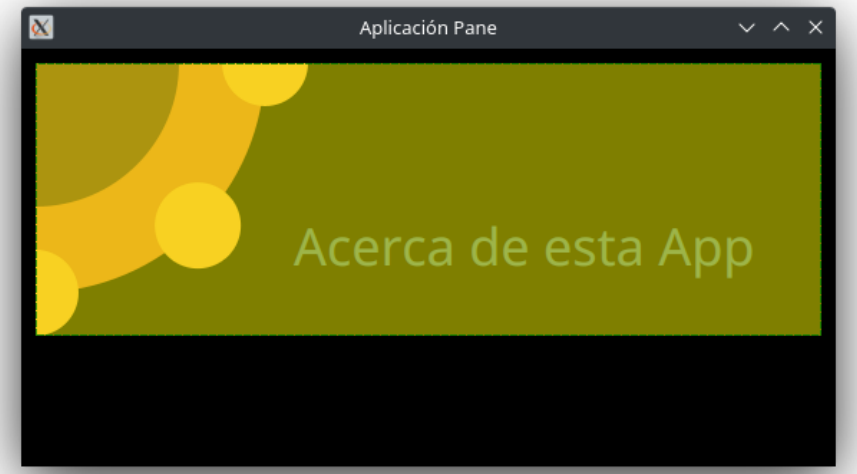
Esta captura de pantalla muestra una vista *Acerca de*. La vista *Acerca de* contiene un `Hyperlink` en el medio de la pantalla "Acerca de esta aplicación". La vista *Acerca de* utiliza varias formas JavaFX para formar un diseño que se recorta para que parezca una tarjeta de presentación.



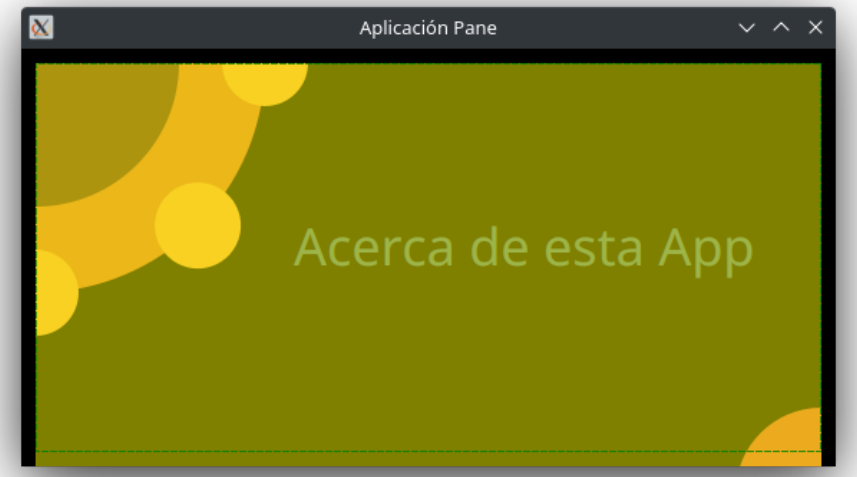
© 2025 David Martínez licensed under CC BY-NC-SA 4.0

TAMAÑO DEL PANEL

A diferencia de la mayoría de los contenedores, `Pane` cambia de tamaño para adaptarse a su contenido y no al revés. Esta imagen es una captura de pantalla de Scenic View tomada antes de agregar el Arco inferior derecho. El `Pane` es el área resaltada en amarillo. Tenga en cuenta que no ocupa la totalidad `Stage`.



A continuación se muestra una captura de pantalla tomada después de agregar la esquina inferior derecha `Arc`. Esto `Arc` se colocó más cerca del borde inferior derecho del archivo `Stage`. Esto obliga al Panel a estirarse para acomodar los contenidos expandidos.



EL PANEL (`Pane`)

El contenedor más externo de la vista *Acerca de* es un `VBox` cuyo único contenido es el archivo `Pane`. El `VBox` se utiliza para encajar en el conjunto `Stage` y proporciona un fondo.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
VBox vbox = new VBox();
vbox.setPadding( new Insets( 10 ) );
vbox.setBackground(
    new Background(
        new BackgroundFill(Color.BLACK, new CornerRadii(0), new Insets(0))
        ));

Pane p = new Pane();
```

LAS FORMAS (Shape)

En la parte superior izquierda de la pantalla, hay un grupo de 4 Arcos y 1 Círculo. Este código posiciona `largeArc` en (0,0) a través de los argumentos `centerX` y `centerY` en el constructor `Arc`. Observa que `backgroundArc` también se coloca en (0,0) y aparece debajo de `largeArc`. `Pane` no intenta eliminar el conflicto de formas superpuestas y, en este caso, lo que se busca es la superposición. `smArc1` se coloca en (0,160), que está abajo en el eje Y. `smArc2` está posicionado en (160,0) que está justo en el eje X. `smCircle` se coloca a la misma distancia que `smArc1` y `smArc2`, pero en un ángulo de 45 grados.

```
Arc largeArc = new Arc(0, 0, 100, 100, 270, 90);
largeArc.setType(ArcType.ROUND);

Arc backgroundArc = new Arc(0, 0, 160, 160, 270, 90 );
backgroundArc.setType( ArcType.ROUND );

Arc smArc1 = new Arc( 0, 160, 30, 30, 270, 180);
smArc1.setType(ArcType.ROUND);

Circle smCircle = new Circle(160/Math.sqrt(2.0), 160/Math.sqrt(2.0), 30,Color.web("0xF2A444"));

Arc smArc2 = new Arc( 160, 0, 30, 30, 180, 180);
smArc2.setType(ArcType.ROUND);
```

En la parte inferior derecha el `Arc` se coloca en función de la altura total del archivo `Stage`. Los 20 restados de la altura son los 10 píxeles de `Insets` (Margen) de `VBox` (10 para la izquierda + 10 para la derecha).

```
Arc medArc = new Arc(568-20, 320-20, 60, 60, 90, 90);
medArc.setType(ArcType.ROUND);

primaryStage.setWidth(568);
primaryStage.setHeight(320);
```

EL HIPERVÍNCULO

El `Hyperlink` está posicionado compensado el centro (284,160) que es el ancho y alto de `Stage` ambos dividido por dos. Esto coloca el texto del `Hyperlink` en el cuadrante inferior derecho de la pantalla, por lo que se necesita un desplazamiento basado en el ancho y el alto de `Hyperlink`. Las dimensiones del `Hyperlink` no están disponibles hasta que se muestra la pantalla, por lo que se realiza un ajuste posterior a la visualización de la posición.

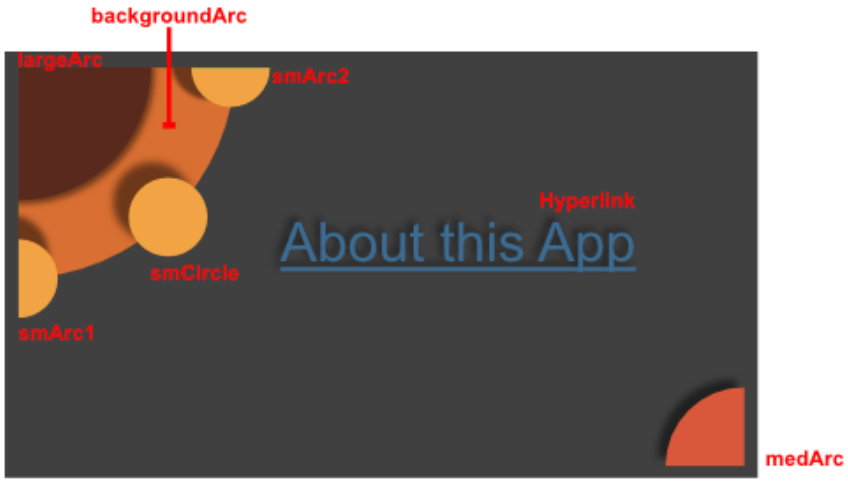
```
Hyperlink hyperlink = new Hyperlink("About this App");

primaryStage.setOnShown( (evt) -> {
     hyperlink.setLayoutX( 284 - (hyperlink.getWidth()/3) );
     hyperlink.setLayoutY( 160 - hyperlink.getHeight() );
});
```

El `Hyperlink` no está colocado en el verdadero centro de la pantalla. El valor de `layoutX` se basa en una operación de división por tres que lo aleja de las formas de la esquina superior izquierda.

ORDEN Z

Como se mencionó anteriormente, `Pane` admite la superposición de los hijos. Esta imagen muestra la vista *Acerca de* con profundidad.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



El orden z en este ejemplo está determinado por el orden en que se agregan los elementos secundarios al archivo `Pane`. `backgroundArc` está oscurecido por elementos agregados más tarde, más notablemente `largeArc`. Para reorganizar los elementos secundarios, use los métodos `toFront()` y `toBack()` después de agregar los elementos al archivo `Pane`.

```
p.getChildren().addAll( backgroundArc, largeArc, smArc1, smCircle, smArc2, hyperlink, medArc );
vbox.getChildren().add( p );
```

Recomendación

Al comenzar con JavaFX, es tentador construir un diseño absoluto. Tenga en cuenta que los diseños absolutos son frágiles y, a menudo, se rompen cuando se cambia el tamaño de la pantalla (como en este caso) o cuando se agregan elementos durante la fase de mantenimiento del software. Sin embargo, existen buenas razones para utilizar el posicionamiento absoluto. El juego es uno de esos usos. En un juego, puede ajustar la coordenada (x,y) de una `Shape` para mover una pieza del juego por la pantalla.

CÓDIGO COMPLETO

Esta es la aplicación JavaFX completa y principal.

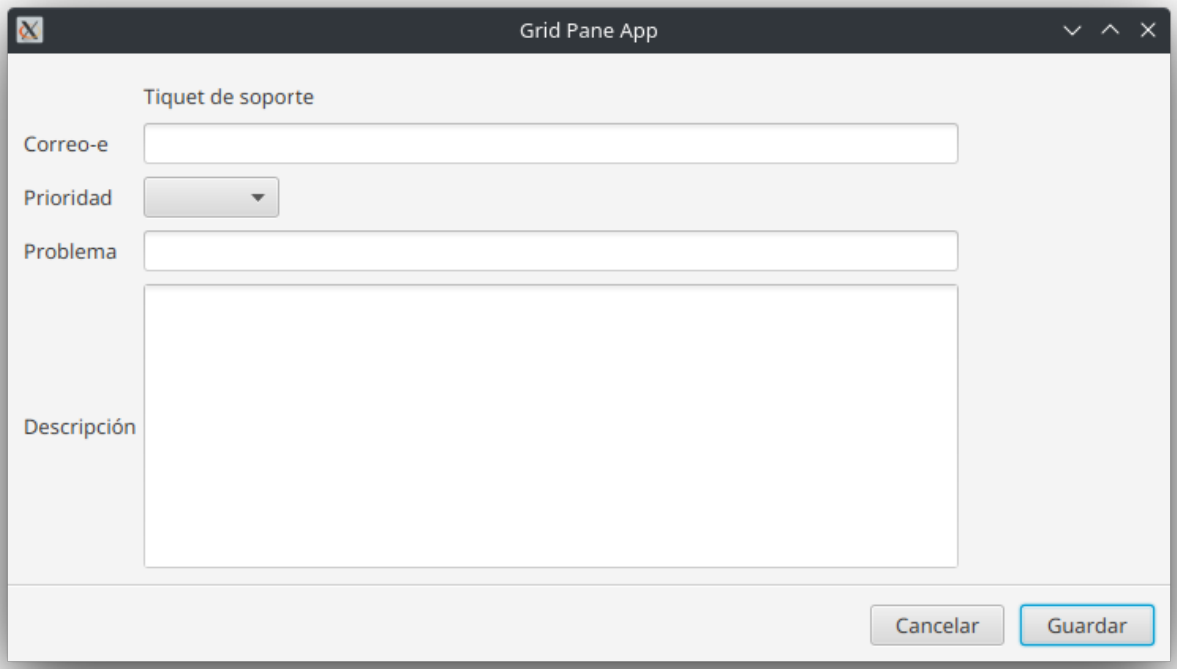
Consulta el código completo de PaneApp.

GridPane

Los formularios en las aplicaciones comerciales a menudo usan un diseño que imita un registro de base de datos. Para cada columna de una tabla, se agrega un encabezado en el lado izquierdo que coincide con un valor de fila en el lado derecho. JavaFX tiene un control de propósito especial llamado `GridPane` para este tipo de diseño que mantiene los contenidos alineados por fila y columna. `GridPane` también admite expansión para diseños más complejos.

La siguiente captura de pantalla muestra un diseño básico de `GridPane`. En el lado izquierdo del formulario, hay una columna de nombres de campo: Correo-e, Prioridad, Problema, Descripción. En el lado derecho del formulario, hay una columna de controles que mostrará el valor del campo correspondiente. Los nombres de campo son de tipo `Label` y los controles de valor son una mezcla que incluye `TextField`, `TextArea` y `ComboBox`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

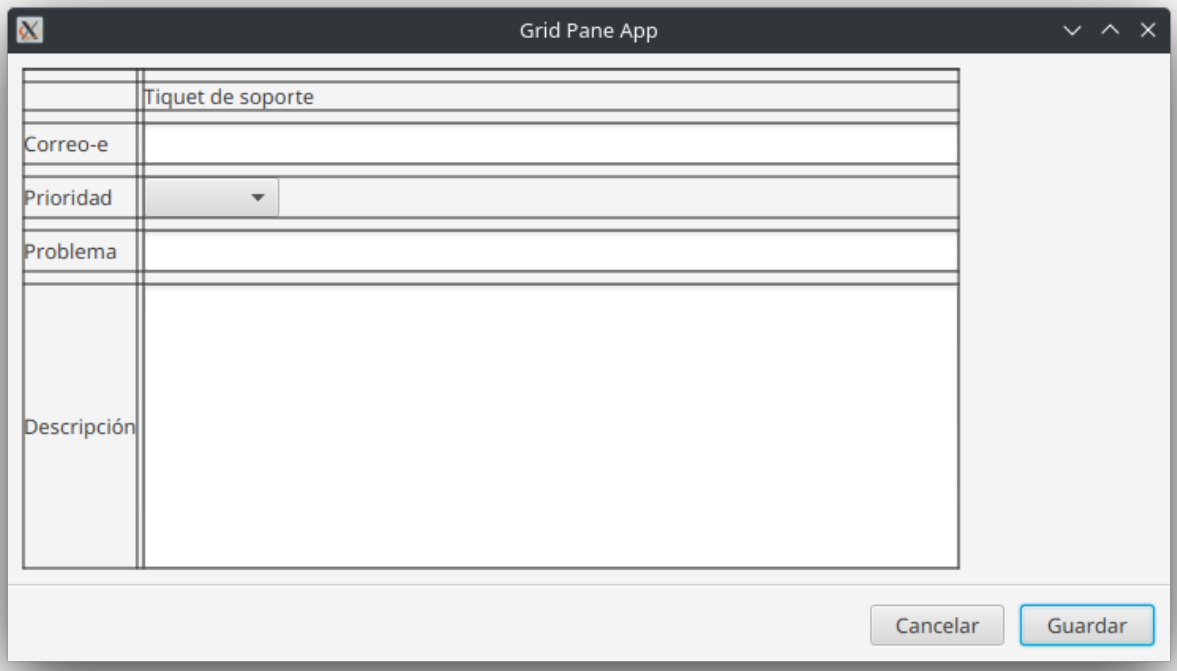


El siguiente código muestra los objetos creados para el formulario. "vbox" es la raíz del `Scene` y también contendrá el `ButtonBar` en la base del formulario.

```
VBox vbox = new VBox();
GridPane gp = new GridPane();

Label lblTitle = new Label("Tiquet de soporte");

Label lblEmail = new Label("Correo-e");
TextField tfEmail = new TextField();

Label lblPriority = new Label("Prioridad");
ObservableList<String> priorities = FXCollections.observableArrayList("Media", "Alta", "Baja");
ComboBox<String> cbPriority = new ComboBox<>(priorities);

Label lblProblem = new Label("Problema");
TextField tfProblem = new TextField();

Label lblDescription = new Label("Descripción");
TextArea taDescription = new TextArea();
```

GridPane tiene un método útil `setGridLinesVisible()` que muestra la estructura de la cuadrícula y los espacios. Es especialmente útil en diseños más complejos donde se involucra la expansión porque los espacios en las asignaciones de filas/columnas pueden causar cambios en el diseño.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



ESPACIADO

Como contenedor, `GridPane` tiene una propiedad de relleno que se puede configurar para rodear el contenedor `GridPane` con espacios en blanco. `setPadding()` tomará un objeto `Inset` como parámetro. En este ejemplo, se aplican 10 píxeles de espacio en blanco a todos los lados, por lo que se usa un constructor de formato corto para `Inset`.

Dentro de `GridPane`, `vgap` y `hgap` controlan los espacios. El `hgap` se establece en 4 para mantener los campos cerca de sus valores. `vgap` es un poco más grande para ayudar con la navegación del mouse.

```
1  gp.setPadding( new Insets(10) );
2  gp.setHgap( 4 );
3  gp.setVgap( 8 );
```

Para mantener consistente la parte inferior del formulario, `Priority.ALWAYS` se establece a en la clase VBox sobre el `GridPane` (recuerda que es estático). Sin embargo, esto no cambiará el tamaño de las filas individuales. Para especificaciones de cambio de tamaño individuales, debes usar `ColumnConstraints` y `RowConstraints`.

```
1  VBox.setVgrow(gp, Priority.ALWAYS );
```

ADICIÓN DE ELEMENTOS

A diferencia de los contenedores como `BorderPane` o `HBox`, los nodos deben especificar su posición dentro del contenedor `GridPane`. Esto se hace con el método `add()` en `GridPane` y no con el método `add()` en una propiedad secundaria del contenedor. El método `add()` de `GridPane` recibe una posición de columna de base cero y una posición de fila de base cero. En este código ponemos dos declaraciones en la misma línea para facilitar la lectura.

```
1  gp.add( lblTitle,       1, 1);  // empty item at 0,0
2  gp.add( lblEmail,       0, 2); gp.add(tfEmail,       1, 2);
3  gp.add( lblPriority,    0, 3); gp.add( cbPriority,   1, 3);
4  gp.add( lblProblem,     0, 4); gp.add( tfProblem,    1, 4);
5  gp.add( lblDescription, 0, 5); gp.add( taDescription, 1, 5);
```

`lblTitle` se coloca en la segunda columna de la primera fila. No hay ninguna entrada en la primera columna de la primera fila.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Las adiciones posteriores se presentan por parejas. Los objetos de nombre de campo `Label` se colocan en la primera columna (índice de columna=0) y los controles de valor se colocan en la segunda columna (índice de columna=1). Las filas se agregan por el segundo valor incrementado. Por ejemplo, `lblPriority` se coloca en la cuarta fila junto con su `ComboBox`.

`GridPane` es un contenedor importante en el diseño de aplicaciones empresariales JavaFX. Cuando tenga un requisito de pares de nombre/valor, `GridPane` será una manera más fácil de organizar la estructura del formulario.

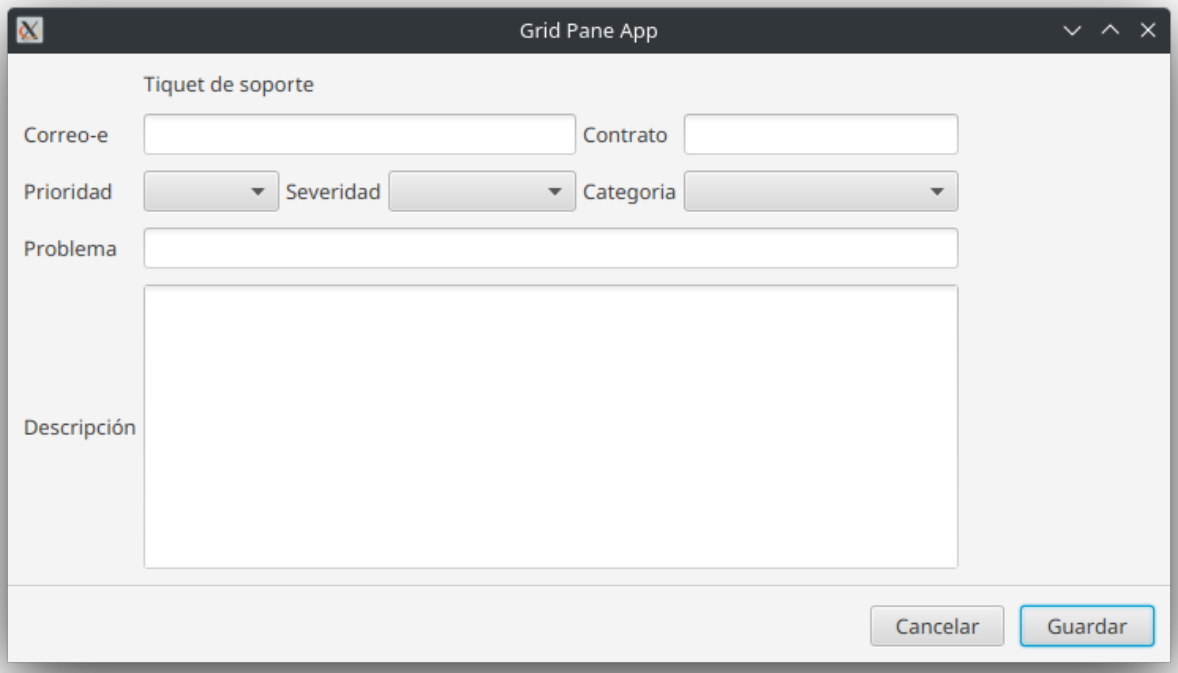
CÓDIGO COMPLETO

La siguiente clase es el código completo del ejemplo. Esto incluye la definición de la `ButtonBar` que no se presentó en las secciones anteriores enfocadas en `GridPane`.

Consulta el código completo de GridPaneApp.

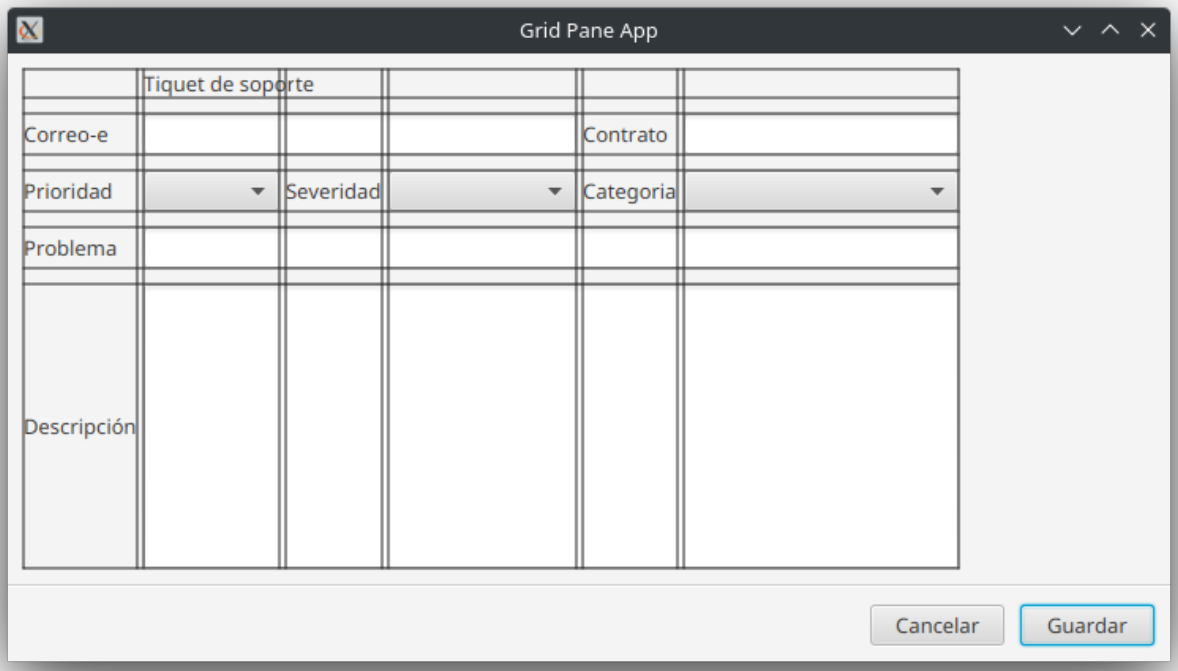
GridPane Spanning (expansión)

Para formularios más complejos implementados con `GridPane`, se admite la expansión. La expansión permite que un control reclame el espacio de columnas vecinas (`colspan`) y filas vecinas (`rowspan`). Esta captura de pantalla muestra un formulario que amplía el ejemplo de la sección anterior. El diseño de dos columnas de la versión anterior se reemplazó por un diseño de varias columnas. Los campos como Problema y Descripción conservan la estructura original. Pero se agregaron controles a las filas que anteriormente contenían solo Correo electrónico y Prioridad.



Al activar las líneas de la cuadrícula, observe que la cuadrícula anterior de dos columnas se reemplaza con una cuadrícula de seis columnas. La tercera fila que contiene seis elementos (3 pares de nombre de campo/valor) dicta la estructura. El resto del formulario utilizará la expansión para completar el espacio en blanco.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Hay un poco más de Vgap para ayudar al usuario a seleccionar los controles `ComboBox`. Como en la versión anterior, los controles se agregan al `GridPane` con el método `add()`. Se especifica una columna y una fila. Repasa los índices ya que no es evidente, ya que se espera que se llenen los vacíos mediante el contenido expandido.

Las definiciones de expansión se establecen mediante un método estático en `GridPane`. Hay un método similar para hacer la expansión de filas. El título ocupará 5 columnas, al igual que el problema y la descripción. El correo electrónico comparte una fila con el contrato, pero ocupará más columnas. La tercera fila de ComboBoxes es un conjunto de tres pares de campo/valor, cada uno de los cuales ocupa una columna.

```
GridPane.setColumnSpan( lblTitle, 5 );
GridPane.setColumnSpan( tfEmail, 3 );
GridPane.setColumnSpan( tfProblem, 5 );
GridPane.setColumnSpan( taDescription, 5 );
```

Alternativamente, una variación del método add() tendrá argumentos columnSpan y rowSpan para evitar la subsiguiente llamada al método estático.

Este ejemplo ampliado `GridPane` demostró la expansión de columnas. La misma capacidad está disponible para la expansión de filas, lo que permitiría que un control reclame espacio vertical adicional. La expansión mantiene los controles alineados incluso en los casos en que varía el número de elementos en una fila (o columna) determinada.

CÓDIGO COMPLETO

El siguiente es el código completo para el ejemplo de GridPane de expansión.

Consulta el código completo de GridPaneAppv2.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
        //Expansiones
        GridPane.setColumnSpan(lblTitle, 5);
        GridPane.setColumnSpan(tfEmail, 3);
        GridPane.setColumnSpan(tfProblem, 5);
        GridPane.setColumnSpan(taDescription, 5);

        Separator sep = new Separator(); // hr

        ButtonBar buttonBar = new ButtonBar();
        buttonBar.setPadding(new Insets(10));

        Button saveButton = new Button("Guardar");
        Button cancelButton = new Button("Cancelar");

        buttonBar.setButtonData(saveButton, ButtonBar.ButtonData.OK_DONE);
        buttonBar.setButtonData(cancelButton, ButtonBar.ButtonData.CANCEL_CLOSE);

        buttonBar.getButtons().addAll(saveButton, cancelButton);

        vbox.getChildren().addAll(gp, sep, buttonBar);

        //para mostrar las lineas de estructura descomenta la siguiente linea
        gp.setGridLinesVisible(true);

        Scene scene = new Scene(vbox);

        primaryStage.setTitle("Grid Pane App");
        primaryStage.setScene(scene);
        primaryStage.setWidth(736);
        primaryStage.setHeight(414);
        primaryStage.show();

    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

10.1.5 Gestión de excepciones y ventanas modales `Alert`

En JavaFX, las ventanas modales de tipo `Alert` son útiles para mostrar mensajes al usuario (errores, advertencias, información o confirmaciones). Se combinan frecuentemente con la gestión de excepciones para informar de forma amigable cuando ocurre un error en tiempo de ejecución.

Ejemplo práctico:

```
try {
    int resultado = 10 / 0;  // Simula una excepción
} catch (ArithmeticException e) {
    Alert alerta = new Alert(Alert.AlertType.ERROR);
    alerta.setTitle("Error matemático");
    alerta.setHeaderText("No se pudo realizar la operación");
    alerta.setContentText("División entre cero: " + e.getMessage());
    alerta.showAndWait();  // Ventana modal (bloquea hasta que se cierre)
}
```

Tipos de alerta más comunes:

- `ERROR` → Para fallos graves.
- `WARNING` → Para situaciones peligrosas o dudosas.
- `INFORMATION` → Para notificar acciones completadas.
- `CONFIRMATION` → Para preguntar sí/no antes de continuar.

Ventaja: La combinación de `try-catch` con `Alert` mejora la experiencia de usuario, evitando que la aplicación falle silenciosamente o muestre mensajes de consola poco intuitivos.

10.1.6 Estructura de la aplicación

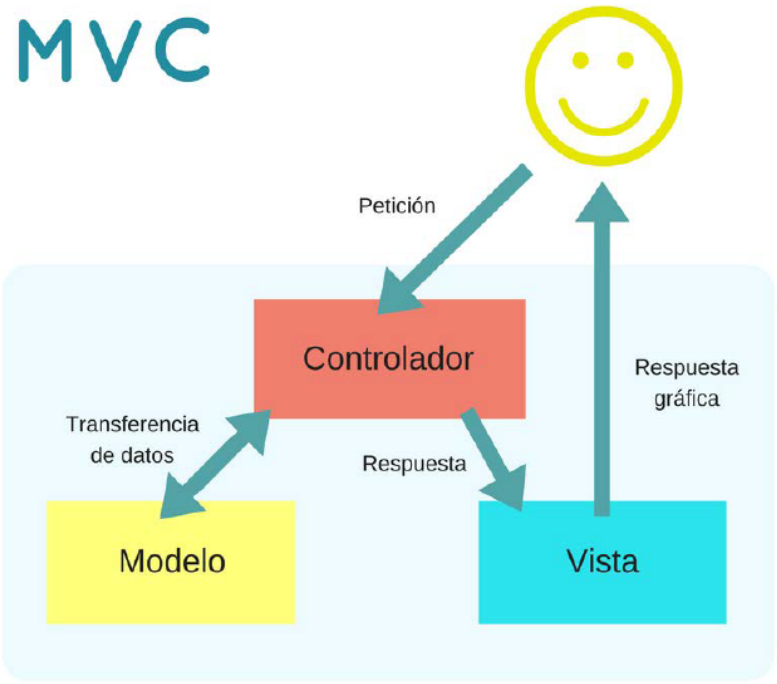
El patrón MVC

Modelo-vista-controlador (**MVC**) es un patrón de arquitectura de software, que separa los datos y principalmente lo que es la lógica de negocio de una aplicación de su representación y el módulo encargado de gestionar los eventos y las comunicaciones. Para ello MVC propone la construcción de tres componentes distintos que son el **modelo**, la **vista** y el **controlador**, es decir, por un lado define componentes para la representación de la información, y por otro lado para la interacción del usuario. Este patrón de arquitectura de software se basa en las ideas de reutilización de código y la separación de conceptos, características que buscan facilitar la tarea de desarrollo de aplicaciones y su posterior mantenimiento.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

De manera genérica, los componentes de MVC se podrían definir como sigue:

- El **Modelo**: Es la representación de la información con la cual el sistema opera, por lo tanto gestiona todos los accesos a dicha información, tanto consultas como actualizaciones, implementando también los privilegios de acceso que se hayan descrito en las especificaciones de la aplicación (lógica de negocio). Envía a la 'vista' aquella parte de la información que en cada momento se le solicita para que sea mostrada (típicamente a un usuario). Las peticiones de acceso o manipulación de información llegan al 'modelo' a través del 'controlador'.
- El **Controlador**: Responde a eventos (usualmente acciones del usuario) e invoca peticiones al 'modelo' cuando se hace alguna solicitud sobre la información (por ejemplo, editar un documento o un registro en una base de datos). También puede enviar comandos a su 'vista' asociada si se solicita un cambio en la forma en que se presenta el 'modelo' (por ejemplo, desplazamiento o scroll por un documento o por los diferentes registros de una base de datos), por tanto se podría decir que el 'controlador' hace de intermediario entre la 'vista' y el 'modelo' (véase ***Middleware***).
- La **Vista**: Presenta el 'modelo' (información y *lógica de negocio*) en un formato adecuado para interactuar (usualmente la interfaz de usuario), por tanto requiere de dicho 'modelo' la información que debe representar como salida.



10.1.7 Ejemplos UD09

Ejemplo01

Programa mínimo "Hola Mundo" en JavaFX.

```
import javafx.application.Application;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.layout.StackPane;
import javafx.scene.text.Text;
import javafx.stage.Stage;

public class HolaMundo extends Application {

    private Parent createContent() {
        return new StackPane(new Text("Hola Mundo"));
    }

    @Override
    public void start(Stage stage) throws Exception {
        stage.setScene(new Scene(createContent(), 400, 400));
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

⬆ Volver a teoría

Ejemplo02

Transformaciones básicas en JavaFX: traslación, rotación y escalado.

```java
package UD09;

import javafx.application.Application;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.stage.Stage;

/**
 *
 * @author David Martínez (wwww.martinezpenya.es|ieseduardoprimo.es)
 */
public class E02_TransformApp extends Application {

    private Parent createContent() {
        Rectangle box = new Rectangle(100, 50, Color.BLUE);

        transform(box);

        return new Pane(box);
    }

    private void transform(Rectangle box) {
        //Aplicaremos las transformaciones aquí

        //Descomentar para traslación
        //box.setTranslateX(100);
        //box.setTranslateY(200);

        //Descomentar para escalado
        //box.setScaleX(1.5);
        //box.setScaleY(1.5);

        //Descomentar para rotación
        //box.setRotate(30);
    }

    @Override
    public void start(Stage stage) throws Exception {
        stage.setScene(new Scene(createContent(), 300, 300, Color.GRAY));
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

⬆ Volver a teoría

Ejemplo03

Creación de un evento personalizado con `Event` y `EventType`.

```java
import javafx.event.Event;
import javafx.event.EventType;

public class E03_EventoUsuario extends Event {

    public static final EventType<E03_EventoUsuario> ANY = new EventType<>(Event.ANY, "ANY");

    public static final EventType<E03_EventoUsuario> LOGIN_SUCCEEDED = new EventType<>(ANY, "LOGIN_SUCCEEDED");

    public static final EventType<E03_EventoUsuario> LOGIN_FAILED = new EventType<>(ANY, "LOGIN_FAILED");

    public E03_EventoUsuario(EventType<? extends Event> eventType) {
        super(eventType);
    }
    // cualquier otro atributo importante como la fecha, la hora...
}
```

⬆ Volver a teoría

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Ejemplo04

Uso del control `Label`: texto, estilo CSS, rotación e imagen.

```java
package UD09;

import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.input.MouseEvent;
import javafx.scene.layout.GridPane;
import javafx.scene.paint.Color;
import javafx.scene.text.Font;
import javafx.stage.Stage;

public class E05_Label extends Application {

    private Parent createContent() {
        GridPane grid = new GridPane();
        grid.setAlignment(Pos.CENTER);
        grid.setHgap(10);
        grid.setVgap(10);
        grid.setPadding(new Insets(25, 25, 25, 25));

        //Creamos la etiqueta vacia
        Label label1 = new Label();
        //añadimos texto una vez creada
        label1.setText("Texto añadido después de la creación, autoajustable");
        //cambiamos la fuente y tamaño
        label1.setFont(new Font("Ubuntu", 12));
        //establecemos su color
        label1.setTextFill(Color.web("#0076a3"));
        //activamos la propiedad de autoajustable a true
        label1.setWrapText(true);
        //añadimos la etiqueta a la columna 0 fila 0
        grid.add(label1, 0, 0);

        //Creamos la etiqueta con texto
        Label label2 = new Label("Etiqueta2");
        grid.add(label2, 1, 0);
        label2.setFont(Font.font("FreeMono", 32));
        label2.setRotate(270);

        //Creamos la etiqueta con imágen
        Image image = new Image("UD09/label.png");
        Label label3 = new Label("Search", new ImageView(image));
        label3.setGraphicTextGap(20);
        grid.add(label3, 2, 0);

        label3.setOnMouseEntered((MouseEvent event) -> {
            label3.setScaleX(1.5);
            label3.setScaleY(1.5);
        });

        label3.setOnMouseExited((MouseEvent event) -> {
            label3.setScaleX(1);
            label3.setScaleY(1);
        });

        return grid;
    }

    @Override
    public void start(Stage stage) throws Exception {
        stage.setScene(new Scene(createContent(), 500, 200));
        stage.setTitle("Ejemplo con Label");
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

⬆ Volver a teoría

Ejemplo05

Manejo de eventos con `Button`: `setOnAction`, `setOnMouseEntered` y `setOnMouseExited`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD09;

import javafx.application.Application;
import javafx.event.ActionEvent;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.effect.DropShadow;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.input.MouseEvent;
import javafx.scene.layout.GridPane;
import javafx.stage.Stage;

public class E06_Button extends Application {

    private Parent createContent() {
        GridPane grid = new GridPane();
        grid.setAlignment(Pos.CENTER_LEFT);
        grid.setHgap(10);
        grid.setVgap(10);
        grid.setPadding(new Insets(25, 25, 25, 25));

        //Creamos el botón vacio
        Button button1 = new Button();
        //añadimos texto una vez creado
        button1.setText("Botón CSS");
        //establecemos la clase correspondiente del css
        button1.getStyleClass().add("button1");
        //añadimos el botón a la columna 0 fila 0 con colspan 3 y rowspan 1
        grid.add(button1, 0, 0, 3, 1);

        //Creamos el botón con texto
        Button button2 = new Button("Sí");
        grid.add(button2, 0, 1);
        Button button3 = new Button("No");
        grid.add(button3, 1, 1);
        //Añadimos el label que cambiará segun el botón presionado
        Label label = new Label("Aceptado");
        grid.add(label, 2, 1);

        //Creamos el botón con texto e imágen
        Image image = new Image("UD09/ok.png");
        Button button4 = new Button("Aceptar", new ImageView(image));
        grid.add(button4, 0, 2, 2, 1);

        //métodos para cambiar el label segun el botón pulsado
        button2.setOnAction((ActionEvent e) -> {
            label.setText("Aceptado");
        });

        button3.setOnAction((ActionEvent e) -> {
            label.setText("Denegado");
        });

        //Creamos el estilo de sombra
        DropShadow shadow = new DropShadow();
        //Añadimos la sombra cuando pasamos sobre el botón
        button4.addEventHandler(MouseEvent.MOUSE_ENTERED, (MouseEvent e) -> {
            button4.setEffect(shadow);
        });

        //Eliminamos la sombra al salir del botón
        button4.addEventHandler(MouseEvent.MOUSE_EXITED, (MouseEvent e) -> {
            button4.setEffect(null);
        });

        return grid;
    }

    @Override
    public void start(Stage stage) throws Exception {
        Scene scene = new Scene(createContent(), 300, 200);
        scene.getStylesheets().add("UD09/style.css");
        stage.setScene(scene);

        stage.setTitle("Ejemplo con Button");
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

⬆ Volver a teoría

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Ejemplo06

Uso de `RadioButton` con agrupación y selección única.

```java
package UD09;

import javafx.application.Application;
import javafx.beans.value.ObservableValue;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.control.RadioButton;
import javafx.scene.control.Toggle;
import javafx.scene.control.ToggleGroup;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.layout.GridPane;
import javafx.stage.Stage;

public class E07_RadioButton extends Application {

    private Parent createContent() {
        GridPane grid = new GridPane();
        grid.setAlignment(Pos.CENTER_LEFT);
        grid.setHgap(10);
        grid.setVgap(10);
        grid.setPadding(new Insets(25, 25, 25, 25));

        //Creamos el botón vacio
        RadioButton rButton1 = new RadioButton();
        //añadimos texto una vez creado
        rButton1.setText("Coche");
        //añadimos el RadioButton a la columna 0 fila 0 con colspan 3 y rowspan 1
        grid.add(rButton1, 0, 0);

        //Creamos los RadioButton con texto
        RadioButton rButton2 = new RadioButton("Moto");
        grid.add(rButton2, 0, 1);
        //Creamos un RadioButton con imágen
        RadioButton rButton3 = new RadioButton("A pie");
        grid.add(rButton3, 0, 2);

        //Añadimos las imágenes a los Radio Button
        //ImageView imageCoche = new ImageView("UD09/coche.png");
        //rButton1.setGraphic(imageCoche);
        //Creamos el grupo de alternancia
        final ToggleGroup grupo = new ToggleGroup();
        rButton1.setToggleGroup(grupo);
        //rButton1.setSelected(true); //si queremos que la primera opción este marcada por defecto
        rButton2.setToggleGroup(grupo);
        rButton3.setToggleGroup(grupo);

        //Añadimos un valor personalizado a cada control con el nombre de la imágen correspondiente
        rButton1.setUserData("coche");
        rButton2.setUserData("moto");
        rButton3.setUserData("pie");

        //Añadimos una imágen que cambiara al cambiar la selección
        ImageView image = new ImageView();
        grid.add(image, 1, 0, 1, 3);

        //añadimos el listener al grupo para que capture el evento cuando se cambie la selección
        grupo.selectedTogglePropery().addListener(
                (ObservableValue<? extends Toggle> ov, Toggle old_toggle, Toggle new_toggle) -> {
                    if (grupo.getSelectedToggle() != null) {
                        image.setImage(new Image("UD09/" + grupo.getSelectedToggle().getUserData().toString() + ".png"));
                    }
                });

        return grid;
    }

    @Override
    public void start(Stage stage) throws Exception {
        Scene scene = new Scene(createContent(), 300, 200);
        stage.setScene(scene);

        stage.setTitle("Ejemplo RadioButton");
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

⬆ Volver a teoría

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Ejemplo07

Uso de `CheckBox` con selección múltiple y evento de cambio.

```java
package UD09;

import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.control.CheckBox;
import javafx.scene.layout.GridPane;
import javafx.stage.Stage;

public class E08_CheckBox extends Application {

    private Parent createContent() {
        GridPane grid = new GridPane();
        grid.setAlignment(Pos.CENTER_LEFT);
        grid.setHgap(10);
        grid.setVgap(10);
        grid.setPadding(new Insets(25, 25, 25, 25));

        //Creamos el CheckBox vacio
        CheckBox check1 = new CheckBox();
        //Añadimos texto una vez creado
        check1.setText("Coche");
        //Añadimos el CheckBox a la columna 0 fila 0
        grid.add(check1, 0, 0);

        //Creamos los CheckBox con texto
        CheckBox check2 = new CheckBox("Moto");
        grid.add(check2, 0, 1);
        //Hademos aparezca marcado por defecto
        CheckBox check3 = new CheckBox("A pie");
        check3.setSelected(true);
        grid.add(check3, 0, 2);

        //Ahora crearemos los 3 checkboxes en un bucle y tendran 3 estados
        final String[] nombres = new String[]{"Coche3", "Moto3", "Pie3"};
        final CheckBox[] checkBox = new CheckBox[nombres.length];

        for (int i = 0; i < nombres.length; i++) {
            final CheckBox cb = checkBox[i] = new CheckBox(nombres[i]);
            cb.setAllowIndeterminate(true);
            grid.add(cb, 1, i);
        }

        return grid;
    }

    @Override
    public void start(Stage stage) throws Exception {
        Scene scene = new Scene(createContent(), 300, 200);
        stage.setScene(scene);

        stage.setTitle("Ejemplo CheckBox");
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

⬆ Volver a teoría

Ejemplo08

Uso de `TextField` para entrada de texto y manejo de eventos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD09;

import javafx.application.Application;
import javafx.event.ActionEvent;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.CheckBox;
import javafx.scene.control.Label;
import javafx.scene.control.PasswordField;
import javafx.scene.control.TextField;
import javafx.scene.layout.GridPane;
import javafx.stage.Stage;

public class E09_TextBox extends Application {

    private Parent createContent() {
        GridPane grid = new GridPane();
        grid.setAlignment(Pos.CENTER_LEFT);
        grid.setHgap(10);
        grid.setVgap(10);
        grid.setPadding(new Insets(25, 25, 25, 25));

        //Creamos el TextkField vacio
        TextField tFNombre = new TextField();
        //Establecemos el número de caracteres que mostrará por defecto
        tFNombre.setPrefColumnCount(10);
        //definimos setPromptText para que indique la información que espera el campo
        tFNombre.setPromptText("Nombre");
        grid.add(tFNombre, 0, 0);

        //Creamos el campo PasswordField que no mostrará por pantalla la información
        PasswordField tFApellidos = new PasswordField();
        tFApellidos.setPrefColumnCount(10);
        tFApellidos.setPromptText("Apellidos");
        grid.add(tFApellidos, 0, 1);

        TextField tFComentario = new TextField();
        //Establecemos el contenido por defecto del campo de texto
        tFComentario.setText("Comentario por defecto");
        //Sería lo mismo que haber creado el TextField de esta manera:
        //TextField tFComentario = new TextField("Comentario por defecto");
        tFComentario.setPrefColumnCount(10);
        tFComentario.setPromptText("Comentario");
        grid.add(tFComentario, 0, 2);

        Button btnMostraTexto = new Button("Mostrar apellidos");
        Label label = new Label();
        btnMostraTexto.setOnAction((ActionEvent e) -> {
            label.setText(tFApellidos.getText());
        });

        grid.add(label, 1, 0);
        grid.add(btnMostraTexto, 1, 1);

        return grid;

    }

    @Override
    public void start(Stage stage) throws Exception {
        Scene scene = new Scene(createContent(), 500, 200);
        stage.setScene(scene);

        stage.setTitle("Ejemplo TextField y PasswordField");
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

⬆ Volver a teoría

Ejemplo09

Clase `Cliente` para el manejo de datos de cliente en una aplicación JavaFX.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD09._02_VBoxHBox;

public class Cliente {
    private String nombre;
    private String apellidos;

    public Cliente(String nombre, String apellidos) {
        this.nombre = nombre;
        this.apellidos = apellidos;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getApellidos() {
        return apellidos;
    }

    public void setApellidos(String apellidos) {
        this.apellidos = apellidos;
    }
}
```

⬆ Volver a teoría

Ejemplo10

Layout con `VBox` y `HBox`: disposición vertical y horizontal de nodos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD09._02_VBoxHBox;

import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Hyperlink;
import javafx.scene.control.Separator;
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.HBox;
import javafx.scene.layout.Priority;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class VBoxAndHBoxApp extends Application {

    @Override
    public void start(Stage primaryStage) throws Exception {

        VBox vbox = new VBox();

        HBox controlesArriba = new HBox();
        VBox.setMargin( controlesArriba, new Insets(10.0d) );
        controlesArriba.setAlignment( Pos.BOTTOM_LEFT );

        Button btnActualizar = new Button("Actualizar");

        HBox topRightControls = new HBox();
        HBox.setHgrow(topRightControls, Priority.ALWAYS );
        topRightControls.setAlignment( Pos.BOTTOM_RIGHT );
        Hyperlink lnkCerrarSesion = new Hyperlink("Cerrar sesión");
        topRightControls.getChildren().add( lnkCerrarSesion );

        controlesArriba.getChildren().addAll( btnActualizar, topRightControls );

        TableView<Cliente> tblClientes = new TableView<>();
        tblClientes.setColumnResizePolicy(TableView.CONSTRAINED_RESIZE_POLICY);
        VBox.setMargin( tblClientes, new Insets(0.0d, 10.0d, 10.0d, 10.0d) );
        VBox.setVgrow( tblClientes, Priority.ALWAYS );

        TableColumn<Cliente, String> columnaApellidos = new TableColumn<>("Apellidos");
        columnaApellidos.setCellValueFactory(new PropertyValueFactory<>("apellidos"));

        TableColumn<Cliente, String> columnaNombre = new TableColumn<>("Nombre");
        columnaNombre.setCellValueFactory(new PropertyValueFactory<>("nombre"));

        tblClientes.getColumns().addAll( columnaApellidos, columnaNombre );

        Separator sep = new Separator();

        HBox controlesAbajo = new HBox();
        controlesAbajo.setAlignment(Pos.BOTTOM_RIGHT );
        VBox.setMargin( controlesAbajo, new Insets(10.0d) );

        Button btnCerrar = new Button("Cerrar");

        controlesAbajo.getChildren().add( btnCerrar );

        vbox.getChildren().addAll(
                controlesArriba,
                tblClientes,
                sep,
                controlesAbajo
        );

        Scene scene = new Scene(vbox );

        primaryStage.setScene( scene );
        primaryStage.setWidth( 800 );
        primaryStage.setHeight( 600 );
        primaryStage.setTitle("Aplicación con VBox y HBox");
        primaryStage.setOnShown( (evt) -> loadTable(tblClientes) );
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }

    private void loadTable(TableView<Cliente> tblCustomers) {
        tblCustomers.getItems().add(new Cliente("David", "Martinez"));
        tblCustomers.getItems().add(new Cliente("Ada", "Lovelace"));
        tblCustomers.getItems().add(new Cliente("Alan", "Turing"));
    }
}
```

⬆ Volver a teoría

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Ejemplo11

Layout con `StackPane`: apilamiento de nodos superpuestos.

```java
package UD09._03_StackPane;

import javafx.application.Application;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.scene.shape.Rectangle;
import javafx.stage.Stage;

public class StackPaneApp extends Application {
    @Override
    public void start(Stage stage) throws Exception {
        StackPane pane = new StackPane(
                new Rectangle(200, 100, Color.BLACK),
                new Circle(40, Color.RED),
                new Button("Hello StackPane")
        );

        //pane.setAlignment(Pos.CENTER_LEFT);

        stage.setScene(new Scene(pane, 300, 300));
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

⬆ Volver a teoría

Ejemplo12

Layout con `Pane`: posicionamiento absoluto de nodos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD09._04_Pane;

import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.Hyperlink;
import javafx.scene.layout.Background;
import javafx.scene.layout.BackgroundFill;
import javafx.scene.layout.Border;
import javafx.scene.layout.CornerRadii;
import javafx.scene.layout.Pane;
import javafx.scene.layout.VBox;
import javafx.scene.paint.Color;
import javafx.scene.shape.Arc;
import javafx.scene.shape.ArcType;
import javafx.scene.shape.Circle;
import javafx.scene.text.Font;
import javafx.stage.Stage;

public class PaneApp extends Application {

    @Override
    public void start(Stage primaryStage) throws Exception {

        VBox vbox = new VBox();
        vbox.setPadding( new Insets( 10 ) );
        vbox.setBackground(
            new Background(
                new BackgroundFill(Color.BLACK, new CornerRadii(0), new Insets(0))
                ));

        Pane p = new Pane();

        Arc largeArc = new Arc(0, 0, 100, 100, 270, 90);
        largeArc.setFill(Color.web("0x59291E"));
        largeArc.setType(ArcType.ROUND);

        Arc backgroundArc = new Arc(0, 0, 160, 160, 270, 90 );
        backgroundArc.setFill( Color.web("0xD96F32") );
        backgroundArc.setType( ArcType.ROUND );

        Arc smArc1 = new Arc( 0, 160, 30, 30, 270, 180);
        smArc1.setFill(Color.web("0xF2A444"));
        smArc1.setType(ArcType.ROUND);

        Circle smCircle = new Circle(
            160/Math.sqrt(2.0), 160/Math.sqrt(2.0), 30,Color.web("0xF2A444")
            );

        Arc smArc2 = new Arc( 160, 0, 30, 30, 180, 180);
        smArc2.setFill(Color.web("0xF2A444"));
        smArc2.setType(ArcType.ROUND);

        Hyperlink hyperlink = new Hyperlink("Acerca de esta App");
        hyperlink.setFont( Font.font(36) );
        hyperlink.setTextFill( Color.web("0x3E6C93") );
        hyperlink.setBorder( Border.EMPTY );

        Arc medArc = new Arc(568-20, 320-20, 60, 60, 90, 90);
        medArc.setFill(Color.web("0xD9583B"));
        medArc.setType(ArcType.ROUND);

        p.getChildren().addAll( backgroundArc, largeArc, smArc1, smCircle,
            smArc2, hyperlink, medArc );

        vbox.getChildren().add( p );

        Scene scene = new Scene(vbox);
        scene.setFill(Color.BLACK);

        primaryStage.setTitle("Aplicación Pane");
        primaryStage.setScene( scene );
        primaryStage.setWidth( 568 );
        primaryStage.setHeight( 320 );
        primaryStage.setOnShown( (evt) -> {
             hyperlink.setLayoutX( 284 - (hyperlink.getWidth()/3) );
             hyperlink.setLayoutY( 160 - hyperlink.getHeight() );
        });
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

⬆ Volver a teoría

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Ejemplo13

Layout con `GridPane`: disposición en cuadrícula de nodos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package UD09._05_GridPane;

import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.ButtonBar;
import javafx.scene.control.ComboBox;
import javafx.scene.control.Label;
import javafx.scene.control.Separator;
import javafx.scene.control.TextArea;
import javafx.scene.control.TextField;
import javafx.scene.layout.GridPane;
import javafx.scene.layout.Priority;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class GridPaneApp extends Application {

    @Override
    public void start(Stage primaryStage) throws Exception {

        VBox vbox = new VBox();

        GridPane gp = new GridPane();
        gp.setPadding(new Insets(10));
        gp.setHgap(4);
        gp.setVgap(8);

        VBox.setVgrow(gp, Priority.ALWAYS);

        Label lblTitle = new Label("Tiquet de soporte");

        Label lblEmail = new Label("Correo-e");
        TextField tfEmail = new TextField();

        Label lblPriority = new Label("Prioridad");
        ObservableList<String> priorities
                = FXCollections.observableArrayList("Media", "Alta", "Baja");
        ComboBox<String> cbPriority = new ComboBox<>(priorities);

        Label lblProblem = new Label("Problema");
        TextField tfProblem = new TextField();

        Label lblDescription = new Label("Descripción");
        TextArea taDescription = new TextArea();

        gp.add(lblTitle, 1, 1);  // empty item at 0,0
        gp.add(lblEmail, 0, 2);
        gp.add(tfEmail, 1, 2);
        gp.add(lblPriority, 0, 3);
        gp.add(cbPriority, 1, 3);
        gp.add(lblProblem, 0, 4);
        gp.add(tfProblem, 1, 4);
        gp.add(lblDescription, 0, 5);
        gp.add(taDescription, 1, 5);

        Separator sep = new Separator(); // hr

        ButtonBar buttonBar = new ButtonBar();
        buttonBar.setPadding(new Insets(10));

        Button saveButton = new Button("Guardar");
        Button cancelButton = new Button("Cancelar");

        buttonBar.setButtonData(saveButton, ButtonBar.ButtonData.OK_DONE);
        buttonBar.setButtonData(cancelButton, ButtonBar.ButtonData.CANCEL_CLOSE);

        buttonBar.getButtons().addAll(saveButton, cancelButton);

        vbox.getChildren().addAll(gp, sep, buttonBar);

        //para mostrar las lineas de estructura descomenta la siguiente linea
        //gp.setGridLinesVisible(true);

        Scene scene = new Scene(vbox);

        primaryStage.setTitle("Grid Pane App");
        primaryStage.setScene(scene);
        primaryStage.setWidth(736);
        primaryStage.setHeight(414);
        primaryStage.show();

    }

    public static void main(String[] args) {
        launch(args);
    }
}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

⬆ Volver a teoría

Ejemplo14

Layout con `GridPane` avanzado: celdas combinadas y `setColumnSpan`.

```
Package UD09._05_GridPane;

import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.ButtonBar;
import javafx.scene.control.ComboBox;
import javafx.scene.control.Label;
import javafx.scene.control.Separator;
import javafx.scene.control.TextArea;
import javafx.scene.control.TextField;
import javafx.scene.layout.GridPane;
import javafx.scene.layout.Priority;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class GridPaneAppv2 extends Application {

    @Override
    public void start(Stage primaryStage) throws Exception {

        VBox vbox = new VBox();

        GridPane gp = new GridPane();
        gp.setPadding(new Insets(10));
        gp.setHgap(4);
        gp.setVgap(10);

        VBox.setVgrow(gp, Priority.ALWAYS);

        Label lblTitle = new Label("Tiquet de soporte");

        Label lblEmail = new Label("Correo-e");
        TextField tfEmail = new TextField();

        Label lblContract = new Label("Contrato");
        TextField tfContract = new TextField();

        Label lblPriority = new Label("Prioridad");
        ObservableList<String> priorities
                = FXCollections.observableArrayList("Media", "Alta", "Baja");
        ComboBox<String> cbPriority = new ComboBox<>(priorities);

        Label lblSeverity = new Label("Severidad");
        ObservableList<String> severities
                = FXCollections.observableArrayList("Bloqueante", "Salvable", "No importa");
        ComboBox<String> cbSeverity = new ComboBox<>(severities);

        Label lblCategory = new Label("Categoria");
        ObservableList<String> categories
                = FXCollections.observableArrayList("Defecto", "Nueva funcionalidad");
        ComboBox<String> cbCategory = new ComboBox<>(categories);

        Label lblProblem = new Label("Problema");
        TextField tfProblem = new TextField();

        Label lblDescription = new Label("Descripción");
        TextArea taDescription = new TextArea();

        gp.add(lblTitle, 1, 0);  // empty item at 0,0

        gp.add(lblEmail, 0, 1);
        gp.add(tfEmail, 1, 1);
        gp.add(lblContract, 4, 1);
        gp.add(tfContract, 5, 1);

        gp.add(lblPriority, 0, 2);
        gp.add(cbPriority, 1, 2);
        gp.add(lblSeverity, 2, 2);
        gp.add(cbSeverity, 3, 2);
        gp.add(lblCategory, 4, 2);
        gp.add(cbCategory, 5, 2);

        gp.add(lblProblem, 0, 3);
        gp.add(tfProblem, 1, 3);
        gp.add(lblDescription, 0, 4);
        gp.add(taDescription, 1, 4);
```

⬆ Volver a teoría

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

10.1.8 Resumen — Conceptos clave

Concepto	Definición
Stage	Ventana principal de la aplicación JavaFX
Scene	Contenedor del grafo de escena
Node	Elemento del grafo de escena (botón, texto, imagen...)
Layout	Contenedor que organiza los nodos hijos
FXML	Lenguaje de marcado para definir interfaces JavaFX
SceneBuilder	Herramienta visual para diseñar interfaces FXML
Evento	Acción del usuario que desencadena una respuesta

10.1.9 Autoevaluación

- [x] Comprendo la arquitectura Stage-Scene-Node
- [x] Creo interfaces con diferentes layouts
- [x] Manejo eventos de botones y teclado
- [x] Diseño interfaces con Scene Builder
- [x] Separo la interfaz (FXML) de la lógica (Java)
- [x] Creo proyectos JavaFX con Maven

10.1.10 Vídeos recomendados

Canal	Vídeo	Contenido
EducaMadrid	JavaFX (introducción)	Introducción a JavaFX
Píldoras Informáticas	Curso Java — Swing / Interfaces gráficas	Vídeos 100–150: creación de interfaces gráficas (conceptos transferibles a JavaFX)
Píldoras Informáticas	Curso Java 2026 — Interfaces gráficas	Nueva edición: componentes Swing, eventos, layouts

20 de julio de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

10.2 Ejercicios de la UD09

10.2.1 Cuestiones generales

Implementa las aplicaciones gráficas siguiendo las especificaciones de diseño y funcionalidad descritas en cada ejercicio. Ten en cuenta lo siguiente:

- Los aspectos del diseño que no se describan quedan a tu elección. En todo caso, intenta que la interfaz sea sencilla de utilizar para el usuario.
- Renombra todos los componentes para que se entienda su uso. En lugar de `TextField1`, `Button1` y `Label1` es mejor utilizar abreviaturas: `txtEdad`, `btnSuma` y `lblRes`.
- Valida la información introducida por el usuario y avísale en caso de error.
- Maneja las posibles excepciones que puedan producirse.
- Puedes implementar clases adicionales si lo consideras oportuno.

10.2.2 Ejercicios (con `SceneBuilder`)

1. Crea con JavaFX la siguiente pantalla.

Cuando el usuario escriba números (controlarlo con excepciones) y pulse en sumar, debe mostrar en el `txtResultado` el resultado final.

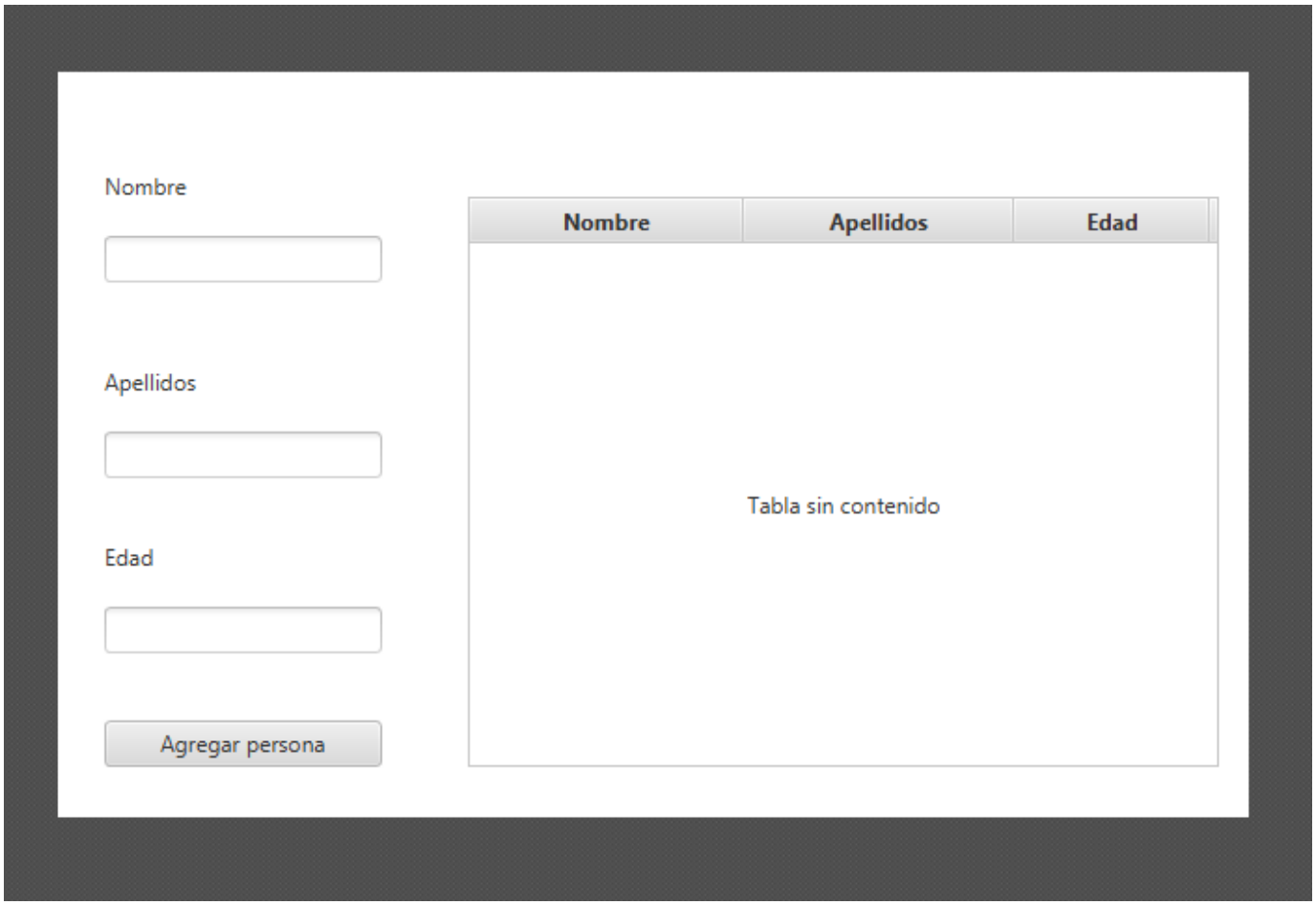
1. Crea con `JavaFX` la siguiente pantalla.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Cuando el usuario escriba números (controlarlo con excepciones) y según que operación elija, deberá hacer la operación y mostrarlo en el txtResultado. Tener en cuenta que la división, debemos controlar que el segundo operando no debe ser 0. El proyecto debe seguir el modelo MVC.

1. Crea la siguiente pantalla con `JavaFX`.

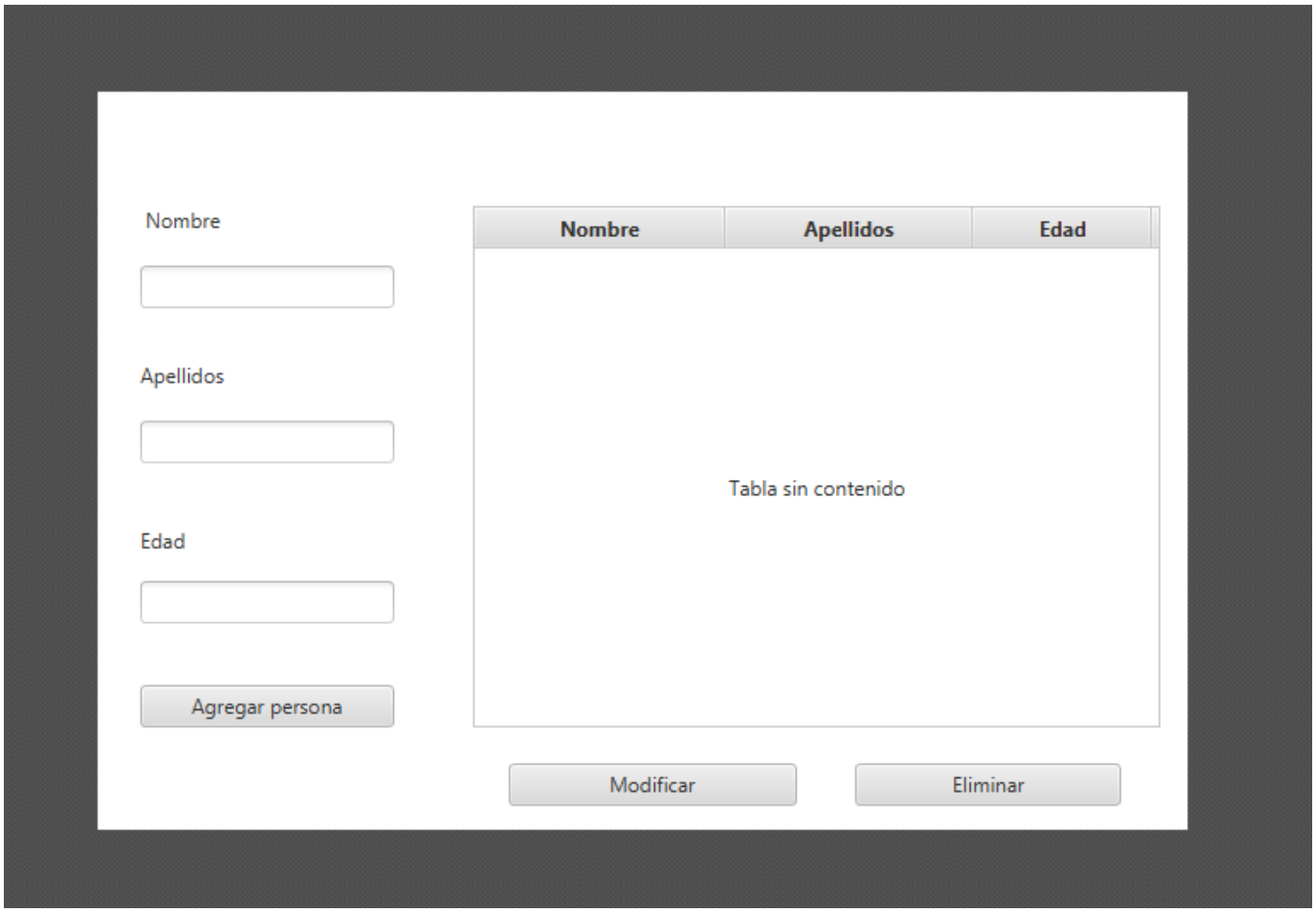
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Al pulsar en el botón de agregar persona, creara un objeto `Persona` y lo insertara en la tabla.

1. Dado el ejercicio anterior, agrega lo siguiente.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



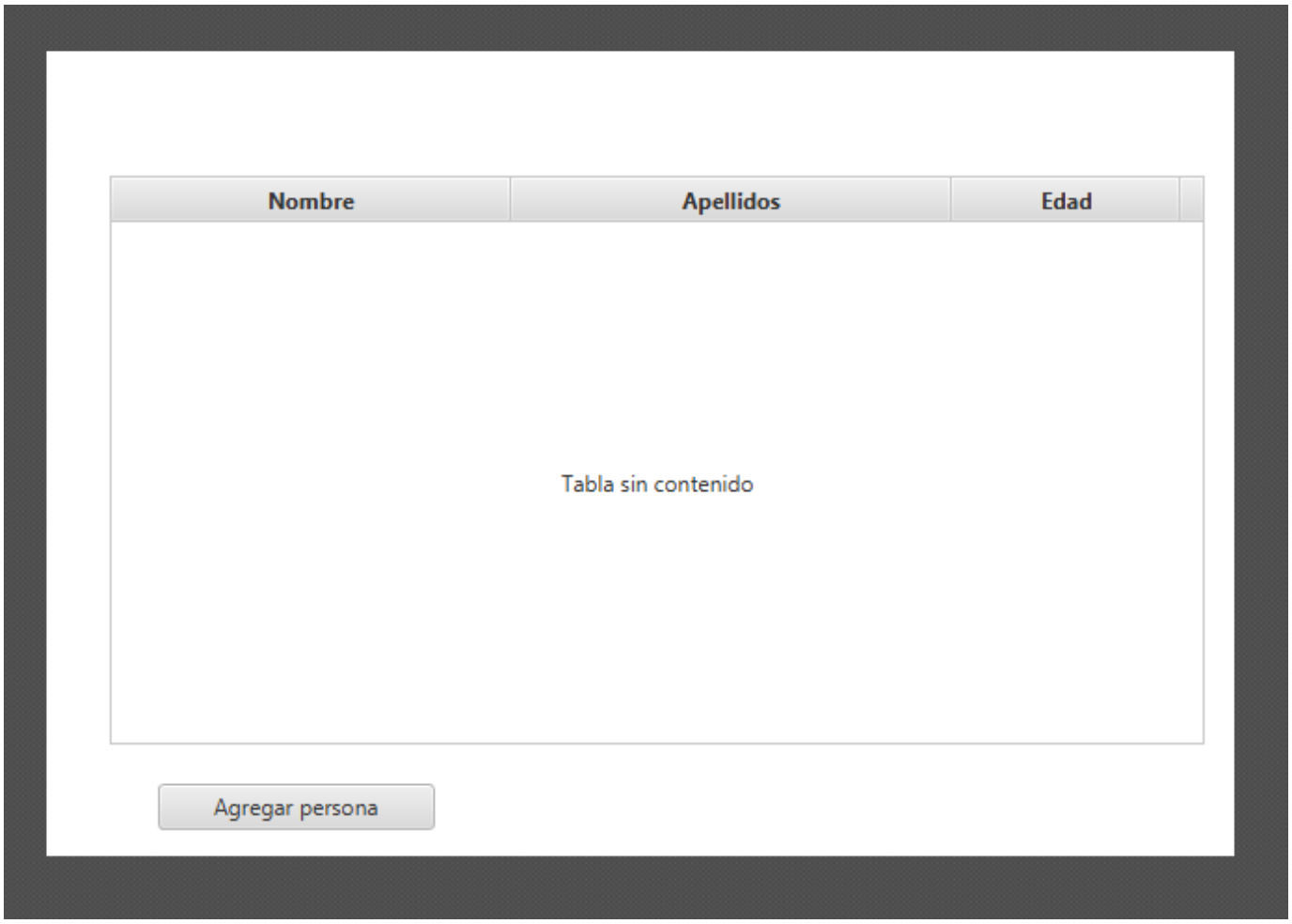
Al seleccionar en un registro, mostrara los datos en los txt.

Al pulsar el botón Modificar, el registro seleccionado se modificara con los datos que hay en los txt (editando el objeto).

Al pulsar el botón Eliminar, el registro seleccionado se eliminara de la tabla.

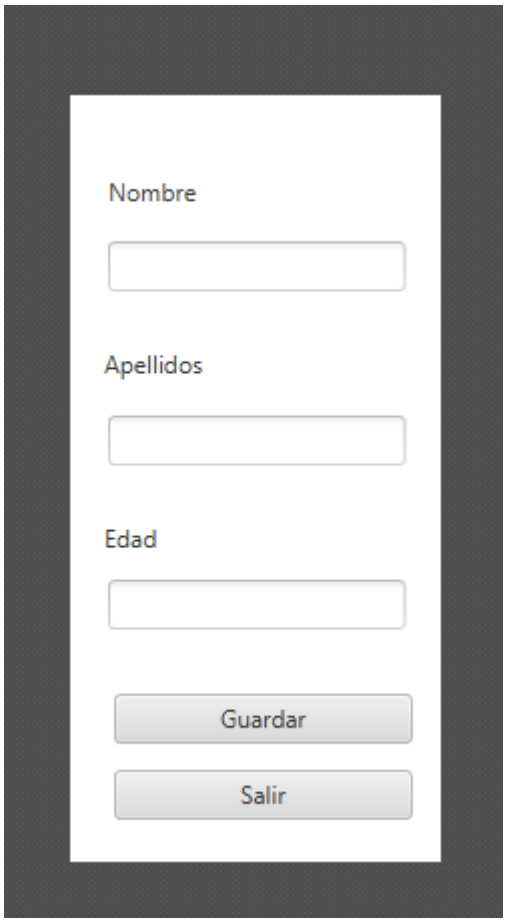
1. Dado el ejercicio anterior, modifica la ventana.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Al pulsar el botón de agregar Persona, se mostrara la siguiente ventana.

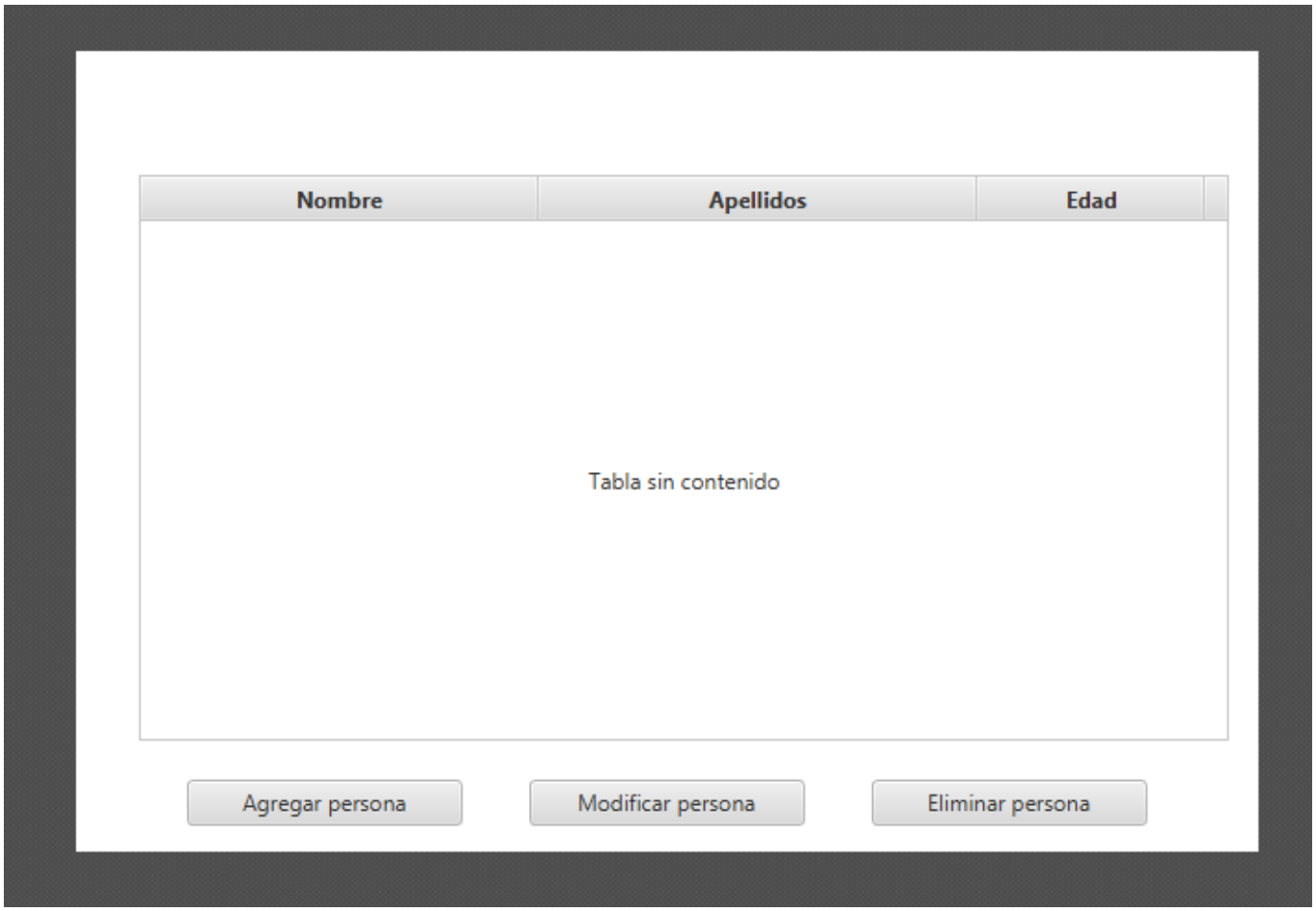
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Cuando rellenamos los datos, cerrara la ventana y añadirá la persona a la tabla.

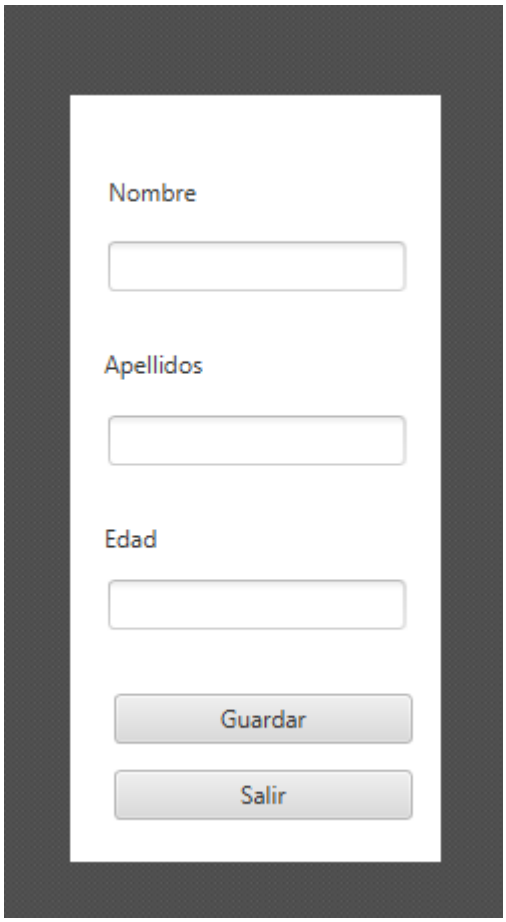
1. Dado el ejercicio anterior, modifica la ventana.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Al pulsar el botón de agregar Persona, se mostrara la siguiente ventana.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



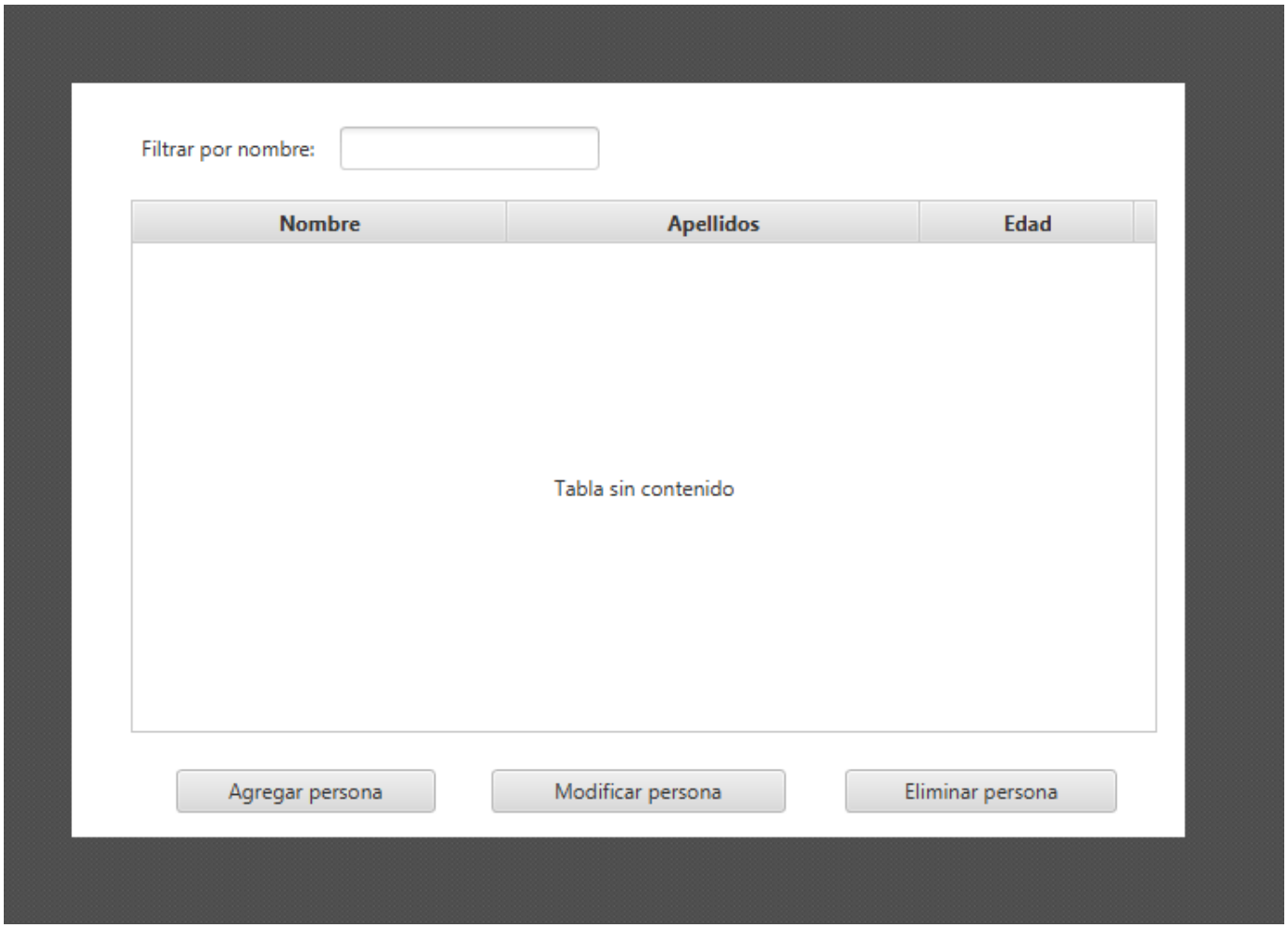
Cuando rellenamos los datos, cerrara la ventana y añadirá la persona a la tabla.

Cuando pulsemos modificar, se abrirá la ventana anterior, pero con los datos cargados y al guardar editara el objeto seleccionado.

Cuando pulsemos eliminar, eliminar el objeto de la tabla.

1. Dado el ejercicio anterior, modifica la ventana (se mantiene el dialogo de persona).

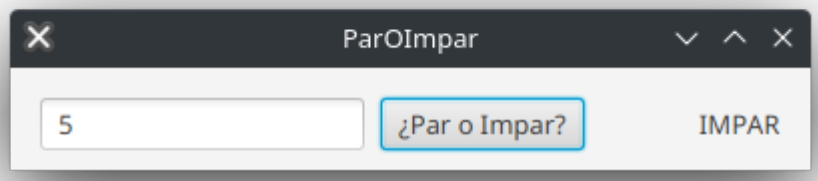
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Cuando escribamos en el txt de arriba, filtrara los nombre de las personas que contengan el texto que tu indiques.

10.2.3 Actividades

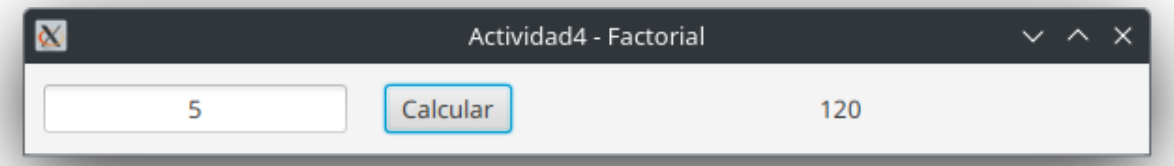
1. `ParOImpar` Aplicación gráfica que permita introducir un número entero y luego saber si dicho número es par o impar. Utiliza un `TextField` para introducir el valor, un `Button` con el texto "¿par o impar?" y un `Label` para mostrar "PAR" o "IMPAR" según el caso.



1. `Minicalculadora1` Aplicación gráfica que permita introducir dos números reales y calcular el resultado de su suma, resta, multiplicación o división. Utiliza un Button distinto para cada operación y un único Label para mostrar el resultado.
2. `Minicalculadora2` Aplicación gráfica que permite introducir dos números enteros (A y B) y permita realizar tres cálculos distintos: suma de A y B, producto de A x B y exponencial A^B (A elevado a B). Utiliza tres `RadioButton` (uno para cada cálculo) y un botón "¡Calcula!".

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

3. `Factorial` Aplicación de uso de JavaFX con FXML y controlador para calcular el factorial de un número.



1. `ValidarletraNIF` Aplicación gráfica que permita introducir un NIF (8 números y una letra) e indique si la letra de dicho NIF es válida.
2. `DadosDeRol` Aplicación gráfica que permita al usuario simular que lanza un dado de juegos de rol. Podrá elegir entre dado de 6 caras (de 1 a 6), dado de 8 caras (de 1 a 8), dado de 10 caras (de 1 a 10), dado de 12 caras (de 1 a 12) y dado de 20 caras (de 1 a 20). Utiliza un botón distinto para cada tipo de dado. Muestra en cada botón una imagen de cada dado. Puedes encontrar las imágenes fácilmente haciendo una búsqueda en Internet.
3. `InicioDeSesion` Aplicación gráfica que simule una ventana de inicio de sesión y registro de usuarios. El usuario podrá introducir su nombre de usuario, contraseña (PasswordField) y hacer click en un botón de “Iniciar sesión”. Muestra el resultado del intento de inicio de sesión en un Label.

Los usuarios registrados y sus contraseñas estarán en el archivo ‘users.txt’. Crea unos pocos usuarios de ejemplo para probar la aplicación. No está permitido utilizar espacios ni en los nombres de usuario ni en las contraseñas.

Extra: Añade un botón “Crear usuario” que registre un nuevo usuario.

1. `BurguerMenApp` Implementa una aplicación gráfica que simule una aplicación de pedidos de menús de hamburguesas. La aplicación permitirá configurar un solo menú. El usuario podrá elegir el tipo de hamburguesa, pan, patatas y bebida. También hay elementos opcionales/extra.

El precio de un menú básico es de 8 euros pero algunas opciones tienen un coste adicional. Se mostrará el precio total del menú con las opciones elegidas, el IVA en € (21% adicional) y el precio de venta al público (tras sumar el IVA).

Las opciones obligatorias a elegir son:

- Hamburguesa a elegir: pollo, cerdo, ternera (+1€) o vegana (+1€).
- Pan a elegir: normal, integral o centeno.
- Patatas a elegir: fritas, gajo y caseras (+1 €).
- Bebida a elegir: refresco de cola, refresco de naranja, refresco de limón, agua y cerveza.

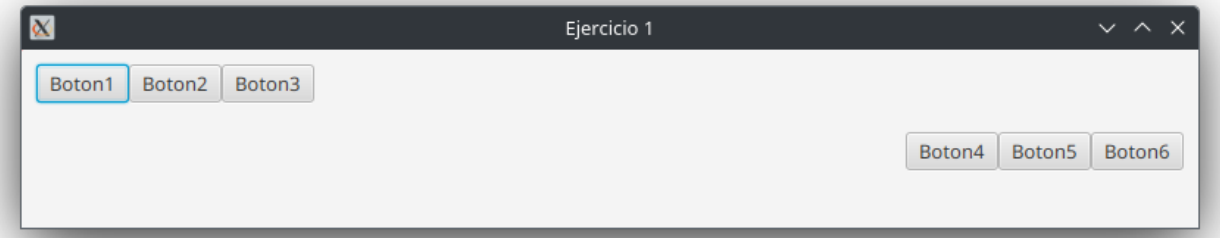
Las opciones extra/adicionales son:

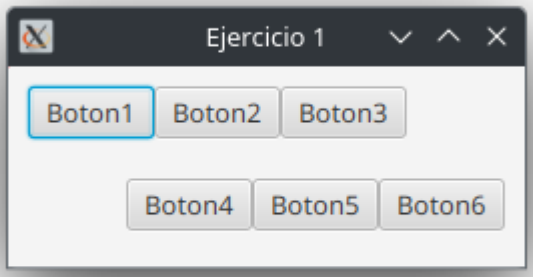
- Hamburguesa doble (+2 €).
- Extra de queso (+0,50 €).
- Extra de patatas (+1 €).
- Salsas: ketchup, barbacoa, mostaza y thai. Pueden pedirse varias de cada (+0,50 cada una).
- Reparto a domicilio (precio estándar) o recogida en el local (-20% sobre el precio final).

10.2.4 Más ejercicios (Sin `SceneBuilder`)

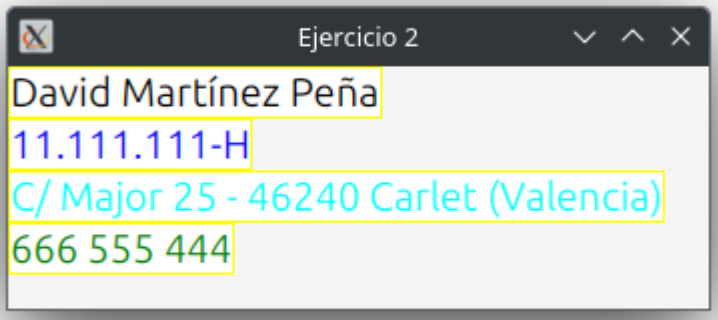
1. Ejercicio 1 Escriba un programa que muestre la siguiente `stage` usando más de un contenedor. Los tres botones superiores se alinearan a la parte izquierda de la ventana, y los tres inferiores a la parte derecha.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0





1. Ejercicio 2 Escriba un programa que muestre cuatro líneas de texto con cuatro `Label`: - Escribe tu nombre, DNI, dirección, teléfono en cada una de las etiquetas. - Establece el fondo de las etiquetas en blanco. - Establece el color del texto de las etiquetas en negro, azul, cian, verde respectivamente. - Establece la fuente de cada etiqueta en TimesNewRoman/Ubuntu, negrita y 20 píxeles. - Establece el borde de cada etiqueta en un borde de línea con color amarillo.



1. Ejercicio 3 Muestre un `stage` que contenga nueve etiquetas. Una etiqueta puede mostrar un icono de imagen para X, un icono de imagen para O, o nada. Lo que se muestra se decide al azar. Utilizar el Método `Math.random()` para generar un número entero 0, 1 o 2, que corresponde a un icono de imagen X, un icono O o nada (una imagen vacía).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

10 de enero de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

10.3 Talleres

10.3.1 Taller UD09_02: Scene Builder, ScenicView y FXMLManager

Introducción

En el desarrollo de aplicaciones con JavaFX, la eficiencia y la claridad en la creación de interfaces gráficas son fundamentales. Herramientas como **SceneBuilder** permiten diseñar interfaces de usuario de manera visual y rápida, arrastrando y soltando componentes, lo que facilita la creación de layouts complejos sin necesidad de escribir manualmente todo el código FXML. Por otro lado, **ScenicView** es una herramienta invaluable para depurar y analizar la estructura de la interfaz gráfica en tiempo de ejecución, permitiendo identificar problemas de diseño o rendimiento. Finalmente, **FXMLManager** ayuda a gestionar y organizar los archivos FXML, promoviendo un código más modular y mantenible. Juntas, estas herramientas integradas en IntelliJ IDEA optimizan el flujo de trabajo, permitiendo a los desarrolladores centrarse en la lógica de la aplicación mientras mantienen un diseño de interfaz limpio y funcional.

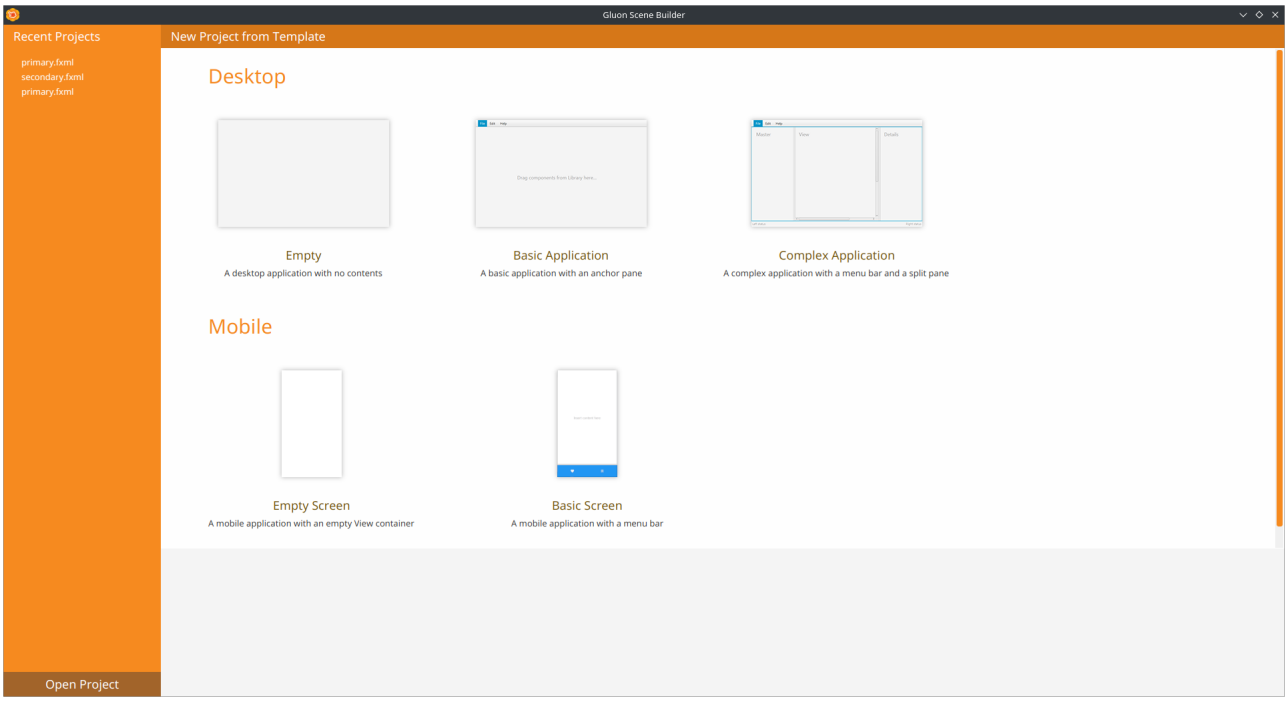
SceneBuilder

Scene Builder es una alternativa orientada al diseño que puede ser más productiva. Además es multiplataforma y está disponible para GNU/Linux, Windows y Mac. Scene Builder funciona con el ecosistema JavaFX: controles oficiales, proyectos comunitarios y ofertas de Gluon que incluyen Gluon Mobile, Gluon Desktop y Gluon CloudLink.

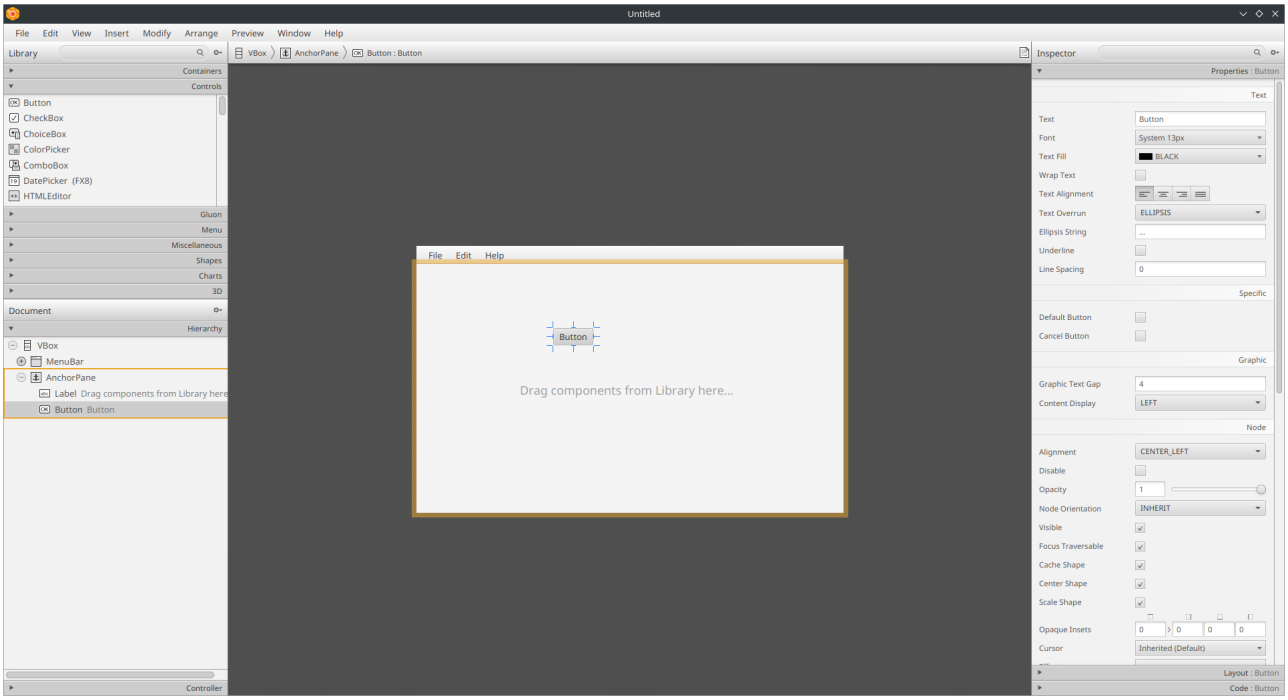
El diseño de la interfaz de usuario *drag&drop* permite una iteración rápida. La separación de los archivos de diseño y lógica permite que los miembros del equipo se concentren rápida y fácilmente en su capa específica de desarrollo de aplicaciones.

Scene Builder es gratuito y de código abierto, pero cuenta con el respaldo de Gluon. Están disponibles ofertas de soporte comercial, que incluyen formación y servicios de consultoría personalizados.

Descarga e información: https://gluonhq.com/products/scene-builder/



© 2025 David Martínez licensed under CC BY-NC-SA 4.0

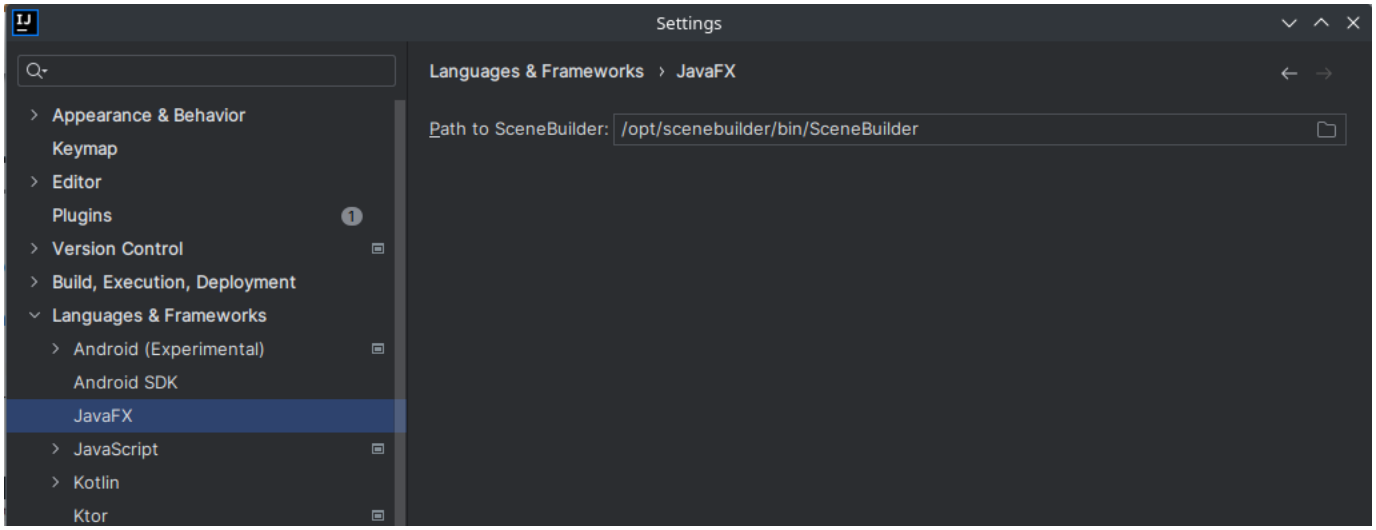


CONFIGURAR EN `IntelliJ` LA LOCALIZACIÓN DE `SceneBuilder`

Con el fin de que cuando se abra un archivo `FXML` desde `NetBeans` se muestre directamente con la herramienta `SceneBuider`, se debe indicar en la configuración de NetBeans en qué carpeta se encuentra `SceneBuider`.

En el artículo Using Scene Builder with IntelliJ IDEA de la web de Oracle se puede obtener también información sobre los pasos a seguir.

SceneBuilder se encuentra instalada por defecto en la carpeta `C:\Users\TU_USUARIO\AppData\Local\SceneBuilder\SceneBuilder.exe` (en Windows), tambien es posible usar esta aplicación en Mac o Linux. Lo más importante es que conozcas la ubicación del ejecutable y lo configures en IntelliJ:

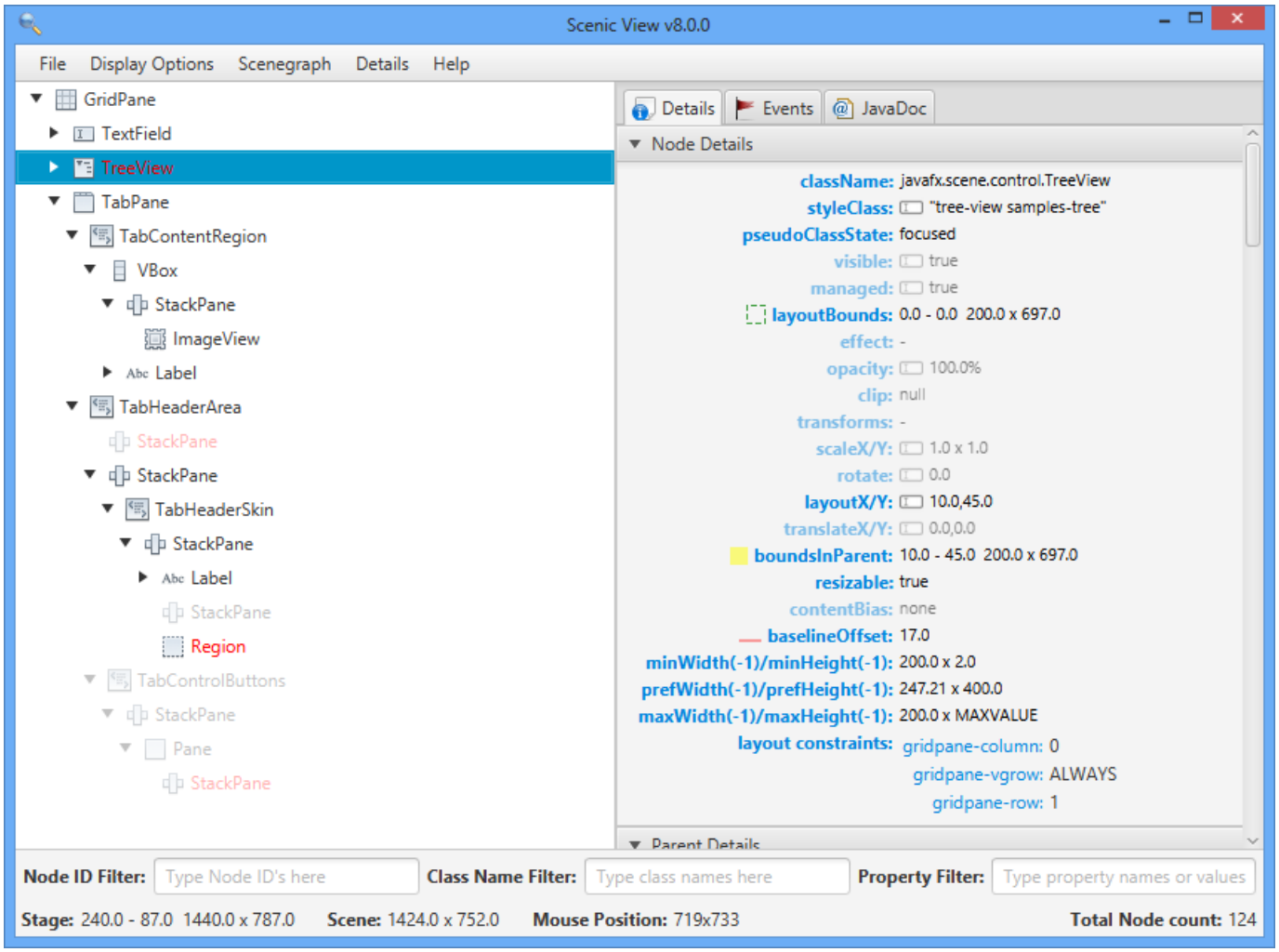


Puedes acceder a las opciones de configuración de NetBeans en el menú `Settings` -> `Languages & Frameworks` -> `JavaFx`. Ahí accede a la sección `Path to SceneBuilder` y escribe la ruta donde se encuentre el ejecutable de SceneBuilder (en la imágen superior tienes el ejemplo de mi máquina linux).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

`ScenicView`

Scenic View es una aplicación `JavaFX` diseñada para simplificar la comprensión del estado actual del gráfico de escena de su aplicación y también para manipular fácilmente las propiedades del gráfico de escena sin tener que seguir editando su código. Esto le permite encontrar errores y hacer que las cosas sean perfectas sin tener que hacer el baile de compilación, verificación y compilación.



DESCARGAR E INSTALAR `ScenicView`

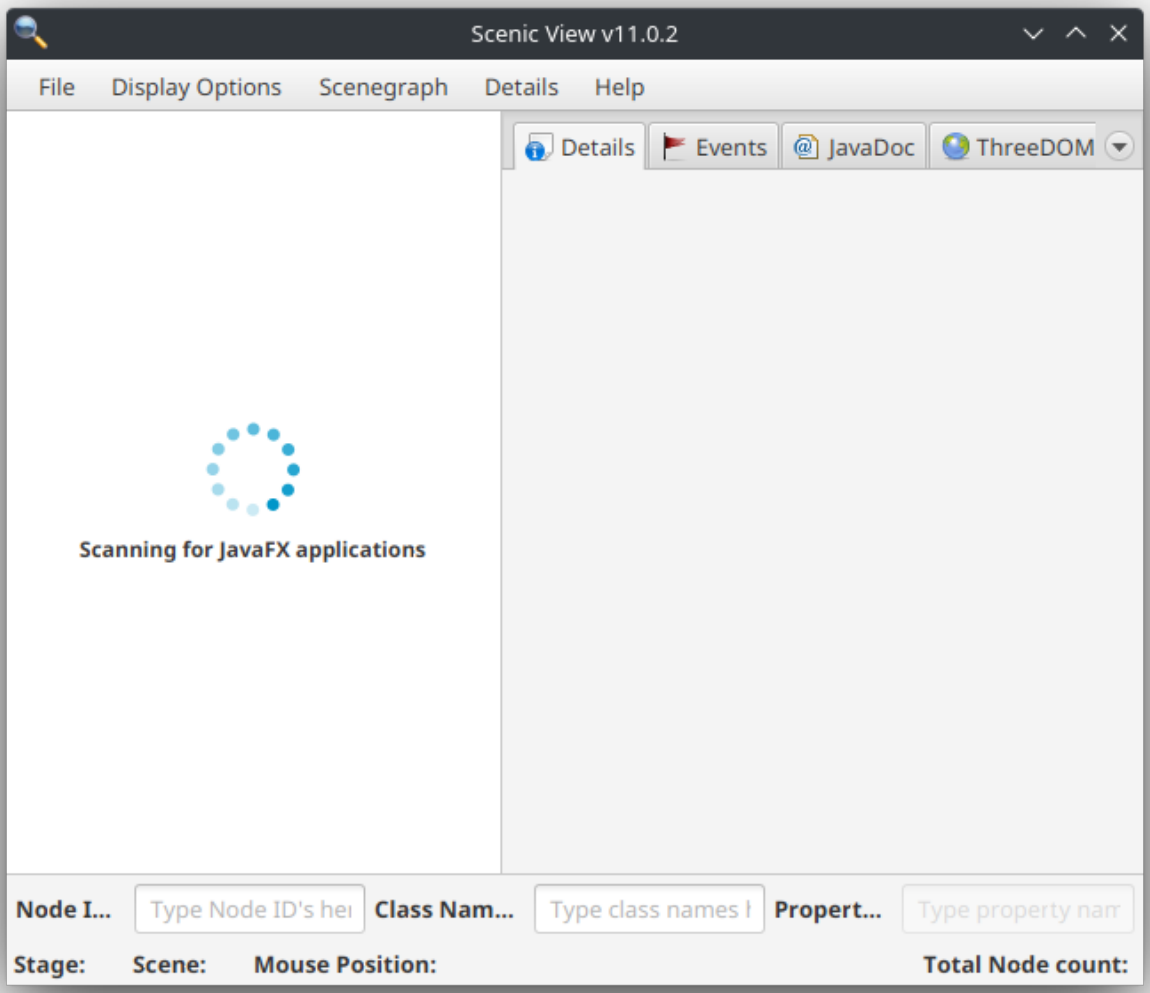
Puedes descargar la versión adecuada para tu sistema operativo desde https://github.com/JonathanGiles/scenic-view

En realidad ScenicView se distribuye como una aplicación portable. Eso quiere decir que no es necesario instalarla, sino directamente ejecutar la aplicación desde la carpeta `bin` según el sistema operativo en el que nos encontremos.

USAR `ScenicView`

Para examinar nuestra aplicación `JavaFX` con `ScenicView`, debemos en primer lugar ejecutar la aplicación `ScenicView`:

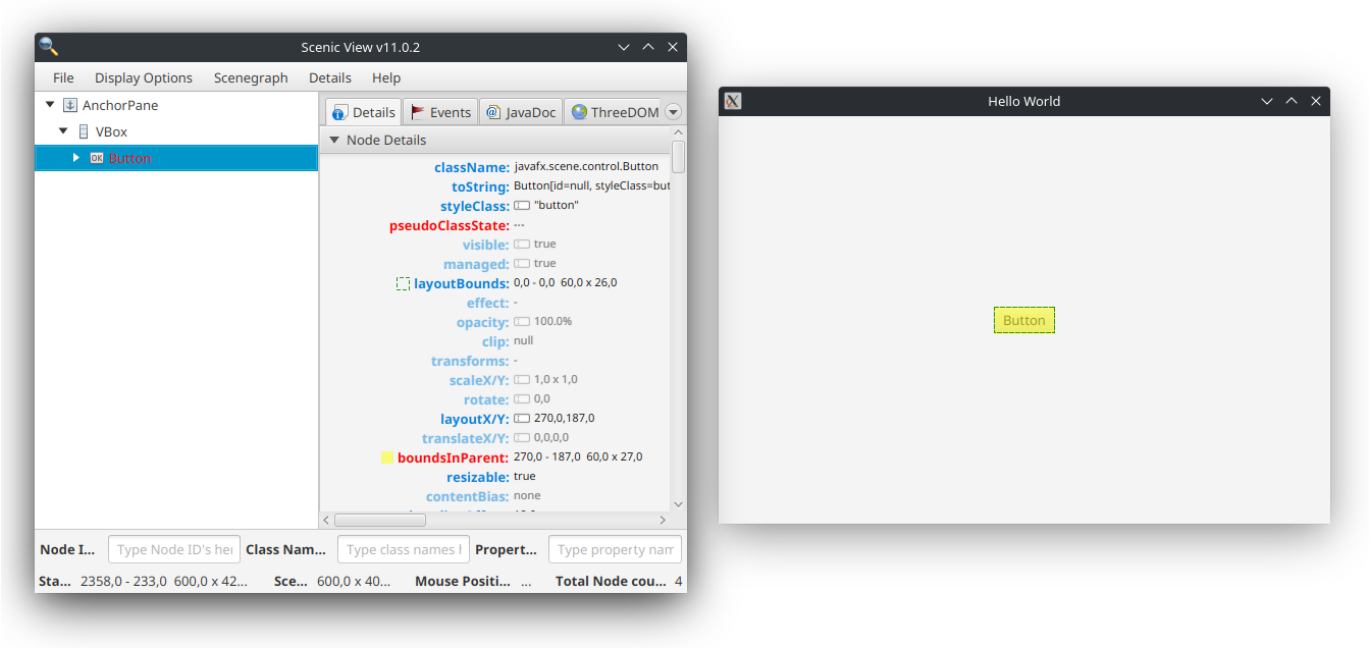
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Una vez ejecutada la aplicación comienza a buscar aplicaciones que se esten ejecutando y que usen `JavaFX`.

En este punto solo queda que ejecutemos nuestra aplicación `JavaFX` (por ejemplo `HolaFX`) y `ScenicView` detectará la aplicación y nos mostrará toda su información y nos permitirá realizar modificaciones:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



`FXMLManager`

`FXMLManager` ayuda a gestionar y organizar los archivos FXML.

Para instalar `FXMLManager` solo debes ir al apartado `Plugins` de IntelliJ y buscar en el `Market` el nombre "`FXMLManager`" y marcarlo para instalar.

Nos permite añadir/eliminar código al controlador a partir de las modificaciones que realizamos a la vista con el `SceneBuilder`.

Una vez realizadas las modificaciones en nuestra vista (y guardados los cambios), solo tenemos que pulsar el botón derecho sobre el fichero `*.fxml` correspondiente y elegir la opción "**Update Controller from FXML**".

Esto hará que se añadan las etiquetas `@FXML` con todos los componentes y métodos definidos en la vista.

Videos relacionados

- https://www.youtube.com/playlist?list=PLNjWMbvTJAIjLRW2qyuc4DEgFVW5YFRSR
- https://www.youtube.com/playlist?list=PLaxZkGlLWHGUWZxuadN3J7KKaICRlhz5-

Tarea

Sigue esta guía para instalar en tu ordenador `SceneBuilder`, `ScenicView` y `FXMLManager`.

Genera una memoria en pdf con capturas/explicaciones en las que se pueda ver en funcionamiento SceneBuilder (dede IntelliJ) y ScenicView en tu ordenador. `FXMLManager` lo usaremos en el siguiente Taller, pero puedes añadir capturas demostrando que lo has instalado como plugin.

Envía el archivo pdf a la tarea de Aules.

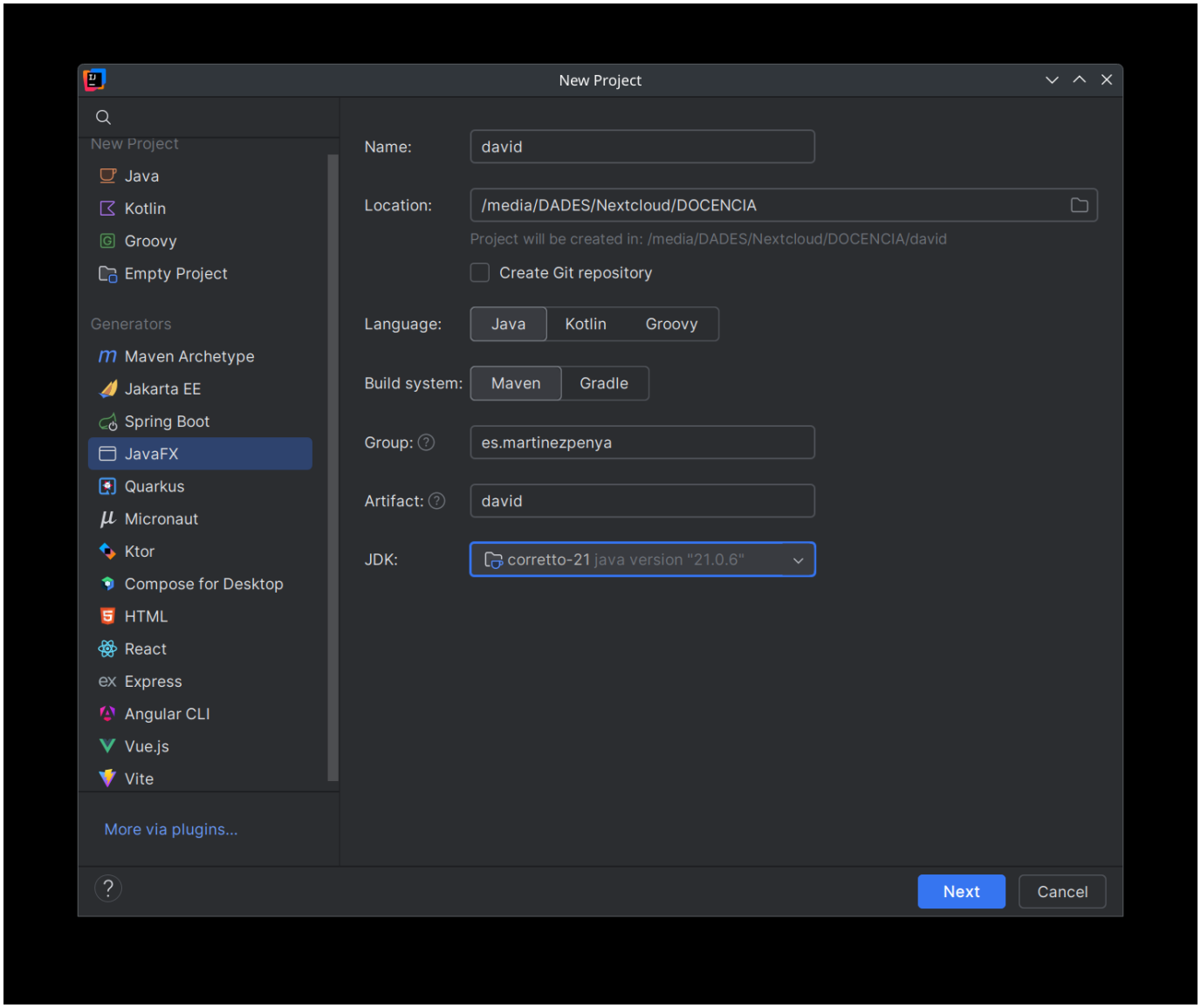
6 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

10.3.2 Taller UD09_03: Proyecto JavaFX (con Maven)

Crea un proyecto de JavaFx (con Maven)

Cuando creas un proyecto en Intellij, el mismo IDE ofrece un tipo de proyecto JavaFx:

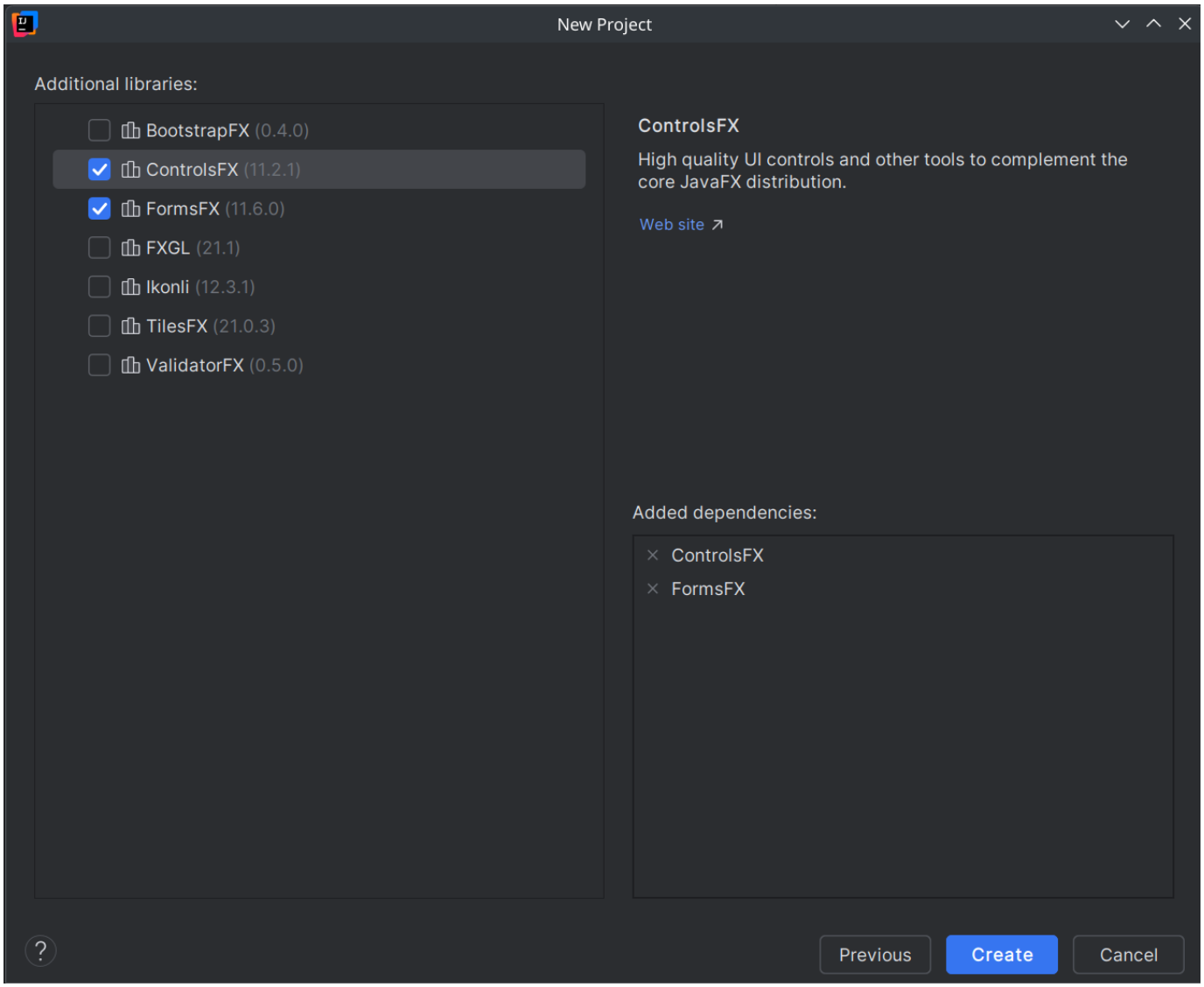


> **Importante**
>
> Fíjate que al elegir el tipo de proyecto JavaFX a la parte derecha se selecciona el gestor de librerías Maven. Más adelante veremos como gestionar manualmente Maven, de momento dejaremos que IntelliJ genere el projecto Maven para JavaFx por nosotros.

Solo debes fijarte en escribir tu nombre y apellidos como en la imagen y seleccionar la versión 21 del jdk.

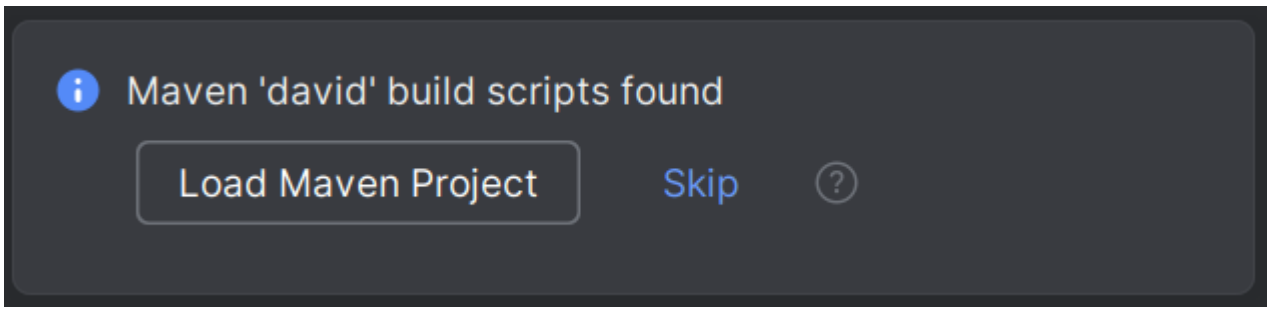
En la siguiente ventana debes marcar al menos estas dos opciones:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Y por último pulsar el botón [Create]

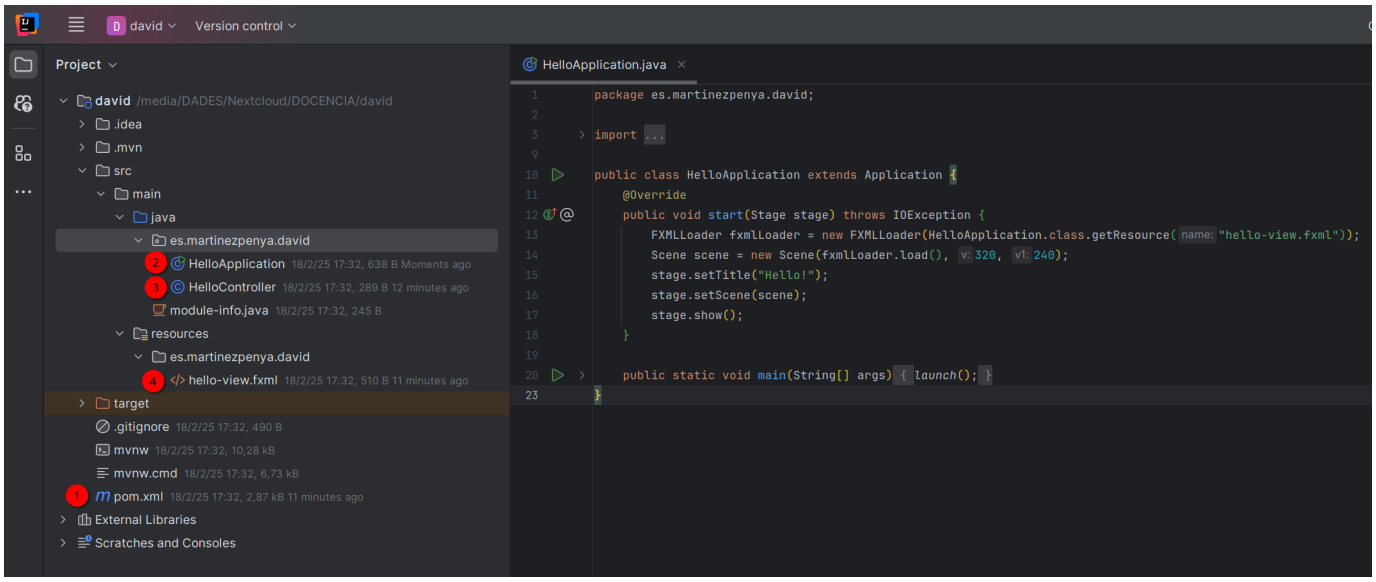
Si esperas unos segundos aparecerá una notificación en la parte inferior derecha:



Debes elegir la opción Load Maven Project, y esperar a que procese las dependencias solicitadas.

Estructura del proyecto JavaFx recien creado con el asistente de IntelliJ:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



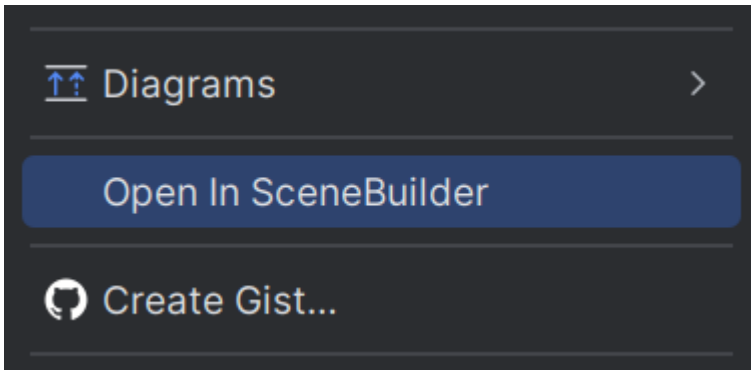
En la imágen anterior puedes encontrar:

1. fichero `pom.xml` que gestiona las dependencias del proyecto maven.
2. `HelloApplication` clase que contiene el main de la aplicación JavaFx
3. `HelloController` clase con el controlador de la aplicación
4. `hello-view.fxml` archivo que contiene el formulario (vista) de la apliación

El proyecto sigue el patrón MVC que hemos visto en teoria, solo que es tan simple que no tiene ningún Modelo.

Modificar el formulario con `SceneBuilder`

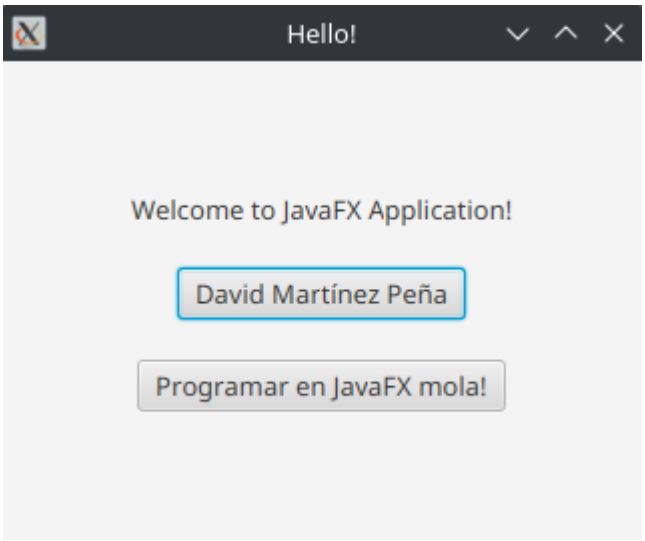
Si has configurado correctamente `SceneBuilder` en tu ordenador para usarlo desde IntelliJ, deberias poder pulsar con el botón derecho del ratón sobre el archivo (4) (`hello-view.fxml`) y elegir la opción `Abrir en SceneBuilder`:



Usa `FXMLManager`

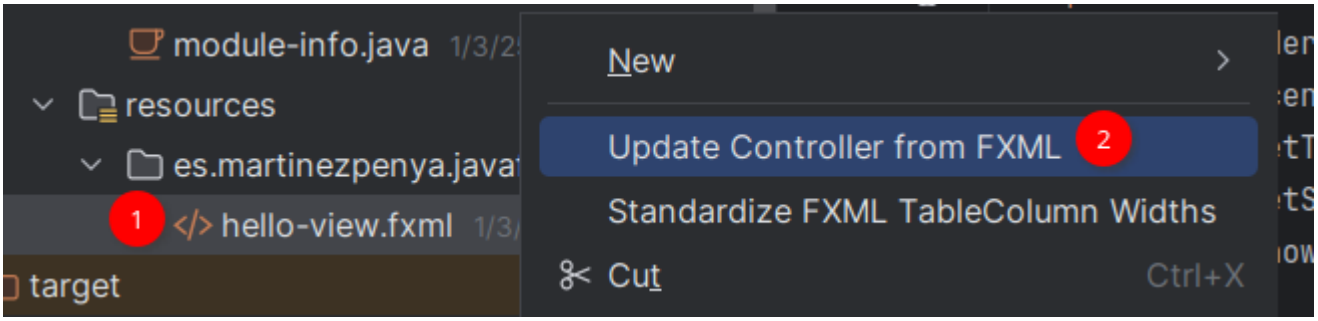
Ahora, vamos a añadir cualquier componente a nuestra interfaz (preferiblemente mediante `SceneBuilder`), en nuestro caso, por ejemplo, otro botón (al que llamaremos "mola":

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

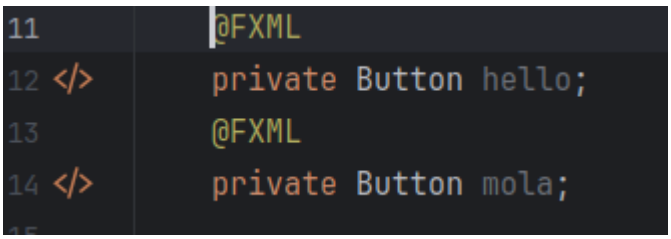


Ahora nuestra vista tiene un componente más que nuestro controlador, y aquí es donde entra en acción `FXMLManager` ya que en lugar de estar pendientes de ir añadiendo nuestras modificaciones en la Vista a nuestro Controlador, `FXMLManager` se encargará de gestionarlo.

Hacemos clic derecho sobre el archivo `hello-view.fxml` y elegimos la opción:



Ahora si abrimos de nuevo el fichero HelloController.java aparte del botón hello (que ya aparecia anteriormente) aparecerá el nuevo boton "mola":



Tarea

Usa el asistente de IntelliJ para crear proyecto `JavaFX` con maven. Modifica el formulario con `SceneBuilder` y cambia el texto del botón por tu nombre y apellidos. Añade un nuevo botón (con el nombre que quieras), pon el texto que quieras, y con `FXMLManager` añade el código correspondiente al `HelloController.java`.

Genera un zip con el proyecto de IntelliJ (o la carpeta src del IDE que uses). Envía el archivo zip a la tarea de Aules.

6 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

10.3.3 Taller UD09_04: Calculadora en JavaFX (IntelliJ)

Introducción

Vamos a intentar juntar todo lo aprendido en una guía para realizar una aplicación `JavaFX` con `SceneBuilder` i `IntelliJ`, siguiendo el modelo `MVC`.

Necesitaras:

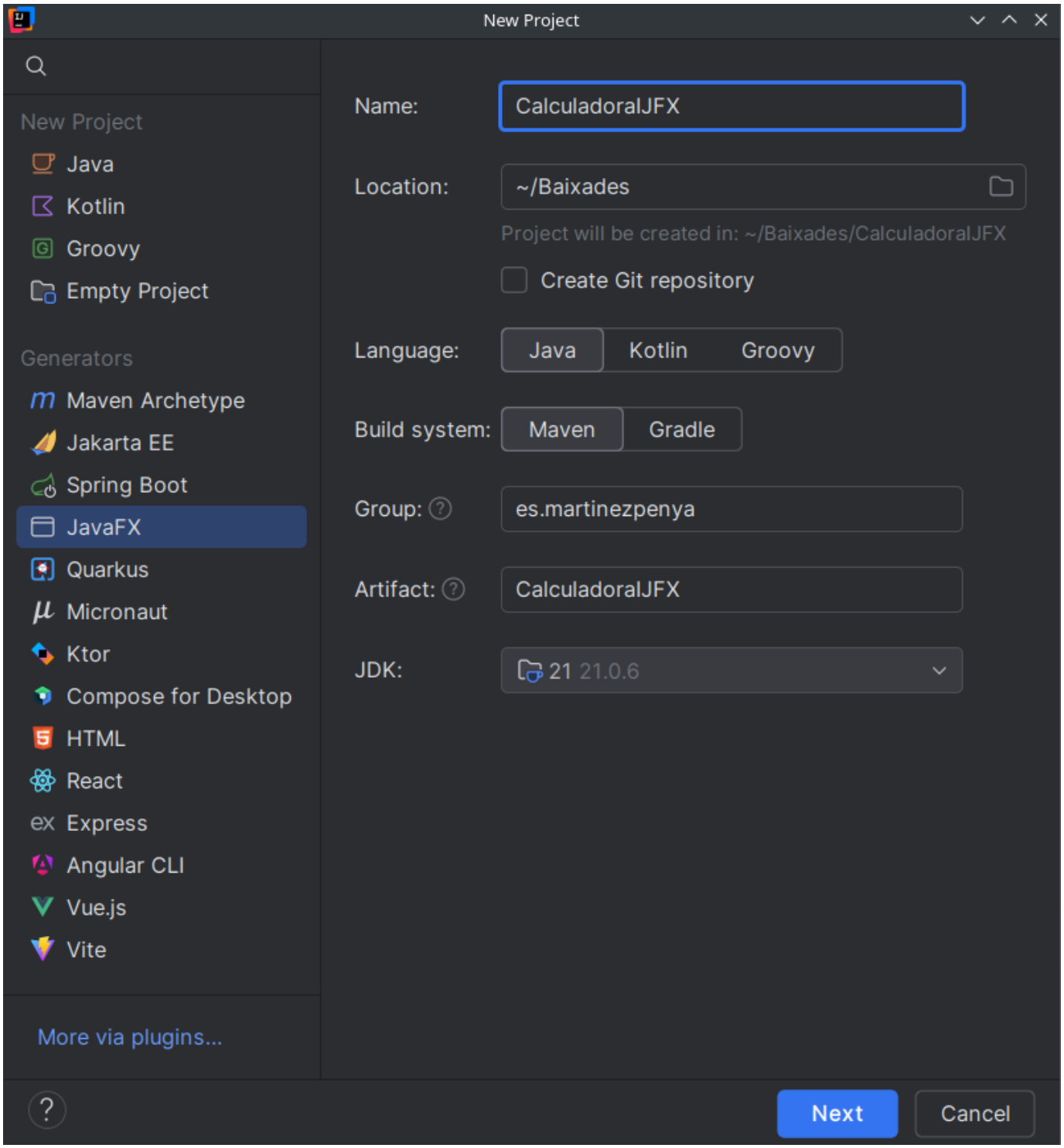
- IntelliJ Ultimate (seguramente con la comunity también funcione)
- OpenJDK 21 (seguramente funcionará con una posterior)
- JavaFx 21 LTS
- `SceneBuilder`
- `FXMLManager`
- `ScenicView`

Crear proyecto

Vamos a crear el proyecto con el template de `JavaFX`.

En nuestro caso llamamos al proyecto `CalculadoraIJFX`, yo como Group pongo "`es.martinezpenya`", pero tu puedes poner tu "dominio" (tus apellidos) y recuerda en la siguiente pantalla marcar: `ControlsFX` y `FormsFX`

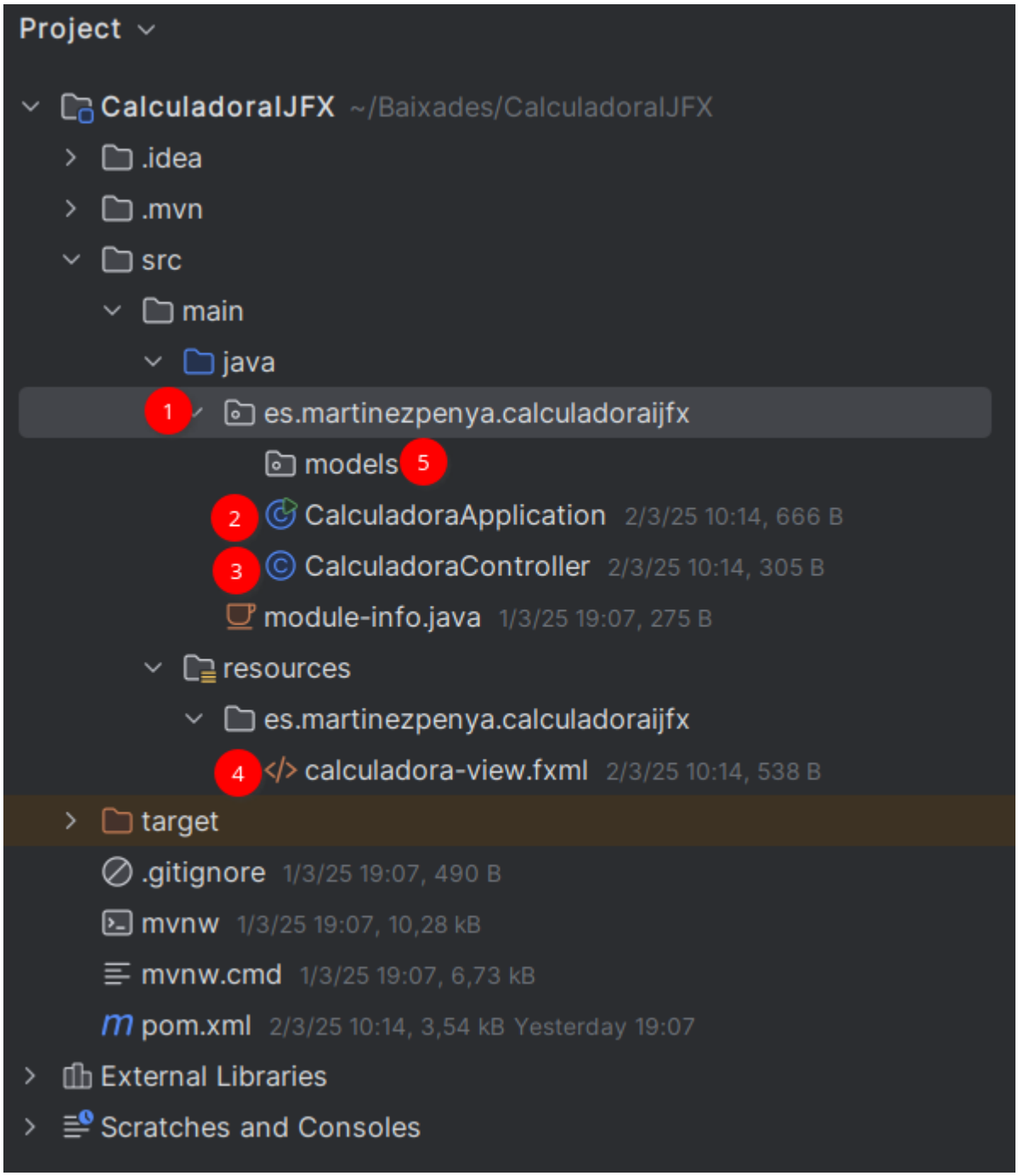
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



El asistente de IntelliJ sigue el modelo **MVC**, pero vamos a renombrar las clases que crea por defecto y crearemos un package (carpeta) donde incluiremos los modelos (el proyecto por defecto es tan simple que no tiene).

Cuando hagas todo eso tu proyecto debería tener un aspecto similar a este::

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Si lanzamos la aplicacion `CalculadoraApplication` (que es la única clase con `main`) solo deberíamos ver una ventana con un botón **Hello**, y si lo pulsamos aparece el texto **"Welcome to JavaFX Application!"**

Modelo

Para la calculadora necesitaremos un modelo que se encargue de realizar las distintas operaciones de nuestra calculadora. Para ello crearemos un nuevo fichero `Operaciones.java` dentro del paquete `models` con el siguiente contenido:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

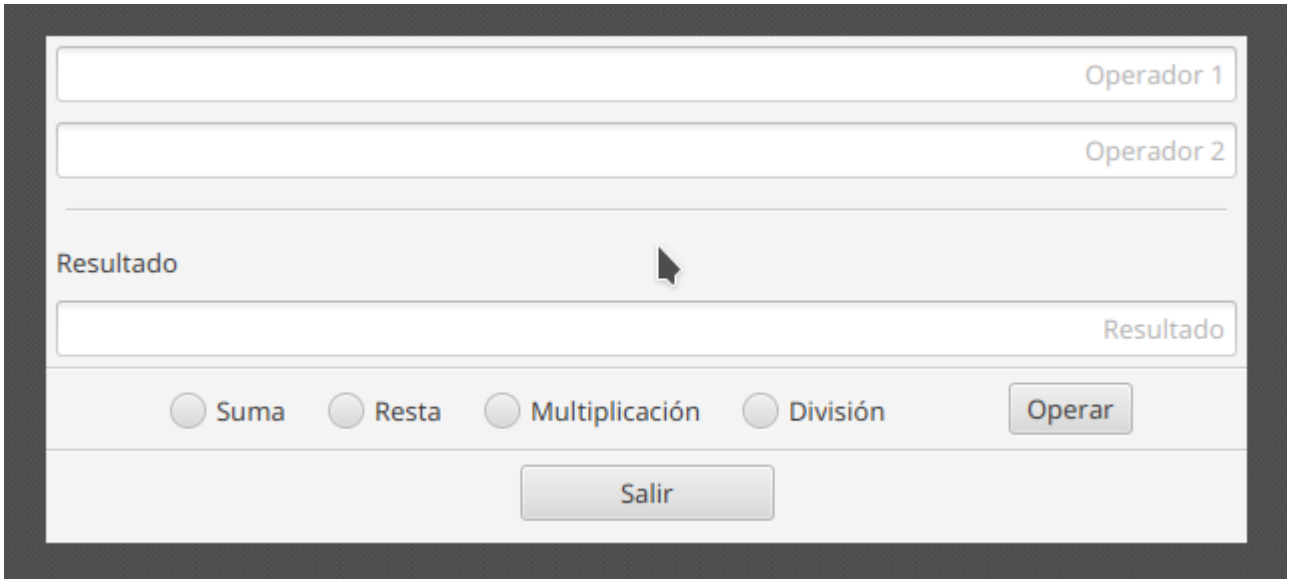
```java
package es.martinezpenya.calculadoraijfx.models;

public class Operaciones {
    private double operador1;
    private double operador2;

    public Operaciones(double operador1, double operador2) {
        this.operador1 = operador1;
        this.operador2 = operador2;
    }

    public double getOperador1() {
        return operador1;
    }

    public void setOperador1(double operador1) {
        this.operador1 = operador1;
    }

    public double getOperador2() {
        return operador2;
    }

    public void setOperador2(double operador2) {
        this.operador2 = operador2;
    }

    public double suma(){
        return this.operador1+this.operador2;
    }
    public double resta(){
        return this.operador1-this.operador2;
    }
    public double multiplicacion(){
        return this.operador1*this.operador2;
    }
    public double division(){
        return this.operador1/this.operador2;
    }
}
```

Fíjate que este es un modelo muy simple, con dos atributos, un constructor, sus *getters* y *setters* y las cuatro operaciones básicas de nuestra calculadora (`suma` , `resta` , `multiplicacion` y `division`).

Vista

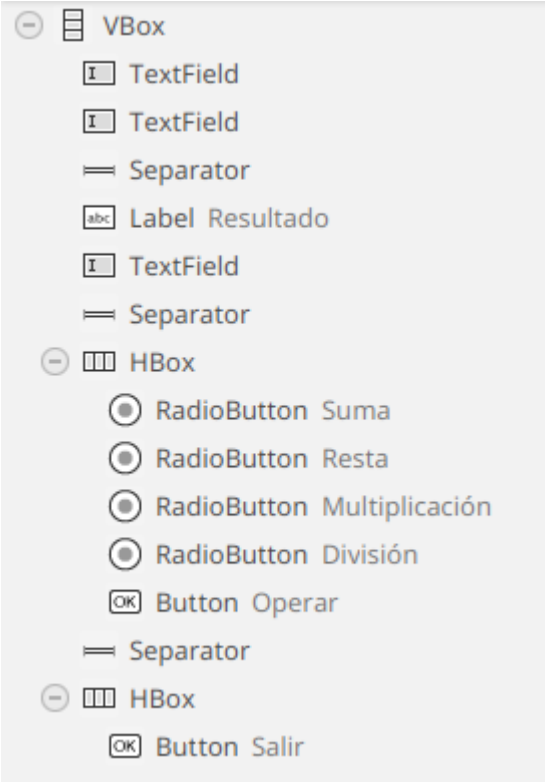
Ahora vamos a editar el fichero `calculadora-view.fxml` del paquete `resources` . Para ello, pulsamos el botón derecho sobre el nuevo fichero y elegimos la opción `Open in SceneBuilder` por defecto y el botón **Hello**.

Ahora, usando contenedores y componentes básicos deberías crear una ventana similar a esta:



Este ejemplo tiene la siguiente jerarquía:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



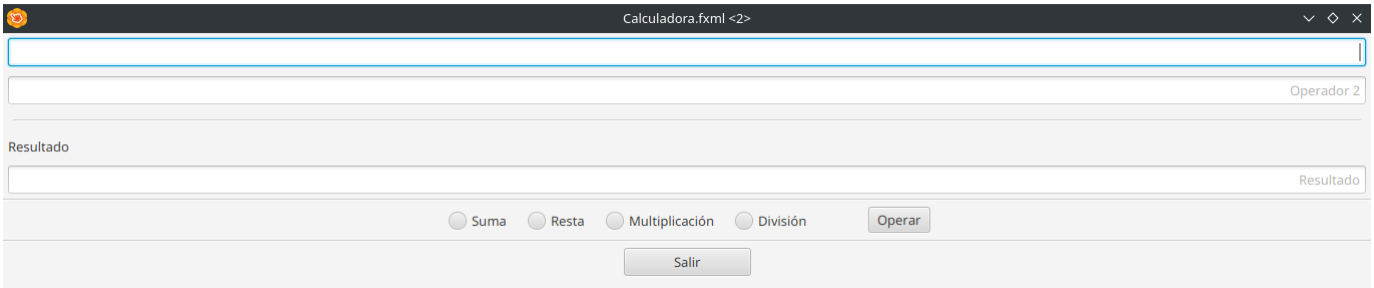
Recuerda dar nombre a todos los componentes en la pestaña `code` al campo `fx:id`.

`txtOperador1`, `txtOperador2` y `txtResultado` para los `TextField's`

`rbSuma`, `rbResta`, `rbMultiplicación`, `rbDivision` para los `RadioButton's`

Desactiva el `txtResultado`, para que no sea editable.

Crea los contenedores y ajusta sus alineaciones, así como los márgenes y espaciadores de los elementos que contienen, de manera que si amplias la ventana al máximo quede algo similar a esto:



También debes añadir la acciones `ON ACTION` dentro de la pestaña `code` para los botones:

`btnSalir` : # `salir`

`btnOperar` : # `operar`

> **Errores**
>
> Puede que tengas errores en el fichero `fxml`, de momento es normal, continua con el proceso.

Controladores

`CalculadoraController.java`

Actualizar el controlador para la vista es muy sencillo y automático (si dispones del plugin `FXMLManager`).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Añade el texto `fx:controller="es.martinezpenya.calculadoraijfx.CalculadoraController"` al final de la linea del `VBOX` del archivo `calculadora-view.fxml`:

```
<VBox maxHeight="-Infinity" maxWidth="-Infinity" minHeight="-Infinity" minWidth="-Infinity" prefHeight="253.0" prefWidth="600.0" xmlns="http://
javafx.com/javafx/21" xmlns:fx="http://javafx.com/fxml/1" fx:controller="es.martinezpenya.calculadoraijfx.CalculadoraController">
```

Una vez configurado todo esto debes hacer clic derecho sobre el archivo `calculadora-view.fxml` y elegir la opción "`Update Controller from FXML`" (Esto deberás hacerlo cada vez que realices un cambio en el fichero `FXML`).

> **Atención**
>
> Es posible que al generar el código para el controlador, no se agreguen automáticamente los imports, debes hacerlo tu a partir de la ayuda que te proporciona el IDE, pero recuerda que todos los imports deben estar relacionados con `JavaFx` y no con `AWT` (Antigüa tecnología de Java para interfaces gráficas)

Ahora, dentro del `CalculadoraController.java` debemos primero eliminar la acción que traía el código de ejemplo:

```
@FXML
protected void onHelloButtonClick() {
    welcomeText.setText("Welcome to JavaFX Application!");
}
```

Y agregaremos el código necesario para gestionar las acciones de los botones, y además asegurarnos que los *radio buttons* son auto-excluyentes:

Acción `salir`:

```
    @FXML
    public void salir(ActionEvent actionEvent) {
        Stage stage = (Stage) btnSalir.getScene().getWindow();
        stage.close();
    }
```

Acción `operar`:

```
     @FXML
     public void operar(ActionEvent actionEvent) {
         try {
             double op1 = Double.parseDouble(this.txtOperador1.getText());
             double op2 = Double.parseDouble(this.txtOperador2.getText());
             Operaciones op = new Operaciones(op1, op2);
             if (this.rbSuma.isSelected()) {
                 this.txtResultado.setText(String.valueOf(op.suma()));
             } else if (this.rbResta.isSelected()) {
                 this.txtResultado.setText(String.valueOf(op.resta()));
             } else if (this.rbMultiplicacion.isSelected()) {
                 this.txtResultado.setText(String.valueOf(op.multiplicacion()));
             } else if (this.rbDivision.isSelected()) {
                 if (op2 != 0) {
                     this.txtResultado.setText(String.valueOf(op.division()));
                 } else {
                     Alert alert = new Alert(Alert.AlertType.ERROR);
                     alert.setHeaderText(null);
                     alert.setTitle("Error");
                     alert.setContentText("El operador 2 no puede ser 0.");
                     alert.showAndWait();
                 }
             }
         } catch (NumberFormatException numberFormatException) {
             Alert alert = new Alert(Alert.AlertType.ERROR);
             alert.setHeaderText(null);
             alert.setTitle("Error");
             alert.setContentText("Formato incorrecto de algun operando");
             alert.showAndWait();
         }
     }
```

Recuerda realizar el *import* del `models.Operaciones` y de todos los componentes de `javaFx` necesarios:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
import es.martinezpenya.calculadoraijfx.models.Operaciones;
import javafx.event.ActionEvent;
import javafx.fxml.FXML;
import javafx.scene.control.Alert;
import javafx.scene.control.Button;
import javafx.scene.control.RadioButton;
import javafx.scene.control.TextField;
import javafx.stage.Stage;
```

Acción `initialize`:

```
@FXML
public void initialize() {
    ToggleGroup tgRadio = new ToggleGroup();
    rbSuma.setToggleGroup(tgRadio);
    rbMultiplicacion.setToggleGroup(tgRadio);
    rbResta.setToggleGroup(tgRadio);
    rbDivision.setToggleGroup(tgRadio);
}
```

El método `initialize` será llamado al instanciar el controlador y generará un `ToggleGroup` de manera que solo podamos seleccionar una de las cuatro opciones disponibles.

`main.java`

Por último solo nos queda modificar la clase `CalculadoraApplication`, que contendrá el método `main` que lanzará la aplicación JavaFX. Una vez modificado (sobre todo el título y tamaño de la ventana), debería quedar algo parecido a esto:

```
package es.martinezpenya.calculadoraijfx;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.stage.Stage;

import java.io.IOException;

public class CalculadoraApplication extends Application {
    @Override
    public void start(Stage stage) throws IOException {
        FXMLLoader fxmlLoader = new FXMLLoader(CalculadoraApplication.class.getResource("calculadora-view.fxml"));
        Scene scene = new Scene(fxmlLoader.load(), 600, 253);
        stage.setTitle("Calculadora de David");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}
```

Fíjate como se carga el recurso desde una "ruta" relativa a partir del nombre de la clase. Te dejo aquí debajo un cuadro resumen con las diferentes maneras de cargar recursos en Java.

Different ways to load classpath resources in Java

A comparison of different ways of resources loading in Java

Followings are the preferred ways to load resources in classpath.

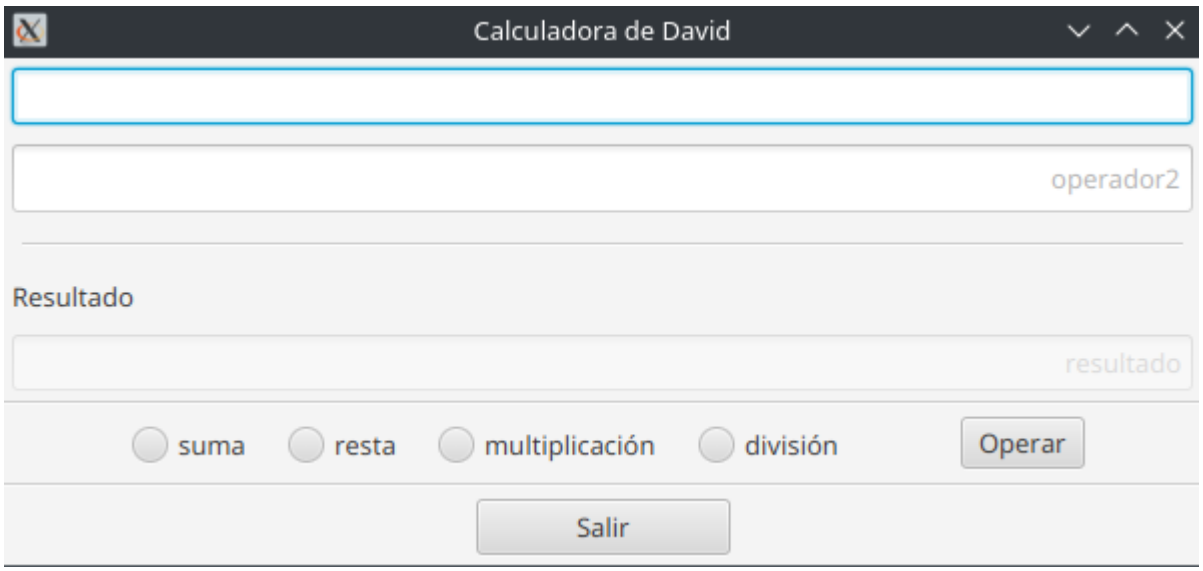
- `this.getClass().getResource(resourceName)`: It tries to find the resource in the same package as 'this' class unless we use absolute path starting with '/'
- `Thread.currentThread().getContextClassLoader().getResource(resourceName)`: A ClassLoader can be passed (shared) when creating a new thread using `Thread.setContextClassLoader`, so that different thread contexts can load each other classes/resources. If not set, the default is the ClassLoader context of the parent Thread. This method is not appropriate if we want to load resources inside the packages unless we use complete paths starting from root.
- `ClassLoader.getSystemClassLoader().getResource(resourceName)`: `ClassLoader.getSystemClassLoader()` gives the class loader used to start the application. we have to use complete path for the resources starting from root.

If we don't create any threads in the entire application, the main thread will end up with the system class loader as their context class loader.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Primer lanzamiento

Si todo ha ido bien debería aparecer nuestra calculadora en pantalla:

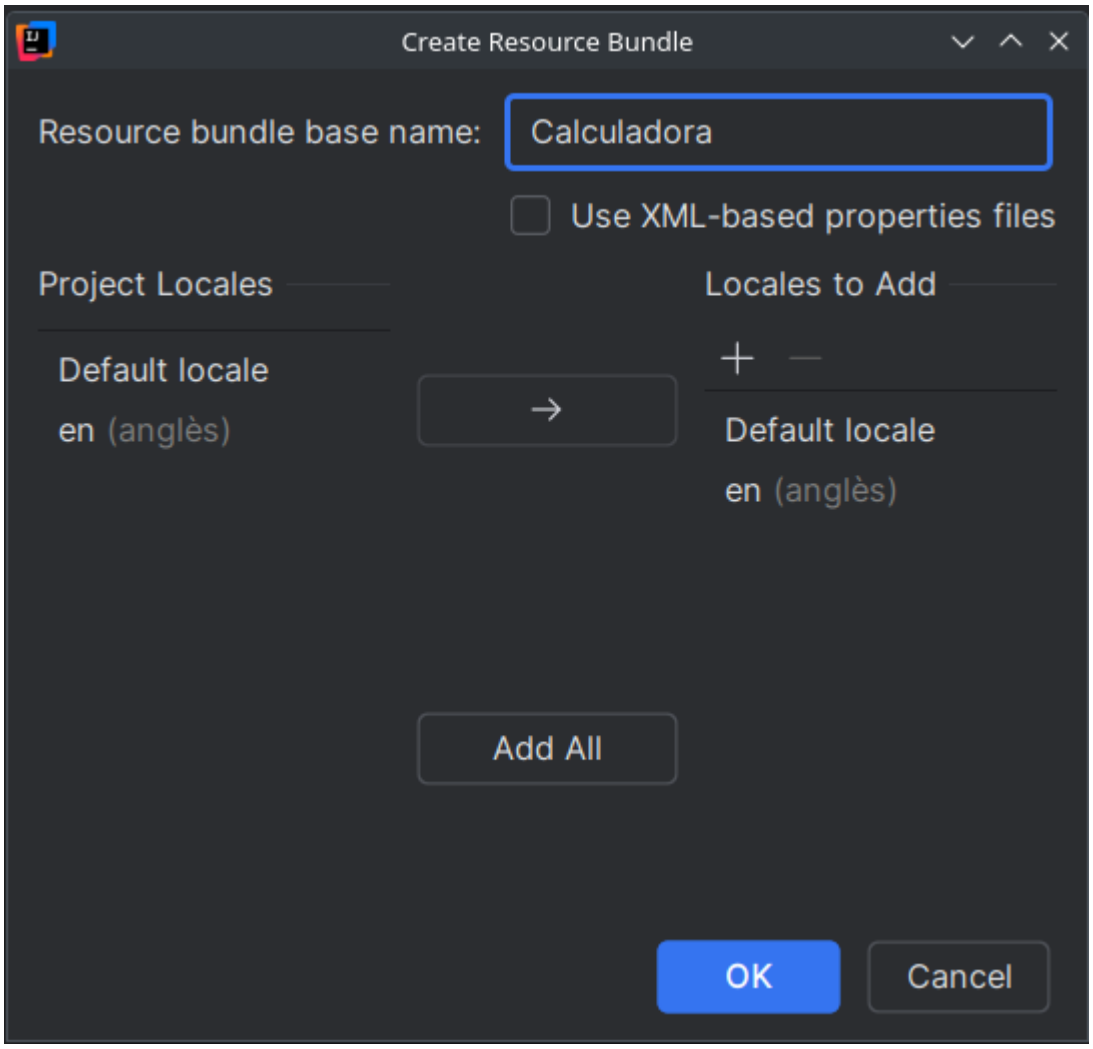


Internacionalización `I18N` del formulario (ampliación voluntaria)

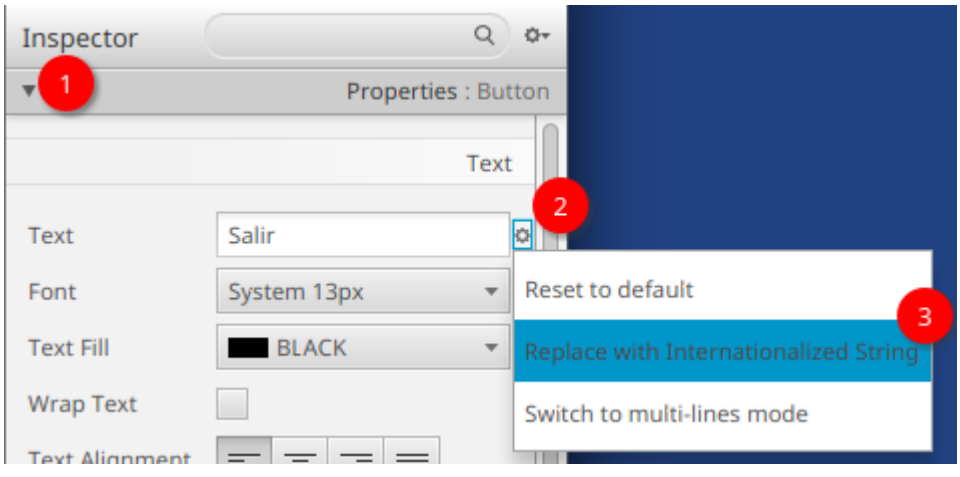
En las aplicaciones de interfaz gráfica es muy recomendable (y relativamente fácil) añadir la funcionalidad multi-idioma. Para ilustrar como se hace ayudados de `SceneBuilder` y `JavaFX`, haremos que nuestra calculadora funcione en Español e Inglés.

Dentro del paquete de recursos (resources) vamos a añadir un `Resource Bundle`, este tipo de recurso siempre acaba con la cadena "`.properties`" para que IntelliJ lo reconozca como un recurso de propiedades, así que nosotros pondremos solo `Calculadora`, y el IDE añadirá el "`.properties`". Consideraremos que el lenguaje por defecto será el Español, así que solo debemos añadir un lenguaje más que será el "Ingles" (en). Por último podemos elegir guardar las propiedades en un fichero xml o un fichero de texto plano (desmarcando la opción):

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

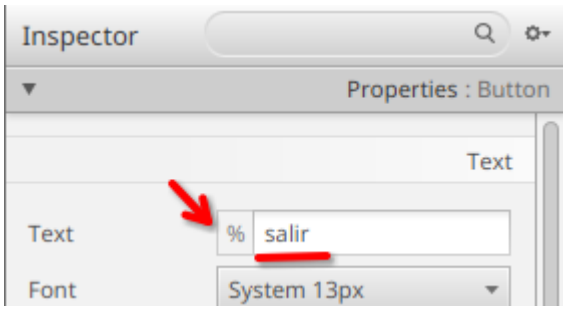


Lo siguiente será convertir todas las cadenas de texto de la interfaz gráfica en "Cadenas internacionalizadas". Abre el formulario `fxml` con `SceneBuilder`, elige por ejemplo el botón `Salir`, y en la propiedad `Text` (dentro de `Properties`) elige la rueda dentada que aparece a la derecha y del menú que aparece elige la opción "Replace with Internationalized String":

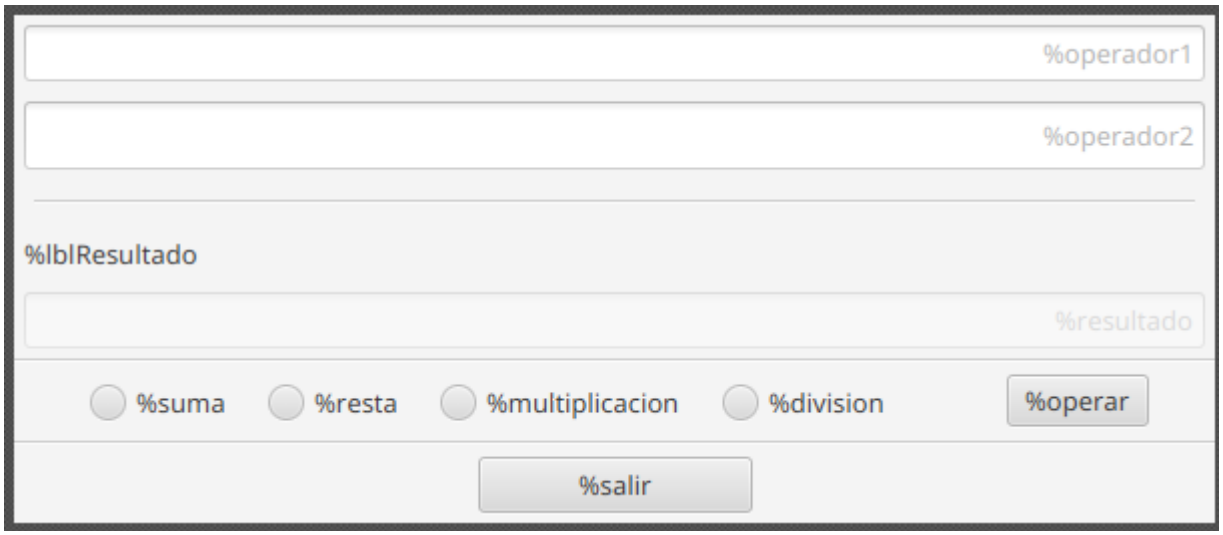


Ahora el campo `Text` deberías dejarlo así:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

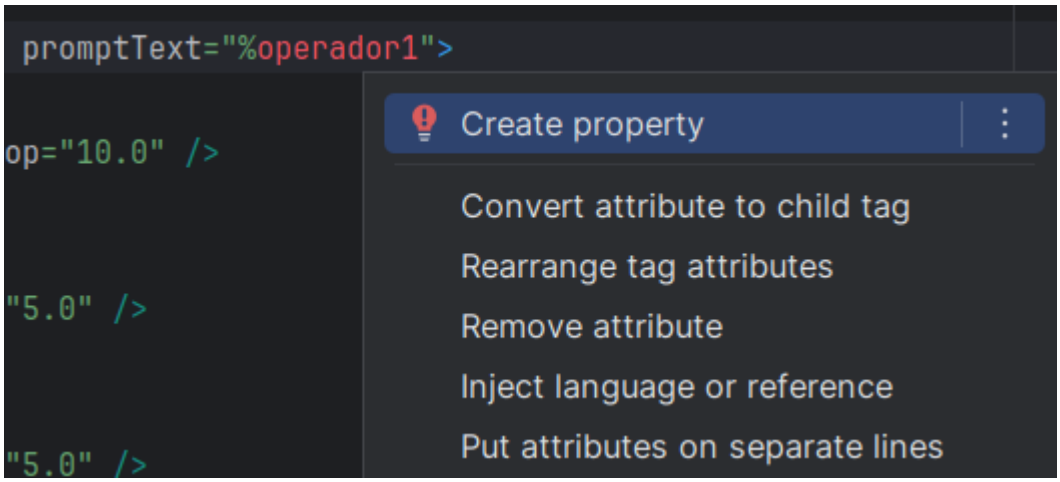


Repite este proceso con todas las partes de la interfaz gráfica que deban estar en dos idiomas.



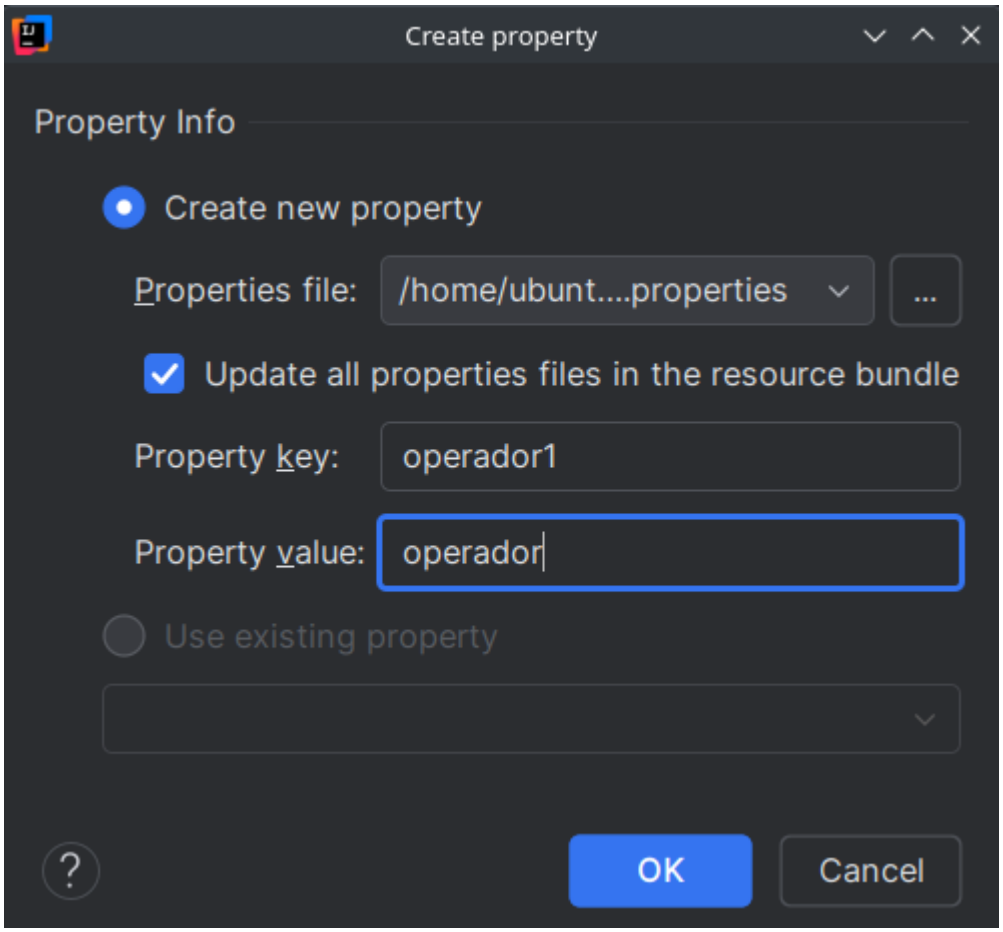
Ahora veras que el código fuente del fichero `fxml` contiene errores en cada una de las lineas que hacen referencia a estas nuevas cadenas de texto internacionales que hemos definido, eso de momento está bien.

Ahora, usando el asistente de IntelliJ, le pedimos que nos ayude a corregir los errores:



Y a continuación (si no te aparece directamente el fichero de propiedades es que no has hecho bien el primer paso):

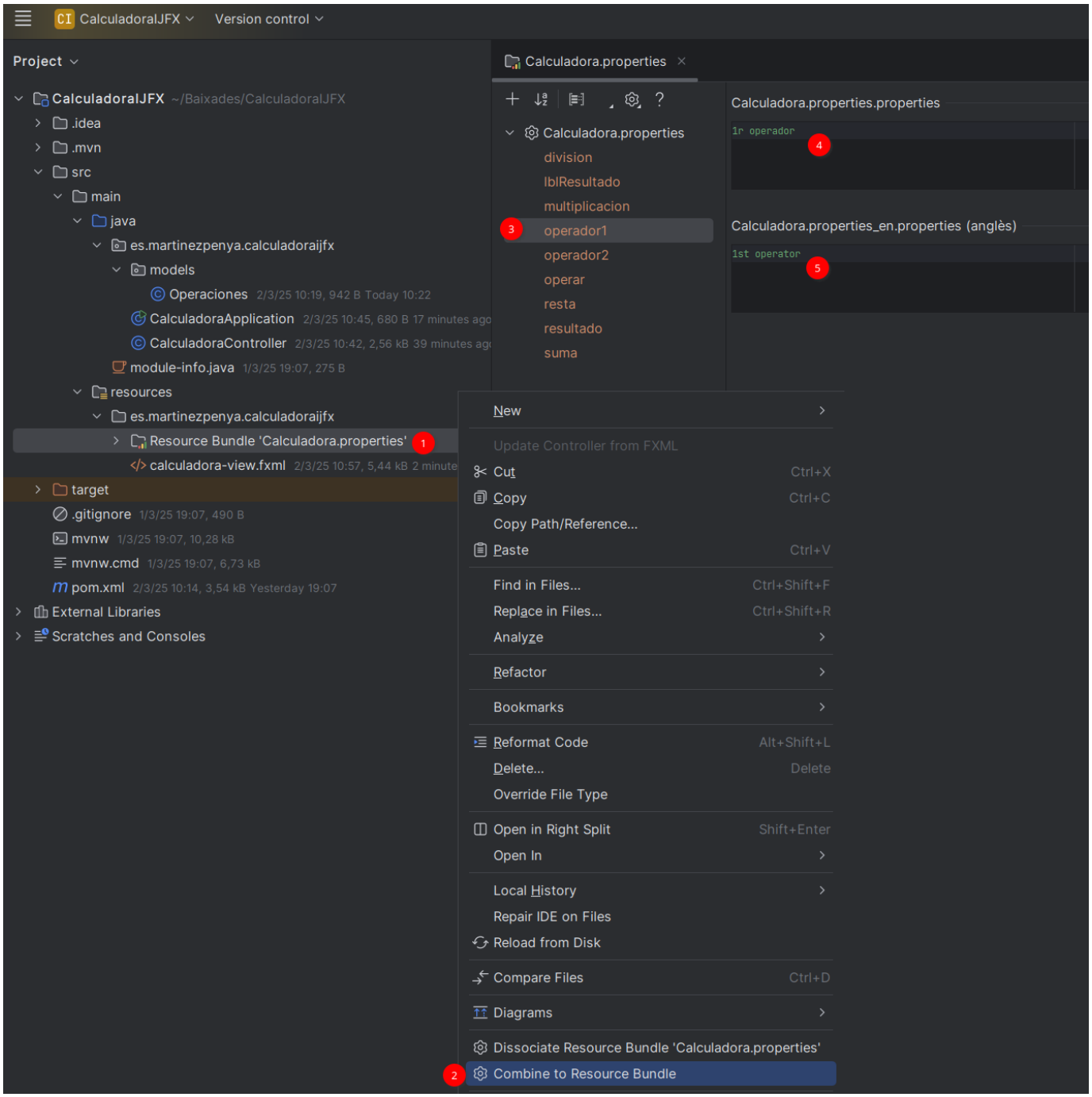
© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Repite el paso anterior para todas las propiedades del FXML que dan error, genera todas las etiquetas con el idioma por defecto (Español).

Ahora solo queda añadir las diferentes traducciones al Inglés, para ello combinaremos todas las propiedades en un único editor:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Para ello:

1. Pulsa botón derecho sobre el recurso
2. Elige la opción "Combine to Resource Bundle"
3. Elegimos la "etiqueta" que queremos traducir
4. Escribimos/modificamos el idioma por defecto (Español)
5. Escribimos/modificamos el idioma en Ingles.

Ahora debemos hacer que el main de la clase `CalculadoraApplication.java` cargue estos recursos al cargar el formulario. Añadiremos la linea del recurso (ResourceBundle) y añadiremos el mismo (bundle) al cargar el formulario:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
public void start(Stage stage) throws IOException {
    ResourceBundle bundle = ResourceBundle.getBundle(getClass().getPackage().getName()+"." + "Calculadora");
    FXMLLoader fxmlLoader = new FXMLLoader(CalculadoraApplication.class.getResource("calculadora-view.fxml"), bundle);
    Scene scene = new Scene(fxmlLoader.load(), 600, 253);
    stage.setTitle("Calculadora de David");
    stage.setScene(scene);
    stage.show();
}
```

Ahora, al lanzar la aplicación, aparecerá en el idioma español por defecto, a no ser que nuestro sistema esté configurado en Inglés. Podemos forzar que se ejecute por ejemplo siempre en inglés añadiendo las siguientes `VM Options` al entorno de ejecución del proyecto:

```
-Duser.country=UK -Duser.language=en
```

Internacionalización de los mensajes de error (ampliación voluntaria)

En el punto anterior hemos traducido el formulario a dos idiomas, pero los mensajes de error siguen saliendo solo en Español. Para corregirlo necesitamos añadir más propiedades al `ResourceBundle` "**Calculadora**" (al menos una para el título de las ventanas de Error, y otras dos con los mensajes que necesita el controlador de la Calculadora, "formato incorrecto de los operadores" y "el operador 2 no puede ser 0"). Una vez creadas las propiedades, solo debemos añadir al controlador la linea para que cargue los recursos:

```
ResourceBundle bundle = ResourceBundle.getBundle(getClass().getPackage().getName()+"." + "Calculadora");
```

Y luego cambiar el método `operar` para que haga uso de ellas en lugar de los Strings:

```
@FXML
public void operar(ActionEvent actionEvent) {
    try {
        double op1 = Double.parseDouble(this.txtOperador1.getText());
        double op2 = Double.parseDouble(this.txtOperador2.getText());
        Operaciones op = new Operaciones(op1, op2);
        if (this.rbSuma.isSelected()) {
            this.txtResultado.setText(String.valueOf(op.suma()));
        } else if (this.rbResta.isSelected()) {
            this.txtResultado.setText(String.valueOf(op.resta()));
        } else if (this.rbMultiplicacion.isSelected()) {
            this.txtResultado.setText(String.valueOf(op.multiplicacion()));
        } else if (this.rbDivision.isSelected()) {
            if (op2 != 0) {
                this.txtResultado.setText(String.valueOf(op.division()));
            } else {
                Alert alert = new Alert(Alert.AlertType.ERROR);
                alert.setHeaderText(null);
                alert.setTitle(bundle.getString("error"));
                alert.setContentText(bundle.getString("operador2Non0"));
                alert.showAndWait();
            }
        }
    } catch (NumberFormatException numberFormatException) {
        Alert alert = new Alert(Alert.AlertType.ERROR);
        alert.setHeaderText(null);
        alert.setTitle(bundle.getString("error"));
        alert.setContentText(bundle.getString("formatError"));
        alert.showAndWait();
    }
}
```

En mi ejemplo las propiedades del `ResourceBundle` se llaman `error`, `operador2Non0` y `formatError`.

Videos relacionados

- https://www.youtube.com/playlist?list=PLNjWMbvTJAIjLRW2qyuc4DEgFVW5YFRSR
- https://www.youtube.com/playlist?list=PLaxZkGlLWHGUWZxuadN3J7KKaICRlhz5-

Tarea

La tarea consiste en seguir la guía y crear tu aplicación en IntelliJ, pon como título de aplicación tu nombre y apellidos.

Envía el proyecto en `zip` y un `pdf` explicando las partes que te han parecido más complicadas al realizar la práctica, que parte te ha parecido más interesante, cual te gustaría ampliar y alguna captura mostrándola en funcionamiento.

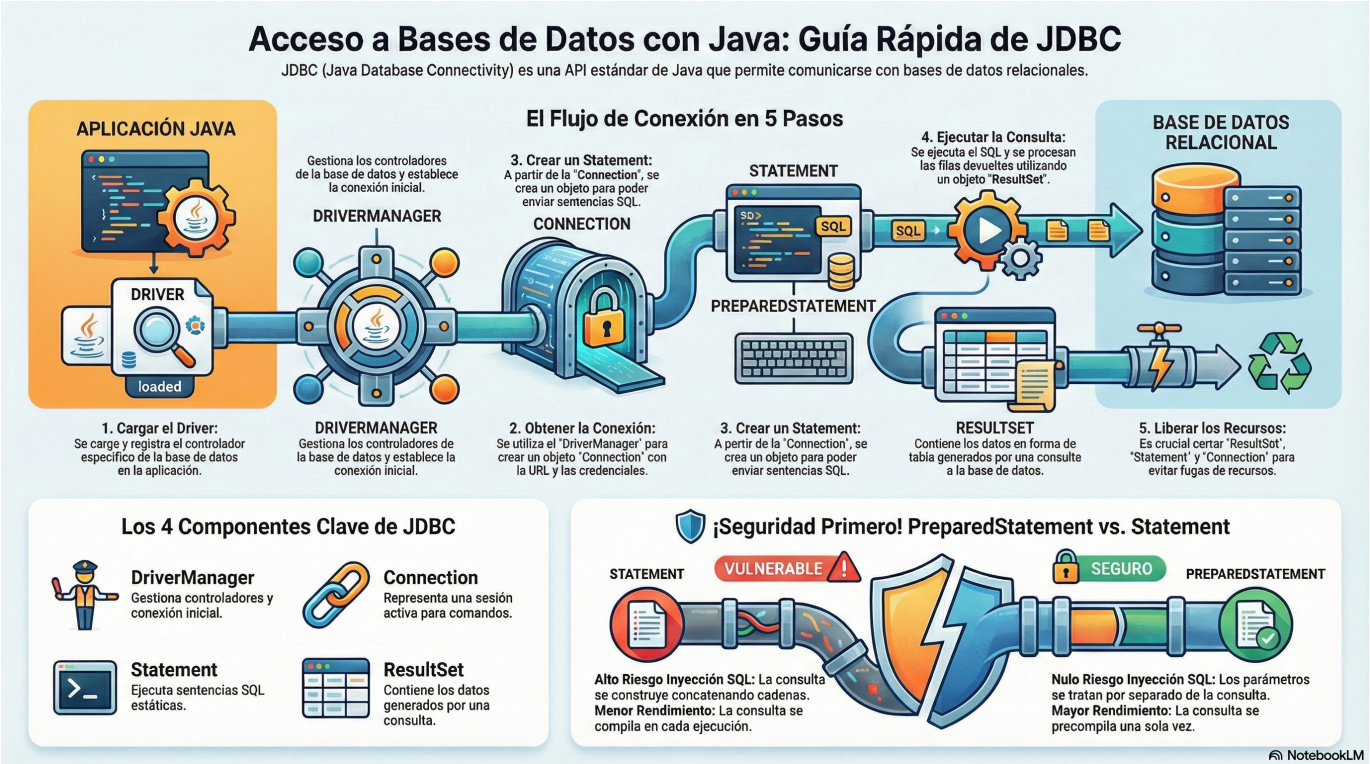
© 2025 David Martínez licensed under CC BY-NC-SA 4.0

6 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

11. UD10

11.1 Acceso a Bases de Datos Relacionales



Al finalizar esta unidad serás capaz de...

- Conocer los fundamentos de JDBC
- Establecer conexiones con bases de datos
- Ejecutar consultas SQL con Statement y PreparedStatement
- Recuperar y mostrar resultados con ResultSet
- Realizar operaciones CRUD (insert, select, update, delete)
- Utilizar el patrón DAO para separar la lógica de acceso a datos
- Conectar con bases de datos en la nube (AWS)

11.1.1 Introducción

Hoy en día, la mayoría de aplicaciones informáticas necesitan almacenar y gestionar gran cantidad de datos. Esos datos, se suelen guardar en **bases de datos relacionales**, ya que éstas son las más extendidas actualmente.

Las bases de datos relacionales permiten organizar los datos en **tablas** y esas tablas y datos se relacionan mediante campos clave. Además se trabaja con el lenguaje estándar conocido como **SQL**, para poder realizar las consultas que deseemos a la base de datos.

Una base de datos relacional se puede definir de una manera simple como aquella que presenta la información en tablas con filas y columnas.

Una tabla es una serie de **filas** y **columnas** , en la que cada fila es un **registro** y cada columna es un **campo**. Un campo representa un dato de los elementos almacenados en la tabla (*NSS*, *nombre*, *etc.*). Cada registro representa un elemento de la tabla (el equipo Real Madrid, el equipo Real Murcia, etc.)

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

No se permite que pueda aparecer dos o más veces el mismo registro, por lo que uno o más campos de la tabla forman lo que se conoce como **clave primaria** (atributo que se elige como identificador en una tabla, de manera que no haya dos registros iguales, sino que se diferencien al menos en esa clave. Por ejemplo, en el caso de una tabla que guarda datos de personas, el número de la seguridad social, podría elegirse como clave primaria, pues sabemos que aunque haya dos personas llamadas, por ejemplo, Juan Pérez Pérez, estamos seguros de que su número de seguridad social será distinto).

El sistema gestor de bases de datos, en inglés conocido como: **Database Management System** (**DBMS**) , gestiona el modo en que los datos se almacenan, mantienen y recuperan.

En el caso de una base de datos relacional, el sistema gestor de base de datos se denomina: **Relational Database Management System** (**RDBMS**).

Tradicionalmente, la programación de bases de datos ha sido como una Torre de Babel: gran cantidad de productos de bases de datos en el mercado, y cada uno “hablando” en su lenguaje privado con las aplicaciones.

Java, mediante **JDBC** (Java Database Connectivity, API que permite la ejecución de operaciones sobre bases de datos desde el lenguaje de programación Java, independientemente del sistema operativo donde se ejecute o de la base de datos a la cual se accede), permite simplificar el acceso a base de datos , proporcionando un lenguaje mediante el cual las aplicaciones pueden comunicarse con motores de bases de datos. Sun desarrolló este API para el acceso a bases de datos, con tres objetivos principales en mente:

- Ser un API con soporte de SQL: poder construir sentencias SQL e insertarlas dentro de llamadas al API de Java,
- Aprovechar la experiencia de los APIs de bases de datos existentes,
- Ser sencillo.

Qué es una API?

API: *Application Programming Interface* (Interfaz de Programación de Aplicaciones). Conjunto de reglas y protocolos que permite a diferentes aplicaciones o sistemas comunicarse entre sí. En pocas palabras, actúa como un intermediario que permite que dos programas informáticos se comuniquen y compartan datos entre ellos de manera segura y eficiente. Las APIs se utilizan comúnmente en el desarrollo de software para permitir la integración de diferentes sistemas, la creación de aplicaciones de terceros y la automatización de procesos.

Un ejemplo sencillo de API podría ser el servicio de pronóstico del tiempo proporcionado por una compañía meteorológica. Imagina que tienes una aplicación de clima en tu teléfono. Esta aplicación necesita mostrar el pronóstico del tiempo actualizado, pero no tiene la capacidad de predecir el clima por sí misma.

En lugar de eso, la aplicación utiliza una API proporcionada por una empresa meteorológica. Esta API permite que la aplicación envíe una solicitud con la ubicación actual del usuario y, a cambio, recibe datos sobre el clima en esa ubicación. La API proporciona estos datos en un formato estructurado, como JSON o XML, que la aplicación puede interpretar y mostrar de manera comprensible para el usuario.

En resumen, la aplicación de clima utiliza la API de la empresa meteorológica para obtener datos actualizados sobre el pronóstico del tiempo sin tener que desarrollar su propio sistema de predicción meteorológica. La API actúa como un puente entre la aplicación y los recursos de la compañía meteorológica, permitiendo que la aplicación acceda y utilice esos recursos de manera fácil y eficiente.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Bases de datos NoSQL vs Bases de datos relacionales

Las bases de datos NoSQL (Not Only SQL) son sistemas de gestión de bases de datos que no siguen el modelo relacional tradicional. Están diseñadas para manejar datos no estructurados o semiestructurados y para proporcionar flexibilidad en la forma en que se almacenan y acceden a los datos. Existen diferentes tipos:

- **Documentales:** Almacenan datos en documentos similares a JSON, como MongoDB.
- **Clave-Valor:** Almacenan pares de clave-valor, como Redis.
- **Columnar:** Almacenan datos en columnas en lugar de filas, como Cassandra.
- **Grafos:** Almacenan datos en estructuras de grafos, como Neo4j.

Entre sus ventajas, podemos encontrar las siguientes:

- **Escalabilidad Horizontal:** Pueden manejar grandes volúmenes de datos mediante la distribución de la carga en múltiples servidores.
- **Flexibilidad:** No requieren un esquema fijo, lo que permite manejar datos heterogéneos y evolucionar el modelo de datos fácilmente.
- **Alto Rendimiento:** Optimizadas para operaciones de lectura y escritura rápidas.

Diferencias con Bases de Datos Relacionales

Referente al Modelo de Datos, las **Relacionales** utilizan un modelo tabular con filas y columnas, y relaciones definidas mediante claves primarias y foráneas. Mientras que las **NoSQL** utilizan diversos modelos de datos (documentos, clave-valor, columnar, grafos) que permiten una mayor flexibilidad.

En cuanto a su Esquema, las **Relacionales:** tienen esquemas rígidos y predefinidos. Cambiar el esquema puede ser complejo. Por contra, las **NoSQL** tienen esquemas dinámicos o inexistentes, permitiendo agregar nuevos campos sobre la marcha sin necesidad de modificar el esquema de la base de datos.

Las Consultas en las **Relacionales** utilizan SQL (Structured Query Language) para realizar consultas complejas y uniones entre tablas. Sin embargo, las **NoSQL** utilizan diferentes lenguajes de consulta o API específicos para el tipo de base de datos, que pueden ser menos complejos y más rápidos para ciertas operaciones.

Si hablamos de Escalabilidad, las **Relacionales** tienen escalabilidad vertical (mejorar el hardware del servidor). Pero las **NoSQL** cuentan con escalabilidad horizontal (añadir más servidores al clúster).

Por último, referente a la Integridad y Transacciones, las **Relacionales** cuentan con una fuerte consistencia y soporte robusto para transacciones ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad). Por contra, las **NoSQL** en muchos casos, priorizan la disponibilidad y la partición tolerante sobre la consistencia (sistema BASE: Basic Availability, Soft state, Eventual consistency).

Conexión a las BBDD: conectores

Dejemos de momento de lado el desfase Objeto-Relacional y centrémonos ahora en el acceso a Base de Datos Relacionales desde los lenguajes de programación. Lo razonaremos en general y lo aplicaremos a Java.

Desde la década de los 80 que existen a pleno rendimiento las bases de datos relacionales. Casi todos los Sistemas Gestores de Bases de Datos (excepto los más pequeños como Access, SQLite o Base de LibreOffice) utilizan la arquitectura cliente-servidor. Esto significa que hay un ordenador central donde está instalado el Sistema Gestor de Bases de Datos Relacional que actúa como servidor, y habrá muchos clientes que se conectarán al servidor haciendo peticiones sobre la Base de Datos.

Los Sistemas Gestores de Bases de Datos inicialmente disponían de lenguajes de programación propios para poder hacer los accesos desde los clientes. Era muy consistente, pero a base de ser muy poco operativo:

- La empresa desarrolladora del SGBD debían mantener un lenguaje de programación, que resultaba necesariamente muy costoso, si no querían que quedara desfasado.
- Las empresas usuarias del SGBD, que se conectaban como clientes, se encontraban muy ligadas al servidor para tener que utilizar el lenguaje de programación para acceder al servidor, lo que no siempre se ajustaba a sus necesidades. Además, el plantearse cambiar de servidor, significaba que había que rehacer todos los programas, y por tanto una tarea de muchísima envergadura.

Para poder ser más operativos, había que desvincular los lenguajes de programación de los Sistemas Gestores de Bases de Datos utilizando unos estándares de conexión.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

11.1.2 JDBC

Java puede conectarse con distintos SGBD y en diferentes sistemas operativos. Independientemente del método en que se almacenen los datos debe existir siempre un **mediador** entre la aplicación y el sistema de base de datos y en Java esa función la realiza **JDBC**.

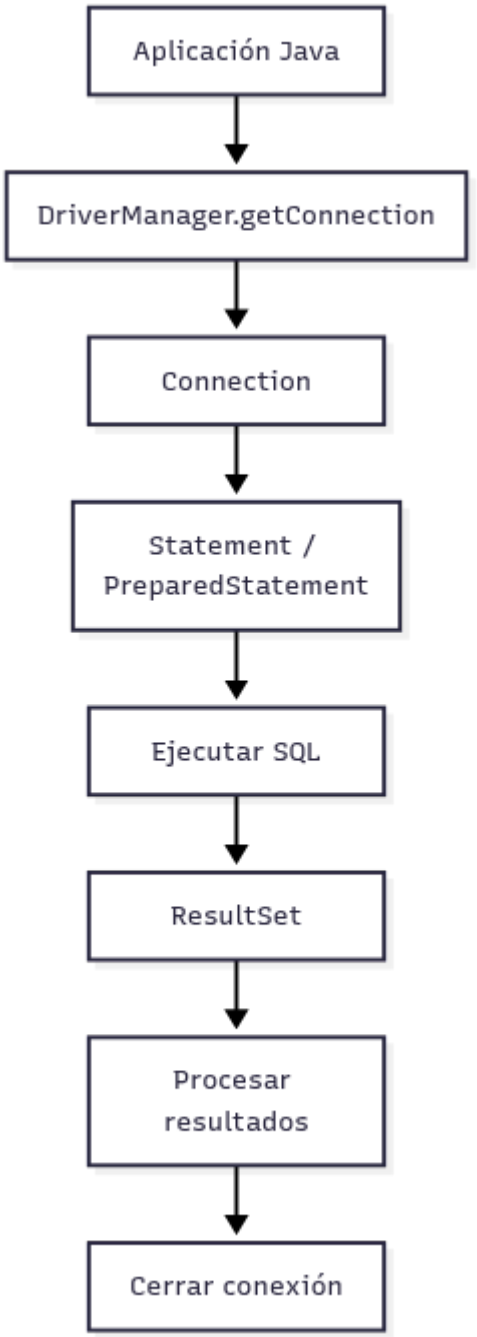
> **Importante**
>
> Para la conexión a las bases de datos utilizaremos la API estándar de JAVA denominada **JDBC** (*Java Data Base Connectivity*).

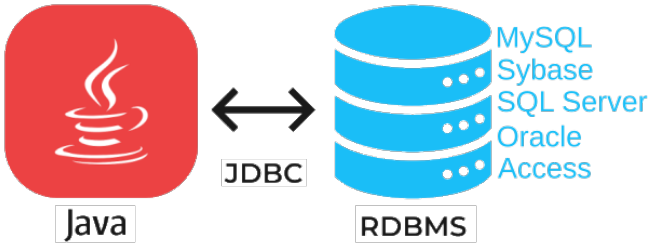
JDBC es un API incluido dentro del lenguaje Java para el acceso a bases de datos. Consiste en un conjunto de clases e interfaces escritas en Java que ofrecen un completo API para la programación con bases de datos, por lo tanto es la única solución 100% Java que permite el acceso a bases de datos.

JDBC es una especificación formada por una colección de interfaces y clases abstractas, que todos los fabricantes de drivers deben implementar si quieren realizar una implementación de su driver 100% Java y compatible con JDBC (JDBC-compliant driver). Debido a que JDBC está escrito completamente en Java también posee la ventaja de ser independiente de la plataforma.

> **Importante**
>
> No será necesario escribir un programa para cada tipo de base de datos, una misma aplicación escrita utilizando JDBC podrá manejar bases de datos Oracle, Sybase, SQL Server, etc.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

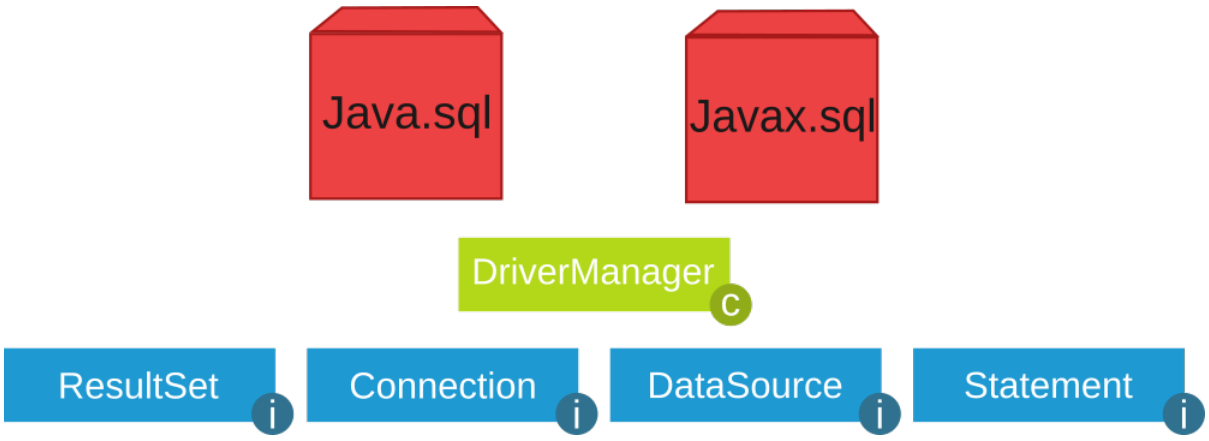




Además podrá ejecutarse en cualquier sistema operativo que posea una Máquina Virtual de Java, es decir, serán aplicaciones completamente independientes de la plataforma. Otras API'S que se suelen utilizar bastante para el acceso a bases de datos son DAO (Data Access Objects) y RDO (Remote Data Objects), y ADO (ActiveX Data Objects), pero el problema que ofrecen estas soluciones es que sólo son para plataformas Windows.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

JDBC tiene sus clases en el paquete `java.sql` y otras extensiones en el paquete `javax.sql`.



Funciones del JDBC

Básicamente el API JDBC hace posible la realización de las siguientes tareas:

- Establecer una conexión con una base de datos.
- Enviar sentencias SQL.
- Manipular datos.
- Procesar los resultados de la ejecución de las sentencias.

Drivers JDBC

Los drivers nos permiten conectarnos con una base de datos determinada. Existen **cuatro tipos de drivers JDBC**, cada tipo presenta una filosofía de trabajo diferente. A continuación se pasa a comentar cada uno de los drivers:

- **JDBC-ODBC bridge plus ODBC driver** (tipo 1): permite al programador acceder a fuentes de datos ODBC existentes mediante JDBC. El JDBC-ODBC Bridge (puente JDBC-ODBC) implementa operaciones JDBC traduciéndolas a operaciones ODBC, se encuentra dentro del paquete `sun.jdbc.odbc` y contiene librerías nativas para acceder a ODBC.

  Al ser usuario de ODBC depende de las dll de ODBC y eso limita la cantidad de plataformas en donde se puede ejecutar la aplicación.
- **Native-API partly-Java driver** (tipo 2): son similares a los drivers de tipo1, en tanto en cuanto también necesitan una configuración en la máquina cliente. Este tipo de driver convierte llamadas JDBC a llamadas de Oracle, Sybase, Informix, DB2 u otros SGBD. Tampoco se pueden utilizar dentro de applets al poseer código nativo.
- **JDBC-Net pure Java driver** (tipo 3): Estos controladores están escritos en Java y se encargan de convertir las llamadas JDBC a un protocolo independiente de la base de datos y en la aplicación servidora utilizan las funciones nativas del sistema de gestión de base de datos mediante el uso de una biblioteca JDBC en el servidor. La ventaja de esta opción es la portabilidad.
- **JDBC de Java cliente** (tipo 4): Estos controladores están escritos en Java y se encargan de convertir las llamadas JDBC a un protocolo independiente de la base de datos y en la aplicación servidora utilizan las funciones nativas del sistema de gestión de base de datos sin necesidad de bibliotecas. La ventaja de esta opción es la portabilidad. Son como los drivers de tipo 3 pero sin la figura del intermediario y tampoco requieren ninguna configuración en la máquina cliente. Los drivers de tipo 4 se pueden utilizar para servidores Web de tamaño pequeño y medio, así como para intranets.

Instalación controlador

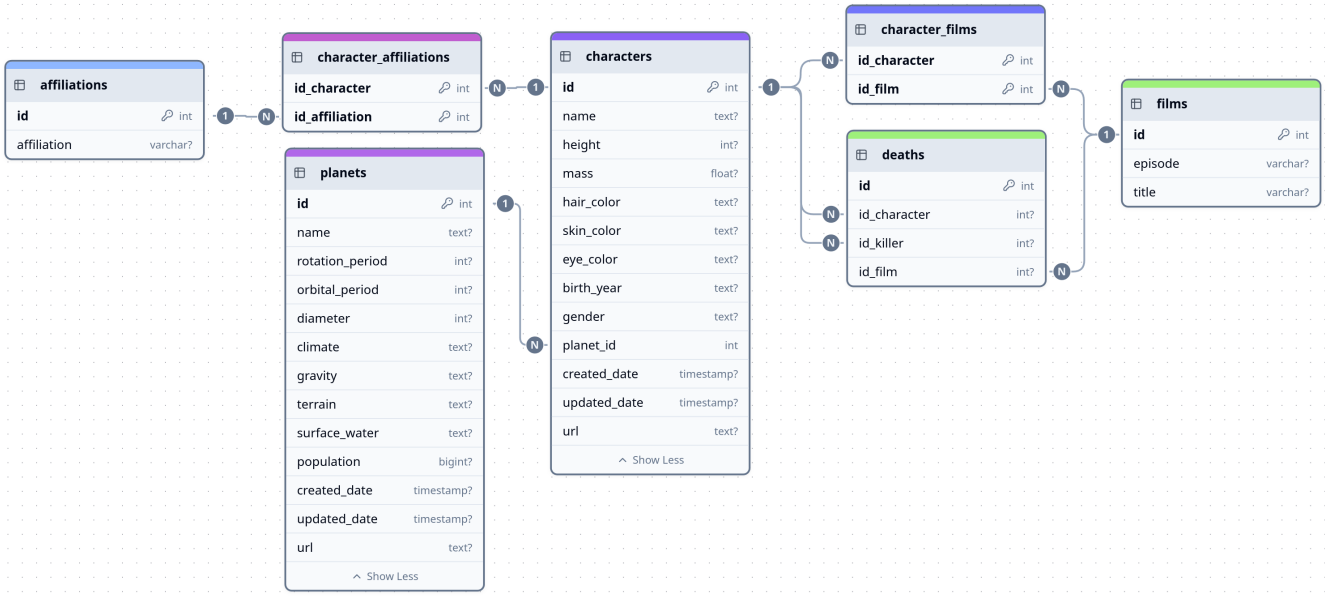
Consulta en el taller UD10_T01_Conectores_ES indicaciones sobre como instalar el controlador que necesites según tu IDE, sistema operativo y gestor de BBDD.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Base de datos de pruebas `starwars`

Todo el código fuente que veras a continuación se basa en el taller UD10_T02_AWS_IntelliJ_ES, que crea la BBDD "*StarWars*" en un servidor `MariaDB` alojado en la nube de `Amazon`. Para replicar las pruebas necesitaras la dirección de tu instáncia de BBDD (algo parecido a *databasedmp.cipxbdkxiaqy.us-east-1.rds.amazonaws.com*), el puerto (por defecto *3306*), el usuario (*admin*) y la contraseña (*123456Ab$*). Asegurate de haber completado primero el taller para poder probar el código.

Aquí te dejo el diagrama de la BD. Deberías familiarizarte con él antes de seguir.



Carga del controlador JDBC y conexión con la BD

El primer paso para conectarnos a una base de datos mediante JDBC es cargar el controlador apropiado. Estos controladores se distribuyen en un archivo `.jar` que provee el fabricante del SGBD y deben estar accesibles por la aplicación.

Para cargar el controlador (MySQL) se usan las siguientes sentencias:

Consulta el código completo de UD10_01_ConectarMySQL.

Observamos las siguientes cuestiones:

- Como ya hemos comentado alguna vez, la sentencia `Class.forName()` no sería necesaria en muchas aplicaciones. Pero nos asegura que hemos cargado el driver, y por tanto el `DriverManager` la sabrá manejar
- El `DriverManager` es capaz de encontrar el driver adecuado a través de la url proporcionada (sobre todo si el driver está cargado en memoria), y es quien nos proporciona el objeto `Connection` por medio del método `getConnection()`. Hay otra manera de obtener el `Connection` por medio del objeto `Driver`, como veremos más adelante, pero también será pasando indirectamente por `DriverManager`.
- Si no se encuentra la clase del driver (por no tenerlo en las librerías del proyecto, o haber escrito mal su nombre) se producirá la excepción `ClassNotFoundException`. Es conveniente tratarla con `try ... catch`.
- Si no se puede establecer la conexión por alguna razón se producirá la excepción `SQLException`. Al igual que en el caso anterior, es conveniente tratarla con `try ... catch`.
- El objeto `Connection` mantendrá una conexión con la Base de Datos desde el momento de la creación hasta el momento de cerrarla con `close()`. Es muy importante cerrar la conexión, no sólo para liberar la memoria de nuestro ordenador (que al cerrar la aplicación liberaría), sino sobre todo para cerrar la sesión abierta en el Servidor de Bases de Datos.

CONEXIÓN ALTERNATIVA MEDIANTE `Driver`

Una manera de conectar alternativa a las anteriores es utilizando el objeto `Driver`. La clase `java.sql.Driver` pertenece a la **API JDBC**, pero no es instanciable, y tan sólo es una interfaz, para que las clases `Driver` de los contenedores hereden de ella e implementen la manera exacta de acceder al SGBD correspondiente. Como no es instanciable (no podemos hacer `new Driver()`) la manera de crearlo es a través del método `getDriver()` del `DriverManager`, que seleccionará el driver adecuado a partir de la url. Ya sólo quedarán definir algunas propiedades, como el usuario y la contraseña, y obtener el `Connection` por medio del método `connect()`

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

La manera de conectar a través de un objeto `Driver` es más larga, pero más completa ya que se podrían especificar más cosas. Y quizás ayude a entender el montaje de los controladores de los diferentes SGBD en Java.

Consulta el código completo de UD10_02_ConectarMySQLDriver.

11.1.3 Patrones de diseño aplicables

Patrón `Singleton`

El patrón `Singleton` es un patrón de diseño que asegura que una clase tenga **una única instancia** durante toda la ejecución del programa, y proporciona un **punto de acceso global** a esa instancia. Esto es especialmente útil cuando queremos controlar el acceso a recursos compartidos, como una conexión a una base de datos.

¿Por qué usar `Singleton` en el acceso a bases de datos?

Cuando trabajamos con bases de datos relacionales, es común que necesitemos una **única conexión** o un **pool de conexiones** para interactuar con la base de datos. Si no controlamos cómo se crean estas conexiones, podríamos terminar con múltiples instancias de conexiones abiertas, lo que puede llevar a:

1. **Consumo excesivo de recursos**: Cada conexión consume memoria y recursos del sistema.
2. **Problemas de consistencia**: Si hay múltiples conexiones, podríamos tener inconsistencias en los datos.
3. **Dificultad para gestionar**: Sería complicado cerrar y controlar todas las conexiones abiertas.

Vamos a implementar una clase `DatabaseConnection` que siga el patrón Singleton. Esta clase se encargará de gestionar la conexión a la base de datos.

Consulta el código completo de DatabaseConnection.

Explicación paso a paso

1. **Instancia única**: La variable `instance` es estática y privada, lo que significa que pertenece a la clase y no puede ser accedida directamente desde fuera. Esto asegura que solo exista una instancia de `DatabaseConnection`.
2. **Constructor privado**: Al hacer el constructor privado, evitamos que otras clases puedan crear instancias de `DatabaseConnection` usando `new DatabaseConnection()`. Esto garantiza que la única forma de obtener la instancia sea a través del método `getInstance()`.
3. **Método `getInstance()`**: Este método es estático y se encarga de crear la instancia única si no existe, o devolver la existente si ya fue creada. Esto asegura que siempre trabajemos con la misma conexión.
4. **Conexión a la base de datos**: La conexión se establece en el constructor privado, y podemos acceder a ella a través del método `getConnection()`.
5. **Cerrar la conexión**: Aunque no es estrictamente necesario, es buena práctica proporcionar un método para cerrar la conexión cuando ya no la necesitemos.

¿Cómo usamos esta clase?

Aquí tienes un ejemplo de cómo usar la clase `DatabaseConnection` en tu aplicación:

Consulta el código completo de DatabaseConnectionTest.

Patrón `Pool Object` (DataBase Pool Connection `DBPC`)

El patrón Pool de Objetos es un patrón de diseño que tiene como objetivo **reutilizar objetos** que son costosos de crear, como conexiones a bases de datos, hilos, o sockets. En lugar de crear y destruir estos objetos repetidamente, el patrón mantiene un **conjunto de objetos ya creados** (un "pool") que pueden ser prestados y devueltos cuando ya no se necesiten.

¿Por qué usar un Pool de Objetos en el acceso a bases de datos?

Cuando trabajamos con bases de datos, establecer una nueva conexión es una operación costosa en términos de tiempo y recursos. Si abrimos y cerramos conexiones constantemente, el rendimiento de la aplicación se verá afectado. Aquí es donde entra en juego el **Database Connection Pooling (DBCP)**.

Un **Database Connection Pool** es una implementación específica del patrón Pool de Objetos que se encarga de gestionar múltiples conexiones a la base de datos. En lugar de crear una nueva conexión cada vez que se necesita, el pool:

1. **Mantiene un conjunto de conexiones abiertas** y listas para ser usadas.
2. **Presta una conexión** cuando un componente de la aplicación la solicita.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

3. **Recupera la conexión** cuando el componente termina de usarla, permitiendo que sea reutilizada por otros componentes.

¿Qué es HikariCP?

HikariCP es una librería de conexión a bases de datos que implementa un **Database Connection Pool**. Es conocida por ser extremadamente rápida, ligera y eficiente en comparación con otras soluciones como Apache DBCP o C3P0. HikariCP se ha convertido en el estándar de facto para aplicaciones Java que requieren alta concurrencia y rendimiento.

Además en IntelliJ con maven es muy sencillo, ya que solo debemos agregar esta dependencia al `pom.xml`:

```
<!-- HikariCP -->
<!-- https://mvnrepository.com/artifact/com.zaxxer/HikariCP -->
<dependency>
    <groupId>com.zaxxer</groupId>
    <artifactId>HikariCP</artifactId>
    <version>6.2.1</version>
</dependency>
```

Ventajas de usar un pool de conexiones:

- **Mejora del rendimiento**: Al reutilizar conexiones existentes, se evita el costo de crear y cerrar conexiones repetidamente.
- **Control de recursos**: El pool limita el número máximo de conexiones, evitando que la base de datos se sature.
- **Escalabilidad**: Permite que múltiples hilos o componentes de la aplicación accedan a la base de datos de manera concurrente sin problemas.
- **Manejo de errores**: El pool puede manejar conexiones caídas o inválidas, reemplazándolas automáticamente.

Consulta el código completo de HikariCPSingleton.

¿Cómo usamos esta clase?

Aquí tienes un ejemplo de cómo usar la clase `HikariCPSingleton` en tu aplicación:

Consulta el código completo de HikariCPSingletonTest.

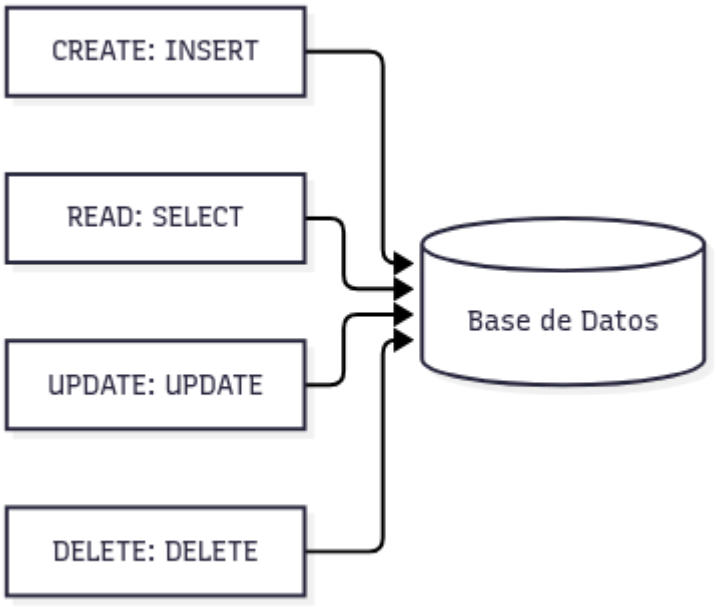
> **Recomendación**
>
> Este último Patrón `Pool Object` será el que usaremos para el resto de ejemplos de la unidad, y el que te recomiendo que uses. Si no te convence mi opinión, puedes ver el vídeo de makigas que tienes más abajo.

Patrón `DAO`

Proporciona una abstracción para las operaciones CRUD (Create, Read, Update, Delete) con la base de datos. Este patrón separa la lógica de negocio de la lógica de acceso a datos, facilitando el mantenimiento y la escalabilidad del código.

El patrón DAO se utiliza para encapsular todo el acceso a la base de datos en una clase separada. Esto permite que la lógica de negocio interactúe con la base de datos a través de métodos definidos en el DAO, sin preocuparse por los detalles de la implementación de la base de datos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Ejemplo para una clase `Film` sencilla

Consulta el código completo de Film.

```
La interfaz de `FilmDAO`:
```

Consulta el código completo de FilmDAO.

```
!!! question "Reflexiona"
    Dejo aquí una cuestión en el aire para que reflexiones, si todas las entidades van a usar una Interfaz similar... que solución propones para definir
una única Interfaz DAO que sirva para todas las entidades? (pista: Genéricos)

Y por último la implementación de la interfaz en `FilmDAOImplementado`:
```

Consulta el código completo de FilmDAOImplementado.

Entre las ventajas de usar el patrón encontramos:

- **Separación de responsabilidades**: La lógica de acceso a datos se separa de la lógica de negocio.
- **Reutilización de código**: Los métodos de acceso a datos pueden ser reutilizados por diferentes partes de la aplicación.
- **Facilidad de mantenimiento**: Cambiar la implementación de la base de datos no afecta la lógica de negocio.

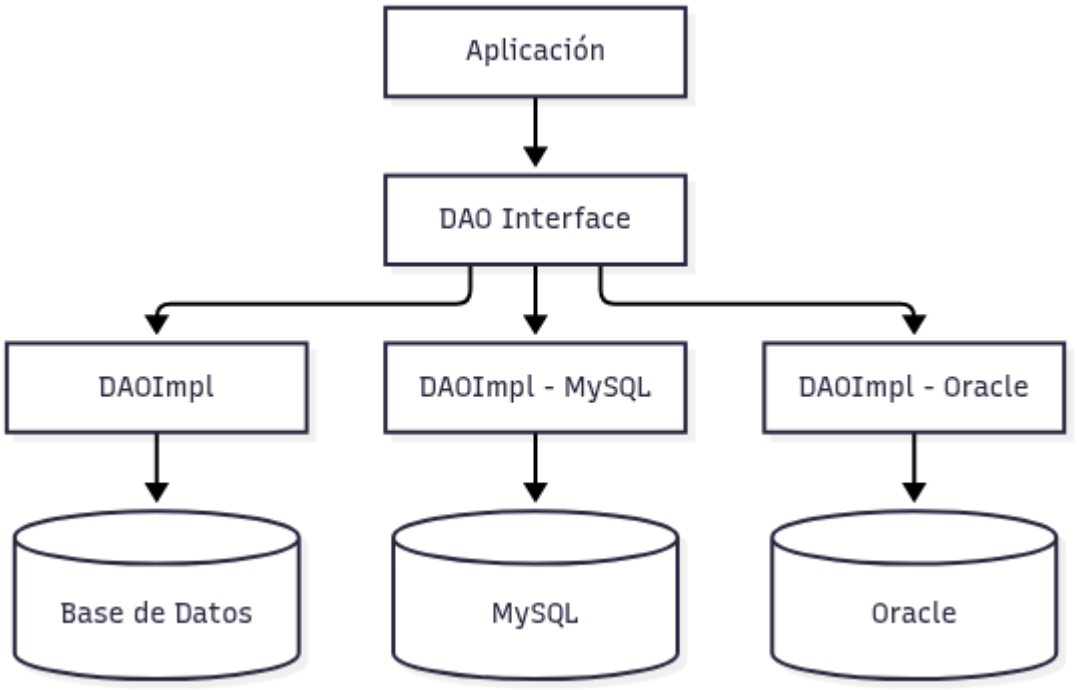
Con la siguiente clase `TestFilmDAOImplementado.java` podemos probar los métodos implementados:

Consulta el código completo de TestFilmDAOImplementado.

El resultado será:

```
Insertando el film...
Film insertado: The Force Awakens

Recuperando el film con ID 7...
Film recuperado: The Force Awakens (VII)

Modificando el film...
Film modificado: The Force Awakens

Recuperando el film modificado...
Film modificado recuperado: The Force Awakens (Episode VII)

Eliminando el film con ID 7...
Film eliminado.

Intentando recuperar el film eliminado...
El film fue eliminado correctamente.
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0



11.1.4 Acceso a BBDD

En este apartado se ofrece una introducción a los aspectos fundamentales del acceso a bases de datos mediante código Java. En los siguientes apartados se explicarán algunos aspectos en mayor detalle, sobre todo los relacionados con las clases `Statement` y `ResultSet`.

Cargar el `Driver`

En un proyecto Java que realice conexiones a bases de datos es necesario, antes que nada, utilizar `Class.forname(…)` para cargar dinámicamente el `Driver` que vamos a utilizar. Esto solo es necesario hacerlo una vez en nuestro programa. Puede lanzar excepciones por lo que es necesario utilizar un bloque `try-catch`.

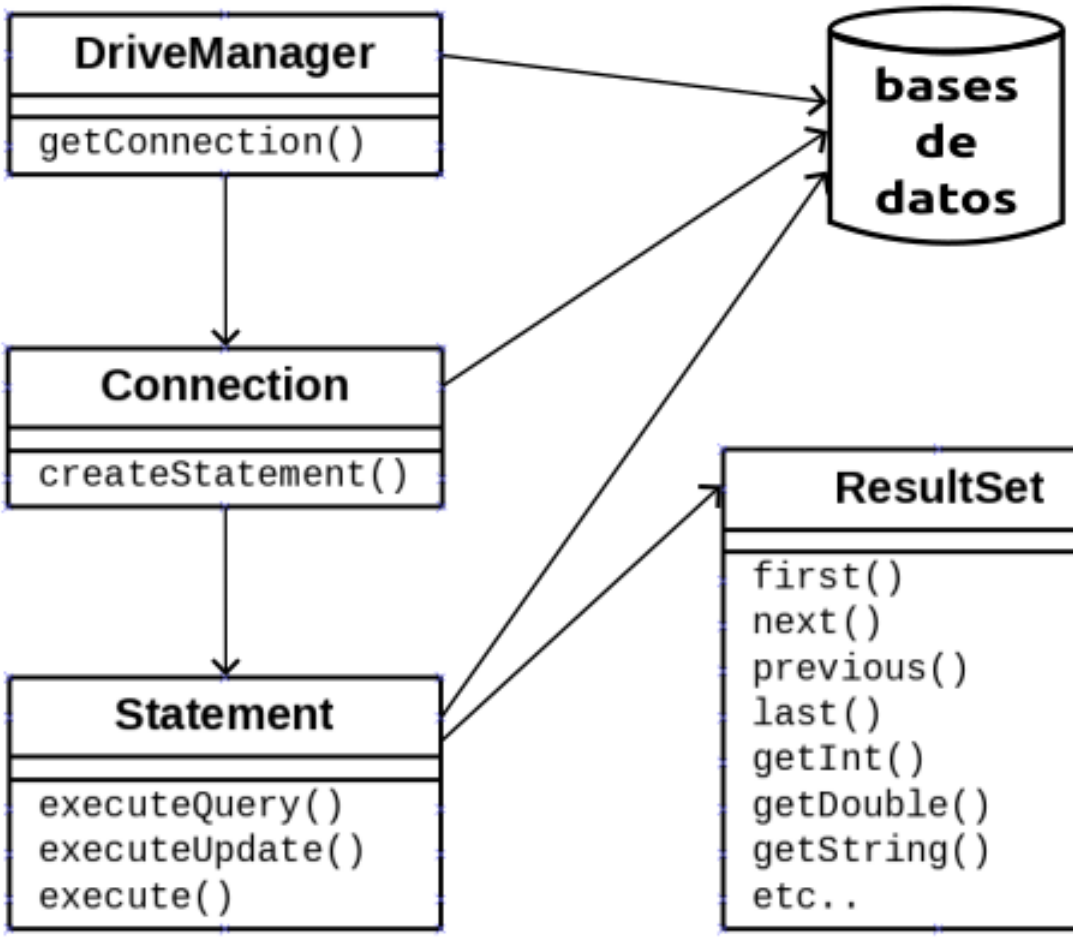
```
try {
    Class<?> c = Class.forName("com.mysql.cj.jdbc.Driver");
    System.out.println("Cargado " + c.getName());
} catch (Exception e) {
    // manejamos el error
}
```

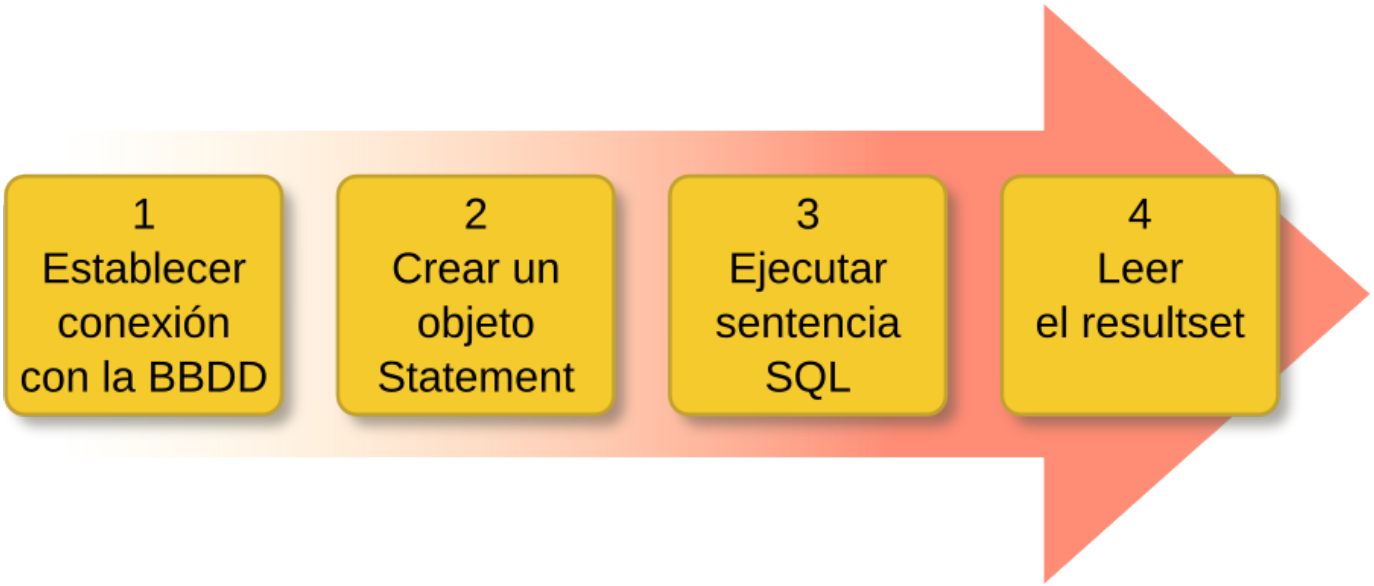
Hay que tener en cuenta que las clases y métodos utilizados para conectarse a una base de datos (explicados más adelante) funcionan con todos los drivers disponibles para Java (JDBC es solo uno, hay muchos más). Esto es posible ya que el estándar de Java solo los define como interfaces (*interface*) y cada librería driver los implementa (define las clases y su código). Por ello es necesario utilizar `Class.forName(…)` para indicarle a Java qué driver vamos a utilizar.

Este nivel de asbtracción facilita el desarrollo de proyectos ya que si necesitáramos utilizar otro sistema de base de datos (que no fuera *MySQL*) solo necesitaríamos cambiar la línea de código que carga el driver y poco más. Si cada sistema de base de datos necesitara que utilizáramos distintas clases y métodos todo sería mucho más complicado.

Las cuatro clases fundamentales que toda aplicación Java necesita para conectarse a una base de datos y ejecutar sentencias son: **`DriverManager`**, **`Connection`**, **`Statement`** y **`ResultSet`**.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0





Clase `DriverManager`

Paso 1: Establecer conexión con la BBDD

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
/* Para MySQL:
     jdbc  --> driver
     mysql --> protocolo driver
     localhost --> dirección del servidor
     3306 --> puerto
     gestionPedidos --> nombre de la BD
*/
jdbc:mysql:localhost:3306/gestionPedidos

/* Para SQL Server:
     jdbc  --> driver
     odbc --> protocolo driver
     DSN_gestionPedidos --> Nombre de la BD
*/
jdbc:odbc:DSN_gestionPedidos

/* Para Oracle:
     jdbc  --> driver
     oracle --> protocolo driver
     juan --> nombre de usuario
     @servidor --> dirección del servidor
     3306 --> puerto
     gestionPedidos --> nombre de la BD
*/
jdbc:oracle:juan@servidor:3306:gestionPedidos
```

Vamos a necesitar información adicional como son los datos de *usuario* y *contraseña*.

La clase `java.sql.DriverManager` es la capa gestora del driver JDBC. Se encarga de manejar el Driver apropiado y **permite crear conexiones con una base de datos** mediante el método estático `getConnection()` que tiene dos variantes:

```
//opcion 1
DriveManager.getConnection(String URL)

//opcion2
DriveManager.getConnection(String URL, String user, String password)
```

Este método intentará establecer una conexión con la base de datos según el URL indicado. Opcionalmente se le puede pasar el usuario y contraseña como argumento (también se puede indicar en la propia URL). Si la conexión es satisfactoria devolverá un objeto `Connection`.

Ejemplo de conexión a la base de datos *prueba* en localhost:

```
//mysql://<username>:<password>@<host>:<port>/<db_name>
String url = "jdbc:mysql://localhost:3306/prueba";
Connection conn = DriverManager.getConnection(url,"root","");
```

Este método puede lanzar dos tipos de excepciones (que habrá que manejar con un *try-catch*):

- `SQLException`: la conexión no ha podido producirse. Puede ser por multitud de motivos como una URL mal formada, un error en la red, host o puerto incorrecto, base de datos no existente, usuario y contraseña no válidos, etc.
- `SQLTimeOutException`: se ha superado el `LoginTimeout` sin recibir respuesta del servidor.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Recomendación

Aquí podemos ver un vídeo en el que Makigas explica porqué en el mundo real no se usa `DriverManager.GetConnection()`:

Clase `Connection`

Paso 2. Crear un objeto `Statement`

Un objeto `java.sql.Connection` **representa una sesión de conexión con una base de datos**. Una aplicación puede tener tantas conexiones como necesite, ya sea con una o varias bases de datos.

El método más relevante es `createStatement()` que devuelve un objeto `Statement` asociado a dicha conexión que permite ejecutar sentencias SQL. El método `createStatement()` puede lanzar excepciones de tipo `SQLException`.

```
1  Statement st = conn.createStatement();
```

Cuando ya no la necesitemos es aconsejable **cerrar la conexión con `close()`** para liberar recursos:

```
1  conn.close();
```

Clase `Statement`

Paso 3. Ejecutar sentencia SQL

Un objeto `java.sql.Statement` permite **ejecutar sentencias SQL en la base de datos** a través de la conexión con la que se creó el `Statement` (ver apartado anterior). Los tres métodos más comunes de ejecución de sentencias SQL son `executeQuery(…)`, `executeUpdate(…)` y `execute(…)`. Pueden lanzar excepciones de tipo `SQLException` y `SQLTimeoutException`.

- `ResultSet executeQuery(String sql)`: ejecuta la sentencia sql indicada (de tipo *SELECT*). Devuelve un objeto `ResultSet` con los datos proporcionados por el servidor.

```
1  ResultSet rs = st.executeQuery("SELECT * FROM vendedores");
```

- `int executeUpdate(String sql)`: ejecuta la sentencia SQL indicada (de tipo *DML* como por ejemplo *INSERT*, *UPDATE* o *DELETE*). Devuelve un el número de registros que han sido afectados (insertados, modificados o eliminados).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
int nr = st.executeUpdate ("INSERT INTO vendedores VALUES (1,'Pedro Gil', '2017-04-11', 15000);")
```

Cuando ya no lo necesitemos es aconsejable **cerrar el `Statement` con `close()`** para liberar recursos:

```
st.close();
```

> **Definición**
>
> Podríamos decir que este `ResultSet` es una especie de *tabla virtual* que se almacena en memoria con la información en su interior.

Clase `ResultSet`

Paso 4. Leer el `Resultset`

Un objeto `java.sql.ResultSet` contiene un conjunto de resultados (datos) obtenidos tras ejecutar una sentencia SQL, normalmente de tipo SELECT. Es una **estructura de datos en forma de tabla** con **registros (filas)** que podemos recorrer para acceder a la información de sus **campos (columnas)**.

`ResultSet` utiliza internamente un cursor que apunta al *registro actual* sobre el que podemos operar. Inicialmente dicho cursor está situado antes de la primera fila y disponemos de varios métodos para desplazar el cursor. El más común es `next()`:

- `boolean next()`: mueve el cursor al siguiente registro. Devuelve `true` si fue posible y `false` en caso contrario (si ya llegamos al final de la tabla).

Algunos de los métodos para obtener los datos del registro actual son:

- `String getString(String columnLabel)`: devuelve un dato `String` de la columna indicada por su nombre.

Por ejemplo:

```
rs.getString("nombre");
```

- `String getString(int columnIndex)`: devuelve un dato `String` de la columna indicada por su indice

> **Ojo!**
>
> La primera columna es la 1, no la cero 🤦🏾‍♂️

Por ejemplo:

```
rs.getString(2);
```

Existen métodos análogos a los anteriores para obtener valores de tipo `int`, `long`, `float`, `double`, `boolean`, `Date`, `Time`, `Array`, etc. Pueden consultarse todos en la documentación oficial de Java.

- `int getInt(String columnLabel)`
- `int getInt(int columnIndex)`
- `double getDouble(String columnLabel)`
- `double getDouble(int columnIndex)`
- `boolean getBoolean(String columnLabel)`
- `boolean getBoolean(int columnIndex)`
- `Date getDate(String columnLabel)`
- `Date getDate(int columnIndex)`
- etc.

Más adelante veremos cómo se realiza la modificación e inserción de datos.

Todos estos métodos pueden lanzar una `SQLException`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Veamos un ejemplo de cómo recorrer un `ResultSet` llamado `rs` y mostrarlo por pantalla:

```
while(rs.next()) {
    int id = rs.getInt("id");
    String nombre = rs.getString("nombre");
    Date fecha = rs.getDate("fecha_ingreso");
    float salario = rs.getFloat("salario");
    System.out.println(id + " " + nombre + " " + fecha + " " + salario);
}
```

11.1.5 Navegabilidad y concurrencia

Cuando invocamos a `createStatement()` **sin argumentos**, como hemos visto anteriormente, al ejecutar sentencias SQL obtendremos un `ResultSet` **por defecto en el que el cursor solo puede moverse hacia adelante y los datos son de solo lectura**. A veces esto no es suficiente y necesitamos mayor funcionalidad.

Por ello el método `createStatement()` está sobrecargado (existen varias versiones de dicho método) lo cual nos permite invocarlo con argumentos en los que podemos especificar el funcionamiento.

- **`Statement createStatement (int resultSetType, int resultSetConcurrency)`**: devuelve un objeto `Statement` cuyos objetos `ResultSet` serán del tipo y concurrencia especificados. Los valores válidos son constantes definidas en `ResultSet`.

Valores válidos para el **argumento** **`resultSetType`** indica el tipo de `ResultSet`:

- **`ResultSet.TYPE_FORWARD_ONLY`**: `ResultSet` por defecto, *forward-only* y *no-actualizable*.
- Solo permite movimiento hacia delante con `next()`.
- Sus datos NO se actualizan. Es decir, no reflejará cambios producidos en la base de datos. Contiene una instantánea del momento en el que se realizó la consulta.
- **`ResultSet.TYPE_SCROLL_INSENSITIVE`**: `ResultSet` desplazable y no actualizable.
- Permite libertad de movimiento del cursor con otros métodos como `first()`, `previous()`, `last()`, etc. además de `next()`.
- Sus datos NO se actualizan, como en el caso anterior.
- **`ResultSet.TYPE_SCROLL_SENSITIVE`**: `ResultSet` desplazable y actualizable.
- Permite libertad de movimientos del cursor, como en el caso anterior.
- Sus datos SÍ se actualizan. Es decir, mientras el `ResultSet` esté abierto se actualizará automáticamente con los cambios producidos en la base de datos. Esto puede suceder incluso mientras se está recorriendo el `ResultSet`, lo cual puede ser conveniente o contraproducente según el caso.

Diferencia entre `TYPE_SCROLL_INSENSITIVE` y `TYPE_SCROLL_SENSITIVE`

Consulta el código completo de EjemploScrollInsensitive.

```
En este ejemplo, incluso si la base de datos cambia mientras el programa está esperando (durante el `Thread.sleep(10000)`), el ResultSet no reflejará
esos cambios cuando se vuelva a consultar la primera fila.

En caso de cambiar el valor `TYPE_SCROLL_INSENSITIVE` por `TYPE_SCROLL_SENSITIVE`, si hay cambios en la base de datos durante el tiempo de espera, el
ResultSet reflejará esos cambios cuando se vuelva a consultar la primera fila. Por ejemplo, si se actualiza el nombre del primer registro en la  base de
datos mientras el programa espera, el nuevo nombre aparecerá en  la salida.

Estos ejemplos demuestran cómo `TYPE_SCROLL_INSENSITIVE` no  refleja cambios en la base de datos después de su creación, mientras que
`TYPE_SCROLL_SENSITIVE` sí lo hace.
```

El **argumento** **`ResultSet.Concurrency`** indica la concurrencia del `ResultSet`:

- **`ResultSet.CONCUR_READ_ONLY`**: solo lectura. Es el valor por defecto.
- **`ResultSet.CONCUR_UPDATABLE`**: permite modificar los datos almacenados en el `ResultSet` para luego aplicar los cambios sobre la base de datos (más adelante se verá cómo).

Importante

El `ResultSet` por defecto que se obtiene con `createStatement()` sin argumentos es el mismo que con `createStatement(ResultSet.TYPE_FORWARD_ONLY, ResultSet.CONCUR_READ_ONLY)`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

11.1.6 Consultas (`Query`)

Navegación de un `ResultSet`

Como ya se ha visto, en un objeto *ResultSet* se encuentran los resultados de la ejecución de una sentencia SQL. Por lo tanto, un objeto *ResultSet* contiene las filas que satisfacen las condiciones de una sentencia SQL, y ofrece métodos de navegación por los registros como *next()* que desplaza el cursos al siguiente registro del *ResultSet*.

Además de este método de desplazamiento básico, existen otros de desplazamiento libre que podremos utilizar siempre y cuando el *ResultSet* sea de tipo `ResultSet.TYPE_SCROLL_INSENSITIVE` o *ResultSet.TYPE_SCROLL_SENSITIVE* como se ha dicho antes.

Algunos de estos métodos son:

- **`void beforeFirst()`** : mueve el cursor antes de la primera fila.
- **`boolean first()`** : mueve el cursor a la primera fila.
- **`boolean next()`** : mueve el cursor a la siguiente fila. Permitido en todos los tipos de ResultSet.
- **`boolean previous()`** : mueve el cursor a la fila anterior.
- **`boolean last()`** : mueve el cursor a la última fila.
- **`void afterLast()`** : mover el cursor después de la última fila.
- **`boolean absolute(int row)`** : posiciona el cursor en el número de registro indicado. Hay que tener en cuenta que el primer registro es el 1, no el cero.

  > **`absolute(n)`**
  >
  > `absolute(7)` desplazará el cursor al séptimo registro. Si valor es negativo se posiciona en el número de registro indicado pero empezando a contar desde el final (el último es el -1). Por ejemplo si tiene 10 registros y llamamos `absolute(-2)` se desplazará al registro número 9.

- **`boolean relative(int registros)`** : desplaza el cursor un número relativo de registros, que puede ser positivo o negativo.

  > **`relative(n)`**
  >
  > Si el cursor está en el registro 5 y llamamos a `relative(10)` se desplazará al registro número 15. Si luego llamamos a `relative(-4)` se desplazará al registro 11.

Los métodos que devuelven un tipo `boolean` devolverán `true` si ha sido posible mover el cursor a un registro válido, y `false` en caso contrario, por ejemplo si no tiene ningún registro o hemos saltado a un número de registro que no existe.

Todos estos métodos pueden producir una excepción de tipo `SQLException`.

También existen otros métodos relacionados con la posición del cursor.

- **`int getRow()`** : devuelve el número de registro actual. Cero si no hay registro actual.
- **`boolean isBeforeFirst()`** : devuelve *true* si el cursor está antes del primer registro.
- **`boolean isFirst()`** : devuelve *true* si el cursor está en el primer registro.
- **`boolean isLast()`** : devuelve *true* si el cursor está en el último registro.
- **`boolean isAfterLast()`** : devuelve *true* si el cursor está después del último registro.

Obteniendo datos del `ResultSet`

Los métodos `getXXX()` ofrecen los medios para recuperar los valores de las columnas (campos) de la fila (registro) actual del `ResultSet`. No es necesario que las columnas sean obtenidas utilizando un orden determinado.

Para designar una columna podemos utilizar su nombre o bien su número (empezando por 1).

Por ejemplo si la segunda columna de un objeto `ResultSet` se llama `titulo` y almacena datos de tipo `String`, se podrá recuperar su valor de las dos formas siguientes:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
// rs es un objeto ResultSet
String valor = rs.getString(2);
String valor = rs.getString("titulo");
```

> **Importante**
>
> Es importante tener en cuenta que las columnas se numeran de izquierda a derecha y que la primera es la número 1, no la cero. También que las columnas no son case sensitive, es decir, no distinguen entre mayúsculas y minúsculas.

> **Ejemplo**
>
> La información referente a las columnas de un `ResultSet` se puede obtener llamando al **método** `getMetaData()` que devolverá un objeto `ResultSetMetaData` que contendrá el número, tipo y propiedades de las columnas del `ResultSet`.

Consulta el código completo de EjemploResultSetMetaData.

```
Esto provocará la siguiente salida por pantalla:

```
Número de columnas: 3
Columna 1:
Nombre: id
Tipo: INT
Tamaño display: 10
Nullable: false
Columna 2:
Nombre: episode
Tipo: VARCHAR
Tamaño display: 12
Nullable: true
Columna 3:
Nombre: title
Tipo: VARCHAR
Tamaño display: 30
Nullable: true
1 Episode I The Phantom Menace
2 Episode II Attack of the Clones
3 Episode III Revenge of the Sith
4 Episode IV A New Hope
5 Episode V The Empire Strikes Back
6 Episode VI Return of the Jedi
```
```

Si conocemos el nombre de una columna, pero no su índice, el método `findColumn()` puede ser utilizado para obtener el número de columna, pasándole como argumento un objeto `String` que sea el nombre de la columna correspondiente, este método nos devolverá un entero que será el índice correspondiente a la columna.

Tipos de datos y conversiones

Cuando se lanza un método `getXXX()` determinado sobre un objeto `ResultSet` para obtener el valor de un campo del registro actual, el driver JDBC convierte el dato que se quiere recuperar al tipo Java especificado y entonces devuelve un valor Java adecuado. Por ejemplo si utilizamos el método `getString()` y el tipo del dato en la base de datos es *`VARCHAR`*, el driver JDBC convertirá el dato `VARCHAR` de tipo SQL a un objeto `String` de Java.

Algo parecido sucede con otros tipos de datos SQL como por ejemplo `DATE`. Podremos acceder a él tanto con `getDate()` como con `getString()`. La diferencia es que el primero devolverá un objeto Java de tipo `Date` y el segundo devolverá un `String`.

Siempre que sea posible el driver JDBC convertirá el tipo de dato almacenado en la base de datos al tipo solicitado por el método `getXXX()`, pero hay conversiones que no se pueden realizar y lanzarán una excepción, como por ejemplo si intentamos hacer un `getInt()` sobre un campo que no contiene un valor numérico.

Sentencias que no devuelven datos

Las ejecutamos con el método `executeUpdate`. Serán todas las sentencias SQL **excepto el SELECT**, que es la de consulta. Es decir, nos servirá para las siguientes sentencias:

- Sentencias que cambian las estructuras internas de la BD donde se guardan los datos (instrucciones conocidas con las siglas **DDL**, del inglés **Data Definition Language**), como por ejemplo `CREATE TABLE`, `CREATE VIEW`, `ALTER TABLE`, `DROP TABLE`, ...,

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- Sentencias para otorgar permisos a los usuarios existentes o crear otros nuevos (subgrupo de instrucciones conocidas como **DCL** o **Data Control Language**), como por ejemplo `GRANT`.
- Y también las sentencias para modificar los datos guardados utilizando las instrucciones `INSERT`, `UPDATE` y `DELETE`.

Aunque se trata de sentencias muy dispares, desde el punto de vista de la comunicación con el SGBD se comportan de manera muy similar, siguiendo el siguiente patrón:

1. **Instanciación del** `Statement` a partir de una conexión activa.
2. **Ejecución de una sentencia SQL** pasada por parámetro al método `executeUpdate`.
3. **Cierre del objeto** `Statement` instanciado.

Miremos este ejemplo, en el que

Vamos a crear una tabla muy sencilla en la Base de Datos MySql

```
[...]
public void createTable() throws SQLException{
   Statement st = con.createStatement();
   st.executeUpdate("CREATE TABLE T1 (c1 varchar(50))");
   st.close();
}
[...]
```

Asegurar la liberación de recursos

Las instancias de `Connection` y las de `Statement` guardan, en memoria, mucha información relacionada con las ejecuciones realizadas. Además, mientras continúan activas mantienen en el SGBD una sesión abierta, que supondrá un conjunto importante de recursos abiertos, destinados a servir de forma eficiente las peticiones de los clientes. Es importante cerrar estos objetos para liberar recursos tanto del cliente como del servidor.

Si en un mismo método debemos cerrar un objeto `Statement` y el `Connection` a partir del cual la hemos creado, se deberá cerrar primero el `Statement` y después el `Connection`. Si lo hacemos al revés, cuando intentamos cerrar el `Statement` nos saltará una excepción de tipo `SQLException`, ya que el cierre de la conexión le habría dejado inaccesible.

Además de respetar el orden, asegurar la liberación de los recursos situando las operaciones de cierre dentro de un bloque `finally`. De este modo, aunque se produzcan errores, no se dejarán de ejecutar las instrucciones de cierre.

Hay que tener en cuenta todavía un detalle más cuando sea necesario realizar el cierre de varios objetos a la vez. En este caso, aunque las situamos una tras otra, todas las instrucciones de cierre dentro del bloque `finally`, no sería suficiente garantía para asegurar la ejecución de todos los cierres, ya que, si mientras se produce el cierre de un los objetos se lanza una excepción, los objetos invocados en una posición posterior a la del que se ha producido el error no se cerrarán.

La solución de este problema pasa por evitar el lanzamiento de cualquier excepción durante el proceso de cierre. Una posible forma es encapsular cada cierre entre sentencias `try-catch` dentro del `finally`.

Consulta el código completo de LiberacionRecursos.

Consulta el código completo de LiberacionRecursosTryWithResources.

11.1.7 Modificación (`update`)

Para poder modificar los datos que contiene un `ResultSet` necesitamos un `ResultSet` de tipo modificable. Para ello debemos utilizar la constante `ResultSet.CONCUR_UPDATABLE` al llamar al método `createStatement()` como se ha visto antes.

Para modificar los valores de un registro existente se utilizan una serie de métodos **`updateXXX()`** de `ResultSet`. Las XXX indican el tipo del dato y hay tantos distintos como sucede con los métodos `getXXX()` de este mismo interfaz: `updateString()`, `updateInt()`, `updateDouble()`, `updateDate()`, etc.

La diferencia es que **los métodos `updateXXX()` necesitan dos argumentos**:

- La columna que deseamos actualizar (por su nombre o por su número de columna).
- El valor que queremos almacenar en dicha columna (del tipo que sea).

Por ejemplo para modificar el campo `edad` almacenando el entero `28` habría que llamar al siguiente método, suponiendo que `rs` es un objeto `ResultSet`:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
rs.updateInt("edad", 28);
```

También podría hacerse de la siguiente manera, suponiendo que la columna `edad` es la segunda:

```
rs.updateInt(2, 28);
```

Los métodos `updateXXX()` no devuelven ningún valor (son de tipo `void`). Si se produce algún error se lanzará una `SQLException`.

Posteriormente hay que **llamar a `updateRow()` para que los cambios realizados se apliquen sobre la base de datos**. El *Driver JDBC* se encargará de ejecutar las sentencias SQL necesarias. Esta es una característica muy potente ya que nos facilita enormemente la tarea de modificar los datos de una base de datos.

En resumen, el proceso para realizar la modificación de una fila de un `ResultSet` es el siguiente:

1. **Desplazamos el cursor** al registro que queremos modificar.
2. Llamamos a todos los métodos `updateXXX(...)` que necesitemos.
3. Llamamos a `updateRow()` para que los cambios se apliquen a la base de datos.

> **Importante**
>
> Hay que llamar a `updateRow()` antes de desplazar el cursor**. Si desplazamos el cursor antes de llamar a `updateRow()`, se perderán los cambios.
>
> Si queremos **cancelar las modificaciones de un registro del `ResultSet`** podemos llamar a `cancelRowUpdates()`, que cancela todas las modificaciones realizadas sobre el registro actual.
>
> Si ya hemos llamado a `updateRow()` el método `cancelRowUpdates()` no tendrá ningún efecto.
>
> El siguiente código de ejemplo muestra cómo modificar el campo `title` del último registro de un `ResultSet` que contiene el resultado de una *SELECT* sobre la tabla de `films`:

Consulta el código completo de EjemploUpdateRow.

11.1.8 Inserción (`insert`)

Para insertar nuevos registros necesitaremos utilizar, al menos, estos dos métodos:

- **`void moveToInsertRow()`**: desplaza el cursor al *registro de inserción*. Es un registro especial utilizado para insertar nuevos registros en el `ResultSet`. Posteriormente tendremos que llamar a los métodos `updateXXX()` ya conocidos para establecer los valores del registro de inserción. Para finalizar hay que llamar a `insertRow()`.
- **`void insertRow()`**: inserta el *registro de inserción* en el `ResultSet`, pasando a ser un registro normal más, y también lo inserta en la base de datos.

Consulta el código completo de EjemploMoveToInsertRow.

Los campos cuyo valor no se haya establecido con `updateXXX()` tendrán un valor `NULL`. Si en la base de datos dicho campo no está configurado para admitir nulos se producirá una `SQLException`.

Tras insertar nuestro nuevo registro en el objeto `ResultSet` podremos volver a la anterior posición en la que se encontraba el cursor (antes de invocar `moveToInsertRow()`) llamando al método `moveToCurrentRow()`. Este método sólo se puede utilizar en combinación con `moveToInsertRow()`. El Ejemplo16 muestra el código completo.

Consulta el código completo de EjemploInsercionConSQL.

11.1.9 Borrado (`delete`)

Para eliminar un registro solo hay que desplazar el cursor al registro deseado y llamar al método:

- **`void deleteRow()`**: elimina el registro actual del `ResultSet` y también de la base de datos.

Consulta el código completo de EjemploDeleteRow.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

11.1.10 Sentencias predefinidas

Para solucionar el problema de crear sentencias sql complejas, se utiliza `PreparedStatement`.

JDBC dispone de un objeto derivado del `Statement` que se llama `PreparedStatement`, al que se le pasa la sentencia SQL en el momento de crearlo, no en el momento de ejecutar la sentencia (como pasaba con `Statement`). Y además esta sentencia puede admitir parámetros, lo que nos puede ir muy bien en determinadas ocasiones.

De cualquier modo, `PreparedStatement` presenta ventajas sobre su antecesor `Statement` cuando nos toque trabajar con sentencias que se hayan de ejecutar varias veces. La razón es que cualquier sentencia SQL, cuando se envía al SGBD será compilada antes de ser ejecutada.

- Utilizando un objeto `Statement`, cada vez que hacemos una ejecución de una sentencia, ya sea vía `executeUpdate` o bien vía `executeQuery`, el SGBD la compilará, ya que le llegará en forma de cadena de caracteres.
- En cambio, al `PreparedStament` la sentencia nunca varía y por lo tanto se puede compilar y guardar dentro del mismo objeto, por lo que las siguientes veces que se ejecute no habrá que compilarse. Esto reducirá sensiblemente el tiempo de ejecución.

En algunos sistemas gestores, además, usar `PreparedStatements` puede llegar a suponer más ventajas, ya que utilizan la secuencia de bytes de la sentencia para detectar si se trata de una sentencia nueva o ya se ha servido con anterioridad. De esta manera se propicia que el sistema guarde las respuestas en la memoria caché, de manera que se puedan entregar de forma más rápida.

La principal diferencia de los objetos `PreparedStatement` en relación a los `Statement`, es que en los primeros se les pasa la sentencia SQL predefinida en el momento de crearlo. Como la sentencia queda predefinida, ni los métodos `executeUpdate` ni `executeQuery` requerirán ningún parámetro. Es decir, justo al revés que en el `Statement`.

Los parámetros de la sentencia se marcarán con el símbolo de interrogación (`?`) Y se identificarán por la posición que ocupan en la sentencia, empezando a contar desde la izquierda a partir del número 1. El valor de los parámetros se asignará utilizando el método específico, de acuerdo con el tipo de datos a asignar. El nombre empezará por `set` y continuará con el nombre del tipo de datos (ejemplos: `setString`, `setInt`, `setLong`, `setBoolean` ...). Todos estos métodos siguen la misma sintaxis:

```
setXXXX(<posiciónEnLaSentenciaSQL>, <valor>);
```

con PreparedStatement

Este es el mismo método para insertar y eliminar un `film` pero usando `PreparedStatement`:

Consulta el código completo de EjemploInsercionBorradoConPreparedStatement.

Ventajas de `PreparedStatement` desde el Punto de Vista de Seguridad

`PreparedStatement` ofrece varias ventajas en términos de seguridad, principalmente al prevenir ataques de inyección SQL.

Prevención de Inyección SQL:

Definición

La **inyección SQL** es un tipo de ataque en el que un atacante inserta o "inyecta" código SQL malicioso en una consulta a través de entradas de usuario. Esto puede permitir a un atacante ejecutar comandos SQL no autorizados, acceder a datos sensibles o manipular la base de datos de formas inesperadas.

Ejemplo de Inyección SQL:

Supongamos que tienes un código que construye una consulta SQL concatenando cadenas de texto:

```
String query = "SELECT * FROM usuarios WHERE username = '" + username + "' AND password = '" + password + "'";
```

Si un atacante proporciona un `username` como `' OR '1'='1` y una `password` como `' OR '1'='1`, la consulta resultante sería:

```
SELECT * FROM usuarios WHERE username = '' OR '1'='1' AND password = '' OR '1'='1'
```

Esta consulta siempre devolvería todos los registros de la tabla `usuarios`, lo que podría permitir al atacante acceder a datos que no deberían estar disponibles.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

11.1.11 Ejemplos UD10

Ejemplo01

Conexión básica a MySQL usando `DriverManager`.

```java
package es.martinezpenya.ejemplos.UD10._02_JDBC;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class UD10_01_ConectarMySQL {
    public static void main(String[] args) {
        try {
            // Dependiendo de a qué tipo de SGBD queramos conectar cargaremos un controlador u otro
            // Intentar cargar el driver de MySQL
            Class<?> c = Class.forName("com.mysql.cj.jdbc.Driver");
            System.out.println("Cargado " + c.getName());

            //Definir la url de conexión y los parámetros de usuario y contraseña
            String host = "jdbc:mysql://databasedmp.cipxbdkxiaqy.us-east-1.rds.amazonaws.com:3306/starwars";
            String username = "admin";
            String password = "123456Ab$";
            Connection con = DriverManager.getConnection(host, username, password);

            System.out.println("Conexión completada");
            //cerrar la conexión
            con.close();
        } catch (ClassNotFoundException cnfe) {
            System.out.println(cnfe.getMessage());
        } catch (SQLException ex) {
            System.out.println("ERROR al conectar: " + ex.getMessage());
        }
    }
}
```

⬆ Volver a teoría

Ejemplo02

Conexión alternativa a MySQL mediante objeto `Driver`.

```java
package es.martinezpenya.ejemplos.UD10._02_JDBC;

import java.sql.Connection;
import java.sql.Driver;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.Properties;

public class UD10_02_ConectarMySQLDriver {
    public static void main(String[] args) {
        //Definir la url de conexión y los parámetros de usuario y contraseña
        String url = "jdbc:mysql://databasedmp.cipxbdkxiaqy.us-east-1.rds.amazonaws.com:3306/starwars";
        String username = "admin";
        String password = "123456Ab$";

        try {
            Driver driver = DriverManager.getDriver(url);
            //Establecer las propiedades de la conexión
            Properties properties = new Properties();
            properties.setProperty("user", username);
            properties.setProperty("password", password);
            //Conectar
            Connection con = driver.connect(url, properties);
            System.out.println("Conexión completada a través de Driver");
            //Cerrar la conexión
            con.close();
        } catch (SQLException ex) {
            System.out.println("ERROR al conectar: " + ex.getMessage());
        }
    }
}
```

⬆ Volver a teoría

Ejemplo03

Patrón Singleton para gestionar la conexión a la base de datos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package es.martinezpenya.ejemplos.UD10._03_Patrones._01_Singleton;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DatabaseConnection {

    // 1. Declaramos una instancia estática y privada de la clase
    private static DatabaseConnection instance;
    private Connection connection;

    // 2. Hacemos el constructor privado para evitar que se creen instancias desde fuera
    private DatabaseConnection() {
        try {
            // Establecemos la conexión a la base de datos
            String url = "jdbc:mysql://databasedmp.cipxbdkxiaqy.us-east-1.rds.amazonaws.com:3306/starwars";
            String user = "admin";
            String password = "123456Ab$";
            this.connection = DriverManager.getConnection(url, user, password);
            System.out.println("Conexión a la base de datos establecida.");
        } catch (SQLException e) {
            System.out.println("Error al conectar a la base de datos: " + e.getMessage());
        }
    }

    // 3. Método estático para obtener la instancia única (Singleton)
    public static DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }

    // 4. Método para obtener la conexión
    public Connection getConnection() {
        return connection;
    }

    // 5. Método para cerrar la conexión (opcional)
    public void closeConnection() {
        if (connection != null) {
            try {
                connection.close();
                System.out.println("Conexión cerrada.");
            } catch (SQLException e) {
                System.out.println("Error al cerrar la conexión: " + e.getMessage());
            }
        }
    }
}
```

⬆ Volver a teoría

Ejemplo04

Prueba de la clase `DatabaseConnection` Singleton.

```java
package es.martinezpenya.ejemplos.UD10._03_Patrones._01_Singleton;

import java.sql.Connection;

public class DatabaseConnectionTest {
    public class Main {
        public static void main(String[] args) {
            // Obtenemos la instancia única de DatabaseConnection
            DatabaseConnection dbConnection = DatabaseConnection.getInstance();

            // Usamos la conexión para interactuar con la base de datos
            Connection connection = dbConnection.getConnection();
            // Aquí podríamos ejecutar consultas SQL, por ejemplo:
            // Statement statement = connection.createStatement();
            // ResultSet resultSet = statement.executeQuery("SELECT * FROM usuarios");

            // Cerramos la conexión cuando ya no la necesitemos
            dbConnection.closeConnection();
        }
    }
}
```

⬆ Volver a teoría

Ejemplo05

Pool de conexiones con HikariCP y patrón Singleton.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package es.martinezpenya.ejemplos.UD10._03_Patrones._02_PoolObject;

import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
import java.sql.Connection;
import java.sql.SQLException;

public class HikariCPSingleton {

    private static HikariDataSource dataSource;

    // Configuración del pool de conexiones
    static {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl("jdbc:mariadb://databasedmp.cipxbdkxiaqy.us-east-1.rds.amazonaws.com:3306/starwars"); // URL de la base de datos
        config.setUsername("admin"); // Usuario de la base de datos
        config.setPassword("123456Ab$"); // Contraseña de la base de datos
        config.setMaximumPoolSize(10); // Número máximo de conexiones en el pool
        config.setMinimumIdle(2); // Número mínimo de conexiones inactivas en el pool
        config.setIdleTimeout(30000); // Tiempo de espera para conexiones inactivas (en milisegundos)
        config.setMaxLifetime(1800000); // Tiempo máximo de vida de una conexión (en milisegundos)
        config.setConnectionTimeout(30000); // Tiempo de espera para obtener una conexión (en milisegundos)

        dataSource = new HikariDataSource(config);
    }

    // Método para obtener una conexión del pool
    public static Connection getConnection() throws SQLException {
        return dataSource.getConnection();
    }

    // Método para cerrar el pool de conexiones (opcional)
    public static void closeDataSource() {
        if (dataSource != null) {
            dataSource.close();
        }
    }
}
```

⬆ Volver a teoría

Ejemplo06

Prueba de la clase `HikariCPSingleton`.

```java
package es.martinezpenya.ejemplos.UD10._03_Patrones._02_PoolObject;

import java.sql.Connection;
import java.sql.SQLException;

public class HikariCPSingletonTest {

    private static Connection connection;

    public static void main(String[] args) {
        try {
            connection = HikariCPSingleton.getConnection();
            // Hacemos algo con la conexión
        } catch (SQLException e) {
            System.out.println("ERROR al conectar: " + e.getMessage());
        }
    }
}
```

⬆ Volver a teoría

Ejemplo07

Modelo POJO para la entidad `Film`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package es.martinezpenya.ejemplos.UD10._03_Patrones._03_DAO;

    public class Film {
     private int id;
     private String episode;
     private String title;

     // Getters y setters
     public int getId() {
         return id;
     }

     public void setId(int id) {
         this.id = id;
     }

     public String getEpisode() {
         return episode;
     }

     public void setEpisode(String episode) {
         this.episode = episode;
     }

     public String getTitle() {
         return title;
     }

     public void setTitle(String title) {
         this.title = title;
     }
    }
```

⬆ Volver a teoría

Ejemplo08

Interfaz DAO para operaciones CRUD de `Film`.

```java
package es.martinezpenya.ejemplos.UD10._03_Patrones._02_DAO;

    public interface FilmDAO {
     void agregarFilm(Film film);         // Crear una nueva película
        Film obtenerFilm(int id);            // Obtener una película por su ID
     void actualizarFilm(Film film);      // Actualizar una película existente
        void eliminarFilm(int id);           // Eliminar una película por su ID
    }
```

⬆ Volver a teoría

Ejemplo09

Implementación de la interfaz `FilmDAO` con JDBC.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package es.martinezpenya.ejemplos.UD10._03_Patrones._03_DAO;

    import es.martinezpenya.ejemplos.UD10._03_Patrones._02_PoolObject.HikariCPSingleton;

    import java.sql.Connection;
    import java.sql.PreparedStatement;
    import java.sql.ResultSet;
    import java.sql.SQLException;

    public class FilmDAOImplementado implements FilmDAO {

        private Connection connection;

        public FilmDAOImplementado() {
            try {
                connection = HikariCPSingleton.getConnection();
            } catch (SQLException e) {
                System.out.println("ERROR al conectar: " + e.getMessage());
            }
        }

        @Override
        public void agregarFilm(Film film) {
            try {
                PreparedStatement ps = connection.prepareStatement("INSERT INTO films (id, episode, title) VALUES (?,?,?)");
                ps.setInt(1, film.getId());
                ps.setString(2, film.getEpisode());
                ps.setString(3, film.getTitle());
                ps.executeUpdate();
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }

        @Override
        public Film obtenerFilm(int id) {
            Film film = null;
            try {
                PreparedStatement ps = connection.prepareStatement("SELECT * FROM films WHERE id = ?");
                ps.setInt(1, id);
                ResultSet rs = ps.executeQuery();
                if (rs.next()) {
                    film = new Film();
                    film.setId(rs.getInt("id"));
                    film.setEpisode(rs.getString("episode"));
                    film.setTitle(rs.getString("title"));
                }
            } catch (SQLException e) {
                e.printStackTrace();
            }
            return film;
        }

        @Override
        public void actualizarFilm(Film film) {
            try {
                PreparedStatement ps = connection.prepareStatement("UPDATE films SET episode = ?, title = ? WHERE id = ?");
                ps.setString(1, film.getEpisode());
                ps.setString(2, film.getTitle());
                ps.setInt(3, film.getId());
                ps.executeUpdate();
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }

        @Override
        public void eliminarFilm(int id) {
            try {
                PreparedStatement ps = connection.prepareStatement("DELETE FROM films WHERE id = ?");
                ps.setInt(1, id);
                ps.executeUpdate();
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
```

⬆ Volver a teoría

Ejemplo10

Prueba completa del DAO: insertar, obtener, actualizar y eliminar películas.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package es.martinezpenya.ejemplos.UD10._03_Patrones._03_DAO;

public class TestFilmDAOImplementado {

    public static void main(String[] args) {
        // Crear una instancia de FilmDAOImplementado
        FilmDAOImplementado filmDAO = new FilmDAOImplementado();

        // Crear un film ficticio para el episodio 7
        Film film = new Film();
        film.setId(7); // Cambiamos el episodio a 7
        film.setEpisode("VII");
        film.setTitle("The Force Awakens"); // Cambiamos el título

        // 1. Insertar el film en la base de datos
        System.out.println("Insertando el film...");
        filmDAO.agregarFilm(film);
        System.out.println("Film insertado: " + film.getTitle());

        // 2. Recuperar el film recién insertado (asumiendo que el ID es 1)
        System.out.println("\nRecuperando el film con ID 7...");
        Film filmRecuperado = filmDAO.obtenerFilm(7);
        if (filmRecuperado != null) {
            System.out.println("Film recuperado: " + filmRecuperado.getTitle() + " (" + filmRecuperado.getEpisode() + ")");
        } else {
            System.out.println("No se encontró el film con ID 7.");
        }

        // 3. Modificar el film recuperado
        if (filmRecuperado != null) {
            System.out.println("\nModificando el film...");
            filmRecuperado.setEpisode("Episode VII"); // Cambiamos el título para agregar más detalle
            filmDAO.actualizarFilm(filmRecuperado);
            System.out.println("Film modificado: " + filmRecuperado.getTitle());

            // Verificar la modificación
            System.out.println("\nRecuperando el film modificado...");
            Film filmModificado = filmDAO.obtenerFilm(filmRecuperado.getId());
            if (filmModificado != null) {
                System.out.println("Film modificado recuperado: " + filmModificado.getTitle() + " (" + filmModificado.getEpisode() + ")");
            } else {
                System.out.println("No se encontró el film modificado.");
            }
        }

        // 4. Eliminar el film
        if (filmRecuperado != null) {
            System.out.println("\nEliminando el film con ID " + filmRecuperado.getId() + "...");
            filmDAO.eliminarFilm(filmRecuperado.getId());
            System.out.println("Film eliminado.");

            // Verificar la eliminación
            System.out.println("\nIntentando recuperar el film eliminado...");
            Film filmEliminado = filmDAO.obtenerFilm(filmRecuperado.getId());
            if (filmEliminado == null) {
                System.out.println("El film fue eliminado correctamente.");
            } else {
                System.out.println("El film no fue eliminado.");
            }
        }
    }
}
```

⬆ Volver a teoría

Ejemplo11

Navegación de un `ResultSet` con scroll insensitive.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package es.martinezpenya.ejemplos.UD10._05_NavegabilidadConcurrencia;
    import es.martinezpenya.ejemplos.UD10._03_Patrones._02_PoolObject.HikariCPSingleton;

    import java.sql.*;

    public class EjemploScrollInsensitive {

        public static void main(String[] args) {
            try (Connection con = HikariCPSingleton.getConnection();
                 Statement stmt = con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY);
                 ResultSet rs = stmt.executeQuery("SELECT id, name FROM characters")) {

                // Mover a la primera fila
                if (rs.first()) {
                    System.out.println("primera fila: " + rs.getInt("id") + ", " + rs.getString("name"));
                }

                // Mover a la última fila
                if (rs.last()) {
                    System.out.println("última fila: " + rs.getInt("id") + ", " + rs.getString("name"));
                }

                // Simulamos un retraso y actualizamos la base de datos (en otra sesión)
                System.out.println("Esperando las actualizaciones...");
                Thread.sleep(10000); // Esperar 10 segundos

                // Mover a la primera fila otra vez
                if (rs.first()) {
                    System.out.println("primera fila después de esperar: " + rs.getInt("id") + ", " + rs.getString("name"));
                }

            } catch (SQLException | InterruptedException ex) {
                ex.printStackTrace();
            }
        }
    }
```

⬆ Volver a teoría

Ejemplo12

Obtención de metadatos de un `ResultSet`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package es.martinezpenya.ejemplos.UD10._06_Consultas;

    import es.martinezpenya.ejemplos.UD10._03_Patrones._02_PoolObject.HikariCPSingleton;

    import java.sql.*;

    public class EjemploResultSetMetaData {
     public static void main(String[] args) {
         try (Connection con = HikariCPSingleton.getConnection();
              Statement stmt = con.createStatement();
              ResultSet rs = stmt.executeQuery("SELECT id, episode, title FROM films")) {

             // Obtener metadata del ResulSet
             ResultSetMetaData rsmd = rs.getMetaData();

             // Obtener el número de columnas
             int columnCount = rsmd.getColumnCount();
             System.out.println("Número de columnas: " + columnCount);

             // Listar las columnas de detalles
             for (int i = 1; i <= columnCount; i++) {
                 String columnName = rsmd.getColumnName(i);
                 String columnType = rsmd.getColumnTypeName(i);
                 int columnDisplaySize = rsmd.getColumnDisplaySize(i);
                 boolean isNullable = rsmd.isNullable(i) == ResultSetMetaData.columnNullable;

                 System.out.println("Columna " + i + ":");
                 System.out.println("  Nombre: " + columnName);
                 System.out.println("  Tipo: " + columnType);
                 System.out.println("  Tamaño display: " + columnDisplaySize);
                 System.out.println("  Nullable: " + isNullable);
             }

             // Iterar sobre el conjunto de resultados
             while (rs.next()) {
                 for (int i = 1; i <= columnCount; i++) {
                     System.out.print(rs.getString(i) + " ");
                 }
                 System.out.println();
             }

         } catch (SQLException ex) {
             System.out.println("Error de SQL: " + ex.getMessage());
         }
      }
     }
```

⬆ Volver a teoría

Ejemplo13

Liberación correcta de recursos JDBC (Connection, Statement, ResultSet).

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package es.martinezpenya.ejemplos.UD10._06_Consultas;

    import es.martinezpenya.ejemplos.UD10._03_Patrones._02_PoolObject.HikariCPSingleton;

    import java.sql.Connection;
    import java.sql.ResultSet;
    import java.sql.SQLException;
    import java.sql.Statement;

    public class LiberacionRecursos {
        static Statement stmt = null;
        static ResultSet rs = null;
        static Connection con;

        public static void main(String[] args) {
            try {
                con = HikariCPSingleton.getConnection();
                stmt = con.createStatement();
                rs = stmt.executeQuery("SELECT id, name, hair_color FROM characters");

                while (rs.next()) {
                    System.out.print(rs.getInt(1) + "\t");
                    System.out.print(rs.getString(2) + "\t");
                    System.out.println(rs.getString(3));
                }
            } catch (SQLException ex) {
                System.out.println("Error de SQL: " + ex.getMessage());
            } finally {
                try {
                    //Siempre se debe cerrar todo lo abierto
                    if (rs != null) {
                        rs.close();
                    }
                } catch (java.sql.SQLException ex) {
                    System.out.println("ERROR: " + ex.getMessage());
                }
                try {
                    //Siempre se debe cerrar todo lo abierto
                    if (stmt != null) {
                        stmt.close();
                    }
                } catch (java.sql.SQLException ex) {
                    System.out.println("ERROR: " + ex.getMessage());
                }
            }
        }
    }
```

⬆ Volver a teoría

Ejemplo14

Liberación de recursos usando try-with-resources.

```java
package es.martinezpenya.ejemplos.UD10._06_Consultas;

    import es.martinezpenya.ejemplos.UD10._03_Patrones._02_PoolObject.HikariCPSingleton;

    import java.sql.Connection;
    import java.sql.ResultSet;
    import java.sql.SQLException;
    import java.sql.Statement;

    public class LiberacionRecursosTryWithResources {
        public static void main(String[] args) {
            try (Connection con = HikariCPSingleton.getConnection();
                 Statement stmt = con.createStatement();
                 ResultSet rs = stmt.executeQuery("SELECT id, name, hair_color FROM characters");) {

                while (rs.next()) {
                    System.out.print(rs.getInt(1) + "\t");
                    System.out.print(rs.getString(2) + "\t");
                    System.out.println(rs.getString(3));
                }
            } catch (SQLException ex) {
                System.out.println("Error de SQL: " + ex.getMessage());
            }
        }
    }
```

⬆ Volver a teoría

Ejemplo15

Actualización de registros con `executeUpdate`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package es.martinezpenya.ejemplos.UD10._07_Modificacion;

    import es.martinezpenya.ejemplos.UD10._03_Patrones._02_PoolObject.HikariCPSingleton;

    import java.sql.*;

    public class EjemploUpdateRow {

        public static void main(String[] args) {
            String sql = "SELECT * FROM films";
            try (Connection con = HikariCPSingleton.getConnection();
                 // Creamos un Statement scrollable y modificable
                 Statement st = con.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_UPDATABLE);
                 // Ejecutamos un SELECT y obtenemos la tabla films en un ResultSet
                 ResultSet rs = st.executeQuery(sql);) {
                // Nos movemos al último registro y lo actualizamos
                rs.last();
                rs.updateString(3, "Return Of The Jedi"); // Ponemos todas las palabras en mayúsculas
                rs.updateRow();
            } catch (SQLException ex) {
                System.out.println("Error de SQL: " + ex.getMessage());
            }
        }
    }
```

⬆ Volver a teoría

Ejemplo16

Inserción de registros con `moveToInsertRow`.

```java
package es.martinezpenya.ejemplos.UD10._08_Insercion;

import es.martinezpenya.ejemplos.UD10._03_Patrones._02_PoolObject.HikariCPSingleton;

import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class EjemploMoveToInsertRow {
    public static void main(String[] args) {
        String sql = "SELECT * FROM films";
        try (Connection con = HikariCPSingleton.getConnection();
             Statement st = con.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_UPDATABLE);
             ResultSet rs = st.executeQuery(sql);) {
            // Creamos un nuevo registro y lo insertamos
            rs.moveToInsertRow();
            rs.updateInt(1, 7);
            rs.updateString(2, "Episode VII");
            rs.updateString(3, "The force awakens");
            rs.insertRow();
        } catch (SQLException ex) {
            System.out.println("Error de SQL: " + ex.getMessage());
        }
    }
}
```

⬆ Volver a teoría

Ejemplo17

Borrado de registros con `deleteRow`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package es.martinezpenya.ejemplos.UD10._09_Borrado;

import es.martinezpenya.ejemplos.UD10._03_Patrones._02_PoolObject.HikariCPSingleton;

import java.sql.*;

public class EjemploDeleteRow {

    public static void main(String[] args) {
        String sql = "SELECT * FROM films";
        try (Connection con = HikariCPSingleton.getConnection();
             Statement st = con.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_UPDATABLE);
             ResultSet rs = st.executeQuery(sql);) {
            // Desplazamos el cursor al tercer registro
            rs.absolute(3);
            rs.deleteRow();
        } catch (SQLException ex) {
            System.out.println("Error de SQL: " + ex.getMessage());
        }
    }
}
```

⬆ Volver a teoría

Ejemplo18

Inserción de registros mediante sentencias SQL. Este ejemplo muestra cómo usar sentencias SQL para realizar la inserción de nuevos registros, ya veremos que esta no es la forma más aconsejable por seguridad.

```java
package es.martinezpenya.ejemplos.UD10._08_Insercion;

    import es.martinezpenya.ejemplos.UD10._03_Patrones._02_PoolObject.HikariCPSingleton;

    import java.sql.*;

    public class EjemploInsercionConSQL {

        public static void insertFilmFijo(){
            String sql = "INSERT INTO films (id, episode, title) VALUES (7, 'Episode VII', 'The force awakens')";
            try (Connection con = HikariCPSingleton.getConnection();
                 // Creamos un Statement scrollable y modificable
                 Statement st = con.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_UPDATABLE);
                 // Ejecutamos un SELECT y obtenemos la tabla films en un ResultSet
                 ResultSet rs = st.executeQuery(sql);) {
            } catch (SQLException ex) {
                System.out.println("ERROR al insertar el film: " + ex.getMessage());
            }
        }

        public static void insertFilmParametros(int id, String episode, String title){
            // OJO!! Forma no recomendada por problemas de seguridad (SQL Injection)
            String sql = "INSERT INTO films (id, episode, title) VALUES (" + id + ", '" + episode + "', '" + title + "')";
            try (Connection con = HikariCPSingleton.getConnection();
                 // Creamos un Statement scrollable y modificable
                 Statement st = con.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_UPDATABLE);
                 // Ejecutamos un SELECT y obtenemos la tabla films en un ResultSet
                 ResultSet rs = st.executeQuery(sql);) {
            } catch (SQLException e) {
                System.out.println("Se ha producido un error al insertar el film. Mensaje: " + e.getMessage());
            }
        }

        public static void main(String[] args) {
            insertFilmFijo();
            insertFilmParametros(8, "Episode VIII", "The last Jedi");
        }
    }
```

⬆ Volver a teoría

Ejemplo19

Inserción y borrado con `PreparedStatement`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package es.martinezpenya.ejemplos.UD10._10_Predefinidas;

    import java.sql.Connection;
    import java.sql.PreparedStatement;
    import java.sql.SQLException;
    import java.util.Scanner;

    import es.martinezpenya.ejemplos.UD10._03_Patrones._02_PoolObject.HikariCPSingleton;

    public class EjemploInsercionBorradoConPreparedStatement {

        public static void insertFilmPrepared(int id, String episode, String title) {
            String sql = "INSERT INTO films (id, episode, title) VALUES (?,?,?)";
            try (Connection con = HikariCPSingleton.getConnection(); // Obtener conexión del pool
                 PreparedStatement pst = con.prepareStatement(sql)) {
             // Asignamos los valores a los parámetros
                pst.setInt(1, id);
                pst.setString(2, episode);
                pst.setString(3, title);
             // Ejecutamos la sentencia preparada sin peligro de SQL Injection
                pst.executeUpdate();
            } catch (SQLException e) {
                System.out.println("Se ha producido un error al insertar el film. Mensaje: " + e.getMessage());
            }
        }

        public static void deleteFilmPrepared(int id) {
            String sql = "DELETE FROM films WHERE id = ?";
            try (Connection con = HikariCPSingleton.getConnection(); // Obtener conexión del pool
                 PreparedStatement pst = con.prepareStatement(sql)) {
                // Asignamos el valor al parámetro
                pst.setInt(1, id);
                // Ejecutamos la sentencia preparada sin peligro de SQL Injection
                pst.executeUpdate();
            } catch (SQLException e) {
                System.out.println("Se ha producido un error al borrar el film. Mensaje: " + e.getMessage());
            }
        }

     public static void main(String[] args) {
            insertFilmPrepared(8, "Episode VIII", "The last Jedi");
            Scanner sc = new Scanner(System.in);
            System.out.println("Pausa, pulsa ENTER para borrar el film introducido....");
         sc.nextLine();
            deleteFilmPrepared(8);
        }
    }
```

⬆ Volver a teoría

11.1.12 Resumen — Conceptos clave

Concepto	Definición
JDBC	API de Java para conectar y operar con bases de datos relacionales
Connection	Objeto que representa la conexión a la base de datos
Statement	Objeto para ejecutar sentencias SQL
PreparedStatement	Statement precompilado que evita inyección SQL
ResultSet	Conjunto de resultados devueltos por una consulta
DAO	Patrón de diseño para abstraer el acceso a datos
CRUD	Create, Read, Update, Delete — operaciones básicas sobre datos

11.1.13 Autoevaluación

- Establezco conexiones JDBC con bases de datos
- Ejecuto consultas SQL con Statement y PreparedStatement
- Recorro ResultSets y extraigo información
- Realizo operaciones CRUD completas
- Implemento el patrón DAO
- Comprendo la importancia de PreparedStatement frente a Statement

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

11.1.14 Vídeos recomendados

Canal	Vídeo	Contenido
makigas	Serie JDBC Moderno	12 vídeos (2h): driver, conexión, PreparedStatement, transacciones, DataSource
makigas	Serie JDBC (2016)	24 vídeos: instalación MySQL, CRUD completo con DAO, PostgreSQL
Píldoras Informáticas	JDBC I (vídeo 201)	Acceso a bases de datos con JDBC
DiscoDurodeRoer	Curso JDBC — playlist	Conexión, consultas, CRUD con JDBC

21 de julio de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

11.2 Ejercicios de la UD10

Paso previo inicial [REQUERIDO!]

Para la realización de los ejercicios deberás crear una base de datos en tu **Sistema Gestor de Bases de Datos** [SGBD] (MySql/MariaDB) de nombre `prXXYY_tuNombre` (donde XXYY hace referencia al curso en mi caso `pr2526_DavidMartinez`) .

Tu SGBD puedes escogerlo tu mismo, mi recomendación es que uses AWS para tener tu base de datos siempre disponible en la nube.

De tu SGBD necesitarás saber:

- `username` : tu nombre de usuario
- `password` : tu contraseña
- `url` : dirección donde se encuentra tu SGBD
- `port` : puerto en el que escucha tu SGBD
- `database_name` : nombre de la BBDD (un SGBD puede gestionar más de una Base de Datos, en mi caso `pr2425_DavidMartinez`)

Descarga el fichero sql **tablas.sql** e impórtalas en tu SGBD. Para ello, ve a la pestaña de phpMyAdmin `Importa` , selecciona el fichero descargado anteriormente y ejecútalo.

11.2.1 Actividades

UD10_00_GestionProveedores

Tenemos nuestra base de datos **tablas.sql** que almacena información sobre `proveedores` . La tabla `proveedores` tiene las siguientes columnas:

- `id` : Identificador único del empleado (entero). (AUTOINCREMENTAL)
- `nombre` : Nombre del empleado (cadena de texto).
- `fecha_ingreso` : Fecha de ingreso en el sistema (fecha)
- `salario` : Salario del empleado (decimal).

Crea un ejercicio de nombre `UD10_00_GestionProveedores` en el que se liste todos los `proveedores` .

UD10_01_GestionEmpleados

Tenemos nuestra base de datos **tablas.sql** que almacena información sobre empleados. La tabla empleados tiene las siguientes columnas:

- `id` : Identificador único del empleado (entero). (AUTOINCREMENTAL)
- `nombre` : Nombre del empleado (cadena de texto).
- `salario` : Salario del empleado (decimal).

Es escribir un programa `UD10_01_GestionEmpleados` que realice las siguientes operaciones utilizando diferentes tipos de resultado y opciones de concurrencia:

- `listarEmpleados` : Mostrar en la consola todos los empleados y sus salarios.
- `actualizarSalarios` : Incrementar el salario de todos los empleados en un 10%.
- `eliminarEmpleados` : Eliminar todos los empleados cuyo salario sea menor que 3000€.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Consejo

En el `main` ejecuta por este orden:

1. `listarEmpleados`
2. `actualizarEmpleados`
3. `listarEmpleados`
4. `eliminarEmpleados`
5. `listarEmpleados`

Para probar

Puedes implementar cada operación utilizando un tipo de resultado y opción de concurrencia diferente para familiarizarte con su uso.

No olvides manejar las excepciones `SQLException` adecuadamente.

Por ejemplo, podrías probar las siguientes operaciones:

- Lista todos los empleados junto con sus salarios utilizando un `ResultSet` de tipo `TYPE_SCROLL_SENSITIVE` y opción de concurrencia `CONCUR_READ_ONLY`.
- Actualiza los salarios de todos los empleados incrementándolos en un 10% utilizando un ResultSet de tipo `TYPE_FORWARD_ONLY` y opción de concurrencia `CONCUR_UPDATABLE`.
- Elimina todos los empleados cuyo salario sea menor a 3000€ utilizando un `Statement` estándar sin necesidad de un `ResultSet`.

No olvides

1. Manejar las excepciones `SQLException` adecuadamente.
2. Ajustar la cadena de conexión a tu base de datos y reemplazar "usuario" y "contraseña" con las credenciales adecuadas.

UD10_02_GestionProductos

En la base de datos **tablas.sql** se almacena información sobre `productos`. La tabla `productos` tiene las siguientes columnas:

- `id`: Identificador único del producto (entero). (AUTOINCREMENTAL)
- `nombre`: Nombre del producto (cadena de texto).
- `precio`: Precio del producto (decimal).

Tu tarea es escribir un programa `UD10_02_GestionProductos` que realice las siguientes operaciones utilizando los métodos proporcionados:

1. `mostrarProductosPorPagina()`: Mostrar una página de productos cada vez que el usuario lo solicite. Cada página debe contener 5 productos. Implementa las funciones para mover el cursor a la primera página, página siguiente, página anterior, última página y una página específica utilizando el método `absolute(int row)`.
2. `buscarProductoPorNombre(String nombre)`: Permitir al usuario buscar un producto por su nombre. Utiliza el método `relative(int registros)` para desplazar el cursor hacia adelante o hacia atrás según la coincidencia del nombre.

UD10_03_GestionAlumnos

En la base de datos **tablas.sql** se almacena información sobre `alumnos`. La tabla `alumnos` tiene las siguientes columnas:

- `id`: Identificador único del alumno (entero). (AUTOINCREMENTAL)
- `nombre`: Nombre del alumno (cadena de texto).
- `edad`: Edad del alumno (entero).

Tu tarea es escribir un programa `UD10_03_GestionAlumnos` que realice las siguientes operaciones utilizando los métodos proporcionados:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- **Mostrar la información del alumno más joven y más viejo**: Utiliza los métodos `first()` y `last()` para mover el cursor a la primera y última fila respectivamente y obtener la información del alumno más joven y más viejo.
- **Desplazarse por los alumnos en orden inverso de edad**: Muestra la información de los alumnos en orden inverso de edad. Utiliza el método `previous()` para desplazarte hacia atrás a través de los registros.

UD10_04_GestionLibros

En la base de datos **tablas.sql** se almacena información sobre `libros`. La tabla `libros` tiene las siguientes columnas:

- `id`: Identificador único del libro (entero). (AUTOINCREMENTAL)
- `titulo`: Título del libro (cadena de texto).
- `autor`: Nombre del autor del libro (cadena de texto).
- `anio_publicacion`: Año de publicación del libro (entero).

Tu tarea es escribir un programa `UD10_04_GestionLibros` que realice las siguientes operaciones utilizando los métodos proporcionados:

- `mostrarLibrosPorDecada(int decada)`: Permite al usuario ingresar una década y mostrar todos los libros publicados en esa década.

> **Sugerencia**
>
> Puedes realizar el método `mostrarLibrosPorDecada()` de dos formas:
>
> 1. Utiliza el método `createStatement()` para crear el `Resultset` con el atributo `ResultSet.TYPE_SCROLL_INSENSITIVE`. Utiliza dentro los métodos `afterLast()` y `previous()` para mover el cursor al final y luego retroceder, así puedes comenzar desde la última fila.
> 2. Utiliza el método `preparedStatement(sql)` con una consulta en la que se listen los libros comprendidos en una década y ordenados de forma descendente por el `anio_publiacion`.

`buscarLibroPorAutor(String autor)`: Permite al usuario ingresar el nombre de un autor y muestra todos los libros escritos por ese autor.

> **Sugerencia**
>
> Puedes realizar el método `buscarLibroPorAutor()` de dos formas:
>
> 1. Utiliza el método `createStatement()` para crear el `Resultset` con el atributo `ResultSet.TYPE_SCROLL_INSENSITIVE`. Utiliza dentro el método `relative(int registros)` para desplazarte a través de los registros según las coincidencias del autor.
> 2. Utiliza el método `preparedStatement(sql)` con una consulta en la que se listen los libros que contengan la cadena autor dentro del campo autor.

UD10_05_GestionVentas

En la base de datos **tablas.sql** se almacena información sobre `ventas`. La tabla `ventas` tiene las siguientes columnas:

- `id`: Identificador único de la venta (entero). (AUTOINCREMENTAL)
- `producto`: Nombre del producto vendido (cadena de texto).
- `cantidad`: Cantidad de productos vendidos (entero).
- `total`: Total de la venta (decimal).

Tu tarea es escribir un programa `UD10_05_GestionVentas` que realice las siguientes operaciones utilizando los métodos proporcionados:

- Calcular el total de ventas: Utiliza el método `next()` para recorrer todas las ventas y sumar los totales para obtener el total general de ventas.
- Buscar ventas por producto: Permite al usuario ingresar el nombre de un producto y muestra todas las ventas asociadas a ese producto. Utiliza el método `relative(int registros)` para desplazarte a través de los registros según las coincidencias del producto.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

UD10_06_GestionPedidos

En la base de datos **tablas.sql** se almacena información sobre `pedidos`. La tabla `pedidos` tiene las siguientes columnas:

- `id`: Identificador único del pedido (entero). (AUTOINCREMENTAL)
- `cliente`: Nombre del cliente que realizó el pedido (cadena de texto).
- `producto`: Nombre del producto pedido (cadena de texto).
- `cantidad`: Cantidad del producto solicitada en el pedido (entero).
- `fecha`: Fecha en que se realizó el pedido (fecha).

Tu tarea es escribir un programa `UD_06_GestionPedidos` que realice las siguientes operaciones utilizando los métodos proporcionados:

- Listar pedidos por cliente: Permite al usuario ingresar el nombre de un cliente y mostrar todos los pedidos realizados por ese cliente. Utiliza el método `relative(int registros)` para desplazarte a través de los registros según las coincidencias del cliente.
- Buscar pedidos por fecha: Permite al usuario ingresar una fecha y mostrar todos los pedidos realizados en esa fecha. Utiliza el método `afterLast()` y `previous()` para mover el cursor al final y luego retroceder, así puedes comenzar desde la última fila.

UD10_07_GestionEmpleados (continuación)

Continuando con el ejercicio de gestión de empleados, copia el programa `UD10_01_GestionEmpleados`, cambia el nombre a `UD10_07_GestionEmpleados` y agrega algunas funcionalidades adicionales:

- **Mostrar información del empleado por ID**: Permite al usuario ingresar el ID de un empleado y muestra toda la información relacionada con ese empleado. Utiliza el método `absolute(int row)` para posicionarte en el registro del empleado especificado.
- **Buscar empleados por salario**: Permite al usuario ingresar un rango de salarios y mostrar todos los empleados cuyo salario esté dentro de ese rango. Utiliza el método `next()` para recorrer todas las filas y filtrar los empleados según el criterio de salario.

UD10_08_GestionEstudiantes

En la base de datos **tablas.sql** se almacena información sobre `estudiantes`. La tabla `estudiantes` tiene las siguientes columnas:

- `id`: Identificador único del estudiante (entero). (AUTOINCREMENTAL)
- `nombre`: Nombre del estudiante (cadena de texto).
- `edad`: Edad del estudiante (entero).
- `promedio`: Promedio de calificaciones del estudiante (decimal).

Tu tarea es escribir un programa `UD_08_GestionEstudiantes` que realice las siguientes operaciones utilizando los métodos proporcionados:

- **Mostrar la posición actual del estudiante**: Muestra la posición del estudiante actual en el conjunto de resultados. Utiliza el método `getRow()` para obtener el número de registro actual.
- **Validar la posición del cursor**: Verifica si el cursor está antes del primer registro, en el primer registro, en el último registro o después del último registro. Utiliza los métodos `isBeforeFirst()`, `isFirst()`, `isLast()` e `isAfterLast()` para realizar estas verificaciones.

UD10_09_GestionProductos (continuación)

Continuando con el ejercicio `UD10_02_GestionProductos`, copia el programa y cambia el nombre a `UD10_09_GestionProductos` y agrega algunas funcionalidades adicionales:

- **Mostrar el número total de productos**: Muestra el número total de productos en la base de datos. Utiliza el método `getRow()` para obtener el número de registro actual y `last()` para mover el cursor a la última fila.
- **Verificar si hay productos disponibles**: Verifica si hay algún producto disponible en la base de datos. Utiliza los métodos `isBeforeFirst()` e `isAfterLast()` para determinar si el cursor está antes del primer registro o después del último registro, respectivamente.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

UD10_10_GestionEmpleados (continuación)

Continuando con el ejercicio de gestión de empleados `UD10_07_GestionEmpleados` haz una copia y nómbralo `UD10_10_GestionEmpleados` y agrega algunas funcionalidades adicionales:

- **Verificar si hay empleados en la base de datos**: Verifica si hay algún empleado registrado en la base de datos. Utiliza los métodos `isBeforeFirst()` e `isAfterLast()` para determinar si el cursor está antes del primer registro o después del último registro, respectivamente.
- **Mostrar el primer empleado**: Muestra la información del primer empleado en la base de datos. Utiliza el método `first()` para mover el cursor al primer registro y luego muestra la información del empleado.

UD10_11_GestionVendedores

En la base de datos **tablas.sql** se almacena información sobre `vendedores`. La tabla `vendedores` tiene las siguientes columnas:

- `id`: Identificador único del cliente (entero). (AUTOINCREMENTAL)
- `nombre`: Nombre del cliente (cadena de texto).
- `fecha_ingreso`: Fecha de ingreso en el sistema (fecha)
- `salario`: Salario del vendedor (decimal).

Tu tarea es escribir un programa `UD10_11_GestionVendedores` que realice las siguientes operaciones utilizando los métodos proporcionados:

- **Mostrar la posición actual del vendedor**: Muestra la posición actual del vendedor en el conjunto de resultados. Utiliza el método `getRow()` para obtener el número de registro actual.
- **Mostrar información del último cliente**: Muestra la información del último cliente en la base de datos. Utiliza el método `last()` para mover el cursor al último registro y luego muestra la información del cliente.

21 de julio de 2026

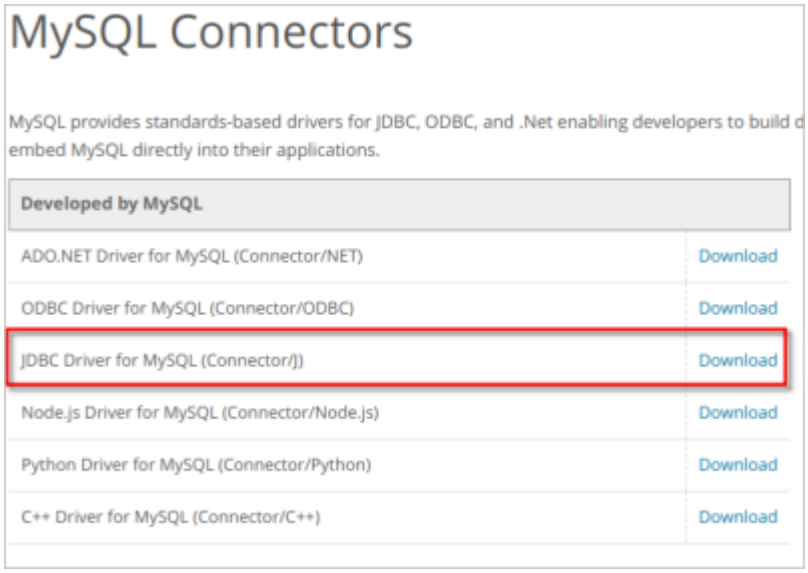
© 2025 David Martínez licensed under CC BY-NC-SA 4.0

11.3 Talleres

11.3.1 Taller UD10_02: Conectores

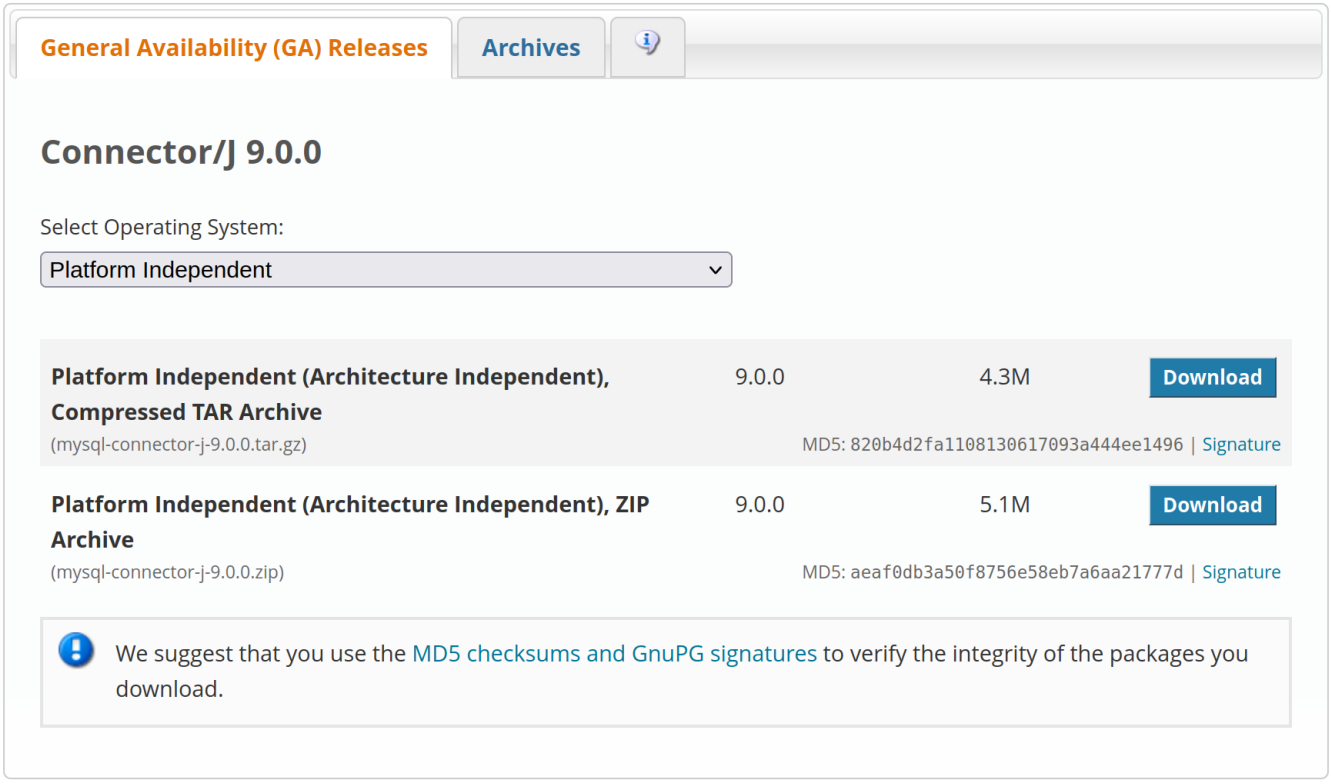
Controlador MySql (Manualmente - Jar)

1. El primer paso es descargar desde https://www.mysql.com/products/connector/ el conector apropiado.

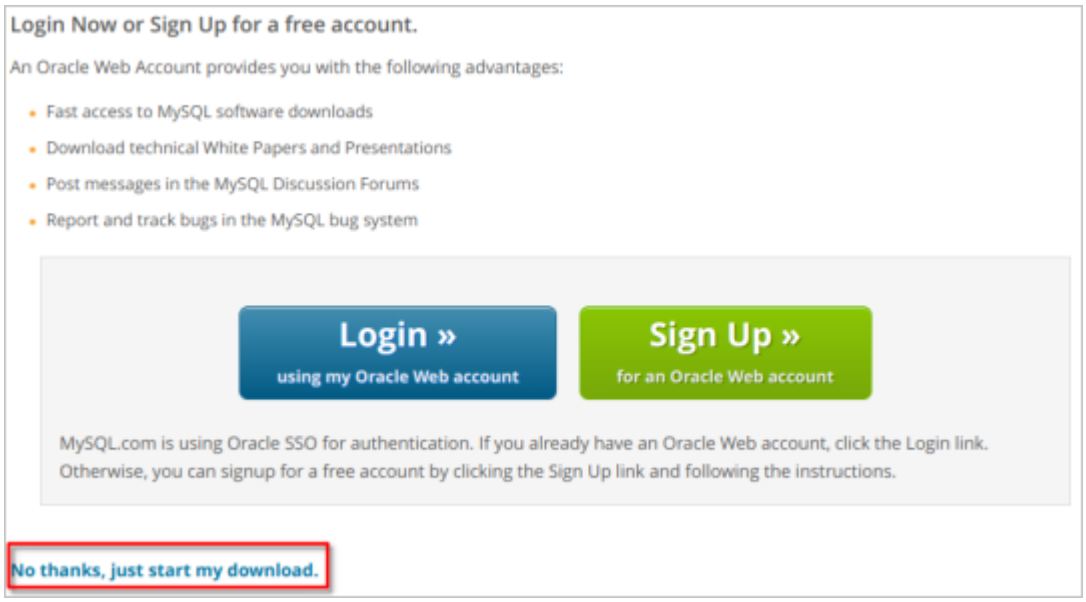


1. Elegiremos la opción independiente de la Plataforma, que nos provee del archivo jar necesario (comprimido en tar.gz o zip, según nos interese):

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



1. Haz clic en **Donwload** y selecciona la opción: **No thanks, just start download**

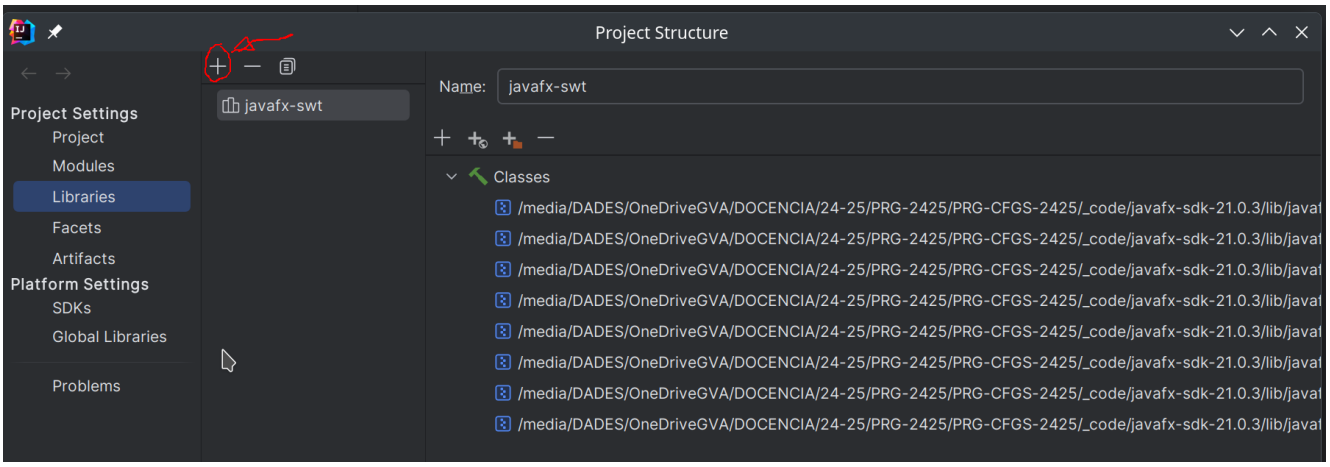


1. Ahora descomprimimos el archivo descargado, que contendrá el archivo jar que nos interesa:

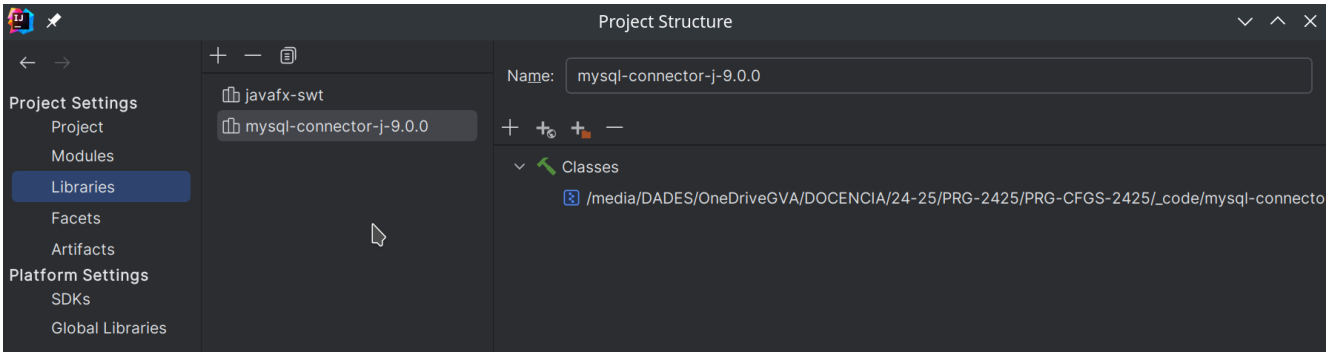
`mysql-connector-j-9.0.0.jar`

1. Ahora deberemos añadir la librería JDBC a nuestro proyecto. Este punto dependerá del IDE que estemos usando, en mi caso con IntelliJ seria en `File/Project Structure (Ctrl+Alt+Shift+S)` , y pulsar el botón `+` para añadir la libreria.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



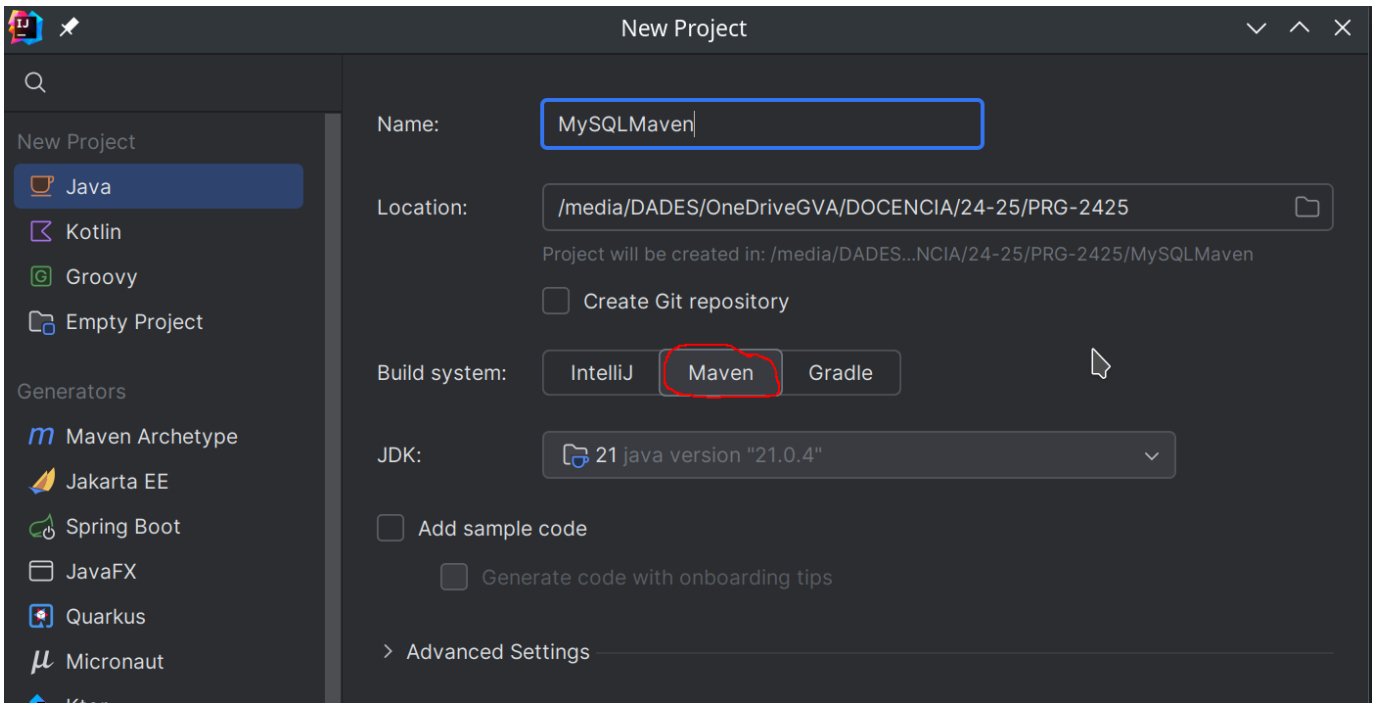
2. Busca en la carpeta descomprimida el archivo `jar` y selecciónalo, te informará de que la librería se añadirá a tu proyecto (pulsa OK) y debería quedar algo así:



Controlador MySql (IntelliJ - Maven)

1. Crear un Nuevo Proyecto con Maven:

- Al crear un nuevo proyecto en IntelliJ, selecciona "Maven" como sistema de construcción.



- IntelliJ generará la estructura del proyecto y el archivo `pom.xml`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

2. Importar un Proyecto Existente con Maven:

- Si ya tienes un proyecto con un `pom.xml`, simplemente importa el proyecto en IntelliJ y selecciona el archivo `pom.xml`. IntelliJ configurará el proyecto automáticamente.

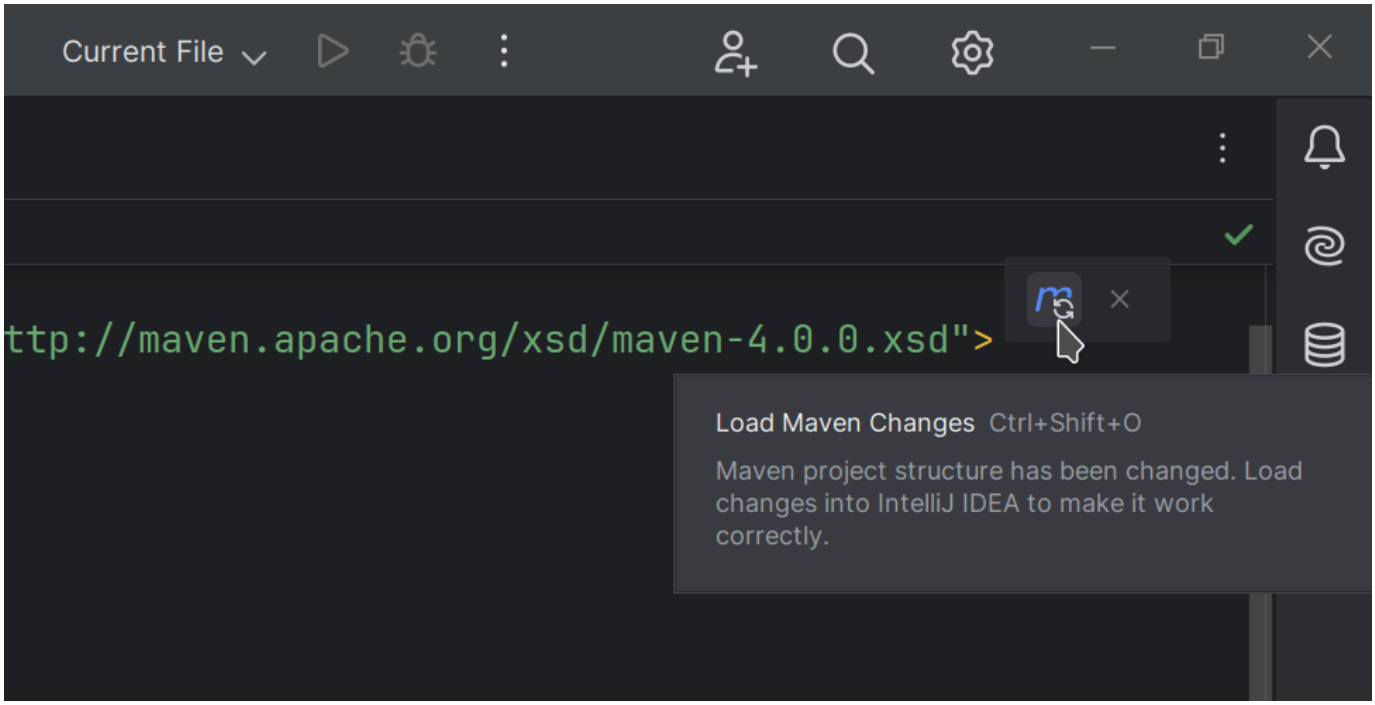
3. Agregar Dependencias:

- Abre el archivo `pom.xml` y agrega las dependencias necesarias dentro de la sección `<dependencies>` (Aquí se muestran dos, MySQL y MariaDB, por ejemplo, agrega solo una!). IntelliJ descargará y configurará automáticamente las dependencias.

```
[...]
    <dependencies>
        <!-- Dependencia de MySQL -->
        <dependency>
            <groupId>mysql</groupId>
            <artifactId>mysql-connector-java</artifactId>
            <version>8.0.33</version>
        </dependency>
        <!-- Dependencia de MariaDB -->
        <dependency>
            <groupId>org.mariadb.jdbc</groupId>
            <artifactId>mariadb-java-client</artifactId>
            <version>3.5.2</version>
        </dependency>
    </dependencies>
[...]
```

1. Actualizar Dependencias:

- Si cambias el archivo `pom.xml`, IntelliJ mostrará un botón para cargar los cambios:



Tarea

Ahora que conoces las diferencias entre agregar una libreria manualmente o mediante Maven, genera un documento pdf con el siguiente contenido:

1. Después de haber probado las dos maneras de agregar el conector `MySQL` a tu proyecto, explica que método te ha resultado más difícil. Luego, investiga si en el repositorio de central de Maven, siempre está disponible la última versión del controlador `MySQL`, crees que esto puede llegar a ser un problema?
2. Explica en tu documento como agregar los controladores (mediante Maven) para bases de datos Oracle (explica las diferencias entre el `Ojdbc8` y el `Ojdbc11`) y `SQLite`. (Capturas de pantalla y explicaciones, y capturas que demuestren que el proyecto tiene las librerías agregadas).

Envia tu fichero pdf a la tarea de Aules.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

6 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

11.3.2 Taller UD10_03: BBDD en la nube (AWS) con IntelliJ

Introducción

La intención de este documento es la de dar una perspectiva más realista del uso del acceso a datos, ya que en lugar de usar la misma máquina del alumno como servidor de BBDD, vamos a desplegar el servidor MariaDB en una máquina alojada en la nube de Amazon (AWS).

Requisitos

Para realizar esta práctica guiada necesitamos:

- Acceso al Learner Lab proporcionado por el profesor. (https://awsacademy.instructure.com)
- Conocimientos sobre las BBDD, IP's y puertos.
- Un dispositivo local con capacidad de ejecutar un cliente de BBDD, con acceso a los puertos e Ip's de AWS (Ojo con la red de conselleria)

Guía paso a paso

PREPARAR EL ENTORNO DE LA NUBE

Iniciar Laboratorio

Lo primero que necesitamos es arrancar el laboratorio, para ello Accedemos al LMS del awsacademy, buscamos el Curso facilitado por el docente, accedemos a sus contenidos y a continuación al Learner Lab. (Si es la primera vez que accedemos debemos aceptar los términos de uso).

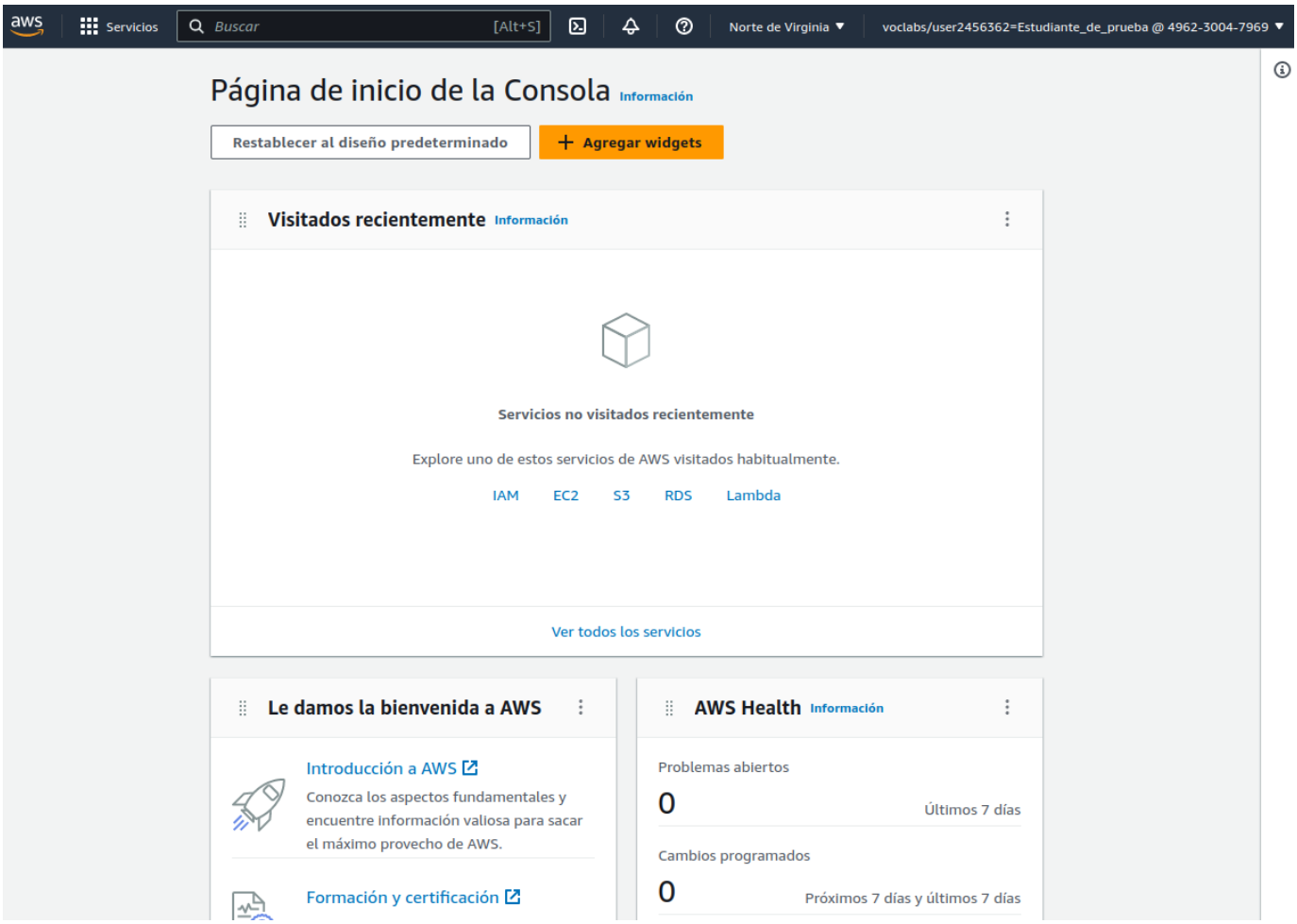
Inicialmente el laboratorio está en rojo:

Elegimos la opción `Start Lab` y esperamos a que aparezca el laboratorio en verde:

Por defecto el Learner Lab nos proporciona 100 dolares de saldo, y un tiempo de 4 horas, tras el cual se detendrán la mayoría de servicios que tengamos en marcha. Pero mientras quede saldo podemos volver a iniciar el Laboratorio y dispondremos de 4 horas más.

Una vez aparece en verde podemos hacer click sobre las letras AWS y aparecerá el Dashboard de AWS (debemos permitir las ventanas emergentes):

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

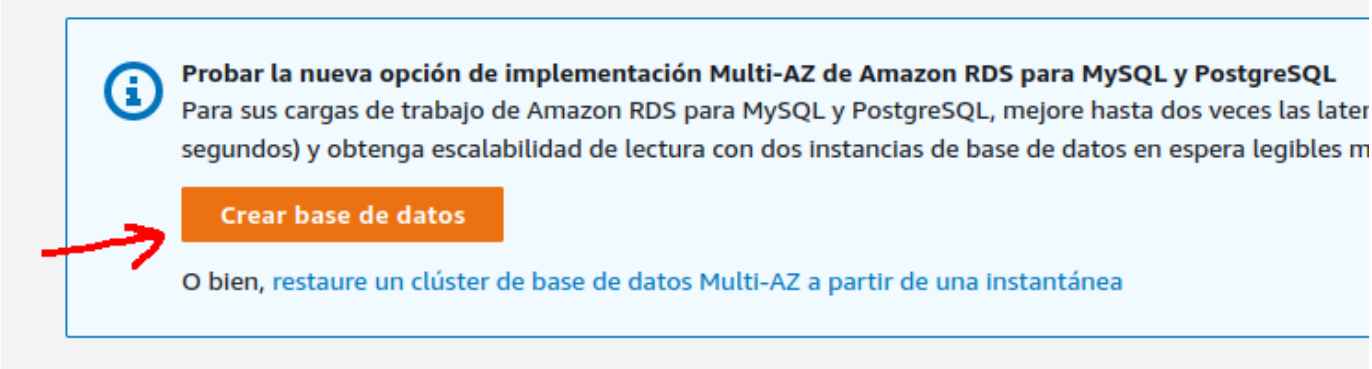


Crear una BD en AWS

Debemos buscar el apartado Amazon RDS (ahora se llama Aurora y RDS):

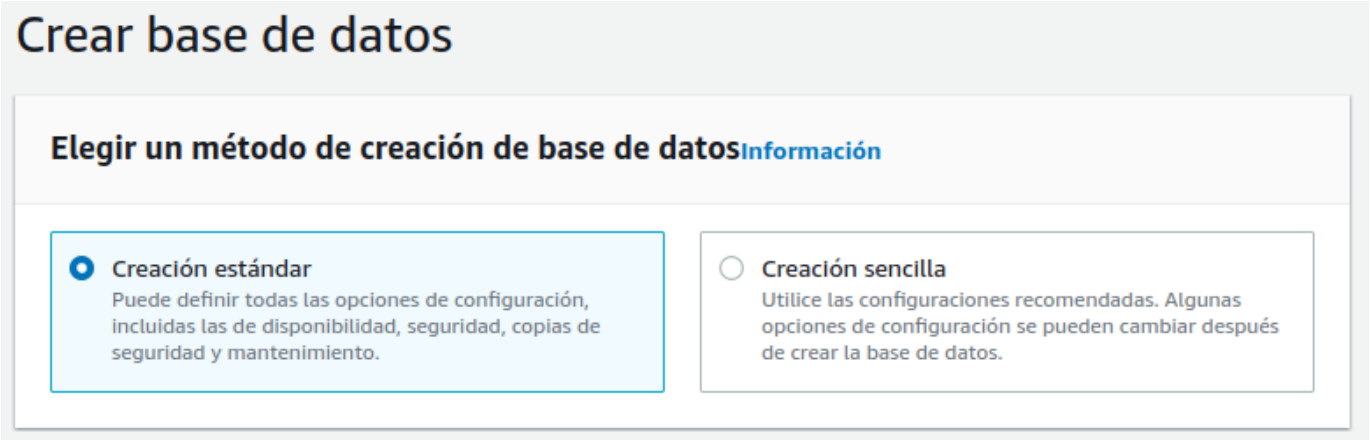
Una vez aquí debemos pulsar el botón `[Crear base de datos]`:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



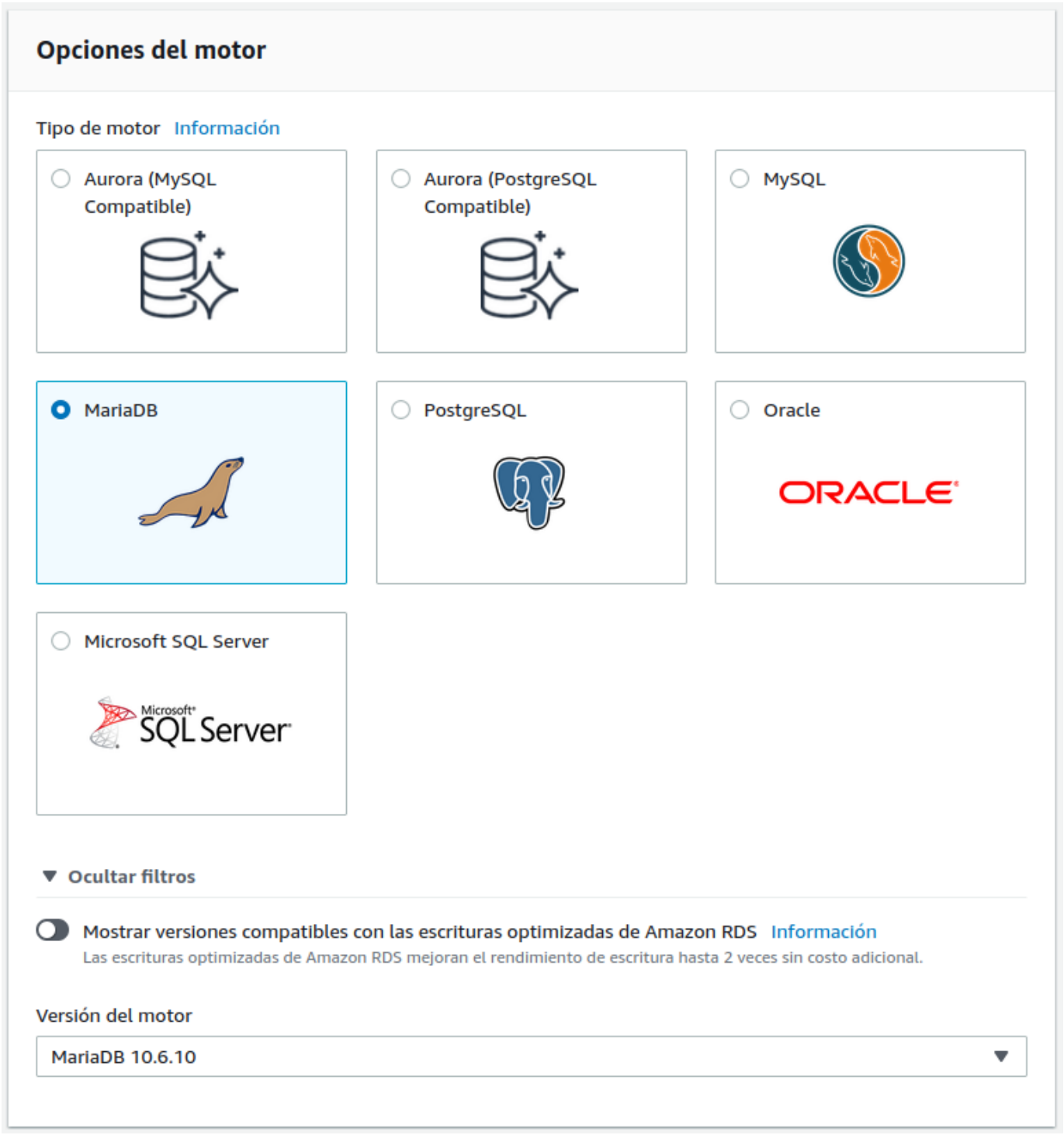
Ahora debemos elegir las opciones para nuestra base de datos:

Al `Crear base de datos` dejamos la `Creación estándar` por defecto:



En `Opciones del motor` elegimos `MariaDB` (Como versión puedes elegir la más moderna que aparezca):

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



En `Plantillas` elegimos la `Capa gratuita`:

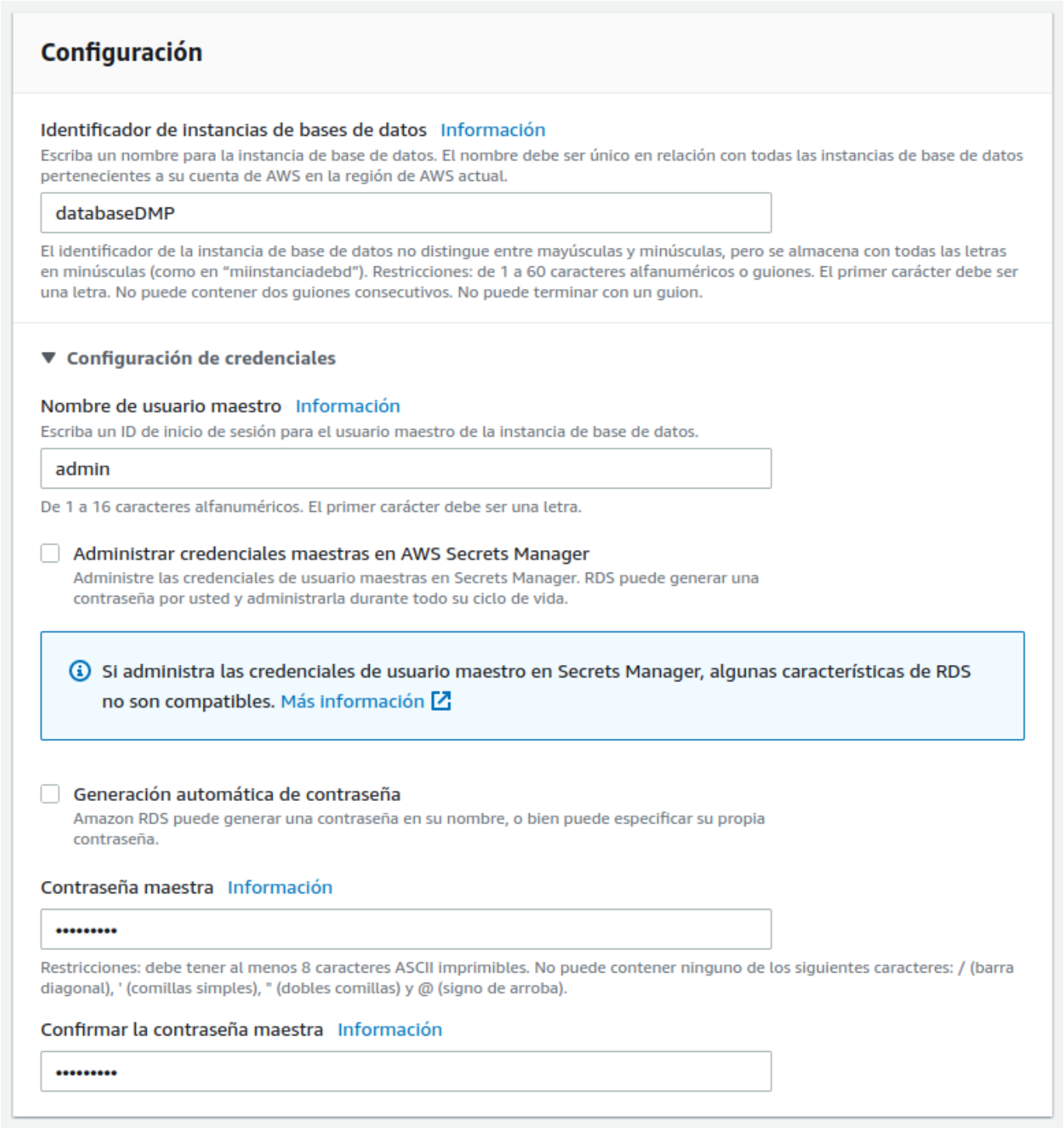
© 2025 David Martínez licensed under CC BY-NC-SA 4.0

A continuación, en `Configuración`:

Elegimos el `Identificador de instancias de bases de datos`, podemos poner el que más nos guste, en mi caso `databaseDMP`, también debemos indicar el `username`, en mi caso `admin`, y por último la `Contraseña maestra` y su confirmación, que deberá tener más de 8 carácteres ASCII imprimibles, en mi caso `123456Ab$`.

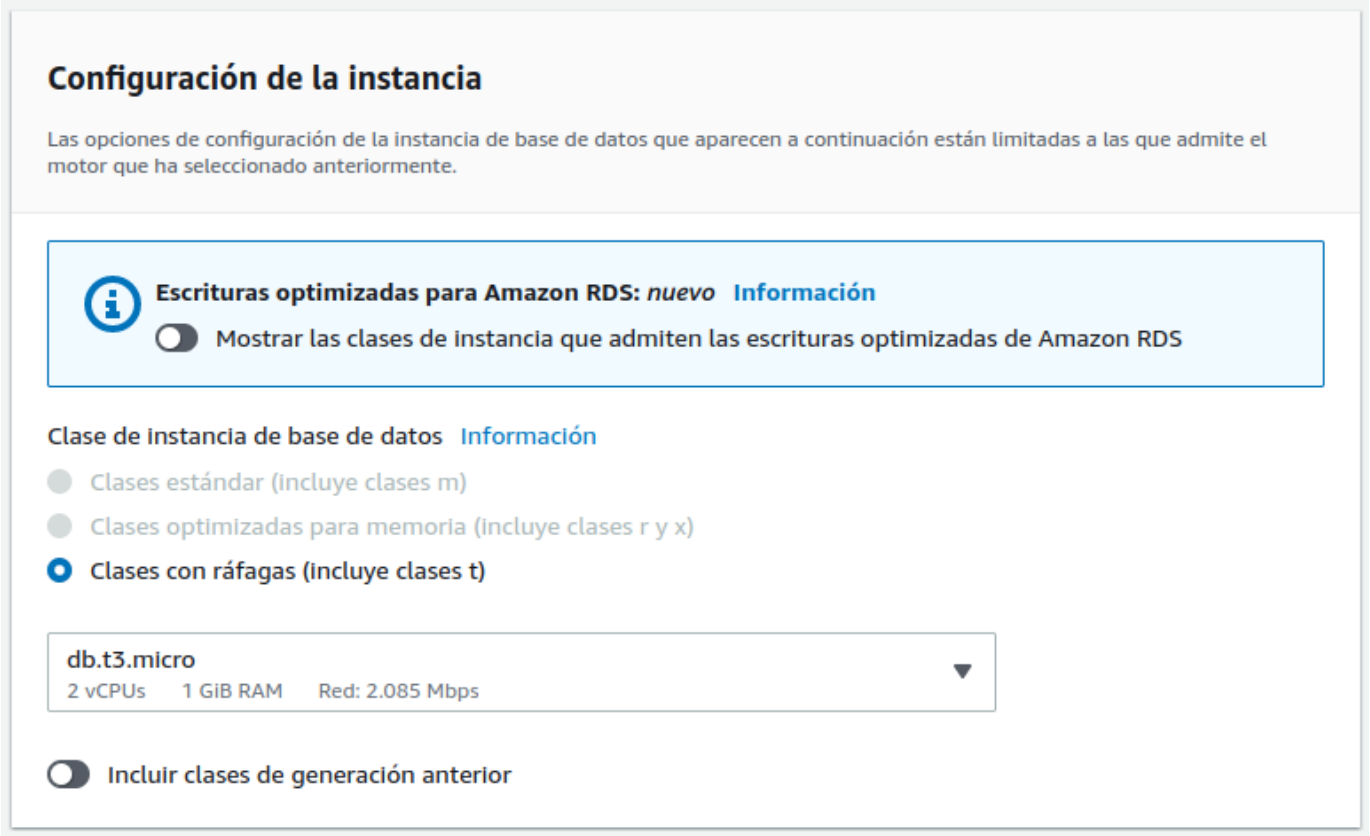
> **Atención**
>
> Anota todos estos datos en un lugar seguro, los necesitarás más adelante.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

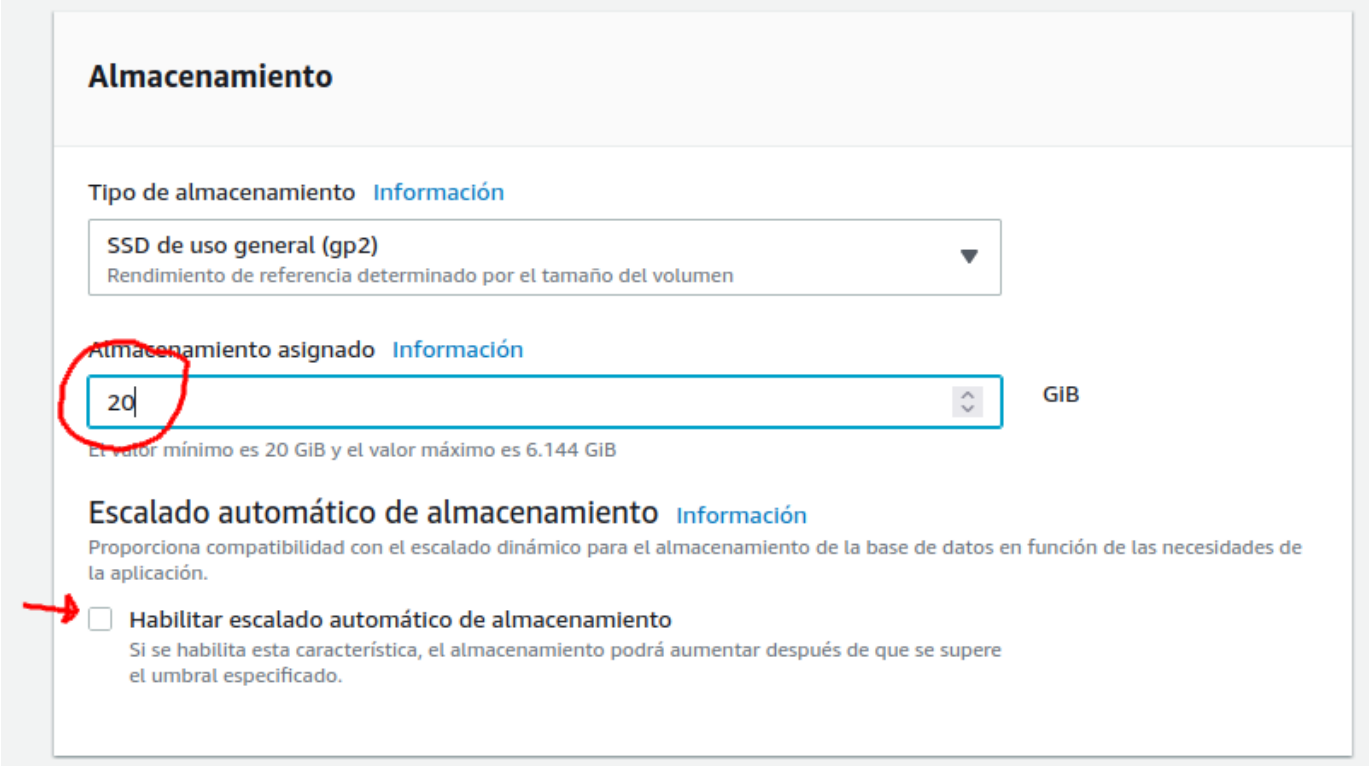


En la `Configuración de la instancia` dejamos todo por defecto (ahora db.t4g.micro):

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



En `Almacenamiento` dejamos el Almacenamiento asignado en 20GiB y deshabilitamos el escalado automático del almacenamiento:

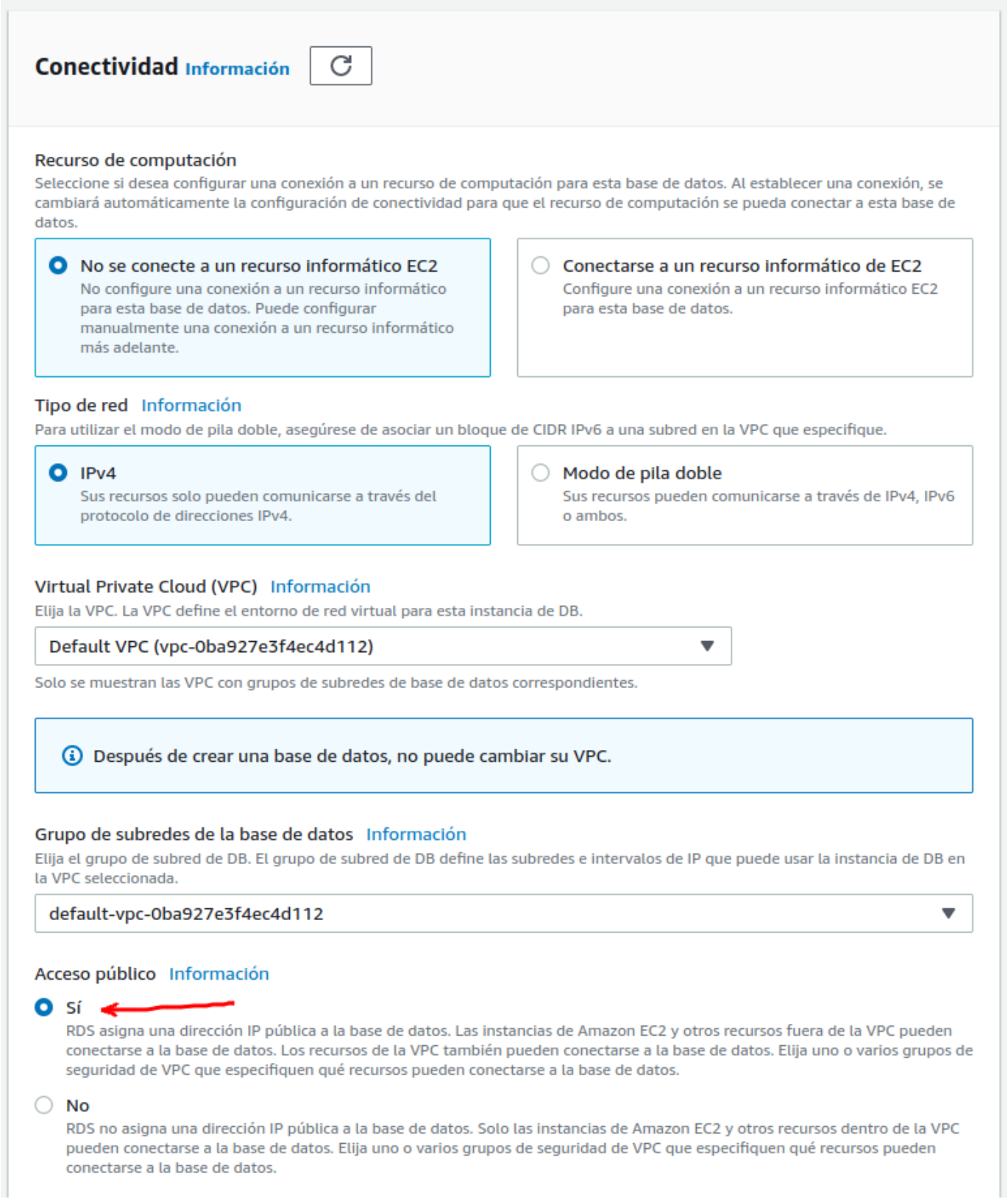


Dejamos `Disponibilidad y durabilidad` por defecto.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

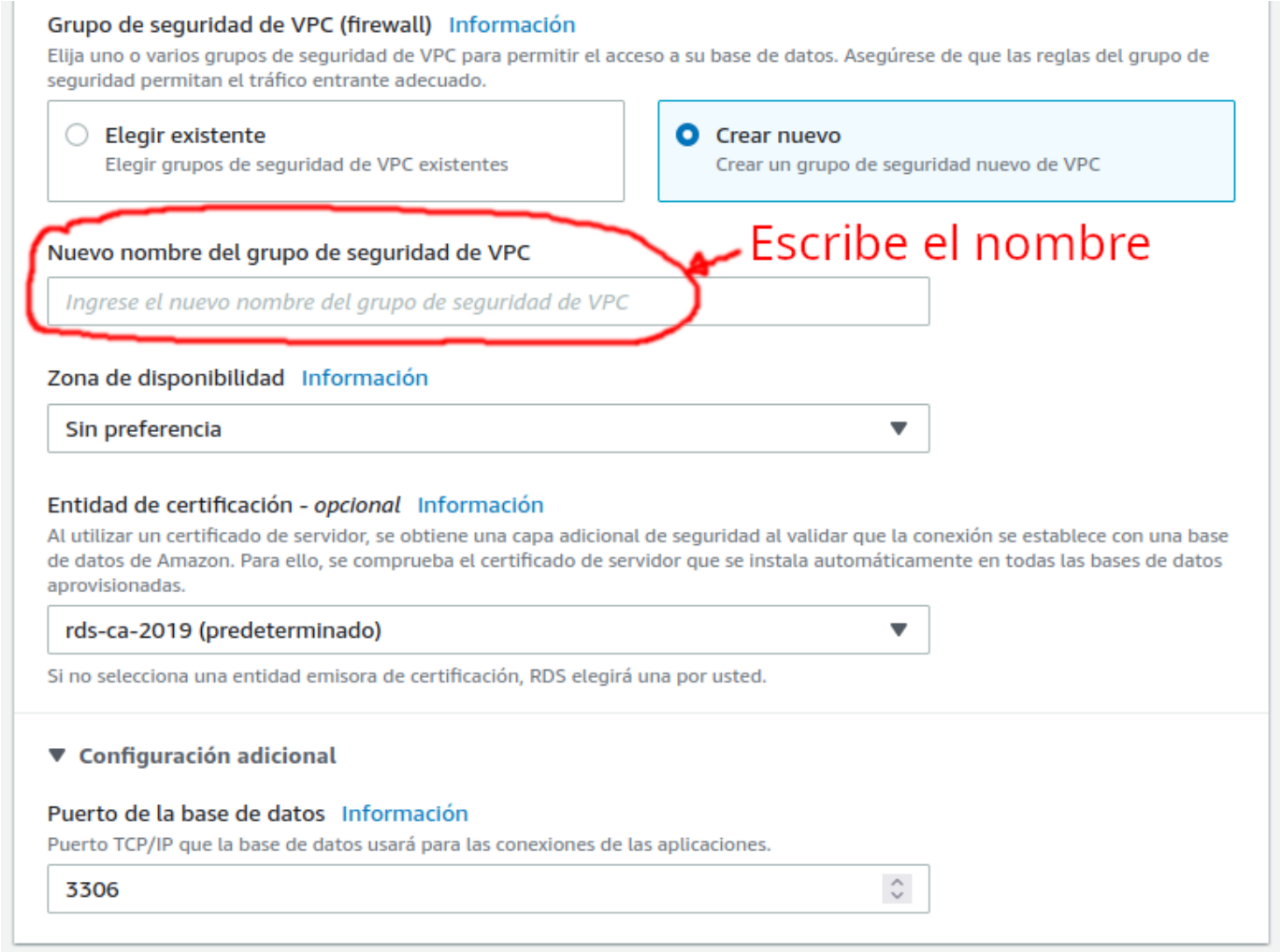
En el apartado `Conectividad` dejamos todo por defecto excepto la opción de `Acceso público` que debemos indicar que `Sí`:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



En la parte baja elegiremos crear un nuevo `Grupo de seguridad de VPC` y el puerto `3306`, el resto de opciones por defecto:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

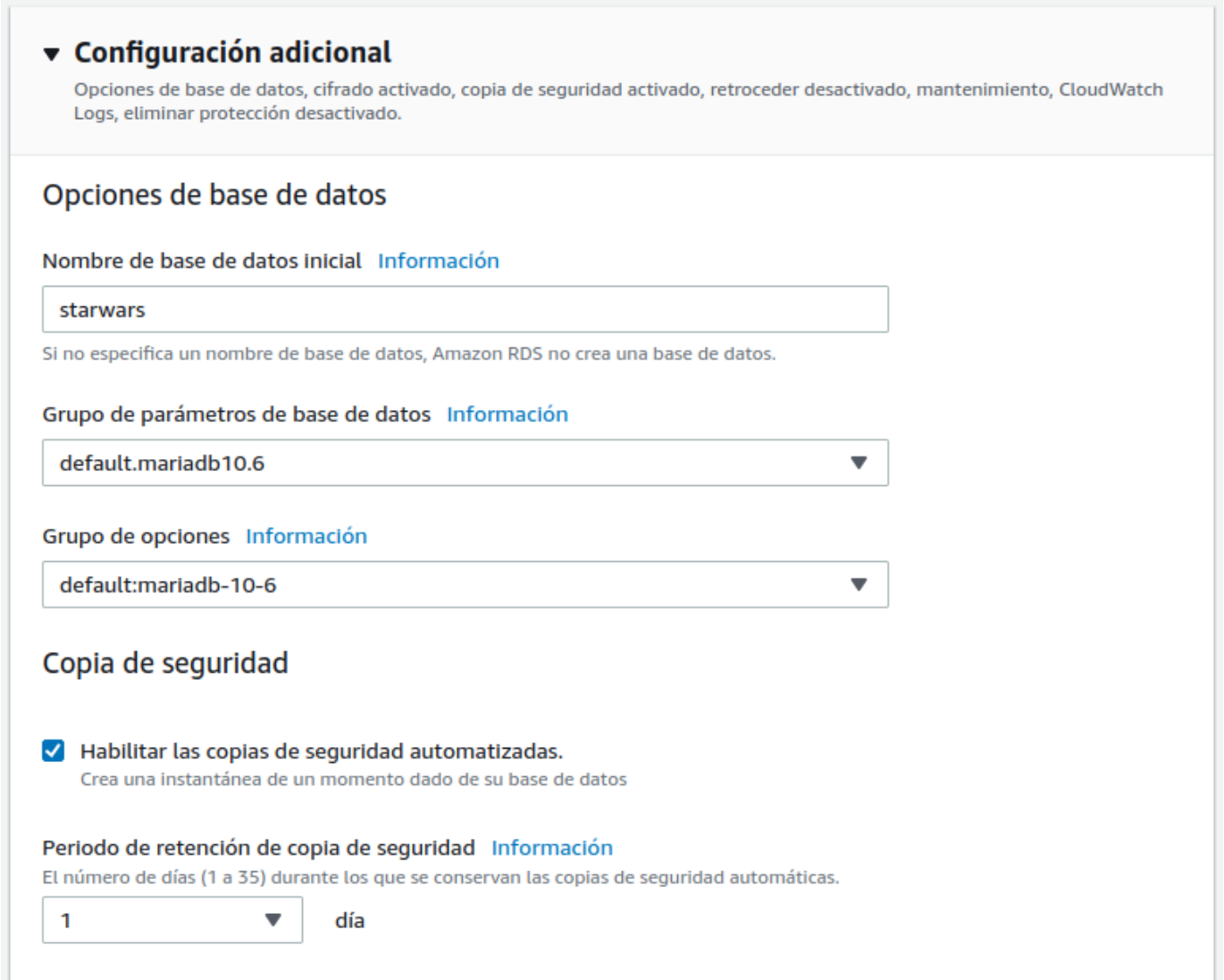


Ojo

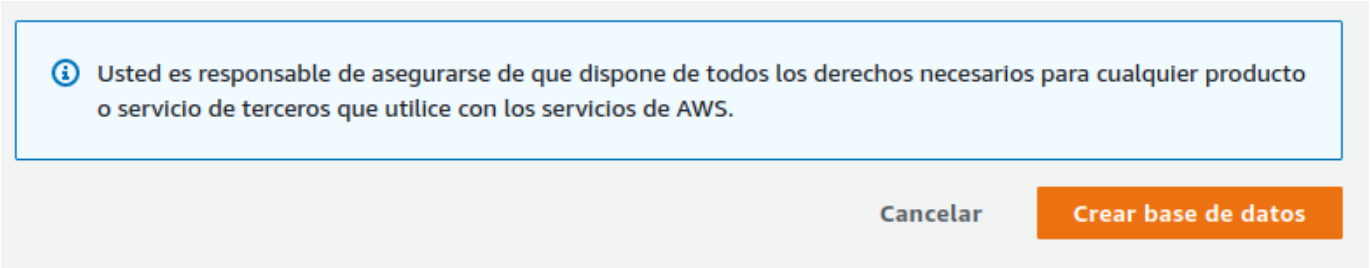
Aunque el puerto 3306 debería estar abierto el la red del instituto, he encontrado ocasiones en las que no ha funcionado correctamente, si te sucede esto, puedes cambiar al puerto 6000 o alguno de los que te sugiera tu profesor.

En el apartado `Configuración adicional` indicamos `starwars` como nombre de la base de datos inicial y cambiamos el periodo de retención de copias a `1 dia`:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

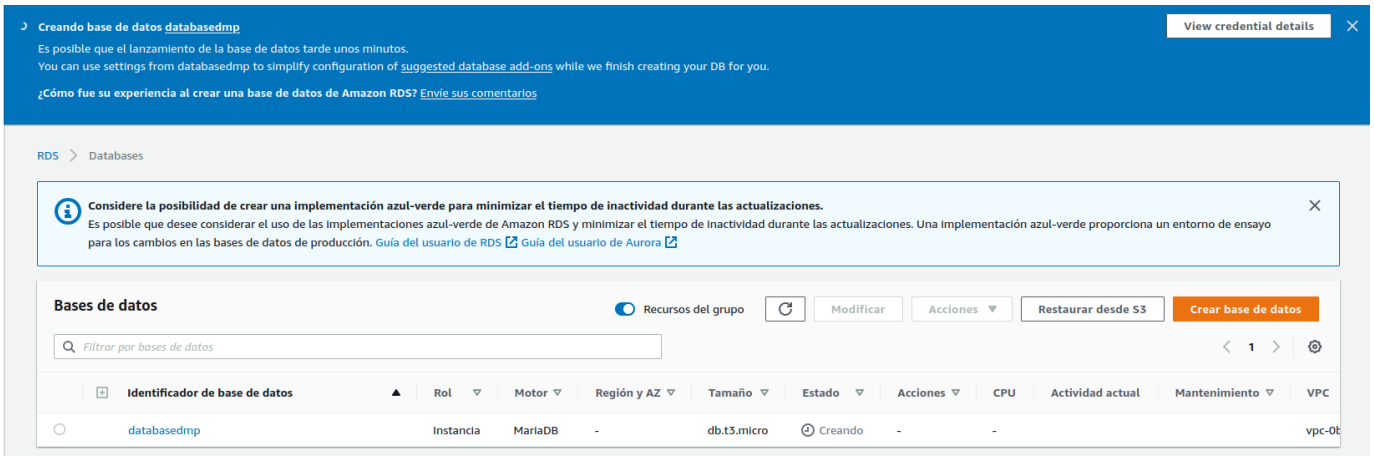


dejamos el resto de opciones por defecto y pulsaremos el botón del final `[Crear base de datos]`:

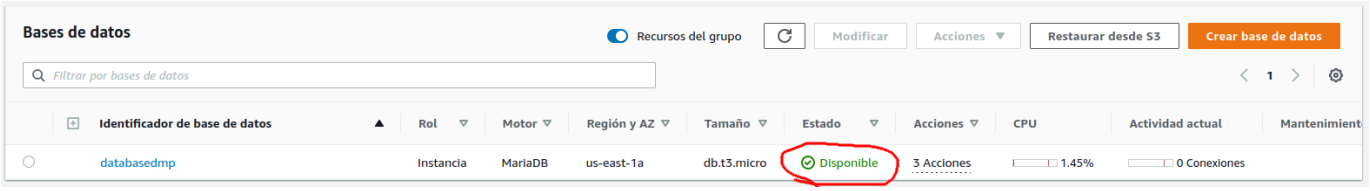


Si todo ha ido bien AWS nos indica que se está creando la BD y puede tardar varios minutos:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

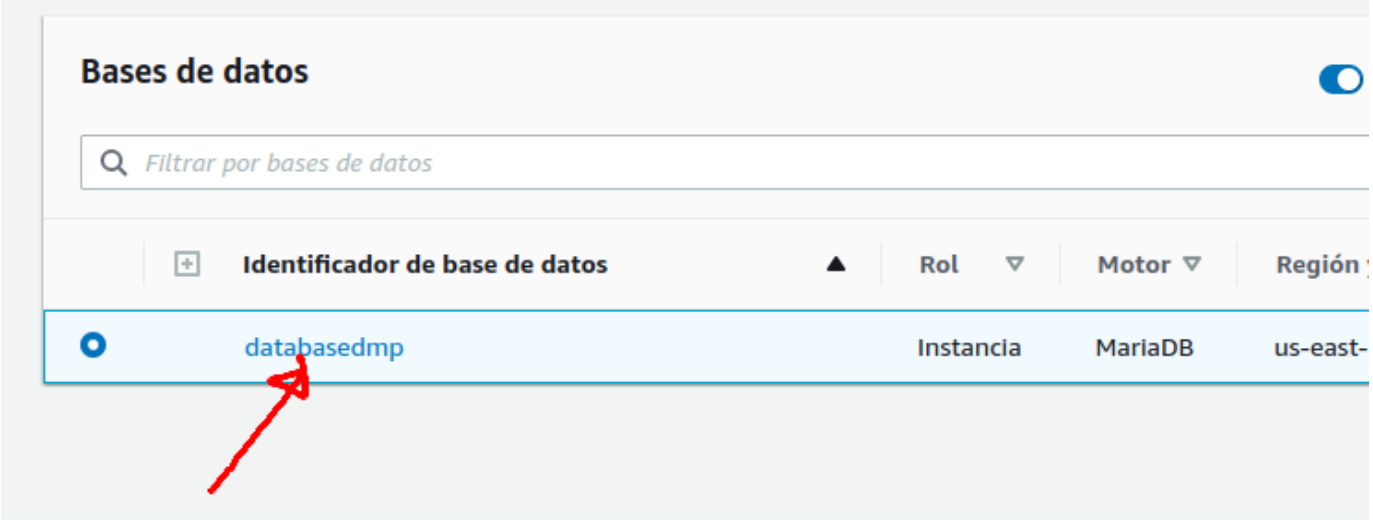


Una vez aparezca el estado en `Disponible`, podremos pasar al siguiente paso:



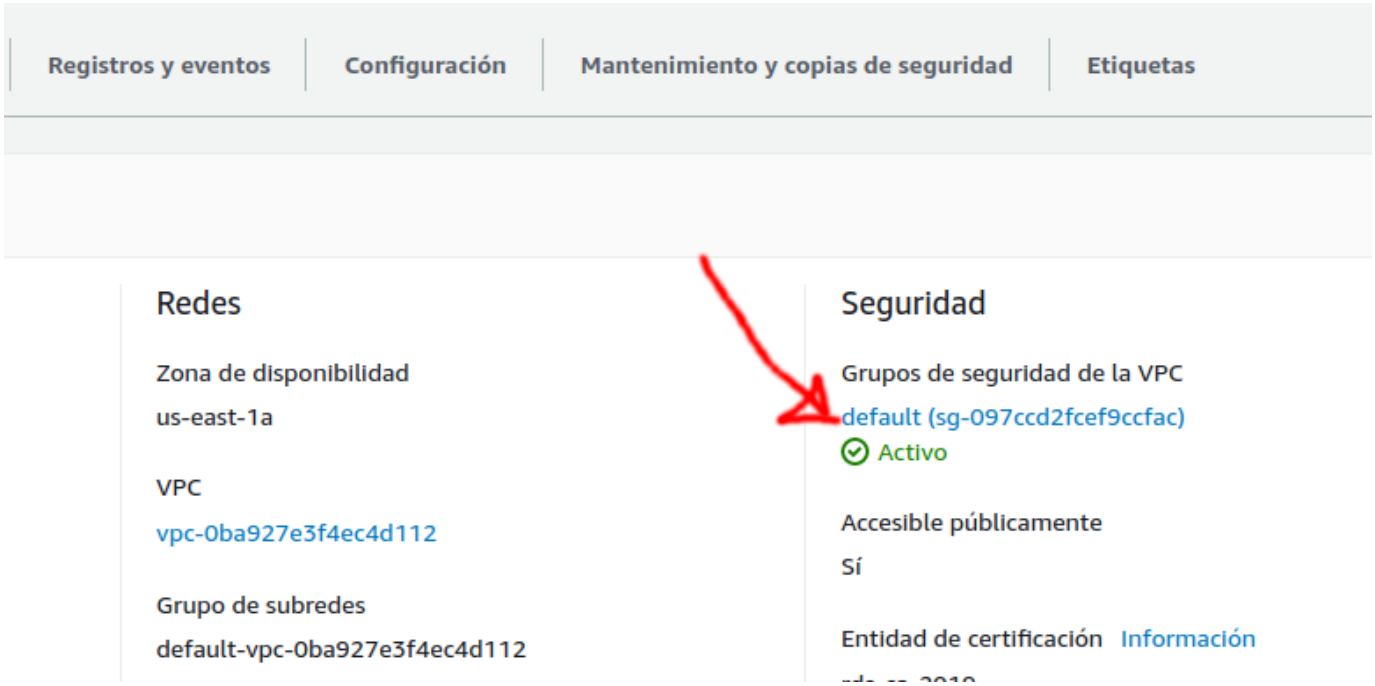
Abrir puertos para el acceso público

Ahora abriremos el puerto necesario para la que la BD sea accesible desde el exterior, para ello haremos clic sobre el nombre de la base de datos:

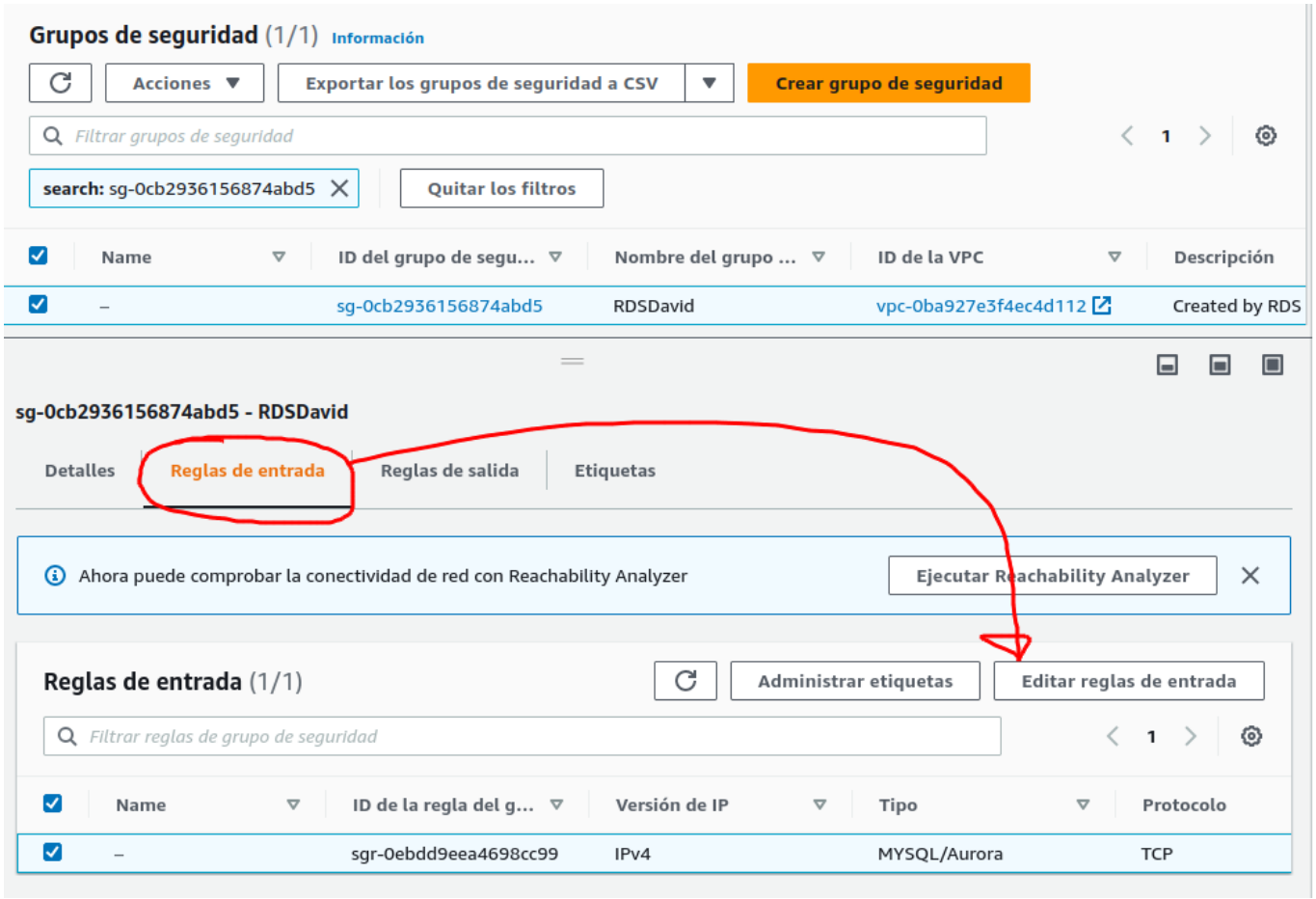


Y en la siguiente ventana hacemos click sobre el `Grupo de seguridad de la VPC` :

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

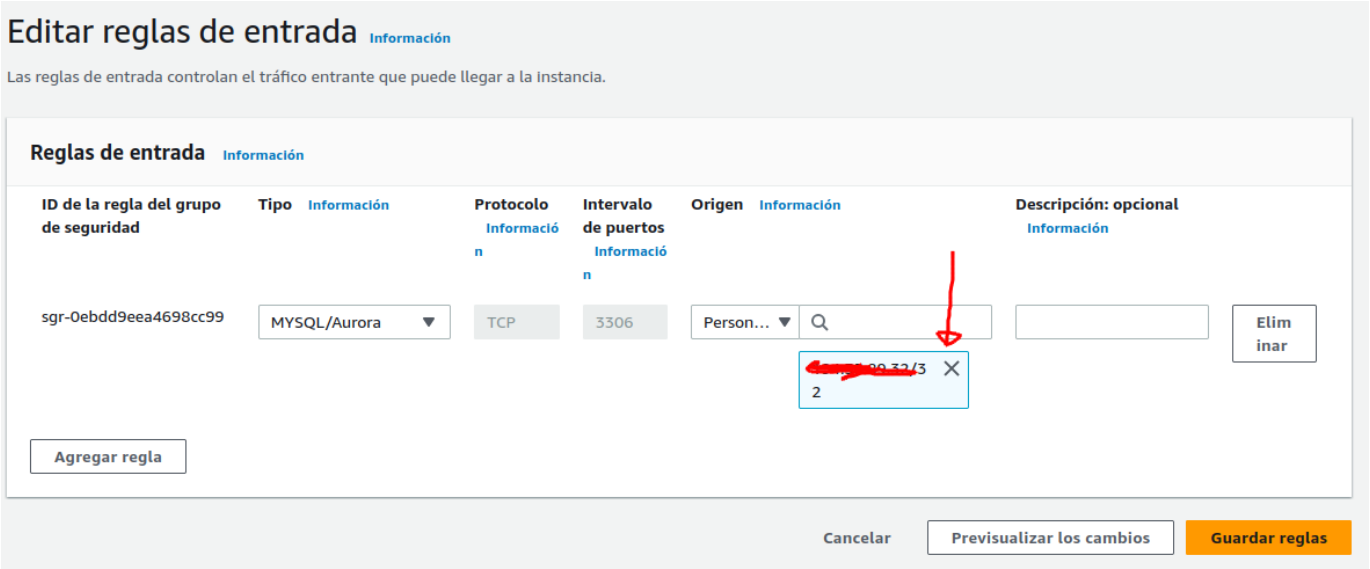


Una vez estamos en el apartado de Grupos de seguridad, debemos elegir la pestaña `Reglas de entrada`, y el botón `Editar reglas de entrada`:

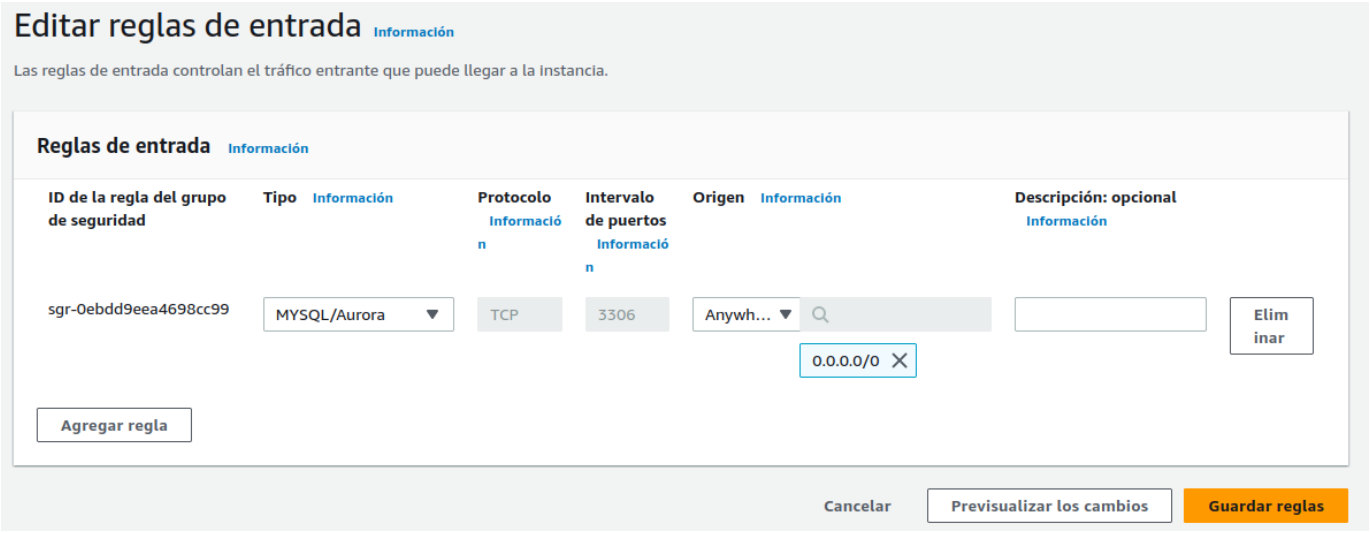


Si hemos seguido correctamente la guia debería aparecer el puerto 3306 abierto, pero solo para la ip pública desde la que estamos accediendo, así que deberemos pulsar la x para eliminar esa ip:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Y por último agrega la ip 0.0.0.0 para que sea acesible desde cualquier lugar:

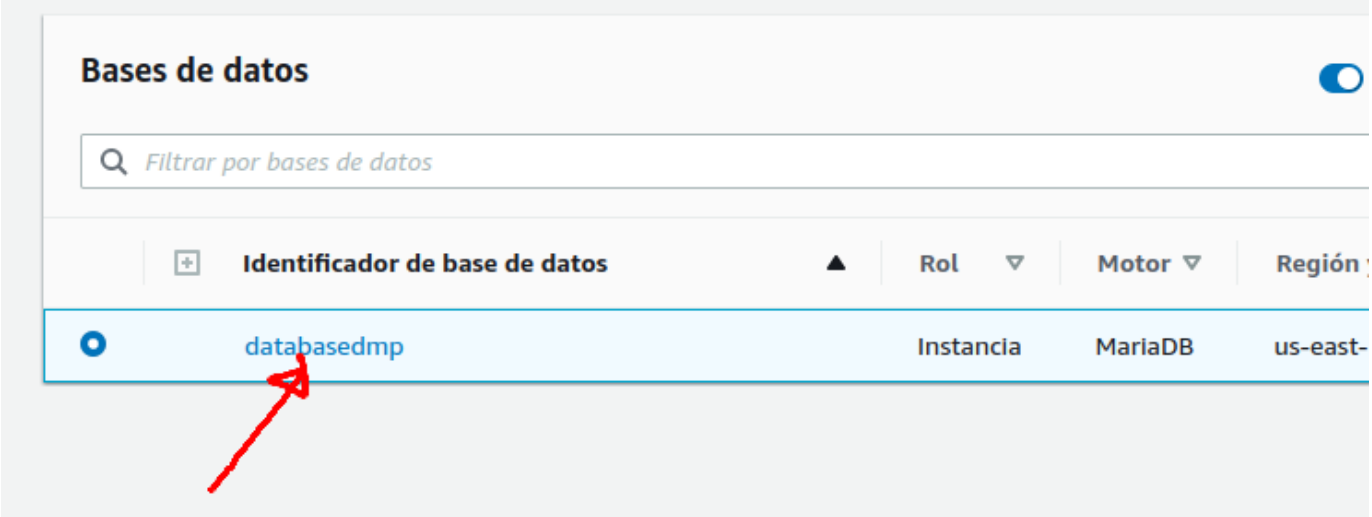


Dirección pública de la BBDD

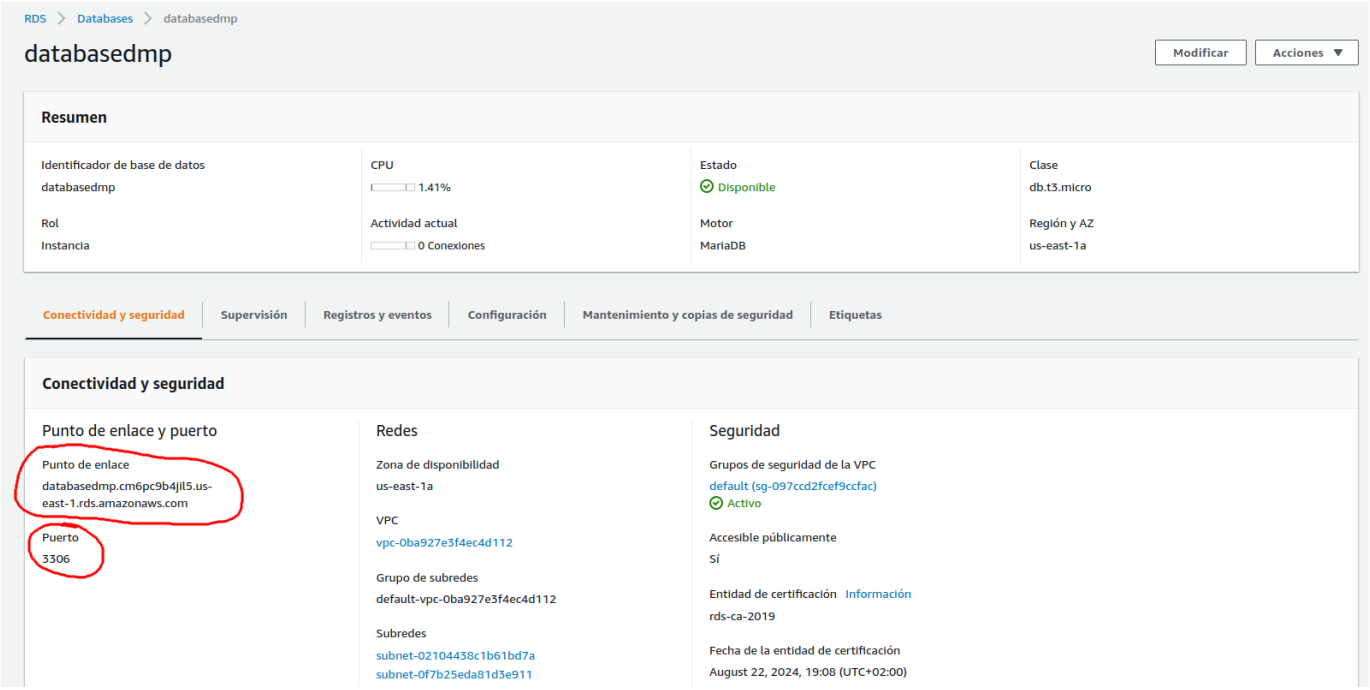
Ahora necesitamos la URL de acceso a la BD desde el exterior, primero volvemos al apartado RDS:

y a continuación haremos clic sobre el nombre de la base de datos:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Y en la siguiente ventana encontraremos los datos que necesitamos, por un lado el `Punto de enlace` (`databasedmp.cm6pc9b4jil5.us-east-1.rds.amazonaws.com`) y por otro el `Puerto` (`3306`):



Con toda esta información podemos realizar conexiones desde el exterior.

CARGA DE DATOS DESDE INTELLIJ

Instalar MySQL WorkBench

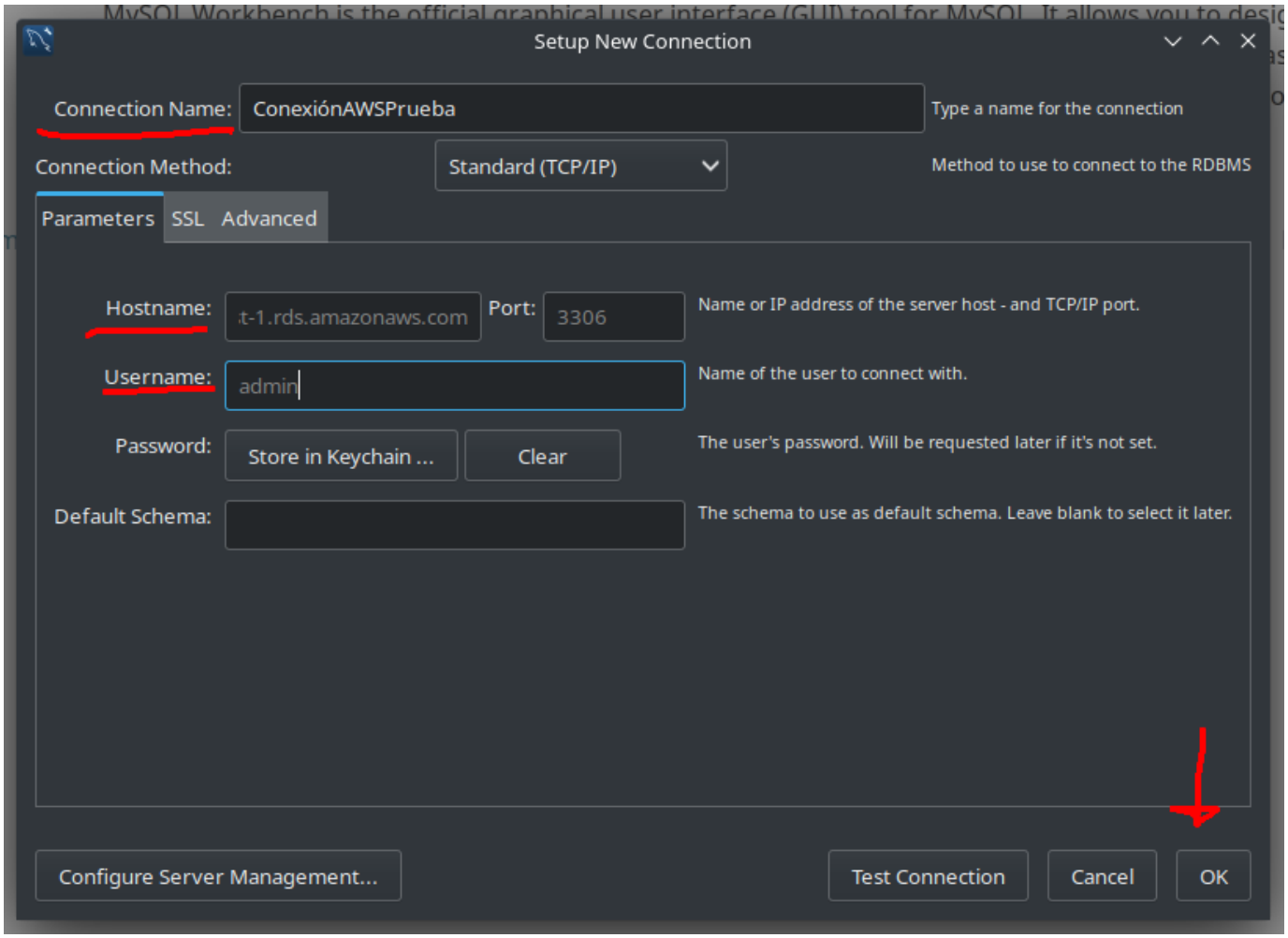
Dependiendo de nuestro sistema operativo deberemos usar un procedimiento u otro, lo mejor es consultar la página web y seguir sus instrucciones:

https://www.mysql.com/products/workbench/

Conexión de prueba

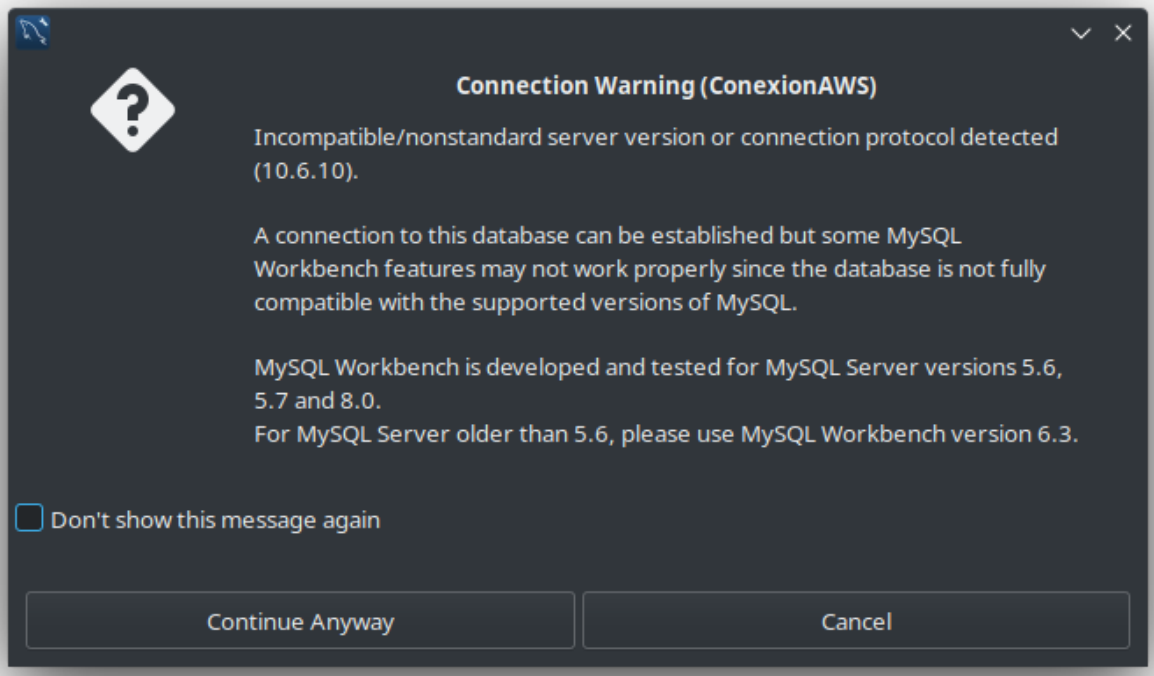
Una vez instalado nuestro cliente configuraremos una nueva conexión con los datos que hemos guardado en pasos anteriores, indicaremos el `Connection Name` (nombre de la conexión), `Hostname` (URL de la base de datos), y `username` (usuario de la base de datos). Al pulsar el botón `Test Connection`, nos permite probar la conexión y nos pedirá el `password`, y si pulsamos `OK` se guardará.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



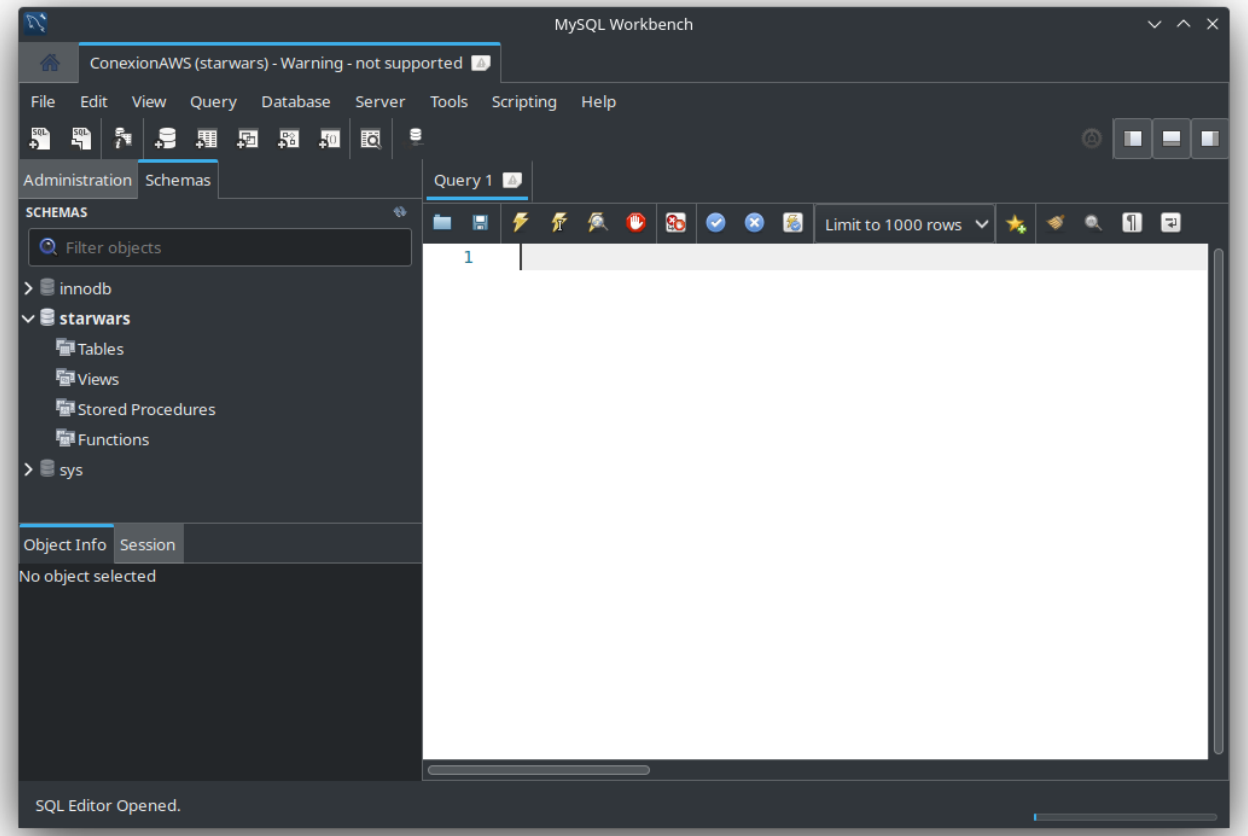
Es posible que nos indique que puede haber problemas de compatibilidad, pero podemos dar a `Continue anyway` sin preocuparnos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Y deberíamos ver nuestro entorno de modo similar a este:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

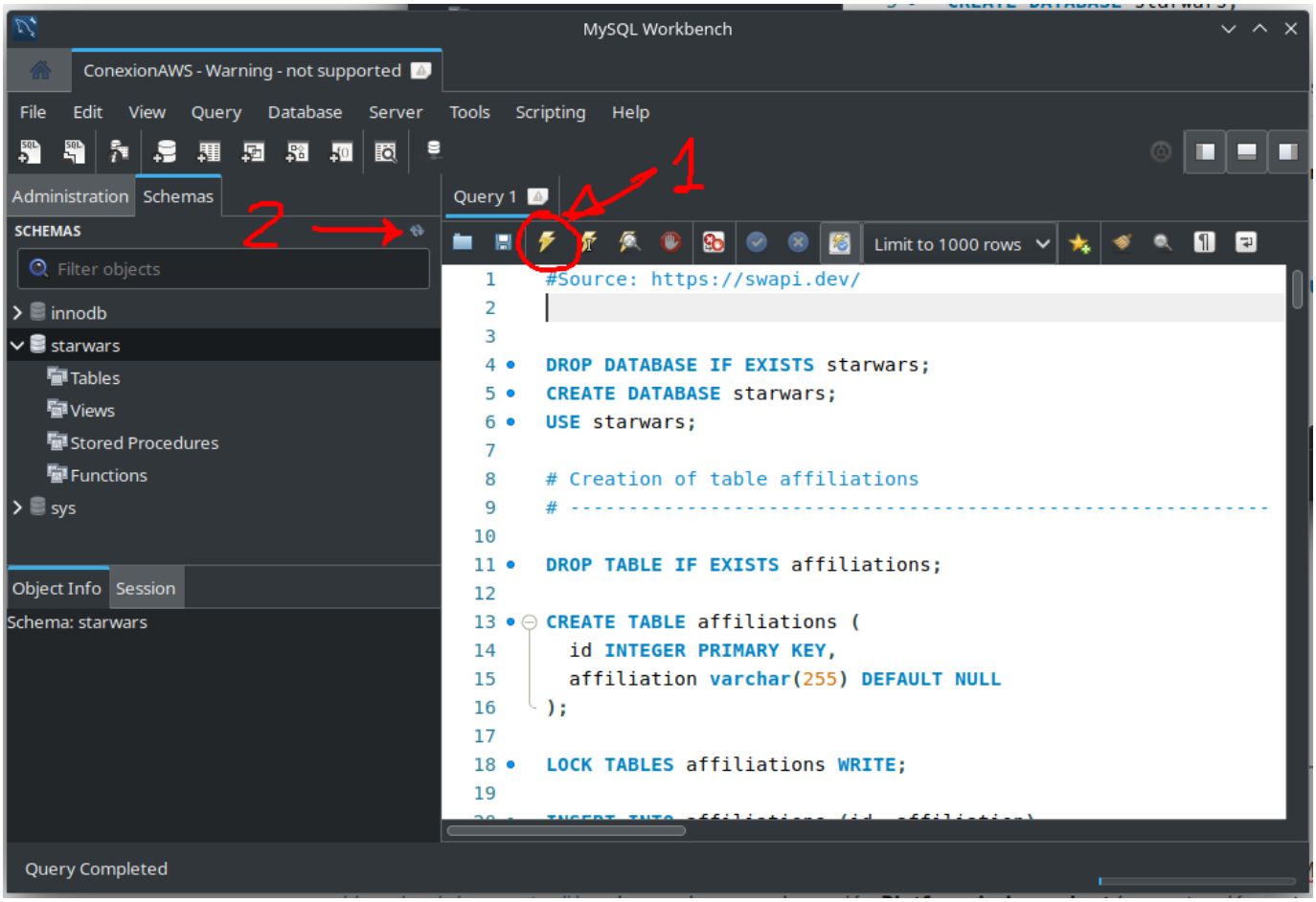


Importar datos de muestra

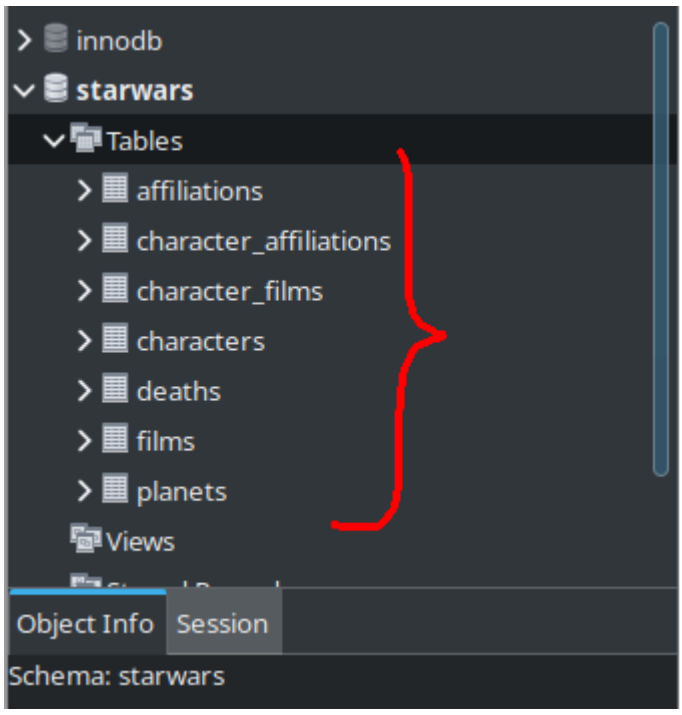
Para no trabajar con una base de datos vacía partiremos de una base de datos con información que podemos encontrar en el siguiente enlace: https://github.com/miguelmarti/FreakDataBases

En concreto usaremos la de StarWars.sql, la descargaremos de github, abriremos el archivo con nuestro editor preferido, copiaremos todo su contenido y lo pegaremos en la ventana blanca del entorno. A continuación pulsamos el rayo para ejecutar el script, y por último el botón para refrescar el contenido de las tablas:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Una vez hecho todo esto debería tener al menos 7 tablas:



Cuando comprobemos que tenemos la 7 tablas, podemos seguir.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

PREPARAR EL CLIENTE LOCAL

Descargar MySQL connector

Tal y como vimos en el taller UD10_T01_Conectores_ES, añade el conector a tu proyecto (mediante jar o maven, como prefieras.)

Código del cliente

En nuestro IDE preferido creamos un nuevo archivo `AWSTest.java` con el siguiente código:

```
package UD10.AWS;

import java.sql.*;
import java.util.Scanner;

/**
 *
 * @author David Martínez (wwww.martinezpenya.es|ieseduardoprimo.es)
 */
public class AWSTest {

    private static final String AWSDNS = "databasedmp.cm6pc9b4jil5.us-east-1.rds.amazonaws.com";
    private static final String DBNAME = "starwars";
    private static final int PUERTO = 3306;
    private static final String USERNAME = "admin";
    private static final String PASSWORD = "123456Ab$";

    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        try (Connection con = DriverManager.getConnection("jdbc:mysql://" + AWSDNS + ":" + PUERTO + "/" + DBNAME, USERNAME, PASSWORD);) {
            Statement stmt = con.createStatement();
            //Query sin parámetros
            ResultSet rs = stmt.executeQuery("select * from films");
            System.out.println("Estas son las películas de Star Wars");
            while (rs.next()) {
                System.out.println(rs.getInt(1) + "  " + rs.getString(2) + "  " + rs.getString(3));
            }
            System.out.print("Elige un número de película y te mostraré los personajes que aparecen en ella: ");
            String numPelicula = sc.nextLine();
            PreparedStatement pStmt = con.prepareStatement("SELECT C.id, name, C.height, C.mass, C.hair_color, C.skin_color "
                    + "FROM starwars.characters C, starwars.character_films CF, starwars.films F "
                    + "WHERE F.id=CF.id_film AND CF.id_character=C.id "
                    + "AND CF.id_film = ?;");
            pStmt.setString(1, numPelicula);
            //Query con parámetros
            ResultSet rs2 = pStmt.executeQuery();
            while (rs2.next()) {
                System.out.println(rs2.getInt(1) + "  " + rs2.getString(2) + "  " + rs2.getInt(3) + "  " + rs2.getFloat(4) + "  " + rs2.getString(5) +
"  " + rs2.getString(6));
            }
            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

> **Recuerda**
>
> Recuerda cambiar la constante `AWSDNS`, `DBNAME`, `PUERTO`, `USERNAME` y `PASSWORD` por el `String` o `int` que corresponda con tu configuración.

EJECUCIÓN DE PRUEBA

Desde el punto de vista del cliente

Una vez ejecutado el cliente debe aparecer algo similar a esto:

```
Estas son las películas de Star Wars
1  Episode I  The Phantom Menace
2  Episode II  Attack of the Clones
3  Episode III  Revenge of the Sith
4  Episode IV  A New Hope
5  Episode V  The Empire Strikes Back
6  Episode VI  Return of the Jedi
Elige un número de película y te mostraré los personajes que aparecen en ella:
```

Y si a continuación elegimos por ejemplo la película 4:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
Elige un número de película y te mostraré los personajes que aparecen en ella: 4
1  Luke Skywalker  172  77.0  blond  fair
2  C-3PO  167  75.0  n/a  gold
3  R2-D2  96  32.0  n/a  white, blue
4  Darth Vader  202  136.0  none  white
5  Leia Organa  150  49.0  brown  light
6  Owen Lars  178  120.0  brown, grey  light
7  Whitesun lars  165  75.0  brown  light
8  R5-D4  97  32.0  n/a  white, red
9  Biggs Darklighter  183  84.0  black  light
10  Obi-Wan Kenobi  182  77.0  auburn, white  fair
12  Wilhuff Tarkin  180  0.0  auburn, grey  fair
13  Chewbacca  228  112.0  brown  null
14  Han Solo  180  80.0  brown  fair
15  Greedo  173  74.0  n/a  green
18  Wedge Antilles  170  77.0  brown  fair
19  Jek Tono Porkins  180  110.0  brown  fair
81  Raymus Antilles  188  79.0  brown  light
89  Cornelius Evazan  0  0.0  null  null
90  Ponda Baba  0  0.0  null  null
```

Warning SLF4J

Si te aparece el mensaje siguiente:

```
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
```

Aunque es un **warning** y no debe afectar al funcionamiento de tus programas, resolver el error es tan sencillo como añadir una dependencia más:

```
<!-- https://mvnrepository.com/artifact/org.slf4j/slf4j-api -->
<dependency>
 <groupId>org.slf4j</groupId>
 <artifactId>slf4j-simple</artifactId>
 <version>2.0.17</version>
</dependency>
```

Tarea

Ahora que has visto como crear una BBDD en AWS:

1. Genera una nueva BBDD con el motor que prefieras (MySQL, MariaDB, Oracle, etc...)
2. Carga una Base de Datos que puedas conseguir por tu cuenta, que crees desde cero, o a partir de las que te proporciona el profesor. La condición es que debe tener al menos 3 tablas, al menos dos de ellas deben estar relacionadas, y debes tener al menos 10 registros (información) en cada una de las tablas.

Comenta con el profesor (por correo electrónico) la BBDD que usaras para evitar que dos alumnos usen la misma (en caso que uses una que encuentres por Internet). 3. Esta nueva Base de Datos será la que usaras para completar la siguiente tarea, así que quizá deberías echarle un ojo antes de tomar la decisión.

Envía una memoria en fichero **pdf** explicando los pasos seguidos (con capturas) a la tarea de **Aules** correspondiente.

6 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

11.3.3 Taller UD10_04: Patron DAO (CRUD completo)

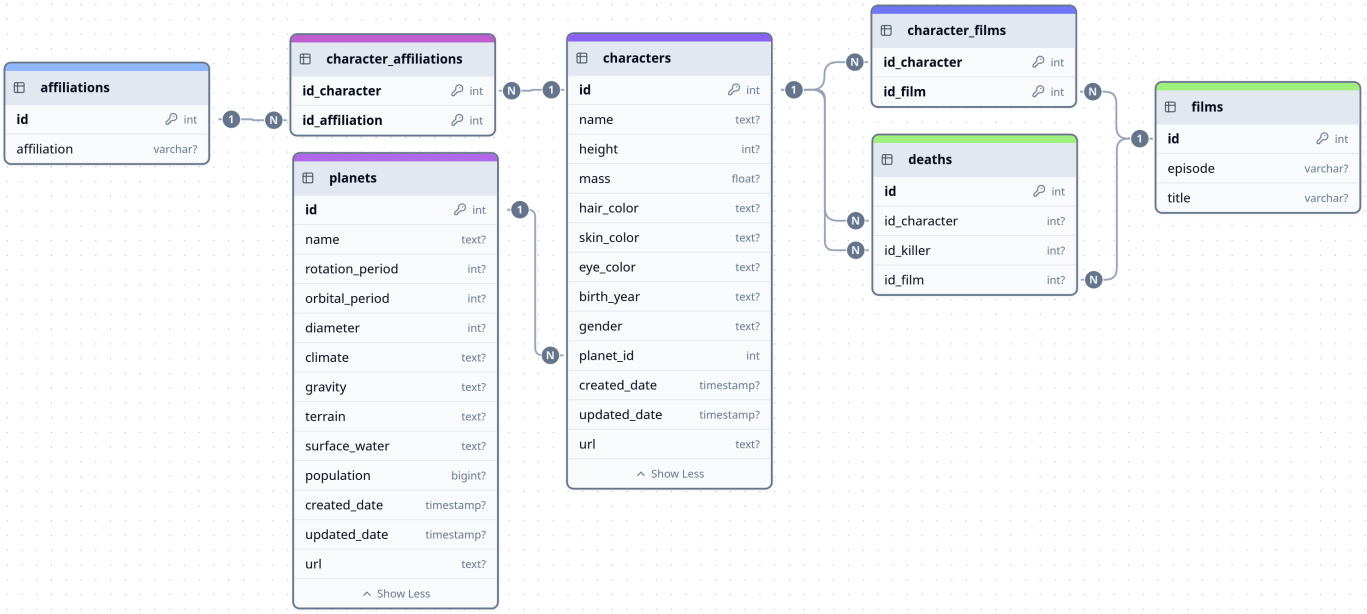
Introducción

En el apartado correspondiente de teoria, ya estudiamos los patrones `Singleton` y `DAO`.

> **Importante**
>
> Para realizar el ejemplo que seguiremos en este apartado usaremos la BBDD starwars. Pero tu deberás replicar este ejemplo para la BBDD que creaste en el anterior taller UD10_T02_AWS_IntelliJ_ES.

En programación existen una serie de estándares denominados Patrones de Diseño que debes conocer para poder programar según estos patrones y no reinventar la rueda.

Esquema de la BBDD



Implementación paso a paso

Paso 0: Definir la clase `DbConnect` (`Singleton`), usando HikariCP

Para simplificar la implementar y conexión a la base de datos podemos crear la clase `DbConnect` (como ya hemos implementado anteriormente en algunos ejercicios):

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package es.martinezpenya.ejemplos.UD10.starwars;

import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
import java.sql.Connection;
import java.sql.SQLException;

public class DBConnect {

    private static HikariDataSource dataSource;

    // Configuración del pool de conexiones
    static {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl("jdbc:mariadb://databasedmp.cipxbdkxiaqy.us-east-1.rds.amazonaws.com:6000/starwars"); // URL de la base de datos
        config.setUsername("admin"); // Usuario de la base de datos
        config.setPassword("123456Ab$"); // Contraseña de la base de datos
        config.setMaximumPoolSize(20); // Número máximo de conexiones en el pool
        config.setMinimumIdle(2); // Número mínimo de conexiones inactivas en el pool
        config.setIdleTimeout(3000); // Tiempo de espera para conexiones inactivas (en milisegundos)
        config.setMaxLifetime(180000); // Tiempo máximo de vida de una conexión (en milisegundos)
        config.setConnectionTimeout(300000); // Tiempo de espera para obtener una conexión (en milisegundos)

        dataSource = new HikariDataSource(config);
    }

    // Método para obtener una conexión del pool
    public static Connection getConnection() throws SQLException {
        return dataSource.getConnection();
    }

    // Método para cerrar el pool de conexiones (opcional)
    public static void closeDataSource() {
        if (dataSource != null) {
            dataSource.close();
        }
    }
}
```

Paso 1: Definir las clases de las entidades

Definiremos las clases `Film`, `Planet`, `Character` y `Characterfilm`, pero tu deberás hacerlo con todas tus tablas (o al menos 3 de ellas, y 2 deben estar relacionadaS), deberás incluir todos los campos de las tablas, los constructores que estimes oportuno, los getters y setters y un metodo `toString()`:

Film

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package es.martinezpenya.ejemplos.UD10.starwars;

public class Film {
    private int id;
    private String episode;
    private String title;

    public Film(int id, String episode, String title) {
        this.id = id;
        this.episode = episode;
        this.title = title;
    }

    public Film(int id, String title) {
        this.id = id;
        this.title = title;
    }

    // Getters y setters
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getEpisode() {
        return episode;
    }

    public void setEpisode(String episode) {
        this.episode = episode;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    @Override
    public String toString() {
        return "Film{" +
                "id=" + id +
                ", episode='" + episode + '\'' +
                ", title='" + title + '\'' +
                '}';
    }
}
```

Planet

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package es.martinezpenya.ejemplos.UD10.starwars;

public class Planet {
    private int id;
    private String name;
    private String rotation_period;
    private String orbital_period;
    private String diameter;
    private String climate;
    private String gravity;
    private String terrain;
    private String surface_water;
    private String population;
    private String created_date;
    private String updated_date;
    private String url;

    public Planet(int id, String name, String rotation_period, String orbital_period, String diameter, String climate, String gravity, String terrain, S
tring surface_water, String population) {
        this.id = id;
        this.name = name;
        this.rotation_period = rotation_period;
        this.orbital_period = orbital_period;
        this.diameter = diameter;
        this.climate = climate;
        this.gravity = gravity;
        this.terrain = terrain;
        this.surface_water = surface_water;
        this.population = population;
    }

    public Planet(int id, String name) {
        this.id = id;
        this.name = name;
    }

    // Getters y setters
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getRotation_period() {
        return rotation_period;
    }

    public void setRotation_period(String rotation_period) {
        this.rotation_period = rotation_period;
    }

    public String getOrbital_period() {
        return orbital_period;
    }

    public void setOrbital_period(String orbital_period) {
        this.orbital_period = orbital_period;
    }

    public String getDiameter() {
        return diameter;
    }

    public void setDiameter(String diameter) {
        this.diameter = diameter;
    }

    public String getClimate() {
        return climate;
    }
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
    public void setClimate(String climate) {
        this.climate = climate;
    }

    public String getGravity() {
        return gravity;
    }

    public void setGravity(String gravity) {
        this.gravity = gravity;
    }

    public String getTerrain() {
        return terrain;
    }

    public void setTerrain(String terrain) {
        this.terrain = terrain;
    }

    public String getSurface_water() {
        return surface_water;
    }

    public void setSurface_water(String surface_water) {
        this.surface_water = surface_water;
    }

    public String getPopulation() {
        return population;
    }

    public void setPopulation(String population) {
        this.population = population;
    }

    public String getCreated_date() {
        return created_date;
    }

    public void setCreated_date(String created_date) {
        this.created_date = created_date;
    }

    public String getUpdated_date() {
        return updated_date;
    }

    public void setUpdated_date(String updated_date) {
        this.updated_date = updated_date;
    }

    public String getUrl() {
        return url;
    }

    public void setUrl(String url) {
        this.url = url;
    }

    @Override
    public String toString() {
        return "Planet{" +
                "id=" + id +
                ", name='" + name + '\'' +
                ", rotation_period='" + rotation_period + '\'' +
                ", orbital_period='" + orbital_period + '\'' +
                ", diameter='" + diameter + '\'' +
                ", climate='" + climate + '\'' +
                ", gravity='" + gravity + '\'' +
                ", terrain='" + terrain + '\'' +
                ", surface_water='" + surface_water + '\'' +
                ", population='" + population + '\'' +
                ", created_date='" + created_date + '\'' +
                ", updated_date='" + updated_date + '\'' +
                ", url='" + url + '\'' +
                '}';
    }
}
```

Character

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package es.martinezpenya.ejemplos.UD10.starwars;

import java.time.LocalDateTime;

public class Character {
    private int id;
    private String name;
    private int height;
    private double mass;
    private String hair_color;
    private String skin_color;
    private String eye_color;
    private String birth_year;
    private String gender;
    private Planet planet_id;
    private LocalDateTime created_date;
    private LocalDateTime updated_date;
    private String url;

    public Character(int id, String name, int height, double mass, String hair_color, String skin_color, String eye_color, String birth_year, String gen
der, Planet planet_id) {
        this.id = id;
        this.name = name;
        this.height = height;
        this.mass = mass;
        this.hair_color = hair_color;
        this.skin_color = skin_color;
        this.eye_color = eye_color;
        this.birth_year = birth_year;
        this.gender = gender;
        this.planet_id = planet_id;
    }

    public Character(int id, String name) {
        this.id = id;
        this.name = name;
    }

    // Getters y setters
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getHeight() {
        return height;
    }

    public void setHeight(int height) {
        this.height = height;
    }

    public double getMass() {
        return mass;
    }

    public void setMass(double mass) {
        this.mass = mass;
    }

    public String getHair_color() {
        return hair_color;
    }

    public void setHair_color(String hair_color) {
        this.hair_color = hair_color;
    }

    public String getSkin_color() {
        return skin_color;
    }
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
    public void setSkin_color(String skin_color) {
        this.skin_color = skin_color;
    }

    public String getEye_color() {
        return eye_color;
    }

    public void setEye_color(String eye_color) {
        this.eye_color = eye_color;
    }

    public String getBirth_year() {
        return birth_year;
    }

    public void setBirth_year(String birth_year) {
        this.birth_year = birth_year;
    }

    public String getGender() {
        return gender;
    }

    public void setGender(String gender) {
        this.gender = gender;
    }

    public Planet getPlanet_id() {
        return planet_id;
    }

    public void setPlanet_id(Planet planet_id) {
        this.planet_id = planet_id;
    }

    public LocalDateTime getCreated_date() {
        return created_date;
    }

    public void setCreated_date(LocalDateTime created_date) {
        this.created_date = created_date;
    }

    public LocalDateTime getUpdated_date() {
        return updated_date;
    }

    public void setUpdated_date(LocalDateTime updated_date) {
        this.updated_date = updated_date;
    }

    public String getUrl() {
        return url;
    }

    public void setUrl(String url) {
        this.url = url;
    }

    @Override
    public String toString() {
        return "Character{" +
                "id=" + id +
                ", name='" + name + '\'' +
                ", height=" + height +
                ", mass=" + mass +
                ", hair_color='" + hair_color + '\'' +
                ", skin_color='" + skin_color + '\'' +
                ", eye_color='" + eye_color + '\'' +
                ", birth_year='" + birth_year + '\'' +
                ", gender='" + gender + '\'' +
                ", planet='" + planet_id + '\'' +
                ", created_date=" + created_date +
                ", updated_date=" + updated_date +
                ", url='" + url + '\'' +
                '}';

    }
}
```

CharacterFilm

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package es.martinezpenya.ejemplos.UD10.starwars;

public class CharacterFilm {
    private Character character;
    private Film film;

    public CharacterFilm(Character character, Film film) {
        this.character = character;
        this.film = film;
    }

    public Character getCharacter() {
        return character;
    }

    public void setCharacter(Character character) {
        this.character = character;
    }

    public Film getFilm() {
        return film;
    }

    public void setFilm(Film film) {
        this.film = film;
    }

    @Override
    public String toString() {
        return "CharacterFilm{" +
                "character=" + character +
                ", film=" + film +
                '}';
    }
}
```

Repite el proceso anterior para todas las clases de tu BD.

Paso 2: Definir la interface DAO

Luego, definiremos la interfaz DAO para después implementarlas en cada entidad.

DAO

```java
package es.martinezpenya.ejemplos.UD10.starwars;

import java.sql.SQLException;
import java.util.ArrayList;

public interface DAO<T> {
    void crear(T entidad) throws SQLException;;         // C: Create
    T obtener(int id) throws SQLException;;             // R: Read
    ArrayList<T> obtenerTodos() throws SQLException;;   // R: Read (all)
    void actualizar(T entidad) throws SQLException;;    // U: Update
    void eliminar(int id) throws SQLException;;         // D: Delete
}
```

Paso 3: Implementar las clases DAO

Luego, implementaremos las clases DAO para cada entidad e interactuando con la base de datos.

FilmDAO

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package es.martinezpenya.ejemplos.UD10.starwars;

import java.sql.*;
import java.util.ArrayList;

public class FilmDAO implements DAO<Film> {
    private static final String INSERT_QUERY = "INSERT INTO films (id, episode, title) VALUES (?, ?, ?)";
    private static final String SELECT_BY_ID_QUERY = "SELECT * FROM films WHERE id = ?";
    private static final String SELECT_ALL_QUERY = "SELECT * FROM films";
    private static final String UPDATE_QUERY = "UPDATE films SET episode = ?, title = ? WHERE id = ?";
    private static final String DELETE_QUERY = "DELETE FROM films WHERE id = ?";
    private Connection con;

    public FilmDAO () {
        try {
            con = DBConnect.getConnection();
        } catch (SQLException e) {
            System.out.println("ERROR al conectar: " + e.getMessage());
        }
    }

    @Override
    public void crear(Film film) throws SQLException {
        try (PreparedStatement pst = con.prepareStatement(INSERT_QUERY, Statement.RETURN_GENERATED_KEYS)) {

            pst.setInt(1, film.getId());
            pst.setString(2, film.getEpisode());
            pst.setString(3, film.getTitle());

            int filasInsertadas = pst.executeUpdate();
            /* El siguiente fragmento serviria para el caso en que la tabla tuviera un campo autoincremental
              con pst.getGeneratedKeys() obtenemos el id generado y se lo podemos asignar al objeto
              if (filasInsertadas > 0) {
                ResultSet rs = pst.getGeneratedKeys();
                if (rs.next()) {
                    int id = rs.getInt(1);
                    film.setId(id);
                }
            }
            */
        }
    }

    @Override
    public Film obtener(int id) throws SQLException {
        Film usuario = null;
        try (PreparedStatement pst = con.prepareStatement(SELECT_BY_ID_QUERY)) {

            pst.setInt(1, id);
            ResultSet rs = pst.executeQuery();
            if (rs.next()) {
                int idDevuelto = rs.getInt("id");
                String episode = rs.getString("episode");
                String title = rs.getString("title");
                usuario = new Film(id, episode, title);
            }
        }
        return usuario;
    }

    @Override
    public ArrayList<Film> obtenerTodos() throws SQLException {
        ArrayList<Film> films = new ArrayList<>();
        try (Statement st = con.createStatement();
             ResultSet rs = st.executeQuery(SELECT_ALL_QUERY)) {
            while (rs.next()) {
                int id = rs.getInt("id");
                String episode = rs.getString("episode");
                String title = rs.getString("title");
                Film film = new Film(id, episode, title);
                films.add(film);
            }
        }
        return films;
    }

    @Override
    public void actualizar(Film usuario) throws SQLException {
        try (PreparedStatement pst = con.prepareStatement(UPDATE_QUERY)) {
            pst.setString(1, usuario.getEpisode());
            pst.setString(2, usuario.getTitle());
            pst.setInt(3, usuario.getId());
            pst.executeUpdate();
        }
    }
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
    @Override
    public void eliminar(int id) throws SQLException {
        try (PreparedStatement pst = con.prepareStatement(DELETE_QUERY)) {
            pst.setInt(1, id);
            pst.executeUpdate();
        }
    }
}
```

PlanetDAO

```
package es.martinezpenya.ejemplos.UD10.starwars;

import java.sql.*;
import java.util.ArrayList;

public class PlanetDAO implements DAO<Planet> {
    private static final String INSERT_QUERY = "INSERT INTO planets (id, name, rotation_period, orbital_period, diameter, climate, gravity, terrain,
surface_water, population) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
    private static final String SELECT_BY_ID_QUERY = "SELECT * FROM planets WHERE id = ?";
    private static final String SELECT_ALL_QUERY = "SELECT * FROM planets";
    private static final String UPDATE_QUERY =
"UPDATE planets SET name = ?, rotation_period = ?, orbital_period = ?, diameter = ?, climate = ?, gravity = ?, terrain = ?, surface_water = ?,
population = ? WHERE id = ?";
    private static final String DELETE_QUERY = "DELETE FROM planets WHERE id = ?";
    private Connection con;

    public PlanetDAO () {
        try {
            con = DBConnect.getConnection();
        } catch (SQLException e) {
            System.out.println("ERROR al conectar: " + e.getMessage());
        }
    }

    public void crear(Planet planet) throws SQLException {
        try (PreparedStatement pst = con.prepareStatement(INSERT_QUERY, Statement.RETURN_GENERATED_KEYS)) {

            pst.setInt(1, planet.getId());
            pst.setString(2, planet.getName());
            pst.setString(3, planet.getRotation_period());
            pst.setString(4, planet.getOrbital_period());
            pst.setString(5, planet.getDiameter());
            pst.setString(6, planet.getClimate());
            pst.setString(7, planet.getGravity());
            pst.setString(8, planet.getTerrain());
            pst.setString(9, planet.getSurface_water());
            pst.setString(10, planet.getPopulation());

            int filasInsertadas = pst.executeUpdate();
            if (filasInsertadas > 0) {
                ResultSet rs = pst.getGeneratedKeys();
                if (rs.next()) {
                    int id = rs.getInt(1);
                    planet.setId(id);
                }
            }
        }
    }

    public Planet obtener(int id) throws SQLException {
        Planet planet = null;
        try (PreparedStatement pst = con.prepareStatement(SELECT_BY_ID_QUERY)) {

            pst.setInt(1, id);
            ResultSet rs = pst.executeQuery();
            if (rs.next()) {
                int idDevuelto = rs.getInt("id");
                String name = rs.getString("name");
                String rotation_period = rs.getString("rotation_period");
                String orbital_period = rs.getString("orbital_period");
                String diameter = rs.getString("diameter");
                String climate = rs.getString("climate");
                String gravity = rs.getString("gravity");
                String terrain = rs.getString("terrain");
                String surface_water = rs.getString("surface_water");
                String population = rs.getString("population");
                planet = new Planet(id, name, rotation_period, orbital_period, diameter, climate, gravity, terrain, surface_water, population);
            }
        }
        return planet;
    }
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
    @Override
    public ArrayList<Planet> obtenerTodos() throws SQLException {
        ArrayList<Planet> planets = new ArrayList<>();
        try (Statement st = con.createStatement();
             ResultSet rs = st.executeQuery(SELECT_ALL_QUERY)) {
            while (rs.next()) {
                int id = rs.getInt("id");
                String name = rs.getString("name");
                String rotation_period = rs.getString("rotation_period");
                String orbital_period = rs.getString("orbital_period");
                String diameter = rs.getString("diameter");
                String climate = rs.getString("climate");
                String gravity = rs.getString("gravity");
                String terrain = rs.getString("terrain");
                String surface_water = rs.getString("surface_water");
                String population = rs.getString("population");
                Planet planet = new Planet(id, name, rotation_period, orbital_period, diameter, climate, gravity, terrain, surface_water, population);
                planets.add(planet);
            }
        }
        return planets;
    }

    public void actualizar(Planet planet) throws SQLException {
        try (PreparedStatement pst = con.prepareStatement(UPDATE_QUERY)) {

            pst.setString(1, planet.getName());
            pst.setString(2, planet.getRotation_period());
            pst.setString(3, planet.getOrbital_period());
            pst.setString(4, planet.getDiameter());
            pst.setString(5, planet.getClimate());
            pst.setString(6, planet.getGravity());
            pst.setString(7, planet.getTerrain());
            pst.setString(8, planet.getSurface_water());
            pst.setString(9, planet.getPopulation());
            pst.setInt(10, planet.getId());

            pst.executeUpdate();
        }
    }

    public void eliminar(int id) throws SQLException {
        try (PreparedStatement pst = con.prepareStatement(DELETE_QUERY)) {

            pst.setInt(1, id);
            pst.executeUpdate();
        }
    }
}
```

CharacterDAO

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package es.martinezpenya.ejemplos.UD10.starwars;

import java.sql.*;
import java.util.ArrayList;

public class CharacterDAO implements DAO<Character> {
    private static final String INSERT_QUERY = "INSERT INTO characters (id, name, height, mass, hair_color, skin_color, eye_color, birth_year, gender,
planet_id) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
    private static final String SELECT_BY_ID_QUERY = "SELECT * FROM characters WHERE id = ?";
    private static final String SELECT_ALL_QUERY = "SELECT * FROM characters";
    private static final String UPDATE_QUERY = "UPDATE characters SET name = ?, height = ?, mass = ?, hair_color = ?, skin_color = ?, eye_color = ?,
birth_year = ?, gender= ?, planet_id = ? WHERE id = ?";
    private static final String DELETE_QUERY = "DELETE FROM characters WHERE id = ?";
    private Connection con;

    public CharacterDAO() {
        try {
            con = DBConnect.getConnection();
        } catch (SQLException e) {
            System.out.println("ERROR al conectar: " + e.getMessage());
        }
    }

    public void crear(Character character) throws SQLException {
        try (PreparedStatement pst = con.prepareStatement(INSERT_QUERY, Statement.RETURN_GENERATED_KEYS)) {

            pst.setInt(1, character.getId());
            pst.setString(2, character.getName());
            pst.setInt(3, character.getHeight());
            pst.setDouble(4, character.getMass());
            pst.setString(5, character.getHair_color());
            pst.setString(6, character.getSkin_color());
            pst.setString(7, character.getEye_color());
            pst.setString(8, character.getBirth_year());
            pst.setString(9, character.getGender());
            pst.setInt(10, character.getPlanet_id().getId());

            int filasInsertadas = pst.executeUpdate();
            if (filasInsertadas > 0) {
                ResultSet rs = pst.getGeneratedKeys();
                if (rs.next()) {
                    int id = rs.getInt(1);
                    character.setId(id);
                }
            }
        }
    }

    @Override
    public Character obtener(int id) throws SQLException {
        Character character = null;
        try (PreparedStatement pst = con.prepareStatement(SELECT_BY_ID_QUERY)) {

            pst.setInt(1, id);
            PlanetDAO planetDAO = new PlanetDAO();
            ResultSet rs = pst.executeQuery();
            if (rs.next()) {
                int idDevuelto = rs.getInt("id");
                String name = rs.getString("name");
                int height = rs.getInt("height");
                double mass = rs.getDouble("mass");
                String hair_color = rs.getString("hair_color");
                String skin_color = rs.getString("skin_color");
                String eye_color = rs.getString("eye_color");
                String birth_year = rs.getString("birth_year");
                String gender = rs.getString("gender");
                Planet planet = planetDAO.obtener(rs.getInt("planet_id")); //buscamos el planeta por su id
                character = new Character(id, name, height, mass, hair_color, skin_color, eye_color, birth_year, gender, planet);
            }
        }
        return character;
    }
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
    @Override
    public ArrayList<Character> obtenerTodos() throws SQLException {
        ArrayList<Character> characters = new ArrayList<>();
        PlanetDAO planetDAO = new PlanetDAO();
        try (Statement st = con.createStatement();
             ResultSet rs = st.executeQuery(SELECT_ALL_QUERY)) {
            while (rs.next()) {
                int id = rs.getInt("id");
                String name = rs.getString("name");
                int height = rs.getInt("height");
                double mass = rs.getDouble("mass");
                String hair_color = rs.getString("hair_color");
                String skin_color = rs.getString("skin_color");
                String eye_color = rs.getString("eye_color");
                String birth_year = rs.getString("birth_year");
                String gender = rs.getString("gender");
                int planet_id = rs.getInt("planet_id");
                Planet planet = null;
                if (!rs.wasNull()){
                    planet = planetDAO.obtener(planet_id); //buscamos el planeta por su id
                }
                Character character = new Character(id, name, height, mass, hair_color, skin_color, eye_color, birth_year, gender, planet);
                characters.add(character);
            }
        }
        return characters;
    }

    @Override
    public void actualizar(Character entidad) throws SQLException {
        try (PreparedStatement pst = con.prepareStatement(UPDATE_QUERY)) {
            pst.setString(1, entidad.getName());
            pst.setInt(2, entidad.getHeight());
            pst.setDouble(3, entidad.getMass());
            pst.setString(4, entidad.getHair_color());
            pst.setString(5, entidad.getSkin_color());
            pst.setString(6, entidad.getEye_color());
            pst.setString(7, entidad.getBirth_year());
            pst.setString(8, entidad.getGender());
            pst.setInt(9, entidad.getPlanet_id().getId());
            pst.setInt(10, entidad.getId());
            pst.executeUpdate();
        }
    }

    @Override
    public void eliminar(int id) throws SQLException {
        try (PreparedStatement pst = con.prepareStatement(DELETE_QUERY)) {
            pst.setInt(1, id);
            pst.executeUpdate();
        }
    }

}
```

CharacterFilmDAO

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package es.martinezpenya.ejemplos.UD10.starwars;

import java.sql.*;
import java.util.ArrayList;

public class CharacterFilmDAO implements DAO<CharacterFilm> {
    private static final String INSERT_QUERY = "INSERT INTO character_films (id_character, id_film) VALUES (?, ?)";
    private static final String SELECT_ALL_QUERY_BY_CHARACTER = "SELECT * FROM character_films WHERE id_character = ?";
    private static final String SELECT_ALL_QUERY_BY_FILM = "SELECT * FROM character_films WHERE id_film = ?";
    private static final String DELETE_QUERY_BY_CHARACTER = "DELETE FROM character_films WHERE id_character = ?";
    private static final String DELETE_QUERY_BY_FILM = "DELETE FROM character_films WHERE id_film = ?";
    public static final String DELETE_QUERY_BY_CHARACTER_AND_FILM = "DELETE FROM character_films WHERE id_character = ? AND id_film = ?";

    private Connection con;

    public CharacterFilmDAO() {
        try {
            con = DBConnect.getConnection();
        } catch (SQLException e) {
            System.out.println("ERROR al conectar: " + e.getMessage());
        }
    }
    @Override
    public void crear(CharacterFilm entidad) throws SQLException {
        try (PreparedStatement pst = con.prepareStatement(INSERT_QUERY, Statement.RETURN_GENERATED_KEYS)) {

            pst.setInt(1, entidad.getCharacter().getId());
            pst.setInt(2, entidad.getFilm().getId());

            ResultSet rs = pst.executeQuery();
        }
    }

    @Override
    public CharacterFilm obtener(int id) throws SQLException {
        //No tiene sentido en una tabla N a N
        return null;
    }

    @Override
    public ArrayList<CharacterFilm> obtenerTodos() throws SQLException {
        //No tiene sentido en una tabla N a N
        return null;
    }

    public ArrayList<CharacterFilm> obtenerTodosPorCharacter(int id_character) throws SQLException {
        ArrayList<CharacterFilm> characterFilms = new ArrayList<>();
        try (PreparedStatement pst = con.prepareStatement(SELECT_ALL_QUERY_BY_CHARACTER)) {
            pst.setInt(1, id_character);
            ResultSet rs = pst.executeQuery();
            CharacterDAO characterDAO = new CharacterDAO();
            Character character = characterDAO.obtener(id_character);
            FilmDAO filmDAO = new FilmDAO();
            while (rs.next()) {
                int id_film = rs.getInt("id_film");
                Film film = filmDAO.obtener(id_film);
                characterFilms.add(new CharacterFilm(character, film));
            }
        }
        return characterFilms;
    }
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
    public ArrayList<CharacterFilm> obtenerTodosPorFilm(int id_film) throws SQLException {
        ArrayList<CharacterFilm> characterFilms = new ArrayList<>();
        try (PreparedStatement pst = con.prepareStatement(SELECT_ALL_QUERY_BY_FILM)) {
            pst.setInt(1, id_film);
            ResultSet rs = pst.executeQuery();
            CharacterDAO characterDAO = new CharacterDAO();
            FilmDAO filmDAO = new FilmDAO();
            Film film = filmDAO.obtener(id_film);
            while (rs.next()) {
                int id_character = rs.getInt("id_character");
                Character character = characterDAO.obtener(id_character);
                characterFilms.add(new CharacterFilm(character, film));
            }
        }
        return characterFilms;
    }

    @Override
    public void actualizar(CharacterFilm entidad) throws SQLException {
        //No tiene sentido en una tabla N a N
    }

    @Override
    public void eliminar(int id) throws SQLException {
        //No tiene sentido en una tabla N a N
    }

    public void eliminarPorCharacterYFilm(int idCharacter, int idFilm) throws SQLException {
        try (PreparedStatement pst = con.prepareStatement(DELETE_QUERY_BY_CHARACTER_AND_FILM)) {
            pst.setInt(1, idCharacter);
            pst.setInt(2, idFilm);
            pst.executeUpdate();
        }
    }

    public void eliminarPorCharacter(int id) throws SQLException {
        try (PreparedStatement pst = con.prepareStatement(DELETE_QUERY_BY_CHARACTER)) {
            pst.setInt(1, id);
            pst.executeUpdate();
        }
    }

    public void eliminarPorFilm(int id) throws SQLException {
        try (PreparedStatement pst = con.prepareStatement(DELETE_QUERY_BY_FILM)) {
            pst.setInt(1, id);
            pst.executeUpdate();
        }
    }
}
```

Paso 4: Implementar la lógica de la aplicación

Finalmente, implementaremos la lógica de la aplicación en una clase principal `Main` donde podremos interactuar con los DAOs y la base de datos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
package es.martinezpenya.ejemplos.UD10.starwars;

import java.sql.Connection;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        try (Connection con = DBConnect.getConnection()) {
            while (true) {
                menuPrincipal();
                int opcion = entrada.nextInt();
                entrada.nextLine(); // Limpiar el buffer del entrada
                switch (opcion) {
                    case 1:
                        gestionarFilms();
                        break;
                    case 2:
                        gestionarCharacters();
                        break;
                    case 3:
                        gestionarCharacterFilms();
                        break;
                    case 4:
                        gestionarPlanetas();
                        break;
                    case 0:
                        System.out.println("Saliendo...");
                        entrada.close();
                        System.exit(0);
                    default:
                        System.out.println("Opción inválida. Intenta de nuevo.");
                }
            }
        } catch (SQLException e) {
            System.out.println("Error en la conexión con la base de datos: " + e.getMessage());
        }
    }

    private static void gestionarCharacterFilms() {
        Scanner entrada = new Scanner(System.in);
        CharacterFilmDAO characterFilmDAO = new CharacterFilmDAO();
        while (true) {
            menuCharacterFilms();
            int opcion = entrada.nextInt();
            entrada.nextLine(); // Limpiar el buffer del entrada
            switch (opcion) {
                case 1:
                    System.out.print("Introduce el id del character: ");
                    int id_character = entrada.nextInt();
                    entrada.nextLine();
                    System.out.print("Introduce el id del film: ");
                    int id_film = entrada.nextInt();
                    entrada.nextLine();
                    Character character = null;
                    Film film = null;
                    try {
                        character = new CharacterDAO().obtener(id_character);
                    } catch (SQLException e) {
                        System.out.println("Error al obtener el character: " + e.getMessage());
                    }
                    try {
                        film = new FilmDAO().obtener(id_film);
                    } catch (SQLException e) {
                        System.out.println("Error al obtener el film: " + e.getMessage());
                    }
                    if (character == null || film == null) {
                        System.out.println("No se pudo crear el CharacterFilm porque el Character o el Film no existen.");

                    } else {
                        CharacterFilm nuevoCharacterFilm = new CharacterFilm(character, film);
                        try {
                            characterFilmDAO.crear(nuevoCharacterFilm);
                            System.out.println("CharacterFilm creado");
                        } catch (SQLException e) {
                            System.out.println("Error al crear characterFilm: " + e.getMessage());
                        }
                    }
                    break;
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
                case 2:
                    System.out.print("Introduce el ID del Character para ver todos los Films en los que aparece: ");
                    int idCharacter = entrada.nextInt();
                    entrada.nextLine(); // Limpiar el buffer del entrada
                    try {
                        ArrayList<CharacterFilm> characterFilms = characterFilmDAO.obtenerTodosPorCharacter(idCharacter);
                        if (!characterFilms.isEmpty()) {
                            System.out.println("Listado de CharacterFilm:");
                            for (CharacterFilm cf : characterFilms) {
                                System.out.println(cf);
                            }
                        } else {
                            System.out.println("No hay CharacterFilm registrados.");
                        }
                    } catch (SQLException e) {
                        System.out.println("Error al obtener todos los CharacterFilm del Character: " + e.getMessage());
                    }
                    break;
                case 3:
                    System.out.print("Introduce el ID del Film para ver todos los Characters que aparecen: ");
                    int idFilm = entrada.nextInt();
                    entrada.nextLine(); // Limpiar el buffer del entrada
                    try {
                        ArrayList<CharacterFilm> characterFilms = characterFilmDAO.obtenerTodosPorFilm(idFilm);
                        if (!characterFilms.isEmpty()) {
                            System.out.println("Listado de CharacterFilm:");
                            for (CharacterFilm cf : characterFilms) {
                                System.out.println(cf);
                            }
                        } else {
                            System.out.println("No hay CharacterFilm registrados.");
                        }
                    } catch (SQLException e) {
                        System.out.println("Error al obtener todos los CharacterFilm del Film: " + e.getMessage());
                    }
                    break;
                case 4:
                    System.out.print("Introduce el ID del Character para eliminar todos los CharacterFilm asociados: ");
                    int idCharacterEliminar = entrada.nextInt();
                    entrada.nextLine(); // Limpiar el buffer del entrada
                    try {
                        characterFilmDAO.eliminarPorCharacter(idCharacterEliminar);
                        System.out.println("CharacterFilm eliminados correctamente.");
                    } catch (SQLException e) {
                        System.out.println("Error al eliminar CharacterFilm: " + e.getMessage());
                    }
                    break;
                case 5:
                    System.out.print("Introduce el ID del Film para eliminar todos los CharacterFilm asociados: ");
                    int idFilmEliminar = entrada.nextInt();
                    entrada.nextLine(); // Limpiar el buffer del entrada
                    try {
                        characterFilmDAO.eliminarPorFilm(idFilmEliminar);
                        System.out.println("CharacterFilm eliminados correctamente.");
                    } catch (SQLException e) {
                        System.out.println("Error al eliminar CharacterFilm: " + e.getMessage());
                    }
                    break;
                case 6:
                    System.out.print("Introduce el ID del Character del CharacterFilm a eliminar: ");
                    int idCharacterCharacterFilmEliminar = entrada.nextInt();
                    System.out.print("Introduce el ID del Film del CharacterFilm a eliminar: ");
                    int idFilmCharacterFilmEliminar = entrada.nextInt();
                    entrada.nextLine(); // Limpiar el buffer del entrada
                    try {
                        characterFilmDAO.eliminarPorCharacterYFilm(idCharacterCharacterFilmEliminar, idFilmCharacterFilmEliminar);
                        System.out.println("CharacterFilm eliminado correctamente.");
                    } catch (SQLException e) {
                        System.out.println("Error al eliminar CharacterFilm: " + e.getMessage());
                    }
                    break;
            }
        }
    }
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
    private static void gestionarFilms() {
        Scanner entrada = new Scanner(System.in);
        FilmDAO filmDAO = new FilmDAO();
        while (true) {
            menuFilms();
            int opcion = entrada.nextInt();
            entrada.nextLine(); // Limpiar el buffer del entrada
            switch (opcion) {
                case 1:
                    System.out.print("Introduce el id del film: ");
                    int id = entrada.nextInt();
                    entrada.nextLine();
                    System.out.print("Introduce el episode del film: ");
                    String episode = entrada.nextLine();
                    System.out.print("Introduce el title del film: ");
                    String title = entrada.nextLine();
                    Film nuevoFilm = new Film(id, episode, title); // El ID se genera automáticamente
                    try {
                        filmDAO.crear(nuevoFilm);
                        System.out.println("Usuario creado con ID: " + nuevoFilm.getId());
                    } catch (SQLException e) {
                        System.out.println("Error al crear usuario: " + e.getMessage());
                    }
                    break;
                case 2:
                    System.out.print("Introduce el ID del film a consultar: ");
                    int idFilm = entrada.nextInt();
                    entrada.nextLine(); // Limpiar el buffer del entrada
                    try {
                        Film film = filmDAO.obtener(idFilm);
                        if (film != null) {
                            System.out.println("Film encontrado:");
                            System.out.println(film);
                        } else {
                            System.out.println("No se encontró ningún film con ese ID.");
                        }
                    } catch (SQLException e) {
                        System.out.println("Error al obtener el film: " + e.getMessage());
                    }
                    break;
                case 3:
                    try {
                        ArrayList<Film> films = filmDAO.obtenerTodos();
                        if (!films.isEmpty()) {
                            System.out.println("Listado de Usuarios:");
                            for (Film u : films) {
                                System.out.println(u);
                            }
                        } else {
                            System.out.println("No hay films registrados.");
                        }
                    } catch (SQLException e) {
                        System.out.println("Error al obtener todos los films: " + e.getMessage());
                    }
                    break;
                case 4:
                    System.out.print("Introduce el ID del film a actualizar: ");
                    int idActualizar = entrada.nextInt();
                    entrada.nextLine(); // Limpiar el buffer del entrada
                    try {
                        Film filmActualizar = filmDAO.obtener(idActualizar);
                        if (filmActualizar != null) {
                            System.out.print("Introduce el nuevo episode del film (deja en blanco para mantener el actual): ");
                            String nuevoEpisode = entrada.nextLine();
                            if (!nuevoEpisode.isEmpty()) {
                                filmActualizar.setEpisode(nuevoEpisode);
                            }
                            System.out.print("Introduce el nuevo title del film (deja en blanco para mantener el actual): ");
                            String nuevoTitle = entrada.nextLine();
                            if (!nuevoTitle.isEmpty()) {
                                filmActualizar.setTitle(nuevoTitle);
                            }
                            filmDAO.actualizar(filmActualizar);
                            System.out.println("Film actualizado correctamente.");
                        } else {
                            System.out.println("No se encontró ningún Film con ese ID.");
                        }
                    } catch (SQLException e) {
                        System.out.println("Error al actualizar Film: " + e.getMessage());
                    }
                    break;
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
                case 5:
                    System.out.print("Introduce el ID del film a eliminar: ");
                    int idEliminar = entrada.nextInt();
                    entrada.nextLine(); // Limpiar el buffer del entrada
                    try {
                        Film filmEliminar = filmDAO.obtener(idEliminar);
                        if (filmEliminar != null) {
                            // eliminar primero los character_films y deaths asociados al film
                            // Character_filmDAO.eliminarCharacter_filmsPorFilm(idEliminar);
                            // luego eliminar el film
                            filmDAO.eliminar(idEliminar);
                            System.out.println("Film eliminado correctamente.");
                        } else {
                            System.out.println("No se encontró ningún film con ese ID.");
                        }
                    } catch (SQLException e) {
                        System.out.println("Error al eliminar film: " + e.getMessage());
                    }
                    break;
                case 0:
                    return;
                default:
                    System.out.println("Opción inválida. Intenta de nuevo.");
            }
        }
    }

    private static void gestionarPlanetas() {
        Scanner entrada = new Scanner(System.in);
        PlanetDAO planetDAO = new PlanetDAO();
        while (true) {
            menuPlanets();
            int opcion = entrada.nextInt();
            entrada.nextLine(); // Limpiar el buffer del entrada
            switch (opcion) {
                case 1:
                    System.out.print("Introduce el id del planeta: ");
                    int id = entrada.nextInt();
                    entrada.nextLine();
                    System.out.print("Introduce el nombre del planeta: ");
                    String name = entrada.nextLine();
                    System.out.print("Introduce el rotation_period del planeta: ");
                    String rotation_period = entrada.nextLine();
                    System.out.print("Introduce el orbital_period del planeta: ");
                    String orbital_period = entrada.nextLine();
                    System.out.print("Introduce el diameter del planeta: ");
                    String diameter = entrada.nextLine();
                    System.out.print("Introduce el climate del planeta: ");
                    String climate = entrada.nextLine();
                    System.out.print("Introduce el gravity del planeta: ");
                    String gravity = entrada.nextLine();
                    System.out.print("Introduce el terrain del planeta: ");
                    String terrain = entrada.nextLine();
                    System.out.print("Introduce el surface_water del planeta: ");
                    String surface_water = entrada.nextLine();
                    System.out.print("Introduce el population del planeta: ");
                    String population = entrada.nextLine();
                    Planet nuevoPlanet = new Planet(id, name, rotation_period, orbital_period, diameter, climate, gravity, terrain, surface_water, popul
ation); // El ID se genera automáticamente
                    try {
                        planetDAO.crear(nuevoPlanet);
                        System.out.println("Planeta creado con ID: " + nuevoPlanet.getId());
                    } catch (SQLException e) {
                        System.out.println("Error al crear planeta: " + e.getMessage());
                    }
                    break;
                case 2:
                    System.out.print("Introduce el ID del planeta a consultar: ");
                    int idPlanet = entrada.nextInt();
                    entrada.nextLine(); // Limpiar el buffer del entrada
                    try {
                        Planet planet = planetDAO.obtener(idPlanet);
                        if (planet != null) {
                            System.out.println("Planeta encontrado:");
                            System.out.println(planet);
                        } else {
                            System.out.println("No se encontró ningún planeta con ese ID.");
                        }
                    } catch (SQLException e) {
                        System.out.println("Error al obtener el planeta: " + e.getMessage());
                    }
                    break;
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
                case 3:
                    try {
                        ArrayList<Planet> planets = planetDAO.obtenerTodos();
                        if (!planets.isEmpty()) {
                            System.out.println("Listado de Planetas:");
                            for (Planet u : planets) {
                                System.out.println(u);
                            }
                        } else {
                            System.out.println("No hay planetas registrados.");
                        }
                    } catch (SQLException e) {
                        System.out.println("Error al obtener todos los planetas: " + e.getMessage());
                    }
                    break;
                case 4:
                    System.out.print("Introduce el ID del planeta a actualizar: ");
                    int idActualizar = entrada.nextInt();
                    entrada.nextLine(); // Limpiar el buffer del entrada
                    try {
                        Planet planetActualizar = planetDAO.obtener(idActualizar);
                        if (planetActualizar != null) {
                            System.out.print("Introduce el nuevo nombre del planeta (deja en blanco para mantener el actual): ");
                            String nuevoName = entrada.nextLine();
                            if (!nuevoName.isEmpty()) {
                                planetActualizar.setName(nuevoName);
                            }
                            System.out.print("Introduce el nuevo rotation_period del planeta (deja en blanco para mantener el actual): ");
                            String nuevoRotation_period = entrada.nextLine();
                            if (!nuevoRotation_period.isEmpty()) {
                                planetActualizar.setRotation_period(nuevoRotation_period);
                            }
                            System.out.print("Introduce el nuevo orbital_period del planeta (deja en blanco para mantener el actual): ");
                            String nuevoOrbital_period = entrada.nextLine();
                            if (!nuevoOrbital_period.isEmpty()) {
                                planetActualizar.setOrbital_period(nuevoOrbital_period);
                            }
                            System.out.print("Introduce el nuevo diameter del planeta (deja en blanco para mantener el actual): ");
                            String nuevoDiameter = entrada.nextLine();
                            if (!nuevoDiameter.isEmpty()) {
                                planetActualizar.setDiameter(nuevoDiameter);
                            }
                            System.out.print("Introduce el nuevo climate del planeta (deja en blanco para mantener el actual): ");
                            String nuevoClimate = entrada.nextLine();
                            if (!nuevoClimate.isEmpty()) {
                                planetActualizar.setClimate(nuevoClimate);
                            }
                            System.out.print("Introduce el nuevo gravity del planeta (deja en blanco para mantener el actual): ");
                            String nuevoGravity = entrada.nextLine();
                            if (!nuevoGravity.isEmpty()) {
                                planetActualizar.setGravity(nuevoGravity);
                            }
                            System.out.print("Introduce el nuevo terrain del planeta (deja en blanco para mantener el actual): ");
                            String nuevoTerrain = entrada.nextLine();
                            if (!nuevoTerrain.isEmpty()) {
                                planetActualizar.setTerrain(nuevoTerrain);
                            }
                            System.out.print("Introduce el nuevo surface_water del planeta (deja en blanco para mantener el actual): ");
                            String nuevoSurface_water = entrada.nextLine();
                            if (!nuevoSurface_water.isEmpty()) {
                                planetActualizar.setSurface_water(nuevoSurface_water);
                            }
                            System.out.print("Introduce el nuevo population del planeta (deja en blanco para mantener el actual): ");
                            String nuevoPopulation = entrada.nextLine();
                            if (!nuevoPopulation.isEmpty()) {
                                planetActualizar.setPopulation(nuevoPopulation);
                            }
                            planetDAO.actualizar(planetActualizar);
                            System.out.println("Planeta actualizado correctamente.");
                        } else {
                            System.out.println("No se encontró ningún Planeta con ese ID.");
                        }
                    } catch (SQLException e) {
                        System.out.println("Error al actualizar Planeta: " + e.getMessage());
                    }
                    break;
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
                case 5:
                    System.out.print("Introduce el ID del planeta a eliminar: ");
                    int idEliminar = entrada.nextInt();
                    entrada.nextLine(); // Limpiar el buffer del entrada
                    try {
                        Planet planetEliminar = planetDAO.obtener(idEliminar);
                        if (planetEliminar != null) {
                            // todo: Revisar si quiero eliminar Characters al eliminar su Planeta Origen.
                            planetDAO.eliminar(idEliminar);
                            System.out.println("Planeta eliminado correctamente.");
                        } else {
                            System.out.println("No se encontró ningún planeta con ese ID.");
                        }
                    } catch (SQLException e) {
                        System.out.println("Error al eliminar planeta: " + e.getMessage());
                    }
                    break;
                case 0:
                    return;
                default:
                    System.out.println("Opción inválida. Intenta de nuevo.");
            }
        }
    }

    private static void gestionarCharacters() {
        Scanner entrada = new Scanner(System.in);
        CharacterDAO characterDAO = new CharacterDAO();
        while (true) {
            menuCharacters();
            PlanetDAO planetDAO = new PlanetDAO();
            int opcion = entrada.nextInt();
            entrada.nextLine(); // Limpiar el buffer del entrada
            switch (opcion) {
                case 1:
                    System.out.print("Introduce el id del character: ");
                    int id = entrada.nextInt();
                    entrada.nextLine();
                    System.out.print("Introduce el name del character: ");
                    String name = entrada.nextLine();
                    System.out.print("Introduce el height del character: ");
                    int height = Integer.parseInt(entrada.nextLine());
                    System.out.print("Introduce el mass del character: ");
                    int mass = Integer.parseInt(entrada.nextLine());
                    System.out.print("Introduce el hair_color del character: ");
                    String hair_color = entrada.nextLine();
                    System.out.print("Introduce el skin_color del character: ");
                    String skin_color = entrada.nextLine();
                    System.out.print("Introduce el eye_color del character: ");
                    String eye_color = entrada.nextLine();
                    System.out.print("Introduce el birth_year del character: ");
                    String birth_year = entrada.nextLine();
                    System.out.print("Introduce el gender del character: ");
                    String gender = entrada.nextLine();
                    System.out.print("Introduce el id del planeta del character: ");
                    int planet_id = Integer.parseInt(entrada.nextLine());
                    Character nuevoCharacter = null;
                    try {
                        nuevoCharacter = new Character(id, name, height, mass, skin_color, hair_color, eye_color, birth_year, gender,
planetDAO.obtener(planet_id));
                    } catch (SQLException e) {
                        System.out.println("Error al obtener el planeta: " + e.getMessage());
                    }
                    try {
                        characterDAO.crear(nuevoCharacter);
                        System.out.println("Character creado con ID: " + nuevoCharacter.getId());
                    } catch (SQLException e) {
                        System.out.println("Error al crear character: " + e.getMessage());
                    }
                    break;
                case 2:
                    System.out.print("Introduce el ID del character a consultar: ");
                    int idCharacter = entrada.nextInt();
                    entrada.nextLine(); // Limpiar el buffer del entrada
                    try {
                        Character character = characterDAO.obtener(idCharacter);
                        if (character != null) {
                            System.out.println("Character encontrado:");
                            System.out.println(character);
                        } else {
                            System.out.println("No se encontró ningún character con ese ID.");
                        }
                    } catch (SQLException e) {
                        System.out.println("Error al obtener el character: " + e.getMessage());
                    }
                    break;
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
                case 3:
                    try {
                        ArrayList<Character> characters = characterDAO.obtenerTodos();
                        if (!characters.isEmpty()) {
                            System.out.println("Listado de Characters:");
                            for (Character u : characters) {
                                System.out.println(u);
                            }
                        } else {
                            System.out.println("No hay characters registrados.");
                        }
                    } catch (SQLException e) {
                        System.out.println("Error al obtener todos los characters: " + e.getMessage());
                    }
                    break;
                case 4:
                    System.out.print("Introduce el ID del character a actualizar: ");
                    int idActualizar = entrada.nextInt();
                    entrada.nextLine(); // Limpiar el buffer del entrada
                    try {
                        Character characterActualizar = characterDAO.obtener(idActualizar);
                        if (characterActualizar != null) {
                            System.out.print("Introduce el nuevo name del character (deja en blanco para mantener el actual): ");
                            String nuevoName = entrada.nextLine();
                            if (!nuevoName.isEmpty()) {
                                characterActualizar.setName(nuevoName);
                            }
                            System.out.print("Introduce el nuevo height del character (deja en blanco para mantener el actual): ");
                            String nuevoHeight = entrada.nextLine();
                            if (!nuevoHeight.isEmpty()) {
                                characterActualizar.setHeight(Integer.parseInt(nuevoHeight));
                            }
                            System.out.print("Introduce el nuevo mass del character (deja en blanco para mantener el actual): ");
                            String nuevoMass = entrada.nextLine();
                            if (!nuevoMass.isEmpty()) {
                                characterActualizar.setMass(Double.parseDouble(nuevoMass));
                            }
                            System.out.print("Introduce el nuevo hair_color del character (deja en blanco para mantener el actual): ");
                            String nuevoHair_color = entrada.nextLine();
                            if (!nuevoHair_color.isEmpty()) {
                                characterActualizar.setHair_color(nuevoHair_color);
                            }
                            System.out.print("Introduce el nuevo skin_color del character (deja en blanco para mantener el actual): ");
                            String nuevoSkin_color = entrada.nextLine();
                            if (!nuevoSkin_color.isEmpty()) {
                                characterActualizar.setSkin_color(nuevoSkin_color);
                            }
                            System.out.print("Introduce el nuevo eye_color del character (deja en blanco para mantener el actual): ");
                            String nuevoEye_color = entrada.nextLine();
                            if (!nuevoEye_color.isEmpty()) {
                                characterActualizar.setEye_color(nuevoEye_color);
                            }
                            System.out.print("Introduce el nuevo birth_year del character (deja en blanco para mantener el actual): ");
                            String nuevoBirth_year = entrada.nextLine();
                            if (!nuevoBirth_year.isEmpty()) {
                                characterActualizar.setBirth_year(nuevoBirth_year);
                            }
                            System.out.print("Introduce el nuevo gender del character (deja en blanco para mantener el actual): ");
                            String nuevoGender = entrada.nextLine();
                            if (!nuevoGender.isEmpty()) {
                                characterActualizar.setGender(nuevoGender);
                            }
                            System.out.print("Introduce el nuevo id del planeta del character (deja en blanco para mantener el actual): ");
                            String nuevoPlanet_id = entrada.nextLine();
                            if (!nuevoPlanet_id.isEmpty()) {
                                Planet p;
                                if ((p=planetDAO.obtener(Integer.parseInt(nuevoPlanet_id)))!=null) {
                                    characterActualizar.setPlanet_id(p);
                                }
                            }
                            characterDAO.actualizar(characterActualizar);
                            System.out.println("Character actualizado correctamente.");
                        } else {
                            System.out.println("No se encontró ningún Character con ese ID.");
                        }
                    } catch (SQLException e) {
                        System.out.println("Error al actualizar Character: " + e.getMessage());
                    }
                    break;
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
                case 5:
                    System.out.print("Introduce el ID del character a eliminar: ");
                    int idEliminar = entrada.nextInt();
                    entrada.nextLine(); // Limpiar el buffer del entrada
                    try {
                        Character characterEliminar = characterDAO.obtener(idEliminar);
                        if (characterEliminar != null) {
                            characterDAO.eliminar(idEliminar);
                            System.out.println("Character eliminado correctamente.");
                        } else {
                            System.out.println("No se encontró ningún character con ese ID.");
                        }
                    } catch (SQLException e) {
                        System.out.println("Error al eliminar character: " + e.getMessage());
                    }
                    break;
                case 0:
                    return;
                default:
                    System.out.println("Opción inválida. Intenta de nuevo.");
            }
        }
    }

    // MENÚ PRINCIPAL: menuPrincipal()
    private static void menuPrincipal() {
        System.out.println("\nMenú Principal:");
        System.out.println("1. Gestionar Films");
        System.out.println("2. Gestionar Characters");
        System.out.println("3. Gestionar Character_films");
        System.out.println("4. Gestionar Planets");
        System.out.println("0. Salir");
        System.out.print("Selecciona una opción:  ");
    }

    // MENÚ SECUNDARIO: menuFilms()
    private static void menuFilms() {
        System.out.println("\nMenú de Films:");
        System.out.println("1. Crear film");
        System.out.println("2. Consultar film por ID");
        System.out.println("3. Listar todos los films");
        System.out.println("4. Actualizar film");
        System.out.println("5. Eliminar film");
        System.out.println("0. Volver al menú principal");
        System.out.print("Selecciona una opción:  ");
    }

    //MENU SECUNDARIO: menuPlanets()
    private static void menuPlanets() {
        System.out.println("\nMenú de Planets:");
        System.out.println("1. Crear planeta");
        System.out.println("2. Consultar planeta por ID");
        System.out.println("3. Listar todos los planetas");
        System.out.println("4. Actualizar planeta");
        System.out.println("5. Eliminar planeta");
        System.out.println("0. Volver al menú principal");
        System.out.print("Selecciona una opción:  ");
    }

    // MENÚ SECUNDARIO: menuCharacters()
    private static void menuCharacters() {
        System.out.println("\nMenú de Characters:");
        System.out.println("1. Crear character");
        System.out.println("2. Consultar character por ID");
        System.out.println("3. Listar todos los characters");
        System.out.println("4. Actualizar character");
        System.out.println("5. Eliminar character");
        System.out.println("0. Volver al menú principal");
        System.out.print("Selecciona una opción:  ");
    }

    // MENÚ SECUNDARIO: menuCharacterFilms()
    private static void menuCharacterFilms() {
        System.out.println("\nMenú de CharacterFilm:");
        System.out.println("1. Crear characterFilm");
        System.out.println("2. Consultar characterFilm por Character");
        System.out.println("3. Consultar characterFilm por Film");
        System.out.println("4. Eliminar characterFilm por Character");
        System.out.println("5. Eliminar characterFilm por Film");
        System.out.println("6. Eliminar characterFilm por Character y Film");
        System.out.println("0. Volver al menú principal");
        System.out.print("Selecciona una opción:  ");
    }
}
```

Consideraciones

- **HikariCP y SQL Queries:** Hemos utilizado HikariCP para conectarnos y realizar operaciones en la base de datos. Es importante manejar excepciones y cerrar correctamente las conexiones y recursos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- **Patrón DAO:** Este patrón nos ayuda a mantener un código organizado y aislado, separando la lógica de acceso a datos de la lógica de negocio.
- **Lógica de la aplicación:** En la clase `Main`, hemos implementado un menú interactivo que permite al usuario gestionar *films*, *characters*, *planets* y *character_films* utilizando los métodos proporcionados por los DAO's.
- **Adaptabilidad:** Puedes expandir este ejemplo completando las tablas que faltan, añadiendo más funcionalidades o haciendo ajustes según los requisitos de tu aplicación.
- **Fuera de la norma**: Fíjate en la lógica de la tabla `character_films` (tabla N a N), fíjate que no sigue exactamente el patrón DAO, pero eso no es un problema, la interfaz DAO marca un mínimo, pero no es completa, la podemos expandir con nuevas funcionalidades.
- **Eliminación y actualización en cascada**: En este ejemplo no se han contemplado estos casos, se podrían gestionar directamente con el gestor de las bases de datos, al definir las tablas y su comportamiento, o bien simular esta funcionalidad desde sentencias SQL en la lógica del programa.
- **Autonuméricos**: El ejemplo que se ha desarrollado más arriba no tiene ninguna tabla con un campo autoincremental (como suelen tener algunas claves primarias), pero en el código fuente del método `crear` de la clase `FilmDAO` puedes ver un fragmento comentado sobre como podrias gestionar el echo de que no sepas el identificador del registro que insertas en la BBDD hasta que realmente se haya insertado, como recuperarlo y mostrarlo al usuario.
- **Tablas Relacionadas**: Fíjate en el comportamiento que tiene la clase `Character` respecto del `Planet` (o el resto de relaciones), no solo se muestra el `planet_id` de la tabla `Character`, sino que se recupera también el resto de información del planeta.
- **Localización de la BBDD**. En este caso he apostado por alojar la BBDD en un servidor AWS, pero si lo prefieres puedes hacerlo localmente. Eso si, asegurate que le das toda la información necesaria al docente para que pueda reproducir tu entorno y poder evaluar que tu proyecto funciona como se espera.
- **Rendimiento**: En según que casos, si las tablas tienen muchísima información pueden provocar salidas muy lentas, o incluso errores de *timeout*.

Tarea

Teniendo todo lo anterior en cuenta debes...

1. Crear un nuevo proyecto con un nombre identificativo que incluya tu nombre y/o apellidos. En mi caso podria llamarse "`StarWarsDavid`".
2. Sigue las instrucciones para crear las clases básicas para cada una de las tablas de tu BBDD.
3. Genera la clase de conexión a la BBDD.
4. Define la interfaz para gestionar el patrón `DAO`.
5. Implementa las clases que a su vez implementan la interfaz `DAO` para cada una de las tablas.
6. Genera una clase con el `main` que hará que todo funcione con menús y gestión completa del CRUD de tu BBDD.
7. Añade un archivo `.sql` que permita reconstruir la BBDD desde cero, debe incluir tanto el DDL como la inserción de información en las tablas.

Envía a la tarea de Aules:

1. Una memoria en fichero **pdf** explicando los pasos seguidos (con capturas), explicando la estructura de la BBDD, las tablas que vas a implementar, y toda la información que creas relevante, así como tu opinión personal sobre el proyecto (dificultades, futuras ampliaciones, limitaciones conocidas, etc.)
2. Un fichero **zip** con el proyecto gestionado con `maven` en **IntelliJ**. Asegurate de incluir todo lo necesario para que el profesor pueda reproducir y poner en funcionamiento el proyecto. No olvides el fichero `.sql` (o similar) para poder generar la BBDD de nuevo. Este archivo generalmente lo puedes obtener exportando la BBDD o generando una copia de seguridad en SQL.

6 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

12. UD11

12.1 Bases de Datos Orientadas a Objetos

Al finalizar esta unidad serás capaz de...

- Conocer las características de las bases de datos orientadas a objetos
- Instalar y configurar ObjectDB
- Crear bases de datos y almacenar objetos
- Realizar consultas con OQL
- Actualizar y eliminar objetos persistentes
- Conocer JDO y JPA como estándares de persistencia
- Comparar BDOO con BDR (relacionales)

12.1.1 Introducción

Si bien las BD relacionales son las más populares y las que más aceptan, su utilización dentro de una aplicación orientada a objetos implica un proceso de traducción del diagrama UML original a un modelo relacional, totalmente basado en tablas. En esta traducción se pierden muchas de las funcionalidades básicas de la orientación a objetos, que deben simularse de algún modo: referencias a objetos, clases asociativas, listas de objetos, herencia, etc. Cuando el diagrama es de cierta complejidad, la traducción puede resultar muy complicada.

Para resolver este problema se encuentran las BD orientadas a objetos (BDOO). Éstas, en lugar de organizar los datos en tablas, los organizan exactamente tal y como lo hace un diagrama UML, mediante la definición del conjunto de clases y relaciones entre ellas. Por tanto, no es necesario hacer ninguna traducción.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

> **Aclaración**
>
> Para evitar confusiones, utilizamos el término `BDR` para referirnos explícitamente a una BD relacional y el término `BDOO` para referirnos a una BD orientada a objetos, de forma que ambos queden diferenciados.

Actualmente, la aplicación de `BDOO` se limita a ámbitos muy concretos, especialmente a los vinculados a áreas científicas. Su implantación en aplicaciones comerciales a nivel general es muy baja. Uno de los principales problemas de las BDOO es que los fabricantes tienden a crear soluciones incompatibles, que no obedecen a ninguna especificación concreta. Al contrario que en el caso de las BDR, es muy posible que una aplicación cliente hecha por el producto de un fabricante concreto no funcione sobre una BDOO de otro fabricante. De hecho, a las BDOO que soportan Java no se accede mediante JDBC, puesto que este mecanismo es específico para BDR, sino que normalmente se accede usando bibliotecas específicas para cada fabricante. Java tiene una especificación para BDOO llamada JDO (Java Data Objects).

Las **bases de datos** `NoSQL` (**N**ot **O**nly **SQL**) son sistemas de almacenamiento diseñados para manejar datos no estructurados o semiestructurados, destacando por su escalabilidad horizontal, esquemas flexibles y alto rendimiento en entornos distribuidos. Incluyen modelos como documentos (`MongoDB`), clave-valor (`Redis`), columnas (`Cassandra`) y grafos (`Neo4j`), siendo ideales para Big Data, aplicaciones en tiempo real y escenarios donde las bases relacionales tradicionales resultan inflexibles.

12.1.2 Los lenguajes `ODL` y `OQL`

De la misma manera que existe `SQL` como lenguaje estándar para acceder a una `BDR` independientemente del fabricante, existe un lenguaje para acceder a los datos de una `BDOO` : el lenguaje de consultas a objetos (object query language, `OQL`) . Adicionalmente, existe el lenguaje de descripción de objetos (object description language, `ODL`), que sirve para especificar el formato de una `BDOO` : qué tipo de objetos puede contener y sus relaciones. Desgraciadamente, si bien estos lenguajes están especificados, y como ya se ha dicho, no se puede contar con que cualquier fabricante realmente los soporte. De ninguna forma llegan al grado de aceptación de SQL.

El lenguaje `ODL`

El lenguaje `ODL` se utiliza para definir clases de objetos persistentes en una `BDOO` , de modo que sus objetos se puedan almacenar. Dentro de la declaración de cada clase se incluye:

- El nombre de la clase.
- Declaraciones opcionales de claves primarias.
- La declaración de la extensión: el nombre del conjunto de instancias existentes.
- Declaraciones de elementos: atributos, relaciones o métodos.

La sintaxis es la siguiente (entre corchetes se indican campos opcionales):

```
class nombreClase [(key nombreClavePrimaria)] {
    attribute tipoAtributo nombreAtributo;
    ...
    relationship tipo<nombreClaseDestino> nombreRelacion;
    ...
    tipoRetorno nombreMetodo(params) [raises (tipoExcepcion)]
    ...
}
```

Como puede apreciarse, simplemente es un cambio de sintaxis respecto al lenguaje Java propiamente, pero la mayoría de elementos de una clase son claramente identificables.

La única diferencia es la declaración explícita de las relaciones en forma de la palabra clave `relationship` , en contraste en Java, que se traducen a atributos. Existen diferentes tipos de relaciones según la cardinalidad que se quiere expresar. De hecho, cada uno de estos tipos tiene una cierta correspondencia con las clases que se usan en Java para implementar relaciones. Para cardinalidad 1, basta con poner el nombre de la clase destino. En caso de cardinalidad múltiple, se puede elegir entre distintos tipos:

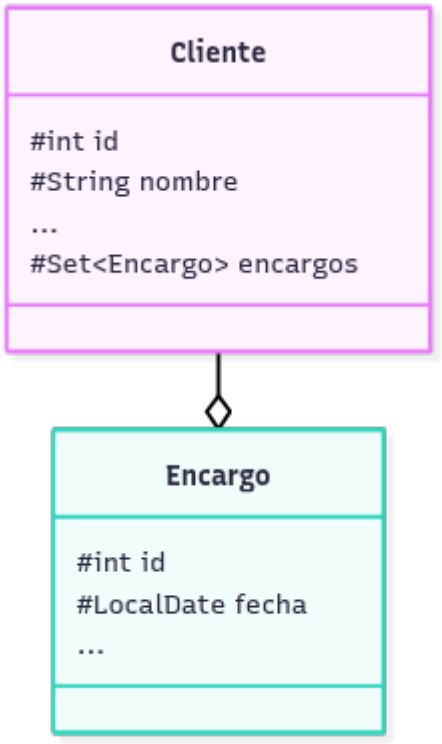
- `<nombreClaseDestino>` , si la relación es sólo a un único objeto.
- `Set<nombreClaseDestino>` , un conjunto no ordenado sin repeticiones.
- `Bag<nombreClaseDestino>` , un conjunto no ordenado con repeticiones.
- `List<nombreClaseDestino>` , un conjunto ordenado con repeticiones, con inserciones eficientes.
- `Array<nombreClaseDestino>` , un conjunto ordenado con repeticiones.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Su significado es el mismo que el de las clases homónimas de Java (capacidad de haber repeticiones de elementos, ordenadas o no por índice, etc.). Normalmente, la más usada es `Set<nombreClaseDestino>`.

A continuación se muestra cómo podría representarse dos clases interdependientes llamadas `Cliente` y `Encargo`, que almacenan datos en una aplicación de gestión de clientes, mediante `ODL`. Observa el Ejemplo01 para ver la implementación completa.

La representación en UML seria algo parecido a esto:



La herencia entre clases se inicia en su declaración mediante la palabra clave extends seguido del nombre de la superclase:

```
class nombreClase extends nombreSuperClase {
    ...
}
```

El lenguaje `OQL`

El lenguaje `OQL` se limita a permitir consultas sobre una `BDOO`. Su operador principal es `SELECT`, el cual tiene una gran similitud con el equivalente `SQL`. Sin embargo, tiene algunas particularidades debidas a la forma en que se estructuran los datos mediante la orientación a objetos (por ejemplo, no hay tablas, por supuesto).

La sintaxis general es:

```
SELECT valor1,valor2,...
FROM lista_de_colecciones y nombres_para_miembros_típicos
WHERE condición
```

Dado que ahora ya no hay tablas, es necesario tener en cuenta dos cosas. Por un lado, la lista de colecciones especificada en el apartado `FROM` corresponde a alguna de las clases declaradas. Junto al nombre de esta clase se especifica la variable que se utilizará en los términos `SELECT` y `WHERE` para referenciar valores. Por otro lado, los valores que se quieren consultar o comparar son atributos de clases, por lo que la forma de referirse a ellos es mediante la nomenclatura: `nombreClase.nombreAtributo`. Esta posibilidad también puede utilizarse para indicar sus relaciones.

Por ejemplo, en una aplicación de gestión de clientes, si se quiere consultar a los clientes de la `BDOO` de acuerdo con la definición de sus clases, se puede hacer:

Observa el Ejemplo02 para más consultas de ejemplo con OQL.

También es posible acceder a los encargos por medio de los clientes, siguiendo su relación:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

12.1.3 La librería `ObjectDB`

A pesar de los esfuerzos por estandarizar el uso de las `BDOO`, no se puede decir que actualmente haya ningún equivalente al lenguaje `SQL`. Aunque sobre el papel están `ODL` y `OQL`, en la práctica ahora mismo no hay ninguna lengua franca que se pueda garantizar que está soportada, al menos en sus aspectos fundamentales, por todas las bases de datos, todo y que después cada fabricante pueda añadir igualmente sus propias extensiones propietarias. Por tanto, cada tipo de `BDOO` ofrece su propio sistema para acceder a los objetos almacenados. Afortunadamente, como verá, esto no es muy problemático, ya que la ventaja de usar una `BDOO` es poder crear código donde operar con objetos persistentes; es casi igual que trabajar con objetos en memoria, y, por tanto, los mecanismos que ofrecen las diferentes `BDOO` a menudo son muy parecidos a trabajar con objetos directamente en memoria. Lo que varía son las librerías de clases a utilizar, pero no la idea general. En este apartado se verá un caso concreto de acceso a una `BDOO` llamada **ObjectDB**

ObjectDB

ObjectDB es un potente sistema de gestión de bases de datos orientado a objetos (`ODBMS`). Es compacto, fiable, fácil de usar y extremadamente rápido. **ObjectDB** proporciona todos los servicios estándar de administración de bases de datos (almacenamiento y recuperación, transacciones, administración de bloqueos, procesamiento de consultas, etc.), pero de una manera que facilita el desarrollo y acelera las aplicaciones.

Características clave de la base de datos ObjectDB

- Sistema de gestión de bases de datos orientado a objetos (ODBMS) 100% puro Java.
- Sin API propietaria - administrado únicamente por API de Java estándar (JPA 2 / JDO 2).
- Extremadamente rápido: más rápido que cualquier otro producto JPA/JDO.
- Adecuado para archivos de bases de datos que van desde kilobytes hasta terabytes.
- Admite tanto el modo Cliente-Servidor como el modo Integrado.
- JAR único sin dependencias externas.
- La base de datos se almacena como un único archivo.
- Capacidades avanzadas de consulta e indexación.
- Efectivo en entornos multiusuario con mucha carga.
- Puede integrarse fácilmente en aplicaciones de cualquier tipo y tamaño.
- Probado con Tomcat, Jetty, GlassFish, JBoss y Spring.

ObjectDB, que es la `BDOO` elegida permite tanto `JDO` como `JPA` (Java Persistence API). Aunque parece que el mercado evoluciona del lado de JPA y es la modalidad más usada para dotar de persistencia a los datos. Por tanto en este tema usaremos **ObjectDB** con **JPA**.

ObjectDB se puede descargar y utilizar **sin coste (incluso comercialmente)** con la restricción de un máximo de **10 clases de entidad** y **un millón de objetos de entidad** por archivo de base de datos. Esto podría ser útil para proyectos pequeños, tareas académicas, evaluación y aprendizaje. **ObjectDB** es un software comercial y su uso sin estas restricciones requiere la compra de una licencia.

Se recomienda probar **ObjectDB** antes de comprar una licencia.

Más información sobre tipos de licencias: https://www.objectdb.com/database/purchase

Usar `ObjectDB` con `Maven`

Para usar `ObjectDB` en nuestros proyectos, lo podemos hacer de manera fácil a través de `Maven` (consulta talleres anteriores si tienes dudas sobre como usar `Maven`). Debes añadir un repositorio y una dependencia:

```
[...]
    <repositories>
        <repository>
            <id>objectdb</id>
            <name>ObjectDB Repository</name>
            <url>https://m2.objectdb.com</url>
        </repository>
    </repositories>

    <dependencies>
        <dependency>
            <groupId>com.objectdb</groupId>
            <artifactId>objectdb</artifactId>
            <version>2.8.1</version>
        </dependency>
    </dependencies>
[...]
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Consulta el Taller 1 de esta unidad para realizar una introducción rápida al uso de `ObjectDB` (con `Maven`).

12.1.4 Anotaciones para `ObjectDB`

Como hemos explicado anteriormente, cada BDOO utilitza su propio lenguaje ODL y OQL, a continuación presentaremos el modo de definir la estructura de la BDOO que tiene `ObjectDB`, insertando anotaciones (palabras reservadas que comienza por `@`), para definir la estructura y la persistencia de los objetos en la BDOO.

Ampliación

Aquí, por su extensión, solo veremos una parte de todo lo que se puede hacer con ObjectDB y JPA, si te interesa profundiza más, te recomiendo que acudas al manual online que tienen publicado en su web, y en el que se basan estos apuntes: https://house.objectdb.com/java/jpa

¿Por qué ObjectDB?

¿Porqué usamos DBOO y en concreto `ObjectDB` si requiere la compra de una licencia y además no se usa mucho en el mercado?

La ventaja de usar `ObjectDB` es que es plenamente compatible con `JPA`, y sus anotaciones son muy similares a las que deberias usar con otros `ORM`, como por ejemplo `Hibernate`.

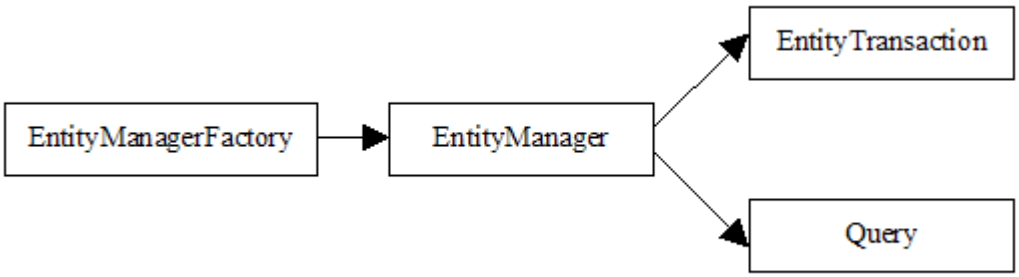
`Hibernate` es un `ORM` que si se usa mucho a nivel comercial, pero no cumple con los requisitos que marca el Real Decreto que regula el ciclo superior de Desarrollo de Aplicaciones Multiplataforma.

De esta manera al estudiar `ObjectDB` nos sirve para cuando necesites usar `Hibernate` y cubrimos los requisitos del RD.

Usando JPA

CONEXIÓN DE BASE DE DATOS USANDO `JPA`

Trabajar con Java Persistence API (JPA) consiste en utilizar las siguientes interfaces:



Descripción general

Una conexión a una base de datos está representada por una instancia EntityManager, que también proporciona funcionalidad para realizar operaciones en una base de datos. Muchas aplicaciones requieren múltiples conexiones a bases de datos durante su vida útil. Por ejemplo, en una aplicación web es común establecer una conexión de base de datos separada, utilizando una instancia "EntityManager" separada, para cada solicitud HTTP.

La función principal de una instancia de EntityManagerFactory es admitir la creación de múltiples instancias de EntityManager. Una `EntityManagerFactory` se construye para una base de datos específica y, al administrar los recursos de manera eficiente (por ejemplo, un grupo de sockets), proporciona una manera eficiente de construir múltiples instancias de `EntityManager` para esa base de datos. La creación de instancias de `EntityManagerFactory` en sí puede ser menos eficiente, pero es una operación única. Una vez construido, puede servir para toda la aplicación.

Las operaciones que modifican el contenido de una base de datos requieren transacciones activas. Las transacciones son administradas por una instancia de EntityTransaction obtenida del `EntityManager`.

Una instancia de `EntityManager` también funciona como una fábrica para instancias de Query, que son necesarias para ejecutar consultas en la base de datos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Cada implementación de JPA define clases que implementan estas interfaces. Cuando utiliza `ObjectDB`, trabaja con instancias de clases de `ObjectDB` que implementan estas interfaces y, debido a que se utilizan interfaces JPA estándar, su aplicación es portable.

`EntityManagerFactory`

Una instancia de EntityManagerFactory se obtiene utilizando un método estático de la clase base de JPA, Persistence:

```
1  EntityManagerFactory emf = Persistence.createEntityManagerFactory("objectdb:myDbFile.odb");
```

La instancia EntityManagerFactory, cuando se construye, abre la base de datos. Si la base de datos aún no existe, se crea un nuevo archivo de base de datos. Cuando la aplicación termine de usar `EntityManagerFactory`, debe cerrarse. Al cerrar `EntityManagerFactory`, se cierra el archivo de base de datos.

URL de conexión

El método createEntityManagerFactory toma como argumento el nombre de una unidad de persistencia. Como extensión, `ObjectDB` permite especificar una URL (o ruta) de base de datos directamente, evitando la necesidad de una unidad de persistencia. `ObjectDB` considera que cualquier cadena que comience con `objectdb:` o termine con `.odb` o `.objectdb` es una URL de base de datos en lugar de un nombre de unidad de persistencia.

Para utilizar `ObjectDB` integrado directamente en su aplicación (modo integrado), se debe especificar una ruta absoluta o relativa de un archivo de base de datos local (por ejemplo, `"my.odb"`). Especificar el protocolo `objectdb:` como prefijo (p. ej., `"objectdb:my.odb"`) es opcional si la extensión del nombre del archivo de la base de datos es `odb` u `objectdb` y es necesario para otras extensiones de nombre de archivo (p. ej., `"objectdb:mi-db.tmp"`).

Para utilizar el modo cliente-servidor, se debe especificar una URL con el formato `objectdb://host:puerto/ruta`. En este caso, se espera que se ejecute un Servidor de base de datos ObjectDB en una máquina llamada host (podría ser un nombre de dominio o una dirección IP) y escuchando en el puerto especificado (el valor predeterminado es 6136 cuando no se especifica). La ruta indica la ubicación del archivo de base de datos en el servidor, en relación con la ruta raíz de datos del servidor.

Administrador de entidades

Una instancia de EntityManager puede representar una conexión remota a un servidor de base de datos remoto (en modo cliente-servidor) o una conexión local a un archivo de base de datos local (en modo incrustado). La funcionalidad en ambos casos es la misma.

La instancia `EntityManager` se obtiene de la instancia propietaria `EntityManagerFactory`. Llamar al método close es esencial para liberar recursos (como un socket en modo cliente-servidor) a la `EntityManagerFactory` propietaria.

Entidad de Transacción

Las operaciones que afectan el contenido de la base de datos (almacenar, actualizar, eliminar) deben realizarse dentro de una transacción activa. La interfaz EntityTransaction representa y administra transacciones de bases de datos. Cada EntityManager contiene una única instancia adjunta `EntityTransaction` que está disponible a través de getTransaction(). Una transacción se inicia con una llamada a begin y finaliza con una llamada a commit o rollback.

Todas las operaciones en la base de datos dentro de estos límites están asociadas con esa transacción y se mantienen en la memoria hasta que finaliza la transacción. Si la transacción finaliza con una "reversión" (roll-back), todas las modificaciones a la base de datos se descartan. Sin embargo, de forma predeterminada, la instancia en memoria de la entidad gestionada no se ve afectada por la reversión y no vuelve a su estado modificado previamente.

Finalizar una transacción con un "commit" propaga todas las modificaciones físicamente a la base de datos. Si por algún motivo falla una "commit", la transacción se revierte automáticamente (incluidas las modificaciones que ya se han propagado a la base de datos antes del error) y se genera una RollbackException.

Consulta el Ejemplo03 para ver el ciclo completo de conexión a ObjectDB.

`TRABAJAR CON OBJETOS DE ENTIDAD` `JPA`

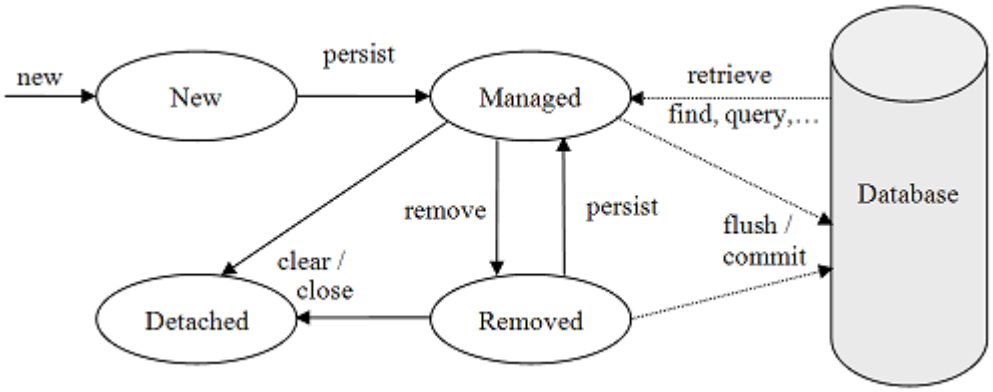
Los objetos de entidad son instancias en memoria de clases de entidad (clases persistentes definidas por el usuario), que pueden representar objetos físicos en la base de datos.

Administrar una base de datos de objetos `ObjectDB` usando JPA requiere el uso de objetos de entidad para muchas operaciones, incluido el almacenamiento, recuperación, actualización y eliminación de objetos de la base de datos.

Ciclo de vida del objeto de entidad

El ciclo de vida de los objetos de entidad consta de cuatro estados: Nuevo, Administrado, Eliminado y Separado.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



Cuando se crea inicialmente un objeto de entidad, su estado es **Nuevo**. En este estado, el objeto aún no está asociado con un EntityManager y no tiene representación en la base de datos.

Un objeto de entidad pasa a ser **Administrado** cuando persiste en la base de datos a través del método `EntityManager's` persist, que debe ser invocado dentro de una transacción activa. Al confirmar la transacción, el `EntityManager` propietario almacena el nuevo objeto de entidad en la base de datos.

Los objetos de entidad recuperados de la base de datos por un "EntityManager" también están en el estado **Administrado**.

Si un objeto de entidad administrada se modifica dentro de una transacción activa, el "EntityManager" propietario detecta el cambio y la actualización se propaga a la base de datos al confirmar la transacción.

Un objeto de entidad administrada también se puede recuperar de la base de datos y marcar para su eliminación, utilizando el método remove dentro de una transacción activa. El objeto de entidad cambia su estado de Administrado a **Eliminado** y se elimina físicamente de la base de datos durante la confirmación de la transacción.

El último estado, **Separado**, representa objetos de entidad que se han desconectado del `EntityManager`. Por ejemplo, todos los objetos administrados de un "EntityManager" se desconectan cuando se cierra el "EntityManager".

El contexto de persistencia

El contexto de persistencia es la colección de todos los objetos administrados de un "EntityManager". Si un objeto de entidad que debe recuperarse ya existe en el contexto de persistencia, el objeto de entidad gestionado existente se devuelve sin acceder realmente a la base de datos (excepto la recuperación mediante refresh, que siempre requiere acceder a la base de datos).

El contexto de persistencia se puede borrar utilizando el método clear:

```
1  em.clear();
```

Cuando se borra el contexto de persistencia, todas sus entidades administradas se desconectan y se descarta cualquier cambio en los objetos de entidad que no se hayan vaciado a la base de datos.

Clases de entidad `JPA`

CLASES DE ENTIDAD

Una clase de entidad es una clase Java ordinaria definida por el usuario cuyas instancias se pueden almacenar en la base de datos. La forma sencilla de declarar una clase como entidad es marcarla con la anotación @Entity.

Requisitos de clase de entidad

Una clase de entidad JPA persistente:

- debe ser una clase de nivel superior (es decir, no una clase anidada/interna).
- debe tener un constructor sin argumentos `público` o `protegido`.
- no puede ser `final` y no puede tener métodos "finales" o variables de instancia "finales".

`ObjectDB` es un poco menos restrictivo:

- Se permiten clases de entidades anidadas estáticas (se prohíben las clases internas no estáticas).
- Las variables de instancia (no estáticas) no pueden ser finales, pero las clases y métodos pueden ser finales.
- En la mayoría de los casos, `ObjectDB` puede superar la falta de un constructor sin argumentos.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Aparte de estas restricciones, una clase de entidad es como cualquier otra clase de Java. Puede extender otra clase de entidad o una clase definida por el usuario que no sea una entidad (pero no clases del sistema, como `ArrayList`) e implementar cualquier interfaz. Puede contener constructores, métodos, campos y tipos anidados con cualquier modificador de acceso (público, protegido, paquete o privado) y puede ser concreta o abstracta.

TIPOS DE DATOS JAVA SIMPLES

Todos los siguientes tipos de datos Java simples son persistentes:

- Tipos primitivos: `boolean`, `byte`, `short`, `char`, `int`, `long`, `float` y `double`.
- Clases contenedoras (`Wrappers`) equivalentes del paquete `java.lang`: `Boolean`, `Byte`, `Short`, `Char`, `Integer`, `Long`, `Float` y `Double`.
- `java.math.BigInteger`, `java.math.BigDecimal`.
- `java.lang.String`.
- `java.util.Date`, `java.util.Calendar`, `java.sql.Date`, `java.sql.Time`, `java.sql.Timestamp`.

Los tipos de fecha y hora se analizan con más detalle en el siguiente párrafo.

TIPOS DE FECHA Y HORA (TEMPORALES)

Las clases de fecha y hora `java.sql` representan diferentes partes de fechas y horas:

- `java.sql.Date`: representa solo la fecha (por ejemplo, `2019-12-31`).
- `java.sql.Time`: representa solo el tiempo (por ejemplo, `23:59:59`).
- `java.sql.Timestamp`: representa la fecha y la hora (por ejemplo, `2019-12-31 23:59:59`). Los tipos `java.util.Date` y `java.util.Calendar`, por otro lado, son genéricos y pueden representar cualquiera de los anteriores, usando la anotación @Temporal.

Fechas puras persistentes (sin la parte de tiempo), ya sea usando el tipo `java.sql.Date` o especificando @Temporal La anotación TemporalType.DATE tiene varios beneficios:

- Ahorra espacio en la base de datos.
- Es más eficiente (el almacenamiento y la recuperación son más rápidos).
- Simplifica las consultas sobre fechas y rangos de fechas.

Cuando se almacena una entidad, sus campos de fecha y hora se ajustan automáticamente al modo solicitado. Por ejemplo, los campos `fecha1`, `fecha2` y `fecha3` anteriores pueden inicializarse como `new Date()`, es decir, con fecha y hora. Su parte de tiempo se descarta cuando se almacenan en la base de datos.

Puedes ver el uso completo de `@Temporal` en el Ejemplo04.

TIPOS DE VALORES MÚLTIPLES

Los siguientes tipos de valores múltiples son persistentes:

- Tipos de colección del paquete `java.util`: `ArrayList`, `Vector`, `Stack`, `LinkedList`, `ArrayDeque`, `PriorityQueue`, `HashSet`, `LinkedHashSet`, `TreeSet`.
- Tipos de mapas del paquete `java.util`: `HashMap`, `HashTable`, `WeakHashMap`, `IdentityHashMap`, `LinkedHashMap`, `TreeMap` y `Properties`.
- Arrays (incluidos los multidimensionales (Matrices)).

Se admiten tipos de colecciones y mapas tanto genéricos (por ejemplo, `ArrayList<String>`) como no genéricos (por ejemplo, `ArrayList`), siempre que sus valores (es decir, elementos en colecciones y matrices y claves y valores en mapas) sean valores nulos o instancias de tipos persistentes.

Además de las clases de colección y mapa que son totalmente compatibles con `ObjectDB`, se puede utilizar cualquier otra clase que implemente `java.util.Collection` o `java.util.Map` al almacenar entidades. Si se utiliza una colección o un tipo de mapa no compatible, `ObjectDB` cambiará a un tipo compatible similar cuando se recuperen los datos de la base de datos.

Por ejemplo, el método `Arrays.asList` devuelve una instancia de un tipo de colección Java interna que no es compatible con `ObjectDB`. Sin embargo, se puede almacenar la siguiente entidad:

```
1  private List<String> palabras = Arrays.asList("casa", "manzana", "perro", "gato");
```

Cuando la entidad se recupera de la base de datos, la lista se creará como una "ArrayList". En este caso, es esencial usar una interfaz (`List<String>`) para el tipo de campo para permitir el cambio al tipo de colección admitido cuando se recupera la

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

entidad. En realidad, JPA requiere declarar campos de mapas y colecciones persistentes solo como tipos de interfaz (es decir, `java.util.Collection`, `java.util.List`, `java.util.Set`, `java.util.Map`), y Esa también es una buena práctica cuando se trabaja con `ObjectDB`.

Campos de entidad JPA

Los campos de clases persistentes definidas por el usuario (clases de entidad) se pueden clasificar en los siguientes cinco grupos:

- Campos *transient*
- Campos persistentes
- Compos incrustables
- Campos inversos (mapeados por)
- Campos de clave primaria (ID)
- Campo de versión

CAMPOS *TRANSIENT*

Son campos que no participan en la persistencia y sus valores nunca se almacenan en la base de datos (similar a los campos *transient* en Java que no participan en la serialización). Los campos de entidad estática y final siempre se consideran *transient*. Otros campos se pueden declarar explícitamente como *transient* usando el modificador `transient` de Java (que también afecta la serialización) o la anotación JPA @Transient (que sólo afecta la persistencia).

El ejemplo anterior contiene solo campos de entidad *transient* (no persistentes) sin contenido real para almacenar en la base de datos.

CAMPOS PERSISTENTES

Cada campo de entidad no estático y no final es persistente de forma predeterminada a menos que se especifique explícitamente lo contrario (por ejemplo, mediante el uso de la anotación @Transient).

Almacenar un objeto de entidad en la base de datos no almacena métodos ni código. Sólo se almacena el estado persistente del objeto de entidad, tal como lo reflejan sus campos persistentes (incluidos los campos persistentes que se heredan de las clases antecesoras).

Cuando un objeto de entidad se almacena en la base de datos, cada campo persistente debe contener "nulo" o un valor de uno de los tipos persistentes admitidos. `ObjectDB` admite campos persistentes con cualquier tipo estático declarado, incluido un `java.lang.Object` genérico, siempre que el tipo del valor real en tiempo de ejecución sea persistente (o `null`).

Cada campo persistente se puede marcar con una de las siguientes anotaciones:

- OneToOne, ManyToOne - para referencias de tipos de entidad.
- OneToMany, ManyToMany - para colecciones y mapas de tipos de entidades.
- Básico para cualquier otro tipo persistente. En JPA solo Básico es opcional, mientras que las otras anotaciones anteriores son obligatorias cuando corresponda. `ObjectDB`, sin embargo, no exige el uso de ninguna de estas anotaciones, por lo que son útiles sólo para clases que también se utilizan con un proveedor ORM JPA (como Hibernate) o para cambiar la configuración de campo predeterminada.

La declaración de clase de entidad anterior demuestra el uso de anotaciones de campos y relaciones para cambiar el comportamiento predeterminado. Los valores "nulo" están permitidos de forma predeterminada. Al especificar "`optional=false`" (como se ha visto para "campo1") se genera una excepción en cualquier intento de almacenar una entidad con un valor "nulo" en ese campo. Las configuraciones de cascada y recuperación se explican más adelante.

CAMPOS INCRUSTABLES

Especifica un campo o propiedad persistente de una entidad cuyo valor es una instancia de una clase integrable. La clase integrable debe estar anotada como Integrable.

Estan muy relacionados con las clases incrustables.

CLASES INCRUSTABLES (EMBEDDABLE CLASSES)

Las clases incrustables son clases persistentes definidas por el usuario que funcionan como tipos. Al igual que con otros tipos que no son de entidad, las instancias de una clase **incrustable** solo pueden almacenarse en la base de datos como objetos **embedded**, es decir, como parte de un objeto de entidad.

Una clase se declara como **incrustable** marcándola con la anotación Embeddable. Y desde el punto de vista de la entidad que incluye este campo usaremos `@Embedded`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Las instancias de clases integrables siempre se integran en otros objetos de entidad y no requieren asignación de espacio ni operaciones de almacenamiento y recuperación independientes. Por lo tanto, el uso de clases integrables puede ahorrar espacio en la base de datos y mejorar la eficiencia.

Sin embargo, las clases integrables no tienen una identidad (clave principal) propia, lo que conlleva algunas limitaciones (por ejemplo, sus instancias no pueden ser compartidas por diferentes objetos de entidad y no pueden consultarse directamente). Por lo tanto, la decisión de declarar una clase como entidad o integrable debe analizarse caso por caso.

Observa el Ejemplo05 para ver ejemplos de campos transient, embedded y embeddable.

CAMPOS INVERSOS

Los campos inversos (o mapeados por) contienen datos que no se almacenan como parte de la entidad en la base de datos, pero que aún están disponibles después de su recuperación mediante una consulta automática especial.

> ⚠ **Rendimiento**
>
> La navegación a través de campos inversos es **mucho menos eficiente** que la navegación a través de campos persistentes ordinarios, ya que requiere la ejecución de consultas. Los campos inversos son esenciales para los campos de colección cuando se usan implementaciones ORM JPA, **pero no cuando se usa `ObjectDB`**.

CAMPO DE VERSIÓN

`ObjectDB` mantiene un número de versión para cada objeto de entidad. La versión inicial de un nuevo objeto de entidad (cuando se almacena en la base de datos por primera vez) es 1. Por cada transacción en la que se modifica un objeto de entidad, su número de versión aumenta automáticamente en uno.

Puede exponer las versiones de los objetos de entidad y hacer que sus valores sean accesibles para su aplicación marcando los campos de versión en sus clases de entidad con la anotación @Version. Además, un campo de versión debe tener un tipo numérico:

```
1  @Version long version;
```

Si existe un campo de versión, `ObjectDB` inyecta automáticamente el valor de la versión en ese campo. Los campos de versión deben ser tratados como de solo lectura por la aplicación y no se deben escribir métodos mutadores en ellos. Solo se permite un campo de versión por clase de entidad. De hecho, un único campo de versión por jerarquía de clases de entidad es suficiente porque las subclases heredan un campo de versión.

Puedes ver un ejemplo completo de clave primaria y campo de versión en el Ejemplo06.

ACCESO A LAS PROPIEDADES (ATRIBUTOS)

Cuando una entidad se almacena en la base de datos, los datos se extraen de los campos persistentes de esa entidad. Asimismo, cuando se recupera una entidad, los campos persistentes se inicializan con datos de la base de datos.

De forma predeterminada, `ObjectDB` accede a los campos directamente, pero también se admite el acceso a los campos indirectamente como propiedades utilizando los métodos `get` y `set`. En algunas implementaciones de `JPA`, como Hibernate, el uso del acceso a propiedades puede tener algunos beneficios de rendimiento. Este no es el caso de `ObjectDB`. Por lo tanto, considerando la complejidad adicional que implica configurar y mantener el acceso a la propiedad, generalmente se prefiere el modo de acceso al campo predeterminado.

Clave primaria `JPA`

Cada objeto de entidad que se almacena en la base de datos tiene una clave principal. Una vez asignada, la clave principal no se puede modificar. Representa el objeto de entidad mientras exista en la base de datos.

Como base de datos de objetos, `ObjectDB` admite ID de objetos implícitos, por lo que no se requiere una clave primaria definida explícitamente. Pero `ObjectDB` también admite claves primarias JPA estándar explícitas, incluidas claves primarias compuestas y automática generación de valores secuenciales. Esta es una característica muy poderosa de `ObjectDB` que está ausente en otras bases de datos orientadas a objetos.

IDENTIFICACIÓN DE ENTIDAD

Cada objeto de entidad en la base de datos se identifica de forma única (y puede recuperarse de la base de datos) mediante la combinación de su tipo y su clave principal. Los valores de clave principal son únicos por clase de entidad. Sin embargo, las instancias de diferentes clases de entidades pueden compartir el mismo valor de clave principal.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Sólo los objetos de entidad tienen claves primarias. Las instancias de otros tipos persistentes siempre se almacenan como parte de los objetos de entidad que los contienen y no tienen su propia identidad separada.

CLAVE PRIMARIA AUTOMÁTICA

De forma predeterminada, la clave principal es un número secuencial de 64 bits (`long`) que `ObjectDB` establece automáticamente para cada nuevo objeto de entidad que se almacena en la base de datos. La clave principal del primer objeto de entidad en la base de datos es 1, la clave principal del segundo objeto de entidad es 2, etc. Los valores de clave principal **no** se reciclan cuando los objetos de entidad se eliminan de la base de datos.

Se puede acceder al valor de clave principal de una entidad declarando un campo de clave principal:

```
1  @Id @GeneratedValue long id; // Es generado por ObjectDB
```

La anotación @Id marca un campo como campo de clave principal. Cuando se define un campo de clave principal, `ObjectDB` inyecta automáticamente el valor de la clave principal en ese campo.

La anotación @GeneratedValue especifica que `ObjectDB` asigna automáticamente la clave principal.

CLAVE PRINCIPAL DEL CONJUNTO DE APLICACIONES

Si una entidad tiene un campo de clave principal que no está marcado con @GeneratedValue, el valor de clave principal automático no se genera y la aplicación es responsable de establezca una clave principal inicializando el campo de clave principal. Esto debe hacerse antes de cualquier intento de persistir el objeto de entidad:

```
1  @Id long id2; // Es generado por la aplicación
```

Operaciones CRUD con `JPA`

ALMACENAMIENTO DE OBJETOS DE ENTIDAD `JPA`

Persistencia explícita

El siguiente código almacena una instancia de la clase de entidad "Empleado" en la base de datos. Observa el Ejemplo07 para más detalles.

La instancia `Empleado` se construye como un objeto Java ordinario y su estado inicial es Nuevo. Una llamada explícita a persist asocia el objeto con un EntityManager `em` y cambia su estado a **Administrado**. El nuevo objeto de entidad se almacena en la base de datos cuando se confirma la transacción.

Objetos incrustados referenciados

El siguiente código almacena una instancia de "`Empleado`" con una referencia a una instancia de "`Direccion`". Consulta de nuevo el Ejemplo07 para ver ambos casos completos.

Las instancias de tipos persistentes que no sean clases de entidad se almacenan automáticamente incrustadas en los objetos de entidad que las contienen. Por lo tanto, si `Dirección` se define como una clase incrustable, el objeto de entidad `Empleado` se almacena automáticamente en la base de datos con su instancia de `Direccion` como un objeto incrustado.

Tenga en cuenta que los objetos incrustados no pueden ser compartidos por varios objetos de entidad. Cada objeto de entidad contenedora debe tener sus propios objetos incrustados.

RECUPERAR OBJETOS DE ENTIDAD `JPA`

La API de persistencia de Java (JPA) proporciona varias formas de recuperar objetos de la base de datos. La recuperación de objetos no requiere una transacción activa porque no cambia el contenido de la base de datos.

Recuperación por clase y clave principal

Cada objeto de entidad puede identificarse y recuperarse de forma única mediante la combinación de su clase y su clave principal. Dado un EntityManager `em` , el siguiente fragmento de código demuestra la recuperación de un objeto "`Empleado`" cuya clave principal es 3. Consulta el Ejemplo08 para más detalles sobre recuperación y fetch types.

No es necesario convertir el objeto recuperado a `Empleado` porque find se define como devolver una instancia de la misma clase que toma como primer argumento (usando genéricos).

Recuperación mediante Eager Fetch (Recuperación ansiosa)

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

La recuperación de un objeto de entidad de la base de datos puede provocar la recuperación automática de objetos de entidad adicionales. De forma predeterminada, una operación de recuperación se realiza automáticamente en cascada a través de todos los campos persistentes que no son de colección y de mapa (es decir, a través de relaciones uno a uno y muchos a uno). Por lo tanto, cuando se recupera un objeto de entidad, también se recuperan todos los objetos de entidad a los que se puede acceder desde él mediante la navegación a través de campos persistentes de mapa y no de colección. En teoría, en algunas situaciones extremas esto podría provocar la recuperación de toda la base de datos en la memoria, lo que suele ser inaceptable.

Un campo de referencia persistente se puede excluir de esta recuperación automática en cascada mediante el uso de un tipo de recuperación diferida (**Lazy Fetch**). El valor predeterminado para campos no mapeados y que no son de colección es FetchType.EAGER, lo que indica que la operación de recuperación se realiza en cascada a través del campo. Especificando explícitamente FetchType.LAZY en ya sea en las anotaciones @OneToOne o @ManyToOne excluye el campo de participar en la recuperación en cascada.

Por otro lado, la política de recuperación predeterminada de los campos mapeados y colecciones persistentes es `FetchType.LAZY`. Por lo tanto, de forma predeterminada, cuando se recupera un objeto de entidad, cualquier otro objeto de entidad a los que haga referencia a través de su colección y los campos de mapa no se recuperan con él.

Esto se puede cambiar mediante una configuración explícita de `FetchType.EAGER`.

Recuperación por Navegación y Acceso

Se puede acceder libremente a todos los campos persistentes de un objeto de entidad, independientemente de la política de recuperación actual, siempre que el "EntityManager" esté abierto. Esto también incluye campos que hacen referencia a objetos de entidad que aún no se han cargado desde la base de datos y están representados por objetos vacios. Si el `EntityManager` está abierto cuando se accede por primera vez a un objeto vacio, su contenido se recupera automáticamente de la base de datos y todos sus campos persistentes se inicializan.

Desde el punto de vista del desarrollador, parece que todo el esquema de objetos está presente en la memoria. Esta ilusión, que se basa en la activación y recuperación transparente y diferida de objetos por parte de `ObjectDB`, ayuda a ocultar parte de la interacción directa con la base de datos y facilita la programación de la base de datos.

Acceder a un campo persistente en un objeto vacio (por ejemplo, el nombre del "gerente" en el ejemplo anterior) provoca la recuperación del contenido faltante de la base de datos y la inicialización de todos los campos persistentes.

Como se ve, todo el esquema de objetos está disponible para la navegación, independientemente de la política de recuperación. Sin embargo, la política de recuperación sí afecta el rendimiento.

ACTUALIZACIÓN DE OBJETOS DE ENTIDAD JPA

La modificación de objetos de entidad existentes que están almacenados en la base de datos se basa en la persistencia transparente, lo que significa que los cambios se detectan y manejan automáticamente.

Actualización transparente

Una vez que un objeto de entidad se recupera de la base de datos (sin importar de qué manera), simplemente se puede modificar en la memoria desde dentro de una transacción activa. Observa el Ejemplo09 para un ejemplo completo.

El objeto de entidad se actualiza físicamente en la base de datos cuando se confirma la transacción. Si la transacción se revierte y no se confirma, la actualización se descarta.

Seguimiento automático de cambios

Como se muestra arriba, una actualización se logra modificando un objeto de entidad gestionada desde dentro de una transacción activa. No se invoca ningún método de EntityManager para informar la actualización. Por lo tanto, para poder aplicar actualizaciones de la base de datos al confirmar, `ObjectDB` debe detectar automáticamente los cambios en las entidades administradas.

Sin embargo, detectar cambios en las matrices requiere el uso de instantáneas incluso si las clases de entidad están mejoradas. Por lo tanto, por motivos de eficiencia, el comportamiento predeterminado de ObjectDB ignora los cambios de matriz cuando se utilizan clases de entidad mejoradas.

Generalmente se recomienda utilizar colecciones en lugar de matrices cuando se utiliza JPA. Las colecciones son más portátiles para las implementaciones ORM JPA y brindan un mejor soporte de seguimiento automático de cambios.

ELIMINACIÓN DE OBJETOS DE ENTIDAD JPA

Los objetos de entidad existentes se pueden eliminar de la base de datos ya sea explícitamente invocando el método remove o implícitamente como resultado de una operación en cascada.

Eliminación explícita

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Para eliminar un objeto de la base de datos, primero debe recuperarse (sin importar de qué manera) y luego, en una transacción activa, se puede eliminar usando eliminar. Consulta el Ejemplo10 para más detalles.

El objeto de entidad se elimina físicamente de la base de datos cuando se confirma la transacción. También se eliminan los objetos incrustados contenidos en el objeto de entidad. Si la transacción se revierte y no se confirma, el objeto no se elimina.

Consultas JPA

API DE CONSULTA JPA

Las consultas se representan en JPA 2 mediante dos interfaces: la antigua interfaz Query, que era la única interfaz disponible para representar consultas en JPA 1, y la nueva interfaz TypedQuery que se introdujo en JPA 2. La interfaz TypedQuery amplía la interfaz Query.

En JPA 2, la interfaz Query debe usarse principalmente cuando el tipo de resultado de la consulta es desconocido o cuando una consulta devuelve resultados polimórficos y el mínimo común conocido. El denominador de todos los objetos resultantes es "Objeto". Cuando se espera un tipo de resultado más específico, las consultas normalmente deben utilizar la interfaz TypedQuery. Es más fácil ejecutar consultas y procesar los resultados de la consulta de forma segura cuando se utiliza la interfaz `TypedQuery`.

Creación de consultas con createQuery

Como ocurre con la mayoría de las otras operaciones en JPA, el uso de consultas comienza con un EntityManager (representado por `em` en los siguientes fragmentos de código), que sirve como fábrica tanto para Query como para TypedQuery. Observa el Ejemplo11 para consultas JPQL básicas.

```
Query q1 = em.createQuery("SELECT e FROM Empleado e");
List results1 = q1.getResultList();

TypedQuery<Empleado> q2 = em.createQuery("SELECT e FROM Empleado e", Empleado.class);
List<Empleado> results2 = q2.getResultList();
```

En el código anterior, la misma consulta JPQL que recupera todos los objetos "Empleado" en la base de datos está representada por "q1" y "q2". Al crear una instancia de TypedQuery, el tipo de resultado esperado debe pasarse como argumento adicional, como se demostró para `q2`. Debido a que, en este caso, se conoce el tipo de resultado (la consulta devuelve sólo objetos `Empleado`), se prefiere una `TypedQuery`.

Hay otra ventaja de utilizar `TypedQuery` en ObjectDB. En el contexto de las consultas anteriores, si aún no hay instancias de "Empleado" en la base de datos y se desconoce la clase "Empleado" como una clase de entidad administrada - sólo la variante `TypedQuery` es válida porque introduce la clase `Country` en ObjectDB.

JPQL dinámico, API de criterios y consultas con nombre

La creación de consultas pasando cadenas de consulta JPQL directamente al método `createQuery`, como se muestra arriba, se denomina en JPA construcción de consultas dinámicas porque la cadena de consulta se puede crear dinámicamente en tiempo de ejecución.

La API de criterios JPA proporciona una forma alternativa de crear consultas dinámicas, basadas en objetos Java que representan elementos de consulta (reemplazando JPQL basado en cadenas).

JPA también proporciona una forma de crear consultas estáticas, como consultas con nombre, utilizando @NamedQuery y anotaciones @NamedQueries. Se considera una buena práctica en JPA preferir consultas con nombre a consultas dinámicas cuando sea posible.

EJECUCIÓN DE CONSULTAS JPA

La interfaz Query define dos métodos para ejecutar consultas SELECT:

- Query.getSingleResult: para usar cuando se espera exactamente un objeto de resultado.
- Query.getResultList - para uso general en cualquier otro caso.

De manera similar, la interfaz TypedQuery define los siguientes métodos:

- TypedQuery.getSingleResult: para usar cuando se espera exactamente un objeto de resultado.
- TypedQuery.getResultList - para uso general en cualquier otro caso.

Además, la interfaz de consulta define un método para ejecutar consultas DELETE y UPDATE:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

- Query.executeUpdate: para ejecutar solo consultas DELETE y UPDATE. **Ejecución de consulta ordinaria (con getResultList)**

La siguiente consulta recupera todos los objetos Empleado en la base de datos. La consulta debe ejecutarse utilizando el método getResultList, ya que esperamos recibir múltiples objetos a cambio.

Tanto Query como TypedQuery definen un `getResultList` método, pero la versión de Query devuelve una lista de resultados de un tipo sin formato (no genérico) en lugar de un tipo parametrizado (genérico).

Un intento de convertir los `resultados` anteriores a un tipo parametrizado (`List<Country>`) provocará una advertencia de compilación. Sin embargo, si se utiliza la nueva interfaz TypedQuery, la conversión no es necesaria y se evita la advertencia.

La colección de resultados de la consulta funciona como cualquier otra colección Java normal. Por ejemplo, una colección de resultados de un tipo parametrizado se puede iterar fácilmente utilizando un bucle for mejorado.

Tenga en cuenta que para imprimir solo los nombres de los Empleados, sería necesaria una consulta utilizando projection y recuperando los nombres de los empleados directamente en lugar de recuperar las instancias completas del Empleado. más eficiente.

Ejecución de consulta de resultado único (con `getSingleResult`)

El método getResultList (que se analizó anteriormente) también se puede utilizar para ejecutar consultas que devuelven un único objeto de resultado. En este caso, el objeto de resultado debe extraerse de la colección de resultados después de la ejecución de la consulta (por ejemplo, mediante `results.get(0)`). Para eliminar esta operación de rutina, JPA proporciona un método adicional, getSingleResult, como método más conveniente cuando se espera exactamente un objeto de resultado.

La siguiente consulta agregada siempre devuelve un único objeto de resultado, que es un objeto "Long" que refleja el número de objetos "Empleado" en la base de datos.

Tenga en cuenta que cuando una consulta devuelve un único objeto, puede resultar tentador utilizar Query en lugar de TypedQuery, incluso cuando se conoce el tipo de resultado, porque la conversión de un solo objeto es fácil y el código es simple.

Una consulta `COUNT` agregada siempre devuelve un resultado, por definición. En otros casos, nuestra expectativa de obtener un resultado de objeto único podría fallar, dependiendo del contenido de la base de datos.

Sin embargo, la exactitud de esta suposición depende del contenido de la base de datos. Si la base de datos contiene varios objetos "Empleado" con el nombre "David", se genera una NonUniqueResultException. Por otro lado, si no hay ningún resultado, se genera una NoResultException. Por lo tanto, el uso de getSingleResult requiere cierta precaución y, si existe alguna posibilidad de que se produzcan estas excepciones, deben detectarse y manejarse.

ELIMINAR y ACTUALIZAR ejecución de consultas (con ejecutarActualización)

Consultas ELIMINAR y ACTUALIZAR se ejecutan utilizando el método executeUpdate.

Se genera una TransactionRequiredException si no hay ninguna transacción activa.

En caso de éxito, el método `executeUpdate` devuelve la cantidad de objetos que la consulta actualizó o eliminó.

Consulta el Ejemplo13 para ver operaciones DELETE y UPDATE mediante `executeUpdate` .

La sección Estructura de consulta explica ELIMINAR y ACTUALIZAR consultas con más detalle.

PARÁMETROS DE CONSULTA EN JPA

Los parámetros de consulta permiten la definición de consultas reutilizables. Estas consultas se pueden ejecutar con diferentes valores de parámetros para obtener diferentes resultados. Ejecutar la misma consulta varias veces con diferentes valores de parámetros (argumentos) es más eficiente que usar una nueva cadena de consulta para cada ejecución de consulta, porque elimina la necesidad de compilaciones repetidas de consultas.

Parámetros con nombre (:nombre)

El siguiente método recupera un objeto "Empleado" de la base de datos por su nombre. Puedes verlo en el Ejemplo12.

La cláusula WHERE reduce los resultados de la consulta a objetos "Empleado" cuyo valor del campo de nombre es igual a ":nombre", que es un parámetro que sirve como marcador de posición para un valor real. Antes de que se pueda ejecutar la consulta, se debe establecer un valor de parámetro utilizando el método setParameter. El método setParameter admite el encadenamiento de métodos (devolviendo el mismo TypedQuery instancia en la que se invocó), por lo que la invocación de getSingleResult se puede encadenar a la misma expresión.

Los parámetros con nombre se pueden identificar fácilmente en una cadena de consulta por su forma especial, que son dos puntos (:) seguidos de un identificador JPQL válido que sirve como nombre del parámetro. JPA no proporciona una API para

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

definir los parámetros explícitamente (excepto cuando se utiliza la API de criterios), por lo que los parámetros de consulta se definen implícitamente al aparecer en la cadena de consulta. El tipo de parámetro se infiere por el contexto. En el ejemplo anterior, una comparación de `:nombre` con un campo cuyo tipo es `String` indica que el tipo de `:nombre` en sí es `String`.

Las consultas pueden incluir varios parámetros y cada parámetro puede aparecer varias veces en la cadena de consulta. Una consulta solo se puede ejecutar después de establecer valores para todos sus parámetros (sin importar el orden).

Parámetros ordinales (`?indice`)

Además de los parámetros con nombre, cuya forma es `:nombre`, JPQL también admite parámetros ordinales, cuya forma es `?index`. El Ejemplo12 muestra ambos tipos.

La forma de los parámetros ordinales es un signo de interrogación (?) seguido de un número int positivo. Aparte de la notación diferente, los parámetros con nombre y los parámetros ordinales son idénticos.

Los parámetros con nombre pueden proporcionar un valor agregado a la claridad de la cadena de consulta (suponiendo que se seleccionen nombres significativos). Por lo tanto, se prefieren a los parámetros ordinales.

Puedes ver ejemplos de ambos tipos de parámetros en el Ejemplo12.

API DE CRITERIOS JPA FRENTE A JPQL

Las consultas JPQL se definen como cadenas, de manera similar a SQL. Las consultas de criterios JPA, por otro lado, se definen mediante la creación de instancias de objetos Java que representan elementos de consulta.

Una ventaja importante de utilizar la API de criterios es que los errores se pueden detectar antes, durante la compilación y no en tiempo de ejecución. Por otro lado, para muchos desarrolladores las consultas JPQL basadas en cadenas, que son muy similares a las consultas SQL, son más fáciles de usar y comprender.

Para consultas estáticas simples, pueden ser preferibles las consultas JPQL basadas en cadenas (por ejemplo, como consultas con nombre). Para consultas dinámicas que se crean en tiempo de ejecución, es posible que se prefiera la API de criterios.

Por ejemplo, se espera que la creación de una consulta dinámica basada en campos que un usuario completa en tiempo de ejecución en un formulario que contiene muchos campos opcionales sea más limpia cuando se utiliza la API de criterios JPA, porque elimina la necesidad de crear la consulta utilizando muchas concatenaciones de cadenas. operaciones.

Las consultas JPQL basadas en cadenas y las consultas basadas en criterios JPA son igualmente poderosas y eficientes. Por tanto, elegir un método u otro también es una cuestión de preferencia personal.

12.1.5 Comparativa BDR vs BDOO vs ORM

Aquí tienes una tabla comparativa con las **características, ventajas e inconvenientes** de las **bases de datos relacionales (RDBMS)**, las **bases de datos de objetos (ODBMS)** y los **ORM (como Hibernate)**:

Aspecto	BD Relacionales (BDR)	BD de Objetos (BDOO)	ORM (ej: Hibernate)
Estructura	Datos en tablas (filas/columnas)	Almacena objetos directamente	Mapea objetos ↔ tablas relacionales
Lenguaje	SQL	OQL (Object Query Language)	API en código OO (Java, C#, etc.)
Modelo de datos	Rígido (esquema definido)	Flexible (objetos dinámicos)	Adapta objetos a esquema relacional
Rendimiento	Bueno para consultas complejas	Rápido en operaciones OO	Overhead por abstracción
Escalabilidad	Vertical (mejor con ACID)	Horizontal (menos común)	Depende de la implementación
Ventajas	- ACID garantizado - Amplio soporte - Madurez	- Sin "impedance mismatch" - Mejor integración OO	- Reduce código SQL - Portabilidad entre BD
Inconvenientes	- Menos flexible para datos complejos - Joins costosos	- Menos adopción - Limitado para informes	- Consultas ineficientes - Curva de aprendizaje
Mejor uso	Datos estructurados, transacciones	Apps con dominio OO complejo	Proyectos OO con BD relacional

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

12.1.6 Ejemplos

Ejemplo01

Definición de clases con ODL.

```
class Cliente (key id) {
     attribute int id;
     attribute String nombre;
     attribute String direccionPostal;
     attribute String eMail;
     attribute String telefono;

     relationship Set<Encargo> encargos;

     String getId();
     ...
}

class Encargo (key id)  {
     attribute int id;
     attribute LocalDate fecha;
     ...
}
```

⬆ Volver a teoría

Ejemplo02

Consultas básicas con OQL.

```
SELECT c.direccionPostal, c.telefono
FROM Clientes c
WHERE c.nombre = "Cliente1"
```

```
SELECT e.fecha
FROM Clientes c, c.encargos e
WHERE c.eMail = "email1@dominio.com"
```

⬆ Volver a teoría

Ejemplo03

Conexión a ObjectDB con JPA (EntityManagerFactory y EntityManager).

```
EntityManagerFactory emf = Persistence.createEntityManagerFactory("objectdb:myDbFile.odb");
```

Apertura y cierre del gestor de entidades:

```
EntityManager em = emf.createEntityManager();
try {
    // TODO: Usar la EntityManager para acceder a la BDOO
}
finally {
    em.close();
}
emf.close();
```

Manejo de transacciones:

```
try {
    em.getTransaction().begin();
    // Operaciones que modifican la base de datos se realizan aquí.
    em.getTransaction().commit();
}
finally {
    if (em.getTransaction().isActive())
        em.getTransaction().rollback();
}
```

⬆ Volver a teoría

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Ejemplo04

Tipos de fecha y hora con `@Temporal`.

```
// Fecha:
java.sql.Date date1;
@Temporal(TemporalType.DATE) java.util.Date date2;
@Temporal(TemporalType.DATE) java.util.Calendar date3;

// Hora:
java.sql.Time time1;
@Temporal(TemporalType.TIME) java.util.Date time2;
@Temporal(TemporalType.TIME) java.util.Calendar time3;

// Fecha y hora:
java.sql.Timestamp dateAndTime1;
@Temporal(TemporalType.TIMESTAMP) java.util.Date dateAndTime2;
@Temporal(TemporalType.TIMESTAMP) java.util.Calendar dateAndTime3;
```

⬆ Volver a teoría

Ejemplo05

Campos `@Transient`, `@Embedded` y clase `@Embeddable`.

Campos no persistentes:

```
static int transient1;              // no persistente por ser estático
final int transient2 = 0;           // no persistente por ser final
transient int transient3;           // no persistente por ser transient
@Transient int transient4;          // no persistente por la etiqueta @Transient
```

Anotaciones de campo y relación:

```
@Basic(optional=false) Integer campo1;
@OneToOne(cascade=CascadeType.ALL) Entidad campo2;
@OneToMany(fetch=FetchType.EAGER) ArrayList<Entidad> campo3;
```

Clase incrustable:

```
@Embeddable
public class Direccion {
    protected String calle;
    protected String ciudad;
    protected String pais;
    protected String codPostal;
}
```

Campo incrustado:

```
@Embedded Direccion direccion;
```

⬆ Volver a teoría

Ejemplo06

Clave primaria con `@Id`, `@GeneratedValue` y `@Version`.

Clave primaria autogenerada:

```
@Id @GeneratedValue long id;    // Generado por ObjectDB
```

Clave primaria asignada por la aplicación:

```
@Id long id2;                   // Generado por la aplicación
```

Campo de versión:

```
@Version long version;
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

⬆ Volver a teoría

Ejemplo07

Almacenamiento de objetos con `em.persist()`.

Persistencia explícita:

```
Empleado empleado = new Empleado("David", "Martinez", "Peña");

em.getTransaction().begin();
em.persist(empleado);
em.getTransaction().commit();
```

Persistencia con objeto incrustado:

```
Empleado empleado2 = new Empleado("David", "Martinez", "Peña");
Direccion direccion = new Direccion("Carlet", "España");
empleado2.setDireccion(direccion);

em.getTransaction().begin();
em.persist(empleado2);
em.getTransaction().commit();
```

⬆ Volver a teoría

Ejemplo08

Recuperación de objetos con `em.find()` y tipos de fetch.

Recuperación por clave primaria:

```
Empleado empleado3 = em.find(Empleado.class, 3);
```

Fetch diferido (lazy):

```
@Entity
public class Empleado {
    [...]
    @ManyToOne(fetch=FetchType.LAZY)
    private Empleado gerente;
    [...]
```

Fetch eager (ansioso):

```
@Entity
public class Empleado {
    [...]
    @ManyToMany(fetch=FetchType.EAGER)
    private ArrayList<Proyecto> proyectos;
    [...]
}
```

Navegación desde un objeto recuperado:

```
Empleado empleado3 = em.find(Empleado.class, 3);
Empleado gerente3 = empleado3.getGerente();
String nombreGerente = gerente3.getNombre();
```

⬆ Volver a teoría

Ejemplo09

Actualización de objetos JPA.

```
Empleado empleado4 = em.find(Empleado.class, 2);
em.getTransaction().begin();
empleado4.setApellido1("Otroapellido");
em.getTransaction().commit();
```

⬆ Volver a teoría

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Ejemplo10

Eliminación de objetos JPA.

```
Empleado empleado5 = em.find(Empleado.class, 1);
em.getTransaction().begin();
em.remove(empleado5);
em.getTransaction().commit();
```

⬆ Volver a teoría

Ejemplo11

Consultas JPQL básicas.

Consulta con `TypedQuery` y `getResultList`:

```
TypedQuery<Empleado> q2 = em.createQuery("SELECT e FROM Empleado e", Empleado.class);
List<Empleado> results2 = q2.getResultList();
```

Consulta con `Query` sin tipo:

```
Query q1 = em.createQuery("SELECT e FROM Empleado e");
List results1 = q1.getResultList();
```

Iteración sobre resultados:

```
for (Empleado e : results2) {
    System.out.println(e.getNombre());
}
```

Consulta con `getSingleResult`:

```
TypedQuery<Long> query3 = em.createQuery("SELECT COUNT(e) FROM Empleado e", Long.class);
long numeroEmpleados = query3.getSingleResult();
```

⬆ Volver a teoría

Ejemplo12

Consultas JPQL con parámetros.

Parámetro con nombre (`:nombre`):

```
public Empleado getEmpleadoPorNombre(EntityManager em, String nombre) {
    TypedQuery<Empleado> query = em.createQuery(
        "SELECT e FROM Empleado e WHERE e.nombre = :nombre", Empleado.class);
    return query.setParameter("nombre", nombre).getSingleResult();
}
```

Parámetro ordinal (`?1`):

```
public Empleado getEmpleadoPorNombreOrdinal(EntityManager em, String nombre) {
    TypedQuery<Empleado> query = em.createQuery(
        "SELECT e FROM Empleado e WHERE e.nombre = ?1", Empleado.class);
    return query.setParameter(1, nombre).getSingleResult();
}
```

⬆ Volver a teoría

Ejemplo13

Consultas DELETE y UPDATE con `executeUpdate`.

Eliminación masiva:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
em.getTransaction().begin();
int count = em.createQuery("DELETE FROM Empleado").executeUpdate();
em.getTransaction().commit();
```

Actualización masiva:

```
em.getTransaction().begin();
int count2 = em.createQuery(
    "UPDATE Empleado SET apellido1 = 'Martinez'").executeUpdate();
em.getTransaction().commit();
```

⬆ Volver a teoría

12.1.7 Resumen — Conceptos clave

Concepto	Definición
BDOO	Base de datos que almacena objetos directamente, sin necesidad de mapeo
ObjectDB	Sistema gestor de bases de datos orientadas a objetos para Java
OQL	Lenguaje de consulta para BDOO similar a SQL
Persistencia	Capacidad de un objeto de mantener su estado más allá de la ejecución
JDO	Java Data Objects — estándar para persistencia de objetos
Entity	Objeto que puede ser persistido en una base de datos

12.1.8 Autoevaluación

- [ ] Entiendo qué es una BDOO y en qué se diferencia de una BDR
- [ ] He instalado ObjectDB correctamente
- [ ] Persisto objetos en ObjectDB
- [ ] Realizo consultas con OQL
- [ ] Actualizo y elimino objetos de la base de datos
- [ ] Conozco los estándares JDO y JPA

12.1.9 Vídeos recomendados

Canal	Vídeo	Contenido
makigas	Serie JDBC Moderno	12 vídeos (2h): driver, conexión, PreparedStatement, transacciones, DataSource
Píldoras Informáticas	Curso Spring — Hibernate (vídeo 46)	Acceso a datos con Hibernate/JPA
YouTube — Varios	Playlist JPA + Hibernate	Curso completo de JPA e Hibernate
ObjectDB	Documentación oficial	Tutoriales y guías de ObjectDB JPA

20 de julio de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

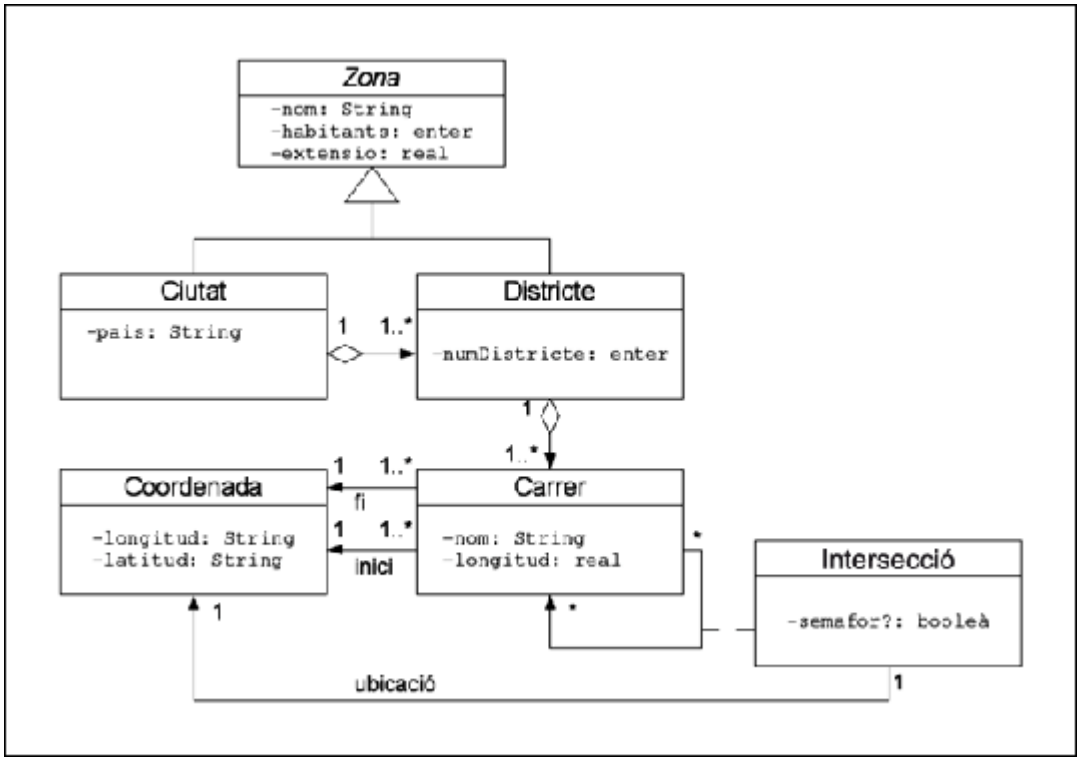
12.2 Ejercicios de la UD11

12.2.1 Actividades

Definición de clases mediante ODL

El objetivo de esta actividad es ver cómo especificar clases usando el lenguaje ODL.

Genere la declaración ODL de las clases del diagrama siguiente:



SOLUCIÓN

```
class Zona(key nom) {
    attribute String nom;
    attribute integer habitants;
    attribute float extensió;
}

class Ciutat extends Zona {
    attribute string pais;
    relationship set<Districte> districtes;
}

class Districte extends Zona {
    attribute integer numDistricte;
    relationship set<Carrer> carrers;
}

class Coordenada {
    attribute String longitud;
    attribute String latitud;
}

class Carrer {
    attribute String nom;
    attribute float longitud;
    relationship <Coordenada> inici;
    relationship <Coordenada> fi;
    relationship Set<Intersecció> interseccions;
}

class Intersecció {
    attribute boolean semàfor;
    relationship <Carrer> carrerIntersectat;
    relationship <Coordenada> ubicació;
}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Instalación y uso de db4o

El objetivo de esta actividad es ver cómo usar las librerías de *db4o* para trabajar con BDOO mediante su IDE.

Vaya a la página de db4o y descargue sus bibliotecas y la documentación. Lea la documentación para ver cómo puede utilizar las bibliotecas para generar programas mediante su IDE. Una vez hecho, ejecute el siguiente código para ver que todo está correcto:

```
import com.db4o.*;

public class ProvaDB4O {

  private class Data {
    private int dia;
    private int mes;
    private int any;

    public Data(int d, int m, int a) {
      dia = d;
      mes = m;
      any = a;
    }

    @Override
    public String toString() {
      return dia + "-" + mes + "-" + any;
    }
  }

  public void start() throws Exception {
    ObjectContainer db = Db4oEmbedded.openFile("BDOOClients.db4o");
    try {
      Data d = new Data(23, 11, 2011);
      db.store(d);
      d = new Data(5, 9, 1975);
      db.store(d);
      d = new Data(4, 9, 1978);
      db.store(d);

      System.out.println("Les dates del setembre són:");
      d = new Data(0,9,0);
      ObjectSet<Data> res = db.queryByExample(d);
      while (res.hasNext()) {
        System.out.println(res.next());
      }

    } finally {
      db.close();
    }
  }

  public static void main (String[] args) throws Exception {
    ProvaDB4O prova = new ProvaDB4O();
    prova.start();
  }

}
```

Gestión de grupos de trabajo de estudiantes mediante db4o

El objetivo de esta actividad es ver cómo gestionar la persistencia de objetos usando la BDOO *db4o*.

Se dispone de las dos clases `Estudiant` y `GrupTreball`, cuyo código se muestra a continuación. A partir de este código, se desea crear una aplicación que, mediante una BDOO db4o gestione a qué grupos de trabajo pertenecen diferentes estudiantes dentro de una escuela. Dado un grupo de trabajo, éste puede tener asignados varios estudiantes, pero todo estudiante sólo tiene un (pero siempre uno) único grupo de trabajo. La aplicación debe poder hacer lo siguiente:

- Dar de alta a un nuevo estudiante. A la hora de asignarle un grupo, si el nombre indicado no existe, se crea un grupo nuevo. Si existe, al estudiante se le asigna ese grupo.
- Reasignar un estudiante a otro grupo de trabajo. Este grupo ya debe existir. Si, al hacerlo, el grupo antiguo queda sin miembros, debe borrarse de la BDOO.
- Listar a todos los grupos existentes.
- Listar a todos los estudiantes (ya qué grupo pertenecen).

Se considera que los nombres de los grupos y estudiantes son únicos en el sistema. No puede haber nombres repetidos. En base a la descripción, también hay que remarcar que la única forma de crear grupos nuevos es añadiendo nuevos estudiantes.

Para realizar esta tarea, el código fuente de estas dos clases no puede modificarse en absoluto.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
public class Estudiant {
  private String nom;
  private GrupTreball grup;

  public Estudiant (String n, GrupTreball gt) {
    nom = n;
    grup = gt;
    grup.sumaEstudiant();
  }

  public String getNom() {
    return nom;
  }

  public GrupTreball getGrupTreball() {
    return grup;
  }

  public void reassignaGrup(GrupTreball gt) {
    if (grup != gt) {
      grup.restaEstudiant();
      grup = gt;
      grup.sumaEstudiant();
    }

  }

  @Override
  public String toString() {
    return nom + " -> " + grup;
  }

}
public class GrupTreball {
  private String nom;
  private String tema;
  private int numEstudiants = 0;

  public GrupTreball(String n, String tm) {
    nom = n;
    tema = tm;
  }

  public void sumaEstudiant() {
    numEstudiants++;
  }

  public void restaEstudiant() {
    numEstudiants--;
  }

  public int getNumEstudiants() {
    return numEstudiants;
  }

  @Override
  public String toString() {
    return nom + " - " + tema;
  }

}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

SOLUCIÓN

```java
import java.util.*;
import com.db4o.*;
import com.db4o.config.EmbeddedConfiguration;
import com.db4o.query.Predicate;

public class GestionarTreballs {

  public void start() {
    EmbeddedConfiguration config = Db4oEmbedded.newConfiguration();
    config.common().objectClass(Estudiant.class).cascadeOnUpdate(true);
    ObjectContainer db = Db4oEmbedded.openFile("BDOOClientsEx3.db4o");

    boolean executar = true;

    while (executar) {
      System.out.println("[1] Alta de nou estudiant.");
      System.out.println("[2] Reassignar el grup d'un estudiant.");
      System.out.println("[3] Llistar Estudiants.");
      System.out.println("[4] Llistar Grups.");
      System.out.println("[0] Sortir.");
      switch(llegirEnter("Tria una opció")) {
        case 0:
          executar = false;
          break;
        case 1:
          altaEstudiant(db);
          break;
        case 2:
          reassignaGrup(db);
          break;
        case 3:
          llistaEstudiants(db);
          break;
        case 4:
          llistaGrups(db);
          break;
        default:
          System.out.println("Opció incorrecta.");
      }
    }

    db.close();
  }

  public void altaEstudiant(ObjectContainer db) {
    final String nom = llegirText("Nom del nou estudiant");

    Predicate p = new Predicate<Estudiant>() {
      @Override
      public boolean match(Estudiant e) {
        //Codi per al criteri de cerca
        return nom.equals(e.getNom());
      }
    };

    ObjectSet<Estudiant> estudiants = db.query(p);

    if (estudiants.size() > 0) {
      System.out.println("Aquest estudiant ja existeix.");
    } else {
      String nomGrup = llegirText("Nom del seu grup");
      GrupTreball tr = new GrupTreball(nomGrup, null);
      ObjectSet<GrupTreball> grups = db.queryByExample(tr);
      GrupTreball nouGrup = null;
      if (grups.size() > 0) {
        nouGrup = grups.next();
        System.out.println("El nou estudiant s'ha afegit a un grup ja existent");
      } else {
        String tema = llegirText("Aquest grup és nou. Temàtica del grup? ");
        nouGrup = new GrupTreball(nomGrup, tema);
      }
      Estudiant estudiant = new Estudiant(nom, nouGrup);
      db.store(estudiant);
    }
  }

  public void reassignaGrup(ObjectContainer db) {
    final String nom = llegirText("Nom del nou estudiant");

    Predicate p = new Predicate<Estudiant>() {
      @Override
      public boolean match(Estudiant e) {
        //Codi per al criteri de cerca
        return nom.equals(e.getNom());
      }
    };
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
    ObjectSet<Estudiant> estudiants = db.query(p);


    if (estudiants.size() == 0) {
      System.out.println("Aquest estudiant no existeix.");
    } else {
      Estudiant estudiant = estudiants.next();
      String nomGrup = llegirText("Nom del seu nou grup");
      GrupTreball tr = new GrupTreball(nomGrup, null);
      ObjectSet<GrupTreball> grups = db.queryByExample(tr);
      if (grups.size() == 0) {
        System.out.println("Aquest grup no existeix.");
      } else {
        GrupTreball nouGrup = grups.next();
        GrupTreball anticGrup = estudiant.getGrupTreball();
        estudiant.reassignaGrup(nouGrup);
        //Queda algú a l'antic grup?
        if (anticGrup.getNumEstudiants() == 0) {
          db.delete(anticGrup);
        }
        db.store(estudiant);
      }
    }
  }

  public void llistaEstudiants(ObjectContainer db) {
    Predicate p = new Predicate<Estudiant>() {
      @Override
      public boolean match(Estudiant e) {
        //Codi pel criteri de cerca
        return true;
      }
    };

    ObjectSet<Estudiant> estudiants = db.query(p);

    while (estudiants.hasNext()) {
      System.out.println(estudiants.next());
    }
  }

  public void llistaGrups(ObjectContainer db) {
    GrupTreball e = new GrupTreball(null, null);
    ObjectSet<GrupTreball> grups = db.queryByExample(e);
    while (grups.hasNext()) {
      System.out.println(grups.next());
    }
  }

  public String llegirText(String pregunta) {
    Scanner in = new Scanner(System.in);
    String res = null;
    while (res == null) {
      System.out.print(pregunta + ": ");
      res = in.next();
      if ("".equals(res))
        res = null;
      in.nextLine();
    }
    return res;
  }

 public int llegirEnter(String pregunta) {
    Scanner in = new Scanner(System.in);
    int res = -1;
    while (res == -1) {
      System.out.print(pregunta + ": ");
      try {
        res = in.nextInt();
      } catch (Exception ex) {
        System.out.println("Entrada invàlida.");
      }
      in.nextLine();
    }
    return res;
  }

  public static void main (String[] args) {
    GestionarTreballs programa = new GestionarTreballs();
    programa.start();
  }
}
```

Gestión de clientes y encargos mediante db4o

El objetivo de esta actividad es ver cómo gestionar la persistencia de objetos usando la BDOO *db4o*.

Haga un programa que gestione los objetos de una base de datos de clientes de acuerdo al siguiente diagrama:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

El programa debe poder dar las siguientes opciones:

- Añadir un nuevo tipo de producto.
- Añadir un nuevo cliente.
- Añadir un nuevo encargo a un cliente; el encargo puede contener más de un tipo de producto asociado.
- Listar todos los tipos de producto en el sistema.
- Listar los datos de un cliente y todos sus encargos hasta ahora.

Los tipos de producto y los clientes se identifican de forma única por el nombre (sólo puede haber un único objeto con un nombre dado). Para este ejercicio no es necesario gestionar la persistencia de actualizaciones entre diferentes ejecuciones.

SOLUCIÓN

```java
public class TipusProducte {
  private String nom;
  private float preu;

  public TipusProducte(String n, float p) {
    nom = n;
    preu = p;
  }

  @Override
  public String toString() {
    return  nom + ":" + preu;
  }
}
```

```java
public class Quantitat {
  private TipusProducte tipusProducte;
  private int nre;

  public Quantitat(TipusProducte tp, int n) {
    tipusProducte = tp;
    nre = n;
  }

  @Override
  public String toString() {
    return tipusProducte.toString() + " - " + nre;
  }
}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
import java.util.*;

public class Client {
  private String nom;
  private String aPostal;
  private String aMail;
  private String telefon;
  private List<Encarrec> liComandes = new LinkedList<Encarrec>();

  public Client(String n, String ap, String am, String t) {
    nom = n;
    aPostal = ap;
    aMail = am;
    telefon = t;
  }

  public String getNom() {
    return nom;
  }

  public String getAPostal() {
    return aPostal;
  }

  public String getAMail() {
    return aMail;
  }

  public String getTelefon() {
    return telefon;
  }

  public int getNreComandes() {
    return liComandes.size();
  }

  public void addComanda(Encarrec e) {
    liComandes.add(e);
  }

  public List<Encarrec> getComandes() {
    return liComandes;
  }

  @Override
  public String toString() {
    String res = nom + " : " + aPostal + " : (" + aMail +
          ", " + telefon + ")\n";
    Iterator<Encarrec> it = liComandes.iterator();
    while (it.hasNext()) {
      Encarrec e = it.next();
      res += e.toString() + "\n";
     }
    return res;
  }
}
```

```java
import java.util.*;

public class Encarrec {

  private Date data;
  private List<Quantitat> productes = new LinkedList();

  public Encarrec () {
    data = new Date();
  }

  public void addProductes(TipusProducte tp, int n) {
    productes.add(new Quantitat(tp, n));
  }

  @Override
  public String toString() {
    String res = "Data = " + data + ":\n" ;
    Iterator<Quantitat> it = productes.iterator();
    while (it.hasNext()) {
      res += it.next().toString() + "\n";
    }
    return res;
  }
}
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
import java.util.*;
import com.db4o.*;
import com.db4o.config.EmbeddedConfiguration;

public class GestionarClients {

  public void start() {
    EmbeddedConfiguration config = Db4oEmbedded.newConfiguration();
    config.common().objectClass(Client.class).cascadeOnUpdate(true);
    config.common().objectClass(Encarrec.class).cascadeOnUpdate(true);
    config.common().objectClass(Quantitat.class).cascadeOnUpdate(true);
    config.common().objectClass(TipusProducte.class).cascadeOnUpdate(true);
    ObjectContainer db = Db4oEmbedded.openFile("BDOOClientsEx4.db4o");

    boolean executar = true;

    while (executar) {
      System.out.println("[1] Afegir Tipus de Producte.");
      System.out.println("[2] Afegir Client.");
      System.out.println("[3] Afegir Encàrrec.");
      System.out.println("[4] Llistar tipus de productes a la BDOO.");
      System.out.println("[5] Mostrar client i els seus encàrrecs.");
      System.out.println("[0] Sortir.");
      switch(llegirEnter("Tria una opció")) {
        case 0:
          executar = false;
          break;
        case 1:
          afegirProducte(db);
          break;
        case 2:
          afegirClient(db);
          break;
        case 3:
          afegirEncarrec(db);
          break;
        case 4:
          llistarProductes(db);
          break;
        case 5:
          llistarClient(db);
          break;
        default:
          System.out.println("Opció incorrecta.");
      }
    }
    db.close();
  }

  public void afegirProducte(ObjectContainer db) {
    String nom = llegirText("Nom del Producte");
    //Cal cercar primer si ja existeix a la BDOO, o se'n crearà una còpia!
    TipusProducte p = new TipusProducte(nom, 0);
    ObjectSet<TipusProducte> prods = db.queryByExample(p);
    if (prods.size() > 0) {
      System.out.println("Aquest tipus de producte ja existeix.");
    } else {
      float preu = llegirReal("Preu");
      p = new TipusProducte(nom, preu);
      db.store(p);
      System.out.println("Producte emmagatzemat");
    }
  }

  public void afegirClient(ObjectContainer db) {
    String nom = llegirText("Nom del Client");
    //Cal cercar primer si ja existeix a la BDOO, o es crearà una còpia!
    Client c = new Client(nom, null, null, null);
    ObjectSet<Client> clis = db.queryByExample(c);
    if (clis.size() > 0) {
      System.out.println("Aquest client ja existeix.");
    } else {
      String ap = llegirText("Adreça");
      String am = llegirText("E-Mail");
      String tl = llegirText("Telefon");
      c = new Client(nom, ap, am, tl);
      db.store(c);
      System.out.println("Client emmagatzemat");
    }
  }
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
  public void afegirEncarrec(ObjectContainer db) {
    String nom = llegirText("Nom del Client");
    //Cal cercar si el client existeix
    Client c = new Client(nom, null, null, null);
    ObjectSet<Client> clis = db.queryByExample(c);
    if (clis.size() == 0) {
      System.out.println("Aquest client no existeix.");
    } else {
      Client client = clis.next();
      boolean afegir = true;
      Encarrec encarrec = new Encarrec();
      while (afegir) {
        nom = llegirText("Nom del Producte (escriu \"fi\" per acabar)");
        if ("fi".equals(nom)) {
          afegir = false;
        } else {
          TipusProducte p = new TipusProducte(nom, 0);
          ObjectSet<TipusProducte> prods = db.queryByExample(p);
          if (prods.size() == 0) {
            System.out.println("Aquest producte no existeix.");
          } else {
            TipusProducte prod = prods.next();
            int nre = llegirEnter("Quantiat");
            encarrec.addProductes(prod, nre);
          }
        }
      }
      client.addComanda(encarrec);
      db.store(client);
      System.out.println("Encàrrec emmagatzemat");
    }
  }

  public void llistarProductes(ObjectContainer db) {
    TipusProducte p = new TipusProducte(null, 0);
    ObjectSet<TipusProducte> prods = db.queryByExample(p);
    System.out.println("Llista de tipus de productes:");
    while (prods.hasNext()) {
      System.out.println(prods.next());
    }
  }

  public void llistarClient(ObjectContainer db) {
    String nom = llegirText("Nom del Client");
    //Cal cercar si el client existeix
    Client c = new Client(nom, null, null, null);
    ObjectSet<Client> clis = db.queryByExample(c);
    if (clis.size() == 0) {
      System.out.println("Aquest client no existeix.");
    } else {
      Client client = clis.next();
      System.out.println(client);
    }
  }

  public String llegirText(String pregunta) {
    Scanner in = new Scanner(System.in);
    String res = null;
    while (res == null) {
      System.out.print(pregunta + ": ");
      res = in.next();
      if ("".equals(res))
        res = null;
      in.nextLine();
    }
    return res;
  }

 public int llegirEnter(String pregunta) {
    Scanner in = new Scanner(System.in);
    int res = -1;
    while (res == -1) {
      System.out.print(pregunta + ": ");
      try {
        res = in.nextInt();
      } catch (Exception ex) {
        System.out.println("Entrada invàlida.");
      }
      in.nextLine();
    }
    return res;
  }
```


© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```
  public float llegirReal(String pregunta) {
     Scanner in = new Scanner(System.in);
     float res = -1;
     while (res == -1) {
       System.out.print(pregunta + ": ");
       try {
         res = in.nextFloat();
       } catch (Exception ex) {
         System.out.println("Entrada invàlida.");
       }
       in.nextLine();
     }
     return res;
   }

   public static void main (String[] args) {
     GestionarClients programa = new GestionarClients();
     programa.start();
   }
}
```

Instalación y uso de JDO

El objetivo de esta actividad es ver cómo usar una BDDOO basada en JDO mediante su IDE.

Vaya a la página de BDOO JDOInstruments y descargue sus bibliotecas y la documentación. Lea la documentación para ver cómo puede utilizar las bibliotecas para generar programas mediante su IDE. Una vez hecho, ejecute correctamente el ejemplo proporcionado con las librerías (en la carpeta “linuxexample” o “windiowsexample”, según su sistema operativo).

12.2.2 Ejercicios

Ejercicio 1

A partir del siguiente fragmento de ODL, que describe en términos generales un juego donde jugadores se desplazan por un laberinto:

```
class Jugador {
  attribute string nom;
  relationship Bag<Item> items;
  relationship <Sala> ubicació;
}

class Item (key nom) {
  attribute string nom;
  attribute float pes;
}

class Sala {
  attribute string nom;
  attribute string descripció;
  relationship Set<Sortida> sortides;
}

class Sortida {
  attribute string nom;
  attribute string descripció;
  relationship <Sortida> onVa;
}
```

Marque si son verdaderas (V) o falsas (F) las siguientes opciones. Siguiendo estrictamente las definiciones...

Núm	Pregunta	V	F
1	Un jugador puede tener varios elementos con el mismo nombre.		f
2	Un jugador puede tener varios ítems con el mismo peso.	v	
3	Una sala puede tener distintas salidas con el mismo nombre.		f
4	Una salida puede ir a parar a varias salas, pero nunca repetidas.		f
5	En el programa puede haber salas con el mismo nombre.	v	
6	En el programa puede haber salas con la misma descripción.	v	
7		v	

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Núm	Pregunta	V	F
	En el programa puede haber jugadores con el mismo nombre.		
8	En el programa puede haber jugadores con un mismo ítem.	v	
9	En el programa puede haber jugadores con la misma ubicación.	v	
10	No puede haber ningún ítem en ninguna sala.	v	

Ejercicio 2

Dada la consulta OQL siguiente:

```
SELECT c.adreçaPostal, c.telefon
DE Clientes c
DONDE c.nom = "Cliente1"
```

Elija la opción correcta. Si se desea obtener estrictamente los mismos datos resultantes...

Núm	Pregunta	
1	En una BDOO *db4o*, es posible hacerlo usando una búsqueda por ejemplo.	
2	En una BDOO *db4o*, es posible hacerlo usando una búsqueda nativa.	
3	En una BDOO basada en JDO, es posible hacerlo usando una búsqueda `query.execute(…)`.	
4	Ninguna de las anteriores afirmaciones es cierta.	esta

Ejercicio 3

Se dispone de la siguiente clase, cuyos objetos están almacenados en una BDOO *db4o*. Para gestionarlos, sólo se desea utilizar búsquedas por ejemplo (*Queries-By-Example*).

```
public class Assignatura {
  private String nom;
  private String tematica;
  private int credits;
  private int nreCopsOberta;
  private List<Estudiants> estudiantsMatriculatsActualment;
}
```

Marque si son verdaderas (V) o falsas (F) las siguientes opciones:

Núm	Pregunta	V	F
1	Es posible buscar la asignatura que tiene un nombre concreto.		f
2	Es posible buscar las asignaturas con una temática concreta.	v	
3	Es posible buscar las asignaturas de seis créditos.	v	
4	Es posible buscar la asignatura con menos créditos.		f
5	Es posible buscar las asignaturas que nunca se han abierto.		f
6	Es posible buscar las asignaturas abiertas al menos una vez.		f
7	Es posible buscar la asignatura con más estudiantes en la actualidad.		f
8	Es posible buscar las asignaturas sin estudiantes en la actualidad.		f

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Ejercicio 4

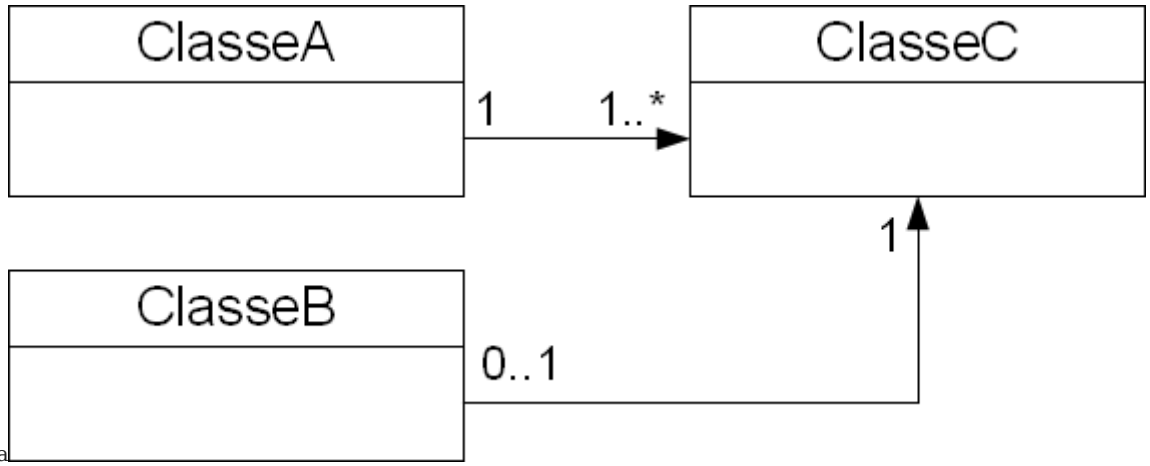
Complete las afirmaciones con la palabra que considere más adecuada:

Núm	Pregunta	Respuesta
1	Al abrir una BDOO *db4o* se obtiene un...	

2	**Al abrir una BDOO JDO se obtiene un...**
3	Para realizar búsquedas simples en una BDOO *db4o* se usa el método...
4	Para realizar búsquedas complejas en una BDOO *db4o* se usa el método...
5	Para realizar búsquedas en una BDOO JDO se usa el método...
6	Para almacenar objetos en una BDOO *db4o* se usa el método...
7	Para almacenar objetos en una BDOO JDO se usa el método...

Ejercicio 5

Dado el siguiente diagrama de clases y fragmento de código de una BDOO *db4o*:



Figura

```
1  private ObjectContainer db = ...;
2  ...
3
4  public void eliminaA(ClasseA a) {
5    db.delete(a);
6    Iterator<ClasseC> it = a.getC().iterator();
7    while (it.hasNext()) {
8      ClasseC c = it.next();
9      db.delete(c);
10   }
11 }
```

Elija la opción correcta:

Núm	Pregunta	
1	Este código es correcto y no crea ninguna inconsistencia en BDOO.	
2	Este código es incorrecto; siempre habrá inconsistencias en la BDOO.	
3	Este código es incorrecto, pero no siempre creará inconsistencias en BDOO.	ESTA
4	Este código es redundante, ya que no es necesario el bucle.	

12.2.3 Ejercicios propuestos

1. Investiga la sintaxis de uso de la API Criteria y propón algún ejemplo.
2. ¿Es posible mezclar parámetros por nombre y parámetros posicionales en una query?

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

3. Investiga sobre el marcado de clases con la etiqueta @Mapped. ¿Qué significado tiene?
4. ¿Para qué se utilizan los scripts enhancer.bat y enhancer.sh?
5. Indica qué significado tiene marcar un campo persistente con alguna de las siguientes marcas: @OneToO0ne, @ManyToOne, @OnetoMany, @ManyToMany, Basic. Propón ejemplos ilustrativos.

12.2.4 Supuestos prácticos

1. Se necesita implementar un programa que administre la información de los vehículos y personas que los conducen cuando entran en un recinto privado. De los vehículos se desea almacenar su identificador (número entero), la matrícula (cadena de caracteres), la fecha de matriculación (fecha) y la persona que lo conduce (clase persona). De los vehículos que sean coches, además, se desea almacenar el tipo de energía que lo propulsa (enumerado: GASOLINA, DIÉSEL, ELÉCTRICO, HÍBRIDO). De las personas que conducen los vehículos se desea almacenar su nombre (cadena de caracteres) y su DNI (cadena de caracteres). De aquellas personas que además sean trabajadores se desea almacenar su número de registro personal (número entero). Implementa las clases entidad que permitan manejar y volcar esta información a una base de datos denominada control, así como un programa que añada los datos de 4 vehículos (2 de ellos coches) y 4 conductores (2 de ellos trabajadores). Utiliza el explorador ObJjectDB Explorer para comprobar que se han introducido correctamente.
2. Sobre el supuesto práctico anterior, añade al programa principal un menú que ofrezca al usuario la posibilidad de dar de alta nuevos vehículos y personas, consultar los datos de los vehículos almacenados en el sistema, modificar los mismos datos y eliminar los datos de determinados vehículos, incluidos coches, a partir de su identificador.
3. Crea un programa que permita migrar los datos de los trabajadores de una empresa contenidos en un fichero de texto a una base de datos ObjectDB. Los campos contenidos en el fichero de texto vienen separados por el carácter 4 y son los siguientes: DNI, Apellido1, Apellido2, Nombre, FechaNacimiento, Fechalncorporación, Puesto. Selecciona el tipo de dato persistente que mejor se adapte a la información a almacenar.
4. A partir de la base de datos del supuesto práctico anterior, escribe un programa que permita recuperar los datos de los trabajadores según determinados parámetros que serán introducidos por teclado conforme a la opción elegida en un menú de opciones. El menú constará de 3 opciones:

- “Filtrar mayores de edad”: Devolverá únicamente los datos de los trabajadores de más de 18 años a fecha actual.
- “Filtrar por DNI igual a...”: Devolverá únicamente los datos del trabajador cuyo DNI coincida con el leído.
- “Filtrar por antigúedad mayor a...”: Devolverá los datos de los trabajadores cuya antigúedad en el puesto sea superior a la leída por teclado.

Emplea @NamedQuery para una de ellas y querys dinámicas con parámetros por nombre para las otras dos.

1. Modifica el programa del supuesto práctico anterior para que añada la opción “Alta trabajador” al menú y permita leer los datos desde teclado para almacenarlos en la base de datos.

12.2.5 Auto evaluación

1. ¿Cuál de las siguientes no es una declaración de tipo persistente?

a) `java.sql.Date` fecha.

b) `@Temporal(TemporalType.DATE) java.util.Date fecha`.

c) `@Temporal(TemporalType.DATE) java.util.Calendar fecha`.

d) Todas las opciones anteriores son declaraciones de tipo persistente.

1. ¿Cuál de los siguientes no es un requisito de una clase entidad?

a) Tener un constructor parametrizado.

b) Sus campos deben ser privados.

c) Sus propiedades deben ser accesibles mediante métodos get y set que siguen una convención de nomenclatura estándar.

d) Ser serializable.

1. ¿Cómo debe marcarse el campo de una clase entidad cuyo tipo sea una clase definida por el programador y del que no se desea que sea almacenado como clase entidad?

a) `@Embedded`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

b) `@Embeddable`.

c) `@Serializable`,

d) `@Entity`

1. De las siguientes estrategias de generación de clave primaria, ¿cuál es equivalente a `@GeneratedValue`?

a) Omitir la marca es la equivalencia.

b) `@GeneratedValue(strategy=GenerationType.IDENTITY)`.

c) `@GeneratedValue(strategy=GenerationType.AUTO)`.

d) `@GeneratedValue(strategy=GenerationType.SEQUENCE, generator="seq")`.

1. ¿En qué parte de una sentencia SELECT se pueden emplear las funciones agregadas? a) SELECT.

b) FROM.

c) AS.

d) Todas las opciones son correctas.

1. ¿Cuál de los siguientes métodos propaga los cambios a la base de datos? a) `persist`.

b) `begin`.

c) `rollback`.

d) `commit`.

1. ¿Cuál de las siguientes query utiliza parámetros posicionales de forma correcta? a) `em.createQuery("SELECT e FROM Compra c WHERE p.precio = *1")`.

b) `em.createQuery("SELECT € FROM Compra c WFIERE p.precio =? 1")`.

c) `em.createQuery("SELECT e FROM Compra c WHERE p.precio = '*1'")`.

d) `em.createQuery(“SELECT € FROM Compra c WHERE p.precio = '?1'”)`.

1. ¿Qué operación del conjunto de operaciones CRUD no necesita de un commit? a) C (*Create*).

b) R (*Read*).

c) U (*Update*).

d) D (*Delete*).

1. ¿Cuál de las siguientes no es una opción válida para establecer la ruta de la base de datos?

a) Utilizar la variable `$objectdb`.

b) Utilizar directamente el path de la base de datos (“`C:\ ...`”).

c) Utilizar una URL.

d) Todas las opciones anteriores son válidas para establecer la ruta de la base de datos.

1. ¿Qué métodos permiten establecer los cambios que se ejecutan como una transacción?

a) `em.getTransaction().begin()`; y `em.getTransaction().end()`. b) `em.getTransaction().start()`; y `em.getTransaction().end()`. c) `em.getTransaction().begin()`; y `em.getTransaction().commit()`. d) `em.getTransaction().start()`; y `em.getTransaction().commit()`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

SOLUCIONES:

1.	a	b	c	**d**	5.	**a**	b	c	d		
2.	**a**	b	c	d	6.	a	b	c	**d**	9.	a	b	c	**d**
3.	**a**	b	c	d	7.	a	**b**	c	d	10.	a	b	c	**d**
4.	a	b	**c**	d	8.	a	**b**	c	d					

10 de enero de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

12.3 Talleres

12.3.1 Taller UD11_02: Añadir ObjectDB a un proyecto IntelliJ (Maven)

Que es ObjectDB

ObjectDB es un potente sistema de gestión de bases de datos orientado a objetos (`ODBMS`). Es compacto, fiable, fácil de usar y extremadamente rápido. **ObjectDB** proporciona todos los servicios estándar de administración de bases de datos (almacenamiento y recuperación, transacciones, administración de bloqueos, procesamiento de consultas, etc.), pero de una manera que facilita el desarrollo y acelera las aplicaciones.

CARACTERÍSTICAS CLAVE DE LA BASE DE DATOS OBJECTDB

- Sistema de gestión de bases de datos orientado a objetos (ODBMS) 100% puro Java.
- Sin API propietaria - administrado únicamente por API de Java estándar (JPA 2 / JDO 2).
- Extremadamente rápido: más rápido que cualquier otro producto JPA/JDO.
- Adecuado para archivos de bases de datos que van desde kilobytes hasta terabytes.
- Admite tanto el modo Cliente-Servidor como el modo Integrado.
- JAR único sin dependencias externas.
- La base de datos se almacena como un único archivo.
- Capacidades avanzadas de consulta e indexación.
- Efectivo en entornos multiusuario con mucha carga.
- Puede integrarse fácilmente en aplicaciones de cualquier tipo y tamaño.
- Probado con Tomcat, Jetty, GlassFish, JBoss y Spring.

ObjectDB se puede descargar y utilizar **sin coste (incluso comercialmente)** con la restricción de un máximo de **10 clases de entidad** y **un millón de objetos de entidad** por archivo de base de datos. Esto podría ser útil para proyectos pequeños, tareas académicas, evaluación y aprendizaje. **ObjectDB** es un software comercial y su uso sin estas restricciones requiere la compra de una licencia.

Se recomienda probar **ObjectDB** antes de comprar una licencia.

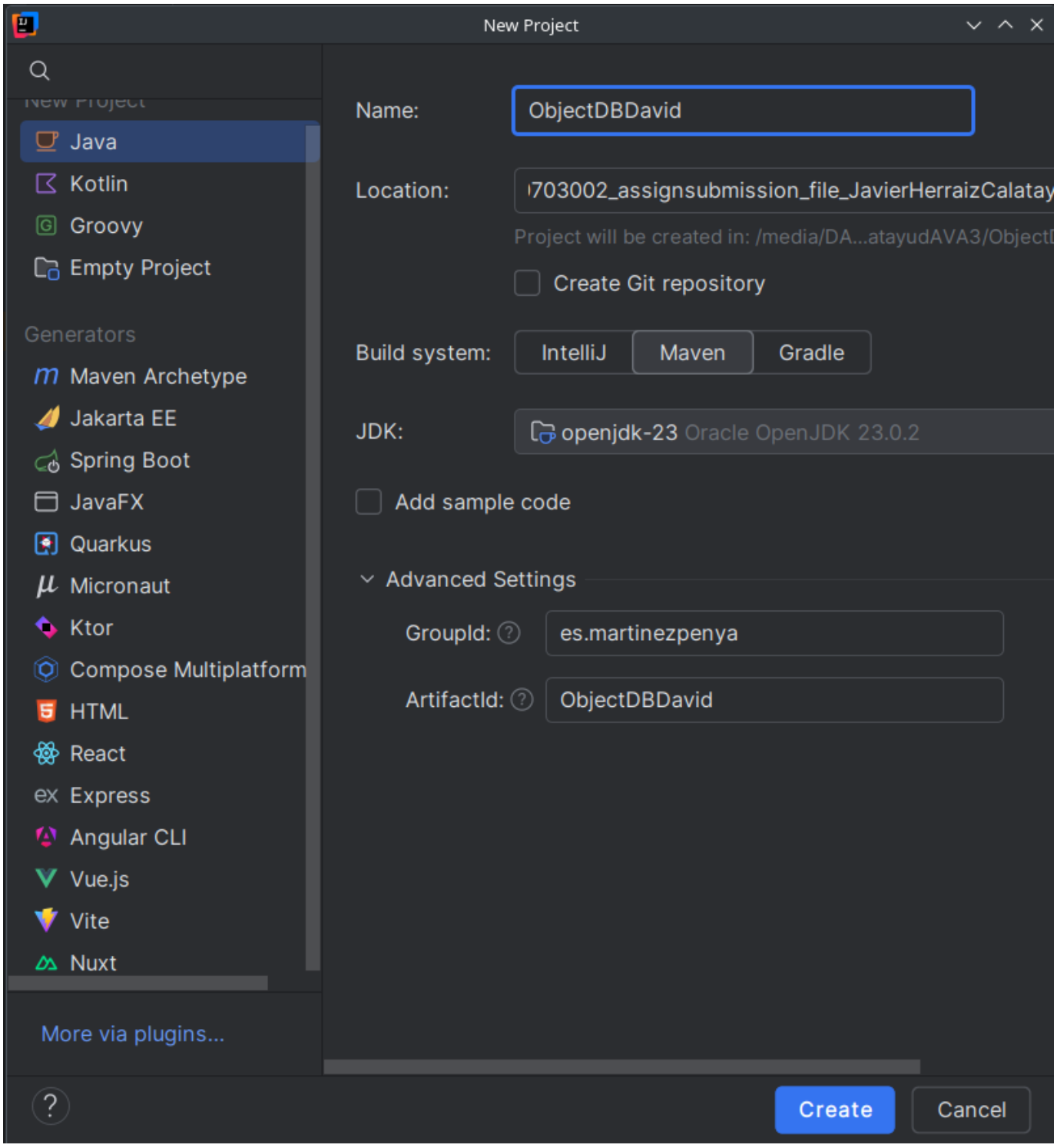
Más información sobre tipos de licencias: https://www.objectdb.com/database/purchase

Añadir ObjectDB mediante Maven

1. Crear un Nuevo Proyecto con Maven:

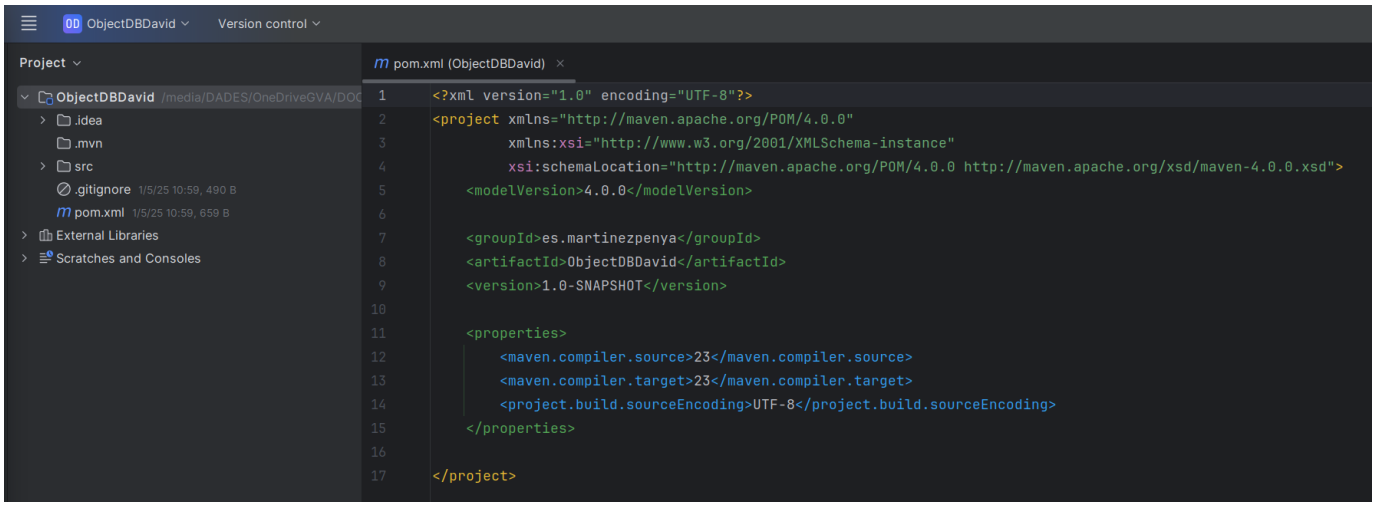
- Al crear un nuevo proyecto en IntelliJ, selecciona "Maven" como sistema de construcción.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



- IntelliJ generará la estructura del proyecto y el archivo `pom.xml`.

© 2025 David Martínez licensed under CC BY-NC-SA 4.0



2. Agregar Dependencias:

- Abre el archivo `pom.xml` y agrega el repositorio y las dependencias necesarias.

```
[...]
    <repositories>
        <repository>
            <id>objectdb</id>
            <name>ObjectDB Repository</name>
            <url>https://m2.objectdb.com</url>
        </repository>
    </repositories>

    <dependencies>
        <dependency>
            <groupId>com.objectdb</groupId>
            <artifactId>objectdb</artifactId>
            <version>2.8.1</version>
        </dependency>
    </dependencies>
[...]
```

3. Actualizar Dependencias:

- Si cambias el archivo `pom.xml`, IntelliJ mostrará un botón para cargar los cambios:

Tu primer proyecto con ObjectDB

En la web del proyecto ObjectDB puedes descargar un tutorial con un proyecto ya creado para ver como funciona, pero nosotros lo vamos a crear desde cero.

1. Código fuente de la clase `Punto.java`

Dentro de la carpeta `src/main/java` crea una nueva clase llamada `Punto.java` con el siguiente contenido:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.Id;

@Entity
public class Punto {
    @Id
    @GeneratedValue
    private long id;

    private int x;
    private int y;

    public Punto() {
    }

    Punto(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public Long getId() {
        return id;
    }
    public int getX() {
        return x;
    }
    public int getY() {
        return y;
    }
    public void setId(long id) {
        this.id = id;
    }
    public void setX(int x) {
        this.x = x;
    }
    public void setY(int y) {
        this.y = y;
    }

    @Override
    public String toString() {
        return String.format("(%d, %d)", this.x, this.y);
    }
}
```

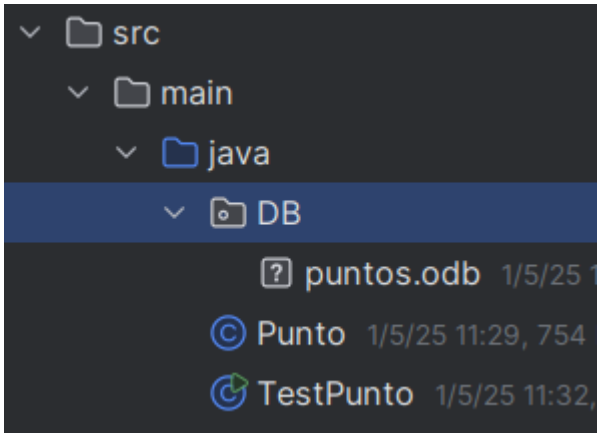
1. En la misma ruta, crea la clase `TestPunto.java`:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

```java
import javax.persistence.*;
import java.util.List;

public class TestPunto {
    public static void main(String[] args) {
        // Abre una conexión a la base de datos
        // Se crea un EntityManagerFactory con la configuración especificada en el archivo de persistencia
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("src/main/java/DB/puntos.odb");
        // Se crea un EntityManager para interactuar con la base de datos
        EntityManager em = emf.createEntityManager();

        // Almacena 1000 objetos Punto en la base de datos
        // Inicia una transacción
        em.getTransaction().begin();
        for (int i = 0; i < 1000; i++) {
            // Crea un nuevo objeto Punto con coordenadas aleatorias entre 0 y 99
            Punto p = new Punto((int) (Math.random() * 100), (int) (Math.random() * 100));
            // Persiste el objeto Punto en la base de datos
            em.persist(p);
        }
        // Confirma la transacción para guardar los cambios
        em.getTransaction().commit();

        // Consulta para contar los objetos Punto
        Query q1 = em.createQuery("SELECT COUNT(p) FROM Punto p");
        // Imprime el resultado de la consulta
        System.out.println("Número total de Puntos: " + q1.getSingleResult());

        // Consulta para calcular el promedio de X
        Query q2 = em.createQuery("SELECT AVG(p.x) FROM Punto p");
        // Imprime el resultado de la consulta
        System.out.println("Media de los puntos X: " + q2.getSingleResult());

        // Consulta para obtener todos los objetos Punto
        TypedQuery<Punto> query = em.createQuery("SELECT p FROM Punto p", Punto.class);
        // Obtiene la lista de resultados
        List<Punto> results = query.getResultList();
        System.out.println("Resultados de la consulta query:");
        for (Punto p : results) {
            // Imprime cada objeto Punto
            System.out.println(p);
        }

        // Cierra la conexión a la base de datos
        // Cierra el EntityManager
        em.close();
        // Cierra el EntityManagerFactory
        emf.close();
    }
}
```

1. Crea la BDOO:

Una vez creado todo el proyecto, y comprobado que no tenga errores de compilación, ejecuta el método `main` de la clase `TestPunto` y veras aparecer una carpeta llamada `DB` en la ruta que contiene un único fichero `puntos.odb`.



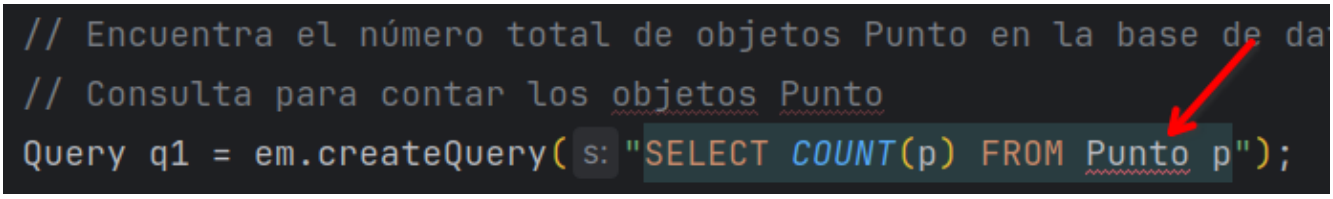
1. El resultado en pantalla debe ser similar a este:

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

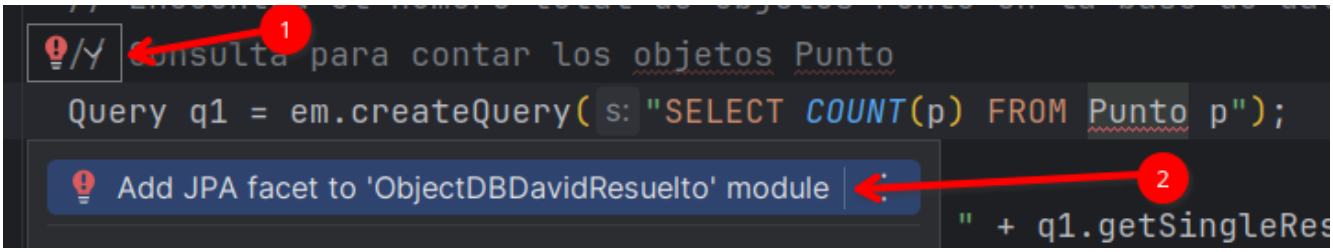
```
Número total de Puntos: 1000
Media de los puntos X: 49.05433333333333
(96, 19)
(40, 65)
(87, 97)
(24, 26)
(46, 19)
(28, 79)
(32, 18)
(51, 18)
(64, 22)
...
```

Fíjate

Puede que hayas observado que la palabra `Punto` dentro de las consultas SQL aparece subrayado en rojo. Eso es porque IntelliJ no detecta la clase Punto como una entidad JPA:



La solución es tan simple como hacer clic en la bombilla roja que aparece, y elegir la opción de que IntelliJ nos añada el complemento necesario de JPA para "entender" la persistencia de datos.



Ten en cuenta que cada vez que ejecutas el código se añaden 1000 puntos más a la BDOO, así que si lo ejecutas un par de veces más tendras 3000 puntos en la base de datos.

Si quieres volver a empezar solo debes borrar el fichero `*.odb` y al volver a ejecutar el código se generará de nuevo el fichero con 1000 puntos.

Tarea

Ahora genera un documento `pdf` con el siguiente contenido:

1. Explica para que sirven las etiquetas (`@Entity` , `@Id` , `@GeneratedValue` y `@Override`) que aparecen en la clase `Punto` .
2. Modifica la clase `TestPunto.java` para que la base de datos se cree en la ruta `bdoo` en luga de `DB` y el fichero `.odb` tenga tu nombre (en mi caso `david.odb`).
3. Modifica el método `main` y añade dos consultas más `q3` , `q4` y `q5` que muestren:
4. `q3` : La media de los puntos Y.
5. `q4` : consulte los puntos en los que la X o la Y sean superiores a 50.
6. `q5` : consulta los puntos ordenando primero descendientemente por la X, y en caso de empate ascendentemente por la Y.

Envía tu fichero `pdf` con explicaciones detallas sobre los cambios requeridos, adjuntando fragmentos del código añadido (con capturas de pantalla o recortes de código fuente) a la tarea de Aules.

6 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

12.3.2 Taller UD11_03: CRUD con ObjectDB

Introducción

Siguiendo con la misma base de datos que usaste el último taller de la unidad 10, vamos a realizar un mini-proyecto en el que puedas comprobar todo lo aprendido en la unidad 11. La idea básica es adaptar el proyecto del taller 3 de la unidad 10 a ObjectDB, pero solo para dos tablas (las que tienen alguna relación).

PASOS A SEGUIR

Los pasos a seguir para realizar el proyecto serán los siguientes:

1. Genera al menos dos clases (una para cada tabla de tu futura DBO) con todas las etiquetas ObjectDB necesarias, añadiendo comentarios al lado de cada una de las etiquetas, justificando porque las estas usando.
2. Genera una o varias clases más para poder realizar un menú similar al de la unidad anterior, que te permita elegir de un menú principal cual de las dos tablas quieres gestionar.
3. Para cada una de las tablas deberás generar el CRUD completo (similar al tema anterior).
4. Solo para una de ellas, además deberás añadir la posibilidad de mantener la relación que existe entre ellas.

Consideraciones adicionales:

1. La BD se generará en una carpeta llamada `BD` dentro de la estructura de tu proyecto y puede ser un único fichero `odb`.
2. Para simplificar el proyecto no será necesario seguir los patrones vistos en la unidad anterior.
3. Recuerda que en el proyecto no solo es necesario programar, sino también comentar lo que vas haciendo.

Ejemplo con dos tablas

Para clarificar las tareas a realizar, seguiremos con la BD de starwars del tema anterior.

1. Generaremos dos tablas (characters/personajes y planets/planetas) con sus etiquetas para cada uno de sus campos y la relación que existe entre ellos, cada personaje tiene un planeta (y un planeta puede tener muchos personajes).
2. En mi caso generaremos una clase principal, con el menú principal que usará las clases del punto siguiente. Esta clase también se encargará de generar contenido en la DBO (en caso de que esté vacía en la primera ejecución)
3. Desarrollaremos una clase para gestionar personajes y otra para gestionar planetas.
4. Desde la clase para gestionar personajes, se permitirá elegir un planeta asociado al personaje (así implementamos la relación entre las dos tablas)

Más consideraciones:

1. El fichero de la `BD` se llamará `starwars.odb` y se guardará en la carpeta `BD` de mi proyecto.
2. Aunque no es necesario, aplicar el patrón Singleton para acceder a la BD es relativamente sencillo e igual al de las BDR, así que unificaremos la conexión en una única clase.

Tarea

Genera un proyecto en `IntelliJ` con las características descritas, exporta el proyecto como `zip` y sube el proyecto a la tarea de AULES.

6 de septiembre de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

13. 👋 Sobre mí...

13.1 👨‍💻 David Martinez Peña

13.2 📬 Contacto:

- 📧 d.martinezpena@edu.gva.es
- ▶️ Youtube
- 💼 LinkedIn
- 🐙 GitHub

IES EDUARDO PRIMO MARQUÉS

👨‍🏫 Profesor de Secundaria, especialidad de Informática.

🏫 Actualmente con destino en IES Eduardo Primo Marqués de Carlet

🤖 Programación © 2025 by David Martínez is licensed under CC BY-NC-SA 4.0

IES EDUARDO PRIMO MARQUÉS INFORMÀTICA I COMUNICACIONS

6 de diciembre de 2025

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

14. Fuentes de información

- Wikipedia
- Programación (Grado Superior) - Juan Carlos Moreno Pérez (Ed. Ra-ma)
- Apuntes IES Henri Matisse (Javi García Jimenez?)
- Apuntes AulaCampus
- Apuntes José Luis Comesaña
- Apuntes IOC Programació bàsica (Joan Arnedo Moreno)
- Apuntes IOC Programació Orientada a Objectes (Joan Arnedo Moreno)
- Apuntes Lionel
- https://arturoblasco.github.io/prg
- https://www.cs.usfca.edu/~galles/visualization/Algorithms.html
- https://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html
- FXDocs
- https://examples.javacodegeeks.com/java-development/desktop-java/javafx/javafx-2d-shape-example/
- https://openjfx.io/openjfx-docs/
- https://docs.oracle.com/javase/8/javafx/user-interface-tutorial
- https://github.com/JonathanGiles/scenic-view
- Apuntes de Jose Antonio Diaz-Alejo
- https://docs.oracle.com/javase/8/scene-builder-2/work-with-java-ides/sb-with-nb.htm#CHEEHIDG
- https://blog.devgenius.io/flutter-internationalization-i18n-with-getx-d3a6465d1282
- https://github.com/openjfx/openjfx-docsopen
- https://github.com/openjfx/samples
- https://www.objectdb.com/java/jpa
- https://awsacademyinstructure.com
- https://dev.mysql.com
- https://aws.amazon.com/es/getting-started/hands-on/create-mysql-db/
- https://github.com/miguelmarti/FreakDataBases
- https://www.jetbrains.com
- NotebookLM
- ChatGPT
- DeepSeek

14.1 Canales de vídeo

Canal	Enlace	Descripción
Píldoras Informáticas	https://www.youtube.com/@pildorasinformaticas	Curso Java completo (edición 2026 en publicación semanal + curso clásico de 280 vídeos)
Programación ATS	https://youtube.com/playlist?list=PLWtYZ2ejMVJkjOuTCzIk61j7XKfpIR74K	Curso Java estructurado: desde introducción hasta colecciones y ficheros
MoureDev	https://youtu.be/JOAqpdM36wI	Curso completo Java 8h — ideal como visión global
makigas	https://www.makigas.es/series	Series Java IO y JDBC Moderno
EducaMadrid Mediateca	https://mediateca.educa.madrid.org/	Vídeos educativos de profesores (POO, arrays, ficheros, colecciones)
DiscoDurodeRoer	https://www.youtube.com/@DiscoDurodeRoer	

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

Canal	Enlace	Descripción
		Listas de reproducción Java: ficheros, JDBC, etc.

20 de julio de 2026

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

https://martinezpenya.es/1DAMProgramacion/

David Martinez Peña

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

 © 2025 David Martínez licensed under CC BY-NC-SA 4.0